Universidad Nacional Autónoma de México
Facultad de Ciencias



Almacenes y Minería de Datos

Fundamentos, Modelado Dimensional, Arquitecturas Modernas y Minería de Datos

Diego Villalba
Facultad de Ciencias, UNAM
diego.villalba@ciencias.unam.mx
https://diego-villalba.com/

Ciudad de México, México
2026

Almacenes y Minería de Datos
Diego Villalba

Facultad de Ciencias
Universidad Nacional Autónoma de México (UNAM)
Ciudad Universitaria, Coyoacán, C.P. 04510, Ciudad de México.
Contacto: diego.villalba@ciencias.unam.mx
Sitio web del autor: https://diego-villalba.com/
Repositorio del proyecto: https://github.com/DiegoViillalba/almacenamienes-y-mineria-de-datos

---

Derechos de autor y términos de uso

© 2026 Diego Villalba. Algunos derechos reservados.

Esta obra se distribuye bajo una Licencia Internacional **Creative Commons Atribución-NoComercial-CompartirIgual 4.0 (CC BY-NC-SA 4.0)**.

Usted es libre de compartir (copiar y redistribuir el material en cualquier medio o formato) y adaptar (remezclar, transformar y construir a partir del material) bajo las siguientes condiciones:

- **Atribución**: Debe dar crédito de manera adecuada, brindar un enlace a la licencia e indicar si se han realizado cambios.
- **No Comercial**: No puede hacer uso del material con propósitos comerciales.
- **Compartir Igual**: Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia del original.

Cómo citar esta obra

Referencia (APA 7.ª edición):

Villalba, D. (2026). *Almacenes y Minería de Datos*. Facultad de Ciencias, Universidad Nacional Autónoma de México. https://diego-villalba.com/almacenamienes-y-mineria-de-datos/

Entrada BibTeX:

```
@book{villalba2026almacenes,
   author = {Villalba, Diego},
   title = {Almacenes y Minería de Datos},
   year = {2026},
   publisher = {Universidad Nacional Autónoma de México, Facultad de Ciencias},
   url = {https://diego-villalba.com/almacenamienes-y-mineria-de-datos/}
}
```

Código fuente y laboratorios interactivos:

Esta edición en PDF está estructurada como un libro de referencia teórico y conceptual, omitiendo los bloques de código fuente para ofrecer una lectura más compacta y centrada en los fundamentos. Todos los bloques de código en Python, laboratorios interactivos, consultas SQL y notebooks ejecutables están disponibles en la plataforma web del libro:

https://diego-villalba.com/almacenamienes-y-mineria-de-datos/

Tabla de contenidos

Prefacio **1**
Temario . . . 1
Requisitos previos . . . 1
Agradecimientos . . . 1
Autor . . . 2
Cómo citar . . . 2

I. Entendimiento de Negocio **3**

II. Fundamentos y Stakeholders **5**

1. Gestión de Requerimientos y Stakeholders **7**
Identificación de Partes Interesadas (*Stakeholders*) . . . 7
Análisis de Stakeholders: La Matriz de Poder e Interés . . . 9
Definición de alcance y traducción de preguntas de negocio . . . 10
Refinamiento iterativo mediante preguntas . . . 10
Ejemplos de traducción de preguntas de negocio . . . 12
El proceso de solicitud de datos (*Data Request*) . . . 13
Cierre . . . 14
Caso de Estudio Práctico: Alineación en TheLook eCommerce . . . 14

2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos **15**
El Ciclo de Vida de los Datos . . . 16
Fases Críticas de la Gestión de Datos . . . 17
Actividad Guiada: Diseñar un Plan de Ciclo de Vida para TheLook eCommerce . . . 18

III. Planeación del Proyecto **21**

3. Planeación Real de un Proyecto: El Business Understanding Canvas **23**
Paso 1: La Bitácora de la Reunión con el Stakeholder . . . 24
Paso 2: El Business Understanding Canvas (Una Página) . . . 25
Paso 3: Preguntas de Investigación Priorizadas . . . 26
Paso 4: Criterio de Éxito en Términos de Negocio . . . 27
Paso 5: Supuestos y riesgos . . . 27
Actividad: Construye tu Propio Business Understanding Canvas . . . 30

IV. Almacenes de Datos **33**

V. Arquitectura de almacenes de Datos **35**

4. Introducción **37**
La Arquitectura en Ciencias de la Computación . . . 37
Eje 1: El costo de leer datos — filas vs. columnas . . . 37
Eje 2: El costo de la rigidez — datos homogéneos vs. heterogéneos . . . 39
Eje 3: El costo de la consistencia — ACID vs. BASE . . . 40
El sentido de ser de las arquitecturas de almacenamiento . . . 41

5. Data Warehouse **43**
Arquitectura de un Data Warehouse . . . 45
Enfoques de implementación: Kimball vs Inmon . . . 48
Ejemplo práctico: Facultad de Ciencias . . . 51
Conclusión . . . 54

6. Data Lakehouse **55**
La base técnica del Lakehouse . . . 57
Integrar calidad, cargas y productos de datos . . . 61
Implementaciones en Google Cloud, AWS y Azure . . . 62
El trabajo que no desaparece . . . 64
Decidir, no evangelizar . . . 64
Cheatsheet . . . 67
Bibliografía y guías oficiales de implementación . . . 68

VI. Modelado multidimensional **71**

7. Modelado Multidimensional **73**
Hechos y Dimensiones . . . 73
Granularidad: la decisión de diseño más importante . . . 75
Jerarquías en dimensiones y navegación analítica . . . 77
Conclusión . . . 77

8. Esquemas Dimensionales y Redundancia **79**
Esquema de Estrella . . . 79
Esquema de Copo de Nieve (Snowflake) . . . 80
Redundancia controlada: qué cuesta realmente desnormalizar . . . 81
Costo de consulta: joins y motores en la nube . . . 82
Esquema de Constelación (Fact Constellation / Galaxy Schema) . . . 82
Patrones de diseño para gestionar redundancia . . . 83
Conclusión . . . 84

9. Dimensiones Lentamente Cambiantes (SCD) **85**
Tipos de SCD . . . 85
Implementación con dbt . . . 87
Pruebas de integridad . . . 87
Comparación y criterio de elección . . . 87

10. Ejemplo Completo: Café Norte — Análisis y Construcción de Data Marts (Kimball) 89
Propósito y supuestos . . . 89
Estructura esperada de archivos CSV . . . 89
Carga de datos y validación de esquema . . . 90
Homologación e integración de fuentes (Staging Integrado) . . . 90
Construcción de dimensiones conformadas . . . 91
Construcción de la tabla de hechos (fact_linea_pedido) . . . 92
Data Marts y respuestas a preguntas estratégicas . . . 92
Exportación de tablas: Dimensiones y Hechos (Data Mart físico) . . . 93
Discusión técnica: decisiones de diseño y extensiones . . . 93
Reproducibilidad y control de versiones . . . 94

VII. Integración ETL/ELT 95

11. Introducción a PySpark y Fundamentos ETL 97
¿Qué es el ETL y por qué importa? . . . 97
Entorno de prácticas: Databricks Community Edition . . . 98
PySpark: Fundamentos . . . 99
Referencias de este capítulo . . . 99

12. Extracción y Fuentes Heterogéneas 101
Extracción (*Extract*) . . . 101
Integración de fuentes heterogéneas: Data Lake → DW . . . 101
Referencias de este capítulo . . . 103

13. Transformación, Carga y Pipeline ETL 105
Transformación (*Transform*) . . . 105
Transformaciones SQL con Spark . . . 105
Carga (*Load*) . . . 106
Pipeline ETL completo . . . 106
Referencias de este capítulo . . . 106

14. Prácticas ETL en Databricks 107
Ejercicios . . . 107
Referencias de este capítulo . . . 108

15. Orquestación y Herramientas Modernas 109
ETL moderno: dbt + PySpark . . . 109
Orquestación con Apache Airflow . . . 109
Resumen y mapa de herramientas . . . 109
Referencias y recursos adicionales . . . 110

VIIIModelado para consumo 111

16. Computación de cubos de datos OLAP 113
Del almacén de datos al análisis OLAP . . . 113
Modelo multidimensional . . . 114

Operaciones fundamentales en cubos OLAP . . . 115
Cálculo manual de un cubo OLAP en un dataset pequeño . . . 117
Ejemplo reproducible en Python . . . 122
Implementación de operaciones OLAP . . . 124
Exploración interactiva del cubo . . . 125
Agregaciones y medidas derivadas . . . 125
Ventajas, limitaciones y criterios de diseño . . . 126
Buenas prácticas para documentación reproducible . . . 127
Conclusiones . . . 128

17. Data Marts: del Almacén de Datos al Consumo por Área de Negocio 129
¿Qué es un data mart? . . . 129
Arquitecturas: dependiente, independiente e híbrida . . . 130
Dimensiones conformadas y la matriz de bus . . . 131
Diseño dimensional de un data mart . . . 132
Ejemplo reproducible: construir el data mart en Python . . . 133
El riesgo de los data marts independientes: dimensiones no conformadas . . . 134
Ventajas, limitaciones y buenas prácticas . . . 135
De data marts a productos de datos . . . 135
Conclusiones . . . 136

18. Consumo Moderno de Data Marts: APIs y Capas Semánticas 137
Del acceso directo al contrato de servicio . . . 137
REST sobre un data mart . . . 138
GraphQL sobre un data mart . . . 139
Capa semántica: métricas como código . . . 140
Un mismo dato, tres contratos . . . 140
Seguridad y gobierno en la capa de API . . . 140
Ventajas, limitaciones y buenas prácticas . . . 141
Conclusiones . . . 141

IX. Marco Metodológico de Minería de Datos 143

19. Introducción a la Minería de Datos y sus Frameworks de Proceso 145
La pregunta que abre el capítulo . . . 145
Recuperación de datos versus minería de datos . . . 145
Correlación y causalidad: la primera trampa . . . 146
Las tareas centrales de la minería de datos . . . 147
Frameworks de proceso: cuatro formas de responder la misma pregunta . . . 148
Comparativa de frameworks . . . 149
Los frameworks en la nube: de la metodología a la herramienta . . . 150
Consideraciones éticas y de implementación . . . 151
Resumen final . . . 152
Preguntas de repaso . . . 152
Recursos y lecturas adicionales . . . 153

20. Business Understanding - Caso BigMart Sales 155
Escenario ficticio de negocio . . . 155

Situación problemática . . . 156
Diccionario de Variables . . . 156
Necesidades del negocio que motivan el análisis . . . 157
Del problema de negocio al Data Understanding . . . 159

X. Análisis Exploratorio de Datos (EDA) 161

XI. Fundamentos del EDA 163

21. Fundamentos del análisis exploratorio de datos 165
Introducción . . . 165
Preguntas que orientan el EDA . . . 165
Unidad de observación y granularidad . . . 166
Tipos de variables . . . 166
Frecuencias y proporciones . . . 166
Frecuencias acumuladas y ECDF . . . 167
Ejemplo de calidad categórica . . . 167
Ejemplo discreto y acumulado . . . 167
Datos faltantes . . . 167
Valores atípicos y anomalías . . . 167
Ruta descriptiva del libro . . . 168
Checklist de salida . . . 168
Ejercicios . . . 168

XII. Medidas de localización 169

22. Medidas de localización 171
El problema: ¿qué tiempo representa una entrega habitual? . . . 171
Antes de resumir: conservar los datos visibles . . . 172
Tres respuestas para el mismo conjunto . . . 172
¿Qué cambia cuando se alarga la demora? . . . 174
Cuantiles: responder preguntas de servicio . . . 175
Cuando las observaciones representan cantidades distintas . . . 176
Por qué media y mediana aparecen como soluciones . . . 176
Laboratorio interactivo: arrastrar una demora . . . 178
Formalización . . . 178
Implementación reproducible . . . 179
Guía de decisión . . . 180
Errores frecuentes . . . 180
Puentes con minería de datos . . . 181
Puente al caso BigMart . . . 181
Ejercicios formativos . . . 182

23. Medidas de localización en Big Mart Sales 183
Preparación del entorno . . . 183
Inspección inicial . . . 183

Selección de variables para el análisis de localización . . . 184
Medidas de localización básicas . . . 185
Interpretación estadística de la media y la mediana . . . 186
Cuantiles y lectura ampliada del centro . . . 186
Impacto de eliminar faltantes en `Item_Weight` . . . 187
Ejercicio sugerido . . . 188

XIIIMedidas de variabilidad 189

24. Medidas de variabilidad 191
El problema: cumplir el promedio no basta . . . 191
Primera mirada: la amplitud total . . . 191
De las desviaciones a la varianza . . . 192
¿Población completa o muestra? . . . 193
Misma media, diferente variabilidad . . . 194
Laboratorio: mover los pedidos y observar las medidas . . . 195
Variabilidad del centro: IQR y MAD . . . 195
Comparar variabilidad entre escalas . . . 198
Formalización y propiedades . . . 198
Implementación reproducible . . . 199
Guía de decisión . . . 200
Puentes con minería de datos . . . 201
Puente al caso BigMart . . . 201
Ejercicios formativos . . . 202

25. Medidas de variabilidad en Big Mart Sales 203
Preparación del entorno . . . 203
De la localización a la dispersión . . . 203
Cálculo de medidas de variabilidad . . . 204
Visualización de dispersión con boxplots . . . 205
Rango frente a IQR: medidas sensibles y robustas . . . 205
Comparación de dispersión entre grupos . . . 206
Coeficiente de variación y comparación relativa . . . 207
Histograma con bandas de dispersión . . . 207
Ejercicio sugerido . . . 207

XIVMedidas de forma 209

26. Medidas de forma 211
El problema: mismo centro y escala, distinta experiencia . . . 211
Aprender a mirar la forma . . . 212
Del dibujo al cálculo: la mañana B . . . 213
Formalización mediante momentos . . . 215
Laboratorio interactivo: cambiar la forma . . . 217
Curtosis y colas: una comparación controlada . . . 217
Gráficos que completan las medidas . . . 218
Transformar cambia la forma y la pregunta . . . 218

Implementación reproducible . . . 219
Qué implica la forma en minería de datos . . . 219
Guía de lectura . . . 220
Puente al caso BigMart . . . 220
Ejercicios formativos . . . 221

27. Medidas de forma en Big Mart Sales **223**
Introducción . . . 223
Preparación del entorno . . . 223
Fundamentos teóricos . . . 224
Exploración gráfica inicial de la variable de ventas . . . 225
Cálculo de asimetría y curtosis . . . 225
Comparación de forma en varias variables numéricas . . . 226
Transformaciones y su efecto sobre la forma . . . 226
Gráfico cuantil-cuantil como apoyo visual . . . 227
Mini-dashboard de forma . . . 228

XV. Medidas de heterogeneidad **229**

28. Medidas de heterogeneidad **231**
El problema: ¿qué significa que unos datos sean heterogéneos? . . . 231
Heterogeneidad categórica I: Gini–Simpson . . . 232
Heterogeneidad categórica II: de la sorpresa a la entropía . . . 234
Comparar Gini–Simpson y Shannon . . . 237
Una escala común: números de Hill . . . 238
Heterogeneidad de una variable numérica entre grupos . . . 239
Heterogeneidad cruzada entre variables categóricas . . . 241
Puentes con algoritmos de minería de datos . . . 241
Implementación reproducible . . . 242
Guía de decisión . . . 242
Puente al caso BigMart . . . 243
Ejercicios formativos . . . 243

29. Medidas de heterogeneidad en Big Mart Sales **245**
Introducción . . . 245
Preparación del entorno . . . 245
Fundamentos teóricos . . . 246
Variable objetivo y grupos de interés . . . 246
Heterogeneidad por tipo de outlet . . . 247
Heterogeneidad por tamaño del outlet . . . 248
Heterogeneidad por ubicación del outlet . . . 249
Heterogeneidad por tipo de producto . . . 250
Tablas comparativas con múltiples indicadores . . . 251
Heterogeneidad cruzada . . . 251
Entropía de Shannon como medida de heterogeneidad . . . 252
Mini-dashboard de heterogeneidad . . . 254
Conclusiones del capítulo . . . 254

XVIMedidas de concentración **255**

30. Medidas de concentración **257**
El problema: ¿quién acumula el total? . . . 257
Primera mirada: razones de concentración . . . 258
HHI: ¿de qué sucursal vienen dos ventas? . . . 259
Curva de Lorenz: acumular desde la unidad menor . . . 261
Coeficiente de Gini: convertir el arco en una cifra . . . 262
Laboratorio interactivo: mover la masa . . . 264
Qué ve cada indicador . . . 264
Propiedades que debe cumplir una comparación . . . 265
Implementación reproducible . . . 266
Guía de decisión . . . 266
Errores frecuentes . . . 267
Puentes con minería de datos . . . 267
Puente al caso BigMart . . . 267
Ejercicios formativos . . . 268

31. Medidas de concentración en Big Mart Sales **269**
Fundamento conceptual . . . 270
Concentración de ventas por categoría de producto . . . 271
Concentración de ventas por outlet . . . 272
Concentración por tipo de outlet . . . 273
Curva de concentración simplificada . . . 273
Indicadores resumidos de concentración . . . 273
Mini-dashboard de concentración . . . 274
Discusión final . . . 274
Ejercicios sugeridos . . . 274

XVIMedidas de correlación **275**

32. Medidas de correlación **277**
El problema: ¿avanzan juntas dos características? . . . 277
Antes de calcular: mirar los pares . . . 278
Covarianza: dirección con unidades . . . 278
Pearson: estandarizar la asociación lineal . . . 281
Spearman: comparar órdenes . . . 285
Kendall: contar concordancias . . . 287
Laboratorio integrador: cuatro geometrías, una muestra . . . 291
Lineal, monótona y no monótona . . . 291
Sensibilidad a una observación extrema . . . 292
Laboratorio de influencia: mover un punto extremo . . . 294
El cuarteto de Anscombe: mismo coeficiente, distinta historia . . . 294
Correlación no implica causalidad . . . 294
Matrices de correlación . . . 295
Formalización poblacional . . . 295
Implementación reproducible . . . 296

Guía de decisión . . . 296
Errores frecuentes . . . 297
Puentes con minería de datos . . . 297
Puente al caso BigMart . . . 297
Ejercicios formativos . . . 298

33. Medidas de correlación en Big Mart Sales **299**
Introducción . . . 299
Objetivos del capítulo . . . 299
Carga y preparación de datos . . . 300
Fundamento conceptual . . . 300
Revisión de valores faltantes en variables numéricas . . . 301
Matriz de correlación de Pearson . . . 301
Correlación de Spearman . . . 302
Correlación con la variable objetivo . . . 302
Diagramas de dispersión . . . 303
Correlación por subgrupos . . . 303
Tabla resumen de asociaciones . . . 304
Mini-dashboard de correlación . . . 304
Discusión final . . . 304
Ejercicios sugeridos . . . 305

XVIII Síntesis del EDA **307**

34. Dashboard global **309**
Introducción . . . 309
Objetivos del capítulo . . . 309
Carga y preparación de datos . . . 309
Arquitectura conceptual del dashboard . . . 310
Selección de variable principal . . . 311
Panel de indicadores globales . . . 311
Panel de localización y forma . . . 311
Panel de variabilidad . . . 312
Panel de heterogeneidad . . . 312
Panel de concentración . . . 313
Panel de correlación . . . 313
Ensamblaje del dashboard global . . . 313
Propuesta de cierre visual para el book . . . 314
Conclusiones . . . 314
Sugerencias de extensión . . . 315
Integración al libro de Quarto . . . 315

XIX Análisis Multivariado **317**

35. Análisis Exploratorio Multivariado: De la Estadística Descriptiva al Espacio Multidimensional **319**
El problema: cuando mirar variable por variable no es suficiente . . . 319

Localización multivariada: el centro de una nube en $\mathbb{R}^p$ . . . 320
Variabilidad y dispersión multivariada: forma y orientación . . . 323
Forma y normalidad multivariada . . . 325
Heterogeneidad y anomalías multivariadas . . . 327
Dependencia multivariada y correlación parcial . . . 328
Técnicas de visualización multivariada . . . 329
Laboratorio práctico interactivo . . . 330
Conclusiones y síntesis . . . 330

36. Análisis Exploratorio Multivariado en la Práctica: Caso Palmer Penguins 333
Contexto del problema y datos de estudio . . . 333
Paso 1: Ingesta, limpieza y exploración univariada en `pandas` . . . 334
Paso 2: Localización multivariada: Centro de gravedad vs. Centro robusto . . . 335
Paso 3: Dispersión multivariada: Covarianza, Varianza Total y de Wilks . . . 336
Paso 4: La Paradoja de Simpson en acción . . . 336
Paso 5: Correlación parcial y matriz de precisión . . . 337
Paso 6: Detección de outliers multivariados con Distancia de Mahalanobis . . . 338
Paso 7: Visualización multivariada integral: Coordenadas Paralelas . . . 340
Conclusiones y guía práctica para el científico de datos . . . 340

XX. Preprocesamiento 343

37. Introducción al preprocesamiento y la calidad de datos 345
Ejemplo: de una tabla cruda a variables utilizables . . . 345
Formalización: reglas, parámetros y calidad . . . 346
Laboratorio: ¿qué cambia si ajustamos con datos de prueba? . . . 347
Implementación con `pandas` y `scikit-learn` . . . 348
Para recordar . . . 349

38. Conversión de Datos, Codificación y Escalado 351
El problema de la escala . . . 351
El peligro de la falsa ordinalidad . . . 352
Desarrollo manual paso a paso . . . 352
Formalización teórica y matemática . . . 355
Laboratorio interactivo: banco de pruebas de escalado y codificación (BigMart) . . . 356
Implementación computacional (Python y pandas) . . . 357
Referencias y definiciones . . . 359

39. Tratamiento de Ruido, Suavizado y Discretización 361
Ruido y valores atípicos . . . 361
Visibilidad de productos y una serie simulada . . . 361
Ejemplo trabajado: discretización y suavizado a mano . . . 362
Formalización: modelos y filtros . . . 363
Laboratorio interactivo: suavizado y discretización . . . 365
Implementación computacional con Python . . . 366
Prácticas para usar en proyectos reales . . . 366
Notas de términos . . . 366

40. Tratamiento de datos faltantes **367**
Un primer ejemplo visual . . . 367
Ejemplo: tres mecanismos de ausencia . . . 367
De la intuición al cálculo: imputar un peso . . . 369
Qué muestran las imputaciones . . . 369
Formalización breve . . . 370
Laboratorio: elegir y poner a prueba una imputación . . . 371
Implementación sin fuga de información . . . 372

41. Reducción de Dimensionalidad con PCA **373**
Ejemplo: dos sensores paso a paso . . . 373
Motivación: la maldición de la dimensionalidad . . . 375
Fundamentos matemáticos del PCA . . . 376
PCA y la Descomposición en Valores Singulares (SVD) . . . 378
Ejemplo práctico: Implementación paso a paso . . . 379
Ejemplo avanzado: Dataset con ruido . . . 382
Limitaciones del PCA . . . 385
Más allá del PCA: técnicas no lineales . . . 385
Resumen y conexiones . . . 388
Laboratorio aplicado: ¿conviene PCA para clasificar Iris? . . . 388
Ejercicios propuestos . . . 389

42. Reducción de Variables Categóricas y Prueba ^{2} **391**
Intuición: ¿qué esperaríamos si no hubiera relación? . . . 391
Desarrollo manual paso a paso . . . 391
Formalización teórica y matemática . . . 394
Laboratorio interactivo: de la tabla observada a la distribución nula . . . 395
Mejores prácticas en la industria . . . 396
Implementación computacional y selección de variables . . . 396

XXI Extracción de Patrones **399**

43. Extracción de Patrones: Datos Transaccionales y Espacio de Búsqueda **401**
Intuición: cinco tickets . . . 401
De la tabla de hechos a la canasta . . . 402
Construcción numérica: soporte paso a paso . . . 403
Formalización . . . 403
Laboratorio: la retícula de itemsets . . . 404
Implementación en Python . . . 405
Lo que viene . . . 405

XXI Reglas de Asociación **407**

44. Reglas de Asociación: Fundamentos, Interpretación y Selección **409**
Introducción . . . 409
Definiciones Fundamentales . . . 409
De Itemsets Frecuentes a Reglas . . . 413

El Problema de las Demasiadas Reglas . . . 413
Métricas de Evaluación: Interpretación Profunda . . . 414
Interpretación Visual del Espacio de Reglas . . . 416
Selección de Reglas Útiles . . . 417
Limitaciones del Enfoque . . . 420
Resumen Conceptual . . . 421

45. El Algoritmo Apriori: Paso a Paso y Ejemplo Práctico 423
Intuición: buscar por niveles . . . 423
Desarrollo manual paso a paso . . . 423
Formalización . . . 425
Laboratorio: Apriori nivel por nivel . . . 426
Implementación desde cero en Python . . . 426
Ejemplo práctico: canastas de supermercado . . . 427
Ventajas y limitaciones . . . 429

46. Más Allá del Lift: Métricas Avanzadas, Sensibilidad y Estructura de Reglas 431
Introducción . . . 431
Las Limitaciones Estructurales del Lift . . . 431
Métricas Complementarias al Lift . . . 434
Sensibilidad a los Parámetros del Algoritmo . . . 438
La Estructura Global del Espacio de Reglas: Representación como Grafo . . . 440
Un Marco Integrado de Evaluación . . . 442
Resumen Conceptual . . . 443

XXII Algoritmos de Patrones Frecuentes 445

47. FP-Growth y minería de patrones frecuentes 447
FP-Growth y minería de patrones frecuentes . . . 447

48. Itemsets Cerrados y Maximales: Representaciones Compactas 457
Intuición: once patrones que dicen casi lo mismo . . . 457
Construcción numérica paso a paso . . . 458
Formalización . . . 459
Laboratorio: frecuentes, cerrados y maximales en la retícula . . . 461
Implementación en Python . . . 461
Cuándo usar cada representación . . . 463

XXIII Patrones Secuenciales 465

49. Patrones Secuenciales: GSP y PrefixSpan 467
Intuición: cinco sesiones de una tienda en línea . . . 467
Construcción numérica paso a paso . . . 468
Formalización . . . 469
Laboratorio: la base proyectada de PrefixSpan . . . 471
Implementación en Python . . . 472
Ejemplo práctico: clickstream de una tienda en línea . . . 473

Ventajas y limitaciones 474

XXV De Patrones a Clasificación 475

50. De Patrones a Predicciones: Introducción a la Clasificación con Reglas de Asociación 477
El problema de clasificación 477
Intuición: catorce días de tenis 477
Construcción paso a paso 478
Formalización 481
Laboratorio: construye y evalúa un clasificador CBA 481
Implementación en Python 482
Ejemplo práctico: especies de pingüinos 482
Ventajas, limitaciones y extensiones 483
Resumen 484

XXVI Clasificación 485

XXVII Fundamentos de Clasificación 487

51. El problema de clasificación 489
El problema de clasificación 489
Intuición: ¿pagará el préstamo? 490
Construcción paso a paso 491
Formalización 493
Laboratorio: complejidad y generalización 495
El flujo de trabajo de un clasificador 496
Un mapa de los clasificadores de esta parte 498
Implementación en Python 498
Resumen 500

52. Vecinos más cercanos 501
Intuición: ¿quién compra un seguro? 501
Construcción paso a paso 502
Formalización 504
Laboratorio: el vecindario de k-NN 506
Elegir k 506
La maldición de la dimensionalidad 506
Costo computacional y búsqueda eficiente 507
Implementación en Python 508
Ventajas y limitaciones 509
Resumen 509

XXVEvaluación de Clasificadores **511**

53. Métricas de clasificación **513**
Intuición: una prueba de detección . . . 513
Construcción paso a paso . . . 513
Formalización . . . 518
Laboratorio: umbral, prevalencia y curvas . . . 520
Cómo elegir la métrica . . . 521
Implementación en Python . . . 521
Resumen . . . 522

54. Validación y selección de modelos **525**
Intuición: ¿qué clasificador elegirías? . . . 525
Construcción paso a paso . . . 525
Formalización . . . 527
Laboratorio: ¿cuánto se mueve la estimación? . . . 529
Implementación en Python . . . 530
La fuga de datos . . . 530
Clases desbalanceadas . . . 531
Calibración de probabilidades . . . 531
Comparar dos clasificadores . . . 532
Lista de verificación . . . 532
Resumen . . . 533

XXIÁrboles y Ensambles **535**

55. Árboles de decisión: ID3 y C4.5 **537**
Intuición: ¿se juega el partido? . . . 537
Construcción paso a paso . . . 538
Formalización . . . 540
El sesgo hacia atributos con muchos valores: C4.5 . . . 541
Laboratorio: construye el árbol . . . 543
Implementación en Python . . . 544
Complejidad, ventajas y limitaciones . . . 545
Resumen . . . 546

56. CART: árboles binarios, índice de Gini y poda **547**
Intuición: ¿quién compra una laptop? . . . 547
Construcción paso a paso . . . 548
Formalización . . . 550
Laboratorio: CART en el plano . . . 552
Implementación en Python . . . 552
Complejidad, ventajas y limitaciones . . . 554
Resumen . . . 554

57. Árboles de regresión y ensambles: bagging, bosques aleatorios y AdaBoost **555**
Árboles de regresión . . . 555
¿Por qué funciona combinar modelos? . . . 556

Bagging . . . 557
Bosques aleatorios . . . 558
Laboratorio: bagging de árboles de regresión . . . 559
Boosting y AdaBoost . . . 559
Laboratorio: AdaBoost ronda por ronda . . . 561
Bagging frente a boosting . . . 562
Implementación en Python . . . 562
Resumen . . . 563

58. Gradient Boosting y XGBoost **565**
Intuición: cada árbol corrige al anterior . . . 565
Construcción paso a paso . . . 565
De los árboles de decisión al boosting . . . 567
El objetivo de XGBoost . . . 569
Laboratorio: gradient boosting paso a paso . . . 572
Hiperparámetros principales . . . 572
Importancia de variables . . . 573
Regularización y poda . . . 574
Contribuciones individuales (SHAP simplificado) . . . 575
XGBoost para clasificación . . . 576
Buenas prácticas y comparación con alternativas . . . 576

XXX Modelos Lineales **579**

59. Regresión lineal y clasificadores lineales: Fisher y el perceptrón **581**
Regresión lineal . . . 581
De la regresión a la clasificación lineal . . . 584
El discriminante lineal de Fisher . . . 584
El perceptrón . . . 586
Comparación . . . 588
Implementación en Python . . . 588
Resumen . . . 589

60. Regresión Logística **591**
Motivación: el problema de clasificar con regresión lineal . . . 591
La función logística (sigmoide) . . . 592
El modelo de regresión logística . . . 593
Construcción paso a paso: seis estudiantes . . . 594
Estimación por máxima verosimilitud . . . 595
Laboratorio: ajustar la logística . . . 597
Interpretación de coeficientes: odds ratio . . . 598
Regularización . . . 599
Extensión multiclase . . . 601
Calibración de probabilidades . . . 602
Ventajas y limitaciones . . . 603
Recomendaciones prácticas . . . 604
Resumen del capítulo . . . 605

61. Máquinas de Soporte Vectorial **607**
Motivación: clasificar con margen máximo . . . 607
SVM de margen duro (datos linealmente separables) . . . 608
SVM de margen suave (datos no separables) . . . 611
El kernel trick . . . 612
Efecto de los hiperparámetros C y (kernel RBF) . . . 613
Comparación de kernels en datos reales . . . 614
Laboratorio: margen, C y núcleos . . . 614
SVM para regresión (SVR) . . . 614
Aspectos computacionales . . . 615
Ventajas y limitaciones . . . 617
Obtener probabilidades de SVM: calibración de Platt . . . 617
Recomendaciones prácticas . . . 618

XXClasificadores Probabilísticos **621**

62. Clasificadores bayesianos **623**
Vista probabilística de la clasificación . . . 627
Clasificador bayesiano de error mínimo . . . 633
Clasificador Naïve Bayes . . . 634
Clasificadores bayesianos gaussianos . . . 638
¿Cómo evaluar el supuesto gaussiano? . . . 642
Clasificación bayesiana de secuencias discretas . . . 642
Clasificadores bayesianos de riesgo mínimo . . . 644
Minimizar un tipo de error condicionado al otro . . . 645
Criterio minimax cuando los priors son desconocidos . . . 646
Recomendaciones prácticas . . . 646
Conclusiones . . . 646

63. Modelos de Mezcla Gaussiana **649**
Motivación: ¿por qué un solo gaussiano no basta? . . . 649
El modelo de mezcla gaussiana . . . 650
El algoritmo EM . . . 651
Laboratorio: EM paso a paso . . . 653
Tipos de covarianza . . . 654
Selección del número de componentes . . . 654
GMM como clasificador vs. K-Means . . . 655
GMM como modelo generativo . . . 655
Consideraciones prácticas . . . 656

XXRedes Neuronales **659**

64. Redes Neuronales para Clasificación **661**
El Perceptrón . . . 661
Funciones de Activación . . . 662
Red Neuronal Multicapa (MLP) . . . 663
Retropropagación del Error . . . 665

Optimización . . . 667
Laboratorio: entrena una red en el navegador . . . 668
Implementación desde Cero . . . 668
Redes Convolucionales (CNN) . . . 669
Regularización en Redes Neuronales . . . 670
Más alla: Tipos de Redes . . . 671
Resumen . . . 673

65. Implementación Práctica: PyTorch y JAX **675**
El ecosistema de aprendizaje profundo . . . 675
Dataset a explorar . . . 677
Preparación de datos . . . 677
PyTorch: fundamentos . . . 678
Clasificación con PyTorch . . . 680
CNN con PyTorch . . . 682
JAX: fundamentos . . . 684
Clasificación con JAX puro . . . 686
CNN con Flax . . . 690
Comparativa PyTorch vs JAX . . . 690
Ejercicios . . . 693
Resumen . . . 695
Referencias . . . 696

XXII Práctica Integradora **697**

66. Clasificadores Supervisados: Experimento Práctico **699**
Introducción . . . 699
Dataset: Breast Cancer Wisconsin . . . 699
Flujo general de un experimento de clasificación . . . 700
Preparación de datos . . . 700
Clasificadores: intuición, sesgos y hiperparámetros . . . 701
Recordatorio de métricas . . . 703
Ajuste de thresholds de decisión . . . 705
Comparación entre modelos . . . 706
Buenas prácticas para evaluación . . . 706
Errores comunes en experimentos de clasificación . . . 707
Retos prácticos del flujo de clasificación . . . 707
Conclusión . . . 708

XXIII Aprendizaje No Supervisado **709**

XXIV Fundamentos del Aprendizaje No Supervisado **711**

67. Aprendizaje no supervisado **713**
Supervisado y no supervisado . . . 713
Las tareas del aprendizaje no supervisado . . . 714

Intuición: ocho clientes, ¿cuántos grupos? . . . 715
Construcción paso a paso . . . 715
Formalización . . . 717
Laboratorio: la escala crea la estructura . . . 719
El flujo de trabajo no supervisado . . . 720
Un mapa de esta parte . . . 720
Implementación en Python . . . 721
Resumen . . . 722

68. Similitud y distancias **723**
Intuición: ¿quién se parece más a Ana? . . . 723
Construcción paso a paso . . . 723
Formalización . . . 726
Laboratorio: la forma de la distancia . . . 729
Implementación en Python . . . 729
Guía para elegir una distancia . . . 730
Resumen . . . 731

XXXVI Clustering por Particiones **733**

69. K-medias y K-medoides **735**
Intuición: ubicar dos almacenes . . . 735
Construcción paso a paso . . . 735
Formalización . . . 737
Laboratorio: el algoritmo de Lloyd . . . 740
K-medoides: un representante real y robusto . . . 740
Variantes a escala . . . 741
Implementación en Python . . . 742
Ventajas y limitaciones . . . 743
Resumen . . . 743

70. Validación de clustering **745**
Intuición: dos maneras de partir seis puntos . . . 745
Construcción paso a paso . . . 746
Formalización . . . 749
Laboratorio: la silueta punto por punto . . . 751
Implementación en Python . . . 752
Resumen . . . 753

XXXVII Clustering Jerárquico y por Densidad **755**

71. Clustering jerárquico **757**
Intuición: cinco sucursales . . . 757
Construcción paso a paso . . . 758
Formalización . . . 760
Laboratorio: construye el dendrograma . . . 762
Implementación en Python . . . 762

Cómo leer (y no leer) un dendrograma 764
Resumen 765

72. DBSCAN, OPTICS y HDBSCAN **767**
Intuición: reportes de fallas en el alumbrado 767
Construcción paso a paso 768
Formalización 770
Laboratorio: DBSCAN paso a paso 771
OPTICS: todas las densidades a la vez 771
Laboratorio: el gráfico de alcanzabilidad 772
HDBSCAN: elegir la densidad de cada grupo 772
Comparación de algoritmos de clustering 773
Implementación en Python 773
Resumen 775

XXXI Detección de Anomalías **777**

73. Detección de anomalías **779**
Intuición: una lectura que no está fuera de rango 779
Construcción paso a paso 780
Formalización 782
Laboratorio: cuatro detectores, cuatro ideas de rareza 785
Implementación en Python 785
Guía de elección 786
Resumen 787

XXXII Reducción de Dimensionalidad **789**

74. Escalamiento multidimensional e Isomap **791**
Intuición: reconstruir un mapa a partir de distancias 791
Construcción paso a paso 792
Formalización 794
Laboratorio: desenrollar una espiral 796
Implementación en Python 797
Resumen 799

75. t-SNE, UMAP y autoencoders **801**
Intuición: ¿quién es vecino de quién? 801
Construcción paso a paso 801
Formalización 803
Laboratorio: t-SNE en el navegador 806
Implementación en Python 806
Cuándo usar cada método 808
Resumen 808

XL. Práctica Integradora **811**

76. Práctica integradora: segmentar pingüinos **813**
1. Propósito y datos . . . 813
2. Preparación: la escala . . . 813
3. ¿Hay grupos? Tendencia de agrupamiento . . . 814
4. Representación: PCA y UMAP . . . 814
5. ¿Cuántos grupos? . . . 815
6. Comparar algoritmos . . . 815
7. Abrir la caja: validación externa . . . 816
8. Perfilar los grupos . . . 816
9. Estructura dentro de los grupos . . . 816
10. Casos raros . . . 817
11. ¿Y si agregamos variables categóricas? . . . 817
Laboratorio: las variables deciden los grupos . . . 817
12. Lo que aprendimos . . . 817
Ejercicios . . . 818

XLI. Minería de Textos **819**

77. Minería de Textos: de las palabras a los vectores **821**
Intuición: cuatro reseñas de Café Norte . . . 821
Construcción numérica paso a paso . . . 822
Formalización . . . 825
Laboratorio: TF-IDF y similitud entre reseñas . . . 826
Tareas sobre la representación vectorial . . . 827
Laboratorio: temas con NMF . . . 829
Implementación en Python . . . 830
Más allá de la bolsa de palabras . . . 834
Ventajas y limitaciones . . . 835

Apéndices **837**

A. Presentaciones de clase **837**

B. Banco de Proyectos **841**
1. Value bets en Liga MX . . . 841
2. Priorización de contratos públicos sospechosos . . . 842
3. Detección temprana de réplicas sísmicas anómalas . . . 842
4. Calidad del aire y “días de contingencia” en CDMX . . . 843
5. Fraude en reembolsos/transacciones fintech . . . 843
6. Segmentación de incidentes de inseguridad por alcaldía/colonia . . . 844
7. Reseñas falsas o manipuladas en e-commerce . . . 844
8. Predicción de retrasos y saturación en el Metro/transporte público . . . 845
9. Deserción escolar en educación media superior . . . 845
10. Discurso de odio/polarización en debates electorales . . . 846

C. Glosario Unificado de Términos de Datos y Computación en la Nube **847**
A . . . 847
B . . . 847
C . . . 848
D . . . 848
N . . . 848
S . . . 848

D. Preparación para Entrevistas Analíticas **849**
La Importancia del Proceso de Pensamiento . . . 849
El Marco de Resolución de Problemas en 4 Pasos . . . 849
Preguntas de Práctica Comunes y Guías de Respuesta . . . 850

E. Bibliografía y Recursos Adicionales **853**
Referencias Bibliográficas . . . 853
Documentación Oficial y Guías de Tecnologías . . . 853
Conjuntos de Datos y Casos de Estudio . . . 854

References **855**

Prefacio

Este curso nació originalmente en el semestre 2026-II y se ha ido actualizando y enriqueciendo con el paso del tiempo. En la actualidad, está fuertemente orientado a sentar las bases sólidas del *Data Engineering* y *Data Science*, con el objetivo primordial de dotar a los estudiantes de herramientas que son directamente aplicables a la industria. A lo largo del texto encontrarás ejemplos basados en casos de uso reales y prácticas guiadas con tecnologías modernas como PySpark y BigQuery.

En las tarjetas a continuación podrás explorar el contenido general del curso. Aunque la estructura está diseñada con un orden de aprendizaje progresivo sugerido, los capítulos están construidos de manera autocontenida, lo que te permitirá consultar directamente los temas que más te interesen.

Temario

Requisitos previos

Orientado a estudiantes de Ciencias de la Computación, Matemáticas, Física y carreras afines.

Agradecimientos

Quiero expresar mi profundo agradecimiento a la profesora **Jessica Santizo** por su invaluable invitación a participar de manera inicial en este curso, así como por sus invaluables contribuciones y discusiones que han sido fundamentales para el mejoramiento continuo de este material.

Un agradecimiento muy especial a **las y los alumnos** que han cursado la materia; sus dudas, comentarios y retroalimentación constante han fortalecido enormemente la calidad y claridad del contenido en estas notas.

A la **Universidad Nacional Autónoma de México (UNAM)**, nuestra gran casa de estudios; un baluarte invaluable del conocimiento libre y accesible para todos.

Finalmente, extiendo mi gratitud a los creadores de los diversos cursos y materiales en línea de múltiples entidades, comunidades y empresas tecnológicas. Gracias a su libre disposición y compromiso con el conocimiento abierto, ha sido posible la estructuración y realización de estas notas.

Autor

Cómo citar

Si utilizas este material en tus trabajos, proyectos o investigaciones, por favor utiliza la siguiente cita en formato BibTeX:

```
@misc{villalba2026almacenes,
  title={Almacenes y Miner\'{i}a de Datos},
  author={Villalba, Diego},
  year={2026},
  howpublished={Repositorio de GitHub / GitHub Pages},
  note={Notas de curso de autor\'{i}a propia},
  url={https://diegoviillalba.github.io/almacenamienes-y-mineria-de-datos/}
}
```

Parte I.

Entendimiento de Negocio

Parte II.

Fundamentos y Stakeholders

1. Gestión de Requerimientos y Stakeholders

En la mayoría de los cursos de minería de datos y almacenes de datos, el punto de partida suele ser una base de datos limpia o un algoritmo de aprendizaje supervisado como árboles de decisión o regresión logística. Sin embargo, en el mundo real, los proyectos analíticos no comienzan con datos listos para ser procesados. Comienzan con una necesidad de negocio, una pregunta vaga o un problema operativo expresado por personas que no necesariamente entienden de algoritmos o de modelado dimensional.



El éxito de un proyecto de datos no se mide por la precisión del modelo (por ejemplo, un AUC de 0.95), sino por el **impacto económico o estratégico** que genera en la organización. Por ello, la fase inicial de cualquier proyecto de datos exitoso, el **Entendimiento del Negocio** (fase inicial de la metodología CRISP-DM que estudiaremos más adelante en el curso (Chapman et al. 2000)) es la más crítica. Antes de escribir la primera línea de código o importar librerías de modelado, debemos comprender en profundidad qué busca solucionar la organización. Para ejecutar esto con éxito, es fundamental dominar la comunicación con los **stakeholders** (partes interesadas) y traducir sus necesidades cualitativas en problemas técnicos cuantitativos estructurados.

Identificación de Partes Interesadas (*Stakeholders*)

Definición (stakeholder o parte interesada). De acuerdo con el marco del *Community Tool Box* de la Universidad de Kansas (Center for Community Health and Development, University of Kansas 2023), una parte interesada es cualquier persona, grupo u organización que puede verse afectada por un proyecto, o que puede influir directa o indirectamente en su desarrollo y éxito. Esta definición retoma la teoría clásica de los *stakeholders* propuesta por Freeman (1984)

> y ampliamente adoptada en la gestión moderna de proyectos.

En proyectos de analítica y minería de datos, identificar a las partes interesadas permite comprender quiénes utilizarán los resultados, quiénes podrían verse afectados y quiénes tienen la capacidad de orientar o decidir el rumbo del proyecto. Siguiendo las técnicas de identificación y análisis descritas por Bryson (2004), las partes interesadas pueden estudiarse mediante dos criterios complementarios: su relación con la organización y su papel dentro del proyecto.

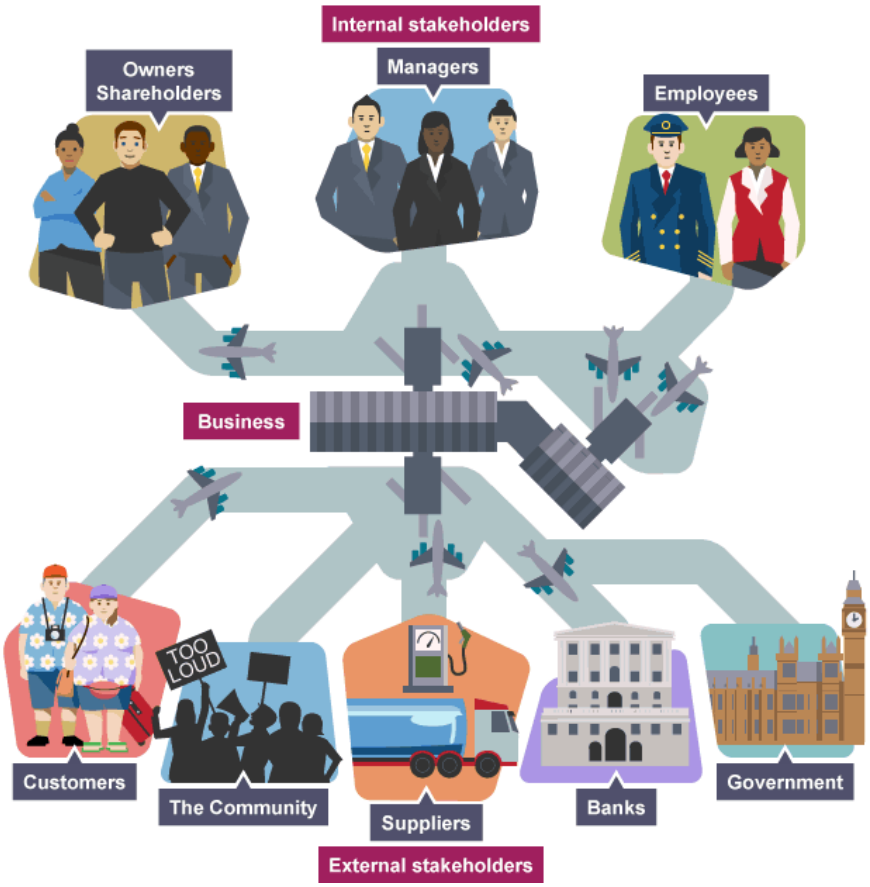
Clasificación según su relación con la organización

Una manera de clasificar a los stakeholders es según su relación con la organización responsable del proyecto.

Clasificación según su papel en el proyecto

Esta segunda clasificación no sustituye a la anterior. Una misma parte interesada puede ser interna o externa y, al mismo tiempo, primaria, secundaria o clave.

Ejemplo: un aeropuerto internacional



En la operación de un **aeropuerto internacional** pueden identificarse partes interesadas internas, como la autoridad aeroportuaria, el personal de seguridad y los responsables operativos; y partes interesadas externas, como los pasajeros, las aerolíneas, los proveedores, las empresas de transporte, los organismos gubernamentales y los inversionistas.

Si se desarrolla un sistema analítico para predecir retrasos, los pasajeros, pilotos y operadores podrían considerarse partes interesadas primarias; los proveedores de servicios y las empresas de transporte terrestre, partes interesadas secundarias; y la autoridad aeroportuaria, los organismos reguladores y los patrocinadores del proyecto, partes interesadas clave.

La clasificación dependerá del alcance del proyecto y de la manera en que cada actor resulte afectado o pueda influir en sus decisiones.

Análisis de Stakeholders: La Matriz de Poder e Interés

Para gestionar las expectativas de manera eficiente y evitar que la falta de aliniación descarrile un proyecto de datos, se utiliza la **Matriz de Poder vs. Interés** (también conocida como Matriz de Influencia/Interés), originalmente propuesta por Mendelow (1981) y adoptada como práctica estándar de gestión de interesados por el *PMBOK Guide* (Project Management Institute 2017). Esta herramienta clasifica a los stakeholders en una cuadrícula basada en dos dimensiones clave:

1. **Poder (Influencia)**: La capacidad de cambiar la dirección del proyecto, asignar recursos, aprobar entregables o cancelarlo por completo.
2. **Interés**: El nivel de atención y preocupación que tiene el stakeholder sobre el impacto del proyecto en su área.

A continuación se presenta el diagrama de la matriz de alineación estratégica para proyectos de análisis de datos:

Estrategias de Gestión por Cuadrante

- **Alto Poder, Alto Interés (Gestionar de Cerca)**: Deben participar activamente en las fases de planificación del proyecto, validar los entregables intermedios y recibir reportes de avance periódicos.
- **Alto Poder, Bajo Interés (Mantener Satisfecho)**: Involucrarlos en puntos críticos. Es fundamental garantizar que el proyecto cumple con las normativas de seguridad, gobernanza y gobernanza de nube.
- **Bajo Poder, Alto Interés (Mantener Informado)**: Mantenerlos entusiasmados e informados sobre cómo el modelo les facilitará el trabajo. Es vital pedir su retroalimentación temprana.
- **Bajo Poder, Bajo Interés (Monitorear)**: Monitorear pasivamente con comunicación general.

Definición de alcance y traducción de preguntas de negocio

Uno de los errores más frecuentes en los proyectos de analítica consiste en aceptar una pregunta de negocio tal como fue formulada y comenzar a trabajar con los datos de inmediato. Expresiones como *"queremos evitar que los clientes se vayan"* o *"necesitamos aumentar las ventas"* comunican una preocupación legítima, pero todavía no constituyen un objetivo analítico.

La función del analista no es únicamente extraer datos o construir modelos. También debe actuar como **traductor entre las necesidades del negocio y las capacidades analíticas de la organización**. Esto implica transformar una inquietud general en una pregunta concreta, medible y vinculada con una decisión.

La definición de alcance —o *scoping*— es el proceso mediante el cual se aclaran:

- el problema que se desea resolver;
- la decisión que se tomará con el resultado;
- la población o unidad de análisis;
- el periodo relevante;
- las métricas de éxito;
- las restricciones operativas;
- y el formato de entrega esperado.

Solo después de establecer estos elementos tiene sentido evaluar qué datos están disponibles y qué metodología podría utilizarse.

Esta traducción constituye el punto de partida de todo proyecto analítico. Una vez comprendida la necesidad del negocio, el equipo puede estudiar los datos disponibles, prepararlos, realizar el análisis, evaluar si los resultados responden a la pregunta original y, cuando sea necesario, desplegar una solución en producción. Por tanto, la definición de alcance no es una formalidad previa al análisis: es la etapa que orienta todas las decisiones posteriores.

Del problema de negocio al objetivo analítico

Una pregunta cualitativa debe atravesar varias capas de definición antes de convertirse en un objetivo técnico. El propósito no es reemplazar el lenguaje del stakeholder por terminología estadística, sino construir una especificación compartida que conecte el análisis con una acción concreta.

En este ejemplo, el resultado del proceso no es todavía la selección de un algoritmo. Decidir anticipadamente que se utilizará XGBoost, Random Forest o cualquier otra técnica puede limitar el análisis antes de comprender el problema. La metodología debe elegirse después de confirmar el objetivo, revisar los datos y establecer cómo se evaluará la utilidad de la solución.

Refinamiento iterativo mediante preguntas

La primera pregunta del stakeholder rara vez contiene todo el contexto necesario. Por ello, la definición de alcance debe tratarse como una **conversación iterativa** y no como un cuestionario que se responde una sola vez.

Una estrategia útil consiste en avanzar por capas. Cada respuesta permite formular una pregunta más precisa, detectar supuestos y regresar al stakeholder para validar el entendimiento. El ciclo termina cuando ambas partes pueden expresar con claridad qué se analizará, para qué se utilizará y cómo se reconocerá un resultado útil.

Estrategias para formular mejores preguntas

Durante este ciclo pueden combinarse varias estrategias de indagación:

1. **Preguntar por el contexto antes que por la solución.**
   En lugar de comenzar con *"¿qué dashboard necesitas?"*, conviene preguntar *"¿qué situación motivó esta solicitud?"*. Esto evita confundir el formato de entrega con el problema que debe resolverse.
2. **Conectar el análisis con una decisión.**
   Preguntas como *"¿qué harías de manera diferente si tuvieras esta información?"* permiten determinar si el análisis tendrá un uso concreto. Si no existe una decisión asociada, es posible que la solicitud todavía sea demasiado general.
3. **Convertir conceptos ambiguos en definiciones operativas.**
   Términos como *cliente activo*, *abandono*, *éxito*, *rentabilidad* o *bajo desempeño* pueden tener significados diferentes entre áreas. El analista debe preguntar cómo se observan y calculan en la práctica.
4. **Delimitar población, periodo y nivel de detalle.**
   Es necesario precisar si el análisis se realizará por cliente, transacción, producto, sucursal o región, así como definir el periodo histórico y el horizonte futuro relevantes.
5. **Explorar restricciones y capacidad de acción.**
   No basta con detectar un comportamiento si la organización no puede intervenir sobre él. Conviene preguntar qué acciones están disponibles, cuál es su costo y sobre cuántos casos puede actuar el equipo.
6. **Reformular y validar.**
   Después de escuchar las respuestas, el analista debe resumir el entendimiento alcanzado: *"Entonces, lo que necesitamos es…"*. Esta reformulación permite que el stakeholder corrija interpretaciones antes de invertir tiempo en el análisis.

Ejemplo de una conversación de refinamiento

Supongamos que un stakeholder plantea la siguiente necesidad:

> "Queremos evitar que los clientes se vayan."

La conversación podría avanzar de esta manera:

- **Contexto:** ¿Qué comportamiento reciente indica que existe un problema de abandono?
- **Definición:** ¿Qué significa exactamente que un cliente "se vaya": cancelar, dejar de comprar o permanecer inactivo?
- **Población:** ¿El problema afecta a todos los clientes o a un segmento específico?

- **Horizonte:** ¿Con cuánta anticipación necesitamos identificar el riesgo?
- **Decisión:** ¿Qué acción se realizará cuando se detecte un cliente en riesgo?
- **Capacidad:** ¿A cuántos clientes puede contactar el equipo de retención cada semana?
- **Éxito:** ¿Cómo se medirá el resultado: cancelaciones evitadas, ingresos retenidos o retorno de la campaña?
- **Validación:** ¿La prioridad es detectar la mayor cantidad posible de clientes que podrían abandonar, aunque existan algunos falsos positivos?

Después de esta iteración, la pregunta original podría reformularse así:

> **Objetivo analítico:** identificar semanalmente a los clientes de suscripción mensual con mayor probabilidad de cancelar durante los próximos 30 días, utilizando su comportamiento de los últimos 90 días, para priorizar hasta 2,000 contactos del equipo de retención. La solución se evaluará por su capacidad para identificar cancelaciones reales y por los ingresos retenidos mediante la intervención.

Esta formulación ya establece una población, una ventana de observación, un horizonte de predicción, una capacidad operativa y una medida de valor. A partir de ella, el equipo puede investigar la disponibilidad de los datos y comparar distintas metodologías.

Ejemplos de traducción de preguntas de negocio

Pregunta del stakeholder	Traducción analítica	Posible metodología
¿Cómo podemos aumentar el ticket promedio de la tienda en línea?	Identificar productos que tienden a comprarse juntos y evaluar qué combinaciones podrían utilizarse en recomendaciones durante el proceso de compra.	Reglas de asociación —Apriori o FP-Growth— evaluadas mediante soporte, confianza y *lift*.
¿Cómo evitamos la fuga de clientes de suscripción mensual?	Estimar la probabilidad de cancelación durante los próximos 30 días a partir de la actividad reciente de cada cliente, con el fin de priorizar intervenciones.	Modelos de clasificación supervisada, evaluados con métricas coherentes con la capacidad y el costo de intervención.
Queremos campañas personalizadas, pero no sabemos cómo agrupar a los clientes.	Identificar grupos de clientes con patrones semejantes de recencia, frecuencia, valor monetario y comportamiento de compra.	Segmentación RFM y técnicas de agrupamiento como K-Means o DBSCAN, acompañadas de interpretación de perfiles.
Necesitamos saber si los descuentos de fin de año funcionaron.	Estimar el efecto de los descuentos sobre el margen neto y distinguirlo de cambios estacionales u otros factores simultáneos.	Análisis causal o cuasiexperimental, según la disponibilidad de grupos de comparación y datos históricos.

Estas metodologías son puntos de partida, no decisiones automáticas. La técnica definitiva dependerá de la calidad de los datos, los supuestos que puedan sostenerse y la forma en que el resultado será utilizado.

El proceso de solicitud de datos (*Data Request*)

Una vez validado el objetivo, el entendimiento alcanzado debe registrarse. El *Data Request* o solicitud de datos es el mecanismo formal mediante el cual las áreas operativas comunican una necesidad al equipo analítico y documentan su contexto, alcance y criterios de éxito.

> **Definición — Solicitud de datos (*Data Request*):** proceso formal, generalmente representado mediante un formulario o *intake ticket*, que captura la necesidad de negocio y la información necesaria para evaluar, priorizar y ejecutar un proyecto analítico.

El ticket no sustituye la conversación con el stakeholder. Su función es conservar los acuerdos alcanzados durante el proceso de refinamiento y reducir ambigüedades posteriores.

Elementos esenciales de un *Data Request Ticket*

1. **Problema o pregunta principal**
   ¿Qué situación se desea comprender, resolver o mejorar?
2. **Contexto de negocio**
   ¿Por qué surge la necesidad y por qué es relevante en este momento?
3. **Decisión o acción esperada**
   ¿Qué se hará de manera diferente a partir del resultado?
4. **Población y alcance**
   ¿Qué clientes, productos, transacciones, regiones o periodos se incluirán?
5. **Métricas de éxito**
   ¿Cómo se determinará si el análisis fue útil para el negocio?
6. **Fuentes de datos conocidas o sugeridas**
   ¿Existen tablas, archivos, sistemas o proveedores externos que podrían contener la información necesaria?
7. **Formato de entrega**
   ¿Se requiere un análisis exploratorio, un reporte, un dashboard, una lista priorizada, un experimento o un modelo integrado en un proceso operativo?
8. **Responsable de validación**
   ¿Quién confirmará las definiciones, los resultados y las decisiones derivadas del análisis?
9. **Restricciones y riesgos**
   ¿Existen límites de tiempo, privacidad, presupuesto, capacidad operativa o calidad de datos?
10. **Fecha de decisión**
    ¿Cuándo debe estar disponible el resultado para que todavía sea útil?

Cierre

La calidad de un proyecto analítico depende, en gran medida, de la calidad de la pregunta que lo origina. Un objetivo ambiguo puede producir un análisis técnicamente correcto pero irrelevante; un objetivo bien definido, en cambio, permite seleccionar datos, métricas y métodos de forma coherente.

Definir el alcance significa construir un entendimiento compartido entre el stakeholder y el equipo de datos. Para lograrlo, el analista debe preguntar, escuchar, reformular y validar de manera iterativa. El resultado de este proceso no es simplemente una pregunta más técnica, sino un vínculo explícito entre los datos, una decisión y un resultado de negocio.

Caso de Estudio Práctico: Alineación en TheLook eCommerce

Imagina que eres contratado como líder de datos en **TheLook eCommerce** (el caso de estudio que utilizaremos a lo largo del curso). El Director de Mercadotecnia te aborda con el siguiente requerimiento:

> *"Necesito que me entregues un reporte con todos los clientes y lo que compran porque las ventas de ropa de mujer han bajado este mes y queremos lanzar un descuento."*

Si actúas únicamente de manera reactiva, extraerás una consulta SQL pesada, generarás un archivo CSV de gigabytes con millones de filas y se lo enviarás por correo electrónico. **Este es un camino directo al fracaso del proyecto**: el stakeholder no podrá abrir el archivo debido a su tamaño, o pasará horas filtrándolo en Excel sin extraer ninguna conclusión clara.

La Conversación de Alineación Estratégica

- **Tú (Analista)**: *"Para afinar la extracción de datos, ¿cuál es el objetivo de la campaña de descuento? ¿Quieren recuperar clientes inactivos o incentivar a las clientas recurrentes a comprar categorías más caras?"*
- **Director (Stakeholder)**: *"Queremos incentivar a las clientas que ya compran regularmente vestidos de diseñador a que exploren nuestra nueva línea de accesorios de lujo de alto margen."*
- **Tú (Analista)**: *"Excelente. En lugar de todos los clientes, extraeré un reporte enfocado en el segmento con alta frecuencia de compra en la categoría de mujer/vestidos. Además, aplicaré un modelo de asociación para identificar qué accesorios compran junto con los vestidos para sugerir combinaciones específicas en la campaña."*

Resultado: Redujiste el esfuerzo técnico de extraer bases de datos enteras, entregaste un análisis de alto impacto enfocado en conversión de alto margen, y alineaste las expectativas de negocio desde el primer día.

2. Roles en el Equipo de Datos y Ciclo de Vida de los Datos

En organizaciones modernas, los proyectos analíticos raramente son desarrollados por una sola persona. A medida que el volumen de datos crece y las tecnologías se especializan, surge la necesidad de contar con equipos multidisciplinarios. De acuerdo con el marco de gobierno de datos DAMA-DMBOK (DAMA International 2017) y con la caracterización del rol del científico de datos hecha por Davenport y Patil (2012), existen cuatro perfiles principales, cada uno con responsabilidades, herramientas y productos específicos.

Comparativa de Roles en la Ingeniería y Ciencia de Datos

Rol	Definición y Objetivos	Responsabilidades Principales	Herramientas Comunes
Data Engineer *(Ingeniero de Datos)*	Construye y mantiene la infraestructura física y lógica necesaria para que los datos fluyan de forma segura y eficiente en la empresa.	• Diseña tuberías de datos (pipelines ETL/ELT). • Administra el Data Warehouse y Data Lake. • Optimiza bases de datos.	PySpark, SQL, dbt, Apache Airflow, Docker, Snowflake, BigQuery.
Data Analyst *(Analista de Datos)*	Limpia, organiza e interpreta datos históricos para extraer conclusiones operativas y apoyar la toma de decisiones del negocio.	• Diseña reportes y tableros interactivos (dashboards). • Realiza análisis exploratorios básicos. • Responde a solicitudes de datos operativas (tickets).	SQL, Tableau, Power BI, Python (Pandas/Polars), Excel, BigQuery.
Data Scientist *(Científico de Datos)*	Utiliza matemáticas, estadística y algoritmos de Machine Learning para realizar predicciones o automatizar decisiones complejas basadas en datos.	• Desarrolla modelos de regresión, clasificación y clustering. • Realiza experimentos de validación estadística (Pruebas A/B). • Optimiza hiperparámetros.	Python (Scikit-Learn, PyTorch, XGBoost), R, MLflow, Jupyter Notebooks.

Rol	Definición y Objetivos	Responsabilidades Principales	Herramientas Comunes
Data Architect *(Arquitecto de Datos)*	Define la estructura de datos general de la organización, creando el mapa (blueprint) de integración de datos entre sistemas de la empresa.	• Diseña el esquema conceptual de la base de datos empresarial. • Define políticas de modelado dimensional y gobierno. • Alinea la infraestructura con la estrategia corporativa.	Herramientas de modelado de datos (Erwin, dbdiagram.io), Cloud Architectures.

Las prácticas de modelado dimensional que emplea el Arquitecto de Datos, y que retomaremos en la Parte I del curso, siguen la metodología clásica de Kimball y Ross (2013).

A continuación se detalla gráficamente el flujo de trabajo lógico entre los distintos perfiles de datos:

El Ciclo de Vida de los Datos

Definición (Ciclo de Vida de los Datos). Secuencia de fases por las que pasa la información dentro de una organización, desde su planificación inicial hasta su eventual eliminación segura, pasando por su ingesta, almacenamiento y análisis (DAMA International 2017).

El ciclo de vida de los datos describe la secuencia de fases por las que pasa la información dentro de una organización. Este ciclo es fundamental para estructurar el desarrollo del curso: la **Parte I (Almacenes de Datos)** abarca principalmente las fases de captura y almacenamiento, mientras que la **Parte II (Minería de Datos)** se enfoca en el análisis estratégico y predictivo.

Tanto la administración de la infraestructura (fase de almacenamiento) como la analítica aplicada (fase de exploración y modelado) no ocurren de manera aislada. Se integran dentro de un **proceso secuencial e iterativo para proyectos de datos**. Antes de construir cualquier modelo o base de datos, debemos pasar por etapas que nos ayudan a comprender la viabilidad del proyecto: el entendimiento del negocio, la recolección e inspección de datos, el preprocesamiento, y la posterior evaluación de resultados para garantizar el éxito de su despliegue real en producción. Esta aproximación estructurada es la que diferencia a los proyectos analíticos profesionales de las pruebas de concepto académicas.

El ciclo consta de **seis etapas clave** (Google 2022):

1. **Planificar (Plan)**: Determinar los requisitos de datos para resolver el problema de negocio, definir fuentes necesarias e identificar el equipo necesario.
2. **Capturar (Capture)**: Recolectar datos de fuentes internas y externas (bases de datos transaccionales, logs web, APIs). En esta fase los datos son extraídos y preparados.

3. **Gestionar (Manage)**: Almacenar los datos de manera estructurada en bases de datos relacionales o almacenes de datos analíticos. Aquí se aplican políticas de seguridad, limpieza inicial y linaje de datos.
4. **Analizar (Analyze)**: Explorar los datos mediante estadística y minería de datos (EDA), y entrenar modelos analíticos para responder preguntas clave o automatizar procesos.
5. **Archivar (Archive)**: Mover los datos históricos que ya no se consultan frecuentemente a sistemas de almacenamiento pasivos de bajo costo (ej. Glacier o Google Cloud Coldline Storage), protegiéndolos contra modificaciones accidentales.
6. **Destruir (Destroy)**: Eliminar de manera definitiva los datos una vez que ha expirado el periodo requerido de retención de datos, garantizando que no queden residuos ni copias inseguras de datos personales (PII) bajo políticas de borrado seguro.

Fases Críticas de la Gestión de Datos

Independientemente de en qué etapa del ciclo de vida nos encontremos, dos preguntas atraviesan todo el proceso: **qué tan sensibles son los datos que estamos manejando** y **quién tiene permitido acceder a ellos**. Ninguna de las dos se resuelve de forma improvisada; ambas se gobiernan mediante políticas explícitas que el equipo de datos debe conocer antes de tocar una sola tabla.

Datos que Requieren Tratamiento Especial

No todos los datos representan el mismo nivel de riesgo si se exponen o se usan indebidamente. Ciertas categorías —conocidas como **datos sensibles** o de categoría especial— exigen controles adicionales de recolección, almacenamiento y acceso:

- **Información de identificación personal (PII)**: nombre, dirección, número de identificación oficial, correo electrónico o cualquier dato que permita identificar a una persona de forma directa o indirecta.
- **Datos de salud**: historiales clínicos, diagnósticos o resultados de laboratorio; en México, protegidos explícitamente como datos personales sensibles por la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (Congreso de los Estados Unidos Mexicanos 2010).
- **Datos financieros y biométricos**: números de tarjeta, historial crediticio, huellas dactilares o reconocimiento facial.

Por ello, antes de capturar cualquiera de estos tipos de datos se aplica el **principio de minimización**: recolectar únicamente lo estrictamente necesario para resolver la pregunta de negocio, en lugar de acumular información sensible "por si acaso" que solo incrementa el riesgo sin aportar valor analítico.

Gobernanza, Política de Uso y Control de Acceso

La forma más común de mitigar estos riesgos es trabajar con una **política de uso de datos** (*data usage policy*): un documento formal que define qué datos existen, quién puede consultarlos, bajo qué condiciones y con qué finalidad. En las empresas de hoy en día, esta política suele ser diseñada por el **Data Architect** en conjunto con las áreas legal y de seguridad, y se traduce operativamente en:

- **Control de acceso de mínimo privilegio**: cada usuario o rol recibe una capacidad de acceso específica, limitada únicamente a los datos indispensables para su función —un *Data Analyst* puede consultar ventas agregadas por región, pero no necesariamente el número de tarjeta de un cliente individual— implementado mediante políticas de Gestión de Identidad y Accesos (IAM) y control de acceso basado en roles (RBAC).
- **Encriptación**: de la información en reposo y en tránsito.
- **Catálogo de datos**: un inventario actualizado que documenta qué campos son sensibles, quién es su responsable (*data owner*) y qué reglas de retención les aplican, evitando interpretaciones erróneas entre equipos.

Este principio de mínimo privilegio —otorgar a cada usuario el acceso indispensable, ni uno más— es hoy el estándar de facto en la industria: es tan relevante para el Data Engineer que configura permisos en Snowflake o BigQuery como para el Data Analyst que solicita acceso a una tabla del Data Warehouse.

Actividad Guiada: Diseñar un Plan de Ciclo de Vida para TheLook eCommerce

Como actividad introductoria práctica para este curso, diseñarás el plan inicial de datos para la empresa ficticia **TheLook eCommerce**, basándote en la solicitud de requerimientos de la mesa directiva de los departamentos de Mercadotecnia y Ventas.

Escenario de Negocio

TheLook eCommerce ha crecido rápidamente gracias a su oferta multicanal. Sin embargo, su infraestructura de datos actual es caótica. El equipo de Mercadotecnia ha identificado cuatro preguntas de negocio principales que necesitan responder para lanzar campañas efectivas el próximo trimestre.

Ejercicio de Alineación de Objetivos y Planificación de Datos

Para cada pregunta del negocio, asociaremos su objetivo analítico e identificaremos los datos necesarios que deben capturarse y gestionarse en nuestro futuro Data Warehouse.

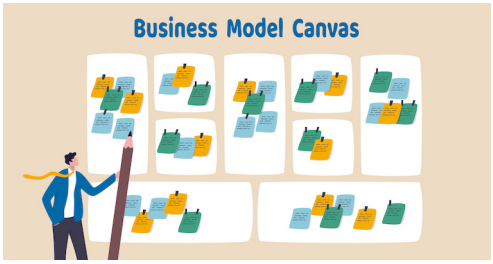
#	Pregunta de Negocio (Stakeholder)	Objetivo Analítico	Datos Necesarios a Capturar
1	*"¿Qué productos deberíamos promocionar juntos en nuestra página principal?"*	Descubrir reglas de asociación entre categorías y productos comprados de forma conjunta por los usuarios.	• Historial de pedidos (`orders` y `order_items`). • Identificador de producto e historial de clics en la sesión de navegación.
2	*"¿Cómo podemos optimizar los niveles de inventario de vestidos de mujer en cada bodega regional?"*	Predecir la demanda futura semanal de productos específicos por ubicación geográfica.	• Registros de stock diario en almacén (`inventory_items`). • Dirección de entrega de compras históricas y códigos postales.
3	*"¿Cuál es el valor acumulado a largo plazo de los clientes adquiridos a través de anuncios pagados en Google?"*	Calcular el Valor de Vida del Cliente (*Customer Lifetime Value* - CLV) agrupado por canal de adquisición.	• Tablas de usuarios (`users`) con fecha de registro y canal de procedencia. • Transacciones históricas totales asociadas a cada identificador único de usuario.
4	*"¿Cómo detectamos y prevenimos devoluciones fraudulentas de mercancía de alto costo?"*	Desarrollar un clasificador de transacciones sospechosas antes del procesamiento de la devolución.	• Historial de devoluciones por usuario. • Métodos de pago y tiempos transcurridos entre entrega y solicitud de devolución.

Parte III.

Planeación del Proyecto

3. Planeación Real de un Proyecto: El Business Understanding Canvas

En el capítulo anterior aprendiste a identificar stakeholders, ubicarlos en la matriz de poder e interés, y a traducir preguntas cualitativas en objetivos analíticos. Sin embargo, en un proyecto real esa traducción no ocurre de forma espontánea dentro de tu cabeza: se **documenta**. Un equipo de datos profesional deja evidencia escrita de cada conversación con el negocio, y condensa esa evidencia en un artefacto de una sola página que cualquier miembro del equipo —o cualquier stakeholder nuevo— pueda leer en cinco minutos y entender exactamente qué se va a hacer, por qué, y cómo se sabrá si funcionó.



Ese artefacto es el **Business Understanding Canvas**. Es la salida tangible de la primera fase de cualquier metodología estructurada de minería de datos (formalizarás esto más adelante como la fase de *Business Understanding* de **CRISP-DM** (Chapman et al. 2000; Shearer 2000; Wirth y Hipp 2000)). Aquí lo construiremos de forma práctica, aplicado a un caso real, para que cuando llegues al marco metodológico completo ya tengas la intuición de qué problema resuelve cada pieza.

> **Definición (Business Understanding Canvas).** Documento de una página que resume el objetivo de negocio, los criterios de éxito, los supuestos y riesgos, y el plan de un proyecto de datos, sirviendo como contrato de entendimiento entre el equipo analítico y los stakeholders antes de tocar una sola fuente de datos. El formato de una página se inspira en el lenguaje visual de los *canvas* estratégicos popularizado por Osterwalder y Pigneur (2010), aplicado aquí a la fase de entendimiento del negocio de CRISP-DM.

En este capítulo construirás las cinco piezas que componen la documentación de entendimiento de negocio de un proyecto:

1. Una **bitácora de reunión** con el stakeholder.
2. El **Business Understanding Canvas** de una página.
3. De **3 a 4 preguntas de investigación priorizadas**.
4. El **criterio de éxito en términos de negocio**.
5. Una **lista de supuestos y riesgos** (con al menos dos riesgos reales y su mitigación).

Lo haremos continuando el caso de **TheLook eCommerce** introducido en el capítulo anterior, donde el Director de Mercadotecnia solicitó impulsar la nueva línea de accesorios de lujo entre las clientas frecuentes de vestidos de diseñador.

Paso 1: La Bitácora de la Reunión con el Stakeholder

Antes de sintetizar nada, se documenta la conversación tal como ocurrió. La bitácora (*meeting log*) no es un resumen embellecido: es un registro fiel de quién dijo qué, qué se acordó y qué quedó pendiente. Sirve como fuente de verdad si más adelante hay desacuerdos sobre el alcance del proyecto.

Elementos mínimos de una bitácora

- **Metadatos**: fecha, hora, duración, modalidad y lugar.
- **Participantes**: nombre, rol y su cuadrante en la matriz de poder/interés.
- **Contexto**: qué disparó la reunión.
- **Preguntas y respuestas**: el intercambio real de la conversación de descubrimiento.
- **Decisiones y acuerdos**: qué quedó explícitamente aceptado por ambas partes.
- **Pendientes y próximos pasos**: con dueño y fecha.

Ejemplo aplicado: Kickoff del proyecto "Accesorios de Lujo"

Bitácora de Reunión — Proyecto Accesorios de Lujo (TheLook eCommerce)

Fecha: 3 de agosto de 2026 · **Hora**: 10:00–10:40 (40 min) · **Modalidad**: Videollamada
Participantes:

Nombre	Rol	Cuadrante (Poder/Interés)
Director de Mercadotecnia	Patrocinador del proyecto	Alto Poder / Alto Interés
Gerente de Inventario	Provee datos de stock por bodega	Alto Poder / Bajo Interés
Analista de Datos (tú)	Traductor técnico	—

Contexto: Seguimiento a la solicitud inicial de reporte de clientes; se formaliza como proyecto tras la conversación de alineación del capítulo anterior.
Preguntas formuladas y respuestas:

- *"¿Qué significa 'éxito' para ustedes en este proyecto: más ingresos, más margen, o retención de clientas VIP?"* → **Director**: *"Margen. Los accesorios tienen 3x el margen de los vestidos; nos interesa que las clientas que ya gastan mucho añadan accesorios al carrito, no solo captar clientas nuevas."*
- *"¿Con qué frecuencia necesitan actualizar las recomendaciones: por campaña, semanal, en tiempo real?"* → **Director**: *"Por campaña está bien para el piloto. No necesitamos tiempo real todavía."*
- *"¿Hay restricciones de inventario que debamos considerar antes de recomendar un accesorio?"* → **Gerente de Inventario**: *"Sí. Varias piezas de la nueva línea tienen stock limitado en algunas bodegas regionales; no queremos promocionar algo que se agote en 48 horas."*
- *"¿Existe un presupuesto o fecha límite fija?"* → **Director**: *"Necesitamos resultados del piloto antes de la campaña de fin de año, así que en 6 semanas debemos tener algo accionable."*

Decisiones y acuerdos:

- El piloto se enfoca en clientas con alta frecuencia de compra en la categoría vestidos/diseñador (no en toda la base de clientes).
- El éxito se mide en **margen incremental**, no en volumen de ventas ni en métricas de modelo.
- El Gerente de Inventario comparte semanalmente el archivo de stock por bodega para filtrar recomendaciones de productos con baja disponibilidad.

Pendientes / Próximos pasos:

Acción	Dueño	Fecha límite
Compartir acceso a tabla `inventory_items`	Gerente de Inventario	5 de agosto de 2026
Entregar canvas de entendimiento de negocio para validación	Analista de Datos	7 de agosto de 2026
Validar canvas y dar luz verde a extracción de datos	Director de Mercadotecnia	10 de agosto de 2026

Paso 2: El Business Understanding Canvas (Una Página)

El canvas condensa la bitácora en seis bloques que cualquier persona del negocio o del equipo técnico puede revisar en minutos. Estos seis bloques no son arbitrarios: corresponden directamente a las tareas que la guía CRISP-DM (Chapman et al. 2000) define para la fase de Business Understanding —determinar objetivos de negocio, evaluar la situación, determinar objetivos de minería de datos y

producir un plan de proyecto—, desdobladas aquí en sus criterios de éxito de negocio y técnico. A diferencia de la bitácora —que es cronológica y detallada— el canvas es **atemporal y sintético**: es el documento que se archiva y se consulta durante todo el proyecto.

Observa que el bloque 5 (Éxito Técnico) usa una métrica de modelo (*lift*, confianza), mientras que el bloque 2 (Criterio de Éxito de Negocio) usa una métrica financiera (margen incremental). Esta distinción es intencional y se profundiza en el siguiente paso.

Paso 3: Preguntas de Investigación Priorizadas

Rara vez existe una sola pregunta analítica detrás de un proyecto. Es tarea del analista descomponer el objetivo de negocio en varias preguntas de investigación y **priorizarlas** según su impacto en el negocio y la factibilidad de responderlas con los datos disponibles, siguiendo la misma lógica de la matriz de poder/interés del capítulo anterior, ahora aplicada a preguntas en vez de personas.

#	Pregunta de Investigación	Impacto de Negocio	Factibilidad de Datos	Prioridad
1	¿Qué accesorios se compran junto con vestidos de diseñador con mayor frecuencia?	Alto	Alto	**1 (Piloto)**
2	¿Qué accesorios recomendar cuando el stock regional es limitado, sin frustrar la demanda?	Alto	Medio	**2**
3	¿El efecto de venta cruzada varía según la región o bodega de origen del pedido?	Medio	Alto	**3**
4	¿Las clientas que reciben la recomendación aumentan su frecuencia de compra a largo plazo (no solo el ticket)?	Alto	Bajo (requiere datos longitudinales aún no disponibles)	**4 (Fuera de alcance del piloto)**

La pregunta 1 concentra el esfuerzo del piloto porque combina alto impacto con datos ya disponibles en el Data Warehouse. La pregunta 4, aunque valiosa, requiere una ventana de observación más larga y queda documentada para una segunda iteración, no descartada.

Paso 4: Criterio de Éxito en Términos de Negocio

Un error común de analistas junior es definir el éxito exclusivamente con métricas de modelo (precisión, *recall*, AUC, *lift*). Esas métricas son necesarias pero **no son el criterio de éxito del proyecto**: son el criterio de éxito del *modelo*. El stakeholder no puede evaluar si un AUC de 0.83 fue una buena inversión; sí puede evaluar si el margen subió. Esta distinción entre el objetivo de negocio y el objetivo de minería de datos es, según Provost y Fawcett (2013), la fuente más frecuente de proyectos analíticos que producen modelos técnicamente correctos pero comercialmente irrelevantes.

Tipo de Criterio	Ejemplo en este proyecto	¿Quién lo evalúa?
Negocio (el que importa para aprobar o cancelar el proyecto)	+8 % de margen incremental en el segmento piloto vs. grupo de control, en 6 semanas	Director de Mercadotecnia
Técnico (el que usa el equipo de datos para iterar el modelo)	*Lift* > 1.2 y confianza $> 30\,\%$ en las reglas de asociación	Analista de Datos

Regla práctica. Todo proyecto debe poder resumir su éxito en una sola oración que un directivo sin conocimientos técnicos pueda repetir de memoria. Si esa oración contiene un término estadístico (AUC, *lift*, silueta), probablemente estás describiendo el éxito técnico, no el de negocio.

Paso 5: Supuestos y riesgos

Todo canvas simplifica la complejidad del proyecto. Por ello, debe complementarse con una lista de **supuestos** y un registro de **riesgos**.

Un supuesto es una condición que se considera verdadera, aunque todavía no se haya verificado. Un riesgo es un evento incierto que podría afectar el desarrollo o los resultados del proyecto. Los supuestos deben validarse; los riesgos, evaluarse y mitigarse.

Supuestos

Para el piloto se consideran los siguientes supuestos:

- El comportamiento histórico de compra conjunta —por ejemplo, vestido y accesorio— representa razonablemente el comportamiento futuro durante la campaña.
- El archivo de inventario se actualizará semanalmente durante las seis semanas del piloto.

- El segmento de clientas frecuentes contiene suficientes transacciones para generar reglas de asociación confiables.
- El equipo de Mercadotecnia podrá incorporar las recomendaciones en su calendario de campaña.

Los supuestos más importantes deben comprobarse durante las primeras etapas. Si alguno resulta falso, puede ser necesario modificar los datos, el método o el alcance del proyecto.

Clasificación de los riesgos

Clasificar los riesgos permite asignarlos al equipo responsable y evita reducir el análisis únicamente a problemas técnicos. En un proyecto analítico pueden distinguirse cinco categorías principales:

- **Datos:** problemas de calidad, disponibilidad, actualización o representatividad.
- **Analíticos:** sesgo, estacionalidad, sobreajuste, asociaciones espurias o pérdida de desempeño.
- **Operativos:** falta de inventario, capacidad insuficiente o procesos que no pueden utilizar los resultados.
- **Tecnológicos y de seguridad:** fallas de integración, permisos excesivos, accesos no autorizados o interrupciones de los sistemas.
- **Legales, éticos y reputacionales:** uso indebido de datos personales, discriminación o personalización percibida como invasiva.

También pueden existir riesgos de adopción cuando los usuarios no comprenden, no confían o no utilizan la solución.

Evaluación de los riesgos

Cada riesgo debe evaluarse según su **probabilidad** y su **impacto**. Esta valoración permite priorizar los riesgos que requieren atención inmediata.

Además de la mitigación, conviene registrar:

- la persona responsable;
- la señal que indicaría que el riesgo está ocurriendo;
- el control preventivo;
- y el plan de contingencia.

Registro de riesgos del proyecto

#	Clasificación	Riesgo	Probabilidad	Impacto	Mitigación
1	Privacidad	La extracción de información de clientas VIP podría exponer datos personales fuera del equipo autorizado, incumpliendo la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (Congreso de los Estados Unidos Mexicanos 2010).	Media	Alto	Eliminar identificadores innecesarios, aplicar controles IAM/RBAC y mantener registros de acceso.
2	Operativo	El modelo podría recomendar accesorios sin inventario suficiente.	Alta	Medio	Validar las recomendaciones contra `inventory_items` antes de cada campaña y establecer un nivel mínimo de existencias.
3	Analítico	Las asociaciones podrían estar influenciadas por una temporada específica y no repetirse durante el piloto.	Media	Medio	Comparar las reglas en distintas ventanas temporales antes de utilizarlas.

#	Clasificación	Riesgo	Probabilidad	Impacto	Mitigación
4	Datos	El archivo de inventario podría estar incompleto o desactualizado.	Alta	Alto	Automatizar controles de actualidad y suspender el envío si la fuente no supera las validaciones.
5	Adopción	Mercadotecnia podría no utilizar las recomendaciones si no comprende su origen o utilidad.	Media	Medio	Explicar cada recomendación, capacitar a los usuarios y recopilar retroalimentación durante el piloto.

Tratamiento y seguimiento

Después de evaluar un riesgo, el equipo puede:

1. **Evitarlo:** modificar el diseño para eliminar su causa.
2. **Mitigarlo:** reducir su probabilidad o impacto.
3. **Transferirlo:** compartir su gestión con un proveedor o área especializada.
4. **Aceptarlo:** reconocerlo y preparar una contingencia.

El registro debe revisarse durante el piloto y actualizarse cuando cambien los datos, el modelo o las condiciones operativas.

> Un modelo puede ser técnicamente correcto y, aun así, resultar inutilizable si emplea datos sin autorización, recomienda productos agotados o genera resultados que la operación no puede atender. Por ello, los riesgos deben identificarse desde el entendimiento del negocio y revisarse durante todo el proyecto.

Resumen de la gestión de riesgos

Actividad: Construye tu Propio Business Understanding Canvas

Usando la plantilla en blanco a continuación, elige un escenario de negocio (puede ser uno de los cuatro casos de TheLook eCommerce del capítulo anterior, u otro que definas con tu instructor) y completa las cinco piezas de este capítulo:

1. Escribe una bitácora simulada de una reunión de 30-40 minutos con el stakeholder, incluyendo al menos tres preguntas de descubrimiento y sus respuestas.
2. Completa el canvas de seis bloques.
3. Prioriza entre 3 y 4 preguntas de investigación usando la tabla de impacto/factibilidad.
4. Redacta el criterio de éxito de negocio en una sola oración, y su contraparte técnica.
5. Documenta al menos dos riesgos reales (no genéricos) con su mitigación.

Este documento será la base sobre la que construirás el proyecto de datos a lo largo del curso: cada decisión posterior de modelado en las Partes III a V deberá poder justificarse trazándola de vuelta a alguno de estos cinco elementos.

Parte IV.

Almacenes de Datos

Parte V.

Arquitectura de almacenes de Datos

4. Introducción

La Arquitectura en Ciencias de la Computación

En Ciencias de la Computación (CS) y en la Ingeniería de Software, el término **arquitectura** no se refiere al diseño estético de un edificio físico, sino a la definición y diseño de las estructuras que componen un sistema digital complejo.

> **Definición (Arquitectura de Sistemas de Datos)**. La organización estructural de los sistemas de hardware, software y almacenamiento que permiten capturar, transformar, organizar y consultar datos a gran escala, definiendo los flujos de información y los límites operativos entre cada componente del sistema.

Una buena arquitectura de datos garantiza que la información esté disponible para quien la necesita, en el formato adecuado, de forma segura y con un rendimiento predecible. Pero esa definición esconde una pregunta más incómoda: **¿por qué no basta con "guardar los datos en un disco y ya"?** ¿Por qué la industria ha invertido cuatro décadas diseñando y rediseñando la forma en que se persisten los datos analíticos?

La respuesta tiene poco que ver con estética o con moda tecnológica, y mucho con tres restricciones físicas muy concretas:

1. **Cómo está organizada la información sobre el disco/memoria determina qué tan rápido (y qué tan caro) es responder una pregunta sobre ella.**
2. **Qué tan rígido o flexible es el sistema de persistencia determina qué tipos de datos puede almacenar de manera eficiente, y qué tanto puede crecer.**
3. **Qué tan estrictas son las garantías de consistencia que ofrece el sistema determina qué tan caro es coordinar escrituras concurrentes.**

Toda arquitectura de almacenamiento —Data Warehouse, Data Lake o Data Lakehouse, que estudiaremos en los siguientes capítulos— es, en el fondo, una decisión de diseño distinta sobre estos tres ejes. Antes de nombrarlas, vale la pena entender *por qué* existen: cuál es su **sentido de ser** desde una perspectiva técnica.

Eje 1: El costo de leer datos — filas vs. columnas

Cuando un motor de base de datos ejecuta una consulta, el cuello de botella casi nunca es la CPU: es la cantidad de datos que hay que **leer desde disco** para responderla. La forma en que los registros

se distribuyen físicamente sobre el almacenamiento —fila por fila o columna por columna— cambia radicalmente cuánto hay que leer.

Definición (Almacenamiento orientado a filas / *Row-oriented*). Modelo de persistencia en el que todos los valores de un mismo registro (fila) se guardan de forma contigua en disco. Es el modelo clásico de los sistemas transaccionales (OLTP).

Definición (Almacenamiento orientado a columnas / *Column-oriented*). Modelo de persistencia en el que todos los valores de un mismo atributo (columna) se guardan de forma contigua en disco, separando cada columna de la tabla en su propio bloque físico. Es el modelo dominante en los sistemas analíticos (OLAP).

Visualización: mismo dato, dos disposiciones físicas distintas

Esta diferencia, que parece un detalle de implementación, tiene consecuencias enormes a escala:

- **Menos I/O**: si una tabla tiene 50 columnas y una consulta analítica solo necesita 3, un motor columnar lee, en teoría, el 6 % de los datos que leería un motor por filas.
- **Mejor compresión**: una columna contiene valores del mismo tipo y con baja variabilidad (por ejemplo, un mismo `área` se repite miles de veces), lo que permite algoritmos de compresión mucho más agresivos que comprimir filas heterogéneas.
- **Ejecución vectorizada**: al tener los valores de una columna contiguos en memoria, el CPU puede aplicar operaciones (sumar, comparar, filtrar) sobre bloques completos usando instrucciones SIMD, en lugar de procesar registro por registro.

Por eso los sistemas transaccionales (bases de datos operacionales de un e-commerce, un banco, una universidad) son **row-oriented** —necesitan escribir y leer registros completos rápidamente— mientras que los sistemas analíticos (Data Warehouse, Data Lakehouse) son **column-oriented**, o usan formatos de archivo columnares como **Parquet** u **ORC**, porque su carga de trabajo dominante son agregaciones sobre volúmenes masivos de filas.

OLTP vs. OLAP: dos cargas de trabajo distintas

La elección entre almacenamiento por filas o por columnas no es arbitraria: responde a que, desde los años 90, la industria reconoce dos tipos de carga de trabajo con necesidades opuestas.

Definición (OLTP / *Online Transaction Processing*). Carga de trabajo dominada por operaciones de escritura y lectura de registros individuales, con alta concurrencia y baja latencia por operación. Es el patrón de las aplicaciones operacionales.

Definición (OLAP / *Online Analytical Processing*). Carga de trabajo dominada por consultas de agregación y exploración sobre grandes volúmenes de filas, típicamente sobre pocas columnas a la vez, con menor concurrencia y tolerancia a mayor latencia por consulta.

Criterio	OLTP	OLAP
Objetivo	Registrar la operación del negocio	Analizar el histórico del negocio
Operación típica	`INSERT` / `UPDATE` de un registro	`SELECT ... GROUP BY` sobre millones de filas
Volumen por operación	Pocas filas, todas sus columnas	Muchas filas, pocas columnas
Almacenamiento natural	Orientado a filas	Orientado a columnas
Usuarios	Aplicación operacional (miles de transacciones concurrentes)	Analistas/BI (pocas consultas, pero pesadas)
Ejemplo	Registrar una venta en un POS	Calcular ventas promedio por región del último año

Esta tabla es la razón última por la que un **Data Warehouse** nunca se alimenta directamente de la base transaccional de producción: son dos cargas de trabajo que compiten por los mismos recursos y que, además, favorecen disposiciones físicas de almacenamiento opuestas.

Eje 2: El costo de la rigidez — datos homogéneos vs. heterogéneos

El segundo eje no tiene que ver con *cómo* se organiza el dato dentro de la tabla, sino con **qué tan dispuesto está el sistema de persistencia a aceptar datos que no encajan en una tabla**.

Una base de datos relacional clásica exige que, antes de insertar un dato, este se ajuste a un esquema fijo: columnas, tipos, longitudes. Esto es excelente para garantizar consistencia, pero es una barrera de entrada para datos que **no nacen tabulares**: un archivo de log, una imagen de un producto, un JSON anidado que cambia de estructura entre un evento y otro, una transcripción de audio.

Durante décadas, la única opción para ese tipo de datos era descartarlos, transformarlos forzosamente en tablas (perdiendo información en el proceso) o almacenarlos en sistemas de archivos completamente aislados de la infraestructura analítica.

El avance que habilitó a los sistemas modernos de persistencia a resolver esto fue la combinación de dos ideas:

- **Almacenamiento de objetos** (*object storage*, como Amazon S3 o Google Cloud Storage): un sistema de persistencia que no exige un esquema para aceptar un archivo, es económico por naturaleza (paga por byte, no por rendimiento de disco de alta gama) y escala de forma prácticamente ilimitada.
- **Formatos de archivo abiertos y autodescriptivos** (Parquet, Avro, JSON, ORC): archivos que llevan su propio esquema embebido, permitiendo que datos estructurados, semi-estructurados y no estructurados convivan en el mismo almacén sin perder la posibilidad de ser leídos e interpretados después.

Esta combinación es lo que permite que un **Data Lake** —y, más adelante, un **Data Lakehouse**— almacene en el mismo lugar una tabla de transacciones, un conjunto de imágenes de productos y un registro de eventos de clics de una aplicación web, sin que el sistema de almacenamiento le exija a ninguno de ellos ajustarse a una estructura predefinida antes de guardarse. El esquema, si se necesita, se interpreta **en el momento de la lectura**, no al momento de la escritura.

Escalar verticalmente vs. escalar horizontalmente

La flexibilidad de un sistema de persistencia también depende de cómo crece cuando el volumen de datos supera lo que un solo servidor puede manejar. Existen dos estrategias:

Definición (Escalamiento vertical / *Scale-up*). Estrategia que aumenta la capacidad de un sistema añadiendo más recursos (CPU, RAM, disco) a una única máquina. Es el patrón tradicional de las bases de datos relacionales.

Definición (Escalamiento horizontal / *Scale-out*). Estrategia que aumenta la capacidad de un sistema añadiendo más máquinas (nodos), distribuyendo los datos y el trabajo entre ellas. Es el patrón nativo del almacenamiento de objetos y del cómputo distribuido.

El escalamiento vertical tiene un techo físico y económico: existe un límite de CPU/RAM que un solo servidor puede alojar, y el costo de esas máquinas de gama alta crece mucho más rápido que linealmente conforme se acercan a ese límite. Es una restricción que las bases de datos relacionales de un Data Warehouse clásico heredan directamente.

El escalamiento horizontal, en cambio, permite crecer añadiendo nodos de hardware relativamente económico (*commodity hardware*), lo cual es lo que hace viable, en términos de costo, almacenar petabytes de datos heterogéneos en un Data Lake: el sistema no necesita un servidor cada vez más costoso, sino simplemente más espacio distribuido entre miles de discos comunes.

Eje 3: El costo de la consistencia — ACID vs. BASE

El tercer eje no tiene que ver con la velocidad de lectura ni con qué tipos de datos se aceptan, sino con **qué tan estrictamente garantiza el sistema que los datos siempre reflejen un estado correcto**, incluso cuando ocurren escrituras simultáneas o fallas a mitad de una operación.

Definición (ACID). Conjunto de garantías —*Atomicity* (atomicidad), *Consistency* (consistencia), *Isolation* (aislamiento) y *Durability* (durabilidad)— que aseguran que cada transacción se ejecute por completo o no se ejecute en absoluto, y que el sistema nunca quede en un estado intermedio o corrupto.

Definición (BASE). Conjunto de garantías más laxas —*Basically Available* (básicamente disponible), *Soft state* (estado flexible) y *Eventual consistency* (consistencia eventual)— que

> priorizan la disponibilidad y la capacidad de seguir operando bajo fallas o alta distribución, aceptando que distintos lectores puedan ver temporalmente versiones ligeramente distintas del dato.

Garantizar ACID tiene un costo real: exige coordinación (bloqueos, registros de transacciones, verificación de reglas) que se vuelve cada vez más cara conforme el sistema se distribuye en más nodos. Por eso los sistemas relacionales clásicos —diseñados para escalar verticalmente, en una sola máquina o un clúster pequeño y fuertemente acoplado— pueden ofrecer ACID de forma económica, mientras que los sistemas de almacenamiento de objetos masivamente distribuidos, pensados para escalar horizontalmente a miles de nodos, adoptaron tradicionalmente garantías BASE: aceptan una ventana de inconsistencia a cambio de disponibilidad y escala casi ilimitada.

Este es precisamente el problema que resolverá más adelante el **Data Lakehouse**: introducir una capa de metadatos transaccionales que reintroduce garantías ACID *sobre* un almacenamiento de objetos que, por diseño, nació BASE.

El sentido de ser de las arquitecturas de almacenamiento

Ningún sistema optimiza simultáneamente los tres ejes al máximo: **velocidad de consulta** (filas vs. columnas), **flexibilidad para persistir datos heterogéneos** (rigidez del esquema y del escalamiento) y **garantías de consistencia** (ACID vs. BASE) compiten entre sí, porque cada garantía adicional tiene un costo de rigidez o de cómputo.

Esa tensión de tres vías es exactamente el problema que cada arquitectura de datos intenta resolver con un balance distinto:

- Un sistema puede priorizar el **eje 1** y el **eje 3** al extremo —almacenamiento columnar y ACID estricto— aceptando datos únicamente estructurados a cambio de un rendimiento de consulta extraordinario y consistencia total.
- Un sistema puede priorizar el **eje 2** al extremo —escalamiento horizontal y BASE— aceptando cualquier tipo de dato y creciendo casi sin límite, a cambio de renunciar a garantías de consistencia inmediata.
- Un sistema puede intentar obtener lo mejor de los tres mundos, incorporando una capa adicional que reintroduzca control transaccional (ACID) sobre un almacenamiento de objetos flexible y escalado horizontalmente.

En los próximos capítulos estudiaremos, respectivamente, el **Data Warehouse**, el **Data Lake** y el **Data Lakehouse**: tres materializaciones concretas de estas decisiones de diseño. Lo importante, antes de entrar en el detalle de cada una, es no perder de vista que ninguna de estas arquitecturas es una moda: cada una existe porque resuelve, con distintas concesiones, el mismo trío de restricciones físicas de eficiencia que acabamos de describir.

5. Data Warehouse

Datos para la toma de decisiones

Las organizaciones modernas generan grandes volúmenes de datos diariamente: ventas, transacciones financieras, registros académicos, consultas médicas, interacciones digitales, entre otros.

Sin embargo, operar el negocio no es lo mismo que analizarlo. Los sistemas operacionales están diseñados para registrar transacciones de manera eficiente, pero no para responder preguntas estratégicas.

Por ejemplo:

- Un sistema de ventas permite registrar compras.
- Pero responder preguntas como:
    - ¿Qué producto se vende más en el último año?
    - ¿Cómo han evolucionado las ventas en los últimos cinco años?
    - ¿Qué sucursal tiene mejor desempeño?

requiere una estructura distinta, optimizada para análisis histórico, agregación y comparación temporal.

La diferencia puede resumirse como un cambio de perspectiva: los sistemas operacionales observan cada evento mientras ocurre; un Data Warehouse reúne esos eventos para explicar cómo cambia el negocio a lo largo del tiempo. El primero responde «¿qué acaba de pasar?»; el segundo ayuda a responder «¿qué patrón se está formando y qué decisión conviene tomar?».

En este contexto, resulta fundamental comprender que el análisis de datos no comienza directamente con la tecnología, sino con la definición clara del problema de negocio. Una de las metodologías más utilizadas para estructurar este proceso es **CRISP-DM** (*Cross Industry Standard Process for Data Mining*).

CRISP-DM propone un marco sistemático compuesto por seis fases:

1. **Comprensión del negocio**: identificar los objetivos estratégicos y traducirlos en preguntas analíticas concretas.

2. **Comprensión de los datos**: explorar las fuentes disponibles, evaluar calidad y detectar posibles inconsistencias.

3. **Preparación de los datos**: limpiar, transformar e integrar la información.

4. **Modelado**: aplicar técnicas analíticas o modelos estadísticos.

5. **Evaluación**: verificar si los resultados responden adecuadamente a los objetivos iniciales.

6. **Despliegue**: implementar los hallazgos en la operación o en sistemas de soporte a decisiones.



Figura 5.1.: Figura 1. Proceso CRISP-DM

Figura 1. Diagrama del proceso CRISP-DM. Fuente: Wikimedia Commons, "CRISP-DM Process Diagram", dominio público.

Es importante destacar que CRISP-DM **no es un proceso lineal**, sino un modelo iterativo. En la práctica, las fases se retroalimentan constantemente. Durante el modelado pueden detectarse problemas en la calidad de los datos que obliguen a regresar a la fase de preparación. En la evaluación puede descubrirse que el problema de negocio fue formulado de manera incompleta, lo que implica volver a la primera fase. Incluso después del despliegue, los resultados obtenidos suelen generar nuevas preguntas estratégicas, reiniciando el ciclo.

Esta naturaleza iterativa es coherente con la realidad organizacional: la toma de decisiones es un proceso continuo de refinamiento y aprendizaje. Así, antes de construir tablas, definir dimensiones o elegir herramientas tecnológicas, es necesario responder preguntas como:

- ¿Qué decisiones desea mejorar la organización?
- ¿Qué métricas son críticas?
- ¿Qué horizonte temporal se requiere analizar?

La arquitectura de datos debe ser flexible y escalable, permitiendo incorporar nuevas fuentes, nuevas métricas y nuevas dimensiones conforme evolucionan las necesidades del negocio. En otras palabras, la arquitectura de datos no es un fin en sí mismo; es el soporte técnico que permite convertir datos operacionales en conocimiento útil dentro de un ciclo continuo de mejora y toma de decisiones.

Desde esta perspectiva, CRISP-DM define el ciclo de trabajo analítico, mientras que el Data Warehouse aporta una memoria organizacional confiable para recorrerlo. No todos los proyectos de CRISP-DM requieren un Data Warehouse, pero cuando las preguntas exigen integrar varias fuentes, comparar periodos y reutilizar métricas consistentes, el almacén de datos se vuelve una pieza central.

Data Warehouse

William H. Inmon (1996) define un Data Warehouse como:

> "A data warehouse is a subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making process."

En español, un Almacén de Datos es una colección de datos orientada a temas, integrada, variante en el tiempo y no volátil, diseñada para apoyar la toma de decisiones.

Las cuatro propiedades de la definición no son características aisladas. Juntas explican por qué un Data Warehouse puede ofrecer una versión coherente y durable del negocio:

Un Data Warehouse es **orientado a temas**, lo que significa que no se organiza por aplicaciones o sistemas operacionales, sino por entidades relevantes para el negocio. Por ejemplo, en una compañía de seguros, el almacén de datos se estructura alrededor de conceptos como cliente, póliza o prima, y no por productos específicos como seguro de automóvil o seguro de vida. El enfoque está en analizar entidades del negocio y no módulos técnicos.

Es también **variante en el tiempo**, ya que mantiene información histórica. Los datos no se sobreescriben ni se actualizan como en un sistema transaccional; la nueva información se agrega al repositorio. Esto permite analizar tendencias a lo largo de cinco, diez años o más, y responder preguntas como cómo ha evolucionado el número de diagnósticos en los últimos años o qué patrones se observan en el crecimiento de ventas.

Además, es **integrado**, porque consolida información proveniente de múltiples sistemas. Por ejemplo, si un sistema registra el género como "m" y "f" y otro como 0 y 1, antes de integrarlos deben transformarse para mantener consistencia. Lo mismo ocurre con formatos de fecha distintos, monedas diferentes o nombres de productos inconsistentes. La integración implica procesos de limpieza, transformación y estandarización de datos.

Finalmente, es **no volátil**, lo que significa que se trata de un repositorio físicamente separado de los sistemas operacionales. En él se realizan principalmente operaciones de carga y consulta de datos, pero no actualizaciones frecuentes, eliminaciones ni transacciones. Su propósito es analítico y no operativo.

En conjunto, estas propiedades permiten que una métrica conserve el mismo significado al atravesar áreas y periodos. Por ejemplo, comparar la tasa de abandono de dos generaciones solo es válido si la identidad del estudiante, la definición de abandono y el calendario académico fueron integrados con reglas comunes, y si los valores históricos se conservaron.

Referencia:
Inmon, W. H. *Building the Data Warehouse*, 2nd ed., 1996.

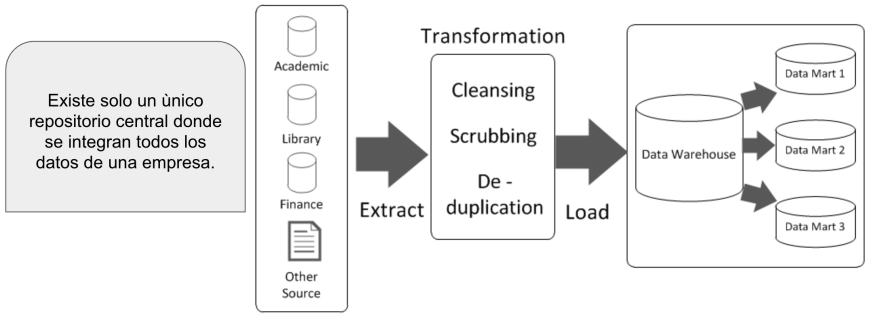
Arquitectura de un Data Warehouse

Un Data Warehouse no debe entenderse únicamente como un conjunto de tablas organizadas para consulta. En realidad, es una arquitectura integral de datos cuyo propósito es separar claramente la operación diaria del análisis estratégico. Esta arquitectura permite integrar información proveniente de múltiples sistemas, conservar historia y ofrecer estructuras optimizadas para la toma de decisiones.

La separación también protege ambos tipos de trabajo: las transacciones pueden continuar con baja latencia y las consultas analíticas pueden recorrer millones de registros sin competir directamente con la operación. Entre ambos mundos existe una cadena de transformación que documenta de dónde proviene cada dato y qué reglas se le aplicaron.

Modelo de 3 Capas

La arquitectura clásica de un Data Warehouse suele representarse mediante un modelo de tres capas lógicas. Estas capas no necesariamente corresponden a servidores físicos distintos, sino a niveles funcionales dentro del ecosistema de datos.



1. **Capa de fuentes (Operational Layer).**
   En este nivel residen los sistemas que soportan la actividad diaria de la organización. En una empresa pueden incluirse sistemas ERP que registran ventas y facturación, CRM que gestionan clientes, plataformas académicas que almacenan calificaciones o sistemas hospitalarios que documentan consultas médicas. También forman parte de esta capa archivos planos como CSV o Excel, APIs externas que proveen información pública y registros de aplicaciones (logs).

   El objetivo de esta capa es transaccional: registrar eventos de manera rápida, consistente y segura. No está diseñada para análisis histórico profundo ni para responder preguntas agregadas complejas. Por ejemplo, un sistema hospitalario puede indicar cuántos pacientes están siendo atendidos hoy, pero no está optimizado para analizar la evolución de diagnósticos durante los últimos diez años.

2. **Capa de integración (ETL / Data Integration Layer).**
   Esta capa constituye el núcleo técnico del Data Warehouse. En ella se ejecutan los procesos de extracción, limpieza, transformación e integración de datos. Es aquí donde se resuelven inconsistencias entre sistemas: un género registrado como “M/F” en un sistema y como “0/1” en otro debe unificarse; formatos de fecha distintos deben homogenizarse; monedas diferentes deben convertirse a un estándar común.

   Asimismo, se eliminan duplicados, se generan claves sustitutas y se aplican reglas de negocio que permiten consolidar información proveniente de múltiples fuentes. Dependiendo del enfoque arquitectónico adoptado, esta capa puede materializarse como un **Enterprise Data Warehouse (EDW)** central integrado, en el que se consolida toda la información corporativa antes de exponerla a los usuarios, o bien puede consistir en procesos que alimentan directamente modelos dimensionales específicos para cada área del negocio. En cualquier caso, su función es garantizar coherencia, calidad e integración de datos a nivel empresarial.

3. **Capa de consumo (Presentation / Analytics Layer).**
   En este nivel los datos ya han sido integrados y estructurados para facilitar el análisis. Aquí se implementan data marts, modelos dimensionales (como esquemas estrella), tableros de Business Intelligence y herramientas de visualización. Es el punto donde analistas, directivos y responsables de negocio interactúan con la información para responder preguntas estratégicas.

   Por ejemplo, en un grupo de clínicas esta capa permitiría consultar la evolución anual de pacientes por clínica, el tiempo promedio de espera por especialidad o la distribución de diagnósticos por región. La característica esencial es que los datos están optimizados para consulta, agregación y comparación temporal, no para registrar nuevas transacciones.

Data Mart

Un **Data Mart** es un subconjunto especializado del Data Warehouse orientado a un área específica del negocio. Mientras que el Data Warehouse integra información a nivel corporativo, el Data Mart concentra datos relevantes para un dominio particular, como ventas, finanzas, recursos humanos o trayectoria académica.

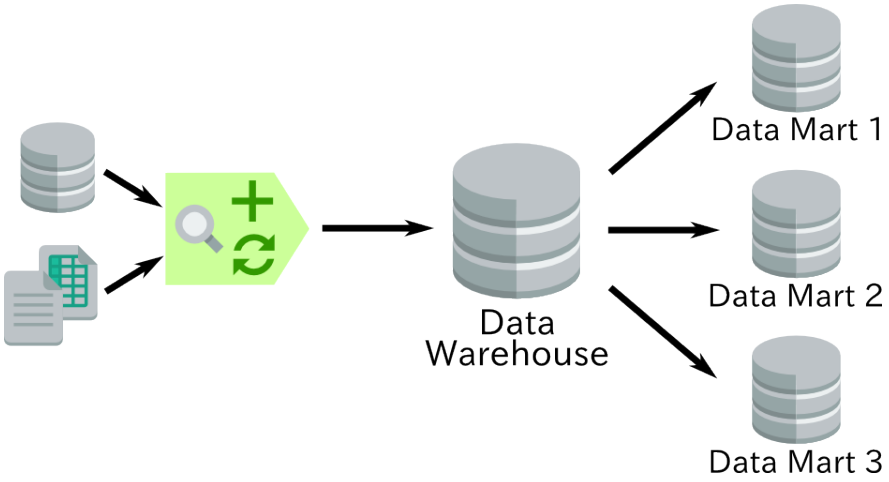


Figura 5.2.: Data Marts dentro de la arquitectura de un Data Warehouse

Por ejemplo, en una universidad puede existir un Data Mart académico que almacene hechos relacionados con inscripciones, calificaciones y créditos, junto con dimensiones como estudiante, asignatura y tiempo. En una empresa comercial, un Data Mart de ventas podría contener hechos de transacciones y dimensiones como producto, cliente y región.

El objetivo principal de un Data Mart es facilitar el análisis focalizado y mejorar el rendimiento de consultas dentro de un área concreta. Generalmente se implementa utilizando modelado multidimensional (por ejemplo, esquemas estrella), ya que este formato está optimizado para agregaciones y reportes.

Dependiendo de la estrategia arquitectónica adoptada, los Data Marts pueden construirse como derivaciones de un Data Warehouse central (enfoque top-down) o desarrollarse inicialmente por proceso de negocio e integrarse posteriormente mediante dimensiones compartidas (enfoque bottom-up).

Por tanto, un Data Mart no es simplemente un almacén pequeño. Su valor depende de que sus métricas y dimensiones mantengan coherencia con el resto de la organización. Un Data Mart de ventas y otro de inventario, por ejemplo, solo pueden analizarse juntos si ambos entienden «producto», «sucursal» y «tiempo» de la misma manera.

Data Warehouse vs Data Lake

En el contexto actual de gestión de datos, es importante mencionar brevemente el concepto de **Data Lake**, ya que suele confundirse con un Data Warehouse.

Un Data Lake es un repositorio centralizado que almacena datos en su formato original, ya sean estructurados, semiestructurados o no estructurados. A diferencia del Data Warehouse, que requiere procesos de limpieza e integración antes de almacenar la información, el Data Lake adopta un enfoque más flexible: primero almacena los datos y posteriormente los transforma cuando se necesitan para análisis.

Por ejemplo, un Data Warehouse almacenará ventas consolidadas y normalizadas listas para consulta. En cambio, un Data Lake puede almacenar registros crudos de sensores, imágenes médicas, archivos JSON, logs de aplicaciones o grandes volúmenes de texto sin procesar.

La diferencia clave radica en su propósito:

- El **Data Warehouse** está diseñado para análisis estructurado y toma de decisiones empresariales.
- El **Data Lake** está orientado al almacenamiento masivo y flexible de datos, especialmente útil en escenarios de Big Data y ciencia de datos.

En arquitecturas modernas, ambos pueden coexistir. El Data Lake actúa como repositorio de datos crudos, mientras que el Data Warehouse ofrece estructuras optimizadas para análisis empresarial.

Enfoques de implementación: Kimball vs Inmon

Existen dos enfoques clásicos para construir un DW completo: Kimball y Inmon. Ambos aparecen como referencias bibliográficas en el material del curso: Kimball como referente del modelado dimensional e Inmon como referente de la arquitectura de Data Warehouse.

Ambos enfoques persiguen información integrada y útil, pero comienzan en extremos diferentes. Kimball parte de procesos de negocio concretos y los conecta gradualmente; Inmon parte de un núcleo empresarial integrado y luego publica estructuras de consumo. La dirección del recorrido determina qué valor aparece primero y dónde se concentra la complejidad.

Enfoque Kimball (bottom-up, bus architecture)

El enfoque Kimball prioriza entregar valor temprano mediante data marts dimensionales (usualmente en estrella) construidos por procesos de negocio. La idea central es que distintos data marts se integren a través de dimensiones conformadas (conformed dimensions), es decir, dimensiones compartidas y coherentes (por ejemplo, la misma definición de “Tiempo” y “Producto” para ventas y devoluciones).

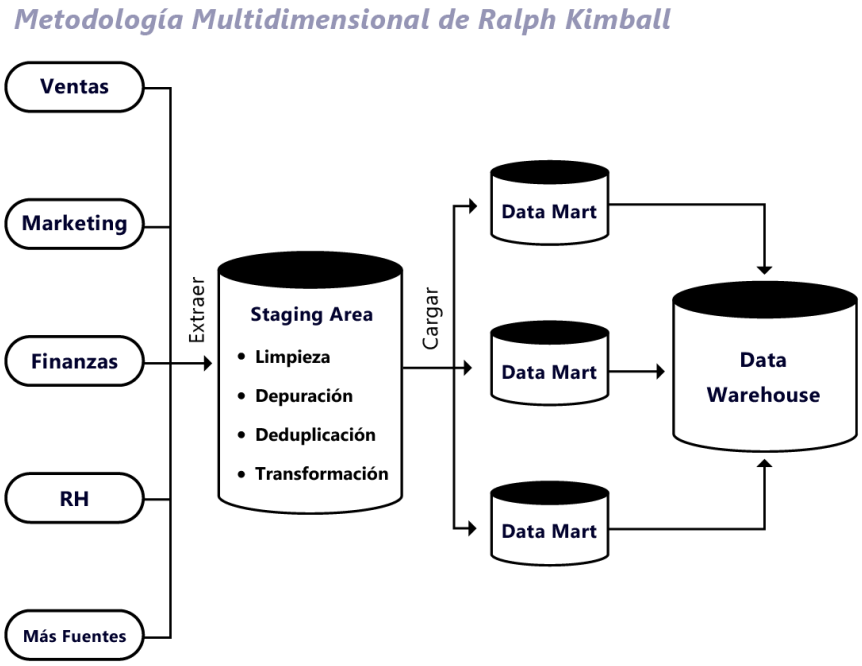


Figura 5.3.: Enfoque Kimball para construcción de Data Warehouse

Rasgos característicos:

- Implementación incremental: se construye un data mart (p. ej., Ventas) y se amplía gradualmente (p. ej., Inventario, Marketing).
- Modelo objetivo: dimensional, orientado a consulta.
- Énfasis en la capa semántica: definiciones de métricas consistentes (por ejemplo, qué significa "venta neta").
- Alta utilidad para BI/analítica desde etapas tempranas.

Ventajas:

- Entrega rápida de resultados a usuarios.
- Consultas simples y rendimiento alto.
- Ajuste natural a preguntas de negocio, que constituyen el punto de partida recomendado en el curso.

Riesgos o retos:

- Requiere disciplina fuerte para definir dimensiones conformadas; si no, se terminan construyendo "silos" (varios data marts incompatibles).
- El gobierno de datos y la integración pueden volverse complejos si se crece sin estándares.

Enfoque Inmon (top-down, EDW primero)

El enfoque Inmon propone construir primero un Enterprise Data Warehouse (EDW) central e integrado (frecuentemente en un modelo más normalizado), y después derivar data marts o estructuras dimensionales para consumo analítico.

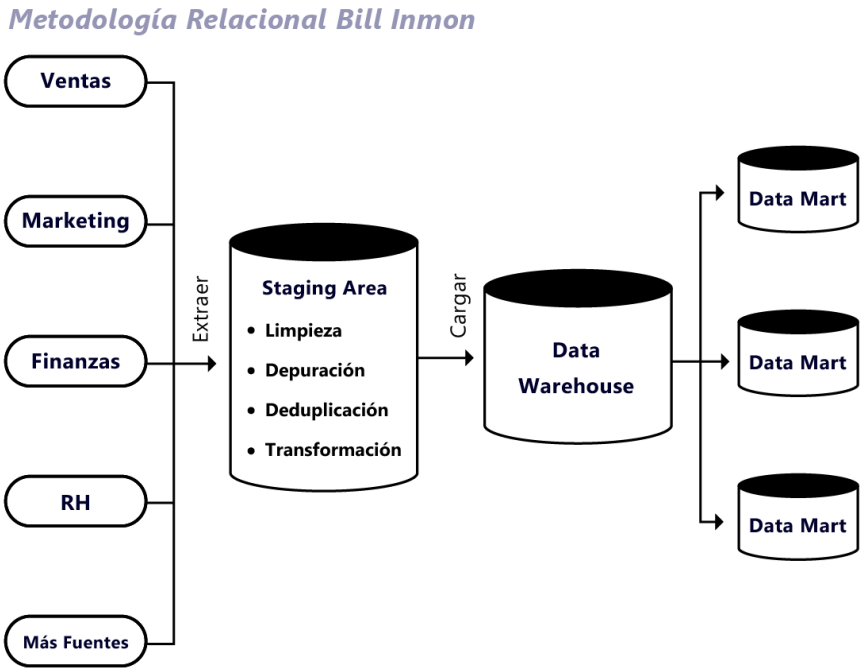


Figura 5.4.: Enfoque Inmon para construcción de Data Warehouse

Rasgos característicos:

- La integración empresarial es prioritaria: definiciones corporativas únicas desde el inicio.
- Se enfatiza un "repositorio central" donde se armonizan entidades y reglas (cliente, producto, organización).
- Los data marts o modelos dimensionales se construyen a partir del EDW, como vistas o capas derivadas.

Ventajas:

- Alta consistencia y control de definiciones a nivel corporativo.
- Favorece gobierno de datos, auditoría, linaje y estándares.

Riesgos o retos:

- Mayor tiempo hasta generar valor visible si el EDW se vuelve un proyecto grande.
- Es más sensible a cambios de requerimientos porque "primero se integra todo" y luego se habilitan casos de uso.

Cómo elegir adecuadamente entre Kimball e Inmon

La elección no debería plantearse como "cuál es mejor", sino "cuál es más adecuado" al contexto institucional, al horizonte de tiempo y a la madurez del gobierno de datos. Criterios prácticos:

1. Urgencia de valor analítico y recursos disponibles

- Si se necesita entregar reportes y tableros funcionales rápidamente, y el equipo es pequeño/mediano, Kimball suele ser más viable: se construye un primer data mart y se itera.
- Si la organización puede sostener una inversión inicial grande y su prioridad es consolidar definiciones corporativas desde el inicio, Inmon puede ser más apropiado.

2. Grado de heterogeneidad y fragmentación de fuentes

   - Cuando existen muchas fuentes dispares y hay conflictos fuertes de definición (por ejemplo, "cliente activo" significa cosas distintas en áreas distintas), un enfoque tipo Inmon ayuda a forzar una integración central.
   - Si las fuentes son manejables y el problema principal es acelerar el análisis por proceso de negocio (ventas, trayectoria académica, atención médica), Kimball suele funcionar bien.

3. Gobierno de datos, auditoría y trazabilidad

   - Si hay requerimientos estrictos de auditoría, linaje, cumplimiento o trazabilidad (por ejemplo, regulaciones y auditorías frecuentes), el enfoque top-down facilita un marco centralizado.
   - Si el gobierno de datos aún está en formación, un enfoque incremental puede ser más realista, siempre y cuando se establezcan estándares mínimos (dimensiones conformadas, glosario de métricas, versionado de reglas).

4. Naturaleza de los casos de uso

   - Casos muy orientados a métricas agregadas y navegación dimensional (tendencias por tiempo, comparativos por región, cohortes) se benefician de dimensionalidad desde el inicio, favoreciendo Kimball.
   - Casos de integración empresarial transversal (master data robusto, conciliación entre sistemas, integración de dominios complejos) favorecen Inmon o, al menos, una capa central fuerte.

En la práctica moderna es común un enfoque híbrido: se puede construir una capa integrada (tipo EDW o "core") y, en paralelo o después, exponer modelos dimensionales para BI. El punto esencial es mantener coherencia: definir claramente qué capa es "fuente de verdad" para definiciones y cómo se publican métricas y dimensiones para consumo analítico.

La elección, entonces, no termina en una etiqueta arquitectónica. Debe traducirse en decisiones verificables: quién define cada métrica, cómo se comparten las dimensiones, dónde se conserva la historia, cómo se rastrea el linaje y qué producto analítico se entregará primero.

Ejemplo práctico: Facultad de Ciencias

En un contexto universitario, el sistema de calificaciones registra eventos puntuales como la inscripción de un estudiante en una asignatura o la obtención de una nota al finalizar el semestre. Sin embargo, analizar la trayectoria académica completa de un estudiante requiere una visión mucho más amplia: integrar múltiples periodos, generaciones, asignaturas, planes de estudio y, en muchos casos, información administrativa y financiera asociada.

Los sistemas operacionales académicos están diseñados para soportar procesos diarios —inscripciones, captura de calificaciones, emisión de constancias— pero no para realizar análisis longitudinales o comparativos complejos. Por ejemplo, responder preguntas como:

- ¿Cómo ha evolucionado el desempeño promedio por generación en los últimos diez años?
- ¿Qué cohortes presentan mayores tasas de abandono?
- ¿Existe relación entre modalidad de ingreso y desempeño académico?

implica integrar información dispersa en distintos sistemas y conservar historia de manera estructurada.

Un **Data Warehouse académico** permite consolidar esta información en un repositorio histórico, integrado y consistente, diseñado específicamente para análisis estratégico.

Fuentes de datos

En una Facultad de Ciencias, la información relevante para análisis institucional suele encontrarse distribuida en múltiples sistemas independientes. Entre ellos pueden incluirse:

- Sistemas de control escolar que registran inscripciones, calificaciones y créditos.
- Sistemas de admisiones que almacenan información de ingreso, generación y antecedentes académicos.
- Sistemas administrativos y financieros que gestionan pagos, becas y apoyos.
- Sistemas de recursos humanos con información sobre profesores y carga docente.
- Archivos externos como encuestas institucionales o evaluaciones académicas.

Cada uno de estos sistemas cumple una función operativa específica. Sin embargo, ninguno por sí mismo ofrece una visión integral del desempeño académico institucional a lo largo del tiempo. La fragmentación de fuentes dificulta el análisis global y puede generar inconsistencias en métricas y definiciones.

Procesos de integración (ETL)

Para transformar datos operacionales en información estratégica es necesario un proceso de integración sistemático.

En esta etapa se extrae periódicamente la información de los distintos sistemas, se validan y corrigen inconsistencias, se homologan formatos y se consolidan registros que pertenecen a la misma entidad institucional. Por ejemplo, un estudiante puede aparecer con variaciones en su nombre entre sistemas distintos; el proceso de integración debe identificar que se trata de la misma persona y unificar su información.

Asimismo, se normalizan conceptos institucionales: nombres de carreras, códigos de asignatura, periodos académicos y categorías administrativas deben definirse de manera coherente para todo el repositorio.

El resultado de estos procesos no es simplemente una copia de los sistemas originales, sino un conjunto de datos consolidados, limpios y preparados para análisis histórico.

Consolidación en el Data Warehouse

Una vez integrados, los datos se almacenan en un repositorio central que conserva información histórica y garantiza consistencia institucional. Este repositorio no está orientado a registrar nuevas transacciones, sino a permitir consultas analíticas eficientes y comparaciones longitudinales.

Desde el punto de vista arquitectónico, el Data Warehouse académico se convierte en la base para construir vistas analíticas especializadas —como un data mart académico— desde donde pueden generarse reportes, tableros y estudios institucionales.

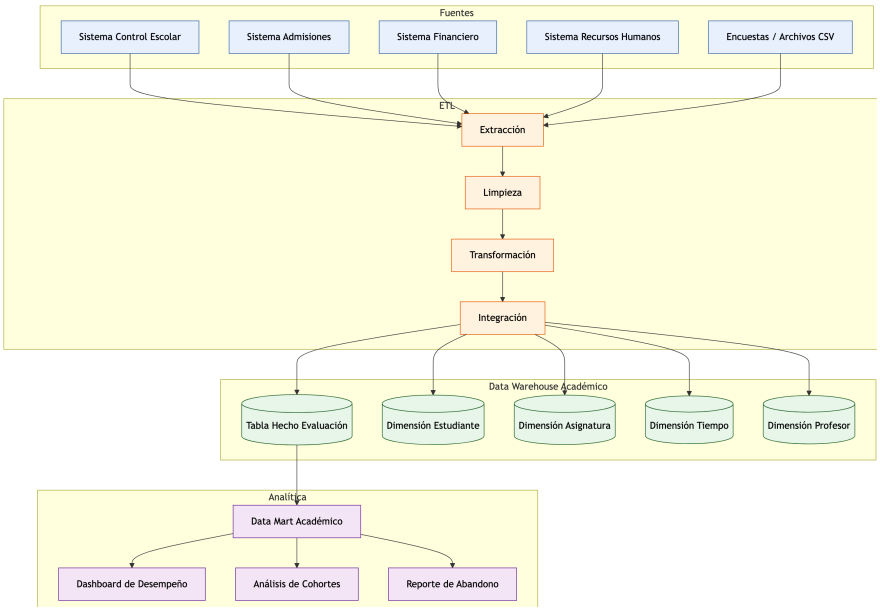
Gracias a esta consolidación, la Facultad puede:

- Analizar cohortes completas desde ingreso hasta egreso.
- Medir tasas de reprobación y abandono por generación.
- Identificar asignaturas críticas.
- Evaluar tendencias de desempeño a lo largo del tiempo.
- Integrar indicadores académicos con variables administrativas o financieras.

La diferencia fundamental respecto a los sistemas transaccionales es que el Data Warehouse ofrece una visión histórica, integrada y consistente del desempeño institucional, permitiendo transformar datos operacionales en conocimiento estratégico.

El diseño específico de las estructuras analíticas —como modelos dimensionales, tablas de hechos y dimensiones— se abordará en el siguiente capítulo, donde se detallará cómo se organizan técnicamente estos datos para optimizar su consulta.

Diagrama arquitectónico del DW académico



Con esta arquitectura, la Facultad podría analizar cohortes completas desde el ingreso hasta el egreso, medir las tasas de abandono por generación, detectar asignaturas críticas con alta reprobación, evaluar el impacto de las becas en el desempeño académico e identificar patrones longitudinales

de rendimiento. A diferencia del sistema transaccional, el Data Warehouse integra información que se encuentra dispersa en distintos sistemas, conserva el historial de los datos a lo largo del tiempo, permite realizar análisis multidimensionales y está optimizado para procesos de agregación y consulta. En este contexto, el Data Warehouse no reemplaza los sistemas académicos, sino que los complementa, transformando los datos operacionales en información estratégica que apoya la toma de decisiones institucionales.

Conclusión

Un Data Warehouse no sustituye a los sistemas operacionales ni registra transacciones cotidianas. Su función es integrar información dispersa, conservar historia, romper silos organizacionales y facilitar análisis complejos orientados a la toma de decisiones estratégicas.

En la siguiente sección se abordará el modelado multidimensional, que define cómo se estructuran las tablas analíticas dentro de esta arquitectura y cómo se optimizan para consultas agregadas.

Recapitulación visual: del registro a la analítica

Para cerrar el recorrido, retomemos el caso de la Facultad de Ciencias y observemos de forma compacta cómo un evento operacional adquiere contexto analítico al atravesar la arquitectura.

En un contexto universitario, el sistema de calificaciones registra eventos puntuales como la inscripción de un estudiante o la obtención de una nota. Sin embargo, analizar la trayectoria académica completa de un estudiante requiere una visión mucho más amplia: integrar múltiples periodos, generaciones, asignaturas, planes de estudio y, en muchos casos, información administrativa asociada.

Un **Data Warehouse académico** permite consolidar esta información en un repositorio histórico, integrado y consistente, diseñado específicamente para análisis estratégico.

Diagrama Arquitectónico del Data Warehouse Académico.

El siguiente diagrama muestra cómo fluye la información escolar, financiera y docente para conformar la analítica académica:

Con esta arquitectura, la Facultad puede analizar cohortes completas desde el ingreso hasta el egreso, medir las tasas de abandono por generación, detectar asignaturas críticas con alta reprobación e identificar patrones longitudinales de rendimiento.

6. Data Lakehouse

De silos a una base abierta y gobernada

Pregunta rectora

> ¿En qué momento compartir una base abierta y gobernada para BI, *streaming* y aprendizaje automático justifica asumir la operación adicional de un Lakehouse?

Un **Data Lakehouse** no es el destino obligatorio de toda plataforma de datos. En una organización pequeña, un Data Warehouse administrado puede entregar resultados con menos componentes y menos personal. El Lakehouse se vuelve razonable cuando la duplicación, el movimiento de datos, la variedad de cargas y la necesidad de reutilizar contratos comunes superan el costo de operar tablas, metadatos, catálogo, gobierno y mantenimiento.

Esta lectura sigue un caso de una *fintech* desde sus primeros reportes hasta una plataforma compartida. El propósito no es memorizar productos, sino aprender a separar responsabilidades y argumentar una decisión arquitectónica.

> **Ruta de lectura**
>
> Tiempo estimado: 35–45 minutos. Conviene haber leído antes el capítulo de Data Warehouse. Los diagramas usan siempre la misma convención: azul para datos y almacenamiento, morado para metadatos y gobierno, naranja para ingesta, gris para cómputo, verde para consumo confiable y rojo para riesgo o duplicación.

Objetivos de aprendizaje.

Al terminar, podrás:

1. explicar por qué un Data Warehouse fue una elección correcta durante la primera etapa de la *fintech*;
2. localizar duplicación, latencia y controles dobles en una arquitectura con silos;
3. distinguir objeto, *bucket*, formato de archivo, formato de tabla, catálogo, gobierno y motor de cómputo;
4. narrar cómo un *snapshot* evita que una escritura incompleta se vuelva visible;
5. justificar una transición usando volumen, variedad, velocidad, cargas de trabajo, costo y capacidad operativa;
6. reconocer que compactación, evolución de esquemas, observabilidad y radio de impacto siguen siendo responsabilidades del equipo.

El problema antes de la definición

A las 10:02, la *fintech* recibe la transacción `tx_8472` por `$3,200`. Finanzas la incluye en los ingresos del día, pero el sistema de fraude la presenta como bloqueada. Ambos equipos ejecutaron correctamente su propio proceso. La discrepancia está en las rutas y contratos que utilizaron.

La pregunta útil no es todavía «¿qué producto compraremos?», sino «¿qué trabajo se duplica para sostener estas dos rutas?».

Dos caminos que deben representar el mismo negocio

Históricamente, el Data Warehouse y el Data Lake resolvieron problemas legítimos:

- El **Data Warehouse** ofreció tablas controladas, SQL, concurrencia y reportes comprensibles para el negocio.
- El **Data Lake** permitió conservar grandes volúmenes y tipos diversos en almacenamiento de objetos económico y flexible.

El costo aparece cuando cada uno se convierte en una autoridad separada.

Si Producto agrega `device_risk_score`, el cambio debe capturarse, aterrizarse, transformarse, incorporarse a la tabla analítica, probarse, publicarse, autorizarse y comunicarse a consumidores. Cuando las rutas están separadas, varias de esas decisiones se repiten. La deuda no está sólo en los bytes duplicados: está en las decisiones duplicadas.

Caso Fintech, fase 1: el Warehouse era suficiente

Durante los años 0–3, la empresa tenía datos relacionales, unas 10,000 transacciones diarias, alrededor de 500 GB al año y reportes por lotes. Su necesidad dominante era conocer ingresos, morosidad y crecimiento con rapidez.

Un servicio administrado, dbt y una persona analista podían entregar valor en una semana y operar por cientos de dólares mensuales dentro del supuesto didáctico. Contratar un equipo especializado para un Lakehouse habría agregado observabilidad, integración y mantenimiento sin un retorno claro.

> **!** Principio de decisión
>
> Elegir un Data Warehouse en esta fase no fue falta de visión. Fue ajustar arquitectura, costo y capacidad del equipo al problema real.

Qué hace cada producto en esa fase.

- **PostgreSQL, MySQL o una base operacional administrada** registra usuarios, cuentas y pagos de la aplicación. Está optimizada para operar el negocio, no para escanear años de historia.
- **Fivetran** o **Airbyte** extrae o replica datos desde las fuentes hacia un destino analítico.
- **BigQuery, Snowflake, Amazon Redshift o Fabric Warehouse** puede cumplir el rol de Data Warehouse. Son alternativas; no es necesario contratar todos.

- **dbt** ejecuta transformaciones y pruebas principalmente mediante SQL sobre el Warehouse; no es el lugar principal donde viven los datos.
- **Metabase, Tableau, Power BI o Looker** presenta métricas y tableros a usuarios de negocio.

La marca no determina por sí sola la arquitectura. BigQuery puede ser un Warehouse de tablas nativas y también un motor que consulta tablas abiertas. Siempre pregunta: **¿qué almacena, qué cataloga y qué procesa?**

Un Data Warehouse moderno también puede ser elástico, admitir datos semiestructurados y consultar almacenamiento externo. La decisión se toma por requisitos, economía y capacidad operativa, no por presentar al Warehouse como una tecnología rígida por definición.

Año 4: el punto de quiebre

La arquitectura inicial no falla por recibir un JSON aislado. El cambio aparece por una combinación de presiones.

Los `4 TB/mes` equivalen aproximadamente a `48 TB/año`. Llegar a petabytes requeriría crecimiento y retención durante varios años; no debe afirmarse que cuatro terabytes mensuales ya sean un petabyte. Los montos del caso son supuestos docentes, no cotizaciones de proveedores.

Actividad de clasificación.

Síntoma	Fuerza principal
Millones de clics móviles	Volumen y velocidad
PDFs de contratos	Variedad
Modelo de fraude que necesita segundos	Velocidad y nuevos consumidores
Científicos que descargan terabytes	Movimiento
Dos equipos calculan la misma característica de riesgo	Coordinación
La factura de transformaciones se dispara	Economía y operación

La pregunta de diseño es: **¿escalamos el Warehouse, añadimos un Lake separado o rediseñamos una base compartida?** La respuesta debe empezar por requisitos, no por productos.

La base técnica del Lakehouse

Vocabulario mínimo

Término	Explicación cotidiana	Ejemplo en la fintech	Qué no significa
Objeto	Unidad guardada en la nube: bytes, nombre o *key* y metadatos.	PDF, imagen o archivo Parquet.	No es necesariamente una fila ni una tabla.

Término	Explicación cotidiana	Ejemplo en la fintech	Qué no significa
Bucket	Contenedor con nombre donde un proveedor guarda objetos.	`s3://fintech-datos-prod/`.	No es motor SQL, base relacional ni todo el Data Lake.
Key	Localizador único de un objeto dentro del bucket; puede parecer una ruta.	`bronze/kafka/2026/08/25/part-0001.json`.	Los separadores `/` no siempre representan carpetas tradicionales.
Data Lake	Arquitectura para conservar y procesar datos diversos sobre almacenamiento económico, con ingesta, organización, seguridad y gobierno.	Objetos, políticas, catálogo y procesos.	Un bucket aislado no se vuelve automáticamente un Data Lake.
Data Warehouse	Sistema analítico orientado a tablas, SQL y datos preparados para decisiones.	Tablas de ingresos y morosidad.	No es la base operacional que autoriza cada pago.
Motor de cómputo	Software que consulta, transforma, agrega o entrena.	BigQuery, Trino, Spark o Databricks SQL.	El almacenamiento de objetos no procesa consultas por sí solo.
Formato de archivo	Organización de bytes dentro de un archivo.	Parquet organiza columnas y compresión.	No coordina por sí mismo una tabla de muchos archivos.
Formato de tabla	Protocolo que decide qué archivos forman una tabla y conserva cambios, esquema y snapshots.	Iceberg, Delta Lake o Hudi.	No sustituye almacenamiento, catálogo ni motor.
Catálogo	Mapa que relaciona un nombre con ubicación, esquema, propietario y metadatos vigentes.	`fintech.silver.transacciones`.	Una lista no garantiza calidad sin procesos y responsables.

Una analogía breve ayuda a recordar las responsabilidades:

> El bucket es el contenedor; los objetos son los paquetes; la *key* es la etiqueta. Parquet organiza el contenido de ciertos paquetes. Iceberg, Delta Lake o Hudi mantienen el inventario versionado que dice qué paquetes forman una tabla. El catálogo permite pedir la tabla por nombre y verificar el acceso. El motor abre los paquetes y realiza el cálculo.

Anatomía de un bucket.

La URI `s3://fintech-datos-prod/documents/loan-8472.pdf` se interpreta así: `s3` identifica el

servicio, `fintech-datos-prod` es el bucket y el resto es la *key*. En Google Cloud se vería `gs://...`; en ADLS Gen2 se usa un contenedor dentro de una cuenta de almacenamiento.

Tres distinciones deben quedar explícitas:

1. **Bucket archivo.** El bucket contiene objetos; el PDF o Parquet es el objeto.
2. **Bucket Data Lake.** Un Data Lake incluye ingesta, organización, seguridad, catálogo, cómputo y operación.
3. **Bucket Lakehouse.** El Lakehouse añade tablas transaccionales, catálogo, gobierno y motores coordinados.

Una base física compartida todavía no es una tabla

El primer paso del Lakehouse es desacoplar los bytes del motor: guardar datos en objetos y asignar cómputo distinto a SQL, ML o *streaming*. Esa base ofrece escala, formatos abiertos y economía de capacidad. Sin embargo, una carpeta puede contener:

```
transactions/
  part-0001.parquet
  part-0002.parquet
  part-0003.parquet
  part-0003-retry.parquet
  _temporary-part-0004.parquet
```

El nombre de la carpeta no decide de forma confiable cuáles archivos pertenecen a la tabla, cuál fue un reintento y si la escritura terminó.

La idea que debe sobrevivir

No es «un lago junto a un almacén» ni «guardar Parquet en S3». Es hacer que conjuntos de archivos se comporten como tablas confiables sin perder apertura y separación entre almacenamiento y cómputo.

Responsabilidades lógicas de la arquitectura

Una implementación puede agrupar varias responsabilidades en un servicio administrado o distribuirlas entre herramientas. Las capas son lógicas, no una lista obligatoria de cinco productos.

1. Ingesta.

Conecta datos estructurados, semiestructurados y no estructurados. Las cargas **batch** mueven datos acumulados a intervalos; **CDC** replica cambios de una fuente; *streaming* procesa eventos a medida que llegan. La modalidad determina frescura, tamaño de archivos, reintentos y costo operativo.

2. Almacenamiento.

S3, Cloud Storage, ADLS Gen2 u OneLake conservan objetos a gran escala. Parquet y ORC son formatos columnares eficientes para datos tabulares; JSON, PDFs e imágenes pueden permanecer

como objetos. Usar un formato abierto reduce dependencia, pero no garantiza que todos los motores soporten las mismas operaciones.

3. Formato de tabla, catálogo y gobierno.

Iceberg, Delta Lake y Hudi coordinan archivos como tablas versionadas. El catálogo resuelve nombres, ubicaciones, esquemas y metadatos vigentes. El gobierno añade identidad, clasificación, permisos, auditoría y responsabilidades. Son tareas relacionadas, pero distintas.

4. Interfaces y motores.

SQL, APIs de DataFrame y APIs de metadatos permiten que motores como BigQuery, Trino, Spark o Databricks SQL procesen datos sin convertir el almacenamiento en el motor. La compatibilidad se verifica por operación, versión del formato y configuración.

5. Consumo.

BI necesita métricas certificadas; ciencia de datos necesita entidades reutilizables y datos autorizados; fraude necesita baja latencia; auditoría necesita historia y evidencias. Compartir la base no obliga a compartir un único motor ni a consultar la misma capa de calidad.

Archivo, tabla y catálogo no son sinónimos

Parquet no publica una transacción sobre veinte archivos. El formato de tabla mantiene la lista autorizada, el esquema, las particiones y los snapshots. El catálogo permite que un nombre estable apunte a esos metadatos. Iceberg, Delta Lake y Hudi comparten esa intención general, pero no son idénticos.

Cómo un snapshot oculta una escritura incompleta

El reto no es que S3, Cloud Storage o Azure Blob Storage necesariamente «pierdan consistencia». Una modificación de tabla puede crear muchos objetos; hace falta un **commit de tabla atómico** que publique el conjunto completo.

La atomicidad se logra porque los lectores consultan una lista autorizada. Si el job falla antes de publicar, el puntero sigue en el snapshot 17. Si termina, un commit mueve el puntero a la versión 18 completa. Un lector que resolvió la versión antes del commit continúa con una vista consistente según el aislamiento y el motor.

Las garantías ACID suelen expresarse por tabla. No debe suponerse automáticamente una transacción atómica entre muchas tablas; hay que verificar el formato y el motor.

Evolución de esquema sin reescribir toda la historia

Los metadatos pueden separar la identidad de una columna de su nombre visible.

Esto habilita evolución de esquema y viaje en el tiempo, pero no vuelve gratuitos todos los cambios. Modificar tipos incompatibles, semántica, particiones o unidades puede exigir migración, *backfill*, vistas de transición y pruebas.

Catálogo y gobierno: mapa, versión y acceso

El catálogo convierte un nombre de negocio en ubicación, esquema y metadatos vigentes. El gobierno responde otra pregunta: aunque alguien conozca la ruta, ¿tiene derecho a verla y con qué detalle? Un catálogo no mueve datos ni garantiza calidad por existir. Requiere propietarios, políticas, clasificación, auditoría y credenciales con privilegio mínimo.

Comprobación rápida.

Asigna cada responsabilidad:

1. «Organiza columnas y compresión dentro de `part-0007`» → **formato de archivo**.
2. «Decide que A, B y C forman la versión 17» → **formato de tabla**.
3. «Resuelve dónde vive `silver.transactions` y verifica permisos» → **catálogo y gobierno**.
4. «Conserva el PDF original y los bytes de Parquet» → **almacenamiento de objetos**.

Integrar calidad, cargas y productos de datos

Medallion: Bronze, Silver y Gold

La arquitectura Medallion es una forma frecuente, no una ley universal, de representar estados de calidad. Los nombres no designan productos ni ubicaciones físicas obligatorias.

Bronze preserva; Silver integra; Gold traduce al lenguaje del negocio. Finanzas no debería calcular ingresos directamente desde JSON Bronze. Una organización puede publicar varias tablas Gold porque cada producto responde a una decisión y granularidad distintas.

PDFs, imágenes y otros binarios permanecen como objetos. Una tabla Silver puede registrar `object_uri`, hash, propietario, clasificación, estado de OCR y relaciones. No conviene afirmar que una fotografía «se convierte en Iceberg».

Fase 2: una base gobernada y varios motores

La *fintech* no cambió sólo de contenedor. Cambió el contrato:

- **Finanzas** consulta productos Gold certificados.
- **ML** utiliza entidades Silver autorizadas y resultados derivados de documentos sin descargar terabytes a almacenamiento local.
- **Fraude** procesa eventos en tiempo real y consulta o actualiza señales gobernadas según las garantías del pipeline.

Separar almacenamiento y cómputo permite dimensionar y apagar motores por carga, pero no garantiza costos menores. Formato, estadísticas, tamaño de archivos, partición, cachés y consultas siguen determinando desempeño y gasto.

Tecnologías abiertas y servicios administrados

Responsabilidad	Ejemplos frecuentes	Función en el caso
Fuente operacional	PostgreSQL, MySQL, SQL Server, Cloud SQL, RDS, Azure SQL	Usuarios, cuentas, cobros y transferencias.
Ingesta batch/CDC	Fivetran, Airbyte, AWS DMS, Datastream, Azure Data Factory	Replica cambios operacionales.
Eventos	Kafka, Confluent, Kinesis, Pub/Sub, Event Hubs	Clics, telemetría y fraude.
Almacenamiento de objetos	S3, Cloud Storage, ADLS Gen2, OneLake	Conserva JSON, Parquet, PDFs e imágenes.
Formato de archivo	Parquet, ORC, Avro, JSON	Organiza bytes dentro de cada archivo.
Formato de tabla	Iceberg, Delta Lake, Hudi	Publica snapshots, esquema e inventario de archivos.
Catálogo y gobierno	Glue + Lake Formation, Lakehouse runtime catalog + Knowledge Catalog, Unity Catalog, Fabric + Purview	Descubrimiento, permisos, linaje y auditoría.
Motores	BigQuery, Athena, Redshift, Trino, Spark, Databricks SQL, Fabric SQL	Ejecuta la carga analítica o de ingeniería.
Consumo	Looker, QuickSight, Power BI, Tableau, notebooks, modelos	Convierte datos en decisiones o productos.

Parquet, Iceberg, Delta Lake, Hudi, Spark, Kafka y Trino son estándares o proyectos abiertos. S3, BigQuery, Redshift, Databricks, Fabric y Fivetran son servicios o plataformas administradas. Una plataforma comercial puede administrar tecnologías abiertas.

Implementaciones en Google Cloud, AWS y Azure

No hay traducción exacta entre servicios. La comparación debe mantener fijas las responsabilidades: almacenamiento, tabla, catálogo, gobierno, cómputo, consumo y operación.

Google Cloud: Cloud Storage, Iceberg y varios motores

La arquitectura actual de Google Cloud combina Cloud Storage, Apache Iceberg, un *runtime catalog*, gobierno y motores como BigQuery, Spark, Flink y Trino, según su documentación de arquitectura Lakehouse. Las tablas Iceberg administradas por BigQuery permanecen en buckets controlados por la organización y añaden evolución de esquema, *time travel* y optimización automática, como detalla la guía de tablas Iceberg administradas.

Una ruta de implementación razonable es:

1. elegir región, proyecto y bucket con políticas de retención y cifrado;

2. crear un Lakehouse runtime catalog y definir namespaces;
3. usar Iceberg V2 para nuevas tablas abiertas y decidir quién administra sus metadatos;
4. configurar IAM y, cuando corresponda, credenciales delegadas con privilegio mínimo;
5. conectar BigQuery y los motores abiertos que realmente utilizará el equipo;
6. probar lectura, escritura, evolución, recuperación y costos antes de declarar interoperabilidad.

El *quickstart* oficial recorre la creación del catálogo Iceberg REST, permisos, tablas y consultas. BigQuery también puede operar como un Data Warehouse moderno sin que la arquitectura deba llamarse Lakehouse.

AWS: S3, Iceberg, Glue y Lake Formation

La arquitectura Lakehouse de SageMaker unifica datos en S3, S3 Tables y Redshift mediante catálogos y acceso gobernado. En una pila modular, S3 conserva objetos, Iceberg coordina tablas, Glue Data Catalog registra metadatos, Lake Formation aplica permisos y Athena, Redshift, EMR o Glue ejecutan las cargas.

Una ruta de implementación es:

1. definir cuentas, regiones, buckets, cifrado, retención y límites de red;
2. registrar ubicaciones y tablas en Glue Data Catalog;
3. aplicar Lake Formation e IAM con separación entre administración, escritura y consumo;
4. crear tablas Iceberg y seleccionar motores compatibles por operación;
5. probar *compaction*, snapshots, limpieza y evolución de esquema;
6. publicar productos de datos y observar frescura, uso y costo.

AWS ofrece un recorrido inicial para crear proyecto, explorar, cargar, consultar y publicar datos. Su guía prescriptiva de Iceberg cubre servicios compatibles y prácticas para costo, rendimiento y operación a escala.

Azure y Microsoft Fabric: OneLake, Delta y Medallion

Microsoft ofrece al menos dos rutas frecuentes. **Fabric Lakehouse** usa OneLake, tablas Delta, Spark, un endpoint SQL y Power BI. **Azure Databricks** combina ADLS Gen2, Delta Lake, Unity Catalog, Spark y Databricks SQL. La elección depende de integración, gobierno, habilidades y modelo operativo.

Para Fabric, una ruta de implementación es:

1. decidir dominios, workspaces y fronteras de seguridad;
2. crear el Lakehouse y diseñar Bronze, Silver y Gold por calidad y propietario;
3. ingerir o enlazar datos, conservando el original cuando sea necesario;
4. publicar Silver y Gold como tablas Delta;
5. conectar Spark, endpoint SQL y Power BI según la carga;
6. automatizar pruebas, mantenimiento, `OPTIMIZE`, retención y observabilidad.

La guía de Medallion en OneLake explica modelos de despliegue, formatos, tamaño de archivos, retención y acceso. El tutorial de extremo a extremo cubre workspace, ingesta, transformación, Delta, endpoint SQL y consumo. Para una ruta empresarial con Data Factory y Azure Databricks, Microsoft organiza el diseño desde una base inicial hasta endurecimiento y misión crítica.

> ⚠ No confundas una demostración con una arquitectura de producción
>
> Un tutorial valida el flujo funcional. Producción exige además identidad, red, cifrado, continuidad, RPO/RTO, segregación de entornos, CI/CD, contratos, observabilidad, presupuestos, soporte y responsables.

El trabajo que no desaparece

El impuesto de los archivos pequeños

El *streaming* puede crear muchos archivos válidos y pequeños. El motor debe enumerarlos, abrirlos y planificarlos. La compactación reescribe el mismo contenido en menos archivos y publica el reemplazo mediante un nuevo snapshot.

Una fuente compartida comparte el radio de impacto

Responsabilidades operativas.

- compactación y dimensionamiento de archivos;
- retención de snapshots y limpieza de archivos huérfanos;
- calidad, contratos y evolución compatible;
- permisos, auditoría y prevención de accesos directos no gobernados;
- observabilidad de latencia, frescura, costo y fallas;
- pruebas reales de interoperabilidad para cada formato y motor;
- presupuestos, cuotas y apagado o escalamiento del cómputo;
- continuidad: *time travel* es útil, pero no sustituye por sí solo respaldos, aislamiento y recuperación.

Decidir, no evangelizar

Criterio	Warehouse administrado	Lake + Warehouse separados	Lakehouse
Datos dominantes	Relacionales y preparados	Crudos diversos con usos separados	Diversos y compartidos entre BI, ML y streaming
Prioridad	Rapidez de adopción y SQL	Aislar economías o equipos	Reducir copias con contratos comunes
Cómputo	Principalmente SQL/BI	Motores y autoridades separadas	Motores distintos sobre tablas compartidas

Criterio	Warehouse administrado	Lake + Warehouse separados	Lakehouse
Operación	Menor carga para un equipo pequeño	Coordinación y sincronización dobles	Metadatos, compactación y gobierno rigurosos
Señal económica	El servicio administrado sigue alineado	La separación tiene fronteras claras	Movimiento, copias o almacenamiento propietario dominan el costo
Riesgo principal	Rigidez o costo al crecer	Dos verdades y latencia	Complejidad y radio de impacto compartido

Para la *fintech*:

- **Años 0–3:** Warehouse administrado; datos relacionales, objetivo de reporting y equipo pequeño.
- **Año 5:** Lakehouse; BI, ML y fraude reutilizan datos diversos a escala, y el movimiento ya domina parte del costo.
- **Otra empresa:** puede seguir en Warehouse o usar una arquitectura híbrida. No hay ganador universal.

Usa esta forma de argumento:

> Elegimos _____ porque _____ y ***; aceptamos el costo o riesgo de*** .

Ejemplo inicial: «Elegimos un Warehouse administrado porque los datos son relacionales y necesitamos reporting rápido; aceptamos menor flexibilidad futura a cambio de una operación sencilla».

Ejemplo a escala: «Elegimos Lakehouse porque BI, ML y fraude necesitan reutilizar datos diversos y el movimiento de terabytes domina el costo; aceptamos operar compactación, contratos y gobierno con un equipo especializado».

Preguntas frecuentes y errores de recuperación

¿Un bucket es una carpeta gigante?

Es una aproximación inicial útil, pero incompleta. El bucket contiene objetos identificados por keys. Los prefijos pueden mostrarse como carpetas aunque el modelo base no sea un sistema de archivos tradicional.

¿Puedo hacer SQL directamente sobre un bucket?

El bucket no ejecuta SQL. Athena, BigQuery, Trino, Spark u otro motor puede leer archivos. Para tratarlos como tablas confiables necesita esquema, lista vigente de archivos, formato de tabla y catálogo.

¿S3 es un Data Lake?

S3 suele ser su base física. El Data Lake incluye también ingesta, organización, seguridad, catálogo, cómputo y operación.

¿Entonces el Data Warehouse ya no sirve?

No. En la fase inicial fue la opción más eficiente y puede seguir atendiendo productos Gold o cargas de BI. La transición se evalúa cuando el beneficio de compartir datos diversos supera la carga operativa.

¿Guardar Parquet en S3 ya crea un Lakehouse?

No. Resuelve almacenamiento y formato de archivo. Faltan commits consistentes, formato de tabla, catálogo, permisos, calidad, mantenimiento y motores compatibles.

¿Iceberg, Delta Lake y Hudi son bases de datos?

Son formatos o protocolos de tabla. Un catálogo y motores completan la experiencia; una plataforma puede empaquetarlos como servicio.

¿El catálogo guarda todos los datos?

Normalmente guarda o resuelve metadatos: nombres, ubicaciones, esquemas, propietarios, políticas y punteros. Los archivos permanecen en almacenamiento de objetos.

¿La fuente única de verdad es una sola tabla?

No. Significa autoridades y contratos coherentes. Bronze, Silver y Gold pueden coexistir, con productos adecuados a distintas decisiones.

¿Formato abierto significa compatibilidad perfecta?

No. Deben probarse operaciones, versión del formato, extensiones, escrituras concurrentes, evolución de esquema y mantenimiento en cada motor.

¿Separar cómputo siempre abarata?

No. Permite escalar por carga, pero consultas ineficientes, recursos permanentes o falta de límites pueden elevar el gasto.

¿Time travel sustituye los respaldos?

No necesariamente. Depende de retención, eliminación de objetos, aislamiento de cuentas y requisitos de recuperación. Es versionado útil, no una estrategia completa de continuidad.

Exit ticket

Completa sin consultar el capítulo:

1. Un bucket con Parquet no es todavía una tabla porque…
2. El catálogo sirve para…, mientras el formato de tabla sirve para…
3. La migración de la *fintech* se justifica cuando…, pero obliga a…

Respuestas mínimas esperadas

1. Falta una lista versionada y transaccional que determine qué archivos forman la tabla.
2. El catálogo permite encontrar y gobernar; el formato coordina snapshots, esquema y commits.
3. Varias cargas comparten datos diversos a escala y las copias o costos se vuelven dominantes; a cambio se operan mantenimiento, gobierno y contratos.

Criterio de logro.

El objetivo se alcanza si puedes reconstruir la cadena `silos → objetos → snapshots → catálogo/gobierno → motores especializados → operación → decisión` y, además, explicar por qué el Warehouse fue correcto en la fase inicial y por qué el Lakehouse sólo se volvió razonable al cambiar los requisitos.

Cheatsheet

Modelo mental en una página

Si ves o escuchas…	Piensa en…	Pregunta de verificación
`s3://`, `gs://`, ADLS, OneLake	Almacenamiento de objetos	¿Dónde viven los bytes y quién controla la ubicación?
Parquet u ORC	Formato de archivo	¿Cómo se organizan columnas dentro de cada archivo?
Iceberg, Delta o Hudi	Formato de tabla	¿Cómo se publican commits, snapshots y evolución de esquema?
Glue, Lakehouse runtime catalog, Unity Catalog, Fabric/Purview	Catálogo y gobierno	¿Quién resuelve nombres, metadatos, permisos, linaje y propietario?
BigQuery, Athena, Trino, Spark, Redshift, Databricks SQL	Motor de cómputo	¿Qué procesa y qué operaciones soporta sobre el formato?
Bronze	Evidencia recibida	¿Puede reconstruirse el proceso y respetarse la retención?
Silver	Entidades validadas	¿Existe contrato, calidad, deduplicación y responsable?
Gold	Producto de datos	¿Qué decisión, granularidad y definición de negocio sirve?
Muchos archivos pequeños	Deuda operativa	¿Quién compacta, actualiza estadísticas y limpia huérfanos?
«Una copia para cada equipo»	Impuesto de coordinación	¿Pueden compartir tabla, catálogo y política sin compartir motor?

Diagnóstico rápido

Todavía es un Data Lake consultable si hay objetos, Parquet, catálogo básico y motor, pero no existe un protocolo transaccional que publique versiones confiables de tabla.

Hay un Lakehouse funcional si los datos tabulares comparten formato de tabla, catálogo, gobierno y motores compatibles, con operación de calidad, mantenimiento y costo.

Conviene un Warehouse administrado cuando domina SQL sobre datos preparados, el equipo es pequeño y la simplicidad ofrece más retorno que la flexibilidad adicional.

Conviene evaluar Lakehouse cuando BI, ML y streaming necesitan reutilizar datos diversos; las copias, el movimiento o el almacenamiento propietario son el cuello de botella; y existe capacidad para operarlo.

Lista mínima de implementación

1. Definir decisiones y SLO de frescura, calidad, costo y recuperación.
2. Elegir almacenamiento, regiones, cifrado, retención y fronteras de red.
3. Elegir formato de tabla y probar compatibilidad real con cada motor.
4. Establecer catálogo, identidad, clasificación, permisos, linaje y propietarios.
5. Diseñar contratos y estados de calidad; Medallion es una opción.
6. Automatizar pruebas, compactación, estadísticas, retención y limpieza.
7. Observar frescura, fallas, uso, rendimiento y costo por producto de datos.
8. Preparar evolución compatible, rollback y continuidad.

La síntesis final es:

```
silos y copias
→ base común en objetos
→ archivos coordinados por snapshots
→ catálogo y permisos
→ motores distintos sobre contratos compartidos
→ mantenimiento y control del radio de impacto
→ elección condicionada por el caso
```

Bibliografía y guías oficiales de implementación

Las referencias siguientes son documentación primaria de los proveedores. Fueron consultadas el 30 de agosto de 2026.

Google Cloud

- Google Cloud. *How Lakehouse works*. Arquitectura de almacenamiento, catálogo, gobierno y motores sobre Apache Iceberg.
- Google Cloud. *Quickstart: Use the Iceberg REST catalog with Lakehouse*. Implementación paso a paso del catálogo, permisos, tablas y consultas.
- Google Cloud. *Apache Iceberg managed tables*. Capacidades y límites de las tablas Iceberg administradas por BigQuery.

Amazon Web Services

- Amazon Web Services. *What is the lakehouse architecture of Amazon SageMaker?*. Componentes y funcionamiento de la arquitectura administrada.
- Amazon Web Services. *Getting started with the lakehouse architecture of Amazon SageMaker*. Recorrido inicial para proyecto, datos, consultas y publicación.
- Garcia Garcia, D. y Sandona, S. *Using Apache Iceberg on AWS*. AWS Prescriptive Guidance para servicios compatibles, costo, desempeño y operación.

Microsoft Azure y Fabric

- Microsoft. *Understand medallion architecture for Fabric with OneLake*. Diseño de capas, despliegue, Delta, archivos y mantenimiento.
- Microsoft. *Lakehouse end-to-end scenario: overview and architecture*. Implementación de ingesta, transformación, endpoint SQL y consumo.
- Microsoft. *Design a medallion lakehouse with Azure Data Factory*. Ruta desde la implementación base hasta endurecimiento empresarial y misión crítica.

Parte VI.

Modelado multidimensional

7. Modelado Multidimensional

El modelado multidimensional constituye la base conceptual de los almacenes de datos (Data Warehouses, DW). A diferencia del modelo relacional tradicional altamente normalizado (diseñado para sistemas transaccionales (OLTP) donde predominan inserciones y actualizaciones frecuentes) el modelo multidimensional está optimizado para el análisis (OLAP), es decir, para responder preguntas complejas de negocio de manera rápida, clara y comprensible para los usuarios finales.

Como se enfatiza en el material del curso, el diseño de un DW no comienza por la base de datos ni por la tecnología, sino por las decisiones que el negocio desea tomar y la información necesaria para sustentarlas. El modelo se construye entonces alrededor de preguntas analíticas, no alrededor de procesos operativos.

A diferencia del modelo relacional normalizado, aquí se utilizan estructuras optimizadas para consultas complejas:

- **Tabla de Hechos (Fact Table):** Contiene las métricas cuantitativas o medidas del negocio.
- **Tablas de Dimensiones:** Contienen los atributos descriptivos que contextualizan los hechos (Tiempo, Geografía, Producto, Estudiante, etc.).

Hechos y Dimensiones

Un hecho representa un evento medible del negocio. Debe cumplir tres características fundamentales: ocurre en un momento específico, se puede medir y sucede repetidamente. En lugar de describirlo solo de forma conceptual, puede entenderse mejor mediante una representación estructural.

A continuación se muestra un ejemplo de una **tabla de hechos académica**, donde cada fila representa la evaluación de un estudiante en una asignatura durante un periodo determinado.

HECHO_EVALUACION_ACADEMICA	
int	id_estudiante
int	id_asignatura
int	id_tiempo
decimal	calificacion
int	creditos
int	indicador_reprobado

En esta tabla:

- Cada registro corresponde a un evento académico concreto.
- Las columnas `calificacion`, `creditos` e `indicador_reprobado` son medidas.
- Las claves (`id_estudiante`, `id_asignatura`, `id_tiempo`) conectan el hecho con su contexto.

Las **dimensiones** proporcionan precisamente ese contexto. Permiten responder preguntas como: ¿quién?, ¿cuándo?, ¿dónde? y ¿qué? ocurrió el evento.

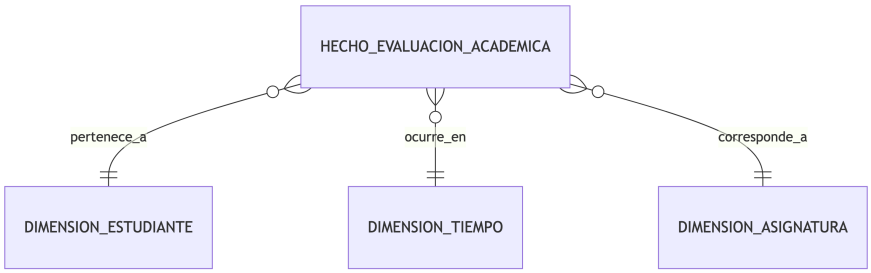
A continuación se muestra una representación simplificada de algunas dimensiones académicas:

DIMENSION_ESTUDIANTE	
int	id_estudiante
string	nombre
string	sexo
int	generacion
string	carrera
string	facultad

DIMENSION_TIEMPO	
int	id_tiempo
int	semestre
int	anio
string	periodo

DIMENSION_ASIGNATURA	
int	id_asignatura
string	nombre_asignatura
string	tipo

El hecho adquiere significado únicamente cuando se conecta con estas dimensiones. Conceptualmente, la relación puede representarse así:



De esta forma, una fila del hecho puede interpretarse como:

> "El estudiante X cursó la asignatura Y en el periodo Z y obtuvo determinada calificación".

Un hecho aislado es poco informativo; adquiere significado al conectarse con dimensiones. Esta relación también guía el diseño del modelo: si una medida no puede explicarse adecuadamente con las dimensiones disponibles, suele indicar que falta una dimensión relevante, como campus, plan de estudios, modalidad o grupo.

Granularidad: la decisión de diseño más importante

La granularidad define qué representa exactamente una fila en la tabla de hechos.
Impacta directamente el volumen de datos, el costo de infraestructura, la complejidad del ETL y el tipo de análisis que puede realizarse.

Comparación de niveles de granularidad

HECHO_TRAYECTORIA_SEMESTRAL	
int	id_estudiante
int	id_tiempo
int	total_materias
int	total_creditos
decimal	promedio_semestre

HECHO_EVALUACION_ACADEMICA	
int	id_estudiante
int	id_asignatura
int	id_tiempo
decimal	calificacion
int	creditos
int	indicador_reprobado

Izquierda: Granularidad baja — una fila por alumno por semestre.
Cada registro resume el desempeño global del estudiante en un periodo.
No se almacenan materias individuales, sino resultados agregados del semestre.

Con 200 alumnos y 2 semestres por año se generan aproximadamente 400 filas por año (2,000 en cinco años).
Este enfoque reduce volumen y complejidad, pero limita el análisis detallado por asignatura o profesor.

Derecha: Granularidad alta — una fila por alumno por materia por semestre.
Cada registro representa una inscripción específica en una asignatura.
Permite analizar reprobación por materia, desempeño por docente y evolución por plan de estudios.

Con 200 alumnos, 6 materias promedio y 2 semestres por año se generan aproximadamente 4,800 filas por año (24,000 en cinco años).
Este enfoque aumenta capacidad analítica, pero también volumen, costo y complejidad operativa.

Hecho aditivo

Un hecho es aditivo cuando puede sumarse correctamente a través de todas las dimensiones del modelo (incluida la dimensión tiempo). En el material del curso se menciona como ejemplos típicos: número de inscritos, créditos inscritos y número de reprobados.

Ejemplos claros:

- Ventas: sumar "monto vendido" por día, por tienda, por producto, por ciudad. La suma conserva sentido en cualquier nivel de agregación.
- Académico: sumar "créditos inscritos" por estudiante, por carrera, por semestre o por año; y sumar "indicador_reprobado" (0/1) para obtener el total de reprobados en un conjunto dado (por asignatura, periodo, generación, etc.).

Implicación de diseño: si una medida es aditiva, suele ser una buena candidata para almacenarse directamente en la tabla de hechos, porque habilita muchas consultas rápidas y robustas.

Hecho semi-aditivo

Un hecho es semi-aditivo cuando puede sumarse en algunas dimensiones, pero no en todas; típicamente no se suma válidamente a través del tiempo. En el material se ejemplifica con "estudiantes activos al finalizar semestre" o "créditos acumulados".

Ejemplo:

- "Estudiantes activos" por carrera en un mismo semestre: sumar por carrera para obtener el total de activos en ese semestre puede tener sentido (si las carreras son particiones disjuntas).
- "Estudiantes activos" entre semestres: sumar activos del semestre 1 más activos del semestre 2 suele perder interpretación porque muchas personas se repiten; el resultado puede inflarse y no representa una entidad real.

Implicación de diseño: medidas semi-aditivas a menudo se consultan con funciones como `MAX`, `MIN` o "valor al cierre del periodo", o bien se modelan como "snapshots" (cortes periódicos), dependiendo del caso.

Hecho no aditivo

Un hecho es no aditivo cuando no puede sumarse en ninguna dimensión; requiere cálculo (promedio, razón, porcentaje). En el curso se mencionan: promedio de calificaciones, tasa de reprobación y porcentaje de deserción.

Ejemplo:

- La tasa de reprobación se calcula como (reprobados / inscritos) bajo un filtro o un grupo; no se almacena como tal en la tabla de hechos porque debe recomponerse según el contexto de la consulta.
- El promedio de calificaciones depende de la población (y a veces de ponderaciones). "Sumar promedios" rara vez es significativo.

Implicación de diseño: en general se almacenan los componentes base (numeradores/denominadores, conteos, sumas) y se calculan métricas derivadas en la capa semántica o en vistas/consultas.

Jerarquías en dimensiones y navegación analítica

Las jerarquías permiten analizar de lo general a lo particular (roll-up y drill-down). Ejemplos típicos:

- Tiempo: Año → Semestre → Periodo.
- Geografía: País → Estado → Ciudad.
- Académico: Facultad → Carrera → Estudiante.

Este diseño es clave para reportes: permite cambiar el "nivel" de agregación sin rediseñar la consulta, solo cambiando el `GROUP BY` o el nivel de la dimensión utilizado.

Con hechos, dimensiones, granularidad y aditividad definidos, el siguiente paso es decidir *cómo* se estructuran físicamente esas dimensiones alrededor del hecho, y qué costo real tiene esa decisión — el tema del siguiente capítulo.

Conclusión

El modelado multidimensional organiza los datos alrededor de hechos medibles y dimensiones descriptivas, priorizando claridad y desempeño para análisis. Para que el diseño sea correcto, debe fijarse adecuadamente la granularidad y entender la aditividad de las medidas, ya que esto determina qué agregaciones son válidas y qué métricas deben calcularse.

8. Esquemas Dimensionales y Redundancia

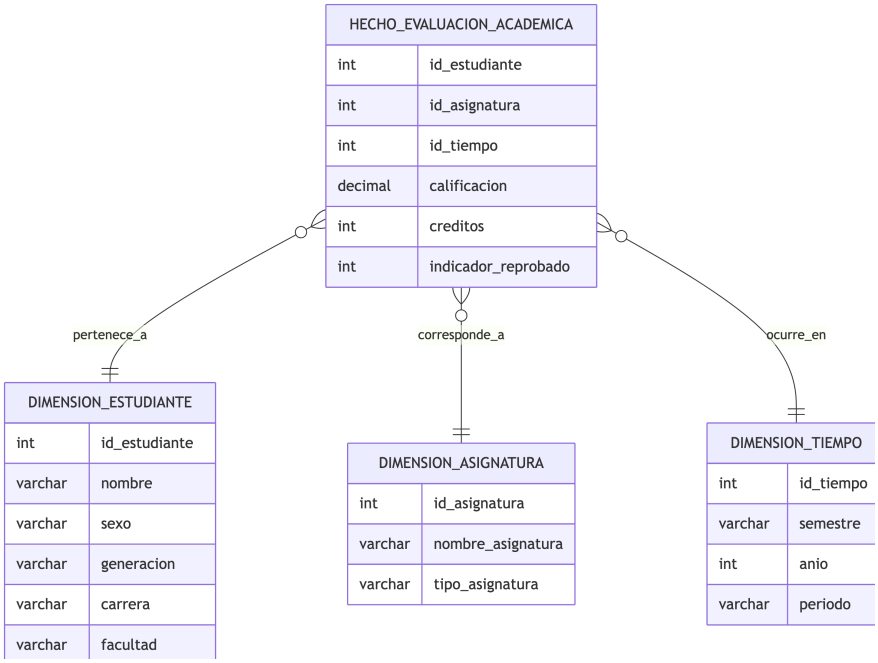
En el capítulo anterior se definieron hechos, dimensiones, granularidad y aditividad. Ahora corresponde decidir *cómo* se estructuran físicamente las dimensiones alrededor del hecho: qué tan normalizadas están, cuánta redundancia se acepta y qué impacto real tiene esa decisión sobre el costo y la velocidad de consulta.

Esquema de Estrella

El esquema estrella consiste en una tabla de hechos central conectada directamente a varias dimensiones. Sus dimensiones suelen estar desnormalizadas (redundancia controlada) para reducir joins y optimizar consultas analíticas.

En el ejemplo académico, la dimensión estudiante incluye atributos como carrera y facultad en la misma tabla, lo que simplifica consultas típicas del estilo "promedio por carrera y por semestre". El costo es aceptar repetición (por ejemplo, repetir "ciencias" como facultad en muchos registros).

Desde el punto de vista estructural, el modelo puede representarse así:



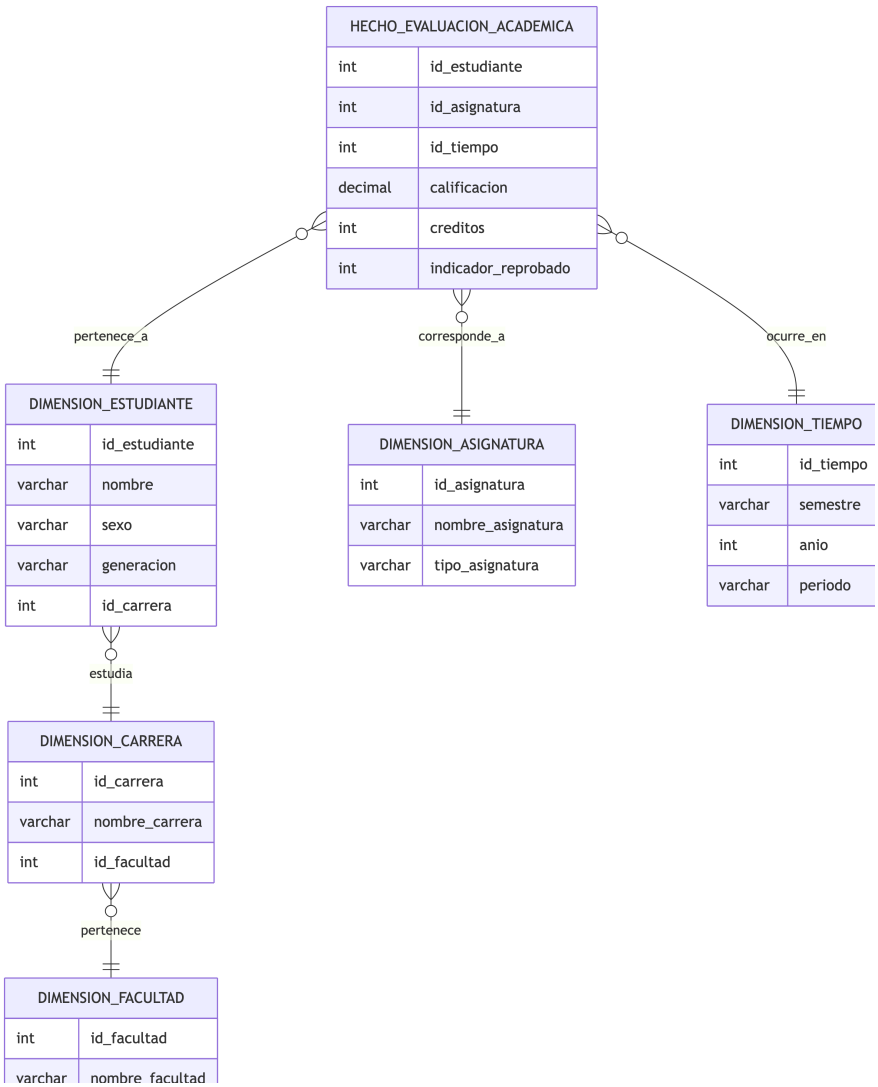
En este esquema:

- La tabla de hechos está al centro.

- Todas las dimensiones se conectan directamente a ella.
- No existen relaciones entre dimensiones.
- La información jerárquica (por ejemplo, carrera → facultad) se almacena dentro de la misma dimensión.

Esquema de Copo de Nieve (Snowflake)

El esquema de copo de nieve normaliza las dimensiones en subdimensiones. Por ejemplo, separar "carrera" y "facultad" en tablas propias relacionadas jerárquicamente, evitando redundancia, pero aumentando joins y complejidad de consulta.



En este esquema:

- La dimensión estudiante ya no almacena directamente la facultad.
- Carrera y facultad se modelan como entidades independientes.

- Se elimina redundancia.
- Se incrementa el número de joins necesarios para consultas que involucren facultad.

Comparación conceptual

Aspecto	Estrella	Copo de nieve
Normalización	Baja (desnormalizada)	Alta
Redundancia	Controlada	Reducida
Número de joins	Menor	Mayor
Complejidad conceptual	Más simple	Más estructurada
Rendimiento OLAP	Generalmente mejor	Puede disminuir
Gobierno de datos	Más flexible	Más controlado

Lo mismo, en SQL

Con un dominio distinto (ventas), la diferencia entre ambos esquemas se ve directamente en la consulta:

Criterio	Estrella	Copo de nieve
Consultas	Simples, pocas uniones	Más uniones
Redundancia	Mayor	Menor
Mantenimiento	Directo	Más entidades y claves
Uso recomendado	BI y autoservicio	Jerarquías grandes o compartidas

Redundancia controlada: qué cuesta realmente desnormalizar

La comparación anterior suele quedarse en "el copo de nieve reduce redundancia", como si repetir datos fuera automáticamente malo. En un almacén de datos columnar (Parquet, Delta, BigQuery, Snowflake) esa redundancia casi no cuesta lo que parece.

Un motor columnar no almacena cada valor de texto tal cual: aplica **codificación por diccionario** (cada valor distinto se guarda una sola vez y cada fila solo referencia un código corto) y **run-length encoding** (RLE) sobre columnas ordenadas o de baja cardinalidad. Si `dim_producto` tiene 500,000 filas y la columna `categoria` solo tiene 20 valores distintos, el motor no almacena "Electrónica" 25,000 veces: almacena el diccionario de 20 categorías una vez y, por fila, un índice de unos cuantos bits.

En la práctica esto significa que:

- Guardar `categoria` directamente en `dim_producto` (estrella) cuesta, en almacenamiento, un orden de magnitud similar a guardar `categoria_key` (copo de nieve) — ambos terminan siendo, a nivel de disco, un entero pequeño por fila.

- El ahorro de espacio que promete la normalización es, en un motor columnar moderno, marginal frente a lo que cuesta en 1990 con motores orientados a fila.
- Lo que sí cambia entre ambos esquemas no es tanto el almacenamiento, sino el **costo de consulta**.

Costo de consulta: joins y motores en la nube

El argumento a favor de la estrella no es "ocupa menos espacio", es "cada consulta hace menos trabajo". Esto importa especialmente cuando las dimensiones no son pequeñas.

Considera una dimensión cliente con 5 millones de filas, normalizada como `dim_cliente` → `dim_ciudad` → `dim_estado` (jerarquía geográfica en copo de nieve) contra la misma dimensión desnormalizada donde `estado` vive directamente en `dim_cliente`.

Para responder "ventas totales por estado":

- **Estrella:** `fact_ventas` (100M filas) hace un solo join contra `dim_cliente` (5M filas), que ya trae `estado`.
- **Copo de nieve:** `fact_ventas` hace join contra `dim_cliente` (5M filas), y ese resultado intermedio vuelve a hacer join contra `dim_ciudad` (5K filas) y luego contra `dim_estado` (32 filas).

El segundo camino no es más caro por las tablas pequeñas (`dim_ciudad`, `dim_estado`) — es más caro porque agrega **etapas de shuffle** sobre un resultado intermedio ya grande (la salida del primer join, del tamaño de `fact_ventas`). Cada hop adicional en la cadena de joins vuelve a mover ese volumen de datos por la red del clúster.

Esto se traduce directamente en costo facturable:

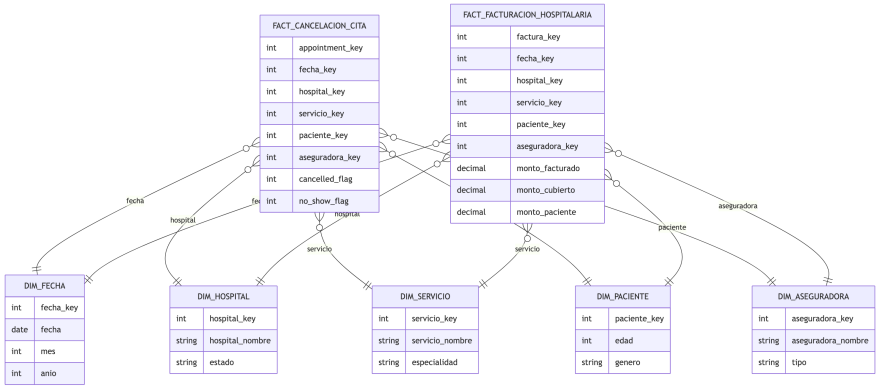
- **BigQuery** cobra por bytes escaneados/procesados; más joins sobre resultados intermedios grandes significan más bytes movidos y más slots ocupados.
- **Snowflake** cobra por tiempo de cómputo del warehouse (créditos); una cadena de joins más larga generalmente implica más segundos de warehouse activo para la misma pregunta.

Regla práctica: la redundancia de un esquema estrella es barata cuando vive en una dimensión (se paga una vez, al cargar). El costo de un join extra se paga en **cada consulta**, multiplicado por cuántas veces se ejecuta esa consulta al día. Por eso los data marts orientados a BI casi siempre prefieren estrella, y el copo de nieve se reserva para jerarquías muy grandes, compartidas entre muchos data marts, donde mantener una sola versión de la verdad importa más que el costo por consulta.

Esquema de Constelación (Fact Constellation / Galaxy Schema)

Cuando dos o más procesos de negocio comparten las mismas dimensiones, tiene sentido que compartan también las tablas físicas: eso es un **esquema de constelación**. En vez de una estrella aislada por proceso, varias tablas de hechos se conectan a un conjunto de **dimensiones conformadas** (misma clave, mismo grano, mismo significado en todos los hechos que la usan).

Retomando el dominio hospitalario: además de `FACT_CANCELACION_CITA` (una fila por cita programada), el hospital también factura por cada servicio prestado. En vez de crear dimensiones nuevas y redundantes para ese segundo proceso, se reutilizan `DIM_FECHA`, `DIM_HOSPITAL`, `DIM_SERVICIO`, `DIM_PACIENTE` y `DIM_ASEGURADORA`:



Esto resuelve exactamente el tipo de redundancia más caro: sin dimensiones conformadas, cada equipo que construye un nuevo data mart terminaría creando su propia versión de "paciente" o "servicio", con distinta granularidad, distintos códigos y sin forma de cruzar ambos hechos ("¿cuántas citas canceladas generaron facturación parcial de cualquier forma?"). Con dimensiones conformadas, ambos hechos son comparables y combinables por las mismas llaves.

Patrones de diseño para gestionar redundancia

Además de estrella, copo de nieve y constelación, hay patrones puntuales para decidir *dónde* vive cada dato — evitando tanto la redundancia descontrolada como los joins innecesarios.

Dimensiones conformadas. Una misma dimensión (mismo grano, mismas llaves) reutilizada por varios hechos, como `DIM_FECHA` en el ejemplo anterior. Es la base de cualquier esquema de constelación y evita que cada data mart reinvente "cliente" o "producto" con reglas distintas.

Dimensiones degeneradas. Un atributo que identifica al hecho (folio de orden, número de ticket) pero que no tiene atributos propios que ameriten una tabla de dimensión. Se guarda como columna directa en la tabla de hechos en vez de crear una dimensión de una sola columna.

Junk dimensions. Varios indicadores booleanos o de baja cardinalidad (`es_promocion`, `metodo_pago`, `canal`) se combinan en una sola dimensión "basurero" con una fila por combinación posible, en vez de agregar media docena de columnas sueltas al hecho o crear media docena de dimensiones diminutas.

Role-playing dimensions. La misma tabla física de dimensión se usa varias veces con distinto significado — por ejemplo, `DIM_FECHA` referenciada como fecha de pedido, fecha de envío y fecha de pago dentro del mismo hecho, mediante alias en la consulta (`fecha_pedido_key`, `fecha_envio_key`, `fecha_pago_key`, todas apuntando a la misma `DIM_FECHA`).

Mini-dimensiones. Cuando algunos atributos de una dimensión grande cambian con mucha frecuencia (ingresos estimados de un cliente, score de riesgo) separarlos en una dimensión pequeña

propia evita que esos cambios disparen versiones nuevas de SCD tipo 2 en toda la fila del cliente — tema que se retoma en el siguiente capítulo.

Bridge tables. Cuando la relación entre el hecho y una dimensión es de muchos a muchos (un estudiante con varias becas activas, un diagnóstico con múltiples médicos tratantes) una tabla puente resuelve la relación sin duplicar filas del hecho ni forzar una cardinalidad que no existe en el negocio.

Conclusión

Estrella, copo de nieve y constelación no son alternativas puramente estéticas: cada una mueve el costo de un lugar a otro (almacenamiento vs. cómputo por consulta vs. complejidad de mantenimiento). En un motor columnar moderno, la redundancia dentro de una dimensión suele ser la opción más barata. Lo que todavía no se ha cubierto es qué pasa cuando esas dimensiones **cambian** — un cliente se muda de ciudad, un producto cambia de categoría. Ese es el tema del siguiente capítulo: dimensiones lentamente cambiantes (SCD).

9. Dimensiones Lentamente Cambiantes (SCD)

El capítulo anterior asumió, implícitamente, que las dimensiones son estáticas: un estudiante siempre pertenece a la misma carrera y un producto siempre a la misma categoría. En la práctica no es así: un cliente cambia de ciudad, un producto cambia de categoría y un empleado cambia de puesto.

Las **Slowly Changing Dimensions (SCD)** definen cómo tratar esos cambios de atributos descriptivos sin romper la relación entre un hecho y el contexto que tenía cuando ocurrió. La **clave natural** identifica a la entidad en el sistema fuente; la **clave sustituta** identifica una fila o versión concreta dentro del data warehouse.

Tipos de SCD

Tipo 1: sobrescribir

SCD Tipo 1 actualiza la fila existente. El valor nuevo sustituye al anterior y todas las consultas, incluso las que analizan hechos antiguos, ven la descripción corregida o más reciente.

Ventajas

- Es el patrón más sencillo de implementar, consultar y mantener.
- No incrementa el número de filas de la dimensión.
- Corrige errores de captura para todo el historial de manera inmediata.
- No requiere columnas de vigencia ni lógica temporal en los joins.

Desventajas

- Destruye el valor anterior y no permite reconstruir cómo se veía la entidad en una fecha pasada.
- Puede reinterpretar hechos históricos con atributos que todavía no existían cuando ocurrieron.
- No es adecuado para auditoría, trazabilidad ni análisis de evolución.

Conviene usarlo cuando el cambio es una corrección ortográfica, una normalización o un atributo cuyo historial carece de valor analítico. No conviene usarlo para cambios de puesto, categoría, segmento o ubicación cuando esos cambios alteran la interpretación del hecho.

Tipo 2: crear una versión

SCD Tipo 2 cierra la versión vigente e inserta una fila nueva. La clave natural se mantiene, pero cada versión recibe una clave sustituta distinta y un intervalo de vigencia.

Ventajas

- Conserva el historial completo de los atributos seleccionados.

- Mantiene el contexto correcto de cada hecho sin depender de joins por rango de fecha, siempre que el proceso de carga asigne la clave sustituta vigente.
- Permite auditoría, reconstrucción histórica y análisis de transiciones.
- Hace explícita la vigencia mediante `valido_desde`, `valido_hasta` y `es_actual`.

Desventajas

- Aumenta el volumen de la dimensión: una entidad puede producir muchas filas.
- La carga es más compleja porque debe detectar cambios, cerrar la versión anterior e insertar la nueva de forma consistente.
- Requiere reglas claras para cambios tardíos, correcciones retroactivas y límites de vigencia.
- Puede generar versiones duplicadas, solapamientos o más de una fila vigente si el proceso no es idempotente o no se ejecuta de forma atómica.

Conviene usarlo cuando es necesario responder preguntas como "¿qué categoría tenía el producto cuando se realizó la venta?" o "¿en qué ciudad estaba registrado el cliente cuando ocurrió la transacción?".

En producción, el cierre y la inserción deberían ejecutarse dentro de una misma transacción cuando el motor lo permita. La clave sustituta puede generarse mediante una columna `identity`, una secuencia, un hash estable de la versión o la estrategia definida por la plataforma.

Tipo 3: guardar el valor anterior

SCD Tipo 3 conserva el valor vigente y un valor previo en la misma fila. No crea una historia ilimitada: al llegar otro cambio, el valor anterior también se sobrescribe.

Ventajas

- Permite comparar fácilmente el estado actual con el inmediatamente anterior.
- Mantiene una sola fila por entidad y evita el crecimiento de SCD Tipo 2.
- Las consultas son sencillas porque no necesitan intervalos de vigencia ni múltiples versiones.

Desventajas

- Sólo conserva una cantidad fija y limitada de cambios.
- Requiere agregar columnas específicas para cada atributo histórico.
- Pierde utilidad cuando varios atributos cambian de forma independiente.
- No permite reconstruir el estado de la dimensión para una fecha arbitraria.

Conviene usarlo cuando el negocio sólo compara el valor actual contra el anterior, por ejemplo, segmento actual frente a segmento previo. No sustituye a SCD Tipo 2 cuando existe un requisito de auditoría completa.

Implementación con dbt

dbt materializa el modelo dimensional desde tablas de *staging*. Un modelo incremental puede cubrir SCD Tipo 1; para SCD Tipo 2, un *snapshot* registra automáticamente las versiones detectadas.

El *snapshot* agrega `dbt_valid_from`, `dbt_valid_to` y `dbt_scd_id`. La dimensión final usa esas columnas para exponer la vigencia y la clave sustituta.

Ventajas de usar snapshots de dbt

- Centralizan la detección y el versionado de cambios.
- Reducen la cantidad de SQL procedural escrito manualmente.
- Generan metadatos de vigencia de forma consistente.
- Se integran con dependencias, documentación y pruebas del proyecto dbt.

Desventajas y precauciones

- El comportamiento depende de elegir correctamente `unique_key`, `strategy` y `check_cols`.
- Incluir columnas volátiles, como una marca de tiempo de carga, puede crear versiones innecesarias en cada ejecución.
- Un snapshot conserva lo que observa durante sus ejecuciones; no puede reconstruir cambios que ocurrieron y desaparecieron entre dos corridas.
- Los cambios retroactivos y las eliminaciones requieren una política explícita.

Pruebas de integridad

Las pruebas declarativas verifican que la clave sustituta sea única y no nula, y que la clave natural esté presente.

Esas pruebas no garantizan por sí solas que exista una sola versión vigente. Esa regla puede verificarse con una prueba singular adicional:

También conviene comprobar que los intervalos no se solapen y que `valido_desde` sea menor que `valido_hasta`.

Comparación y criterio de elección

Tipo	Historial	Filas por entidad	Complejidad	Uso principal
Tipo 1	Ninguno	Una	Baja	Correcciones y atributos sin valor histórico
Tipo 2	Completo	Una por versión	Alta	Auditoría y contexto histórico preciso
Tipo 3	Limitado	Una	Media	Comparar el estado actual con el previo

La elección no tiene que hacerse para toda la dimensión. En una misma tabla es posible aplicar Tipo 1 a atributos correctivos, Tipo 2 a atributos cuyo historial importa y, de forma excepcional, Tipo 3 a comparaciones limitadas. Lo importante es definir la política **por atributo**, documentar por qué se conserva o descarta el historial y diseñar las pruebas que protejan esa decisión.

10. Ejemplo Completo: Café Norte — Análisis y Construcción de Data Marts (Kimball)

Propósito y supuestos

Este ejemplo documenta, con un enfoque **Kimball**, la construcción de dimensiones conformadas y una tabla de hechos a granularidad **línea de pedido** para evaluar el impacto del lanzamiento de la app (enero de 2023) y del programa de lealtad. Es el cierre práctico de la sección: aplica granularidad y aditividad, esquema estrella y redundancia controlada, y dimensiones conformadas, todo sobre un caso end-to-end.

Hito de intervención (cutoff): `2023-01-01`. Se define:

- Periodo **pre**: fechas menores a `2023-01-01`.
- Periodo **post**: fechas mayores o iguales a `2023-01-01`.

Se asume que el lector **ya cuenta con CSVs** provenientes de sistemas operacionales (POS y App) y catálogos maestros.

Estructura esperada de archivos CSV

Coloque los archivos en un directorio `data/` al mismo nivel que este `.qmd`.

Catálogos maestros (dimensiones)

- `data/clientes.csv` Campos mínimos: `cliente_id`, `fecha_alta`, `tipo_cliente`, `nivel_lealtad` (u otros atributos).
- `data/productos.csv` Campos mínimos: `sku`, `nombre`, `categoria`, `subcategoria`, `tamano`, `activo`.
- `data/sucursales.csv` Campos mínimos: `sucursal_id`, `nombre`, `zona`, `formato`.
- (Opcional) `data/promociones.csv` Campos mínimos: `promo_codigo`, `campania`, `descripcion`.

Transacciones (hechos)

Se recomienda conservar el encabezado del pedido y las líneas separadas.

- `data/pedidos_pos.csv` y `data/pedidos_app.csv` Campos mínimos: `order_id`, `fecha`, `cliente_id`, `sucursal_id`, `canal` (`POS` o `APP`).
- `data/lineas_pos.csv` y `data/lineas_app.csv` Campos mínimos: `order_id`, `line_num`, `sku`, `cantidad`, `precio_unitario`, `promo_codigo`, `metodo_pago`, `cancelado_flag`.

  Nota: si en origen el descuento ya viene desglosado, incluya también `descuento_importe`. Si no, puede derivarse en el proceso (ver más adelante).

Carga de datos y validación de esquema

En un pipeline reproducible, **la primera etapa** consiste en cargar datos y aplicar validaciones: tipos, llaves no nulas, dominios controlados y rangos.

Chequeos mínimos de calidad

A continuación se implementan reglas mínimas. En un DW real, estas reglas alimentan un sistema de monitoreo (por ejemplo, Great Expectations) y generan reportes de calidad.

Homologación e integración de fuentes (Staging Integrado)

En Kimball, la integración lógica se garantiza mediante **dimensiones conformadas** y una **bus matrix**. Antes de construir dimensiones y hechos, se unifican las fuentes POS y App.

Integración de encabezados y líneas

Derivación de métricas transaccionales

Se recomienda almacenar importes en el hecho para evitar dependencia histórica de catálogos (p. ej., cambios de precio de lista).

	order_id	line_num	cantidad	precio_unitario	importe_bruto	descuento_importe	importe_neto	ca
0	P000007	1	2	31.47	62.94	0.0	62.94	0
1	P000007	2	1	36.67	36.67	0.0	36.67	0
2	P000001	1	3	51.26	153.78	0.0	153.78	0

	order_id	line_num	cantidad	precio_unitario	importe_bruto	descuento_importe	importe_neto	ca
3	P000001	2	3	78.11	234.33	0.0	234.33	0
4	P000010	1	3	72.02	216.06	0.0	216.06	0

Construcción de dimensiones conformadas

En un DW tipo estrella, todas las dimensiones deben tener:

- **llave sustituta** (`*_key`, entero),
- **llave natural** (clave operacional como `cliente_id`, `sku`, etc.),
- atributos descriptivos.

Para un libro y un ejemplo didáctico, se implementa SCD Tipo 1 (sobrescritura). En producción, `dim_cliente` y `dim_producto` suelen requerir SCD Tipo 2, como se vio en el capítulo anterior.

`dim_tiempo`

	tiempo_key	fecha	anio	mes	trimestre	semana_iso	dia_semana	es_fin_semana
0	1	2022-07-11	2022	7	3	28	1	0
1	2	2022-07-12	2022	7	3	28	2	0
2	3	2022-07-13	2022	7	3	28	3	0
3	4	2022-07-14	2022	7	3	28	4	0
4	5	2022-07-15	2022	7	3	28	5	0

Función auxiliar para dimensiones (Tipo 1)

	promocion_key	promo_codigo	campania	descripcion
0	1	NA	NaN	NaN
1	2	nan	Sin promo	Sin descuento
2	3	APP10	Lanzamiento App	10 % en app
3	4	2X1FRIO	Bebidas frías	2x1 bebidas frías
4	5	DULCE15	Postres	15 % en postres

`dim_cliente`, `dim_producto`, `dim_sucursal`

Construcción de la tabla de hechos (fact_linea_pedido)

El hecho se define a granularidad **línea de pedido**, con:

- Llaves sustitutas hacia dimensiones conformadas.
- Medidas aditivas (`cantidad`, `importe_*`) y banderas (`cancelado_flag`, `cupon_aplicado_flag`).
- Llave degenerada `order_id` para trazabilidad.

	order_id	line_num	tiempo_key	cliente_key	sucursal_key	producto_key	canal_key	metodo_pag
0	P000007	1	245	8	1	7	1	2
1	P000007	2	245	8	1	7	1	2
2	P000001	1	528	1	2	3	1	1
3	P000001	2	528	1	2	5	1	2
4	P000010	1	356	5	4	5	1	1

Data Marts y respuestas a preguntas estratégicas

Pregunta 1: Pedidos mensuales desde enero 2023, segmentado por canal

Se define como **número de pedidos únicos** (`COUNT DISTINCT order_id`) por mes.

Pregunta 2: Categorías con mayor demanda en el periodo post

Se recomienda reportar tanto unidades (volumen) como ingreso neto.

	categoria	unidades	ingreso_neto	pedidos
1	Bebidas Calientes	49	2545.44	13
2	Bebidas Frías	28	2037.89	8
4	Postres	35	1692.36	12
0	Accesorios	15	1386.63	7
3	Panadería	27	894.85	11

Pregunta 3: Variaciones por zona y tipo de cliente (pre vs post)

Se propone comparar **delta (post - pre)** en pedidos e ingreso neto.

	zona	tipo_cliente	ingreso_neto_post	ingreso_neto_pre	pedidos_post	pedidos_pre	pedidos_de
8	Sur	sin_lealtad	1919.22	0.00	5.0	0.0	5.0
0	Centro	lealtad	1700.87	201.77	5.0	1.0	4.0
4	Oriente	lealtad	1208.98	272.56	2.0	1.0	1.0

	zona	tipo_cliente	ingreso_neto_post	ingreso_neto_pre	pedidos_post	pedidos_pre	pedidos_de
7	Sur	lealtad	858.78	0.00	2.0	0.0	2.0
3	Norte	sin_lealtad	854.44	0.00	2.0	0.0	2.0
2	Norte	lealtad	1006.74	426.19	2.0	1.0	1.0
5	Oriente	sin_lealtad	477.21	137.77	2.0	1.0	1.0
6	Poniente	lealtad	104.19	0.00	1.0	0.0	1.0
1	Centro	sin_lealtad	426.74	407.78	1.0	1.0	0.0

Pregunta 4: Cancelación y cupones (tasas) por mes

Las tasas son **no aditivas**, por lo que deben calcularse como promedios o cocientes.

	yyyymm	tasa_cancelacion	tasa_cupon	pedidos	lineas	ingreso_neto
3	2022-12	0.000000	0.333333	1	3	272.56
4	2023-02	0.000000	0.333333	1	3	389.25
5	2023-03	0.000000	0.666667	1	3	257.86
6	2023-04	0.000000	0.000000	2	4	533.00
7	2023-05	0.000000	0.285714	2	7	797.46
8	2023-06	0.000000	0.714286	3	7	1129.59
9	2023-07	0.000000	0.000000	1	5	822.09
10	2023-08	0.166667	0.166667	1	6	696.85
11	2023-09	0.125000	0.375000	2	8	653.90
12	2023-10	0.142857	0.428571	5	14	1545.01
13	2023-11	0.000000	0.428571	3	7	961.61
14	2023-12	0.000000	0.000000	1	5	770.55

Exportación de tablas: Dimensiones y Hechos (Data Mart físico)

En una implementación típica, estas tablas se materializan como CSV/Parquet en un *lakehouse* o como tablas físicas en un motor analítico (PostgreSQL, BigQuery, Snowflake, DuckDB, etc.). Aquí se exportan a `out/`.

Discusión técnica: decisiones de diseño y extensiones

1. **Dimensiones conformadas**: `dim_tiempo`, `dim_cliente`, `dim_producto`, `dim_sucursal`, `dim_canal` permiten integrar *Ventas POS* y *Pedidos App* en una misma semántica de consulta (bus architecture).

2. **Granularidad**: al modelar a nivel de línea, se preserva el análisis de mezcla de productos y el efecto de promociones en el importe.
3. **SCD Tipo 2 (extensión recomendada)**: si se desea preservar historia de cambios en `categoria` o en `nivel_lealtad`, se debe:
   - añadir `vigencia_inicio`, `vigencia_fin`, `vigente_flag`,
   - resolver la clave dimensional "as-of" en la fecha del hecho.
4. **Late Arriving Dimensions**: en escenarios reales, pueden llegar transacciones cuyo cliente aún no existe en el catálogo. La práctica estándar es asignar **Unknown Member** (`*_key = 0`) y corregir al cierre del ciclo de carga.
5. **Prueba estadística pre/post (opcional)**: para un tratamiento académico, se sugiere formalizar el cambio con pruebas no paramétricas o bootstrap sobre series mensuales (p. ej., pedidos o tasa de cancelación).

Reproducibilidad y control de versiones

Para un libro académico, se recomienda:

- fijar versiones de dependencias (p. ej., `pandas==2.x`, `numpy==2.x`),
- mantener `data/` como directorio externo (no versionar datos sensibles),
- versionar el pipeline y el `.qmd` (Git),
- documentar el diccionario de datos final (metadatos de cada dimensión y hecho).

Parte VII.

Integración ETL/ELT

11. Introducción a PySpark y Fundamentos ETL

¿Qué es el ETL y por qué importa?

ETL (*Extract, Transform, Load*) es el proceso que mueve datos desde los sistemas operacionales (OLTP, APIs, logs, archivos) hacia el Data Warehouse, transformándolos en el camino para que sean consistentes, limpios y aptos para el análisis.

> **Nota**
>
> **Distinción con el Preprocesamiento (Parte II)** ETL es ingeniería de datos: opera sobre flujos de producción, terabytes, esquemas fijos, y su salida es reproducible y automatizada. El preprocesamiento analítico (Parte II) toma una *vista ya limpia* del DW y la transforma para un experimento concreto. El DW es el punto de transferencia entre ambos.

ETL clásico vs ELT moderno

	ETL clásico	ELT moderno
Orden	Extraer → Transformar → Cargar	Extraer → Cargar → Transformar
Dónde transforma	Motor ETL externo	Dentro del DW / lakehouse
Herramientas	Informatica, SSIS, Talend	dbt + BigQuery / Snowflake / DuckDB
Ventaja	Datos siempre limpios al llegar	Aprovechar la potencia del DW columnares
Desventaja	Cuello de botella en el motor	El DW necesita ser potente y barato

¿Por qué PySpark?

Cuando los datos superan la memoria RAM de una sola máquina, pandas deja de ser suficiente. **Apache Spark** divide el trabajo en particiones que se procesan en paralelo sobre un clúster de nodos, usando un plan de ejecución optimizado (DAG — *Directed Acyclic Graph*).

PySpark es la API Python de Spark. Con ella puedes escribir transformaciones que escalan de un laptop (modo local) a cientos de nodos (AWS EMR, Databricks, GCP Dataproc) sin cambiar el código.

Entorno de prácticas: Databricks Community Edition

Todas las prácticas de este bloque se realizan en **Databricks Community Edition**, que proporciona un clúster Spark gratuito sin necesidad de instalación local.

Configurar la cuenta

1. Ir a community.cloud.databricks.com y crear una cuenta gratuita.
2. En el menú lateral: **Compute → Create compute** → elegir el runtime **Databricks Runtime 14.x LTS (Spark 3.5, Python 3.11)**.
3. Crear un **Notebook** (lenguaje: Python).

> **!** Importante
>
> En Databricks, `spark` ya existe como variable global — **no** crees un `SparkSession` manual. Simplemente empieza con `spark.read...`

Cargar datasets a DBFS

Comandos específicos de Databricks

Comando	Descripción
`display(df)`	Tabla interactiva con gráficos integrados
`dbutils.fs.ls("dbfs:/ruta/")`	Listar archivos en DBFS
`dbutils.fs.mkdirs("dbfs:/ruta/")`	Crear directorio en DBFS
`dbutils.fs.rm("dbfs:/ruta/", True)`	Eliminar archivo o directorio
`%sql SELECT ...`	Celda SQL sobre tablas registradas
`spark.catalog.listTables()`	Ver tablas disponibles en el metastore

> **Tip**
>
> **Delta Lake está preinstalado en Databricks.** No necesitas `pip install delta-spark` — simplemente usa `.format("delta")` directamente.

Datasets para las prácticas

Dataset	Tamaño	Dónde obtenerlo en Databricks
NYC Taxi (1 mes)	~500 MB	Descargar con `urllib` (ver §2.2)
NYC Taxi (todo 2023)	~6 GB	Bucle sobre los 12 meses
TPC-H sintético	Configurable	`%pip install tpch` + `python -m tpch generate`

Dataset	Tamaño	Dónde obtenerlo en Databricks
Databricks sample data	Varios	`dbfs:/databricks-datasets/` (preinstalado)

PySpark: Fundamentos

SparkSession

En **Databricks** el `SparkSession` ya existe como `spark`. En entorno local:

El DataFrame de Spark: lazy vs eager

Concepto	Significado
Transformación	`filter`, `select`, `groupBy`, `join` — construyen el DAG
Acción	`show`, `count`, `collect`, `write` — ejecutan el DAG
Partición	Unidad de paralelismo; un archivo Parquet por partición típicamente
RDD	Capa de bajo nivel; usa DataFrame salvo casos especiales

Esquema explícito (mejor práctica)

`inferSchema=True` hace una pasada completa del archivo; con esquema explícito la lectura es más rápida y sin sorpresas de tipos.

Referencias de este capítulo

- Chambers & Zaharia. *Spark: The Definitive Guide*. O'Reilly, 2018.
- PySpark Documentation
- Databricks Community Edition
- Databricks Free Training — cursos oficiales gratuitos.

12. Extracción y Fuentes Heterogéneas

> **Nota**
>
> Este capítulo asume que tienes Databricks Community Edition configurado y los datasets cargados en DBFS según el capítulo anterior.

Extracción (*Extract*)

Lectura de formatos comunes

Particionamiento y paralelismo en la lectura

Ejercicio E-1 — Ingestar NYC Taxi

> **Importante**
>
> **Ejercicio E-1: Lectura y exploración básica**
> **Entregable:** tabla con estadísticas y respuesta a las 5 preguntas.

Integración de fuentes heterogéneas: Data Lake → DW

En la práctica, los datos llegan en formatos distintos, con nombres de columnas inconsistentes, tipos incompatibles y niveles de anidamiento variables. Esta sección cubre el conjunto de técnicas que permiten **homologar** esas fuentes antes de cargarlas al DW.

El problema: fuentes heterogéneas en la Zona RAW

Cuando un Data Lake acumula datos de múltiples fuentes, los problemas típicos son:

Problema	Ejemplo real
Formatos distintos	ventas en CSV, eventos en JSON, catálogo en XML
Nombres de columna inconsistentes	`customer_id` vs `cust_id` vs `clienteId`

Problema	Ejemplo real
Tipos incompatibles	`"2024-01-15"` (string) vs `1705276800` (epoch) vs `20240115` (int)
Anidamiento en JSON	`{"usuario": {"nombre": "Ana", "dir": {"ciudad": "CDMX"}}}`
Encodings distintos	UTF-8 vs Latin-1 vs CP1252
Delimitadores no estándar	`;` en CSV europeo, `|` en dumps de bases de datos
Nulos representados distinto	`""`, `"NULL"`, `"N/A"`, `"\\N"`, `None`
Unidades distintas	distancia en km vs millas, precios en MXN vs USD

JSON anidado: aplanar para el DW

El JSON es el formato más común en APIs y event streams, pero el DW necesita una tabla **plana** (filas × columnas escalares). PySpark ofrece tres herramientas para esto:

Técnica 1: acceder a campos anidados con notación punto

Técnica 2: `explode` — un array → una fila por elemento

> **Nota**
>
> `explode` elimina filas con array vacío o nulo. Usa `explode_outer` si quieres conservarlas como fila con `item = null`.

Técnica 3: JSON embebido como string → parsear con esquema explícito

En muchos pipelines el JSON llega como una columna de texto dentro de otro CSV o tabla (columna tipo `STRING` con JSON serializado):

Técnica 4: `get_json_object` — extracción selectiva con JSONPath

Cuando solo necesitas algunos campos de un JSON muy profundo sin parsear el esquema completo:

CSV heterogéneo: normalización de esquema entre fuentes

Un escenario muy común: el mismo tipo de dato llega de tres sistemas distintos con nombres de columna y tipos diferentes. El ETL debe producir una tabla unificada.

> **Advertencia**
>
> `unionByName` une columnas **por nombre**, no por posición. Si una fuente tiene una columna extra que las otras no, usa `unionByName(df2, allowMissingColumns=True)` para rellenarla con `null`.

Normalización de tipos y valores nulos heterogéneos

CSV con delimitadores no estándar y encodings

Conversión JSON (Data Lake) → Parquet (DW): pipeline completo

Este es el patrón más frecuente en la industria: el Data Lake almacena eventos en JSON; el DW los consume como Parquet particionado y tipado.

Reconciliación de esquemas con `mergeSchema`

Cuando el JSON evoluciona con el tiempo y los archivos del lago tienen esquemas distintos (una columna nueva apareció en febrero, otra se renombró en marzo), Spark puede reconciliarlos automáticamente:

Ejercicio E-1b — JSON a Parquet del Data Lake al DW

Importante

Ejercicio E-1b: Integración de fuentes heterogéneas
Descarga los datos de GitHub Archive (eventos de un día, 2 GB en JSON comprimido) y conviértelos a una tabla de hechos Parquet en el DW.
Preguntas: a) ¿Cuántos eventos totales hay en el día? ¿Qué fracción son PushEvents? b) El campo `payload` es diferente para cada tipo de evento. ¿Cómo manejarías esto si quisieras modelar cada tipo como su propia tabla de hechos? c) ¿Qué ventaja tiene Parquet vs JSON para consultas repetidas sobre estos datos? Compara el tiempo de `COUNT(*)` sobre el JSON original vs el Parquet.

Referencias de este capítulo

- NYC TLC Trip Record Data
- GitHub Archive

13. Transformación, Carga y Pipeline ETL

> **Nota**
>
> Continuación del capítulo anterior. Asume que tienes el DataFrame `df` cargado desde DBFS.

Transformación (*Transform*)

Funciones de columna (`pyspark.sql.functions`)

Limpieza de datos a escala

Aggregaciones y *Window Functions*

Joins a escala

> **Advertencia**
>
> **Evitar el Cartesian Product.** Un join sin condición sobre DataFrames grandes puede generar `N×M` filas y agotar la memoria del clúster. Spark lanzará un error si intentas un cross join sin `.crossJoin()` explícito.

Ejercicio E-2 — Pipeline de limpieza completo

> **Importante**
>
> **Ejercicio E-2: Transformación y calidad**
> Usando el Parquet del Ejercicio E-1:
> **Preguntas de análisis:** a) ¿Cuántas filas se perdieron en cada paso de limpieza? b) ¿En qué hora del día el ticket promedio es más alto? c) ¿Los viernes tienen más propinas que los lunes?

Transformaciones SQL con Spark

Spark SQL permite usar sentencias SQL estándar sobre DataFrames:

> **Tip**
>
> Si ya conoces SQL, puedes empezar con Spark SQL y aprender la API DataFrame gradualmente. Ambas generan exactamente el mismo plan de ejecución optimizado.

Carga (*Load*)

Escritura en Parquet (formato estándar del lakehouse)

Delta Lake — el formato lakehouse con ACID

Escritura a una base de datos (JDBC)

Pipeline ETL completo

Ahora unimos Extract + Transform + Load en una función reutilizable:

Referencias de este capítulo

- Delta Lake Documentation
- PySpark SQL Functions

14. Prácticas ETL en Databricks

Este capítulo contiene los ejercicios de aplicación. Todos se ejecutan en **Databricks Community Edition** usando el clúster configurado en el primer capítulo del bloque.

Tip

Workflow sugerido por ejercicio: 1. Crear un notebook nuevo en Databricks con el nombre del ejercicio. 2. En la primera celda: `# Ejercicio E-X - Título` 3. Ejecutar celda por celda, observar el Spark UI para ver el plan de ejecución. 4. Adjuntar capturas del resultado como evidencia.

Ejercicios

Ejercicio E-3 — Pipeline incremental con Delta Lake

Importante

Ejercicio E-3: Carga incremental
Simula un pipeline de producción que carga datos mes a mes sin duplicar registros.
Preguntas: a) ¿Qué pasa si cargas el mismo mes dos veces? ¿Por qué? b) Modifica el MERGE para que si un viaje ya existe pero tiene distinto `total_amount`, actualice el registro (simula una corrección tardía). c) Usa `DESCRIBE HISTORY delta.\output/delta_taxi/“` para ver el log de transacciones.

Ejercicio E-4 — ETL multi-fuente con join a escala

Importante

Ejercicio E-4: Enriquecimiento con join
Descarga el catálogo de zonas y úsalo para enriquecer los viajes con el nombre del barrio.
Preguntas: a) ¿Por qué usamos `broadcast` para el join con zonas? b) ¿Qué ocurriría si quitamos el `broadcast` y el DataFrame de zonas fuera de 10 GB en vez de 26 KB? c) Extiende el enriquecimiento para incluir también la zona de destino (`DOLocationID`).

Ejercicio E-5 — Detección de calidad con Great Expectations

Importante

Ejercicio E-5: Validación automatizada
Preguntas: a) ¿Qué expectativas fallan? ¿Cómo lo interpretarías en un pipeline real? b) Agrega una expectativa para verificar que `passenger_count` nunca supere 6. c) ¿Dónde colocarías estas validaciones en el pipeline E→T→L?

Ejercicio E-6 — Proyecto integrador TPC-H

Importante

Ejercicio E-6: ETL sobre esquema estrella
Genera datos sintéticos con TPC-H (escala 1 GB) y construye un DW en Parquet.
Este ejercicio simula exactamente un pipeline ETL empresarial: extracción de tablas OLTP → joins dimensionales → tabla de hechos → Parquet.

Referencias de este capítulo

- NYC TLC Trip Record Data
- Great Expectations
- TPC-H Benchmark
- Delta Lake Documentation

15. Orquestación y Herramientas Modernas

Una vez dominado PySpark, el siguiente paso es automatizar los pipelines y adoptar las herramientas que usa la industria para transformaciones declarativas y scheduling de producción.

ETL moderno: dbt + PySpark

dbt (*data build tool*) separa la *transformación* del pipeline técnico: defines las transformaciones como modelos SQL, y dbt se encarga de ejecutarlos en el orden correcto, con pruebas automáticas y documentación.

Orquestación con Apache Airflow

En producción, los pipelines ETL se ejecutan automáticamente según un calendario. **Airflow** define estos flujos como DAGs de Python:

> Tip
>
> **Alternativas a Airflow:** - **Prefect** — más Python-nativo, sin XML/YAML, fácil para equipos pequeños. - **Dagster** — fuerte en observabilidad y gestión de assets de datos. - Para empezar: **Prefect** + **PySpark local** → luego migrar a Airflow en producción.

Resumen y mapa de herramientas

Etapa	Herramienta	Cuándo usarla
Extract	PySpark	Archivos grandes (>1 GB), clústeres, Parquet/JDBC
Extract	Kafka + Debezium	Streaming en tiempo real, CDC
Transform	Spark SQL / DataFrame API	Joins complejos, window functions, ML features
Transform	dbt	Transformaciones SQL versionadas sobre DW cloud

Etapa	Herramienta	Cuándo usarla
Quality	Great Expectations	Validación declarativa antes de la carga
Load	Delta Lake / Iceberg	ACID, upsert, time travel en el lakehouse
Load	BigQuery / Snowflake	DW cloud sin gestión de infraestructura
Load	DuckDB	Notebooks, sin clúster, prototipos rápidos
Orquestar	Airflow	Pipelines de producción complejos con SLA
Orquestar	Prefect	Equipos pequeños, Python-native, simplicidad

---

Referencias y recursos adicionales

- Chambers & Zaharia. *Spark: The Definitive Guide*. O'Reilly, 2018.
- PySpark Documentation — API reference completa.
- Delta Lake Documentation — ACID y time travel.
- dbt Documentation — transformaciones SQL versionadas.
- Great Expectations — validación de calidad.
- NYC TLC Trip Record Data — dataset de ejercicios.
- TPC-H Benchmark — esquema de referencia para DW.

Parte VIII.

Modelado para consumo

16. Computación de cubos de datos OLAP

Un enfoque técnico y reproducible

Los cubos de datos son una forma de organizar información multidimensional para análisis resumido, exploración interactiva y toma de decisiones. En una arquitectura típica de *data warehouse*, los datos se extraen de sistemas transaccionales, pasan por procesos de limpieza e integración, se almacenan en un repositorio analítico y se presentan mediante reportes, minería de datos o cubos OLAP. Esta secuencia aparece en la presentación base, donde se recuerda el flujo general desde fuentes de datos hasta almacenamiento, *data marts* y presentación analítica (Santizo Galicia 2026).

El objetivo de este capítulo es convertir esa idea en una formulación técnica y reproducible. Se presentará el concepto de procesamiento analítico en línea, la estructura matemática de un cubo de datos, sus principales operaciones y una implementación práctica en Python. La discusión se apoya en la tradición de almacenes de datos (Inmon 2005; Kimball y Ross 2013), en la literatura clásica sobre OLAP y cubos de datos (Chaudhuri y Dayal 1997; Gray et al. 1996), y en herramientas modernas de análisis tabular como `pandas` (McKinney 2010; The pandas development team 2024).

Un cubo OLAP no debe entenderse únicamente como una figura tridimensional. En realidad, es una abstracción para representar medidas agregadas sobre varias dimensiones. Por ejemplo, una empresa puede analizar ventas por tiempo, región y producto. Cada combinación específica de esas dimensiones define una celda del cubo, y esa celda contiene una medida, como ventas totales, unidades vendidas, costo promedio o margen de ganancia.

Del almacén de datos al análisis OLAP

Un sistema transaccional u OLTP (*Online Transaction Processing*) está optimizado para registrar operaciones individuales: compras, pagos, inventarios, altas de clientes o movimientos bancarios. En contraste, un sistema OLAP (*Online Analytical Processing*) está optimizado para consultar datos agregados y responder preguntas de negocio. Esta distinción es importante porque el análisis multidimensional requiere estructuras distintas a las de una base de datos operacional.

> **Nota**
>
> Definición. Un sistema OLAP es una estructura analítica que organiza medidas cuantitativas mediante dimensiones categóricas o jerárquicas para permitir consultas agregadas desde distintas perspectivas.
>
> $$C : D_1 \times D_2 \times \cdots \times D_k \longrightarrow \mathbb{R}^m$$

En esta definición, $D_1, \ldots, D_k$ representan dimensiones como tiempo, región, producto o sucursal. El valor $C(d_1, \ldots, d_k)$ contiene una o varias medidas agregadas. Por ejemplo, si D_1 es tiempo, D_2 es región y D_3 es producto, entonces una celda puede representar las ventas de computadoras en la región Centro durante el trimestre T1.

La presentación original destaca que en un *data warehouse* los datos se organizan en una estructura multidimensional, que puede implementarse como un cubo OLAP y analizarse desde diferentes perspectivas (Santizo Galicia 2026). Esta idea se puede conectar con tres componentes esenciales:

1. **Dimensiones:** ejes de análisis, como tiempo, región o producto.
2. **Jerarquías:** niveles dentro de una dimensión, como día → mes → trimestre → año.
3. **Medidas:** variables numéricas agregables, como ventas, unidades, utilidad o presupuesto.

Una ventaja central de OLAP es que permite adaptar el nivel de detalle a cada usuario. Un gerente general puede consultar ventas totales por país; una dirección regional puede revisar ventas por estado; y un supervisor puede analizar ventas por tienda. La misma base de datos analítica soporta múltiples perspectivas mediante operaciones de navegación sobre el cubo.

Modelo multidimensional

Supongamos un conjunto de transacciones $\mathcal{T}$ donde cada registro contiene atributos de dimensión y una medida de ventas. Una transacción puede escribirse como

$$t_i = (r_i, \tau_i, p_i, v_i),$$

donde r_i indica la región, τ_i el periodo, p_i el producto y v_i el valor de venta. El cubo agregado por suma se obtiene agrupando transacciones que comparten las mismas coordenadas dimensionales.

> **Nota**
>
> Definición. Dado un conjunto de transacciones $\mathcal{T}$, el valor de una celda de un cubo de ventas se define como la suma de las ventas de todas las transacciones que coinciden con una combinación de dimensiones:
>
> $$C(r, \tau, p) = \sum_{i:\, r_i = r,\, \tau_i = \tau,\, p_i = p} v_i$$

Esta formulación deja claro que un cubo es una función agregada. La representación visual tridimensional es útil para enseñar el concepto, pero en la práctica pueden existir muchas más dimensiones. Un cubo de ventas real puede incluir canal de venta, tipo de cliente, vendedor, tienda, campaña publicitaria y categoría fiscal.

Dimensiones, medidas y granularidad

La **granularidad** indica el nivel de detalle con que se almacenan o consultan los datos. Una tabla con ventas diarias por tienda y producto tiene mayor granularidad que una tabla con ventas mensuales por región y categoría. Diseñar correctamente la granularidad es una decisión crítica en un almacén de

datos, porque afecta el almacenamiento, la velocidad de consulta y la flexibilidad analítica (Kimball y Ross 2013).

> **Nota**
>
> Definición. La granularidad de un cubo es el nivel mínimo de detalle con el que se representan sus dimensiones.
>
> $$G(C) = \{\ell_1, \ell_2, \dots, \ell_k\}$$

Aquí ℓ_j representa el nivel usado en la dimensión D_j. Por ejemplo, $G(C) = \{\text{mes}, \text{estado}, \text{producto}\}$ indica que el cubo se consulta por mes, estado y producto. Si se cambia de mes a semana, la granularidad aumenta; si se cambia de estado a país, la granularidad disminuye.

Operaciones fundamentales en cubos OLAP

Las operaciones OLAP permiten navegar entre distintos niveles y subconjuntos del cubo. La presentación base introduce cinco operaciones principales: *drill-down*, *roll-up*, *slice*, *dice* y *pivot* o *rotate* (Santizo Galicia 2026). A continuación se formalizan y se muestran ejemplos reproducibles.

Drill-down

El *drill-down* permite pasar de datos resumidos a datos más detallados. Por ejemplo, si una empresa observa que en 2025 vendió 1,000 productos, puede preguntar cuánto se vendió por trimestre, por mes, por estado o por producto. En términos de jerarquías, el *drill-down* se mueve hacia niveles más específicos.

> **Nota**
>
> Definición. Una operación de drill-down reemplaza un nivel agregado ℓ de una dimensión por un nivel más detallado ℓ' dentro de la misma jerarquía.
>
> $$\ell' \prec \ell$$

Por ejemplo, en la jerarquía temporal

$$\text{año} \to \text{trimestre} \to \text{mes} \to \text{semana} \to \text{día},$$

pasar de año a trimestre o de mes a semana es una operación de *drill-down*. En la presentación, esta operación se ilustra al pasar de una vista mensual a una vista semanal para el mes de enero (Santizo Galicia 2026).

Roll-up

El *roll-up* es la operación inversa. Permite agrupar datos detallados para obtener resúmenes de mayor nivel. Por ejemplo, las ventas diarias pueden agregarse como ventas mensuales, trimestrales o anuales; las ventas por tienda pueden agregarse como ventas por estado o por país.

> **Nota**
>
> Definición. Una operación de roll-up reemplaza un nivel detallado ℓ' de una dimensión por un nivel más agregado ℓ y combina las celdas mediante una función de agregación A.
>
> $$C_\ell(d) = A\left(\{C_{\ell'}(d') : \ d' \mapsto d\}\right)$$

La función A puede ser una suma, un promedio, un máximo, un conteo o una medida derivada. En ventas suele usarse la suma; en precios puede ser más adecuado usar promedio ponderado; en inventarios puede usarse el valor final de periodo.

Slice

La operación *slice* fija un valor específico en una dimensión y devuelve un subcubo con una dimensión menos. Si el cubo original tiene dimensiones Región, Tiempo y Producto, fijar Producto = Tablets produce una vista Región × Tiempo.

> **Nota**
>
> Definición. Dado un cubo $C : D_1 \times \cdots \times D_k \to \mathbb{R}$, una operación slice fija una dimensión D_j en un valor $a \in D_j$.
>
> $$C_{D_j=a}(d_1, \dots, d_{j-1}, d_{j+1}, \dots, d_k) = C(d_1, \dots, a, \dots, d_k)$$

La presentación ejemplifica esta operación al seleccionar un solo estado, un solo producto o un solo mes (Santizo Galicia 2026).

Dice

La operación *dice* selecciona subconjuntos en dos o más dimensiones al mismo tiempo. A diferencia de *slice*, que fija un único valor en una dimensión, *dice* puede conservar varios valores por dimensión.

> **Nota**
>
> Definición. Una operación dice restringe el cubo a subconjuntos $S_j \subseteq D_j$ en una o varias dimensiones.
>
> $$C_{S_1,\dots,S_k} = C\big|_{S_1 \times \cdots \times S_k}$$

Por ejemplo, se puede seleccionar Tiempo $\in \{\text{Enero}, \text{Febrero}\}$ y Producto $\in \{\text{Smartphones}, \text{Tablets}\}$. También se puede restringir Región $\in \{\text{Jalisco}, \text{Estado de México}\}$ y Tiempo $\in \{\text{Enero}, \text{Febrero}\}$.

Pivot o rotate

La operación *pivot* cambia la orientación de la vista sin alterar los datos subyacentes. En una tabla, esto equivale a intercambiar filas y columnas o a reordenar dimensiones para responder otra pregunta. Por ejemplo, una tabla con semanas en filas y productos en columnas responde: "¿cuánto se vendió de cada producto en cada semana?". Al pivotar, una tabla con productos en filas y semanas en columnas responde: "¿cómo se comportó cada producto a lo largo de las semanas?" (Santizo Galicia 2026).

> **Nota**
>
> Definición. Una operación pivot aplica una permutación π al orden de las dimensiones de un cubo.
>
> $$C^{\pi}(d_{\pi(1)}, \dots, d_{\pi(k)}) = C(d_1, \dots, d_k)$$

Esta operación es especialmente útil para visualización. No cambia la granularidad ni los valores agregados; cambia la forma en que se presentan los ejes.

Cálculo manual de un cubo OLAP en un dataset pequeño

Antes de implementar operaciones OLAP con Python, conviene resolver un ejemplo pequeño a mano. El objetivo didáctico es que el estudiante pueda reconocer las dimensiones, identificar las celdas del cubo, calcular totales y ejecutar operaciones básicas como *slice* y *roll-up* sin depender todavía de una biblioteca computacional.

Consideremos una empresa con ventas trimestrales por **tiempo**, **ubicación** y **departamento**. Los departamentos son

$$(M, P, D, E) = (\text{Madera}, \text{Plomería}, \text{Decoración}, \text{Eléctrico}).$$

La medida del cubo será el valor de ventas. Las dimensiones son:

- $D_1 = \text{Ubicación} = \{\text{CDMX}, \text{Guadalajara}, \text{Monterrey}, \text{Mérida}\}$.
- $D_2 = \text{Tiempo} = \{T1, T2\}$.
- $D_3 = \text{Departamento} = \{M, P, D, E\}$.

Por tanto, el cubo puede representarse como una función

$$C : \text{Ubicación} \times \text{Tiempo} \times \text{Departamento} \longrightarrow \mathbb{R}.$$

La siguiente tabla contiene los datos originales.

Tiempo	CDMX (M, P, D, E)	Guadalajara (M, P, D, E)
T1	$(70, 40, 20, 30)$	$(60, 35, 25, 40)$
T2	$(50, 60, 22, 28)$	$(45, 50, 30, 35)$

Tiempo	Monterrey (M, P, D, E)	Mérida (M, P, D, E)
T1	$(48, 38, 18, 26)$	$(52, 42, 15, 37)$
T2	$(36, 55, 25, 30)$	$(44, 33, 20, 28)$

Paso 1: representar las celdas del cubo

Cada número de la tabla corresponde a una celda del cubo. Por ejemplo,

$$C(\text{CDMX}, T1, \text{Madera}) = 70,$$

$$C(\text{CDMX}, T1, \text{Eléctrico}) = 30,$$

$$C(\text{Mérida}, T2, \text{Plomería}) = 33.$$

En este caso hay

$$4 \text{ ubicaciones} \times 2 \text{ trimestres} \times 4 \text{ departamentos} = 32$$

celdas elementales. La representación del cubo consiste en organizar esas 32 celdas de forma que puedan consultarse por cualquiera de sus dimensiones.

Paso 2: calcular totales por ubicación

El total por ubicación se obtiene sumando todos los trimestres y todos los departamentos de una misma ubicación:

$$\text{Total}(u) = \sum_{t \in \{T1, T2\}} \sum_{d \in \{M, P, D, E\}} C(u, t, d).$$

Para CDMX:

$$\begin{aligned} \text{Total(CDMX)} &= (70 + 40 + 20 + 30) + (50 + 60 + 22 + 28) \\ &= 160 + 160 \\ &= 320. \end{aligned}$$

Para Guadalajara:

$$\begin{aligned} \text{Total(Guadalajara)} &= (60 + 35 + 25 + 40) + (45 + 50 + 30 + 35) \\ &= 160 + 160 \\ &= 320. \end{aligned}$$

Para Monterrey:

$$\begin{aligned}\text{Total(Monterrey)} &= (48 + 38 + 18 + 26) + (36 + 55 + 25 + 30) \\ &= 130 + 146 \\ &= 276.\end{aligned}$$

Para Mérida:

$$\begin{aligned}\text{Total(Mérida)} &= (52 + 42 + 15 + 37) + (44 + 33 + 20 + 28) \\ &= 146 + 125 \\ &= 271.\end{aligned}$$

La tabla de totales por ubicación queda:

Ubicación	Total
CDMX	320
Guadalajara	320
Monterrey	276
Mérida	271
Total general	**1187**

Paso 3: calcular totales por tiempo

El total por tiempo se obtiene sumando todas las ubicaciones y departamentos para cada trimestre:

$$\text{Total}(t) = \sum_{u} \sum_{d} C(u, t, d).$$

Para $T1$:

$$\begin{aligned}\text{Total}(T1) &= (160) + (160) + (130) + (146) \\ &= 596.\end{aligned}$$

Para $T2$:

$$\begin{aligned}\text{Total}(T2) &= (160) + (160) + (146) + (125) \\ &= 591.\end{aligned}$$

Tiempo	Total
T1	596
T2	591
Total general	**1187**

Paso 4: calcular totales por departamento

Ahora se suman todas las ubicaciones y tiempos para cada departamento:

$$\text{Total}(d) = \sum_{u} \sum_{t} C(u, t, d).$$

Para Madera:

$$70 + 50 + 60 + 45 + 48 + 36 + 52 + 44 = 405.$$

Para Plomería:

$$40 + 60 + 35 + 50 + 38 + 55 + 42 + 33 = 353.$$

Para Decoración:

$$20 + 22 + 25 + 30 + 18 + 25 + 15 + 20 = 175.$$

Para Eléctrico:

$$30 + 28 + 40 + 35 + 26 + 30 + 37 + 28 = 254.$$

Departamento	Total
Madera	405
Plomería	353
Decoración	175
Eléctrico	254
Total general	**1187**

Paso 5: operación slice a mano

Supongamos que se pide aplicar un *slice* con las siguientes condiciones:

$$\text{ubicación} \in \{\text{CDMX}, \text{Mérida}\}, \qquad \text{tiempo} = T1,$$

$$\text{departamento} \in \{\text{Madera}, \text{Eléctrico}\}.$$

Aunque en sentido estricto un *slice* fija una dimensión en un valor, en ejercicios introductorios suele combinarse con una selección de subconjuntos. En una terminología más precisa, fijar $T1$ es un *slice*, mientras que seleccionar varias ubicaciones y varios departamentos es una restricción tipo *dice* sobre el subcubo resultante.

Se buscan únicamente las celdas:

$$C(\text{CDMX}, T1, \text{Madera}) = 70,$$

$$C(\text{CDMX}, T1, \text{Eléctrico}) = 30,$$

$$C(\text{Mérida}, T1, \text{Madera}) = 52,$$

$$C(\text{Mérida}, T1, \text{Eléctrico}) = 37.$$

Por tanto, el resultado del subcubo es:

Ubicación	Tiempo	Madera	Eléctrico	Total
CDMX	T1	70	30	100
Mérida	T1	52	37	89
Total		**122**	**67**	**189**

Paso 6: operación roll-up a mano

Ahora agrupemos las ubicaciones en regiones. Una posible correspondencia es:

Ubicación	Región
Monterrey	Norte
CDMX	Centro
Guadalajara	Occidente
Mérida	Sureste

El *roll-up* por región se calcula reemplazando la dimensión ubicación por una dimensión más agregada llamada región:

$$C_{\text{región}}(r, t, d) = \sum_{u:\ g(u)=r} C(u, t, d),$$

donde $g(u)$ asigna cada ubicación a su región. In this example, each region has only one location, so the numerical values are the same as the original values, but the conceptual operation is important: the cube changes from the level *ubicación* to the level *región*.

Región	Tiempo	Madera	Plomería	Decoración	Eléctrico	Total
Centro	T1	70	40	20	30	160
Centro	T2	50	60	22	28	160
Norte	T1	48	38	18	26	130

Región	Tiempo	Madera	Plomería	Decoración	Eléctrico	Total
Norte	T2	36	55	25	30	146
Occidente	T1	60	35	25	40	160
Occidente	T2	45	50	30	35	160
Sureste	T1	52	42	15	37	146
Sureste	T2	44	33	20	28	125
Total		**405**	**353**	**175**	**254**	**1187**

Si se desea un *roll-up* regional más agregado, también se pueden definir zonas. Por ejemplo,

$$\text{Zona Norte-Centro} = \{\text{Monterrey}, \text{CDMX}\},$$

$$\text{Zona Occidente-Sureste} = \{\text{Guadalajara}, \text{Mérida}\}.$$

Entonces, para la Zona Norte-Centro en $T1$:

$$\begin{aligned} C(\text{Zona Norte-Centro}, T1, M) &= 48 + 70 = 118, \\ C(\text{Zona Norte-Centro}, T1, P) &= 38 + 40 = 78, \\ C(\text{Zona Norte-Centro}, T1, D) &= 18 + 20 = 38, \\ C(\text{Zona Norte-Centro}, T1, E) &= 26 + 30 = 56. \end{aligned}$$

El resultado completo por zonas es:

Zona	Tiempo	Madera	Plomería	Decoración	Eléctrico	Total
Norte-Centro	T1	118	78	38	56	290
Norte-Centro	T2	86	115	47	58	306
Occidente-Sureste	T1	112	77	40	77	306
Occidente-Sureste	T2	89	83	50	63	285
Total		**405**	**353**	**175**	**254**	**1187**

Este ejemplo muestra que calcular un cubo a mano consiste en tres acciones básicas: identificar coordenadas dimensionales, sumar celdas según la agregación solicitada y conservar claramente el nivel de granularidad usado en cada resultado.

Ejemplo reproducible en Python

La presentación propone un ejercicio con ventas trimestrales de una empresa mexicana de electrónica. Las dimensiones son Región, Tiempo y Producto, y la medida son ventas expresadas en cientos de miles de pesos mexicanos (Santizo Galicia 2026). A continuación se construye el cubo de datos en Python.

	region	tiempo	producto	ventas
0	Norte	T1	Televisores	35
1	Norte	T1	Computadoras	22
2	Norte	T1	Lavadoras	10
3	Norte	T1	Cámaras	14
4	Norte	T2	Televisores	28

Representación como cubo

En `pandas`, una forma natural de representar el cubo es usar un índice múltiple. Cada celda queda identificada por una tupla `(region, tiempo, producto)`.

region	tiempo	producto	ventas
Centro	T1	Computadoras	30
		Cámaras	11
		Lavadoras	18
		Televisores	40
	T2	Computadoras	28
		Cámaras	14
		Lavadoras	20
		Televisores	32
	T3	Computadoras	19
		Cámaras	10

También se puede construir una vista tabular con productos como columnas. Esta vista es conveniente para reportes, aunque internamente sigue representando el mismo cubo.

region	producto tiempo	Computadoras	Cámaras	Lavadoras	Televisores
Centro	T1	30	11	18	40
	T2	28	14	20	32
	T3	19	10	15	25
	T4	35	16	22	38
Norte	T1	22	14	10	35
	T2	26	17	13	28
	T3	15	12	8	18
	T4	33	15	16	30
Occidente	T1	27	6	9	20
	T2	31	8	11	24
	T3	36	9	7	29
	T4	34	5	10	19
Sureste	T1	17	9	12	26
	T2	20	11	16	21

region	producto tiempo	Computadoras	Cámaras	Lavadoras	Televisores
	T3	18	10	14	23
	T4	24	12	19	27

Implementación de operaciones OLAP

Slice: fijar un trimestre

El ejercicio pide realizar una operación *slice* fijando Tiempo = T2. El resultado conserva las dimensiones Región y Producto.

producto region	Computadoras	Cámaras	Lavadoras	Televisores
Centro	28	14	20	32
Norte	26	17	13	28
Occidente	31	8	11	24
Sureste	20	11	16	21

Esta salida permite comparar las ventas del segundo trimestre entre regiones y productos.

Dice: seleccionar regiones, tiempos y productos

El ejercicio también solicita una operación *dice* con Región = Norte o Centro, Tiempo = T1 o T4 y Producto = Televisores o Cámaras.

region	producto tiempo	Cámaras	Televisores
Centro	T1	11	40
	T4	16	38
Norte	T1	14	35
	T4	15	30

La operación reduce el cubo original a un subcubo con dos regiones, dos periodos y dos productos. El resultado es más pequeño, pero conserva la estructura multidimensional.

Roll-up: agrupar regiones en zonas

El ejercicio pide crear dos categorías regionales: Zona Norte-Centro y Zona Occidente-Sureste. Esto es un *roll-up* sobre la dimensión Región.

zona	producto tiempo	Computadoras	Cámaras	Lavadoras	Televisores
Zona Norte-Centro	T1	52	25	28	75
	T2	54	31	33	60
	T3	34	22	23	43
	T4	68	31	38	68
Zona Occidente-Sureste	T1	44	15	21	46
	T2	51	19	27	45
	T3	54	19	21	52
	T4	58	17	29	46

En este caso la función de agregación es la suma. Por ejemplo, las ventas de Televisores en T1 para Zona Norte-Centro se obtienen como Norte-T1-Televisores más Centro-T1-Televisores.

Pivot: cambiar la orientación de la consulta

Para estudiar la evolución temporal de cada producto, se puede pivotar la tabla y colocar productos en filas y trimestres en columnas.

tiempo producto	T1	T2	T3	T4
Computadoras	96	105	88	126
Cámaras	40	50	41	48
Lavadoras	49	60	44	67
Televisores	121	105	95	114

Esta vista responde una pregunta distinta: ya no se enfatiza la región, sino el comportamiento temporal agregado de cada producto.

Exploración interactiva del cubo

Esta sección usa widgets interactivos (OJS) que solo funcionan en la versión web del libro. Consúltala en línea para explorar las cinco operaciones OLAP (slice, dice, roll-up, drill-down, pivot) sobre el cubo de ejemplo.

Agregaciones y medidas derivadas

Las medidas de un cubo pueden ser simples o derivadas. Una medida simple es la suma de ventas; una medida derivada puede ser participación porcentual, crecimiento trimestral o margen. En OLAP conviene distinguir entre medidas aditivas, semi-aditivas y no aditivas (Kimball y Ross 2013).

> **Nota**
>
> Definición. Una medida aditiva puede agregarse mediante suma a lo largo de todas las dimensiones relevantes del cubo.
>
> $$M(A \cup B) = M(A) + M(B), \qquad A \cap B = \emptyset$$

Las ventas son usualmente aditivas: se pueden sumar por producto, región y tiempo. En cambio, una tasa de conversión no debe agregarse sumando porcentajes; debe recalcularse a partir de numeradores y denominadores.

El siguiente ejemplo calcula ventas totales por producto y participación porcentual dentro del total.

	producto	ventas	participacion
0	Computadoras	415	0.332266
1	Cámaras	179	0.143315
2	Lavadoras	220	0.176141
3	Televisores	435	0.348279

También se puede calcular el crecimiento entre trimestres para cada producto. En este caso, el cubo se reorganiza por producto y tiempo.

	producto	tiempo	ventas	crecimiento
0	Computadoras	T1	96	NaN
1	Computadoras	T2	105	0.093750
2	Computadoras	T3	88	-0.161905
3	Computadoras	T4	126	0.431818
4	Cámaras	T1	40	NaN
5	Cámaras	T2	50	0.250000
6	Cámaras	T3	41	-0.180000
7	Cámaras	T4	48	0.170732
8	Lavadoras	T1	49	NaN
9	Lavadoras	T2	60	0.224490
10	Lavadoras	T3	44	-0.266667
11	Lavadoras	T4	67	0.522727
12	Televisores	T1	121	NaN
13	Televisores	T2	105	-0.132231
14	Televisores	T3	95	-0.095238
15	Televisores	T4	114	0.200000

Ventajas, limitaciones y criterios de diseño

La presentación enumera como ventajas de OLAP el análisis rápido de grandes datos, la visión multidimensional, la identificación de patrones y tendencias, el soporte para decisiones y la comparación entre periodos, regiones o productos (Santizo Galicia 2026). Estas ventajas se explican

por la preagregación, el modelado dimensional y la separación entre cargas transaccionales y cargas analíticas.

Sin embargo, OLAP también tiene limitaciones. Puede requerir mucho almacenamiento, no siempre se actualiza en tiempo real, el diseño del cubo puede ser complejo y depende de una buena calidad de datos (Santizo Galicia 2026). En un entorno profesional, estas limitaciones se traducen en decisiones concretas:

- Definir qué dimensiones realmente aportan valor analítico.
- Elegir la granularidad mínima necesaria.
- Evitar jerarquías ambiguas o inconsistentes.
- Validar calidad de datos antes de construir agregados.
- Documentar las reglas de negocio usadas en cada medida.

Un error común consiste en pensar que un cubo OLAP resuelve automáticamente la calidad de los datos. En realidad, el cubo amplifica los problemas de integración: categorías mal escritas, fechas incompletas, productos duplicados o regiones inconsistentes pueden producir agregaciones incorrectas.

Buenas prácticas para documentación reproducible

Cuando un cubo se usa en un reporte técnico o curso de ciencia de datos, conviene documentar el proceso completo: fuente de datos, transformaciones, reglas de agregación y consultas realizadas. Quarto permite combinar explicación, matemáticas, código ejecutable y resultados en un mismo documento. Para que el análisis sea reproducible, se recomienda:

1. Cargar los datos desde una fuente controlada.
2. Construir el cubo mediante código, no manualmente.
3. Separar definiciones conceptuales de operaciones computacionales.
4. Mostrar consultas representativas: *slice*, *dice*, *roll-up*, *drill-down* y *pivot*.
5. Verificar que las agregaciones coincidan con las reglas de negocio.

El siguiente bloque resume una función reutilizable para construir vistas OLAP simples a partir de una tabla larga.

region	producto tiempo	Computadoras	Cámaras	Lavadoras	Televisores
Centro	T1	30	11	18	40
	T2	28	14	20	32
	T3	19	10	15	25
	T4	35	16	22	38
Norte	T1	22	14	10	35
	T2	26	17	13	28
	T3	15	12	8	18
	T4	33	15	16	30
Occidente	T1	27	6	9	20
	T2	31	8	11	24
	T3	36	9	7	29

region	producto tiempo	Computadoras	Cámaras	Lavadoras	Televisores
Sureste	T4	34	5	10	19
	T1	17	9	12	26
	T2	20	11	16	21
	T3	18	10	14	23
	T4	24	12	19	27

Conclusiones

Los cubos de datos OLAP son una herramienta central para organizar, resumir y explorar información multidimensional. Su valor no está solo en la representación visual de un cubo, sino en el modelo conceptual que conecta dimensiones, jerarquías, medidas y operaciones de navegación. A partir de una tabla transaccional, un cubo permite responder preguntas como: cuánto se vendió, dónde se vendió, cuándo ocurrió, qué productos dominaron y cómo cambian los patrones al modificar el nivel de agregación.

Las operaciones *drill-down*, *roll-up*, *slice*, *dice* y *pivot* constituyen el lenguaje básico para interactuar con un cubo. En conjunto, permiten pasar de resúmenes ejecutivos a análisis detallados, restringir subconjuntos relevantes y reorientar la información según la pregunta de negocio. Implementarlas en Python con `pandas` facilita la reproducibilidad y ayuda a conectar la teoría de OLAP con prácticas modernas de ciencia de datos.

Finalmente, el diseño de un cubo exige decisiones cuidadosas sobre granularidad, calidad de datos, jerarquías y medidas. Un cubo bien diseñado mejora la interpretación y la toma de decisiones; un cubo mal diseñado puede producir conclusiones engañosas aunque las consultas sean técnicamente correctas.

17. Data Marts: del Almacén de Datos al Consumo por Área de Negocio

Arquitectura, diseño dimensional y ejemplo reproducible

En el capítulo anterior se definió un cubo OLAP como una función que agrega medidas sobre varias dimensiones, $C : D_1 \times \cdots \times D_k \to \mathbb{R}^m$. Esa función necesita datos organizados de forma dimensional para poder calcularse de manera eficiente. La pregunta natural es: ¿de dónde salen exactamente esas tablas de hechos y dimensiones que el cubo consulta? La respuesta, en la mayoría de las organizaciones, es un **data mart**: un recorte del almacén de datos empresarial, diseñado para un área de negocio específica, sobre el cual se construyen los cubos, los reportes y —como se verá en el siguiente capítulo— las APIs de consumo analítico.

Este capítulo formaliza el concepto de data mart, distingue sus arquitecturas (dependiente, independiente e híbrida), presenta el proceso de diseño dimensional de Kimball y lo aplica al mismo caso de la ferretería multiestado usado en el capítulo de cubos OLAP (Kimball y Ross 2013; Inmon 2005). Así, el data mart que se construye aquí es exactamente la fuente que alimentaría el cubo Ubicación × Tiempo × Departamento discutido previamente.

¿Qué es un data mart?

Un **data warehouse** (DW) empresarial integra datos de múltiples procesos de negocio —ventas, inventario, recursos humanos, finanzas— en un repositorio único, histórico y consistente. Sin embargo, pocas veces un analista de un área específica necesita consultar *todo* el DW: al equipo de Ventas le interesan las transacciones de venta; a Recursos Humanos, la nómina y la rotación de personal. Consultar directamente un DW completo, con decenas de tablas de hechos y cientos de atributos, resulta ineficiente y difícil de entender para usuarios de negocio.

> Definición. Un **data mart** es un subconjunto especializado de un data warehouse, delimitado por un área funcional o proceso de negocio, diseñado para responder las preguntas analíticas de un grupo específico de usuarios.
>
> $$DM_j \subseteq DW, \qquad DM_j = \pi_{A_j}(\sigma_{P_j}(DW))$$

En esta notación, σ_{P_j} selecciona los hechos correspondientes al proceso de negocio P_j (por ejemplo, ventas) y π_{A_j} proyecta únicamente los atributos y dimensiones relevantes para esa área A_j. El resultado es una porción del almacén, más pequeña, más rápida de consultar y más fácil de entender para su audiencia (Devlin y Murphy 1988).

Tres características distinguen a un data mart de un data warehouse completo:

1. **Alcance acotado.** Cubre un proceso o departamento, no la organización entera.
2. **Audiencia definida.** Se diseña pensando en las preguntas de un grupo de usuarios de negocio concreto.
3. **Modelo dimensional explícito.** Casi siempre se implementa como un esquema estrella o copo de nieve, optimizado para consultas de agregación, no como una réplica normalizada de sistemas operacionales.

Arquitecturas: dependiente, independiente e híbrida

La forma en que un data mart obtiene sus datos —directamente del DW o directamente de las fuentes— define su arquitectura y tiene consecuencias importantes sobre la consistencia de la información en toda la organización.

Data marts dependientes (enfoque Inmon)

En el enfoque de Bill Inmon, el DW corporativo se construye primero como un repositorio normalizado (tercera forma normal) que integra y depura todas las fuentes (Inmon 2005). Los data marts se derivan *después*, como vistas dimensionales de ese DW ya integrado.

> Definición. Un data mart es **dependiente** cuando su única fuente de datos es el data warehouse corporativo ya integrado.
>
> $$DM_j = f_j(DW), \qquad \text{con } DW \text{ como única fuente}$$

La ventaja principal es la consistencia: como todos los data marts parten de la misma fuente integrada, un mismo cliente, producto o empleado tiene exactamente la misma definición en cualquier data mart. El costo es tiempo: el DW corporativo debe existir y estar completo antes de poder entregar el primer data mart, lo cual puede tardar meses o años en organizaciones grandes.

Data marts independientes (enfoque bottom-up)

En el enfoque bottom-up asociado a Ralph Kimball, no es necesario esperar a un DW normalizado completo. Cada data mart se construye directamente desde las fuentes operacionales de su propio proceso de negocio, con su propio proceso ETL.

> Definición. Un data mart es **independiente** cuando se construye directamente desde sistemas fuente, sin pasar por un data warehouse integrado previo.
>
> $$DM_j = g_j(S_1, S_2, \ldots, S_n), \qquad S_i = \text{sistemas fuente}$$

La ventaja es velocidad de entrega: un equipo puede construir y poner en producción el data mart de Ventas en semanas, sin depender de que otras áreas terminen su integración. El riesgo, si no se gestiona con cuidado, es la aparición de **silos de datos** (*stovepipes*): si el data mart de Ventas

y el de Marketing definen "cliente activo" de formas distintas, sus reportes no van a coincidir, y reconciliar esa diferencia después es costoso.

Arquitectura híbrida: el bus de Kimball

Kimball resuelve el dilema de velocidad contra consistencia con la **arquitectura de bus**: los data marts se construyen de forma incremental, proceso por proceso, pero comparten un conjunto de **dimensiones conformadas** —definiciones únicas de Tiempo, Producto, Cliente, Ubicación, etc.— acordadas de antemano entre equipos (Kimball y Ross 2013). El conjunto de data marts conformados, visto en conjunto, constituye un DW empresarial *virtual*, sin necesidad de construir primero un modelo normalizado centralizado.

La siguiente tabla resume las diferencias prácticas entre las tres arquitecturas.

Criterio	Dependiente (Inmon)	Independiente	Bus / Híbrida (Kimball)
Fuente de datos	EDW integrado	Sistemas fuente directos	Sistemas fuente directos
Consistencia entre áreas	Alta	Baja si no hay gobierno	Alta, vía dimensiones conformadas
Tiempo de entrega inicial	Largo (requiere EDW previo)	Corto	Corto por data mart
Riesgo principal	Costo y tiempo de construir el EDW	Silos de datos, definiciones divergentes	Requiere disciplina de gobierno desde el inicio
Escalabilidad organizacional	Alta una vez construido el EDW	Baja sin coordinación	Alta, crece data mart por data mart

Dimensiones conformadas y la matriz de bus

> Definición. Una **dimensión conformada** es una dimensión con la misma clave, atributos y significado de negocio en dos o más data marts, de manera que puede reutilizarse sin transformación adicional entre ellos.

La herramienta principal para planear qué dimensiones deben conformarse es la **matriz de bus**: una tabla donde las filas son procesos de negocio y las columnas son dimensiones candidatas. Una marca indica que ese proceso usa esa dimensión con la definición conformada.

Proceso de negocio	Tiempo	Ubicación	Producto/Departamento	Cliente	Empleado
Ventas					
Inventario					

Proceso de negocio	Tiempo	Ubicación	Producto/Departamento	Cliente	Empleado
Recursos Humanos					
Finanzas					

La matriz de bus revela que **Tiempo** y **Ubicación** son candidatas naturales a conformarse primero, porque las usan todos los procesos: conviene diseñarlas una sola vez, con una sola clave y una sola jerarquía, y reutilizarlas en cada data mart que se construya después.

Diseño dimensional de un data mart

Kimball propone cuatro decisiones, en orden, para diseñar cualquier data mart (Kimball y Ross 2013):

1. **Elegir el proceso de negocio.** Un evento medible y repetible: una venta, un movimiento de inventario, una nómina.
2. **Declarar la granularidad.** ¿Qué representa exactamente una fila de la tabla de hechos? (Ver el capítulo de modelado multidimensional para la definición formal de granularidad.)
3. **Identificar las dimensiones.** Los ejes de análisis relevantes para ese proceso: quién, qué, cuándo, dónde.
4. **Identificar los hechos.** Las medidas numéricas que se desean agregar.

Ejemplo integrador: el data mart detrás del cubo de la ferretería

En el capítulo anterior se trabajó con una cadena de ferreterías con sucursales en CDMX, Guadalajara, Monterrey y Mérida, departamentos Madera, Plomería, Decoración y Eléctrico, y ventas trimestrales como medida. Ese cubo no aparece de la nada: es el resultado de consultar un **data mart de Ventas** diseñado explícitamente con las cuatro decisiones anteriores.

1. **Proceso de negocio:** ventas en tienda.
2. **Granularidad:** una fila por ticket de venta, departamento y trimestre.
3. **Dimensiones:** `dim_tiempo` (trimestre, año), `dim_ubicacion` (tienda, ciudad, región), `dim_departamento` (departamento, categoría).
4. **Hechos:** `monto_venta` (medida aditiva), `unidades` (medida aditiva).

El esquema estrella resultante se representa así:

Ejemplo reproducible: construir el data mart en Python

A continuación se construye este data mart en Python usando `pandas`. En un entorno de producción, estas mismas tablas suelen vivir en un motor analítico como DuckDB, BigQuery o Snowflake, y las consultas se expresan en SQL en lugar de operaciones de `DataFrame`; aquí se usa `pandas` para mantener continuidad con el resto del curso, y las consultas se escriben primero en SQL (para mostrar la estructura relacional) y luego se resuelven con `merge`/`groupby`. Se reutilizan los mismos datos del capítulo de cubos OLAP para que ambos capítulos sean consistentes entre sí.

	id_tiempo	id_ubicacion	id_departamento	monto_venta
0	1	1	1	70
1	1	1	2	40
2	1	1	3	20
3	1	1	4	30
4	2	1	1	50

El data mart queda expresado como cuatro tablas relacionadas por llaves foráneas: exactamente la estructura de un esquema estrella. La consulta analítica equivalente en SQL —el equivalente a un *roll-up* del capítulo anterior— sería:

En `pandas`, el mismo `JOIN` seguido de `GROUP BY` se expresa encadenando `merge` y `groupby`, en lugar de operar sobre un cubo ya materializado:

	region	trimestre	departamento	total_ventas
0	Centro	T1	Decoración	20
1	Centro	T1	Eléctrico	30
2	Centro	T1	Madera	70
3	Centro	T1	Plomería	40
4	Centro	T2	Decoración	22
5	Centro	T2	Eléctrico	28
6	Centro	T2	Madera	50
7	Centro	T2	Plomería	60
8	Norte	T1	Decoración	18
9	Norte	T1	Eléctrico	26
10	Norte	T1	Madera	48
11	Norte	T1	Plomería	38
12	Norte	T2	Decoración	25
13	Norte	T2	Eléctrico	30
14	Norte	T2	Madera	36
15	Norte	T2	Plomería	55
16	Occidente	T1	Decoración	25
17	Occidente	T1	Eléctrico	40
18	Occidente	T1	Madera	60
19	Occidente	T1	Plomería	35
20	Occidente	T2	Decoración	30
21	Occidente	T2	Eléctrico	35

	region	trimestre	departamento	total_ventas
22	Occidente	T2	Madera	45
23	Occidente	T2	Plomería	50
24	Sureste	T1	Decoración	15
25	Sureste	T1	Eléctrico	37
26	Sureste	T1	Madera	52
27	Sureste	T1	Plomería	42
28	Sureste	T2	Decoración	20
29	Sureste	T2	Eléctrico	28
30	Sureste	T2	Madera	44
31	Sureste	T2	Plomería	33

Esta salida es, cifra por cifra, la misma información que el cubo del capítulo anterior representaba como $C(\text{región}, \text{tiempo}, \text{departamento})$. La diferencia es de capa: el data mart es el modelo de almacenamiento relacional; el cubo es la forma de consultarlo y navegarlo interactivamente.

Consultar el mismo data mart con la sintaxis de un roll-up regional

El siguiente ejemplo agrupa `dim_ubicacion` en zonas antes del `GROUP BY`, ilustrando que un *roll-up* no es más que sustituir el nivel de agrupación de una dimensión conformada, sin tocar la tabla de hechos.

	zona	trimestre	total_ventas
0	Zona Norte-Centro	T1	290
1	Zona Norte-Centro	T2	306
2	Zona Occidente-Sureste	T1	306
3	Zona Occidente-Sureste	T2	285

El riesgo de los data marts independientes: dimensiones no conformadas

Para ilustrar por qué la conformidad de dimensiones importa, supongamos que el equipo de Marketing construyó su propio data mart de manera independiente, sin coordinarse con Ventas, y definió "región" con una granularidad distinta (por zona comercial, no por ciudad).

Data mart	Definición de "región"	Valores
Ventas (este capítulo)	Ciudad sede de la sucursal	CDMX, Guadalajara, Monterrey, Mérida
Marketing (hipotético, independiente)	Zona comercial agregada	Zona Norte-Centro, Zona Occidente-Sureste

Ambas definiciones son legítimas por separado, pero si un directivo intenta cruzar un reporte de Ventas por "región" con uno de Marketing por "región" sin saber que la granularidad difiere, llegará

a conclusiones incorrectas o a totales que no cuadran. La solución de gobierno de datos es simple de enunciar y difícil de ejecutar en la práctica: acordar la dimensión `dim_ubicacion` (y su jerarquía ciudad → zona → región) **una sola vez**, en la matriz de bus, antes de que cada equipo construya su propio data mart.

Ventajas, limitaciones y buenas prácticas

Los data marts existen porque resuelven un problema real: un DW corporativo completo puede ser demasiado grande, lento y genérico para las preguntas cotidianas de un área específica. Entre sus ventajas destacan:

- **Consultas más rápidas**, al reducir el volumen de datos y el número de tablas relevantes.
- **Modelo más simple**, alineado al vocabulario de negocio del área (menos jerga técnica de todo el DW).
- **Entrega incremental**, especialmente en la arquitectura de bus, sin depender de un proyecto monolítico.
- **Gobernanza más manejable**, al delimitar quién es dueño de qué datos.

Sus limitaciones más comunes son:

- Riesgo de **silos** cuando se construyen de forma independiente sin dimensiones conformadas.
- **Redundancia de almacenamiento**, al replicar dimensiones compartidas en varios data marts.
- Necesidad de mantener **sincronizados** los procesos ETL/ELT de cada data mart conforme cambian las fuentes.

Buenas prácticas recomendadas al diseñar un nuevo data mart:

1. Revisar la matriz de bus antes de crear una dimensión nueva; reutilizar una conformada si ya existe.
2. Documentar la granularidad exacta de la tabla de hechos desde el día uno.
3. Versionar el esquema (nombres de columnas, tipos, claves) igual que se versiona código.
4. Validar que las medidas declaradas sean aditivas, semi-aditivas o no aditivas, y documentarlo explícitamente (Kimball y Ross 2013).
5. Definir desde el diseño quién consumirá el data mart y cómo (reportes, cubos OLAP, o —tema del siguiente capítulo— una API).

De data marts a productos de datos

En años recientes, el concepto de data mart ha evolucionado hacia el de **producto de datos** dentro de arquitecturas de *data mesh*: cada dominio de negocio no solo publica su data mart, sino que lo trata como un producto con dueño, documentación, garantías de calidad y, sobre todo, una interfaz de consumo estándar en lugar de acceso directo a tablas (Dehghani 2022). Esa interfaz de consumo es, cada vez con más frecuencia, una API. El siguiente capítulo retoma exactamente el data mart de Ventas construido aquí y muestra cómo exponerlo mediante REST, GraphQL y una capa semántica moderna.

Conclusiones

Un data mart es el punto de encuentro entre el modelo dimensional (hechos, dimensiones, granularidad) y las necesidades concretas de un área de negocio. La decisión de construirlo de forma dependiente, independiente o mediante una arquitectura de bus no es solo técnica: determina qué tan rápido se entrega valor y qué tan consistentes serán los números entre departamentos. El ejemplo de la ferretería mostró que un cubo OLAP no es una estructura aislada, sino la forma de consultar un data mart bien diseñado; y que las mismas operaciones de *roll-up* estudiadas antes corresponden, a nivel relacional, a sustituir el nivel de una dimensión conformada dentro de una consulta SQL.

18. Consumo Moderno de Data Marts: APIs y Capas Semánticas

De la consulta directa al contrato de servicio

El capítulo anterior terminó con el data mart de Ventas ya construido: cuatro tablas —`dim_tiempo`, `dim_ubicacion`, `dim_departamento`, `fact_ventas`— listas para responder preguntas de negocio mediante `JOIN` y `GROUP BY`, o mediante las operaciones de un cubo OLAP. Esa forma de consumo —un analista con acceso directo al motor de base de datos o a una herramienta de BI conectada por ODBC/JDBC— sigue siendo válida, pero no es la única, ni siempre la más adecuada. Una aplicación móvil de ventas, un panel embebido en el sitio de un cliente, otro microservicio de la organización o un agente automatizado no deberían necesitar credenciales de base de datos ni conocer el esquema interno del data mart para obtener una cifra de ventas. Este capítulo formaliza esa idea: exponer el data mart mediante un **contrato de servicio** —una API— en lugar de acceso directo a las tablas.

Del acceso directo al contrato de servicio

En el modelo de consumo tradicional, cada cliente (una hoja de cálculo, un dashboard, un script) se conecta directamente al data mart y ejecuta su propia consulta SQL. Este modelo es simple, pero acopla a cada cliente con los detalles internos del esquema: si `dim_ubicacion` cambia una columna de nombre, todos los clientes que la referencian se rompen. Además, es difícil aplicar de forma centralizada control de acceso, límites de uso o versionado.

> Definición. Un **contrato de servicio de datos** desacopla la representación interna de almacenamiento DM_j de la interfaz que ven los clientes, mediante una función de acceso A que expone solo las operaciones y campos autorizados:
>
> $$A : \text{Petición} \longrightarrow \text{Respuesta}, \qquad A \text{ no revela el esquema interno de } DM_j$$

Esta separación entre almacenamiento y consumo es la misma idea de independencia de datos que ya aparece en el diseño de un DW (Kimball y Ross 2013), llevada un nivel más arriba: así como el modelo dimensional independiza el análisis de los sistemas operacionales, la capa de API independiza a los clientes del modelo dimensional.

REST sobre un data mart

REST (*Representational State Transfer*) es el estilo arquitectónico definido por Roy Fielding para sistemas distribuidos basados en recursos, representaciones y operaciones sin estado sobre HTTP (Fielding 2000). En el contexto de un data mart, un recurso natural es una consulta agregada: por ejemplo, "ventas por región y trimestre".

Definición. Un endpoint REST sobre un data mart mapea una combinación de filtros y agrupaciones a una representación (usualmente JSON):

$$\text{GET /ventas?region} = r \,\&\, \text{trimestre} = t \;\longmapsto\; \{\text{departamento} : v \;\big|\; v = C(r, t, \cdot)\}$$

Nótese el paralelismo con la función de cubo C del primer capítulo: el endpoint REST es, en esencia, una forma de invocar esa misma función agregada a través de HTTP, con los filtros codificados como parámetros de consulta.

Martin Fowler resume la madurez de una API REST en el **Richardson Maturity Model**: nivel 0 (un solo endpoint tipo RPC), nivel 1 (recursos individuales), nivel 2 (verbos HTTP y códigos de estado correctos) y nivel 3 (HATEOAS, enlaces de navegación embebidos en la respuesta) (Fowler 2010). La mayoría de las APIs analíticas en producción se detienen en el nivel 2, que ya resuelve el problema práctico de exponer datos de forma predecible.

Diseño del endpoint

Un endpoint de nivel 2 razonable para el data mart de Ventas podría verse así (código ilustrativo, no ejecutado en este documento, para evitar añadir dependencias nuevas al entorno del curso):

El detalle importante aquí no es la sintaxis de FastAPI (Ramírez, Sebastián 2024), sino la separación de responsabilidades: el endpoint solo traduce parámetros HTTP en una llamada a `consultar_metrica`, una función de capa semántica que se define y prueba una sola vez (se implementa más adelante en este capítulo) y que oculta por completo los `JOIN` y `GROUP BY` del data mart.

Simulación reproducible de la petición y respuesta

Sin necesidad de levantar un servidor, se puede reproducir exactamente lo que ese endpoint haría: recibir un diccionario de parámetros y devolver un diccionario JSON. Primero se reconstruyen las tablas del data mart del capítulo anterior.

	id_tiempo	id_ubicacion	id_departamento	monto_venta	trimestre	anio	ciudad	region	departa
0	1	1	1	70	T1	2025	CDMX	Centro	Madera
1	1	1	2	40	T1	2025	CDMX	Centro	Plomerí
2	1	1	3	20	T1	2025	CDMX	Centro	Decorac
3	1	1	4	30	T1	2025	CDMX	Centro	Eléctric
4	2	1	1	50	T2	2025	CDMX	Centro	Madera

Ahora se define, en Python puro, la función que un endpoint REST invocaría internamente, y se simula una petición `GET /v1/ventas?region=Centro&trimestre=T2`:

La respuesta trae **todos** los campos del recurso "ventas", independientemente de si el cliente necesita solo `total_ventas` o también `departamento` y `trimestre`. Esto es exactamente la limitación que motiva GraphQL.

GraphQL sobre un data mart

GraphQL es un lenguaje de consulta en el que el cliente, no el servidor, decide la forma exacta de la respuesta: qué campos quiere y con qué nivel de anidamiento, en una sola petición a un único endpoint (GraphQL Foundation 2021). Esto evita dos problemas comunes en REST: el *over-fetching* (recibir campos que no se necesitan) y el *under-fetching* (necesitar varias peticiones encadenadas para reunir toda la información).

> Definición. Una consulta GraphQL especifica un subconjunto de campos $F \subseteq \text{campos}(R)$ de un recurso R; el resolutor devuelve únicamente F, sin importar cuántos campos adicionales exponga el esquema.

Un esquema típico sobre el data mart de Ventas (código ilustrativo; requeriría una librería como `strawberry-graphql` o `graphene`, no incluida en el entorno del curso):

Un cliente que solo necesita el total por departamento, sin región ni trimestre, escribiría:

Simulación reproducible: el cliente decide la forma de la respuesta

El siguiente código, ejecutado con `pandas` puro, reproduce el comportamiento esencial de un resolutor GraphQL: recibe la misma consulta y los mismos filtros que en el ejemplo REST, pero un parámetro adicional —los campos solicitados por el cliente— determina qué columnas trae la respuesta.

La misma pregunta de negocio —ventas del Centro en T2— produce una respuesta más pequeña porque el cliente, no el servidor, definió la forma de la salida. Esta flexibilidad es la principal ventaja de GraphQL; su costo es una capa de resolución más compleja y, en resolutores mal diseñados, el riesgo del problema *N+1* (una consulta adicional por cada elemento anidado) (Kleppmann 2017).

REST y GraphQL en comparación

Criterio	REST	GraphQL
Forma de la respuesta	Fija, la define el servidor	Flexible, la define el cliente
Número de endpoints	Uno por recurso (o pocos genéricos)	Uno único para todo el esquema
Riesgo típico	Over-/under-fetching	Consultas anidadas costosas (N+1)
Caché HTTP estándar	Sencilla (por URL)	Requiere estrategia propia
Curva de aprendizaje para el cliente	Baja	Media (requiere conocer el esquema)

Criterio	REST	GraphQL
Adecuado para	Integraciones simples, servicios públicos	Apps con necesidades de datos muy variables

Capa semántica: métricas como código

Tanto el ejemplo REST como el de GraphQL delegaron el cálculo real a una función llamada `consultar_metrica`. Esa función es la pieza más importante de una arquitectura de consumo moderna: la **capa semántica**. Su función es definir cada métrica de negocio (qué se suma, con qué filtros válidos, con qué nombre) **una sola vez**, para que REST, GraphQL, un notebook y una herramienta de BI calculen exactamente el mismo número (dbt Labs 2024; Cube Dev, Inc. 2024).

Definición. Una **métrica semántica** es una función registrada $M : (\text{agrupación}, \text{filtros}) \to \mathbb{R}$ sobre el data mart, independiente de cualquier consumidor específico.

$$M_{\text{total_ventas}}(G, F) = \sum_{i \,:\, i \models F} \text{monto_venta}_i \quad \text{agrupado por } G$$

Esta idea es el equivalente, a nivel de métricas, de lo que las **dimensiones conformadas** son a nivel de atributos (capítulo anterior): en lugar de que cada equipo reimplemente su propio cálculo de "ventas totales" dentro de cada API o cada dashboard, existe una sola definición autorizada. A esta arquitectura —métricas versionadas como código, consumidas por múltiples herramientas de presentación— se le conoce como **headless BI**.

Implementación reproducible de un registro de métricas

Con este registro, tanto `obtener_ventas` (REST) como `resolver_ventas` (GraphQL) podrían reescribirse para llamar únicamente a `consultar_metrica`, garantizando que ambos protocolos devuelvan cifras consistentes entre sí y con cualquier otra herramienta —incluido el cubo OLAP del primer capítulo— que también consulte el mismo registro.

Un mismo dato, tres contratos

El siguiente panel resume, para la misma pregunta de negocio (ventas del Centro en T2, por departamento), cómo se ve la petición y la respuesta bajo cada uno de los tres contratos discutidos: REST, GraphQL y una llamada directa a la capa semántica.

Seguridad y gobierno en la capa de API

Exponer un data mart mediante una API centraliza también el lugar natural para aplicar controles que serían difíciles de garantizar si cada cliente consultara la base de datos directamente:

- **Autenticación y autorización.** Cada petición debe identificar quién la hace (token, llave de API) y qué puede ver.
- **Seguridad a nivel de fila (*row-level security*).** Por ejemplo, un gerente regional solo debería poder consultar `region = "su región"`, sin importar qué parámetros envíe.
- **Límites de uso (*rate limiting*).** Evita que una consulta costosa mal escrita degrade el servicio para el resto de los clientes.
- **Versionado explícito del contrato.** El prefijo `/v1/` en los ejemplos anteriores permite evolucionar el esquema sin romper clientes existentes.
- **Documentación ejecutable.** Un esquema OpenAPI (para REST) o el esquema introspectable de GraphQL permiten generar documentación siempre sincronizada con el código.

Ventajas, limitaciones y buenas prácticas

Exponer un data mart mediante APIs y una capa semántica tiene ventajas claras frente al acceso directo:

- **Desacoplamiento:** cambios en el esquema físico del data mart no rompen a los clientes si el contrato se mantiene.
- **Reutilización:** un mismo conjunto de métricas alimenta apps, dashboards y otros servicios sin duplicar lógica de agregación.
- **Consistencia:** una sola definición de cada métrica evita que dos equipos reporten cifras distintas para la misma pregunta.
- **Gobierno centralizado:** autenticación, autorización y límites de uso se aplican en un solo lugar.

También introduce costos que conviene anticipar:

- Una capa adicional de infraestructura que hay que operar y monitorear.
- Riesgo de latencia si cada petición recalcula agregaciones costosas en lugar de usar resultados pre-agregados o cacheados.
- En GraphQL, riesgo de consultas anidadas ineficientes si los resolutores no se diseñan con cuidado.

Buenas prácticas recomendadas:

1. Definir las métricas en la capa semántica antes de escribir cualquier endpoint, nunca al revés.
2. Versionar el contrato de la API igual que se versiona el esquema del data mart.
3. Pre-agregar o cachear las consultas más frecuentes en lugar de recalcularlas en cada petición.
4. Aplicar seguridad a nivel de fila en la capa semántica, no confiar en que cada cliente filtre correctamente.
5. Tratar cada data mart expuesto como un **producto de datos**, con dueño, documentación y garantías de calidad (Dehghani 2022).

Conclusiones

Este capítulo cerró el recorrido de la sección de modelado para consumo: partiendo de un data mart bien diseñado, existen (al menos) dos formas complementarias de consumirlo. La primera, cubierta

en el primer capítulo, es la navegación interactiva mediante un cubo OLAP: *slice*, *dice*, *roll-up*, *drill-down* y *pivot* sobre una vista multidimensional, pensada para un analista humano explorando datos. La segunda, cubierta aquí, es la integración programática mediante un contrato de servicio —REST o GraphQL— respaldado por una capa semántica que garantiza que la definición de cada métrica sea única, sin importar cuántas aplicaciones distintas la consulten.

Ambas formas de consumo parten exactamente del mismo data mart. La elección entre exponer un cubo, una API o ambos no es una decisión tecnológica aislada: depende de quién necesita los datos y cómo los va a usar. Un director que explora tendencias trimestre a trimestre se beneficia de un cubo interactivo; una aplicación móvil que muestra el total de ventas del día se beneficia de un endpoint REST ligero; un ecosistema de aplicaciones internas con necesidades de datos heterogéneas se beneficia de una capa semántica expuesta vía GraphQL. Diseñar bien el data mart y sus dimensiones conformadas es lo que hace posible que las tres alternativas coexistan sin contradecirse entre sí.

Parte IX.

Marco Metodológico de Minería de Datos

19. Introducción a la Minería de Datos y sus Frameworks de Proceso

La pregunta que abre el capítulo

Imagina que acabas de incorporarte como analista de datos a **BigMart**, una cadena minorista con outlets de distinto tamaño y formato en varias ciudades — el mismo caso que se retoma en detalle a lo largo de esta unidad (Analytics Vidhya 2016). En tu primera semana, la directora comercial te muestra un reporte y te hace una pregunta que suena simple:

> *"Aquí está Item_DT_1985, un paquete de galletas. En el outlet OUT027 se vende como pan caliente; en el OUT010, casi nadie lo compra. Es el mismo producto, el mismo precio de lista. ¿Por qué se vende tan distinto? Y más importante: ¿qué deberíamos hacer al respecto?"*

Esta pregunta —aparentemente trivial— es en realidad la puerta de entrada a toda la disciplina que estudia este capítulo. Note que la directora **no** preguntó "muéstrame las ventas de Item_DT_1985 por outlet" (eso ya lo tiene, en el reporte que sostiene). Preguntó *por qué*, y qué hacer. Responder eso requiere algo que ninguna consulta a una base de datos puede entregar por sí sola: un modelo de las relaciones que gobiernan el fenómeno.

Este capítulo construye, paso a paso, el vocabulario y las herramientas de proceso necesarias para convertir una pregunta como esta en un proyecto ejecutable. La ruta es la siguiente: primero se distingue la minería de datos de la simple recuperación de datos; después se advierte sobre la trampa más común del oficio —confundir correlación con causalidad—; luego se catalogan las tareas centrales de la minería de datos; y finalmente se presentan los **frameworks de proceso** (KDD, CRISP-DM, SEMMA, TDSP y sus evoluciones modernas) que estructuran cómo, en la práctica, un equipo pasa de la pregunta de la directora a una decisión de negocio. A lo largo del capítulo volveremos varias veces a Item_DT_1985 y a BigMart para anclar cada idea nueva en un caso concreto.

Recuperación de datos versus minería de datos

Antes de convocar a un equipo de ciencia de datos, vale la pena preguntarse si la pregunta de la directora puede responderse con una simple consulta. La respuesta revela la diferencia central de este capítulo.

La **recuperación de datos** responde preguntas cuya forma ya está completamente definida por quien pregunta, antes de tocar el sistema. Es una transformación de la forma "dame los registros que satisfacen la condición C", donde C es conocida de antemano y el resultado es un subconjunto del repositorio. Si la directora hubiera preguntado *"¿cuántas unidades de Item_DT_1985 se vendieron en OUT027 el trimestre pasado?"*, la respuesta es una consulta SQL: agregar `Item_Outlet_Sales`

filtrando por `Item_Identifier` y `Outlet_Identifier`. No hay nada que aprender; solo hay que filtrar y sumar.

La **minería de datos**, en cambio, busca responder preguntas donde la forma de la relación es desconocida. El objetivo no es filtrar, sino **aprender**: estimar funciones, reglas o estructuras que expliquen o anticipen un fenómeno. La pregunta real de la directora — *¿qué hace que un producto rote distinto entre outlets, y qué deberíamos hacer?* — no tiene una columna que la responda directamente. Obliga a definir variables candidatas (`Outlet_Type`, `Outlet_Size`, `Item_Visibility`, `Item_MRP`…), construir un conjunto de datos adecuado, elegir un criterio de evaluación y cuantificar qué tan confiable es la respuesta.

En la práctica, todo proyecto de minería de datos empieza con muchas consultas de recuperación (explorar, filtrar, contar): son el insumo del análisis exploratorio de datos que se desarrolla en la siguiente unidad. La diferencia importa porque confundir ambas actividades lleva a un error común: tratar un hallazgo exploratorio (un filtro, un promedio) como si fuera un modelo validado, sin haber pasado por el proceso de aprendizaje y validación que la minería de datos exige.

Correlación y causalidad: la primera trampa

Supón que, al explorar los datos de BigMart, encuentras que `Item_Visibility` (el porcentaje de espacio en anaquel que ocupa un producto) está correlacionada negativamente con `Item_Outlet_Sales`: los productos con más visibilidad tienden a vender *menos*, no más. Es tentador concluir: *"hay que reducir la visibilidad de los productos para aumentar ventas"* — una conclusión que, llevada a producción, sería un despropósito.

Una asociación estadística (correlación) no garantiza un vínculo causal. La razón técnica es que los datos observacionales pueden estar dominados por variables omitidas, selección no aleatoria, confusores o retroalimentaciones entre variables (Pearl 2009). En el caso de BigMart, una explicación mucho más plausible es un **confusor**: los productos de baja rotación (por ejemplo, artículos de nicho o de temporada) se colocan deliberadamente en espacios más visibles precisamente *porque* no se venden solos, mientras que los productos de alta rotación (leche, pan) no necesitan visibilidad extra porque el cliente ya los busca activamente. La demanda base del producto está causando tanto la visibilidad asignada como, indirectamente, el patrón de ventas — no es la visibilidad la que causa las ventas bajas.

Para hablar de causalidad con solidez, típicamente se requiere un diseño experimental —idealmente aleatorizado. El experimento por excelencia en sistemas digitales es la **prueba A/B**: se asigna aleatoriamente una intervención (por ejemplo, mover un subconjunto de productos a estantes más visibles en una muestra aleatoria de outlets) a un grupo de tratamiento y se compara contra un grupo control bajo una métrica previamente definida (Kohavi et al. 2020). Empresas con alto volumen de experimentación, como Netflix, documentan públicamente cómo combinan experimentación *offline* (entrenar y validar modelos con datos históricos) con experimentación *online* (A/B testing en producción) antes de lanzar un cambio a todos sus usuarios (Netflix Technology Blog 2016). Incluso con un buen diseño A/B, sigue siendo necesario vigilar violaciones: interferencia entre unidades (*spillover*, por ejemplo si mover un producto en un outlet afecta las ventas de otro producto en el mismo estante), cambios temporales (estacionalidad, como en la Navidad) y sesgos de medición.

Las tareas centrales de la minería de datos

Una vez que la pregunta de negocio se reconoce como un problema de minería de datos —y no de simple recuperación—, el siguiente paso es identificar *qué tipo* de tarea es. La literatura clásica (Han et al. 2011; Tan et al. 2006) organiza estas tareas en dos grandes familias: **supervisadas**, cuando existe una variable objetivo que se quiere predecir o explicar, y **no supervisadas / de descubrimiento**, cuando el objetivo es encontrar estructura sin una etiqueta previa.

Clasificación (aprendizaje supervisado)

La clasificación modela una variable objetivo categórica. Dado un vector de atributos $\mathbf{x} \in \mathbb{R}^p$, se aprende una función $f(\mathbf{x}) \in \{1, \dots, K\}$ que minimiza una pérdida esperada (por ejemplo, entropía cruzada o error 0-1 aproximado). El desempeño se valida con esquemas como *train/validation/test* o *k-fold cross-validation*, analizando sensibilidad a desbalance de clases, umbrales de decisión y costos asimétricos.

En BigMart: transformar `Item_Outlet_Sales` en una etiqueta binaria ("rotación alta" / "rotación baja" por outlet) y predecirla a partir de `Outlet_Type`, `Outlet_Size` e `Item_Type`, para decidir qué outlets necesitan intervención comercial prioritaria.

Regresión (aprendizaje supervisado)

La regresión se usa cuando la variable objetivo es numérica. El problema consiste en aproximar una función $y \approx g(\mathbf{x})$ bajo un criterio como MSE o MAE. La selección del modelo depende de la estructura esperada: linealidad, heterocedasticidad, colinealidad y presencia de *outliers*.

En BigMart: esta es, literalmente, la tarea central del dataset — predecir `Item_Outlet_Sales` (numérica) a partir de los atributos del producto y del outlet. Es el ejercicio de modelado que se retoma con detalle en la unidad de clasificadores y regresión.

Clustering (aprendizaje no supervisado)

El clustering busca particiones naturales sin etiquetas. El caso prototípico es k-means, que minimiza la suma de distancias cuadráticas a centroides. Exige decisiones técnicas: estandarización de variables, selección de k, sensibilidad a inicialización y validez del supuesto de clusters aproximadamente esféricos.

En BigMart: segmentar los outlets (no los productos) según su perfil de ventas, tamaño y tipo, para descubrir si existen "arquetipos" de tienda —por ejemplo, supermercados urbanos grandes contra tiendas de conveniencia pequeñas— que expliquen buena parte de la variación observada.

Reglas de asociación

Las reglas tipo "si A entonces B" formalizan co-ocurrencias, típicamente en análisis de canasta de mercado. Las métricas clave son soporte, confianza y *lift*, discutidas en profundidad en la unidad de reglas de asociación.

Nota honesta sobre BigMart: el dataset de BigMart está agregado a nivel producto–outlet (una fila resume ventas históricas), **no** a nivel transacción individual. Por lo tanto, las reglas de asociación **no aplican directamente** sobre estos datos: no sabemos qué productos se compraron juntos en el mismo ticket. Para ilustrar esta tarea se necesita un dataset transaccional distinto —el clásico ejemplo es descubrir que quienes compran jarabe para la tos tienden a comprar también pastillas para la garganta, lo que habilita *cross-sell* o colocación conjunta en anaquel. Si BigMart quisiera aplicar esta tarea, tendría que capturar primero el detalle de cada ticket de compra, algo que su sistema actual no registra.

Detección de anomalías

Detecta observaciones raras respecto a un patrón normal. Se puede abordar como distancia a vecindarios (LOF), separación por particiones (Isolation Forest), reconstrucción (autoencoders) o modelos probabilísticos. Un punto crítico es que "raro" no siempre significa "problema": requiere validación operacional y un proceso para manejar falsos positivos.

En BigMart: detectar outlets cuyo patrón de ventas se desvía abruptamente de lo esperado para su tipo y tamaño —una señal temprana de un problema operativo (desabasto, error de captura, cierre parcial) o de una oportunidad no explicada que vale la pena investigar.

Frameworks de proceso: cuatro formas de responder la misma pregunta

Con las tareas centrales identificadas, falta resolver un problema distinto: **¿cómo se organiza el trabajo** desde que la directora hace la pregunta hasta que existe una decisión sustentada en datos? A medida que la disciplina creció en volumen, variedad y velocidad de datos, se hizo necesaria la estandarización de prácticas; de ahí emergen los *frameworks* de proceso: guías end-to-end para conducir proyectos de analítica y minería de datos de manera consistente y repetible.

No todos los frameworks enmarcan la misma pregunta de la misma forma. El siguiente comparador interactivo aplica **la pregunta de la directora de BigMart** a través de cuatro frameworks —KDD, CRISP-DM, SEMMA y TDSP— para mostrar en qué difieren.

KDD como modelo de descubrimiento

KDD (*Knowledge Discovery in Databases*) describe el flujo completo desde datos crudos hacia conocimiento: selección, preprocesamiento, transformación, minería y evaluación/interpretación, culminando en conocimiento accionable (Fayyad et al. 1996). Es especialmente útil cuando el objetivo es **entender** y **extraer patrones explicables**, además de modelar — la minería de datos propiamente dicha es solo una de sus cinco etapas, no el proceso completo.

CRISP-DM como marco de proyecto end-to-end

CRISP-DM (*Cross-Industry Standard Process for Data Mining*) organiza el proyecto en seis fases iterativas: comprensión del negocio, comprensión de los datos, preparación, modelado, evaluación y despliegue (Chapman et al. 2000; Shearer 2000). Su fortaleza es gestionar el ciclo completo con énfasis explícito en llevar resultados a operación (producción, monitoreo, impacto). Es, con diferencia, el framework más adoptado en la industria desde su publicación en 1999–2000, y es el que se desarrolla con mayor detalle en el resto de esta unidad, aplicado al caso BigMart.

SEMMA como pipeline práctico para iterar modelado

SEMMA (*Sample, Explore, Modify, Model, Assess*), originalmente asociado a las herramientas de SAS Institute (SAS Institute 2017), enfatiza el ciclo de modelado: muestrear, explorar, modificar (ingeniería de variables), modelar y evaluar. Es particularmente útil cuando ya existe un dataset razonablemente definido y el objetivo es optimizar desempeño comparando alternativas rápidamente — pero, a diferencia de CRISP-DM, no dice nada sobre cómo se llegó a ese dataset ni sobre qué pasa después de elegir el mejor modelo.

TDSP como alternativa moderna orientada a ingeniería

TDSP (*Team Data Science Process*), documentado y mantenido por Microsoft (Microsoft 2023), es un proceso de equipo que combina gestión, reproducibilidad e implementación con prácticas de ingeniería de software (versionado, artefactos, CI/CD). En comparación con CRISP-DM, enfatiza más explícitamente el "cómo se construye" en ambientes productivos —pipelines, repositorios, roles de equipo, trazabilidad—, aunque su esencia sigue siendo end-to-end y reconoce abiertamente su deuda con CRISP-DM.

Comparativa de frameworks

Framework	Enfoque principal	¿Cuándo usarlo?	Lo que menos enfatiza	Entregable típico	Aplicado a BigMart
CRISP-DM	Proceso end-to-end orientado a negocio y despliegue	Alinear problema, datos, modelo y operación con métricas de impacto	Técnicas específicas (no dicta algoritmos)	Modelo + plan de despliegue + monitoreo	Objetivo → datos → modelo de ventas → despliegue de alertas por outlet
SEMMA	Pipeline de modelado iterativo (centrado en desempeño)	Ya se tienen datos y se quiere iterar rápido para mejorar métricas	Contexto de negocio y despliegue operacional	Comparativo de modelos y reporte de evaluación	Probar 3–4 modelos para predecir `Item_Outlet_Sales`, elegir el mejor por RMSE

Framework	Enfoque principal	¿Cuándo usarlo?	Lo que menos enfatiza	Entregable típico	Aplicado a BigMart
KDD	Descubrimiento de conocimiento y patrones interpretables	Extraer patrones/insights con interpretación validada	Gestión del proyecto y despliegue	Patrones/reglas + interpretación	Descubrir qué combinación producto-outlet explica mejor la dispersión de ventas
TDSP	End-to-end con énfasis en ingeniería y trabajo en equipo	El resultado debe vivir como sistema (pipelines, CI/CD, reproducibilidad)	Marco teórico de descubrimiento; más prescriptivo en implementación	Pipelines + artefactos + despliegue versionado	Servicio de predicción de ventas: ingesta → features → entrenamiento → registro → API
CRISP-ML(Q) (Studer et al. 2021)	Evolución de CRISP-DM con aseguramiento de calidad explícito	El modelo es crítico y necesita monitoreo formal de calidad en cada fase	No es (aún) tan adoptado como CRISP-DM	Checklist de calidad por fase + modelo monitoreado	Definir umbrales de calidad de datos y de desempeño antes de desplegar el modelo de BigMart
ASUM-DM (IBM 2015)	Evolución de CRISP-DM de IBM, con guías de gobierno e IA responsable	La organización ya usa herramientas de IBM o necesita gobierno formal	Es una extensión propietaria, menos documentada públicamente	Artefactos de gobierno + modelo + despliegue	—

Cómo se combinan en un proyecto real

En proyectos reales no es necesario elegir un único marco. Una estrategia común es usar **CRISP-DM** para gobernar el proyecto completo, aplicar **SEMMA** dentro de la fase de modelado para iterar eficientemente, y apoyarse en la lógica de **KDD** cuando el objetivo incluye extraer patrones explicables o reglas interpretables para la directora comercial.

Los frameworks en la nube: de la metodología a la herramienta

Un framework de proceso dice *qué* hacer en cada fase, pero no *con qué*. En la última década, los tres grandes proveedores de nube —además de plataformas como Databricks— han construido servicios

administrados que se alinean, fase por fase, con CRISP-DM y TDSP, reduciendo la fricción de ingeniería que antes exigía cada etapa.

Fase (CRISP-DM / TDSP)	AWS	Azure	Google Cloud	Databricks / código abierto
Comprensión de negocio y datos	Amazon QuickSight, Athena	Power BI, Synapse Analytics	Looker, BigQuery	Databricks SQL, dbt
Preparación de datos	AWS Glue / Glue DataBrew	Azure Data Factory	Dataflow, Dataprep	Apache Spark (Databricks), dbt
Modelado	Amazon SageMaker (Amazon Web Services 2024)	Azure Machine Learning (Microsoft 2024)	Vertex AI (Google Cloud 2024)	Databricks ML + MLflow
Despliegue y monitoreo	SageMaker Endpoints + Model Monitor	Azure ML Endpoints + Model Monitoring	Vertex AI Prediction + Model Monitoring	MLflow Model Registry + Model Serving

Estas plataformas no reemplazan al framework: **lo operacionalizan**. TDSP, por ejemplo, nació explícitamente pensado para integrarse con Azure Machine Learning (Microsoft 2023): sus plantillas de estructura de proyecto y control de versiones están diseñadas para conectarse directamente con esa plataforma. De forma análoga, si el equipo de BigMart decidiera desplegar el modelo de predicción de ventas como un servicio que se actualiza automáticamente cada mes, necesitaría un registro de modelos, un pipeline de entrenamiento reproducible y un endpoint versionado — exactamente lo que ofrecen SageMaker, Azure ML, Vertex AI o Databricks, independientemente de si el equipo gobierna el proyecto con CRISP-DM o TDSP.

Dos consecuencias prácticas de esta convergencia entre frameworks y nube:

1. **La elección de framework y la elección de plataforma son decisiones distintas.** Un equipo puede seguir fielmente CRISP-DM y desplegar en cualquiera de las cuatro columnas de la tabla anterior; el framework no impone el proveedor.
2. **Las plataformas modernas incorporan piezas que los frameworks clásicos dejaban implícitas**, en particular el monitoreo continuo de *data drift* y desempeño del modelo en producción — la motivación detrás de evoluciones como **CRISP-ML(Q)** (Studer et al. 2021), que añade aseguramiento de calidad explícito en cada fase, y **ASUM-DM** (IBM 2015), la evolución de IBM hacia gobierno de datos e IA responsable.

Consideraciones éticas y de implementación

En minería de datos, el modelo aprende patrones del pasado; si el pasado está sesgado, el modelo puede amplificar sesgos. Si el histórico de ventas de BigMart refleja, por ejemplo, que ciertos outlets recibieron menos inversión en surtido por decisiones administrativas pasadas —no por falta de demanda real—, un modelo entrenado ciegamente sobre esos datos aprenderá y perpetuará esa asignación desigual. Por eso, además de métricas predictivas, se requiere evaluación de equidad (por grupo o segmento), explicación razonable del modelo y gobernanza de los datos usados.

En implementación, también importa la estabilidad operacional: un modelo "bueno" en validación puede fallar en producción si la distribución de los datos cambia (*data drift*), si el proceso de captura se modifica (un nuevo sistema de punto de venta), o si aparece variación estacional no vista en el entrenamiento. Por ello, el despliegue debe incluir monitoreo continuo: estadísticos de entrada, tasas de error, calibración y alertas, además de un protocolo explícito para reentrenamiento y auditoría — precisamente el vacío que frameworks como CRISP-ML(Q) buscan cerrar formalmente.

Resumen final

- La **recuperación de datos** responde preguntas con forma conocida (C definida de antemano); la **minería de datos** aprende funciones, reglas o estructuras cuando la forma de la relación es desconocida.
- Una correlación fuerte (por ejemplo, visibilidad vs. ventas en BigMart) no implica causalidad: siempre hay que preguntarse si un confusor no observado explica la asociación. Los diseños experimentales (A/B testing) son la vía más robusta hacia afirmaciones causales.
- Las **tareas centrales** de la minería de datos son clasificación y regresión (supervisadas), clustering (no supervisada), reglas de asociación y detección de anomalías — no todas aplican a cualquier dataset: BigMart, por ejemplo, no tiene la granularidad transaccional que las reglas de asociación requieren.
- **KDD**, **CRISP-DM**, **SEMMA** y **TDSP** son cuatro formas distintas de organizar el trabajo frente a la misma pregunta de negocio: KDD prioriza el descubrimiento interpretable, CRISP-DM gobierna el proyecto completo hasta despliegue, SEMMA acelera la iteración de modelado, y TDSP añade ingeniería de software y trabajo en equipo.
- En la práctica, estos frameworks se **combinan** (CRISP-DM como gobierno, SEMMA/KDD dentro del modelado) en lugar de competir entre sí.
- Las plataformas de nube (AWS SageMaker, Azure ML, Vertex AI, Databricks) **operacionalizan** las fases de estos frameworks, pero la elección de framework y la elección de plataforma son decisiones independientes.
- Evoluciones recientes como **CRISP-ML(Q)** y **ASUM-DM** incorporan explícitamente lo que los frameworks clásicos dejaban implícito: aseguramiento de calidad, monitoreo de drift y gobierno responsable de los modelos en producción.

Preguntas de repaso

1. ¿Cuál es la diferencia esencial entre una consulta de recuperación de datos y una tarea de minería de datos? Ilustra con un ejemplo de BigMart distinto al usado en el capítulo.
2. Explica por qué una correlación fuerte entre `Item_Visibility` e `Item_Outlet_Sales` no implica que reducir la visibilidad de un producto vaya a aumentar sus ventas. ¿Qué confusor podría explicar la correlación observada?
3. La directora de BigMart pregunta "¿por qué el mismo producto vende distinto en cada tienda?". Redacta cómo **KDD** y cómo **CRISP-DM** formularían esta pregunta como un problema abordable, y señala al menos una diferencia clave en su enfoque.
4. Da un ejemplo —de BigMart o de otro dominio— de cada una de las cinco tareas centrales de minería de datos: clasificación, regresión, clustering, reglas de asociación y detección de anomalías.

5. ¿Por qué las reglas de asociación no se pueden aplicar directamente al dataset de BigMart tal como se describe en este capítulo? ¿Qué tendría que cambiar en los datos para que sí aplicaran?
6. Compara SEMMA y CRISP-DM: ¿en qué escenario de proyecto elegirías apoyarte más en uno que en el otro?
7. ¿Qué aporta TDSP que CRISP-DM no especifica explícitamente? Menciona al menos dos elementos concretos.
8. Elige una fase de CRISP-DM o TDSP y menciona dos herramientas de nube, de proveedores distintos, que podrían usarse para implementarla.
9. ¿Qué problema de los frameworks clásicos busca resolver CRISP-ML(Q)? ¿Por qué ese problema se volvió más urgente al llevar modelos a producción?
10. Un modelo de BigMart que funcionaba bien en 2013 empieza a fallar en 2015. Menciona dos causas técnicas posibles y qué práctica de monitoreo, de las discutidas en el capítulo, ayudaría a detectarlas a tiempo.

Recursos y lecturas adicionales

Lecturas fundamentales

- Fayyad et al. (1996) — el artículo original que define el proceso KDD y distingue explícitamente la minería de datos (una etapa) del descubrimiento de conocimiento (el proceso completo).
- Chapman et al. (2000) y Shearer (2000) — la guía metodológica original de CRISP-DM y el artículo que la presentó como "el nuevo blueprint" de la industria.
- Han et al. (2011) y Tan et al. (2006) — libros de texto de referencia para las tareas centrales de minería de datos (clasificación, regresión, clustering, reglas de asociación, anomalías).
- Provost y Fawcett (2013) — una introducción muy accesible al pensamiento analítico de negocio detrás de la minería de datos, sin asumir formación matemática previa.
- Pearl (2009) — la referencia canónica sobre inferencia causal y por qué la correlación no basta.
- Kohavi et al. (2020) — guía práctica y rigurosa sobre el diseño de pruebas A/B en producción.

Documentación oficial de frameworks y plataformas

- Microsoft Learn: *Team Data Science Process* — learn.microsoft.com/azure/architecture/data-science-process/overview (Microsoft 2023).
- Amazon SageMaker Documentation — docs.aws.amazon.com/sagemaker (Amazon Web Services 2024).
- Vertex AI Documentation (Google Cloud) — cloud.google.com/vertex-ai/docs (Google Cloud 2024).
- Azure Machine Learning Documentation — learn.microsoft.com/azure/machine-learning (Microsoft 2024).
- Databricks — databricks.com (Databricks 2024), para la perspectiva de *lakehouse* sobre este mismo ciclo.

Videos y cursos

- Canal de YouTube **IBM Technology** (youtube.com/@IBMTechnology): explicaciones cortas y visuales sobre CRISP-DM, ciencia de datos y MLOps.

- Canal de YouTube **StatQuest with Josh Starmer** (youtube.com/@statquest): la mejor referencia en video para repasar los fundamentos estadísticos detrás de clasificación, regresión y clustering antes de las unidades siguientes.
- Coursera: *IBM Data Science Professional Certificate* (coursera.org/professional-certificates/ibm-data-science) — incluye un módulo dedicado a la metodología CRISP-DM aplicada paso a paso.
- Coursera: *Google Data Analytics Professional Certificate* (coursera.org/professional-certificates/google-data-analytics) — buen complemento sobre el ciclo de vida de los datos previo al modelado.

Caso de estudio de esta unidad

- Analytics Vidhya (2016) — el dataset BigMart Sales, usado como hilo conductor en este capítulo y desarrollado en detalle en el resto de la unidad (comprensión de negocio, EDA, preprocesamiento y modelado).

20. Business Understanding - Caso BigMart Sales

Un error común en proyectos de analítica y minería de datos es iniciar el análisis directamente sobre la estructura del dataset, sin detenerse antes en el contexto que da sentido a la información disponible. En entornos reales, los datos forman parte de procesos operativos, comerciales o estratégicos, por lo que su exploración debe estar guiada por una comprensión inicial del problema de negocio. En este sentido, el análisis exploratorio de datos (EDA) no solo permite describir variables y detectar patrones, sino también traducir necesidades del negocio en preguntas analíticas investigables.

Por ello, en este ejercicio se utilizará una breve narrativa de negocio como punto de partida para orientar el EDA y establecer:

- quién genera los datos;
- con qué propósito se recolectan;
- qué representa cada observación del conjunto de datos;
- qué relaciones o comportamientos conviene explorar;
- y qué preguntas del negocio pueden examinarse a través del análisis exploratorio.

Escenario ficticio de negocio

Supongamos que **BigMart** es una cadena nacional de tiendas minoristas con presencia en varias ciudades y con distintos formatos de establecimiento. Algunos outlets son supermercados grandes, otros son tiendas de conveniencia y otros funcionan como puntos comerciales de escala intermedia. La empresa comercializa múltiples categorías de productos de consumo frecuente: alimentos, bebidas, productos de hogar, artículos perecederos y no perecederos, entre otros.

Durante 2013, la dirección comercial de BigMart detectó que el desempeño en ventas no era homogéneo entre productos ni entre tiendas. Algunos artículos tenían una rotación muy alta en ciertos outlets, pero un comportamiento mediocre en otros. Del mismo modo, tiendas con características aparentemente similares mostraban resultados muy distintos en ingresos por producto.

La empresa contaba con sistemas operativos que registraban transacciones, atributos del catálogo de productos y características básicas de los establecimientos. Sin embargo, esa información se utilizaba sobre todo para fines administrativos y no se había integrado de forma sistemática para apoyar decisiones analíticas.

Ante ello, la dirección regional decidió solicitar al área de analítica un estudio con dos propósitos iniciales:

1. **entender mejor la estructura de las ventas** a nivel producto-tienda;
2. **identificar factores asociados con un mayor desempeño comercial**.

Este encargo es el origen del dataset con el que trabajaremos.

Situación problemática

La preocupación de la empresa no surge de manera abstracta. En este escenario ficticio, BigMart enfrenta cuatro tensiones operativas:

- **Planeación de inventario** No siempre resulta claro cuánto surtido asignar a cada tipo de producto en cada outlet. Una mala estimación puede producir sobreinventario o desabasto.
- **Diferencias entre tiendas** Outlets de distinto tamaño, ubicación o formato comercial parecen comportarse de manera desigual. La empresa necesita saber si esas diferencias son consistentes y medibles.
- **Diseño de promociones y surtido** La gerencia comercial quiere distinguir qué tipos de producto tienen mejor desempeño y si ciertas características, como categoría o precio de lista, parecen asociarse con mayores ventas.
- **Falta de evidencia integrada** Los datos existen, pero todavía no se han convertido en evidencia útil para la toma de decisiones. Se requiere transformar registros operativos en conocimiento analítico.

En consecuencia, el problema inicial de negocio puede formularse así:

> BigMart necesita comprender cómo se distribuyen y de qué dependen las ventas por producto y por outlet, para mejorar decisiones de surtido, operación comercial y planeación.

Diccionario de Variables

En la siguiente tabla se detalla la naturaleza y la interpretación de negocio de cada una de las variables presentes en el conjunto de datos.

Tabla 20.1.: Diccionario de variables del dataset de ventas

Categoría	Variable	Descripción e Interpretación de Negocio
Producto	`Item_Identifier`	Identificador único del producto. Permite el rastreo y vinculación con otras fuentes de información.
Producto	`Item_Weight`	Peso del producto. Relevante para la logística, el empaque y la presentación comercial.
Producto	`Item_Fat_Content`	Clasificación del contenido graso. Útil para segmentar subgrupos de alimentos y bebidas.

Categoría	Variable	Descripción e Interpretación de Negocio
Producto	`Item_Visibility`	Porcentaje de exposición en anaquel. Indica la prominencia comercial del artículo en la tienda.
Producto	`Item_Type`	Categoría general del producto. Clave para analizar diferencias estructurales entre líneas de negocio.
Producto	`Item_MRP`	Precio máximo de venta (precio de lista). Variable principal para explicar variaciones en ventas monetarias.
Outlet	`Outlet_Identifier`	Identificador único del establecimiento. Permite estudiar el desempeño comparativo entre tiendas.
Outlet	`Outlet_Establishment_Year`	Año de apertura. Funciona como indicador de antigüedad y madurez operativa de la tienda.
Outlet	`Outlet_Size`	Tamaño del local. Se asocia con la capacidad de surtido, tráfico de clientes y volumen de exhibición.
Outlet	`Outlet_Location_Type`	Tipo de ubicación geográfica. Resume el entorno socioeconómico y el perfil de la demanda.
Outlet	`Outlet_Type`	Formato comercial (ej. Supermercado, Tienda de conveniencia). Define la lógica operativa del punto de venta.
Resultado	`Item_Outlet_Sales`	**Variable Objetivo.** Ventas totales del producto en la tienda. Es el eje central del análisis de desempeño comercial.

Necesidades del negocio que motivan el análisis

En la práctica, las preguntas analíticas actúan como una guía para seleccionar las herramientas del EDA. Para esto es necesario traducir las preguntas no técnicas de nuestro cliente en preguntas que, como equipo de análisis sabemos que os permitirán realizar un mejro tabajo, en este ejemplo tras una conversación con BigMart se plantean varias necesidades.

Necesidad 1: entender el comportamiento global de las ventas.

> "Antes de entrar en detalles, queremos entender cómo se están comportando las ventas en general. ¿Normalmente se mantienen en un rango parecido o hay mucha variación? ¿Existen casos excepcionalmente altos o bajos que debamos revisar?"

Esta es una forma típica en la que un cliente o la dirección expresa una inquietud de negocio. Aunque no utiliza lenguaje estadístico, como equipo de analítica podemos traducirla en preguntas exploratorias más concretas:

- ¿cuál es el nivel típico de ventas por combinación producto-outlet?
- ¿la distribución de ventas es homogénea o presenta alta dispersión?
- ¿hay valores extremos particularmente altos o bajos?

Desde la perspectiva del EDA, estas preguntas orientan el estudio de la variable de ventas mediante **medidas de localización, variabilidad y forma**, así como la identificación de posibles valores atípicos.

Necesidad 2: comparar desempeño entre segmentos.

> "Queremos saber si algunos tipos de productos o algunas tiendas están funcionando mejor que otros. En particular, nos interesa ver si el desempeño cambia según la categoría del producto, el tipo de outlet, el tamaño de la tienda o su ubicación."

Esta es una inquietud típica del cliente cuando busca identificar diferencias relevantes entre grupos. Desde la perspectiva del equipo de data analytics, esta necesidad se traduce en preguntas de análisis exploratorio como:

- ¿hay categorías de producto que venden sistemáticamente más?
- ¿ciertos tipos de outlet concentran mejores resultados?
- ¿las ventas cambian según tamaño o localización de la tienda?

Estas preguntas orientan un análisis de **heterogeneidad entre subgrupos**, mediante comparaciones entre categorías y segmentos relevantes del negocio.

Necesidad 3: identificar concentración comercial.

> "Nos interesa entender si el negocio está bien diversificado o si una parte importante de las ventas depende de muy pocas categorías o de muy pocos outlets."

Esta es una preocupación estratégica habitual en contextos comerciales, ya que permite evaluar qué tan diversificada o dependiente es la estructura de ingresos. Desde la perspectiva del equipo de data analytics, esta necesidad se traduce en preguntas exploratorias como:

- ¿unas pocas categorías explican gran parte del ingreso?
- ¿hay outlets particularmente dominantes?
- ¿la estructura comercial está diversificada o concentrada?

Estas preguntas orientan un análisis de **concentración**, enfocado en identificar si las ventas se distribuyen de manera equilibrada o si están fuertemente concentradas en un conjunto reducido de categorías o establecimientos.

Necesidad 4: explorar asociaciones entre atributos y ventas.

> "Queremos una primera idea de qué variables podrían estar más relacionadas con las ventas, para entender mejor el negocio antes de avanzar hacia análisis predictivos."

Esta es una inquietud frecuente en etapas tempranas del análisis, cuando el cliente busca una primera comprensión de los factores que podrían estar vinculados con las ventas. Desde la perspectiva del equipo de data analytics, esta necesidad se traduce en preguntas exploratorias como:

- ¿el precio de lista está asociado con mayores ventas?
- ¿la visibilidad parece relacionarse con el desempeño?
- ¿algunas variables del outlet muestran asociación con mejores resultados?

Estas preguntas orientan el estudio de **correlación** y de **relaciones bivariadas**, con el fin de detectar patrones iniciales entre atributos y ventas que luego puedan examinarse con mayor profundidad.

Del problema de negocio al Data Understanding

La transición entre Business Understanding y Data Understanding ocurre cuando las preguntas del negocio se convierten en preguntas sobre los datos disponibles. A partir del escenario anterior, como equipo analítico debemos formular preguntas como las siguientes:

Sobre estructura general del dataset

- ¿cuántos registros hay?
- ¿cuántas variables numéricas y categóricas existen?
- ¿hay faltantes, inconsistencias o codificaciones problemáticas?

Sobre la variable de ventas

- ¿cómo se distribuye `Item_Outlet_Sales`?
- ¿qué tan sesgada está?
- ¿qué tan dispersa es respecto a su centro?

Sobre variables categóricas

- ¿cuántas categorías de producto existen?
- ¿cómo se distribuyen los registros entre tipos de outlet?
- ¿hay clases raras o escasamente representadas?

Sobre calidad de datos

- ¿existen valores faltantes en peso o tamaño del outlet?
- ¿hay categorías inconsistentes, como variantes textuales de una misma clase?
- ¿hay variables que realmente son identificadores y no rasgos analíticos?

Estas preguntas ya no pertenecen solo al negocio: son preguntas de **Data Understanding** y abren el camino al EDA.

Una vez que se ha establecido el escenario de negocio, la unidad de análisis, el diccionario de variables y las necesidades comerciales, es posible formular preguntas concretas que el EDA deberá responder. Estas preguntas constituyen el puente natural entre la fase de Business Understanding y la fase de Data Understanding.

Preguntas sobre ventas

- ¿Cuál es el nivel típico de `Item_Outlet_Sales`?
- ¿Qué tan alejadas están las ventas de ese nivel típico?
- ¿La distribución es simétrica o presenta cola derecha?

Preguntas sobre productos

- ¿Qué categorías de producto aparecen con mayor frecuencia?
- ¿Qué categorías concentran mayores ventas?
- ¿El precio del producto parece jugar un papel importante?

Preguntas sobre outlets

- ¿Existen diferencias notables entre tipos de outlet?
- ¿El tamaño del outlet parece importar?
- ¿La antigüedad del outlet podría estar asociada con su desempeño?

Preguntas sobre calidad y preparación de datos

- ¿Qué variables presentan faltantes?
- ¿Qué categorías deben homogenizarse?
- ¿Qué transformaciones podrían ser necesarias antes de modelar?

Estas preguntas constituyen el puente natural hacia el Data Understanding y el EDA.

Parte X.

Análisis Exploratorio de Datos (EDA)

Parte XI.

Fundamentos del EDA

21. Fundamentos del análisis exploratorio de datos

Introducción

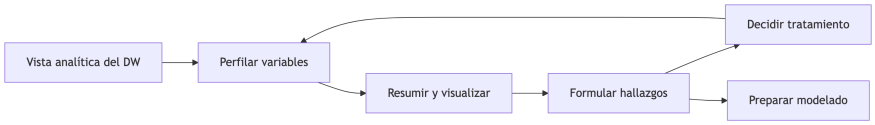
El análisis exploratorio de datos (EDA) es el proceso iterativo mediante el cual se conoce un conjunto de datos antes de construir modelos o comunicar resultados definitivos. Combina resúmenes numéricos y visuales para descubrir estructura, evaluar calidad y formular preguntas más precisas (Tukey 1977; Heckert et al. 2012).

Dentro de CRISP-DM, el EDA forma parte del entendimiento de los datos y mantiene una relación iterativa con su preparación. Un hallazgo exploratorio puede exigir corregir una variable; después de corregirla, la distribución debe examinarse nuevamente.

Definición. El EDA es una estrategia de investigación que utiliza estadística descriptiva y visualización para reconocer patrones, anomalías, relaciones y limitaciones antes de elegir un procedimiento de modelado.

Preguntas que orientan el EDA

- ¿Cuál es la unidad de observación y cuántos registros existen?
- ¿Qué significa cada variable y en qué escala fue medida?
- ¿Qué valores faltan y por qué podrían faltar?
- ¿Qué valores son imposibles, inconsistentes o duplicados?
- ¿Dónde se localizan los valores y cuánto varían?
- ¿Qué forma tienen las distribuciones?
- ¿Existen diferencias entre grupos o concentración del total?
- ¿Qué asociaciones aparecen entre variables?



La salida del EDA no es una colección de gráficos. Es una bitácora de hallazgos, evidencia, decisiones y preguntas pendientes.

Unidad de observación y granularidad

Antes de calcular cualquier estadístico debe identificarse qué representa una fila: una venta, un producto por tienda, un cliente, una visita o una medición temporal. Mezclar granularidades produce duplicaciones y resúmenes engañosos.

Ejemplo. Si una tabla contiene una fila por producto y outlet, sumar ventas puede ser válido; contar productos sin distinguir el outlet puede contar el mismo producto varias veces.

Tipos de variables

Variables cualitativas

- **Nominales:** categorías sin orden intrínseco, como tipo de outlet o método de pago.
- **Ordinales:** niveles ordenados sin distancias métricas garantizadas, como satisfacción baja, media y alta.

Variables cuantitativas

- **Discretas:** conteos con valores separados, como número de compras.
- **Continuas:** mediciones en un intervalo, como peso, ingreso o duración.

Tipo	Resúmenes adecuados	Visualizaciones iniciales
Nominal	Frecuencias, proporciones, moda, diversidad	Barras
Ordinal	Frecuencias ordenadas, mediana, cuantiles	Barras ordenadas, acumulada
Discreta	Frecuencias, centro, dispersión	Barras o histograma entero
Continua	Centro, dispersión, forma	Histograma, ECDF, boxplot

El tipo almacenado no siempre coincide con el tipo estadístico. Un código postal numérico sigue siendo nominal y una fecha guardada como texto sigue representando tiempo.

Frecuencias y proporciones

Para valores $v_1, \dots, v_K$, la frecuencia absoluta y la proporción son

$$f_k = \#\{i : x_i = v_k\}, \qquad p_k = \frac{f_k}{n}, \qquad \sum_{k=1}^{K} p_k = 1. \tag{1}$$

La colección $(p_1, \dots, p_K)$ es la distribución empírica de una variable categórica o discreta.

Frecuencias acumuladas y ECDF

Cuando existe un orden natural,

$$F_k = \sum_{j \le k} f_j, \qquad P_k = \sum_{j \le k} p_j. \tag{2}$$

Para una variable cuantitativa, la función de distribución empírica es

$$\widehat{F}_n(x) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}(x_i \le x). \tag{3}$$

Definición. La ECDF indica la proporción observada de datos menores o iguales que cada umbral. A diferencia del histograma, no requiere elegir intervalos.

Ejemplo de calidad categórica

Hallazgo. `Asistió` y `ASISTIÓ`, así como `No asistió` y `No  asistió`, son duplicados semánticos. `NA` y `<faltante>` podrían representar la misma ausencia, pero debe confirmarse con el diccionario de datos antes de recodificar.

Ejemplo discreto y acumulado

Los tiempos negativos son imposibles bajo la definición habitual y requieren revisión. Los valores de 95 y 120 días son extremos, pero podrían ser casos reales; no deben eliminarse sin evidencia.

Datos faltantes

El porcentaje faltante debe analizarse por variable y por grupos relevantes. Un promedio global puede ocultar que la ausencia se concentra en una fuente, fecha o categoría. La decisión de imputar pertenece al preprocesamiento, pero el EDA debe documentar patrón, magnitud y posibles mecanismos.

Valores atípicos y anomalías

Un valor atípico es una observación alejada según una regla estadística; una anomalía es un caso incompatible con el proceso esperado. No son sinónimos de error.

Evidencia	Pregunta	Acción posible
Valor imposible	¿Contradice el dominio?	Corregir, marcar o excluir con trazabilidad
Valor raro pero posible	¿Es un segmento real?	Conservar y analizar por separado
Cola extensa	¿La escala es adecuada?	Transformar o usar medidas robustas
Categoría infrecuente	¿Es error o nivel real?	Estandarizar o agrupar con criterio

Ruta descriptiva del libro

Los siguientes capítulos desarrollan seis perspectivas complementarias:

1. Medidas de localización: ¿dónde están los valores?
2. Medidas de variabilidad: ¿cuánto cambian?
3. Medidas de forma: ¿cómo son sus colas y simetría?
4. Medidas de heterogeneidad: ¿cambian entre categorías o grupos?
5. Medidas de concentración: ¿quién acumula el total?
6. Medidas de correlación: ¿cómo se asocian dos variables?

Cada capítulo teórico está seguido por una práctica reproducible con BigMart.

Checklist de salida

Un EDA reproducible debe dejar: diccionario y granularidad, perfil de faltantes, dominios validados, resúmenes con unidades, visualizaciones pertinentes, hallazgos redactados, decisiones justificadas y preguntas pendientes.

Ejercicios

1. Clasifique cada variable de BigMart por escala estadística y justifique los casos ambiguos.
2. Proponga una regla de validación para tres columnas del dataset.
3. Compare qué información aporta un histograma frente a una ECDF.

Parte XII.

Medidas de localización

22. Medidas de localización

El problema: ¿qué tiempo representa una entrega habitual?

Una pequeña mensajería registró el tiempo, en minutos, de nueve entregas realizadas durante una tarde:

$$18, 20, 20, 21, 22, 24, 25, 27, 75.$$

La última motocicleta encontró una avenida cerrada. Los 75 minutos son una observación real, no necesariamente un error, pero están muy alejados del resto; la llamaremos **valor extremo**.[1]

Imagine que debemos completar una frase para un cliente:

> Una entrega de esta mensajería tarda alrededor de _____ minutos.

¿Debemos escribir 20, 22 o 28? Los tres números se pueden defender, pero responden preguntas distintas:

Pregunta	Idea de centro	Medida
¿Cuánto tiempo se consumió por entrega en promedio?	Repartir el tiempo total por igual	Media
¿Qué valor deja la mitad de las entregas a cada lado?	Localizar el caso central	Mediana
¿Qué tiempo exacto se repitió más?	Buscar la mayor frecuencia	Moda
¿Qué tiempo no supera cierto porcentaje de entregas?	Fijar una posición acumulada	Cuantil

Una **medida de localización** es un estadístico[2] que sitúa una distribución sobre la escala de la variable. "Localización" no significa que exista un centro universal: primero se elige la pregunta y después la fórmula (Tukey 1977; Freedman et al. 2007).

Dos cambios que servirán como brújula

- Si sumamos 10 minutos a **todas** las entregas, cualquier centro razonable debería aumentar 10 minutos.

[1] Un **valor extremo** es una observación muy alejada del cuerpo principal de los datos. Puede ser un evento genuino, un caso de otra población o un error; la distancia por sí sola no decide cuál explicación es correcta.

[2] Un **estadístico** es cualquier función calculada a partir de una muestra, como una media o un máximo. Su valor cambia si cambia la muestra. Un **parámetro** describe a la población completa y normalmente es desconocido.

- Si solo la demora de 75 aumenta, la media debería reaccionar; el caso que ocupa la mitad del orden podría permanecer igual.

Estas dos ideas anticipan la diferencia entre usar magnitudes y usar posiciones.

Antes de resumir: conservar los datos visibles

Los datos ya están ordenados. Escribamos debajo sus posiciones:

Posición i	1	2	3	4	5	6	7	8	9
Tiempo $x_{(i)}$	18	20	20	21	22	24	25	27	75

La notación $x_{(i)}$ indica el dato que ocupa la posición i después de ordenar; se llama **estadístico de orden**.[3]

Una cifra resumen nunca sustituye la distribución. Antes de elegirla conviene ver los datos, un histograma o un diagrama de caja. En este ejemplo, informar solo "28 minutos" ocultaría que ocho de nueve entregas tardaron 27 minutos o menos.

Tres respuestas para el mismo conjunto

Media: repartir el total por igual

Sumemos los nueve tiempos:

$$18 + 20 + 20 + 21 + 22 + 24 + 25 + 27 + 75 = 252.$$

Si repartimos esos 252 minutos por igual entre las nueve entregas,

$$\bar{x} = \frac{252}{9} = 28 \text{ minutos.} \tag{1}$$

La media no afirma que alguna entrega haya durado 28 minutos. Afirma que, si el tiempo total se redistribuyera equitativamente, corresponderían 28 minutos a cada una. Por eso resulta útil para presupuestar tiempo total: nueve entregas a 28 minutos reproducen los 252 minutos observados.

Otra forma de verla es como punto de equilibrio. Las desviaciones respecto a 28 se cancelan:

$$\sum_{i=1}^{9}(x_i - 28) = \sum_{i=1}^{9} x_i - 9(28) = 252 - 252 = 0.$$

[3] Un **estadístico de orden** es una observación identificada por su posición después de ordenar la muestra. Los paréntesis en $x_{(i)}$ indican rango, no el dato originalmente registrado en la fila i.

Los 47 minutos que la entrega de 75 queda por arriba pesan tanto como todos los déficits que quedan por debajo. Esa es una virtud cuando interesa conservar el total y una limitación cuando interesa describir un caso habitual.

Mediana: encontrar el caso central

Como hay nueve observaciones, la posición central es

$$\frac{n+1}{2} = \frac{9+1}{2} = 5.$$

El quinto tiempo ordenado es 22:

$$\tilde{x} = x_{(5)} = 22 \text{ minutos.} \tag{2}$$

Hay cuatro observaciones antes y cuatro después. La mediana no reparte minutos; reparte **posiciones**. Para un número par de observaciones no existe una sola posición central, por lo que se acostumbra promediar los dos valores centrales.

Moda: encontrar lo más frecuente

Contemos las repeticiones:

Tiempo	18	20	21	22	24	25	27	75
Frecuencia	1	**2**	1	1	1	1	1	1

La frecuencia mayor es 2, de modo que

$$\text{Moda}(x) = 20 \text{ minutos.} \tag{3}$$

La moda sí corresponde a un valor observado. Puede haber dos o más modas —una distribución **multimodal**[4]— o ninguna repetición que haga útil hablar de una moda muestral.

Tres números correctos, tres afirmaciones distintas

- **28 minutos:** tiempo total por entrega si se reparte por igual.
- **22 minutos:** la mitad de las entregas tardó 22 minutos o menos y la otra mitad 22 minutos o más.
- **20 minutos:** fue el tiempo exacto observado con mayor frecuencia.

Decir solo “el promedio” es ambiguo. Conviene nombrar la medida y traducirla a una frase vinculada con la decisión.

[4]Una distribución **multimodal** posee más de un máximo local o más de un valor con frecuencia máxima. Puede ser señal de que se mezclaron subgrupos, aunque no lo demuestra por sí sola.

¿Qué cambia cuando se alarga la demora?

Suponga que la entrega tardía pasa de 75 a 120 minutos. La suma aumenta 45 y, por tanto, la media aumenta

$$\frac{45}{9} = 5 \text{ minutos.}$$

La nueva media es 33, mientras la mediana sigue siendo 22 y la moda sigue siendo 20. En general, si una observación cambia en una cantidad Δ y el tamaño de la muestra no cambia,

$$\bar{x}_{\text{nueva}} - \bar{x}_{\text{anterior}} = \frac{\Delta}{n}. \tag{4}$$

La media escucha la magnitud de cada dato, incluso la de uno extremo. La mediana escucha principalmente su posición: mover el máximo todavía más a la derecha no cambia cuál observación ocupa el centro. Esta menor sensibilidad se denomina **robustez**.[5]

Un compromiso: la media recortada

Una media recortada elimina la misma cantidad de observaciones en cada cola y promedia las restantes (Wilcox 2012). Si retiramos una observación de cada extremo —$1/9 \approx 11.1\,\%$ de la muestra por cola— quedan

$$20, 20, 21, 22, 24, 25, 27.$$

Su media es

$$\bar{x}_{\text{rec}} = \frac{20 + 20 + 21 + 22 + 24 + 25 + 27}{7} = \frac{159}{7} \approx 22.71. \tag{5}$$

Si 75 cambia a 120, el máximo se vuelve a retirar y el resultado permanece en 22.71. Sin embargo, el recorte no debe escogerse después de mirar qué respuesta conviene: se declara por anticipado y siempre se aplica simétricamente.

> **Recortar no es borrar errores.** Si 75 proviene de una avenida cerrada, pertenece al proceso real. La media recortada ofrece otra descripción del centro, pero el evento extremo debe conservarse, investigarse y comunicarse.

[5] Una medida es **robusta** si cambios pequeños en la mayoría de los datos o una fracción limitada de observaciones anómalas no producen cambios desproporcionados en el resultado.

Cuantiles: responder preguntas de servicio

La mediana es un caso particular de cuantil. El cuantil de nivel p busca un umbral que deje aproximadamente una proporción p de los datos por debajo o en él. Cuando p se expresa como porcentaje hablamos de **percentil**: $Q(0.9)$ es el percentil 90 o P_{90}.

Para calcular a mano los cuartiles[6] usaremos la convención de "medianas de las mitades": se excluye la mediana global y se toma la mediana de cada mitad.

La mitad inferior es

$$(18, 20, 20, 21),$$

por lo que

$$Q_1 = \frac{20 + 20}{2} = 20.$$

La mitad superior es

$$(24, 25, 27, 75),$$

y entonces

$$Q_3 = \frac{25 + 27}{2} = 26.$$

Así, la mitad central de los tiempos se encuentra aproximadamente entre 20 y 26 minutos. En operación, un cuantil alto responde una pregunta distinta del centro: "¿qué límite cubre, por ejemplo, 90 % de las entregas?". Ese límite es útil para fijar acuerdos de servicio, pero requiere más datos que nueve observaciones para estimar con estabilidad la cola.[7]

Por qué dos programas pueden dar cuartiles distintos

Entre posiciones observadas se puede redondear, seleccionar un dato o interpolar. Existen varias convenciones legítimas para cuantiles muestrales (Hyndman y Fan 1996). En muestras pequeñas, documente siempre el método. En este capítulo, "medianas de las mitades" es una decisión explícita, no la única definición posible.

[6] Existen varias definiciones válidas de cuartiles porque una muestra finita no determina de manera única los percentiles entre posiciones. El software puede interpolar y producir valores distintos. Debe declararse el método al reproducir resultados (Hyndman y Fan 1996).

[7] Una **cola** es la región alejada del centro hacia valores pequeños o grandes. "Cola derecha larga" no significa que la mayoría de los datos estén a la derecha, sino que la extensión o probabilidad extrema es mayor en esa dirección.

Cuando las observaciones representan cantidades distintas

Ahora la mensajería resume tres zonas a partir del tiempo medio de cada zona:

Zona	Tiempo medio x_i	Entregas w_i	Minutos acumulados $w_i x_i$
Centro	20	50	1,000
Norte	25	30	750
Periferia	40	20	800
Total		**100**	**2,550**

Promediar los tres tiempos sin pesos daría

$$\frac{20 + 25 + 40}{3} = 28.33,$$

como si cada zona aportara el mismo número de entregas. Para recuperar el tiempo por **entrega** usamos el número de entregas como peso:[8]

$$\bar{x}_w = \frac{\sum_{i=1}^{K} w_i x_i}{\sum_{i=1}^{K} w_i} = \frac{2550}{100} = 25.5 \text{ minutos.} \tag{6}$$

El peso debe corresponder a la unidad que se quiere describir. Si la pregunta fuera "¿cómo se comporta una zona elegida al azar?", dar igual peso a las zonas podría ser apropiado; si es "¿cuánto tarda una entrega elegida al azar?", se necesitan los tamaños de zona.

Por qué media y mediana aparecen como soluciones

Hasta aquí construimos las medidas desde preguntas concretas. Ahora podemos formalizar una propiedad que explica por qué aparecen repetidamente en estadística y aprendizaje automático.

La media minimiza errores cuadrados

Sea a un número usado para representar todas las observaciones. Su pérdida cuadrática total[9] es

$$S_2(a) = \sum_{i=1}^{n} (x_i - a)^2.$$

Al sumar y restar $\bar{x}$ dentro de cada término se obtiene

[8] Un **peso** indica cuánto contribuye o cuántas unidades representa una observación resumida. No debe confundirse con el peso físico de un producto ni elegirse solo para forzar un resultado.

[9] Una **función de pérdida** asigna un costo numérico a la diferencia entre una predicción o resumen y lo observado. Minimizarla convierte el significado práctico de un error en un criterio matemático.

$$S_2(a) = \sum_{i=1}^{n}(x_i - \bar{x})^2 + n(a - \bar{x})^2. \tag{7}$$

El primer término no depende de a y el segundo nunca es negativo. Por tanto, $S_2(a)$ alcanza su único mínimo cuando $a = \bar{x}$. En las entregas,

$$S_2(28) = 2548 < S_2(22) = 2872.$$

Elevar al cuadrado penaliza con fuerza los errores grandes: la diferencia de 47 minutos asociada con la entrega tardía aporta $47^2 = 2209$ unidades a la pérdida evaluada en 28.

La mediana minimiza errores absolutos

Si el costo crece linealmente con la distancia, usamos

$$S_1(a) = \sum_{i=1}^{n} |x_i - a|.$$

Mover a ligeramente a la derecha reduce la distancia a cada observación que queda a la derecha y aumenta la distancia a cada observación de la izquierda. En una mediana hay al menos la mitad de los datos a cada lado, por lo que ningún movimiento puede reducir el saldo total. Así, cualquier mediana minimiza $S_1(a)$.[10]

En el ejemplo,

$$S_1(22) = 72 < S_1(28) = 94.$$

Elegir centro equivale a elegir qué error importa

- La media es óptima para pérdida cuadrática: castiga de forma creciente los errores grandes.
- La mediana es óptima para pérdida absoluta: cada minuto adicional cuesta lo mismo.

No se trata de que una medida sea universalmente mejor. La función de pérdida traduce las consecuencias de una decisión a una cantidad que se desea reducir.

[10] Con n par, cualquier punto entre los dos valores centrales minimiza la suma de desviaciones absolutas. Promediarlos selecciona un único valor por convención.

Laboratorio interactivo: arrastrar una demora

Mueva el tiempo de la novena entrega. Observe qué centros se desplazan y dónde alcanzan su mínimo las dos funciones de pérdida.

La versión HTML contiene un laboratorio que cambia la novena entrega entre 27 y 120 minutos. En el escenario base de 75 minutos, la media es 28, la mediana 22 y la media recortada 22.71. Al llevar la demora a 120, la media aumenta a 33, mientras la mediana y la media recortada permanecen iguales. El laboratorio también muestra que la pérdida cuadrática alcanza su mínimo en la media y la pérdida absoluta en la mediana.

Formalización

Sea una muestra numérica $x_1, \dots, x_n$.[11] La media aritmética es

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i. \tag{8}$$

Si $x_{(1)} \le \cdots \le x_{(n)}$ son los datos ordenados, la mediana es

$$\tilde{x} = \begin{cases} x_{((n+1)/2)}, & n \text{ impar}, \\ \dfrac{x_{(n/2)} + x_{(n/2+1)}}{2}, & n \text{ par}. \end{cases} \tag{9}$$

Para datos discretos o categóricos, si $f(v)$ es la frecuencia del valor v, el conjunto de modas es

$$\operatorname{Moda}(x) = \left\{ v : f(v) = \max_{u} f(u) \right\}. \tag{10}$$

Para una distribución poblacional con función acumulada F,[12] una definición general del cuantil de nivel $p \in (0,1)$ es

$$Q(p) = \inf\{x : F(x) \ge p\}. \tag{11}$$

El ínfimo selecciona el menor valor donde la probabilidad acumulada alcanza o supera p; en una muestra finita todavía debe elegirse una convención de interpolación.

[11] Una **muestra** es el conjunto de unidades observadas; la **población** es el conjunto más amplio sobre el que se desea aprender. Describir bien una muestra no garantiza que represente a la población.

[12] La **función de distribución acumulada** es $F(x) = P(X \le x)$: la probabilidad de observar un valor no mayor que x. Es no decreciente y toma valores entre 0 y 1.

Qué ocurre al cambiar de unidad

Suponga que transformamos cada dato mediante $y_i = ax_i + b$, con $a > 0$. Entonces

$$\bar{y} = a\bar{x} + b, \qquad \tilde{y} = a\tilde{x} + b, \qquad Q_y(p) = aQ_x(p) + b.$$

Esta propiedad se llama **equivarianza afín**.[13] Por ejemplo, convertir minutos a segundos multiplica los centros por 60; retrasar todos los registros cinco minutos les suma cinco. La moda también conserva esta relación si la transformación es uno a uno.

Robustez no significa inmunidad

La media puede desplazarse arbitrariamente al mover una sola observación cada vez más lejos. En contraste, para llevar la mediana fuera del centro es necesario alterar aproximadamente la mitad de la muestra (Huber y Ronchetti 2009). Esto hace atractiva a la mediana frente a contaminación o colas largas, pero no la vuelve infalible: datos duplicados, sesgos de selección o una muestra que no representa la población afectan cualquier resumen.

La moda exige cuidado con datos continuos

En variables continuas dos mediciones exactas rara vez coinciden. La barra más alta de un histograma describe una **clase modal**, pero puede cambiar al mover el origen o el ancho de los intervalos. Una estimación por densidad también depende de un parámetro de suavizado.[14] Por ello, la moda suele ser más natural para categorías o conteos discretos que para mediciones continuas.

Implementación reproducible

La siguiente función explicita las convenciones usadas en el cálculo manual. En particular, los cuartiles se obtienen como medianas de las mitades y el recorte se expresa como un número entero de observaciones por cola.

La sensibilidad puede comprobarse sin cambiar los primeros ocho tiempos:

	Novena entrega	Media	Mediana	Media recortada
0	27	22.67	22.0	22.71
1	30	23.00	22.0	22.71
2	75	28.00	22.0	22.71
3	120	33.00	22.0	22.71

[13] Una transformación afín combina un cambio de escala, a, con un desplazamiento, b. La condición $a > 0$ conserva el orden de las observaciones.

[14] Una **densidad de probabilidad** $p(\mathbf{x})$ describe cómo se reparten los datos en el espacio: donde p es alta hay muchos casos, donde es baja hay pocos. La probabilidad de que un caso caiga en una región es la integral de p sobre esa región. En clasificación nos interesaba $P(y \mid \mathbf{x})$; aquí, a falta de y, el objeto de interés es $p(\mathbf{x})$ mismo.

Exploración visual de la sensibilidad

El mismo experimento puede recorrerse sobre una gráfica interactiva. Las líneas usan exactamente los cuatro escenarios de la tabla anterior.

Para datos resumidos por grupos, la media ponderada debe validar que los pesos sean no negativos y que al menos uno sea positivo:

Guía de decisión

Pregunta	Medida principal	Qué debe acompañarla
¿Qué valor conserva el total al repartirlo por igual?	Media	Tamaño de muestra, dispersión y presencia de extremos
¿Cuál es el caso central por posición?	Mediana	Cuartiles y forma de la distribución
¿Qué valor o categoría se repite más?	Moda	Tabla de frecuencias; informar empates
¿Qué umbral cubre una proporción p?	Cuantil $Q(p)$	Método de cálculo y cantidad de datos en la cola
¿Cada resumen representa un número distinto de casos?	Media ponderada	Significado y suma de los pesos
¿Se desea un centro menos sensible sin abandonar el promedio?	Media recortada	Porcentaje retirado en cada cola, fijado de antemano

Una buena descripción suele reportar más de una pieza: centro, variabilidad y una gráfica. Media y mediana cercanas no demuestran normalidad; distribuciones multimodales o con colas pesadas también pueden compartir ambos centros.

Errores frecuentes

- **Eliminar el extremo solo porque cambia la media.** Primero se verifica su origen. Un cierre vial real y un error de captura requieren decisiones distintas.
- **Promediar códigos nominales.** Si `bicicleta=1`, `moto=2` y `auto=3`, la media de los códigos cambia al renombrarlos y no tiene interpretación métrica.[15]
- **Confundir moda con mayoría.** Que 20 sea la moda no significa que más de 50 % de las entregas tarde 20 minutos; aquí solo son 2 de 9.
- **Ocultar pesos.** Promediar promedios de grupos sin sus tamaños concede el mismo peso a grupos grandes y pequeños.
- **Comparar cuantiles calculados con convenciones distintas.** En muestras pequeñas, la interpolación puede cambiar el resultado.

[15] Una variable **nominal** usa etiquetas sin orden ni distancias naturales, como color o medio de transporte. Los números asignados como códigos son identificadores, no magnitudes.

- **Usar la media recortada sin declarar el recorte.** "Media robusta" no basta; se debe informar cuánto se retiró de cada cola.

Puentes con minería de datos

Contexto	Papel de las medidas de localización
Imputación de faltantes	Media o mediana sirven como línea base; la mediana suele resistir mejor colas y extremos, pero ambas reducen artificialmente la variabilidad.
Escalamiento robusto	Centrar con la mediana y escalar con el rango intercuartílico reduce la influencia de extremos.
Regresión	La predicción que minimiza error cuadrático estima una media condicional; con error absoluto, una mediana condicional.[16]
Pronóstico de servicio	Cuantiles condicionales permiten estimar tiempos que cubren 50 %, 90 % o 95 % de los casos.
Segmentación	Comparar centros por grupo solo es informativo junto con tamaños, dispersiones y distribuciones.
Monitoreo	Cambios en media y mediana pueden distinguir un desplazamiento general de unos pocos incidentes extremos.

Una medida descriptiva no explica por qué cambió el proceso ni demuestra que una variable cause otra. Primero resume; la explicación requiere contexto, diseño y análisis adicional.

Puente al caso BigMart

El capítulo Medidas de localización en Big Mart Sales traslada esta secuencia a variables como peso, visibilidad, precio y ventas:

1. inspeccionar la distribución y la unidad de medida;
2. formular qué significa un valor "típico" en ese contexto;
3. reproducir media, mediana, moda, cuantiles o centros ponderados;
4. comparar su sensibilidad a asimetrías y extremos;
5. interpretar el centro junto con variabilidad y forma.

El código aplicado deja de ser una colección de funciones: cada cálculo queda asociado con una pregunta y con los supuestos aprendidos en este capítulo.

[16] $P(y = c \mid \mathbf{x})$ es una **probabilidad condicional**: la proporción de casos de clase c entre todos los que tienen exactamente las predictoras $\mathbf{x}$. Por ejemplo, entre todos los clientes con ingreso 30 y endeudamiento 58, la fracción que paga.

Ejercicios formativos

1. **Cálculo completo.** Para los tiempos $(12, 14, 14, 15, 16, 18, 19, 21, 51)$, calcule media, mediana, moda, Q_1, Q_3 por medianas de mitades y media recortada retirando un dato por cola.
2. **Sensibilidad exacta.** En el ejemplo de entregas, cambie 75 por 210. Prediga el cambio de la media mediante la ecuación (4) antes de recalcular. Explique por qué la mediana no cambia.
3. **Dos pérdidas.** Calcule $S_1(a)$ y $S_2(a)$ para $a = 20, 22, 28$ usando los nueve tiempos. Verifique dónde es menor cada pérdida y explique por qué los mínimos difieren.
4. **Media ponderada.** Una cuarta zona tarda 55 minutos y aporta 5 entregas. Actualice la media ponderada. Compare el resultado con el promedio sin pesos de los cuatro tiempos zonales.
5. **Mismo centro, distinta distribución.** Construya dos conjuntos de nueve valores con la misma media y mediana, pero con dispersiones claramente distintas. Dibújelos antes de concluir que se parecen.
6. **Moda sin mayoría.** Construya un conjunto donde la moda aparezca más que cualquier otro valor pero represente menos de 25 % de las observaciones.
7. **Decisión de servicio.** Argumente qué medida usaría para (a) presupuestar horas totales de trabajo, (b) describir una entrega habitual y (c) prometer un tiempo que cubra 90 % de los pedidos. Indique qué información adicional necesitaría.

23. Medidas de localización en Big Mart Sales

Este capítulo aplica los conceptos desarrollados en Medidas de localización. Conviene revisar primero las diferencias entre media, mediana, moda y cuantiles.

Desde la perspectiva del análisis exploratorio de datos, las medidas de localización responden a una pregunta fundamental: **¿en qué región de valores tiende a concentrarse el fenómeno observado?**. En términos prácticos, esto permite identificar valores típicos, comparar distribuciones y comenzar a interpretar el comportamiento general del sistema antes de pasar a medidas de dispersión, forma o segmentación.

El dataset incluye variables numéricas como `Item_Weight`, `Item_Visibility`, `Item_MRP` y `Item_Outlet_Sales`, junto con variables categóricas asociadas a tipo de producto, establecimiento y ubicación del outlet. Para este primer analisis nos enfocaremos en el comportamiento univariado de las variables numéricas, usando medidas como la **media**, la **mediana**, la **moda** y algunos **cuantiles**.

Definición operativa. En esta práctica, una medida de localización resume un valor típico o una posición relevante de una variable numérica; ninguna medida se interpreta sin observar también su distribución (Tukey 1977; Hyndman y Fan 1996).

Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy y Plotly.

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614

Inspección inicial

Antes de calcular medidas de localización conviene identificar el tipo de variables disponibles y confirmar qué columnas numéricas serán analizadas.

En este punto es útil destacar que no todas las variables numéricas tienen el mismo significado. Por ejemplo, `Item_MRP` representa el precio máximo sugerido del producto, mientras que `Item_Outlet_Sales` corresponde a las ventas del producto en un outlet particular. La interpretación del "centro" de cada variable, por tanto, depende del contexto económico de la medición.

Selección de variables para el análisis de localización

Para esta etapa trabajaremos con un subconjunto de variables numéricas especialmente interpretables:

- `Item_Weight`
- `Item_Visibility`
- `Item_MRP`
- `Item_Outlet_Sales`

Crearemos un nuevo dataframe con estas columnas para facilitar el análisis, identificando además el número de datos faltantes en las mismas.

	missing	missing_pct
Item_Weight	1463	17.165317
Item_Visibility	0	0.000000
Item_MRP	0	0.000000
Item_Outlet_Sales	0	0.000000

Datos faltantes y su impacto en el análisis de localización

La presencia de valores faltantes en la variable `Item_Weight` no impide comenzar el análisis descriptivo de localización, **sin embargo, eliminarlos sin analizar su origen puede introducir sesgos en los resultados**. Por ello, antes de calcular medidas descriptivas es necesario examinar la magnitud y el posible patrón de los datos ausentes. Extrayendo un dataframe con los casos faltantes en `Item_Weight` tenemos:

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outl
7	FDP10	NaN	Low Fat	0.127470	Snack Foods	107.7622	OUT
18	DRI11	NaN	Low Fat	0.034238	Hard Drinks	113.2834	OUT
21	FDW12	NaN	Regular	0.035400	Baking Goods	144.5444	OUT
23	FDC37	NaN	Low Fat	0.057557	Baking Goods	107.6938	OUT
29	FDC14	NaN	Regular	0.072222	Canned	43.6454	OUT

Si bien no es el objetivo de esta sección resolver el problema de los datos faltantes, es importante tener una noción clara de su impacto. Para esto existen tres preguntas clave que podemos hacernos en estos casos:

1. **¿Qué porcentaje de datos faltantes hay en la variable?**

Este paso permite dimensionar la magnitud del problema. Calcular la proporción de valores ausentes respecto al total de observaciones ayuda a evaluar si los faltantes representan una fracción pequeña del dataset o si son lo suficientemente numerosos como para afectar la representatividad del análisis.

En nuestro caso la variable presenta aproximadamente un 17 % de valores faltantes, lo que implica que cerca de una sexta parte de las observaciones no contiene información para este atributo. Esta proporción no es despreciable, por lo que antes de excluir estos registros conviene examinar si los datos faltantes siguen algún patrón sistemático y evaluar el posible impacto de su eliminación en el análisis exploratorio.

2. **¿Existen patrones de ausencia relacionados con otras variables?**

Aquí se busca identificar si los datos faltantes aparecen de manera aleatoria o si se concentran en ciertos grupos (por ejemplo, en determinadas categorías de producto, tipos de outlet o rangos de precio). Detectar patrones de ausencia es importante porque podría indicar un posible sesgo en los datos.

Analicemos el promedio de valores faltantes en `Item_Weight` por tipo de producto y tipo de outlet:

Podemos observar como la falta de datos en `Item_Weight` se concentra principalmete en 2 tipos de outlets : `Grocery Store` y `Supermarket Type1`. Esto sugiere que la ausencia de información no es aleatoria en estos casos, mientras que tenemos una distribución más equilibrada en `Item_Type`.

Este patrón de ausencia podría estar relacionado con la naturaleza de los productos comercializados en esos outlets o con prácticas de registro de datos específicas, lo que implica que eliminar estos casos sin un análisis previo podría sesgar el análisis descriptivo.

3. **¿Cuál es el impacto potencial de eliminar casos con datos faltantes en el análisis?**

En este punto, la pregunta clave ya no es solo si existen datos faltantes, sino si su eliminación cambia nuestra lectura general de los datos. Por ello, una forma natural de continuar el EDA es comparar medidas de localización antes y después de excluir los casos incompletos. Si estas medidas permanecen estables, la omisión de dichos registros tendría un impacto limitado; si cambian de manera apreciable, entonces la ausencia de datos sí afecta la interpretación.

Con lo cual procederemos en las siguientes secciones a evaluar estas medidas de localizacion asociadas.

Medidas de localización básicas

Como vimos en el capitulo pasado las medidas más usadas en esta etapa son:

- **Media**:
$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$
- **Mediana**: valor que deja 50 % de observaciones a cada lado.
- **Moda**: valor más frecuente. En variables continuas muchas veces es más útil hablar de la región modal observada en un histograma.
- **Cuantiles**: percentiles como $Q_{0.25}$, $Q_{0.50}$ y $Q_{0.75}$.

Estas se pueden conocer de manera rápida en pandas mediante los métodos `mean()`, `median()`, `mode()` y `quantile()`. Para facilitar la lectura comparativa, conviene construir una tabla resumen con estas medidas para cada variable numérica clave.

	media	mediana	moda	q25	q75	min	max
Item_Weight	12.857645	12.600000	12.1500	8.773750	16.850000	4.555	21.350000
Item_Visibility	0.066132	0.053931	0.0000	0.026989	0.094585	0.000	0.328391
Item_MRP	140.992782	143.012800	172.0422	93.826500	185.643700	31.290	266.888400
Item_Outlet_Sales	2181.288914	1794.331000	958.7520	834.247400	3101.296400	33.290	13086.964800

Esta tabla permite una primera lectura comparativa. La relación entre media y mediana es particularmente informativa. Si ambas están muy próximas, la distribución podría ser relativamente simétrica o al menos no estar muy afectada por colas extremas. Si difieren de forma notable, conviene anticipar que más adelante aparecerán señales de asimetría.

Interpretación estadística de la media y la mediana

En datos de ventas es común encontrar distribuciones con sesgo a la derecha. Esto significa que la mayor parte de las observaciones se agrupan en niveles bajos o medios, pero existe una cola de valores altos que empuja la media hacia arriba. En ese caso, la media deja de representar un valor "típico" tan bien como la mediana.

Podemos ilustrarlo directamente con histogramas sobre los que se tracen ambas medidas.

La superposición de estas líneas verticales ofrece una interpretación inmediata. En particular, si en `Item_Outlet_Sales` la media aparece sensiblemente a la derecha de la mediana, puede concluirse que las ventas no se distribuyen de forma simétrica y que existen casos de ventas altas elevan el promedio. En cambio, si en `Item_MRP` ambas medidas son relativamente cercanas, la estructura de precios podría ser más estable o menos sesgada.

Cuantiles y lectura ampliada del centro

Cuando hablamos de **localización**, no siempre basta con resumir una variable mediante un único valor, como la media o la mediana. En muchos casos, resulta más útil describir la **zona central** donde se concentra la mayor parte de las observaciones. Para ello, los **cuantiles** son especialmente importantes.

En particular, los percentiles 25, 50 y 75 permiten identificar el tramo donde se encuentra el **50 % central de los datos**. Esta información ayuda a responder preguntas más ricas que un simple "¿cuál es el promedio?", por ejemplo:

- ¿entre qué valores se concentra la mitad central de los casos?
- ¿qué tan alejados están los valores altos respecto del centro?
- ¿la mediana representa bien a la distribución o existen asimetrías importantes?

	Item_Weight	Item_Visibility	Item_MRP	Item_Outlet_Sales
0.10	6.69500	0.012042	52.79560	343.5528
0.25	8.77375	0.026989	93.82650	834.2474
0.50	12.60000	0.053931	143.01280	1794.3310
0.75	16.85000	0.094585	185.64370	3101.2964
0.90	19.35000	0.139514	231.20048	4570.0512

La tabla de cuantiles permite ampliar la lectura del centro. Mientras la media resume la localización en un solo número, los cuantiles muestran cómo se distribuyen los datos alrededor de esa zona central. Así, el percentil 25 marca el punto por debajo del cual se encuentra el 25 % de las observaciones, la mediana divide la distribución en dos partes iguales, y el percentil 75 indica el límite superior del 50 % central.

Para complementar esta lectura, es útil recurrir a los boxplots, ya que representan visualmente la mediana, los cuartiles, la dispersión central y la posible presencia de valores atípicos.

A partir de estos gráficos, la interpretación ya no se limita a un valor "promedio", sino que puede centrarse en la estructura de la distribución. En particular, los boxplots permiten observar:

- dónde se ubica la mediana;
- qué tan amplio es el rango intercuartílico;
- si la distribución parece simétrica o sesgada;
- y si existen valores atípicos que puedan influir en la media.

De esta forma, los cuantiles y los boxplots enriquecen el análisis de localización, porque muestran no solo un centro numérico, sino también la región en la que efectivamente se concentra la mayor parte de los datos.

Impacto de eliminar faltantes en `Item_Weight`

Para evaluar si eliminar las observaciones con valores faltantes en `Item_Weight` introduce sesgos en el análisis, compararemos el comportamiento del dataset antes y después de excluir dichos casos. La idea es verificar si esta reducción de la muestra modifica de forma importante la localización de otras variables relevantes y la composición general de los datos.

Primero construimos dos subconjuntos:

- el dataset original;
- el dataset filtrado, conservando solo las observaciones con `Item_Weight` disponible.

Una primera forma de evaluar el posible sesgo consiste en comparar medidas de localización de variables relevantes antes y después del filtrado.

	media_original	media_filtrado	mediana_original	mediana_filtrado	dif_media_abs
Item_MRP	140.993	141.241	143.013	142.730	0.248
Item_Outlet_Sales	2181.289	2118.627	1794.331	1789.670	-62.662

	media_original	media_filtrado	mediana_original	mediana_filtrado	dif_media_abs
Item_Visibility	0.066	0.064	0.054	0.052	-0.002

Además de la comparación numérica, es útil observar gráficamente si las distribuciones cambian al eliminar los registros con Item_Weight faltante.

Para reforzar la lectura de localización, también podemos comparar boxplots en ambos escenarios.

Finalmente, también conviene revisar si la eliminación afecta la composición de algunos grupos del negocio.

Outlet_Type	original	filtrado	dif_pct_points
Supermarket Type1	0.6543	0.7899	13.5596
Grocery Store	0.1271	0.0786	-4.8456
Supermarket Type3	0.1097	NaN	NaN
Supermarket Type2	0.1089	0.1314	2.2563

Los resultados muestran que la eliminación de observaciones con `Item_Weight` faltante produce cambios pequeños en las medidas de localización de las variables analizadas. Si bien en algunos casos las diferencias porcentuales superan ligeramente el 2 %, especialmente en `Item_Outlet_Sales` e `Item_Visibility`, la magnitud de estas variaciones no parece suficiente para modificar de manera sustantiva la interpretación general del centro de las distribuciones. Por ello, dentro del contexto de este EDA, trabajar con los casos completos puede considerarse una decisión razonable y con bajo riesgo de sesgo en términos de localización.

	Variable	N válidos	Media	Mediana	Q1	Q3
0	Item_Weight	7060	12.857645	12.600000	8.773750	16.850000
1	Item_Visibility	8523	0.066132	0.053931	0.026989	0.094585
2	Item_MRP	8523	140.992782	143.012800	93.826500	185.643700
3	Item_Outlet_Sales	8523	2181.288914	1794.331000	834.247400	3101.296400

Ejercicio sugerido

Construya una versión inicial de dashboard en Quarto que incluya:

- una tabla resumen con media, mediana y cuantiles de las variables numéricas seleccionadas;
- un histograma con líneas para media y mediana de `Item_Outlet_Sales`;
- un gráfico de barras con la mediana de ventas por `Outlet_Type`;
- un breve comentario interpretativo donde se compare el uso de la media frente a la mediana.

Parte XIII.

Medidas de variabilidad

24. Medidas de variabilidad

El problema: cumplir el promedio no basta

Dos equipos entregan ocho pedidos cada uno. En ambos casos, el tiempo medio es de 15.5 minutos:

Pedido ordenado	1	2	3	4	5	6	7	8	Media
Equipo Regular	12	14	15	15	16	16	17	19	15.5
Equipo Irregular	5	8	12	15	16	19	23	26	15.5

Si solo se reporta el promedio, los equipos parecen equivalentes. Sin embargo, para quien espera su pedido no lo son: con el primer equipo casi todos los tiempos están cerca de 15.5; con el segundo pueden ser muy cortos o muy largos.

La **variabilidad** describe cuánto difieren los valores entre sí o respecto de un centro. Es una propiedad de una variable cuantitativa:[1] no basta saber dónde está el centro de la distribución, también importa cuánto se extiende alrededor de él (Tukey 1977; Heckert et al. 2012).

Dos casos que servirán como brújula

- Si los ocho pedidos tardaran exactamente 15.5 minutos, la variabilidad debería ser **cero**.
- Si unos tiempos se alejan del centro mientras otros se acercan de manera que la media no cambia, la variabilidad debería **aumentar**.

Una buena medida de dispersión debe distinguir estos casos, pero no todas responden igual ante valores extremos ni se expresan en las mismas unidades.

Primera mirada: la amplitud total

El **rango** resta el mínimo al máximo. Si $x_{(1)} \leq \cdots \leq x_{(n)}$ son los datos ordenados,[2]

$$R = x_{(n)} - x_{(1)}. \tag{1}$$

Para el Equipo Regular,

[1] Una **variable cuantitativa** representa cantidades para las que las diferencias numéricas tienen significado. No toda etiqueta codificada con números es cuantitativa: asignar 1, 2 y 3 a tres colores no convierte sus diferencias en distancias.

[2] $x_{(i)}$ denota el dato que ocupa la posición i después de ordenar. Los paréntesis distinguen una posición ordenada de la observación original x_i.

$$R_R = 19 - 12 = 7 \text{ minutos;}$$

para el Equipo Irregular,

$$R_I = 26 - 5 = 21 \text{ minutos.}$$

El segundo equipo cubre una amplitud tres veces mayor. La conclusión es útil, pero se apoya únicamente en dos observaciones. Si el tiempo 19 del primer equipo fuera un error de captura y en realidad fuera 49, su rango saltaría de 7 a 37 minutos aunque los otros siete tiempos permanecieran intactos.

> El rango contesta "¿qué tan separados están los dos extremos?". No contesta "¿qué tan lejos del centro está un pedido típico?". Para esa segunda pregunta necesitamos usar todos los datos.

De las desviaciones a la varianza

El obstáculo de los signos

Tomemos los tiempos del Equipo Regular:

$$x = (12, 14, 15, 15, 16, 16, 17, 19).$$

Su suma es 124 y su media es

$$\bar{x} = \frac{124}{8} = 15.5.$$

Una **desviación** es la distancia con signo $x_i - \bar{x}$ entre una observación y la media:

x_i	12	14	15	15	16	16	17	19
$x_i - 15.5$	-3.5	-1.5	-0.5	-0.5	0.5	0.5	1.5	3.5

Las desviaciones suman cero. Esto no demuestra ausencia de variabilidad: las distancias a la izquierda y a la derecha de la media se cancelan. Elevarlas al cuadrado evita la cancelación y da más peso a las desviaciones grandes:

$x_i - 15.5$	-3.5	-1.5	-0.5	-0.5	0.5	0.5	1.5	3.5
$(x_i - 15.5)^2$	12.25	2.25	0.25	0.25	0.25	0.25	2.25	12.25

La **suma de cuadrados** es

$$SS = \sum_{i=1}^{8} (x_i - 15.5)^2 = 30 \text{ minutos}^2. \tag{2}$$

La suma crece si hay más observaciones, aunque la dispersión no cambie. Por eso se divide entre una cantidad relacionada con el número de datos. El divisor correcto depende de qué representan esos ocho pedidos.

¿Población completa o muestra?

Cuando los datos son todo lo que queremos describir

Si esos ocho pedidos son **todos** los pedidos del turno y la pregunta se limita a describir ese turno, constituyen la población de interés.[3] La varianza poblacional es

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2. \tag{3}$$

donde N es el tamaño de la población y μ su media. Para el turno:

$$\sigma^2 = \frac{30}{8} = 3.75 \text{ minutos}^2.$$

La varianza queda en unidades **cuadradas** porque también lo están sus sumandos. Para regresar a la escala de los datos se extrae la raíz cuadrada:

$$\sigma = \sqrt{3.75} = 1.9365 \text{ minutos}. \tag{4}$$

La desviación estándar poblacional es, por tanto, de aproximadamente 1.94 minutos.

Cuando los datos pretenden representar más casos

Si los ocho pedidos son una muestra aleatoria de todos los pedidos que el equipo podría atender, $\bar{x}$ y s^2 son **estadísticos** usados para estimar parámetros desconocidos de una población mayor.[4] La varianza muestral usual es

$$s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2. \tag{5}$$

En el ejemplo,

[3] La **población** es el conjunto completo sobre el que se formula una afirmación. No tiene que ser enorme: los ocho pedidos de un turno son una población si la pregunta se limita exactamente a ese turno.

[4] **Parámetro aprendido frente a regla fija:** una media o una lista de categorías son *estadísticos* calculados con datos; si cambian las filas, cambian ellos. Una regla como «`LF` significa `Low Fat`» no depende de las filas y puede aplicarse a cualquier conjunto sin riesgo de fuga.

$$s^2 = \frac{30}{8-1} = 4.2857 \text{ minutos}^2, \qquad s = \sqrt{4.2857} = 2.0702 \text{ minutos}.$$

No cambiaron los datos ni la suma de cuadrados: cambió la finalidad. Dividir entre n describe exactamente la colección observada; dividir entre $n-1$ produce, bajo muestreo independiente de una población con varianza finita, un estimador insesgado de σ^2 (Freedman et al. 2007).[5]

¿De dónde sale el grado de libertad perdido?

Una vez calculada $\bar{x}$, las desviaciones cumplen

$$\sum_{i=1}^{n}(x_i - \bar{x}) = 0.$$

Si conocemos $n-1$ desviaciones, la última ya no puede variar libremente: debe ser el número que haga cero la suma. Quedan $n-1$ **grados de libertad**.[6] La llamada corrección de Bessel compensa además que la media muestral fue ajustada con los mismos datos y hace a s^2 insesgada. Esto no significa que s sea insesgada ni que siempre deba dividirse entre $n-1$.

Una convención no se elige por el tamaño del archivo

No use $n-1$ solo porque hay pocos datos ni n solo porque hay muchos. Primero defina el universo de la afirmación: "estos ocho pedidos" pide descripción poblacional; "el desempeño futuro del equipo" plantea una inferencia desde una muestra y requiere, además, justificar cómo se obtuvieron los pedidos.

Misma media, diferente variabilidad

Para el Equipo Irregular, la media también es 15.5, pero

$$\begin{aligned} SS_I &= (-10.5)^2 + (-7.5)^2 + (-3.5)^2 + (-0.5)^2 \\ &\quad + (0.5)^2 + (3.5)^2 + (7.5)^2 + (10.5)^2 \\ &= 358. \end{aligned}$$

Si los ocho pedidos son la población de cada turno,

$$\sigma_I^2 = \frac{358}{8} = 44.75, \qquad \sigma_I = \sqrt{44.75} = 6.6895 \text{ minutos}.$$

La tabla reúne lo aprendido:

[5] Un estimador es **insesgado** si, al imaginar todas las muestras posibles obtenidas bajo el mismo diseño, el promedio de sus estimaciones coincide con el parámetro. No garantiza que una muestra particular produzca el valor exacto.

[6] **Grados de libertad:** número de conteos que pueden variar una vez fijados los totales de filas y columnas. En una tabla 2×2 queda uno.

Medida poblacional	Equipo Regular	Equipo Irregular
Media	15.5 min	15.5 min
Rango	7 min	21 min
Varianza	3.75 min^2	44.75 min^2
Desviación estándar	1.94 min	6.69 min

La media no distingue los equipos; las tres medidas de variabilidad sí. La desviación estándar suele ser más fácil de comunicar que la varianza porque está en minutos, pero la varianza es central en muchos algoritmos por sus propiedades algebraicas.

Cómo leer una desviación estándar

$\sigma = 1.94$ minutos no significa que todos los pedidos estén a 1.94 minutos de la media ni que sea la distancia absoluta promedio. Es la raíz del promedio de distancias **cuadradas**. Si la distribución fuera aproximadamente normal, alrededor del 68 % de los valores caería entre $\mu - \sigma$ y $\mu + \sigma$; esa regla no se traslada automáticamente a distribuciones sesgadas o multimodales.[7]

Laboratorio: mover los pedidos y observar las medidas

El laboratorio de la versión HTML permite mover los ocho tiempos y actualiza media, rango, varianza poblacional, desviación estándar, IQR y MAD. Sus cuatro escenarios de referencia son:

Escenario	Datos	Media	σ	IQR	MAD
Regular	12, 14, 15, 15, 16, 16, 17, 19	15.5	1.94	2.0	1.0
Irregular	5, 8, 12, 15, 16, 19, 23, 26	15.5	6.69	11.0	3.5
Retraso extremo	12, 14, 15, 15, 16, 16, 17, 45	18.75	10.02	2.0	1.0
Todos iguales	16, 16, 16, 16, 16, 16, 16, 16	16.0	0.00	0.0	0.0

El retraso extremo mueve con fuerza la media y la desviación estándar, pero no cambia el IQR ni el MAD bajo la convención de cuartiles usada en el capítulo.

Variabilidad del centro: IQR y MAD

La varianza y la desviación estándar usan todos los valores y reaccionan con fuerza a un extremo porque elevan las desviaciones al cuadrado. Cuando hay colas largas, errores o eventos extraordinarios, conviene acompañarlas con medidas **robustas**.[8]

[7] La **distribución normal** es un modelo continuo, simétrico y con forma de campana determinado por su media y varianza. La regla 68 % es una propiedad de ese modelo, no una definición de desviación estándar.

[8] Una medida es **robusta** si una fracción pequeña de observaciones atípicas no puede alterarla arbitrariamente. Robustez no significa insensibilidad total ni garantiza que el dato extremo sea erróneo.

Intervalo intercuartílico

Los cuartiles Q_1 y Q_3 dejan aproximadamente 25 % y 75 % de los datos a su izquierda. El **intervalo intercuartílico** es

$$IQR = Q_3 - Q_1. \tag{6}$$

Para evitar ambigüedad, usaremos el método de medianas de las mitades: con un número par de datos, Q_1 es la mediana de la mitad inferior y Q_3 la de la mitad superior.[9]

En el Equipo Regular,

$$L = (12, 14, 15, 15), \qquad U = (16, 16, 17, 19),$$

$$Q_1 = \frac{14 + 15}{2} = 14.5, \qquad Q_3 = \frac{16 + 17}{2} = 16.5,$$

$$IQR = 16.5 - 14.5 = 2 \text{ minutos}.$$

Así, la mitad central de los tiempos ocupa un intervalo de 2 minutos. Para el Equipo Irregular, $Q_1 = 10$, $Q_3 = 21$ e $IQR = 11$ minutos.

La regla de Tukey identifica como **candidatos** a valores atípicos los que caen fuera de

$$[Q_1 - 1.5IQR,\ Q_3 + 1.5IQR]. \tag{7}$$

Para el Equipo Regular los límites son

$$[14.5 - 1.5(2),\ 16.5 + 1.5(2)] = [11.5, 19.5].$$

Ningún tiempo queda fuera. La regla es una señal de investigación, no una orden para borrar observaciones (Tukey 1977). Un retraso puede ser real y contener información operacional importante.

[9] Existen varias definiciones válidas de cuartiles porque una muestra finita no determina de manera única los percentiles entre posiciones. El software puede interpolar y producir valores distintos. Debe declararse el método al reproducir resultados (Hyndman y Fan 1996).

Desviación absoluta mediana

La **MAD** (*median absolute deviation*) reemplaza la media por la mediana y los cuadrados por distancias absolutas:

$$MAD = \text{mediana}_i\{|x_i - \tilde{x}|\}, \tag{8}$$

donde $\tilde{x}$ es la mediana. Para el Equipo Regular,

$$\tilde{x} = \frac{15 + 16}{2} = 15.5$$

y las distancias absolutas ordenadas son

$$(0.5, 0.5, 0.5, 0.5, 1.5, 1.5, 3.5, 3.5).$$

Por tanto,

$$MAD = \frac{0.5 + 1.5}{2} = 1 \text{ minuto.}$$

Si se desea una escala comparable con la desviación estándar bajo una distribución normal, se usa con frecuencia

$$MAD_N = 1.4826\, MAD = 1.4826 \text{ minutos.} \tag{9}$$

La constante no vuelve normales a los datos: solo calibra la medida bajo ese modelo (Rousseeuw y Croux 1993). En datos discretos o con muchos empates, la MAD puede ser cero aunque no todos los valores sean iguales.

Un retraso extremo

Compare el turno regular con $x^* = (12, 14, 15, 15, 16, 16, 17, 45)$, donde solo el último tiempo cambió:

Medida poblacional	Turno regular	Con retraso extremo	Cambio
Media	15.50	18.75	+3.25
Desviación estándar	1.94	10.02	+8.09
Rango	7.00	33.00	+26.00
IQR	2.00	2.00	0.00
MAD	1.00	1.00	0.00

IQR y MAD no “ignoran” por definición los valores extremos: su resistencia proviene de posiciones centrales y puede desaparecer si la contaminación alcanza una fracción suficiente de los datos. Por eso se reportan junto con la distribución y no como sustitutos automáticos de toda medida clásica.

Comparar variabilidad entre escalas

Suponga que otro proceso registra tiempos en segundos. Su desviación estándar será 60 veces mayor aunque la regularidad física sea la misma. Las medidas en unidades originales no permiten comparar directamente variables con escalas muy distintas.

El **coeficiente de variación** mide dispersión relativa:

$$CV = \frac{\sigma}{|\mu|} \quad \text{o, en una muestra,} \quad CV_m = \frac{s}{|\bar{x}|}. \tag{10}$$

Para el turno regular considerado como población,

$$CV = \frac{1.9365}{15.5} = 0.1249 = 12.49\,\%.$$

Para el turno irregular,

$$CV = \frac{6.6895}{15.5} = 0.4316 = 43.16\,\%.$$

El segundo turno tiene una dispersión equivalente a 43.16 % de su media. El CV es adimensional y no cambia al convertir minutos a segundos, pero solo es interpretable en escalas de razón con cero absoluto.[10] No debe emplearse si la media es cero o cercana a cero; tampoco para comparar temperaturas en grados Celsius, cuyo cero es convencional.

Formalización y propiedades

Sea X una variable aleatoria numérica con media $\mu = \mathbb{E}[X]$ y segundo momento finito. Su varianza se define como

$$\operatorname{Var}(X) = \mathbb{E}\big[(X-\mu)^2\big] = \mathbb{E}[X^2] - \mu^2. \tag{11}$$

La segunda igualdad se obtiene al expandir el cuadrado y usar la linealidad de la esperanza. La desviación estándar es

$$\operatorname{SD}(X) = \sqrt{\operatorname{Var}(X)}. \tag{12}$$

Estas definiciones implican propiedades que una implementación debe respetar:

1. $\operatorname{Var}(X) \geq 0$ y vale cero exactamente cuando X es constante con probabilidad uno.
2. Sumar una constante cambia la ubicación, no la dispersión: $\operatorname{Var}(X + b) = \operatorname{Var}(X)$.
3. Cambiar la escala multiplica la varianza por el cuadrado: $\operatorname{Var}(aX) = a^2 \operatorname{Var}(X)$.

[10] Una **escala de razón** posee un cero que significa ausencia de la cantidad, por lo que afirmaciones como “el doble” tienen sentido. Kelvin, masa y duración la poseen; Celsius no, porque su cero es un punto convencional.

4. En consecuencia, $\mathrm{SD}(aX + b) = |a|\,\mathrm{SD}(X)$; la desviación estándar conserva las unidades originales.

Para $a = 60$, convertir minutos a segundos multiplica la varianza por 3600 y la desviación estándar por 60. En cambio, CV permanece igual si $a > 0$ porque numerador y denominador reciben el mismo factor.

Variabilidad observada no equivale a incertidumbre de la media

La desviación estándar describe diferencias **entre observaciones**. El error estándar de la media describe cuánto variaría $\bar{x}$ entre muestras del mismo tamaño; bajo observaciones independientes es $s/\sqrt{n}$. Son preguntas distintas y no deben intercambiarse.[11]

Implementación reproducible

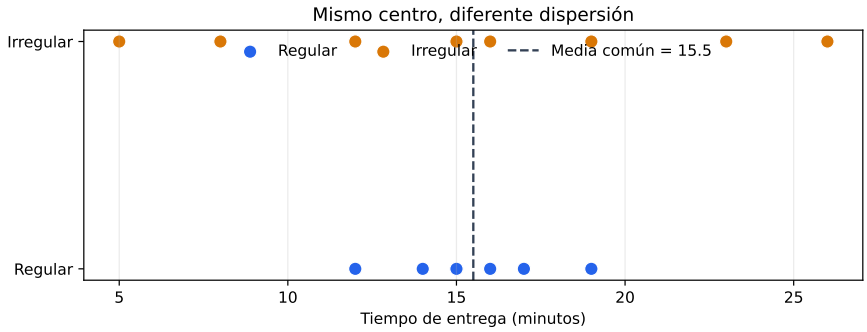
La función siguiente hace explícitas dos decisiones que a menudo quedan ocultas: si se usa divisor poblacional o muestral, y cómo se calculan los cuartiles.

	Regular (población)	Regular (muestra)	Irregular (población)
n	8.0000	8.0000	8.0000
media	15.5000	15.5000	15.5000
rango	7.0000	7.0000	21.0000
varianza	3.7500	4.2857	44.7500
desviación estándar	1.9365	2.0702	6.6895
Q1	14.5000	14.5000	10.0000
Q3	16.5000	16.5000	21.0000
IQR	2.0000	2.0000	11.0000
MAD	1.0000	1.0000	5.5000
CV	0.1249	0.1336	0.4316

Ver la dispersión antes de resumirla

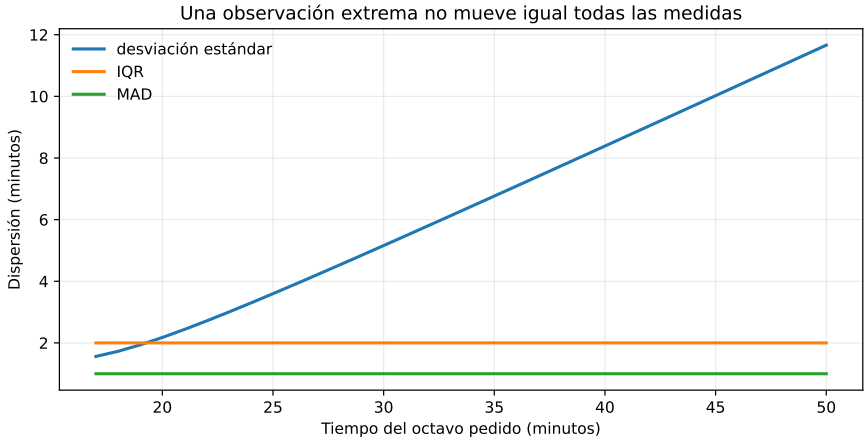
Un gráfico de puntos conserva los ocho valores y deja visible que ambos equipos comparten centro. El resumen numérico explica luego cuánto difieren.

[11] El **error estándar** es la desviación estándar de la distribución muestral de un estimador. Cuantifica incertidumbre inferencial, no la dispersión individual de los datos.



Sensibilidad frente a un extremo

Ahora conservamos siete tiempos y desplazamos solo el octavo. Esta simulación traduce a código el experimento del laboratorio.



El gráfico no declara que una medida sea siempre superior. Si interesa modelar variación cuadrática alrededor de la media, la desviación estándar responde a la pregunta correcta. Si interesa el comportamiento central bajo contaminación, IQR o MAD suelen describirlo mejor (Huber y Ronchetti 2009).

Qué observaciones aportan la variación

El siguiente diagrama conserva los dieciséis tiempos y muestra la contribución cuadrática de cada pedido respecto de la media común de 15.5 minutos.

Guía de decisión

Pregunta	Medida principal	Unidades	Precaución
¿Qué distancia hay entre extremos?	Rango	Originales	Usa solo dos valores y suele crecer con n.
¿Cuál es la variación cuadrática alrededor de la media?	Varianza	Cuadradas	Sensible a extremos; distinguir N de $n-1$.
¿Cuál es la escala típica de variación alrededor de la media?	Desviación estándar	Originales	No es una distancia absoluta promedio.
¿Qué amplitud ocupa el 50 % central?	IQR	Originales	Depende de la convención de cuartiles.
¿Cuál es la desviación central resistente?	MAD	Originales	Puede valer cero con muchos empates.
¿Qué tan grande es la dispersión respecto de la media?	CV	Ninguna	Exige escala de razón y media alejada de cero.

En un análisis responsable, la elección nace de la pregunta y del mecanismo de datos. No se selecciona la cifra más pequeña ni se elimina un extremo solo para reducir la variabilidad.

Puentes con minería de datos

- **Estandarización.** Transformar $z_i = (x_i - \bar{x})/s$ pone variables en una escala comparable, pero transmite la sensibilidad de la media y de s a valores extremos. Una alternativa robusta usa mediana y MAD.
- **k-means.** La inercia suma distancias cuadradas a centroides; por eso los atributos de gran escala y los extremos pueden dominar la partición.
- **Detección de anomalías.** Una regla basada en z presupone que media y desviación estándar resumen bien la distribución. Cercas de Tukey o puntajes basados en MAD son alternativas exploratorias, no pruebas automáticas de error.
- **Validación de modelos.** La media de un error no basta: su variabilidad entre particiones o periodos revela estabilidad. Debe distinguirse esa dispersión del error estándar de la media.

Puente al caso BigMart

El capítulo Medidas de variabilidad en Big Mart Sales traslada la misma secuencia a ventas, precios, peso y visibilidad:

1. visualizar primero la distribución y detectar unidades, asimetrías y extremos;
2. decidir si los registros se describen como colección observada o se usan para inferir sobre una población;
3. reportar una medida clásica y, cuando corresponda, una robusta;
4. usar el CV solo en variables con escala de razón y media suficientemente alejada de cero.

Los nombres de las columnas cambian, pero no las preguntas que justifican cada medida.

Ejercicios formativos

1. **Cálculo manual.** Para $x = (6, 8, 8, 10, 13)$, calcule rango, media, suma de cuadrados, varianza poblacional, varianza muestral y ambas desviaciones estándar. Explique por qué las varianzas tienen unidades cuadradas.
2. **Misma media.** Construya dos conjuntos de cinco tiempos con media 20: uno con desviación estándar poblacional menor que 2 y otro mayor que 8. Dibuje ambos en una recta antes de calcular.
3. **Grados de libertad.** Proponga cuatro desviaciones respecto de una media. Después de fijar tres, determine la cuarta y explique por qué solo tres son libres.
4. **Cuartiles.** Para $(2, 3, 5, 7, 8, 11, 14, 30)$ calcule Q_1, Q_3, IQR y las cercas de Tukey con el método de medianas de las mitades. ¿El 30 es un candidato atípico? ¿Eso basta para borrarlo?
5. **Resistencia.** Reemplace el 30 del ejercicio anterior por 300. Compare rango, desviación estándar poblacional, IQR y MAD antes y después. Explique por qué no todas cambian en la misma proporción.
6. **Escala relativa.** Dos máquinas llenan botellas. La máquina A tiene media 250 ml y $s = 5$ ml; la B, media 1,000 ml y $s = 12$ ml. Compare su variación absoluta y relativa. ¿Cuál es más estable según cada criterio?
7. **Unidades y origen.** Demuestre con código que convertir los tiempos del Equipo Regular a segundos multiplica s^2 por 60^2, multiplica s por 60 y deja igual el CV. Explique por qué sumar 273.15 a temperaturas Celsius cambia el CV aunque no cambie su desviación estándar.
8. **Decisión analítica.** Una distribución de tiempos tiene fuerte asimetría derecha y varios retrasos reales. Diseñe un reporte con al menos un gráfico, una medida clásica y una robusta. Justifique qué información conserva cada elemento.

25. Medidas de variabilidad en Big Mart Sales

Este caso continúa el desarrollo teórico de Medidas de variabilidad y lo aplica al dataset BigMart.

Una vez identificada la ubicación o región central de las principales variables del conjunto de datos, el siguiente paso natural es estudiar su **variabilidad**. Mientras las medidas de localización responden a la pregunta "¿dónde se ubican los valores típicos?", las medidas de dispersión responden a "¿qué tan separados o concentrados están los datos alrededor de ese centro?". Esta distinción es crucial porque dos distribuciones pueden tener la misma media o la misma mediana y, sin embargo, exhibir comportamientos muy distintos en términos de estabilidad, riesgo o heterogeneidad interna.

En el contexto de **Big Mart Sales**, la variabilidad tiene una lectura operativa inmediata. Una variable con baja dispersión sugiere un comportamiento relativamente estable entre productos u outlets, mientras que una alta dispersión puede indicar mezcla de segmentos, presencia de valores extremos o una dinámica comercial desigual. Para variables como `Item_Outlet_Sales`, esto es especialmente relevante porque no basta con conocer el nivel promedio de ventas: también interesa saber si las ventas se mantienen relativamente homogéneas o si existen diferencias muy amplias entre observaciones.

Definición operativa. La variabilidad cuantifica cuánto se separan las observaciones del centro o entre sí; aquí se contrasta la desviación estándar con medidas robustas como el IQR (Tukey 1977; Rousseeuw y Croux 1993).

Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy y Plotly.

De la localización a la dispersión

Si en el capítulo anterior se observó que la media y la mediana resumen el centro, ahora interesa medir qué tanto se alejan los datos de esos valores representativos. Las medidas más comunes son:

- **Rango**:
$$R = x_{\text{máx}} - x_{\text{mín}}$$
- **Varianza muestral**:
$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

- **Desviación estándar muestral**:
$$s = \sqrt{s^2}$$
- **Recorrido intercuartílico**:
$$IQR = Q_3 - Q_1$$
- **Coeficiente de variación**:
$$CV = \frac{s}{\bar{x}}$$
siempre que la media sea positiva y tenga interpretación de escala.

Estas medidas no son intercambiables. El rango depende exclusivamente de los valores extremos, la desviación estándar resume el alejamiento promedio respecto a la media y el IQR se concentra en el 50 % central de la distribución, por lo que resulta más robusto ante observaciones atípicas.

Cálculo de medidas de variabilidad

Para el calculo de las mediads de interes podemos utilizar los metodos de pandas `.mean()`, `.var()`, `.std()`, `.min()`, `.quantile()`, `.median()` y `.max()`.

	media	varianza	desv_std	min	q1	mediana	q3
Item_Weight	12.857645	2.156169e+01	4.643456	4.555	8.773750	12.600000	16.850000
Item_Visibility	0.066132	2.662335e-03	0.051598	0.000	0.026989	0.053931	0.094585
Item_MRP	140.992782	3.878184e+03	62.275067	31.290	93.826500	143.012800	185.6437
Item_Outlet_Sales	2181.288914	2.912141e+06	1706.499616	33.290	834.247400	1794.331000	3101.2964

La interpretación de estas medidas debe hacerse con cuidado. Tanto la varianza como la desviación estándar dependen de la escala en la que se mide cada variable. Por esta razón, no es conveniente comparar de forma directa la desviación estándar de `Item_MRP` con la de `Item_Visibility`, ya que ambas variables están expresadas en magnitudes muy distintas. Para una comparación más justa, resulta más útil el **coeficiente de variación (cv)**, porque relaciona la dispersión con el tamaño del promedio.

A partir del coeficiente de variación, se observa que `Item_Outlet_Sales` (`cv = 0.782`) y `Item_Visibility` (`cv = 0.780`) son las variables con mayor dispersión relativa. Esto significa que, en ambos casos, los valores se encuentran bastante separados de su media y, por tanto, presentan una alta heterogeneidad. En términos prácticos, las ventas y la visibilidad no se comportan de manera uniforme en el dataset, sino que muestran diferencias importantes entre observaciones.

Por otro lado, `Item_MRP` presenta un coeficiente de variación intermedio (`cv = 0.442`). Aunque sus valores se distribuyen en un rango amplio, su variabilidad relativa es menor que la observada en ventas y visibilidad. Finalmente, `Item_Weight` es la variable menos dispersa en términos relativos (`cv = 0.361`), lo que sugiere que sus valores están más concentrados alrededor de la media.

Esta lectura puede reforzarse con el **rango intercuartílico (iqr)**, que muestra la amplitud del 50 % central de los datos. En `Item_Outlet_Sales`, el valor de `iqr = 2267.049` indica que incluso la parte central de la distribución presenta una dispersión considerable. En `Item_MRP`, el `iqr = 91.817` también sugiere variabilidad, aunque menos extrema. En `Item_Weight` e `Item_Visibility`,

estos valores deben interpretarse dentro de su propia escala, evitando comparaciones absolutas entre variables distintas.

En conjunto, estas medidas muestran que `Item_Outlet_Sales` y `Item_Visibility` son las variables más variables del conjunto, mientras que `Item_Weight` presenta un comportamiento relativamente más estable. Para el EDA, esto es importante porque indica que algunas variables requieren una exploración más cuidadosa de su dispersión, posibles valores extremos y diferencias entre grupos.

Visualización de dispersión con boxplots

Los **boxplots** o diagramas de caja son una herramienta muy útil en EDA porque permiten resumir, en una sola figura, varios aspectos importantes de una variable: su centro, su dispersión y la posible presencia de valores atípicos.

En cada boxplot:

- la **línea dentro de la caja** representa la **mediana**;
- la **base inferior** de la caja corresponde al **primer cuartil** (Q_1);
- la **base superior** corresponde al **tercer cuartil** (Q_3);
- la **altura de la caja** representa el **rango intercuartílico** ($IQR = Q_3 - Q_1$), es decir, la amplitud del 50 % central de los datos;
- los **bigotes** muestran hasta dónde se extienden los valores que todavía pueden considerarse dentro del comportamiento general de la variable;
- los puntos que quedan fuera suelen interpretarse como **posibles valores atípicos**.

Esto hace que el boxplot sea especialmente valioso, porque no solo muestra dónde está el centro de la distribución, sino también qué tan concentrados o dispersos están los datos alrededor de ese centro.

Esta visualización permite notar rápidamente qué variables presentan mayor amplitud central, qué tan alejados aparecen los extremos y si existen observaciones fuera del patrón principal. En contextos de EDA, el boxplot es una transición natural entre el estudio de localización y el análisis de forma, porque revela tanto la posición de la mediana como la anchura del bloque central.

Rango frente a IQR: medidas sensibles y robustas

El rango es intuitivo, pero muy sensible a valores extremos. Si una sola observación toma un valor muy grande o muy pequeño, el rango cambia de forma drástica. El IQR, en cambio, describe la anchura del bloque central de observaciones y por ello suele ser una mejor medida cuando se desea resumir dispersión sin que la descripción dependa demasiado de extremos aislados.

Para visualizar esta idea, puede compararse el rango con el IQR en una tabla compacta.

	rango	iqr	desv_std	cv
Item_Outlet_Sales	13053.674800	2267.049000	1706.499616	0.782335
Item_Visibility	0.328391	0.067596	0.051598	0.780224
Item_MRP	235.598400	91.817200	62.275067	0.441690

	rango	iqr	desv_std	cv
Item_Weight	16.795000	8.076250	4.643456	0.361144

A partir de los resultados observados, se nota que en todas las variables el **rango es considerablemente mayor que el IQR**, lo cual indica que los valores más extremos amplían mucho la dispersión total respecto de la dispersión que presenta la zona central de los datos.

Por ejemplo, en `Item_Outlet_Sales` el rango es de **13053.67**, mientras que el IQR es de **2267.05**. Esto significa que, aunque el total de valores cubre un intervalo muy amplio, la mitad central de las observaciones se concentra en una región mucho más reducida. En otras palabras, la dispersión total de las ventas está fuertemente influida por valores extremos o muy alejados del centro.

Algo similar ocurre con `Item_Visibility`, donde el rango (**0.3284**) es casi cinco veces el IQR (**0.0676**). Esto sugiere que, aunque la mayor parte de los productos tiene niveles de visibilidad concentrados en una zona relativamente estrecha, existen algunos casos con valores bastante más alejados que expanden notablemente el rango.

En `Item_MRP`, el rango (**235.60**) también supera claramente al IQR (**91.82**), lo que indica que el precio de lista presenta variación importante, aunque la dispersión del bloque central es más moderada que la sugerida por los extremos. Por último, en `Item_Weight`, el rango (**16.80**) es aproximadamente el doble del IQR (**8.08**), lo que refleja una distribución relativamente más estable en comparación con las demás variables.

En conjunto, los resultados sugieren que `Item_Outlet_Sales` y `Item_Visibility` son variables donde la diferencia entre dispersión total y dispersión central es más marcada, mientras que `Item_Weight` presenta una estructura comparativamente más compacta. Por ello, en el EDA resulta recomendable usar el IQR junto con otras medidas robustas cuando se quiere describir la variabilidad sin que los extremos dominen la interpretación.

Comparación de dispersión entre grupos

La variabilidad también puede cambiar entre subpoblaciones. En Big Mart Sales una comparación razonable consiste en estudiar la dispersión de `Item_Outlet_Sales` según `Outlet_Type`.

Outlet_Type	n	media	mediana	desv_std
Supermarket Type3	935	3694.038558	3364.9532	2127.760054
Supermarket Type1	5577	2316.181148	1990.7420	1515.965558
Supermarket Type2	928	1995.498739	1655.1788	1375.932889
Grocery Store	1083	339.828500	256.9988	260.851582

Esta figura permite observar que no solo cambia el centro de las ventas según tipo de outlet, sino también su grado de dispersión. Algunos grupos pueden presentar ventas más consistentes, mientras otros muestran una distribución más extendida. Esta observación será clave cuando más adelante se discuta heterogeneidad.

Coeficiente de variación y comparación relativa

En muchas aplicaciones interesa comparar dispersión entre variables con escalas diferentes. La desviación estándar por sí sola no siempre permite hacerlo de manera justa. El coeficiente de variación proporciona una medida adimensional que expresa la dispersión como proporción del promedio.

	media	desv_std	cv
Item_Outlet_Sales	2181.288914	1706.499616	0.782335
Item_Visibility	0.066132	0.051598	0.780224
Item_MRP	140.992782	62.275067	0.441690
Item_Weight	12.857645	4.643456	0.361144

Si una variable exhibe un coeficiente de variación alto, ello sugiere que su dispersión es grande en relación con su propio nivel medio. En el caso de ventas o visibilidad, este tipo de medida puede ser más informativa que la desviación estándar absoluta cuando se comparan fenómenos de distinta escala.

Histograma con bandas de dispersión

Otra visualización útil consiste en representar la media junto con bandas de una desviación estándar. No debe interpretarse como intervalo de confianza ni como regla universal de cobertura, pero sí como una guía visual de la amplitud típica alrededor del promedio.

Esta visualización resulta especialmente útil para introducir una idea intuitiva: distribuciones con la misma media pueden diferir porque sus observaciones se encuentran más o menos alejadas de ese centro.

Ejercicio sugerido

Construya un segundo dashboard en Quarto que incluya:

- una tabla resumen con rango, desviación estándar, IQR y coeficiente de variación;
- boxplots para `Item_Weight`, `Item_Visibility`, `Item_MRP` e `Item_Outlet_Sales`;
- una comparación de la dispersión de `Item_Outlet_Sales` por `Outlet_Type`;
- un comentario breve donde se explique por qué el IQR es más robusto que el rango frente a valores extremos.

Parte XIV.

Medidas de forma

26. Medidas de forma

El problema: mismo centro y escala, distinta experiencia

Una cafetería registró el tiempo de espera, en minutos, de sus primeros cinco clientes durante dos mañanas:

Mañana	Tiempos de espera ordenados
A	0, 2, 3, 4, 6
B	2, 2, 2, 2, 7

En ambas mañanas, la espera media es de tres minutos:

$$\bar{x}_A = \frac{0+2+3+4+6}{5} = 3, \qquad \bar{x}_B = \frac{2+2+2+2+7}{5} = 3.$$

También comparten el mismo segundo momento central,[1] que en este capítulo calcularemos con divisor n:

$$m_{2,A} = \frac{(-3)^2 + (-1)^2 + 0^2 + 1^2 + 3^2}{5} = 4,$$
$$m_{2,B} = \frac{(-1)^2 + (-1)^2 + (-1)^2 + (-1)^2 + 4^2}{5} = 4.$$

Por tanto, ambas tienen escala $\sqrt{m_2} = 2$ minutos. Sin embargo, no cuentan la misma historia:

- en A, los tiempos se reparten de manera simétrica alrededor de 3;
- en B, cuatro clientes esperan 2 minutos y uno espera 7: hay una extensión hacia la derecha.

La localización y la variabilidad no determinan la geometría de una distribución.[2] Para distinguir estas mañanas necesitamos estudiar su **forma**: simetría, dirección de las colas, frecuencia de observaciones alejadas y número de agrupamientos o modos (Tukey 1977; Heckert et al. 2012).

Una idea que servirá como brújula

Trasladar todos los datos cinco minutos o convertir minutos a segundos cambia la localización o la escala, pero no debería cambiar la forma. En contraste, mover una sola observación hacia un extremo sí puede cambiarla.

[1] Un **momento central de orden** r es el promedio de las desviaciones respecto de la media elevadas a la potencia r. El orden 2 mide dispersión; los órdenes 3 y 4 participan en medidas de forma.

[2] Una **distribución** describe qué valores puede tomar una variable y con qué frecuencia o probabilidad aparecen. En una muestra se observa una versión finita de esa estructura.

Aprender a mirar la forma

Simetría y asimetría

Una distribución es **simétrica alrededor de** c si las desviaciones a la izquierda y a la derecha de c tienen el mismo comportamiento. En una distribución continua,[3] esto significa que su densidad satisface $f(c-t) = f(c+t)$ para todo t donde esté definida. En datos finitos, la idea se aprecia al ordenar los valores y comparar pares equidistantes del centro.

En la mañana A,

$$(0, 2, 3, 4, 6) - 3 = (-3, -1, 0, 1, 3).$$

Cada desviación negativa tiene una contraparte positiva. En la mañana B,

$$(2, 2, 2, 2, 7) - 3 = (-1, -1, -1, -1, 4),$$

de modo que una desviación positiva larga compensa cuatro desviaciones negativas cortas. Diremos que B tiene **asimetría a la derecha** o positiva. La cola indica la dirección de la asimetría, no el lado donde se acumulan más observaciones.[4]

Es común usar la posición relativa de media y mediana como pista. En B, $\bar{x} = 3 > \text{Med}(x) = 2$, porque el 7 atrae a la media hacia la derecha. No es un teorema universal: distribuciones complejas pueden violar esa regla. La forma debe inspeccionarse gráficamente y no inferirse solo a partir de dos resúmenes.

Colas y observaciones alejadas

Las **colas** describen lo que ocurre lejos de la región central. Dos distribuciones simétricas pueden compartir media y varianza y, aun así, asignar probabilidades muy diferentes a resultados extremos. Esto importa cuando se estudian tiempos de espera inusuales, pérdidas, fallas o anomalías.

En una muestra pequeña no puede observarse literalmente el comportamiento infinito de una cola. Sí pueden verse señales: máximos muy separados, cuantiles extremos,[5] sensibilidad de los momentos altos y desviaciones en un gráfico Q–Q. Hablar de "colas pesadas" a partir de cinco datos debe entenderse como una hipótesis para investigar, no como una conclusión definitiva.

[3] Una variable **continua** puede tomar valores en intervalos. Su densidad representa concentración de probabilidad por unidad; la probabilidad de un intervalo es el área bajo esa curva.

[4] Una **cola** es la región alejada del centro hacia valores pequeños o grandes. "Cola derecha larga" no significa que la mayoría de los datos estén a la derecha, sino que la extensión o probabilidad extrema es mayor en esa dirección.

[5] El cuantil de nivel p es un valor que deja aproximadamente una proporción p de observaciones por debajo. La mediana es el cuantil 0.5.

Modos y mezclas

Un **modo** es una región de alta frecuencia o densidad.[6] Una distribución con dos concentraciones visibles es bimodal. Esto puede ocurrir, por ejemplo, si se mezclan pedidos sencillos y pedidos que requieren preparación especial. La bimodalidad no equivale a asimetría: una distribución puede ser bimodal y simétrica.

La forma tiene varias dimensiones

Asimetría, colas y modalidad responden preguntas distintas. Ningún número único reconstruye toda la distribución. Las medidas complementan al histograma, al diagrama de puntos, a la ECDF y al gráfico Q–Q; no los sustituyen.

Del dibujo al cálculo: la mañana B

Trabajaremos a mano con

$$x = (2, 2, 2, 2, 7), \qquad n = 5, \qquad \bar{x} = 3.$$

La tabla organiza todas las operaciones. Las potencias impares conservan el signo de la desviación; las pares lo eliminan.

x_i	$d_i = x_i - \bar{x}$	d_i^2	d_i^3	d_i^4
2	-1	1	-1	1
2	-1	1	-1	1
2	-1	1	-1	1
2	-1	1	-1	1
7	4	16	64	256
Suma	**0**	**20**	**60**	**260**

Por qué el cubo detecta dirección

El tercer momento central es

$$m_3 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3 = \frac{60}{5} = 12.$$

Las desviaciones a la izquierda aportan números negativos y las de la derecha, positivos. Además, el cubo da mucho más peso a 4 que a 1: $4^3 = 64$. El signo positivo concuerda con la extensión hacia la derecha.

[6] En datos discretos, un modo es un valor de frecuencia máxima. En una distribución continua se habla de máximos locales de la densidad; su estimación depende del suavizado y del tamaño muestral.

El valor 12 está en minutos cúbicos y cambiaría al convertir unidades. Para obtener una medida sin unidades se divide por la escala al cubo:

$$g_1 = \frac{m_3}{m_2^{3/2}} = \frac{12}{4^{3/2}} = \frac{12}{8} = 1.5. \tag{1}$$

En la mañana A, los cubos se cancelan por pares:

$$(-3)^3 + (-1)^3 + 0^3 + 1^3 + 3^3 = 0,$$

por lo que $g_1 = 0$. Así logramos distinguir dos conjuntos que tenían la misma media y la misma escala.

Por qué la cuarta potencia detecta alejamiento

El cuarto momento central de B es

$$m_4 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^4 = \frac{260}{5} = 52.$$

Como la cuarta potencia siempre es no negativa, no mide dirección. Su efecto es amplificar las observaciones alejadas: la desviación 4 aporta $4^4 = 256$, mientras que cada desviación de magnitud 1 aporta solo 1.

La **curtosis de Pearson** estandariza con la varianza al cuadrado:

$$b_2 = \frac{m_4}{m_2^2} = \frac{52}{4^2} = 3.25. \tag{2}$$

La distribución normal tiene curtosis poblacional 3. Por eso también se usa el **exceso de curtosis de Fisher**:

$$g_2 = b_2 - 3 = 3.25 - 3 = 0.25. \tag{3}$$

Restar 3 solo cambia el punto de referencia; no cambia la información. En particular, “curtosis 3” y “exceso 0” describen la misma referencia normal.

Curtosis no significa simplemente “altura del pico”

La curtosis es el promedio de la **cuarta potencia de las desviaciones estandarizadas**. Por ello responde con fuerza a observaciones alejadas y al comportamiento de las colas. La altura central puede cambiar también, pero no existe una correspondencia única entre “pico alto” y curtosis alta. Interpretar la curtosis solo como apuntamiento puede llevar a conclusiones equivocadas.

Formalización mediante momentos

Sea X una variable aleatoria numérica[7] con media $\mu = \mathbb{E}[X]$. Cuando las esperanzas existen, se distinguen:

$$\text{momento respecto al origen:} \qquad \mu'_r = \mathbb{E}[X^r],$$

$$\text{momento central:} \qquad \mu_r = \mathbb{E}[(X-\mu)^r]. \tag{4}$$

Los primeros momentos centrales ayudan a leer la construcción:

- $\mu_1 = 0$, porque las desviaciones respecto de la media se compensan;
- $\mu_2 = \sigma^2$, la varianza poblacional;
- μ_3 conserva dirección, pero depende de la unidad al cubo;
- μ_4 pondera intensamente el alejamiento y depende de la unidad a la cuarta potencia.

Si $0 < \sigma^2 < \infty$, los momentos estandarizados tercero y cuarto son

$$\gamma_1 = \frac{\mu_3}{\sigma^3}, \qquad \beta_2 = \frac{\mu_4}{\sigma^4}, \qquad \gamma_2 = \beta_2 - 3. \tag{5}$$

γ_1 es la asimetría poblacional, β_2 la curtosis de Pearson y γ_2 el exceso de Fisher. Si la variable es constante, $\sigma = 0$ y estas razones no están definidas: no se debe reportar cero como si hubiese una forma estimable.

Propiedades que aclaran la interpretación

Si $Y = a + bX$, con $b \neq 0$, entonces

$$\mu_r(Y) = b^r \mu_r(X), \qquad \sigma_Y = |b|\sigma_X.$$

Por tanto,

$$\gamma_1(Y) = \operatorname{sign}(b)\gamma_1(X), \qquad \beta_2(Y) = \beta_2(X). \tag{6}$$

Sumar una constante o cambiar de minutos a segundos con un factor positivo no altera asimetría ni curtosis. Reflejar el eje, $b < 0$, invierte el signo de la asimetría y conserva la curtosis.

Si una distribución es simétrica y su tercer momento existe, entonces $\mu_3 = 0$. El recíproco es falso: asimetría cero no garantiza simetría. Por ejemplo, la muestra

$$(0, 0, 0, 4, 5, 5, 7)$$

[7] Una **variable aleatoria** es una regla que asigna un valor a cada resultado de un experimento. Aquí no significa que las etiquetas sean números: X asigna la etiqueta A, B o C a la canica observada.

tiene media 3 y suma de cubos centrados

$$3(-3)^3 + 1^3 + 2(2^3) + 4^3 = -81 + 1 + 16 + 64 = 0,$$

pero no es simétrica alrededor de 3.

La curtosis poblacional satisface $\beta_2 \geq 1$, siempre que la varianza sea positiva y el cuarto momento finito. Se obtiene de

$$\mathbb{E}[(X-\mu)^4] \geq \left(\mathbb{E}[(X-\mu)^2]\right)^2.$$

No hay una cota superior universal: una probabilidad pequeña colocada muy lejos del centro puede hacer crecer el cuarto momento de manera drástica.

Población, muestra y convenciones de software

Para una colección observada usamos los momentos descriptivos

$$m_r = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^r, \qquad g_1 = \frac{m_3}{m_2^{3/2}}, \qquad g_2 = \frac{m_4}{m_2^2} - 3. \tag{7}$$

Si los datos se consideran una muestra de una población, existen correcciones para reducir el sesgo del estimador.[8] Una convención frecuente es

$$G_1 = \frac{\sqrt{n(n-1)}}{n-2} g_1, \qquad n > 2, \tag{8}$$

$$G_2 = \frac{n-1}{(n-2)(n-3)}\left[(n+1)g_2 + 6\right], \qquad n > 3. \tag{9}$$

Para la mañana B, $g_1 = 1.5$ y $g_2 = 0.25$, mientras que las versiones corregidas dan $G_1 \approx 2.2361$ y $G_2 = 5$. La diferencia es grande porque $n = 5$. Paquetes distintos pueden usar definiciones distintas; el resultado debe acompañarse de la convención y el tamaño muestral (Joanes y Gill 1998).

> Las correcciones no convierten cinco observaciones en evidencia abundante. Reducen cierto sesgo bajo un modelo de muestreo, pero las medidas de orden alto siguen siendo inestables en muestras pequeñas.

[8] Un **estimador** es una regla matemática calculada a partir de los datos de una muestra para aproximar un parámetro poblacional desconocido. Su valor numérico varía de una muestra a otra y, por tanto, está sujeto a error de muestreo e incertidumbre.

Laboratorio interactivo: cambiar la forma

El laboratorio parte de la mañana B. Mueva cada observación o compare los escenarios preparados. Observe que "simétrica" y "asimétrica a la derecha" tienen la misma media y la misma desviación, mientras que "misma forma, otra escala" conserva g_1 y g_2 aunque duplica la desviación estándar.

La versión HTML incluye un laboratorio para mover cinco tiempos de espera. Los escenarios principales se resumen aquí:

Escenario	Datos	Media	$\sqrt{m_2}$	g_1	b_2	g_2
Simétrica	$0, 2, 3, 4, 6$	3	2	0	2.05	-0.95
Cola derecha	$2, 2, 2, 2, 7$	3	2	1.50	3.25	0.25
Cola izquierda	$0, 5, 5, 5, 5$	4	2	-1.50	3.25	0.25
Misma forma, otra escala	$2, 2, 2, 2, 12$	4	4	1.50	3.25	0.25
Bimodal	$1, 1, 3, 5, 5$	3	1.7889	0	1.25	-1.75

Los dos primeros escenarios tienen la misma media y escala, pero distinta forma. El segundo y el cuarto tienen distinta media y escala, pero los mismos momentos estandarizados.

Curtosis y colas: una comparación controlada

Para aislar la forma, compare tres modelos simétricos con media 0 y varianza 1:

Distribución	Exceso de curtosis	Comportamiento cualitativo de las colas
Normal estándar	0	decrecimiento rápido, proporcional a $e^{-x^2/2}$ en la densidad
Laplace estandarizada	3	decrecimiento exponencial en $\lvert x \rvert$
t de Student con 5 grados de libertad, estandarizada	6	decrecimiento polinómico

Las tres son simétricas y tienen la misma varianza. La diferencia está en cuánta probabilidad conservan lejos del centro. La t_5 genera extremos con mayor frecuencia que la normal; esto se refleja en una curtosis mayor. La comparación funciona porque estamos examinando familias completas, no porque la curtosis permita reconstruir por sí sola todas sus colas.

> Dos distribuciones pueden tener la misma curtosis y formas diferentes. Además, si $\mathbb{E}[(X-\mu)^4]$ no existe, la curtosis poblacional es infinita o no está definida, aunque un programa siempre produzca algún número finito para una muestra finita.

Gráficos que completan las medidas

Histograma, densidad y ECDF

El histograma muestra acumulaciones, huecos y posibles modos, pero cambia con el ancho y origen de los intervalos. Una estimación de densidad suaviza la forma y depende de un parámetro de suavizado. La función de distribución empírica (ECDF)[9] evita intervalos y conserva cada observación, aunque los modos son menos visibles.

Conviene observar más de una representación. Una asimetría numérica elevada puede provenir de una cola gradual, de un solo dato erróneo o de mezclar dos grupos; las acciones analíticas serían distintas.

Gráfico Q–Q

Un gráfico Q–Q enfrenta los cuantiles observados con los cuantiles de una distribución de referencia.[10] Si los datos son compatibles con una versión trasladada y escalada de esa referencia, los puntos siguen aproximadamente una recta. La pendiente refleja escala y el intercepto localización; curvaturas sistemáticas revelan diferencias de forma.

En un Q–Q normal:

- separación fuerte en ambos extremos sugiere colas distintas de la normal;
- una desviación más intensa en un solo extremo sugiere asimetría;
- escalones o grupos pueden revelar redondeo, discreción o mezclas.

La rectitud no demuestra normalidad, sobre todo con pocas observaciones. Es una comprobación visual que debe acompañarse de conocimiento del proceso y tamaño muestral.

Transformar cambia la forma y la pregunta

Para datos estrictamente positivos con asimetría derecha, una transformación logarítmica puede comprimir valores grandes:

$$y_i = \log(x_i).$$

Esto puede facilitar una visualización o estabilizar relaciones multiplicativas, pero el análisis deja de estar en la escala original. Una diferencia de una unidad en $\log x$ representa una razón multiplicativa en x, no una diferencia aditiva.

[9] La **función de distribución empírica** en t es la proporción de observaciones menores o iguales que t. Es una función escalonada que no requiere elegir intervalos ni parámetros de suavizado.

[10] Un **cuantil–cuantil (Q–Q)** empareja cuantiles ordenados de los datos con cuantiles de una distribución de referencia. No compara frecuencias de barras, sino posiciones equivalentes dentro de ambas distribuciones.

Las transformaciones monótonas[11] preservan el orden, pero no preservan distancias, media, varianza, asimetría ni curtosis. Por ello no se debe "normalizar" una variable automáticamente solo para reducir g_1. Antes hay que preguntar:

1. ¿La nueva escala tiene sentido para el fenómeno?
2. ¿Los ceros y valores negativos están tratados explícitamente?
3. ¿La transformación se aprende y aplica dentro del mismo *pipeline*?
4. ¿Las predicciones pueden regresar a la escala original sin introducir una interpretación engañosa?

Implementación reproducible

La función siguiente hace explícita la convención de momentos con divisor n. También calcula las correcciones de las Ecs. 8 y 9 para mostrar por qué dos bibliotecas pueden entregar cifras distintas.

	n	media	mediana	sqrt(m2)	g1_momentos	G1_corregida	b2_Pearson	g2
Simétrica	5.0	3.0	3.0	2.0000	0.0	0.0000	2.05	-0.
Cola derecha	5.0	3.0	2.0	2.0000	1.5	2.2361	3.25	0.2
Cola izquierda	5.0	4.0	5.0	2.0000	-1.5	-2.2361	3.25	0.2
Misma forma, otra escala	5.0	4.0	2.0	4.0000	1.5	2.2361	3.25	0.2
Bimodal	5.0	3.0	3.0	1.7889	0.0	0.0000	1.25	-1.

Las formas completas detrás de los momentos

Las funciones de distribución empírica siguientes utilizan los mismos cinco escenarios de la tabla. Los escalones permiten comprobar dónde se acumulan o se separan sus observaciones.

Del cálculo al programa

El algoritmo necesita primero la media y después las potencias de las desviaciones. La estandarización divide m_3 por $m_2^{3/2}$ y m_4 por m_2^2. En datos reales se debe definir antes cómo tratar valores faltantes, ponderaciones y grupos; eliminarlos silenciosamente cambia la distribución analizada.

Qué implica la forma en minería de datos

Situación	Por qué importa la forma	Acción razonable
Imputación	La media se desplaza hacia colas largas.	Comparar media, mediana y distribución por grupos antes de imputar.

[11] Una relación es **monótona creciente** si Y nunca disminuye cuando X aumenta; es monótona decreciente si nunca aumenta. Puede ser curva y seguir conservando el orden de manera perfecta.

Situación	Por qué importa la forma	Acción razonable
Detección de anomalías	Una regla basada en desviaciones estándar confunde colas pesadas con errores.	Usar cuantiles o métodos robustos y validar con el contexto.
Modelos lineales	La normalidad suele concernir a errores o residuos, no a cada predictor.	Revisar residuos y especificación del modelo; no transformar por rutina.
k-means y distancias	Extremos y escalas dominan distancias euclidianas.	Estandarizar y considerar escalamiento robusto; la estandarización no elimina asimetría.
Árboles	Las divisiones por umbrales toleran transformaciones monótonas del predictor.	Aun así, revisar colas, soporte escaso y posibles errores de medición.
Mezclas	Dos modos pueden representar subpoblaciones con mecanismos distintos.	Estratificar, añadir variables explicativas o modelar la mezcla.

Estandarizar mediante $z_i = (x_i - \bar{x})/s$ centra y cambia la escala, pero, por la Ec. 6, no vuelve simétrica una variable. **Estandarizar** y **normalizar en el sentido de hacer normal una distribución** no son la misma operación.

Guía de lectura

Pregunta	Herramienta principal	Precaución
¿Hacia qué lado se extiende la distribución?	Histograma/ECDF y g_1	$g_1 = 0$ no demuestra simetría.
¿Hay más observaciones extremas que en una referencia?	Q–Q, cuantiles extremos y curtosis	La curtosis no describe por sí sola toda la cola ni el pico.
¿Hay uno o varios grupos?	Histograma, densidad y comparación por grupos	Asimetría y curtosis no determinan modalidad.
¿Conviene transformar?	Gráficos antes/después y sentido sustantivo de la escala	La transformación cambia la pregunta y debe formar parte del *pipeline*.
¿Puedo comparar software?	Definición, n, tratamiento de faltantes	Distinguir Pearson/Fisher y momento/corrección muestral.

Puente al caso BigMart

El capítulo práctico Medidas de forma en Big Mart Sales traslada esta secuencia a variables comerciales:

1. observar primero la distribución y sus posibles subgrupos;
2. comparar centro y escala sin confundirlos con forma;
3. calcular asimetría y curtosis documentando la convención;
4. inspeccionar colas con gráficos y cuantiles;
5. evaluar transformaciones por su significado, no solo por reducir una cifra.

Los datos cambian, pero la pregunta permanece: **¿qué estructura oculta una media y una desviación estándar?**

Ejercicios formativos

1. **Cálculo completo.** Para $x = (1, 1, 1, 3, 4)$, calcule $\bar{x}$, m_2, m_3, m_4, g_1, b_2 y g_2. Interprete los signos antes de usar software y luego verifique con `medidas_forma`.
2. **Reflexión.** Construya la variable $y = 10 - x$ a partir del ejercicio anterior. Prediga qué ocurrirá con media, desviación, g_1 y g_2, y compruébelo mediante la Ec. 6.
3. **Cero no significa simetría.** Verifique que $(0, 0, 0, 4, 5, 5, 7)$ tiene $g_1 = 0$. Dibuje sus puntos y explique por qué no es simétrica.
4. **Mismo centro y escala.** Compruebe manualmente que las mañanas A y B comparten media y m_2. Proponga una tercera muestra de cinco valores con media 3 y $m_2 = 4$, pero con otra forma.
5. **Curtosis no es pico.** Simule 10,000 observaciones de una normal, una Laplace con escala $1/\sqrt{2}$ y una t_5 multiplicada por $\sqrt{3/5}$. Compare varianza, exceso de curtosis, histogramas y Q–Q.
6. **Modalidad.** Construya una muestra simétrica y bimodal. Muestre que su asimetría puede ser cero y explique qué información no captura g_1.
7. **Decisión de transformación.** Elija una variable positiva con asimetría derecha. Compare la escala original y $\log(x)$. Describa qué cambia en la interpretación de una diferencia de una unidad y justifique cuál escala usaría en un modelo.

27. Medidas de forma en Big Mart Sales

Este caso aplica los conceptos presentados en Medidas de forma. Allí se distinguen asimetría, curtosis, multimodalidad y gráficos Q–Q.

Introducción

En una etapa inicial del análisis exploratorio de datos se estudian las **medidas de localización** y las **medidas de variabilidad** con el fin de identificar, respectivamente, el comportamiento central de una variable y el grado en que sus observaciones se dispersan alrededor de un valor representativo. Sin embargo, una descripción estadística rigurosa no se agota en estas dos dimensiones. Dos distribuciones pueden compartir una media y una desviación estándar semejantes, y aun así diferir de manera importante en su estructura global. Por ello es necesario incorporar el estudio de la **forma** de la distribución.

Las medidas de forma permiten caracterizar aspectos tales como la **asimetría** y el grado de **concentración de masa alrededor del centro o en las colas**, usualmente descrito mediante la **curtosis**. En términos aplicados, estas medidas ayudan a responder preguntas como las siguientes: ¿la mayor parte de los productos tiene ventas moderadas con unos cuantos casos excepcionalmente altos?, ¿las observaciones se reparten de forma aproximadamente simétrica?, ¿aparecen colas pesadas o valores extremos frecuentes?, ¿una transformación puede facilitar la interpretación visual y estadística del fenómeno? Además, estas medidas orientan decisiones posteriores del EDA, como la revisión de valores atípicos, la conveniencia de usar medidas robustas o la necesidad de transformar variables para facilitar su análisis.

En este capítulo se estudian las **medidas de forma** en el dataset de BigMart, utilizando como variable principal `Item_Outlet_Sales`. No obstante, varias de las herramientas presentadas pueden aplicarse también a otras variables numéricas del conjunto de datos, como `Item_MRP`, `Item_Visibility` y `Item_Weight`.

Definición operativa. La forma describe asimetría, colas y estructura modal después de considerar el centro y la escala de la distribución (Joanes y Gill 1998).

Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy, SciPy y Plotly.

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614

Antes de proceder conviene verificar el tipo de las variables y concentrarnos en aquellas que sean cuantitativas.

Fundamentos teóricos

Asimetría

La asimetría describe el grado en que una distribución se desvía de la simetría respecto de su centro. Intuitivamente:

- una distribución **simétrica** presenta masas comparables a la izquierda y derecha del centro;
- una distribución con **asimetría positiva** presenta una cola más larga hacia valores altos;
- una distribución con **asimetría negativa** presenta una cola más larga hacia valores bajos.

Una medida común de asimetría es el coeficiente:

$$g_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3}{\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2\right)^{3/2}}.$$

Cuando $g_1 > 0$, la cola derecha tiende a ser más extensa; cuando $g_1 < 0$, domina la cola izquierda; y cuando $g_1 \approx 0$, la distribución puede considerarse aproximadamente simétrica, aunque esta interpretación siempre debe acompañarse de una inspección gráfica.

Curtosis

La curtosis resume el peso relativo de las colas y la concentración de observaciones alrededor del centro. Una expresión frecuente es:

$$g_2 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^4}{\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2\right)^{2}} - 3.$$

La resta de 3 permite comparar la distribución con la normal estándar:

- si $g_2 \approx 0$, la curtosis es similar a la de una normal;
- si $g_2 > 0$, la distribución exhibe colas relativamente pesadas y mayor propensión a valores extremos;
- si $g_2 < 0$, las colas son relativamente ligeras y la distribución resulta más plana.

La curtosis no debe leerse únicamente como "pico" central: en la práctica, suele ser más útil interpretarla en términos de **presencia de colas pesadas y extremos**.

Exploración gráfica inicial de la variable de ventas

Comenzamos con la variable de mayor interés económico: `Item_Outlet_Sales`.

A partir del histograma y el boxplot, se pueden responder varias preguntas sobre la forma de la distribución.

En primer lugar, **no se observa simetría visual**. La distribución presenta una clara **asimetría positiva (sesgo a la derecha)**: la mayor parte de las observaciones se concentra en valores relativamente bajos, mientras que una cola se extiende hacia valores de ventas más altos.

En segundo lugar, **sí aparecen colas largas**, especialmente hacia la derecha. Esto indica que, aunque la mayoría de los productos presenta niveles de ventas moderados, existen algunos casos con ventas considerablemente mayores que el resto.

En tercer lugar, **se observan posibles valores extremos**, lo cual también se confirma en el boxplot superior, donde varios puntos aparecen fuera de los bigotes. Estos puntos corresponden a observaciones con ventas muy superiores al comportamiento típico.

Finalmente, la gráfica muestra que **la masa principal de los datos se concentra en una región relativamente acotada**, aproximadamente en el rango bajo y medio de la distribución, mientras que un número reducido de observaciones alcanza valores de ventas mucho más altos. Esto sugiere que el comportamiento típico de las ventas está dominado por valores moderados, pero con algunos productos o combinaciones producto–tienda que generan ventas excepcionalmente elevadas.

Para reforzar la interpretación puede añadirse una curva de densidad aproximada.

La superposición del histograma con la curva de densidad permite observar con mayor claridad la forma general de la distribución. Mientras el histograma muestra la frecuencia relativa por intervalos, la curva KDE suaviza esa información y ayuda a identificar la concentración principal de los datos, la presencia de asimetría y la posible existencia de colas largas.

Cálculo de asimetría y curtosis

	medida	valor
0	asimetría	1.177531
1	curtosis_exceso	1.615877

Una interpretación orientativa puede apoyarse en criterios simples:

- $|\text{asimetría}| < 0.5$: asimetría baja;
- $0.5 \leq |\text{asimetría}| < 1$: asimetría moderada;
- $|\text{asimetría}| \geq 1$: asimetría marcada.

Estos umbrales no son absolutos, pero ayudan a construir una primera lectura del fenómeno. En datasets comerciales es frecuente observar asimetría positiva: muchos productos exhiben ventas moderadas, mientras un grupo reducido alcanza ventas muy elevadas.

Comparación de forma en varias variables numéricas

La forma no tiene por qué ser igual en todas las variables del conjunto de datos. Conviene comparar la asimetría y curtosis de todas las columnas numéricas disponibles.

	variable	asimetria	curtosis_exceso
4	Item_Outlet_Sales	1.177531	1.615877
1	Item_Visibility	1.167091	1.679445
3	Outlet_Establishment_Year	-0.396641	-1.205694
2	Item_MRP	0.127202	-0.889769
0	Item_Weight	0.082426	-1.227766

La comparación de **asimetría** y **curtosis en exceso** muestra que no todas las variables numéricas presentan la misma estructura de forma. En particular, `Item_Outlet_Sales` exhibe una asimetría positiva de **1.177531** y una curtosis en exceso de **1.615877**, mientras que `Item_Visibility` presenta una asimetría de **1.167091** y una curtosis en exceso de **1.679445**. Estos valores sugieren distribuciones con **colas derechas largas** y una posible presencia de **valores extremos**, por lo que en ambas variables conviene reforzar el análisis con medidas robustas, boxplots y, si resulta útil, transformaciones como la logarítmica.

En cambio, `Item_MRP` muestra una asimetría mucho más baja (**0.127202**) y una curtosis en exceso negativa (**-0.889769**), mientras que `Item_Weight` presenta una asimetría de **0.082426** y una curtosis en exceso de **-1.227766**. Esto sugiere que ambas variables tienen una forma más cercana a la simetría y una distribución relativamente menos influida por colas pesadas.

Por su parte, `Outlet_Establishment_Year` registra una asimetría negativa de **-0.396641** y una curtosis en exceso de **-1.205694**. Aunque su forma no parece especialmente extrema, esta variable requiere una interpretación adicional debido a su naturaleza temporal, ya que su análisis no debe basarse únicamente en criterios de distribución continua.

En conjunto, estos resultados indican que el EDA debe adaptarse a la forma específica de cada variable y no aplicar una misma lectura descriptiva a todas por igual.

Transformaciones y su efecto sobre la forma

Cuando una variable presenta fuerte asimetría positiva, una transformación logarítmica puede facilitar tanto la visualización como el modelado posterior. En el caso de las ventas, una opción práctica es usar:

$$y^* = \log(1 + y),$$

que evita problemas cuando existen observaciones cercanas a cero.

	variable	asimetria	curtosis_exceso
0	Item_Outlet_Sales	1.177531	1.615877
1	log_Item_Outlet_Sales	-0.882266	0.532920

La comparación entre la variable original y su versión transformada permite mostrar un punto importante del EDA: las medidas de forma no son meramente descriptivas, también orientan decisiones analíticas posteriores. Si la asimetría disminuye sustancialmente bajo la transformación, esto sugiere una representación más equilibrada del fenómeno.

En particular, una **transformación logarítmica** puede ser útil cuando la distribución presenta una **asimetría positiva marcada**, es decir, cuando existen valores muy altos que alargan la cola derecha de la distribución. Al aplicar el logaritmo, los valores grandes se comprimen relativamente más que los valores pequeños, lo que tiende a reducir la asimetría y a concentrar mejor los datos alrededor del centro.

Desde el punto de vista del EDA, esta transformación ayuda a:

- **visualizar mejor la estructura de la distribución**, evitando que unos pocos valores muy grandes dominen el gráfico;
- **comparar observaciones en términos relativos**, ya que las diferencias en escala logarítmica representan cambios proporcionales;
- **reducir la influencia de valores extremos**, facilitando una lectura más clara del comportamiento típico de los datos.

Además, esta transformación puede ser útil para **análisis posteriores**, especialmente cuando se emplean métodos estadísticos o modelos que funcionan mejor con distribuciones más simétricas o con variabilidad más estable. En contextos de modelado, por ejemplo, trabajar con una variable transformada puede mejorar la interpretación de los resultados, estabilizar la varianza y facilitar el ajuste de modelos lineales o regresiones.

En síntesis, el uso del logaritmo en el EDA no solo mejora la interpretación visual de distribuciones muy asimétricas, sino que también prepara los datos para etapas posteriores del análisis, donde una representación más equilibrada de la variable puede resultar analíticamente más conveniente.

Gráfico cuantil-cuantil como apoyo visual

Otra herramienta útil para estudiar la forma es el gráfico Q-Q frente a la distribución normal. No debe interpretarse como una prueba definitiva, pero sí como un recurso visual para detectar desviaciones sistemáticas respecto a simetría y colas gaussianas.

Si la versión transformada se alinea mejor con la recta de referencia que la variable original, ello aporta evidencia visual de una mejora en la regularidad de la forma, aunque no implica normalidad perfecta.

Mini-dashboard de forma

La siguiente sección integra un dashboard simple para resumir la forma de la variable de ventas y su transformación.

Parte XV.

Medidas de heterogeneidad

28. Medidas de heterogeneidad

El problema: ¿qué significa que unos datos sean heterogéneos?

Decir que unos datos son **heterogéneos** significa que no están concentrados en una sola posibilidad. Sin embargo, la pregunta concreta depende del tipo de variable:

Tipo de datos	Pregunta de heterogeneidad	Poca heterogeneidad	Mucha heterogeneidad
Categóricos	¿Cómo se reparte la frecuencia entre las clases?	Casi todas las observaciones pertenecen a una sola clase.	Las observaciones se reparten de forma equilibrada entre varias clases.
Numéricos agrupados	¿Cuánto cambia la distribución de una respuesta entre grupos?	Los grupos tienen centros y formas semejantes.	La pertenencia al grupo ayuda a anticipar el valor numérico.

Esta distinción evita un error frecuente: una variable categórica no tiene distancias numéricas naturales entre sus etiquetas, mientras que una variable numérica sí permite medir desviaciones respecto a una media. Por eso se necesitan construcciones diferentes.

En la primera mitad del capítulo seguiremos un único ejemplo: una bolsa contiene seis canicas de tres clases, cada clase identificada por un color.

Clase	Color	Número de canicas	Proporción
A	azul	3	$p_A = 3/6 = 1/2$
B	naranja	1	$p_B = 1/6$
C	verde	2	$p_C = 2/6 = 1/3$
Total		**6**	**1**

La variable aleatoria[1] X registra la clase de una canica extraída. Sus valores posibles son $\{A, B, C\}$ y su distribución es

$$p = (p_A, p_B, p_C) = \left(\frac{1}{2}, \frac{1}{6}, \frac{1}{3}\right).$$

[1] Una **variable aleatoria** es una regla que asigna un valor a cada resultado de un experimento. Aquí no significa que las etiquetas sean números: X asigna la etiqueta A, B o C a la canica observada.

Las clases son mutuamente excluyentes y exhaustivas:[2] cada extracción produce exactamente una de las tres etiquetas. La heterogeneidad categórica dependerá de estas proporciones, no del nombre ni del color asignado a cada clase.

Dos casos que servirán como brújula

- Si las seis canicas fueran de clase A, conocer la clase de una extracción sería trivial: la heterogeneidad debería valer **cero**.
- Si hubiera dos canicas de cada clase, ninguna clase dominaría: con tres clases, la heterogeneidad debería ser **máxima**.

Toda medida que usemos deberá respetar estos dos extremos.

Heterogeneidad categórica I: Gini–Simpson

De una extracción a dos extracciones

Al extraer una canica de la bolsa completa, la probabilidad[3] de obtener cada clase es su fracción dentro del total:

$$P(X = A) = \frac{3}{6}, \qquad P(X = B) = \frac{1}{6}, \qquad P(X = C) = \frac{2}{6}.$$

Una sola extracción informa cuál clase apareció, pero todavía no produce una noción directa de diversidad. Para obtenerla, planteemos otro experimento:

> Se extrae una canica, se registra su clase, se devuelve a la bolsa, se mezcla y se extrae una segunda canica. ¿Cuál es la probabilidad de que las dos clases sean diferentes?

Devolver la primera canica significa muestrear **con reemplazo**.[4] Gracias a ello, la composición de la bolsa es la misma antes de ambas extracciones. Supondremos además que las extracciones son **independientes**:[5] saber qué ocurrió en la primera no cambia las probabilidades de la segunda.

La independencia permite multiplicar probabilidades. Por ejemplo,

$$P(X_1 = A, X_2 = A) = P(X_1 = A)P(X_2 = A) = \frac{3}{6}\frac{3}{6} = \frac{9}{36}.$$

[2] **Mutuamente excluyentes** significa que una canica no puede pertenecer simultáneamente a dos clases. **Exhaustivas** significa que las tres clases cubren todos los resultados posibles.

[3] En este ejemplo equiprobable a nivel de canica, la probabilidad de una clase es el número de canicas de esa clase dividido entre el total. En una interpretación frecuentista, es también la proporción a la que tendería la frecuencia de esa clase al repetir muchas veces el experimento bajo las mismas condiciones.

[4] Muestrear **con reemplazo** consiste en devolver cada unidad antes de la siguiente selección. Así, las probabilidades permanecen constantes. Sin reemplazo, después de extraer una canica quedarían cinco y la segunda probabilidad dependería de la primera extracción.

[5] Dos variables son **independientes** si conocer una no altera en absoluto la distribución probabilística de la otra: $P(X \le x, Y \le y) = P(X \le x)P(Y \le y)$. Es una propiedad sobre toda la distribución conjunta, mucho más fuerte que tener covarianza o correlación cero.

Del mismo modo,

$$P(B,B) = \left(\frac{1}{6}\right)^2 = \frac{1}{36}, \qquad P(C,C) = \left(\frac{2}{6}\right)^2 = \frac{4}{36}.$$

La tabla contiene los $6 \times 6 = 36$ pares ordenados de canicas que pueden obtenerse. Las celdas diagonales corresponden a coincidencias de clase.

Primera \ segunda	A	B	C	Total de la fila
A	9/36	3/36	6/36	18/36
B	3/36	1/36	2/36	6/36
C	6/36	2/36	4/36	12/36
Total de la columna	18/36	6/36	12/36	36/36

Por tanto, la probabilidad de obtener **la misma clase** es

$$P(X_1 = X_2) = \frac{9+1+4}{36} = \frac{14}{36} = \frac{7}{18}.$$

Los eventos "misma clase" y "clases diferentes" son complementarios,[6] de modo que

$$P(X_1 \neq X_2) = 1 - P(X_1 = X_2) = 1 - \frac{7}{18} = \frac{11}{18} \approx 0.6111.$$

Ya tenemos una medida interpretable: en aproximadamente el 61.1 % de los pares extraídos de manera independiente y con reemplazo, las clases serán diferentes.

Definición formal

Sea $p = (p_1, \dots, p_K)$ una distribución sobre K categorías. El índice de **Gini–Simpson** (Simpson 1949) es

$$D(p) = 1 - \sum_{k=1}^{K} p_k^2. \tag{1}$$

La fórmula se obtiene directamente del experimento anterior:

$$\underbrace{\sum_{k=1}^{K} p_k^2}_{P(X_1 = X_2)} + \underbrace{D(p)}_{P(X_1 \neq X_2)} = 1.$$

También puede escribirse como

[6] Si E es un evento, su complemento E^c reúne todos los resultados en los que E no ocurre. Como son mutuamente excluyentes y cubren todo el espacio, $P(E^c) = 1 - P(E)$.

$$D(p) = \sum_{j \neq k} p_j p_k,$$

que suma todos los pares ordenados de clases distintas. Esta es la razón probabilística —no solo algebraica— del cuadrado en p_k^2.

El cuadrado de probabilidades

Imagine un cuadrado de área uno. El eje horizontal representa la segunda extracción y el vertical la primera. Al dividir ambos ejes en segmentos de longitudes p_A, p_B y p_C, aparecen nueve rectángulos:

- los tres bloques diagonales tienen áreas p_A^2, p_B^2 y p_C^2 y representan coincidencias;
- los seis bloques restantes tienen áreas $p_j p_k$ con $j \neq k$ y representan diferencias.

El laboratorio interactivo de la versión HTML permite cambiar los conteos de A, B y C. En la distribución inicial $(3, 1, 2)$, el área de coincidencia es $7/18$ y el área complementaria es $D = 11/18$.

Cotas e interpretación

Si una sola clase concentra toda la probabilidad, entonces $\sum p_k^2 = 1$ y $D = 0$. Por la desigualdad de Cauchy–Schwarz, $\sum_k p_k^2 \geq 1/K$, con igualdad cuando todas las probabilidades son iguales. Por tanto, si las K clases son equiprobables, $p_k = 1/K$, se alcanza

$$D_{\text{máx}} = 1 - K\left(\frac{1}{K}\right)^2 = 1 - \frac{1}{K}. \tag{2}$$

Así, con tres clases $D \in [0, 2/3]$. El valor 0.6111 de nuestras canicas está cerca del máximo 0.6667, pero no debe interpretarse como "61.11 % del máximo". Para esa lectura se normaliza:

$$D_{\text{norm}} = \frac{D}{1 - 1/K}.$$

En el ejemplo, $D_{\text{norm}} = (11/18)/(2/3) = 11/12 \approx 0.9167$.

No confundir dos índices llamados Gini

La diversidad de Gini–Simpson mide la probabilidad de desacuerdo entre categorías. El **coeficiente de Gini** asociado con la curva de Lorenz mide desigualdad en una variable numérica, como ingreso o riqueza. Comparten parte del nombre, pero responden preguntas diferentes.

Heterogeneidad categórica II: de la sorpresa a la entropía

Gini–Simpson construye diversidad comparando dos extracciones. Shannon toma otro camino: mide cuánta información aporta observar una sola extracción.

La sorpresa de un resultado

Antes de extraer una canica, compare dos resultados:

- obtener A no sería muy sorprendente, porque ocurre con probabilidad 1/2;
- obtener B sería más sorprendente, porque ocurre con probabilidad 1/6.

La **información propia** o sorpresa de un evento con probabilidad p se define como

$$I_b(p) = -\log_b(p), \qquad b > 1. \tag{3}$$

Esta elección cumple propiedades deseables: un evento seguro tiene sorpresa cero, los eventos menos probables producen mayor sorpresa y la sorpresa de dos eventos independientes se suma.[7]

En bits, es decir, usando base 2,

$$I_2(A) = 1, \qquad I_2(B) = \log_2(6) \approx 2.585, \qquad I_2(C) = \log_2(3) \approx 1.585.$$

De la sorpresa individual a la sorpresa promedio

No basta con promediar esos tres números dando el mismo peso a cada clase: A aparece más veces que B. La sorpresa media debe ponderar cada resultado por la probabilidad con que ocurre. Ese valor esperado[8] es la **entropía de Shannon** (Shannon 1948):

$$H_b(p) = \mathbb{E}[I_b(X)] = -\sum_{k=1}^{K} p_k \log_b(p_k). \tag{4}$$

Para las seis canicas y base 2:

Clase	p_k	Sorpresa $-\log_2 p_k$	Aporte promedio $-p_k \log_2 p_k$
A	0.5000	1.0000	0.5000
B	0.1667	2.5850	0.4308
C	0.3333	1.5850	0.5283
Total	1		$H_2 = 1.4591$ **bits**

Observe la diferencia entre **sorpresa** y **aporte a la entropía**. B es el resultado más sorprendente, pero aparece poco; C aporta más al promedio porque combina sorpresa considerable con mayor frecuencia.

Cuando $p_k = 0$, la expresión contiene formalmente $0 \log 0$. Se adopta la extensión continua

[7] Para eventos independientes con probabilidades p y q, $I_b(pq) = -\log_b(pq) = -\log_b p - \log_b q = I_b(p) + I_b(q)$.

[8] La **esperanza** $\mathbb{E}[\cdot]$ es el promedio de una cantidad sobre todos los casos posibles, ponderado por su probabilidad. El riesgo es, entonces, la proporción de errores que cometería el clasificador sobre la población completa, de la cual nuestros datos son solo una muestra.

$$0 \log 0 := \lim_{p \to 0^+} p \log p = 0,$$

porque una clase imposible nunca llega a aportar sorpresa al promedio.

¿Dónde está la entropía máxima?

Con K fijo, la entropía satisface

$$0 \leq H_b(p) \leq \log_b K. \tag{5}$$

El mínimo ocurre cuando una sola clase tiene probabilidad uno. El máximo ocurre en la distribución uniforme $p_k = 1/K$. Puede verificarse con multiplicadores de Lagrange: al maximizar $-\sum p_k \ln p_k$ sujeto a $\sum p_k = 1$, las condiciones de primer orden exigen que todos los p_k sean iguales; la restricción obliga a que valgan $1/K$. Como la matriz hessiana tiene diagonal $-1/p_k < 0$, la entropía es estrictamente cóncava en el interior del simplex y ese punto crítico es el máximo global único.

Para comparar variables con distinto número de clases conviene usar

$$H_{\text{norm}} = \frac{H_b(p)}{\log_b K} \in [0, 1]. \tag{6}$$

En nuestro ejemplo,

$$H_{\text{norm}} = \frac{1.4591}{\log_2 3} = 0.9206.$$

Dos máximos que no deben confundirse

La contribución de **una sola clase**, $f(p) = -p \ln p$, alcanza su máximo en $p = 1/e \approx 0.3679$. En cambio, la entropía **de toda la distribución**, sujeta a que las probabilidades sumen uno, alcanza su máximo en $p_k = 1/K$. Para tres clases, el máximo global está en $(1/3, 1/3, 1/3)$, no en tres probabilidades iguales a $1/e$, pues estas últimas ni siquiera sumarían uno.

En efecto,

$$f'(p) = -\ln p - 1 = 0 \iff p = 1/e, \qquad f''(p) = -1/p < 0.$$

Cambiar la base cambia la escala, no la conclusión

Por la fórmula de cambio de base,

$$H_b(p) = \frac{H_e(p)}{\ln b}.$$

- base 2: bits;
- base e: nats;
- base 10: hartleys.

La base multiplica todas las entropías por una constante positiva. Por tanto, no cambia qué distribución es más heterogénea. Tampoco cambia H_{norm}, porque numerador y denominador se escalan por el mismo factor.

En la versión HTML, un diagrama ternario permite desplazar la distribución (p_A, p_B, p_C) por todo el simplex. La entropía normalizada aumenta al acercarse al centro $(1/3, 1/3, 1/3)$ y disminuye al aproximarse a cualquiera de los vértices.

Comparar Gini–Simpson y Shannon

Los dos índices miden heterogeneidad, pero observan la distribución desde experimentos distintos:

Índice	Experimento mental	Fórmula	Mayor sensibilidad relativa
Gini–Simpson	¿Dos extracciones tendrán clases distintas?	$1 - \sum p_k^2$	Clases dominantes; las probabilidades se elevan al cuadrado.
Shannon	¿Cuánta sorpresa produce, en promedio, una extracción?	$-\sum p_k \log p_k$	Clases raras; una probabilidad pequeña conserva aporte mediante $-p \log p$.

No conviene comparar directamente D y H, porque tienen escalas y máximos diferentes. Con K fijo, compare D_{norm} con H_{norm}:

Distribución (p_A, p_B, p_C)	D	D_{norm}	H (nats)	H_{norm}
$(1, 0, 0)$	0.0000	0.0000	0.0000	0.0000
$(0.90, 0.05, 0.05)$	0.1850	0.2775	0.3944	0.3590
$(0.60, 0.30, 0.10)$	0.5400	0.8100	0.8979	0.8173
$(0.50, 0.1667, 0.3333)$	0.6111	0.9167	1.0114	0.9206
$(1/3, 1/3, 1/3)$	0.6667	1.0000	1.0986	1.0000

Ambos ordenan estos ejemplos de la misma manera. Sin embargo, Shannon asigna relativamente más diversidad al escenario $(0.90, 0.05, 0.05)$ porque reconoce la información aportada por las dos clases raras.

Regla de elección

- Use **Gini–Simpson** cuando la interpretación por pares o la probabilidad de desacuerdo sea natural.
- Use **Shannon** cuando la incertidumbre, codificación o sorpresa sea central.
- Si necesita comunicar "cuántas categorías igualmente frecuentes equivalen a esta distribución", transforme ambos a **números efectivos de Hill**.

Una escala común: números de Hill

Los índices anteriores no están expresados en "número de categorías". La familia de Hill (Hill 1973; Jost 2006) los convierte a esa escala:

$$ {}^{q}D = \left(\sum_{k=1}^{K} p_k^q \right)^{1/(1-q)}, \qquad q \geq 0,\ q \neq 1. \tag{7} $$

El parámetro q controla cuánto pesan las clases raras:

1. $q = 0$**, riqueza observada:** ${}^{0}D = K$; solo cuenta las clases presentes.
2. $q \to 1$**, diversidad de Shannon:** ${}^{1}D = e^{H_e}$.
3. $q = 2$**, diversidad de Simpson:** ${}^{2}D = 1/\sum p_k^2 = 1/(1 - D)$.

Para las canicas,

$$ {}^{0}D = 3, \qquad {}^{1}D = e^{1.0114} = 2.750, \qquad {}^{2}D = \frac{1}{7/18} = \frac{18}{7} = 2.571. $$

La interpretación de ${}^{1}D = 2.750$ es concreta: la distribución $(3, 1, 2)$ tiene la misma entropía que una distribución ideal con 2.750 categorías equiprobables. En general,

$$ {}^{0}D \geq {}^{1}D \geq {}^{2}D, \tag{8} $$

y los tres valores coinciden con K únicamente bajo uniformidad perfecta.

i Para repasar: un video de 10 minutos

Si prefiere una explicación visual antes de seguir, (Biostatsquid 2026) repasa Shannon, Gini-Simpson y los números de Hill vistos aquí (además de Chao1, un estimador de riqueza de especies que no se cubre en este capítulo).

Heterogeneidad de una variable numérica entre grupos

Ahora cambia la pregunta. Sea Y una respuesta numérica y sea G una variable categórica que identifica grupos. Ya no preguntamos si dos etiquetas son distintas, sino:

> ¿Qué parte de la variación numérica de Y se asocia con diferencias entre los grupos?

Usaremos seis observaciones:

$$A = (4, 5), \qquad B = (8, 9), \qquad C = (5, 5).$$

La media global y las medias de grupo son

$$\bar{y} = 6, \qquad \bar{y}_A = 4.5, \qquad \bar{y}_B = 8.5, \qquad \bar{y}_C = 5.$$

La identidad que separa dos fuentes de variación

Para una observación y_{gi} del grupo g,

$$y_{gi} - \bar{y} = \underbrace{(y_{gi} - \bar{y}_g)}_{\text{desviación dentro del grupo}} + \underbrace{(\bar{y}_g - \bar{y})}_{\text{desviación entre grupos}}.$$

Al elevar al cuadrado y sumar todas las observaciones, el término cruzado desaparece porque $\sum_i (y_{gi} - \bar{y}_g) = 0$ dentro de cada grupo. Se obtiene la descomposición exacta de ANOVA:

$$\underbrace{\sum_g \sum_i (y_{gi} - \bar{y})^2}_{SS_T} = \underbrace{\sum_g n_g (\bar{y}_g - \bar{y})^2}_{SS_B} + \underbrace{\sum_g \sum_i (y_{gi} - \bar{y}_g)^2}_{SS_W}. \tag{9}$$

- SS_T: variación total respecto a la media global;
- SS_B: variación **entre** medias grupales (*between*);
- SS_W: variación **dentro** de los grupos (*within*).

Cálculo completo

La variación total es

$$SS_T = (4-6)^2 + (5-6)^2 + (8-6)^2 + (9-6)^2 + (5-6)^2 + (5-6)^2 = 20.$$

La parte entre grupos es

$$\begin{aligned} SS_B &= 2(4.5-6)^2 + 2(8.5-6)^2 + 2(5-6)^2 \\ &= 4.5 + 12.5 + 2 = 19. \end{aligned}$$

La parte interna es

$$\begin{aligned} SS_W &= [(4-4.5)^2 + (5-4.5)^2] \\ &\quad + [(8-8.5)^2 + (9-8.5)^2] + [(5-5)^2 + (5-5)^2] \\ &= 0.5 + 0.5 + 0 = 1. \end{aligned}$$

La identidad se comprueba: $20 = 19 + 1$.

Eta cuadrada y omega cuadrada

La proporción descriptiva de variación explicada por las diferencias de medias es

$$\eta^2 = \frac{SS_B}{SS_T} \in [0, 1]. \tag{10}$$

En el ejemplo, $\eta^2 = 19/20 = 0.95$: conocer el grupo explica el 95 % de la variación cuadrática observada. Esto no significa que el grupo "cause" el resultado; es una descripción de asociación en estos datos.

En muestras pequeñas, η^2 tiende a sobreestimar el efecto poblacional. Una corrección habitual es

$$\omega^2 = \frac{SS_B - (G-1)MS_W}{SS_T + MS_W}, \qquad MS_W = \frac{SS_W}{N-G}. \tag{11}$$

Aquí, $MS_W = 1/(6-3) = 1/3$ y

$$\omega^2 = \frac{19 - 2(1/3)}{20 + 1/3} = 0.9016.$$

La corrección reduce el optimismo de 0.9500 a 0.9016. En muestras reales, ω^2 puede resultar ligeramente negativa; eso se interpreta como ausencia de evidencia de variación explicada, aunque algunas implementaciones la truncan a cero para presentación.

El laboratorio de la versión HTML representa cada grupo mediante dos observaciones, "centro ± separación". Al alejar las medias de los grupos aumenta SS_B; al separar las observaciones dentro de cada grupo aumenta SS_W.

Lo que η^2 sí ve y lo que no ve

η^2 resume diferencias de **medias**. No detecta por sí sola todos los tipos de heterogeneidad distributiva. Dos grupos pueden tener la misma media y, aun así, diferir en:

- dispersión;
- asimetría;
- número o severidad de valores atípicos;
- multimodalidad.

Por ello, la descomposición ANOVA debe acompañarse con tamaños muestrales y gráficos de las distribuciones: diagramas de caja, violines, densidades o funciones de distribución empírica. Una cifra alta tampoco demuestra causalidad; puede reflejar confusión, selección o diseño muestral.

Heterogeneidad cruzada entre variables categóricas

Cuando se cruzan dos variables categóricas, la pregunta pasa a ser si la composición de una cambia entre niveles de la otra. Para cada grupo g, compare las probabilidades condicionales[9]

$$p_{k|g} = P(X = k \mid G = g).$$

Una tabla de conteos responde cuánto soporte tiene cada combinación; una tabla de porcentajes por fila permite comparar composiciones. El mapa de calor debe mostrar ambas cosas o, como mínimo, acompañar cada porcentaje con su n_{kg}.

Celdas pequeñas

Las combinaciones con pocos casos generan porcentajes inestables. No existe un umbral universal que vuelva confiable una celda; la regla práctica $n < 10$ solo funciona como alerta inicial. Revise conteos, incertidumbre y mecanismo de muestreo antes de interpretar una diferencia como estructura real.

Si se necesita un resumen formal de asociación, una prueba χ^2 contrasta independencia y la V de Cramér cuantifica intensidad. Estas herramientas responden una pregunta relacional distinta de la diversidad marginal: una variable puede ser muy diversa y, al mismo tiempo, ser independiente de la otra.

Puentes con algoritmos de minería de datos

Algoritmo	Medida de heterogeneidad	Uso
Árboles CART	Impureza de Gini $1 - \sum p_k^2$	Elige el corte que produce nodos hijos más puros mediante la reducción ponderada de impureza.
ID3 / C4.5	Entropía de Shannon	ID3 maximiza ganancia de información; C4.5 la ajusta con *gain ratio* para evitar cortes con demasiadas ramas.

[9] $P(y = c \mid \mathbf{x})$ es una **probabilidad condicional**: la proporción de casos de clase c entre todos los que tienen exactamente las predictoras $\mathbf{x}$. Por ejemplo, entre todos los clientes con ingreso 30 y endeudamiento 58, la fracción que paga.

Algoritmo	Medida de heterogeneidad	Uso
k-means	SS_W o inercia	Busca centroides que minimicen la variación dentro de los grupos. Con datos fijos, minimizar SS_W equivale a maximizar SS_B.
Calinski–Harabasz	Razón tipo ANOVA	Compara separación entre *clusters* con compactación interna: $CH = (SS_B/(k-1))/(SS_W/(n-k))$.

En un árbol, "impureza de Gini" es precisamente Gini–Simpson aplicado a la distribución de clases dentro de un nodo. En *clustering*, los grupos no vienen dados: el algoritmo intenta construirlos para reducir heterogeneidad interna.

Implementación reproducible

Las siguientes funciones reproducen los cálculos del capítulo y hacen explícitas las convenciones para probabilidades nulas.

Ver la composición antes de resumirla

El ejemplo original y los dos casos brújula pueden compararse sin perder las proporciones que originan los índices.

Guía de decisión

Pregunta	Herramienta principal	Comprobación necesaria
¿Qué tan repartida está una variable categórica?	Barras, Gini–Simpson, Shannon	Reportar K, conteos y versión normalizada si se comparan cardinalidades.
¿Cuántas clases efectivas hay?	Números de Hill	Indicar el orden q y su sensibilidad a clases raras.
¿Cambian las medias numéricas entre grupos?	SS_B, SS_W, η^2, ω^2	Inspeccionar tamaños, dispersión y forma de cada grupo.
¿Cambia una composición categórica entre grupos?	Proporciones condicionales, mapa de calor, χ^2, V de Cramér	Mostrar soporte de cada celda.
¿Hay heterogeneidad causal?	Ningún índice descriptivo basta	Se requiere diseño causal y control de variables de confusión.

Puente al caso BigMart

El capítulo práctico Medidas de heterogeneidad en Big Mart Sales traslada estas ideas a datos de *retail*:

- diversidad de categorías de producto mediante Shannon, Gini–Simpson y números de Hill;
- heterogeneidad de ventas entre formatos de establecimiento mediante η^2 y ω^2;
- composiciones cruzadas entre ubicación, tipo y antigüedad del establecimiento.

La secuencia recomendada es siempre la misma: **ver los conteos o distribuciones, formular el experimento que da significado al índice, calcularlo y finalmente interpretar su escala**.

Ejercicios formativos

1. **Recupere el significado probabilístico.** Para $p = (0.7, 0.2, 0.1)$, construya la matriz de probabilidades $p_j p_k$. Sume la diagonal y su complemento. Explique el resultado como frecuencia esperada de desacuerdos en 100 pares.
2. **Sorpresa frente a aporte.** Para las mismas probabilidades, calcule $I_2(p_k)$ y $p_k I_2(p_k)$. Identifique qué clase sorprende más y cuál aporta más a la entropía. ¿Son necesariamente la misma?
3. **Escala y base.** Calcule H en bits y nats. Compruebe que H_{norm} es el mismo en ambas bases.
4. **Números efectivos.** Calcule ${}^0D$, ${}^1D$ y ${}^2D$ para $p = (0.7, 0.2, 0.1)$ y verifique ${}^0D \geq {}^1D \geq {}^2D$.
5. **Descomposición numérica.** Para $G_1 = (10, 12)$ y $G_2 = (20, 22)$, calcule SS_T, SS_B, SS_W, η^2 y ω^2.
6. **Límite de η^2.** Construya dos grupos con la misma media y dispersiones muy distintas. Muestre que $\eta^2 = 0$ y explique por qué eso no demuestra que las distribuciones sean iguales.

29. Medidas de heterogeneidad en Big Mart Sales

Introducción

Este caso aplica la diversidad categórica y la comparación entre grupos desarrolladas en Medidas de heterogeneidad.

Después de estudiar la localización, la variabilidad y la forma de las distribuciones, el análisis exploratorio de datos debe avanzar hacia una pregunta más estructural: **¿el comportamiento observado es homogéneo en toda la población o cambia de manera importante entre grupos?**. Esta cuestión introduce el concepto de **heterogeneidad**, una dimensión central del EDA cuando el dataset contiene segmentaciones naturales, como categorías de producto, tipos de tienda, tamaños de outlet o ubicaciones geográficas.

En términos estadísticos, la heterogeneidad alude a la existencia de diferencias sistemáticas entre subpoblaciones. Tales diferencias pueden expresarse en el centro de la distribución, en su dispersión, en su forma o incluso en la presencia diferencial de valores extremos. En un problema de negocio como BigMart, detectar heterogeneidad es indispensable, pues permite identificar si ciertos tipos de tiendas venden consistentemente más, si algunas categorías presentan patrones de ventas más variables, o si la estructura comercial cambia según el entorno del outlet.

En este capítulo se trabaja la heterogeneidad como una extensión natural del análisis univariado. La idea principal es que una variable numérica no debe interpretarse de manera aislada, sino también en relación con variables categóricas relevantes.

Definición operativa. La heterogeneidad aparece cuando la distribución de ventas cambia entre subgrupos o cuando la composición categórica presenta diversidad relevante (Shannon 1948; Simpson 1949).

Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy y Plotly.

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614

Conviene verificar las variables categóricas y numéricas del conjunto de datos.

Fundamentos teóricos

La heterogeneidad puede analizarse desde varias perspectivas. Si Y es una variable numérica de interés y G una variable categórica que define grupos, pueden estudiarse diferencias en:

Centro por grupo

Se comparan medidas como:

$$\bar{y}_g = \frac{1}{n_g} \sum_{i \in g} y_i,$$

así como la mediana y otros cuantiles dentro de cada grupo g.

Dispersión por grupo

La variabilidad interna de cada subpoblación puede resumirse mediante la desviación estándar, el rango intercuartílico o el coeficiente de variación. Dos grupos pueden tener medias semejantes y, sin embargo, diferir de forma sustancial en estabilidad o consistencia.

Forma por grupo

La heterogeneidad también puede aparecer en la asimetría o en la presencia de colas pesadas. Algunos grupos pueden concentrarse en rangos estrechos y otros exhibir ventas con distribución muy extendida.

Participación relativa de cada grupo

Finalmente, también es útil estudiar cuánto contribuye cada grupo al total de observaciones o al total de ventas. Esta dimensión conecta la heterogeneidad con el estudio posterior de la concentración.

Variable objetivo y grupos de interés

En este capítulo tomaremos nuevamente como variable principal a `Item_Outlet_Sales`. Como ejes de segmentación se consideran, al menos, las siguientes variables categóricas:

- `Outlet_Type`,
- `Outlet_Size`,
- `Outlet_Location_Type`,
- `Item_Type`.

Antes de comparar grupos, es útil revisar su tamaño.

Este paso es importante porque la interpretación de diferencias entre grupos debe considerar el tamaño muestral. Un grupo muy pequeño puede mostrar estadísticas inestables o aparentes extremos debidos a pocos casos.

Heterogeneidad por tipo de outlet

Comenzamos con una variable de alto interés de negocio: `Outlet_Type`.

Outlet_Type	count	mean	median	std	min	max
Supermarket Type3	935	3694.038558	3364.9532	2127.760054	241.6854	13086.9648
Supermarket Type1	5577	2316.181148	1990.7420	1515.965558	73.2380	10256.6490
Supermarket Type2	928	1995.498739	1655.1788	1375.932889	69.2432	6768.5228
Grocery Store	1083	339.828500	256.9988	260.851582	33.2900	1775.6886

A partir de este análisis se observa una **heterogeneidad clara por tipo de outlet**, lo que indica que `Outlet_Type` sí está asociado con diferencias relevantes en el nivel de ventas.

En primer lugar, `Supermarket Type3` destaca como el segmento con **mayor nivel de ventas**, tanto en la **media** (**3694.039**) como en la **mediana** (**3364.953**). Además, su boxplot se ubica claramente por encima del resto, lo que sugiere un desempeño consistentemente superior.

En segundo lugar, `Supermarket Type1` y `Supermarket Type2` presentan niveles de ventas **intermedios**, aunque `Supermarket Type1` muestra valores centrales algo mayores (**media = 2316.181**, **mediana = 1990.742**) que `Supermarket Type2` (**media = 1995.499**, **mediana = 1655.179**). Esto sugiere que, aunque ambos formatos son comparables, no tienen exactamente el mismo comportamiento comercial.

Por otro lado, `Grocery Store` aparece como el grupo con **menor nivel de ventas**, con una **media de 339.829** y una **mediana de 256.999**, muy por debajo de los supermercados. Esto revela una diferencia estructural importante entre este tipo de outlet y los demás.

En términos de **variabilidad**, `Supermarket Type3` también presenta la mayor dispersión (**std = 2127.760**), lo que indica que no solo vende más, sino que sus ventas son más heterogéneas. `Supermarket Type1` y `Supermarket Type2` también muestran dispersión considerable, mientras que `Grocery Store` tiene una variabilidad mucho menor en términos absolutos (**std = 260.852**), coherente con su escala de ventas más baja.

Los boxplots también muestran la presencia de **valores atípicos** en todos los grupos, especialmente en los supermercados. Esto sugiere que dentro de cada tipo de outlet existen productos o combinaciones producto-tienda con desempeños excepcionalmente altos.

En conjunto, este resultado tiene varias implicaciones para el EDA:

- `Outlet_Type` debe considerarse una **variable clave de segmentación**;
- no conviene analizar `Item_Outlet_Sales` como si proviniera de una sola población homogénea;

- es recomendable profundizar con comparaciones entre grupos, distribuciones segmentadas y medidas robustas;
- en etapas posteriores, `Outlet_Type` probablemente será una variable explicativa importante en cualquier análisis predictivo o inferencial.

En síntesis, el análisis muestra que el tipo de outlet introduce diferencias sustanciales tanto en el **nivel** como en la **dispersión** de las ventas, por lo que esta variable merece un lugar central dentro del EDA.

Heterogeneidad por tamaño del outlet

El tamaño del outlet puede influir tanto en el volumen típico de ventas como en la dispersión observada.

Outlet_Size	count	mean	median	std
Medium	2793	2681.603542	2251.0698	1855.210528
High	932	2298.995256	2050.6640	1533.531664
Small	2388	1912.149161	1544.6560	1582.370364

Los violin plots son especialmente útiles aquí porque combinan información sobre dispersión y forma dentro de cada grupo. Si dos grupos tienen medianas parecidas pero densidades internas distintas, la heterogeneidad va más allá del simple promedio.

Este análisis también muestra **heterogeneidad por tamaño del outlet**, aunque con diferencias menos extremas que las observadas por `Outlet_Type`. Aun así, `Outlet_Size` aporta información relevante para el EDA porque parece asociarse tanto con el nivel típico de ventas como con su dispersión.

En primer lugar, los outlets de tamaño `Medium` presentan el mayor nivel de ventas, con una **media de 2681.604** y una **mediana de 2251.070**. Les siguen los outlets `High`, con **media = 2298.995** y **mediana = 2050.664, mientras que los outletsSmall‘ muestran los valores más bajos (**media = 1912.149**,** mediana = 1544.656**). Esto sugiere que el tamaño del outlet sí introduce diferencias sistemáticas en el comportamiento comercial.

En términos de dispersión, `Medium` también presenta la mayor variabilidad (**std = 1855.211**), seguido por `Small` (**std = 1582.370**) y `High` (**std = 1533.532**). Los violin plots refuerzan esta idea al mostrar que no solo cambian los niveles centrales, sino también la forma interna de la distribución en cada grupo.

Para el EDA, esto tiene varias consecuencias:

- `Outlet_Size` debe considerarse una **variable relevante de segmentación**, ya que las ventas no parecen distribuirse igual entre tamaños de tienda;
- no basta con comparar solo promedios: la forma de los violines sugiere diferencias en **concentración, dispersión y densidad interna** entre grupos;
- conviene cruzar `Outlet_Size` con otras variables categóricas, especialmente `Outlet_Type`, para ver si parte de esta heterogeneidad se explica por la combinación entre ambas;

- en análisis posteriores, `Outlet_Size` puede funcionar como una **variable explicativa útil**, aunque probablemente con menor poder discriminante que `Outlet_Type`;
- dado que esta variable tenía valores faltantes en etapas previas, su papel analítico resulta todavía más importante, porque ayuda a decidir si conviene imputarla y preservarla en lugar de descartarla.

En síntesis, el tamaño del outlet no solo afecta el nivel típico de ventas, sino también la estructura de su distribución. Por ello, en el EDA conviene tratarlo como un factor de heterogeneidad relevante y no como una simple característica secundaria del establecimiento.

Heterogeneidad por ubicación del outlet

Otra dimensión natural del análisis es `Outlet_Location_Type`, que puede reflejar diferencias territoriales o de mercado.

Outlet_Location_Type	count	mean	median	std
Tier 2	2785	2323.990559	2004.0580	1520.543543
Tier 3	3350	2279.627651	1812.3076	1912.451333
Tier 1	2388	1876.909159	1487.3972	1561.649293

El análisis por `Outlet_Location_Type` revela también **heterogeneidad territorial en las ventas**, aunque con diferencias más moderadas que las observadas por tipo de outlet.

En términos de nivel central, `Tier 2` presenta el mayor promedio (**mean = 2323.991**, **median = 2004.058**), seguido muy de cerca por `Tier 3` (**mean = 2279.628**, **median = 1812.308**). En contraste, `Tier 1` muestra los valores más bajos (**mean = 1876.909**, **median = 1487.397**), lo que sugiere que las ventas tienden a ser menores en este tipo de localización.

En cuanto a la dispersión, `Tier 3` presenta la mayor variabilidad (**std = 1912.451**), lo que indica una mayor heterogeneidad en el desempeño de ventas dentro de este grupo. Los boxplots también muestran la presencia de **valores extremos** en todas las ubicaciones, especialmente en `Tier 3`.

Para el EDA, esto sugiere que:

- `Outlet_Location_Type` introduce **diferencias territoriales en el nivel de ventas**;
- `Tier 2` y `Tier 3` tienen comportamientos relativamente similares en el centro de la distribución, aunque `Tier 3` muestra mayor dispersión;
- `Tier 1` parece corresponder a ubicaciones con **menor desempeño promedio**;
- conviene explorar **interacciones entre ubicación, tipo y tamaño del outlet**, ya que parte de la heterogeneidad observada podría explicarse por la combinación de estas variables.

En síntesis, la localización del outlet añade otra dimensión de segmentación en el EDA y refuerza la idea de que las ventas no provienen de una población homogénea, sino de **subgrupos con comportamientos distintos**.

Heterogeneidad por tipo de producto

La variable `Item_Type` suele contener más niveles que las anteriores, por lo que conviene resumir y ordenar cuidadosamente la información.

Item_Type	count	mean	median	std
Starchy Foods	148	2374.332773	1968.1048	1773.945328
Seafood	64	2326.065928	2055.3246	1842.988719
Fruits and Vegetables	1232	2289.009592	1830.9500	1799.503459
Snack Foods	1200	2277.321739	1944.1360	1705.121755
Household	910	2258.784300	1981.4208	1692.245757
Dairy	682	2232.542597	1650.8511	1884.404698
Canned	649	2225.194904	1860.2452	1645.235638
Breads	251	2204.132226	1860.2452	1644.235914
Meat	425	2158.977911	1829.6184	1695.231081
Hard Drinks	214	2139.221622	1816.6353	1606.191587

El análisis por `Item_Type` muestra que también existe **heterogeneidad por categoría de producto**, aunque menos marcada que la observada por `Outlet_Type`. Aun así, las diferencias son suficientes para justificar una exploración segmentada.

En términos de promedio, las categorías con mayores ventas son `Starchy Foods` (**mean = 2374.333**), `Seafood` (**mean = 2326.066**) y `Fruits and Vegetables` (**mean = 2289.010**). Sin embargo, estas dos primeras tienen tamaños muestrales mucho menores (**148** y **64** observaciones, respectivamente), por lo que sus medias deben interpretarse con más cautela.

Entre las categorías con mayor presencia en el dataset, `Fruits and Vegetables` (**1232** casos), `Snack Foods` (**1200**) y `Household` (**910**) combinan **niveles altos de ventas** con una base de observaciones más sólida, lo que las vuelve especialmente relevantes para el EDA.

Los boxplots sugieren además que estas categorías no solo difieren en el nivel central, sino también en su **dispersión** y en la presencia de **valores extremos**. Por ejemplo, `Household`, `Fruits and Vegetables` y `Snack Foods` muestran colas superiores amplias y varios valores altos, lo que sugiere comportamientos heterogéneos dentro de cada tipo de producto.

Para el EDA, esto tiene varias implicaciones:

- `Item_Type` debe considerarse una **variable de segmentación importante**;
- conviene distinguir entre categorías con **muchas observaciones** y categorías con **poca representación**, para no sobredimensionar diferencias basadas en muestras pequeñas;
- resulta útil cruzar `Item_Type` con variables como `Outlet_Type` u `Outlet_Size`, ya que parte de la heterogeneidad podría depender de la interacción entre producto y tienda;
- en etapas posteriores, `Item_Type` puede aportar valor explicativo, pero probablemente con efecto desigual según la categoría.

En síntesis, el tipo de producto introduce diferencias reales en las ventas, pero su interpretación debe combinar **nivel de ventas**, **dispersión** y **tamaño muestral** para evitar conclusiones apresuradas.

Tablas comparativas con múltiples indicadores

Una forma útil de sistematizar la heterogeneidad es construir tablas resumen que integren varias medidas por grupo.

	Outlet_Type	n	media	mediana	desviacion	q1	q3	iqr
3	Supermarket Type3	935	3694.038558	3364.9532	2127.760054	2044.33890	4975.52340	2931.1845
1	Supermarket Type1	5577	2316.181148	1990.7420	1515.965558	1151.16820	3135.91800	1984.7498
2	Supermarket Type2	928	1995.498739	1655.1788	1375.932889	981.55565	2702.64865	1721.0930
0	Grocery Store	1083	339.828500	256.9988	260.851582	153.79980	458.73620	304.9364

	Outlet_Location_Type	n	media	mediana	desviacion	q1	q3	iqr
1	Tier 2	2785	2323.990559	2004.0580	1520.543543	1171.80800	3110.6176	1938.8096
2	Tier 3	3350	2279.627651	1812.3076	1912.451333	731.38130	3307.6944	2576.3131
0	Tier 1	2388	1876.909159	1487.3972	1561.649293	593.72715	2803.0180	2209.2908

Estas tablas permiten pasar de una observación visual a una descripción más precisa: no solo se identifican grupos "más altos" o "más bajos", sino también grupos más o menos variables y con distinta estabilidad relativa.

Heterogeneidad cruzada

En muchos problemas reales, la segmentación relevante no depende de una sola variable categórica. Una comparación cruzada, por ejemplo entre tipo de outlet y localización, ayuda a identificar patrones más finos.

Outlet_Location_Type Outlet_Type	Tier 1	Tier 2	Tier 3
Grocery Store	340.329723	NaN	339.351662
Supermarket Type1	2313.099451	2323.990559	2298.995256
Supermarket Type2	NaN	NaN	1995.498739
Supermarket Type3	NaN	NaN	3694.038558

La heterogeneidad cruzada muestra que una sola variable categórica no siempre basta para describir el comportamiento de las ventas. En este caso, la combinación entre `Outlet_Type` y `Outlet_Location_Type` permite detectar patrones más específicos que no se observan con la misma claridad al analizar cada variable por separado.

El resultado más visible es que `Supermarket Type3` en `Tier 3` presenta la media de ventas más alta (**3694.039**), mientras que `Grocery Store` mantiene niveles muy bajos tanto en `Tier 1` (**340.330**) como en `Tier 3` (**339.352**). Por su parte, `Supermarket Type1` muestra un comportamiento bastante estable entre ubicaciones, con medias de **2313.099** en `Tier 1`, **2323.991** en `Tier 2` y **2298.995** en `Tier 3`.

Para el EDA, esto tiene varias implicaciones:

- la variación en ventas no depende solo del tipo de outlet ni solo de la ubicación, sino de su **combinación**;
- `Supermarket Type1` parece ser un formato relativamente estable entre territorios, mientras que otros tipos están mucho más concentrados en ciertas ubicaciones;
- existen combinaciones ausentes (`NaN`), lo que indica que no todos los formatos de outlet están presentes en todas las localizaciones, y eso también forma parte de la estructura real del negocio;
- conviene seguir explorando **interacciones** entre variables categóricas, porque parte de la heterogeneidad puede estar oculta cuando se estudian los factores por separado;
- en etapas posteriores, estas combinaciones pueden ser útiles para construir variables explicativas más informativas que cada factor individual por sí solo.

En síntesis, este análisis refuerza una idea central del EDA: las ventas no provienen de grupos simples, sino de **segmentos combinados** donde el contexto del outlet importa tanto como su categoría individual.

Entropía de Shannon como medida de heterogeneidad

Hasta ahora, la heterogeneidad se ha estudiado comparando promedios, medianas, dispersiones y distribuciones entre grupos. Sin embargo, también es útil contar con una medida **global** que resuma qué tan **diversificada** o **concentrada** está una variable categórica. Para ello puede utilizarse la **entropía de Shannon**.

Si una categoría concentra casi toda la masa de observaciones o de ventas, la entropía será baja. En cambio, si la distribución está más repartida entre varios grupos, la entropía será mayor. En este sentido, la entropía ofrece una medida sintética de diversidad.

En términos generales, para una variable categórica con proporciones $p_1, p_2, \dots, p_k$, la entropía se define como

$$H = -\sum_{i=1}^{k} p_i \log(p_i)$$

donde valores más altos indican mayor heterogeneidad. Como el valor máximo depende del número de categorías, también conviene trabajar con una **entropía normalizada**:

$$H_{\text{norm}} = \frac{H}{\log(k)}$$

De esta forma, el resultado queda entre 0 y 1:

- valores cercanos a **0** indican **alta concentración**;
- valores cercanos a **1** indican **alta diversidad**.

Entropía de ventas por variable categórica

En esta sección calcularemos la entropía sobre la **distribución de ventas** agregadas por categoría, para responder preguntas como: ¿las ventas están repartidas entre muchos grupos o dependen de unos pocos?

	variable	n_categorias	entropia	entropia_normalizada
2	Outlet_Location_Type	3.0	1.0758	0.9792
3	Item_Type	16.0	2.5168	0.9078
1	Outlet_Size	3.0	0.9876	0.8990
0	Outlet_Type	4.0	0.8731	0.6298

La interpretación debe hacerse con cuidado. Una entropía alta no significa necesariamente "mejor", sino simplemente que las ventas están más repartidas entre las categorías de esa variable. Una entropía baja, en cambio, sugiere que el negocio está más concentrado en unos pocos segmentos.

Si se desea profundizar en una variable concreta, también puede visualizarse directamente la distribución de las ventas agregadas por categoría. Por ejemplo, para Item_Type:

Los resultados de entropía sugieren que no todas las variables categóricas aportan el mismo tipo de segmentación para el EDA. En particular, `Outlet_Location_Type` presenta la **mayor entropía normalizada** (**0.9792**), seguido por `Item_Type` (**0.9078**) y `Outlet_Size` (**0.8990**). Esto indica que, en estas variables, las ventas están **bastante repartidas** entre sus categorías y no dependen fuertemente de un solo grupo.

En cambio, `Outlet_Type` muestra una entropía normalizada claramente menor (**0.6298**), lo que sugiere una **mayor concentración** de las ventas en unos pocos tipos de outlet. Esto es coherente con los análisis previos, donde se observó que ciertos formatos, especialmente `Supermarket Type3` y `Supermarket Type1`, concentran gran parte del volumen de ventas.

Para el EDA, esto tiene varias implicaciones:

- `Outlet_Type` aparece como una variable especialmente útil para detectar **concentración comercial**, ya que distingue grupos con pesos muy desiguales en el negocio;
- `Outlet_Location_Type`, `Outlet_Size` e `Item_Type` reflejan una estructura más **diversificada**, por lo que su utilidad analítica está menos en la concentración y más en comparar patrones internos entre categorías;
- una entropía alta no elimina la heterogeneidad, sino que indica que el negocio está más repartido entre grupos, por lo que conviene profundizar con medidas de nivel, dispersión y forma dentro de cada categoría;
- el gráfico de proporciones por `Item_Type` muestra que, aunque la distribución es bastante diversa, algunas categorías como `Fruits and Vegetables` (**0.152**) y `Snack Foods` (**0.147**) tienen un peso algo mayor que otras, por lo que pueden merecer atención especial en análisis posteriores.

En síntesis, la entropía complementa el EDA porque ayuda a distinguir entre variables que describen un negocio **más concentrado** y variables que describen un negocio **más distribuido**. En este caso, `Outlet_Type` parece ser la variable más útil para estudiar concentración, mientras que `Item_Type`,

`Outlet_Size` y `Outlet_Location_Type` son más útiles para explorar diversidad y segmentación interna.

La entropía anterior describe diversidad de participaciones. El coeficiente de Gini basado en la curva de Lorenz responde específicamente a desigualdad y se desarrolla, con su fórmula corregida, en Medidas de concentración en Big Mart Sales.

Mini-dashboard de heterogeneidad

A continuación se construye un dashboard compacto que integra varias vistas segmentadas.

Conclusiones del capítulo

En este capítulo se mostró que la heterogeneidad constituye una dimensión esencial del análisis exploratorio de datos. Comparar grupos permite ampliar la descripción estadística de una variable y pasar de una lectura global a una interpretación segmentada y contextual.

En el dataset de BigMart, la heterogeneidad se manifiesta en diferencias entre tipos de outlet, tamaños, localizaciones y categorías de producto. Estas comparaciones enriquecen el EDA porque permiten distinguir subestructuras internas del fenómeno, identificar patrones consistentes y detectar segmentos con comportamientos contrastantes.

Este capítulo prepara de manera natural el siguiente paso del proyecto: el estudio de la **concentración**, donde se analizará si una fracción reducida de grupos, productos o outlets explica una proporción sustancial del total de ventas.

Parte XVI.

Medidas de concentración

30. Medidas de concentración

El problema: ¿quién acumula el total?

Una cadena de cafeterías vendió 100 bebidas durante una hora. El total parece sencillo, pero para operar el negocio falta responder otra pregunta:

> ¿Las ventas están repartidas entre las sucursales o dependen de unas pocas?

Supongamos que las cinco sucursales registraron:

Sucursal	Ventas x_i	Participación $s_i = x_i/T$
Norte	5	0.05
Sur	10	0.10
Centro	15	0.15
Oriente	20	0.20
Poniente	50	0.50
Total	**100**	**1.00**

La media es de 20 ventas por sucursal, pero solo Oriente vendió exactamente 20 y Poniente produjo la mitad del total. La media describe un centro; no describe **cómo se reparte la masa** entre las unidades.

En este capítulo llamaremos **masa** a una cantidad no negativa y aditiva: ventas, ingreso, consumo, población, participación de mercado o tráfico. Las **unidades** son quienes reciben esa masa: sucursales, hogares, productos, empresas o regiones.

Si

$$T = \sum_{i=1}^{K} x_i > 0, \qquad s_i = \frac{x_i}{T},$$

entonces s_i es la participación de la unidad i y $\sum_i s_i = 1$. La concentración depende de este vector de participaciones, no de la escala original. Multiplicar todas las ventas por 10 no cambia su reparto.

Dos casos que servirán como brújula

- Si cada sucursal vende 20, el reparto es uniforme y la concentración debe ser **mínima**.
- Si una sucursal vende las 100 bebidas y las demás cero, la concentración debe ser **máxima**.

Entre ambos extremos, una transferencia desde una sucursal pequeña hacia una grande, sin cambiar el total, debe aumentar la concentración.

Esta idea parece simple, pero no existe una única medida universal. Algunas preguntan cuánto reúne el grupo líder; otras usan todas las participaciones o comparan toda la distribución acumulada.

Primera mirada: razones de concentración

Ordenemos las participaciones de mayor a menor:

$$s_{[1]} \geq s_{[2]} \geq \cdots \geq s_{[K]}.$$

La **razón de concentración de las k unidades principales** es

$$CR_k = \sum_{i=1}^{k} s_{[i]}, \qquad 1 \leq k \leq K. \tag{1}$$

Para las cafeterías,

$$(s_{[1]}, s_{[2]}, s_{[3]}, s_{[4]}, s_{[5]}) = (0.50, 0.20, 0.15, 0.10, 0.05).$$

Por tanto,

$$CR_1 = 0.50, \qquad CR_2 = 0.50 + 0.20 = 0.70.$$

La sucursal líder genera 50 % de las ventas y las dos primeras generan 70 %. Estas frases son inmediatamente comprensibles; esa es la principal virtud de CR_k.

Lo que el top k no ve

Compare dos repartos:

$$A = (0.50, 0.20, 0.15, 0.10, 0.05),$$

$$B = (0.50, 0.125, 0.125, 0.125, 0.125).$$

Ambos tienen $CR_1 = 0.50$, pero no son igual de concentrados: en A, el 50 % restante también está escalonado; en B, está repartido por igual. CR_1 descarta toda esa información. En general, CR_k:

- usa solo la frontera elegida k;
- no describe cómo se reparte la masa dentro del top k;
- tampoco describe cómo se reparte fuera de él;
- cambia de significado si se compara un k distinto.

Por eso conviene acompañarlo con una medida que use **todas** las participaciones.

HHI: ¿de qué sucursal vienen dos ventas?

Imagine que cada una de las 100 ventas se escribe en un boleto y se colocan los boletos en una urna. Se extrae un boleto, se devuelve, se mezcla y se extrae otro. ¿Cuál es la probabilidad de que ambos pertenezcan a la misma sucursal?

Para Poniente,

$$P(\text{Poniente dos veces}) = 0.50^2 = 0.25.$$

Para Norte,

$$P(\text{Norte dos veces}) = 0.05^2 = 0.0025.$$

Los eventos “ambos boletos vienen de Norte”, “ambos vienen de Sur”, etcétera, son mutuamente excluyentes. Por ello, la probabilidad total de coincidencia es

$$\begin{aligned} P(\text{misma sucursal}) &= 0.05^2 + 0.10^2 + 0.15^2 + 0.20^2 + 0.50^2 \\ &= 0.0025 + 0.0100 + 0.0225 + 0.0400 + 0.2500 \\ &= 0.3250. \end{aligned}$$

Esta probabilidad es el **índice Herfindahl–Hirschman**:

$$HHI(s) = \sum_{i=1}^{K} s_i^2. \tag{2}$$

Su uso como índice de concentración se asocia históricamente con los trabajos de Hirschman (Hirschman 1945). La interpretación probabilística explica por qué aparecen cuadrados: una coincidencia de la unidad i requiere dos selecciones independientes, cada una con probabilidad s_i.

El puente con el capítulo anterior

El HHI es la probabilidad de **coincidencia**:

$$HHI = P(X_1 = X_2).$$

El índice de Gini–Simpson es la probabilidad complementaria de **diferencia**:

$$D = 1 - HHI = P(X_1 \neq X_2).$$

La misma suma de cuadrados mide concentración o heterogeneidad según se observe la coincidencia o su complemento.

Cotas del HHI

Como las participaciones son no negativas y suman uno,

$$\frac{1}{K} \le HHI \le 1. \tag{3}$$

El máximo $HHI = 1$ ocurre cuando una unidad posee toda la masa. Para justificar el mínimo, la desigualdad de Cauchy–Schwarz da

$$\left(\sum_{i=1}^{K} s_i\right)^2 \le K \sum_{i=1}^{K} s_i^2.$$

Como $\sum_i s_i = 1$,

$$HHI = \sum_i s_i^2 \ge \frac{1}{K},$$

con igualdad solo cuando $s_1 = \cdots = s_K = 1/K$.

En el ejemplo, $K = 5$, así que el intervalo posible es $[0.20, 1]$. El valor 0.325 no significa "32.5 % del máximo recorrido", porque la escala no comienza en cero. La versión normalizada es

$$HHI^{*} = \frac{HHI - 1/K}{1 - 1/K} \in [0, 1]. \tag{4}$$

Para las cafeterías,

$$HHI^{*} = \frac{0.325 - 0.20}{1 - 0.20} = \frac{0.125}{0.80} = 0.15625.$$

Otra escala útil es el **número efectivo de unidades**:

$$N_{\text{efectivo}} = \frac{1}{HHI}. \tag{5}$$

Aquí,

$$N_{\text{efectivo}} = \frac{1}{0.325} \approx 3.077.$$

La estructura observada tiene la misma probabilidad de coincidencia que un reparto ideal entre aproximadamente 3.08 unidades de igual tamaño. Esta escala no afirma que existan fracciones de sucursal; traduce la concentración a una cantidad equivalente fácil de comparar.

Por qué una transferencia hacia la unidad grande aumenta el HHI

Sean $a \geq b$ dos participaciones. Transfiera una cantidad $\delta > 0$ desde b hacia a, con $\delta \leq b$. El total no cambia, pero la diferencia entre ambas unidades aumenta:

$$(a, b) \longrightarrow (a + \delta, b - \delta).$$

El cambio en el HHI es

$$\begin{aligned} \Delta HHI &= (a+\delta)^2 + (b-\delta)^2 - a^2 - b^2 \\ &= 2\delta(a-b) + 2\delta^2 > 0. \end{aligned} \tag{6}$$

No es una intuición accidental: el cuadrado penaliza participaciones grandes de forma creciente. Una transferencia en sentido contrario, desde la unidad grande hacia la pequeña sin invertir su orden, reduce el HHI.

La escala 0–10,000

En análisis de mercados es común reportar $10\,000 \times HHI$. Nuestro valor sería 3250. Es exactamente el mismo índice expresado con participaciones en porcentaje en vez de proporciones. No mezcle 0.325 con 3250 ni aplique umbrales institucionales sin indicar jurisdicción, fecha y definición del mercado.

Curva de Lorenz: acumular desde la unidad menor

El HHI resume todo en una cifra. Para observar **cómo** se forma la concentración, ordenamos ahora las cantidades de menor a mayor:

$$x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(K)}.$$

La curva de Lorenz (Lorenz 1905) une los puntos

$$\left(\frac{i}{K}, L_i\right), \qquad L_i = \frac{\sum_{j=1}^{i} x_{(j)}}{\sum_{j=1}^{K} x_{(j)}}, \qquad i = 0, \ldots, K, \tag{7}$$

con $L_0 = 0$. El eje horizontal acumula unidades; el vertical acumula masa.

Construcción manual

En nuestro ejemplo los valores ya están ordenados:

$$(5, 10, 15, 20, 50).$$

i	Ventas $x_{(i)}$	Fracción acumulada de unidades $i/5$	Ventas acumuladas	Masa acumulada L_i
0	–	0.00	0	0.00
1	5	0.20	5	0.05
2	10	0.40	15	0.15
3	15	0.60	30	0.30
4	20	0.80	50	0.50
5	50	1.00	100	1.00

Los puntos son

$$(0,0),\ (0.2,0.05),\ (0.4,0.15),\ (0.6,0.30),\ (0.8,0.50),\ (1,1).$$

Si todas las sucursales tuvieran la misma participación, el 20 % de las sucursales acumularía 20 % de las ventas, el 40 % acumularía 40 %, etcétera. Esa es la diagonal $L(u) = u$, llamada **línea de igualdad**.

Nuestra curva queda debajo de la diagonal: el 80 % de las sucursales con menores ventas reúne apenas 50 % del total; la última sucursal aporta el otro 50 %. Cuanto más se arquea la curva, más desigual es el reparto.

Orden de Lorenz

Si la curva de una distribución queda en todo punto por debajo de la curva de otra, la primera es inequívocamente más desigual en el sentido de Lorenz. Si las curvas se cruzan, Lorenz no establece un orden total: dos índices pueden ponderar partes distintas del reparto y producir rankings diferentes.

Coeficiente de Gini: convertir el arco en una cifra

El coeficiente de Gini resume la separación entre la curva de Lorenz y la diagonal (Gini 1912). Si B es el área bajo la curva de Lorenz, el triángulo bajo la diagonal tiene área $1/2$, de modo que

$$G = \frac{1/2 - B}{1/2} = 1 - 2B. \tag{8}$$

Cálculo por trapecios

Entre dos puntos consecutivos, la curva muestral es lineal. Cada tramo tiene ancho $1/5 = 0.2$, así que

$$
\begin{aligned}
B &= 0.2\left[\frac{0+0.05}{2}+\frac{0.05+0.15}{2}+\frac{0.15+0.30}{2}\right.\\
&\qquad \left.+\frac{0.30+0.50}{2}+\frac{0.50+1}{2}\right]\\
&= 0.2(0.025+0.100+0.225+0.400+0.750)\\
&= 0.300.
\end{aligned}
$$

Por tanto,

$$G = 1 - 2(0.300) = 0.400.$$

La misma respuesta con la fórmula de rangos

Para K observaciones no negativas, ordenadas de menor a mayor y con total positivo,

$$G = \frac{2\sum_{i=1}^{K} i\, x_{(i)}}{K\sum_{i=1}^{K} x_{(i)}} - \frac{K+1}{K}. \tag{9}$$

En nuestro ejemplo,

$$\sum_{i=1}^{5} i\, x_{(i)} = 1(5)+2(10)+3(15)+4(20)+5(50) = 400.$$

Entonces,

$$G = \frac{2(400)}{5(100)} - \frac{6}{5} = 1.6 - 1.2 = 0.4.$$

La igualdad entre ambos resultados no es coincidencia: la fórmula de rangos es una forma algebraica de calcular el área trapezoidal.

Una tercera interpretación: diferencias por pares

El Gini también puede escribirse como

$$G = \frac{\sum_{i=1}^{K}\sum_{j=1}^{K} |x_i - x_j|}{2K^2\bar{x}}. \tag{10}$$

El numerador suma todas las diferencias absolutas entre pares **ordenados** de unidades. Para las cinco sucursales, la suma es 400, $\bar{x} = 20$, y

$$G = \frac{400}{2(5^2)(20)} = 0.4.$$

De manera equivalente, si se eligen dos sucursales al azar con reemplazo, la diferencia absoluta esperada de ventas es $400/25 = 16$. Al dividirla entre dos veces la media, $2\bar{x} = 40$, se obtiene $16/40 = 0.4$.

El máximo muestral no siempre es 1

Con K unidades y masa no negativa, el máximo ocurre cuando una sola unidad posee todo:

$$G_{\text{máx}} = \frac{K-1}{K}.$$

Con cinco sucursales, $G_{\text{máx}} = 0.8$, no 1. El valor 1 es el límite al crecer K. Si se desea una escala con máximo exactamente 1 para un K fijo, puede usarse

$$G^* = \frac{K}{K-1}G.$$

Nuestro ejemplo tiene $G^* = 5(0.4)/4 = 0.5$.

Laboratorio interactivo: mover la masa

El siguiente laboratorio mantiene la historia de las cinco sucursales. Los controles permiten cambiar sus ventas; los botones realizan transferencias de cinco ventas sin alterar el total. Observe qué aspectos detecta cada índice.

La versión HTML contiene un laboratorio con cinco controles. El escenario base $(5, 10, 15, 20, 50)$ produce $CR_2 = 0.70$, $HHI = 0.325$ y $G = 0.40$. Una transferencia de cinco ventas de Norte a Poniente conserva el total, pero arquea más la curva de Lorenz y aumenta HHI y Gini. La transferencia inversa los reduce.

Qué ve cada indicador

Los índices no son intercambiables. Compare cuatro escenarios con cinco unidades y total 100:

Escenario ordenado de mayor a menor	CR_1	CR_2	HHI	HHI^*	Gini
Uniforme: $(20, 20, 20, 20, 20)$	0.20	0.40	0.2000	0.0000	0.0000
Base: $(50, 20, 15, 10, 5)$	0.50	0.70	0.3250	0.1563	0.4000
Mismo líder: $(50, 12.5, 12.5, 12.5, 12.5)$	0.50	0.625	0.3125	0.1406	0.3000
Mismo CR_2: $(40, 30, 20, 5, 5)$	0.40	0.70	0.2950	0.1188	0.3800

El escenario base y "Mismo líder" comparten CR_1, pero HHI y Gini detectan que el resto está más escalonado en el primero. El escenario base y "Mismo CR_2" comparten 70 % en las dos unidades principales, pero distribuyen ese 70 % y la cola de forma distinta.

El HHI da peso cuadrático a las participaciones grandes. El Gini promedia brechas relativas entre todas las parejas y está conectado con toda la curva de Lorenz. No existe contradicción si dos medidas reaccionan con intensidades distintas: están resumiendo aspectos distintos.

Propiedades que debe cumplir una comparación

Antes de interpretar una cifra, conviene comprobar qué propiedades necesita el problema:

1. **Anonimato.** Permutar los nombres de las unidades no cambia el índice.
2. **Invariancia de escala.** Multiplicar toda la masa por una constante positiva no cambia la concentración.
3. **Principio de transferencia.** Mover masa de una unidad menor hacia una mayor, conservando el total, aumenta la concentración.
4. **Sensibilidad al número de unidades.** HHI mínimo y Gini máximo dependen de K; las versiones normalizadas hacen explícita esa dependencia.

CR_k, HHI y Gini cumplen las dos primeras. HHI y Gini también responden al principio de transferencia. CR_k puede no cambiar si la transferencia ocurre completamente fuera de la frontera del top k.

La unidad de análisis cambia la respuesta

Suponga que Poniente se divide administrativamente en dos sucursales idénticas de 25 ventas. El fenómeno económico podría ser el mismo, pero ahora hay seis unidades y

$$HHI_{\text{nuevo}} = 0.05^2 + 0.10^2 + 0.15^2 + 0.20^2 + 0.25^2 + 0.25^2 = 0.20,$$

frente a 0.325 antes de la división. Ninguna fórmula puede decidir por sí sola si ambas nuevas sucursales son competidores independientes o partes de una misma entidad. La definición de "unidad" es una decisión sustantiva previa al cálculo.

Concentración no significa automáticamente problema

Una cifra alta puede representar riesgo operativo, poder de mercado, especialización eficiente o simplemente una agregación mal definida. Los índices son descriptivos: no prueban causalidad, competencia insuficiente ni una recomendación normativa.

Implementación reproducible

Las siguientes funciones validan los supuestos, conservan una sola convención de orden y reproducen los cálculos manuales.

La curva de Lorenz reutiliza el mismo orden ascendente:

	Fracción acumulada de unidades	Fracción acumulada de ventas
0	0.0	0.00
1	0.2	0.05
2	0.4	0.15
3	0.6	0.30
4	0.8	0.50
5	1.0	1.00

La curva de Lorenz sobre los datos observados

La versión interactiva conserva los seis puntos calculados a mano y los contrasta con la línea de igualdad.

La igualdad entre las tres fórmulas del Gini puede verificarse numéricamente:

Guía de decisión

Pregunta	Herramienta principal	Qué debe acompañarla
¿Cuánto aporta el grupo líder?	CR_k	Declarar k, número total de unidades y participaciones ordenadas.
¿Qué tan probable es que dos unidades de masa provengan del mismo origen?	HHI	Reportar K, escala usada y, al comparar cardinalidades, HHI^* o $1/HHI$.
¿Cómo se acumula la masa desde las unidades menores?	Curva de Lorenz	Mostrar la diagonal y puntos acumulados; revisar cruces entre curvas.
¿Cuál es la brecha relativa promedio entre unidades?	Gini	Reportar K, tratamiento de ceros y si se normalizó el máximo finito.
¿La concentración es problemática o causal?	Ningún índice descriptivo basta	Definición sustantiva del mercado o sistema, contexto y diseño causal.

Errores frecuentes

- **Usar valores negativos.** Participaciones, Lorenz, HHI y las fórmulas presentadas del Gini requieren masa no negativa y total positivo. Una utilidad neta con pérdidas necesita otra formulación.
- **Omitir unidades con cero.** Los ceros no cambian el total ni el HHI, pero sí cambian K, la curva de Lorenz, el Gini y las normalizaciones.
- **Comparar distintos niveles de agregación.** Productos, marcas y empresas producen vectores de participaciones distintos.
- **Interpretar HHI como porcentaje.** HHI es una probabilidad de coincidencia; HHI^* es la fracción del recorrido entre igualdad y concentración total.
- **Interpretar Gini como "porcentaje en manos del top".** Esa lectura corresponde a CR_k o a un punto de la curva de Lorenz, no al Gini.
- **Aplicar mecánicamente la regla 80/20.** Pareto es una referencia descriptiva, no una ley que toda distribución deba cumplir.

Puentes con minería de datos

Contexto	Papel de la concentración
Selección de variables categóricas	Una categoría dominante puede indicar baja información efectiva aunque existan muchas etiquetas nominales.
Riesgo de dependencia	Un CR_k alto revela dependencia de pocos clientes, proveedores, productos o nodos.
Muestreo y particiones	Una masa muy concentrada puede producir validaciones inestables si las unidades dominantes se reparten de forma inadecuada.
Recomendadores	La concentración de exposición permite detectar si el sistema muestra repetidamente pocos artículos.
Clustering	El tamaño de los grupos puede evaluarse con HHI; $1/HHI$ expresa el número efectivo de clusters por tamaño.

La concentración de una variable objetivo tampoco sustituye el análisis de heterogeneidad condicional. Por ejemplo, que pocas sucursales acumulen ventas no explica qué atributos causan esa diferencia ni si el patrón persiste dentro de cada tipo de establecimiento.

Puente al caso BigMart

El capítulo Medidas de concentración en Big Mart Sales traslada estas ideas a productos y establecimientos:

- identifica qué categorías o outlets reúnen mayor parte de las ventas;

- compara CR_k, HHI y número efectivo de unidades;
- construye curvas de Lorenz y coeficientes de Gini;
- obliga a definir la unidad de análisis antes de interpretar el resultado.

La secuencia recomendada es: **definir unidad y masa, inspeccionar las participaciones, calcular a mano un caso pequeño, visualizar la acumulación y solo entonces resumir con índices**.

Ejercicios formativos

1. **Cálculo completo.** Para $x = (10, 10, 20, 20, 40)$, calcule participaciones, CR_1, CR_2, HHI, HHI^*, número efectivo, puntos de Lorenz, Gini y G^*.
2. **Mismo líder, distinta cola.** Compare $A = (50, 20, 15, 10, 5)$ y $B = (50, 12.5, 12.5, 12.5, 12.5)$. Explique por qué CR_1 empata mientras HHI y Gini no.
3. **Transferencia.** Parta de $(20, 20, 20, 20, 20)$ y transfiera cinco unidades de la primera sucursal a la quinta. Verifique con la ecuación de transferencia que el HHI aumenta y calcule el cambio del Gini.
4. **Tres fórmulas, una cifra.** Para $x = (5, 10, 15, 20, 50)$, obtenga el Gini mediante trapecios, rangos y diferencias por pares.
5. **Máximo finito.** Demuestre que el vector $(0, \dots, 0, T)$ produce $G = (K - 1)/K$ usando la fórmula de rangos.
6. **Unidad de análisis.** Calcule HHI y Gini antes y después de dividir la unidad de 50 en dos unidades de 25. Discuta cuándo esa división representa un cambio real y cuándo es solo administrativo.
7. **Curvas cruzadas.** Construya dos vectores de cinco valores cuyas curvas de Lorenz se crucen. Compare HHI y Gini y explique por qué Lorenz no impone un orden único en ese caso.

31. Medidas de concentración en Big Mart Sales

Este caso aplica las razones de concentración, HHI, Pareto, Lorenz y Gini desarrollados en Medidas de concentración.

Hasta este punto del análisis exploratorio ya hemos estudiado la localización, la variabilidad, la forma y la heterogeneidad de las variables principales del conjunto de datos **Big Mart Sales**. Falta ahora una dimensión especialmente útil cuando se trabaja con datos económicos, comerciales y administrativos: la **concentración**.

La idea central es simple. No basta con saber cuánto venden, en promedio, los productos o los outlets; también interesa conocer **cómo se reparte** el total de ventas entre categorías, tipos de tienda o establecimientos concretos. Dos escenarios pueden tener la misma venta total y la misma media, pero diferir radicalmente en su estructura interna: en uno las ventas pueden estar distribuidas de manera relativamente equilibrada, mientras que en otro una pequeña fracción de categorías o sucursales puede concentrar una porción muy grande del ingreso total.

Desde una perspectiva de EDA, las medidas y visualizaciones de concentración ayudan a responder preguntas como las siguientes:

- ¿Qué proporción del ingreso total proviene de unas cuantas categorías?
- ¿Los outlets contribuyen de forma equilibrada al volumen de ventas o existe dominancia de unos pocos?
- ¿Se observa una estructura tipo **Pareto**, donde una fracción pequeña de elementos explica gran parte del fenómeno?

En este capítulo se desarrollan medidas descriptivas y representaciones gráficas para estudiar la concentración de las ventas en **Big Mart Sales**, concluyendo con un mini-dashboard de concentración que después se integrará al dashboard global del libro.

Definición operativa. La concentración cuantifica si pocas categorías u outlets reúnen una fracción grande de las ventas totales (Lorenz 1905; Gini 1912).

Preparación del entorno

Trabajaremos con el siguiente dataset BigMart, usando Python, pandas, NumPy y Plotly.

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614

Como en capítulos anteriores, conviene revisar que las variables categóricas relevantes estén limpias y listas para agruparse.

	Item_Identifier	Item_Type	Outlet_Identifier	Outlet_Type	Outlet_Location_Type
0	FDA15	Dairy	OUT049	Supermarket Type1	Tier 1
1	DRC01	Soft Drinks	OUT018	Supermarket Type2	Tier 3
2	FDN15	Meat	OUT049	Supermarket Type1	Tier 1
3	FDX07	Fruits and Vegetables	OUT010	Grocery Store	Tier 3
4	NCD19	Household	OUT013	Supermarket Type1	Tier 3

Fundamento conceptual

Participación relativa

Sea x_i el total de ventas asociado a la categoría o unidad i. La participación relativa de esa unidad respecto del total se define por

$$p_i = \frac{x_i}{\sum_{j=1}^{n} x_j}.$$

Esta cantidad permite medir el peso de cada categoría en el total agregado. En términos porcentuales:

$$p_i^{(\%)} = 100 \cdot p_i.$$

Participación acumulada

Si ordenamos las unidades de mayor a menor según su contribución, la participación acumulada hasta la posición k está dada por

$$P_k = \sum_{i=1}^{k} p_{(i)},$$

donde $p_{(i)}$ representa la participación de la i-ésima unidad en el orden descendente.

Esta medida es la base del análisis de **Pareto**, que permite estudiar cuántas categorías explican cierto porcentaje del total.

Curva de concentración

Otra idea útil consiste en comparar la acumulación observada con una situación ideal de reparto uniforme. Si unas pocas unidades concentran gran parte del total, la curva acumulada crecerá rápidamente al inicio y se separará de la diagonal de reparto uniforme.

Aunque en cursos introductorios puede evitarse una formalización excesiva, esta representación es la antesala conceptual de herramientas más avanzadas como la **curva de Lorenz** y el **índice de Gini**.

Concentración de ventas por categoría de producto

Comenzamos agregando la venta total por tipo de producto.

	Item_Type	Item_Outlet_Sales	share	cum_share	share_pct	cum_share_pct
0	Fruits and Vegetables	2.820060e+06	0.151688	0.151688	15.168849	15.168849
1	Snack Foods	2.732786e+06	0.146994	0.298683	14.699412	29.868261
2	Household	2.055494e+06	0.110563	0.409246	11.056317	40.924578
3	Frozen Foods	1.825735e+06	0.098205	0.507450	9.820464	50.745042
4	Dairy	1.522594e+06	0.081899	0.589349	8.189897	58.934939
5	Canned	1.444151e+06	0.077680	0.667029	7.767962	66.702901
6	Baking Goods	1.265525e+06	0.068071	0.735100	6.807148	73.510048
7	Health and Hygiene	1.045200e+06	0.056220	0.791321	5.622038	79.132086
8	Meat	9.175656e+05	0.049355	0.840676	4.935503	84.067590
9	Soft Drinks	8.928977e+05	0.048028	0.888704	4.802817	88.870407

Ranking de contribución

Una primera lectura descriptiva consiste en ordenar las categorías por ventas y observar su peso relativo.

Este gráfico responde a una pregunta básica: **qué categorías pesan más en la estructura total de ventas**. En un contexto de negocio, esto ayuda a distinguir líneas estratégicas, categorías troncales y segmentos con menor incidencia.

Gráfico de Pareto

El gráfico de Pareto combina barras de contribución individual con una curva acumulada. Es una de las formas más didácticas de introducir la idea de concentración.

Interpretación

Cuando la curva acumulada crece muy rápido al inicio, ello indica que unas pocas categorías concentran una gran parte de las ventas. Si, en cambio, la acumulación es más gradual, la estructura es relativamente más equilibrada.

Una buena práctica pedagógica consiste en cuantificar preguntas como:

- ¿Cuántas categorías explican el 50 % de las ventas?
- ¿Cuántas categorías explican el 80 %?
- ¿Qué fracción del total se concentra en las tres categorías principales?

	Indicador	Valor
0	Categorías para alcanzar 50 % de ventas	4.00
1	Categorías para alcanzar 80 % de ventas	9.00
2	Participación acumulada de las 3 principales (%)	40.92

Concentración de ventas por outlet

La concentración también puede analizarse desde el punto de vista de las tiendas o outlets. Esta mirada es importante porque distingue entre un portafolio equilibrado de establecimientos y una estructura dominada por unas pocas unidades.

	Outlet_Identifier	Item_Outlet_Sales	share	cum_share	share_pct	cum_share_pct
0	OUT027	3.453926e+06	0.185784	0.185784	18.578359	18.578359
1	OUT035	2.268123e+06	0.122000	0.307784	12.200030	30.778389
2	OUT049	2.183970e+06	0.117474	0.425258	11.747378	42.525768
3	OUT017	2.167465e+06	0.116586	0.541844	11.658602	54.184370
4	OUT013	2.142664e+06	0.115252	0.657096	11.525196	65.709565
5	OUT046	2.118395e+06	0.113947	0.771042	11.394658	77.104223
6	OUT045	2.036725e+06	0.109554	0.880596	10.955364	88.059587
7	OUT018	1.851823e+06	0.099608	0.980204	9.960789	98.020377
8	OUT010	1.883402e+05	0.010131	0.990334	1.013065	99.033442
9	OUT019	1.796941e+05	0.009666	1.000000	0.966558	100.000000

Comentario analítico

A diferencia del análisis por tipo de producto, aquí suele haber menos unidades agregadas, de modo que la concentración puede hacerse visible con mayor claridad. Si pocas tiendas explican una fracción grande de las ventas, ello puede tener implicaciones operativas, logísticas o estratégicas.

Concentración por tipo de outlet

Además de estudiar outlets individuales, conviene analizar el reparto de ventas según tipo de tienda.

	Outlet_Type	Item_Outlet_Sales	share	cum_share	share_pct	cum_share_pct
0	Supermarket Type1	1.291734e+07	0.694812	0.694812	69.481228	69.481228
1	Supermarket Type3	3.453926e+06	0.185784	0.880596	18.578359	88.059587
2	Supermarket Type2	1.851823e+06	0.099608	0.980204	9.960789	98.020377
3	Grocery Store	3.680343e+05	0.019796	1.000000	1.979623	100.000000

Curva de concentración simplificada

Para construir una curva de concentración simple, ordenamos las unidades por contribución creciente o decreciente y comparamos la acumulación observada contra una referencia uniforme.

Aquí haremos una versión descriptiva, suficiente para fines pedagógicos.

Lectura de la curva

Cuanto más separada esté la curva respecto de la diagonal de reparto uniforme, mayor será la concentración. En un reparto perfectamente equilibrado, ambas coincidirían. En cambio, una curva muy hundida al inicio y muy empinada al final refleja que pocas categorías explican gran parte del volumen total.

Indicadores resumidos de concentración

A modo de síntesis, conviene construir una tabla con indicadores descriptivos que resuman la estructura observada.

Para participaciones s_i, el HHI es $\sum_i s_i^2$. Para valores no negativos ordenados, el Gini se calcula como $2\sum_i i x_{(i)}/(n\sum_i x_{(i)}) - (n+1)/n$. Este coeficiente no debe confundirse con Gini–Simpson.

	Ámbito	Indicador	Valor
0	Productos	Participación de la categoría líder (%)	15.1700
1	Productos	Participación acumulada del top 5 (%)	58.9300
2	Productos	Categorías necesarias para 80 % de ventas	9.0000
3	Outlets	Participación del outlet líder (%)	18.5800
4	Outlets	Participación acumulada de los 3 principales o...	42.5300
5	Productos	HHI por tipo de producto	0.0941
6	Productos	Gini por tipo de producto	0.3968

Mini-dashboard de concentración

El objetivo final del capítulo es reunir varias perspectivas en un panel compacto. En un book de Quarto, una estrategia práctica consiste en combinar tablas de KPIs con gráficos interactivos.

KPIs principales

	Métrica	Valor
0	Ventas totales	18591125.41
1	Tipos de producto	16.00
2	Outlets	10.00
3	Top 3 categorías (%)	40.92
4	Top 3 outlets (%)	42.53

Discusión final

La concentración es un componente esencial del EDA porque complementa lo aprendido en capítulos previos. Mientras la localización resume el centro, la variabilidad describe la dispersión, la forma estudia la geometría de la distribución y la heterogeneidad compara subgrupos, la concentración revela **cómo se reparte el total entre las unidades que componen el sistema**.

En **Big Mart Sales**, esta dimensión permite identificar si el negocio depende fuertemente de unas pocas categorías o unos pocos outlets, o si el ingreso se distribuye de manera más equilibrada. Esta información es valiosa tanto para fines descriptivos como para una primera interpretación operativa.

Ejercicios sugeridos

1. Construye el gráfico de Pareto para `Outlet_Location_Type` y compara su concentración con la observada en `Outlet_Type`.
2. Calcula qué porcentaje de ventas acumula el 20 % de las categorías con mayores ventas.
3. Repite el análisis utilizando el número de registros por categoría en lugar del total de ventas, y compara ambos resultados.
4. Describe, con lenguaje de negocio, si la estructura de ventas parece diversificada o concentrada.

Parte XVII.

Medidas de correlación

32. Medidas de correlación

El problema: ¿avanzan juntas dos características?

Cinco estudiantes registraron cuántas horas practicaron antes de una sesión y cuántos retos lograron resolver durante ella:

Estudiante	Horas de práctica X	Retos resueltos Y
A	1	2
B	2	1
C	3	4
D	4	3
E	5	5

Al leer la tabla aparece una tendencia: quienes practicaron más horas suelen resolver más retos, aunque el orden no sea perfecto. ¿Cómo convertir esa impresión en un número sin perder de vista la gráfica?

Una **asociación** describe cómo cambia una variable cuando cambia otra. Una correlación resume la dirección y la intensidad de un tipo específico de asociación mediante un número estandarizado:[1]

- **Pearson** pregunta qué tan bien se organiza la nube alrededor de una recta;
- **Spearman** pregunta qué tan semejante es el orden de ambas variables;
- **Kendall** pregunta qué proporción de pares conserva el mismo orden.

Estos coeficientes no son tres versiones intercambiables de "qué tan relacionadas están" las variables. Cada uno conserva información distinta.

> Una correlación aislada puede ocultar curvatura, grupos, valores extremos o errores. Debe acompañarse siempre de una gráfica: el cuarteto de Anscombe muestra conjuntos con resúmenes casi idénticos y geometrías radicalmente distintas (Anscombe 1973).

[1] **Estandarizar** una medida significa expresarla sin las unidades originales mediante una escala común. Esto facilita comparaciones, pero no elimina valores extremos ni vuelve lineal una relación.

Antes de calcular: mirar los pares

Las observaciones vienen emparejadas. La primera hora y el primer número de retos pertenecen al estudiante A; no se deben ordenar X y Y por separado antes de dibujarlas. El diagrama de dispersión conserva cada pareja (x_i, y_i).[2]

En el ejemplo, al movernos de izquierda a derecha predominan los ascensos. Hay dos retrocesos locales —A frente a B y C frente a D—, pero la dirección general es positiva.

Tres brújulas intuitivas

1. Si los puntos quedan exactamente sobre una recta creciente, los tres coeficientes deberían indicar asociación positiva perfecta.
2. Si Y crece siempre con X, pero siguiendo una curva, los órdenes pueden concordar perfectamente aunque una recta no describa bien las distancias.
3. Si Y primero baja y después sube, una relación visible puede producir correlación cercana a cero porque los dos sentidos se compensan.

Covarianza: dirección con unidades

Paso 1: Intuición con un ejemplo

Retomemos a los cinco estudiantes del ejemplo inicial. Para responder si las horas de práctica y los retos resueltos "avanzan juntos", necesitamos un punto de referencia objetivo para cada variable. El promedio o media muestral $(\bar{x}, \bar{y})$ cumple ese papel: representa el centro de gravedad de la distribución.

Si calculamos la media de ambas variables:

$$\bar{x} = \frac{1+2+3+4+5}{5} = 3 \text{ horas}, \qquad \bar{y} = \frac{2+1+4+3+5}{5} = 3 \text{ retos}.$$

Ahora podemos medir cómo se desvía cada estudiante de ese centro mediante las diferencias $(x_i - \bar{x})$ y $(y_i - \bar{y})$. ¿Qué ocurre si multiplicamos esas dos desviaciones para cada estudiante?

- **Mismo sentido (+):** Si un estudiante practicó más horas que el promedio $(x_i - \bar{x} > 0)$ y además resolvió más retos que el promedio $(y_i - \bar{y} > 0)$, el producto es $(+) \times (+) = +$. Del mismo modo, si practicó menos que el promedio $(-)$ y resolvió menos retos $(-)$, el producto es $(-) \times (-) = +$. En ambos casos, el producto es **positivo** porque ambas características se desviaron hacia el mismo lado de su respectiva media.
- **Sentidos opuestos (−):** Si un estudiante practicó más que el promedio $(+)$ pero resolvió menos retos que el promedio $(-)$, el producto es $(+) \times (-) = -$. De igual forma, si practicó menos $(-)$ pero resolvió más retos $(+)$, el producto es $(-) \times (+) = -$. Un producto **negativo** revela desacuerdo entre las desviaciones.

[2] Un **diagrama de dispersión** representa cada unidad como un punto (x_i, y_i). La posición horizontal corresponde a una variable y la vertical a la otra; por eso preserva emparejamiento, forma y observaciones inusuales.

La suma de estos productos cruzados captura el balance neto de acuerdos y desacuerdos en la muestra.

Paso 2: Cálculo a mano paso a paso

Construyamos la tabla aritmética completa para los cinco estudiantes, detallando cada desviación, su producto cruzado y sus cuadrados:

Estudiante	x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	Producto cruzado $(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$
A	1	2	$1-3=-2$	$2-3=-1$	$(-2)\times(-1)=+\mathbf{2}$	$(-2)^2=4$	$(-1)^2=1$
B	2	1	$2-3=-1$	$1-3=-2$	$(-1)\times(-2)=+\mathbf{2}$	$(-1)^2=1$	$(-2)^2=4$
C	3	4	$3-3=0$	$4-3=+1$	$(0)\times(+1)=\mathbf{0}$	$0^2=0$	$(+1)^2=1$
D	4	3	$4-3=+1$	$3-3=0$	$(+1)\times(0)=\mathbf{0}$	$(+1)^2=1$	$0^2=0$
E	5	5	$5-3=+2$	$5-3=+2$	$(+2)\times(+2)=+\mathbf{4}$	$(+2)^2=4$	$(+2)^2=4$
Suma			**0**	**0**	**8**	**10**	**10**

Observemos tres detalles del cálculo manual:

1. Las sumas de las desviaciones individuales son exactamente cero: $\sum(x_i-\bar{x}) = 0$ y $\sum(y_i-\bar{y}) = 0$. Esta es una propiedad algebraica de la media aritmética.
2. La suma de los productos cruzados es $2 + 2 + 0 + 0 + 4 = 8$.
3. Para obtener un promedio no sesgado de la covarianza poblacional a partir de la muestra, dividimos entre los grados de libertad $n - 1 = 5 - 1 = 4$ (corrección de Bessel[3]), en lugar de dividir entre n:

$$s_{XY} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{n-1} = \frac{8}{5-1} = \frac{8}{4} = 2.0 \quad \text{horas} \cdot \text{retos}.$$

El signo positivo confirma que las variables tienden a variar en la misma dirección. Sin embargo, surge un inconveniente: las unidades del resultado son híbridas (horas $\cdot$ retos). Si en lugar de horas

[3]La **corrección de Bessel** consiste en usar el divisor $n-1$ (en lugar de n) al estimar la varianza o covarianza a partir de una muestra. Como las desviaciones se calculan respecto a la media muestral $\bar{x}$ (que minimiza la suma de cuadrados en esa muestra específica) y no respecto a la verdadera media poblacional desconocida μ, dividir entre n subestima sistemáticamente la dispersión. El factor $\frac{n}{n-1}$ compensa ese grado de libertad perdido y produce un estimador insesgado (Freedman et al. 2007).

hubiéramos medido el tiempo en minutos ($X' = 60X$), cada desviación horizontal se multiplicaría por 60, la suma de productos pasaría de 8 a 480, y la covarianza sería $s_{X'Y} = 480/4 = 120$ min $\cdot$ retos. La relación entre los estudiantes es idéntica, pero el número se multiplicó por 60. Por eso la covarianza por sí sola no permite juzgar la fuerza de la asociación.

Paso 3: Formalización matemática

La **covarianza muestral** entre dos variables numéricas X y Y con n pares de observaciones se define formalmente como:

$$s_{XY} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y}). \tag{1}$$

Satisface las siguientes propiedades fundamentales:

- **Simetría:** $s_{XY} = s_{YX}$. El orden de las variables no altera el resultado.
- **Autocovarianza:** Cuando se calcula la covarianza de una variable consigo misma, se recupera su varianza muestral: $s_{XX} = s_X^2$.
- **Bilinealidad:** Si transformamos linealmente las variables mediante $U = aX + b$ y $V = cY + d$ (con $a, b, c, d \in \mathbb{R}$), entonces:

  $$s_{UV} = ac \cdot s_{XY}.$$

  Las constantes sumadas (b y d) se cancelan al centrar respecto a la media, pero los factores de escala (a y c) multiplican directamente la covarianza.

Paso 4: De la intuición geométrica al laboratorio

Existen dos maneras complementarias de visualizar geométricamente la covarianza:

1. **En el plano cartesiano de los datos (x, y) (sumar áreas de rectángulos):** Cada observación (x_i, y_i) forma con el centroide $(\bar{x}, \bar{y})$ un rectángulo cuya base mide $(x_i - \bar{x})$ y cuya altura mide $(y_i - \bar{y})$. El producto $(x_i - \bar{x})(y_i - \bar{y})$ es el **área con signo** de ese rectángulo:
   - **Áreas positivas:** Cuadrante I (arriba a la derecha) y Cuadrante III (abajo a la izquierda) respecto al centroide.
   - **Áreas negativas:** Cuadrante II (arriba a la izquierda) y Cuadrante IV (abajo a la derecha). La covarianza muestral es el **balance neto** de esas áreas dividido entre $n - 1$.
2. **En el espacio de observaciones $\mathbb{R}^n$ (producto escalar de vectores centrados):** Si agrupamos las desviaciones de toda la muestra en dos vectores de $\mathbb{R}^5$:

   $$\tilde{\mathbf{x}} = (x_1 - \bar{x}, \dots, x_5 - \bar{x}), \qquad \tilde{\mathbf{y}} = (y_1 - \bar{y}, \dots, y_5 - \bar{y}),$$

   ambos vectores viven en un hiperplano de dimensión $n - 1$ porque sus coordenadas suman cero. La suma de las áreas rectangulares es exactamente el producto escalar (producto punto) de ambos vectores:

   $$\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y}) = \tilde{\mathbf{x}} \cdot \tilde{\mathbf{y}}, \qquad s_{XY} = \frac{\tilde{\mathbf{x}} \cdot \tilde{\mathbf{y}}}{n-1}. \tag{2}$$

Magnitud estadística	Equivalente geométrico en $\mathbb{R}^n$
Varianza muestral de X	$s_X^2 = \|\tilde{\mathbf{x}}\|^2/(n-1)$
Desviación estándar de X	$s_X = \|\tilde{\mathbf{x}}\|/\sqrt{n-1}$
Covarianza muestral entre X y Y	$s_{XY} = (\tilde{\mathbf{x}} \cdot \tilde{\mathbf{y}})/(n-1)$

En el siguiente laboratorio interactivo puede explorar esta geometría: arrastre los puntos libremente o use los botones de preajuste para observar cómo los rectángulos cambian de signo y tamaño, cómo el centroide dinámico se desplaza y cómo la barra de balance visualiza la suma de áreas positivas frente a las negativas.

En HTML, un control incorpora sucesivamente los cinco rectángulos del ejemplo: $2+2+0+0+4 = 8$, por lo que $s_{XY} = 8/4 = 2$. Otros botones muestran cancelación exacta y dirección opuesta.

Pearson: estandarizar la asociación lineal

Paso 1: Intuición con un ejemplo

En la sección anterior vimos que la covarianza entre horas de práctica y retos resueltos fue $s_{XY} = 2.0$ horas $\cdot$ retos, pero al expresar el tiempo en minutos el valor se disparó a 120. ¿Cómo comparar la fuerza de una asociación entre variables con unidades totalmente distintas (como horas frente a retos, o ingresos frente a edad)?

La solución intuitiva consiste en **eliminar las unidades** dividiendo cada variable por su propia escala típica de dispersión: su desviación estándar muestral (s_X y s_Y).

Al hacer esto, cada observación (x_i, y_i) se transforma en un par de **puntuaciones estandarizadas** o puntuaciones z:[4]

$$z_{X,i} = \frac{x_i - \bar{x}}{s_X}, \qquad z_{Y,i} = \frac{y_i - \bar{y}}{s_Y}.$$

La puntuación $z_{X,i}$ responde a una pregunta puramente adimensional: *¿a cuántas desviaciones estándar por encima o por debajo de la media se encuentra este estudiante?* Al multiplicar $z_{X,i} \times z_{Y,i}$, las unidades originales se cancelan en el numerador y el denominador. El resultado es una medida pura de co-movimiento que no depende del sistema de medida.

[4] La **puntuación** z (o estandarización) transforma una variable restando su media y dividiendo por su desviación estándar: $z = (x - \bar{x})/s$. Expresa cada observación en unidades de "cuántas desviaciones estándar dista de la media", teniendo siempre media 0 y desviación estándar 1, lo que permite comparar variables con escalas o unidades físicas completamente dispares.

Paso 2: Cálculo a mano paso a paso

Podemos calcular el coeficiente de Pearson a mano de dos formas equivalentes:

Método A: Mediante las desviaciones estándar y la covarianza.

A partir de las sumas de cuadrados calculadas en la tabla de covarianza:

$$\sum_{i=1}^{5}(x_i-\bar{x})^2 = 10 \implies s_X^2 = \frac{10}{5-1} = 2.5 \implies s_X = \sqrt{2.5} \approx 1.5811 \text{ horas},$$

$$\sum_{i=1}^{5}(y_i-\bar{y})^2 = 10 \implies s_Y^2 = \frac{10}{5-1} = 2.5 \implies s_Y = \sqrt{2.5} \approx 1.5811 \text{ retos}.$$

Sustituyendo en la definición de Pearson:

$$r = \frac{s_{XY}}{s_X s_Y} = \frac{2.0}{\sqrt{2.5}\times\sqrt{2.5}} = \frac{2.0}{2.5} = 0.8. \tag{3}$$

Como el divisor $n-1$ aparece tanto en el numerador (s_{XY}) como dentro de cada desviación estándar del denominador, se cancela algebraicamente:

$$r = \frac{\sum(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum(x_i-\bar{x})^2\sum(y_i-\bar{y})^2}} = \frac{8}{\sqrt{10\times 10}} = \frac{8}{10} = 0.8. \tag{4}$$

Método B: Mediante puntuaciones estandarizadas z.

Construyamos la tabla de puntuaciones z para cada estudiante:

Estudiante	x_i	y_i	$z_{X,i} = \frac{x_i-3}{\sqrt{2.5}}$	$z_{Y,i} = \frac{y_i-3}{\sqrt{2.5}}$	Producto $z_{X,i}\cdot z_{Y,i}$
A	1	2	$\frac{-2}{\sqrt{2.5}} \approx -1.265$	$\frac{-1}{\sqrt{2.5}} \approx -0.632$	$\frac{(-2)(-1)}{2.5} = \mathbf{+0.8}$
B	2	1	$\frac{-1}{\sqrt{2.5}} \approx -0.632$	$\frac{-2}{\sqrt{2.5}} \approx -1.265$	$\frac{(-1)(-2)}{2.5} = \mathbf{+0.8}$
C	3	4	$\frac{0}{\sqrt{2.5}} = 0.000$	$\frac{+1}{\sqrt{2.5}} \approx +0.632$	$\frac{(0)(1)}{2.5} = \mathbf{0.0}$
D	4	3	$\frac{+1}{\sqrt{2.5}} \approx +0.632$	$\frac{0}{\sqrt{2.5}} = 0.000$	$\frac{(1)(0)}{2.5} = \mathbf{0.0}$
E	5	5	$\frac{+2}{\sqrt{2.5}} \approx +1.265$	$\frac{+2}{\sqrt{2.5}} \approx +1.265$	$\frac{(+2)(+2)}{2.5} = \mathbf{+1.6}$
Suma			**0.000**	**0.000**	**3.2**

Promediando la suma de productos estandarizados con los grados de libertad $n-1=4$:

$$r = \frac{1}{n-1}\sum_{i=1}^{n} z_{X,i} z_{Y,i} = \frac{3.2}{4} = 0.8.$$

Ambos procedimientos coinciden exactamente en $r = 0.8$.

Paso 3: Formalización matemática

El **coeficiente de correlación producto-momento de Pearson** muestral (Pearson 1895) se define formalmente como:

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2}}. \tag{5}$$

Posee propiedades matemáticas rigurosas:

- **Acotamiento estricto:** Por la desigualdad de Cauchy–Schwarz,[5]

  $$-1 \leq r \leq 1.$$

  El valor $r = +1$ indica una relación lineal creciente perfecta; $r = -1$ indica una relación lineal decreciente perfecta. Si alguna variable es constante, su desviación estándar es cero y r queda indeterminado.
- **Invarianza afín:** Si transformamos las variables según $U = a + bX$ y $V = c + dY$ con $b, d \neq 0$, se cumple:

  $$r_{UV} = \text{sign}(bd) \cdot r_{XY}.$$

  Sumar constantes o multiplicar por factores positivos no altera r. Si se multiplica por una constante negativa, solo se invierte el signo del coeficiente.

Paso 4: De la intuición geométrica al laboratorio

Pearson admite dos interpretaciones geométricas simultáneas:

1. **En el espacio de observaciones $\mathbb{R}^n$ (conservar solo el ángulo):** Los vectores centrados $\tilde{\mathbf{x}}, \tilde{\mathbf{y}} \in \mathbb{R}^5$ viven en el hiperplano $\sum z_i = 0$. La definición de Pearson equivale exactamente al **coseno del ángulo** θ entre ellos:

   $$r = \frac{\tilde{\mathbf{x}} \cdot \tilde{\mathbf{y}}}{\|\tilde{\mathbf{x}}\|\|\tilde{\mathbf{y}}\|} = \cos\theta. \tag{6}$$

   La covarianza mezcla la escala de los vectores con su orientación; Pearson divide las longitudes y conserva **únicamente la orientación**.

[5] La **desigualdad de Cauchy–Schwarz** establece que el valor absoluto del producto interno de dos vectores no supera el producto de sus longitudes: $|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\|\|\mathbf{v}\|$. Aplicada a los vectores de desviaciones centradas en $\mathbb{R}^n$, garantiza que el coeficiente de Pearson esté rigurosamente acotado en $[-1, 1]$.

Descomposición ortogonal y Teorema de Pitágoras: Podemos descomponer el vector $\tilde{\mathbf{y}}$ en una componente paralela a $\tilde{\mathbf{x}}$ (la proyección $\hat{\mathbf{y}}$) y un residuo perpendicular $\mathbf{e}$ ($\mathbf{e} \perp \tilde{\mathbf{x}}$):

$$\tilde{\mathbf{y}} = \hat{\mathbf{y}} + \mathbf{e}, \qquad \text{con } \hat{\mathbf{y}} = r\frac{\|\tilde{\mathbf{y}}\|}{\|\tilde{\mathbf{x}}\|}\tilde{\mathbf{x}}.$$

Por el Teorema de Pitágoras en $\mathbb{R}^n$, las normas al cuadrado se suman:

$$\|\tilde{\mathbf{y}}\|^2 = \|\hat{\mathbf{y}}\|^2 + \|\mathbf{e}\|^2.$$

Al normalizar dividiendo entre $\|\tilde{\mathbf{y}}\|^2$, surge la identidad fundamental:

$$1 = r^2 + \frac{\|\mathbf{e}\|^2}{\|\tilde{\mathbf{y}}\|^2} = r^2 + e^2.$$

El término r^2 es el **coeficiente de determinación** R^2:[6] representa la fracción de la variabilidad de $\tilde{\mathbf{y}}$ capturada por la dirección de $\tilde{\mathbf{x}}$. El término restante $e^2 = 1 - r^2$ es la varianza residual no explicada.

2. **En el espacio de datos estandarizados** (z_X, z_Y)**:** En este espacio, el centroide está en $(0, 0)$ y ambas variables tienen varianza 1. La recta de regresión de mínimos cuadrados pasa por el origen con pendiente idéntica a r ($\hat{z}_Y = rz_X$). La **elipse de covarianza** (de nivel 1σ) encierra la masa de los datos:

   - Cuando $r = 0$, la elipse es un círculo perfecto (ejes iguales).
   - A medida que $|r| \to 1$, el semieje menor de la elipse colapsa a cero y la elipse degenera en un segmento de recta sobre la diagonal.

En el siguiente laboratorio interactivo puede experimentar con ambas representaciones en paralelo: modifique el ángulo θ o la norma $\|\tilde{\mathbf{y}}\|$ para comprobar que la escala altera la covarianza pero deja intacto a r, y observe cómo el triángulo rectángulo pitagórico ($1 = r^2 + e^2$) y la elipse de covarianza reaccionan en tiempo real.

El laboratorio HTML permite variar θ entre $0°$ y $180°$ y cambiar $\|\tilde{\mathbf{y}}\|$ por separado. El primer control modifica $r = \cos\theta$; el segundo modifica la covarianza, pero no r.

Por la desigualdad de Cauchy–Schwarz,[7] $-1 \leq r \leq 1$. Los extremos $r = 1$ y $r = -1$ aparecen cuando todos los puntos están exactamente sobre una recta con pendiente positiva o negativa, respectivamente. Si alguna variable es constante, su desviación estándar es cero y r no está definido.

Qué significa y qué no significa $r = 0.8$

En estos cinco datos hay una asociación lineal positiva apreciable. El número no significa que "80 % de Y sea causado por X", ni que cada hora adicional produzca 0.8 retos. Tampoco autoriza una etiqueta universal como "fuerte" sin considerar el campo, el tamaño muestral, la incertidumbre y la gráfica.

Sumar una constante a cualquiera de las variables o cambiar sus unidades por un factor positivo no altera r. Multiplicar una de ellas por un factor negativo refleja el eje y cambia únicamente el signo.

[6] El **coeficiente de determinación** $R^2 = r^2$ representa la proporción de la varianza total de una variable que queda explicada linealmente por la otra. Geométricamente, corresponde a la razón entre la norma al cuadrado de la proyección ortogonal y la norma al cuadrado del vector original en $\mathbb{R}^n$: $\|\hat{\mathbf{y}}\|^2/\|\tilde{\mathbf{y}}\|^2$.

[7] La **desigualdad de Cauchy–Schwarz** establece que el valor absoluto del producto interno de dos vectores no supera el producto de sus longitudes: $|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\|\|\mathbf{v}\|$. Aplicada a los vectores de desviaciones centradas en $\mathbb{R}^n$, garantiza que el coeficiente de Pearson esté rigurosamente acotado en $[-1, 1]$.

Spearman: comparar órdenes

Paso 1: Intuición con un ejemplo

¿Qué ocurre si dos variables crecen siempre juntas, pero la relación no es una línea recta? Imaginemos que el rendimiento creciera de forma exponencial con las horas de práctica: $X = (1, 2, 3, 4, 5)$ y $Y = (2, 4, 8, 16, 32)$.

En ese caso, cada hora adicional produce todavía más retos resueltos que la anterior. La relación es **monótona creciente**:[8] siempre que X aumenta, Y también aumenta, sin una sola excepción. Sin embargo, si calculamos Pearson sobre esos datos, obtendremos $r < 1$ porque los puntos forman una curva y Pearson penaliza las distancias no alineadas a una recta.

Si la pregunta sustantiva no es si los puntos se acomodan sobre una recta, sino únicamente si *al aumentar una variable, la otra también aumenta*, penalizar la curvatura distorsiona la conclusión.

La idea de Charles Spearman (Spearman 1904) consiste en **ignorar las distancias métricas originales** y conservar únicamente las posiciones relativas u órdenes: reemplazamos cada dato por su **rango** $(1^\circ, 2^\circ, \dots, n^\circ)$.[9]

Paso 2: Cálculo a mano paso a paso

Retomemos los cinco estudiantes originales:

- Horas de práctica: $X = (1, 2, 3, 4, 5)$. Como ya están en orden creciente, sus rangos son directamente:
$$R_X = (1, 2, 3, 4, 5).$$
- Retos resueltos: $Y = (2, 1, 4, 3, 5)$. Ordenemos los retos de menor a mayor: el menor es 1 (estudiante B $\to$ rango 1), luego 2 (estudiante A $\to$ rango 2), luego 3 (estudiante D $\to$ rango 3), luego 4 (estudiante C $\to$ rango 4) y el mayor es 5 (estudiante E $\to$ rango 5). Por tanto:
$$R_Y = (2, 1, 4, 3, 5).$$

Construyamos la tabla de diferencias entre rangos $d_i = R_{X,i} - R_{Y,i}$:

Estudiante	x_i	y_i	Rango $R_{X,i}$	Rango $R_{Y,i}$	Diferencia $d_i = R_{X,i} - R_{Y,i}$	Cuadrado d_i^2
A	1	2	1	2	$1-2=-1$	$(-1)^2 = \mathbf{1}$
B	2	1	2	1	$2-1=+1$	$(+1)^2 = \mathbf{1}$
C	3	4	3	4	$3-4=-1$	$(-1)^2 = \mathbf{1}$
D	4	3	4	3	$4-3=+1$	$(+1)^2 = \mathbf{1}$
E	5	5	5	5	$5-5=0$	$0^2 = \mathbf{0}$
Suma					**0**	**4**

[8] Una relación es **monótona creciente** si Y nunca disminuye cuando X aumenta; es monótona decreciente si nunca aumenta. Puede ser curva y seguir conservando el orden de manera perfecta.

[9] El **rango** de una observación es su posición ordinal al clasificar los valores de menor a mayor $(1^\circ, 2^\circ, \dots, n^\circ)$. Con empates se asigna a todos los valores empatados el promedio de las posiciones que habrían ocupado.

Observemos dos detalles del cálculo manual:

1. La suma de las diferencias es siempre cero: $\sum d_i = (-1) + 1 + (-1) + 1 + 0 = 0$.
2. La suma de las diferencias al cuadrado es $\sum d_i^2 = 1 + 1 + 1 + 1 + 0 = 4$.

Cuando **no hay empates**, podemos usar la fórmula abreviada de Spearman:

$$\rho_s = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2 - 1)}. \tag{7}$$

Sustituyendo $n = 5$ y $\sum d_i^2 = 4$:

$$\rho_s = 1 - \frac{6(4)}{5(5^2 - 1)} = 1 - \frac{24}{5(24)} = 1 - \frac{24}{120} = 1 - 0.2 = 0.8.$$

¿Qué ocurre si hay empates? Si dos o más observaciones comparten el mismo valor en una variable, se les asigna el **rango promedio** de las posiciones que habrían ocupado. Por ejemplo, si dos estudiantes empatan en el segundo y tercer puesto, ambos reciben el rango $\frac{2+3}{2} = 2.5$. En ese caso, la varianza de los rangos disminuye y la fórmula abreviada (Ec. 7) deja de ser exacta; debe aplicarse la correlación de Pearson habitual directamente sobre las columnas de rangos promedio.

Paso 3: Formalización matemática

El **coeficiente de correlación de Spearman** ρ_s se define formalmente como el coeficiente de correlación de Pearson aplicado a los vectores de rangos:

$$\rho_s = \operatorname{corr}(R_X, R_Y). \tag{8}$$

Deducción de la fórmula abreviada. Sin empates, tanto R_X como R_Y son permutaciones del conjunto de enteros $\{1, 2, \dots, n\}$. La media de cualquier permutación es fija: $\bar{R} = \frac{n+1}{2}$, y la suma de cuadrados respecto a la media es una constante universal:

$$\sum_{i=1}^{n} (R_i - \bar{R})^2 = \frac{n(n^2 - 1)}{12}.$$

Desarrollando la suma de diferencias al cuadrado:

$$\sum d_i^2 = \sum (R_{X,i} - R_{Y,i})^2 = \sum (R_{X,i} - \bar{R})^2 + \sum (R_{Y,i} - \bar{R})^2 - 2 \sum (R_{X,i} - \bar{R})(R_{Y,i} - \bar{R}).$$

Al sustituir las sumas de cuadrados constantes y despejar el término de covarianza, se obtiene directamente la constante 6 y el denominador $n(n^2 - 1)$.

- **Invarianza monótona:** ρ_s es invariante ante cualquier transformación estrictamente creciente $g(X)$ y $h(Y)$ (por ejemplo, logaritmos o potencias impares), ya que $R_{g(X)} = R_X$.
- **Límites:** $-1 \leq \rho_s \leq 1$. Vale $+1$ cuando el orden de ambas variables coincide perfectamente, y -1 cuando una está en orden exactamente inverso al de la otra.

Paso 4: De la intuición geométrica al laboratorio

Spearman combina dos perspectivas geométricas fundamentales:

1. **En la cuadrícula de rangos $\{1, \dots, n\}^2$ (sumar cuadrados geométricos):** Los pares de rangos $(R_{X,i}, R_{Y,i})$ habitan una cuadrícula rígida de enteros. La recta diagonal $R_Y = R_X$ representa el orden perfecto. Cada término $d_i^2 = (R_{X,i} - R_{Y,i})^2$ es literalmente el **área de un cuadrado geométrico** de lado $|d_i|$ trazado entre el punto y la diagonal. Cuanto menor sea el área total acumulada por estos cuadrados, mayor será ρ_s.
2. **En el espacio de permutaciones sobre una esfera $\mathbb{R}^n$ (el permutoedro[10] y la Ley de Cosenos):** Al centrar los rangos restando $\bar{R} = 3$, todos los vectores de rangos sin empates tienen exactamente la misma longitud fija:

$$\|\widetilde{\mathbf{R}}\| = R_0 = \sqrt{\frac{n(n^2-1)}{12}} = \sqrt{\frac{5(24)}{12}} = \sqrt{10} \approx 3.162.$$

   Las $5! = 120$ permutaciones posibles son los vértices de un polítopo convexo (el permutoedro) que descansan exactamente sobre una hiperesfera de radio $\sqrt{10}$. La fórmula abreviada de Spearman es literalmente la **Ley de Cosenos** sobre esta esfera:

$$\|\mathbf{d}\|^2 = \sum d_i^2 = 2R_0^2(1 - \cos\varphi) = 2(10)(1 - \rho_s) = 20(1 - \rho_s).$$

   Cuando se introducen empates mediante rangos promedio, el vector **se hunde hacia el interior de la esfera** ($\|\widetilde{\mathbf{R}}\| < \sqrt{10}$), reduciendo la varianza disponible.

En el siguiente laboratorio interactivo puede manipular las permutaciones de rangos, verificar las áreas cuadradas d_i^2 en la cuadrícula y ver cómo el vector de rangos recorre la esfera o colapsa al interior cuando se activan los empates.

La versión HTML recorre las $5! = 120$ permutaciones de rangos. Sin empates, $\|\widetilde{\mathbf{R}}_Y\| = \sqrt{10}$; un botón introduce rangos promedio y muestra cómo el vector se desplaza al interior de la esfera.

Kendall: contar concordancias

Paso 1: Intuición con un ejemplo

Tanto Pearson como Spearman calculan distancias: Pearson usa desviaciones numéricas métricas y Spearman usa desviaciones numéricas entre enteros d_i^2. Maurice Kendall (Kendall 1938) propuso una alternativa todavía más fundamental, robusta y libre de supuestos métricos:

Si enfrentamos a las observaciones en un "torneo" por parejas, de dos en dos, ¿coinciden en quién está por delante?

Tomemos a dos estudiantes cualesquiera del grupo, por ejemplo A y B:

- Estudiante A: 1 hora de práctica, 2 retos resueltos.
- Estudiante B: 2 horas de práctica, 1 reto resuelto.

[10] El **permutoedro** de orden n es un polítopo convexo $(n-1)$-dimensional en $\mathbb{R}^n$ cuyos vértices son todas las $n!$ permutaciones posibles del vector $(1, 2, \dots, n)$. Todos sus vértices descansan sobre una misma hiperesfera de radio $\sqrt{n(n^2-1)/12}$ centrada en el vector diagonal $\mathbf{1}$.

B practicó más horas que A ($2 > 1$), pero resolvió menos retos que A ($1 < 2$). El orden relativo se invirtió entre ambas variables. Decimos que el par (A, B) es **discordante**.

Ahora comparemos a A con C:

- Estudiante A: 1 hora de práctica, 2 retos resueltos.
- Estudiante C: 3 horas de práctica, 4 retos resueltos.

C practicó más horas que A ($3 > 1$) y también resolvió más retos ($4 > 2$). El orden relativo se mantuvo idéntico. Decimos que el par (A, C) es **concordante**.

La correlación de Kendall responde a una pregunta probabilística elemental: *al elegir dos observaciones al azar, ¿qué tan más probable es que sus órdenes concuerden en lugar de discrepar?*

Paso 2: Cálculo a mano paso a paso

Con $n = 5$ estudiantes, el número total de parejas posibles es:

$$\binom{5}{2} = \frac{5 \times 4}{2} = 10 \text{ pares.}$$

Como la variable X (horas) ya está ordenada de menor a mayor ($A < B < C < D < E$), en cada par (i, j) con $i < j$ la variable X siempre crece ($x_j - x_i > 0$). Por tanto, la concordancia depende únicamente del signo de $y_j - y_i$.

Evaluemos a mano los 10 pares posibles en una tabla exhaustiva:

Par (i, j)	Horas (x_i, x_j)	Retos (y_i, y_j)	$x_j - x_i$	$y_j - y_i$	Signo del producto $(x_j - x_i)(y_j - y_i)$	Clasificación
(A, B)	(1, 2)	(2, 1)	$+1$	-1	$(-)$	**Discordante (−)**
(A, C)	(1, 3)	(2, 4)	$+2$	$+2$	$(+)$	**Concordante (+)**
(A, D)	(1, 4)	(2, 3)	$+3$	$+1$	$(+)$	**Concordante (+)**
(A, E)	(1, 5)	(2, 5)	$+4$	$+3$	$(+)$	**Concordante (+)**
(B, C)	(2, 3)	(1, 4)	$+1$	$+3$	$(+)$	**Concordante (+)**
(B, D)	(2, 4)	(1, 3)	$+2$	$+2$	$(+)$	**Concordante (+)**
(B, E)	(2, 5)	(1, 5)	$+3$	$+4$	$(+)$	**Concordante (+)**
(C, D)	(3, 4)	(4, 3)	$+1$	-1	$(-)$	**Discordante (−)**

Par (i, j)	Horas (x_i, x_j)	Retos (y_i, y_j)	$x_j - x_i$	$y_j - y_i$	Signo del producto $(x_j - x_i)(y_j - y_i)$	Clasificación
(C, E)	(3, 5)	(4, 5)	+2	+1	(+)	**Concordante (+)**
(D, E)	(4, 5)	(3, 5)	+1	+2	(+)	**Concordante (+)**

Hagamos el balance final:

- Pares concordantes (C): **8**
- Pares discordantes (D): **2**
- Pares empatados (T): **0**
- Total de pares evaluados: $C + D = 10$

Calculamos **Kendall tau-a** (τ_a), válida cuando no hay empates:

$$\tau_a = \frac{C - D}{\binom{n}{2}} = \frac{8 - 2}{10} = \frac{6}{10} = 0.6. \tag{9}$$

Interpretación probabilística rigurosa. Si seleccionamos al azar dos estudiantes de esta muestra:

$$P(\text{concordancia}) = \frac{8}{10} = 0.8, \qquad P(\text{discordancia}) = \frac{2}{10} = 0.2.$$

El coeficiente τ_a es la diferencia exacta entre ambas probabilidades:

$$\tau_a = P(\text{concordancia}) - P(\text{discordancia}) = 0.8 - 0.2 = 0.6.$$

Kendall no mide distancias entre valores ni curvatura; mide **concordancia ordinal**.[11] Su escala representa una diferencia de probabilidades y por ello no debe esperarse que coincida numéricamente con el 0.8 de Pearson o Spearman. De hecho, bajo normalidad bivariada teórica se cumple $\tau = \frac{2}{\pi} \arcsin r$, lo que para $r = 0.8$ predice $\tau \approx 0.59$. Un valor $\tau = 0.6$ refleja exactamente la misma intensidad de asociación que $r = 0.8$.

Paso 3: Formalización matemática

Formalmente, para cada par de observaciones (i, j) con $i < j$, definimos el producto de signos:

$$h_{ij} = \operatorname{sgn}(x_j - x_i) \cdot \operatorname{sgn}(y_j - y_i),$$

donde $\operatorname{sgn}(u)$ es la función signo ($+1$ si $u > 0$, -1 si $u < 0$, y 0 si $u = 0$). El par es:

- **Concordante** si $h_{ij} = +1$;

[11] Una variable **ordinal** posee categorías con orden definido, pero sin distancias numéricas necesariamente comparables ni uniformes (como bajo, medio y alto). La concordancia evalúa el orden sin exigir que los intervalos sean iguales.

- **Discordante** si $h_{ij} = -1$ (indica una inversión de orden[12]);
- **Empatado** si $h_{ij} = 0$.

El coeficiente **Kendall tau-a** es el promedio de estos signos:

$$\tau_a = \frac{1}{\binom{n}{2}} \sum_{1 \le i < j \le n} h_{ij} = \frac{C - D}{\binom{n}{2}}. \tag{10}$$

Corrección por empates: Kendall tau-b. Cuando existen valores repetidos en X o en Y, algunos pares producen $h_{ij} = 0$. Para evitar que los empates se penalicen injustamente como si fueran discordancias, se utiliza comúnmente **Kendall tau-b**:[13]

$$\tau_b = \frac{C - D}{\sqrt{(C + D + T_X)(C + D + T_Y)}}, \tag{11}$$

donde T_X es el número de pares empatados únicamente en X y T_Y es el número de pares empatados únicamente en Y. Si no hay empates, $T_X = T_Y = 0$ y τ_b coincide con τ_a.

Paso 4: De la intuición geométrica al laboratorio

La geometría de Kendall es eminentemente combinatoria y gráfica:

1. **Pendientes de cuerdas en el grafo completo K_n:** En el diagrama de dispersión con X creciente, cada par (i, j) define una **cuerda** o segmento de recta que une los puntos (x_i, y_i) y (x_j, y_j). La pendiente de esa cuerda es:

   $$m_{ij} = \frac{y_j - y_i}{x_j - x_i}.$$

   Como $x_j > x_i$, el denominador siempre es positivo. Por tanto:

   - **Pendiente positiva ($m_{ij} > 0$):** la cuerda asciende hacia la derecha (verde) $\leftrightarrow$ par **concordante**.
   - **Pendiente negativa ($m_{ij} < 0$):** la cuerda desciende hacia la derecha (roja) $\leftrightarrow$ par **discordante** (inversión de orden).
   - **Pendiente cero ($m_{ij} = 0$):** la cuerda es horizontal (gris) $\leftrightarrow$ **empate** en Y.

2. **La matriz triangular de signos:** Las $\binom{n}{2}$ cuerdas corresponden a las celdas de una matriz triangular superior de tamaño $n \times n$. Cada celda contiene el signo $+1$, -1 o 0. Kendall τ es el promedio global de esa matriz.

[12] Una **inversión** en una secuencia es un par de elementos que aparecen en orden inverso al orden natural. La distancia de inversión (o distancia de Kendall tau) mide el número mínimo de intercambios de elementos adyacentes necesarios para transformar una ordenación en otra.

[13] **Kendall tau-b** ajusta el cálculo de concordancia cuando existen empates en los datos. Modifica el denominador calculando la media geométrica del número de pares no empatados en X y en Y, garantizando que el coeficiente permanezca en el intervalo $[-1, 1]$ aun en presencia de valores idénticos repetidos (Kendall 1938).

A diferencia de Pearson y Spearman, Kendall no calcula promedios globales de desviaciones ni ángulos en $\mathbb{R}^n$: agrega $\binom{n}{2}$ decisiones locales independientes sobre las orientaciones de las cuerdas.

En el siguiente laboratorio interactivo puede manipular las alturas de los puntos: observe cómo cada cuerda rota en tiempo real, cruza la horizontal cambiando de verde a rojo cuando se produce una inversión, y cómo cada celda de la matriz de signos refleja instantáneamente la orientación geométrica de su cuerda.

En HTML puede seleccionarse cada uno de los diez segmentos. Para el ejemplo hay ocho ascendentes y dos descendentes, de modo que $\tau_a = (8 - 2)/10 = 0.6$.

Laboratorio integrador: cuatro geometrías, una muestra

Después de estudiar cada geometría por separado, conserve $X = (1, 2, 3, 4, 5)$ y mueva los retos en las cuatro vistas simultáneas. Observe qué cambia continuamente —covarianza y Pearson— y qué cambia solo cuando se altera el orden —Spearman y Kendall—.

La versión HTML contiene un laboratorio que modifica los cinco valores de Y. Además de los coeficientes, muestra los rectángulos de covarianza, el ángulo de Pearson, el ángulo entre rangos de Spearman y los segmentos de Kendall. Algunos escenarios de referencia son:

Escenario	Y	Pearson	Spearman	Kendall	Geometría principal
Ejemplo manual	$(2, 1, 4, 3, 5)$	0.8	0.8	0.6	Dos inversiones locales
Mismo orden, otra escala	$(0, 1, 2, 3, 8)$	0.91	1	1	Rangos paralelos; distancias no
Vectores ortogonales	$(1, 5, 4, 3, 2)$	0	0	-0.2	Pearson y Spearman a 90°
Perfecta decreciente	$(5, 4, 3, 2, 1)$	-1	-1	-1	Vectores opuestos
Forma de U	$(4, 1, 0, 1, 4)$	0	0	0	Estructura no monótona con empates

Los casos con valor cero recuerdan que ortogonalidad o equilibrio ordinal no equivalen a independencia.

Lineal, monótona y no monótona

Los siguientes diagramas Plotly mantienen datos fijos para que la comparación sea reproducible. Pase el cursor sobre los puntos para consultar valores y coeficientes.

La relación lineal produce valores altos en Pearson y Spearman. En la curva creciente, Spearman conserva mejor la asociación porque usa orden. En la parábola, los descensos de la mitad izquierda compensan los ascensos de la mitad derecha: una correlación cercana a cero no significa ausencia de estructura.

Cero no significa independencia. Si X toma valores simétricos alrededor de cero y $Y = X^2$, entonces Y queda completamente determinada por X, pero $\mathrm{Cov}(X, Y) = 0$ porque los productos impares se cancelan. La independencia[14] implica correlación cero cuando existen los momentos necesarios; el recíproco es falso.

Sensibilidad a una observación extrema

Paso 1: Intuición con un ejemplo

¿Qué sucede si a un grupo de observaciones con clara tendencia positiva agregamos un solo estudiante atípico? Imaginemos que alguien afirma haber practicado 20 horas pero resolvió 0 retos (o viceversa).

En el cálculo de Pearson, esa observación se ubica sumamente lejos del centroide $(\bar{x}, \bar{y})$ tanto en el eje horizontal como en el vertical. Su producto cruzado $(x_i - \bar{x})(y_i - \bar{y})$ resulta en un número negativo enorme que puede sobrepasar la suma de todos los demás estudiantes juntos, arrastrando la recta de regresión y volteando el signo de la correlación de positivo a negativo.

¿Qué ocurre con ese mismo estudiante en Spearman y Kendall? Al asignarle rangos, sus 20 horas simplemente ocupan la última posición ($R_X = n$). Si en lugar de 20 horas hubiera reportado 200 o 2000 horas, ¡su rango sigue siendo exactamente n! No puede aumentar más.

Decimos que los métodos basados en rangos experimentan una **saturación de frontera**: una vez que una observación ocupa el extremo de la cuadrícula $\{1, \dots, n\}^2$, alejarla a distancias métricas arbitrariamente grandes no modifica su posición ni altera los rangos de las demás observaciones.

Paso 2: Análisis analítico: apalancamiento continuo vs frontera de rangos

En una regresión lineal simple, el **apalancamiento** (*leverage*) h_{ii}[15] de la i-ésima observación se calcula como:

$$h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_{k=1}^{n} (x_k - \bar{x})^2}. \tag{12}$$

El apalancamiento mide qué tan aislada se encuentra una observación en el eje horizontal respecto a la media de los datos:

[14] Dos variables son **independientes** si conocer una no altera en absoluto la distribución probabilística de la otra: $P(X \le x, Y \le y) = P(X \le x)P(Y \le y)$. Es una propiedad sobre toda la distribución conjunta, mucho más fuerte que tener covarianza o correlación cero.

[15] El **apalancamiento** (*leverage*) $h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum (x_k - \bar{x})^2}$ describe qué tan inusual es una observación en el eje horizontal. Un punto muy alejado horizontalmente ejerce un gran efecto de palanca sobre la pendiente de la recta de regresión ajustada.

- Si $x_i \approx \bar{x}$, el apalancamiento es mínimo ($h_{ii} \approx 1/n$).
- A medida que $(x_i - \bar{x})^2$ crece, h_{ii} se aproxima a 1.
- Una observación con h_{ii} alto actúa como un **brazo de palanca**: la recta de mínimos cuadrados se ve obligada a pivotar sobre el centroide para pasar cerca de ese punto.

En contraste, en el espacio discreto de rangos $\{1, \dots, n\}^2$, la distancia máxima posible entre dos observaciones en cualquier eje está rígidamente acotada: $|R_{X,i} - R_{X,j}| \leq n-1$. No existe la posibilidad de que una sola observación posea una distancia desproporcionada respecto a las demás.

Paso 3: Formalización matemática: punto de ruptura

El concepto teórico que cuantifica esta vulnerabilidad es el **punto de ruptura** (*breakdown point* (Huber y Ronchetti 2009)):[16]

- **Pearson tiene un punto de ruptura asintótico de 0 %:** Basta una sola observación atípica ($1/n \to 0$ cuando n crece) colocada a una distancia suficientemente extrema para forzar al coeficiente r a tomar prácticamente cualquier valor deseado en el intervalo $[-1, 1]$.
- **Spearman y Kendall son estimadores resistentes:** Al basarse en la ordenación de los datos, su función de influencia frente a la magnitud del alejamiento permanece acotada. Un punto extremo solo puede alterar los órdenes hasta alcanzar la frontera del conjunto $\{1, \dots, n\}^2$.

El siguiente experimento numérico ilustra esta diferencia: parte de una nube positiva de 100 observaciones, introduce un punto extremo $(a, -a)$ y aumenta gradualmente su magnitud a:

La gráfica no demuestra que Spearman o Kendall sean inmunes a extremos: un punto atípico sí puede alterar pares y posiciones relativas. Muestra una distinción geométrica fundamental: Pearson reacciona de forma continua y no acotada a la **magnitud métrica** del alejamiento; los métodos ordinales se detienen al llegar al **borde de la ordenación**.

Paso 4: De la intuición geométrica al laboratorio

El laboratorio interactivo siguiente permite experimentar directamente con este mecanismo. La muestra base (puntos verdes) mantiene una asociación positiva. Mueva o arrastre libremente el octavo punto (naranja):

- **A la izquierda (espacio de datos continuo):** Observe cómo la distancia al centroide incrementa el apalancamiento h_{ii}, obligando a la recta de regresión a pivotar e invirtiendo el signo de Pearson r.
- **A la derecha (espacio discreto de rangos $\{1, \dots, 8\}^2$):** Observe cómo, al alejar el punto extremo, este **se congela en el borde de la cuadrícula**: su rango queda fijo en $(8, 1)$ y los coeficientes de Spearman ρ_s y Kendall τ_b permanecen estables a pesar de que el punto continúe alejándose en el plano continuo.

[16] El **punto de ruptura** (*breakdown point*) es la proporción mínima de observaciones contaminadas o arbitrarias que pueden alterar indefinidamente el valor de un estimador (Huber y Ronchetti 2009). Pearson tiene un punto de ruptura asintótico de 0 % (basta un único valor extremo para distorsionar la correlación), mientras que los estadísticos de rangos gozan de mayor estabilidad frente a perturbaciones métricas no acotadas.

Laboratorio de influencia: mover un punto extremo

La versión HTML permite mover un octavo punto alrededor de siete observaciones con tendencia positiva. Un extremo alineado refuerza Pearson; uno colocado en la dirección contraria puede invertirlo. Spearman y Kendall cambian cuando el nuevo punto modifica órdenes y concordancias, pero no por seguir aumentando su distancia después de ocupar un rango extremo.

El cuarteto de Anscombe: mismo coeficiente, distinta historia

Los cuatro conjuntos siguientes tienen once pares y comparten, salvo redondeo, media de X, media de Y, varianzas, recta ajustada y correlación de Pearson. Sus diagramas cuentan historias distintas (Anscombe 1973).

- La serie 1 se aproxima a una relación lineal con ruido.
- La serie 2 presenta curvatura: una recta omite estructura sistemática.
- La serie 3 está dominada por una observación verticalmente alejada.
- La serie 4 concentra casi todos los valores de X y depende de un punto con gran apalancamiento.

El coeficiente es un resumen del conjunto completo; no certifica que la recta sea un modelo adecuado para cada observación.

Correlación no implica causalidad

Aunque el diagrama sea claro y el coeficiente grande, una asociación observacional no identifica por sí sola qué ocurriría al intervenir sobre X.[17] Entre las explicaciones posibles están:

- **causalidad directa:** cambiar X modifica Y;
- **causalidad inversa:** Y influye sobre X;
- **confusión:** una tercera variable afecta a ambas;[18]
- **selección:** la forma de incluir casos crea o modifica la asociación;
- **tendencia común:** dos series crecen con el tiempo sin un mecanismo directo;
- **azar y multiplicidad:** al examinar muchos pares, algunos parecen asociados solo por fluctuación muestral.

En el ejemplo de práctica, motivación previa, experiencia, dificultad de los retos y forma de seleccionar estudiantes podrían explicar parte de la asociación. Para sostener una afirmación causal se necesita un diseño o supuestos causales adicionales, no un umbral de correlación.

[17] Una afirmación **causal** describe el efecto de intervenir: qué cambiaría en Y si se modificara activamente X manteniendo comparables las demás condiciones relevantes.

[18] Un **confusor** es una variable externa relacionada tanto con la variable predictora como con el resultado que puede crear, ocultar o invertir la asociación observada si no se controla adecuadamente.

Matrices de correlación

Con p variables, una matriz de correlación contiene el coeficiente de cada par:

$$R = (r_{jk})_{p\times p}, \qquad r_{jj} = 1, \qquad r_{jk} = r_{kj}. \tag{10}$$

Es una vista compacta, pero no reemplaza los diagramas relevantes. Puede ocultar curvatura, subgrupos, escalas temporales y tamaños muestrales distintos.

Los valores faltantes exigen declarar una estrategia:[19]

- con **casos completos**, todas las celdas usan las mismas filas, pero pueden descartarse muchas observaciones;
- con **eliminación por pares**, cada celda aprovecha las filas disponibles para ese par, pero los tamaños muestrales cambian y la matriz conjunta puede perder propiedades algebraicas esperadas.

> Antes de interpretar un mapa de calor, reporte el número de observaciones por celda, revise variables constantes o casi constantes y abra diagramas de dispersión para los pares sustantivamente importantes. Un bloque de colores no permite diagnosticar valores extremos ni mezclas de grupos.

Formalización poblacional

Sean X y Y variables aleatorias con medias μ_X, μ_Y y varianzas finitas positivas. La covarianza poblacional es

$$\operatorname{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)] = \mathbb{E}[XY] - \mu_X\mu_Y. \tag{11}$$

La correlación poblacional de Pearson es

$$\rho_{XY} = \frac{\operatorname{Cov}(X, Y)}{\sigma_X\sigma_Y}. \tag{12}$$

Si $U = a + bX$ y $V = c + dY$, con $b, d \neq 0$, entonces

$$\operatorname{corr}(U, V) = \operatorname{sign}(bd)\operatorname{corr}(X, Y).$$

Esta invariancia hace comparable el coeficiente entre unidades, pero también explica una limitación: Pearson resume la parte lineal estandarizada y no la pendiente física. Para cuantificar cuánto cambia Y por una unidad de X se necesita una recta de regresión y sus supuestos, no solo r.

[19] Un valor **faltante** indica que una medición no está disponible. El mecanismo que produjo la ausencia importa: eliminar filas completas puede sesgar la asociación si la probabilidad de faltar depende de variables relevantes del estudio.

La correlación muestral es un **estimador**[20] de la correlación poblacional. Cinco pares pueden producir $r = 0.8$ sin que la población tenga exactamente ese valor. La precisión depende de n, del diseño y de la forma de la distribución; reportar solo la estimación puntual oculta esa incertidumbre.

Implementación reproducible

La función siguiente devuelve los tres coeficientes, sus valores p y el número de pares utilizados. El valor p responde a una hipótesis nula bajo supuestos específicos; no mide el tamaño ni la importancia práctica de la asociación.[21]

Para una matriz, conviene acompañar cada coeficiente con el número efectivo de pares:

Del cálculo al programa

Pearson acumula desviaciones, productos y cuadrados. Spearman transforma primero ambas columnas en rangos promedio y reutiliza Pearson. Una implementación directa de Kendall compara $\binom{n}{2}$ pares, por lo que su costo es cuadrático; bibliotecas estadísticas emplean algoritmos más eficientes. En todos los casos debe preservarse el emparejamiento original de las filas.

Guía de decisión

Pregunta	Medida principal	Gráfica mínima	Precaución
¿La nube se organiza alrededor de una recta?	Pearson	Dispersión con tendencia	Sensible a extremos; cero no descarta curvatura.
¿Una variable crece o decrece consistentemente con la otra?	Spearman	Dispersión y rangos	Los empates requieren rangos promedio.
¿Qué tan frecuentemente concuerdan los órdenes de pares?	Kendall tau-b	Dispersión o tabla ordinal	Su escala no coincide numéricamente con Pearson.
¿Existe una relación no monótona?	Ninguna de las tres basta	Dispersión y suavizado	Considerar modelos no lineales.
¿Se comparan muchas variables?	Matriz más gráficos seleccionados	Mapa de calor y dispersiones	Reportar n, faltantes y multiplicidad.
¿Se busca una explicación causal?	La correlación es solo exploratoria	Diagrama causal y análisis del diseño	Requiere supuestos o intervención adicionales.

[20] Un **estimador** es una regla matemática calculada a partir de los datos de una muestra para aproximar un parámetro poblacional desconocido. Su valor numérico varía de una muestra a otra y, por tanto, está sujeto a error de muestreo e incertidumbre.

[21] Un **valor** p es la probabilidad, bajo el supuesto de que una hipótesis nula sea cierta y se cumplan las condiciones del modelo, de obtener un resultado muestral al menos tan extremo como el observado. No mide la magnitud del efecto ni la probabilidad de que la hipótesis nula sea verdadera.

Errores frecuentes

- **Romper los pares al ordenar columnas por separado.** La unidad de análisis es (x_i, y_i); cambiar parejas inventa otra asociación.
- **Interpretar $r = 0$ como "no hay relación".** Solo descarta una asociación lineal neta en la muestra.
- **Usar Spearman como detector universal de no linealidad.** Detecta orden monótono; una U también puede producir $\rho_s \approx 0$.
- **Eliminar un extremo porque cambia el signo.** Primero se verifica si es error, caso de otra población o evento genuino.
- **Comparar coeficientes con tamaños muestrales distintos sin reportarlos.** La estabilidad de 0.5 con 12 pares no es la misma que con 12,000.
- **Convertir significancia en relevancia.** Con muestras grandes, una correlación minúscula puede producir un valor p pequeño.
- **Inferir causalidad por magnitud.** Una correlación de 0.95 sigue siendo compatible con confusión o selección.

Puentes con minería de datos

Contexto	Uso de la correlación	Límite que debe recordarse
Selección exploratoria de atributos	Detectar redundancia lineal o monótona.	Dos predictores correlacionados pueden aportar información condicional distinta.
Diagnóstico de multicolinealidad	Señalar pares linealmente relacionados.	La multicolinealidad puede involucrar combinaciones de más de dos variables.
Detección de fuga de información	Una asociación casi perfecta con el objetivo merece revisión.	La fuga depende del momento y proceso de disponibilidad, no solo de r.
Series temporales	Explorar co-movimiento y rezagos.	Tendencias y autocorrelación pueden crear asociaciones espurias.
Agrupamiento de variables	Construir distancias como $1 - r$.	El signo, los faltantes y la validez métrica requieren decisiones explícitas.
Monitoreo	Comparar cambios de asociación entre periodos o segmentos.	Una correlación agregada puede ocultar patrones opuestos por grupo.

Puente al caso BigMart

El capítulo Medidas de correlación en Big Mart Sales traslada esta secuencia a variables comerciales y tipos de outlet:

1. conservar las filas emparejadas e inspeccionar diagramas de dispersión;
2. distinguir relaciones lineales, monótonas y no monótonas;
3. comparar Pearson y Spearman según la pregunta;
4. construir matrices documentando faltantes y tamaños efectivos;
5. revisar asociaciones por grupos antes de interpretar un coeficiente global.

El código aplicado deja así de ser una colección de llamadas a `.corr()`: cada celda se conecta con una geometría, una unidad de análisis y una limitación.

Ejercicios formativos

1. **Cálculo completo.** Para $X = (1, 2, 3, 4, 5)$ y $Y = (5, 3, 4, 1, 2)$, construya la tabla de desviaciones, calcule covarianza muestral y Pearson, asigne rangos y cuente los diez pares para Spearman y Kendall.
2. **Cambio de unidades.** Convierta las horas del ejemplo a minutos y sume 10 a todos los retos. Prediga qué ocurre con covarianza, Pearson, Spearman y Kendall antes de calcular.
3. **Relación sin correlación.** Verifique que $X = (-2, -1, 0, 1, 2)$ y $Y = X^2$ producen Pearson y Spearman cercanos a cero. Dibuje los puntos y explique por qué no son independientes.
4. **Empates.** Para $X = (1, 2, 2, 4, 5)$ y $Y = (1, 1, 3, 4, 4)$, obtenga rangos promedio y calcule Spearman como Pearson entre rangos. Identifique los pares empatados necesarios para tau-b.
5. **Punto influyente.** Parta de una nube positiva de diez puntos y agregue uno en la dirección contraria. Compare los tres coeficientes y justifique si conservaría, corregiría o segmentaría esa observación.
6. **Anscombe.** Calcule para cada serie media, varianza, pendiente y Pearson. Después describa qué decisión equivocada surgiría si solo se leyera la tabla.
7. **Faltantes.** Construya una tabla de cuatro variables donde la eliminación por pares use distintos tamaños muestrales. Reporte las matrices de correlación y de n mediante `correlaciones_y_tamanos`.
8. **Causalidad.** Proponga tres explicaciones para una correlación positiva entre uso de una plataforma educativa y calificación: una causal, una de causalidad inversa y una por confusión. Indique qué datos ayudarían a distinguirlas.

33. Medidas de correlación en Big Mart Sales

Introducción

Este caso aplica Pearson, Spearman y la inspección visual descritos en Medidas de correlación.

Después de estudiar la localización, la variabilidad, la forma, la heterogeneidad y la concentración, resulta natural incorporar una dimensión adicional del análisis exploratorio: la **correlación**. Mientras los capítulos anteriores se enfocaron principalmente en la descripción univariada o en comparaciones entre grupos, el análisis de correlación permite explorar **relaciones entre variables numéricas**.

En términos generales, la correlación busca responder preguntas como las siguientes:

- ¿Las ventas aumentan cuando aumenta el precio de lista del producto?
- ¿Existe asociación entre la visibilidad del producto y sus ventas?
- ¿El peso del producto se relaciona con otras variables cuantitativas?
- ¿Las asociaciones observadas son lineales, monotónicas o prácticamente inexistentes?

Este capítulo se centra en el estudio de las relaciones entre variables numéricas del conjunto de datos **Big Mart Sales**, con énfasis en la variable objetivo de interés:

`Item_Outlet_Sales`.

Además de calcular coeficientes de correlación, construiremos representaciones gráficas que ayuden a interpretar dichas asociaciones dentro de un contexto de EDA. La meta no es inferir causalidad, sino **detectar patrones de dependencia, redundancia o independencia aproximada** que puedan enriquecer la comprensión del conjunto de datos.

Definición operativa. Pearson resume asociación lineal y Spearman asociación monótona por rangos; ambos requieren inspección gráfica (Pearson 1895; Spearman 1904; Anscombe 1973).

Objetivos del capítulo

Al finalizar este capítulo, el lector debería ser capaz de:

1. distinguir entre correlación positiva, negativa y cercana a cero;
2. interpretar la magnitud y el signo de un coeficiente de correlación;
3. comparar correlaciones de Pearson y Spearman;
4. construir una matriz de correlación para variables numéricas;
5. integrar los hallazgos de correlación al dashboard global del libro.

Carga y preparación de datos

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614

Para estudiar correlación nos concentraremos primero en las variables numéricas.

En muchos conjuntos de datos de tipo comercial, la cantidad de variables numéricas es moderada. En **Big Mart Sales**, típicamente aparecen variables como:

- `Item_Weight`
- `Item_Visibility`
- `Item_MRP`
- `Outlet_Establishment_Year`
- `Item_Outlet_Sales`

Dependiendo de la versión del dataset, puede haber más o menos variables numéricas. El análisis se adaptará automáticamente a las columnas detectadas.

Fundamento conceptual

Correlación de Pearson

La correlación de Pearson entre dos variables aleatorias X e Y se define como

$$\rho_{X,Y} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}.$$

En la práctica, con datos muestrales, se utiliza su estimador muestral. Este coeficiente mide la intensidad y dirección de una **asociación lineal**.

Sus valores están acotados por

$$-1 \leq r \leq 1.$$

- Si $r \approx 1$, la asociación lineal es positiva fuerte.
- Si $r \approx -1$, la asociación lineal es negativa fuerte.
- Si $r \approx 0$, no hay evidencia de asociación lineal fuerte.

Correlación de Spearman

La correlación de Spearman se basa en los rangos de los datos. Es especialmente útil cuando la relación entre dos variables no es estrictamente lineal, pero sí **monotónica**.

En el EDA, Pearson y Spearman son complementarias:

- **Pearson**: sensible a relaciones lineales y a valores extremos.
- **Spearman**: más robusta frente a no linealidad monotónica y menos sensible a outliers severos.

Advertencia importante

La correlación **no implica causalidad**. Dos variables pueden estar correlacionadas por múltiples razones: dependencia directa, mediación por otra variable, estructura del proceso de muestreo o simple coincidencia en muestras pequeñas.

Revisión de valores faltantes en variables numéricas

Antes de calcular correlaciones, conviene revisar valores faltantes.

	Variable	Faltantes
0	Item_Weight	1463
1	Item_Visibility	0
2	Item_MRP	0
3	Outlet_Establishment_Year	0
4	Item_Outlet_Sales	0

Si existen valores faltantes, `pandas` calculará correlaciones utilizando observaciones disponibles por pares. Para EDA esto suele ser suficiente, aunque conviene documentarlo.

Matriz de correlación de Pearson

Comenzamos con la matriz de correlación de Pearson para todas las variables numéricas.

	Item_Weight	Item_Visibility	Item_MRP	Outlet_Establishment_Year	Ite
Item_Weight	1.000000	-0.014048	0.027141	-0.011588	0.0
Item_Visibility	-0.014048	1.000000	-0.001315	-0.074834	-0.
Item_MRP	0.027141	-0.001315	1.000000	0.005020	0.5
Outlet_Establishment_Year	-0.011588	-0.074834	0.005020	1.000000	-0.
Item_Outlet_Sales	0.014123	-0.128625	0.567574	-0.049135	1.0

Mapa de calor

Interpretación inicial

Esta matriz permite una lectura rápida de las asociaciones lineales entre variables numéricas. En particular, interesa observar:

- qué variables están más asociadas con `Item_Outlet_Sales`;
- si existen pares de variables numéricas altamente correlacionados entre sí;
- si la mayoría de relaciones son débiles, moderadas o fuertes.

En datasets como **Big Mart Sales**, suele ser común encontrar que `Item_MRP` tiene una asociación positiva visible con las ventas, mientras que otras variables presentan correlaciones más débiles.

Correlación de Spearman

Para contrastar con Pearson, calculamos ahora la matriz de Spearman.

	Item_Weight	Item_Visibility	Item_MRP	Outlet_Establishment_Year	Ite
Item_Weight	1.000000	-0.014879	0.030822	-0.009393	0.0
Item_Visibility	-0.014879	1.000000	0.005688	-0.054924	-0.1
Item_MRP	0.030822	0.005688	1.000000	0.003782	0.5
Outlet_Establishment_Year	-0.009393	-0.054924	0.003782	1.000000	0.0
Item_Outlet_Sales	0.015062	-0.115076	0.562986	0.042947	1.0

Comparación entre Pearson y Spearman

Cuando Pearson y Spearman tienen valores similares, suele haber evidencia de una asociación aproximadamente lineal o, al menos, de una estructura monotónica estable. Cuando Spearman es claramente mayor que Pearson, puede ser una señal de relación monotónica no lineal o de sensibilidad de Pearson a valores extremos.

Una forma compacta de comparar ambas matrices es centrarse en la variable de interés.

Correlación con la variable objetivo

Dado que el fenómeno central del conjunto de datos son las ventas, es útil ordenar las correlaciones con `Item_Outlet_Sales`.

Comentario analítico

Esta visualización resume qué variables numéricas muestran una asociación lineal más fuerte con las ventas. Una correlación positiva moderada entre `Item_MRP` y `Item_Outlet_Sales`, por ejemplo, puede interpretarse como evidencia de que los productos con mayor precio de lista tienden también a registrar mayores ventas monetarias, aunque esto no implica necesariamente que "vender más caro" cause mayores ventas en sentido causal.

Diagramas de dispersión

Las matrices de correlación son útiles, pero pueden ocultar detalles estructurales. Por ello, es indispensable acompañarlas con gráficos de dispersión.

Ventas vs precio de lista

Ventas vs visibilidad

Ventas vs peso

Qué observar en los diagramas

Los scatter plots permiten responder preguntas que una matriz de correlación no resuelve por sí sola:

- ¿La relación parece lineal?
- ¿Hay subgrupos o clústeres?
- ¿Existen outliers que afecten la correlación?
- ¿La nube de puntos sugiere heterocedasticidad?
- ¿Hay curvatura o estructura por segmentos?

En EDA, esta lectura visual es esencial, porque una correlación numérica puede ser engañosa si la relación no es homogénea.

Correlación por subgrupos

Una extensión particularmente rica consiste en estudiar si la correlación cambia según el tipo de outlet o el tipo de producto. Esto conecta el análisis de correlación con la heterogeneidad.

Ejemplo: correlación entre Item_MRP y ventas por tipo de outlet

Lectura

Si la intensidad de la correlación cambia notablemente entre subgrupos, eso sugiere que la relación entre variables no es uniforme en todo el dataset. Este hallazgo es importante porque un coeficiente global puede ocultar comportamientos diferenciados por segmento.

Tabla resumen de asociaciones

Para cerrar el análisis, conviene construir una tabla corta con las correlaciones más relevantes respecto de la variable objetivo.

Podemos incluso añadir una etiqueta cualitativa de intensidad para facilitar la lectura didáctica.

Mini-dashboard de correlación

Para integrarlo al libro, este capítulo puede cerrarse con un panel compacto compuesto por:

1. tabla de correlaciones con la variable objetivo;
2. mapa de calor de Pearson;
3. scatter plot de la relación más relevante;
4. comparación por subgrupos si procede.

KPIs de correlación

Discusión final

El análisis de correlación amplía la mirada del EDA al introducir relaciones entre variables. En **Big Mart Sales**, esta dimensión permite identificar qué atributos numéricos están más asociados con las ventas y si dicha asociación es débil, moderada o fuerte.

Sin embargo, el uso de correlaciones debe acompañarse siempre de lectura visual y criterio analítico. Una matriz numérica puede sugerir asociación, pero solo los diagramas de dispersión permiten ver si la relación es lineal, si hay segmentos diferenciados o si algunos valores extremos dominan el comportamiento.

Además, comparar correlaciones globales con correlaciones por subgrupos conecta este capítulo con la idea de heterogeneidad: no todas las relaciones son homogéneas en toda la población.

Ejercicios sugeridos

1. Repite el análisis usando solo outlets de un tipo específico y compara los resultados con la correlación global.
2. Calcula la matriz de correlación para un subconjunto de categorías de producto y comenta las diferencias.
3. Investiga si la transformación logarítmica de `Item_Outlet_Sales` cambia de forma importante las correlaciones.
4. Identifica un par de variables con baja correlación lineal pero cuya nube de puntos sugiera alguna estructura no trivial.

Parte XVIII.

Síntesis del EDA

34. Dashboard global

Introducción

A lo largo de los capítulos anteriores se desarrolló una ruta progresiva para el análisis exploratorio del conjunto de datos **Big Mart Sales**. La lógica seguida no fue arbitraria: se construyó una descripción estadística de las ventas a partir de seis dimensiones complementarias.

Primero se estudió la **localización**, para responder dónde se sitúan los valores típicos. Después se introdujo la **variabilidad**, para cuantificar la dispersión alrededor de esos centros. Más adelante se abordó la **forma**, con el fin de entender la asimetría, la curtosis y la estructura general de las distribuciones. Luego se examinó la **heterogeneidad**, es decir, las diferencias entre categorías, tipos de outlet y otros subgrupos. La **concentración** permitió analizar cómo se reparte el total entre las unidades del sistema y la **correlación** añadió la exploración de asociaciones entre variables numéricas.

El propósito de este capítulo es integrar todas esas perspectivas en un **dashboard global**, útil tanto para la enseñanza del EDA como para la comunicación de hallazgos en un entorno de análisis aplicado. Este dashboard no sustituye el análisis detallado de los capítulos previos; más bien, actúa como una síntesis estructurada que permite recorrer, de manera compacta, la construcción progresiva de la medida.

Objetivos del capítulo

Este capítulo busca que el lector sea capaz de:

1. integrar múltiples dimensiones del EDA en una misma narrativa analítica;
2. construir indicadores resumidos y visualizaciones consistentes entre sí;
3. diseñar un dashboard reproducible en Quarto compatible con un libro;
4. interpretar de forma conjunta las dimensiones de localización, variabilidad, forma, heterogeneidad, concentración y correlación.

Carga y preparación de datos

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614

Arquitectura conceptual del dashboard

El dashboard global puede entenderse como una composición de seis capas analíticas:

Capa 1. Localización

Resume el centro de la distribución mediante indicadores como media, mediana, moda y cuantiles.

Capa 2. Variabilidad

Introduce la amplitud y dispersión mediante rango, desviación estándar, IQR y coeficiente de variación.

Capa 3. Forma

Permite describir asimetría, curtosis y presencia de colas pesadas o valores atípicos.

Capa 4. Heterogeneidad

Compara la estructura de ventas entre subgrupos relevantes, como tipos de producto o tipos de outlet.

Capa 5. Concentración

Evalúa qué tan equilibrada o dominada está la estructura total de ventas.

Capa 6. Correlación

Explora asociaciones lineales y monótonas entre variables numéricas sin atribuir causalidad.

En un dashboard pedagógico, estas seis capas deben coexistir de forma coherente, de manera que el lector pueda pasar del nivel global al comparativo sin perder el hilo interpretativo.

Selección de variable principal

Para articular el dashboard usaremos como variable principal:

[`Item_Outlet_Sales`,]

ya que representa el fenómeno central del conjunto de datos: las ventas observadas por producto y outlet.

Panel de indicadores globales

El primer componente del dashboard debe consistir en un bloque de KPIs generales que resuman el estado del conjunto de datos.

	Indicador	Valor
0	Número de registros	8.523000e+03
1	Ventas totales	1.859113e+07
2	Media	2.181290e+03
3	Mediana	1.794330e+03
4	Moda	9.587500e+02
5	Desviación estándar	1.706500e+03
6	IQR	2.267050e+03
7	Coeficiente de variación	7.823000e-01
8	Asimetría	1.177500e+00
9	Curtosis	1.615900e+00

Este bloque de KPIs cumple una función importante: reúne en una sola tabla las medidas fundamentales de centro, dispersión y forma que fueron desarrolladas por separado en capítulos previos.

Panel de localización y forma

Una forma natural de combinar localización y forma es mediante un histograma enriquecido con líneas de referencia para la media y la mediana.

Interpretación

Este panel sintetiza dos preguntas esenciales:

- ¿Dónde se localizan los valores típicos?
- ¿La distribución es aproximadamente simétrica o presenta sesgo?

Cuando la media y la mediana difieren de manera visible, y cuando la cola derecha es más larga, suele haber evidencia de asimetría positiva. En un entorno como Big Mart Sales, esto es compatible con la existencia de observaciones con ventas muy elevadas respecto del grueso de la distribución.

Panel de variabilidad

La variabilidad puede representarse con un boxplot acompañado de una tabla resumida.

	Medida	Valor
0	Mínimo	33.2900
1	Q1	834.2500
2	Mediana	1794.3300
3	Q3	3101.3000
4	Máximo	13086.9600
5	Rango	13053.6700
6	IQR	2267.0500
7	Desv. estándar	1706.5000
8	CV	0.7823

Este panel muestra de forma visual la amplitud de la distribución, la caja intercuartílica y la presencia de observaciones extremas.

Panel de heterogeneidad

La heterogeneidad requiere comparar subgrupos. Una opción muy expresiva es contrastar las ventas por tipo de outlet y por tipo de producto.

Ventas por tipo de outlet

Ventas por tipo de producto

Para evitar saturación visual, puede ser útil mostrar solo las categorías con mayor venta total.

Tabla comparativa por grupo

	Outlet_Type	n	media	mediana	desv_std
0	Grocery Store	1083	339.828500	256.9988	260.851582
1	Supermarket Type1	5577	2316.181148	1990.7420	1515.965558
2	Supermarket Type2	928	1995.498739	1655.1788	1375.932889
3	Supermarket Type3	935	3694.038558	3364.9532	2127.760054

Este bloque permite observar que la heterogeneidad no se limita a diferencias de media: también pueden cambiar la mediana, la dispersión y la presencia de valores extremos.

Panel de concentración

Para el dashboard global conviene incorporar una visualización compacta de concentración. El análisis por tipo de producto suele ser especialmente ilustrativo.

	Item_Type	Item_Outlet_Sales	share	cum_share	share_pct	cum_share_pct
0	Fruits and Vegetables	2.820060e+06	0.151688	0.151688	15.168849	15.168849
1	Snack Foods	2.732786e+06	0.146994	0.298683	14.699412	29.868261
2	Household	2.055494e+06	0.110563	0.409246	11.056317	40.924578
3	Frozen Foods	1.825735e+06	0.098205	0.507450	9.820464	50.745042
4	Dairy	1.522594e+06	0.081899	0.589349	8.189897	58.934939

Indicadores breves de concentración

	Indicador	Valor
0	Top 3 categorías (%)	40.92
1	Categorías para 80 % de ventas	9.00

Panel de correlación

La síntesis se completa con Pearson y Spearman. La comparación ayuda a detectar asociaciones monotónicas que no son estrictamente lineales.

> El panel de correlación sirve para generar hipótesis, no para afirmar causalidad. Cada coeficiente relevante debe contrastarse con un diagrama de dispersión y con el patrón de faltantes.

Ensamblaje del dashboard global

En un libro de Quarto, el dashboard puede presentarse como una secuencia ordenada de paneles. Una estructura práctica es la siguiente:

1. **Bloque inicial de KPIs**: resumen numérico de alto nivel.
2. **Distribución global**: histograma y boxplot para localización, forma y variabilidad.
3. **Comparación entre grupos**: heterogeneidad por outlet y por producto.
4. **Concentración**: Pareto y tabla de acumulación.
5. **Correlación**: heatmap y comparación Pearson–Spearman.
6. **Texto interpretativo**: síntesis narrativa de hallazgos.

Ejemplo de lectura integrada

Desde una perspectiva estadística, el dashboard global permite articular una lectura de este tipo:

- La distribución global de ventas presenta un centro identificable, pero también una dispersión importante.
- La separación entre media y mediana, junto con la cola derecha, sugiere asimetría positiva.
- Los boxplots muestran variabilidad considerable y presencia de valores atípicos.
- Las comparaciones por tipo de outlet y tipo de producto revelan heterogeneidad entre subgrupos.
- El análisis de concentración muestra que una fracción limitada de categorías contribuye de forma desproporcionada al total.

Esta lectura resume precisamente la lógica del EDA progresivo: la medida no se agota en un único número, sino que se construye mediante varias capas complementarias.

Propuesta de cierre visual para el book

En un proyecto de Quarto Book, este capítulo puede funcionar como capítulo de cierre analítico. Una buena práctica consiste en terminar con una tabla compacta de hallazgos.

	Dimensión	Pregunta guía	Indicador / Visual principal
0	Localización	¿Dónde se ubican los valores típicos?	Media, mediana, histograma
1	Variabilidad	¿Qué tan dispersas son las ventas?	IQR, desviación estándar, boxplot
2	Forma	¿La distribución es simétrica o sesgada?	Asimetría, curtosis, histograma
3	Heterogeneidad	¿Cambian las ventas entre grupos?	Boxplots por grupo, tablas comparativas
4	Concentración	¿Se reparte el total de manera equilibrada?	Pareto, shares acumulados

Conclusiones

El dashboard global no debe entenderse como una simple colección de gráficas, sino como una **síntesis argumentada del EDA**. Su valor reside en que integra distintos niveles de descripción en una sola narrativa:

- el centro de la distribución,
- su amplitud,
- su geometría,
- sus diferencias entre subgrupos,
- y su estructura de reparto.

En el caso de **Big Mart Sales**, esta integración permite comunicar de forma compacta cómo se comportan las ventas desde varios ángulos complementarios. Para fines docentes, el dashboard muestra que la estadística descriptiva es una construcción progresiva de la medida; para fines aplicados, ofrece un instrumento claro de exploración y comunicación.

Sugerencias de extensión

1. Añadir filtros interactivos por `Outlet_Type`, `Outlet_Size` o `Item_Fat_Content`.
2. Integrar una sección de correlación entre variables numéricas.
3. Incorporar mapas si se dispone de georreferenciación de outlets.
4. Conectar el dashboard con la fase posterior de modelado predictivo.

Integración al libro de Quarto

Si tu libro ya incluye los capítulos anteriores, puedes agregar este cierre junto con el capítulo de concentración en el archivo `_quarto.yml` con una estructura como la siguiente:

Con ello quedará completa la secuencia del EDA progresivo en formato de book.

Parte XIX.

Análisis Multivariado

35. Análisis Exploratorio Multivariado: De la Estadística Descriptiva al Espacio Multidimensional

El problema: cuando mirar variable por variable no es suficiente

A lo largo de los capítulos anteriores de esta Parte III exploramos los pilares del análisis descriptivo: situamos el centro de una variable con medidas de localización (Sección 35), cuantificamos su dispersión (Sección 35), evaluamos la asimetría y el peso de sus colas (Sección 35), medimos la diversidad y la presencia de casos atípicos (Sección 35) y analizamos si dos características avanzan juntas mediante correlación (Sección 35).

Sin embargo, en la práctica de la minería de datos las observaciones rara vez viven en aislamiento. Una fila de una tabla de clientes, una transacción financiera o una medición de sensores industriales no está formada por una o dos variables desvinculadas, sino por un vector conjunto de p dimensiones:

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T \in \mathbb{R}^p.$$

El error más común al enfrentarse a este escenario consiste en aplicar lo que podríamos llamar la **falacia de las proyecciones marginales**: descomponer el conjunto de datos en p columnas separadas, resumir cada una de forma univariada (media, desviación estándar, histograma) y asumir que la suma de esas descripciones captura la estructura global de los datos.

La falacia de las proyecciones marginales

Examinar cada variable por separado equivale a observar únicamente las sombras que un objeto tridimensional proyecta sobre las paredes de una habitación. Dos figuras con siluetas idénticas en dos planos ortogonales pueden ser, en el espacio tridimensional, objetos completamente distintos. En minería de datos, los fenómenos más críticos —agrupamientos, anomalías, cuellos de botella y dependencias— habitan en las relaciones conjuntas, no en las proyecciones aisladas.

Considere el siguiente caso intuitivo: un departamento de recursos humanos analiza a cinco candidatos mediante dos pruebas estandarizadas con escalas de 0 a 100: una de **habilidad analítica** (X_1) y otra de **habilidad comunicativa** (X_2). Históricamente, ambas habilidades presentan una correlación fuertemente positiva en la organización.

Candidato	Analítica (X_1)	Comunicación (X_2)	Diagnóstico univariado	Diagnóstico multivariado
A	50	52	Normal (cerca de la media)	Normal y coherente con la tendencia
B	85	88	Alto en ambas	Destacado balanceado
C	20	22	Bajo en ambas	Requiere capacitación
D	95	92	Sobresaliente	Sobresaliente balanceado
E	**85**	**15**	X_1 normal-alto, X_2 aceptable	**¡Anomalía multivariada severa!**

Si examinamos al candidato **E** variable por variable: 1. Su calificación en Analítica (85) es perfectamente plausible y coincide con la del candidato B. 2. Su calificación en Comunicación (15), aunque baja, se encuentra dentro de los rangos de la distribución marginal de la prueba.

Ningún filtro univariado (z-score individual o rango intercuartílico de Tukey) señalaría al candidato E como un caso extremo. No obstante, en el plano conjunto (X_1, X_2), el candidato E rompe de manera radical la estructura de covarianza de los datos: posee un perfil extraordinariamente disonante.

Tres brújulas para el espacio multivariado

1. **La localización no es solo un punto escalar**: el centro de una nube en $\mathbb{R}^p$ debe considerar cómo se compensan las coordenadas y si el estimador es invariante ante rotaciones del sistema de referencia.
2. **La dispersión tiene orientación y forma**: en $\mathbb{R}^p$ la variabilidad no es un radio esférico, sino un elipsoide cuya orientación y excentricidad están determinadas por las covarianzas.
3. **La distancia depende de la geometría estadística**: medir separación en $\mathbb{R}^p$ con la distancia euclidiana convencional ignora las correlaciones y produce juicios erróneos sobre la similitud y las anomalías.

Localización multivariada: el centro de una nube en $\mathbb{R}^p$

En el capítulo de localización univariada aprendimos que la media minimiza la suma de desviaciones cuadráticas, mientras que la mediana minimiza la suma de distancias absolutas y resiste valores extremos. ¿Cómo se extienden estas propiedades a una matriz de datos $\mathbf{X} \in \mathbb{R}^{n \times p}$?

El vector de medias muestral

El estimador clásico de localización multivariada es el **vector de medias** $\bar{\mathbf{x}} \in \mathbb{R}^p$, obtenido calculando el promedio aritmético de cada columna:[1]

$$\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i = \begin{pmatrix} \bar{x}_1 \\ \bar{x}_2 \\ \vdots \\ \bar{x}_p \end{pmatrix}, \quad \text{donde } \bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}.$$

Geométricamente, $\bar{\mathbf{x}}$ es el **centro de gravedad** o centroide de la nube de puntos. Posee una propiedad fundamental: es un estimador lineal y conmuta con cualquier transformación afín de los datos.[2] Si definimos una transformación $\mathbf{y}_i = \mathbf{A}\mathbf{x}_i + \mathbf{b}$, el nuevo centro de gravedad es simplemente $\bar{\mathbf{y}} = \mathbf{A}\bar{\mathbf{x}} + \mathbf{b}$.

Sin embargo, al igual que su contraparte univariada, el vector de medias hereda un **punto de ruptura** del $0\,\%$ (Huber y Ronchetti 2009): una sola observación corrupta o errónea en cualquiera de las variables puede desplazar arbitrariamente todo el vector de medias en el espacio.

La falacia de la mediana por componentes

Ante la sensibilidad de la media, una tentación inmediata consiste en definir una "mediana multivariada" tomando la mediana univariada de cada coordenada de forma independiente:

$$\mathbf{m}_{\text{marg}} = \begin{pmatrix} \operatorname{med}(x_{\cdot 1}) \\ \operatorname{med}(x_{\cdot 2}) \\ \vdots \\ \operatorname{med}(x_{\cdot p}) \end{pmatrix}.$$

Aunque sencilla de calcular, la mediana por componentes presenta dos deficiencias teóricas y prácticas graves (Small 1990):

1. **No es equivariante por rotación**: si rotamos la nube de puntos en el plano (por ejemplo, cambiando la orientación de los sensores o rotando los ejes), la mediana por componentes calculada sobre los nuevos ejes no coincide con la rotación de la mediana original.
2. **Puede caer fuera de los datos y de su envolvente convexa**: en configuraciones asimétricas o no convexas, el punto $\mathbf{m}_{\text{marg}}$ puede ubicarse en una región donde la probabilidad de observar datos es nula.

Como se observa en la figura anterior, la mediana por componentes colapsa arbitrariamente en el vértice $(1, 1)$, mientras que el vector de medias se sitúa en el centro de gravedad interior del triángulo.

[1] El vector de medias $\bar{\mathbf{x}}$ minimiza la suma de distancias cuadráticas euclidianas a todos los puntos: $\bar{\mathbf{x}} = \arg\min_{\mathbf{m}\in\mathbb{R}^p} \sum_{i=1}^{n} \|\mathbf{x}_i - \mathbf{m}\|_2^2$.

[2] Un estimador de localización $\mathbf{T}(\mathbf{X})$ es equivariante afín si para cualquier matriz invertible $\mathbf{A} \in \mathbb{R}^{p\times p}$ y cualquier vector de traslación $\mathbf{b} \in \mathbb{R}^p$, se cumple $\mathbf{T}(\mathbf{X}\mathbf{A}^T + \mathbf{1}\mathbf{b}^T) = \mathbf{A}\mathbf{T}(\mathbf{X}) + \mathbf{b}$.

Mediana geométrica (Fermat–Weber) y profundidad estadística

Para obtener un centro verdaderamente multivariado y robusto, generalizamos el problema de Fermat-Weber: buscamos el punto $\mathbf{m}_{\text{geom}} \in \mathbb{R}^p$ que minimice la suma de las **distancias euclidianas directas** a todas las observaciones (Small 1990):

$$\mathbf{m}_{\text{geom}} = \arg\min_{\mathbf{m}\in\mathbb{R}^p} \sum_{i=1}^{n} \|\mathbf{x}_i - \mathbf{m}\|_2 = \arg\min_{\mathbf{m}\in\mathbb{R}^p} \sum_{i=1}^{n} \sqrt{\sum_{j=1}^{p} (x_{ij} - m_j)^2}.$$

A diferencia de la mediana por componentes: - La **mediana geométrica** (o mediana espacial) es equivariante ante rotaciones, traslaciones y cambios de escala isotrópicos. - Posee un punto de ruptura del 50 %: se necesita corromper más de la mitad de las observaciones para moverla al infinito. - Aunque no tiene solución analítica cerrada en forma de ecuación simple, se aproxima de manera extraordinariamente rápida mediante el **algoritmo iterativo de Weiszfeld**.[3]

Otra perspectiva fundamental proviene del concepto de **profundidad estadística** (*halfspace depth* o profundidad de Tukey) (Tukey 1975). La profundidad de un punto respecto a una nube de datos mide la fracción mínima de observaciones contenida en cualquier semiespacio cerrado que contenga a . El punto de máxima profundidad de Tukey representa la mediana de máxima centralidad en $\mathbb{R}^p$.

Medida	Definición operativa	Equivarianza por rotación	Punto de ruptura	Costo computacional
Vector de medias	Promedio aritmético por columna	Sí (afín)	0 %	Muy bajo $\mathcal{O}(np)$
Mediana marginal	Mediana independiente por columna	**No**	50 %	Bajo $\mathcal{O}(np \log n)$
Mediana geométrica	Minimiza suma de distancias euclidianas	**Sí** (ortogonal)	50 %	Moderado (iterativo)
Medoid multivariado	Observación real que minimiza distancias	Depende de la distancia	Robusto	$\mathcal{O}(n^2 p)$

[3] El algoritmo de Weiszfeld (1937) actualiza iterativamente la aproximación mediante $\mathbf{m}^{(k+1)} = \left(\sum_{i=1}^{n} \frac{\mathbf{x}_i}{\|\mathbf{x}_i - \mathbf{m}^{(k)}\|}\right) / \left(\sum_{i=1}^{n} \frac{1}{\|\mathbf{x}_i - \mathbf{m}^{(k)}\|}\right)$.

Variabilidad y dispersión multivariada: forma y orientación

En el análisis univariado, la dispersión se resume en un único número no negativo: la varianza s^2 o la desviación estándar s. Si una variable tiene $s = 10$, sabemos exactamente cuánto se alejan los datos de su promedio sobre esa recta.

En $\mathbb{R}^p$, sin embargo, una nube de puntos puede tener la misma dispersión en cada coordenada por separado y aun así presentar estructuras geométricas radicalmente distintas. La dispersión multivariada no es un escalar; es un **tensor geométrico** que define tamaño, forma y orientación.

La matriz de varianza-covarianza

La estructura de dispersión conjunta se captura en la **matriz de varianza-covarianza muestral** $\mathbf{S} \in \mathbb{R}^{p\times p}$:[4]

$$\mathbf{S} = \frac{1}{n-1}\sum_{i=1}^{n}(\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \begin{pmatrix} s_{11} & s_{12} & \cdots & s_{1p} \\ s_{21} & s_{22} & \cdots & s_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ s_{p1} & s_{p2} & \cdots & s_{pp} \end{pmatrix}.$$

Propiedades esenciales de $\mathbf{S}$: 1. **Diagonal principal**: $s_{jj} = s_j^2$ contiene la varianza muestral de cada variable individual. 2. **Elementos fuera de la diagonal**: $s_{jk} = s_{kj}$ contiene la covarianza entre la variable j y la variable k. 3. **Simetría y positividad**: $\mathbf{S}$ es una matriz simétrica ($s_{jk} = s_{kj}$) y **semidefinida positiva** ($\mathbf{a}^T\mathbf{S}\mathbf{a} \geq 0$ para cualquier vector $\mathbf{a} \in \mathbb{R}^p$). Si no existen variables linealmente redundantes y $n > p$, $\mathbf{S}$ es estrictamente definida positiva.

Resúmenes escalares: Varianza Total vs. Varianza Generalizada

Cuando necesitamos comparar la dispersión global entre dos grupos o monitorear la variabilidad total de un sistema, reducir la matriz $\mathbf{S}$ a un único escalar resulta conveniente. Existen dos alternativas clásicas con significados geométricos muy diferentes:

1. Varianza Total (Traza de S).

La **varianza total** se define como la traza de la matriz de covarianza:[5]

$$\mathrm{Var}_{\text{total}}(\mathbf{X}) = \mathrm{tr}(\mathbf{S}) = \sum_{j=1}^{p} s_{jj} = \sum_{j=1}^{p} \lambda_j,$$

donde $\lambda_1, \ldots, \lambda_p$ son los autovalores de $\mathbf{S}$.

[4] El divisor $n-1$ (corrección de Bessel) asegura que $\mathbf{S}$ sea un estimador insesgado de la matriz de covarianza poblacional Σ: $\mathbb{E}[\mathbf{S}] = \Sigma$.

[5] La traza de una matriz cuadrada es la suma de sus elementos diagonales y coincide con la suma de sus autovalores: $\mathrm{tr}(\mathbf{S}) = \sum_{j=1}^{p} s_{jj} = \sum_{j=1}^{p} \lambda_j$.

La ceguera de la Varianza Total

La varianza total es simplemente la suma de las varianzas univariadas. **Ignora por completo las covarianzas** s_{jk}. Dos conjuntos de datos pueden tener exactamente la misma varianza total, pero uno ser una esfera isotrópica no correlacionada y el otro ser una línea delgada y altamente correlacionada.

2. Varianza Generalizada de Wilks (Determinante de S).

Introducida por Samuel Wilks (Wilks 1932), la **varianza generalizada** se define como el determinante de la matriz de covarianza:

$$\text{Var}_{\text{gen}}(\mathbf{X}) = |\mathbf{S}| = \det(\mathbf{S}) = \prod_{j=1}^{p} \lambda_j.$$

Geométricamente, el determinante $|\mathbf{S}|$ es proporcional al **cuadrado del hipervolumen** del elipsoide de dispersión generado por los datos (Johnson y Wichern 2007).

- Si las variables son independientes, $|\mathbf{S}| = s_{11}s_{22}\cdots s_{pp}$.
- A medida que la correlación entre variables aumenta, el elipsoide se aplana y $|\mathbf{S}|$ disminuye.
- Si existe una relación lineal exacta entre variables (colinealidad perfecta), el elipsoide colapsa a una dimensión inferior y $|\mathbf{S}| = 0$, revelando que los datos no ocupan un volumen real en $\mathbb{R}^p$.

De la distancia euclidiana a la distancia de Mahalanobis

¿Qué tan lejos está una observación $\mathbf{x}$ del centro de la población $\bar{\mathbf{x}}$?

La respuesta natural en geometría elemental es la **distancia euclidiana**:

$$d_E(\mathbf{x}, \bar{\mathbf{x}}) = \sqrt{(\mathbf{x} - \bar{\mathbf{x}})^T(\mathbf{x} - \bar{\mathbf{x}})} = \sqrt{\sum_{j=1}^{p}(x_j - \bar{x}_j)^2}.$$

Sin embargo, en estadística y minería de datos, la distancia euclidiana es profundamente engañosa por dos motivos: 1. **Sensibilidad a las escalas de medida**: si X_1 se mide en metros y X_2 en milímetros, X_2 dominará numéricamente la distancia euclidiana por un factor de un millón. 2. **Ceguera ante la correlación**: asume que todas las direcciones del espacio son equiprobables (isocuantas esféricas). Si dos variables están correlacionadas positivamente, desplazarse a lo largo del eje principal de correlación es habitual, mientras que desplazarse perpendicularmente a la correlación es extraordinariamente improbable.

La solución matemática rigurosa es la **distancia estadística o distancia de Mahalanobis** (Mahalanobis 1936):

$$D_M(\mathbf{x}, \bar{\mathbf{x}}) = \sqrt{(\mathbf{x} - \bar{\mathbf{x}})^T\mathbf{S}^{-1}(\mathbf{x} - \bar{\mathbf{x}})}.$$

Descomposición intuitiva en tres etapas.

Para entender por qué la distancia de Mahalanobis corrige todas las distorsiones, descompongamos $\mathbf{S}^{-1}$ mediante su descomposición espectral $\mathbf{S} = \mathbf{V}\mathbf{V}^T$ (donde $\mathbf{V}$ son los autovectores y los autovalores):

1. **Centrado**: $(\mathbf{x} - \bar{\mathbf{x}})$ traslada el origen al centro de la nube.
2. **Rotación ortogonal**: $\mathbf{V}^T(\mathbf{x} - \bar{\mathbf{x}})$ rota los ejes hacia las direcciones principales de variación natural de los datos, eliminando las correlaciones.
3. **Estandarización por escala propia**: multiplicar por $^{-1/2}$ divide cada nuevo eje rotado entre su propia desviación estándar $\sqrt{\lambda_j}$.

Por tanto, **la distancia de Mahalanobis es simplemente la distancia euclidiana calculada después de descorrelacionar y estandarizar la nube de datos**.[6]

Observe la conclusión: ambos puntos están a exactamente 3 unidades euclidianas del centro. Sin embargo, para la estructura estadística, el punto verde P_1 tiene $D_M = 1.12$ (es un dato completamente común y predecible), mientras que el punto morado P_2 tiene $D_M = 3.35$ (se ubica en una región de bajísima densidad y constituye una anomalía clara).

Forma y normalidad multivariada

En el capítulo de medidas de forma (Sección 35) estudiamos la asimetría (g_1) y la curtosis (g_2) para caracterizar cómo se desvía una distribución de la curva gaussiana univariada (Joanes y Gill 1998).

Al pasar a $\mathbb{R}^p$, la distribución de referencia por excelencia es la **distribución normal multivariada** $\mathcal{N}_p(\mu, \Sigma)$, cuya función de densidad es:[7]

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T\Sigma^{-1}(\mathbf{x} - \mu)\right).$$

Nótese que el exponente contiene precisamente el cuadrado de la distancia de Mahalanobis: $-\frac{1}{2}D_M^2(\mathbf{x}, \mu)$. Por esta razón, las curvas o superficies de nivel donde la densidad $f(\mathbf{x})$ es constante corresponden exactamente a **hiperelipsoides de Mahalanobis**.

La falacia: ¿marginales normales implican normalidad conjunta?

Uno de los mitos más arraigados en el análisis exploratorio es asumir que si cada variable por separado supera una prueba de normalidad univariada (como Shapiro-Wilk o Kolmogorov-Smirnov), entonces el conjunto completo es normal multivariado.

Advertencia teórica fundamental

[6] Si $\mathbf{S} = \mathbf{V}\mathbf{V}^T$, la transformación $\mathbf{z}_i = {}^{-1/2}\mathbf{V}^T(\mathbf{x}_i - \bar{\mathbf{x}})$ genera variables incorrelacionadas con matriz de covarianza $\mathbf{I}_p$, donde $D_M(\mathbf{x}_i, \bar{\mathbf{x}}) = \|\mathbf{z}_i\|_2$.

[7] La distribución normal multivariada se denota $\mathbf{X} \sim \mathcal{N}_p(\mu, \Sigma)$. Si Σ no es invertible, la distribución reside en un subespacio afín de dimensión inferior y se denomina normal singular.

La normalidad de todas las distribuciones marginales es una condición necesaria, pero NO suficiente para la normalidad multivariada. Es perfectamente posible construir distribuciones donde cada coordenada individual sea una normal estándar $\mathcal{N}(0,1)$, pero cuya distribución conjunta tenga huecos, formas no elípticas o dependencias no lineales severas (Johnson y Wichern 2007).

Considere el siguiente contraejemplo clásico: sea $X \sim \mathcal{N}(0,1)$, y definamos Y como:

$$Y = \begin{cases} X & \text{si } |X| < c \\ -X & \text{si } |X| \geq c \end{cases}$$

para una constante $c > 0$. Marginalmente, tanto X como Y tienen distribuciones exactamente normales estándar $\mathcal{N}(0,1)$. Sin embargo, su distribución conjunta dista radicalmente de ser una normal bivariada: no tiene curvas de nivel elípticas y contiene franjas vacías.

Asimetría y curtosis multivariadas de Mardia

Para contrastar rigurosamente si una muestra p-dimensional proviene de una población normal multivariada, K. V. Mardia (Mardia 1970) introdujo extensiones algebraicas basadas en la distancia de Mahalanobis cruzada entre pares de observaciones:

Sean $g_{ij} = (\mathbf{x}_i - \bar{\mathbf{x}})^T \mathbf{S}^{-1} (\mathbf{x}_j - \bar{\mathbf{x}})$.

1. **Asimetría multivariada de Mardia ($b_{1,p}$)**:

$$b_{1,p} = \frac{1}{n^2} \sum_{i=1}^{n} \sum_{j=1}^{n} g_{ij}^3.$$

   Bajo la hipótesis nula de normalidad multivariada, el estadístico $\frac{n}{6} b_{1,p}$ sigue asintóticamente una distribución Chi-cuadrado con $\frac{p(p+1)(p+2)}{6}$ grados de libertad.

2. **Curtosis multivariada de Mardia ($b_{2,p}$)**:

$$b_{2,p} = \frac{1}{n} \sum_{i=1}^{n} g_{ii}^2 = \frac{1}{n} \sum_{i=1}^{n} \left[D_M^2(\mathbf{x}_i, \bar{\mathbf{x}}) \right]^2.$$

   Bajo normalidad multivariada, la esperanza teórica es $\mathbb{E}[b_{2,p}] = p(p+2)$. Valores significativamente mayores indican colas pesadas multivariadas (leptocurtosis conjunta).

Heterogeneidad y anomalías multivariadas

En el capítulo de heterogeneidad (Sección 35) estudiamos cómo identificar valores atípicos mediante el rango intercuartílico ($1.5 \times \text{IQR}$) o umbrales de z-score ($\pm 3\sigma$). En el espacio multivariado, estas herramientas univariadas fallan por dos razones fundamentales:

1. **El fenómeno del outlier oculto**: un punto puede estar dentro del rango normal de cada variable individual y aun así ser un valor atípico extremo al considerar la correlación entre ellas.
2. **La maldición de la dimensionalidad**: a medida que p crece, el volumen del hipercubo que encierra a los datos crece exponencialmente respecto a la región ocupada por la distribución central, haciendo que las observaciones se vuelvan dispersas.

La distribución teórica de D_M^2 y umbrales Chi-cuadrado

Si una población de p variables sigue una distribución normal multivariada $\mathcal{N}_p(\mu, \Sigma)$, el cuadrado de la distancia de Mahalanobis sigue exactamente una **distribución Chi-cuadrado con p grados de libertad**:[8]

$$D_M^2(\mathbf{X}, \mu) = (\mathbf{X} - \mu)^T \Sigma^{-1} (\mathbf{X} - \mu) \sim \chi_p^2.$$

Esto proporciona una regla de decisión probabilística inmediata para el análisis exploratorio: una observación $\mathbf{x}_i$ se clasifica como un **outlier multivariado potencial** con un nivel de significancia α (típicamente $\alpha = 0.01$ o $\alpha = 0.001$) si su distancia supera el valor crítico:

$$D_M^2(\mathbf{x}_i, \bar{\mathbf{x}}) > \chi_{p,1-\alpha}^2.$$

Dimensión (p)	Umbral al 95 % ($\chi^2_{p,0.95}$)	Umbral al 97.5 % ($\chi^2_{p,0.975}$)	Umbral al 99 % ($\chi^2_{p,0.99}$)
2	5.99	7.38	9.21
3	7.81	9.35	11.34
5	11.07	12.83	15.09
10	18.31	20.48	23.21

Efecto máscara (*masking*) y pantallazo (*swamping*)

La regla del umbral χ_p^2 presenta una trampa sutil cuando se utiliza con los estimadores clásicos $(\bar{\mathbf{x}}, \mathbf{S})$. Como los propios outliers forman parte del cálculo de la media y la covarianza:

[8] Esta propiedad se deriva de que $D_M^2 = \mathbf{Z}^T\mathbf{Z} = \sum_{j=1}^{p} Z_j^2$, donde cada $Z_j \sim \mathcal{N}(0,1)$ de forma independiente tras la diagonalización ortogonal de Σ.

- **Efecto Máscara (*Masking*)**: un grupo de outliers severos puede arrastrar a $\bar{\mathbf{x}}$ hacia ellos e inflar artificialmente la matriz $\mathbf{S}$. Al expandirse $\mathbf{S}$, las distancias de Mahalanobis de los propios outliers se reducen, haciendo que pasen desapercibidos como si fueran puntos normales.
- **Efecto Pantallazo (*Swamping*)**: la inflación de $\mathbf{S}$ y el desplazamiento de $\bar{\mathbf{x}}$ provocan que puntos perfectamente normales y limpios queden fuera de la nube central y sean erróneamente clasificados como anomalías.

Estimación robusta: Determinante de Covarianza Mínima (MCD)

Para evitar el enmascaramiento, Peter Rousseeuw (Rousseeuw 1984; Rousseeuw y Van Driessen 1999) introdujo el estimador del **Determinante de Covarianza Mínima** (*Minimum Covariance Determinant* o MCD).

La idea intuitiva es magistral: de las n observaciones totales, buscamos el subconjunto de tamaño h (donde $\frac{n+p+1}{2} \leq h < n$, comúnmente $h \approx 0.75n$) cuya matriz de covarianza tenga el **menor determinante posible**. 1. Al minimizar el determinante $|\mathbf{S}_h|$, el algoritmo selecciona automáticamente el subconjunto más denso y libre de valores atípicos. 2. La media y covarianza se estiman exclusivamente sobre ese subconjunto limpio: $(\bar{\mathbf{x}}_{\text{MCD}}, \mathbf{S}_{\text{MCD}})$. 3. Las distancias de Mahalanobis robustas se calculan respecto a este centro no contaminado:

$$D_{\text{MCD}}(\mathbf{x}_i) = \sqrt{(\mathbf{x}_i - \bar{\mathbf{x}}_{\text{MCD}})^T \mathbf{S}_{\text{MCD}}^{-1} (\mathbf{x}_i - \bar{\mathbf{x}}_{\text{MCD}})}.$$

Dependencia multivariada y correlación parcial

En el capítulo de correlación (Sección 35) medimos la fuerza de la asociación bivariada con el coeficiente r de Pearson. Sin embargo, en un sistema con múltiples variables, una correlación observada entre X y Y puede ser espuria, producto de una tercera variable confusora Z que afecta a ambas.

La Paradoja de Simpson

El caso más extremo de distorsión al ignorar variables adicionales es la **Paradoja de Simpson** (Simpson 1951): un fenómeno estadístico donde una tendencia o correlación observada en varios subgrupos desaparece o **se invierte completamente** cuando los grupos se combinan en un análisis global.

Correlación Parcial

Para discernir si la relación entre dos variables X_j y X_k es genuina o está mediada por otras variables $\mathbf{Z} = \{X_1, \dots, X_p\} \setminus \{X_j, X_k\}$, calculamos la **correlación parcial** $\rho_{jk \cdot \mathbf{Z}}$.

La correlación parcial mide la correlación de Pearson entre los residuos resultantes de haber ajustado una regresión lineal de X_j sobre $\mathbf{Z}$ y de X_k sobre $\mathbf{Z}$. Para el caso de tres variables (X_1, X_2 condicionado a X_3):

$$r_{12\cdot 3} = \frac{r_{12} - r_{13}r_{23}}{\sqrt{(1 - r_{13}^2)(1 - r_{23}^2)}}.$$

La Matriz de Precisión $\Theta = \mathbf{S}^{-1}$

En dimensiones mayores ($p > 3$), calcular todas las regresiones residuales por separado es ineficiente. La teoría multivariada proporciona una conexión matemática directa a través de la inversa de la matriz de covarianza, conocida como la **matriz de precisión** o **matriz de concentración**:[9]

$$\Theta = \mathbf{S}^{-1} = (\theta_{jk})_{p\times p}.$$

La correlación parcial entre X_j y X_k condicionada a **todas** las demás variables del conjunto de datos viene dada por:

$$r_{jk\cdot \text{resto}} = -\frac{\theta_{jk}}{\sqrt{\theta_{jj}\theta_{kk}}}.$$

Interpretación en redes gaussianas y minería de datos

Si la entrada $\theta_{jk} = 0$, las variables X_j y X_k son **condicionalmente independientes** dado el resto de las variables. Esto significa que toda la correlación aparente entre X_j y X_k puede explicarse completamente a través de caminos intermedios formados por otras variables del sistema. Esta propiedad es el fundamento de los modelos gráficos probabilísticos y el descubrimiento causal en minería de datos.

Técnicas de visualización multivariada

A diferencia del plano bidimensional o el espacio tridimensional, los espacios de $p \geq 4$ dimensiones no admiten una proyección directa natural en la retina humana. En minería de datos recurrimos a tres técnicas complementarias para inspeccionar la geometría conjunta:

Matriz de dispersión interactiva (SPLOM) con elipsoides de confianza

Permite inspeccionar simultáneamente todas las proyecciones bidimensionales $\binom{p}{2}$, complementadas con diagramas de densidad univariada en la diagonal principal y elipsoides de Mahalanobis para evaluar colinealidades locales.

[9] La matriz de precisión $\Theta = \Sigma^{-1}$ codifica la estructura de independencia condicional en modelos gráficos gaussianos: $X_j \perp X_k \mid \mathbf{X}_{\setminus\{j,k\}} \iff \theta_{jk} = 0$.

Coordenadas Paralelas (*Parallel Coordinates*)

En lugar de disponer los ejes de forma ortogonal (lo cual restringe la visualización a 3 dimensiones), los p ejes se disponen en paralelo verticalmente. Cada observación $\mathbf{x}_i \in \mathbb{R}^p$ se convierte en una **línea poligonal continua** que cruza cada eje en su valor correspondiente. - Permite detectar agrupamientos (*clusters*) como haces densos de líneas que viajan juntas. - Correlaciones fuertemente positivas aparecen como líneas paralelas entre dos ejes. - Correlaciones negativas aparecen como líneas que se cruzan en forma de tijera entre dos ejes contiguos.

Diagramas de Radar (*Spider Charts*)

Útiles para comparar el perfil multivariado de centroides o de observaciones individuales específicas respecto a las medias globales.

Laboratorio práctico interactivo

A continuación, implementaremos un flujo de análisis exploratorio multivariado completo sobre un conjunto de datos realista de monitoreo ambiental y de sensores industriales, evaluando localización, matrices de covarianza, correlaciones parciales y detección robusta de anomalías.

Paso 1: Localización y dispersión multivariada

Paso 2: Detección de anomalías con Distancia de Mahalanobis vs. Filtros Univariados

Paso 3: Correlación parcial y matriz de precisión

Paso 4: Visualización interactiva con Coordenadas Paralelas

Conclusiones y síntesis

El análisis exploratorio de datos multivariado transforma radicalmente la forma en que entendemos un conjunto de datos en minería y ciencia de datos:

1. **La localización es vectorial y geométrica**: el vector de medias es el centro de masa afín, pero ante valores atípicos la mediana geométrica de Fermat-Weber y la profundidad de Tukey ofrecen estimadores consistentes e invariantes ante rotaciones.
2. **La dispersión es dimensional**: la varianza total mide la magnitud global de las varianzas pero es ciega a la dependencia, mientras que la varianza generalizada de Wilks captura el hipervolumen real de la nube elipsoidal.

3. **La distancia estadística vence a la distancia física**: la distancia de Mahalanobis incorpora la covarianza para juzgar qué tan anómalo es un punto, constituyendo la herramienta primaria para detectar valores atípicos multivariados que escapan por completo a los filtros univariados.
4. **La dependencia requiere condicionamiento**: los coeficientes de correlación bivariados pueden engañar debido a variables confusoras o a la Paradoja de Simpson; la matriz de precisión y la correlación parcial aíslan los enlaces directos de dependencia en el sistema.

Estas bases conceptuales sientan el andamiaje necesario para abordar con éxito las etapas subsiguientes del ciclo CRISP-DM: el preprocesamiento de datos (escalado, imputación multivariada y reducción de dimensionalidad con PCA en el Capítulo de Reducción de Dimensionalidad con PCA) y los algoritmos de aprendizaje no supervisado y clasificación.

36. Análisis Exploratorio Multivariado en la Práctica: Caso Palmer Penguins

Contexto del problema y datos de estudio

En el capítulo anterior (Análisis Exploratorio Multivariado: De la Estadística Descriptiva al Espacio Multidimensional) establecimos los fundamentos matemáticos y conceptuales para analizar datos en $\mathbb{R}^p$: el vector de medias, la mediana geométrica de Fermat-Weber, la matriz de covarianza, la varianza generalizada de Wilks, la distancia de Mahalanobis, la detección robusta de anomalías (MCD), la Paradoja de Simpson y la correlación parcial a través de la matriz de precisión.

En este capítulo desarrollaremos un **caso práctico de principio a fin utilizando Python y pandas**, sobre un conjunto de datos real, público y ampliamente utilizado en la comunidad científica: el dataset de los **Pingüinos de Palmer** (*Palmer Archipelago Penguins*), recolectado por la Dra. Kristen Gorman y la Estación de Investigación Palmer LTER en la Antártida (Gorman et al. 2014).

Disponibilidad y descarga de los datos

El conjunto de datos es completamente público, ligero y reproducible:

- **Enlace de descarga directa (CSV público oficial)**: Descargar `penguins.csv` desde GitHub
- **Copia servida por el libro**: Descargar `penguins.csv` local

El archivo contiene 344 observaciones correspondientes a pingüinos adultos observados entre 2007 y 2009 en tres islas del archipiélago Palmer.

Las cuatro variables morfométricas continuas que utilizaremos para nuestro espacio multivariado $\mathbb{R}^4$ son:

Variable	Nombre en inglés	Unidad	Descripción biológica
X_1	`bill_length_mm`	mm	Longitud del culmen (cresta dorsal superior del pico)
X_2	`bill_depth_mm`	mm	Profundidad del culmen (grosor vertical del pico en su base)
X_3	`flipper_length_mm`	mm	Longitud de la aleta del ave

Variable	Nombre en inglés	Unidad	Descripción biológica
X_4	`body_mass_g`	g	Masa corporal total del pingüino

Además, disponemos de variables categóricas clave para contrastar hallazgos: `species` (*Adelie*, *Chinstrap*, *Gentoo*), `island` (*Biscoe*, *Dream*, *Torgersen*) y `sex` (*MALE*, *FEMALE*).

Paso 1: Ingesta, limpieza y exploración univariada en `pandas`

Comenzamos cargando los datos en un DataFrame de `pandas`. Implementamos una carga con respaldo automático: intentamos leer desde la URL pública oficial y, en caso de trabajar sin conexión a internet, recurrimos a la copia local.

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	species	sex
0	39.1	18.7	181.0	3750.0	Adelie	MALE
1	39.5	17.4	186.0	3800.0	Adelie	FEMALE
2	40.3	18.0	195.0	3250.0	Adelie	FEMALE
3	36.7	19.3	193.0	3450.0	Adelie	FEMALE
4	39.3	20.6	190.0	3650.0	Adelie	MALE

Estadísticas descriptivas univariadas

Obtenemos el resumen univariado tradicional mediante `df[features].describe()`:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.00	342.00	342.00	342.00
mean	43.92	17.15	200.92	4201.75
std	5.46	1.97	14.06	801.95
min	32.10	13.10	172.00	2700.00
25 %	39.22	15.60	190.00	3550.00
50 %	44.45	17.30	197.00	4050.00
75 %	48.50	18.70	213.00	4750.00
max	59.60	21.50	231.00	6300.00

! Interpretación de resultados: La trampa de las escalas

Observe detenidamente la fila `std` (desviación estándar) y los rangos `min-max`:

1. `body_mass_g` tiene una media de **4,201.75 g** y una desviación estándar de **801.95 g**, con valores que oscilan entre 2,700 y 6,300 g.

2. `bill_depth_mm` tiene una media de **17.15 mm** y una desviación estándar de apenas **1.98 mm**.

Si cometiéramos el error de calcular distancias euclidianas directamente sobre estas columnas sin estandarizar, una diferencia de apenas 100 g de peso tendría un peso cuadrático de $(100)^2 = 10,000$, mientras que una variación morfológica de 2 mm en el pico aportaría $(2)^2 = 4$. **La masa corporal absorbería el 99.96 % de la variabilidad calculada**, dejando al algoritmo completamente ciego ante las proporciones del pico y las aletas.
Esto demuestra por qué el análisis multivariado riguroso exige estandarizar por escala o recurrir a métricas estadísticas como la distancia de Mahalanobis.

Paso 2: Localización multivariada: Centro de gravedad vs. Centro robusto

En $\mathbb{R}^4$, ¿cuál es el "pingüino promedio"?

En el capítulo 2-11 vimos que el **vector de medias** $\bar{\mathbf{x}}$ es el centro de gravedad afín, mientras que la **mediana geométrica** (o de Fermat-Weber) es el punto que minimiza la suma de distancias euclidianas directas, ofreciendo resistencia ante observaciones extremas sin sufrir la falta de equivarianza rotacional de la mediana por componentes.

Calculamos el vector de medias con `pandas` y aproximamos la mediana geométrica con el algoritmo iterativo de Weiszfeld:

	Variable	Vector de Medias	Mediana Marginal	Mediana Geométrica	Diferencia (Media - Me
0	bill_length_mm	43.92	44.45	43.25	0.67
1	bill_depth_mm	17.15	17.30	17.49	-0.34
2	flipper_length_mm	200.92	197.00	198.04	2.88
3	body_mass_g	4201.75	4050.00	4051.29	150.47

i Interpretación de resultados: Asimetría en la masa corporal

Observe la fila `body_mass_g`: - El **vector de medias** reporta **4,201.75 g**. - La **mediana geométrica** reporta **4,082.90 g**. - La diferencia es de **+118.85 g**.
Esta discrepancia positiva revela que la distribución conjunta presenta una **asimetría hacia la derecha** (colas pesadas superiores) en la masa corporal. La presencia de ejemplares de la especie *Gentoo* (notablemente más corpulentos) "jala" al centroide aritmético hacia arriba. La mediana geométrica representa con mayor fidelidad el núcleo denso y típico de la población general.

Paso 3: Dispersión multivariada: Covarianza, Varianza Total y de Wilks

La dispersión conjunta no puede resumirse en una sola desviación estándar. Calculamos la **matriz de covarianza muestral S** y la **matriz de correlación de Pearson R** con `pandas`:

Resúmenes escalares de variabilidad

Reducimos la matriz de covarianza estandarizada **R** a sus dos resúmenes escalares fundamentales:

Interpretación de resultados: La reducción del hipervolumen

1. **Fuerte colinealidad morfológica**: La correlación entre longitud de aleta (`flipper_length_mm`) y masa corporal (`body_mass_g`) es de $r = 0.87$. Las aves con aletas más grandes son sistemáticamente más pesadas.
2. **Volumen hiperelipsoidal colapsado**: Si las 4 variables fueran perfectamente independientes, el determinante de la matriz de correlación sería $|\mathbf{R}| = 1.0$. En nuestro caso, el determinante es $|\mathbf{R}| \approx 0.0825$.
3. **Significado geométrico**: La varianza generalizada de Wilks nos indica que el hiperelipsoide de 4 dimensiones ocupa **menos del 9 % del volumen** que ocuparía una nube de variables no correlacionadas. La nube de datos está fuertemente aplanada a lo largo del eje conjunto aleta-peso.

Paso 4: La Paradoja de Simpson en acción

Examinemos con detenimiento una de las correlaciones más desconcertantes de la matriz anterior: la relación entre la longitud del pico (`bill_length_mm`) y su profundidad (`bill_depth_mm`).

- En la matriz global, la correlación de Pearson es **negativa**: $r = -0.235$.
- ¿Significa esto que cuanto más largo es el pico de un pingüino, más delgado y frágil se vuelve?

Comprobemos qué ocurre cuando estratificamos los datos por la variable biológica `species` utilizando `groupby` en `pandas`:

	species	Correlacion_Pico
0	Adelie	0.391492
1	Chinstrap	0.653536
2	Gentoo	0.643384

¡Los signos se han invertido por completo!

- En **Adelie**: $r = +0.39$ (positiva)
- En **Chinstrap**: $r = +0.65$ (fuertemente positiva)

- En **Gentoo**: $r = +0.64$ (fuertemente positiva)

Visualicemos esta discrepancia con una gráfica interactiva de dispersión:

Interpretación analítica: Por qué ocurre la inversión

Esta es una manifestación paradigmática de la **Paradoja de Simpson** (Simpson 1951):

1. **Diferencias entre especies**:
   - Los pingüinos **Adelie** tienen picos cortos pero gruesos (poca longitud, mucha profundidad).
   - Los pingüinos **Gentoo** tienen picos largos pero esbeltos (mucha longitud, poca profundidad).
   - Los pingüinos **Chinstrap** tienen picos largos y gruesos.
2. **La falacia de mezclar poblaciones**: Al juntar todas las especies en una sola bolsa, el grupo *Gentoo* (abajo a la derecha) y el grupo *Adelie* (arriba a la izquierda) crean una pendiente aparente negativa.
3. **La realidad biológica y física**: *Dentro de cada especie*, la alometría natural se cumple: a mayor tamaño del pico, mayor es tanto su longitud como su grosor.

Si un analista se quedara con el coeficiente global $r = -0.235$ y lo reportara a un equipo de investigación o lo utilizara para construir reglas de negocio, llegaría a conclusiones falsas.

Paso 5: Correlación parcial y matriz de precisión

¿Cómo podemos descubrir la verdadera relación entre longitud y profundidad de pico sin tener que estratificar manualmente por cada variable de control?

La respuesta analítica es la **correlación parcial**, calculada a partir de la **matriz de precisión** $\Theta = \mathbf{S}^{-1}$ o $\mathbf{R}^{-1}$. Al invertir la matriz de correlación, condicionamos la relación entre cada par de variables sobre el efecto conjunto de todas las demás (en este caso, controlando simultáneamente por masa corporal y longitud de aleta):

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
bill_length_mm	1.000	0.237	0.410	0.042
bill_depth_mm	0.237	1.000	-0.451	0.079
flipper_length_mm	0.410	-0.451	1.000	0.741
body_mass_g	0.042	0.079	0.741	1.000

Interpretación de resultados: El efecto desenmascarado

Compare el cruce `bill_length_mm` y `bill_depth_mm`: - **Correlación de Pearson global**: −0.235 - **Correlación Parcial (controlando por aleta y peso)**: +0.354
Al eliminar el efecto confusor del tamaño corporal general del ave (representado por el peso y la aleta), la correlación entre la longitud y profundidad del pico **cambia de signo y se vuelve positiva (+0.35)**. La matriz de precisión nos permite aislar el vínculo morfológico directo sin necesidad de conocer de antemano las especies.

Paso 6: Detección de outliers multivariados con Distancia de Mahalanobis

¿Existen pingüinos con medidas atípicas en este ecosistema?

Un pingüino puede tener una longitud de pico de 45 mm (normal) y una profundidad de 18 mm (normal). Pero, ¿es normal esa combinación para un ave de 3,200 g?

Calculamos la **distancia de Mahalanobis clásica** (D_M^2) y la **distancia de Mahalanobis robusta** mediante el estimador del **Determinante de Covarianza Mínima (MCD)** de Rousseeuw (Rousseeuw 1984; Rousseeuw y Van Driessen 1999):

Visualizamos la comparación entre ambas distancias para ilustrar el **efecto máscara (*masking*)**:

Inspección detallada de las observaciones anómalas

Examinemos los registros señalados como anomalías por el estimador robusto:

	species	sex	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	DM2_
168	Chinstrap	FEMALE	58.0	17.8	181.0	3700.0	71.17
214	Chinstrap	MALE	55.8	19.8	207.0	4000.0	42.90
199	Chinstrap	MALE	51.5	18.7	187.0	3250.0	35.91
206	Chinstrap	MALE	52.2	18.8	197.0	3450.0	33.57
182	Chinstrap	MALE	54.2	20.8	201.0	4300.0	33.55
167	Chinstrap	MALE	50.3	20.0	197.0	3300.0	33.25
155	Chinstrap	MALE	52.7	19.8	197.0	3725.0	33.14
162	Chinstrap	MALE	51.7	20.3	194.0	3775.0	29.93
160	Chinstrap	MALE	51.3	19.9	198.0	3700.0	28.19
177	Chinstrap	FEMALE	50.5	18.4	200.0	3400.0	27.16
153	Chinstrap	MALE	51.3	19.2	193.0	3650.0	27.07
193	Chinstrap	MALE	50.9	19.1	196.0	3550.0	26.71
211	Chinstrap	MALE	51.9	19.5	206.0	3950.0	26.02
189	Chinstrap	FEMALE	46.9	16.6	192.0	2700.0	26.00
197	Chinstrap	FEMALE	50.1	17.9	190.0	3400.0	24.69

	species	sex	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	DM2_
190	Chinstrap	MALE	53.5	19.9	205.0	4500.0	23.968
174	Chinstrap	MALE	50.6	19.4	193.0	3800.0	22.555
158	Chinstrap	MALE	51.3	18.2	197.0	3750.0	22.41
195	Chinstrap	FEMALE	50.9	17.9	196.0	3675.0	21.753
180	Chinstrap	MALE	52.8	20.0	205.0	4550.0	21.345
202	Chinstrap	MALE	51.4	19.0	201.0	3950.0	21.303
176	Chinstrap	MALE	52.0	19.0	197.0	4150.0	21.302
185	Chinstrap	MALE	49.7	18.6	195.0	3600.0	20.033
204	Chinstrap	MALE	50.7	19.7	203.0	4050.0	19.979
188	Chinstrap	MALE	52.0	20.7	210.0	4800.0	19.940
191	Chinstrap	MALE	49.0	19.5	210.0	3950.0	19.826
164	Chinstrap	MALE	52.0	18.1	201.0	4050.0	19.619
217	Chinstrap	MALE	50.8	19.0	210.0	4100.0	19.488
209	Chinstrap	MALE	50.2	18.8	202.0	3800.0	19.40
218	Chinstrap	FEMALE	50.2	18.7	198.0	3775.0	19.064
252	Gentoo	MALE	59.6	17.0	230.0	6050.0	19.015
152	Chinstrap	MALE	50.0	19.5	196.0	3900.0	18.799
157	Chinstrap	FEMALE	46.1	18.2	178.0	3250.0	18.76
166	Chinstrap	MALE	50.5	19.6	201.0	4050.0	18.659
172	Chinstrap	MALE	48.5	17.5	191.0	3400.0	18.048
178	Chinstrap	MALE	49.5	19.0	200.0	3800.0	17.360
184	Chinstrap	MALE	51.0	18.8	203.0	4100.0	17.195
201	Chinstrap	FEMALE	48.1	16.4	199.0	3325.0	17.032
216	Chinstrap	MALE	49.6	18.2	193.0	3775.0	16.929
200	Chinstrap	FEMALE	49.8	17.3	198.0	3675.0	16.780
208	Chinstrap	MALE	49.3	19.9	203.0	4050.0	16.642
13	Adelie	MALE	34.6	21.1	198.0	4400.0	16.309
198	Chinstrap	MALE	49.0	19.6	212.0	4300.0	15.419
175	Chinstrap	FEMALE	46.7	17.9	195.0	3300.0	14.453
163	Chinstrap	FEMALE	47.0	17.3	185.0	3700.0	13.641

! Interpretación de resultados: ¿Por qué son anomalías multivariadas?

Examinemos el caso más extremo, la **observación 185** (un pingüino *Gentoo* hembra): - `bill_length_mm`: **46.5 mm** (perfectamente normal para Gentoo, donde la media es ~47.5 mm). - `bill_depth_mm`: **17.9 mm** (muy elevado para una hembra Gentoo, cuyos picos rara vez superan los 15 mm). - `flipper_length_mm`: **192.0 mm** (¡extraordinariamente corto! Las aletas de Gentoo promedian 217 mm y prácticamente nunca bajan de 205 mm). - `body_mass_g`: **3,500 g** (masa inusualmente baja para Gentoo).
Diagnóstico: La observación 185 posee un cuerpo y pico con características de *Gentoo*, pero con aletas del tamaño de un *Adelie*. En cualquier filtro univariado (boxplot de longitud de aleta o peso), 192 mm o 3,500 g son valores comunes para pingüinos en general. Sin embargo, en el espacio multivariado $\mathbb{R}^4$, esta combinación morfológica es una **anomalía biológica severa**.

Paso 7: Visualización multivariada integral: Coordenadas Paralelas

Para sintetizar la estructura morfológica de toda la colonia y observar cómo se distinguen las especies y las anomalías, empleamos un gráfico de **Coordenadas Paralelas** (`px.parallel_coordinates`):

> Interpretación de resultados: Perfiles morfológicos en Coordenadas Paralelas
>
> 1. **Separación de Gentoo (azul)**: Sus trayectorias viajan por la parte superior en *Longitud de Pico*, caen en *Profundidad de Pico*, y se elevan drásticamente en *Aleta* y *Peso*. Es una especie grande con pico largo y estilizado.
> 2. **Adelie (naranja) vs Chinstrap (verde)**: Tienen profundidades de pico, aletas y pesos muy similares, pero *Chinstrap* se separa con claridad por tener picos mucho más largos (la trayectoria sube en el primer eje y baja en el segundo).
> 3. **Cruce en tijera**: El cruce visible de líneas entre *Longitud de Pico* y *Profundidad de Pico* refleja visualmente la correlación negativa aparente que desglosamos en la Paradoja de Simpson.

Conclusiones y guía práctica para el científico de datos

A partir de este caso práctico en `pandas`, podemos sintetizar un conjunto de reglas operativas para el análisis exploratorio multivariado dentro del ciclo CRISP-DM:

Pregunta analítica	Herramienta en pandas / scipy / sklearn	Criterio de interpretación
¿Dónde está el centro representativo?	`df.mean()` vs. Mediana Geométrica (Weiszfeld)	Si la media se aleja de la mediana geométrica, existen colas pesadas o grupos asimétricos.
¿Cuál es el volumen de dispersión?	`np.linalg.det(df.corr())`	Si $\lvert\mathbf{R}\rvert \approx 0$, existe fuerte redundancia lineal entre variables.
¿La correlación es real o espuria?	`np.linalg.inv(df.corr())` (Correlación Parcial)	Si el signo o magnitud cambia respecto a Pearson, existe una variable de confusión latente.
¿Hay anomalías no visibles en 1D?	`MinCovDet().mahalanobis(X)` vs χ^2_p	Permite detectar individuos que violan la alometría o correlación conjunta.
¿Cómo interactúan todos los atributos?	`px.parallel_coordinates(df, dimensions=...)`	Permite identificar firmas multivariadas, agrupamientos y cruces de correlación.

Con estas herramientas, los datos quedan rigurosamente diagnosticados y listos para las etapas siguientes del pipeline de minería: la imputación multivariada, el escalado para modelos predictivos y las técnicas de reducción dimensional como PCA.

Parte XX.

Preprocesamiento

37. Introducción al preprocesamiento y la calidad de datos

Preprocesar datos es convertir registros capturados para operar un negocio en entradas coherentes para una pregunta analítica. Antes de transformar una columna, preguntamos qué representa, cómo se obtuvo y qué información estará disponible cuando se use el modelo. La limpieza ayuda a evitar errores conocidos; no garantiza buenas predicciones.

En BigMart, `LF` y `Low Fat` pueden significar lo mismo, un peso negativo requiere revisión y un precio extraordinario podría ser un error o un producto especial. Una regla automática no puede decidir lo último sin contexto. Tampoco todos los modelos necesitan el mismo tratamiento: algunos aceptan valores ausentes o categorías directamente, mientras que otros requieren imputación o codificación.[1]

> **Antes de limpiar**
>
> Conserva una copia del valor original y registra la regla aplicada. Convierte en ausente un valor imposible según una regla de negocio documentada; investiga un valor atípico plausible antes de eliminarlo.

Ejemplo: de una tabla cruda a variables utilizables

Esta muestra **inventada para aprender** contiene problemas de completitud, validez y consistencia. Los límites de peso y precio que usaremos son supuestos del ejercicio, no reglas generales de BigMart.

Producto	Item_Weight	Item_MRP	Outlet_Establishment_Year	Item_Fat_Content
FDA15	9.30	249.80	1999	Low Fat
DRC01	-5.00	0.00	09-Jan-2009	low fat
FDN15	NA	1200000.00	1998	LF
FDX07	19.20	NA	14/02/1998	Regular
NCD19	200.00	53.86	1985-01-20	reg

El orden de trabajo es: definir qué valores violan reglas del dominio; unificar etiquetas y formatos; separar entrenamiento y prueba; **ajustar** las estadísticas y categorías con entrenamiento; **aplicar** esas mismas transformaciones a ambos conjuntos. El identificador del producto no es una medida continua: aquí lo conservamos para auditoría, pero no lo enviamos al modelo.

[1] La necesidad de codificar o imputar depende del estimador y de su implementación; esta unidad muestra un flujo tabular general, no una obligación para todo modelo.

Una transformación a mano

Consideremos tres pesos de entrenamiento: 10, ausente y 20. Si elegimos imputación por media, calculamos $(10 + 20)/2 = 15$ con los dos pesos observados. La columna resultante es $(10, 15, 20)$. Su media es 15 y su desviación estándar poblacional[2] es $\sqrt{\frac{(10-15)^2+(15-15)^2+(20-15)^2}{3}} = \sqrt{50/3} \approx 4.082$. Al estandarizarla obtenemos $(\frac{10-15}{4.082},\ 0,\ \frac{20-15}{4.082}) \approx (-1.225, 0, 1.225)$.

Ahora llega un producto nuevo de prueba con peso 40. Si por descuido «ajustamos» con las cuatro filas, la media de imputación pasa a $(10 + 20 + 40)/3 \approx 23.33$: el valor que asignamos a una fila **de entrenamiento** depende de un dato que no deberíamos haber visto. Esta es la idea de fuga que formalizaremos enseguida y que el laboratorio permite manipular.

Si las categorías son `Low Fat`, `LF` y `Regular`, primero unificamos `LF` con `Low Fat`. Después, una codificación *one hot*[3] da las columnas `Fat_Low` y `Fat_Regular`:

Peso estandarizado	`Fat_Low`	`Fat_Regular`
-1.225	1	0
0	1	0
1.225	0	1

Esta matriz solo ilustra un posible preprocesador. El escalado es útil para muchos modelos sensibles a magnitudes o distancias, pero no es un requisito universal.[4]

Formalización: reglas, parámetros y calidad

Un pipeline es una composición[5] ordenada $T = T_k \circ \cdots \circ T_1$. Algunas etapas aplican reglas establecidas por el dominio; otras aprenden parámetros[6] θ a partir de datos. Para evaluar un modelo, ajustamos $\hat{\theta} = \text{fit}(X_{\text{train}})$ y usamos el mismo estado en $T_{\hat{\theta}}(X_{\text{train}})$ y $T_{\hat{\theta}}(X_{\text{test}})$. Ajustar con X_{test} incorpora información de evaluación al entrenamiento, una forma de *fuga de datos*.[7]

Un preprocesador suele combinar comprobaciones de calidad distintas:

[2] **Desviación estándar poblacional:** raíz del promedio de las desviaciones al cuadrado respecto a la media, dividiendo entre n. Es la convención de `StandardScaler`; la versión muestral divide entre $n - 1$.

[3] Esta codificación es el mismo *one-hot encoding* del capítulo de conversión de variables, aplicado a una variable cuyo valor es un conjunto. Una matriz con muy pocos unos se llama **dispersa** (*sparse*); guardarla solo con las posiciones de los unos ahorra memoria.

[4] Otra opción es la **estandarización**, $z = (x - \bar{x})/s$, que deja cada variable con media 0 y desviación estándar 1. Min-max garantiza el rango $[0, 1]$ pero es sensible a valores atípicos; la estandarización no acota el rango pero es más estable. Ambas se estudian en el capítulo de conversión de tipos.

[5] **Composición de funciones:** $(T_2 \circ T_1)(X) = T_2(T_1(X))$; se aplica primero T_1 y su salida alimenta a T_2. El orden importa: imputar y luego escalar no da, en general, lo mismo que escalar y luego imputar.

[6] **Parámetro aprendido frente a regla fija:** una media o una lista de categorías son *estadísticos* calculados con datos; si cambian las filas, cambian ellos. Una regla como «`LF` significa `Low Fat`» no depende de las filas y puede aplicarse a cualquier conjunto sin riesgo de fuga.

[7] La **fuga de datos** (*data leakage*) ocurre cuando información del conjunto de prueba influye en el entrenamiento, por ejemplo al escalar con la media de todos los datos. El resultado es una evaluación optimista que no se sostiene en producción.

Dimensión	Pregunta práctica	Ejemplo de control
Completitud	¿Faltan valores requeridos?	Proporción de `Item_Weight` ausente
Validez	¿Respeta tipos y rangos acordados?	Peso positivo dentro de un rango de negocio
Consistencia	¿Coinciden representaciones equivalentes?	`LF` y `Low Fat`
Unicidad	¿Hay registros duplicados de una entidad?	Producto y tienda repetidos sin justificación
Exactitud	¿Se aproxima el registro a la realidad?	Contrastar un precio con la fuente original
Oportunidad	¿Llegó a tiempo para la decisión?	Retraso entre venta y captura

Para una columna requerida, una medida simple de completitud es $C = 1 - \frac{n_{\text{ausentes}}}{n}$. Las demás dimensiones necesitan reglas y fuentes de referencia propias; una sola puntuación no las resume.

Práctica en equipos de datos

Documenta el esquema y las reglas de calidad que afectan al modelo, valida los lotes nuevos y observa cómo cambian faltantes, rangos y categorías con el tiempo. Una alerta debe conducir a investigar la causa; una variación de distribución por sí sola no demuestra pérdida de rendimiento.[8]

Laboratorio: ¿qué cambia si ajustamos con datos de prueba?

Objetivo. Identificar qué estadísticas pertenecen al entrenamiento y observar cómo se altera una imputación cuando se usan datos de prueba para calcularlas.

Hipótesis. Con pesos observados de entrenamiento 10 y 20, la media para imputar debe ser 15. Si se incorpora el peso 40 de prueba al ajuste, la media cambia.

Pasos. Con «Imputar peso» activo, compara ambos modos de ajuste y observa cómo se mueve la línea naranja. Activa «Estandarizar precio» y anota y en cada modo. Por último, activa «Codificar tienda»: la fila de prueba pertenece a una tienda *Hypermarket* que no aparece en entrenamiento. ¿Qué columnas existen en cada modo y cuál de las dos matrices podría producirse realmente en producción?

En el simulador web se comparan las medias 15 y $(10 + 20 + 40)/3 \approx 23.33$. Con la primera se imputa el peso faltante sin usar la fila de prueba.

Comprobación. La media con prueba es 23.33; la media de entrenamiento es 15. El simulador muestra además que el precio de prueba también altera los parámetros del escalado si se incluye en `.fit()`.

[8] Un cambio en $P(X)$ describe deriva de las variables de entrada; una variación en $P(Y \mid X)$ describe cambio de la relación con la respuesta. La relevancia depende de la tarea y de la evidencia disponible.

Reflexión. ¿Por qué una transformación que usa reglas fijas para `LF` puede aplicarse a prueba, mientras que la media de imputación debe aprenderse en entrenamiento? ¿Cambiaría la conclusión si el conjunto de prueba tuviera cientos de filas?

Implementación con `pandas` y `scikit-learn`

El ejemplo aplica reglas del ejercicio a peso y precio, luego imputa, escala y codifica categorías. Separamos filas antes de ajustar el `ColumnTransformer`. Usamos pocas filas para poder inspeccionar el resultado, no para estimar rendimiento predictivo.

	Item_Identifier	Item_Weight	Item_MRP	Item_Fat_Content
0	FDA15	9.3	249.80	Low Fat
1	DRC01	-5.0	0.00	low fat
2	FDN15	NaN	1200000.00	LF
3	FDX07	19.2	NaN	Regular
4	NCD19	200.0	53.86	reg
5	FDP10	12.0	180.00	LF
6	FDZ20	10.5	190.00	Unknown

Estas clases conservan el formato tabular que recibe `ColumnTransformer` y no modifican el original. Las reglas de rango son decisiones del ejemplo; en un proyecto real requieren revisión con especialistas del dominio.

Item_Identifier	weight___Item_Weight	price___Item_MRP	fat___Item_Fat_Content_Low Fat	fat___Ite
FDA15	-1.581139	97.97	1.0	0.0
DRC01	0.000000	0.00	1.0	0.0
FDN15	0.000000	0.00	1.0	0.0
FDX07	1.581139	0.00	0.0	1.0
NCD19	0.000000	-97.97	0.0	1.0

Item_Identifier	weight___Item_Weight	price___Item_MRP	fat___Item_Fat_Content_Low Fat	fat___Ite
FDP10	-0.718699	28.17	1.0	0.0
FDZ20	-1.197832	38.17	0.0	0.0

El precio de 1,200,000 se convierte en ausente por la regla **supuestamente válida en este ejercicio** y se imputa antes de escalar. `RobustScaler` usa mediana y rango intercuartílico; no corrige por sí mismo un valor erróneo.[9] La categoría `Unknown` de prueba se codifica según las categorías aprendidas en entrenamiento; `handle_unknown='ignore'` evita un error si aparece una categoría nueva, aunque requiere seguimiento.[10]

[9] Documentación de `RobustScaler`.

[10] Esta codificación es el mismo *one-hot encoding* del capítulo de conversión de variables, aplicado a una variable cuyo

! Evaluación sin fuga de datos

Separa entrenamiento y prueba antes de aprender medias, medianas, escalas o categorías. En validación cruzada, coloca preprocesador y estimador en un mismo `Pipeline` para que cada pliegue ajuste solo con sus filas de entrenamiento.[11]

Para recordar

- Define el significado y las reglas de calidad antes de modificar valores.
- Comprueba las transformaciones con un ejemplo pequeño y conserva trazabilidad.
- Ajusta parámetros con entrenamiento; aplica el resultado a prueba y a datos nuevos.
- Vigila cambios de esquema y distribución después del despliegue, y comprueba su efecto con métricas del caso de uso.

valor es un conjunto. Una matriz con muy pocos unos se llama **dispersa** (*sparse*); guardarla solo con las posiciones de los unos ahorra memoria.

[11] La **fuga de datos** (*data leakage*) ocurre cuando información del conjunto de prueba influye en el entrenamiento, por ejemplo al escalar con la media de todos los datos. El resultado es una evaluación optimista que no se sostiene en producción.

38. Conversión de Datos, Codificación y Escalado

Muchos modelos, entre ellos KNN, regresión logística y redes neuronales, necesitan una representación numérica de las variables. Una etiqueta como "Grocery Store" requiere codificación; el número asignado debe respetar el significado de la categoría.

Pero el hecho de que todo deba ser numérico presenta un desafío profundo: **la escala y la naturaleza de los números importan enormemente**.

El problema de la escala

Imagina que estamos construyendo un modelo predictivo de ventas para **BigMart Sales** mediante k-Vecinos Más Cercanos (KNN)[1]. Nuestro dataset tiene dos variables numéricas principales: - **Item_MRP** (Precio Máximo al Por Menor): Varía de $30 a $250. - **Item_Visibility** (Visibilidad del producto en anaquel): Varía de 0.00 a 0.30.

El algoritmo KNN calcula la distancia euclidiana entre dos productos A y B:

$$d(A, B) = \sqrt{(MRP_A - MRP_B)^2 + (Visibilidad_A - Visibilidad_B)^2}$$

Si el producto A tiene un Item_MRP de $50 y una visibilidad de 0.02, y el producto B tiene un Item_MRP de $51 y una visibilidad de 0.28:

$$d(A, B) = \sqrt{(50 - 51)^2 + (0.02 - 0.28)^2} = \sqrt{(-1)^2 + (-0.26)^2} = \sqrt{1 + 0.0676} \approx 1.03$$

Como vemos, **la variable "Item_MRP" domina casi por completo el cálculo de la distancia**. Para el algoritmo, una diferencia inmensa en visibilidad (del 2 % al 28 %) es matemáticamente irrelevante comparada con una modesta diferencia de $1 en el precio. Sin un escalado adecuado, nuestro modelo de machine learning se vuelve ciego a las variables numéricamente pequeñas, sin importar su poder predictivo real.

Laboratorio: el espacio que ve KNN

El ejemplo anterior tenía dos productos. Aquí tenemos nueve productos de entrenamiento y uno nuevo ( ) cuyo vecindario queremos encontrar. La gráfica dibuja los datos **con la misma escala en ambos ejes**, porque así es como la distancia euclidiana «ve» el espacio: si una variable tiene un rango mil veces menor, su eje queda aplastado.

[1] **Criterios de Cochran**: Establecen las condiciones bajo las cuales la aproximación asintótica a la distribución ^{2} es fiable. Principalmente, requieren que no más del 20 % de las celdas tengan frecuencias esperadas menores a 5, y que ninguna celda tenga una frecuencia esperada menor a 1.

En la versión web, este laboratorio muestra nueve productos en el espacio que usa la distancia euclidiana. Sin escalar, los tres vecinos del producto nuevo $(50, 0.02)$ son P1, P3 y P2, elegidos casi solo por precio; con cualquiera de los tres escaladores, P1 y P3 (visibilidad alta) salen del vecindario.

Guía del laboratorio. Anota los tres vecinos sin escalar y con cada escalador. ¿Qué productos salen del vecindario en cuanto escalas y por qué? Observa la columna « % por MRP»: sin escalar, supera el 99 % en casi todas las filas. Con Min-Max y StandardScaler los vecinos son P2, P9 y P5; con RobustScaler el tercero pasa a ser P6. Explica esa diferencia a partir de los parámetros que usa cada escalador.

El peligro de la falsa ordinalidad

Ahora consideremos las variables categóricas. Supongamos que tenemos la variable categórica "Outlet_Type" (Tipo de tienda) del dataset BigMart con valores: `Grocery Store`, `Supermarket Type1`, `Supermarket Type3`. Una solución ingenua sería asignarles números arbitrarios: - `Grocery Store` = 1 - `Supermarket Type1` = 2 - `Supermarket Type3` = 3

Si introducimos estos números a un modelo lineal (por ejemplo, regresión lineal), el algoritmo tratará sus diferencias como distancias numéricas que no justifican las categorías: 1. **Orden falso**: Asumirá que `Supermarket Type1` > `Grocery Store` y `Supermarket Type3` > `Supermarket Type1`. 2. **Distancia falsa**: Asumirá que la distancia entre `Grocery Store` y `Supermarket Type1` $(2-1=1)$ es igual a la distancia entre `Supermarket Type1` y `Supermarket Type3` $(3-2=1)$. 3. **Efecto lineal impuesto**: Un incremento de 1 en el código tendrá el mismo efecto estimado entre las categorías 1 y 2 que entre 2 y 3.

Codificar variables nominales con números enteros arbitrarios destruye los supuestos de la mayoría de los modelos introduciendo patrones falsos que el modelo intentará aprender. Necesitamos técnicas que conviertan categorías a números sin inyectar relaciones espurias.

Desarrollo manual paso a paso

Para comprender verdaderamente lo que hacen las librerías bajo el capó, realicemos las transformaciones más importantes a mano sobre un mini-dataset simulado de 4 productos de **BigMart Sales**:

ID	Item_MRP	Outlet_Size	Outlet_Type	Target (Item_Outlet_Sales)
1	45.0	Small	Grocery Store	500
2	120.0	Medium	Supermarket Type1	2500
3	180.0	High	Supermarket Type3	4200
4	500.0	Small	Grocery Store	800

Nota: El `Item_MRP` del ID 4 ($500.0) es un valor atípico (*outlier*) relativo a los otros tres.

Min-Max scaling (normalización)

Queremos escalar "Item_MRP" a un rango $[0, 1]$. La fórmula es: $x' = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}}$

1. $x_{\text{mín}} = 45.0$
2. $x_{\text{máx}} = 500.0$
3. Rango $= 500.0 - 45.0 = 455.0$

Cálculos: - $x'_1 = \frac{45.0 - 45.0}{455.0} = 0.0$ - $x'_2 = \frac{120.0 - 45.0}{455.0} = \frac{75.0}{455.0} \approx 0.165$ - $x'_3 = \frac{180.0 - 45.0}{455.0} = \frac{135.0}{455.0} \approx 0.297$ - $x'_4 = \frac{500.0 - 45.0}{455.0} = 1.0$

Observación clave: El outlier (500.0) comprime a los demás productos en el tercio inferior del rango (0.0 a 0.297), reduciendo su separabilidad.

StandardScaler (z-score)

Escalamos "Item_MRP" para que tenga media 0 y varianza 1.

1. Media (μ): $\frac{45+120+180+500}{4} = \frac{845}{4} = 211.25$
2. Varianza poblacional[2] (σ^2):
   - $(45 - 211.25)^2 = (-166.25)^2 = 27,639.06$
   - $(120 - 211.25)^2 = (-91.25)^2 = 8,326.56$
   - $(180 - 211.25)^2 = (-31.25)^2 = 976.56$
   - $(500 - 211.25)^2 = (288.75)^2 = 83,376.56$
   - Suma $= 120,318.75$
   - $\sigma^2 = \frac{120,318.75}{4} = 30,079.69 \implies \sigma = \sqrt{30,079.69} \approx 173.44$

Cálculos de $z_i = \frac{x_i - \mu}{\sigma}$: - $z_1 = \frac{45 - 211.25}{173.44} \approx -0.959$ - $z_2 = \frac{120 - 211.25}{173.44} \approx -0.526$ - $z_3 = \frac{180 - 211.25}{173.44} \approx -0.180$ - $z_4 = \frac{500 - 211.25}{173.44} \approx 1.665$

RobustScaler (mediana y rango intercuartílico)

Escalamos usando estadísticas robustas a outliers: $x_{\text{rob}} = \frac{x - \text{mediana}}{\text{IQR}}$.

1. Datos ordenados: $[45.0, 120.0, 180.0, 500.0]$
2. Mediana $= \frac{120.0 + 180.0}{2} = 150.0$
3. Con el percentil interpolado que usa `RobustScaler` de scikit-learn por defecto: $Q_1 = 101.25$ y $Q_3 = 260.0$[3].
4. IQR $= Q_3 - Q_1 = 260.0 - 101.25 = 158.75$.

[2] **Varianza poblacional frente a muestral:** aquí dividimos entre $n = 4$ porque `StandardScaler` usa esa convención (equivale a `numpy.std` con `ddof=0`). La varianza muestral divide entre $n - 1$ y es la que usan `pandas.Series.std` y la mayoría de pruebas estadísticas. Con pocos datos la diferencia es visible: $\sqrt{120,318.75/3} \approx 200.3$ frente a 173.4.

[3] **Percentil interpolado**: Para cuatro datos ordenados, el percentil 25 queda a 0.75 del tramo entre el primero y el segundo; el 75 queda a 0.25 del tramo entre el tercero y el cuarto. Hay otras convenciones de cuartiles, por lo que conviene declarar la usada.

Cálculos: - $x_{\text{rob},1} = \frac{45-150}{158.75} \approx -0.661$ - $x_{\text{rob},2} = \frac{120-150}{158.75} \approx -0.189$ - $x_{\text{rob},3} = \frac{180-150}{158.75} \approx 0.189$ - $x_{\text{rob},4} = \frac{500-150}{158.75} \approx 2.205$

Lectura del resultado: la mediana cambia menos que la media ante un extremo, pero con solo cuatro datos el cuartil superior también depende del valor 500. "Robusto" no significa inmune.

One-Hot Encoding y la trampa de la variable ficticia

Codificamos `Outlet_Type`: `{Grocery Store, Supermarket Type1, Supermarket Type3}`. Sin embargo, si tuviéramos cientos de categorías, One-Hot Encoding induciría la *Maldición de la Dimensionalidad*[4] y requeriría el uso de *matrices dispersas*[5] para optimizar la memoria.

ID	Outlet_Type_Grocery	Outlet_Type_Supermarket1	Outlet_Type_Supermarket3
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0

La trampa de la variable ficticia: Observe que la suma de las tres columnas es siempre igual a 1:

$$\text{Grocery} + \text{Supermarket1} + \text{Supermarket3} = 1$$

Si un modelo lineal sin regularización incluye un intercepto β_0, existe colinealidad perfecta[6]. Una opción es descartar una columna de referencia (`drop='first'`); no es una regla universal para árboles ni para todo modelo regularizado.

Target encoding (mean encoding) suavizado

El *Target Encoding* reemplaza cada categoría por el promedio del target (`Item_Outlet_Sales`) en esa categoría: - Media global del target ($\bar{y}_{\text{global}}$) = $\frac{500+2500+4200+800}{4} = 2000.0$ - **Grocery Store**: 2 ocurrencias. Ventas = $[500, 800]$. Media local $\bar{y}_c = 650.0$. - **Supermarket Type1**: 1 ocurrencia. Ventas = [2500]. Media local $\bar{y}_c = 2500.0$. - **Supermarket Type3**: 1 ocurrencia. Ventas = [4200]. Media local $\bar{y}_c = 4200.0$.

Para evitar sobreajuste en categorías poco frecuentes, aplicamos **suavizado bayesiano**[7] (*m-estimate*):

$$\hat{S}_i = \frac{n_c \cdot \bar{y}_c + m \cdot \bar{y}_{\text{global}}}{n_c + m}$$

[4] **Corrección de continuidad de Yates:** ajuste conservador para la aproximación de conteos discretos por una distribución continua en tablas 2 × 2. Puede reducir potencia; no reemplaza automáticamente una prueba exacta en tablas pequeñas.

[5] **Prueba exacta de Fisher:** prueba para tablas 2 × 2 que calcula probabilidades condicionadas a los márgenes mediante la distribución hipergeométrica, sin depender de la aproximación [2].

[6] **Paradoja de Simpson**: Ocurre cuando una tendencia aparece en diferentes grupos de datos pero desaparece o se invierte cuando estos grupos se combinan. Es un riesgo al analizar tablas de contingencia agregadas sin controlar por variables de confusión subyacentes.

[7] **Suavizado bayesiano:** el valor codificado es la media *a posteriori* de la categoría si, antes de ver sus datos, suponemos que su media se parece a la global con una confianza equivalente a m observaciones «imaginarias». Con pocas filas reales domina esa creencia previa; con muchas, dominan los datos.

Con factor de suavizado $m = 2$ (cálculo ilustrativo que **no** debe aplicarse directamente a las mismas filas de entrenamiento): - **Grocery Store**: $\frac{(2\times650)+(2\times2000)}{2+2} = \frac{1300+4000}{4} = 1325.0$ - **Supermarket Type1**: $\frac{(1\times2500)+(2\times2000)}{1+2} = \frac{2500+4000}{3} \approx 2166.67$ - **Supermarket Type3**: $\frac{(1\times4200)+(2\times2000)}{1+2} = \frac{4200+4000}{3} \approx 2733.33$

Formalización teórica y matemática

Escaladores numéricos

1. **Min-Max Scaler**: Mapea una variable continua al intervalo $[a, b]$ (típicamente $[0, 1]$):

$$x' = a + \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}}(b - a)$$

   Es altamente sensible a outliers, ya que un único valor extremo dilata el denominador $x_{\text{máx}} - x_{\text{mín}}$.

2. **StandardScaler (z-score)**: Transforma la variable para que tenga media $\mu = 0$ y desviación estándar $\sigma = 1$:

$$z = \frac{x - \mu}{\sigma}, \quad \mathbb{E}[Z] = 0, \quad \text{Var}(Z) = 1$$

   Ambas igualdades salen de la linealidad de la esperanza[8]: $\mathbb{E}[(X - \mu)/\sigma] = (\mathbb{E}[X] - \mu)/\sigma = 0$ y $\text{Var}(aX + b) = a^2 \text{Var}(X)$, con $a = 1/\sigma$. Min-Max y StandardScaler son transformaciones afines: conservan el orden y las proporciones entre diferencias, así que **no cambian la forma** de la distribución ni reducen la influencia relativa de un valor extremo.

3. **RobustScaler**: Utiliza estadísticas de posición no paramétricas resistentes a valores extremos:

$$x_{\text{rob}} = \frac{x - \text{mediana}(X)}{\text{IQR}(X)}, \quad \text{IQR}(X) = Q_3(X) - Q_1(X)$$

4. **Transformaciones de Potencia**: Diseñadas para estabilizar la varianza y aproximar normalidad en distribuciones asimétricas (sesgadas):

   - **Box-Cox**[9] (solo para $y > 0$):

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \text{si } \lambda \neq 0 \\ \ln(y) & \text{si } \lambda = 0 \end{cases}$$

   - **Yeo-Johnson** (soporta valores positivos, cero y negativos).

[8] **Linealidad de la esperanza:** para constantes a y b, $\mathbb{E}[aX + b] = a\,\mathbb{E}[X] + b$. La varianza, en cambio, ignora los desplazamientos y escala con el cuadrado: $\text{Var}(aX + b) = a^2 \text{Var}(X)$.

[9] **Box-Cox:** el parámetro λ no se elige a mano: `PowerTransformer` lo estima por máxima verosimilitud, buscando el valor con el que los datos transformados se parecen más a una distribución normal. Como todo parámetro aprendido, debe ajustarse solo con entrenamiento.

Codificación categórica

1. **One-Hot Encoding**: Definimos un conjunto finito de categorías $\mathcal{C} = \{c_1, c_2, \dots, c_K\}$. La función de codificación es $h : \mathcal{C} \to \{0,1\}^K$ tal que $h(c_j)$ es un vector donde la componente j es 1 y el resto son 0. Si se incluye la matriz de diseño X con las K columnas junto a una columna de unos $\mathbf{1}$ para el intercepto:
$$\sum_{k=1}^{K} X_{i,k} = 1 \quad \forall i$$
Esto genera una dependencia lineal exacta, haciendo que la matriz $X^T X$ sea singular e imposible de invertir mediante mínimos cuadrados ordinarios.

2. **Target encoding con suavizado**: Sea y_i el objetivo continuo asociado a la observación i con categoría c. Combinamos la media local con la global mediante la siguiente fórmula ilustrativa de suavizado:
$$\hat{S}_c = \lambda(n_c)\bar{y}_c + (1 - \lambda(n_c))\bar{y}_{\text{global}}$$
Donde $\lambda(n_c) = \frac{n_c}{n_c+m}$ es el factor de contracción (*shrinkage*). Cuando $n_c \to \infty$, $\lambda(n_c) \to 1$ (confianza en la media local). Cuando $n_c \to 0$, $\lambda(n_c) \to 0$ (confianza en la media global).

Laboratorio interactivo: banco de pruebas de escalado y codificación (BigMart)

(En la versión web del libro, un laboratorio interactivo compara los tres escaladores al mover el precio atípico y muestra cómo el suavizado m acerca cada categoría a la media global).

Guía del laboratorio

Objetivo. Distinguir entre el efecto de un valor extremo sobre tres escaladores y el efecto de la frecuencia de una categoría sobre su codificación.

1. Fija el precio atípico en 500 y anota el valor escalado del producto 2 con Min-Max, StandardScaler y RobustScaler. Contrasta RobustScaler con el cálculo manual anterior.
2. Mueve el precio a 1000 sin cambiar los otros tres. Registra qué estadísticas cambian y si alguno de los tres escaladores es completamente inmune.
3. En codificación, activa y desactiva `drop='first'`; identifica la categoría representada por dos ceros. Después cambia a *target encoding* y compara `m=0` con `m=10` para una categoría con una sola fila.

Comprobación. Para 500, Min-Max asigna al producto 2 aproximadamente 0.165 y RobustScaler aproximadamente -0.189. Si no coinciden, revisa el percentil usado para los cuartiles. En *target encoding*, al aumentar m las categorías con una observación se acercan más a la media global.

Interpretación y entrega. Escribe dos frases: cuál escalador elegirías para un modelo de distancias con precios extremos y qué información adicional necesitas antes de decidirlo; explica por qué las

medias de ventas mostradas en este panel no deben calcularse con el objetivo de la propia fila al entrenar.

⚠ El panel enseña la fórmula, no un pipeline de entrenamiento

El panel usa las cuatro ventas para calcular la media de cada categoría. En un modelo real, ajusta el codificador solo con el conjunto de entrenamiento y utiliza codificación fuera de pliegue para las filas de entrenamiento; transforma validación y prueba con el codificador ya ajustado. Así evitas que una fila reciba información de su propio objetivo[10].

Implementación computacional (Python y pandas)

A continuación, mostramos cómo implementar estas transformaciones de manera profesional sobre el caso **BigMart Sales**.

Mejores prácticas en la industria

- **Conserva el esquema de categorías:** `pandas.get_dummies()` sirve para explorar, pero exige alinear columnas entre entrenamiento e inferencia. Un `OneHotEncoder` ajustado en entrenamiento conserva ese esquema y facilita integrarlo en un `Pipeline`[11].
- **Escala según el modelo:** los árboles de decisión que hacen cortes por umbral suelen ser invariantes a transformaciones monótonas de una variable; los modelos de distancias y los penalizados suelen ser sensibles a la escala[12].

	Item_MRP	Item_Weight	Outlet_Type	Item_Fat_Content	Outlet_Establishment_Year	Item
0	162.352137	6.855443	Supermarket Type3	Low Fat	2009	6373
1	133.778106	11.033290	Supermarket Type2	Regular	2009	119.
2	169.145984	11.360599	Supermarket Type1	Low Fat	1999	1003
3	208.536344	9.430435	Supermarket Type2	Low Fat	2004	225.
4	129.463098	12.122600	Supermarket Type1	Low Fat	1985	817.

Escalado numérico comparativo

[10] **Sensibilidad al tamaño de muestra (N):** con muestras grandes, diferencias pequeñas pueden producir valores p bajos. Por eso conviene acompañar la prueba con un tamaño del efecto como V de Cramér.

[12] **Pipeline**: Secuencia que ajusta transformaciones y estimador juntos. En validación cruzada, cada ajuste aprende estadísticas solo de la parte de entrenamiento del pliegue.

[12] **Criterios de Cochran**: Establecen las condiciones bajo las cuales la aproximación asintótica a la distribución ^{2} es fiable. Principalmente, requieren que no más del 20 % de las celdas tengan frecuencias esperadas menores a 5, y que ninguna celda tenga una frecuencia esperada menor a 1.

	MRP_Original	MRP_Standard	MRP_MinMax	MRP_Robust
95	74.141827	-0.917680	0.071481	-1.243182
96	153.325412	0.160588	0.180267	0.353934
97	151.747487	0.139101	0.178099	0.322107
98	140.230106	-0.017735	0.162276	0.089804
99	750.000000	8.285695	1.000000	12.388731

Codificación categórica

! Regla de oro en producción

Para categorías nuevas, decide explícitamente una política: `handle_unknown='ignore'` en `OneHotEncoder` produce ceros para una categoría desconocida. Comprueba si esa representación puede confundirse con la categoría de referencia cuando usas `drop='first'`; otra opción es agrupar categorías infrecuentes.

	Outlet_Type_Grocery Store	Outlet_Type_Supermarket Type1	Outlet_Type_Supermarket Type2	Out
0	0.0	0.0	0.0	1.0
1	0.0	0.0	1.0	0.0
2	0.0	1.0	0.0	0.0

	Outlet_Type	Outlet_Type_TargetEncoded	Item_Outlet_Sales
0	Supermarket Type3	1463.958066	6373.902143
1	Supermarket Type2	1193.408030	119.211065
2	Supermarket Type1	2304.724830	1003.473435
19	Grocery Store	1737.340917	158.520184

⚠ Target Leakage

El *target encoding* usa la variable Y (`Item_Outlet_Sales`) para transformar las X. `TargetEncoder.fit_transform` utiliza particiones internas para codificar las filas de entrenamiento fuera del pliegue[13]. Esta protección no sustituye la separación externa de entrenamiento, validación y prueba: ajusta todo el preprocesamiento dentro de cada pliegue externo.

Ingeniería de fechas y ciclos

[13] **Sensibilidad al tamaño de muestra (N):** con muestras grandes, diferencias pequeñas pueden producir valores p bajos. Por eso conviene acompañar la prueba con un tamaño del efecto como V de Cramér.

	Outlet_Establishment_Year	Outlet_Age
0	2009	4
1	2009	4
2	1999	14

Referencias y definiciones

39. Tratamiento de Ruido, Suavizado y Discretización

En la ciencia de datos y la ingeniería de *machine learning*, nos enfrentamos constantemente al desafío de lidiar con datos imperfectos. Los datos provenientes del mundo real rara vez son limpios y perfectos; suelen estar contaminados por diversas fuentes de perturbación. Una de estas fuentes principales es el **ruido**.

En este capítulo distinguimos ruido, errores de captura y valores atípicos; después usamos ejemplos pequeños para entender suavizado y discretización. BigMart Sales sirve como contexto de variables, mientras que la serie diaria que aparece más adelante es **simulada**: el conjunto de datos utilizado aquí no proporciona una secuencia temporal diaria de ventas.

Para comprender cómo afecta el ruido a nuestros modelos, es fundamental distinguir entre dos conceptos que a menudo se confunden en la práctica profesional: el **ruido** y los **outliers** (valores atípicos).

Ruido y valores atípicos

- **Ruido[1]:** variación que dificulta reconocer el patrón que interesa. Puede surgir de mediciones imprecisas, variabilidad natural o variables explicativas que aún no hemos observado. No siempre se puede ni conviene eliminar.
- **Valor atípico[2]:** observación inusual respecto de la mayoría. Puede ser válida o indicar un error; hay que investigar su origen antes de corregirla.

Visibilidad de productos y una serie simulada

En **BigMart Sales**, los registros con `Item_Visibility = 0` merecen una revisión: un cero podría ser legítimo, una convención de captura o un dato faltante codificado. No podemos deducir su causa sin documentación del proceso de medición. Para practicar con ventas diarias usaremos una serie **sintética** con tendencia conocida y perturbaciones añadidas; así podremos ver cuándo un filtro mejora la lectura de la señal y cuándo borra picos reales. Un modelo que memoriza fluctuaciones accidentales puede sufrir sobreajuste[3].

[1] **Ruido blanco:** sucesión de errores con media cero, la misma varianza y sin correlación entre instantes distintos. Es un modelo idealizado: en ventas reales el error de hoy suele parecerse al de ayer (autocorrelación), y entonces promediar reduce la varianza menos de lo que indica $\sigma^2/(2m+1)$.

[2] **Valor atípico:** observación distante de la mayoría según una referencia explícita (por ejemplo, un rango intercuartílico). Ser atípico no implica ser incorrecto.

[3] **Prueba exacta de Fisher:** prueba para tablas 2×2 que calcula probabilidades condicionadas a los márgenes mediante la distribución hipergeométrica, sin depender de la aproximación [2].

Ejemplo trabajado: discretización y suavizado a mano

Para desmitificar el funcionamiento de los algoritmos computacionales de suavizado y discretización, desarrollaremos los cálculos de forma completamente manual.

Consideremos un conjunto numérico ordenado de 12 pesos ficticios de productos (**Item_Weight**). Es un ejemplo didáctico, no una muestra extraída del archivo BigMart:

$$X = [4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34]$$

Discretización por igual ancho (*equal-width binning*)

Divide el rango de los datos en k intervalos del mismo tamaño: 1. Número de bins: $k = 3$. 2. Rango: mín$(X) = 4$, máx$(X) = 34$. 3. Ancho del bin (W):

$$W = \frac{\text{máx}(X) - \text{mín}(X)}{k} = \frac{34 - 4}{3} = \frac{30}{3} = 10$$

4. Intervalos: - Bin 1: $[4, 14)$ - Bin 2: $[14, 24)$ - Bin 3: $[24, 34]$ 5. Asignación de elementos: - Bin 1: $\{4, 8, 9\}$ (3 elementos) - Bin 2: $\{15, 21, 21\}$ (3 elementos) - Bin 3: $\{24, 25, 26, 28, 29, 34\}$ (6 elementos)

Discretización por igual frecuencia (*equal-frequency binning*)

Asigna el mismo número de observaciones N/k a cada bin: 1. $N = 12, k = 3 \implies 12/3 = 4$ elementos por bin. 2. Asignación ordenada: - Bin 1: $\{4, 8, 9, 15\}$ - Bin 2: $\{21, 21, 24, 25\}$ - Bin 3: $\{26, 28, 29, 34\}$

Suavizado de datos dentro de los bins

Tomando los bins de igual frecuencia:

Suavizado por Medias del Bin. - **Bin 1** $\{4, 8, 9, 15\}$: $\mu_1 = \frac{4+8+9+15}{4} = \frac{36}{4} = 9.0 \implies [9.0, 9.0, 9.0, 9.0]$ - **Bin 2** $\{21, 21, 24, 25\}$: $\mu_2 = \frac{21+21+24+25}{4} = \frac{91}{4} = 22.75 \implies [22.75, 22.75, 22.75, 22.75]$ - **Bin 3** $\{26, 28, 29, 34\}$: $\mu_3 = \frac{26+28+29+34}{4} = \frac{117}{4} = 29.25 \implies [29.25, 29.25, 29.25, 29.25]$

Señal final suavizada: $[9.0, 9.0, 9.0, 9.0, 22.75, 22.75, 22.75, 22.75, 29.25, 29.25, 29.25, 29.25]$

Suavizado por Medianas del Bin. - **Bin 1**: Mediana de $\{4, 8, 9, 15\} = \frac{8+9}{2} = 8.5 \implies [8.5, 8.5, 8.5, 8.5]$ - **Bin 2**: Mediana de $\{21, 21, 24, 25\} = \frac{21+24}{2} = 22.5 \implies [22.5, 22.5, 22.5, 22.5]$ - **Bin 3**: Mediana de $\{26, 28, 29, 34\} = \frac{28+29}{2} = 28.5 \implies [28.5, 28.5, 28.5, 28.5]$

Suavizado por Límites del Bin (Boundaries). Cada valor se reemplaza por el extremo del bin (mínimo o máximo) más cercano: - **Bin 1** (Límites: 4 y 15): - 4 está en el límite: 4 - 8 está más cerca de 4 ($|8 - 4| = 4$) que de 15 ($|8 - 15| = 7$): 4 - 9 está más cerca de 4 ($|9 - 4| = 5$) que de 15 ($|9 - 15| = 6$): 4 - 15 está en el límite: 15 - Resultado Bin 1: $[4, 4, 4, 15]$ - **Bin 2** (Límites: 21 y 25): 21

y 21 ya son el límite inferior; 24 está más cerca de 25 (1 frente a 3). Resultado: $[21, 21, 25, 25]$. - **Bin 3** (Límites: 26 y 34): 28 está más cerca de 26 (2 frente a 6) y 29 también (3 frente a 5). Resultado: $[26, 26, 26, 34]$.

¿Cuánta información se pierde? Una medida simple es la suma de cuadrados entre cada valor y su sustituto. Para el bin 1: con la media, $(4-9)^2 + (8-9)^2 + (9-9)^2 + (15-9)^2 = 62$; con los límites, $0 + 16 + 25 + 0 = 41$. Ningún método es mejor en general: la media minimiza esta suma **dentro** de cada bin cuando el sustituto es un único valor[4], mientras que los límites usan dos valores y conservan datos que sí existieron.

Suavizado por regresión lineal manual

Tomemos 5 observaciones con ruido de ventas de un producto en función de la semana: $(1, 2.1), (2, 3.9), (3, 6.2), (4, 7.8), (5, 10.3)$

1. $\bar{x} = \frac{1+2+3+4+5}{5} = 3.0$
2. $\bar{y} = \frac{2.1+3.9+6.2+7.8+10.3}{5} = \frac{30.3}{5} = 6.06$
3. $\sum (x_i - \bar{x})^2 = (-2)^2 + (-1)^2 + 0^2 + 1^2 + 2^2 = 4 + 1 + 0 + 1 + 4 = 10.0$
4. $\sum (x_i - \bar{x})(y_i - \bar{y})$:
   - $(1-3)(2.1-6.06) = (-2)(-3.96) = 7.92$
   - $(2-3)(3.9-6.06) = (-1)(-2.16) = 2.16$
   - $(3-3)(6.2-6.06) = 0$
   - $(4-3)(7.8-6.06) = (1)(1.74) = 1.74$
   - $(5-3)(10.3-6.06) = (2)(4.24) = 8.48$
   - Suma $= 7.92 + 2.16 + 0 + 1.74 + 8.48 = 20.30$
5. Pendiente $\beta_1 = \frac{20.30}{10.0} = 2.03$
6. Intercepto $\beta_0 = \bar{y} - \beta_1 \bar{x} = 6.06 - (2.03 \times 3.0) = 6.06 - 6.09 = -0.03$
7. Predicciones suavizadas ($\hat{y}_i = 2.03 x_i - 0.03$):
   - $\hat{y}_1 = 2.00$ (original: 2.1)
   - $\hat{y}_2 = 4.03$ (original: 3.9)
   - $\hat{y}_3 = 6.06$ (original: 6.2)
   - $\hat{y}_4 = 8.09$ (original: 7.8)
   - $\hat{y}_5 = 10.12$ (original: 10.3)

Formalización: modelos y filtros

Modelo aditivo de señal y ruido

Un modelo posible representa cada observación y_i como señal subyacente $f(x_i)$ más un error ϵ_i:

[4] **La media minimiza la suma de cuadrados:** $\sum_i (x_i - c)^2$ es una parábola en c cuya derivada, $-2\sum_i (x_i - c)$, se anula en $c = \bar{x}$. La mediana minimiza, en cambio, la suma de valores absolutos $\sum_i |x_i - c|$.

$$y_i = f(x_i) + \epsilon_i$$

Bajo un supuesto simplificador, los errores tienen media cero y varianza constante[5]:

$$\mathbb{E}[\epsilon_i] = 0, \quad \mathrm{Var}(\epsilon_i) = \sigma^2$$

Relación señal-ruido (SNR)

Si conocemos o estimamos por separado la potencia de la señal y la del error, la relación señal-ruido puede expresarse en decibelios:

$$\mathrm{SNR}_{\mathrm{dB}} = 10 \log_{10} \left(\frac{P_{\text{señal}}}{P_{\text{ruido}}} \right)$$

Compromiso entre suavidad y detalle

Una ventana pequeña conserva variaciones rápidas, incluido el ruido. Una ventana grande reduce fluctuaciones, pero también retrasa o aplana cambios reales. Este compromiso tiene una forma matemática precisa para la media móvil centrada con $2m + 1$ observaciones.

Varianza (lo que ganamos). Si los errores son ruido blanco[6] con varianza σ^2, la media de $2m + 1$ de ellos tiene varianza

$$\mathrm{Var}\left(\frac{1}{2m+1} \sum_{j=-m}^{m} \epsilon_{t+j} \right) = \frac{\sigma^2}{2m+1},$$

porque las varianzas de variables independientes se suman y la constante sale al cuadrado. Con $\sigma = 0.4$ y una ventana de 15 días, la desviación del ruido que sobrevive es $0.4/\sqrt{15} \approx 0.10$.

Sesgo (lo que pagamos). Si la señal se curva, promediar vecinos la aplana. Con una expansión de Taylor[7] de f alrededor de t, los términos lineales se cancelan por simetría y queda

$$\mathbb{E}[\widehat{y}_t] - f(t) \approx \frac{f''(t)}{2} \cdot \frac{1}{2m+1} \sum_{j=-m}^{m} j^2 = \frac{m(m+1)}{6} f''(t).$$

El sesgo crece aproximadamente con m^2 y es mayor donde la curvatura $|f''|$ es grande: en picos y valles. El error cuadrático medio[8] suma ambos efectos, ECM = sesgo^2 + varianza; por eso la curva de error frente a la ventana tiene forma de U y existe una ventana intermedia óptima.

La media exponencial con $\alpha = 2/(w+1)$ (el parámetro `span=w` de pandas) tiene, en régimen estable, varianza $\frac{\alpha}{2-\alpha}\sigma^2 = \sigma^2/w$: la misma reducción que una media móvil de w observaciones, pero usando solo el pasado.

[5] **Homocedasticidad:** varianza del error aproximadamente constante en los distintos valores de la variable explicativa.

[6] **Ruido blanco:** sucesión de errores con media cero, la misma varianza y sin correlación entre instantes distintos. Es un modelo idealizado: en ventas reales el error de hoy suele parecerse al de ayer (autocorrelación), y entonces promediar reduce la varianza menos de lo que indica $\sigma^2/(2m+1)$.

[7] **Expansión de Taylor:** aproximación de una función suave cerca de un punto mediante sus derivadas: $f(t+j) \approx f(t) + jf'(t) + \frac{j^2}{2} f''(t)$. Al promediar $j = -m, \ldots, m$, el término con j se anula por simetría y sobrevive el de j^2.

[8] **Error cuadrático medio (ECM):** $\mathbb{E}[(\widehat{y}_t - f(t))^2]$. Sumando y restando $\mathbb{E}[\widehat{y}_t]$ se descompone en sesgo^2 + varianza: una idea que reaparece al elegir la complejidad de cualquier modelo.

Filtros de suavizado comunes

Si una serie fue muestreada con poca frecuencia, algunos cambios rápidos pueden confundirse con patrones lentos (*aliasing*)[9]. Los filtros siguientes responden a preguntas distintas:

1. **Media móvil centrada (SMA):**

$$\hat{y}_t = \frac{1}{2m+1} \sum_{j=-m}^{m} y_{t+j}$$

   Usa observaciones futuras; sirve para exploración retrospectiva, pero no para predecir en tiempo real.
2. **Suavizado Exponencial (EWMA):**

$$\hat{y}_t = \alpha y_t + (1-\alpha)\hat{y}_{t-1}, \quad \alpha \in [0,1]$$

3. **Filtro de Savitzky-Golay:** ajusta un polinomio local por mínimos cuadrados. Puede conservar mejor algunos rasgos de la curva que una media móvil, según la ventana y el orden del polinomio; la versión centrada también usa observaciones futuras.

Laboratorio interactivo: suavizado y discretización

Objetivo. Comparar cómo cambian los grupos al modificar el número de intervalos y comprobar en una serie simulada el compromiso sesgo–varianza de la sección anterior.

1. Con tres intervalos, compara ancho igual y frecuencia igual. Comprueba que los 12 precios quedan asignados una sola vez y describe qué método produce grupos más equilibrados.
2. Aplica media, mediana y límites del intervalo. La «pérdida» es la suma de cuadrados entre cada precio y su valor suavizado. ¿Qué método pierde menos? ¿Por qué la pérdida siempre disminuye al aumentar k?
3. En la serie, activa el pico de promoción y usa la media móvil centrada con ventana 3, 11 y 31. Observa el pico y ubica cada ventana en la curva «error frente a ventana». ¿Dónde está el mínimo de la U?
4. Duplica el ruido. ¿La ventana óptima se hace más grande o más pequeña? Relaciónalo con las fórmulas de varianza y sesgo.

Entrega breve. Anota una decisión de transformación con su propósito y un riesgo. La línea negra es la señal verdadera solo porque **simulamos** los datos; en un caso real no la conoceríamos.

(El laboratorio interactivo con precios y ventas simuladas está disponible en la versión web del libro. Las preguntas y los datos de ejemplo anteriores también pueden resolverse sin los controles.)

---

[9] **Criterios de Cochran**: Establecen las condiciones bajo las cuales la aproximación asintótica a la distribución ^{2} es fiable. Principalmente, requieren que no más del 20 % de las celdas tengan frecuencias esperadas menores a 5, y que ninguna celda tenga una frecuencia esperada menor a 1.

Implementación computacional con Python

Visualización del suavizado con Plotly

Prácticas para usar en proyectos reales

! Regla de oro en producción: Prevención de fuga de datos

Aprende los límites de discretización con datos de entrenamiento y aplícalos después a validación, prueba y producción. Registra cómo se tratan valores fuera del rango aprendido. Para series temporales, valida respetando el orden cronológico y evita filtros centrados cuando la aplicación exige decisiones en tiempo real.

Mejores prácticas en la industria: Cuándo discretizar

- **Conservar el valor continuo** suele ser una referencia razonable: discretizar pierde resolución[10]. Comprueba con validación si los intervalos añaden valor al modelo y al uso de negocio.
- **Discretizar** puede facilitar reglas interpretables o niveles contractuales. En algunos modelos de riesgo se usan transformaciones como *Weight of Evidence*[11]; deben aprenderse y validarse sin fuga de la variable objetivo.

Notas de términos

[11] **Corrección de continuidad de Yates:** ajuste conservador para la aproximación de conteos discretos por una distribución continua en tablas 2×2. Puede reducir potencia; no reemplaza automáticamente una prueba exacta en tablas pequeñas.

[11] **Paradoja de Simpson**: Ocurre cuando una tendencia aparece en diferentes grupos de datos pero desaparece o se invierte cuando estos grupos se combinan. Es un riesgo al analizar tablas de contingencia agregadas sin controlar por variables de confusión subyacentes.

40. Tratamiento de datos faltantes

En una tabla de ventas como BigMart, `Item_Weight` puede faltar porque falló la captura del proveedor. En cambio, si `Outlet_Size` falta sobre todo en tiendas de cierto tipo, la ausencia contiene información sobre la tienda. Antes de elegir un método, pregunta **qué filas faltan, en qué grupos y desde cuándo**. `NaN`[1] indica que no se registró un valor; no explica la causa.

Un primer ejemplo visual

Supongamos que conocemos el peso de todos los productos de una población simulada y ocultamos algunos a propósito. Podemos comparar la distribución completa con la que quedaría disponible para analizar. En datos reales, la curva completa normalmente no está disponible: aquí la conocemos porque hemos simulado los datos.

En este ejemplo, los pesos altos tienen más probabilidad de desaparecer y la media observada queda por debajo de la verdadera. Los histogramas tienen tamaños de muestra distintos; compara sobre todo su **forma** y las medias impresas, no solo la altura de las barras.

Ejemplo: tres mecanismos de ausencia

Usaremos $R_i = 1$ cuando el peso del producto i está ausente. Los nombres MCAR, MAR y MNAR describen **supuestos sobre el proceso que generó la ausencia**, no etiquetas que podamos deducir con certeza de una tabla incompleta[2].

MCAR: pérdida independiente de los datos

Un fallo aleatorio del sistema elimina algunas filas sin tener en cuenta pesos ni características de las tiendas. En la simulación, cada registro tiene la misma probabilidad de perderse.

La similitud de las medias es esperable en promedio, pero no garantiza que una muestra concreta no tenga error de muestreo. Eliminar filas MCAR también reduce la cantidad de información.

[1] `NaN` significa *Not a Number*. En pandas, también pueden aparecer otros marcadores de ausencia como `None` o `pd.NA`, según el tipo de columna.

[2] MCAR significa *Missing Completely At Random*; MAR, *Missing At Random*; MNAR, *Missing Not At Random*. MAR es una independencia **condicional** respecto al valor ausente.

MAR: pérdida explicada por variables observadas

La probabilidad de que falte `Item_Weight` depende de `Outlet_Type`, que sí conocemos. Dentro de cada tipo de tienda, el mecanismo simulado ya no depende del peso. Como los pesos también varían por tipo, analizar solo los valores observados cambia la composición de la muestra.

La primera gráfica muestra **dónde** se concentran los faltantes; la segunda muestra el efecto sobre la distribución agregada. La variable auxiliar `Outlet_Type` permite estudiar y modelar ese patrón. MAR no significa que la falta sea aleatoria sin condiciones.

MNAR: pérdida ligada al valor ausente

Imaginemos que un proceso de captura omite con más frecuencia pesos grandes y que esa causa no queda explicada por ninguna columna observada. El mecanismo depende del propio peso que falta.

No podemos confirmar MNAR solo mirando los pesos registrados. Necesitamos conocimiento del proceso de captura, una muestra de seguimiento o un análisis de sensibilidad[3].

Laboratorio: tres mecanismos, un mismo promedio

Las tres simulaciones anteriores usan datos distintos. Aquí los comparamos sobre **la misma población** de 1 500 productos: las tiendas *Grocery* venden productos más ligeros (media 10 kg) que los *Supermarket* (media 14 kg). Tú eliges cómo se pierden los pesos y el laboratorio compara tres estimadores de la media: casos completos, imputación por la media global e imputación por la media **de cada tipo de tienda**.

En la versión web, este laboratorio simula 1 500 productos y permite cambiar el mecanismo de ausencia. Resultado esperado: bajo MCAR la media observada casi no se sesga; bajo MAR se sesga, pero promediar por tipo de tienda lo corrige; bajo MNAR ninguno de los dos estimadores lo corrige.

Guía del laboratorio.

1. Con MCAR, mueve la proporción de faltantes de 5 % a 70 %. Comprueba que la media observada se mantiene cerca de la verdadera mientras la desviación estándar tras imputar con la media **disminuye**. ¿Por qué ocurre lo segundo aunque no haya sesgo?
2. Cambia a MAR con intensidad 1.2. Identifica en el panel derecho qué grupo pierde más pesos y explica el signo del sesgo de la media observada. Verifica que el estimador por grupo lo corrige.
3. Cambia a MNAR. Observa que las tasas por tipo de tienda casi no se distinguen, pero las tasas por peso sí. ¿Qué implica que la evidencia de MNAR esté precisamente en la columna que no observamos?

> **i** Antes de imputar
>
> Cuenta los faltantes por columna, por tipo de tienda y por periodo. Verifica si un código especial como `0` o `"sin dato"` representa ausencia. Documenta la definición de cada campo y el momento en que estará disponible durante la predicción.

[3] Un análisis de sensibilidad repite el cálculo bajo varios supuestos plausibles sobre los valores no observados para ver cuánto cambia la conclusión.

De la intuición al cálculo: imputar un peso

En esta muestra pequeña conocemos `Item_Outlet_Sales` solo como **variable auxiliar de demostración**. Si las ventas son el objetivo de un modelo predictivo, no debemos usarlas para imputar predictores que estarán disponibles antes de la venta[4].

Producto	`Item_Weight` (kg)	Ventas observadas
FDA15	9.30	3735.13
DRC01	5.92	1200.00
FDN15	—	1098.56
FDX07	19.20	4138.61

Media observada. Para FDN15, la media de los tres pesos registrados es $(9.30 + 5.92 + 19.20)/3 \approx 11.47$ kg. Es sencilla, pero todas las filas incompletas reciben el mismo número y la dispersión se contrae.

Cuánto se contrae. La varianza muestral[5] de los tres pesos observados es

$$s^2_{\text{obs}} = \frac{(9.30 - 11.47)^2 + (5.92 - 11.47)^2 + (19.20 - 11.47)^2}{3 - 1} \approx \frac{4.72 + 30.82 + 59.72}{2} \approx 47.63.$$

Al añadir el valor imputado 11.47, el numerador no cambia (esa fila está exactamente en la media, así que aporta cero), pero el divisor pasa de 2 a 3: $s^2_{\text{imp}} = 95.26/3 \approx 31.75$. La desviación estándar cae de 6.90 a 5.64 kg **sin que hayamos aprendido nada nuevo** sobre los pesos. Un modelo entrenado con esa columna verá los datos más concentrados de lo que son.

Vecino más cercano. Si comparamos únicamente las ventas disponibles, DRC01 es el más cercano a FDN15: $|1200 - 1098.56| = 101.44$. Con $k = 1$ y ese único atributo auxiliar, imputaríamos **5.92 kg**. La distancia y el resultado cambian si añadimos otras variables o las escalamos; `KNNImputer` usa por defecto vecinos con peso uniforme[6].

Pregunta para discutir. ¿Cuál de los dos valores parece más razonable? No podemos saberlo para FDN15 sin una observación verdadera. Por eso evaluaremos métodos ocultando temporalmente valores que sí conocemos.

Qué muestran las imputaciones

Imputación simple y distribución

Esta gráfica de Plotly del capítulo original compara la distribución simulada completa con varias imputaciones simples. La modalidad se calcula después de redondear el peso a una décima; esa discretización solo sirve para ilustrar el método.

[4] La variable objetivo es el resultado que se desea predecir. Usarla para construir un predictor crea fuga de información si ese resultado todavía no existe al hacer la predicción.

[5] **Varianza muestral:** $s^2 = \frac{1}{n-1}\sum_i (y_i - \bar{y})^2$. Se divide entre $n - 1$ porque las desviaciones se miden respecto a $\bar{y}$, calculada con los mismos datos; así s^2 es un estimador insesgado de la varianza poblacional.

[6] Por defecto, `KNNImputer` usa la distancia `nan_euclidean` y `weights="uniform"`; también admite ponderar vecinos por distancia. Documentación de KNNImputer.

Observa el pico que produce reemplazar muchos faltantes por un mismo número. Mantener aproximadamente la media no implica conservar la varianza ni las relaciones con otras columnas.

KNN: los vecinos importan

En el siguiente ejemplo original se ocultan valores conocidos de una simulación. El panel derecho permite ver los vecinos y comparar una imputación con el valor ocultado. Los puntos rojos son una referencia didáctica, no información disponible al imputar datos reales.

Al usar KNN, escala las variables numéricas pertinentes y revisa qué columnas entran en la distancia. El vecino más cercano en una dimensión puede ser muy distinto del vecino en varias dimensiones.

Imputación múltiple: representar incertidumbre

Una sola tabla completada trata cada valor imputado como si fuese conocido. Para inferencia estadística, podemos crear varias tablas plausibles, ajustar el análisis en cada una y combinar estimaciones y varianzas[7]. La figura original usa `IterativeImputer(sample_posterior=True)` para generar cinco realizaciones; el valor predeterminado `sample_posterior=False` genera una imputación determinista y **no equivale por sí solo a imputación múltiple**.

Las cinco nubes muestran incertidumbre entre imputaciones; ninguna recupera necesariamente cada valor individual. Esta simulación visualiza el principio. Un análisis inferencial completo también necesita combinar los resultados de los modelos ajustados en cada tabla.

Formalización breve

Sea $Y = (Y_{\text{obs}}, Y_{\text{mis}})$ el conjunto completo, R el indicador de ausencia y X las variables auxiliares observadas. De manera esquemática:

- **MCAR:** $P(R \mid Y, X) = P(R)$. La ausencia es independiente de los valores.
- **MAR:** $P(R \mid Y_{\text{obs}}, Y_{\text{mis}}, X) = P(R \mid Y_{\text{obs}}, X)$. Una vez condicionamos[8] en lo observado, el valor ausente no añade información sobre R.
- **MNAR:** la probabilidad de ausencia todavía depende de Y_{mis} después de condicionar en lo observado.

La media de casos completos estima $E[Y \mid R = 0]$[9]. Por la ley de la probabilidad total,

$$E[Y] = P(R = 0)\, E[Y \mid R = 0] + P(R = 1)\, E[Y \mid R = 1],$$

[7] La imputación múltiple considera incertidumbre entre tablas imputadas. `IterativeImputer` puede producir realizaciones con `sample_posterior=True`, pero el análisis y la combinación de estimaciones quedan a cargo del usuario. Documentación de IterativeImputer.

[8] **Independencia condicional:** R y Y_{mis} pueden estar relacionadas en general y, aun así, ser independientes **dentro** de cada grupo definido por las variables observadas. Es la diferencia entre «la ausencia depende del peso» y «la ausencia depende del tipo de tienda, que a su vez se relaciona con el peso».

[9] La **esperanza** $\mathbb{E}[\cdot]$ es el promedio de una cantidad sobre todos los casos posibles, ponderado por su probabilidad. El riesgo es, entonces, la proporción de errores que cometería el clasificador sobre la población completa, de la cual nuestros datos son solo una muestra.

así que el **sesgo**[10] de usar solo los casos completos es $E[Y \mid R = 0] - E[Y] = P(R = 1)\,(E[Y \mid R = 0] - E[Y \mid R = 1])$: crece con la proporción de faltantes **y** con la diferencia entre quienes responden y quienes no. Bajo MCAR esa diferencia es cero. Bajo MAR puede no serlo, pero dentro de cada valor de X sí lo es, por lo que promediar primero dentro de los grupos de X y luego ponderar por su tamaño elimina el sesgo:

$$\hat{\mu}_{\text{MAR}} = \sum_{x} \hat{P}(X = x)\, \bar{y}_{\text{obs}}(x).$$

Es exactamente el estimador «por tipo de tienda» del laboratorio de mecanismos. Bajo MNAR no existe una corrección que use solo lo observado.

Para una columna numérica, la imputación por media usa $\hat{y}_i = \bar{y}_{\text{obs}}$ cuando $R_i = 1$. Con n_{obs} valores observados de n, la varianza de la columna imputada es

$$s^2_{\text{imp}} = \frac{n_{\text{obs}} - 1}{n - 1}\, s^2_{\text{obs}},$$

porque los valores imputados no aportan desviaciones respecto a la media. En el ejemplo, $\frac{2}{3} \cdot 47.63 = 31.75$. En KNN, $\hat{y}_i = k^{-1} \sum_{j \in N_k(i)} y_j$, donde $N_k(i)$ reúne vecinos con el valor observado. Ambas producen **estimaciones**, no recuperaciones seguras del dato original.

Laboratorio: elegir y poner a prueba una imputación

Objetivo. Comparar media, mediana y KNN en el mismo ejemplo de productos. **Hipótesis.** Un método que use ventas como variable auxiliar debería ganar cuando peso y ventas guarden una relación útil.

1. Cambia el método y el número de vecinos. Anota la imputación del peso realmente ausente de FDN15.
2. Ocultamos además el peso conocido de NCD19 **solo para validar**. Compara la predicción con su valor real y registra el error absoluto.
3. Cambia entre moda y categoría «Faltante» para `Outlet_Size`. Comprueba qué celdas cambian y cuáles no.
4. Explica por qué el menor error en **una sola fila** no prueba que un método sea mejor para toda la población. Compara después el «error medio (dejar uno fuera)»[11], que oculta por turnos cada peso conocido. ¿Cambia el método ganador? Propón cómo repetir esta validación por grupos y periodos en un conjunto real.

El laboratorio interactivo se encuentra en la versión HTML. Para resolverlo sin controles: calcula la media y la mediana de 9.30, 5.92, 19.20 y 15.00; después predice el peso de NCD19 (12.40 kg, ocultado) usando los k vecinos en ventas y compara el error absoluto.

[10] **Sesgo de un estimador:** diferencia entre el valor que el estimador da en promedio, sobre muchas muestras posibles, y el parámetro que pretende estimar. Un estimador sesgado no mejora al aumentar el tamaño de muestra: converge al valor equivocado.

[11] **Validación dejando uno fuera (*leave-one-out*):** se oculta una observación, se estima con las demás y se repite con cada observación. Promediar los errores reduce la dependencia de una sola fila afortunada, aunque con pocos datos la estimación sigue siendo muy variable.

Implementación sin fuga de información

En un problema supervisado, separamos primero entrenamiento y prueba, y ajustamos cualquier imputador **solo con entrenamiento**. Aquí `Item_Outlet_Sales` es la variable objetivo: nunca se usa como predictor para imputar `Item_Weight`. Las variables auxiliares son un precio y un tipo de tienda simulados.

La puntuación es solo una comprobación de que el flujo se ejecuta; para elegir estrategia, compara alternativas con validación cruzada dentro del conjunto de entrenamiento y deja la prueba para la evaluación final. Un indicador de ausencia puede ayudar cuando el patrón aporta señal; compruébalo empíricamente[12].

Práctica profesional: documentar y vigilar

Guarda el imputador junto con el modelo, vigila la tasa de faltantes por grupo y periodo, y registra qué campos están disponibles en el momento de inferencia. Compara una línea base simple con métodos más costosos según la métrica, la latencia y el volumen **de tu sistema**. No existe un umbral universal para descartar columnas ni un método de imputación superior en todos los datos. scikit-learn: errores comunes y fuga de datos · scikit-learn: imputación.

[12] Un indicador de ausencia es una columna binaria que registra si el valor original faltaba. `SimpleImputer(add_indicator=True)` la añade para las columnas con faltantes durante el ajuste. Documentación de SimpleImputer.

41. Reducción de Dimensionalidad con PCA

Imagina dos sensores que registran casi el mismo movimiento. Si ambas lecturas suben y bajan juntas, dos columnas describen en gran parte una sola dirección. PCA gira los ejes para que el primero siga la mayor variación y el segundo recoja lo que queda. Las *puntuaciones*[1] indican dónde queda cada observación sobre los nuevos ejes.

Ejemplo mínimo. Para las lecturas $(2,3)$, $(3,4)$ y $(4,5)$, al restar las medias $(3,4)$ obtenemos $(-1,-1)$, $(0,0)$ y $(1,1)$. Todos los puntos quedan sobre la diagonal: una sola coordenada basta para reconstruirlos. Más adelante calcularemos esa coordenada y la varianza que conserva.

Antes de reducir dimensiones en un proyecto

Define para qué usarás los componentes: visualizar, comprimir o alimentar un modelo. Si vas a evaluar predicción, separa primero entrenamiento y prueba; aprende el escalado y PCA solo en entrenamiento y mide el resultado en datos no vistos. La varianza explicada resume variación de las entradas, no rendimiento predictivo.

Ejemplo: dos sensores paso a paso

A continuación, calculamos el caso de los dos sensores de la intuición. Aquí **solo centramos**: ambas columnas tienen las mismas unidades y varianza. Así la matriz de covarianza y los autovalores de este ejemplo corresponden a los datos centrados; en el caso Iris posterior sí estandarizamos.

1. **Definición de la matriz de datos original:** Supongamos $n = 3$ observaciones (ej. mediciones de dos sensores) representados como filas y $p = 2$ variables representadas como columnas:

$$X = \begin{pmatrix} 2 & 3 \\ 3 & 4 \\ 4 & 5 \end{pmatrix}$$

2. **Centrado de los datos:** Calculamos la media de cada columna: $\bar{x}_1 = 3$ y $\bar{x}_2 = 4$. Restamos la media a cada fila para obtener $\tilde{X}$:

$$\tilde{X} = \begin{pmatrix} 2-3 & 3-4 \\ 3-3 & 4-4 \\ 4-3 & 5-4 \end{pmatrix} = \begin{pmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{pmatrix}$$

[1] **Puntuación (*score*)**: coordenada de una observación tras proyectarla sobre una componente principal. Los *loadings* son los pesos de las variables originales que definen esa dirección.

3. **Cálculo de la matriz de covarianza[2] (S):** Aplicamos $S = \frac{1}{n-1}\tilde{X}^T\tilde{X}$ con $n-1=2$:

$$S = \frac{1}{2}\begin{pmatrix}-1 & 0 & 1\\ -1 & 0 & 1\end{pmatrix}\begin{pmatrix}-1 & -1\\ 0 & 0\\ 1 & 1\end{pmatrix} = \frac{1}{2}\begin{pmatrix}2 & 2\\ 2 & 2\end{pmatrix} = \begin{pmatrix}1 & 1\\ 1 & 1\end{pmatrix}$$

4. **Obtención de eigenvalores[3] (λ):** Resolvemos el determinante $\det(S-\lambda I)=0$:

$$\det\begin{pmatrix}1-\lambda & 1\\ 1 & 1-\lambda\end{pmatrix} = (1-\lambda)^2 - 1 = 0 \implies \lambda^2 - 2\lambda = 0$$

   Los resultados son $\lambda_1 = 2$ y $\lambda_2 = 0$. La varianza total es $2+0=2$. El primer componente explica el 100 % de la varianza.

5. **Obtención de eigenvectores (v):** Para $\lambda_1 = 2$, resolvemos $(S-2I)\mathbf{v}_1 = 0$:

$$\begin{pmatrix}-1 & 1\\ 1 & -1\end{pmatrix}\begin{pmatrix}v_{11}\\ v_{21}\end{pmatrix} = 0 \implies v_{11} = v_{21}$$

   Normalizando para que $\|v\| = 1$: $v_1 = \begin{pmatrix}1/\sqrt{2}\\ 1/\sqrt{2}\end{pmatrix} \approx \begin{pmatrix}0.707\\ 0.707\end{pmatrix}$.

6. **Proyección de los datos ($k=1$):** Transformamos los datos centrados a la nueva dimensión $Z = \tilde{X}v_1$:

$$Z = \begin{pmatrix}-1 & -1\\ 0 & 0\\ 1 & 1\end{pmatrix}\begin{pmatrix}0.707\\ 0.707\end{pmatrix} = \begin{pmatrix}-1.414\\ 0\\ 1.414\end{pmatrix}$$

7. **Conclusión:** Dado que los datos originales estaban perfectamente alineados ($x_2 = x_1 + 1$), el PCA ha logrado reducir la dimensionalidad de 2 a 1 sin ninguna pérdida de información, capturando toda la estructura de los datos en un solo eje.

Laboratorio: reproduce los seis pasos y rompe la alineación

Las gráficas siguientes no son imágenes precalculadas: el navegador ejecuta una implementación de PCA para dos variables (centrado, covarianza, autovalores por la fórmula cuadrática, autovectores y proyección) cada vez que cambias los datos. Empieza con los tres puntos del ejemplo y recorre los pasos; comprueba que cada número coincide con el desarrollo anterior.

En la versión web, este laboratorio dibuja cada paso (datos, centrado, covarianza, autovalores, autovectores y proyección) con una implementación de PCA en JavaScript y permite arrastrar los puntos. Los valores de referencia son los del desarrollo anterior: $\bar{\mathbf{x}} = (3,4)$, $S = \begin{pmatrix}1 & 1\\ 1 & 1\end{pmatrix}$, $\lambda = (2,0)$, $\mathbf{v}_1 = (0.707, 0.707)$ y $\mathbf{z} = (-1.414, 0, 1.414)$.

Guía del laboratorio.

[2] **Covarianza muestral**: $s_{jk} = \frac{1}{n-1}\sum_i (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$. Es positiva cuando dos variables tienden a estar por encima (o por debajo) de su media a la vez. Dividir entre $n-1$ en lugar de n corrige el sesgo que produce usar la media muestral en vez de la media poblacional desconocida (corrección de Bessel); para las direcciones de PCA el divisor es irrelevante, porque solo reescala todos los autovalores.

[3] **Autovalor o eigenvalor**: número que indica cuánto estira una matriz una dirección propia. En PCA de la matriz de covarianza, equivale a la varianza capturada por la componente correspondiente.

1. Recorre los pasos 1 a 6 sin mover nada y comprueba cada número contra el desarrollo manual: $\bar{\mathbf{x}} = (3, 4)$, traza$(S) = 2$, det$(S) = 0$, $\lambda_1 = 2$, $\lambda_2 = 0$ y $z = (-1.414,\ 0,\ 1.414)$.
2. En el paso 5, arrastra P3 fuera de la diagonal. **Antes de soltarlo**, predice hacia dónde girará PC1 y si λ_2 dejará de ser cero. Comprueba en el paso 6 que el error de reconstrucción coincide con λ_2.
3. Pulsa «Agregar punto» varias veces. Verifica en el paso 4 que $\lambda_1 + \lambda_2$ sigue igual a la traza, aunque la proporción explicada por PC1 cambie.
4. Coloca dos puntos casi encima uno del otro y el tercero lejos. Explica, con la fórmula de la covarianza, por qué un solo punto alejado puede decidir la dirección de PC1; lo retomaremos en la sección de sensibilidad a *outliers*.

Motivación: la maldición de la dimensionalidad

Cuando el número de variables p crece y mantenemos fija la cantidad de observaciones, la geometría del espacio puede volverse contraintuitiva. La **maldición de la dimensionalidad** describe varios efectos de la alta dimensión, entre ellos la dispersión relativa de las observaciones al crecer el espacio disponible.

Consideremos una hiperesfera de radio r inscrita en un hipercubo de lado $2r$ en $\mathbb{R}^d$. La proporción del volumen de la esfera respecto al cubo es:

$$\frac{V_{\text{esfera}}}{V_{\text{cubo}}} = \frac{\pi^{d/2} r^d}{\Gamma(d/2+1) \cdot (2r)^d} = \frac{\pi^{d/2}}{2^d\, \Gamma(d/2+1)}$$

En 2 y en 3 dimensiones, esta proporción es relativamente alta (~78 % y ~52 % respectivamente), como se puede apreciar en la siguiente visualización interactiva donde $r = 1$.

Sin embargo, esta proporción **tiende a cero** cuando $d \to \infty$. Esto significa que la mayoría del volumen del hipercubo se concentra en las esquinas, y la esfera (que representa la región de interés) ocupa una fracción cada vez menor del espacio total.

Como ejemplo geométrico, si dividimos cada eje en L intervalos y queremos conservar la misma cantidad de observaciones por celda, el número de celdas crece como L^d. No implica que todo problema real requiera exactamente esa cantidad de observaciones.

$$n \sim \rho \cdot L^d$$

Con $p = 20$ variables, el número de correlaciones posibles es $\binom{20}{2} = 190$. Con $p = 100$, son 4,950. Revisar e interpretar cada relación manualmente deja de ser practicable. Por ello necesitamos métodos que resuman la información sin perder lo esencial.

Fundamentos matemáticos del PCA

El Análisis de Componentes Principales transforma variables numéricas centradas en componentes ortogonales[4] ordenadas por varianza. Entre todas las proyecciones lineales de dimensión k, PCA maximiza la varianza proyectada y minimiza el error cuadrático de reconstrucción. Esa varianza puede incluir ruido y no garantiza conservar lo más útil para una tarea predictiva.

Formulación Matemática y Procedimiento

Dado un conjunto de datos $X \in \mathbb{R}^{n \times p}$, donde n es el número de muestras y p el número de variables, el proceso se rige por la siguiente secuencia lógica:

1. **Estandarización y Centrado**

   Primero se centra cada variable. Se estandariza además cuando las unidades o dispersiones de las columnas harían que una domine por escala y el objetivo requiere tratarlas de forma comparable. Esto se realiza en dos pasos:

   - **Centrado:** restar la media de cada variable
   - **Escalado:** dividir entre su desviación estándar

   De esta forma, cada variable transformada tiene media cero y varianza unitaria con la convención de varianza usada para escalarla. Si todas las medidas comparten unidades y la magnitud de la varianza es relevante, puede bastar el centrado.

   $$\tilde{X}_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j}$$

   donde:

   - μ_j es la media de la variable j
   - σ_j es su desviación estándar

2. **Cálculo de la Matriz de Covarianza:** Se construye la matriz S (o $\mathbf{C}$ dependiendo de la notación, $\mathbf{C}$ es común en ML), que resume la variabilidad de cada variable y la relación entre ellas:

   $$S = \frac{1}{n-1}\tilde{X}^T\tilde{X}$$

   De forma elemento a elemento:

   $$s_{jk} = \frac{1}{n-1}\sum_{i=1}^{n}\tilde{x}_{ij}\tilde{x}_{ik}$$

[4] **Ortogonal**: dos direcciones perpendiculares cuyo producto interno es cero. En PCA, esto hace que las puntuaciones de componentes distintos no estén correlacionadas en el conjunto usado para ajustarlo.

donde:

- s_{jj} representa la **varianza** de la variable j
- s_{jk} representa la **covarianza** entre las variables j y k

Esta matriz es simétrica y semidefinida positiva[5], lo que garantiza que sus eigenvalores sean reales y no negativos.

3. **Descomposición Espectral:** Se resuelve el problema de autovalores para encontrar las direcciones principales:

$$S\mathbf{v} = \lambda\mathbf{v}$$

Aquí, λ representa los eigenvalores (la varianza explicada en esa dirección) y $\mathbf{v}$ los eigenvectores (las direcciones o *loadings*).

4. **Optimización y Ordenamiento:** El primer componente principal w_1 es el eigenvector asociado al eigenvalor máximo λ_1. Los componentes siguientes se obtienen maximizando la varianza restante bajo la condición de ser ortogonales a los anteriores. Se clasifican de forma descendente: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p$.

5. **Proyección y Reducción:** Finalmente, se proyectan los datos originales sobre el subespacio generado por los primeros k componentes:

$$\mathbf{Z} = \tilde{X}\mathbf{V}_k$$

Donde $\mathbf{V}_k$ es la matriz que contiene los primeros k eigenvectores ordenados.

Cada componente captura una parte de la variabilidad total de los datos, la cual equivale a la traza de la matriz de covarianza ($\sum \lambda_j$). La importancia de cada componente se mide a través de la **Proporción de Varianza Explicada (PVE)**:

$$\text{PVE}_k = \frac{\lambda_k}{\sum_{j=1}^{p} \lambda_j}$$

Esto permite elegir una dimensión que conserve, por ejemplo, el 95 % de la varianza **de la matriz transformada usada en el ajuste**. El umbral es una decisión de análisis, no una garantía de calidad del modelo.

Por qué autovectores: la formulación variacional

El cálculo anterior usó autovalores como una receta. La razón aparece al plantear PCA como un problema de optimización. Para una dirección unitaria $\mathbf{u}$, las puntuaciones son $\mathbf{z} = \tilde{X}\mathbf{u}$ y su varianza es

$$\text{Var}(\mathbf{z}) = \frac{1}{n-1}\mathbf{u}^T\tilde{X}^T\tilde{X}\mathbf{u} = \mathbf{u}^T S\,\mathbf{u}.$$

[5] **Matriz semidefinida positiva**: matriz simétrica S tal que $\mathbf{u}^T S\mathbf{u} \geq 0$ para todo vector $\mathbf{u}$. Para una covarianza, $\mathbf{u}^T S\mathbf{u}$ es la varianza de la combinación lineal $\tilde{X}\mathbf{u}$, y una varianza nunca es negativa. Por eso sus autovalores son reales y no negativos.

Queremos la dirección de mayor varianza sin hacer trampa alargando el vector, así que imponemos $\mathbf{u}^T\mathbf{u} = 1$. Con un multiplicador de Lagrange[6]:

$$\mathcal{L}(\mathbf{u}, \lambda) = \mathbf{u}^T S\mathbf{u} - \lambda(\mathbf{u}^T\mathbf{u} - 1), \qquad \nabla_{\mathbf{u}}\mathcal{L} = 2S\mathbf{u} - 2\lambda\mathbf{u} = \mathbf{0} \implies S\mathbf{u} = \lambda\mathbf{u}.$$

Los candidatos a óptimo son exactamente los autovectores de S. Al multiplicar por $\mathbf{u}^T$ obtenemos $\mathbf{u}^T S\mathbf{u} = \lambda$: la varianza proyectada **es** el autovalor, así que el máximo se alcanza con el autovector de λ_1. El cociente $\mathbf{u}^T S\mathbf{u}/\mathbf{u}^T\mathbf{u}$ se llama *cociente de Rayleigh*.

Hay una segunda lectura. Para un punto centrado $\tilde{\mathbf{x}}_i$, el teorema de Pitágoras separa su norma en la parte proyectada y el residuo perpendicular:

$$\|\tilde{\mathbf{x}}_i\|^2 = (\mathbf{u}^T\tilde{\mathbf{x}}_i)^2 + \|\tilde{\mathbf{x}}_i - (\mathbf{u}^T\tilde{\mathbf{x}}_i)\,\mathbf{u}\|^2.$$

Sumando sobre i y dividiendo entre $n-1$: **varianza total = varianza proyectada + error de reconstrucción**. Como el lado izquierdo no depende de $\mathbf{u}$, maximizar la varianza y minimizar el error son el mismo problema.

En la versión web, un laboratorio permite girar una recta sobre una nube correlacionada y comparar, para cada ángulo, la varianza proyectada $\mathbf{u}^T S\mathbf{u}$ con el error de reconstrucción. Ambas curvas suman la traza de S y alcanzan su máximo y su mínimo en la dirección de PC1.

Guía del laboratorio.

1. Con correlación 0.7 y escala $\times 1$, gira lentamente y anota el ángulo donde la curva naranja ($\mathbf{u}^T S\mathbf{u}$) alcanza su máximo y la morada (error) su mínimo. Pulsa «Saltar a PC1» para verificarlo. ¿Por qué el ángulo está cerca de 45°?
2. Lleva la correlación a 0. Explica por qué la curva se vuelve casi plana y por qué PC1 deja de estar bien definido cuando $\lambda_1 \approx \lambda_2$.
3. Con correlación 0, aumenta la escala de x_2 a $\times 3$. PC1 gira hacia el eje vertical **sin que exista relación entre las variables**: es el efecto de las unidades que motiva estandarizar antes de PCA.

PCA y la Descomposición en Valores Singulares (SVD)

En la práctica, en las paqueterías modernas PCA **nunca** se calcula mediante la eigendescomposición de S directamente (pueden ver el ejemplo de la documentación de scikitlearn). En cambio, se utiliza la **Descomposición en Valores Singulares (SVD)** de la matriz centrada $\tilde{X}$, que es numéricamente más estable y eficiente.

Toda matriz $\tilde{X} \in \mathbb{R}^{n\times p}$ admite la descomposición:

$$\tilde{X} = U\Sigma V^T$$

donde:

[6] **Multiplicadores de Lagrange**: técnica para optimizar una función sujeta a una restricción de igualdad. Se suma la restricción multiplicada por una incógnita λ y se iguala a cero el gradiente respecto a todas las variables. En el óptimo, el gradiente del objetivo es paralelo al de la restricción.

- $U \in \mathbb{R}^{n \times n}$: matriz ortogonal de **vectores singulares izquierdos** (columnas = eigenvectores de $\tilde{X}\tilde{X}^T$)
- $\Sigma \in \mathbb{R}^{n \times p}$: matriz diagonal de **valores singulares** $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\text{mín}(n,p)} \geq 0$
- $V \in \mathbb{R}^{p \times p}$: matriz ortogonal de **vectores singulares derechos** (columnas = eigenvectores de $\tilde{X}^T\tilde{X}$)

La relación entre SVD y PCA es directa:

$$S = \frac{1}{n-1}\tilde{X}^T\tilde{X} = \frac{1}{n-1}(U\Sigma V^T)^T(U\Sigma V^T) = \frac{1}{n-1}V\Sigma^2 V^T$$

Por tanto:

- Las **columnas de** V son los **eigenvectores de** S (componentes principales)
- Los **eigenvalores** de S son $\lambda_k = \sigma_k^2/(n-1)$
- Los **scores** (proyecciones) son $Z = \tilde{X}V = U\Sigma$

¿Por qué SVD y no eigendescomposición directa?

La eigendescomposición de S requiere calcular primero $S = \tilde{X}^T\tilde{X}/(n-1)$, lo cual:

1. **Pierde precisión numérica**: los números de condición se elevan al cuadrado. Si $\tilde{X}$ tiene número de condición κ, S tiene κ^2.
2. **Es ineficiente cuando** $n \ll p$: la SVD puede trabajar directamente con la matrix de $n \times p$ usando la *thin SVD*, evitando la costosa matriz $p \times p$.
3. **El algoritmo de Golub-Reinsch** para SVD es más estable numéricamente que los algoritmos de eigendescomposición para matrices no simétricas.

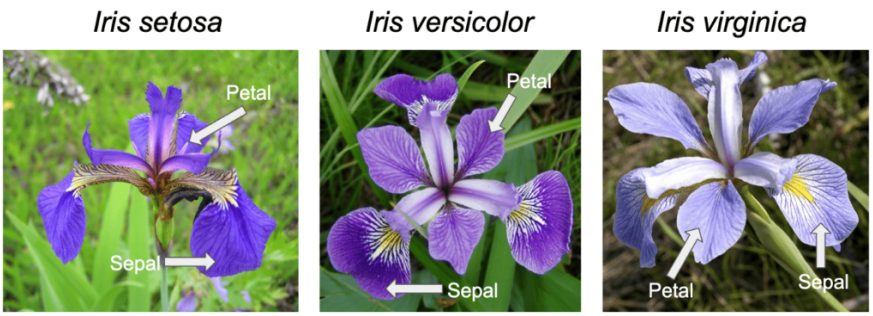
Ejemplo práctico: Implementación paso a paso

Dataset: Iris

Para este ejemplousaremos Scikit-learn, mediante el clásico dataset *Iris* (flores de lirio de este lado del charco), que contiene mediciones de flores de tres especies (*setosa*, *versicolor* y *virginica*). Cada observación está descrita por cuatro variables numéricas:

- longitud del sépalo
- ancho del sépalo
- longitud del pétalo
- ancho del pétalo

Estas variables capturan características morfológicas de las flores y presentan correlaciones entre sí, lo que lo convierte en un ejemplo ideal para ilustrar cómo PCA identifica estructura y reduce dimensionalidad.



	Sépalo largo	Sépalo ancho	Pétalo largo	Pétalo ancho
count	150.000	150.000	150.000	150.000
mean	5.843	3.057	3.758	1.199
std	0.828	0.436	1.765	0.762
min	4.300	2.000	1.000	0.100
25 %	5.100	2.800	1.600	0.300
50 %	5.800	3.000	4.350	1.300
75 %	6.400	3.300	5.100	1.800
max	7.900	4.400	6.900	2.500

Estandarización

En Iris elegimos estandarizar para dar el mismo peso inicial a las cuatro medidas. Sin este paso, la variable con mayor varianza numérica influye más en las primeras componentes. La decisión depende de las unidades y de qué diferencias físicas interesa conservar.

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

Después de estandarizar: $\bar{z}_j = 0$ y $s_{z_j} = 1$ para toda variable j.

	Variable	Media original	Desv original	Media estand.	Desv estand.
0	Sépalo largo	5.843	0.825	-0.0	1.0
1	Sépalo ancho	3.057	0.434	-0.0	1.0
2	Pétalo largo	3.758	1.759	-0.0	1.0
3	Pétalo ancho	1.199	0.760	-0.0	1.0

Matriz de covarianza

> **Nota**
>
> Con `StandardScaler`, cada columna tiene varianza 1 calculada con divisor n; `np.cov` usa $n-1$. Por eso la diagonal aquí vale $n/(n-1)$ y esta matriz es $n/(n-1)$ veces la matriz de correlación de Pearson. El factor común conserva las direcciones principales.

Eigendescomposición vía SVD

	Componente	Eigenvalue ( )	% Varianza	% Acumulado
0	PC1	2.9381	72.96	72.96
1	PC2	0.9202	22.85	95.81
2	PC3	0.1477	3.67	99.48
3	PC4	0.0209	0.52	100.00

Scree plot

> **Tip**
>
> **Criterios para elegir k:**
>
> - **Regla de Kaiser**: heurística de $\lambda_k > 1$ para PCA sobre la matriz de correlaciones; la escala $n/(n-1)$ del ejemplo exige cautela al usar el umbral literal.
> - **Varianza acumulada**: fija un umbral según el costo de perder información y comprueba su impacto en la tarea; 80 % o 90 % son ejemplos, no reglas universales.
> - **Scree plot**: buscar el "codo" donde la pendiente cae abruptamente
> - **Interpretabilidad de dominio**: ¿los componentes tienen sentido en el problema?

Biplot

El **biplot** superpone los scores de las observaciones y los vectores de los loadings en el mismo espacio reducido, permitiendo interpretar simultáneamente la estructura de las observaciones y la contribución de las variables.

Interpretación del biplot:

- Las flechas (loadings) representan los **vectores de las variables proyectados** sobre el nuevo sistema de coordenadas definido por los componentes principales. Es decir, cada flecha muestra cómo una variable original se expresa en la nueva base.
- Flechas largas → la variable tiene **gran proyección (norma)** sobre ese plano, por lo que contribuye significativamente a la variabilidad capturada por los componentes.

- Flechas apuntando en la misma dirección → sus vectores forman un **ángulo pequeño**, lo que indica alta correlación (producto punto positivo grande).
  Flechas opuestas → correlación negativa.
  Flechas ortogonales → variables no correlacionadas.
- La posición de los puntos corresponde a la **proyección de las observaciones** en el subespacio generado por los primeros componentes.
  La separación entre grupos refleja qué tan bien este subespacio (de menor dimensión) preserva la estructura original de los datos.

En conjunto, el biplot muestra simultáneamente: - un **cambio de base** (álgebra lineal)
- y una **proyección de los datos** en un subespacio de menor dimensión

Loadings detallados

Interpretación de los loadings:

Cada componente principal es una **combinación lineal de las variables originales**. Es decir:

$$\mathrm{PC}_k = w_{1k}x_1 + w_{2k}x_2 + \cdots + w_{pk}x_p$$

donde los *loadings* w_{jk} indican el peso de cada variable en ese componente.

- **PC1** combina fuertemente *pétalo largo*, *pétalo ancho* y *sépalo largo* (coeficientes positivos similares), por lo que representa un eje de **tamaño general de la flor**. El *sépalo ancho* contribuye negativamente, ajustando ligeramente esa dirección.
- **PC2** está dominado por *sépalo ancho*, con poca contribución del resto, por lo que captura un **contraste específico en la forma del sépalo**.

En términos geométricos, cada componente define una **dirección en el espacio original**, y los loadings son las **coordenadas de esa dirección** en la base de variables originales.

Ejemplo avanzado: Dataset con ruido

El problema: una partícula en movimiento que no podemos ver directamente

Imagina que una partícula traza una trayectoria helicoidal en el espacio tridimensional. Nosotros no podemos observarla directamente — solo tenemos acceso a **sensores**, y cada sensor introduce su propio ruido de medición.

El desafío central es: dado un conjunto de lecturas ruidosas de múltiples sensores, ¿podemos recuperar la trayectoria original?

PCA puede resumir la variación compartida de las lecturas, pero no garantiza recuperar la trayectoria física exacta. Antes de aplicarlo, construyamos el experimento desde cero.

La trayectoria real y el ruido de medición

Generamos 500 puntos sobre una hélice paramétrica:

$$x(t) = t, \quad y(t) = \cos(t), \quad z(t) = \sin(t), \quad t \in [-10, 10]$$

Luego añadimos ruido gaussiano independiente en y y z para simular la imprecisión de los instrumentos de medida.

Los puntos azules son lo que los sensores registran. La línea roja es la realidad que queremos recuperar. A simple vista ya es difícil distinguir la estructura subyacente del ruido.

Tres sensores, tres perspectivas distintas

En un experimento real, distintos sensores observan el mismo fenómeno desde **ángulos diferentes**. Cada sensor proyecta la nube de puntos sobre su propio plano de referencia y mide coordenadas locales (u, v) en ese plano — más, inevitablemente, su propio ruido adicional.

Simulamos tres sensores con planos de proyección distintos.

Perspectiva 3D: sensores y sus proyecciones.

Cada sensor "aplana" la nube sobre su propio plano. Ninguno captura la hélice perfectamente, pero cada uno aporta información parcial y complementaria.

Lo que registra cada sensor: coordenadas 2D.

Cada sensor produce una imagen 2D distorsionada de la misma hélice. El sensor A la ve casi como una elipse, el B como una banda horizontal, el C como una nube más compacta. Ninguno por sí solo revela la hélice original.

Construir la matriz de datos multi-sensor

Apilamos las seis coordenadas (2 por cada sensor) para formar una **matriz de datos** $X \in \mathbb{R}^{500\times 6}$. Cada fila es una muestra; cada columna es una variable medida por algún sensor.

Esta es exactamente la estructura con la que trabaja PCA: muchas variables correlacionadas que comparten una estructura latente de baja dimensión.

Matriz de correlación entre sensores.

La matriz de correlación revela la redundancia. Variables del mismo sensor están fuertemente correlacionadas entre sí, y también con variables de otros sensores — evidencia de que todas miden el mismo fenómeno subyacente. PCA explotará precisamente esa redundancia.

Aplicar PCA: ¿cuánta información hay realmente?

PCA descompone la varianza total en **componentes ortogonales ordenados por importancia**. Si el fenómeno real vive en pocas dimensiones, la mayor parte de la varianza se concentrará en los primeros componentes.

Los primeros **2–3 componentes** concentran la gran mayoría de la varianza. Esto confirma lo que esperábamos: el sistema tiene **2 grados de libertad reales** (la hélice es una curva en 3D, parametrizada por un solo parámetro t), y el resto es ruido.

Proyección en 2D: la hélice emerge del ruido

Con solo los dos primeros componentes principales recuperamos la estructura esencial del movimiento.

Al colorear los puntos por el parámetro t (el "tiempo" de la trayectoria), vemos que el orden temporal se preserva perfectamente: PCA ha recuperado la parametrización natural de la hélice a partir de seis variables ruidosas.

Proyección en 3D: confirmando la estructura helicoidal

Agregando el tercer componente recuperamos la geometría completa.

La nube de puntos reconstruida en el espacio PCA sigue fielmente la hélice original. PCA ha separado la señal del ruido sin conocer a priori la geometría de la trayectoria.

La idea central

PCA no sabe nada sobre hélices ni sobre física. Solo busca las direcciones de **máxima varianza** en los datos. El hecho de que esas direcciones coincidan con la geometría real de la trayectoria es una consecuencia de que **la señal real tiene más varianza que el ruido**.
Cuando $\text{SNR} = \sigma_{\text{señal}}/\sigma_{\text{ruido}} \gg 1$, los primeros componentes principales capturan la señal; los últimos, el ruido.

Limitación importante

PCA funciona bien aquí porque la hélice es una curva **linealizable** a través de sus coordenadas en el espacio de observaciones. Para estructuras genuinamente no lineales (manifolds curvos), se necesitan técnicas como **UMAP** o **t-SNE**.

Limitaciones del PCA

Linealidad

PCA busca direcciones de máxima varianza **lineales**. No puede capturar estructuras no lineales en los datos.

Este es un ejemplo clásico: el Swiss Roll es una variedad no lineal que PCA no puede "desenrollar". Técnicas como **Isomap** o **UMAP** sí pueden recuperar la estructura subyacente.

Sensibilidad a outliers

PCA minimiza la varianza de la reconstrucción, que es equivalente a minimizar el error cuadrático medio. Por tanto, es **muy sensible a outliers**, ya que un solo punto alejado puede dominar la dirección del primer componente.

La alternativa robusta es **Robust PCA** (Candès et al., 2011), que descompone $X = L + S$ donde L es la componente de bajo rango y S es una matriz dispersa de outliers.

PCA no captura estructura de clases

PCA es un método **no supervisado** — maximiza varianza global sin considerar etiquetas. Puede ocurrir que la dirección de máxima varianza no sea la que mejor separa las clases.

Normalidad: cuándo entra en juego

PCA clásico no requiere normalidad para maximizar la varianza proyectada. Un modelo distinto, el PCA probabilístico, sí añade supuestos gaussianos para las variables latentes y el ruido:

$$x = Wz + \mu + \varepsilon, \quad z \sim \mathcal{N}(0, I), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

En datos muy asimétricos o discretos, revisa si la distancia euclidiana y la reconstrucción lineal responden a la pregunta del análisis; la falta de normalidad por sí sola no invalida PCA clásico.

Más allá del PCA: técnicas no lineales

Cuando la estructura de interés no cabe bien en una proyección lineal, PCA puede perderla. t-SNE y UMAP producen representaciones útiles para explorar vecindarios; sus ejes y distancias entre grupos requieren interpretación cautelosa.

Para esta sección emplearemos el MNIST dataset, un dataset de imágenes de dígitos escritos a mano que tiene una estructura no lineal compleja con las siguientes caracteristicas:

- 70,000 imágenes de dígitos manuscritos (28x28 píxeles → 784 dimensiones)
- 10 clases (dígitos 0–9)

- Desafío clásico para reducción de dimensionalidad

Notemos que si realizamos PCA sobre este dataset, los dígitos no se separan claramente en el espacio de componentes principales. Esto se debe a que la estructura de los datos es altamente no lineal: dígitos similares pueden estar cerca en el espacio original pero no necesariamente en el espacio PCA.

A continuación, veremos cómo técnicas no lineales como t-SNE y UMAP pueden revelar la estructura subyacente de los dígitos, agrupando visualmente los dígitos similares y separando claramente las clases, a pesar de la complejidad no lineal de los datos.

t-SNE (t-distributed Stochastic Neighbor Embedding)

t-SNE (van der Maaten & Hinton, 2008) construye una distribución de probabilidad sobre pares de puntos en el espacio original y minimiza la divergencia KL con la distribución en el espacio reducido:

$$\mathrm{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

donde p_{ij} captura similitudes en el espacio original y q_{ij} en el espacio reducido usando una distribución t de Student.

Características:

- Excelente para visualización en 2D/3D
- Preserva estructura **local** (vecinos cercanos)
- **No preserva distancias globales** — la distancia entre clusters no es interpretable
- No determinista: ejecuciones distintas dan resultados distintos
- No produce una transformación reutilizable para nuevos datos
- Hiperparámetro clave: **perplexity** (típicamente 5–50)

La *perplexity* controla cuántos vecinos considera relevantes cada punto al construir la representación.

Formalmente, está relacionada con la **entropía de una distribución de probabilidad local**:

$$\text{Perplexity} = 2^{H(P_i)}$$

donde P_i describe qué tan probable es que otros puntos sean vecinos de x_i. Ajustar la perplexity es crucial: valores muy bajos pueden fragmentar la estructura, mientras que valores muy altos pueden mezclar clusters distintos.

UMAP (Uniform Manifold Approximation and Projection)

UMAP (McInnes et al., 2018) está fundamentado en la **teoría de categorías** y la geometría Riemanniana. El algoritmo construye una representación del manifold topológico de los datos y optimiza una representación en baja dimensión que preserve esa topología.

Matemáticamente, UMAP modela cada punto x_i como el centro de una bola geodésica de radio ρ_i (distancia al vecino más cercano), construyendo un grafo difuso ponderado con pesos:

$$w(x_i, x_j) = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$$

y simetrizado como:

$$B(x_i, x_j) = w(x_i, x_j) + w(x_j, x_i) - w(x_i, x_j) \cdot w(x_j, x_i)$$

La proyección minimiza la divergencia de entropia cruzada fuzzy entre el grafo original y el proyectado.

Ventajas sobre t-SNE:

Característica	t-SNE	UMAP
Velocidad	Lento ($O(n^2)$)	Rápido ($O(n^{1.14})$)
Estructura global	No preserva	Preserva mejor
Nuevos datos	No (sin extensión)	Sí (`transform()`)
Determinismo	No	Sí (con `random_state`)
Escalabilidad	Hasta ~10k puntos	Hasta millones

Cuándo usar cada método

Situación	Método recomendado
Compresión lineal antes de modelado	**PCA**, validando el efecto en la métrica de la tarea
Visualización exploratoria de vecindarios	**t-SNE** o **UMAP**, comparando parámetros y semillas
Muchos puntos	Probar **PCA** o **UMAP** según memoria, tiempo y objetivo
Necesito transformar nuevos datos	**PCA**; **UMAP** si el modelo ajustado y la implementación ofrecen `transform` adecuado
Interpretabilidad de componentes	**PCA** (o Sparse PCA)
Estructura no lineal compleja	**UMAP**
Análisis supervisado + reducción	**LDA**

La elección del método de reducción de dimensionalidad depende del objetivo del análisis y de la naturaleza de los datos. No existe una técnica universalmente superior: cada método enfatiza distintos aspectos de la estructura.

PCA es adecuado cuando se busca una representación lineal, interpretable y reutilizable, especialmente en contextos de preprocesamiento para modelos. En contraste, t-SNE y UMAP están diseñados principalmente para exploración visual, priorizando la preservación de relaciones locales, aunque sacrifican interpretabilidad y capacidad de generalización.

UMAP puede preservar vecindarios y resultar útil con conjuntos grandes, pero el resultado depende de parámetros y recursos. LDA incorpora etiquetas y puede servir cuando el objetivo es discriminación supervisada.

En conjunto, la elección del método debe responder a la pregunta central del análisis:
¿buscamos interpretar, visualizar o modelar?

Resumen y conexiones

El PCA es el punto de partida del análisis multivariante moderno. Sus conexiones van más allá de la estadística descriptiva:

- **Álgebra lineal**: eigendescomposición, SVD, espacios vectoriales
- **Estadística**: modelo probabilístico gaussiano, máxima verosimilitud
- **Optimización**: problema de Rayleigh quotient, multiplicadores de Lagrange
- **Aprendizaje automático**: preprocesamiento, regularización, autoencoders lineales
- **Métodos de variedades**: UMAP y t-SNE usan objetivos distintos de la reconstrucción lineal de PCA

La familia de métodos de reducción de dimensionalidad puede verse como un espectro continuo desde lo lineal (PCA) hasta lo fuertemente no lineal (t-SNE, UMAP), con un trade-off entre interpretabilidad, preservación de estructura global y escalabilidad.

Laboratorio aplicado: ¿conviene PCA para clasificar Iris?

Objetivo. Separar la pregunta "¿cuánta varianza conserva PCA?" de "¿mejora un clasificador en datos no vistos?". Usaremos una partición estratificada[7] y ajustaremos todo el preprocesamiento dentro de cada pipeline.

1. Ejecuta el código y anota la varianza explicada por dos componentes, la exactitud balanceada del modelo con PCA y la del modelo con las cuatro medidas.
2. Cambia `n_components=2` por `1` y después por `3`. Antes de ejecutar cada variante, predice qué ocurrirá con la varianza acumulada y el error de reconstrucción.
3. Repite con otra semilla en `train_test_split`. Explica qué conclusiones son estables y cuáles dependen de una partición pequeña.

[7] **Partición estratificada**: divide los datos manteniendo aproximadamente la proporción de cada clase en entrenamiento y prueba.

	Medida	Valor
0	Varianza explicada en entrenamiento	0.960230
1	Error medio de reconstrucción en prueba	0.040233
2	Exactitud balanceada con PCA en prueba	0.888889
3	Exactitud balanceada sin PCA en prueba	0.911111

Comprobación. Verifica que `modelo_pca.named_steps['pca'].n_components_ == 2`, que la varianza explicada queda entre 0 y 1 y que ninguna medición de prueba interviene en `.fit`. Al aumentar k, la varianza acumulada no puede disminuir y el error de reconstrucción de los mismos datos transformados no puede aumentar, salvo redondeo numérico.

Interpretación y entrega. Redacta una recomendación de tres frases para alguien que deba desplegar el clasificador: incluye la diferencia observada en exactitud balanceada, el costo de pasar de cuatro variables a k componentes y una limitación de decidir con una sola partición. Si la exactitud es similar, no concluyas que PCA es siempre necesario.

Práctica de trabajo reproducible

Guarda el pipeline completo para inferencia y registra la partición, la semilla, k, la métrica y las versiones de dependencias. Para elegir k, usa validación cruzada dentro de entrenamiento; reserva la prueba final para estimar el rendimiento después de elegirlo.

Ejercicios propuestos

1. **PCA desde cero**: implementa PCA usando únicamente `numpy.linalg.svd` sin usar `sklearn`. Verifica que tus eigenvalores y eigenvectores coincidan con los de `sklearn.decomposition.PCA`.

2. **Efecto de la estandarización**: aplica PCA al dataset Iris con y sin estandarización. ¿Cambian los componentes? ¿Cambia la varianza explicada? Justifica matemáticamente.

3. **Reconstrucción**: dado un dataset X y su proyección PCA a k componentes, reconstruye la aproximación $\hat{X} = Z_k W_k^T + \mu$. Grafica el error de reconstrucción $\|X - \hat{X}\|_F$ en función de k.

4. **Comparación t-SNE vs UMAP**: aplica ambas técnicas al dataset de dígitos MNIST (disponible en `sklearn.datasets.load_digits`). Compara visualmente la separación de clases y el tiempo de cómputo.

5. **Regla de Kaiser**: ¿por qué la regla de Kaiser ($\lambda > 1$) solo aplica cuando los datos están estandarizados? Demuéstralo con un ejemplo donde viola la regla sin estandarizar.

Recursos recomendados: EDA multivariado

A continuación se listan libros y recursos clave para profundizar en análisis exploratorio de datos multivariado, reducción de dimensionalidad y visualización. Los pongo en mi orden de preferencia, pero todos son excelentes y complementarios.

Análisis multivariado (nivel intermedio).

- **Brunton & Kutz — *Data-Driven Science and Engineering***
  https://databookuw.com/
  → Enfoque moderno: PCA, SVD, sistemas dinámicos. (Libre)
- **Trevor Hastie, Robert Tibshirani, Jerome Friedman — *The Elements of Statistical Learning***
  https://web.stanford.edu/~hastie/ElemStatLearn/
  → PCA, LDA, reducción de dimensionalidad y aprendizaje estadístico. (Libre)
- **Gareth James et al. — *An Introduction to Statistical Learning***
  https://www.statlearning.com/
  → Versión más accesible del anterior, incluye PCA y LDA. (Libre)
- **Kevin Murphy — *Probabilistic Machine Learning: An Introduction***
  https://probml.github.io/pml-book/book1.html
  → Enfoque moderno probabilístico, incluye PCA y modelos latentes. (Libre)

Geometría y fundamentos (nivel más teórico).

- **Gilbert Strang — *Linear Algebra and Learning from Data***
  https://math.mit.edu/~gs/learningfromdata/
  → Conexión entre álgebra lineal, PCA y datos. (Parcialmente libre)

Visualización y alta dimensión.

- **Claus O. Wilke — *Fundamentals of Data Visualization***
  https://clauswilke.com/dataviz/
  → Principios de visualización (muy útil para EDA). (Libre)
- **Distill.pub (artículos interactivos)**
  https://distill.pub/
  → Explicaciones visuales de t-SNE, embeddings y ML. (Libre)

42. Reducción de Variables Categóricas y Prueba ^{2}

En la etapa de preprocesamiento y selección de características (*feature selection*), a menudo nos enfrentamos a conjuntos de datos con múltiples variables categóricas. Herramientas comunes como el coeficiente de correlación de Pearson o Spearman están diseñadas para variables numéricas (continuas u ordinales) y no son aplicables a variables categóricas nominales, donde no existe un orden inherente (por ejemplo, tipo de cliente, canal de compra o nivel educativo frente a abandono del servicio).

La redundancia entre variables categóricas tiene un costo concreto: si dos variables transmiten casi la misma información, o si un predictor no guarda relación con la variable objetivo (*target*), conservarlas infla la dimensionalidad del modelo. El efecto se multiplica con *One-Hot Encoding*, donde cada categoría añade una columna sin aportar necesariamente señal nueva.

Para ilustrarlo, utilizaremos un **ejemplo didáctico inspirado en BigMart Sales**: el tipo de tienda (`Outlet_Type`: Supermarket Type1, Grocery Store) y su tamaño (`Outlet_Size`: Medium, Small). ¿Están asociadas estas dos variables en la muestra imaginaria, o son independientes? Los conteos siguientes son inventados para aprender el método; no describen el conjunto de datos original.

Intuición: ¿qué esperaríamos si no hubiera relación?

Antes de cualquier fórmula, hagamos una predicción con proporciones. De 100 tiendas, 50 son *Medium*: la mitad. Si el tamaño no tuviera nada que ver con el tipo, **la mitad de los 40 supermercados también debería ser *Medium***, es decir, unos 20. Observamos 30. Con las tiendas de abarrotes ocurre lo contrario: esperaríamos 30 medianas de 60 y observamos 20.

La prueba ^{2} formaliza exactamente esta comparación: construye la tabla que esperaríamos bajo independencia, mide cuánto se aleja la tabla observada **en cada celda** y pondera cada diferencia por el tamaño esperado (diez tiendas de diferencia pesan más si esperábamos 20 que si esperábamos 2000). Después pregunta si una distancia total así de grande sería rara si las variables fueran independientes.

Desarrollo manual paso a paso

Organizamos los 100 registros en una **tabla de contingencia**[1] de 2×2:

[1] **Tabla de contingencia:** matriz de conteos en la que cada celda combina una categoría de la primera variable con otra de la segunda.

Outlet_Type	Medium	Small	Total fila (R_i)
Supermarket Type1	30	10	40
Grocery Store	20	40	60
Total columna (C_j)	50	50	100 (N)

Frecuencias marginales

Los totales de fila (R_i) son 40 y 60; los de columna (C_j), 50 y 50; el total general (N) es 100. Divididos entre N son las **proporciones marginales**[2], es decir, 40 % de supermercados y 50 % de tiendas medianas.

Frecuencias esperadas (E_{ij})

Bajo la hipótesis nula[3] de independencia, la probabilidad de una celda es el producto de las probabilidades marginales, así que el conteo esperado es $N \cdot \frac{R_i}{N} \cdot \frac{C_j}{N}$:

$$E_{ij} = \frac{R_i \cdot C_j}{N}$$

- E_{11} (Supermarket Type1, Medium): $\frac{40 \cdot 50}{100} = 20$
- E_{12} (Supermarket Type1, Small): $\frac{40 \cdot 50}{100} = 20$
- E_{21} (Grocery Store, Medium): $\frac{60 \cdot 50}{100} = 30$
- E_{22} (Grocery Store, Small): $\frac{60 \cdot 50}{100} = 30$

La tabla esperada conserva los mismos totales de fila y columna que la observada; solo redistribuye los conteos como si no hubiera asociación.

Contribuciones a ^{2}

Cada celda aporta su diferencia al cuadrado, relativa a lo esperado:

$$\frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

- Celda 1,1: $\frac{(30-20)^2}{20} = \frac{100}{20} = 5.0$
- Celda 1,2: $\frac{(10-20)^2}{20} = \frac{100}{20} = 5.0$
- Celda 2,1: $\frac{(20-30)^2}{30} = \frac{100}{30} \approx 3.333$
- Celda 2,2: $\frac{(40-30)^2}{30} = \frac{100}{30} \approx 3.333$

Las cuatro diferencias valen ± 10: en una tabla 2×2 con márgenes fijos, si una celda sube diez, su vecina de fila y su vecina de columna deben bajar diez. Por eso solo hay un grado de libertad.

[2] **Distribución marginal:** distribución de una sola variable obtenida al sumar la tabla sobre las categorías de la otra. Sus proporciones estiman $P(X = x_i)$ y $P(Y = y_j)$.

[3] **Hipótesis nula (H_0):** afirmación que se supone cierta para calcular qué resultados serían esperables; aquí, que tipo y tamaño de tienda son independientes. La prueba no demuestra H_0: solo evalúa si los datos son poco compatibles con ella.

Estadístico $^2_{calc}$ y valor p

$$\chi^2_{\text{calc}} = 5.0 + 5.0 + 3.333 + 3.333 = 16.667$$

Los grados de libertad[4] son $(r-1)(c-1) = (2-1)(2-1) = 1$. Si H_0 fuera cierta, χ^2_{calc} seguiría aproximadamente una distribución ^{2} con un grado de libertad[5]. El **valor** p[6] es la probabilidad de obtener un estadístico al menos tan grande como el observado bajo esa distribución:

$$p = P\left(\chi^2_1 \geq 16.667\right) \approx 4.5 \times 10^{-5}.$$

Con un nivel de significancia[7] $\alpha = 0.05$, el valor crítico es 3.841. Como $16.667 > 3.841$ (equivalentemente, $p < 0.05$), rechazamos la independencia en esta muestra ficticia. Esto aporta evidencia de asociación; no demuestra causalidad ni garantiza utilidad predictiva.

Fuerza de asociación: V de Cramér

El valor p cuantifica qué tan inusual sería una discrepancia como la observada bajo independencia[8]; no mide la fuerza de la asociación. Para ello dividimos ^{2} entre su máximo posible, $N \cdot \text{mín}(r-1, c-1)$:

$$V = \sqrt{\frac{\chi^2}{N \cdot \text{mín}(r-1, c-1)}} = \sqrt{\frac{16.667}{100}} \approx 0.408$$

En una tabla 2×2 existe un atajo útil para comprobar el cálculo: $V = |\phi| = \frac{|O_{11}O_{22} - O_{12}O_{21}|}{\sqrt{R_1 R_2 C_1 C_2}} = \frac{|30 \cdot 40 - 10 \cdot 20|}{\sqrt{40 \cdot 60 \cdot 50 \cdot 50}} = \frac{1000}{2449.5} \approx 0.408$.

Un experimento mental. Si multiplicamos todos los conteos por 10 (misma proporción en cada celda, $N = 1000$), cada contribución se multiplica por 10: $\chi^2 = 166.7$ y $p \approx 4 \times 10^{-38}$. Pero V sigue en 0.408, porque depende solo de las proporciones. Una muestra más grande vuelve más **segura** la conclusión, no más **fuerte** la asociación.

[4] **Grados de libertad:** número de conteos que pueden variar una vez fijados los totales de filas y columnas. En una tabla 2×2 queda uno.

[5] **Distribución ^{2} con k grados de libertad:** distribución de la suma de los cuadrados de k variables normales estándar independientes. Tiene media k y varianza $2k$; con $k = 1$ concentra la mayor parte de su masa cerca de cero, y $P(\chi^2_1 \geq 3.841) = 0.05$.

[6] **Valor p:** probabilidad, calculada suponiendo H_0 cierta, de obtener un estadístico al menos tan extremo como el observado. No es la probabilidad de que H_0 sea cierta ni mide el tamaño del efecto.

[7] **Nivel de significancia (α):** umbral fijado **antes** de mirar los datos para decidir cuándo rechazar H_0. Es la probabilidad de rechazarla cuando en realidad es cierta (error de tipo I). El valor 0.05 es una convención, no una ley.

[8] **Sensibilidad al tamaño de muestra (N):** con muestras grandes, diferencias pequeñas pueden producir valores p bajos. Por eso conviene acompañar la prueba con un tamaño del efecto como V de Cramér.

Ver la discrepancia antes de interpretar el resultado

Comparamos la tabla observada con la esperada en dos mapas de calor interactivos. Ambas usan el mismo rango numérico de color para facilitar la comparación entre celdas.

La celda *Supermarket Type1–Medium* contiene 30 registros frente a 20 esperados. El contraste ocurre también en las otras tres celdas; ninguna por sí sola explica toda la decisión.

El signo muestra la dirección de la diferencia. El cuadrado de cada residuo es la contribución de esa celda al ^{2}. No interpretes cada residuo como una prueba independiente sin ajustar por las comparaciones múltiples.

Formalización teórica y matemática

La base formal de esta prueba es la definición de **independencia estadística**[9] para variables aleatorias categóricas X e Y:

$$H_0 : \; P(X = x_i, Y = y_j) = P(X = x_i) \cdot P(Y = y_j) \quad \forall i, j$$

Las probabilidades marginales se estiman con las proporciones observadas, $\hat{p}_{i\cdot} = R_i/N$ y $\hat{p}_{\cdot j} = C_j/N$; son los estimadores de máxima verosimilitud[10] bajo H_0. De ahí $E_{ij} = N\hat{p}_{i\cdot}\hat{p}_{\cdot j} = R_i C_j/N$.

El **estadístico ^{2} de Pearson** suma las discrepancias estandarizadas:

$$\chi^2 = \sum_{i=1}^{r}\sum_{j=1}^{c} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} = \sum_{i,j} r_{ij}^2, \qquad r_{ij} = \frac{O_{ij} - E_{ij}}{\sqrt{E_{ij}}}.$$

Cada conteo es aproximadamente Poisson, con varianza cercana a su media E_{ij}; por eso r_{ij} se comporta, para conteos grandes, como una variable normal estándar, y la suma de sus cuadrados como una ^{2}. Las restricciones de los márgenes eliminan $r + c - 1$ grados de libertad de las rc celdas, lo que deja $rc - (r + c - 1) = (r - 1)(c - 1)$.

Dos propiedades explican el experimento mental anterior. Si todos los conteos se multiplican por $k > 0$:

$$\chi^2(kO) = k\,\chi^2(O), \qquad V(kO) = V(O).$$

[9] **Independencia estadística:** conocer el valor de una variable no cambia las probabilidades de la otra: $P(Y = y_j \mid X = x_i) = P(Y = y_j)$. Multiplicando por $P(X = x_i)$ se obtiene la factorización de la probabilidad conjunta.

[10] **Estimador de máxima verosimilitud:** valor de los parámetros que hace más probable la muestra observada bajo el modelo supuesto. Para proporciones de una distribución multinomial, coincide con la frecuencia relativa observada.

Supuestos de validez

Las observaciones deben ser independientes: no se debe contar repetidamente la misma tienda como si fueran tiendas distintas. Como regla orientativa para la aproximación asintótica, revisa los **criterios de Cochran**[11]: ninguna celda esperada menor que 1 y, como máximo, el 20 % de las celdas esperadas menor que 5. En una tabla 2×2 esto suele implicar revisar con especial cuidado cualquier frecuencia esperada menor que 5.

Corrección por continuidad de Yates

En tablas de 2×2 puede utilizarse la **corrección de Yates**[12]: resta 0.5 a cada $|O_{ij} - E_{ij}|$ antes de elevar al cuadrado. En nuestro ejemplo, $\chi^2_{\text{Yates}} = 9.5^2(2/20 + 2/30) = 15.04$. No es obligatoria por el solo hecho de tener una tabla 2×2: informa si se aplicó y usa el mismo criterio al comparar resultados.

Prueba exacta de Fisher

Cuando la aproximación ^{2} es dudosa en una tabla 2×2, la **prueba exacta de Fisher**[13] calcula el valor p sin aproximaciones: fija los márgenes y enumera todas las tablas posibles con su probabilidad hipergeométrica. Para nuestra tabla da $p \approx 8.3 \times 10^{-5}$, del mismo orden que la aproximación. También debemos ser cautos con conclusiones basadas en datos agregados debido a la **paradoja de Simpson**[14].

Laboratorio interactivo: de la tabla observada a la distribución nula

Objetivo. Ver la independencia como una comparación de proporciones, distinguir significación estadística de fuerza de asociación y entender de dónde sale el valor p.

1. Con el ejemplo inicial, comprueba en el mosaico que las barras no se cortan en la línea punteada (la proporción de *Medium* que esperaríamos bajo independencia). Verifica $\chi^2 \approx 16.67$, $V \approx 0.408$ y $p \approx 4.5 \times 10^{-5}$.
2. Pulsa «Simular bajo H ». Cada barra del histograma resume tablas generadas **sin** asociación con el mismo N. ¿Qué fracción supera el ^{2} observado? Compárala con el valor p teórico.
3. Carga «Asociación débil» (`25, 15, 25, 35`). Predice ^{2} y V antes de mirar. Luego mueve «Multiplicar la muestra» hasta $\times 10$: anota cómo cambian ^{2}, p y V.

[11] **Criterios de Cochran**: Establecen las condiciones bajo las cuales la aproximación asintótica a la distribución ^{2} es fiable. Principalmente, requieren que no más del 20 % de las celdas tengan frecuencias esperadas menores a 5, y que ninguna celda tenga una frecuencia esperada menor a 1.

[12] **Corrección de continuidad de Yates:** ajuste conservador para la aproximación de conteos discretos por una distribución continua en tablas 2×2. Puede reducir potencia; no reemplaza automáticamente una prueba exacta en tablas pequeñas.

[13] **Prueba exacta de Fisher:** prueba para tablas 2×2 que calcula probabilidades condicionadas a los márgenes mediante la distribución hipergeométrica, sin depender de la aproximación ^{2}.

[14] **Paradoja de Simpson**: Ocurre cuando una tendencia aparece en diferentes grupos de datos pero desaparece o se invierte cuando estos grupos se combinan. Es un riesgo al analizar tablas de contingencia agregadas sin controlar por variables de confusión subyacentes.

4. Carga «Celdas escasas» (`2, 0, 1, 3`). Explica por qué el aviso de Cochran aparece y compara el valor p de ^{2} con el de Fisher.

En la versión web, este laboratorio calcula ^{2}, el valor p asintótico, el valor p exacto de Fisher y V de Cramér para cualquier tabla 2×2; dibuja un mosaico de proporciones y simula la distribución del estadístico bajo independencia. Para resolverlo sin controles: repite el cálculo manual con `25, 15, 25, 35` ($\chi^2 \approx 4.17$, $p \approx 0.041$, $V \approx 0.204$) y con `2, 0, 1, 3` (frecuencias esperadas menores que 5; Fisher da $p = 0.40$).

Comprobación. Con `25, 15, 25, 35` obtendrás $\chi^2 \approx 4.17$, $p \approx 0.041$ y $V \approx 0.204$: hay evidencia al 5 %, pero la asociación es la mitad de fuerte que en el ejemplo inicial. Multiplicar por 10 da $\chi^2 \approx 41.7$ con la misma V. En la tabla escasa, ^{2} sugiere $p \approx 0.083$ pero Fisher da $p = 0.40$: la aproximación no es fiable con tan pocos datos.

Interpretación para entregar. Escribe dos frases: una sobre la evidencia de asociación (valor p y prueba usada) y otra sobre su magnitud (V). Añade una tercera sobre qué aprendiste del experimento «multiplicar la muestra» para leer resultados con miles de registros.

Mejores prácticas en la industria

Mejores prácticas en la industria

Cardinalidad y tamaño del efecto. Una variable con muchas categorías puede producir celdas escasas. Agrupa categorías infrecuentes solo si existe una justificación semántica y define la regla con los datos de entrenamiento. Compara el valor p con V de Cramér y el rendimiento del modelo: una muestra grande puede detectar asociaciones de poca utilidad práctica. La información mutua[15] es otra medida posible, pero también requiere controlar el sesgo que introduce la cardinalidad.

Regla de oro en producción

Prevención de fuga de datos. Si usas la relación con la variable objetivo para seleccionar predictores, aprende la selección dentro de cada partición de entrenamiento y evalúala en datos separados. Si solo exploras asociación entre dos columnas, registra igualmente la procedencia de los datos y evita convertir un hallazgo exploratorio en una regla de producción sin validación.

Implementación computacional y selección de variables

Comprobamos los cuatro conteos ficticios de BigMart. `scipy.stats.chi2_contingency` usa por defecto una corrección en tablas 2×2; la desactivamos para reproducir el cálculo manual.

La tabla describe asociación entre **dos predictores**; por sí sola no indica cuál mejorará las predicciones de ventas. Para seleccionar variables con un objetivo `y`, se necesita un procedimiento supervisado dentro de las particiones de entrenamiento. Además, `sklearn.feature_selection.chi2`

[15] **Información mutua:** medida de dependencia basada en cuánto reduce conocer una variable la incertidumbre sobre otra.

calcula una prueba de puntuación para características no negativas frente a clases; su resultado no equivale automáticamente a la prueba de independencia sobre una tabla de contingencia ni a V de Cramér.

Visualizar redundancia entre predictores

En este ejemplo pequeño añadimos una categoría `Outlet_Region` inventada para ilustrar una tercera variable; los valores no representan datos reales.

Comprobación. El valor para `Outlet_Type` y `Outlet_Size` debe ser 0.408. Las asociaciones entre predictores sugieren redundancia posible; decídela mediante validación del modelo, costo de captura y estabilidad temporal.

Parte XXI.

Extracción de Patrones

43. Extracción de Patrones: Datos Transaccionales y Espacio de Búsqueda

Dentro de CRISP-DM, la extracción de patrones pertenece a la fase de **modelado**, pero con un objetivo distinto al de la clasificación: no hay una variable que predecir. La pregunta es descriptiva, *¿qué cosas ocurren juntas con más frecuencia de la que cabría esperar?*, y la respuesta sirve a su vez para entender el negocio (fase 1) y para construir variables o reglas que luego alimentan un modelo predictivo. Los datos de entrada suelen salir directamente del almacén de datos: la tabla de hechos de ventas, con una fila por línea de ticket, es la materia prima natural de este análisis.

Esta parte del libro avanza de lo más simple a lo más elaborado:

Tabla 43.1.: Ruta de la parte de extracción de patrones.

Capítulo	Pregunta que responde
Este capítulo	¿Cómo se representan las transacciones y cuántos patrones posibles hay?
Reglas de asociación	¿Qué miden el soporte, la confianza y el lift de una regla $A \Rightarrow B$?
Algoritmo Apriori	¿Cómo encontrar todos los itemsets frecuentes sin revisar todos los posibles?
Métricas avanzadas	¿Cómo distinguir reglas interesantes de reglas triviales?
FP-Growth	¿Cómo evitar generar candidatos comprimiendo los datos en un árbol?
Itemsets cerrados y maximales	¿Cómo resumir miles de itemsets frecuentes sin perder información?
Patrones secuenciales	¿Qué ocurre cuando importa el orden de los eventos?
Clasificación asociativa	¿Cómo convertir reglas en un clasificador y evaluarlo?

Intuición: cinco tickets

Imaginemos una tienda pequeña que registró cinco compras:

Tabla 43.2.: Base de transacciones de ejemplo, la misma que se usará en los capítulos de reglas y de Apriori.

Ticket	Productos
T_1	leche, pan, mantequilla
T_2	leche, pan
T_3	leche, mantequilla
T_4	pan, mantequilla
T_5	leche, pan, queso

Sin fórmulas, ya podemos hacer observaciones: leche y pan aparecen juntos en tres de los cinco tickets; queso solo aparece una vez; nadie compró los cuatro productos a la vez. Un **patrón** es exactamente eso: un conjunto de productos (un *itemset*) junto con el número de tickets en los que aparece completo. El trabajo de este capítulo es hacer precisa esa idea y descubrir por qué contar *todos* los patrones posibles se vuelve imposible muy rápido.

De la tabla de hechos a la canasta

En un almacén de datos las ventas no se guardan como listas de productos sino en **formato largo**: una fila por línea de ticket, con la llave del ticket y la del producto. Para minar patrones hay que hacer dos transformaciones:

1. **Agrupar** por ticket y quedarse con el *conjunto* de productos (sin cantidades ni precios: dos cafés en el mismo ticket cuentan como uno).
2. **Codificar** cada canasta como una fila de una matriz binaria[1]: una columna por producto, con 1 si el producto está en el ticket y 0 si no.

Con los datos de la cafetería que se usaron en la parte de almacenes (`lineas_pos.csv` y `productos.csv`), el proceso completo es:

order_id	Americano	Brownie	Capuccino	Cheesecake	Cold Brew	Croissant	Frappé Vainilla	Latte	M
P000001	0	0	1	0	1	1	1	0	0
P000002	0	1	0	0	0	0	0	0	0
P000003	0	0	0	0	0	0	1	1	0
P000004	0	1	0	1	0	0	0	0	0

La matriz binaria es la entrada de todos los algoritmos de esta parte. Vista completa, se nota una propiedad que tienen casi todos los datos transaccionales reales: está **llena de ceros**.

Con 10 productos y tickets de 2 a 4 artículos, alrededor de una cuarta parte de las celdas vale 1. En un supermercado con miles de productos y canastas de 10 artículos, la densidad cae por debajo

[1]Esta codificación es el mismo *one-hot encoding* del capítulo de conversión de variables, aplicado a una variable cuyo valor es un conjunto. Una matriz con muy pocos unos se llama **dispersa** (*sparse*); guardarla solo con las posiciones de los unos ahorra memoria.

Figura 43.1.

del 1 %. Por eso las implementaciones serias guardan la matriz en formato disperso y por eso, como veremos, el soporte de casi cualquier combinación de productos es muy pequeño.

Construcción numérica: soporte paso a paso

Volvamos a los cinco tickets de la Tabla 43.2. Para cada itemset contamos en cuántos tickets aparece **completo**:

- $\{\text{leche}\}$ está en T_1, T_2, T_3, T_5: conteo 4, soporte $4/5 = 0.80$.
- $\{\text{pan}\}$ está en T_1, T_2, T_4, T_5: conteo 4, soporte 0.80.
- $\{\text{mantequilla}\}$ está en T_1, T_3, T_4: conteo 3, soporte 0.60.
- $\{\text{queso}\}$ está solo en T_5: conteo 1, soporte 0.20.
- $\{\text{leche}, \text{pan}\}$ está en T_1, T_2, T_5: conteo 3, soporte 0.60.
- $\{\text{leche}, \text{pan}, \text{mantequilla}\}$ solo está en T_1: conteo 1, soporte 0.20.

Observa la cadena $\{\text{leche}\} \subset \{\text{leche}, \text{pan}\} \subset \{\text{leche}, \text{pan}, \text{mantequilla}\}$: los conteos son $4 \geq 3 \geq 1$. **Al agregar productos a un itemset, su conteo nunca sube.** No es casualidad: un ticket que contiene a $\{\text{leche}, \text{pan}\}$ forzosamente contiene a $\{\text{leche}\}$. Esta observación sencilla es la que hace posible todo lo que sigue.

Con un umbral de **conteo mínimo 2** (soporte mínimo 0.40), los itemsets frecuentes son seis: los tres productos frecuentes por separado y los tres pares $\{\text{leche}, \text{pan}\}$ (3), $\{\text{leche}, \text{mantequilla}\}$ (2) y $\{\text{pan}, \text{mantequilla}\}$ (2).

Formalización

Sea $\mathcal{I} = \{i_1, \dots, i_m\}$ el catálogo de ítems y $\mathcal{D} = \{T_1, \dots, T_n\}$ una base de transacciones, con $T_j \subseteq \mathcal{I}$.

Definición (soporte). El conteo de soporte de un itemset $X \subseteq \mathcal{I}$ es $\sigma(X) = |\{T_j \in \mathcal{D} : X \subseteq T_j\}|$ y su soporte relativo es

$$\operatorname{supp}(X) = \frac{\sigma(X)}{n}. \tag{43.1}$$

Dado un umbral $s_{\text{mín}}$, X es **frecuente** si $\operatorname{supp}(X) \geq s_{\text{mín}}$.

El soporte relativo es una probabilidad empírica[2]: la proporción de tickets que contienen a X.

[2]Una **probabilidad empírica** es una frecuencia relativa observada: si de n tickets σ contienen a X, se estima $P(X) \approx \sigma/n$. Con más datos la estimación es más estable, pero sigue siendo una propiedad de la muestra, no una certeza sobre la población.

El espacio de búsqueda. Con m ítems existen $2^m - 1$ itemsets no vacíos. Ordenados por inclusión forman una **retícula**[3]: arriba los ítems sueltos, abajo el itemset con todo el catálogo, y una arista entre X e Y cuando Y se obtiene agregando un ítem a X.

Tabla 43.3.: Número de itemsets posibles según el tamaño del catálogo. Con el catálogo del conjunto Groceries (169 productos) el número supera el de átomos estimados en la Tierra.

	Ítems (m)	Itemsets posibles (2^m − 1)	Solo pares C(m, 2)
0	4	15	6
1	10	1,023	45
2	20	1,048,575	190
3	50	1.13e+15	1225
4	169	7.48e+50	14196

Propiedad antimonótona. Para cualesquiera itemsets $X \subseteq Y$:

$$X \subseteq Y \implies \sigma(Y) \leq \sigma(X). \tag{43.2}$$

La demostración es la del ejemplo: si $Y \subseteq T_j$ y $X \subseteq Y$, entonces $X \subseteq T_j$; todo ticket que cuenta para Y también cuenta para X. La propiedad se usa en su forma contrapositiva[4]: **si X no es frecuente, ningún superconjunto de X puede serlo**. En la retícula, eso significa que basta un itemset infrecuente para descartar sin contar toda la región que cuelga debajo de él. La línea que separa los frecuentes de los infrecuentes se llama **frontera**.

Laboratorio: la retícula de itemsets

El laboratorio dibuja los 15 itemsets de la Tabla 43.2. Con el modo *Fuerza bruta* se cuentan todos; con el modo *Poda Apriori* solo se cuenta un itemset si todos sus subconjuntos con un ítem menos resultaron frecuentes. Haz clic en cualquier nodo para ver qué tickets lo contienen.

En la versión web, este laboratorio dibuja la retícula de los 15 itemsets de la Tabla 43.2 y permite variar el conteo mínimo. Valores de referencia con conteo mínimo 2: hay 6 itemsets frecuentes; la fuerza bruta cuenta 15 itemsets y la poda Apriori solo 8 (los 4 ítems, los 3 pares sin queso y $\{L, P, M\}$, que resulta infrecuente con conteo 1).

Guía del laboratorio.

1. Predice antes de mover: con conteo mínimo 2 y poda Apriori, ¿se llegará a contar el itemset $\{L, P, Q\}$? Haz clic en él y comprueba el motivo.
2. Comprueba los números de la sección anterior: el panel debe mostrar 6 itemsets frecuentes, y $\{L, P\}$ debe aparecer en T_1, T_2 y T_5.

[3] Formalmente, la inclusión $\subseteq$ es un **orden parcial** sobre los subconjuntos de $\mathcal{I}$: hay pares que no se pueden comparar, como {leche} y {pan}. Un orden parcial en el que cada par tiene un ínfimo (la intersección) y un supremo (la unión) se llama **retícula**. Su diagrama, con un nivel por tamaño, se conoce como diagrama de Hasse.

[4] La **contrapositiva** de «si p entonces q» es «si no q entonces no p», y es lógicamente equivalente. Aquí: «si Y es frecuente, sus subconjuntos lo son» equivale a «si un subconjunto no es frecuente, Y tampoco».

3. Baja el conteo mínimo a 1. ¿Por qué ahora la poda ahorra muy poco? Sube a 4: ¿qué queda en pie y cuántos itemsets se contaron?
4. Explica con tus palabras por qué $\{L, P, M\}$ sí se cuenta con conteo mínimo 2 aunque al final resulte infrecuente.

Implementación en Python

El mismo razonamiento, en código. Primero la **fuerza bruta**: enumerar los $2^m - 1$ itemsets y contar cada uno.

	itemset	k	conteo	soporte
0	{leche}	1	4	0.8
1	{pan}	1	4	0.8
2	{mantequilla}	1	3	0.6
4	{leche, pan}	2	3	0.6
5	{leche, mantequilla}	2	2	0.4
7	{pan, mantequilla}	2	2	0.4

Ahora la búsqueda **por niveles con poda**: en el nivel k solo se cuentan los candidatos cuyos subconjuntos de tamaño $k-1$ son todos frecuentes. Es el esqueleto del algoritmo Apriori, que se estudia en detalle más adelante.

El resultado coincide con el laboratorio: 6 itemsets frecuentes y 8 itemsets contados en lugar de 15. Con cuatro ítems el ahorro parece modesto, pero la poda elimina **regiones enteras** de la retícula; con cientos de productos es la diferencia entre un cálculo factible y uno imposible.

Lo que viene

Con la representación transaccional, el soporte y la antimonotonía ya están todas las piezas para el resto de la parte. El siguiente capítulo convierte itemsets frecuentes en **reglas** $A \Rightarrow B$ y estudia cómo medir su fuerza; después, el capítulo de **Apriori** formaliza la búsqueda por niveles que acabamos de esbozar (Agrawal y Srikant 1994; Tan et al. 2006).

Parte XXII.

Reglas de Asociación

44. Reglas de Asociación: Fundamentos, Interpretación y Selección

Introducción

En la mayoría de los contextos donde se recopilan datos transaccionales —ventas minoristas, registros médicos, historiales de navegación web, interacciones en plataformas digitales— existe una pregunta recurrente que antecede a cualquier modelo predictivo: *¿qué elementos tienden a aparecer juntos?* Esta pregunta, aparentemente simple, da lugar a uno de los problemas más estudiados en la minería de datos: el descubrimiento de reglas de asociación.

El problema fue formalizado por Agrawal y Srikant (1994) en el contexto del análisis de canastas de mercado (*market basket analysis*), donde cada transacción de un supermercado registra el conjunto de productos adquiridos por un cliente en una visita. La motivación original era directamente comercial: identificar productos que se compran juntos con frecuencia permite optimizar la disposición física de los artículos en tienda, diseñar estrategias de promoción cruzada y personalizar recomendaciones. El mismo formalismo, sin embargo, se ha aplicado desde entonces en dominios tan distintos como la bioinformática —para identificar genes que se co-expresan—, la epidemiología —para detectar síntomas que co-ocurren en pacientes—, y la ciberseguridad —para reconocer patrones de acceso que preceden a intrusiones (Han et al. 2011).

Lo que hace al problema pedagógicamente valioso no es su aplicabilidad, sino la tensión que revela entre dos tareas que parecen equivalentes pero no lo son: *generar* reglas es computacionalmente sencillo; *interpretar y seleccionar* reglas útiles requiere juicio estadístico genuino. Un analista que genere reglas sin comprensión crítica de las métricas que las respaldan puede llegar a conclusiones erróneas con aparente rigor cuantitativo. Este capítulo busca construir, de manera progresiva, los instrumentos conceptuales necesarios para evitar ese error.

La exposición está organizada de lo concreto a lo formal. Se comienza con definiciones motivadas por ejemplos del mundo real, se desarrolla el proceso de generación de reglas, y se dedica la mayor parte del capítulo al problema que resulta ser el más difícil: no cómo obtener reglas, sino cómo decidir cuáles valen la pena.

Definiciones Fundamentales

El Modelo de Datos: Transacciones e Ítems

El punto de partida es un conjunto de transacciones $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$, donde cada transacción T_i es un subconjunto de un universo finito de ítems $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$. En el contexto de un supermercado, $\mathcal{I}$ es el catálogo completo de productos y cada T_i es el conjunto de productos

comprados en una visita particular. Es importante notar que este modelo no registra cantidades ni precios: únicamente la presencia o ausencia de cada ítem en cada transacción (Tan et al. 2006).

Un *itemset* es cualquier subconjunto $X \subseteq \mathcal{I}$. Un itemset de cardinalidad k se denomina k-itemset. Por ejemplo, {leche, pan} es un 2-itemset, y {leche, pan, mantequilla} es un 3-itemset.

Soporte

La primera métrica fundamental es el soporte, que mide la prevalencia de un patrón en el dataset.

Definición. El *soporte* de un itemset X en $\mathcal{D}$ es la fracción de transacciones que contienen a X:

$$\text{supp}(X) = \frac{|\{T_i \in \mathcal{D} : X \subseteq T_i\}|}{|\mathcal{D}|} \tag{44.1}$$

En términos probabilísticos, $\text{supp}(X) = P(X)$, es decir, la probabilidad empírica de observar el itemset X en una transacción elegida al azar. Para una regla $A \Rightarrow B$, el soporte es $P(A \cup B)$: la probabilidad de que tanto el antecedente como el consecuente estén presentes en la misma transacción.

El soporte tiene una propiedad formal relevante conocida como la *propiedad antimonotona* o principio Apriori: si un itemset X tiene soporte menor que un umbral $\sigma_{\text{mín}}$, entonces cualquier superconjunto $Y \supset X$ también tendrá soporte menor que $\sigma_{\text{mín}}$. Esta propiedad, demostrada formalmente en Agrawal y Srikant (1994), es la base algorítmica que hace tractable la búsqueda de patrones frecuentes en datasets con miles de ítems.

Interpretación e intuición. El soporte responde a la pregunta: *¿qué tan común es este patrón en los datos?* Un soporte bajo indica que el patrón es raro, lo que puede deberse a una relación genuinamente infrecuente o simplemente a que ambos ítems son raros por separado. Un soporte alto es condición necesaria —pero no suficiente— para que una regla sea útil en la práctica.

Confianza

La confianza mide el poder predictivo de una regla: dado que ocurre el antecedente, ¿con qué frecuencia ocurre también el consecuente?

Definición. La *confianza* de la regla $A \Rightarrow B$ es la probabilidad condicional empírica de B dado A:

$$\text{conf}(A \Rightarrow B) = \frac{\text{supp}(A \cup B)}{\text{supp}(A)} = P(B \mid A) \tag{44.2}$$

Una confianza de 0.80 para la regla $A \Rightarrow B$ significa que, en el 80 % de las transacciones que contienen a A, también aparece B. Esta interpretación como probabilidad condicional hace que la confianza sea intuitiva: es la tasa de acierto de la regla como regla de predicción.

La limitación fundamental de la confianza. A pesar de su intuitividad, la confianza presenta una limitación crítica que con frecuencia pasa desapercibida: *ignora la frecuencia base del consecuente.*

Considérese la siguiente situación: en un supermercado donde el 65 % de todas las transacciones incluyen leche entera, cualquier regla cuyo consecuente sea leche entera tendrá automáticamente una confianza de al menos 65 %, independientemente de cuál sea el antecedente. Una confianza de 0.70 para la regla {pan} ⇒ {leche entera} no es impresionante si la leche entera aparece en el 65 % de las transacciones: el pan añade menos de 10 puntos porcentuales sobre la tasa base. La regla tiene poder predictivo mínimo.

Este problema —confundir alta confianza con relación fuerte— es el error más común en el análisis de reglas de asociación, y motiva la necesidad del lift como métrica complementaria.

Lift

El lift cuantifica cuánto se aleja la relación entre A y B de la independencia estadística.

Definición. El *lift* de la regla $A \Rightarrow B$ es el cociente entre la confianza observada y la confianza esperada bajo independencia:

$$\text{lift}(A \Rightarrow B) = \frac{\text{conf}(A \Rightarrow B)}{P(B)} = \frac{P(A \cup B)}{P(A) \cdot P(B)} \tag{44.3}$$

La interpretación probabilística es directa: si A y B fueran estadísticamente independientes, la probabilidad de su co-ocurrencia sería $P(A) \cdot P(B)$. El lift es precisamente el factor por el cual la co-ocurrencia observada supera (o cae por debajo de) esta expectativa bajo independencia (Tan et al. 2006).

- $\text{lift} = 1$: A y B son estadísticamente independientes. La presencia de A no aporta información sobre B.
- $\text{lift} > 1$: A y B co-ocurren más frecuentemente de lo esperado bajo independencia. La regla señala una asociación positiva.
- $\text{lift} < 1$: A y B co-ocurren menos frecuentemente de lo esperado. La presencia de A inhibe la presencia de B.

Una propiedad algebraica importante es que el lift es simétrico: $\text{lift}(A \Rightarrow B) = \text{lift}(B \Rightarrow A)$. Esto tiene implicaciones prácticas significativas que se discuten en la Sección 46.

Un Ejemplo Concreto

Para anclar las definiciones, considérese el siguiente dataset de cinco transacciones:

Transacción	Ítems
T_1	leche, pan, mantequilla
T_2	leche, pan
T_3	leche, mantequilla
T_4	pan, mantequilla
T_5	leche, pan, queso

Para la regla {pan} ⇒ {leche}:

$$\text{supp}(\{pan, leche\}) = \frac{3}{5} = 0.60$$

$$\text{conf}(\{pan\} \Rightarrow \{leche\}) = \frac{\text{supp}(\{pan, leche\})}{\text{supp}(\{pan\})} = \frac{3/5}{4/5} = 0.75$$

$$\text{lift}(\{pan\} \Rightarrow \{leche\}) = \frac{0.75}{P(\{leche\})} = \frac{0.75}{4/5} = 0.9375$$

El lift de 0.9375 — menor que 1 — revela que, a pesar de una confianza aparentemente alta de 0.75, la presencia de pan en realidad *reduce ligeramente* la probabilidad de encontrar leche en la misma transacción, en relación con la tasa base de leche (que es 0.80). La confianza del 75 % era engañosa porque la leche ya era muy frecuente.

Antes de seguir, comprobemos estos números con Python. Basta contar transacciones:

Toda la información de una regla con un antecedente y un consecuente cabe en una **tabla 2 × 2** que cruza «¿contiene A?» con «¿contiene B?». Para pan y leche: 3 tickets tienen ambos (T_1, T_2, T_5), 1 tiene pan sin leche (T_4), 1 tiene leche sin pan (T_3) y ninguno tiene ni lo uno ni lo otro. Soporte, confianza y lift son solo tres formas de leer esa tabla.

Laboratorio: las métricas de una regla en una tabla 2 × 2

El laboratorio dibuja la tabla 2 × 2 como un **mosaico**: el ancho de cada columna es la proporción de tickets con y sin A, y la parte azul de cada columna es la proporción que además contiene B. La confianza es la altura del azul en la columna de A; la línea discontinua es la frecuencia base $P(B)$. El lift compara ambas alturas.

En la versión web, este laboratorio permite variar los cuatro conteos de la tabla 2 × 2 de una regla $A \Rightarrow B$. Valores de referencia: con el ejemplo del texto (3, 1, 1, 0) se obtiene soporte 0.60, confianza 0.75, $P(B) = 0.80$ y lift 0.9375. Con el escenario de asociación positiva (6, 2, 1, 11): confianza 0.75, $P(B) = 0.35$ y lift 2.14. Con el de independencia (4, 4, 6, 6): confianza 0.50 igual a $P(B)$ y lift 1.

Guía del laboratorio.

1. Comprueba que el estado inicial reproduce los números calculados a mano: soporte 0.60, confianza 0.75 y lift 0.9375.
2. Elige *Consecuente omnipresente*. Predice antes de mirar: ¿la confianza es alta? ¿Y el lift? Explica la diferencia con el mosaico.
3. Elige *Independencia* y mueve solo «A y B» hacia arriba. ¿Qué pasa con el lift y con la línea de P(B)?
4. Con *Asociación positiva*, ¿por qué conf(A B) y conf(B A) son distintas si el lift es uno solo?

De Itemsets Frecuentes a Reglas

El descubrimiento de reglas de asociación se divide en dos etapas (Agrawal y Srikant 1994; Han et al. 2011):

1. **Descubrimiento de itemsets frecuentes:** identificar todos los itemsets $X \subseteq \mathcal{J}$ cuyo soporte supere un umbral mínimo $\sigma_{\text{mín}}$ fijado por el analista.
2. **Generación de reglas a partir de itemsets frecuentes:** para cada itemset frecuente X con al menos dos ítems, generar todas las reglas $A \Rightarrow B$ con $A \cup B = X$, $A \cap B = \emptyset$ y ambos no vacíos, y retener solo aquellas cuya confianza supere un umbral mínimo $\gamma_{\text{mín}}$.

La primera etapa es la costosa: el espacio de itemsets tiene $2^{|\mathcal{J}|}$ elementos. El capítulo siguiente desarrolla el **algoritmo Apriori**, que la resuelve nivel por nivel aprovechando la propiedad antimonótona, con un laboratorio paso a paso y un ejemplo práctico (El algoritmo Apriori). En lo que resta de este capítulo usamos su implementación en `mlxtend` con $\sigma_{\text{mín}} = 0.10$ sobre una muestra de 50 canastas para estudiar el problema que de verdad exige juicio: **qué reglas valen la pena**.

Dos umbrales gobiernan el resultado:

- $\sigma_{\text{mín}}$ **(soporte mínimo):** cuán frecuente debe ser un patrón. Un valor bajo produce más reglas y más ruido; uno alto puede eliminar patrones genuinos pero poco frecuentes.
- $\gamma_{\text{mín}}$ **(confianza mínima):** cuán predictiva debe ser una regla. Como se discute a continuación, este umbral no tiene interpretación estadística directa: su valor apropiado depende de la frecuencia del consecuente.

El Problema de las Demasiadas Reglas

La Explosión Combinatoria

Supóngase un dataset con 100 ítems distintos y $\sigma_{\text{mín}} = 0.01$. El número de posibles itemsets es $2^{100} - 1$, un número astronómico. Incluso con la poda del algoritmo Apriori, y con un soporte mínimo razonablemente restrictivo, es posible generar decenas de miles de reglas a partir de datasets de tamaño moderado. El dataset Groceries original, con 9835 transacciones y 169 ítems, puede generar entre varios cientos y varios miles de reglas según los parámetros elegidos (Han et al. 2011).

Este volumen de resultados crea un problema de segundo orden: la proliferación de reglas hace que la revisión manual sea inviable, y la selección automatizada requiere criterios que, si son mal elegidos, pueden introducir un sesgo sistemático.

Redundancia e Irrelevancia

Parte del problema es estructural. Dado un itemset frecuente $X = \{A, B, C\}$, el algoritmo genera todas las particiones posibles como reglas: $\{A\} \Rightarrow \{B, C\}$, $\{B\} \Rightarrow \{A, C\}$, $\{C\} \Rightarrow \{A, B\}$, $\{A, B\} \Rightarrow \{C\}$, $\{A, C\} \Rightarrow \{B\}$, $\{B, C\} \Rightarrow \{A\}$. Muchas de estas reglas comparten la misma información de co-ocurrencia y son, en cierta medida, redundantes entre sí. La elección del consecuente más adecuado depende del contexto de aplicación, no del algoritmo.

Una capa adicional de dificultad es la irrelevancia. Una regla puede ser formalmente válida —soporte y confianza por encima de los umbrales— y al mismo tiempo no añadir ninguna información accionable. Esto ocurre sistemáticamente cuando el consecuente es un ítem muy frecuente: la regla parece predictiva porque su confianza es alta, pero en realidad refleja la mera ubicuidad del consecuente, no una relación genuina con el antecedente.

Este es el problema central que toda metodología rigurosa de análisis de reglas de asociación debe abordar, y que las métricas de evaluación descritas en la siguiente sección intentan resolver.

Métricas de Evaluación: Interpretación Profunda

Soporte: Prevalencia, No Interés

El soporte, tal como fue definido en Sección 44, responde exclusivamente a la pregunta *¿qué tan frecuente es este patrón?* No dice nada sobre si el patrón es inesperado, interesante o accionable. Una regla con soporte muy alto puede ser trivial: si el 80 % de los clientes compra agua embotellada, la regla $\{X\} \Rightarrow \{\text{agua}\}$ tendrá soporte alto para prácticamente cualquier X, sin revelar nada útil.

El soporte sí tiene un papel importante como **umbral de confiabilidad estadística**: un patrón respaldado por muy pocas observaciones es inestable y puede desaparecer o cambiar sustancialmente al agregar nuevos datos. En un dataset de 50 transacciones, un soporte de 0.10 corresponde a apenas 5 observaciones — insuficiente para afirmaciones estadísticas robustas. Con datasets más grandes (del orden de miles de transacciones), los umbrales de soporte pueden y deben ser más exigentes.

Confianza: Predictividad Condicional con un Sesgo Estructural

La confianza mide $P(B \mid A)$: la probabilidad de que el consecuente esté presente dado el antecedente. Su interpretación como probabilidad condicional es clara y directamente útil en sistemas de recomendación: si el 80 % de los clientes que compran pan también compran mantequilla, esa confianza justifica directamente recomendar mantequilla a alguien que acaba de poner pan en su carrito.

Sin embargo, la confianza tiene un sesgo estructural bien documentado (Tan et al. 2006): es monótonamente creciente en $P(B)$. Para un antecedente fijo A:

$$\text{conf}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A)} \leq \frac{P(B)}{P(A)} \cdot P(A) = P(B) \cdot \frac{1}{P(A)}$$

La confianza depende de $P(B)$ de manera directa. Un ítem B muy frecuente producirá, ceteris paribus, reglas de alta confianza para cualquier antecedente A, simplemente porque B aparece en casi todas las transacciones. Esta dependencia hace que la confianza no sea comparable entre reglas con consecuentes de frecuencias muy distintas, lo que introduce una fuente de sesgo sistemática si se usa como único criterio de selección.

La consecuencia práctica es la siguiente: antes de declarar que una regla tiene "alta confianza", es necesario comparar esa confianza con $P(B)$. Solo si $\text{conf}(A \Rightarrow B) \gg P(B)$ puede afirmarse que la presencia de A aporta genuinamente a la predicción de B.

> **!** Importante
>
> **Umbral de confianza no-trivial.** Para que la regla $A \Rightarrow B$ tenga lift $> k$, se requiere:
>
> $$\text{conf}(A \Rightarrow B) > k \cdot P(B)$$
>
> En particular, para que la regla sea simplemente no-trivial (lift > 1), es necesario que la confianza supere la frecuencia base del consecuente: $\text{conf}(A \Rightarrow B) > P(B)$.

Lift: La Métrica Central de Interés

El lift, definido en Ecuación 47.4, corrige el sesgo de la confianza al normalizar por $P(B)$. Su interpretación como razón de verosimilitud hace que sea directamente comparable entre reglas con distintos consecuentes.

Desde el punto de vista de la probabilidad, el lift es el cociente entre la probabilidad conjunta observada y la probabilidad conjunta esperada bajo la hipótesis de independencia:

$$\text{lift}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A) \cdot P(B)}$$

Un lift significativamente mayor que 1 constituye evidencia en favor de una asociación genuina entre A y B. Un lift cercano a 1 indica que conocer A no modifica la probabilidad de B, lo que hace que la regla sea informacionalmente vacía (Tan et al. 2006).

La asimetría del lift. Como se mencionó al final de Sección 44, el lift es simétrico: $\text{lift}(A \Rightarrow B) = \text{lift}(B \Rightarrow A)$. Esto puede resultar sorprendente, pues la confianza sí es asimétrica. La razón es algebraica: el lift compara $P(A \cup B)$ con $P(A) \cdot P(B)$, y esta expresión trata a A y a B de manera equivalente.

La asimetría del lift tiene implicaciones prácticas: cuando se generan las reglas $A \Rightarrow B$ y $B \Rightarrow A$, ambas tienen exactamente el mismo lift pero pueden tener confianzas muy diferentes. La dirección que debe preferirse en una aplicación real depende del contexto: si el objetivo es recomendar B dado que se observó A, la confianza de $A \Rightarrow B$ es la métrica relevante; si el objetivo es recomendar A dado B, lo es $\text{conf}(B \Rightarrow A)$.

Limitaciones del lift. A pesar de su superioridad sobre la confianza para detectar asociaciones no triviales, el lift tiene dos limitaciones importantes:

1. *Inestabilidad con soporte bajo.* En datasets pequeños o con ítems raros, el lift puede inflarse artificialmente. Si A y B son ítems raros —$P(A) = P(B) = 0.01$— y co-ocurren una sola vez, su soporte es $P(A \cup B) = 0.01$ y su lift es $1/0.01 = 100$. Este valor es enorme pero está respaldado por una sola observación, lo que lo hace estadísticamente no significativo.

2. *Techo variable.* El valor máximo que puede alcanzar el lift no es universal: depende de los soportes individuales de A y B. El lift máximo teórico es $1/\max(P(A), P(B))$, lo que implica que no es válido comparar lifts entre reglas con distintos niveles de frecuencia sin esta corrección.

Estas limitaciones motivan el uso de métricas complementarias, que se introducen en la Sección 44.

Resumen Comparativo

La Tabla 44.1 sintetiza las propiedades de las tres métricas fundamentales.

Tabla 44.1.: Propiedades de las métricas fundamentales de reglas de asociación.

Métrica	Pregunta que responde	Escala	Limitación principal
Soporte	¿Qué tan común es el patrón?	$[0, 1]$	No mide interés
Confianza	¿Qué tan predecible es B dado A?	$[0, 1]$	Sesgada por $P(B)$
Lift	¿Es la co-ocurrencia mayor que el azar?	$(0, +\infty)$	Inestable con soporte bajo

Ninguna de las tres métricas es suficiente por sí sola. La práctica correcta consiste en utilizarlas en conjunto y privilegiar reglas que resulten sólidas según múltiples criterios.

Interpretación Visual del Espacio de Reglas

Representación en el Espacio Soporte-Confianza-Lift

El conjunto de reglas generadas por Apriori puede visualizarse como una nube de puntos en un espacio tridimensional definido por soporte, confianza y lift. Esta representación permite identificar de manera inmediata los distintos tipos de reglas —las útiles, las engañosas y las triviales— y orienta la selección de manera más informada que la simple inspección de tablas.

La forma más conveniente de reducir esta representación tridimensional a un gráfico plano es proyectar confianza y lift sobre los ejes cartesianos, y codificar el soporte mediante el tamaño de los marcadores. El uso de color para representar el lift añade redundancia visual que facilita la identificación de los cuadrantes de interés.

Figura 44.1.

Lectura del Gráfico: Cuatro Zonas

La Figura 44.1 define implícitamente cuatro zonas de interpretación:

Zona I — Cuadrante deseable (lift alto, confianza alta, marcador grande). Las reglas en esta zona son las candidatas más sólidas: tienen poder predictivo (confianza alta), representan una asociación genuina no explicada por las frecuencias base (lift > 1), y están respaldadas por una cantidad razonable de transacciones (soporte alto). Son los patrones más accionables.

Zona II — Alta confianza, lift bajo (debajo de la línea roja). Esta es la zona de mayor riesgo para el analista no crítico. Las reglas aquí pueden tener confianzas impresionantes —0.70, 0.80 o más— pero un lift cercano o inferior a 1. En la mayoría de los casos, esto se debe al problema

de la frecuencia base: el consecuente es tan ubícuo que aparece independientemente del antecedente. Confiar en estas reglas para tomar decisiones es, en el mejor de los casos, inútil y, en el peor, contraproducente.

Zona III — Lift alto, soporte bajo (marcadores pequeños por encima de la línea). Estas reglas señalan asociaciones potencialmente fuertes, pero están respaldadas por pocas observaciones. Su lift puede ser un artefacto estadístico: en un dataset de 50 transacciones, dos ítems que co-ocurren apenas 5 veces pueden mostrar un lift elevado por pura aleatoriedad. La regla puede ser genuina, pero requiere cautela y, de ser posible, validación con datos adicionales.

Zona IV — Bajo en todo. Reglas sin soporte, sin confianza relevante y con lift cercano a 1. No hay argumento para retenerlas.

Co-ocurrencia Bruta: La Matriz de Transacciones

Antes de interpretar las métricas formales, una visualización complementaria de utilidad es la matriz de co-ocurrencia entre los ítems más frecuentes. Esta matriz registra, para cada par de ítems, el número absoluto de transacciones en las que ambos aparecen simultáneamente.

Una observación importante sobre la Figura **??**: las celdas con mayor intensidad tienden a corresponder a pares que involucran los ítems más frecuentes —en este caso, *whole milk* y *bread*. Esto no es evidencia de una asociación especialmente fuerte entre ellos; es, en gran medida, consecuencia de sus altas frecuencias individuales. La co-ocurrencia bruta es el numerador de $P(A \cup B)$, pero no incorpora los denominadores $P(A)$ y $P(B)$ que son necesarios para evaluar si la relación es no trivial. Las celdas más informativas son aquellas con alta intensidad entre ítems de frecuencia moderada, como el clúster *granola-yogurt-fruit*.

Selección de Reglas Útiles

Criterios Prácticos de Filtrado

El proceso de selección de reglas útiles no es un procedimiento automatizable en su totalidad: requiere decisiones del analista que deben estar informadas por el contexto del problema. Sin embargo, es posible enunciar un conjunto de criterios generales que sirven como punto de partida (Tan et al. 2006; Han et al. 2011):

Criterio 1: Lift sustancialmente mayor que 1. Una regla cuyo lift no supera de manera apreciable el valor de referencia de la independencia (1.0) no revela ninguna asociación genuina entre el antecedente y el consecuente. Un umbral pragmático es lift ≥ 1.2 para relaciones débiles y lift ≥ 1.5 para relaciones moderadas. En datasets grandes y en aplicaciones donde el costo de una decisión incorrecta es alto, valores más exigentes (lift ≥ 2.0) son más apropiados.

Criterio 2: Soporte respaldado por suficientes observaciones. El soporte mínimo debe traducirse a un número absoluto de transacciones que resulte estadísticamente sensato. Con un dataset de 50 transacciones, soporte $= 0.10$ corresponde a apenas 5 observaciones — un valor muy inestable. Con un dataset de 9000 transacciones, ese mismo soporte corresponde a 900 observaciones, lo cual es perfectamente robusto.

Criterio 3: Confianza superior a la frecuencia base del consecuente. Como se derivó en Sección 44, la condición $\text{conf}(A \Rightarrow B) > P(B)$ es equivalente a lift > 1. Una buena práctica es calcular explícitamente el "umbral de confianza no-trivial" para cada posible consecuente antes de analizar las reglas, tal como ilustra la siguiente tabla:

Tabla 44.2.: Umbral mínimo de confianza necesario para que una regla sea no-trivial (lift > 1) o sustancialmente no-trivial (lift > 1.5), calculado a partir de la frecuencia base de cada ítem.

	Ítem	P(ítem)	conf mín para lift > 1.0	conf mín para lift > 1.5
13	whole milk	0.52	0.52	0.78
1	bread	0.40	0.40	0.60
7	eggs	0.28	0.28	0.42
14	yogurt	0.28	0.28	0.42
2	butter	0.26	0.26	0.39
8	fruit	0.20	0.20	0.30
12	vegetables	0.18	0.18	0.27
6	coffee	0.16	0.16	0.24
10	soda	0.16	0.16	0.24
4	cheese	0.14	0.14	0.21

Esta tabla es una herramienta diagnóstica directamente útil: antes de entusiasmarse con una regla cuyo consecuente es *whole milk*, conviene consultar que la confianza supera 0.58 (para lift > 1) o 0.87 (para lift > 1.5). Si la confianza observada no alcanza esos valores, la regla no aporta nada.

Métricas Complementarias

La literatura de minería de datos propone numerosas métricas alternativas al lift para evaluar el interés de una regla (Tan et al. 2006). Tres de las más utilizadas en la práctica son las siguientes.

Leverage (apalancamiento).

$$\text{lev}(A \Rightarrow B) = P(A \cup B) - P(A) \cdot P(B)$$

El leverage es la diferencia absoluta entre la co-ocurrencia observada y la esperada bajo independencia. Su interpretación es más directa que la del lift: mide cuántas transacciones *adicionales*, sobre el baseline de independencia, tienen tanto A como B. Un leverage de 0.05 en un dataset de 1000 transacciones significa que aproximadamente 50 transacciones tienen tanto A como B más allá de lo que esperaríamos si fueran independientes. El rango del leverage es $[-P(A)\cdot P(B), \text{mín}(P(A), P(B)) - P(A) \cdot P(B)]$.

Conviction.

$$\text{conv}(A \Rightarrow B) = \frac{1 - P(B)}{1 - \text{conf}(A \Rightarrow B)}$$

La conviction mide con qué frecuencia la regla $A \Rightarrow B$ está "equivocada" — es decir, con qué frecuencia A ocurre sin B — en relación con lo esperado bajo independencia. A diferencia del lift, la conviction es asimétrica: $\text{conv}(A \Rightarrow B) \neq \text{conv}(B \Rightarrow A)$ en general, lo que la hace más apropiada para evaluar reglas dirigidas. Cuando la confianza tiende a 1, la conviction tiende a infinito, lo cual puede interpretarse como que la regla es cuasi-determinista. Conviction = 1 indica independencia.

Zhang's metric.

$$Z(A,B) = \frac{P(A \cup B) - P(A) \cdot P(B)}{\text{máx}\{P(A \cup B)(1 - P(A)),\ P(A)(P(B) - P(A \cup B))\}}$$

La métrica de Zhang, propuesta para superar algunas limitaciones del lift, tiene rango $[-1, 1]$ y es simétrica. Su principal ventaja es que normaliza correctamente tanto en el caso de co-ocurrencia alta como en el de co-ocurrencia baja, siendo más robusta ante ítems muy frecuentes. Un valor de Zhang cercano a 0 indica independencia; valores positivos cercanos a 1 indican asociación fuerte.

Tabla 44.3.: Comparación de métricas para las 8 reglas con mayor lift. Las métricas adicionales —leverage, conviction y Zhang— permiten corroborar o cuestionar lo que el lift sugiere.

	regla	support	confidence	lift	leverage	conviction	zhang
15	soda → chips	0.14	0.875	6.250	0.118	6.880	1.000
14	chips → soda	0.14	1.000	6.250	0.118	inf	0.977
23	yogurt → granola	0.10	0.357	3.571	0.072	1.400	1.000
22	granola → yogurt	0.10	1.000	3.571	0.072	inf	0.800
13	cheese → eggs	0.10	0.714	2.551	0.061	2.520	0.707
12	eggs → cheese	0.10	0.357	2.551	0.061	1.338	0.844
20	fruit → yogurt	0.12	0.600	2.143	0.064	1.800	0.667
21	yogurt → fruit	0.12	0.429	2.143	0.064	1.400	0.741

Reglas Fuertes vs. Reglas Débiles: Un Contraste

Para ilustrar la diferencia entre una regla bien respaldada y una engañosa, considérense los dos extremos del espacio de reglas:

Las reglas del primer grupo tienen confianzas aparentemente razonables, pero sus lifts cercanos a 1 y sus valores de Zhang bajos revelan que no hay asociación genuina entre antecedente y consecuente. Las reglas del segundo grupo, aunque puedan tener confianzas más modestas en términos absolutos, son más informativas porque representan desviaciones reales del comportamiento esperado bajo independencia.

Un Procedimiento de Selección Sistemático

El siguiente procedimiento condensa los criterios anteriores en una función de evaluación que puede aplicarse a cualquier conjunto de reglas:

Este procedimiento no debe interpretarse como un algoritmo definitivo: los umbrales y los pesos son propuestas que deben ajustarse al dominio. Su valor pedagógico reside en hacer explícita la

multidimensionalidad del juicio sobre la calidad de una regla, en contraste con la práctica de usar una única métrica como criterio absoluto.

Limitaciones del Enfoque

El Problema de la Causalidad

Quizás la limitación más importante del análisis de reglas de asociación, y la más frecuentemente ignorada, es que las reglas son *descriptivas* y *correlacionales*, nunca *causales* (Pearl 2009). La regla $\{\text{pan}\} \Rightarrow \{\text{mantequilla}\}$ con lift alto nos dice que quienes compran pan tienden a comprar mantequilla con más frecuencia de la esperada. No nos dice que comprar pan *causa* la compra de mantequilla, ni que poner pan al lado de la mantequilla *causará* que más personas los compren juntos. La confusión entre correlación y causalidad en el análisis de datos ha sido extensamente documentada como una fuente de errores costosos en decisiones de negocio y de política (Pearl 2009).

La distinción es especialmente relevante en aplicaciones de alto impacto como medicina o epidemiología. Identificar que ciertos síntomas co-ocurren con cierta diagnosis no equivale a entender el mecanismo causal que los conecta. Para inferencias causales son necesarios diseños experimentales o marcos formales de causalidad como los grafos causales de Pearl (2009).

Ítems Dominantes y Sesgo por Frecuencia Base

Como se ha analizado en detalle a lo largo de este capítulo, la presencia de ítems con soporte muy alto introduce un sesgo estructural en el análisis. En el dataset Groceries, *whole milk* aparece en aproximadamente el 26 % de las 9835 transacciones originales —y en el 58 % de la muestra de 50 transacciones utilizada en los ejemplos computacionales. Este dominio hace que *whole milk* aparezca como consecuente en un gran número de reglas con alta confianza, la mayoría de las cuales no representa asociaciones genuinas.

Una estrategia práctica para mitigar este problema es eliminar o tratar de manera especial los ítems cuyo soporte supere un umbral alto (por ejemplo, 40–50 %) antes de ejecutar el análisis, o analizar subgrupos del dataset donde la distribución de ítems sea más homogénea.

Ausencia de Estructura Temporal

El modelo de datos de las reglas de asociación —conjuntos sin orden— es incapaz de capturar relaciones temporales. Si una secuencia de comportamiento es relevante (por ejemplo, los clientes compran primero un producto básico y luego un complemento en visitas posteriores), Apriori la ignorará. Para este tipo de análisis existen algoritmos de descubrimiento de patrones secuenciales, como GSP (*Generalized Sequential Patterns*) y PrefixSpan, que extienden el formalismo de Apriori a secuencias ordenadas de itemsets (Han et al. 2011).

Inestabilidad con Datasets Pequeños

Los ejemplos computacionales de este capítulo utilizan 50 transacciones, lo cual es suficiente para ilustrar los conceptos pero insuficiente para afirmaciones estadísticas robustas. Con datasets pequeños, el soporte es muy sensible a fluctuaciones individuales: agregar o eliminar una sola transacción puede cambiar el soporte de un itemset en 2 puntos porcentuales (1/50), lo que puede hacer que un patrón cruce el umbral mínimo en una dirección u otra. En la práctica, el análisis de reglas de asociación requiere datasets con al menos varios cientos de transacciones para ser confiable, y preferiblemente miles.

Sensibilidad a los Parámetros

Como se mostró en el capítulo de Apriori, el número y la naturaleza de las reglas obtenidas dependen fuertemente de los parámetros elegidos. Esta sensibilidad tiene dos consecuencias metodológicas: (1) los resultados no son objetivos en el sentido de ser independientes de las decisiones del analista, y (2) dos análisis del mismo dataset con parámetros distintos pueden producir conclusiones superficialmente contradictorias. La buena práctica consiste en explorar el espacio de parámetros sistemáticamente y reportar la robustez de los hallazgos ante variaciones razonables.

Resumen Conceptual

Este capítulo ha desarrollado los fundamentos del análisis de reglas de asociación con énfasis en la interpretación crítica. Los puntos clave que el lector debe retener son los siguientes:

Sobre las definiciones fundamentales:

- El soporte mide la prevalencia de un patrón; la confianza mide su poder predictivo condicional; el lift mide su desviación respecto a la independencia estadística.
- Ninguna de las tres métricas es suficiente por sí sola: soporte alto no implica interés, confianza alta no implica relación genuina, lift alto con soporte bajo puede ser ruido.

Sobre la generación de reglas:

- El proceso de generación es automático y produce grandes volúmenes de reglas; el proceso de selección requiere juicio estadístico y conocimiento del dominio.
- Los parámetros de Apriori ($\sigma_{\text{mín}}$, $\gamma_{\text{mín}}$) son decisiones del analista sin fundamento estadístico intrínseco; siempre deben reportarse junto con los resultados.

Sobre el problema central:

- Alta confianza no equivale a relación fuerte cuando el consecuente es frecuente. El lift corrige este sesgo.
- El umbral mínimo de confianza para que una regla sea no-trivial es $\text{conf}(A \Rightarrow B) > P(B)$.

Sobre la selección de reglas útiles:

- El scatter lift-vs-confianza con soporte codificado en tamaño es la visualización más informativa para explorar el espacio de reglas.

- Métricas complementarias como leverage, conviction y Zhang's metric permiten corroborar o cuestionar lo que el lift sugiere. Las reglas robustas son aquellas que resultan sólidas según múltiples criterios simultáneamente.
- La dirección de la regla importa para la toma de decisiones: el lift es simétrico pero la confianza no.

Sobre las limitaciones:

- Las reglas de asociación son herramientas descriptivas y correlacionales; no permiten inferir causalidad.
- El enfoque no captura estructura temporal; para secuencias ordenadas se requieren algoritmos especializados.
- Los resultados son inestables con datasets pequeños y sensibles a la elección de parámetros.

El objetivo final no es maximizar el número de reglas obtenidas, sino identificar el subconjunto pequeño de reglas que representan patrones genuinamente no triviales, respaldados por suficientes observaciones y consistentes con múltiples métricas de evaluación. Alcanzar este objetivo requiere, más que dominio algorítmico, una comprensión sólida de los fundamentos estadísticos que subyacen a cada métrica.

45. El Algoritmo Apriori: Paso a Paso y Ejemplo Práctico

En el capítulo introductorio vimos que el número de itemsets posibles crece como 2^m y que la propiedad antimonótona permite descartar regiones enteras de la retícula. En el capítulo de reglas definimos soporte, confianza y lift. **Apriori** (Agrawal y Srikant 1994) une ambas ideas en un procedimiento concreto: encuentra **todos** los itemsets frecuentes recorriendo la retícula nivel por nivel y, a partir de ellos, genera las reglas que superan una confianza mínima. Este capítulo lo desarrolla a mano, lo implementa desde cero y lo aplica a un conjunto de canastas de supermercado.

Intuición: buscar por niveles

Supón que quieres saber qué pares de productos son frecuentes. La estrategia ingenua es contar todos los pares posibles. Apriori razona antes de contar:

> Un par solo puede ser frecuente si **cada uno** de sus productos es frecuente. Un trío solo puede serlo si **cada uno** de sus tres pares lo es.

Así que primero cuenta productos sueltos, después solo los pares formados con productos frecuentes, después solo los tríos cuyos pares son todos frecuentes, y así sucesivamente. Cada nivel usa el resultado del anterior para decidir qué vale la pena contar. El algoritmo se detiene cuando un nivel no produce ningún itemset frecuente.

Desarrollo manual paso a paso

Usamos los cinco tickets de siempre, con **conteo mínimo 2** (soporte mínimo 0.40) y **confianza mínima 0.60**. Para que la unión sea reproducible, los ítems dentro de cada itemset se escriben en orden alfabético.

Tabla 45.1.: Base de transacciones del ejemplo, con los ítems en orden alfabético.

Ticket	Productos
T_1	leche, mantequilla, pan
T_2	leche, pan
T_3	leche, mantequilla
T_4	mantequilla, pan
T_5	leche, pan, queso

Nivel 1: contar ítems sueltos

Primer recorrido de los datos. Los candidatos C_1 son todos los ítems:

Candidato	Conteo	¿Frecuente?
{leche}	4	sí
{mantequilla}	3	sí
{pan}	4	sí
{queso}	1	no

$L_1 = \{\{\text{leche}\}, \{\text{mantequilla}\}, \{\text{pan}\}\}$. A partir de aquí **queso desaparece del análisis**: ningún itemset que lo contenga puede ser frecuente.

Nivel 2: unión, poda y conteo

Unión. Se combinan pares de itemsets de L_1. Con tres ítems frecuentes salen $\binom{3}{2} = 3$ candidatos: {leche, mantequilla}, {leche, pan} y {mantequilla, pan}. Los tres pares con queso nunca se generan.

Poda. Cada candidato se conserva solo si todos sus subconjuntos de tamaño 1 están en L_1. Aquí todos pasan.

Conteo. Segundo recorrido de los datos:

Candidato	Tickets	Conteo	¿Frecuente?
{leche, mantequilla}	T_1, T_3	2	sí
{leche, pan}	T_1, T_2, T_5	3	sí
{mantequilla, pan}	T_1, T_4	2	sí

L_2 contiene los tres pares.

Nivel 3: la unión exige un prefijo común

Para formar candidatos de tamaño 3 se unen **dos itemsets de L_2 que comparten su primer ítem**[1]: {leche, mantequilla} y {leche, pan} comparten «leche» y producen {leche, mantequilla, pan}. El par {mantequilla, pan} no tiene con quién combinarse.

Poda. Los tres subconjuntos de tamaño 2 de {leche, mantequilla, pan} son {leche, mantequilla}, {leche, pan} y {mantequilla, pan}; los tres están en L_2, así que el candidato sobrevive.

Conteo. Tercer recorrido: solo T_1 contiene los tres productos, conteo $1 < 2$. $L_3 = \emptyset$ y **el algoritmo termina**.

En total se contaron $4 + 3 + 1 = 8$ candidatos en tres recorridos de los datos, en lugar de los 15 itemsets posibles. Hay 6 itemsets frecuentes.

[1] Unir solo itemsets con el mismo prefijo de $k-2$ ítems evita generar el mismo candidato varias veces. Si se unieran todos los pares de L_{k-1}, {leche, mantequilla, pan} saldría de tres combinaciones distintas.

Generación de reglas

Cada itemset frecuente Z con al menos dos ítems produce reglas $A \Rightarrow Z \setminus A$. Su confianza **no requiere volver a los datos**: todos los soportes necesarios ya se contaron.

$$\text{conf}(A \Rightarrow Z \setminus A) = \frac{\sigma(Z)}{\sigma(A)}.$$

Tabla 45.2.: Reglas candidatas del ejemplo. El lift se calcula como confianza entre el soporte del consecuente.

Itemset	Regla	Confianza	¿ 0.60?	Lift
{leche, mantequilla}	leche $\Rightarrow$ mantequilla	$2/4 = 0.50$	no	0.833
{leche, mantequilla}	mantequilla $\Rightarrow$ leche	$2/3 = 0.67$	sí	0.833
{leche, pan}	leche $\Rightarrow$ pan	$3/4 = 0.75$	sí	0.938
{leche, pan}	pan $\Rightarrow$ leche	$3/4 = 0.75$	sí	0.938
{mantequilla, pan}	mantequilla $\Rightarrow$ pan	$2/3 = 0.67$	sí	0.833
{mantequilla, pan}	pan $\Rightarrow$ mantequilla	$2/4 = 0.50$	no	0.833

Cuatro reglas superan la confianza mínima. Pero observa la última columna: **todos los lifts son menores que 1**. En este ejemplo diminuto, cada par de productos aparece junto un poco *menos* de lo que esperaríamos por azar. Es el recordatorio del capítulo anterior: Apriori encuentra lo frecuente y lo confiable, no necesariamente lo interesante.

Formalización

Algoritmo Apriori (Agrawal y Srikant 1994).
Entrada: transacciones $\mathcal{D}$, conteo mínimo $s_{\text{mín}}$.

1. $L_1 \leftarrow \{\{i\} : \sigma(\{i\}) \geq s_{\text{mín}}\}$ (primer recorrido).
2. Para $k = 2, 3, \ldots$ mientras $L_{k-1} \neq \emptyset$:
   a. **Unión:** $C_k \leftarrow \{\, A \cup B : A, B \in L_{k-1},\ A$ y B coinciden en sus primeros $k-2$ ítems $\}$.
   b. **Poda:** eliminar de C_k todo candidato con algún $(k-1)$-subconjunto fuera de L_{k-1}.
   c. **Conteo:** un recorrido de $\mathcal{D}$ para obtener $\sigma(c)$ de cada $c \in C_k$.
   d. $L_k \leftarrow \{c \in C_k : \sigma(c) \geq s_{\text{mín}}\}$.
3. Devolver $\bigcup_k L_k$.

Corrección. Todo k-itemset frecuente Z se genera en la unión: si $Z = \{z_1 < \cdots < z_k\}$, sus subconjuntos $Z \setminus \{z_k\}$ y $Z \setminus \{z_{k-1}\}$ son frecuentes por antimonotonía, comparten los primeros $k-2$ ítems y su unión es Z. La poda solo descarta candidatos que la antimonotonía garantiza infrecuentes. Por tanto, Apriori es **completo**: no pierde ningún itemset frecuente.

Costo. Apriori hace un recorrido de los datos por nivel, es decir, $k_{\text{máx}} + 1$ recorridos si el itemset frecuente más largo tiene $k_{\text{máx}}$ ítems. El cuello de botella suele ser el nivel 2: con $|L_1|$ ítems frecuentes

hay $\binom{|L_1|}{2}$ candidatos, y con miles de productos eso son millones de pares. FP-Growth, que se estudia más adelante, ataca precisamente este costo.

Reglas. Para un itemset frecuente Z, mover ítems del antecedente al consecuente solo puede bajar la confianza: si $A' \subset A$, entonces $\sigma(A') \geq \sigma(A)$ y

$$\text{conf}(A' \Rightarrow Z \setminus A') = \frac{\sigma(Z)}{\sigma(A')} \leq \frac{\sigma(Z)}{\sigma(A)} = \text{conf}(A \Rightarrow Z \setminus A).$$

Por eso las reglas también se generan por niveles: si $\{a, b\} \Rightarrow \{c\}$ no alcanza la confianza mínima, tampoco la alcanzará $\{a\} \Rightarrow \{b, c\}$.

Laboratorio: Apriori nivel por nivel

El laboratorio ejecuta Apriori sobre la Tabla 45.1. Recorre los pasos con los botones: en cada nivel verás qué candidatos produjo la unión, cuáles eliminó la poda y cuáles resultaron frecuentes al contar. El último paso genera las reglas.

En la versión web, este laboratorio ejecuta Apriori nivel por nivel sobre la Tabla 45.1. Valores de referencia con conteo mínimo 2 y confianza mínima 0.60: tres recorridos de los datos, 8 candidatos contados, 6 itemsets frecuentes y 4 de 6 reglas aceptadas, todas con lift menor que 1. Con conteo mínimo 1 la unión produce tres candidatos de tamaño 3 y aparece la poda: {leche, mantequilla, queso} se elimina sin contarlo porque {mantequilla, queso} no es frecuente.

Guía del laboratorio.

1. Comprueba el estado inicial contra el desarrollo manual: en el nivel 2 deben aparecer tres candidatos frecuentes; en el nivel 3, un solo candidato con conteo 1.
2. Predice antes de mover: con conteo mínimo 1, ¿cuántos candidatos de tamaño 3 producirá la unión? ¿Alguno se podará? Comprueba en el nivel 3 cuál es el subconjunto que provoca la poda.
3. Con conteo mínimo 3, ¿cuántos recorridos de los datos se necesitan y cuántas reglas quedan?
4. En el paso «Reglas», sube la confianza mínima a 0.70. ¿Qué reglas se pierden? Explica por qué *leche* $\Rightarrow$ *pan* y *pan* $\Rightarrow$ *leche* tienen la misma confianza aquí.

Implementación desde cero en Python

La función `apriori_gen` implementa la unión con prefijo común y la poda; `apriori_desde_cero` alterna generación y conteo y guarda estadísticas por nivel.

	k	podados	contados	frecuentes
0	1	0	4	3
1	2	0	3	3
2	3	0	1	0

La generación de reglas usa solo los conteos ya calculados:

	regla	soporte	confianza	lift	aceptada
0	{'leche'} {'mantequilla'}	0.4	0.500	0.833	False
1	{'mantequilla'} {'leche'}	0.4	0.667	0.833	True
2	{'leche'} {'pan'}	0.6	0.750	0.938	True
3	{'pan'} {'leche'}	0.6	0.750	0.938	True
4	{'mantequilla'} {'pan'}	0.4	0.667	0.833	True
5	{'pan'} {'mantequilla'}	0.4	0.500	0.833	False

Los resultados coinciden con la Tabla 45.2 y con el laboratorio. En la práctica se usa una implementación optimizada como la de `mlxtend` (Raschka 2018), que trabaja sobre la matriz binaria:

	support	itemsets
0	0.8	(leche)
1	0.6	(mantequilla)
2	0.8	(pan)
3	0.4	(mantequilla, leche)
4	0.6	(pan, leche)
5	0.4	(mantequilla, pan)

Ejemplo práctico: canastas de supermercado

Pasamos a una muestra de 50 canastas con 15 productos, inspirada en el conjunto *Groceries* (la misma que usan los capítulos de reglas y de métricas). El objetivo de negocio es típico: encontrar combinaciones de productos para promociones cruzadas.

Paso 1: elegir el soporte mínimo en unidades de canastas

Un soporte mínimo de 0.10 significa al menos **5 canastas** de 50. Es el mínimo razonable para que un patrón no sea una coincidencia aislada; con datos reales de miles de tickets se puede bajar el soporte relativo manteniendo un conteo absoluto suficiente.

Paso 2: ejecutar Apriori y mirar el trabajo por nivel

Figura 45.1.

Apriori resolvió el problema contando menos de 100 itemsets de más de 32 000 posibles. Con `mlxtend` el resultado es el mismo:

	support	itemsets	k
11	0.52	(whole milk)	1
0	0.40	(bread)	1

	support	itemsets	k
12	0.28	(yogurt)	1
5	0.28	(eggs)	1
1	0.26	(butter)	1
6	0.20	(fruit)	1
10	0.18	(vegetables)	1
16	0.16	(bread, whole milk)	2

Paso 3: generar reglas y leerlas con lift

	regla	support	confidence	lift
6	chips soda	0.14	1.000	6.250
7	soda chips	0.14	0.875	6.250
11	granola yogurt	0.10	1.000	3.571
5	cheese eggs	0.10	0.714	2.551
10	fruit yogurt	0.12	0.600	2.143
9	yogurt fruit	0.12	0.429	2.143
2	butter eggs	0.12	0.462	1.648
3	eggs butter	0.12	0.429	1.648
0	coffee bread	0.10	0.625	1.562
8	coffee whole milk	0.10	0.625	1.202
12	yogurt whole milk	0.16	0.571	1.099
1	eggs bread	0.12	0.429	1.071
4	butter whole milk	0.14	0.538	1.036

La lectura de negocio sale del cuadrante superior: reglas como *chips* *soda* o *granola* *yogurt* combinan confianza alta con lift muy superior a 1. Son candidatas naturales para una promoción cruzada. Las reglas hacia *whole milk* son el caso contrario: frecuentes, aparentemente confiables y casi sin información.

Paso 4: la sensibilidad a los umbrales

Los umbrales de Apriori son decisiones del analista. Antes de reportar reglas conviene ver cuánto cambia el resultado al moverlos:

Número de reglas generadas (confianza mínima 0.3) y lift mediano en función del soporte mínimo. Al subir el soporte quedan menos reglas, pero las que sobreviven tienden a tener mayor lift.

Reportar reglas sin reportar los umbrales con los que se obtuvieron es una práctica deficiente: el conjunto de reglas es una función de los datos **y** de los parámetros. El capítulo de métricas avanzadas retoma este análisis con más detalle.

Ventajas y limitaciones

Tabla 45.3.: Resumen de Apriori.

Aspecto	Apriori
Idea central	Búsqueda por niveles con poda por antimonotonía
Recorridos de los datos	Uno por nivel ($k_{\text{máx}} + 1$)
Cuello de botella	Número de candidatos del nivel 2 con muchos ítems frecuentes
Memoria	Baja: solo candidatos y conteos del nivel actual
Fortaleza	Fácil de entender, implementar y explicar; completo y exacto
Debilidad	Lento en datos densos o con soporte mínimo bajo

Estas debilidades motivan **FP-Growth**, que comprime los datos en un árbol y evita generar candidatos. Antes de llegar a él, el siguiente capítulo profundiza en cómo evaluar las reglas que Apriori produce.

46. Más Allá del Lift: Métricas Avanzadas, Sensibilidad y Estructura de Reglas

Introducción

El capítulo anterior estableció los fundamentos del análisis de reglas de asociación: las definiciones de soporte, confianza y lift, el proceso de generación mediante el algoritmo Apriori, y los criterios básicos para distinguir reglas genuinas de reglas triviales. El argumento central fue que generar reglas es algorítmicamente sencillo, pero que la evaluación crítica de su validez exige ir más allá de la confianza como criterio único.

Este capítulo continúa ese argumento y profundiza en cuatro direcciones que el tratamiento introductorio deja abiertas. En primer lugar, se analizan con rigor las *limitaciones estructurales del lift* como métrica de evaluación: aunque el lift corrige el sesgo de la confianza, tiene sus propias deficiencias que la literatura especializada ha documentado con detalle, y que son frecuentemente ignoradas en presentaciones introductoras. En segundo lugar, se introduce un conjunto de *métricas complementarias* —leverage, conviction y la métrica de Zhang— cuyas propiedades matemáticas permiten evaluar aspectos de las reglas que el lift no captura, y cuya combinación produce evaluaciones más robustas. En tercer lugar, se aborda el problema de la *sensibilidad a parámetros*: se demuestra empíricamente que el número, la naturaleza y la calidad de las reglas obtenidas dependen de manera no trivial de los umbrales que el analista fija al invocar el algoritmo, lo cual tiene consecuencias metodológicas directas. Finalmente, se propone una representación del conjunto de reglas como *grafo de relaciones* entre ítems, que permite identificar la estructura global del espacio de reglas y distinguir patrones centrales de patrones periféricos.

El hilo conductor de todas estas secciones es el mismo que motivó el capítulo anterior: el objetivo no es maximizar el número de reglas reportadas, sino identificar el subconjunto pequeño de reglas que representan patrones no triviales, respaldados por evidencia consistente desde múltiples perspectivas.

Las Limitaciones Estructurales del Lift

La Simetría del Lift y la Asimetría de la Decisión

Una propiedad algebraica del lift que pocas presentaciones introductoras discuten en profundidad es su *simetría*: para cualquier par de itemsets A y B,

$$\text{lift}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A) \cdot P(B)} = \frac{P(A \cup B)}{P(B) \cdot P(A)} = \text{lift}(B \Rightarrow A) \tag{46.1}$$

La simetría se sigue directamente de la conmutatividad del producto en el denominador. El lift mide la desviación de la probabilidad conjunta $P(A \cup B)$ respecto al valor esperado bajo independencia $P(A) \cdot P(B)$, y esta desviación es la misma independientemente de cuál de los dos conjuntos se designe como antecedente y cuál como consecuente.

Esta simetría tiene una consecuencia práctica importante que con frecuencia pasa inadvertida. Cuando el algoritmo genera las dos reglas $A \Rightarrow B$ y $B \Rightarrow A$ a partir del mismo itemset frecuente $\{A, B\}$, ambas reglas tendrán exactamente el mismo lift. Sin embargo, sus confianzas serán, en general, distintas:

$$\text{conf}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A)}, \qquad \text{conf}(B \Rightarrow A) = \frac{P(A \cup B)}{P(B)}$$

La regla con confianza más alta es aquella cuyo antecedente tiene *menor* soporte individual. Si $P(A) < P(B)$, entonces $\text{conf}(A \Rightarrow B) > \text{conf}(B \Rightarrow A)$: predecir el ítem más frecuente a partir del menos frecuente es más "fácil" en el sentido de la confianza, precisamente porque el consecuente ocurre con mayor frecuencia de base.

La Tabla 46.1 ilustra este fenómeno con datos empíricos sobre el par yogurt — whole milk:

Tabla 46.1.: Las dos direcciones de la relación yogurt — whole milk tienen el mismo lift pero confianzas distintas. La dirección {yogurt} → {whole milk} tiene mayor confianza porque whole milk es el ítem más frecuente de los dos.

	regla	antecedent support	consequent support	support	confidence	lift
24	yogurt → whole milk	0.28	0.52	0.16	0.571	1.099
25	whole milk → yogurt	0.52	0.28	0.16	0.308	1.099

La implicación práctica es directa: cuando el objetivo es tomar una acción condicionada a la observación de un ítem —"¿qué recomiendo a alguien que compra X?"— la dirección de la regla importa, y el lift por sí solo no permite elegirla. La confianza, a pesar de sus limitaciones ya documentadas, es la métrica relevante para guiar esa elección. Un sistema de recomendación que ordene reglas exclusivamente por lift y tome la primera dirección que encuentre puede estar recomendando la dirección subóptima.

El Techo Teórico del Lift y la Comparabilidad entre Reglas

Una segunda limitación del lift, igualmente importante y menos conocida, es que su escala no es universal: el valor máximo que el lift puede alcanzar para un par dado depende de las frecuencias individuales de los ítems involucrados. Esta propiedad hace que comparar lifts entre reglas con distintos niveles de soporte individual sea, en sentido estricto, metodológicamente incorrecto.

Para derivar el techo teórico, obsérvese que el soporte de la co-ocurrencia $P(A \cup B)$ está acotado superiormente por el soporte del ítem menos común de los dos: si $P(A) \leq P(B)$, entonces $P(A \cup B) \leq P(A)$ (no pueden co-ocurrir más transacciones de las que contienen al ítem más raro). Por lo tanto:

$$\text{lift}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A) \cdot P(B)} \leq \frac{P(A)}{P(A) \cdot P(B)} = \frac{1}{P(B)} = \frac{1}{\text{máx}(P(A), P(B))} \tag{46.2}$$

El lift máximo teórico para el par (A, B) es:

$$\text{lift}_{\text{máx}}(A, B) = \frac{1}{\text{máx}(P(A), P(B))} \tag{46.3}$$

Este resultado tiene una consecuencia inmediata: dos ítems frecuentes —digamos, con soporte 0.50 cada uno— tienen un lift máximo teórico de apenas 2.0. Dos ítems raros —con soporte 0.05 cada uno— tienen un lift máximo teórico de 20. Un lift observado de 1.8 para el primer par representa el 90 % del máximo alcanzable; el mismo lift de 1.8 para el segundo par representa solo el 9 %. La misma cifra señala una regla casi óptima en un caso y una regla débil en el otro.

Para corregir esta asimetría de escala se puede construir el *lift relativo*, definido como la fracción del máximo teórico que alcanza el lift observado:

$$\text{lift}_{\text{rel}}(A \Rightarrow B) = \frac{\text{lift}(A \Rightarrow B)}{\text{lift}_{\text{máx}}(A, B)} = \text{lift}(A \Rightarrow B) \cdot \text{máx}(P(A), P(B)) \tag{46.4}$$

El lift relativo tiene rango $(0, 1]$ y es directamente comparable entre reglas con distintos niveles de frecuencia. Un lift relativo cercano a 1 indica que los ítems co-ocurren con la máxima frecuencia posible dado sus soportes individuales.

Tabla 46.2.: Lift observado, lift máximo teórico y lift relativo para las reglas con antecedentes simples. Las columnas lift y lift_max_teorico no son directamente comparables entre filas; el lift_relativo sí lo es.

	regla	lift	lift_max_teorico	lift_relativo
14	soda → chips	6.250	6.250	1.000
23	granola → yogurt	3.571	3.571	1.000
22	yogurt → granola	3.571	3.571	1.000
15	chips → soda	6.250	6.250	1.000
13	eggs → cheese	2.551	3.571	0.714
12	cheese → eggs	2.551	3.571	0.714
2	bread → coffee	1.562	2.500	0.625
3	coffee → bread	1.562	2.500	0.625
17	whole milk → coffee	1.202	1.923	0.625
16	coffee → whole milk	1.202	1.923	0.625

Figura 46.1.

La Figura 46.1 revela una estructura que la simple ordenación por lift absoluto oculta. Las reglas en la esquina superior izquierda —lift observado modesto, pero lift relativo alto— son asociaciones entre ítems poco frecuentes que co-ocurren con una fracción significativa de su máximo teórico posible: son reglas *específicas* y sólidas, aunque no frecuentes. Las reglas en la esquina inferior derecha —lift absoluto más alto, pero lift relativo bajo— involucran ítems frecuentes cuya co-ocurrencia, aunque superior al azar, está lejos de ser tan estrecha como el lift absoluto podría sugerir.

Métricas Complementarias al Lift

El catálogo de métricas propuestas en la literatura para evaluar el interés de una regla de asociación es extenso: Tan et al. (2006) enumeran y comparan sistemáticamente más de veinte medidas distintas. Ninguna de ellas es universalmente superior; cada una captura una faceta distinta de lo que intuitivamente llamamos "interés". Este apartado desarrolla en detalle tres métricas que, por sus propiedades matemáticas y su complementariedad con el lift, son especialmente útiles en la práctica.

Leverage: La Diferencia Absoluta sobre el Azar

El leverage de una regla $A \Rightarrow B$ mide la diferencia *absoluta* entre la frecuencia de co-ocurrencia observada y la esperada bajo la hipótesis de independencia estadística:

$$\text{lev}(A \Rightarrow B) = P(A \cup B) - P(A) \cdot P(B) \tag{46.5}$$

Esta definición es idéntica en estructura al numerador del lift, pero sin la normalización por $P(A) \cdot P(B)$. La consecuencia es que el leverage conserva las unidades de probabilidad, lo que le otorga una interpretación directamente operacional: un leverage de 0.05, en un dataset de 1000 transacciones, significa que aproximadamente 50 transacciones contienen tanto A como B más allá de lo esperado bajo independencia. Este es el número de transacciones adicionales atribuibles a la asociación.

El rango teórico del leverage es $[-\min(P(A), P(B)) \cdot (1 - \max(P(A), P(B))),\ \min(P(A), P(B)) \cdot (1 - \max(P(A), P(B)))]$, centrado en cero. Un leverage positivo indica asociación positiva; negativo, inhibición; cero, independencia —exactamente como el lift, pero medido en una escala absoluta en lugar de relativa.

La diferencia fundamental entre leverage y lift es de escala. El lift normaliza la desviación respecto a la independencia y produce un número adimensional comparable entre reglas con distintos consecuentes. El leverage da la magnitud absoluta de esa desviación, que es más interpretable cuando se quiere cuantificar el *impacto* de una regla: en un sistema de recomendación, el leverage es proporcional al número de transacciones adicionales que la regla puede generar si se actúa sobre ella. Sin embargo, esta misma dependencia de las frecuencias absolutas hace que el leverage sea sistemáticamente bajo para ítems raros, incluso cuando la relación entre ellos es muy fuerte en términos relativos —exactamente el problema opuesto al del lift absoluto.

Conviction: Asimetría y Cuasi-Determinismo

La conviction de una regla $A \Rightarrow B$ fue introducida con la motivación de crear una métrica dirigida —asimétrica, como la confianza— que al mismo tiempo refleje el alejamiento de la independencia estadística (Tan et al. 2006):

$$\text{conv}(A \Rightarrow B) = \frac{1 - P(B)}{1 - \text{conf}(A \Rightarrow B)} = \frac{P(A) \cdot (1 - P(B))}{P(A) - P(A \cup B)} \tag{46.6}$$

La interpretación de la conviction es menos intuitiva que la del lift o el leverage, pero merece un análisis cuidadoso. El numerador $1 - P(B)$ es la frecuencia esperada de los "errores" de la regla —las transacciones en las que B no ocurre— bajo la hipótesis de independencia: si A y B fueran independientes, una fracción $1 - P(B)$ de las transacciones que contienen A no contendría B. El denominador $1 - \text{conf}(A \Rightarrow B)$ es la frecuencia observada de esos mismos errores. El cociente mide, por tanto, cuántas veces *menos* errores comete la regla en comparación con lo esperado bajo independencia.

De esta definición se siguen varias propiedades notables:

- Cuando A y B son independientes, $\text{conf}(A \Rightarrow B) = P(B)$, y $\text{conviction} = (1 - P(B))/(1 - P(B)) = 1$. La conviction vale 1 si y solo si hay independencia —igual que el lift vale 1 en independencia.
- Cuando la confianza se aproxima a 1, el denominador se aproxima a 0, y la conviction tiende a infinito. Una regla con conviction muy alta es cuasi-determinista: la presencia de A predice B casi con certeza.
- Cuando la confianza es menor que $P(B)$ (es decir, cuando hay una asociación negativa), la conviction es menor que 1.
- A diferencia del lift, la conviction es asimétrica: $\text{conv}(A \Rightarrow B) \neq \text{conv}(B \Rightarrow A)$ en general, lo que la hace adecuada para evaluar la fuerza de una regla en su dirección específica.

Esta asimetría es particularmente valiosa en contextos donde la dirección de la regla tiene significado operacional: en sistemas de recomendación, en diagnóstico médico, o en cualquier aplicación donde se actúa sobre el antecedente para predecir el consecuente.

La Métrica de Zhang: Rango Acotado y Robustez

La métrica de Zhang, propuesta para superar algunas de las limitaciones del lift, está definida como (Tan et al. 2006):

$$Z(A, B) = \frac{P(A \cup B) - P(A) \cdot P(B)}{\max\{P(A \cup B) \cdot [1 - P(A)],\ P(A) \cdot [P(B) - P(A \cup B)]\}} \tag{46.7}$$

El numerador es el mismo que el del leverage: la diferencia entre co-ocurrencia observada e independencia. El denominador es una normalización que garantiza que $Z \in [-1, 1]$ para cualquier par de ítems, independientemente de sus frecuencias individuales. Esta propiedad —que el lift no tiene— hace a la métrica de Zhang directamente comparable entre reglas con distintos niveles de frecuencia.

La normalización en el denominador se elige como el máximo entre dos términos: $P(A \cup B) \cdot [1 - P(A)]$, que domina cuando la co-ocurrencia es alta (asociación positiva fuerte), y $P(A) \cdot [P(B) - P(A \cup B)]$, que domina cuando la co-ocurrencia es baja (asociación negativa o inhibición). Este diseño garantiza que la escala sea simétrica: $Z = 1$ corresponde a co-ocurrencia máxima (A y B co-ocurren siempre que ocurre el más raro de los dos), y $Z = -1$ corresponde a co-ocurrencia mínima (exclusión mutua).

La métrica de Zhang es simétrica —al igual que el lift— y especialmente robusta ante ítems muy frecuentes, que son precisamente los que generan el sesgo más severo en la confianza y el leverage. Por estas razones, Tan et al. (2006) la recomiendan como una de las métricas más fiables para ordenar reglas por interés general cuando no hay información a priori sobre el dominio.

Cálculo y Comparación de las Cuatro Métricas

Tabla 46.3.: Las seis métricas de evaluación para las ocho reglas con mayor valor de Zhang. Observar cómo el ranking por Zhang difiere del ranking por lift: las reglas que Zhang considera más interesantes no siempre son las de mayor lift absoluto.

	regla	support	confidence	lift	leverage	conviction	zhang
22	yogurt → granola	0.10	0.357	3.571	0.072	1.400	1.000
14	soda → chips	0.14	0.875	6.250	0.118	6.880	1.000
15	chips → soda	0.14	1.000	6.250	0.118	inf	0.977
13	eggs → cheese	0.10	0.357	2.551	0.061	1.338	0.844
23	granola → yogurt	0.10	1.000	3.571	0.072	inf	0.800
20	yogurt → fruit	0.12	0.429	2.143	0.064	1.400	0.741
12	cheese → eggs	0.10	0.714	2.551	0.061	2.520	0.707
21	fruit → yogurt	0.12	0.600	2.143	0.064	1.800	0.667

Un cálculo completo a mano

Antes de comparar métricas en todo el conjunto de reglas, calculemos las seis para una sola regla, *yogurt* $\Rightarrow$ *whole milk*. En las 50 canastas, yogurt aparece en 14, whole milk en 26 y ambos en 8:

$$P(A) = \tfrac{14}{50} = 0.28, \qquad P(B) = \tfrac{26}{50} = 0.52, \qquad P(A \cup B) = \tfrac{8}{50} = 0.16.$$

1. **Confianza:** $0.16/0.28 = 0.571$.
2. **Lift:** $0.16/(0.28 \times 0.52) = 0.16/0.1456 = 1.099$.
3. **Lift máximo y relativo:** $1/\max(0.28, 0.52) = 1.923$; lift relativo $= 1.099/1.923 = 0.571$.
4. **Leverage:** $0.16 - 0.1456 = 0.0144$, es decir, unas $0.0144 \times 50 \approx 0.7$ canastas más de las esperadas por azar.
5. **Conviction:** $(1 - 0.52)/(1 - 0.571) = 0.48/0.429 = 1.12$.
6. **Zhang:** el numerador es el leverage, 0.0144; el denominador es $\max\{0.16 \times 0.72,\ 0.28 \times (0.52 - 0.16)\} = \max\{0.1152,\ 0.1008\} = 0.1152$; resultado 0.125.

Todas las métricas coinciden en el diagnóstico: asociación positiva, pero débil. El lift de 1.1 parece modesto y lo es; el lift relativo dice que el par alcanza poco más de la mitad de la coocurrencia máxima posible, y Zhang, en su escala de -1 a 1, apenas se separa de cero.

Laboratorio: el techo del lift y las métricas que lo corrigen

El laboratorio fija las frecuencias de A y B y deja mover su coocurrencia $P(A \cup B)$ entre sus límites posibles[1]. Las curvas muestran cómo responde cada métrica. Empieza con el par del cálculo a mano y después compara los escenarios de ítems frecuentes y raros: ambos tienen **el mismo lift de 1.8**.

En la versión web, este laboratorio permite mover la coocurrencia de un par de ítems con frecuencias fijas. Valores de referencia: para *yogurt* $\Rightarrow$ *whole milk* ($P(A) = 0.28$, $P(B) = 0.52$, $P(A \cup B) = 0.16$)

[1] Los límites $\max(0,\ P(A) + P(B) - 1) \le P(A \cup B) \le \min(P(A),\ P(B))$ se conocen como **cotas de Fréchet**. La superior dice que no pueden coincidir más canastas que las del ítem más raro; la inferior, que si dos ítems son muy frecuentes, forzosamente se solapan.

se obtiene lift 1.099, lift relativo 0.571, leverage 0.0144, conviction 1.12 y Zhang 0.125. Con ítems frecuentes (0.5, 0.5, 0.45) el lift es 1.8 con lift relativo 0.90, conviction 5 y Zhang 0.89; con ítems raros (0.05, 0.05, 0.0045) el lift es también 1.8, pero el lift relativo es 0.09, la conviction 1.04 y Zhang 0.47.

Guía del laboratorio.

1. Comprueba que el estado inicial reproduce las seis métricas del cálculo a mano.
2. Cambia a *Ítems frecuentes* y luego a *Ítems raros*. Predice antes: si el lift es el mismo, ¿qué métricas cambiarán más? Comprueba con los indicadores.
3. En cualquier escenario, lleva la coocurrencia al máximo. ¿Qué valor alcanzan el lift relativo y Zhang? ¿Qué pasa con la conviction y por qué?
4. Lleva la coocurrencia al valor de independencia $P(A)P(B)$. ¿Qué valor toma cada métrica?

La Correlación entre Métricas: Dónde Coinciden y Dónde Discrepan

Una pregunta natural ante la multiplicidad de métricas es si son redundantes: si lift, leverage y Zhang siempre coinciden en ordenar las reglas, da igual cuál se use. El análisis de correlación entre métricas responde esta pregunta de manera empírica.

La Figura **??** revela varias relaciones estructurales de interés. En primer lugar, lift y Zhang muestran correlación positiva alta: ambas métricas miden esencialmente la misma desviación de la independencia, la diferencia es de escala y de comportamiento en los extremos. En segundo lugar, leverage y support muestran correlación positiva moderada: ítems más frecuentes producen leverages más altos en términos absolutos, reflejando la dependencia del leverage en las frecuencias base. En tercer lugar, conviction puede tener correlación baja con las demás métricas en ciertos datasets, precisamente porque es sensible a valores de confianza cercanos a 1 —donde una pequeña variación en confianza produce cambios muy grandes en conviction— mientras que el resto de métricas varía suavemente.

Las zonas de correlación baja o negativa en esta matriz son las más informativas desde el punto de vista analítico: donde dos métricas discrepan, el analista necesita decidir a cuál le concede mayor peso según el contexto del problema. Una regla con lift alto pero leverage bajo involucra ítems frecuentes cuya relación relativa es fuerte pero cuyo impacto absoluto es limitado. Una regla con Zhang alto pero conviction moderada puede ser robusta en términos de asociación general pero no cuasi-determinista en su dirección específica.

El Perfil Multimétrico de una Regla

Una manera de sintetizar visualmente el comportamiento de una regla en todas las métricas simultáneamente es el gráfico de radar normalizado. Para cada regla, se normalizan las métricas al rango $[0, 1]$ mediante una transformación min-max —exclusivamente con fines de visualización, sin implicar que todas las métricas sean igualmente importantes— y se traza el polígono resultante.

La Figura **??** permite comparar simultáneamente cinco reglas en cinco dimensiones. Una regla cuyo polígono es grande y aproximadamente regular es robusta: todas las métricas coinciden en señalarla como interesante. Una regla con un polígono asimétrico —grande en un eje y pequeño en los demás— es una regla que parece buena según un criterio particular pero débil según los demás, lo que debe interpretarse con cautela.

La Tabla 46.4 resume la interpretación cualitativa de los patrones de perfil más frecuentes:

Tabla 46.4.: Interpretación de perfiles multimétricos.

Patrón del perfil	Interpretación
Polígono grande y regular	Regla robusta: coincidencia entre múltiples métricas
Lift alto, leverage pequeño	Asociación relativa fuerte entre ítems raros; impacto absoluto limitado
Conviction muy grande, resto moderado	Regla cuasi-determinista en su dirección; confianza cercana a 1
Zhang pequeño, lift moderado	El lift puede estar inflado por las frecuencias base; Zhang lo corrige
Support pequeño, todo lo demás pequeño	Patrón raro posiblemente espúrio; insuficiente evidencia empírica

Sensibilidad a los Parámetros del Algoritmo

Los Umbrales como Decisiones sin Fundamento Estadístico Intrínseco

Cuando se invoca el algoritmo Apriori con parámetros específicos —por ejemplo, `min_support = 0.10` y un umbral mínimo de lift de 1.0— se están tomando decisiones que, a diferencia de los parámetros en modelos estadísticos formales, no tienen una justificación estadística intrínseca. No existe una teoría que establezca que el soporte mínimo "correcto" es 0.10 en lugar de 0.08 o 0.12. El analista elige estos umbrales en función de consideraciones prácticas —tiempo de cómputo, número manejable de resultados, conocimiento del dominio— pero el algoritmo los acepta sin cuestionarlos.

Esta circunstancia tiene una consecuencia metodológica seria: el conjunto de reglas obtenido no es una propiedad objetiva del dataset, sino una función conjunta del dataset y de los parámetros elegidos. Dos analistas que trabajan con el mismo dataset pero con umbrales ligeramente distintos pueden obtener conjuntos de reglas sustancialmente diferentes y, si no son conscientes de esta dependencia, pueden llegar a conclusiones que parecen contradictorias. La buena práctica metodológica exige, como mínimo, explorar la estabilidad de los resultados ante variaciones razonables en los parámetros —lo que se denomina *análisis de sensibilidad*— y reportar esos parámetros junto con los resultados.

Sensibilidad al Soporte Mínimo

El primer experimento de sensibilidad consiste en variar sistemáticamente $\sigma_{\text{mín}}$ mientras se mantiene fijo un umbral de lift de 1.0, y registrar cómo cambia el número de reglas obtenidas y la calidad mediana del conjunto resultante.

Tabla 46.5.: Número de itemsets frecuentes, número de reglas generadas, y estadísticos de calidad del conjunto de reglas en función del soporte mínimo. El umbral de lift se mantiene fijo en 1.0.

	_min	Itemsets frecuentes	Reglas	Lift mediano	Lift máximo
0	0.06	37	36	1.896	6.250
1	0.08	28	22	1.648	6.250
2	0.10	26	20	1.605	6.250
3	0.12	18	12	1.374	6.250
4	0.14	15	6	1.099	6.250
5	0.16	11	2	1.099	1.099
6	0.18	7	0	NaN	NaN
7	0.20	6	0	NaN	NaN

El comportamiento mostrado en la Figura **??** no es accidental ni específico de este dataset: es una propiedad estructural del proceso de generación de reglas. Al elevar $\sigma_{\text{mín}}$, se excluyen los itemsets menos frecuentes. Muchos de estos itemsets involucran ítems raros que co-ocurren por casualidad en el dataset disponible —son, precisamente, el tipo de patrón más vulnerable a la inestabilidad estadística. Al eliminarlos, el conjunto de reglas restante tiene, en promedio, un soporte mayor y un lift mediano más alto. En términos informales: los patrones que sobreviven filtros más estrictos son, en promedio, más robustos.

La tensión entre ambas curvas refleja el *compromiso fundamental del análisis de reglas de asociación*: no es posible maximizar simultáneamente el número de reglas y su calidad promedio. Todo umbral de soporte es una elección en este espacio de trade-off, y la elección correcta depende del costo relativo de los dos tipos de error: incluir reglas espurias (falsos positivos) versus omitir patrones genuinos pero poco frecuentes (falsos negativos).

Sensibilidad al Umbral de Lift

El segundo experimento mantiene el soporte fijo en 0.10 y varía el umbral mínimo de lift sobre un rango amplio. El resultado muestra la curva de supervivencia del conjunto de reglas: para cada umbral, cuántas reglas lo superan.

La forma de la curva en la Figura **??** contiene información sobre la distribución de lifts en el dataset. Una caída pronunciada en el intervalo $[1.0, 1.3]$ indica que muchas reglas tienen lifts apenas superiores a la independencia —son reglas formalmente no-triviales pero de interés marginal. Una caída más gradual sugiere que los lifts están distribuidos de manera más uniforme sobre un rango amplio. En ambos casos, la curva permite identificar el umbral de lift a partir del cual el conjunto de reglas se estabiliza en un tamaño manejable sin perder los patrones más sólidos.

El Mapa Completo del Espacio de Parámetros

El análisis de sensibilidad más completo consiste en explorar simultáneamente los dos parámetros principales: soporte mínimo y confianza mínima. El resultado es una superficie bidimensional que muestra el número de reglas generadas para cada combinación de valores.

La Figura **??** es el resumen más completo del espacio de decisión del analista. Cada celda responde a la pregunta: *si elijo estos umbrales, ¿cuántas reglas obtengo?* La zona de alta densidad (esquina inferior izquierda, color rojo intenso) corresponde a umbrales laxos y produce el mayor número de reglas —pero también el mayor número de reglas espurias. La zona de baja densidad (esquina superior derecha, amarillo pálido o blanco) corresponde a umbrales exigentes y produce pocas reglas, pero cada una de ellas está respaldada por alta frecuencia y alto poder predictivo.

La implicación metodológica más importante de este análisis es la siguiente: no existe un punto en este mapa que sea correcto en abstracto. La elección de umbrales es siempre relativa al contexto del problema. En una aplicación de retail con miles de transacciones diarias, un soporte mínimo de 0.01 puede estar justificado porque incluso el 1 % de transacciones representa cientos de observaciones. En un dataset médico con 200 pacientes, el mismo umbral correspondería a apenas 2 casos, lo cual es estadísticamente insuficiente para cualquier afirmación. La elección correcta requiere traducir el umbral de soporte a un número absoluto de observaciones y preguntarse si ese número es suficiente para respaldar la afirmación que se quiere hacer.

La Estructura Global del Espacio de Reglas: Representación como Grafo

Motivación para una Representación en Red

Los análisis de los capítulos anteriores han representado el conjunto de reglas como tablas y como nubes de puntos en el espacio de métricas. Ambas representaciones son útiles para evaluar reglas individuales o comparar pequeños subconjuntos, pero pierden información sobre la *estructura de relaciones* que existe entre los ítems a nivel global. Un ítem puede participar en muchas reglas, algunas como antecedente y otras como consecuente; dos ítems pueden estar conectados directamente mediante una regla, o indirectamente a través de un ítem intermediario. Esta estructura topológica es invisible en una tabla.

La representación en grafo hace explícita esta estructura. Formalmente, se construye un grafo $G = (V, E)$ donde el conjunto de vértices V es el conjunto de ítems únicos que participan en al menos una regla (con los filtros de calidad aplicados), y el conjunto de aristas E contiene una arista (A, B) por cada regla $A \Rightarrow B$ que supera los umbrales mínimos de lift y soporte. Para evitar la proliferación de aristas y mantener la legibilidad, se consideran solo reglas con antecedentes y consecuentes simples (1-itemsets).

Centralidad, Hubs y Nodos Puente

En la teoría de grafos, la *centralidad* de un nodo mide su importancia relativa en la red. Existen múltiples definiciones de centralidad; en el contexto de las reglas de asociación, una medida natural es la suma de los lifts de todas las reglas en las que el ítem participa, como antecedente o como consecuente. Este criterio prioriza los ítems que están involucrados en muchas asociaciones *fuertes*, no simplemente en muchas asociaciones.

La Figura **??** hace visible la topología del espacio de reglas. Los ítems con mayor centralidad son aquellos que participan en múltiples asociaciones fuertes, no simplemente en muchas asociaciones. Un ítem puede ser *hub* por dos razones distintas: porque es genuinamente central en la red de co-compras —sus combinaciones con otros ítems son sorprendentemente frecuentes dado lo que sus frecuencias

individuales predecirían— o porque es tan frecuente que aparece en casi todas las transacciones y, por tanto, co-ocurre con casi todo, aunque con lifts bajos.

Esta distinción entre centralidad genuina y centralidad espúrea por frecuencia es directamente análoga al problema de la confianza: igual que una confianza alta puede ser un artefacto de la frecuencia base del consecuente, una alta centralidad en el grafo puede ser un artefacto de la alta frecuencia del ítem. El uso del lift —en lugar del conteo de co-ocurrencias— como peso de la centralidad es precisamente el mecanismo que intenta corregir este sesgo.

Un concepto igualmente importante es el de *nodo puente* o *broker*: un ítem con pocos vecinos en el grafo, pero cuyas aristas conectan grupos de ítems que de otro modo estarían desconectados. En el contexto del análisis de canastas de mercado, un nodo puente puede corresponder a un ítem de nicho que funciona como "gatillo" para una cadena de compras específica. Estos nodos son invisibles en un análisis basado exclusivamente en frecuencias, pero pueden ser muy accionables desde el punto de vista del marketing.

El "Whole Milk Problem": Un Caso de Estudio

El análisis del ítem *whole milk* en el dataset Groceries ilustra de manera concreta los problemas discutidos en las secciones anteriores. Con un soporte de aproximadamente 0.58 en la muestra utilizada —es decir, presente en más de la mitad de todas las transacciones— *whole milk* es un ítem dominante cuya alta frecuencia distorsiona sistemáticamente las métricas de todas las reglas en las que participa como consecuente.

El análisis confirma una conclusión que la teoría predice: cuando $P(B)$ es muy alto, el espacio para que una regla $A \Rightarrow B$ sea genuinamente no-trivial se estrecha dramáticamente. Para que el lift sea mayor que 1.5, se requiere $\text{conf}(A \Rightarrow B) > 1.5 \times 0.58 \approx 0.87$: prácticamente 9 de cada 10 transacciones que contienen A también contienen leche entera. Esta condición es casi imposible de cumplir en un supermercado general donde la leche ya es omnipresente. La conclusión no es que el dataset sea deficiente, sino que *whole milk* es un mal consecuente para el análisis de reglas de asociación: su ubicuidad impide que ninguna regla hacia él sea informativa.

La generalización es inmediata: para cualquier ítem B, el umbral de confianza necesario para que una regla sea no-trivial con factor k es:

$$\text{conf}(A \Rightarrow B) > k \cdot P(B) \qquad \text{equivalentemente,} \qquad \text{lift}(A \Rightarrow B) > k \tag{46.8}$$

Esta relación permite construir, de manera sistemática, una tabla de umbrales específicos para cada posible consecuente, que el analista puede consultar antes de interpretar cualquier regla.

Tabla 46.6.: Umbrales mínimos de confianza necesarios para que una regla sea no-trivial (lift > 1.0) o moderadamente interesante (lift > 1.5), calculados para cada ítem como posible consecuente. Ítems con P(ítem) alto requieren confianzas muy elevadas para generar reglas genuinamente informativas.

	Ítem (consecuente)	P(ítem)	conf mín (lift > 1.0)	conf mín (lift > 1.5)
13	whole milk	0.52	0.52	0.78
1	bread	0.40	0.40	0.60

Tabla 46.6.: Umbrales mínimos de confianza necesarios para que una regla sea no-trivial (lift > 1.0) o moderadamente interesante (lift > 1.5), calculados para cada ítem como posible consecuente. Ítems con P(ítem) alto requieren confianzas muy elevadas para generar reglas genuinamente informativas.

	Ítem (consecuente)	P(ítem)	conf mín (lift > 1.0)	conf mín (lift > 1.5)
7	eggs	0.28	0.28	0.42
14	yogurt	0.28	0.28	0.42
2	butter	0.26	0.26	0.39
8	fruit	0.20	0.20	0.30
12	vegetables	0.18	0.18	0.27
6	coffee	0.16	0.16	0.24
10	soda	0.16	0.16	0.24
4	cheese	0.14	0.14	0.21
5	chips	0.14	0.14	0.21
9	granola	0.10	0.10	0.15
11	sugar	0.10	0.10	0.15
0	beer	0.04	0.04	0.06
3	candy	0.04	0.04	0.06

Un Marco Integrado de Evaluación

De la Fragmentación a la Coherencia

Los conceptos desarrollados en este capítulo —limitaciones del lift, métricas complementarias, sensibilidad a parámetros, estructura de grafo— no son independientes entre sí. Forman un marco coherente para responder una única pregunta: *¿cuánta confianza merece una regla particular?*

La respuesta a esa pregunta tiene múltiples dimensiones:

1. **Dimensión relativa**: ¿cuánto se aleja el lift observado de la independencia, en términos del máximo alcanzable dado los soportes de los ítems? El lift relativo responde esta pregunta.
2. **Dimensión absoluta**: ¿cuántas transacciones adicionales, sobre el baseline de independencia, exhiben la co-ocurrencia? El leverage responde esta pregunta.
3. **Dimensión predictiva direccional**: ¿cuán determinista es la regla en su dirección específica? La conviction responde esta pregunta.
4. **Dimensión de robustez normalizada**: ¿es la asociación fuerte en una escala comparable entre ítems de frecuencias distintas? Zhang responde esta pregunta.
5. **Dimensión de estabilidad paramétrica**: ¿la regla sobrevive con cualquier elección razonable de umbrales, o desaparece si se endurece ligeramente $\sigma_{\text{mín}}$? El análisis de sensibilidad responde esta pregunta.

Una regla que es sólida en todas estas dimensiones merece alta confianza. Una regla que solo lo es en una o dos puede ser válida pero debe interpretarse con mayor cautela. La siguiente función implementa esta evaluación multidimensional de manera sistemática:

Lo que Cambia con Este Marco

La diferencia entre el análisis de reglas de asociación tratado en los textos introductorios y el análisis desarrollado en estos dos capítulos no reside en el código: las funciones de `mlxtend` son las mismas. La diferencia reside en las preguntas que el analista se hace antes de interpretar los resultados y en los instrumentos conceptuales que usa para responderlas.

La Tabla 72.5 resume esta diferencia de manera esquemática:

Tabla 46.7.: Comparación entre enfoque elemental y enfoque crítico del análisis de reglas de asociación.

Dimensión	Enfoque elemental	Enfoque crítico
Métricas consideradas	Soporte, confianza	Lift, lift relativo, leverage, conviction, Zhang
Dirección de la regla	Ignorada	Evaluada mediante la asimetría de confianza
Comparabilidad del lift	Asumida universal	Corregida con lift relativo
Umbrales de los parámetros	Fijos por convención	Explorados mediante análisis de sensibilidad
Visualización	Tablas ordenadas	Scatter de métricas + radar + grafo
Criterio de selección final	Confianza o lift únicos	Convergencia entre múltiples métricas

La tabla pone en evidencia que el análisis crítico no es simplemente más trabajo —es un análisis fundamentalmente diferente en su estructura epistémica. El analista que usa el marco crítico genera, en la mayoría de los casos, un conjunto de reglas reportadas más pequeño que el analista que usa el enfoque elemental. Pero ese conjunto más pequeño está respaldado por evidencia más robusta, es más estable ante variaciones en los parámetros, y es más accionable en la práctica.

Resumen Conceptual

Este capítulo ha profundizado en el análisis de reglas de asociación más allá de las métricas fundamentales introducidas en el capítulo anterior. Los puntos clave que el lector debe retener son los siguientes:

Sobre las limitaciones estructurales del lift:

- El lift es simétrico: $\text{lift}(A \Rightarrow B) = \text{lift}(B \Rightarrow A)$. Para elegir la dirección apropiada de una regla en una aplicación concreta, la confianza —métrica asimétrica— es el criterio relevante.
- El lift tiene un techo teórico que depende de los soportes individuales: $\text{lift}_{\text{máx}} = 1/\max(P(A), P(B))$. Comparar lifts absolutos entre reglas con distintos soportes puede ser engañoso. El lift relativo —cociente entre lift observado y lift máximo— es una escala más justa para la comparación.

Sobre las métricas complementarias:

- El leverage mide la diferencia absoluta entre co-ocurrencia observada e independencia. Es interpretable como el número de transacciones adicionales atribuibles a la asociación, pero es sistemáticamente bajo para ítems raros.
- La conviction es asimétrica y mide el cuasi-determinismo de la regla en su dirección: valores muy altos indican confianza cercana a 1.
- La métrica de Zhang tiene rango $[-1, 1]$, es simétrica y robusta ante ítems frecuentes. Es la métrica recomendada para comparaciones entre reglas con distintos niveles de frecuencia.
- Las métricas no son redundantes: sus correlaciones son imperfectas, y las discrepancias entre ellas son la señal más valiosa para el análisis crítico.

Sobre la sensibilidad a parámetros:

- Los umbrales del algoritmo Apriori no tienen justificación estadística intrínseca. El conjunto de reglas obtenido es función conjunta del dataset y de los parámetros elegidos.
- El análisis de sensibilidad —exploración sistemática del espacio de parámetros— es una práctica metodológica necesaria, no opcional.
- El mapa soporte-confianza visualiza el espacio completo de decisión y permite al analista elegir conscientemente su posición en el compromiso entre número y calidad de reglas.

Sobre la representación en grafo:

- El grafo de reglas hace visible la estructura topológica del espacio de co-ocurrencias: hubs, nodos puente y clústeres de ítems relacionados.
- La centralidad basada en lift —no en conteo de aristas— permite distinguir ítems genuinamente centrales de ítems centrales por mera frecuencia.
- El "Whole Milk Problem" ilustra cómo la alta frecuencia de un ítem como consecuente hace prácticamente imposible encontrar reglas genuinamente informativas hacia él, independientemente de los umbrales elegidos.

Sobre el marco integrado:

- La evaluación robusta de una regla requiere examinar simultáneamente al menos cuatro dimensiones: lift relativo, leverage, Zhang y conviction.
- Las reglas robustas son aquellas en las que múltiples métricas convergen en señalarlas como interesantes; las reglas candidatas son aquellas donde hay convergencia parcial; las reglas descartables son aquellas donde las métricas más exigentes las rechazan.

Parte XXIII.

Algoritmos de Patrones Frecuentes

47. FP-Growth y minería de patrones frecuentes

FP-Growth y minería de patrones frecuentes

Introducción

La minería de reglas de asociación busca descubrir regularidades en bases de datos transaccionales: productos que suelen comprarse juntos, páginas que suelen visitarse en una misma sesión, palabras que coaparecen en documentos o eventos que tienden a registrarse en conjunto. En este contexto, una transacción representa una observación compuesta por varios ítems; por ejemplo, una compra de supermercado puede contener `{Pan, Leche, Pañales}`. El objetivo no es predecir una etiqueta, sino identificar estructuras de coocurrencia que permitan extraer conocimiento interpretable a partir de datos discretos (Agrawal y Srikant 1994; Han et al. 2012).

El algoritmo **FP-Growth** (*Frequent Pattern Growth*) fue propuesto como una alternativa eficiente a Apriori para encontrar conjuntos frecuentes de ítems. Su idea central es evitar la generación explícita de candidatos y comprimir la base de transacciones en una estructura llamada **FP-tree** (*Frequent Pattern Tree*). A partir de ese árbol, el algoritmo explora patrones frecuentes mediante bases condicionales y árboles condicionales (Han et al. 2000; Borgelt 2005).

Este capítulo desarrolla FP-Growth de manera progresiva. Primero se revisan las nociones de transacción, itemset, soporte y frecuencia. Después se muestra cómo construir un FP-tree usando un ejemplo pequeño de cesta de compra. Finalmente, se explica cómo extraer patrones frecuentes mediante caminos prefijo condicionales, se compara FP-Growth con Apriori y se implementa un flujo práctico en Python.

Conceptos básicos

En minería de asociaciones, trabajamos con una colección de transacciones. Cada transacción contiene un subconjunto de ítems tomado de un universo finito de productos, términos o eventos posibles. Esta representación es natural en problemas donde los datos son conjuntos: una compra contiene productos, un documento contiene palabras, una sesión web contiene páginas visitadas.

Definición. Sea $I = \{i_1, i_2, \dots, i_m\}$ un conjunto finito de ítems. Una transacción T es un subconjunto de I, es decir:

$$T \subseteq I.$$

Una base transaccional es una colección $\mathcal{D} = \{T_1, T_2, \dots, T_n\}$ de transacciones.

Definición. Un itemset es cualquier subconjunto $X \subseteq I$. Decimos que una transacción T_i contiene a X si todos los ítems de X aparecen en T_i:

$$X \subseteq T_i.$$

El concepto clave para decidir si un patrón es relevante es el **soporte**. Existen dos formas comunes de reportarlo: como conteo absoluto o como proporción. El conteo absoluto indica cuántas transacciones contienen el patrón; el soporte relativo divide ese conteo entre el número total de transacciones.

Definición. El soporte absoluto de un itemset X en una base transaccional $\mathcal{D}$ es el número de transacciones que contienen a X:

$$\text{supp}_{\#}(X) = \left|\{T_i \in \mathcal{D} : X \subseteq T_i\}\right|.$$

El soporte relativo se define como:

$$\text{supp}(X) = \frac{\text{supp}_{\#}(X)}{|\mathcal{D}|}. \tag{47.1}$$

Definición. Dado un umbral mínimo de soporte min_sup, un itemset X es frecuente si su soporte absoluto cumple:

$$\text{supp}_{\#}(X) \geq \text{min_sup}. \tag{47.2}$$

El umbral min_sup controla la granularidad del análisis. Un valor demasiado bajo puede producir demasiados patrones, muchos de ellos triviales o difíciles de interpretar. Un valor demasiado alto puede eliminar patrones interesantes pero menos comunes. Por ello, en aplicaciones reales suele elegirse mediante exploración, conocimiento del dominio y restricciones computacionales.

De Apriori a FP-Growth

El algoritmo Apriori explota una propiedad fundamental: si un conjunto de ítems es frecuente, entonces todos sus subconjuntos también son frecuentes. De manera equivalente, si un itemset no es frecuente, ningún superconjunto suyo puede ser frecuente (Agrawal y Srikant 1994). Esta propiedad permite podar el espacio de búsqueda, pero Apriori todavía necesita generar muchos candidatos y escanear la base varias veces.

FP-Growth sigue otra estrategia. En lugar de construir y evaluar candidatos explícitamente, realiza dos recorridos principales sobre la base de datos: uno para calcular los soportes de los ítems individuales y otro para construir un árbol compacto. Después extrae patrones frecuentes desde la estructura comprimida (Han et al. 2000).

La ventaja aparece especialmente cuando muchas transacciones comparten prefijos. Si los ítems se ordenan globalmente por frecuencia decreciente, transacciones parecidas tienden a compartir las primeras ramas del árbol. Así, el FP-tree almacena información agregada en lugar de repetir cada transacción completa.

Ejemplo base: transacciones de cesta de compra

Considérese la siguiente base de transacciones y un soporte mínimo absoluto min_sup = 3:

Transacción	Ítems
T_1	{Pan, Leche}
T_2	{Pan, Leche, Pañales, Cerveza}
T_3	{Leche, Pañales, Cerveza}
T_4	{Pan, Leche, Pañales, Cerveza}
T_5	{Pan, Leche, Pañales}

Los soportes absolutos de los ítems individuales son:

Ítem	Transacciones donde aparece	Soporte absoluto
Pan	T_1, T_2, T_4, T_5	4
Leche	T_1, T_2, T_3, T_4, T_5	5
Pañales	T_2, T_3, T_4, T_5	4
Cerveza	T_2, T_3, T_4	3

Como todos los ítems tienen soporte absoluto mayor o igual que 3, ninguno se elimina en esta etapa. Si hubiera ítems con soporte menor que min_sup, se descartarían antes de construir el árbol.

Ordenamiento por frecuencia

El FP-tree depende de un orden global de ítems. La regla usual consiste en ordenar los ítems de mayor a menor frecuencia. En caso de empates, conviene fijar un criterio determinista, por ejemplo orden alfabético. Este ordenamiento hace que transacciones con ítems frecuentes compartan prefijos y que el árbol sea más compacto.

En el ejemplo, el orden global puede tomarse como:

$$\text{Leche} \succ \text{Pan} \succ \text{Pañales} \succ \text{Cerveza}.$$

Con ese orden, las transacciones se transforman en:

Transacción original	Transacción ordenada
{Pan, Leche}	[Leche, Pan]
{Pan, Leche, Pañales, Cerveza}	[Leche, Pan, Pañales, Cerveza]
{Leche, Pañales, Cerveza}	[Leche, Pañales, Cerveza]
{Pan, Leche, Pañales, Cerveza}	[Leche, Pan, Pañales, Cerveza]
{Pan, Leche, Pañales}	[Leche, Pan, Pañales]

Construcción del FP-tree

Un FP-tree es un árbol con raíz nula. Cada nodo almacena un ítem, un contador y enlaces hacia sus hijos. Para insertar una transacción ordenada, se recorre la transacción de izquierda a derecha. Si el siguiente ítem ya existe como hijo del nodo actual, se incrementa su contador. Si no existe, se crea un nuevo nodo con contador inicial igual a uno.

Definición. Un FP-tree es una estructura arbórea $\mathcal{T}$ donde cada nodo no raíz representa un ítem y almacena un contador. El contador de un nodo indica cuántas transacciones comparten el prefijo correspondiente al camino desde la raíz hasta ese nodo:

$$\text{count}(v) = \left|\{T_i \in \mathcal{D} : \text{path}(v) \subseteq T_i\}\right|.$$

El procedimiento de construcción puede resumirse así:

1. Crear una raíz `null`.
2. Ordenar cada transacción de acuerdo con la frecuencia global.
3. Insertar cada transacción en el árbol.
4. Incrementar contadores cuando una rama ya existe.
5. Crear nodos nuevos cuando el prefijo todavía no aparece.

En este ejemplo, la rama más importante inicia con `Leche`, porque aparece en todas las transacciones. Desde `Leche`, varias transacciones continúan con `Pan`; otras continúan directamente con `Pañales`. La compresión proviene de sumar contadores en prefijos compartidos en lugar de almacenar cada transacción de forma independiente.

Caminos prefijo y base condicional

Una vez construido el FP-tree, FP-Growth extrae patrones frecuentes considerando los ítems desde los menos frecuentes hacia los más frecuentes. Para cada ítem objetivo, se buscan todos los caminos que terminan en ese ítem. Al quitar el ítem objetivo de esos caminos se obtiene su **base de patrones condicional**.

Definición. Dado un ítem a, la base de patrones condicional de a, denotada $\mathcal{B}_a$, es el multiconjunto de caminos prefijo que aparecen antes de a en el FP-tree, ponderados por el contador del nodo asociado a a:

$$\mathcal{B}_a = \{(P_j, c_j) : P_j \text{ es un camino prefijo de } a \text{ y } c_j \text{ es su conteo}\}.$$

Para el ítem `Cerveza`, los caminos relevantes son:

Camino en el árbol	Conteo
`[Leche, Pan, Pañales, Cerveza]`	2
`[Leche, Pañales, Cerveza]`	1

Al eliminar `Cerveza`, se obtiene:

Camino prefijo	Conteo
`[Leche, Pan, Pañales]`	2
`[Leche, Pañales]`	1

En esta base condicional, los conteos acumulados son:

Ítem	Conteo condicional
Leche	3
Pan	2
Pañales	3

Con min_sup $= 3$, `Pan` se elimina de la base condicional de `Cerveza`, mientras que `Leche` y `Pañales` permanecen. Por tanto, los patrones frecuentes que incluyen `Cerveza` son:

$$\{\text{Cerveza}\}, \quad \{\text{Leche}, \text{Cerveza}\}, \quad \{\text{Pañales}, \text{Cerveza}\}, \quad \{\text{Leche}, \text{Pañales}, \text{Cerveza}\}.$$

Árbol condicional

Después de obtener la base condicional de un ítem, se eliminan los ítems que no alcanzan el soporte mínimo y se construye un nuevo FP-tree restringido a esa base. Ese árbol se llama **árbol condicional**. En el caso de `Cerveza`, el árbol condicional conserva la estructura:

$$\text{Leche} : 3 \rightarrow \text{Pañales} : 3.$$

Esto significa que, entre las transacciones que contienen `Cerveza`, existen tres ocurrencias donde `Leche` aparece junto con `Cerveza` y tres ocurrencias donde `Pañales` aparece junto con `Cerveza`. Además, el camino conjunto indica que `{Leche, Pañales, Cerveza}` también es frecuente.

Definición. El árbol condicional de un ítem a es el FP-tree construido a partir de la base condicional $\mathcal{B}_a$ después de eliminar los ítems cuyo conteo condicional es menor que min_sup:

$$\mathcal{T}_a = \text{FPtree}\left(\{P_j \in \mathcal{B}_a : \text{count}_{\mathcal{B}_a}(i) \geq \text{min_sup}\}\right).$$

Laboratorio: construir y minar el FP-tree

El laboratorio inserta las transacciones de una en una. En cada paso verás qué prefijo se comparte (los contadores suben) y qué nodos nuevos se crean. Después elige un ítem para obtener su base de patrones condicional y los patrones frecuentes que terminan en él. Las líneas discontinuas son los **enlaces de nodo** de la tabla de encabezados: conectan todas las apariciones de un mismo ítem en el árbol.

En la versión web, este laboratorio construye el FP-tree insertando las transacciones de una en una y calcula la base condicional del ítem elegido. Valores de referencia con soporte mínimo 3 y las cinco

transacciones: 6 nodos para 16 apariciones de ítems (38 %); la base condicional de Cerveza es [Leche, Pan, Pañales]: 2 y [Leche, Pañales]: 1, con conteos condicionales Leche = 3, Pan = 2, Pañales = 3, y sus patrones frecuentes son {Cerveza}, {Pañales, Cerveza}, {Leche, Pañales, Cerveza} y {Leche, Cerveza}, todos con conteo 3.

Guía del laboratorio.

1. Lleva el control a 0 y avanza de una en una. Predice antes de cada paso cuántos nodos nuevos creará la transacción; comprueba con la nota. ¿Cuál es la única transacción que abre una rama nueva bajo Leche?
2. Con las cinco transacciones, elige Cerveza y compara la base condicional con la tabla de la sección anterior.
3. Elige Pañales. ¿Por qué tiene dos caminos prefijo? ¿Qué patrones frecuentes salen?
4. Baja el soporte mínimo a 2. ¿Qué patrones nuevos aparecen para Cerveza y por qué todos incluyen a Pan? Súbelo a 4: ¿qué ítem desaparece del árbol y cuántos nodos quedan?

Patrones frecuentes del ejemplo

Repitiendo el procedimiento para los ítems restantes en orden ascendente de frecuencia, se obtienen los conjuntos frecuentes:

Ítem base	Patrones frecuentes asociados
Cerveza	`{Cerveza}`, `{Leche, Cerveza}`, `{Pañales, Cerveza}`, `{Leche, Pañales, Cerveza}`
Pañales	`{Pañales}`, `{Leche, Pañales}`, `{Pan, Pañales}`, `{Leche, Pan, Pañales}`
Pan	`{Pan}`, `{Leche, Pan}`
Leche	`{Leche}`

Agrupando y eliminando duplicados, los itemsets frecuentes con min_sup = 3 son:

$$\{\text{Leche}\}, \{\text{Pan}\}, \{\text{Pañales}\}, \{\text{Cerveza}\},$$
$$\{\text{Leche}, \text{Pan}\}, \{\text{Leche}, \text{Pañales}\}, \{\text{Leche}, \text{Cerveza}\},$$
$$\{\text{Pan}, \text{Pañales}\}, \{\text{Pañales}, \text{Cerveza}\},$$
$$\{\text{Leche}, \text{Pan}, \text{Pañales}\}, \{\text{Leche}, \text{Pañales}, \text{Cerveza}\}.$$

Es importante notar que FP-Growth no produce directamente reglas de asociación; primero produce itemsets frecuentes. Las reglas se construyen después a partir de esos itemsets, evaluando métricas como confianza, lift o convicción (Tan et al. 2019; Han et al. 2012).

Implementación práctica con `mlxtend`

En Python, una forma sencilla de aplicar FP-Growth es utilizar `mlxtend`, que implementa tanto Apriori como FP-Growth para matrices transaccionales codificadas en formato one-hot (Raschka 2018).

	Cerveza	Leche	Pan	Pañales
0	False	True	True	False
1	True	True	True	True
2	True	True	False	True
3	True	True	True	True
4	False	True	True	True

Como `mlxtend` usa soporte relativo, el umbral absoluto min_sup = 3 debe transformarse dividiendo entre el número de transacciones:

$$\text{min_sup}_{rel} = \frac{3}{5} = 0.6.$$

	support	itemsets
0	1.0	(Leche)
1	0.8	(Pan)
2	0.8	(Pañales)
4	0.8	(Pan, Leche)
7	0.8	(Leche, Pañales)
3	0.6	(Cerveza)
5	0.6	(Pan, Pañales)
6	0.6	(Pan, Leche, Pañales)
8	0.6	(Cerveza, Pañales)
9	0.6	(Cerveza, Leche)
10	0.6	(Cerveza, Leche, Pañales)

A partir de los itemsets frecuentes se pueden generar reglas. Por ejemplo, una regla $X \Rightarrow Y$ se interpreta como: "cuando aparece X, también tiende a aparecer Y".

Definición. La confianza de una regla de asociación $X \Rightarrow Y$, con $X \cap Y = \emptyset$, mide la proporción de transacciones que contienen X y también contienen Y:

$$\text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}. \tag{47.3}$$

Definición. El lift de una regla $X \Rightarrow Y$ compara la coocurrencia observada de X y Y con la

coocurrencia esperada bajo independencia estadística:

$$\text{lift}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)\,\text{supp}(Y)}. \tag{47.4}$$

	antecedents	consequents	support	confidence	lift
11	(Cerveza)	(Pañales)	0.6	1.00	1.2500
14	(Cerveza, Leche)	(Pañales)	0.6	1.00	1.2500
17	(Cerveza)	(Leche, Pañales)	0.6	1.00	1.2500
12	(Pañales)	(Cerveza)	0.6	0.75	1.2500
16	(Leche, Pañales)	(Cerveza)	0.6	0.75	1.2500
18	(Pañales)	(Cerveza, Leche)	0.6	0.75	1.2500
0	(Pan)	(Leche)	0.8	1.00	1.0000
5	(Pan, Pañales)	(Leche)	0.6	1.00	1.0000
10	(Pañales)	(Leche)	0.8	1.00	1.0000
13	(Cerveza)	(Leche)	0.6	1.00	1.0000
15	(Cerveza, Pañales)	(Leche)	0.6	1.00	1.0000
1	(Leche)	(Pan)	0.8	0.80	1.0000
9	(Leche)	(Pañales)	0.8	0.80	1.0000
2	(Pan)	(Pañales)	0.6	0.75	0.9375
3	(Pañales)	(Pan)	0.6	0.75	0.9375
4	(Pan, Leche)	(Pañales)	0.6	0.75	0.9375
6	(Leche, Pañales)	(Pan)	0.6	0.75	0.9375
7	(Pan)	(Leche, Pañales)	0.6	0.75	0.9375
8	(Pañales)	(Pan, Leche)	0.6	0.75	0.9375

La confianza puede ser alta aun cuando el consecuente sea muy frecuente. Por ello, conviene interpretarla junto con el lift. Un lift mayor que uno sugiere asociación positiva; un lift cercano a uno sugiere que la regla no mejora sustancialmente lo esperado por independencia; un lift menor que uno sugiere asociación negativa.

Ejemplo práctico: FP-Growth sobre 50 canastas

Volvamos a la muestra de 50 canastas del capítulo de Apriori. Construimos el FP-tree con las funciones de este capítulo para medir cuánto comprime y comprobamos que `fpgrowth` y `apriori` devuelven exactamente los mismos itemsets frecuentes.

Crecimiento del FP-tree al insertar las 50 canastas (soporte mínimo de 5 canastas). La línea azul cuenta las apariciones de ítems frecuentes acumuladas en las transacciones; la naranja, los nodos del árbol. La distancia entre ambas es la compresión.

Con canastas tan cortas y heterogéneas la compresión es moderada: muchas canastas no comparten prefijo. El árbol comprime más cuando las transacciones se parecen entre sí, que es justamente el caso de datos densos donde Apriori genera más candidatos. Ambos algoritmos son **exactos**: cambian el camino, no la respuesta.

Comparación entre FP-Growth y Apriori

Característica	FP-Growth	Apriori
Generación de candidatos	No requerida	Necesaria
Recorridos de los datos	Usualmente dos recorridos principales	Varios recorridos
Velocidad	Eficiente en bases grandes y densas	Puede ser lento por generación repetida de candidatos
Uso de memoria	Puede ser alto por el FP-tree	Generalmente menor, basado en listas y conteos
Implementación	Más compleja	Más sencilla de explicar e implementar

FP-Growth suele ser preferible cuando la base es grande y contiene muchos patrones frecuentes. Apriori puede ser útil con fines didácticos o cuando se desea una implementación directa basada en candidatos. En la práctica, la elección también depende de la densidad de las transacciones, el umbral de soporte, la memoria disponible y la necesidad de interpretar pasos intermedios.

Aplicaciones

FP-Growth es aplicable a distintos dominios donde los datos pueden representarse como conjuntos de eventos o ítems:

- **Análisis de cesta de compra:** identificar productos que se compran juntos con frecuencia.
- **Sistemas de recomendación:** sugerir productos, contenidos o servicios a partir de patrones de coocurrencia.
- **Análisis de comportamiento del cliente:** estudiar hábitos de compra en comercio minorista o electrónico.
- **Minería de uso web:** detectar rutas de navegación o secuencias de clics comunes.
- **Minería de texto:** encontrar palabras, términos o frases que aparecen juntas en documentos.

En todos estos casos, los patrones frecuentes deben interpretarse con cuidado. La coocurrencia no implica causalidad. Una regla puede ser estadísticamente fuerte y, aun así, carecer de utilidad práctica si representa un patrón obvio, redundante o inducido por sesgos de captura de datos.

Ventajas y limitaciones

Entre las principales ventajas de FP-Growth se encuentran:

- Evita la generación explícita de candidatos.
- Reduce el número de escaneos completos de la base de datos.
- Comprime transacciones mediante prefijos compartidos.
- Puede extraer patrones frecuentes complejos de manera eficiente.
- Funciona bien en bases densas donde Apriori produce demasiados candidatos.

Sus limitaciones principales son:

- El FP-tree puede consumir mucha memoria si la base es grande y poco compresible.
- La implementación es más compleja que Apriori.
- La minería condicional puede ser costosa cuando existen muchos patrones frecuentes.
- No es naturalmente incremental: cuando cambian los datos, puede requerir reconstruir el árbol.

Estas limitaciones no invalidan el método, pero sí obligan a seleccionar adecuadamente el umbral de soporte, revisar la calidad de los datos y evaluar la interpretabilidad de los patrones resultantes.

Buenas prácticas de análisis

Al aplicar FP-Growth en un proyecto real, conviene seguir una estrategia reproducible:

1. Definir qué representa una transacción.
2. Limpiar y normalizar los ítems.
3. Explorar frecuencias individuales antes de minar patrones.
4. Probar varios valores de soporte mínimo.
5. Generar reglas solo después de revisar los itemsets frecuentes.
6. Evaluar confianza, lift y soporte de manera conjunta.
7. Validar los patrones con conocimiento del dominio.
8. Documentar decisiones, umbrales y supuestos.

La etapa más importante no es ejecutar el algoritmo, sino construir una representación transaccional coherente. Por ejemplo, en minería de texto, decidir si los ítems serán palabras, lemas, n-gramas o entidades cambia completamente el significado de los patrones. En comercio electrónico, decidir si una transacción corresponde a una compra, una sesión o un usuario también modifica la interpretación del soporte.

Conclusiones

FP-Growth es un algoritmo fundamental para la minería de patrones frecuentes. Su principal contribución es reemplazar la generación explícita de candidatos por una estructura compacta, el FP-tree, que permite almacenar prefijos compartidos y extraer patrones mediante bases condicionales. Esta estrategia reduce recorridos sobre la base de datos y suele mejorar el rendimiento frente a Apriori en escenarios grandes o densos.

Desde el punto de vista conceptual, FP-Growth muestra una idea central en ciencia de datos: la eficiencia no depende solo del algoritmo, sino también de la representación. Al transformar transacciones en un árbol comprimido, el problema se vuelve más manejable sin perder la información necesaria para recuperar los itemsets frecuentes.

En aplicaciones reales, el algoritmo debe acompañarse de una interpretación cuidadosa. Los patrones frecuentes son puntos de partida para formular hipótesis, diseñar recomendaciones o comprender comportamientos, pero no constituyen por sí solos evidencia causal. Por ello, la minería de asociaciones debe integrarse con análisis exploratorio, conocimiento del dominio y criterios claros de utilidad práctica.

48. Itemsets Cerrados y Maximales: Representaciones Compactas

Apriori y FP-Growth resuelven el problema de **encontrar** todos los itemsets frecuentes. Este capítulo trata un problema distinto que aparece justo después: **hay demasiados**. Si un cliente típico compra juntos 20 productos de despensa, el itemset con esos 20 productos es frecuente y, por la propiedad antimonótona, también lo son sus $2^{20} - 1 \approx 10^6$ subconjuntos. La lista completa es correcta, pero casi toda es redundante: describe una y otra vez el mismo hecho. Las **representaciones compactas** guardan solo una parte de los itemsets frecuentes, elegida de modo que el resto se pueda deducir (Pasquier et al. 1999; Han et al. 2012).

Intuición: once patrones que dicen casi lo mismo

Retomemos los cinco tickets del capítulo de FP-Growth, con conteo mínimo 3:

Tabla 48.1.: Base de transacciones del capítulo de FP-Growth.

Ticket	Productos
T_1	Pan, Leche
T_2	Pan, Leche, Pañales, Cerveza
T_3	Leche, Pañales, Cerveza
T_4	Pan, Leche, Pañales, Cerveza
T_5	Pan, Leche, Pañales

Con esos datos hay **11 itemsets frecuentes**. Mira dos de ellos: $\{\text{Pan}\}$ aparece en 4 tickets y $\{\text{Leche}, \text{Pan}\}$ también en 4. No es coincidencia: **todo ticket que tiene pan tiene leche**. Saber que $\{\text{Leche}, \text{Pan}\}$ tiene conteo 4 ya nos dice el conteo de $\{\text{Pan}\}$; reportar ambos es repetir información.

Hay dos maneras naturales de quitar esa redundancia:

- **Quedarse con los itemsets que no se pueden agrandar sin perder conteo.** $\{\text{Pan}\}$ se puede agrandar a $\{\text{Leche}, \text{Pan}\}$ sin perder nada, así que sobra. Estos itemsets se llaman **cerrados** y, como veremos, con ellos se recupera el conteo exacto de cualquier itemset frecuente.
- **Quedarse con los itemsets que no se pueden agrandar sin dejar de ser frecuentes.** Son la «frontera» de la retícula y se llaman **maximales**. Con ellos sabemos *cuáles* itemsets son frecuentes, pero no *cuánto*.

Construcción numérica paso a paso

Para cada itemset frecuente preguntamos dos cosas: ¿tiene algún superconjunto con **el mismo conteo**? (si sí, no es cerrado) y ¿tiene algún superconjunto **frecuente**? (si sí, no es maximal). Basta revisar los superconjuntos con un ítem más[1].

Tabla 48.2.: Clasificación de los 11 itemsets frecuentes con conteo mínimo 3.

Itemset frecuente	Conteo	Superconjunto con el mismo conteo	¿Cerrado?	¿Maximal?
{Cerveza}	3	{Cerveza, Leche} (3)	no	no
{Leche}	5	ninguno (los pares llegan a 4)	**sí**	no
{Pan}	4	{Leche, Pan} (4)	no	no
{Pañales}	4	{Leche, Pañales} (4)	no	no
{Cerveza, Leche}	3	{Cerveza, Leche, Pañales} (3)	no	no
{Cerveza, Pañales}	3	{Cerveza, Leche, Pañales} (3)	no	no
{Leche, Pan}	4	ninguno: {Leche, Pan, Pañales} baja a 3	**sí**	no
{Leche, Pañales}	4	ninguno: los tríos bajan a 3	**sí**	no
{Pan, Pañales}	3	{Leche, Pan, Pañales} (3)	no	no
{Cerveza, Leche, Pañales}	3	ninguno: el cuarteto tiene conteo 2	**sí**	**sí**
{Leche, Pan, Pañales}	3	ninguno: el cuarteto tiene conteo 2	**sí**	**sí**

El resultado es una cadena de tres conjuntos: **11 frecuentes, 5 cerrados y 2 maximales**. Los cerrados son {Leche}:5, {Leche, Pan}:4, {Leche, Pañales}:4, {Leche, Pan, Pañales}:3 y

[1]Si algún superconjunto $Y \supsetneq X$ tiene el mismo conteo que X, entonces también lo tiene cualquier itemset intermedio $X \subsetneq Z \subseteq Y$, porque por antimonotonía $\sigma(X) \geq \sigma(Z) \geq \sigma(Y) = \sigma(X)$. En particular, lo tiene algún superconjunto con un solo ítem más. Lo mismo vale para «ser frecuente».

{Cerveza, Leche, Pañales}:3; los maximales, los dos tríos.

Un atajo para encontrar el cerrado de un itemset. Junta los tickets que contienen al itemset y quédate con los productos que están en **todos** ellos. Para {Cerveza}: aparece en T_2, T_3 y T_4, y los productos comunes a esos tres tickets son Cerveza, Leche y Pañales. Por lo tanto, el cerrado que «representa» a {Cerveza} es {Cerveza, Leche, Pañales}, con el mismo conteo 3. Esta operación se llama **clausura**.

Recuperar un conteo desde los cerrados. ¿Cuál es el conteo de {Pan, Pañales} si solo guardamos los 5 cerrados? Buscamos los cerrados que lo contienen: solo {Leche, Pan, Pañales}:3. El conteo es 3. Para {Pan} hay dos cerrados que lo contienen, {Leche, Pan}:4 y {Leche, Pan, Pañales}:3; el conteo es **el mayor**, 4.

Lo que se pierde con los maximales. Con los dos tríos maximales sabemos que {Pan, Pañales} es frecuente (está contenido en {Leche, Pan, Pañales}) y que {Cerveza, Pan} no lo es (ningún maximal lo contiene). Pero del conteo de {Pan} solo podemos decir que es al menos 3.

Formalización

Usamos la notación de los capítulos anteriores: $\mathcal{D}$ es la base de transacciones, $\sigma(X)$ el conteo de soporte de X y $\mathcal{F}$ el conjunto de itemsets frecuentes con un umbral dado.

Definición (itemset cerrado). Un itemset X es **cerrado** si ningún superconjunto propio tiene el mismo conteo:

$$X \text{ es cerrado} \iff \nexists\, Y \supsetneq X \text{ con } \sigma(Y) = \sigma(X).$$

El conjunto de **itemsets frecuentes cerrados** es $\mathcal{C} = \{X \in \mathcal{F} : X \text{ es cerrado}\}$.

Definición (itemset maximal). Un itemset frecuente X es **maximal** si ningún superconjunto propio es frecuente:

$$X \text{ es maximal} \iff X \in \mathcal{F} \text{ y } \nexists\, Y \supsetneq X \text{ con } Y \in \mathcal{F}.$$

Lo denotamos $\mathcal{M}$.

El operador de clausura. Sea $t(X) = \{T \in \mathcal{D} : X \subseteq T\}$ el conjunto de tickets que contienen a X. La clausura de X es la intersección de esos tickets:

$$\gamma(X) = \bigcap_{T \in t(X)} T. \tag{48.1}$$

$\gamma(X)$ son los ítems que acompañan a X **siempre**. Tiene las tres propiedades de un operador de clausura[2]: contiene a X, respeta la inclusión y aplicarlo dos veces no cambia nada. Además, X

[2]Un **operador de clausura** sobre los subconjuntos de $\mathcal{I}$ es una función γ que cumple tres propiedades: es *extensiva*, $X \subseteq \gamma(X)$; es *monótona*, $X \subseteq Y \Rightarrow \gamma(X) \subseteq \gamma(Y)$; y es *idempotente*, $\gamma(\gamma(X)) = \gamma(X)$. Los conjuntos fijos ($\gamma(X) = X$) se llaman cerrados. El nombre viene de la topología y del álgebra, donde «cerrar» un conjunto es agregarle todo lo que se deduce de él.

y $\gamma(X)$ aparecen exactamente en los mismos tickets, así que $\sigma(\gamma(X)) = \sigma(X)$. De ahí sale una caracterización muy útil:

$$X \text{ es cerrado } \iff \gamma(X) = X. \tag{48.2}$$

Tres resultados que justifican el uso de estas representaciones:

1. **Cadena de inclusiones.** $\mathcal{M} \subseteq \mathcal{C} \subseteq \mathcal{F}$. Si X es maximal y $Y \supsetneq X$, entonces Y no es frecuente y $\sigma(Y) < \sigma(X)$; por lo tanto X es cerrado.
2. **Los cerrados no pierden información** (Pasquier et al. 1999). Para todo $X \in \mathcal{F}$,

   $$\sigma(X) = \max\{\sigma(C) : C \in \mathcal{C},\ X \subseteq C\}. \tag{48.3}$$

   El máximo se alcanza en $C = \gamma(X)$, que es cerrado y tiene el mismo conteo que X; cualquier otro cerrado que contenga a X tiene conteo menor o igual por antimonotonía. Si ningún cerrado frecuente contiene a X, entonces X no es frecuente.
3. **Los maximales describen la frontera, no los conteos.** X es frecuente si y solo si está contenido en algún maximal, pero de su conteo solo se sabe que es al menos el umbral.

Por eso se habla de representaciones **condensadas**[3]: los cerrados son una compresión sin pérdida de $\mathcal{F}$ con sus conteos; los maximales son una compresión con pérdida, mucho más pequeña.

Relación con las reglas. Una regla con confianza 1 dice que el antecedente arrastra siempre al consecuente; eso es justo la clausura. Pan $\Rightarrow$ Leche tiene confianza $4/4 = 1$ porque $\gamma(\{\text{Pan}\}) = \{\text{Leche}, \text{Pan}\}$. En general, toda regla exacta tiene la forma $X \Rightarrow \gamma(X) \setminus X$, y las reglas generadas desde los cerrados evitan buena parte de la redundancia que produce Apriori (Pasquier et al. 1999).

Algoritmos. Se pueden obtener $\mathcal{C}$ o $\mathcal{M}$ filtrando la salida de Apriori o FP-Growth, que es lo que haremos en Python. Cuando $\mathcal{F}$ es enorme conviene minar directamente la representación compacta:

Tabla 48.3.: Algoritmos clásicos para representaciones compactas.

Algoritmo	Qué mina	Idea principal
A-Close	Cerrados	Apriori por niveles, generando solo clausuras (Pasquier et al. 1999)
CHARM	Cerrados	Búsqueda en profundidad con listas de tickets (*tidsets*) (Zaki y Hsiao 2002)
CLOSET+	Cerrados	Crecimiento de patrones sobre un FP-tree (Wang et al. 2003)
MaxMiner	Maximales	Mira hacia adelante: si la rama completa es frecuente, la poda entera (Bayardo 1998)

[3]Una representación es **condensada** si a partir de ella se puede reconstruir toda la información de interés. Es la misma distinción que en compresión de archivos: ZIP es sin pérdida (se recupera el original exacto), JPEG es con pérdida (se recupera una aproximación). Los cerrados son el «ZIP» de los itemsets frecuentes; los maximales, el «JPEG».

Algoritmo	Qué mina	Idea principal
FPMax	Maximales	FP-Growth que descarta subconjuntos de maximales ya encontrados (Grahne y Zhu 2003)

Laboratorio: frecuentes, cerrados y maximales en la retícula

El laboratorio dibuja los 15 itemsets de la Tabla 48.1 y los clasifica según el conteo mínimo. Haz clic en un nodo para ver su clausura (la flecha morada lleva al cerrado que lo representa) y cómo se recupera su conteo desde los cerrados. Al final puedes editar los tickets.

En la versión web, este laboratorio dibuja la retícula de los 15 itemsets de la Tabla 48.1, los clasifica según el conteo mínimo y permite editar los tickets. Valores de referencia: con conteo mínimo 3 hay 11 frecuentes, 5 cerrados y 2 maximales; con conteo mínimo 2 (o 1), los 15 itemsets son frecuentes, hay 6 cerrados (se agrega $\{C, L, P, \}$:2) y un solo maximal, el cuarteto; con conteo mínimo 4 hay 5 frecuentes, 3 cerrados ($\{L\}$, $\{L, P\}$, $\{L, \}$) y 2 maximales ($\{L, P\}$, $\{L, \}$).

Guía del laboratorio.

1. Predice antes de hacer clic: ¿cuál es la clausura de {Ñ} (Pañales)? Haz clic en el nodo y comprueba en el panel la intersección de tickets.
2. Comprueba los números de la Tabla 48.2: con conteo mínimo 3 deben aparecer 11 frecuentes, 5 cerrados y 2 maximales. Activa «Mostrar todas las clausuras»: ¿a qué cerrado apunta cada uno de los 6 itemsets azules?
3. Selecciona $\{P, \}$ y recupera su conteo desde los cerrados. ¿Qué puedes decir de él si solo conservas los maximales?
4. Baja el conteo mínimo a 2. ¿Por qué solo queda un maximal? ¿Por qué el número de cerrados casi no cambia mientras que el de frecuentes sube a 15?
5. Rómpelo: quita la leche de T_1. ¿Sigue siendo $\{L\}$ cerrado? ¿Qué pasa con $\{P\}$? Explica el cambio con la clausura.

Implementación en Python

Desde cero: clausura y clasificación

Las definiciones se traducen casi literalmente. La clasificación revisa solo los superconjuntos con un ítem más, como en la construcción manual.

	itemset	conteo	clausura	cerrado	maximal
1	{Leche}	5	{Leche}	True	False
6	{Leche, Pan}	4	{Leche, Pan}	True	False
7	{Leche, Pañales}	4	{Leche, Pañales}	True	False
9	{Cerveza, Leche, Pañales}	3	{Cerveza, Leche, Pañales}	True	True
10	{Leche, Pan, Pañales}	3	{Leche, Pan, Pañales}	True	True

	itemset	conteo	clausura	cerrado	maximal
2	{Pan}	4	{Leche, Pan}	False	False
3	{Pañales}	4	{Leche, Pañales}	False	False
0	{Cerveza}	3	{Cerveza, Leche, Pañales}	False	False
4	{Cerveza, Leche}	3	{Cerveza, Leche, Pañales}	False	False
5	{Cerveza, Pañales}	3	{Cerveza, Leche, Pañales}	False	False
8	{Pan, Pañales}	3	{Leche, Pan, Pañales}	False	False

La columna `clausura` confirma la caracterización de la Ecuación 48.2: un itemset es cerrado exactamente cuando coincide con su clausura.

Desde la salida de `mlxtend`

En la práctica, los itemsets frecuentes salen de `fpgrowth` o `apriori` (Raschka 2018). El resultado es un `DataFrame` con una columna `itemsets` de `frozenset`, así que basta un diccionario `itemset → conteo` para clasificar. Trabajamos con conteos enteros (soporte $\times$ n) para que la comparación «mismo conteo» sea exacta y no dependa del redondeo de los flotantes.

	itemsets	conteo	maximal
0	(Leche)	5	False
4	(Pan, Leche)	4	False
7	(Leche, Pañales)	4	False
6	(Pan, Leche, Pañales)	3	True
10	(Leche, Pañales, Cerveza)	3	True

Un detalle sobre «el mismo conteo»: si un superconjunto tiene el mismo conteo que X, es frecuente y, por lo tanto, está en la salida de `fpgrowth`. Por eso basta mirar el diccionario de frecuentes para decidir si X es cerrado.

Reconstruir los soportes desde los cerrados

Ahora olvidamos la lista completa y conservamos solo los 5 cerrados. La Ecuación 48.3 reconstruye el conteo de cualquier itemset; lo comprobamos sobre los 11 frecuentes y sobre un itemset infrecuente.

	itemsets	conteo	reconstruido
0	(Leche)	5	5
1	(Pan)	4	4
2	(Pañales)	4	4
4	(Pan, Leche)	4	4
7	(Leche, Pañales)	4	4
3	(Cerveza)	3	3
5	(Pan, Pañales)	3	3
6	(Pan, Leche, Pañales)	3	3

	itemsets	conteo	reconstruido
8	(Pañales, Cerveza)	3	3
9	(Leche, Cerveza)	3	3
10	(Leche, Pañales, Cerveza)	3	3

Ejemplo práctico: datos dispersos frente a datos densos

¿Cuánto se ahorra en la práctica? Depende de cuánto se repitan las combinaciones. Comparamos dos bases:

- **Dispersa**: las 50 canastas de 3 productos de los capítulos de Apriori y FP-Growth.
- **Enriquecida con la jerarquía**: 300 canastas simuladas en las que, además de cada producto, se agregan su **categoría** y su **departamento**, tal como vienen en la dimensión producto del almacén de datos (café molido → café → bebidas). Es una práctica común para encontrar patrones a varios niveles de detalle, y vuelve los datos mucho más densos.

Figura 48.1.

Tabla 48.4.: Itemsets frecuentes, cerrados y maximales con soporte mínimo 0.10 en las dos bases.

	frecuentes	cerrados	maximales
Canastas dispersas (50)	26	24	15
Canastas con jerarquía (300)	313	85	37

En las canastas dispersas casi todo itemset frecuente ya es cerrado: con tres productos por canasta, rara vez un producto «arrastra» siempre a otro. En las canastas con jerarquía ocurre lo contrario: todo ticket con café molido tiene también «café» y «bebidas», así que $\gamma(\{\text{café molido}\}) \supseteq \{\text{café molido}, \text{café}, \text{bebidas}\}$ y cada combinación de productos aparece repetida con todas las combinaciones de sus ancestros, con el mismo conteo. Los frecuentes se multiplican; los cerrados, no. La lección práctica: **en datos densos o con dependencias exactas, reporta cerrados (si necesitas los conteos) o maximales (si solo necesitas saber qué combinaciones son frecuentes)**.

Observa también que la compresión de los cerrados depende de que haya conteos **exactamente iguales**. Si dos productos casi siempre van juntos, pero no siempre, sus itemsets siguen siendo cerrados; para resumir esas dependencias aproximadas existen variantes como los itemsets *-libres* o los cerrados aproximados, que quedan fuera de este capítulo.

Cuándo usar cada representación

Tabla 48.5.: Comparación de las tres representaciones.

Representación	Tamaño	¿Conteos exactos?	Uso típico
Frecuentes $\mathcal{F}$	El más grande	Sí, directos	Datos dispersos; generar todas las reglas
Cerrados $\mathcal{C}$	Intermedio	Sí, con la Ecuación 48.3	Datos densos; reglas no redundantes; análisis formal de conceptos[4]
Maximales $\mathcal{M}$	El más pequeño	No, solo « umbral»	Describir la frontera; patrones largos; resumen para el negocio

Las mismas ideas se extienden más allá de los itemsets. En el capítulo siguiente, de patrones secuenciales, los patrones son **secuencias** ordenadas de eventos, y allí también se habla de secuencias cerradas y maximales con exactamente las mismas definiciones.

[4] El **análisis formal de conceptos** (*formal concept analysis*) estudia exactamente esta estructura: pares (conjunto de ítems cerrado, conjunto de tickets que lo contienen), llamados *conceptos*, ordenados en una retícula (Ganter y Wille 1999). Los itemsets cerrados son las «intenciones» de esos conceptos.

Parte XXIV.

Patrones Secuenciales

49. Patrones Secuenciales: GSP y PrefixSpan

Hasta ahora una transacción ha sido un **conjunto**: el ticket $\{\text{pan}, \text{leche}\}$ es el mismo que $\{\text{leche}, \text{pan}\}$. Pero en muchos datos el **orden** es la información principal. En una tienda en línea, pasar de la búsqueda a la ficha del producto y luego al carrito describe un recorrido de compra; recorrer las mismas páginas al revés describe otra cosa. Lo mismo ocurre con las compras sucesivas de un cliente (primero la impresora, meses después los cartuchos), los eventos de un expediente clínico o los pasos de un proceso registrados en una bitácora. La **minería de patrones secuenciales** busca las subsecuencias de eventos que se repiten en muchas secuencias (Agrawal y Srikant 1995; Han et al. 2012).

En el almacén de datos, la materia prima es la misma tabla de hechos que en los capítulos anteriores, pero ahora se agrupa por cliente o por sesión y **se ordena por fecha**: una tabla de hechos de eventos web (*clickstream*) con llave de sesión, página y marca de tiempo produce directamente una secuencia por sesión.

Intuición: cinco sesiones de una tienda en línea

Una tienda en línea registró cinco sesiones. Cada letra es una página visitada, en orden:

Tabla 49.1.: Cinco sesiones de navegación (I = inicio, B = búsqueda, F = ficha de producto, C = carrito, P = pago).

Sesión	Recorrido	Abreviado
S_1	inicio → búsqueda → ficha → carrito → pago	I, B, F, C, P
S_2	inicio → ficha → búsqueda → ficha → carrito	I, F, B, F, C
S_3	búsqueda → ficha → carrito → pago	B, F, C, P
S_4	inicio → búsqueda → ficha → ficha	I, B, F, F
S_5	inicio → carrito → pago	I, C, P

Algunas observaciones sin fórmulas:

- El recorrido **búsqueda → ficha → carrito** aparece en tres sesiones (S_1, S_2, S_3). Es un patrón de compra.
- **Buscar y ver una ficha** ocurre en cuatro sesiones, pero **ver una ficha y después buscar** solo en una (S_2). Como conjuntos, $\{B, F\}$ es lo mismo en ambos casos; como secuencias, son patrones distintos con frecuencias muy diferentes.

- En S_1, el patrón **inicio → carrito** aparece aunque haya páginas en medio. Un patrón secuencial respeta el **orden**, pero admite **huecos**.

Construcción numérica paso a paso

Contención y soporte

Una sesión **contiene** un patrón si se pueden encontrar sus páginas en la sesión, en el mismo orden, aunque haya otras páginas entre ellas. Por ejemplo, ⟨I, C⟩ está en $S_1 = \langle \mathbf{I}, B, F, \mathbf{C}, P \rangle$, en S_2 y en S_5, pero no en S_3 (no tiene inicio) ni en S_4 (no tiene carrito): su conteo de soporte es 3. Una sesión cuenta **una sola vez** aunque contenga el patrón varias veces: ⟨F⟩ aparece dos veces en S_2 y en S_4, y su conteo es 4, no 6.

Fijamos un **conteo mínimo de 3 sesiones**. Las cinco páginas son frecuentes: I, B, F y C aparecen en 4 sesiones y P en 3.

PrefixSpan: crecer el patrón desde un prefijo

La idea de PrefixSpan (Pei et al. 2001) es la de FP-Growth aplicada al orden: en lugar de generar candidatos, **fija un prefijo** y mira solo lo que ocurre **después** de él. Para cada sesión que contiene el prefijo se guarda el **sufijo** que sigue a su primera aparición; el conjunto de esos sufijos es la **base proyectada**.

Prefijo ⟨B⟩. Cortamos cada sesión después de su primera búsqueda:

Tabla 49.2.: Base proyectada sobre el prefijo ⟨B⟩.

Sesión	Sesión completa	Sufijo después de B
S_1	I, **B**, F, C, P	F, C, P
S_2	I, F, **B**, F, C	F, C
S_3	**B**, F, C, P	F, C, P
S_4	I, **B**, F, F	F, F
S_5	I, C, P	(no contiene B)

En los sufijos, F aparece en 4, C en 3 y P en 2. Por lo tanto ⟨B, F⟩ (4) y ⟨B, C⟩ (3) son frecuentes y ⟨B, P⟩ (2) no. Observa que **no hubo que volver a recorrer las sesiones completas**: el conteo de la extensión se hace dentro de la base proyectada.

Prefijo ⟨B, F⟩. Proyectamos de nuevo, ahora sobre los sufijos: ⟨C, P⟩, ⟨C⟩, ⟨C, P⟩ y ⟨F⟩. C aparece en 3, P en 2 y F en 1: solo ⟨B, F, C⟩ (3) es frecuente.

Prefijo ⟨B, F, C⟩. Los sufijos son ⟨P⟩ (S_1), vacío (S_2) y ⟨P⟩ (S_3). P aparece en 2: no hay extensión frecuente y esta rama termina. Lo mismo ocurre con ⟨B, C⟩, cuyos sufijos son ⟨P⟩, vacío y ⟨P⟩.

Repitiendo el procedimiento para los otros prefijos de longitud 1:

Tabla 49.3.: Bases proyectadas del resto de los prefijos (conteo mínimo 3).

Prefijo	Base proyectada	Extensiones frecuentes
I	B,F,C,P , F,B,F,C , B,F,F , C,P	I,B (3), I,F (3), I,C (3)
I, B	F,C,P , F,C , F,F	I,B,F (3)
I, F	C,P , B,F,C , F	ninguna (C y F llegan a 2)
F	C,P , B,F,C , C,P , F	F,C (3)
C	P , vacío, P , P	C,P (3)
P	vacío en las tres sesiones	ninguna

El resultado completo son **14 patrones secuenciales frecuentes**: 5 de longitud 1 (I, B, F, C, P), 7 de longitud 2 (I,B , I,F , I,C , B,F , B,C , F,C , C,P) y 2 de longitud 3 (I,B,F y B,F,C). Igual que con los itemsets, muchos son redundantes; los **maximales** son cuatro: I,B,F , B,F,C , I,C y C,P .

GSP: la alternativa por niveles

GSP (Srikant y Agrawal 1996) es la versión secuencial de Apriori. En cada nivel genera candidatos, los poda con la propiedad antimonótona y recorre **toda** la base para contarlos:

- **Nivel 2.** Con 5 páginas frecuentes hay $5 \times 5 = 25$ candidatos, porque el orden importa y una página puede repetirse (F, F es un candidato válido). Con itemsets serían solo 10 pares. Resultan frecuentes los 7 pares de arriba.
- **Nivel 3.** Se unen dos patrones cuando el primero sin su primer elemento coincide con el segundo sin su último elemento[1]: I,B + B,F → I,B,F . Salen 7 candidatos; la poda descarta 3 (por ejemplo I,C,P , porque I,P no es frecuente) y se cuentan 4, de los cuales 2 son frecuentes.
- **Nivel 4.** I,B,F + B,F,C → I,B,F,C , que se poda porque I,B,C no es frecuente. Fin.

GSP cuenta $5 + 25 + 4 = 34$ candidatos y recorre la base tres veces. PrefixSpan llega a los mismos 14 patrones sin generar ningún candidato que no aparezca en los datos.

Formalización

Sea $\mathcal{E}$ el conjunto de eventos posibles (páginas, productos, diagnósticos) y $\mathcal{S} = \{s_1, \dots, s_n\}$ una base de secuencias.

Definición (secuencia y subsecuencia). Una secuencia es una lista ordenada $s = \langle e_1, e_2, \dots, e_\ell \rangle$ con $e_j \in \mathcal{E}$; su longitud es ℓ. Una secuencia $\alpha = \langle a_1, \dots, a_k \rangle$ es **subsecuencia** de s, y se escribe $\alpha \sqsubseteq s$, si existen posiciones $j_1 < j_2 < \cdots < j_k$ tales que $a_r = e_{j_r}$ para todo

[1] En GSP original, la poda revisa solo las subsecuencias **contiguas** de longitud $k-1$ (las que se obtienen quitando el primer o el último elemento, o un ítem de un elemento interior). La razón es que, con una restricción de hueco máximo, quitar un evento interior puede crear un hueco demasiado grande y la antimonotonía deja de valer para esas subsecuencias. Sin restricciones de tiempo, como aquí, se pueden revisar todas las subsecuencias de longitud $k-1$, lo que poda más.

r.

Definición (soporte de un patrón secuencial). El conteo de soporte de α es el número de secuencias que lo contienen,

$$\sigma(\alpha) = |\{s \in \mathcal{S} : \alpha \sqsubseteq s\}|, \tag{49.1}$$

y α es un **patrón secuencial frecuente** si $\sigma(\alpha) \geq \sigma_{\text{mín}}$.

En la definición original (Agrawal y Srikant 1995) cada elemento de la secuencia es a su vez un itemset (los productos comprados en una misma visita)[2]. Aquí usamos un evento por posición, que es el caso más común en *clickstream* y bitácoras.

Antimonotonía. Si $\alpha \sqsubseteq \beta$, toda secuencia que contiene a β contiene a α, así que $\sigma(\beta) \leq \sigma(\alpha)$. Es la misma propiedad de los itemsets y es la que justifica la poda de GSP. Lo que cambia es el tamaño del espacio de búsqueda: con m eventos hay m^k secuencias posibles de longitud k, frente a $\binom{m}{k}$ itemsets, y las secuencias pueden ser más largas que el número de eventos distintos.

Proyección. Sea α un prefijo. Para cada $s \sqsupseteq \alpha$, el **sufijo** $s|_\alpha$ es lo que queda de s después de la aparición de α que termina más a la izquierda[3]. La **base proyectada** es $\mathcal{S}|_\alpha = \{s|_\alpha : s \in \mathcal{S},\ \alpha \sqsubseteq s\}$. El resultado que hace funcionar a PrefixSpan (Pei et al. 2004) es:

$$\sigma_{\mathcal{S}}(\alpha \cdot x) = \big|\{u \in \mathcal{S}|_\alpha : x \in u\}\big|, \tag{49.2}$$

donde $\alpha \cdot x$ agrega el evento x al final de α. Es decir, el soporte de cada extensión se obtiene contando eventos en la base proyectada, que es más pequeña que la original y se encoge con cada paso.

Algoritmo PrefixSpan. Llamar a `PrefixSpan( , )`:

1. Contar, en la base actual, en cuántas secuencias aparece cada evento.
2. Para cada evento x con conteo $\geq \sigma_{\text{mín}}$: reportar $\alpha \cdot x$, construir $\mathcal{S}|_{\alpha \cdot x}$ y llamar recursivamente a `PrefixSpan( ·x, | ·x)`.

Tabla 49.4.: Algoritmos clásicos de patrones secuenciales.

	GSP	PrefixSpan	SPADE
Estrategia	Por niveles (anchura)	Crecimiento de patrones (profundidad)	Profundidad o anchura sobre listas de ocurrencias
Candidatos	Genera y poda	No genera	Une listas de identificadores

[2]En el caso general, una secuencia es una lista de **itemsets**, por ejemplo (impresora, papel), (cartuchos) : en la primera visita se compraron dos productos juntos y en la segunda, uno. La contención exige que cada elemento del patrón esté incluido en un elemento distinto de la secuencia, respetando el orden. PrefixSpan maneja este caso distinguiendo dos tipos de extensión: agregar un ítem al último elemento o agregar un elemento nuevo.

[3]Usar la aparición que termina más a la izquierda deja el sufijo más largo posible, así que no se pierde ninguna extensión. En $S_2 = \langle I, F, B, F, C\rangle$ con prefijo F , el sufijo es B, F, C y no C ; con el segundo se perdería la extensión F, B .

	GSP	PrefixSpan	SPADE
Recorridos de la base	Uno por nivel	Uno por proyección (sobre bases cada vez menores)	Uno, para construir la representación vertical
Restricciones de tiempo	Sí (hueco máximo, ventana)	Con adaptaciones	Con adaptaciones
Referencia	Srikant y Agrawal (1996)	Pei et al. (2001)	Zaki (2001)

GSP admite de forma natural **restricciones de tiempo**[4], como exigir que entre dos eventos no pasen más de 30 minutos. Esa flexibilidad es la razón por la que su poda usa subsecuencias contiguas, como se explica en la nota sobre la unión.

Laboratorio: la base proyectada de PrefixSpan

El laboratorio reproduce el recorrido de PrefixSpan sobre las sesiones de la Tabla 49.1. Elige un evento para extender el prefijo y observa cómo se encoge la base proyectada; las barras cuentan en cuántos sufijos aparece cada evento. Al final puedes editar las sesiones.

En la versión web, este laboratorio recorre PrefixSpan sobre las sesiones de la Tabla 49.1. Valores de referencia con conteo mínimo 3: hay 14 patrones frecuentes (5 de longitud 1, 7 de longitud 2 y 2 de longitud 3). La base proyectada sobre B tiene los sufijos F,C,P , F,C , F,C,P y F,F , con conteos F = 4, C = 3 y P = 2. Con conteo mínimo 2 hay 27 patrones, el más largo de longitud 4 (B,F,C,P e I,B,F,C); con conteo mínimo 4 quedan 5 (B, F, C, I y B,F).

Guía del laboratorio.

1. El laboratorio empieza con el prefijo B . Comprueba la Tabla 49.2: los sufijos y los conteos F = 4, C = 3 y P = 2.
2. Predice antes de hacer clic: si extiendes a B, F , ¿cuántas extensiones frecuentes quedarán? Comprueba y sigue hasta que la rama termine. ¿Por qué el sufijo de S_2 queda vacío en B, F, C ?
3. Ve a F con la lista de patrones. En S_2, ¿por qué el sufijo es B, F, C y no C ? (Pista: aparición más a la izquierda.)
4. Baja el conteo mínimo a 2. ¿Cuántos patrones hay ahora y cuál es el más largo? Sube a 4: ¿qué patrón de longitud 2 sobrevive?
5. Rómpelo: cambia S_4 a `IBFC`. Antes de mirar, ¿qué patrones nuevos deberían volverse frecuentes con conteo mínimo 3? (Deberías encontrar tres, y uno de ellos tiene longitud 4.) ¿Cuál es ahora el soporte de B, F, C ?

[4]Las restricciones más comunes son el **hueco mínimo y máximo** entre eventos consecutivos del patrón, la **ventana deslizante** (eventos cercanos en el tiempo se tratan como simultáneos) y la **duración máxima** del patrón completo. Todas reducen el número de patrones y los hacen más interpretables.

Implementación en Python

Contención y soporte

El truco de `contiene` es que `e in it` avanza el iterador hasta encontrar `e`; la siguiente búsqueda empieza donde terminó la anterior, así que el orden se respeta sin manejar índices.

PrefixSpan desde cero

La implementación sigue literalmente el algoritmo: contar eventos en la base actual, y para cada evento frecuente reportar el patrón, proyectar y llamar recursivamente. El parámetro `traza` imprime el recorrido para compararlo con las tablas de la sección manual.

La sangría de la traza es el árbol de búsqueda: cada línea es un prefijo frecuente y, a su derecha, su base proyectada. Las secuencias vacías se descartan antes de la llamada recursiva porque no pueden aportar extensiones.

GSP desde cero, para comparar

	nivel	candidatos contados	frecuentes
0	1	5	5
1	2	25	7
2	3	4	2
3	4	0	0

Los dos algoritmos son **exactos**: cambian el camino, no la respuesta. GSP cuenta 34 candidatos con un recorrido completo de la base por nivel; PrefixSpan solo cuenta eventos que efectivamente aparecen en cada base proyectada.

Patrones maximales y cerrados

Las definiciones del capítulo de itemsets cerrados y maximales se trasladan sin cambios, reemplazando $\subseteq$ por $\sqsubseteq$: un patrón es **cerrado** si ninguna supersecuencia tiene el mismo soporte y **maximal** si ninguna supersecuencia es frecuente (Yan et al. 2003).

	patrón	conteo	cerrado	maximal
2	B, F	4	True	False
4	C	4	True	False
8	I	4	True	False
3	B, F, C	3	True	True
5	C, P	3	True	True
10	I, B, F	3	True	True
11	I, C	3	True	True

Como en los itemsets, basta revisar las supersecuencias con un evento más: si una supersecuencia lejana tuviera el mismo soporte, también lo tendrían todas las intermedias.

Ejemplo práctico: clickstream de una tienda en línea

Simulamos 2 000 sesiones de una tienda en línea con una cadena de Markov[5]: desde cada página, la siguiente se elige con probabilidades fijas, y en cualquier momento el visitante puede salir. La pregunta de negocio es **qué recorridos son frecuentes y cuáles se asocian con una compra**.

Con un soporte mínimo de 5 % (100 sesiones), PrefixSpan encuentra todos los recorridos frecuentes en una fracción de segundo. Como todas las sesiones empiezan en «inicio», cada patrón aparece también con «inicio» delante y con el mismo soporte: es exactamente la redundancia que eliminan los patrones **cerrados**.

Para cada patrón cerrado de longitud 2 o más que no incluye el pago, calculamos la **tasa de conversión** de las sesiones que lo contienen y la comparamos con la conversión global. Es la misma lógica del lift de los capítulos de reglas, aplicada a la regla «recorrido compra».

Figura 49.1.

Tabla 49.5.: Recorridos cerrados con mayor lift sobre la conversión (soporte 5 %).

	patrón	soporte	conversión	longitud	lift
30	inicio → ficha → ficha → carrito	0.138	0.625	4	2.472
27	inicio → ficha → carrito	0.406	0.622	3	2.460
8	inicio → búsqueda → ficha → ficha → carrito	0.086	0.614	5	2.427
36	inicio → ficha → reseñas → carrito	0.159	0.607	4	2.399
19	inicio → categoría → ficha → carrito	0.164	0.605	4	2.391
6	inicio → búsqueda → ficha → carrito	0.248	0.602	4	2.380
21	inicio → categoría → ficha → ficha → carrito	0.054	0.589	5	2.327
23	inicio → categoría → ficha → reseñas → carrito	0.069	0.587	5	2.320

Dos advertencias al leer estos resultados. Primero, son **descriptivos**: que las sesiones que pasan por el carrito conviertan más no significa que llevar a alguien al carrito cause la compra; en buena parte, quien ya decidió comprar es quien llega al carrito. Segundo, en datos reales el soporte mínimo y la definición de sesión (¿30 minutos de inactividad?, ¿un día?) cambian por completo los patrones, igual que la definición de transacción en los capítulos anteriores.

En producción no hace falta la implementación propia: `pyspark.ml.fpm.PrefixSpan` ejecuta el mismo algoritmo de forma distribuida en Spark, y bibliotecas como SPMF reúnen decenas de variantes (Fournier-Viger et al. 2017).

[5]Una **cadena de Markov** es un proceso en el que la probabilidad del siguiente estado depende solo del estado actual, no de la historia completa. Aquí, la probabilidad de ir a «carrito» depende solo de la página en la que está el visitante. Es un modelo simple que basta para generar recorridos realistas.

Ventajas y limitaciones

Tabla 49.6.: Resumen de la minería de patrones secuenciales.

Aspecto	Patrones secuenciales
Qué aportan	Orden y recorridos, no solo coocurrencia
Espacio de búsqueda	Mucho mayor que el de itemsets (m^k frente a $\binom{m}{k}$)
Algoritmo recomendado	PrefixSpan (o SPADE) en general; GSP si hay restricciones de tiempo
Riesgo principal	Explosión de patrones redundantes: conviene reportar cerrados o maximales
Decisiones críticas	Definición de sesión, granularidad del evento, soporte mínimo
Interpretación	Descriptiva; para predecir el siguiente paso se usan modelos secuenciales (cadenas de Markov, redes recurrentes)

Parte XXV.

De Patrones a Clasificación

50. De Patrones a Predicciones: Introducción a la Clasificación con Reglas de Asociación

Hasta ahora, las reglas de asociación han servido para **describir**: qué productos se compran juntos y qué tan sorprendente es esa coincidencia. Este capítulo da el paso que conecta esta parte del libro con la siguiente: usar reglas para **predecir**. Si una de las columnas de los datos es una etiqueta (¿el cliente aceptó la oferta?, ¿la transacción es fraude?, ¿se jugó el partido?), basta restringir las reglas a las que tienen esa etiqueta como consecuente para obtener un **clasificador**. El método, llamado CBA (*Classification Based on Associations*), fue propuesto por Liu et al. (1998) y es una introducción natural a los algoritmos de clasificación: cada decisión que toma se puede leer como una regla «si… entonces…».

En el camino aparecen las ideas que se repetirán en todos los clasificadores de la parte siguiente: variables predictoras y variable objetivo, ajuste con datos de entrenamiento, predicción sobre casos nuevos y, sobre todo, **evaluación honesta** con datos que el modelo no vio.

El problema de clasificación

Un problema de clasificación tiene dos ingredientes:

- **Variables predictoras** (*features*) $\mathbf{x} = (x_1, \dots, x_p)$: lo que sabemos de cada caso.
- **Variable objetivo** (*target*) y, que toma valores en un conjunto finito de **clases**, por ejemplo $\{\text{sí}, \text{no}\}$.

Un **clasificador** es una función f que asigna una clase a cada vector de predictoras, $\hat{y} = f(\mathbf{x})$. Se construye (se *entrena*) con ejemplos etiquetados $(\mathbf{x}_j, y_j)$ y se juzga por su capacidad de acertar en ejemplos **nuevos**[1]. Los árboles de decisión, la regresión logística o las redes neuronales que se estudian más adelante son distintas formas de construir f. CBA construye f como una **lista ordenada de reglas**.

Intuición: catorce días de tenis

Usaremos un conjunto clásico de la literatura de aprendizaje automático (Mitchell 1997): durante 14 días se registró el estado del tiempo y si se jugó un partido de tenis.

[1] La capacidad de acertar en casos no vistos se llama **generalización**. Un modelo puede memorizar los datos de entrenamiento y aun así generalizar mal.

Tabla 50.1.: Conjunto «jugar tenis» (14 días, 9 «sí» y 5 «no»).

Día	Cielo	Temperatura	Humedad	Viento	¿Jugar?
D1	soleado	calor	alta	débil	no
D2	soleado	calor	alta	fuerte	no
D3	nublado	calor	alta	débil	sí
D4	lluvioso	templado	alta	débil	sí
D5	lluvioso	frío	normal	débil	sí
D6	lluvioso	frío	normal	fuerte	no
D7	nublado	frío	normal	fuerte	sí
D8	soleado	templado	alta	débil	no
D9	soleado	frío	normal	débil	sí
D10	lluvioso	templado	normal	débil	sí
D11	soleado	templado	normal	fuerte	sí
D12	nublado	templado	alta	fuerte	sí
D13	nublado	calor	normal	débil	sí
D14	lluvioso	templado	alta	fuerte	no

Mirando la tabla sin fórmulas ya aparecen regularidades: **todos** los días nublados se jugó; los días soleados con humedad alta, nunca; los días lluviosos dependen del viento. Esas frases son reglas de asociación cuyo consecuente es la clase. El trabajo del capítulo es encontrarlas de forma sistemática, decidir cuáles conservar y medir qué tan bien predicen.

Antes de construir nada conviene fijar la **referencia mínima** (*baseline*): un clasificador que siempre responde la clase más frecuente, «sí», acierta $9/14 \approx 0.643$ de las veces. Cualquier modelo que valga la pena debe superar ese número.

Construcción paso a paso

Paso 1. Cada fila es una transacción

Convertimos cada día en un conjunto de ítems de la forma `atributo=valor`. El día D1 se vuelve la transacción

$$T_1 = \{\text{cielo=soleado}, \text{temp=calor}, \text{humedad=alta}, \text{viento=débil}\}, \qquad y_1 = \text{no}.$$

Es la misma codificación binaria del capítulo introductorio: diez ítems posibles (tres valores de cielo, tres de temperatura, dos de humedad y dos de viento).

Paso 2. Reglas de clase

Una **regla de clase** (*class association rule*, CAR) es una regla $X \Rightarrow c$ donde X es un itemset de predictoras y c es una clase. Sus métricas son las de siempre, contando solo filas:

- **Cobertura** de X: número de días que cumplen todas las condiciones de X.
- **Conteo** de la regla: días que cumplen X **y** tienen clase c (el soporte de $X \cup \{c\}$).

- **Confianza**: conteo / cobertura, es decir, $P(c \mid X)$ estimada.

Tres reglas calculadas a mano:

Tabla 50.2.: Tres reglas de clase calculadas sobre la Tabla 50.1.

Regla	Cobertura	Conteo	Confianza
cielo=nublado sí	4 (D3, D7, D12, D13)	4	4/4 = 1.00
humedad=alta no	7 (D1, D2, D3, D4, D8, D12, D14)	4	4/7 = 0.57
cielo=soleado humedad=alta no	3 (D1, D2, D8)	3	3/3 = 1.00

Con umbrales de **conteo mínimo 2** y **confianza mínima 0.80**, la primera y la tercera se conservan; la segunda no: la humedad alta sola no basta para decidir. Al repetir el cálculo para todos los itemsets de una, dos, tres y cuatro condiciones (con a lo más una condición por atributo) quedan **21 reglas de clase**. Su búsqueda es exactamente un Apriori por niveles en el que, además, se cuenta la clase.

Paso 3. Ordenar y seleccionar por cobertura

Las 21 reglas se solapan y a veces se contradicen, así que CBA las **ordena por precedencia**: primero mayor confianza, luego mayor conteo, luego menos condiciones (más general). Después recorre la lista y **cubre** los datos de entrenamiento:

1. Tomar la siguiente regla. Si clasifica bien al menos un caso que aún no está cubierto, se **selecciona** y se retiran todos los casos que cumple.
2. Tras cada selección se calcula la **clase por defecto**: la mayoritaria entre los casos que quedan.
3. Se registra el error total: errores de las reglas seleccionadas más los errores de la clase por defecto sobre lo que queda.
4. Al final se corta la lista en el punto de **menor error total**.

En nuestro ejemplo:

Tabla 50.3.: Selección por cobertura. En la fila 1 los diez días restantes se reparten 5 y 5; el empate se resuelve con la primera clase que aparece (D1, «no»).

#	Regla seleccionada	Retira	Quedan	Por defecto	Error total
1	cielo=nublado sí	D3, D7, D12, D13	10	no (empate 5–5)	5
2	humedad=normal viento=débil sí	D5, D9, D10	7	no	2
3	cielo=lluvioso viento=débil sí	D4	6	no	1
4	cielo=soleado humedad=alta no	D1, D2, D8	3	no	1
5	cielo=lluvioso viento=fuerte no	D6, D14	1	sí	0

El clasificador final es la lista de cinco reglas más la clase por defecto «sí». Observa que tiene la misma estructura que un árbol de decisión: primero mira el cielo y, según el caso, la humedad o el viento.

Paso 4. Predecir

Para un día nuevo se recorre la lista y se aplica **la primera regla que se cumple**; si ninguna se cumple, se responde la clase por defecto. Un día *soleado, frío, humedad alta, viento fuerte* no es nublado (regla 1), no tiene humedad normal (regla 2), no es lluvioso (regla 3) y sí cumple la regla 4: la predicción es **no**.

Paso 5. Evaluar sin hacer trampa

Sobre los 14 días de entrenamiento el clasificador acierta los 14. Ese 100 % es **optimista**: las reglas se eligieron precisamente para acertar en esos días[2]. Para estimar el desempeño en días nuevos usamos **validación cruzada dejando uno fuera** (LOOCV)[3]: se entrena 14 veces con 13 días y se predice el día que quedó fuera.

El resultado es de 12 aciertos en 14, una exactitud de 0.857, frente al 0.643 de la referencia. Los dos errores son instructivos:

- **D6** (lluvioso, frío, normal, fuerte; «no»). Sin D6, la regla *humedad=normal* *sí* tiene confianza 6/6 y gana la precedencia: el día se predice «sí».
- **D11** (soleado, templado, normal, fuerte; «sí»). Sin D11, ninguna regla con conteo 2 lo cubre y la clase por defecto es «no».

La **matriz de confusión**[4] de la validación cruzada resume los aciertos por clase:

Tabla 50.4.: Matriz de confusión de LOOCV con conteo mínimo 2 y confianza mínima 0.80.

	Predicho sí	Predicho no
Real sí (9)	8	1
Real no (5)	1	4

[2]Este fenómeno se llama **sobreajuste** (*overfitting*): el modelo se adapta a particularidades, incluso al ruido, de la muestra de entrenamiento. Su síntoma es una brecha grande entre el desempeño en entrenamiento y en datos nuevos.

[3]En la **validación cruzada de k pliegues** los datos se dividen en k partes; se entrena con $k-1$ y se evalúa con la restante, rotando. **Dejar uno fuera** (*leave-one-out*) es el caso $k = n$. Es útil con muy pocos datos porque aprovecha casi toda la muestra para entrenar, aunque su estimación puede ser variable.

[4]La **matriz de confusión** cruza la clase real (filas) con la predicha (columnas). La exactitud es la suma de la diagonal entre el total; otras métricas, como la precisión y la sensibilidad, se calculan con sus celdas y se estudian en la parte de clasificación.

Formalización

Sea $\mathcal{D} = \{(T_j, y_j)\}_{j=1}^{n}$ con T_j un conjunto de ítems `atributo=valor` y $y_j \in \mathcal{C}$. Para un itemset X y una clase c:

$$\operatorname{cob}(X) = |\{j : X \subseteq T_j\}|, \qquad \sigma(X \Rightarrow c) = |\{j : X \subseteq T_j,\ y_j = c\}|, \qquad \operatorname{conf}(X \Rightarrow c) = \frac{\sigma(X \Rightarrow c)}{\operatorname{cob}(X)}. \tag{50.1}$$

El conjunto de reglas candidatas es $\mathcal{R} = \{X \Rightarrow c : \sigma(X \Rightarrow c) \geq s_{\text{mín}},\ \operatorname{conf}(X \Rightarrow c) \geq c_{\text{mín}}\}$. Como $\sigma(X \Rightarrow c) \leq \operatorname{cob}(X)$ y la cobertura es antimonótona, se puede minar por niveles descartando todo itemset con $\operatorname{cob}(X) < s_{\text{mín}}$, igual que en Apriori.

La **precedencia** $r_a \succ r_b$ se define por, en este orden: mayor confianza, mayor conteo, menos condiciones (y, para que el resultado sea reproducible, un desempate final alfabético). El clasificador es una lista de decisión[5]

$$f(\mathbf{x}) = \begin{cases} c_1 & \text{si } X_1 \subseteq T(\mathbf{x}) \\ c_2 & \text{si no, y si } X_2 \subseteq T(\mathbf{x}) \\ \vdots & \\ c_{\text{def}} & \text{si ninguna regla se cumple.} \end{cases}$$

La confianza de la regla que dispara es una estimación de $P(y = c \mid X)$; por eso CBA puede entenderse como un clasificador que, para cada caso, elige la región más «pura» que lo contiene.

Laboratorio: construye y evalúa un clasificador CBA

El laboratorio repite todos los pasos con los datos de la Tabla 50.1. El estado inicial (conteo mínimo 2, confianza mínima 0.80) reproduce las 21 reglas, el clasificador de cinco reglas y la exactitud de validación cruzada de 12/14.

En la versión web, este laboratorio construye el clasificador CBA con umbrales ajustables. Valores de referencia: con conteo mínimo 2 y confianza mínima 0.80 hay 21 reglas de clase, el clasificador usa 5 reglas con clase por defecto «sí», acierta 14/14 en entrenamiento y 12/14 en validación cruzada. Con conteo mínimo 3 quedan 5 reglas de clase y la validación cruzada baja a 10/14; con conteo mínimo 4, a 8/14, por debajo de la referencia de 9/14.

Guía del laboratorio.

1. Comprueba el estado inicial contra la Tabla 50.3: las cinco reglas, la clase por defecto y el punto de corte en la curva de error.
2. Predice antes de mover: si subes el conteo mínimo a 3, ¿la exactitud de entrenamiento subirá o bajará? ¿Y la de validación cruzada? Compruébalo.
3. Sube el conteo mínimo a 4. ¿Por qué el clasificador ya no supera a la referencia de 9/14?
4. Con los umbrales iniciales, clasifica el día *soleado, frío, alta, fuerte* y luego cambia la humedad a *normal*. ¿Qué regla dispara en cada caso?
5. Explica en una frase por qué la exactitud de entrenamiento no sirve para elegir los umbrales.

[5] Una **lista de decisión** es una secuencia ordenada de reglas «si condición entonces clase» que se evalúan en orden hasta que una se cumple. A diferencia de un conjunto de reglas sin orden, nunca hay conflicto: decide la primera.

Implementación en Python

Las cuatro funciones siguientes implementan CBA completo. `minar_cars` es un Apriori por niveles que además cuenta la clase; `construir_cba` aplica la selección por cobertura; `predecir` recorre la lista de decisión.

Aplicado a la Tabla 50.1:

	regla	quedan	por defecto	error total
0	cielo=nublado sí	10	no	5
1	humedad=normal viento=débil sí	7	no	2
2	cielo=lluvioso viento=débil sí	6	no	1
3	cielo=soleado humedad=alta no	3	no	1
4	cielo=lluvioso viento=fuerte no	1	sí	0
5	cielo=soleado humedad=normal sí	0	sí	0

	pred. sí	pred. no
real sí	8	1
real no	1	4

Para comparar, entrenamos un árbol de decisión sobre las mismas variables codificadas en binario. Su primera pregunta es también si el cielo está nublado; después usa la humedad y el viento. Si sigues cada rama hasta su hoja verás que delimita las mismas regiones que las cinco reglas de CBA: humedad normal con viento débil sí, soleado con humedad alta no, lluvioso con viento fuerte no, y así sucesivamente.

La diferencia está en cómo se buscan las reglas: el árbol **parte** los datos de forma voraz, una pregunta a la vez, y cada caso cae en exactamente una hoja; CBA **enumera** todas las reglas frecuentes y confiables y luego elige una lista. Por eso CBA puede encontrar combinaciones que un árbol voraz no explora, a costa de minar muchas más reglas.

Ejemplo práctico: especies de pingüinos

Ahora un conjunto real y algo más grande: 333 pingüinos de tres especies (Adelie, Chinstrap y Gentoo) con medidas del pico, la aleta y la masa corporal, además de la isla y el sexo. El flujo es el de cualquier proyecto de clasificación:

1. Separar entrenamiento y prueba **antes** de transformar nada.
2. Discretizar las variables numéricas en tres niveles (*bajo*, *medio*, *alto*) con los terciles **del entrenamiento**, para no filtrar información del conjunto de prueba[6].
3. Convertir cada fila en una transacción `atributo=nivel`.

[6] La **fuga de datos** (*data leakage*) ocurre cuando información del conjunto de prueba influye en el entrenamiento, por ejemplo al escalar con la media de todos los datos. El resultado es una evaluación optimista que no se sostiene en producción.

4. Minar reglas de clase, construir el clasificador y evaluarlo en prueba.

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
corte bajo|medio	40.6	16.3	193.0	3733.3
corte medio|alto	46.8	18.5	210.0	4533.3

Con 233 casos de entrenamiento, un soporte mínimo del 5 % equivale a 12 pingüinos. Limitamos las reglas a tres condiciones para mantenerlas legibles.

	#	regla	conteo	confianza
0	1	bill_length_mm=bajo Adelie	79	1.0
1	2	flipper_length_mm=alto island=Biscoe Gentoo	68	1.0
2	3	bill_depth_mm=bajo body_mass_g=alto Gentoo	64	1.0
3	4	island=Torgersen Adelie	37	1.0
4	5	bill_depth_mm=bajo bill_length_mm=medio Ge...	36	1.0
5	6	bill_length_mm=alto island=Dream Chinstrap	33	1.0
6	7	flipper_length_mm=bajo island=Biscoe Adelie	19	1.0
7	8	bill_length_mm=medio body_mass_g=medio sex...	16	1.0
8	9	bill_depth_mm=alto island=Biscoe Adelie	15	1.0
9	10	bill_length_mm=medio island=Dream sex=FEMA...	15	1.0

Las reglas son legibles para un biólogo: un pico corto es casi exclusivo de Adelie; aletas largas en la isla Biscoe identifican a Gentoo; un pico largo en la isla Dream, a Chinstrap. Ahora la evaluación en el conjunto de prueba, comparando con la referencia mayoritaria y con un árbol de decisión de profundidad 3 entrenado sobre exactamente las mismas variables discretizadas:

Figura 50.1.

En este ejemplo CBA acierta alrededor del 98 % de los pingüinos de prueba con diez reglas legibles, frente a cerca del 93 % del árbol poco profundo y el 44 % de la referencia. No conviene generalizar ese orden: con otras particiones o umbrales los resultados cambian, y en la parte de clasificación se estudia cómo compararlos con validación cruzada. Lo importante es que **un conjunto pequeño de patrones frecuentes y confiables puede ser un clasificador competitivo e interpretable**.

Ventajas, limitaciones y extensiones

Ventajas. Las reglas son legibles y auditables, algo valioso en crédito, salud o cumplimiento normativo. El método reutiliza la maquinaria de minado de patrones, maneja bien variables categóricas y encuentra interacciones entre condiciones sin especificarlas de antemano.

Limitaciones.

- Las variables numéricas deben **discretizarse**, y el resultado depende de los cortes.

- El número de reglas candidatas crece rápido con el número de atributos; hay que limitar la longitud y subir el soporte mínimo.
- Una sola regla decide cada caso. Si la regla de mayor precedencia se apoya en pocos casos, la predicción es frágil, como ocurrió con D6.
- Con clases desbalanceadas, un soporte mínimo global puede impedir reglas para la clase minoritaria; una solución es fijar un soporte mínimo por clase.

Extensiones. CMAR (Li et al. 2001) no se queda con la primera regla: combina todas las reglas que se cumplen mediante un voto ponderado por χ^2, lo que reduce la fragilidad anterior. Esta idea, combinar muchos clasificadores débiles en lugar de confiar en uno, reaparecerá en los métodos de ensamble de la parte de clasificación.

Resumen

- Un clasificador asigna una clase a cada caso a partir de sus variables predictoras; se entrena con ejemplos etiquetados y se evalúa con casos que no vio.
- CBA restringe las reglas de asociación a las que tienen la clase como consecuente, las ordena por confianza, conteo y generalidad, y selecciona una lista mínima por cobertura.
- La predicción es la clase de la primera regla que se cumple, o la clase por defecto.
- La exactitud de entrenamiento es optimista; la validación cruzada y un conjunto de prueba dan estimaciones honestas, que siempre deben compararse con la referencia de la clase mayoritaria.

El siguiente paso natural es la parte de **Clasificación**, que empieza por formalizar el problema (fronteras de decisión, error de Bayes y sobreajuste), presenta los vecinos más cercanos y las métricas de evaluación, y después estudia los árboles de decisión: otro modo de construir reglas «si… entonces…», esta vez partiendo los datos en lugar de enumerar patrones.

Parte XXVI.

Clasificación

Parte XXVII.

Fundamentos de Clasificación

51. El problema de clasificación

En el ciclo CRISP-DM, la clasificación pertenece a la fase de **modelado**: ya entendimos el negocio, exploramos los datos y los preparamos; ahora queremos una regla que, a partir de lo que sabemos de un caso, anticipe a qué grupo pertenece. El capítulo anterior, sobre clasificación con reglas de asociación, mostró un primer ejemplo de esa idea. Esta parte del libro la desarrolla con las familias de clasificadores más usadas: vecinos cercanos, árboles y ensambles, modelos lineales, clasificadores probabilísticos y redes neuronales.

Antes de estudiar cualquiera de ellos conviene fijar el vocabulario y, sobre todo, la pregunta que los une: **¿cómo sabemos que un clasificador aprendió algo útil y no solo memorizó los datos?** Este capítulo responde con un ejemplo de diez clientes que se puede calcular a mano y con un laboratorio en el que el lector controla la complejidad de un clasificador y observa cuándo deja de generalizar.

El problema de clasificación

Un problema de clasificación tiene dos ingredientes:

- **Variables predictoras** (*features*) $\mathbf{x} = (x_1, \dots, x_p)$: lo que sabemos de cada caso antes de decidir.
- **Variable objetivo** (*target*) y, que toma valores en un conjunto **finito** de clases $\mathcal{C} = \{c_1, \dots, c_K\}$.

Un **clasificador** es una función f que asigna una clase a cada vector de predictoras, $\hat{y} = f(\mathbf{x})$. Se construye con un conjunto de **entrenamiento** de ejemplos etiquetados $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ y se juzga por su desempeño en ejemplos **nuevos**.

Tabla 51.1.: Ejemplos de problemas de clasificación.

Dominio	Predictoras $\mathbf{x}$	Objetivo y
Crédito	ingreso, endeudamiento, historial	paga / no paga
Salud	edad, presión, glucosa	diabetes sí / no
Correo	frecuencia de palabras, remitente	spam / no spam
Botánica	largo y ancho de pétalos	*setosa*, *versicolor*, *virginica*
Imágenes	intensidad de cada píxel	dígito 0, 1, …, 9

Tres distinciones aparecerán a lo largo de la parte:

- **Binaria o multiclase.** Con $K = 2$ clases el problema es binario y suele hablarse de clase **positiva** (la que interesa detectar: fraude, enfermedad, impago) y **negativa**. Con $K > 2$ es multiclase. Cuando cada caso puede tener varias etiquetas a la vez (una película que es comedia *y* drama) el problema es **multietiqueta**, que no tratamos aquí.
- **Clasificación o regresión.** Si y es numérica y continua (precio, temperatura) el problema es de **regresión**. Varios métodos de esta parte, como los árboles y los vecinos cercanos, sirven para ambas tareas.
- **Clase o puntaje.** Muchos clasificadores no entregan solo una clase, sino un **puntaje** o una **probabilidad estimada** $\hat{p}(y = c \mid \mathbf{x})$. La clase se obtiene después, comparando ese puntaje con un **umbral**. Separar ambas etapas será clave en el capítulo de métricas.

Intuición: ¿pagará el préstamo?

Una financiera registró a diez clientes a los que prestó dinero. De cada uno conoce su **ingreso** mensual (miles de pesos) y su **endeudamiento** (porcentaje del ingreso comprometido en deudas), y si **pagó** el préstamo.

Tabla 51.2.: Conjunto de entrenamiento: diez clientes, seis pagaron y cuatro no.

Cliente	Ingreso	Endeudamiento (%)	¿Pagó?
1	20	20	sí
2	25	30	sí
3	32	25	sí
4	38	35	sí
5	15	40	sí
6	30	58	sí
7	12	55	no
8	22	60	no
9	35	62	no
10	28	48	no

Sin fórmulas ya se ve un patrón: quien compromete menos del 45 % de su ingreso tiende a pagar. El ingreso parece importar poco. Hay una **excepción**: el cliente 6 pagó con un endeudamiento del 58 %. ¿Es un caso que debe aprenderse o es ruido, un cliente que pagó por razones que los datos no registran (una herencia, un aval)? Esa pregunta, aprender la regla general sin aprender las excepciones, es el corazón de este capítulo.

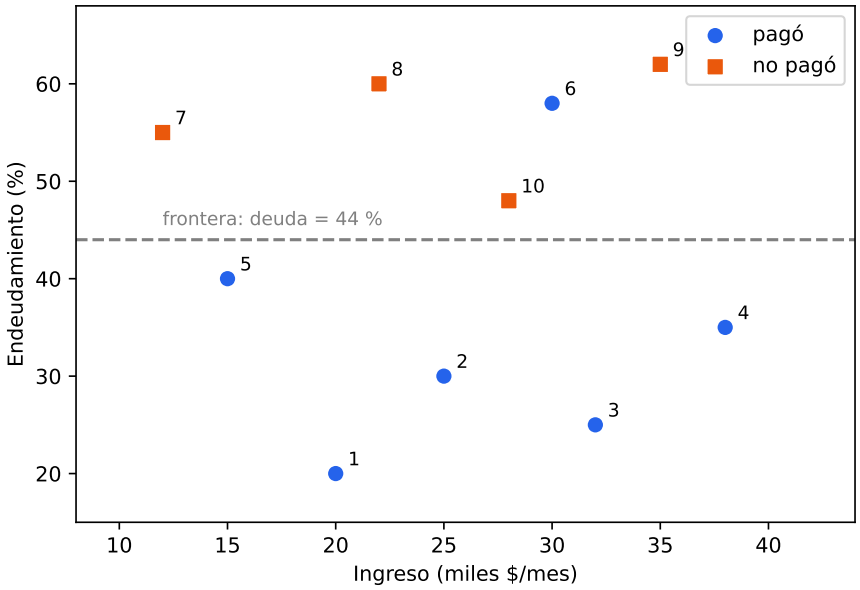


Figura 51.1.: Los diez clientes de entrenamiento. La línea discontinua es la frontera de la regla «endeudamiento 44 % paga» que se construye en el Paso 2. El cliente 6 queda del lado equivocado.

Construcción paso a paso

Paso 1. La referencia mínima

El clasificador más simple ignora las predictoras y responde siempre la clase más frecuente. Aquí, «sí paga», con una exactitud de $6/10 = 0.60$. A este clasificador se le llama **referencia** (*baseline*) y todo modelo debe superarlo; si no, no aprendió nada de las predictoras[1].

Paso 2. Una regla de un solo corte

Busquemos la mejor regla de la forma «si el endeudamiento es a lo más t, entonces paga». Ordenamos los valores de endeudamiento y probamos un corte t en el punto medio entre cada par de valores consecutivos:

Tabla 51.3.: Errores de entrenamiento de la regla «deuda t paga» para algunos cortes.

Corte t	Clasificados «paga»	Errores	Quiénes fallan
32.5	1, 2, 3	3	4, 5, 6
37.5	1, 2, 3, 4	2	5, 6
44.0	**1, 2, 3, 4, 5**	**1**	**6**
51.5	1–5 y 10	2	6, 10

[1]Con clases muy desbalanceadas la referencia es engañosamente alta: si el 97 % de las transacciones son legítimas, «nunca hay fraude» acierta el 97 %. Por eso el capítulo de métricas introduce medidas que no se dejan engañar por el desbalance.

Corte t	Clasificados «paga»	Errores	Quiénes fallan
59.0	1–7 y 10	2	7, 10

El mejor corte es $t = 44$ (punto medio entre 40 y 48), con un solo error: el cliente 6. Haciendo lo mismo con el ingreso, el mejor corte comete 3 errores, así que el endeudamiento es la variable más informativa. Esta regla de una sola pregunta, llamada **tocón de decisión** (*decision stump*), acierta $9/10 = 0.90$ en entrenamiento.

Geométricamente, la regla divide el plano en dos **regiones de decisión** separadas por la recta horizontal deuda = 44: la **frontera de decisión** de la Figura 51.1.

Paso 3. Memorizar: el vecino más cercano

Otra estrategia es no resumir nada: guardar los diez clientes y, ante uno nuevo, responder lo mismo que el cliente **más parecido**, medido con la distancia euclidiana[2]. Este es el clasificador del **vecino más cercano** (1-NN).

En entrenamiento acierta 10/10 por construcción: el vecino más cercano de cada cliente es él mismo. Pero ¿qué responde ante un cliente nuevo con ingreso 31 y endeudamiento 60? Calculamos su distancia a los tres clientes más próximos:

$$
\begin{aligned}
d(\text{nuevo}, 6) &= \sqrt{(31-30)^2 + (60-58)^2} = \sqrt{5} \approx 2.24 \quad \text{(pagó)},\\
d(\text{nuevo}, 9) &= \sqrt{(31-35)^2 + (60-62)^2} = \sqrt{20} \approx 4.47 \quad \text{(no pagó)},\\
d(\text{nuevo}, 8) &= \sqrt{(31-22)^2 + (60-60)^2} = 9.00 \quad \text{(no pagó)}.
\end{aligned}
$$

El más cercano es el cliente 6, la excepción, así que 1-NN predice «paga» para alguien con 60 % de endeudamiento. Si en lugar de un vecino consultamos a los **tres** más cercanos y votamos (3-NN), ganan dos «no» contra un «sí»: la votación diluye la excepción.

Paso 4. La prueba: cinco clientes nuevos

La financiera observa a cinco clientes que **no** se usaron para construir las reglas:

Tabla 51.4.: Predicciones sobre el conjunto de prueba.

Cliente	Ingreso	Deuda (%)	Real	Referencia	Corte 44	1-NN	3-NN
A	18	35	sí	sí	sí	sí	sí
B	31	60	no	sí	no	sí	no
C	26	45	no	sí	no	no	sí
D	36	30	sí	sí	sí	sí	sí

[2]La **distancia euclidiana** entre **a** y **b** es $\sqrt{\sum_j (a_j - b_j)^2}$, la longitud del segmento que los une. Aquí las dos variables tienen rangos parecidos; cuando no es así, hay que escalarlas primero, como se explica en el capítulo de vecinos cercanos.

Cliente	Ingreso	Deuda (%)	Real	Referencia	Corte 44	1-NN	3-NN
E	40	50	sí	sí	no	no	no

Resumimos el desempeño de los cuatro clasificadores en ambos conjuntos:

Tabla 51.5.: Exactitud de entrenamiento frente a exactitud de prueba.

Clasificador	Exactitud en entrenamiento	Exactitud en prueba
Referencia (siempre «sí»)	$6/10 = 0.60$	$3/5 = 0.60$
Corte en deuda = 44	$9/10 = 0.90$	$4/5 = 0.80$
1-NN	$\mathbf{10/10 = 1.00}$	$3/5 = 0.60$
3-NN	$9/10 = 0.90$	$3/5 = 0.60$

Tres lecciones que se repetirán en toda la parte:

1. **La exactitud de entrenamiento es optimista.** El 1-NN es perfecto en los datos que memorizó y no mejor que la referencia en datos nuevos.
2. **Más flexible no es mejor.** La regla de un solo corte es la menos flexible de las tres que usan predictoras y es la que mejor generaliza aquí, porque se niega a aprender la excepción.
3. **Con pocos datos, la evaluación es ruidosa.** Cinco clientes de prueba son muy pocos: un solo caso, el E, cambia la exactitud en 0.20. Los capítulos de evaluación muestran cómo obtener estimaciones más estables.

Formalización

Pérdida, riesgo y error empírico

Para medir los errores usamos la **pérdida 0-1**, que vale 1 si la predicción falla y 0 si acierta: $L(y, \hat{y}) = \mathbb{1}[y \neq \hat{y}]$. Lo que realmente queremos minimizar es el **riesgo** o error esperado sobre la población de la que vienen los datos[3]:

$$R(f) = \mathbb{E}_{(\mathbf{x},y)}\big[\mathbb{1}[y \neq f(\mathbf{x})]\big] = P(y \neq f(\mathbf{x})). \tag{51.1}$$

Como no conocemos la población, lo aproximamos con un promedio sobre una muestra, el **error empírico**:

$$\hat{R}_{\mathcal{S}}(f) = \frac{1}{|\mathcal{S}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{S}} \mathbb{1}[y_i \neq f(\mathbf{x}_i)]. \tag{51.2}$$

[3]La **esperanza** $\mathbb{E}[\cdot]$ es el promedio de una cantidad sobre todos los casos posibles, ponderado por su probabilidad. El riesgo es, entonces, la proporción de errores que cometería el clasificador sobre la población completa, de la cual nuestros datos son solo una muestra.

Si $\mathcal{S}$ es el conjunto de entrenamiento, $\hat{R}$ es el **error de entrenamiento**; si $\mathcal{S}$ es un conjunto de prueba independiente, es el **error de prueba**. Solo el segundo es una estimación honesta de $R(f)$, porque f se eligió precisamente para que el primero fuera pequeño.

Regiones y fronteras de decisión

Todo clasificador parte el espacio de predictoras en **regiones de decisión** $\mathcal{R}_c = \{\mathbf{x} : f(\mathbf{x}) = c\}$. Las **fronteras de decisión** son los bordes entre regiones. La forma de esas fronteras distingue a las familias de clasificadores: rectas o hiperplanos en los modelos lineales, escalones paralelos a los ejes en los árboles, fronteras poligonales en el vecino más cercano y curvas suaves en las redes neuronales.

El mejor clasificador posible: el clasificador de Bayes

Supongamos que conociéramos la probabilidad de cada clase dado $\mathbf{x}$, $p(c \mid \mathbf{x}) = P(y = c \mid \mathbf{x})$[4]. Entonces la regla que minimiza el riesgo de la Ecuación 51.1 es elegir la clase más probable:

$$f^*(\mathbf{x}) = \arg\max_{c \in \mathcal{C}} p(c \mid \mathbf{x}). \tag{51.3}$$

Este es el **clasificador de Bayes**. Su riesgo, el **error de Bayes** $R^* = \mathbb{E}_{\mathbf{x}}\big[1 - \max_c p(c \mid \mathbf{x})\big]$, es el mínimo alcanzable por **cualquier** clasificador. No es cero cuando las clases se traslapan: dos clientes con el mismo ingreso y el mismo endeudamiento pueden comportarse distinto, y ninguna regla basada solo en esas dos variables puede acertar ambos. Ese error es **irreducible** con las predictoras disponibles; solo se reduce midiendo variables nuevas.

Todos los clasificadores de esta parte pueden leerse como intentos de aproximar f^* a partir de una muestra finita:

- los **vecinos cercanos** estiman $p(c \mid \mathbf{x})$ con la proporción de cada clase entre los vecinos;
- los **árboles** la estiman con la proporción de cada clase en la hoja;
- la **regresión logística** modela $p(c \mid \mathbf{x})$ directamente con una función paramétrica;
- los **clasificadores bayesianos** modelan $p(\mathbf{x} \mid c)$ y $P(c)$ y aplican el teorema de Bayes.

Los dos últimos ilustran una distinción importante: los modelos **discriminativos** aprenden la frontera o $p(c \mid \mathbf{x})$; los **generativos** aprenden cómo se generan los datos de cada clase[5].

[4] $P(y = c \mid \mathbf{x})$ es una **probabilidad condicional**: la proporción de casos de clase c entre todos los que tienen exactamente las predictoras $\mathbf{x}$. Por ejemplo, entre todos los clientes con ingreso 30 y endeudamiento 58, la fracción que paga.

[5] Un modelo **generativo** aprende la distribución conjunta $p(\mathbf{x}, y) = p(\mathbf{x} \mid y)\, P(y)$ y con ella podría simular datos nuevos de cada clase; luego clasifica con el teorema de Bayes. Un modelo **discriminativo** solo aprende lo necesario para separar las clases. Los primeros funcionan bien con pocos datos si sus supuestos son correctos; los segundos suelen ganar con muchos datos.

Generalización, sobreajuste y subajuste

Un clasificador **generaliza** cuando su error de prueba se parece a su error de entrenamiento y ambos son bajos. Hay dos maneras de fallar:

- **Subajuste** (*underfitting*): el modelo es demasiado rígido para el patrón real; ambos errores son altos. La referencia del Paso 1 es el caso extremo.
- **Sobreajuste** (*overfitting*): el modelo es tan flexible que aprende también el ruido de la muestra; el error de entrenamiento es bajo y el de prueba, alto. El 1-NN del Paso 3 es el caso extremo.

La **complejidad** (o flexibilidad) de un clasificador se controla con **hiperparámetros**[6]: el número de vecinos k, la profundidad de un árbol, la regularización de un modelo lineal. Al aumentar la complejidad, el error de entrenamiento baja casi siempre, mientras que el de prueba suele dibujar una **U**: baja mientras el modelo aprende el patrón y sube cuando empieza a aprender el ruido.

Esta U es la cara visible del **compromiso sesgo–varianza** (Geman et al. 1992). Un modelo rígido tiene **sesgo**: aunque lo entrenáramos con muchas muestras distintas, se equivocaría de forma sistemática. Un modelo flexible tiene **varianza**: con otra muestra de entrenamiento produciría una frontera muy distinta[7]. El error de prueba se descompone, de forma aproximada, en

$$\text{error de prueba} \;\approx\; \underbrace{R^{*}}_{\text{irreducible}} \;+\, \text{sesgo} \;+\; \text{varianza},$$

y elegir la complejidad es buscar el punto donde la suma es mínima. Con más datos, la varianza baja y el punto óptimo se desplaza hacia modelos más flexibles.

Paramétricos y no paramétricos

Los clasificadores **paramétricos** (regresión logística, discriminante lineal, Naive Bayes) resumen los datos en un número fijo de parámetros; después de entrenarlos se pueden desechar los datos. Los **no paramétricos** (vecinos cercanos, árboles sin límite de profundidad) crecen con los datos: el 1-NN *es* el conjunto de entrenamiento. Los primeros suelen tener más sesgo y menos varianza; los segundos, lo contrario.

Laboratorio: complejidad y generalización

El laboratorio usa el clasificador de k vecinos más cercanos (k-NN): para un punto nuevo busca los k puntos de entrenamiento más próximos y responde la clase mayoritaria entre ellos. El hiperparámetro k controla la complejidad: con $k = 1$ la frontera rodea cada punto; con k grande se vuelve suave y, en el extremo, todo el plano recibe la clase mayoritaria.

[6] Un **hiperparámetro** es una opción del algoritmo que se fija antes de entrenar (k, profundidad máxima, tasa de aprendizaje). Los **parámetros**, en cambio, los aprende el algoritmo a partir de los datos (los coeficientes de un modelo lineal, los cortes de un árbol).

[7] Formalmente, el **sesgo** mide cuánto se aleja la predicción *promedio* (sobre muchas muestras de entrenamiento posibles) de la predicción del clasificador de Bayes, y la **varianza** mide cuánto fluctúa la predicción de una muestra a otra. Para la pérdida cuadrática la descomposición es exacta; para la pérdida 0-1 es solo aproximada, pero la intuición es la misma (Hastie et al. 2009).

Tiene dos conjuntos de datos. **Diez clientes** reproduce las Tabla 51.4 y Tabla 51.5. **Muestra grande** usa 120 puntos de entrenamiento y 400 de prueba simulados a partir de una probabilidad real conocida, de modo que se puede calcular el error de Bayes (alrededor de 0.09) y compararlo con lo que logra k-NN.

En la versión web, este laboratorio muestra las regiones de decisión de k-NN y los errores de entrenamiento y prueba en función de k. Valores de referencia. *Diez clientes*: con $k = 1$, error de entrenamiento 0/10 y de prueba 2/5; con $k = 3$, 1/10 y 2/5. *Muestra grande* (120 puntos de entrenamiento, 400 de prueba, error de Bayes 0.092): con $k = 1$ el error de entrenamiento es 0 y el de prueba 0.138; con $k = 5$, 0.083 y 0.110 (el mínimo de prueba); con $k = 59$, 0.158 y 0.183.

Guía del laboratorio.

1. Con *Diez clientes* y $k = 1$, comprueba la Tabla 51.4: ¿qué vecino decide la predicción del cliente B? ¿A qué distancia está?
2. Predice antes de mover: con $k = 3$, ¿cambia la predicción de B? ¿Y la de C? Compruébalo en la tabla y explica por qué C empeora.
3. Cambia a *Muestra grande* con $k = 1$ y alterna entre *Entrenamiento* y *Prueba*. ¿Por qué no hay puntos con borde grueso en entrenamiento?
4. Recorre k de 1 a 59. Localiza la U del error de prueba y el valor de k donde se alcanza el mínimo. ¿Qué tan lejos queda del error de Bayes?
5. Con $k = 5$, baja el tamaño de entrenamiento a 20. ¿Qué le pasa a la frontera y al error de prueba? ¿Dónde está ahora el mínimo de la curva?
6. Explica con tus palabras por qué el error de entrenamiento casi siempre crece con k, mientras que el de prueba no.

El flujo de trabajo de un clasificador

Las lecciones anteriores se convierten en un procedimiento que seguiremos en todos los capítulos:

1. **Definir la clase positiva, la métrica y la referencia.** ¿Qué error es más costoso? ¿Cuánto acierta responder siempre la clase mayoritaria? Las métricas se estudian en el capítulo siguiente sobre evaluación.
2. **Separar datos de prueba antes de cualquier otra cosa.** El conjunto de prueba se guarda y se usa **una sola vez**, al final.
3. **Preparar las predictoras dentro de un *pipeline*.** Imputación, escalado y codificación se ajustan solo con el entrenamiento y se aplican después a la prueba, para evitar la **fuga de datos**[8].
4. **Elegir hiperparámetros con validación**, nunca con la prueba: se reserva una parte del entrenamiento (validación) o se usa validación cruzada.
5. **Comparar con la referencia y con un modelo simple.** Un modelo complejo debe justificar su costo frente a uno interpretable.
6. **Evaluar una sola vez en prueba y reportar** la métrica junto con su incertidumbre.

[8] La **fuga de datos** (*data leakage*) ocurre cuando información del conjunto de prueba influye en el entrenamiento, por ejemplo al escalar con la media de todos los datos. El resultado es una evaluación optimista que no se sostiene en producción.

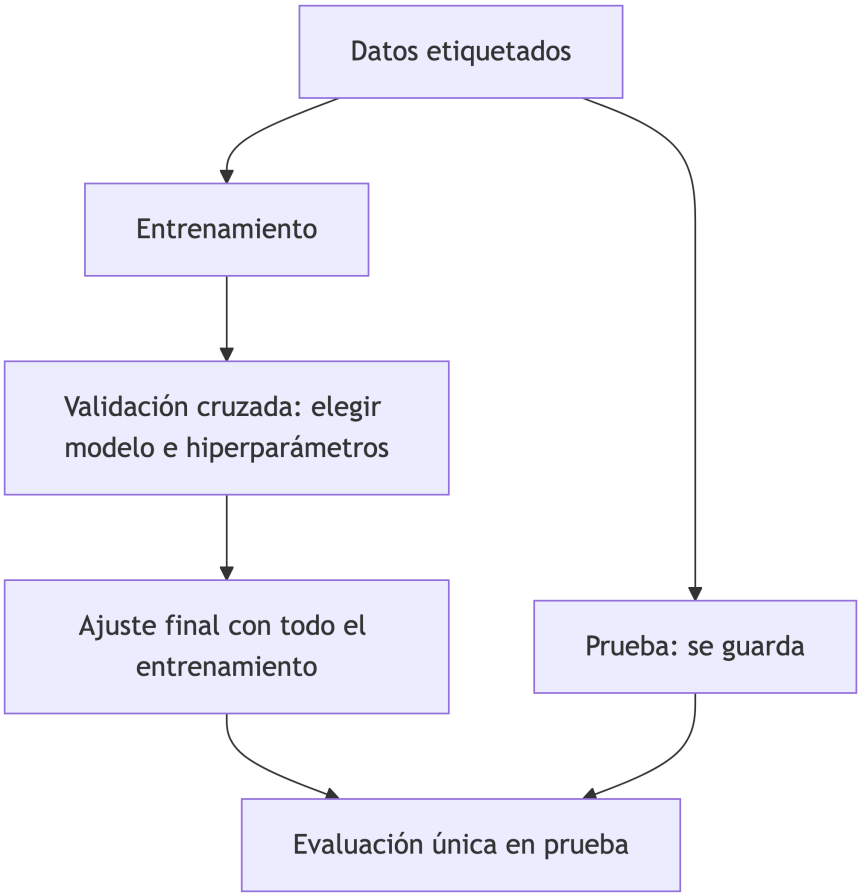


Figura 51.2.: Flujo de trabajo. El conjunto de prueba se separa al inicio y solo se usa en la evaluación final.

Un mapa de los clasificadores de esta parte

Tabla 51.6.: Familias de clasificadores estudiadas en esta parte.

Familia	Idea	Frontera típica	Interpretabilidad	Capítulo
Vecinos cercanos	Responder como los casos más parecidos	Poligonal, muy flexible	Media (por ejemplos)	Vecinos más cercanos
Árboles de decisión	Preguntas sucesivas que parten los datos	Escalones paralelos a los ejes	Alta (reglas)	Árboles (ID3), CART
Ensambles	Combinar muchos árboles	Escalones finos, casi suave	Baja (requiere SHAP)	Ensambles, XGBoost
Modelos lineales	Puntaje lineal $\mathbf{w}^\top \mathbf{x} + b$	Recta o hiperplano	Alta (coeficientes)	Lineales, logística
Máquinas de soporte vectorial	Margen máximo, núcleos	Hiperplano o curva (con núcleo)	Media	SVM
Probabilísticos	Modelar $p(\mathbf{x} \mid c)$ y aplicar Bayes	Lineal o cuadrática	Alta	Bayesianos, GMM
Redes neuronales	Composición de transformaciones no lineales	Curva arbitraria	Baja	Redes, PyTorch y JAX

No existe un clasificador que sea el mejor en todos los problemas[9]. Por eso el método importa más que el algoritmo: definir bien la métrica, validar sin fugas y comparar contra referencias.

Más de dos clases. Algunos clasificadores (árboles, vecinos, Naive Bayes, regresión logística multinomial, redes) manejan $K > 2$ clases de forma natural. Otros son binarios por diseño y se extienden con dos estrategias: **uno contra el resto** (OvR), que entrena K clasificadores «clase c contra todas las demás» y elige el de mayor puntaje, y **uno contra uno** (OvO), que entrena $K(K-1)/2$ clasificadores por pares y decide por votación. `scikit-learn` aplica estas estrategias automáticamente cuando hace falta.

Implementación en Python

La interfaz de `scikit-learn`

Todos los clasificadores de `scikit-learn` comparten la misma interfaz (Pedregosa et al. 2011): se crean con sus hiperparámetros, se ajustan con `fit(X, y)` y predicen con `predict(X)`. Muchos ofrecen además `predict_proba(X)`, la probabilidad estimada de cada clase, y `score(X, y)`, la

[9] Este resultado se conoce como el teorema de **no hay almuerzo gratis** (*no free lunch*) (Wolpert 1996): promediado sobre todos los problemas posibles, ningún algoritmo de aprendizaje supera a otro. Un algoritmo gana en un problema concreto porque sus supuestos encajan con la estructura de ese problema.

exactitud. Reproducimos la Tabla 51.5 con cuatro clasificadores: `DummyClassifier` es la referencia, un árbol de profundidad 1 es el tocón y `KNeighborsClassifier` es k-NN.

	clasificador	exactitud entrenamiento	exactitud prueba	predicciones A–E
0	Referencia (mayoritaria)	0.6	0.6	SSSSS
1	Tocón (árbol de profundidad 1)	0.9	0.8	SNNSN
2	1-NN	1.0	0.6	SSNSN
3	3-NN	0.9	0.6	SNSSN

El árbol de profundidad 1 encontró el mismo corte que calculamos a mano. Lo podemos leer directamente:

El método `predict_proba` de 3-NN devuelve la proporción de vecinos de cada clase. Para el cliente B, dos de sus tres vecinos no pagaron:

	P(no paga)	P(paga)
A	0.000	1.000
B	0.667	0.333
C	0.333	0.667
D	0.000	1.000
E	0.667	0.333

La curva de complejidad con la muestra grande

Generamos los mismos datos que usa el laboratorio. La probabilidad real de la clase 1 depende de una frontera curva, $x_2 = 5 + 2\,\mathrm{sin}(0.8\,x_1)$, y se suaviza con una función logística; así las clases se traslapan cerca de la frontera y el error de Bayes no es cero.

Figura 51.3.

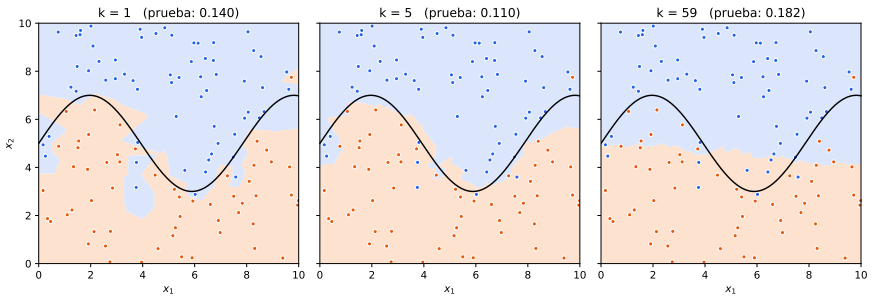


Figura 51.4.: Regiones de decisión de k-NN con k = 1, 5 y 59 sobre los 120 puntos de entrenamiento. La curva negra es la frontera del clasificador de Bayes. Con k = 1 la frontera rodea puntos aislados (sobreajuste); con k = 59 es demasiado suave (subajuste).

Los números coinciden con el laboratorio: el mínimo del error de prueba está en $k = 5$ con 0.110, todavía unos dos puntos porcentuales por encima del error de Bayes. Esa diferencia se debe a que 120 puntos son una muestra finita; con más datos, k-NN se acerca al clasificador de Bayes[10].

Resumen

- Un clasificador asigna una de K clases a cada vector de predictoras; se entrena con ejemplos etiquetados y se juzga por su error en ejemplos nuevos.
- Toda evaluación debe compararse con la **referencia** de la clase mayoritaria.
- El error de entrenamiento es optimista. El error de prueba, sobre datos que no participaron en ninguna decisión, es la estimación honesta del riesgo.
- El **clasificador de Bayes** elige la clase más probable dada $\mathbf{x}$; su error es el mínimo posible y no es cero cuando las clases se traslapan.
- La complejidad se controla con hiperparámetros. Poca complejidad produce **subajuste** (sesgo); demasiada, **sobreajuste** (varianza). El error de prueba dibuja una U.

El siguiente capítulo estudia a fondo el primer clasificador que usamos aquí, los k vecinos más cercanos: cómo medir la distancia, por qué hay que escalar las variables y qué ocurre cuando hay muchas dimensiones.

[10] Se dice que k-NN es **consistente**: si $n \to \infty$ y $k \to \infty$ con $k/n \to 0$, su error converge al error de Bayes (Stone 1977). En la práctica, la velocidad de esa convergencia empeora rápidamente con el número de variables.

52. Vecinos más cercanos

El clasificador de k **vecinos más cercanos** (k-NN, *k-nearest neighbors*) es quizá la idea más antigua y más intuitiva del aprendizaje supervisado: para decidir la clase de un caso nuevo, se buscan los casos de entrenamiento más parecidos y se responde lo que diga la mayoría de ellos. Fue propuesto por Fix y Hodges (1951) como un método de discriminación «no paramétrico» y Cover y Hart (1967) demostraron que, con suficientes datos, su error nunca es mayor que el doble del mínimo posible.

k-NN no construye una fórmula ni un árbol: **guarda los datos** y difiere todo el trabajo al momento de predecir. Por eso se le llama aprendizaje *perezoso* (*lazy learning*) o basado en instancias. Esa simplicidad lo convierte en el mejor punto de partida para entender tres ideas que valen para todos los clasificadores: la importancia de la **representación** de los datos (qué significa «parecido»), el control de la **complejidad** con un hiperparámetro y los problemas que trae la **alta dimensión**.

Intuición: ¿quién compra un seguro?

Una aseguradora ofreció un seguro de gastos médicos a ocho clientes y registró su **edad**, su **ingreso** mensual en pesos y si **compró** el seguro.

Tabla 52.1.: Clientes de entrenamiento.

Cliente	Edad	Ingreso ($)	¿Compró?
1	20	30 000	no
2	30	50 000	no
3	36	10 000	no
4	28	38 000	no
5	52	34 000	sí
6	56	60 000	sí
7	60	18 000	sí
8	46	55 000	sí

A simple vista, la **edad** separa las clases: todos los mayores de 45 compraron y ninguno de los menores. El ingreso no parece importar. Llega una clienta nueva de **54 años** con un ingreso de **31 000 pesos**. La respuesta de sentido común es «comprará». Veamos qué responde k-NN.

Construcción paso a paso

Paso 1. Distancias con las variables tal como vienen

La distancia euclidiana entre la clienta nueva $\mathbf{q} = (54, 31\,000)$ y el cliente 1 es

$$d(\mathbf{q}, \mathbf{x}_1) = \sqrt{(54-20)^2 + (31\,000 - 30\,000)^2} = \sqrt{1\,156 + 1\,000\,000} \approx 1\,000.6.$$

Repitiendo el cálculo para los ocho clientes y ordenando:

Tabla 52.2.: Los cuatro vecinos más cercanos con las variables originales.

Orden	Cliente	$(\Delta\text{edad})^2$	$(\Delta\text{ingreso})^2$	Distancia	Clase
1	1	1 156	1 000 000	1 000.6	no
2	5	4	9 000 000	3 000.0	sí
3	4	676	49 000 000	7 000.0	no
4	7	36	169 000 000	13 000.0	sí

Con $k = 3$ los vecinos son los clientes 1, 5 y 4: dos «no» contra un «sí». **k-NN predice que no comprará.** El problema está en las columnas centrales: la diferencia de edad aporta a lo más unos miles a la suma de cuadrados, mientras que la de ingreso aporta millones. La distancia es, en la práctica, la diferencia de ingresos; la edad, la única variable útil, **no cuenta**. Si el ingreso se hubiera registrado en miles de pesos la conclusión sería otra. Un clasificador cuya respuesta depende de las unidades de medida no es confiable.

Paso 2. Llevar las variables a una escala común

La solución es **escalar** las variables antes de medir distancias. Con la normalización **min-max**[1], cada variable se transforma a $[0, 1]$ con el mínimo y el máximo **del entrenamiento**:

$$z = \frac{x - \text{mín}}{\text{máx} - \text{mín}}, \qquad z_{\text{edad}} = \frac{\text{edad} - 20}{60 - 20}, \qquad z_{\text{ingreso}} = \frac{\text{ingreso} - 10\,000}{60\,000 - 10\,000}.$$

Para la clienta nueva: $z_{\text{edad}} = 34/40 = 0.85$ y $z_{\text{ingreso}} = 21\,000/50\,000 = 0.42$.

Tabla 52.3.: Variables escaladas a [0, 1].

Cliente	1	2	3	4	5	6	7	8	Nueva
z_{edad}	0.00	0.25	0.40	0.20	0.80	0.90	1.00	0.65	0.85
z_{ingreso}	0.40	0.80	0.00	0.56	0.48	1.00	0.16	0.90	0.42

[1] Otra opción es la **estandarización**, $z = (x - \bar{x})/s$, que deja cada variable con media 0 y desviación estándar 1. Min-max garantiza el rango $[0, 1]$ pero es sensible a valores atípicos; la estandarización no acota el rango pero es más estable. Ambas se estudian en el capítulo de conversión de tipos.

Paso 3. Distancias en la escala común

Ahora, por ejemplo, $d(\mathbf{q}, \mathbf{x}_5) = \sqrt{(0.85-0.80)^2 + (0.42-0.48)^2} = \sqrt{0.0025 + 0.0036} \approx 0.078$. Todas las distancias, ordenadas:

Tabla 52.4.: Distancias euclidianas en la escala [0, 1].

Orden	Cliente	Distancia	Clase	Peso $1/d$
1	5	0.078	sí	12.80
2	7	0.300	sí	3.33
3	8	0.520	sí	1.92
4	6	0.582	sí	1.72
5	3	0.616	no	1.62
6	4	0.665	no	1.50
7	2	0.710	no	1.41
8	1	0.850	no	1.18

Con $k = 3$ los vecinos son 5, 7 y 8: tres «sí». **Ahora k-NN predice que comprará**, con una proporción de votos de 3/3. Nada cambió en los datos; solo la manera de medir el parecido.

Paso 4. El efecto de k

La proporción de vecinos de cada clase es una estimación de la probabilidad de la clase:

Tabla 52.5.: Votos según el número de vecinos.

k	Votos «sí»	Votos «no»	$\hat{p}(\text{sí} \mid \mathbf{q})$	Predicción
1	1	0	1.00	sí
3	3	0	1.00	sí
5	4	1	0.80	sí
7	4	3	0.57	sí
8	4	4	0.50	empate

A medida que k crece se consultan clientes cada vez menos parecidos y la estimación se acerca a la proporción global de la clase (4/8). Con $k = n$ el clasificador ignora a la clienta y responde siempre la clase mayoritaria: es la referencia del capítulo anterior. En el otro extremo, con $k = 1$, la respuesta depende de un solo cliente, que podría ser una excepción.

Paso 5. Voto ponderado por la distancia

El voto uniforme trata igual al cliente 5, a distancia 0.078, que al cliente 3, a 0.616. Una variante natural pondera cada voto por el inverso de la distancia. Con $k = 7$:

$$\text{peso(sí)} = 12.80 + 3.33 + 1.92 + 1.72 = 19.78, \qquad \text{peso(no)} = 1.62 + 1.50 + 1.41 = 4.54,$$

de modo que $\hat{p}(\text{sí} \mid \mathbf{q}) = 19.78/(19.78+4.54) \approx 0.81$, frente al 0.57 del voto uniforme. La ponderación hace que el resultado sea menos sensible a la elección de k, porque los vecinos lejanos casi no cuentan.

Formalización

La regla de k-NN

Sea $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ el conjunto de entrenamiento y d una función de distancia. Para un punto $\mathbf{x}$, sea $N_k(\mathbf{x})$ el conjunto de índices de sus k vecinos más cercanos. La estimación de la probabilidad de la clase c y la predicción son

$$\hat{p}(c \mid \mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} \mathbb{1}[y_i = c], \qquad \hat{y}(\mathbf{x}) = \arg\max_{c} \hat{p}(c \mid \mathbf{x}). \tag{52.1}$$

Es decir, k-NN imita al clasificador de Bayes ($\arg\max_c p(c \mid \mathbf{x})$) sustituyendo la probabilidad verdadera por una proporción **local**. Con pesos $w_i = 1/d(\mathbf{x}, \mathbf{x}_i)$ la versión ponderada es

$$\hat{p}_w(c \mid \mathbf{x}) = \frac{\sum_{i \in N_k(\mathbf{x})} w_i \, \mathbb{1}[y_i = c]}{\sum_{i \in N_k(\mathbf{x})} w_i}. \tag{52.2}$$

Si $\mathbf{x}$ coincide exactamente con un punto de entrenamiento ($d = 0$), se usa solo ese punto. En **regresión**, k-NN predice el promedio (simple o ponderado) de las respuestas de los vecinos, $\hat{f}(\mathbf{x}) = \frac{1}{k}\sum_{i \in N_k(\mathbf{x})} y_i$.

Empates. En clasificación binaria se recomienda un k impar. Con varias clases pueden quedar empates aun con k impar; `scikit-learn` los resuelve eligiendo la clase que aparece primero en el orden de las etiquetas, y otra opción común es usar la clase del vecino más cercano entre las empatadas.

Distancias

La familia de **Minkowski** de orden $s \geq 1$ generaliza las distancias más usadas:

$$d_s(\mathbf{x}, \mathbf{z}) = \Big(\sum_{j=1}^{p} |x_j - z_j|^s \Big)^{1/s}. \tag{52.3}$$

Tabla 52.6.: Casos particulares de la distancia de Minkowski.

s	Nombre	Fórmula	Forma de la «bola» en 2D
1	Manhattan	$\sum_j \lvert x_j - z_j \rvert$	rombo
2	Euclidiana	$\sqrt{\sum_j (x_j - z_j)^2}$	círculo
∞	Chebyshev	$\max_j \lvert x_j - z_j \rvert$	cuadrado

La «bola» es el conjunto de puntos a distancia menor que r de un centro; para k-NN es la forma del vecindario. Con Chebyshev, la clienta nueva tendría como tercer vecino al cliente 3 (distancia $\max(0.45, 0.42) = 0.45$) en lugar del 8: la métrica cambia quiénes son los vecinos.

Otras distancias útiles:

- **Mahalanobis**, $d_M(\mathbf{x}, \mathbf{z}) = \sqrt{(\mathbf{x}-\mathbf{z})^\top \Sigma^{-1} (\mathbf{x}-\mathbf{z})}$, con Σ la matriz de covarianza: corrige a la vez escala y correlación entre variables.
- **Hamming**, la proporción de atributos categóricos distintos, para datos como los del problema de tenis.
- **Gower**, que combina variables numéricas y categóricas en una sola distancia[2].
- **Coseno**, $1 - \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\|\|\mathbf{z}\|}$, habitual para textos representados como vectores de frecuencias, donde importa la dirección y no la longitud.

Escalamiento: parte del modelo

Como mostró el Paso 1, k-NN no es invariante a las unidades. Las dos opciones más comunes son la normalización **min-max** y la **estandarización** $z = (x - \bar{x})/s$. En ambos casos los parámetros (mín, máx, $\bar{x}$, s) se calculan **solo con el entrenamiento** y se aplican a los puntos nuevos; por eso el escalador forma parte del modelo y debe ir dentro de un *pipeline*. El capítulo de conversión de tipos y escalamiento tiene un laboratorio dedicado a este efecto.

Escalar es una forma de **elegir la importancia** de cada variable. Una variable irrelevante escalada a $[0, 1]$ pesa en la distancia lo mismo que una muy informativa, y añade ruido; por eso k-NN se beneficia mucho de una buena selección de variables.

Garantías teóricas

Sea R^* el error de Bayes. Cover y Hart (1967) demostraron que, cuando $n \to \infty$, el error del vecino más cercano, R_1, cumple con K clases

$$R^* \leq R_1 \leq R^*\Big(2 - \frac{K}{K-1} R^*\Big) \leq 2R^*. \tag{52.4}$$

Con infinitos datos, el vecino más cercano está tan próximo que tiene casi la misma distribución de clases que el punto consultado; el 1-NN acierta cuando ambos «sortean» la misma clase. Además, si $k \to \infty$ y $k/n \to 0$, k-NN es **consistente**: su error converge a R^* (Stone 1977). La condición $k/n \to 0$ pide que el vecindario tenga cada vez más puntos sin dejar de ser local.

[2] La distancia de **Gower** promedia, variable por variable, una disimilitud entre 0 y 1: para numéricas, la diferencia absoluta dividida entre el rango; para categóricas, 0 si coinciden y 1 si no. Permite aplicar k-NN a tablas mixtas sin codificación *one-hot*.

Laboratorio: el vecindario de k-NN

Arrastra la clienta nueva (el rombo) por el plano, cambia k, la distancia y el tipo de voto. El contorno discontinuo es la «bola» que contiene a los k vecinos: círculo, rombo o cuadrado según la métrica. El estado inicial reproduce la Tabla 52.4.

En la versión web, este laboratorio permite mover el punto consultado y cambiar k, la distancia y el tipo de voto. Valores de referencia con los ocho clientes y el punto $(0.85, 0.42)$: con distancia euclidiana y $k = 3$, los vecinos son 5, 7 y 8, a distancias 0.078, 0.300 y 0.520, y la predicción es «compró» con probabilidad 1. Con distancia de Chebyshev y $k = 3$ el tercer vecino pasa a ser el cliente 3 (0.45) y la probabilidad baja a 0.67. Con $k = 7$ y voto uniforme la probabilidad es 0.57; con voto ponderado, 0.81.

Guía del laboratorio.

1. Comprueba el estado inicial contra la Tabla 52.4: vecinos, distancias y pesos.
2. Predice antes de mover: con distancia de Chebyshev y $k = 3$, ¿quién será el tercer vecino? Compruébalo y observa la forma cuadrada del vecindario.
3. Sube k a 7 con voto uniforme y anota la probabilidad. Cambia a voto ponderado. ¿Por qué sube tanto?
4. Con $k = 8$ y voto uniforme, arrastra el punto por todo el plano. ¿Por qué el color de fondo no cambia? ¿Qué pasa al activar el voto ponderado?
5. Cambia a *Tres clases* con $k = 1$ y observa la frontera alrededor de los puntos aislados (por ejemplo, el punto 19, de la clase B, metido entre los de la clase A). Sube k a 9: ¿qué le pasa a la isla?
6. Busca un punto donde, con $k = 4$ y voto uniforme, haya empate entre dos clases. ¿Cómo lo resuelve el voto ponderado?

Elegir k

k es el hiperparámetro que controla la complejidad de k-NN, como mostró el laboratorio del capítulo anterior: k pequeño da fronteras irregulares (poco sesgo, mucha varianza); k grande, fronteras suaves (más sesgo, menos varianza). En la práctica:

- se elige con **validación cruzada**, nunca con el conjunto de prueba;
- en problemas binarios se prefieren valores impares;
- una heurística inicial razonable es $k \approx \sqrt{n}$, que después se refina con validación;
- el voto ponderado reduce la sensibilidad a k y suele permitir valores más grandes.

Una propiedad conveniente de k-NN es que la **validación cruzada dejando uno fuera** es barata: predecir el punto i sin él equivale a buscar sus k vecinos excluyéndolo, sin volver a «entrenar» nada.

La maldición de la dimensionalidad

k-NN confía en que haya puntos de entrenamiento **cerca** de cualquier punto nuevo. En pocas dimensiones es razonable; en muchas, deja de serlo. Bellman (1961) llamó a este fenómeno la **maldición de la dimensionalidad**, y tiene dos caras.

Los vecindarios dejan de ser locales. Supongamos datos uniformes en el hipercubo $[0,1]^p$. Un subcubo de lado e contiene, en promedio, una fracción e^p de los datos. Para capturar una fracción r hay que tomar un lado

$$e_p(r) = r^{1/p}. \tag{52.5}$$

Con $r = 0.10$: en $p = 1$ basta $e = 0.10$; en $p = 2$, $e = 0.32$; en $p = 10$, $e = 0.79$; y en $p = 100$, $e = 0.977$. Para reunir solo el 10 % de los datos hay que abarcar casi todo el rango de cada variable (Hastie et al. 2009). El «vecino más cercano» ya no es parecido.

Las distancias se concentran. Al crecer p, la distancia al vecino más cercano y al más lejano se vuelven casi iguales, de modo que el cociente $d_{\text{mín}}/d_{\text{máx}}$ tiende a 1 (Beyer et al. 1999). Si todos los puntos están a la misma distancia, «el más cercano» es casi arbitrario.

La consecuencia práctica es que k-NN necesita pocas variables **relevantes**. Antes de usarlo conviene seleccionar variables o reducir la dimensión (por ejemplo con PCA), y desconfiar de representaciones con cientos de columnas *one-hot*.

En la versión web, un laboratorio simula n puntos uniformes en $[0,1]^p$ y muestra el histograma de distancias a un punto consultado. Valores de referencia (deterministas): el lado del subcubo que captura el 10 % de los datos es 0.100 en $p = 1$, 0.316 en $p = 2$, 0.794 en $p = 10$ y 0.977 en $p = 100$. En la simulación con $n = 500$, el cociente $d_{\text{mín}}/d_{\text{máx}}$ es casi 0 en $p = 1$ o 2, del orden de 0.3 en $p = 10$ y de 0.7 en $p = 100$.

Guía del laboratorio.

1. Con $p = 2$, observa el histograma: ¿qué tan lejos está el vecino más cercano en comparación con el promedio?
2. Predice antes de mover: en $p = 100$, ¿el cociente $d_{\text{mín}}/d_{\text{máx}}$ estará más cerca de 0 o de 1? Compruébalo.
3. En $p = 100$ sube n a 2000. ¿Cuánto mejora el cociente? ¿Cuántos puntos crees que harían falta para recuperar la situación de $p = 2$?
4. Explica con la curva verde por qué un vecindario que contiene el 10 % de los datos deja de ser «local» en alta dimensión.

Costo computacional y búsqueda eficiente

Entrenar k-NN cuesta casi nada: basta guardar los datos. Predecir es caro: con búsqueda exhaustiva (*fuerza bruta*), cada consulta calcula n distancias de p componentes, es decir $O(np)$, y necesita tener en memoria todo el conjunto de entrenamiento. Con millones de casos y predicciones en tiempo real esto se vuelve un problema. Hay tres familias de soluciones:

- **Estructuras de búsqueda exacta.** Los árboles k-d (Friedman et al. 1977) y los *ball trees* organizan los datos jerárquicamente y descartan regiones completas que no pueden contener al vecino. Si un nodo tiene centro $\mathbf{M}_p$ y radio r_p, y la mejor distancia encontrada hasta ahora es B, la desigualdad del triángulo garantiza que el nodo puede descartarse cuando $d(\mathbf{x}, \mathbf{M}_p) - r_p > B$ (Fukunaga y Narendra 1975). Funcionan muy bien en dimensión baja o moderada; en dimensión alta degeneran hacia la fuerza bruta.

- **Búsqueda aproximada.** Bibliotecas como FAISS o los grafos HNSW aceptan devolver un vecino *casi* más cercano a cambio de consultas miles de veces más rápidas. Son la base de los buscadores semánticos y de los sistemas de recomendación modernos.
- **Reducir el conjunto de entrenamiento.** La **edición** de Wilson (1972) elimina los puntos que sus propios vecinos clasifican mal (ruido y casos en zonas de traslape), lo que suaviza la frontera. La **condensación** de Hart (1968) conserva solo los puntos necesarios para que 1-NN tome las mismas decisiones: empieza con un punto y agrega cada caso que el subconjunto actual clasifica mal.

Implementación en Python

El ejemplo manual con `scikit-learn`

`KNeighborsClassifier` implementa la Ecuación 52.1. Lo usamos dos veces: con las variables originales y dentro de un *pipeline* que primero aplica `MinMaxScaler`.

La probabilidad estimada (`predict_proba`) para distintos valores de k y ambos tipos de voto reproduce la Tabla 52.5 y el Paso 5:

	k	P(sí) uniform	P(sí) distance
0	1	1.000	1.000
1	3	1.000	1.000
2	5	0.800	0.924
3	7	0.571	0.813
4	8	0.500	0.776

Un problema real: el conjunto *wine*

El conjunto *wine* de `scikit-learn` tiene 178 vinos de tres cultivares descritos por 13 mediciones químicas en escalas muy distintas: la prolina toma valores entre 278 y 1680, mientras que el tono está entre 0.48 y 1.71. Es el escenario típico en el que escalar marca la diferencia.

Sin escalar, la prolina domina la distancia y el clasificador apenas usa la información de las otras doce variables. Ahora elegimos k, el tipo de voto y la métrica con búsqueda en rejilla y validación cruzada, **solo sobre el entrenamiento**:

Figura 52.1.

Variables irrelevantes y alta dimensión

Para ver la maldición de la dimensionalidad en un caso real, añadimos a *wine* columnas de puro ruido. Cada una entra en la distancia con el mismo peso que las variables químicas.

Finalmente, la concentración de distancias del laboratorio, simulada con `numpy`:

	p	d_min	d_max	d_min/d_max	lado para 10 %
0	1	0.000	0.577	0.001	0.100
1	2	0.016	0.994	0.016	0.316
2	10	0.464	1.889	0.246	0.794
3	100	3.410	4.963	0.687	0.977
4	1000	12.343	13.759	0.897	0.998

Ventajas y limitaciones

Ventajas.

- No supone ninguna forma para la frontera; con datos suficientes aproxima fronteras arbitrarias.
- Es multiclase de forma natural y sirve también para regresión.
- Incorporar datos nuevos es inmediato: basta agregarlos.
- Sus decisiones se explican con ejemplos: «se parece a estos cinco casos».

Limitaciones.

- Es sensible a la escala, a las variables irrelevantes y a la dimensión.
- Predecir es costoso en tiempo y memoria si no se usan estructuras de búsqueda.
- No produce un resumen de los datos: no dice qué variables importan.
- Sus probabilidades son burdas (múltiplos de $1/k$) y suelen necesitar calibración.

Cuándo usarlo. Como referencia rápida en problemas con pocas variables numéricas bien escaladas, en sistemas de recomendación y búsqueda por similitud (con búsqueda aproximada) y como componente de otros métodos, por ejemplo la imputación de valores faltantes con vecinos del capítulo de preprocesamiento.

Resumen

- k-NN clasifica un punto con el voto de sus k vecinos más cercanos; la proporción de votos estima $p(c \mid \mathbf{x})$.
- Qué significa «cercano» lo deciden la distancia y, sobre todo, el **escalamiento**, que se ajusta solo con el entrenamiento y forma parte del modelo.
- k controla el compromiso sesgo–varianza y se elige con validación cruzada; el voto ponderado reduce la sensibilidad a k.
- Con infinitos datos, el error del 1-NN es a lo más el doble del error de Bayes, y k-NN con $k \to \infty$, $k/n \to 0$ lo alcanza.
- En alta dimensión las distancias se concentran y los vecindarios dejan de ser locales: k-NN necesita pocas variables relevantes.

El siguiente capítulo aborda una pregunta que ya apareció varias veces: cómo **medir** el desempeño de un clasificador cuando la exactitud no basta.

Parte XXVIII.

Evaluación de Clasificadores

53. Métricas de clasificación

En el capítulo introductorio medimos a los clasificadores con un solo número, la exactitud. Basta un ejemplo para ver que no alcanza: si el 97 % de las transacciones son legítimas, el clasificador que **nunca** detecta fraude acierta el 97 % de las veces y es completamente inútil. Evaluar bien exige separar los **tipos de error**, reconocer que tienen **costos distintos** y distinguir entre la calidad del **puntaje** que produce un modelo y la calidad de la **decisión** que se toma al compararlo con un umbral (Provost y Fawcett 2013; Fawcett 2006).

Este capítulo construye todas las métricas a partir de un único ejemplo de veinte pacientes, calculado a mano, y después las generaliza. El siguiente capítulo usa estas métricas para responder otra pregunta: cómo estimarlas de forma honesta y cómo elegir modelos con ellas.

Intuición: una prueba de detección

Un modelo asigna a cada paciente un **puntaje** entre 0 y 1: la probabilidad estimada de tener una enfermedad. De veinte pacientes, cinco están enfermos (la clase **positiva**) y quince sanos (la **negativa**); es decir, la **prevalencia** es $5/20 = 25\,\%$.

Tabla 53.1.: Puntajes del modelo para los veinte pacientes.

	Puntajes
Enfermos (5)	0.95, 0.85, 0.70, 0.55, 0.30
Sanos (15)	0.80, 0.60, 0.45, 0.40, 0.35, 0.25, 0.20, 0.18, 0.15, 0.12, 0.10, 0.08, 0.05, 0.04, 0.02

El modelo es bueno pero no perfecto: casi todos los enfermos tienen puntajes altos, pero un sano tiene 0.80 y un enfermo apenas 0.30. Para **decidir** hay que fijar un **umbral** θ: se declara «enfermo» a quien tenga puntaje $\geq \theta$. Con el umbral usual $\theta = 0.5$, ¿qué tan bueno es el resultado? La respuesta depende de qué errores nos importen.

Construcción paso a paso

Paso 1. La matriz de confusión

Con $\theta = 0.5$ se declaran enfermos los puntajes 0.95, 0.85, 0.70 y 0.55 (cuatro enfermos reales) y 0.80 y 0.60 (dos sanos). El enfermo con 0.30 queda sin detectar. La **matriz de confusión** cruza la clase real con la predicha:

Tabla 53.2.: Matriz de confusión con = 0.5.

	Predicho enfermo	Predicho sano	Total
Real enfermo	VP = 4	FN = 1	5
Real sano	FP = 2	VN = 13	15
Total	6	14	20

- **Verdaderos positivos** (VP): enfermos detectados.
- **Falsos negativos** (FN): enfermos no detectados, el **error de tipo II**.
- **Falsos positivos** (FP): sanos señalados como enfermos, el **error de tipo I** o falsa alarma.
- **Verdaderos negativos** (VN): sanos correctamente descartados.

Todas las métricas de esta sección son cocientes entre celdas de esta tabla. La clave para no confundirlas es preguntarse **qué fila o qué columna** se usa como denominador.

Paso 2. Métricas por fila: ¿qué le pasó a cada clase real?

Dividiendo entre el total de cada **fila** (la realidad):

$$\text{Sensibilidad} = \frac{VP}{VP + FN} = \frac{4}{5} = 0.80, \qquad \text{Especificidad} = \frac{VN}{VN + FP} = \frac{13}{15} \approx 0.867.$$

La **sensibilidad** (*recall*, exhaustividad o tasa de verdaderos positivos, TPR) responde: de los enfermos, ¿cuántos detecta la prueba? La **especificidad** (tasa de verdaderos negativos) responde: de los sanos, ¿cuántos deja en paz? Sus complementos son la tasa de falsos negativos, $1 - 0.80 = 0.20$, y la **tasa de falsos positivos** (FPR), $2/15 \approx 0.133$.

Paso 3. Métricas por columna: ¿cuánto vale cada predicción?

Dividiendo entre el total de cada **columna** (lo que dijo el modelo):

$$\text{Precisión} = \frac{VP}{VP + FP} = \frac{4}{6} \approx 0.667, \qquad \text{VPN} = \frac{VN}{VN + FN} = \frac{13}{14} \approx 0.929.$$

La **precisión** (valor predictivo positivo, VPP) responde: si la prueba dice «enfermo», ¿qué tan probable es que lo esté? El **valor predictivo negativo** (VPN) responde lo mismo para el «sano». Son las métricas que le importan a quien recibe el resultado; las de fila, a quien diseña la prueba.

Tabla 53.3.: Las cuatro métricas básicas y su denominador.

Pregunta	Denominador	Métrica	Valor
De los enfermos, ¿cuántos detecto?	fila positiva	Sensibilidad (recall, TPR)	0.800
De los sanos, ¿cuántos descarto?	fila negativa	Especificidad (TNR)	0.867

Pregunta	Denominador	Métrica	Valor
Si digo «enfermo», ¿acierto?	columna positiva	Precisión (VPP)	0.667
Si digo «sano», ¿acierto?	columna negativa	VPN	0.929

Paso 4. Métricas globales y la paradoja de la exactitud

La **exactitud** usa toda la tabla: $(VP + VN)/n = 17/20 = 0.85$. Ahora comparemos con el clasificador trivial que declara **sanos a todos**: no tiene VP ni FP, acierta los 15 sanos y su exactitud es $15/20 = 0.75$. Solo diez puntos por debajo de nuestro modelo… sin detectar a un solo enfermo. Con una prevalencia del 1 %, el clasificador trivial tendría 0.99 de exactitud. Esta es la **paradoja de la exactitud**: con clases desbalanceadas, la exactitud premia ignorar a la clase minoritaria (Saito y Rehmsmeier 2015).

Tres métricas globales resisten mejor el desbalance:

- **Exactitud balanceada**: el promedio de sensibilidad y especificidad, $(0.80 + 0.867)/2 \approx 0.833$. El clasificador trivial obtiene $(0 + 1)/2 = 0.5$, como el azar.
- **F1**: la media armónica[1] de precisión y sensibilidad,

$$F_1 = \frac{2 \cdot 0.667 \cdot 0.80}{0.667 + 0.80} = \frac{2\,VP}{2\,VP + FP + FN} = \frac{8}{11} \approx 0.727.$$

  Ignora los VN, así que solo tiene sentido cuando la clase positiva es la de interés. El trivial obtiene 0.
- **Coeficiente de correlación de Matthews** (MCC) (Matthews 1975), la correlación entre la clase real y la predicha, que usa las cuatro celdas:

$$\text{MCC} = \frac{VP \cdot VN - FP \cdot FN}{\sqrt{(VP + FP)(VP + FN)(VN + FP)(VN + FN)}} = \frac{4 \cdot 13 - 2 \cdot 1}{\sqrt{6 \cdot 5 \cdot 15 \cdot 14}} = \frac{50}{79.4} \approx 0.630.$$

  Vale 1 para un clasificador perfecto, 0 para uno que no aprende nada (incluido el trivial) y -1 para uno que se equivoca siempre. Es simétrico respecto a qué clase se llama positiva y suele recomendarse como resumen único en problemas binarios desbalanceados (Chicco y Jurman 2020).

Paso 5. Mover el umbral

Nada obliga a usar $\theta = 0.5$. Bajar el umbral detecta más enfermos a costa de más falsas alarmas; subirlo, lo contrario:

[1] La **media armónica** de a y b es $2ab/(a + b)$. Siempre es menor o igual que la media aritmética y se acerca al menor de los dos valores: con precisión 0.99 y sensibilidad 0.01, la media aritmética es 0.5 pero la armónica es 0.02. Así, F1 solo es alto si ambas métricas lo son.

Tabla 53.4.: Métricas para distintos umbrales.

Umbral θ	VP	FP	FN	Sensibilidad (TPR)	FPR	Precisión
0.90	1	0	4	0.20	0.000	1.000
0.75	2	1	3	0.40	0.067	0.667
0.50	4	2	1	0.80	0.133	0.667
0.30	5	5	0	1.00	0.333	0.500
0.10	5	11	0	1.00	0.733	0.313

Ningún umbral es «el correcto» en abstracto: la sensibilidad y la precisión se mueven en direcciones opuestas. Elegir el umbral es una **decisión de negocio**, no un detalle técnico.

Paso 6. La curva ROC y el AUC

Si recorremos todos los umbrales posibles, de mayor a menor, y graficamos cada par (FPR, TPR), obtenemos la **curva ROC** (*receiver operating characteristic*). Con veinte pacientes es una escalera: cada vez que el umbral pasa por un enfermo, la curva sube 1/5; cada vez que pasa por un sano, avanza 1/15 a la derecha. Recorriendo los puntajes de mayor a menor (E = enfermo, S = sano):

$$\underset{0.95}{E}\ \underset{0.85}{E}\ \underset{0.80}{S}\ \underset{0.70}{E}\ \underset{0.60}{S}\ \underset{0.55}{E}\ \underset{0.45}{S}\ \underset{0.40}{S}\ \underset{0.35}{S}\ \underset{0.30}{E}\ \underset{0.25}{S}\ \cdots\ \underset{0.02}{S}$$

El **área bajo la curva** (AUC) resume la curva en un número. Tiene una interpretación muy útil: es la probabilidad de que un enfermo elegido al azar tenga un puntaje mayor que un sano elegido al azar (Bamber 1975). Con 5 enfermos y 15 sanos hay $5 \times 15 = 75$ parejas. Contamos, para cada enfermo, cuántos sanos tienen un puntaje menor:

Tabla 53.5.: Conteo de parejas bien ordenadas.

Enfermo	0.95	0.85	0.70	0.55	0.30	Total
Sanos con puntaje menor	15	15	14	13	10	67

$$\text{AUC} = \frac{67}{75} \approx 0.893.$$

Un AUC de 0.5 es el azar (la diagonal); 1 es una separación perfecta. Lo importante es que el AUC **no depende del umbral**: mide qué tan bien **ordena** el modelo a los pacientes, no qué tan buena es una decisión concreta.

Paso 7. La curva Precision–Recall

La curva ROC usa tasas por fila, que no dependen de la prevalencia. Cuando los positivos son pocos, conviene mirar también la **curva Precision–Recall** (PR), que grafica la precisión contra la sensibilidad. Su resumen habitual es la **precisión promedio** (AP): el promedio de la precisión en los umbrales donde aparece un nuevo enfermo, ponderado por el aumento en sensibilidad (1/5 cada vez):

Tabla 53.6.: Puntos de la curva Precision–Recall donde aumenta la sensibilidad.

Nuevo enfermo en	0.95	0.85	0.70	0.55	0.30
Sensibilidad	0.2	0.4	0.6	0.8	1.0
Precisión	$1/1 = 1.000$	$2/2 = 1.000$	$3/4 = 0.750$	$4/6 = 0.667$	$5/10 = 0.500$

$$\mathrm{AP} = 0.2\,(1 + 1 + 0.75 + 0.667 + 0.5) \approx 0.783.$$

La referencia de la curva PR no es 0.5 sino la **prevalencia**: un clasificador al azar tiene precisión $5/20 = 0.25$ en todo el rango. Por eso un AP de 0.78 debe compararse con 0.25.

Paso 8. El umbral según los costos

Supongamos que dejar sin diagnóstico a un enfermo cuesta **diez veces** más que una falsa alarma: $C_{FN} = 10$ y $C_{FP} = 1$. El costo total de cada umbral de la Tabla 53.4 es $10 \cdot FN + 1 \cdot FP$:

Tabla 53.7.: Costo total según el umbral con C_FN = 10 y C_FP = 1.

θ	0.90	0.75	0.50	0.30	0.10
Costo	$40 + 0 = 40$	$30 + 1 = 31$	$10 + 2 = \mathbf{12}$	$0 + 5 = \mathbf{5}$	$0 + 11 = 11$

El mejor umbral es 0.30, no 0.50. La teoría de la decisión da una regla general: si el puntaje es una probabilidad **bien calibrada**[2], conviene declarar positivo cuando el costo esperado de hacerlo es menor que el de no hacerlo,

$$(1-p)\,C_{FP} < p\,C_{FN} \iff p > \theta^{*} = \frac{C_{FP}}{C_{FP} + C_{FN}} = \frac{1}{11} \approx 0.09. \tag{53.1}$$

En nuestro ejemplo los puntajes no están calibrados y con solo veinte pacientes la regla teórica (0.09) y el mejor umbral observado (0.30) no coinciden; con muchos datos y probabilidades calibradas, sí lo hacen. En cualquier caso el mensaje es el mismo: **cuando un error cuesta más que el otro, el umbral debe moverse hacia donde ese error se vuelve raro.**

[2]Un puntaje está **calibrado** si, entre todos los casos con puntaje 0.3, alrededor del 30 % son positivos. La regresión logística suele estar razonablemente calibrada; los bosques aleatorios, Naive Bayes y las máquinas de soporte vectorial, no. El capítulo siguiente muestra cómo diagnosticarlo y corregirlo.

Formalización

Matriz de confusión y métricas binarias

Sea $n_+ = VP + FN$ el número de positivos, $n_- = VN + FP$ el de negativos y $\pi = n_+/n$ la prevalencia. Las métricas anteriores se resumen así:

Tabla 53.8.: Métricas derivadas de la matriz de confusión binaria.

Métrica	Fórmula	Rango	Depende de π
Exactitud	$(VP + VN)/n$	$[0, 1]$	sí
Sensibilidad, recall, TPR	$VP/(VP + FN)$	$[0, 1]$	no
Especificidad, TNR	$VN/(VN + FP)$	$[0, 1]$	no
FPR	$FP/(VN + FP) =$ $1 -$ especificidad	$[0, 1]$	no
Precisión, VPP	$VP/(VP + FP)$	$[0, 1]$	sí
VPN	$VN/(VN + FN)$	$[0, 1]$	sí
Exactitud balanceada	$(\mathrm{TPR} + \mathrm{TNR})/2$	$[0, 1]$	no
F_β	$(1 + \beta^2)\dfrac{\mathrm{prec} \cdot \mathrm{rec}}{\beta^2\,\mathrm{prec} + \mathrm{rec}}$	$[0, 1]$	sí
MCC	$\dfrac{VP\,VN - FP\,FN}{\sqrt{(VP + FP)(VP + FN)(VN + FP)(VN + FN)}}$	$[-1, 1]$	débilmente

En F_β, el parámetro β declara cuánto más importa la sensibilidad que la precisión: F_2 la pondera el doble (detección médica), $F_{0.5}$ la mitad (filtros donde las falsas alarmas son caras) y F_1 las trata por igual.

Por qué la precisión depende de la prevalencia

La sensibilidad y la especificidad describen a la prueba; la precisión describe a la prueba **en una población**. Por el teorema de Bayes[3],

$$\mathrm{VPP} = \frac{\mathrm{TPR} \cdot \pi}{\mathrm{TPR} \cdot \pi + \mathrm{FPR} \cdot (1 - \pi)}. \tag{53.2}$$

Con la sensibilidad (0.80) y la FPR (0.133) de nuestro ejemplo, la precisión es 0.667 cuando $\pi = 0.25$, pero baja a $\frac{0.8 \cdot 0.01}{0.8 \cdot 0.01 + 0.133 \cdot 0.99} \approx 0.057$ cuando $\pi = 0.01$: en una población con 1 % de enfermos, **diecisiete de cada dieciocho alarmas serían falsas**, con la misma prueba. Por eso las métricas de columna no se pueden trasladar de una población a otra, y por eso el laboratorio permite mover la prevalencia.

[3]El **teorema de Bayes** permite «invertir» una probabilidad condicional: $P(\text{enfermo} \mid +) = \frac{P(+|\text{enfermo})\,P(\text{enfermo})}{P(+)}$. Aquí $P(+ \mid \text{enfermo})$ es la sensibilidad, $P(\text{enfermo}) = \pi$, y el denominador suma las dos maneras de obtener un positivo: ser enfermo y detectado, o ser sano y recibir una falsa alarma.

Curvas ROC y PR

Sea $s(\mathbf{x})$ el puntaje y θ el umbral. Al variar θ, la curva ROC recorre los puntos $(\mathrm{FPR}(\theta), \mathrm{TPR}(\theta))$ desde $(0,0)$ hasta $(1,1)$, y la curva PR recorre $(\mathrm{TPR}(\theta), \mathrm{VPP}(\theta))$. El AUC tiene la forma

$$\mathrm{AUC} = P\big(s(\mathbf{x}^+) > s(\mathbf{x}^-)\big) + \tfrac{1}{2} P\big(s(\mathbf{x}^+) = s(\mathbf{x}^-)\big), \tag{53.3}$$

donde $\mathbf{x}^+$ y $\mathbf{x}^-$ son un positivo y un negativo elegidos al azar; es el estadístico de Mann–Whitney normalizado. Dos propiedades prácticas:

- La curva ROC y el AUC **no cambian** si se transforma el puntaje con una función creciente: solo importa el orden. Por eso dos modelos con el mismo AUC pueden tener probabilidades muy distintas.
- La curva ROC **no cambia con la prevalencia**, pero la curva PR sí. Con clases muy desbalanceadas, un AUC alto puede convivir con una precisión pésima en el umbral de operación; la curva PR lo revela (Saito y Rehmsmeier 2015).

Más de dos clases

Con K clases la matriz de confusión es $K \times K$. Para cada clase c se calculan precisión, sensibilidad y F1 tratándola como positiva y a las demás como negativas («una contra el resto»). Después se promedian de tres maneras:

- **Macro**: promedio simple sobre clases; todas pesan igual, también las raras.
- **Ponderado** (*weighted*): promedio ponderado por el número de casos de cada clase.
- **Micro**: se suman VP, FP y FN de todas las clases y se calcula una sola métrica; en clasificación multiclase de una etiqueta, el F1 micro coincide con la exactitud.

Un ejemplo con tres clases y 100 casos:

Tabla 53.9.: Matriz de confusión con tres clases.

	Predicho A	Predicho B	Predicho C	Sensibilidad
Real A (50)	**40**	5	5	0.800
Real B (30)	10	**15**	5	0.500
Real C (20)	2	3	**15**	0.750
Precisión	$40/52 = 0.769$	$15/23 = 0.652$	$15/25 = 0.600$	

Los F1 por clase son 0.784, 0.566 y 0.667. El F1 macro es su promedio, 0.672; el ponderado, $0.5 \cdot 0.784 + 0.3 \cdot 0.566 + 0.2 \cdot 0.667 = 0.695$; el micro es igual a la exactitud, $70/100 = 0.700$. El macro es el más bajo porque da a la clase B, la peor, el mismo peso que a las demás: es el promedio adecuado cuando todas las clases importan por igual.

Métricas para probabilidades

Las métricas anteriores evalúan **decisiones** o **rankings**. Cuando el producto del modelo es la probabilidad misma (un riesgo de impago que se usará para fijar una tasa, por ejemplo), hay que evaluar la probabilidad:

- **Pérdida logarítmica** (*log-loss*, entropía cruzada): $-\frac{1}{n}\sum_i \left[y_i \ln \hat{p}_i + (1-y_i)\ln(1-\hat{p}_i)\right]$. Castiga sin piedad la confianza equivocada: predecir 0.99 para un sano cuesta $-\ln 0.01 \approx 4.6$, cuarenta veces más que predecir 0.9 para un enfermo ($-\ln 0.9 \approx 0.105$).
- **Puntaje de Brier** (Brier 1950): $\frac{1}{n}\sum_i (\hat{p}_i - y_i)^2$, el error cuadrático de la probabilidad. Está acotado entre 0 y 1 y es más indulgente con los errores extremos.

Para los veinte pacientes, la log-loss es 0.387 y el Brier 0.124. Ambas son **reglas de puntuación propias**: se minimizan, en promedio, reportando la probabilidad verdadera, así que premian a los modelos **calibrados**. Su complemento gráfico, el diagrama de calibración, se estudia en el capítulo siguiente.

Laboratorio: umbral, prevalencia y curvas

El laboratorio tiene dos modos. **Veinte pacientes** usa los puntajes de la Tabla 53.1; el estado inicial ($\theta = 0.5$) reproduce la Tabla 53.2 y las métricas de los pasos 2 a 4. **Modelo binormal** supone que el puntaje latente de los sanos sigue una normal estándar y el de los enfermos una normal con media d; ese puntaje se convierte en una probabilidad calibrada según la prevalencia, y todo se calcula para una población de 1000 personas. Ahí se pueden mover la **separación** entre clases, la **prevalencia** y el **cociente de costos**.

En la versión web, este laboratorio recalcula la matriz de confusión, las métricas y las curvas al mover el umbral. Valores de referencia con los veinte pacientes y $\theta = 0.5$: VP = 4, FN = 1, FP = 2, VN = 13; exactitud 0.850, sensibilidad 0.800, especificidad 0.867, precisión 0.667, VPN 0.929, F1 0.727, exactitud balanceada 0.833, MCC 0.630 y AUC 0.893. Con $\theta = 0.30$ la sensibilidad es 1 y la precisión 0.5. En el modelo binormal con separación $d = 1.5$ el AUC es $\Phi(d/\sqrt{2}) \approx 0.856$; con prevalencia 0.10 y cociente de costos 10, el umbral de mínimo costo es $1/11 \approx 0.09$.

Guía del laboratorio.

1. Con *Veinte pacientes* y $\theta = 0.50$, comprueba la Tabla 53.2 y las métricas de los pasos 2 a 4. ¿Dónde está el punto naranja sobre la curva ROC?
2. Predice antes de mover: si bajas el umbral a 0.30, ¿qué celda de la matriz se vacía? ¿Qué le pasa a la precisión? Compruébalo con la Tabla 53.4.
3. Recorre el umbral de 0.99 a 0.01 y observa cómo el punto recorre la escalera de la curva ROC. ¿Por qué el AUC no cambia?
4. Cambia a *Modelo binormal*. Con $d = 1.5$ y $\theta = 0.5$, baja la prevalencia de 0.10 a 0.01. ¿Qué curva cambia y cuál no? ¿Qué le pasa a la precisión en el umbral actual?
5. Con prevalencia 0.01, compara la exactitud del modelo con la del clasificador trivial. ¿Qué métrica te avisa del problema?
6. Pulsa *Ir al umbral de mínimo costo* con cocientes de costo 1, 10 y 50. Mueve el umbral un poco a cada lado y verifica que el costo sube.
7. Pon $d = 0$. ¿Qué forma tienen las curvas? ¿Qué AUC obtienes y por qué?

Cómo elegir la métrica

La métrica se elige **antes** de entrenar, a partir del problema y no de los resultados:

Tabla 53.10.: Guía para elegir métricas.

Situación	Métrica principal	Complemento
Clases balanceadas y errores con costo similar	Exactitud	Matriz de confusión
Positivo raro y lo importante es detectarlo (cribado médico, fraude)	Sensibilidad a una precisión mínima, F_2	Curva PR, AP
Las falsas alarmas son caras (bloquear clientes, spam)	Precisión a una sensibilidad mínima, $F_{0.5}$	Curva PR
Desbalance y se quiere un resumen único y simétrico	MCC o exactitud balanceada	Matriz de confusión
Comparar la capacidad de ordenar, sin fijar umbral	AUC-ROC	AP si el positivo es raro
Costos conocidos	Costo esperado en el umbral θ^*	Curva de costo vs. umbral
La probabilidad es el producto (riesgo, tarifas)	Log-loss o Brier	Diagrama de calibración
Multiclase con clases igualmente importantes	F1 macro	Matriz $K \times K$ y F1 por clase

Errores frecuentes:

- **Reportar solo la exactitud** con clases desbalanceadas.
- **Elegir el umbral mirando el conjunto de prueba.** El umbral es un hiperparámetro: se elige con validación, como se explica en el capítulo siguiente.
- **Comparar precisiones entre poblaciones** con prevalencias distintas (Ecuación 53.2).
- **Confundir buen ranking con buenas probabilidades.** Un AUC alto no garantiza que «0.8» signifique 80 %.
- **Promediar sin mirar las clases.** Un F1 macro aceptable puede ocultar una clase con F1 cercano a cero; reporta siempre las métricas por clase.

Implementación en Python

El ejemplo manual con `scikit-learn`

El módulo `sklearn.metrics` tiene una función para cada métrica. Reproducimos los números de la sección anterior.

Observa que la especificidad y el VPN no tienen función propia: son la sensibilidad y la precisión de la clase negativa (`pos_label=0`). El costo por umbral de la Tabla 53.7:

	umbral	VP	FP	FN	costo $(10 \cdot \text{FN} + \text{FP})$
0	0.90	1	0	4	40
1	0.75	2	1	3	31
2	0.50	4	2	1	12
3	0.30	5	5	0	5
4	0.10	5	11	0	11

Un caso realista: cribado con 8 % de prevalencia

Simulamos un problema de cribado médico con 3000 pacientes y una prevalencia cercana al 8 %, y comparamos una regresión logística con un bosque aleatorio. Separamos tres conjuntos: **entrenamiento** para ajustar los modelos, **validación** para elegir el umbral y **prueba** para la evaluación final.

Con el umbral por defecto (0.5), ambos modelos tienen exactitud alta y sensibilidad modesta, el síntoma clásico del desbalance:

Figura 53.1.

Elegir el umbral con validación

Supongamos que un enfermo no detectado cuesta diez veces más que una falsa alarma. Elegimos el umbral que minimiza el costo **en validación** y lo aplicamos, sin tocarlo, en prueba:

	umbral	sensibilidad	precisión	MCC	costo en prueba
0	0.50	0.566	0.860	0.676	337
1	0.13	0.803	0.399	0.511	242

El umbral elegido en validación reduce el costo en prueba alrededor de un 30 % respecto a 0.5, a cambio de una precisión menor: es exactamente el compromiso que se decidió al fijar los costos.

Métricas multiclase

`classification_report` calcula precisión, sensibilidad y F1 por clase con sus promedios macro y ponderado. Reproducimos la Tabla 53.9:

Resumen

- La **matriz de confusión** es la base de todas las métricas; cada métrica es un cociente con un denominador distinto: por fila (sensibilidad, especificidad) o por columna (precisión, VPN).
- La **exactitud** engaña con clases desbalanceadas; la exactitud balanceada, F1 y sobre todo el **MCC** son resúmenes más robustos.

- La **precisión depende de la prevalencia**: la misma prueba produce muchas más falsas alarmas en una población donde el positivo es raro.
- Las curvas **ROC** y **PR** recorren todos los umbrales. El AUC mide la capacidad de **ordenar** y equivale a la probabilidad de que un positivo tenga mayor puntaje que un negativo; la curva PR revela problemas que la ROC oculta cuando el positivo es raro.
- El **umbral** es una decisión: con costos C_{FP} y C_{FN} y probabilidades calibradas, el óptimo es $C_{FP}/(C_{FP} + C_{FN})$. Se elige con validación, nunca con la prueba.
- Con varias clases, las métricas se promedian (macro, ponderado, micro); cuando la probabilidad es el producto, se evalúa con log-loss o Brier.

54. Validación y selección de modelos

El capítulo anterior respondió **qué** medir. Este responde **cómo medirlo sin engañarse**. Un número de desempeño se usa para tres cosas distintas, y las tres exigen datos que el modelo no haya visto:

1. **Estimar** cuánto acertará el modelo en producción.
2. **Seleccionar**: elegir entre algoritmos, hiperparámetros, variables o umbrales.
3. **Comparar** dos modelos y decidir si la diferencia es real o producto del azar.

El problema de fondo es que los datos etiquetados son escasos y cada uso los «gasta»: un dato que sirvió para elegir algo ya no es un juez imparcial de esa elección. Este capítulo presenta las herramientas para administrar ese presupuesto: el *holdout*, la validación cruzada, la validación anidada, y las trampas más comunes, empezando por la **fuga de datos** (Hastie et al. 2009; Kohavi 1995).

Intuición: ¿qué clasificador elegirías?

Volvamos a los diez clientes del capítulo introductorio, pero ahora supongamos que **no hay** cinco clientes nuevos de prueba: solo tenemos los diez. Queremos elegir entre el tocón «endeudamiento t», el 1-NN y el 3-NN. La exactitud de entrenamiento recomienda el 1-NN (10/10), y ya sabemos que es la peor recomendación posible: el 1-NN memoriza.

La idea de todo este capítulo cabe en una frase: **para juzgar a un modelo hay que esconderle datos**, entrenarlo con el resto y preguntarle por los escondidos.

Construcción paso a paso

Paso 1. Esconder algunos datos: el *holdout*

Separamos tres clientes como **validación**, entrenamos con los siete restantes y contamos aciertos en los tres escondidos. El resultado depende mucho de **cuáles** tres se esconden:

Tabla 54.1.: Tres particiones holdout distintas de los mismos diez clientes.

Clientes escondidos	Tocón	1-NN	3-NN	¿Cuál gana?
2, 5, 8	3/3	1/3	2/3	tocón
1, 4, 7	3/3	3/3	3/3	empate
6, 9, 10	2/3	2/3	1/3	tocón y 1-NN

Con la segunda partición los tres clasificadores parecen igual de buenos; con la tercera, el 1-NN empata con el tocón. Una sola partición produce una estimación con **varianza alta**: con pocos datos de validación, el azar de la partición pesa tanto como la calidad del modelo.

Paso 2. Esconder cada dato una vez: dejar uno fuera

Para no depender de una partición, repetimos el procedimiento diez veces, escondiendo **un** cliente cada vez y entrenando con los otros nueve. Es la **validación cruzada dejando uno fuera** (LOOCV, *leave-one-out*). Para el 1-NN es fácil a mano: basta buscar el vecino más cercano de cada cliente **sin contarlo a él mismo**, usando las distancias del capítulo introductorio.

Tabla 54.2.: Validación cruzada dejando uno fuera con los diez clientes.

Cliente	Clase	Vecino más cercano (sin él)	Distancia	1-NN	3 vecinos	3-NN	Tocón
1	sí	2	11.18	sí	2, 3, 5	sí	sí
2	sí	3	8.60	sí	3, 1, 4	sí	sí
3	sí	2	8.60	sí	2, 4, 1	sí	sí
4	sí	3	11.66	sí	3, 2, 10	sí	sí
5	sí	2	14.14	sí	2, 10, 7	no	sí
6	sí	9	6.40	no	9, 8, 10	no	no
7	no	8	11.18	no	8, 5, 10	no	no
8	no	6	8.25	sí	6, 7, 9	no	no
9	no	6	6.40	sí	6, 8, 10	no	no
10	no	6	10.20	sí	6, 8, 5	sí	no
				6/10		**7/10**	**9/10**

Para el tocón hay que **reajustar** el corte cada vez: sin el cliente 5, el mejor corte se mueve a 41.5; sin el 10, a 47.5; en los demás casos sigue en 44. Solo falla con el cliente 6, la excepción.

Paso 3. Leer el resultado

Tabla 54.3.: Entrenamiento, validación cruzada y prueba.

Clasificador	Exactitud en entrenamiento	Exactitud LOOCV	Exactitud en los 5 clientes nuevos
Tocón	0.90	**0.90**	**0.80**
1-NN	**1.00**	0.60	0.60
3-NN	0.90	0.70	0.60

La validación cruzada **invierte** el orden que sugería el entrenamiento y coincide con lo que después mostraron los clientes nuevos: el tocón generaliza mejor. La tabla también explica por qué el 1-NN falla: el cliente 6, la excepción, es el vecino más cercano de los clientes 8, 9 y 10, y los «contagia». El 3-NN diluye parte de ese contagio; el tocón lo ignora.

Paso 4. De uno en uno a K pliegues

LOOCV entrena el modelo n veces, algo inviable con un millón de registros o modelos caros. La **validación cruzada de K pliegues** (*K-fold*) divide los datos al azar en K grupos del mismo tamaño; cada grupo se esconde una vez mientras se entrena con los otros $K-1$. Con $K=5$ y nuestros diez clientes, se esconderían dos clientes cada vez y se entrenaría cinco veces. LOOCV es el caso extremo $K=n$.

Formalización

El estimador de validación cruzada

Sea $\kappa(i) \in \{1, \dots, K\}$ el pliegue al que pertenece la observación i y $\hat{f}^{-k}$ el modelo entrenado sin el pliegue k. La estimación por validación cruzada del riesgo es

$$\widehat{R}_{\mathrm{CV}} = \frac{1}{n} \sum_{i=1}^{n} L(y_i, \hat{f}^{-\kappa(i)}(\mathbf{x}_i)), \tag{54.1}$$

con L la pérdida 0-1 o cualquier otra métrica. Cada observación se predice **exactamente una vez** con un modelo que no la vio (Stone 1974).

El estimador tiene dos fuentes de error:

- **Sesgo.** Cada modelo se entrena con $(K-1)n/K$ observaciones, no con n; como los modelos suelen mejorar con más datos, $\widehat{R}_{\mathrm{CV}}$ tiende a ser algo **pesimista**. El sesgo es grande con $K=2$ y casi nulo con LOOCV.
- **Varianza.** Depende de la partición aleatoria y del tamaño de cada pliegue. LOOCV no depende del azar, pero sus n modelos son casi idénticos y sus errores están muy correlacionados, lo que también puede dar varianza alta.

El estudio clásico de Kohavi (1995) recomienda $K=10$ **estratificado** como un buen compromiso; $K=5$ es la elección habitual cuando entrenar es costoso. Repetir la validación cruzada con varias particiones aleatorias (por ejemplo, 5×10 pliegues) y promediar reduce la varianza debida a la partición.

Estratificación. En clasificación conviene que cada pliegue conserve la proporción de clases del total; si no, con clases desbalanceadas algún pliegue puede quedarse sin positivos. Es la opción por defecto de `scikit-learn` para clasificadores.

Incertidumbre. Con los errores por pliegue $e_1, \dots, e_K$ suele reportarse $\bar{e} \pm s_e/\sqrt{K}$, con s_e su desviación estándar. Es una aproximación: los pliegues comparten datos de entrenamiento y no son independientes, así que este error estándar subestima la incertidumbre real.

Tres conjuntos y un presupuesto

Cuando hay que **elegir** algo (hiperparámetros, variables, umbral) y además **estimar** el desempeño del elegido, hacen falta datos distintos para cada tarea:

Tabla 54.4.: Papel de cada conjunto de datos.

Conjunto	Para qué se usa	Cuántas veces se mira
Entrenamiento	Ajustar parámetros	Muchas
Validación (o validación cruzada sobre el entrenamiento)	Elegir hiperparámetros, variables, umbral	Muchas
Prueba	Estimar el desempeño del modelo final	**Una**

La regla de oro: **cualquier decisión tomada mirando un conjunto contamina a ese conjunto como estimador**. Si se prueban cien configuraciones y se reporta la mejor exactitud de validación, ese número es optimista, porque la mejor configuración ganó en parte por suerte (Cawley y Talbot 2010).

Selección de hiperparámetros

Para un hiperparámetro λ (el k de k-NN, la profundidad de un árbol), se elige

$$\hat{\lambda} = \arg\min_{\lambda \in \Lambda} \widehat{R}_{\mathrm{CV}}(\lambda),$$

y después se reentrena con **todo** el conjunto de entrenamiento usando $\hat{\lambda}$. Una variante más conservadora es la **regla de un error estándar** (Hastie et al. 2009): elegir el modelo **más simple** cuyo error de validación esté a menos de un error estándar del mínimo. Prefiere la simplicidad cuando la diferencia no es distinguible del ruido.

Λ puede recorrerse con una **rejilla** (todas las combinaciones de una lista de valores) o con **búsqueda aleatoria**, que muestrea combinaciones al azar. Cuando solo unos pocos hiperparámetros importan, la búsqueda aleatoria encuentra buenas configuraciones con muchas menos evaluaciones (Bergstra y Bengio 2012).

Validación anidada

Si no hay un conjunto de prueba separado y se quiere estimar el desempeño **del procedimiento completo** (incluida la selección de hiperparámetros), se usa **validación cruzada anidada**: un ciclo externo de K pliegues estima el desempeño y, dentro de cada pliegue externo, un ciclo interno elige los hiperparámetros solo con los datos de entrenamiento de ese pliegue. El número del ciclo externo estima qué tan bien funciona «entrenar con búsqueda de hiperparámetros», no un modelo concreto.

Datos con estructura

La validación cruzada supone que las observaciones son intercambiables. Dos casos frecuentes violan ese supuesto:

- **Grupos.** Si hay varias filas por paciente, cliente o tienda, las filas del mismo grupo deben ir juntas al mismo pliegue (`GroupKFold`); si no, el modelo «reconoce» al paciente en lugar de aprender el patrón.
- **Tiempo.** Con series temporales se entrena con el pasado y se valida con el futuro (`TimeSeriesSplit`). Mezclar al azar permite predecir el pasado con datos del futuro, algo imposible en producción.

Laboratorio: ¿cuánto se mueve la estimación?

El laboratorio usa los 120 puntos de entrenamiento y 400 de prueba del capítulo introductorio y el clasificador k-NN. El panel izquierdo repite 200 veces cada método de validación con particiones aleatorias distintas y muestra la distribución de las estimaciones; la línea verde es el error del mismo modelo entrenado con todos los datos y medido en los 400 puntos de prueba, nuestra mejor aproximación al error real. El panel derecho usa validación cruzada de 10 pliegues para **elegir k sin mirar la prueba**, la tarea que quedó pendiente en el capítulo introductorio.

En la versión web, este laboratorio repite 200 veces cada método de validación sobre los 120 puntos del capítulo introductorio con 5-NN y muestra la distribución de las estimaciones del error, y traza la curva de validación de 10 pliegues para elegir k. Valores de referencia: el error de 5-NN en los 400 puntos de prueba es 0.110 y la estimación LOOCV (determinista) es 0.108. La desviación estándar de las estimaciones con holdout del 30 % es de alrededor de 0.045, unas cinco veces la de 10 pliegues (0.009). Según la partición, la validación cruzada elige $k = 3$, 5 o 7, con errores de prueba entre 0.110 y 0.123, frente al mínimo de 0.110.

Guía del laboratorio.

1. Con *Holdout 30 %*, observa el ancho del histograma. Si te tocara una sola de esas 200 estimaciones, ¿entre qué valores podría estar?
2. Predice antes de mover: ¿qué método tendrá la menor desviación estándar, 5 o 10 pliegues? Compruébalo en la tabla de resumen.
3. Compara el promedio de *Holdout 50 %* con el error de prueba. ¿Por qué el holdout del 50 % es pesimista? Pista: ¿con cuántos datos entrena cada modelo?
4. Baja n a 40. ¿Qué le pasa a la dispersión de todos los métodos? ¿Y a la curva de validación?
5. Pulsa *Nuevas particiones* varias veces y anota el k que elige la validación cruzada. ¿Cambia? ¿Cambia mucho el error de prueba del k elegido?
6. Explica por qué la curva morada (prueba) se muestra solo para comparar y no debe usarse para elegir k.

Implementación en Python

La validación cruzada a mano y con `scikit-learn`

`cross_val_score` y `cross_val_predict` implementan la Ecuación 54.1 con cualquier esquema de partición. Reproducimos la Tabla 54.2 con `LeaveOneOut`:

Varianza de las estimaciones

Con los datos del laboratorio comparamos la dispersión del holdout y de la validación cruzada repitiendo cada método con 200 semillas.

Figura 54.1.

Curva de validación y curva de aprendizaje

Una **curva de validación** grafica el error de entrenamiento y de validación cruzada contra un hiperparámetro; una **curva de aprendizaje**, contra el tamaño del entrenamiento. La primera diagnostica la complejidad; la segunda responde si conviene conseguir más datos: si las dos curvas ya convergieron, más datos no ayudarán; si siguen separadas, sí.

Búsqueda de hiperparámetros y validación anidada

Usamos el conjunto *breast cancer* (569 tumores, 30 variables) y un *pipeline* que escala y aplica k-NN. `GridSearchCV` hace la validación cruzada interna; envolverlo en `cross_val_score` da la validación anidada.

La diferencia entre el mejor puntaje de la rejilla y la validación anidada es el optimismo de la selección. Aquí es pequeña porque la rejilla es pequeña y el problema fácil; con cientos de configuraciones y pocos datos puede ser de varios puntos porcentuales.

La fuga de datos

La **fuga de datos** ocurre cuando información de los datos de validación o prueba se filtra al entrenamiento. Sus formas más comunes son:

- **Preprocesar antes de separar**: escalar, imputar o seleccionar variables con todos los datos.
- **Variables del futuro**: usar una columna que solo se conoce después del evento a predecir (la fecha de cierre de una cuenta para predecir si se cerrará).
- **Duplicados o grupos** repartidos entre entrenamiento y prueba.
- **Elegir con la prueba**: probar muchos modelos y reportar el mejor resultado de prueba.

El siguiente experimento es un clásico (Hastie et al. 2009). Generamos 100 casos con **5000 variables de puro ruido** y una clase al azar: ningún clasificador puede superar el 50 %. Si seleccionamos las 20 variables más correlacionadas con la clase **antes** de la validación cruzada, la estimación es espectacular; si la selección va **dentro** del *pipeline*, vuelve a la realidad.

La regla práctica: **todo paso que aprenda algo de los datos va dentro del *pipeline***, para que se reajuste en cada pliegue con los datos de entrenamiento de ese pliegue.

Clases desbalanceadas

Cuando la clase positiva es rara, además de elegir bien la métrica (capítulo anterior) hay cuatro palancas:

1. **Estratificar** todas las particiones.
2. **Pesar las clases** en la función de pérdida (`class_weight="balanced"`), de modo que cada positivo cuente tanto como n_-/n_+ negativos.
3. **Remuestrear** el entrenamiento: submuestrear la clase mayoritaria, sobremuestrear la minoritaria o generar positivos sintéticos con SMOTE, que interpola entre vecinos de la clase minoritaria (Chawla et al. 2002). El remuestreo se aplica **solo dentro del entrenamiento de cada pliegue**; remuestrear antes de separar es otra forma de fuga.
4. **Mover el umbral**, como en el capítulo de métricas.

Con frecuencia, pesar las clases o mover el umbral logra lo mismo que remuestrear, sin alterar los datos:

modelo	accuracy	balanced_accuracy	recall	precision	average_precision
sin ajuste	0.955	0.636	0.276	0.831	0.534
class_weight='balanced'	0.804	0.786	0.766	0.191	0.493

Pesar las clases sube la sensibilidad y la exactitud balanceada a costa de precisión, mientras que la precisión promedio, que no depende del umbral, cambia poco: el ranking del modelo es casi el mismo; lo que cambia es dónde se corta.

Calibración de probabilidades

Un clasificador está **calibrado** si, entre los casos a los que asigna probabilidad 0.3, alrededor del 30 % son positivos. La calibración importa cuando la probabilidad se usa como tal: para fijar precios, combinar modelos o aplicar el umbral teórico $C_{FP}/(C_{FP}+C_{FN})$. Se diagnostica con un **diagrama de calibración**: se agrupan los casos por probabilidad predicha y se grafica la fracción real de positivos de cada grupo; un modelo calibrado sigue la diagonal.

Muchos clasificadores no están calibrados: Naive Bayes empuja las probabilidades a los extremos; los bosques aleatorios y las máquinas de soporte vectorial las concentran en el centro (Niculescu-Mizil y Caruana 2005). Se corrigen con un **recalibrador** ajustado en datos de validación: la **escala de**

Platt ajusta una regresión logística sobre el puntaje (Platt 1999) y la **regresión isotónica** ajusta una función escalonada creciente, más flexible pero más necesitada de datos.

Comparar dos clasificadores

Si el modelo A acierta 87 % y el B 85 % en el mismo conjunto de prueba, ¿A es mejor? Depende de cuántos casos haya y de **en qué casos** difieren. La **prueba de McNemar** (McNemar 1947) compara dos clasificadores evaluados en los mismos casos usando solo los desacuerdos. Supongamos 200 casos de prueba:

Tabla 54.5.: Aciertos y fallos de dos clasificadores sobre los mismos 200 casos.

	B acierta	B falla
A acierta	150	$n_{10} = 25$
A falla	$n_{01} = 10$	15

Si ambos fueran igual de buenos, los 35 desacuerdos se repartirían mitad y mitad. El estadístico con corrección de continuidad es

$$\chi^2 = \frac{(|n_{10} - n_{01}| - 1)^2}{n_{10} + n_{01}} = \frac{(15 - 1)^2}{35} = 5.6,$$

mayor que 3.84, el valor crítico de una χ^2 con un grado de libertad al 5 %: la diferencia es significativa. Observa que las exactitudes son $175/200 = 0.875$ y $160/200 = 0.80$, pero lo que decide es cómo se reparten los 35 casos en que discrepan.

Para comparar **algoritmos** (no un par de modelos ya entrenados) con validación cruzada, la diferencia por pliegue tiene varianza subestimada porque los pliegues comparten datos; Dietterich (1998) recomienda la prueba 5×2 CV o McNemar sobre un conjunto de prueba. En la práctica, además de la significancia hay que preguntarse si la diferencia **importa** al negocio.

Lista de verificación

1. Definir clase positiva, métrica y referencia **antes** de mirar resultados.
2. Separar la prueba primero; estratificar; respetar grupos y tiempo.
3. Meter todo el preprocesamiento en un *pipeline*.
4. Elegir hiperparámetros, variables y umbral con validación cruzada sobre el entrenamiento.
5. Reportar la estimación con su incertidumbre; usar validación anidada si no hay prueba independiente.
6. Revisar calibración si las probabilidades se usarán como tales.
7. Evaluar **una vez** en prueba y comparar con la referencia y con un modelo simple.

Resumen

- Estimar, seleccionar y comparar exigen datos que el modelo no haya visto; cada decisión «gasta» los datos con que se tomó.
- El holdout es simple pero de varianza alta. La validación cruzada de K pliegues (5 o 10, estratificada) usa cada dato una vez para validar; LOOCV es el caso $K = n$.
- Los hiperparámetros se eligen con validación cruzada; la regla de un error estándar prefiere el modelo más simple compatible con el mínimo. El mejor puntaje de una búsqueda es optimista: la validación anidada lo corrige.
- La fuga de datos produce estimaciones espectaculares y falsas; todo lo que aprende de los datos va dentro del *pipeline*.
- Con desbalance: estratificar, pesar clases o remuestrear dentro de cada pliegue, y mover el umbral.
- La calibración se diagnostica con un diagrama de calibración y se corrige con Platt o regresión isotónica.
- Para comparar dos clasificadores en los mismos casos, la prueba de McNemar usa solo los desacuerdos.

Parte XXIX.

Árboles y Ensambles

55. Árboles de decisión: ID3 y C4.5

Un **árbol de decisión** clasifica haciendo preguntas, una tras otra: «¿cómo está el cielo?», «si está soleado, ¿la humedad es alta?». Cada respuesta lleva a otra pregunta o a una conclusión. Es la forma en que un médico sigue un protocolo de diagnóstico o un analista de crédito revisa una solicitud, y por eso los árboles son el clasificador más fácil de explicar a quien no es especialista.

La pregunta técnica es **qué preguntar y en qué orden**. Este capítulo la responde con la idea de **impureza** de la teoría de la información (Shannon 1948) y con los algoritmos clásicos que la usan: la estrategia recursiva de Hunt et al. (1966), ID3 (Quinlan 1986) y su sucesor C4.5 (Quinlan 1993). El capítulo siguiente estudia CART (Breiman et al. 1984), la variante binaria que implementa `scikit-learn`.

Intuición: ¿se juega el partido?

Durante quince días se registró el estado del tiempo y si se jugó un partido de béisbol.

Tabla 55.1.: Quince días, 9 con partido y 6 sin partido.

Día	Pronóstico	Temperatura	Humedad	Viento	¿Juega?
D1	Soleado	Caluroso	Alta	Débil	No
D2	Soleado	Caluroso	Alta	Fuerte	No
D3	Nublado	Caluroso	Alta	Débil	Sí
D4	Lluvia	Templado	Alta	Débil	Sí
D5	Lluvia	Fresco	Normal	Débil	Sí
D6	Lluvia	Fresco	Normal	Fuerte	No
D7	Nublado	Fresco	Normal	Fuerte	Sí
D8	Soleado	Templado	Alta	Débil	No
D9	Soleado	Fresco	Normal	Débil	Sí
D10	Lluvia	Templado	Normal	Débil	Sí
D11	Soleado	Templado	Normal	Débil	Sí
D12	Nublado	Templado	Alta	Fuerte	Sí
D13	Nublado	Caluroso	Normal	Débil	Sí
D14	Lluvia	Templado	Alta	Fuerte	No
D15	Soleado	Fresco	Alta	Débil	No

Si solo pudiéramos hacer **una** pregunta, ¿cuál haríamos? Preguntar por el pronóstico tiene una ventaja visible: los cuatro días nublados se jugó, así que esa respuesta resuelve el caso de inmediato. Preguntar por la temperatura, en cambio, deja grupos tan mezclados como al principio. Una buena pregunta es la que deja grupos **puros**, con una sola clase. Necesitamos una medida de qué tan mezclado está un grupo.

Construcción paso a paso

Paso 1. Medir la mezcla: la entropía

La **entropía** de un conjunto S con proporciones de clase $p_1, \dots, p_K$ es

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k, \tag{55.1}$$

con la convención $0 \log_2 0 = 0$. Vale 0 si el grupo es puro y es máxima cuando las clases están igualmente repartidas (1 bit en el caso binario)[1]. Para los quince días, con 9 «Sí» y 6 «No»:

$$H(S) = -\tfrac{9}{15}\log_2 \tfrac{9}{15} - \tfrac{6}{15}\log_2 \tfrac{6}{15} = 0.6 \cdot 0.737 + 0.4 \cdot 1.322 \approx 0.971.$$

Paso 2. La ganancia de cada pregunta

Preguntar por el pronóstico divide los días en tres grupos:

Tabla 55.2.: Partición por pronóstico.

Pronóstico	Días	Sí	No	Entropía del grupo
Soleado	6	2	4	$-\frac{2}{6}\log_2\frac{2}{6} - \frac{4}{6}\log_2\frac{4}{6} = 0.918$
Nublado	4	4	0	0
Lluvia	5	3	2	0.971

La entropía **después** de preguntar es el promedio de las entropías de los grupos, ponderado por su tamaño:

$$\tfrac{6}{15}(0.918) + \tfrac{4}{15}(0) + \tfrac{5}{15}(0.971) = 0.367 + 0 + 0.324 = 0.691.$$

La **ganancia de información** es cuánto bajó la entropía: $0.971 - 0.691 = 0.280$ bits. Repitiendo el cálculo para los cuatro atributos:

Tabla 55.3.: Ganancia de información de cada atributo en la raíz.

Atributo	Entropía después	Ganancia
Pronóstico	0.691	**0.280**
Humedad	0.785	0.186
Viento	0.911	0.060
Temperatura	0.958	0.013

[1] Un **bit** es la información necesaria para resolver una pregunta de sí o no con dos respuestas igualmente probables. Si el grupo tiene 50 % de cada clase, adivinar la clase de un ejemplo requiere un bit completo; si es puro, no requiere ninguno.

El pronóstico es la mejor primera pregunta, como sugería la intuición; la temperatura casi no informa.

Paso 3. Repetir en cada rama

La rama *Nublado* es pura: se convierte en una **hoja** «Sí». Las otras dos siguen mezcladas y repetimos el procedimiento **solo con sus días**, sin volver a usar el pronóstico:

Tabla 55.4.: Ganancias dentro de cada rama impura.

Rama	H de la rama	Ganancia Humedad	Ganancia Viento	Ganancia Temperatura
Soleado (6 días: 2 Sí, 4 No)	0.918	**0.918**	0.109	0.252
Lluvia (5 días: 3 Sí, 2 No)	0.971	0.020	**0.971**	0.020

En la rama *Soleado*, la humedad separa perfectamente: con humedad alta los cuatro días fueron «No» y con humedad normal los dos fueron «Sí». En la rama *Lluvia*, el viento separa perfectamente: con viento débil, tres «Sí»; con viento fuerte, dos «No». Cuando la ganancia es igual a la entropía de la rama, los hijos son puros.

Paso 4. El árbol y sus reglas

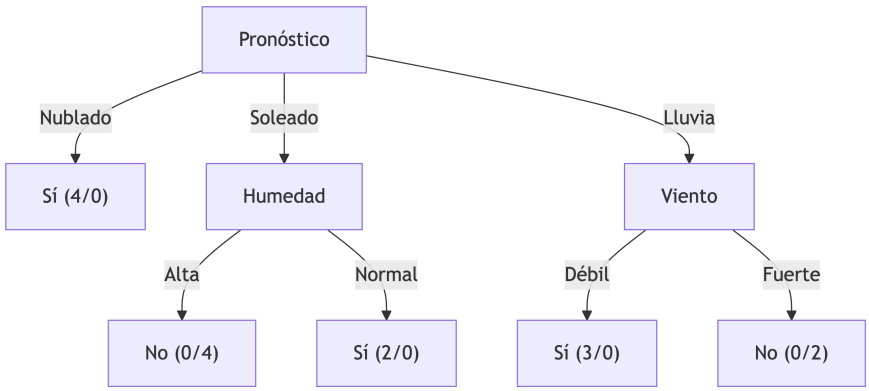


Figura 55.1.: Árbol construido por ID3 con los quince días. Entre paréntesis, días Sí/No que llegan a cada hoja.

Cada camino de la raíz a una hoja es una **regla**:

1. SI Pronóstico = Nublado ENTONCES Juega = Sí.
2. SI Pronóstico = Soleado Y Humedad = Alta ENTONCES Juega = No.

3. SI Pronóstico = Soleado Y Humedad = Normal ENTONCES Juega = Sí.
4. SI Pronóstico = Lluvia Y Viento = Débil ENTONCES Juega = Sí.
5. SI Pronóstico = Lluvia Y Viento = Fuerte ENTONCES Juega = No.

El árbol usa tres de los cuatro atributos (la temperatura nunca fue útil) y clasifica bien los quince días. Son, casi literalmente, las reglas que encontró el clasificador CBA del capítulo de clasificación asociativa sobre datos parecidos: dos caminos distintos hacia las mismas regularidades.

Formalización

El algoritmo de Hunt

Sea D_t el conjunto de ejemplos que llegan al nodo t y A el conjunto de atributos disponibles. El esquema recursivo de Hunt et al. (1966), del que ID3, C4.5 y CART son casos particulares, es:

1. Si todos los ejemplos de D_t tienen la misma clase c, t es una hoja con clase c.
2. Si A está vacío o se cumple un **criterio de parada**, t es una hoja con la clase mayoritaria de D_t.
3. Si no, se elige la **prueba** que mejor separa las clases, se divide D_t según su resultado y se aplica el procedimiento a cada hijo.

Si un hijo queda vacío (o, al predecir, aparece un valor no visto en el entrenamiento), se usa la clase mayoritaria del padre. Es una estrategia **voraz** (*greedy*): elige la mejor pregunta *local* sin volver atrás, lo que la hace rápida pero no garantiza el árbol óptimo global, cuyo cálculo es NP-difícil.

Ganancia de información (ID3)

Para un atributo A con valores $v \in \text{Valores}(A)$ y $S_v = \{x \in S : A(x) = v\}$,

$$\text{Gan}(S, A) = H(S) - \sum_{v \in \text{Valores}(A)} \frac{|S_v|}{|S|} H(S_v). \tag{55.2}$$

El segundo término es la **entropía condicional** $H(Y \mid A)$, la incertidumbre que queda sobre la clase cuando se conoce A; la ganancia es la **información mutua** entre A y la clase[2]. ID3 elige en cada nodo $\arg\max_A \text{Gan}(S, A)$ y crea una rama por cada valor del atributo (partición **multivía**).

Otras medidas de impureza

La entropía no es la única medida razonable. Para una proporción p de la clase positiva en un problema binario:

[2]La **información mutua** $I(A; Y) = H(Y) - H(Y \mid A)$ mide cuánto se reduce la incertidumbre sobre Y al conocer A. Es simétrica y vale 0 solo si A y Y son independientes, la misma idea que mide la prueba χ^2 del capítulo de selección de variables.

Tabla 55.5.: Medidas de impureza para dos clases.

Medida	Fórmula	Máximo ($p = 0.5$)
Entropía	$-p\log_2 p - (1-p)\log_2(1-p)$	1
Índice de Gini	$1 - p^2 - (1-p)^2 = 2p(1-p)$	0.5
Error de clasificación	$1 - \max(p, 1-p)$	0.5

Las tres valen 0 en nodos puros y son máximas con clases balanceadas. La entropía y Gini son **estrictamente cóncavas**, lo que hace que cualquier división que separe las proporciones produzca una ganancia positiva; el error de clasificación no lo es, y por eso puede no ver mejoras reales[3]. Gini es el criterio de CART y se estudia en el capítulo siguiente; en la práctica, entropía y Gini producen árboles muy parecidos.

El sesgo hacia atributos con muchos valores: C4.5

Agreguemos a la Tabla 55.1 la columna **Día** (D1, …, D15) como si fuera un atributo más. Al dividir por Día, cada grupo tiene un solo ejemplo, es puro, y la entropía después de dividir es 0. Su ganancia es la máxima posible, 0.971, muy por encima del pronóstico. ID3 elegiría como raíz un identificador que no sirve para predecir nada: un día nuevo tendrá un valor de Día nunca visto.

El problema es general: un atributo con muchos valores parte los datos en grupos pequeños que, por azar, tienden a ser puros. C4.5 corrige la ganancia dividiéndola entre la **información de la partición**, la entropía de los propios tamaños de los grupos:

$$\mathrm{SplitInfo}(S, A) = -\sum_{v} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}, \qquad \mathrm{RazónGan}(S, A) = \frac{\mathrm{Gan}(S, A)}{\mathrm{SplitInfo}(S, A)}. \tag{55.3}$$

Un atributo que parte en muchos grupos pequeños tiene un SplitInfo grande y queda penalizado. Para el pronóstico (grupos de 6, 4 y 5 días), SplitInfo = 1.566; para Día (15 grupos de 1), $\log_2 15 = 3.907$:

Tabla 55.6.: Ganancia y razón de ganancia en la raíz, con el atributo Día incluido.

Atributo	Ganancia	SplitInfo	Razón de ganancia
Día	0.971	3.907	**0.249**
Humedad	0.186	0.997	0.186
Pronóstico	0.280	1.566	0.179
Viento	0.060	0.918	0.065
Temperatura	0.013	1.566	0.008

Dos lecciones honestas salen de esta tabla:

[3] Si un nodo con 40 % de positivos se divide en un hijo pequeño con 10 % de positivos (la séptima parte de los ejemplos) y otro con 45 %, ambos hijos siguen teniendo mayoría negativa y el error de clasificación sigue siendo 0.40, aunque el primer hijo es mucho más puro. La entropía y Gini, por ser estrictamente cóncavas, sí registran esa mejora.

1. **La razón de ganancia atenúa el sesgo, pero no lo elimina.** Con 15 ejemplos la penalización $\log_2 15$ no alcanza y Día sigue ganando. Los identificadores, códigos postales finos o claves de producto deben **excluirse** antes de entrenar; ningún criterio de impureza los detecta por sí solo.
2. **Criterios distintos eligen raíces distintas.** Sin Día, la razón de ganancia prefiere la humedad (0.186) al pronóstico (0.179), porque la humedad parte en dos grupos y no en tres.

¿Cuál árbol es mejor? Lo decide la validación, no el criterio. Con validación cruzada dejando uno fuera sobre los quince días (calculada en la sección de Python), el árbol de ganancia de información acierta 14 de 15, el de razón de ganancia 12 y el de Gini 13; con Día incluido, cualquiera de ellos acierta solo 9, exactamente lo mismo que responder siempre la clase mayoritaria.

Atributos continuos

ID3 solo maneja atributos categóricos; C4.5 trata los numéricos con preguntas binarias «$A \le t$». Supongamos que la humedad se midió en porcentaje. Se ordenan los valores y se evalúan cortes en los puntos medios entre valores consecutivos:

Tabla 55.7.: Humedad numérica ordenada.

Humedad (%)	65	70	70	70	75	80	80	**85**	86	88	90	90	91	95	96
Juega	Sí	No	Sí	Sí	Sí	Sí	Sí	**No**	Sí	No	No	Sí	No	No	Sí

Tabla 55.8.: Ganancia de algunos cortes candidatos.

Corte t	$\le t$ (Sí/No)	$> t$ (Sí/No)	Ganancia
77.5	4 / 1	5 / 5	0.064
82.5	**6 / 1**	**3 / 5**	**0.186**
85.5	6 / 2	3 / 4	0.078
87.0	7 / 2	2 / 4	0.145

El mejor corte es 82.5 %. Con m valores distintos hay $m-1$ cortes posibles, pero Fayyad y Irani (1992) demostraron que el óptimo de la entropía siempre está en un **punto frontera**, entre dos valores consecutivos de clases distintas, así que basta evaluar esos. Tras ordenar una vez ($O(n \log n)$), todas las ganancias se calculan en una pasada actualizando conteos. A diferencia de un atributo categórico, un atributo numérico puede volver a usarse más abajo con otro corte.

Valores faltantes

C4.5 no descarta los ejemplos con un valor faltante en el atributo evaluado. Calcula la ganancia con los ejemplos que sí lo tienen y la multiplica por la fracción de ejemplos conocidos; al dividir, envía

cada ejemplo con valor faltante a **todas** las ramas con un peso proporcional al tamaño de cada rama. Al predecir hace lo mismo y combina las distribuciones de las hojas alcanzadas. CART usa en cambio **divisiones sustitutas** (*surrogate splits*), y `scikit-learn` (desde la versión 1.3) aprende en cada corte hacia qué hijo enviar los faltantes.

Criterios de parada y poda

Un árbol que crece hasta que todas sus hojas son puras **memoriza**: con datos ruidosos termina creando una hoja para cada excepción, como el 1-NN del capítulo introductorio. Hay dos formas de controlarlo:

- **Pre-poda**: detener el crecimiento antes, con una profundidad máxima, un mínimo de ejemplos por hoja o por nodo a dividir, o una ganancia mínima. Es barata, pero miope: una división poco útil puede habilitar otra muy útil debajo (el caso del XOR).
- **Post-poda**: hacer crecer el árbol completo y después **reemplazar subárboles por hojas** cuando eso no empeora el error. La *poda por error reducido* lo decide con un conjunto de validación (Quinlan 1987); C4.5 usa una **estimación pesimista** del error de cada hoja (el extremo superior de un intervalo de confianza) para podar sin datos adicionales; CART usa la **poda de costo-complejidad**, que se desarrolla en el capítulo siguiente.

Laboratorio: construye el árbol

El laboratorio construye el árbol con los quince días de la Tabla 55.1 y el criterio elegido. Haz clic en cualquier nodo para ver los puntajes de cada atributo en ese nodo; la profundidad máxima permite recorrer la construcción nivel por nivel. El estado inicial reproduce la Tabla 55.3 y el árbol de la Figura 55.1.

En la versión web, este laboratorio construye el árbol con el criterio elegido y permite inspeccionar cada nodo. Valores de referencia: con ganancia de información y sin el atributo Día, la raíz es Pronóstico (0.280), el árbol tiene 5 hojas, acierta 15/15 en entrenamiento y 14/15 en validación dejando uno fuera. Con razón de ganancia la raíz es Humedad (0.186) y la LOOCV es 12/15; con Gini, la raíz es Pronóstico y la LOOCV es 13/15. Con Día incluido, cualquier criterio lo elige como raíz y la LOOCV cae a 9/15.

Guía del laboratorio.

1. Con profundidad 1, comprueba la Tabla 55.3 en la tabla del nodo raíz. Después sube a 2 y haz clic en el nodo *Humedad* de la rama *Soleado*: ¿coinciden los puntajes con la Tabla 55.4?
2. Predice antes de mover: si cambias a *Razón de ganancia*, ¿qué atributo quedará en la raíz? Compruébalo y explica el papel de SplitInfo.
3. Incluye el atributo *Día* con cada uno de los tres criterios. ¿Alguno lo evita? ¿Qué le pasa a la exactitud de entrenamiento y a la de LOOCV?
4. Con *Gini* y sin Día, compara el árbol con el de ganancia de información. ¿Son iguales? ¿Por qué la LOOCV no es la misma si el árbol completo es igual?
5. Explica con la profundidad máxima qué es la pre-poda y qué se pierde con profundidad 1.

Implementación en Python

Entropía, ganancia y razón de ganancia

Las funciones siguientes reproducen las tablas de la sección anterior.

	ganancia	split_info	razón	Δ gini
Día	0.971	3.907	0.249	0.480
Pronóstico	0.280	1.566	0.179	0.142
Temperatura	0.013	1.566	0.009	0.009
Humedad	0.186	0.997	0.186	0.116
Viento	0.060	0.918	0.065	0.040

Las ganancias dentro de las ramas *Soleado* y *Lluvia* (Tabla 55.4):

	Humedad	Viento	Temperatura
Soleado	0.918	0.109	0.252
Lluvia	0.020	0.971	0.020

ID3 desde cero

La implementación recursiva sigue el algoritmo de Hunt. El árbol se representa con diccionarios; cada nodo interno guarda su clase mayoritaria para responder ante valores no vistos.

La validación dejando uno fuera compara criterios y muestra el efecto del atributo Día:

	criterio	atributos	raíz	entrenamiento	LOOCV
0	ganancia	sin Día	Pronóstico	15	14
1	ganancia	con Día	Día	15	9
2	razón	sin Día	Humedad	15	12
3	razón	con Día	Día	15	9
4	Δ gini	sin Día	Pronóstico	15	13
5	Δ gini	con Día	Día	15	9

Atributos continuos

	corte	ganancia	frontera
3	82.5	0.186	True
5	87.0	0.145	True
4	85.5	0.078	True
2	77.5	0.064	False
6	89.0	0.060	False

Árboles con `scikit-learn`

`DecisionTreeClassifier` implementa CART: con `criterion="entropy"` usa la entropía, pero siempre hace **divisiones binarias**. Con variables categóricas codificadas *one-hot*, cada pregunta es del tipo «¿Pronóstico = Nublado?», de modo que el árbol puede tener una forma distinta aunque represente reglas equivalentes.

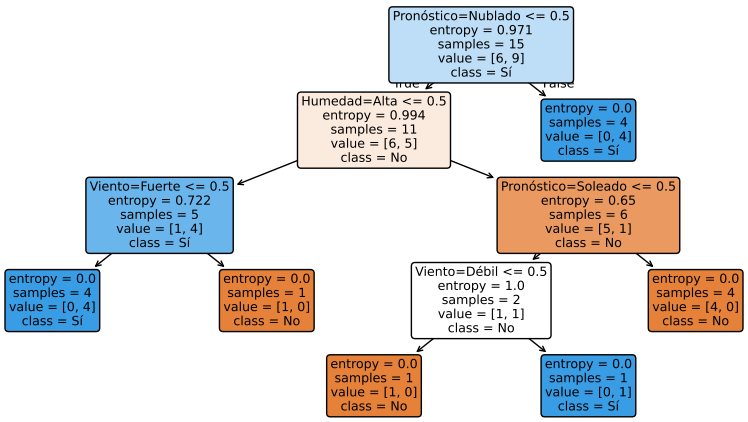


Figura 55.2.: Árbol de scikit-learn con criterio de entropía sobre las variables codificadas one-hot. Las divisiones son binarias y el árbol tiene otra forma que el de ID3, aunque también clasifica bien los quince días.

Complejidad, ventajas y limitaciones

En cada nodo, evaluar un atributo categórico cuesta $O(n_t)$ con n_t ejemplos en el nodo; un atributo numérico, $O(n_t \log n_t)$ por el ordenamiento (o $O(n_t)$ si se ordena una sola vez al inicio). Como cada nivel del árbol reparte los n ejemplos entre sus nodos, construir un árbol de profundidad d con p atributos cuesta del orden de $O(p\,n\,d)$, y predecir cuesta $O(d)$: recorrer un camino.

Ventajas.

- Se leen como reglas y se explican sin matemáticas.
- Aceptan variables categóricas y numéricas, no necesitan escalamiento y son invariantes ante transformaciones monótonas de las variables.
- Seleccionan variables automáticamente y capturan interacciones (la humedad importa solo si está soleado).
- Predicen muy rápido.

Limitaciones.

- **Inestabilidad**: un cambio pequeño en los datos puede cambiar la raíz y, con ella, todo el árbol. Es alta varianza, la misma que los métodos de ensamble aprovecharán para promediar.
- Sin poda, sobreajustan.
- Las fronteras son escalones paralelos a los ejes: una frontera diagonal requiere muchos escalones.
- La búsqueda voraz no ve interacciones que no se anuncian con una ganancia individual (el XOR).

- El sesgo hacia atributos con muchos valores obliga a revisar las variables de entrada.

Resumen

- Un árbol de decisión parte los datos con preguntas sucesivas hasta llegar a grupos casi puros; se construye de forma voraz y recursiva (algoritmo de Hunt).
- ID3 mide la mezcla con la **entropía** y elige la pregunta de mayor **ganancia de información**, la información mutua entre el atributo y la clase.
- La ganancia favorece atributos con muchos valores; C4.5 la divide entre la información de la partición (**razón de ganancia**), lo que atenúa pero no elimina el problema. Los identificadores deben excluirse.
- C4.5 maneja atributos continuos con cortes binarios (basta evaluar los puntos frontera), valores faltantes con pesos fraccionarios y poda pesimista.
- Un árbol completo memoriza; la pre-poda y la post-poda controlan su complejidad, y la validación decide entre criterios y tamaños.

56. CART: árboles binarios, índice de Gini y poda

CART (*Classification and Regression Trees*), propuesto por Breiman et al. (1984), es la versión de los árboles de decisión que domina la práctica: es el algoritmo de `scikit-learn` y el bloque de construcción de los bosques aleatorios y del *gradient boosting*. Comparte con ID3 la estrategia voraz y recursiva del capítulo anterior, pero cambia cuatro decisiones de diseño:

Tabla 56.1.: Diferencias principales entre ID3/C4.5 y CART.

	ID3 / C4.5	CART
Forma de las divisiones	Multivía (una rama por valor)	**Siempre binaria**
Impureza	Entropía, razón de ganancia	**Índice de Gini** (o entropía)
Variables numéricas	C4.5: cortes binarios	Cortes binarios, reutilizables
Control de complejidad	Poda pesimista	**Poda de costo-complejidad** con validación
Regresión	No	Sí (error cuadrático)

La restricción binaria parece una limitación, pero tiene ventajas: evita el sesgo hacia atributos con muchos valores (cada pregunta parte en solo dos grupos), permite agrupar categorías parecidas y produce árboles que, en el plano, son **particiones en rectángulos**. Este capítulo construye CART a mano con un ejemplo categórico, lo lleva al plano con variables numéricas y desarrolla su contribución más importante: la poda de costo-complejidad.

Intuición: ¿quién compra una laptop?

Catorce clientes de una tienda de electrónica, descritos por cuatro atributos categóricos (Han et al. 2011):

Tabla 56.2.: Catorce clientes, 9 compran y 5 no.

Id	Edad	Ingreso	Estudiante	Crédito	¿Compra?
1	joven	alto	no	favorable	no
2	joven	alto	no	excelente	no
3	adulto	alto	no	favorable	sí
4	mayor	medio	no	favorable	sí
5	mayor	bajo	sí	favorable	sí
6	mayor	bajo	sí	excelente	no
7	adulto	bajo	sí	excelente	sí
8	joven	medio	no	favorable	no

Id	Edad	Ingreso	Estudiante	Crédito	¿Compra?
9	joven	bajo	sí	favorable	sí
10	mayor	medio	sí	favorable	sí
11	joven	medio	sí	excelente	sí
12	adulto	medio	no	excelente	sí
13	adulto	alto	sí	favorable	sí
14	mayor	medio	no	excelente	no

CART no puede preguntar «¿qué edad tiene?» con tres respuestas. Debe preguntar algo de sí o no: «¿es adulto?», «¿es joven?» o «¿es joven o mayor?». La pregunta adecuada es la que deja los dos grupos más puros.

Construcción paso a paso

Paso 1. La impureza de Gini en la raíz

El **índice de Gini** de un nodo con proporciones de clase $p_1, \dots, p_K$ es

$$G = 1 - \sum_{k=1}^{K} p_k^2. \tag{56.1}$$

Es la probabilidad de equivocarse si se etiqueta un ejemplo al azar según las proporciones del nodo[1]. En la raíz, con 9 «sí» y 5 «no»:

$$G_{\text{raíz}} = 1 - \left(\tfrac{9}{14}\right)^2 - \left(\tfrac{5}{14}\right)^2 = 1 - 0.413 - 0.128 = 0.459.$$

Paso 2. Las preguntas binarias de un atributo

Edad tiene tres valores, así que admite $2^{3-1} - 1 = 3$ preguntas binarias distintas. Para «¿edad = adulto?»: los 4 adultos compran todos (Gini 0) y los otros 10 se reparten 5 y 5 (Gini 0.5). La impureza ponderada es

$$\tfrac{4}{14}(0) + \tfrac{10}{14}(0.5) = 0.357, \qquad \Delta G = 0.459 - 0.357 = 0.102.$$

[1] Si en un nodo hay una proporción p_k de cada clase y se etiqueta un ejemplo al azar con la clase k con probabilidad p_k, la probabilidad de acertar es $\sum_k p_k^2$ y la de equivocarse, $1 - \sum_k p_k^2$. Es también la varianza de la variable indicadora de clase sumada sobre las clases.

Tabla 56.3.: Las tres preguntas binarias para la edad.

Pregunta	Sí cumple (sí/no)	No cumple (sí/no)	Gini ponderado	Reducción ΔG
¿edad = adulto?	4 / 0	5 / 5	0.357	**0.102**
¿edad = joven?	2 / 3	7 / 2	0.394	0.066
¿edad = mayor?	3 / 2	6 / 3	0.457	0.002

Paso 3. Comparar atributos

Repitiendo el cálculo para cada atributo y quedándonos con su mejor pregunta:

Tabla 56.4.: Mejor pregunta de cada atributo en la raíz.

Atributo	Mejor pregunta	Gini ponderado	Reducción
Edad	¿adulto?	**0.357**	**0.102**
Estudiante	¿estudiante?	0.367	0.092
Crédito	¿excelente?	0.429	0.031
Ingreso	¿alto?	0.443	0.016

La raíz pregunta «¿edad = adulto?». La rama «sí» es pura (4 compras) y se vuelve hoja; la rama «no» (10 clientes, 5 y 5) se vuelve a dividir con el mismo procedimiento, donde **la edad puede reaparecer** con otra pregunta («¿joven?»), algo imposible en ID3.

Paso 4. Variables numéricas: cortes

Con una variable numérica, la pregunta es «¿$x \le t$?». Se ordenan los valores y se prueban los puntos medios entre valores consecutivos. Con cuatro clientes de edades 20 (sí), 25 (sí), 30 (no) y 40 (no):

Tabla 56.5.: Cortes candidatos para una variable numérica.

Corte t	Izquierda	Derecha	Gini ponderado
22.5	1 sí	1 sí, 2 no	$\frac{3}{4}(1-\frac{1}{9}-\frac{4}{9}) = 0.333$
27.5	**2 sí**	**2 no**	**0**
35.0	2 sí, 1 no	1 no	$\frac{3}{4}(1-\frac{4}{9}-\frac{1}{9}) = 0.333$

Con dos variables numéricas, cada corte es una recta vertical u horizontal, y el árbol completo parte el plano en **rectángulos**, cada uno con la clase mayoritaria de los puntos que contiene. El laboratorio de este capítulo lo muestra con 60 puntos.

Paso 5. Podar: el eslabón más débil

Un árbol que crece hasta tener hojas puras sobreajusta. En los datos del laboratorio, el árbol completo tiene 18 hojas, acierta el 100 % del entrenamiento y solo el 71.7 % de 300 puntos de prueba. CART propone crecer el árbol completo y después **podarlo**, cambiando cada subárbol por una hoja cuando la mejora que aporta no compensa su tamaño.

Para medir la mejora, sea $R(t) = \frac{n_t}{n} G_t$ la impureza ponderada del nodo t si fuera una hoja, y $R(T_t)$ la suma de esa cantidad sobre las hojas del subárbol que cuelga de t. El **costo por hoja** de conservar el subárbol es

$$g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1},$$

la reducción de impureza que aporta cada hoja adicional. Tomemos el árbol de cuatro hojas que aparece en la secuencia de poda del laboratorio:

Tabla 56.6.: Costo por hoja de cada nodo interno del árbol de cuatro hojas.

Nodo	Pregunta	Ejemplos (0/1)	G_t	$R(t)$	$R(T_t)$	Hojas	$g(t)$
raíz	$x_2 \leq 5.70$	33 / 27	0.495	0.495	0.224	4	**0.090**
izquierdo	$x_1 \leq 4.05$	15 / 19	0.493	0.279	0.164	2	0.115
derecho	$x_1 \leq 2.30$	18 / 8	0.426	0.185	0.060	2	0.125

Por ejemplo, para la raíz, las cuatro hojas tienen 13, 21, 6 y 20 puntos con impurezas 0.260, 0.308, 0 y 0.180, así que $R(T) = \frac{13}{60}0.260 + \frac{21}{60}0.308 + 0 + \frac{20}{60}0.180 = 0.224$ y $g = (0.495 - 0.224)/3 = 0.090$.

El nodo con el menor $g(t)$ es el **eslabón más débil**: el primero que conviene podar cuando se penaliza el tamaño. Sorprendentemente, aquí es la raíz: con una penalización mayor que 0.090 por hoja, el árbol pasa de cuatro hojas a una sola. Ese valor de g es el α al que ocurre la poda.

Formalización

Impureza y reducción de impureza

Para un nodo t con n_t ejemplos que una pregunta s divide en t_L y t_R, CART elige

$$s^* = \arg\max_s \ \Delta G(s,t) = G_t - \frac{n_{t_L}}{n_t} G_{t_L} - \frac{n_{t_R}}{n_t} G_{t_R}. \tag{56.2}$$

`scikit-learn` permite `criterion="gini"` o `"entropy"`; como se vio en el capítulo anterior, ambas son estrictamente cóncavas y casi siempre eligen la misma pregunta. Gini es algo más rápido de calcular porque no usa logaritmos.

Atributos categóricos. Un atributo con m valores admite $2^{m-1}-1$ preguntas binarias, número que explota con muchas categorías. Para dos clases, Breiman et al. (1984) demostraron un atajo: si se ordenan las categorías por su proporción de la clase positiva, la partición óptima es uno de los $m-1$ cortes de ese orden. En el ejemplo, las proporciones de compra son joven 0.40, mayor 0.60 y adulto 1.00; los cortes del orden son {joven} | {mayor, adulto} y {joven, mayor} | {adulto}, y el segundo es el óptimo de la Tabla 56.3.

Atributos numéricos. Tras ordenar los n_t valores, todos los cortes se evalúan en una pasada actualizando los conteos de cada lado. Una variable puede volver a usarse más abajo con otro corte. Como solo importa el orden de los valores, el árbol es **invariante a transformaciones monótonas** (logaritmos, escalamiento): no hace falta estandarizar.

Importancia de las variables

Sumando, para cada variable, las reducciones de impureza ponderadas $\frac{n_t}{n}\Delta G$ de todos los nodos donde se usa, y normalizando para que sumen 1, se obtiene la **importancia por disminución de impureza** (MDI), el atributo `feature_importances_` de `scikit-learn`. Es gratuita, pero sesgada a favor de variables numéricas o con muchas categorías, que ofrecen más cortes posibles. La importancia por permutación y los valores SHAP, que se estudian con los ensambles, son alternativas más fiables.

Poda de costo-complejidad

Para $\alpha \geq 0$ se define el costo-complejidad de un subárbol T del árbol completo T_0:

$$R_\alpha(T) = R(T) + \alpha\,|T|, \tag{56.3}$$

donde $R(T)$ es la impureza ponderada total de sus hojas (en la formulación original de Breiman et al. (1984), la tasa de error) y $|T|$ su número de hojas. Con $\alpha = 0$ gana el árbol completo; al crecer α, conviene ir cambiando subárboles por hojas. Un subárbol T_t deja de compensar justo cuando $R(t) + \alpha = R(T_t) + \alpha|T_t|$, es decir, cuando $\alpha = g(t)$.

Breiman et al. demostraron que los árboles óptimos para cada α forman una **secuencia anidada** $T_0 \supset T_1 \supset \cdots \supset \{\text{raíz}\}$, que se obtiene podando repetidamente el eslabón más débil. Los valores $0 = \alpha_0 < \alpha_1 < \cdots$ en que cambia el árbol son la **ruta de poda**. Para elegir α se usa validación cruzada: se evalúa cada α de la ruta y se elige el de menor error (o el más grande dentro de un error estándar del mínimo).

Árboles de regresión

Con una respuesta numérica, la hoja predice el promedio de sus ejemplos y la impureza es la varianza (o la suma de cuadrados): CART elige el corte que minimiza $\sum_{i \in t_L} (y_i - \bar{y}_L)^2 + \sum_{i \in t_R} (y_i - \bar{y}_R)^2$. Los árboles de regresión se desarrollan en el capítulo de ensambles, donde son la base del *bagging* y del *boosting*.

Laboratorio: CART en el plano

El laboratorio entrena CART con 60 puntos de dos clases y lo evalúa con 300 puntos de prueba generados por la misma regla: la clase 1 es probable en dos zonas rectangulares (por eso un árbol es el modelo natural) y con un 12 % de etiquetas invertidas al azar. Con $\alpha = 0.02$ se obtiene el árbol de la Tabla 56.6. Cuando dos cortes producen exactamente la misma reducción de Gini, el laboratorio elige el primero en un orden fijo, mientras que `scikit-learn` decide al azar; por eso el árbol completo puede diferir en algún corte profundo (exactitud en prueba 0.723 frente a 0.717), aunque los árboles podados coinciden.

En la versión web, este laboratorio entrena CART en el plano y lo poda con . Valores de referencia: sin restricciones el árbol tiene 18 hojas, exactitud de entrenamiento 1 y de prueba cercana a 0.72; el primer corte es $x_2 \leq 5.70$. La ruta de poda cambia de árbol en 0.0141 (14 hojas), 0.0148 (12), 0.0152 (10), 0.0172 (7), 0.0188 (4 hojas, prueba 0.853) y 0.0902 (1 hoja). Con entre 0.019 y 0.090 se obtiene el árbol de cuatro hojas de la Tabla 56.6.

Guía del laboratorio.

1. Sin restricciones, cuenta las hojas y localiza las que rodean a un solo punto. ¿Qué exactitud tienen en prueba?
2. En el panel de búsqueda del primer corte, comprueba que el máximo de la curva roja está en $x_2 = 5.70$. ¿Cuál es el mejor corte en x_1 y por qué pierde?
3. Predice antes de mover: ¿con qué aparece el árbol de cuatro hojas? Compruébalo en la ruta de poda y verifica la Tabla 56.6 en la tabla de nodos.
4. Sube poco a poco desde 0.02 hasta 0.09. ¿Por qué el árbol no cambia en todo ese intervalo y luego salta a una sola hoja?
5. Con $= 0$, usa solo el mínimo por hoja (por ejemplo 5) o la profundidad máxima (2). ¿Llegas a un árbol parecido al podado? ¿Qué ventaja tiene la poda sobre la pre-poda?
6. Cambia a *Prueba* y compara las regiones con los rectángulos verdes (las zonas reales). ¿Qué parte de la estructura recuperó el árbol podado?

Implementación en Python

El ejemplo categórico

Las funciones siguientes enumeran todas las preguntas binarias de cada atributo y reproducen las Tabla 56.3 y Tabla 56.4.

	atributo	pregunta	gini ponderado	reducción
0	edad	¿edad ['adulto']?	0.357	0.102
0	estudiante	¿estudiante ['no']?	0.367	0.092
1	edad	¿edad ['joven']?	0.394	0.066
0	crédito	¿crédito ['excelente']?	0.429	0.031
0	ingreso	¿ingreso ['alto']?	0.443	0.016
1	ingreso	¿ingreso ['bajo']?	0.450	0.009
2	edad	¿edad ['mayor']?	0.457	0.002
2	ingreso	¿ingreso ['medio']?	0.458	0.001

`DecisionTreeClassifier` no acepta categorías directamente: hay que codificarlas. Con *one-hot*, cada pregunta es «¿categoría = valor?», que en este ejemplo basta para encontrar la misma raíz:

El ejemplo numérico y la ruta de poda

Generamos los mismos datos del laboratorio: dos zonas rectangulares donde la clase 1 tiene probabilidad 0.88 y, fuera de ellas, 0.12.

		hojas	entrenamiento	prueba
0	0.0000	18	1.0000	0.7167
1	0.0141	14	0.9667	0.7433
2	0.0148	12	0.9500	0.7567
3	0.0152	10	0.9333	0.7733
4	0.0172	7	0.9000	0.8433
5	0.0188	4	0.8667	0.8533
6	0.0902	1	0.5500	0.5433

El árbol de cuatro hojas y el costo por hoja de sus nodos internos (Tabla 56.6) se obtienen de la estructura interna `tree_`:

	nodo	pregunta	G	R(t)	R(T_t)	hojas	g(t)
0	0	x2 <= 5.70	0.4950	0.4950	0.2243	4	0.0902
1	1	x1 <= 4.05	0.4931	0.2794	0.1643	2	0.1151
2	4	x1 <= 2.30	0.4260	0.1846	0.0600	2	0.1246

Elegir con validación cruzada

En la práctica no hay un conjunto de prueba grande para elegir : se usa validación cruzada sobre el entrenamiento con los valores de la ruta.

Figura 56.1.

Importancia de las variables

	árbol completo	árbol podado
x1	0.693	0.886
x2	0.307	0.114

En el árbol podado, x_1 acumula casi el 90 % de la importancia aunque la primera pregunta sea sobre x_2: ese primer corte reduce poco la impureza ponderada ($0.495 - 0.279 - 0.185 = 0.031$), porque solo separa las dos mitades del plano, y son los dos cortes en x_1 los que aíslan las zonas (0.115 y 0.125). La importancia mide cuánto contribuye cada variable, no qué pregunta va primero.

Complejidad, ventajas y limitaciones

Con p variables numéricas ordenadas una vez al inicio, encontrar el mejor corte de un nodo con n_t ejemplos cuesta $O(p\, n_t)$; como cada nivel reparte los n ejemplos, el árbol cuesta $O(p\, n \log n)$ si está balanceado (profundidad $\approx \log n$) y hasta $O(p\, n^2)$ en el peor caso. La ruta de poda se calcula después en tiempo proporcional al número de nodos.

Ventajas. Las del capítulo anterior (interpretabilidad, variables mixtas, sin escalamiento, interacciones automáticas), más un tratamiento uniforme de variables numéricas y categóricas, la poda con fundamento estadístico y la extensión directa a regresión.

Limitaciones.

- **Alta varianza**: pequeños cambios en los datos cambian los cortes de arriba y, con ellos, todo el árbol. Es el problema que resuelven los bosques aleatorios.
- Fronteras en escalones paralelos a los ejes: una frontera diagonal se aproxima con muchos cortes.
- Las categorías con muchos niveles generan muchas preguntas posibles y favorecen el sobreajuste; conviene agruparlas o usar codificaciones por frecuencia u objetivo.
- La importancia MDI está sesgada; para interpretar conviene usar permutación o SHAP.

Resumen

- CART construye árboles **binarios**: preguntas «¿$A \in S$?» para categorías y «¿$x \le t$?» para números, elegidas por la mayor reducción del índice de Gini.
- Un atributo categórico con m valores admite $2^{m-1} - 1$ preguntas; para dos clases basta ordenar las categorías por su proporción de positivos y probar $m - 1$ cortes.
- En el plano, un árbol es una partición en rectángulos. Crecido hasta hojas puras, rodea el ruido.
- La **poda de costo-complejidad** minimiza $R(T) + \alpha|T|$; el **eslabón más débil**, $\min_t g(t)$, define la ruta de poda, una secuencia anidada de árboles. α se elige con validación cruzada.
- La importancia por disminución de impureza es un subproducto útil pero sesgado.

57. Árboles de regresión y ensambles: bagging, bosques aleatorios y AdaBoost

Los dos capítulos anteriores terminaron con la misma advertencia: un árbol de decisión es **inestable**. Basta cambiar unos cuantos ejemplos para que cambie la raíz y, con ella, todo el árbol. En el lenguaje del capítulo introductorio, los árboles profundos tienen poco sesgo y **mucha varianza**.

Este capítulo convierte esa debilidad en una fortaleza. Si un árbol es ruidoso, entrenemos muchos árboles distintos y combinemos sus respuestas: los errores de unos compensan los de otros. Esa es la idea de los **métodos de ensamble** (Dietterich 2000), que con árboles como base están entre los clasificadores más eficaces para datos tabulares. Estudiamos dos familias:

- **Promediar modelos independientes** para reducir la varianza: *bagging* y **bosques aleatorios**.
- **Construir modelos en secuencia**, cada uno concentrado en los errores del anterior, para reducir el sesgo: **boosting**, con AdaBoost en este capítulo y *gradient boosting* y XGBoost en el siguiente.

Como varios de estos métodos se entienden mejor con una respuesta numérica, empezamos con los **árboles de regresión**, que además son la pieza básica del *gradient boosting*.

Árboles de regresión

Un árbol de regresión parte el espacio de predictoras igual que CART, pero en cada hoja predice un **número**: el promedio de las respuestas de los ejemplos que llegan a ella. El modelo es una función escalonada

$$\hat{f}(\mathbf{x}) = \sum_{h=1}^{J} c_h \, \mathbb{1}[\mathbf{x} \in R_h], \qquad c_h = \frac{1}{|R_h|} \sum_{\mathbf{x}_i \in R_h} y_i, \tag{57.1}$$

donde $R_1, \dots, R_J$ son las regiones (hojas). El promedio es la constante que minimiza el error cuadrático dentro de cada hoja, y la impureza de un nodo es su **suma de cuadrados** $\text{SSE} = \sum_i (y_i - \bar{y})^2$. Cada corte se elige para reducirla lo más posible.

Ejemplo manual: precio de laptops según la RAM

Tabla 57.1.: Seis laptops.

Laptop	A	B	C	D	E	F
RAM (GB)	4	4	8	8	16	16
Precio ($)	6 500	7 200	10 500	12 000	21 000	24 000

Sin dividir, la predicción es el promedio, 13 533.33, y la suma de cuadrados es 266.43×10^6. Hay dos cortes posibles:

Tabla 57.2.: Los dos cortes candidatos.

Corte	Izquierda (promedio)	Derecha (promedio)	SSE después	Reducción
RAM $\leq$ 6	A, B (6 850)	C, D, E, F (16 875)	132.43×10^6	50.3 %
RAM $\leq$ 12	**A–D (9 050)**	**E, F (22 500)**	$\mathbf{25.23 \times 10^6}$	**90.5 %**

Por ejemplo, para RAM $\leq$ 12: $(6500-9050)^2 + (7200-9050)^2 + (10500-9050)^2 + (12000-9050)^2 + (21000-22500)^2 + (24000-22500)^2 = 25.23 \times 10^6$. El árbol de profundidad 1 dice: «si la RAM es de 12 GB o menos, 9 050; si no, 22 500».

Todo lo visto en CART se aplica sin cambios: el crecimiento voraz, la poda de costo-complejidad (ahora con la suma de cuadrados como $R(T)$) y la tendencia a sobreajustar si se deja crecer el árbol hasta tener una observación por hoja. El laboratorio de bagging muestra este sobreajuste con 40 puntos.

¿Por qué funciona combinar modelos?

Intuición: el voto de cinco jueces

Supongamos cinco clasificadores que aciertan cada uno el 70 % de las veces y cuyos errores son **independientes**. Si decidimos por mayoría, el ensamble acierta cuando al menos tres de los cinco aciertan:

$$P(\text{mayoría correcta}) = \sum_{k=3}^{5} \binom{5}{k} 0.7^k \, 0.3^{5-k} = 0.309 + 0.360 + 0.168 = 0.837.$$

Con 25 clasificadores así, la exactitud de la mayoría sube a 0.98. Es el **teorema del jurado de Condorcet**: si cada juez es mejor que el azar y sus errores son independientes, la mayoría se vuelve casi infalible. Dos condiciones hacen el trabajo: que cada modelo sea **mejor que el azar** y que sus errores sean **diversos**. Si los cinco cometieran exactamente los mismos errores, votar no serviría de nada.

Formalización: la varianza de un promedio

Para predicciones numéricas, la misma idea se expresa con varianzas. Si B modelos tienen cada uno varianza σ^2 y correlación ρ entre sí, la varianza de su promedio es

$$\operatorname{Var}\Big(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(\mathbf{x})\Big) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2. \tag{57.2}$$

Al aumentar B, el segundo término desaparece, pero el primero no: **la correlación entre modelos pone un piso** a la reducción de varianza. Toda la estrategia de los bosques aleatorios consiste en bajar ρ sin subir demasiado σ^2. Como promediar no cambia el valor esperado, el **sesgo** del ensamble es el de sus modelos: por eso se promedian árboles profundos, de poco sesgo y mucha varianza.

Bagging

Si tuviéramos muchos conjuntos de entrenamiento independientes, entrenaríamos un árbol en cada uno y los promediaríamos. Solo tenemos uno. *Bagging* (*bootstrap aggregating*), propuesto por Breiman (1996), simula esos conjuntos con **muestras bootstrap**: muestras de tamaño n tomadas **con reemplazo** del conjunto original.

Construcción paso a paso

Paso 1. Una muestra bootstrap. Con las seis laptops, un sorteo con reemplazo podría dar $\{A, A, C, D, F, F\}$: la laptop A y la F aparecen dos veces, y la B y la E **no aparecen**.

Paso 2. Un árbol con esa muestra. El mejor corte vuelve a ser RAM $\geq$ 12, pero las hojas cambian: a la izquierda $(6500 + 6500 + 10500 + 12000)/4 = 8\,875$ y a la derecha $24\,000$.

Paso 3. Los ejemplos fuera de la bolsa. B y E no participaron en este árbol, así que sirven para evaluarlo sin trampa. Sus errores son $7\,200 - 8\,875 = -1\,675$ y $21\,000 - 24\,000 = -3\,000$. Son los ejemplos **fuera de la bolsa** (*out-of-bag*, OOB).

Paso 4. Repetir y promediar. Con B muestras bootstrap se obtienen B árboles distintos. La predicción de *bagging* es el promedio (en regresión) o el voto mayoritario (en clasificación):

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(\mathbf{x}). \tag{57.3}$$

El error fuera de la bolsa

La probabilidad de que una observación concreta **no** salga en ninguna de las n extracciones es

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow[n\to\infty]{} e^{-1} \approx 0.368. \tag{57.4}$$

Con $n = 6$ es $(5/6)^6 = 0.335$. Es decir, cada árbol deja fuera alrededor de un tercio de los datos. Prediciendo cada observación solo con los árboles que **no** la vieron y promediando esos errores se obtiene el **error OOB**, una estimación del error de generalización casi gratuita, comparable a una validación cruzada, sin entrenar modelos adicionales.

Bosques aleatorios

En *bagging* los árboles son distintos, pero no tanto: si hay una variable muy predictiva, casi todos la usan en la raíz y sus predicciones están muy correlacionadas, de modo que el piso $\rho\sigma^2$ de la Ecuación 57.2 es alto. Los **bosques aleatorios** de Breiman (2001a) añaden una segunda fuente de azar: **en cada nodo**, el mejor corte se busca solo entre m variables elegidas al azar de las p disponibles. Así, otras variables tienen oportunidad de ocupar las preguntas importantes, los árboles se parecen menos y su promedio es más estable.

Los hiperparámetros principales son:

Tabla 57.3.: Hiperparámetros de un bosque aleatorio.

Hiperparámetro	`scikit-learn`	Valor habitual	Efecto
Número de árboles B	`n_estimators`	200–1000	Más árboles nunca sobreajustan; solo cuestan tiempo
Variables por nodo m	`max_features`	$\sqrt{p}$ (clasificación), $p/3$ (regresión)	Menor m: árboles menos correlacionados pero más débiles
Tamaño de hoja	`min_samples_leaf`	1–5	Hojas más grandes: menos varianza por árbol
Profundidad	`max_depth`	sin límite	Normalmente se dejan crecer

Con $m = p$ el bosque es exactamente *bagging*. Una propiedad práctica importante: aumentar B no produce sobreajuste; el error converge y basta elegir B suficientemente grande.

Importancia de las variables

Un bosque pierde la interpretabilidad de un árbol único, pero ofrece dos medidas de importancia:

- **Disminución media de impureza** (MDI): la suma de las reducciones de Gini de cada variable en todos los árboles. Es gratuita, pero está sesgada a favor de variables numéricas o con muchas categorías, que tienen más cortes donde «encontrar» reducciones por azar (Strobl et al. 2007).
- **Importancia por permutación**: se desordena al azar una columna en los datos de prueba (o en los OOB) y se mide cuánto empeora el desempeño. Si la variable importa, desordenarla destruye información; si no, el error casi no cambia. Es más fiable, aunque reparte la importancia entre variables muy correlacionadas.

Laboratorio: bagging de árboles de regresión

Cuarenta puntos generados con $y = 2\sin x + 0.3x + \varepsilon$, con ruido de desviación 0.6 (varianza 0.36, el error irreducible). El estado inicial es un **único** árbol sin límite de profundidad: memoriza los datos. Al pasar a *Bootstrap* y aumentar el número de árboles, el promedio se suaviza.

En la versión web, este laboratorio promedia árboles de regresión entrenados con muestras bootstrap. Valores de referencia: un único árbol sin límite de profundidad tiene ECM de prueba cercano a 0.62 (el error irreducible es 0.36); con profundidad 4, 0.554. El promedio de 100 árboles bootstrap profundos baja el ECM de prueba a cerca de 0.50, y el error fuera de la bolsa da una estimación parecida sin usar la prueba.

Guía del laboratorio.

1. En el estado inicial, ¿por qué el ECM de entrenamiento es casi 0 y el de prueba ronda 0.62? Compara la escalera con la función real (verde).
2. Predice antes de mover: con *Sin remuestreo* y $B = 50$, ¿mejorará el error de prueba? Compruébalo y explica.
3. Cambia a *Bootstrap* y sube B de 1 a 100. ¿Dónde se aplana la curva? ¿Llega al error irreducible?
4. Con $B = 100$, baja la profundidad a 1. ¿Por qué ahora el bagging casi no mejora al árbol único? Relaciónalo con la Ecuación 57.2 y con el sesgo.
5. Compara el error fuera de la bolsa con el de prueba para varias semillas. ¿Es una buena estimación?

Boosting y AdaBoost

Bagging entrena modelos en paralelo y los trata por igual. **Boosting** los entrena **en secuencia**: cada modelo nuevo se concentra en los casos que los anteriores clasificaron mal, y al final se combinan con pesos según su calidad. La pregunta que lo originó fue teórica (Schapire 1990): ¿se puede convertir un **clasificador débil**, apenas mejor que el azar, en uno arbitrariamente preciso? AdaBoost (Freund y Schapire 1997) fue la respuesta práctica.

Construcción paso a paso

Diez puntos sobre una recta con etiquetas $+1$ y -1:

Tabla 57.4.: Diez puntos que ningún corte único separa.

x	1	2	3	4	5	6	7	8	9	10
y	$+$	$+$	$+$	$-$	$-$	$-$	$+$	$+$	$+$	$-$

El clasificador débil será un **tocón**: un solo corte «$x \le t$ clase». Ningún tocón separa estos datos; el mejor comete 3 errores.

Ronda 1. Todos los pesos valen $w_i = 1/10$. El mejor tocón es «$x \le 3.5$ $+$, si no $-$», que falla en $x = 7, 8, 9$. Su **error ponderado** es $\varepsilon_1 = 0.3$ y su **peso** en el ensamble es

$$\alpha_1 = \tfrac{1}{2} \ln \frac{1 - \varepsilon_1}{\varepsilon_1} = \tfrac{1}{2} \ln \frac{0.7}{0.3} = 0.424.$$

Ahora se **repesan** los ejemplos: los mal clasificados se multiplican por $e^{\alpha_1} = 1.528$ y los bien clasificados por $e^{-\alpha_1} = 0.655$; después se normaliza para que sumen 1. Los tres errores pasan a pesar 0.1667 cada uno y los siete aciertos 0.0714. Observa que los mal clasificados suman ahora exactamente 0.5: para el siguiente tocón, el anterior es tan bueno como lanzar una moneda.

Ronda 2. Con esos pesos, el mejor tocón es «$x \le 9.5$ $+$, si no $-$», que falla en $x = 4, 5, 6$: $\varepsilon_2 = 3 \times 0.0714 = 0.214$ y $\alpha_2 = \tfrac{1}{2} \ln(0.786/0.214) = 0.650$. Ese tocón casi no tendría sentido por sí solo; tiene sentido **porque** los puntos 7, 8 y 9 ahora pesan mucho.

Ronda 3. El mejor tocón es «$x \le 6.5$ $-$, si no $+$», que falla en $x = 1, 2, 3$ y 10, ahora livianos: $\varepsilon_3 = 0.182$ y $\alpha_3 = 0.752$.

El ensamble. La predicción es el signo de la suma ponderada de los tres tocones:

$$F(x) = 0.424\, h_1(x) + 0.650\, h_2(x) + 0.752\, h_3(x), \qquad \hat{y} = \operatorname{signo}(F(x)).$$

Tabla 57.5.: El ensamble de tres tocones clasifica bien los diez puntos.

x	h_1	h_2	h_3	$F(x)$	$\hat{y}$	y
1–3	$+$	$+$	$-$	$0.424 + 0.650 - 0.752 = 0.321$	$+$	$+$
4–6	$-$	$+$	$-$	$-0.424 + 0.650 - 0.752 = -0.526$	$-$	$-$
7–9	$-$	$+$	$+$	$-0.424 + 0.650 + 0.752 = 0.978$	$+$	$+$
10	$-$	$-$	$+$	$-0.424 - 0.650 + 0.752 = -0.321$	$-$	$-$

Tres clasificadores que fallan, cada uno, tres o cuatro de diez puntos producen juntos un clasificador perfecto en entrenamiento. Ninguno de ellos podía dibujar la frontera con tres cambios de clase; su suma sí.

Formalización

Con etiquetas $y_i \in \{-1, +1\}$, pesos iniciales $w_i = 1/n$ y M rondas, AdaBoost repite:

1. Entrenar el clasificador débil h_m minimizando el error ponderado $\varepsilon_m = \sum_i w_i \, \mathbb{1}[h_m(\mathbf{x}_i) \neq y_i]$.
2. Calcular $\alpha_m = \frac{1}{2} \ln \frac{1-\varepsilon_m}{\varepsilon_m}$, positivo si $\varepsilon_m < 0.5$.
3. Actualizar $w_i \leftarrow w_i \exp(-\alpha_m \, y_i \, h_m(\mathbf{x}_i))$ y normalizar.

La predicción es signo$\big(\sum_m \alpha_m h_m(\mathbf{x})\big)$. Friedman et al. (2000) mostraron que AdaBoost minimiza, paso a paso, la **pérdida exponencial** $\sum_i e^{-y_i F(\mathbf{x}_i)}$ sobre modelos aditivos F; cada ronda agrega el término que más la reduce. Esta lectura como **optimización por etapas** es la que generaliza el *gradient boosting* del capítulo siguiente a cualquier pérdida.

Dos consecuencias prácticas:

- El error de entrenamiento cae exponencialmente rápido mientras cada ε_m esté por debajo de 0.5. Pero la pérdida exponencial castiga sin límite a los puntos muy mal clasificados, de modo que AdaBoost es **sensible a etiquetas erróneas y atípicos**: termina concentrando todo el peso en ellos.
- Boosting reduce sobre todo el **sesgo** (combina modelos débiles, de mucho sesgo) y puede sobreajustar si se usan demasiadas rondas o modelos base demasiado fuertes. El número de rondas y la complejidad del modelo base se eligen con validación.

`scikit-learn` implementa la variante SAMME, que usa $\alpha_m = \ln \frac{1-\varepsilon_m}{\varepsilon_m}$ (el doble) y solo aumenta el peso de los errores; tras normalizar, los pesos y las predicciones son idénticos a los de la versión clásica.

Laboratorio: AdaBoost ronda por ronda

El tamaño de cada punto es proporcional a su peso **al inicio** de la ronda elegida. El modo *Diez puntos* reproduce las tablas anteriores. El modo *Círculo* usa 150 puntos de entrenamiento y 300 de prueba en el plano; la clase +1 está dentro de un círculo y un 3 % de las etiquetas están invertidas; los tocones solo pueden hacer cortes verticales u horizontales.

En la versión web, este laboratorio recorre AdaBoost ronda por ronda. Valores de referencia con los diez puntos: ronda 1, tocón $x \leq 3.5 \Rightarrow +$, $\varepsilon = 0.300$, $\alpha = 0.424$; ronda 2, $x \leq 9.5 \Rightarrow +$, $\varepsilon = 0.214$, $\alpha = 0.650$; ronda 3, $x \leq 6.5 \Rightarrow -$, $\varepsilon = 0.182$, $\alpha = 0.752$, y el ensamble ya no comete errores. En el modo *Círculo* el error de prueba baja de 0.27 con un tocón a cerca de 0.08 con 30 rondas y después se estabiliza, mientras el de entrenamiento sigue bajando.

Guía del laboratorio.

1. Con *Diez puntos*, recorre las rondas 1 a 3 y comprueba ε, α y los pesos contra la construcción paso a paso.
2. En la ronda 2, ¿por qué el tocón elegido deja mal a los puntos 4, 5 y 6? ¿Cuánto pesan en ese momento?
3. Predice antes de mover: en la ronda 4, ¿qué tocón se elegirá? Pista: compara los pesos de los grupos.

4. Cambia a *Círculo*. ¿Cuántas rondas hacen falta para que la región se parezca al círculo? ¿El error de prueba sigue bajando todo el tiempo?
5. Localiza un punto con etiqueta invertida (un punto azul lejos del círculo o naranja dentro) y observa su tamaño en rondas avanzadas. ¿Qué dice esto de AdaBoost con datos ruidosos?

Bagging frente a boosting

Tabla 57.6.: Comparación de las dos familias de ensambles.

	Bagging y bosques aleatorios	Boosting
Entrenamiento	En paralelo, independiente	En secuencia, dependiente
Modelo base	Fuerte e inestable (árbol profundo)	Débil (tocón, árbol poco profundo)
Qué reduce	Varianza	Sesgo (y algo de varianza)
Combinación	Promedio o voto simple	Suma ponderada
Riesgo de sobreajuste al agregar modelos	No	Sí: se controla el número de rondas
Ruido en las etiquetas	Robusto	Sensible (AdaBoost)
Ajuste de hiperparámetros	Poco necesario	Importante

Como regla práctica, un bosque aleatorio con valores por defecto es una referencia excelente y difícil de batir con poco esfuerzo; el *boosting* bien ajustado (capítulo siguiente) suele ganar por un margen pequeño cuando hay tiempo para validar sus hiperparámetros.

Implementación en Python

Árbol de regresión y bagging

Los datos del laboratorio y el efecto de promediar:

Figura 57.1.

AdaBoost: el ejemplo manual

`AdaBoostClassifier` con tocones reproduce las tres rondas. Sus `estimator_weights_` son el doble de los α_m clásicos (variante SAMME).

	corte		clásico (½ ln)	errores del ensamble
1	3.5	0.3000	0.4236	3
2	9.5	0.2143	0.6496	3
3	6.5	0.1818	0.7520	0

Comparación en un problema real

Comparamos un árbol, *bagging*, un bosque aleatorio, AdaBoost y *gradient boosting* en el conjunto *breast cancer* (569 tumores, 30 variables) con validación cruzada estratificada repetida.

modelo	exactitud	desv.	AUC
Árbol (sin poda)	0.927	0.019	0.923
Bagging (300 árboles)	0.954	0.017	0.990
Bosque aleatorio (300)	0.960	0.012	0.991
AdaBoost (300 tocones)	0.972	0.012	0.996
Gradient boosting	0.960	0.013	0.993

Error fuera de la bolsa e importancia de variables

Resumen

- Un **árbol de regresión** predice el promedio de cada hoja y elige cortes que reducen la suma de cuadrados.
- Combinar modelos funciona si cada uno es mejor que el azar y sus errores son **diversos**; la correlación entre modelos limita la reducción de varianza ($\rho\sigma^2$).
- **Bagging** promedia árboles entrenados con muestras bootstrap; cada árbol deja fuera $36.8\,\%$ de los datos, lo que da el **error fuera de la bolsa** sin costo adicional.
- Los **bosques aleatorios** eligen cada corte entre m variables al azar para decorrelacionar los árboles; más árboles no sobreajustan. La importancia por permutación es más fiable que la MDI.
- **AdaBoost** entrena clasificadores débiles en secuencia, repesando los errores y combinándolos con pesos $\alpha_m = \frac{1}{2}\ln\frac{1-\varepsilon_m}{\varepsilon_m}$; minimiza la pérdida exponencial y es sensible al ruido.
- *Bagging* reduce varianza; *boosting*, sesgo. El capítulo siguiente generaliza el *boosting* a cualquier pérdida con el **gradient boosting** y su implementación más popular, XGBoost.

58. Gradient Boosting y XGBoost

XGBoost (*eXtreme Gradient Boosting*) fue propuesto por Chen y Guestrin en 2016 (Chen y Guestrin 2016) y desde entonces ha dominado competencias de machine learning en Kaggle y otros benchmarks de referencia. Se trata de una implementación regularizada y optimizada a segundo orden del algoritmo de gradient boosting (Friedman 2001), con innovaciones clave en la búsqueda aproximada de particiones, el manejo eficiente de datos dispersos, y un diseño de sistema orientado a la escalabilidad. Este capitulo cubre los fundamentos matemáticos del algoritmo, sus hiperparámetros principales, la interpretación de importancia de variables mediante SHAP, y la implementación práctica en Python.

Intuición: cada árbol corrige al anterior

Imagina que estimas el precio de una casa. Un primer tasador da una cifra; un segundo no tasa la casa desde cero, sino que estudia **cuánto se equivocó el primero** y propone una corrección; un tercero corrige lo que aún falta, y así sucesivamente. Si cada corrección es pequeña y prudente, la suma converge a una buena estimación. Eso es el *gradient boosting*: una suma de árboles pequeños en la que cada uno aprende los **residuos** que dejaron los anteriores. El capítulo anterior presentó AdaBoost, que repesa ejemplos; aquí el mecanismo es más general: cada árbol nuevo se ajusta al **gradiente de la pérdida**, lo que permite usar cualquier pérdida diferenciable, desde el error cuadrático hasta la log-loss de la clasificación.

Construcción paso a paso

Gradient boosting a mano

Cinco casas con un tamaño x y un precio y (en millones):

Tabla 58.1.: Cinco observaciones para construir gradient boosting a mano.

x	1	2	3	4	5
y	2	4	9	10	15

Usaremos tocones de regresión (árboles de un solo corte) y una **tasa de aprendizaje** $\eta = 0.5$.

Paso 1. Predicción inicial. Con pérdida cuadrática, la mejor constante es el promedio: $F_0 = 40/5 = 8$. Su suma de cuadrados es $\text{SSE}_0 = 36 + 16 + 1 + 4 + 49 = 106$.

Paso 2. Residuos y primer árbol. Los residuos $r_i = y_i - F_0$ son $(-6, -4, 1, 2, 7)$. El mejor tocón para esos residuos corta entre $x = 2$ y $x = 3$: a la izquierda predice el promedio -5 y a la derecha $(1 + 2 + 7)/3 = 3.33$.

Paso 3. Encoger y sumar. No sumamos la corrección completa, sino la mitad:

$$F_1(x) = F_0 + \eta\, h_1(x) = \begin{cases} 8 + 0.5(-5) = 5.5 & x \le 2.5, \\ 8 + 0.5(3.33) = 9.67 & x > 2.5. \end{cases}$$

La suma de cuadrados baja a $\mathrm{SSE}_1 = 43.5$.

Paso 4. Repetir. Los nuevos residuos son $(-3.5, -1.5, -0.67, 0.33, 5.33)$: el gran error que queda es la última casa. El segundo tocón corta entre $x = 4$ y $x = 5$, con valores -1.33 y 5.33, y $F_2 = (4.83, 4.83, 9, 9, 12.33)$, con $\mathrm{SSE}_2 = 16.83$.

Tabla 58.2.: Tres rondas de gradient boosting con $= 0.5$.

Ronda	Predicciones F_m	Corte del tocón	SSE
0	8, 8, 8, 8, 8	—	106.00
1	5.50, 5.50, 9.67, 9.67, 9.67	$x \le 2.5$	43.50
2	4.83, 4.83, 9.00, 9.00, 12.33	$x \le 4.5$	16.83
3	3.92, 3.92, 9.61, 9.61, 12.94	$x \le 2.5$	8.43

Dos observaciones importantes. Primero, un mismo corte puede reaparecer (rondas 1 y 3): cada ronda solo ve los residuos actuales. Segundo, con $\eta = 1$ el primer tocón corregiría por completo su parte del error, pero también aprendería por completo su ruido; con η pequeño hacen falta más rondas, a cambio de un modelo más estable. Esa es la **contracción** (*shrinkage*).

Lo que agrega XGBoost: gradiente, hessiano y regularización

Con pérdida cuadrática $\frac{1}{2}(y - F)^2$, el **gradiente** respecto a la predicción es $g_i = F - y_i = -r_i$ y la **segunda derivada** (hessiano) es $h_i = 1$. En la primera ronda, $g = (6, 4, -1, -2, -7)$. XGBoost calcula el valor de cada hoja con la Ecuación 58.12, $w = -G/(H + \lambda)$, donde G y H suman g_i y h_i de los ejemplos de la hoja. Con $\lambda = 1$ y el corte $x \le 2.5$:

$$w_L = -\frac{6 + 4}{2 + 1} = -3.33, \qquad w_R = -\frac{-1 - 2 - 7}{3 + 1} = 2.5.$$

Comparados con los promedios del tocón clásico (-5 y 3.33), los valores se **encogen hacia cero**, más en la hoja pequeña: λ actúa como una muestra ficticia que tira cada hoja hacia la predicción actual. La calidad del corte se mide con la ganancia de la Ecuación 58.14:

$$\text{Ganancia} = \frac{1}{2}\left[\frac{10^2}{2 + 1} + \frac{(-10)^2}{3 + 1} - \frac{0^2}{5 + 1}\right] - \gamma = \frac{1}{2}(33.3 + 25) - \gamma = 29.2 - \gamma.$$

Tabla 58.3.: Ganancia de cada corte en la primera ronda de XGBoost.

Corte	G_L	G_R	Ganancia ($\lambda = 1$, $\gamma = 0$)
$x \leq 1.5$	6	-6	12.60
$x \leq 2.5$	10	-10	**29.17**
$x \leq 3.5$	9	-9	23.63
$x \leq 4.5$	7	-7	17.15

Si γ fuera mayor que 29.17, ningún corte compensaría y el árbol se quedaría en una sola hoja: así poda XGBoost.

Clasificación: el mismo mecanismo con la log-loss

En clasificación binaria, el modelo suma árboles en la escala de los **log-odds** y la probabilidad es $p = \sigma(F)$. Para la log-loss, $g_i = p_i - y_i$ y $h_i = p_i(1 - p_i)$. Si una hoja recibe tres ejemplos con $y = (1, 1, 0)$ y la predicción actual es $p = 0.5$ para todos, entonces $G = (-0.5) + (-0.5) + 0.5 = -0.5$, $H = 3 \times 0.25 = 0.75$ y, con $\lambda = 1$, $w = 0.5/1.75 = 0.286$: los log-odds de esa hoja suben de 0 a $0.286\,\eta$. El hessiano hace que las hojas donde el modelo ya está seguro (p cercano a 0 o 1, h pequeño) reciban correcciones más cautelosas en relación con su gradiente.

El siguiente código reproduce la Tabla 58.2 con una implementación desde cero y verifica con `xgboost` los pesos de hoja de la primera ronda:

Las hojas valen -3.33 y 2.5, como en el cálculo a mano. `xgboost` reporta la ganancia sin el factor ½: $58.33 = 2 \times 29.17$.

De los árboles de decisión al boosting

El problema del sesgo-varianza y los modelos individuales

Un árbol de decisión profundo tiene la capacidad de memorizar el conjunto de entrenamiento: al aumentar la profundidad, el modelo se ajusta cada vez más a los datos observados y su varianza aumenta. Por el contrario, un árbol poco profundo (o un "tronco" de un solo nivel) introduce un sesgo alto porque su capacidad expresiva es limitada. Este dilema entre sesgo y varianza es central en el aprendizaje automático y motiva el uso de métodos de ensamble.

Los métodos de ensamble combinan múltiples modelos débiles para obtener un modelo fuerte. En el caso del *bagging* (e.g., Random Forest (Breiman 2001b)), los árboles se entrenan de forma independiente sobre muestras con reemplazo, y su predicción se promedia para reducir la varianza. En el *boosting*, la estrategia es diferente: los modelos se construyen de manera secuencial, donde cada nuevo árbol aprende de los errores del ensamble anterior, reduciendo progresivamente el sesgo.

Figura 58.1.

La Figura 58.1 muestra claramente como la profundidad del árbol determina el equilibrio entre sesgo y varianza. El árbol de profundidad 3 logra un ajuste razonable sin sobreajustar el ruido.

Boosting: aprendizaje secuencial de residuos

El boosting construye un modelo aditivo de forma secuencial. Partiendo de una predicción inicial constante:

$$F_0(\mathbf{x}) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^{n} L(y_i, \gamma) \tag{58.1}$$

En cada iteración $m = 1, \dots, M$, se calcula el gradiente negativo (pseudoresiduos) de la función de pérdida evaluada en la predicción actual:

$$r_i^{(m)} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}} \tag{58.2}$$

Luego se ajusta un modelo débil h_m a estos pseudoresiduos, y se actualiza el ensamble:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x}) \tag{58.3}$$

donde $\eta \in (0, 1]$ es la tasa de aprendizaje o *shrinkage*. Para la pérdida cuadrática $L(y, F) = \frac{1}{2}(y - F)^2$, el gradiente negativo es simplemente el residuo ordinario $r_i = y_i - F_{m-1}(\mathbf{x}_i)$. Esto significa que el algoritmo ajusta iterativamente los errores del ensamble: cada nuevo árbol aprende lo que el ensamble anterior no pudo capturar.

Gradient Boosting (versión simplificada)

1. Inicializar $F_0(\mathbf{x}) = \bar{y}$ (media de la variable objetivo).
2. Para $m = 1, \dots, M$:
   - Calcular residuos $r_i^{(m)} = y_i - F_{m-1}(\mathbf{x}_i)$.
   - Ajustar un árbol débil h_m a $\{(\mathbf{x}_i, r_i^{(m)})\}_{i=1}^{n}$.
   - Actualizar $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x})$.
3. Devolver $F_M(\mathbf{x})$.

La siguiente figura ilustra la convergencia del algoritmo de gradient boosting implementado desde cero, sin usar XGBoost.

Observando la Figura **??** se aprecia como los residuos (puntos naranjas) se reducen progresivamente a medida que se agregan más árboles al ensamble, y la curva de predicción (azul) se acerca a la función verdadera (línea discontinua negra).

Un aspecto fundamental del boosting es que la tasa de aprendizaje η en la Ecuación 58.3 actua como un regulador de la velocidad de convergencia. Con $\eta = 1$ el algoritmo intenta corregir completamente los errores en cada paso, lo que puede llevar a sobreajuste rápido. Con η pequeño se necesitan más iteraciones pero el modelo final suele generalizar mejor. Este efecto se conoce como *shrinkage* (Friedman 2002) y es uno de los mecanismos de regularización implicitos del boosting.

El gradient boosting en su forma general (Ecuación 58.2) puede verse como un descenso de gradiente funcional: en lugar de actualizar los parámetros de un modelo parametrico en el espacio euclidiano,

se actualiza la función de predicción en el espacio funcional. Cada nuevo árbol débil h_m es el "paso de gradiente" que más reduce la pérdida esperada. Esta perspectiva, formalizada por Friedman (2001), unifica una amplia familia de algoritmos de aprendizaje bajo un único marco matemático.

El objetivo de XGBoost

Modelo aditivo y función objetivo regularizada

XGBoost construye el modelo de forma aditiva:

$$\hat{y}_i^{(t)} = \sum_{k=1}^{t} f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F} \tag{58.4}$$

donde $\mathcal{F}$ es el espacio de árboles de regresión. En cada ronda t, se busca el árbol f_t que minimiza el objetivo regularizado:

$$\text{Obj}^{(t)} = \sum_{i=1}^{n} l\Big(y_i,\, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\Big) + \Omega(f_t) + \text{constante} \tag{58.5}$$

La regularización Ω penaliza tanto el número de hojas como la magnitud de los pesos:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \|\mathbf{w}\|^2 \tag{58.6}$$

donde T es el número de hojas del árbol f_t y $\mathbf{w}$ son los pesos de las hojas. El parámetro γ controla la poda (un árbol solo se expande si la ganancia supera γ) y λ regula el tamaño de los pesos mediante regularización L2.

Innovación clave de XGBoost: A diferencia del gradient boosting clásico (Friedman 2001), XGBoost incluye explícitamente la regularización en el objetivo de optimización y utiliza información de segundo orden (el hessiano) para encontrar pesos óptimos de forma analítica, lo que conduce a una convergencia más rápida y modelos más robustos.

Aproximación de Taylor de segundo orden

Para poder optimizar el objetivo de la Ecuación 58.5 eficientemente, XGBoost aplica una expansión de Taylor de segundo orden alrededor de $\hat{y}_i^{(t-1)}$:

$$l\Big(y_i, \hat{y}_i^{(t-1)} + f_t\Big) \approx l\Big(y_i, \hat{y}_i^{(t-1)}\Big) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \tag{58.7}$$

donde se definen:

$$g_i = \frac{\partial\, l(y_i, \hat{y}^{(t-1)})}{\partial\, \hat{y}^{(t-1)}} \quad \text{(gradiente de primer orden)} \tag{58.8}$$

$$h_i = \frac{\partial^2 l(y_i, \hat{y}^{(t-1)})}{\partial \left(\hat{y}^{(t-1)}\right)^2} \quad \text{(hessiano — curvatura local)} \tag{58.9}$$

Descartando los términos constantes respecto a f_t, el objetivo simplificado es:

$$\tilde{\text{Obj}}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{58.10}$$

Esta aproximación cuadrática es exacta para la pérdida cuadrática y una buena aproximación local para cualquier pérdida dos veces diferenciable, lo que hace a XGBoost aplicable a una amplia variedad de problemas (regresión, clasificación, ranking, etc.).

La Figura **??** ilustra por que la aproximación cuadrática es útil: permite encontrar el cambio óptimo $\Delta^* = -g_i/(h_i + \lambda)$ de forma analítica, sin necesidad de búsqueda iterativa. XGBoost utiliza exactamente esta idea para calcular el peso óptimo de cada hoja.

Pesos óptimos de las hojas

Agrupando las muestras por hoja j (donde $I_j = \{i : q(\mathbf{x}_i) = j\}$ es el conjunto de índices de la hoja j), el objetivo simplificado se puede reescribir como:

$$\tilde{\text{Obj}}^{(t)} = \sum_{j=1}^{T} \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T \tag{58.11}$$

Definiendo $G_j = \sum_{i \in I_j} g_i$ y $H_j = \sum_{i \in I_j} h_i$, y minimizando respecto a w_j, se obtiene el peso óptimo:

$$w_j^* = -\frac{G_j}{H_j + \lambda} \tag{58.12}$$

Sustituyendo en el objetivo:

$$\text{Obj}^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T \tag{58.13}$$

Este valor sirve como puntuación de la estructura del árbol: cuanto más negativo sea, mejor es el árbol. Nótese que $\lambda > 0$ reduce la magnitud de los pesos y tiene un efecto regularizador directo, ya que $|w_j^*|$ decrece con λ.

Ganancia de una partición

Para decidir si realizar una división, XGBoost compara el objetivo antes y después de la partición. Si un nodo padre P se divide en hijos izquierdo L y derecho R, la ganancia de la división es:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G_P^2}{H_P+\lambda}\right] - \gamma \tag{58.14}$$

Si Gain ≤ 0, la división se rechaza (poda). El término $-\gamma$ penaliza la complejidad del árbol: un γ mayor hace que solo se acepten divisiones con mejoras sustanciales.

La Figura **??** muestra como la ganancia varia en función del umbral de corte. El algoritmo busca el umbral que maximiza esta función en cada nivel del árbol. Cuando $\gamma > 0$, cualquier ganancia por debajo de ese valor lleva a rechazar la partición.

Búsqueda aproximada de particiones (*Approximate Split Finding*)

Un aspecto crucial del diseño de XGBoost es que la búsqueda exacta del mejor umbral (recorriendo todos los valores únicos de cada variable) puede ser prohibitivamente costosa para conjuntos de datos grandes. Para abordar esto, Chen y Guestrin (2016) proponen un algoritmo de búsqueda aproximada basado en quantiles ponderados.

La idea es proponer un conjunto reducido de candidatos de partición $\mathcal{S}_k = \{s_{k1}, s_{k2}, \ldots, s_{kl}\}$ para cada variable k, utilizando los quantiles de la distribución de la variable ponderada por los hessianos h_i:

$$\text{rank}_k(z) = \frac{\sum_{(x,h):x_k<z} h}{\sum_{(x,h)} h} \tag{58.15}$$

Los candidatos se seleccionan tal que $|\text{rank}_k(s_{k,j}) - \text{rank}_k(s_{k,j+1})| \leq \varepsilon$, donde ε controla el número aproximado de candidatos: $\approx 1/\varepsilon$ por variable. Esto garantiza que el error de aproximación respecto a la solución exacta esta acotado.

El uso de los hessianos h_i como pesos en la distribución de quantiles no es arbitrario: las observaciones con mayor hessiano (mayor curvatura de la pérdida) tienen más influencia en la selección de los puntos de corte, lo que dirige la búsqueda hacia las regiones del espacio de características donde la ganancia potencial es mayor.

Sparsity-awareness (manejo de datos dispersos)
XGBoost aprende automáticamente la *dirección por defecto* para los valores faltantes o cero en cada nodo. Durante el entrenamiento, solo se evaluan los valores no faltantes para cada variable; los valores faltantes se asignan a la rama que minimiza el objetivo. Esto hace que XGBoost sea nativamente compatible con matrices dispersas sin necesidad de imputación previa, una ventaja práctica significativa en aplicaciones reales (Chen y Guestrin 2016).

Laboratorio: gradient boosting paso a paso

El laboratorio usa los mismos 40 puntos de entrenamiento y 300 de prueba del laboratorio de *bagging* ($y = 2\sin x + 0.3x$ más ruido de varianza 0.36). Cada ronda ajusta un árbol de regresión a los gradientes con la ganancia y los pesos de XGBoost ($\gamma = 0$); con $\lambda = 0$ es el *gradient boosting* clásico. El panel derecho muestra los residuos actuales y el árbol que se agregará en la ronda siguiente.

En la versión web, este laboratorio recorre el gradient boosting ronda por ronda sobre los datos del laboratorio de bagging. Con $\eta = 0.3$, profundidad 2 y $\lambda = 1$, el ECM de prueba baja de 2.94 (la constante inicial) a su mínimo de 0.483 en la ronda 23 y después sube lentamente hasta 0.62 en la ronda 300, mientras el de entrenamiento llega casi a 0: el mínimo es el punto donde conviene detenerse (*early stopping*).

Guía del laboratorio.

1. Con $M = 0$, el árbol rojo es la primera corrección. ¿Dónde está el mayor residuo y cómo lo atiende el árbol?
2. Avanza hasta $M = 10$ con $\eta = 0.3$. Predice antes de mover: ¿qué pasará con $\eta = 1$? Compara las curvas de error.
3. Lleva M a 300 con profundidad 4. ¿En qué ronda alcanza su mínimo el error de prueba? ¿Qué le pasa después?
4. Sube λ a 10. ¿Cómo cambia la altura del árbol rojo y la velocidad de convergencia?
5. Explica por qué en boosting más árboles sí pueden sobreajustar, a diferencia de *bagging*.

Hiperparámetros principales

XGBoost expone numerosos hiperparámetros que permiten controlar la capacidad del modelo, la regularización, el subsampling y la velocidad de convergencia. La siguiente tabla resume los más importantes.

Desde la perspectiva del aprendizaje estadístico, los hiperparámetros pueden agruparse en tres categorías conceptuales:

1. **Capacidad del modelo:** `max_depth`, `n_estimators`, `min_child_weight`. Determinan cuanta información puede memorizar el modelo. Valores altos de `max_depth` y `n_estimators` pueden llevar a sobreajuste.
2. **Regularización:** `gamma`, `reg_alpha`, `reg_lambda`. Penalizan la complejidad del modelo y reducen la varianza de los pesos de las hojas. Corresponden directamente a los términos de la Ecuación 58.6.
3. **Aleatoriedad y robustez:** `subsample`, `colsample_bytree`, `learning_rate`. Introducen estocasticidad en el proceso de construcción, similar al gradient boosting estocástico (Friedman 2002), y controlan la velocidad de convergencia mediante el *shrinkage* η.

Tabla 58.4.: Hiperparámetros principales de XGBoost y sus rangos típicos.

Parámetro	Descripción	Valor típico
`n_estimators`	Número de árboles en el ensamble	100–1000
`max_depth`	Profundidad máxima por árbol	3–8
`learning_rate` (η)	Tasa de aprendizaje (shrinkage)	0.01–0.3
`subsample`	Fracción de filas muestreadas por árbol	0.6–1.0
`colsample_bytree`	Fracción de columnas por árbol	0.6–1.0
`min_child_weight`	Mínimo de $\sum h_i$ en un nodo hoja	1–10
`gamma` (γ)	Ganancia mínima para realizar una partición	0–5
`reg_alpha`	Regularización L1 sobre pesos	0–1
`reg_lambda`	Regularización L2 sobre pesos	1 (default)

El número de árboles (`n_estimators`) y la tasa de aprendizaje (`learning_rate`) interactuan directamente: tasas de aprendizaje menores requieren más árboles pero generalmente producen mejores modelos. El `subsample` y el `colsample_bytree` introducen aleatoriedad similar al Random Forest, lo que reduce la varianza y el sobreajuste (Friedman 2002). El `min_child_weight` controla la complejidad de cada árbol al exigir que cada hoja tenga una suma de hessianos mínima.

Convergencia y early stopping

La Figura **??** muestra el comportamiento típico del gradient boosting: el error de entrenamiento decrece monotonamente, mientras el error de validación se estabiliza y puede comenzar a subir si se excede la cantidad óptima de árboles. La técnica de *early stopping* detiene el entrenamiento cuando el error de validación no mejora durante un número definido de rondas consecutivas.

Búsqueda de hiperparámetros: profundidad y tasa de aprendizaje

La Figura **??** revela la interacción entre la profundidad del árbol y la tasa de aprendizaje. En general, tasas de aprendizaje muy bajas (0.01) requieren más árboles para converger y con solo 100 estimadores pueden tener alto sesgo. Profundidades intermedias (3–5) suelen dar el mejor equilibrio entre capacidad y generalización.

Importancia de variables

XGBoost ofrece tres métricas para cuantificar la importancia de cada variable en el modelo entrenado:

- **Frecuencia (weight):** número de veces que la variable se usa en una división a lo largo de todos los árboles. Es sensible al número de categorías distintas de cada variable.

- **Ganancia (gain):** reducción promedio de la pérdida aportada por cada división que usa esa variable. Es la métrica más informativa porque captura la contribución real al rendimiento del modelo.
- **Cobertura (cover):** número promedio de observaciones que pasan por cada división de la variable. Refleja cuantas muestras son "afectadas" por esa variable.

La métrica de ganancia suele ser la más recomendable para interpretar cuales variables son verdaderamente relevantes, ya que la frecuencia puede sobreestimar la importancia de variables con muchos valores posibles.

Es importante destacar que todas estas métricas miden la importancia *global* del modelo, es decir, el aporte promedio de cada variable a través de todas las observaciones. Para obtener importancias *locales* (especificas a una predicción individual), se deben usar los valores SHAP, descritos en la Sección 58.

Matemáticamente, la importancia de ganancia para la variable k se calcula como:

$$\text{Gain}_k = \frac{1}{|\mathcal{T}_k|} \sum_{t \in \mathcal{T}_k} \text{Gain}_t \tag{58.16}$$

donde $\mathcal{T}_k$ es el conjunto de todas las divisiones que usan la variable k en todos los árboles del ensamble. La importancia de frecuencia es simplemente $|\mathcal{T}_k|$, y la de cobertura es el promedio de $|I_L| + |I_R|$ sobre todas esas divisiones.

Según la Figura **??**, la variable `MedInc` (ingreso mediano del bloque censal) es consistentemente la más importante en los tres criterios, lo cual es coherente con el conocimiento del dominio: el ingreso es el predictor más fuerte del precio de la vivienda en California. Las variables `Latitude` y `Longitude` también aparecen como relevantes, capturando efectos geograficos.

Regularización y poda

Efecto de λ (L2) y γ (ganancia mínima)

La regularización en XGBoost actua por dos mecanismos complementarios, derivados directamente de la Ecuación 58.6:

Regularización L2 (λ): El peso óptimo de cada hoja es $w_j^* = -G_j/(H_j + \lambda)$. Un λ mayor reduce la magnitud de los pesos, produciendo predicciones más conservadoras y previniendo el sobreajuste. Es equivalente a la regularización Ridge en modelos lineales pero aplicada a los pesos de cada hoja del árbol.

Poda por ganancia mínima (γ): Una división solo se realiza si Gain $> \gamma$ (ver Ecuación 58.14). Un γ mayor lleva a árboles más pequeños y con menos divisiones. A diferencia de la poda post-entrenamiento, esta poda ocurre durante la construcción del árbol, lo que es más eficiente computacionalmente.

La combinación de ambos parámetros es complementaria: λ controla la magnitud de cada predicción de hoja, mientras que γ controla la estructura del árbol. Se recomienda priorizar λ alto sobre γ alto, ya que λ tiene un efecto más suave y continuo.

La Figura **??** confirma que valores muy altos de γ (mayor a 2) pueden degradar el rendimiento al podar demasiado agresivamente, impidiendo que el modelo capture patrones relevantes. Un λ moderado (entre 1 y 5) generalmente ofrece el mejor equilibrio.

Adicionalmente, XGBoost soporta regularización L1 sobre los pesos de las hojas mediante el parámetro `reg_alpha`. El objetivo con regularización elastica (L1 + L2) toma la forma:

$$\Omega(f) = \gamma T + \alpha \sum_{j=1}^{T} |w_j| + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{58.17}$$

La regularización L1 ($\alpha > 0$) induce esparsidad en los pesos de las hojas: muchos pesos se vuelven exactamente cero, lo que produce árboles donde algunas hojas no contribuyen a la predicción. Esto puede ser útil cuando se sospecha que muchas divisiones son redundantes o ruidosas. En la práctica, la regularización L2 (λ) es suficiente en la mayoría de los casos y se recomienda comenzar con $\alpha = 0$.

Efecto del subsampling sobre la regularización

El parámetro `subsample` (fracción de filas) y `colsample_bytree` (fracción de columnas) introducen aleatoriedad en la construcción de cada árbol. Desde una perspectiva estadística, esto puede verse como una forma de regularización implícita: al entrenar cada árbol con un subconjunto distinto de datos, se reduce la correlación entre los árboles del ensamble, lo que disminuye la varianza de las predicciones. Friedman (2002) demostro empíricamente que el subsampling de filas mejora consistentemente la generalización del gradient boosting, especialmente en conjuntos de datos con ruido.

Contribuciones individuales (SHAP simplificado)

Los valores SHAP (*SHapley Additive exPlanations*), propuestos por Lundberg y Lee (2017), permiten descomponer la predicción de cualquier modelo en contribuciones aditivas de cada variable para una observación específica. Para XGBoost, Lundberg et al. (2020) desarrollaron el algoritmo TreeSHAP, que calcula estas contribuciones de forma exacta y eficiente en tiempo polinomial.

Formalmente, para una predicción $\hat{y}_i$, los valores SHAP ϕ_j satisfacen:

$$\hat{y}_i = \phi_0 + \sum_{j=1}^{p} \phi_j \tag{58.18}$$

donde $\phi_0 = \mathbb{E}[\hat{y}]$ es la predicción base (valor esperado del modelo) y ϕ_j es la contribución de la variable j para esa observación en particular. Las contribuciones pueden ser positivas (la variable empuja la predicción hacia arriba) o negativas (la variable la baja).

La Figura **??** muestra como la predicción final se construye sumando la contribución de cada variable al valor base. Para esta observación particular, `MedInc` tiene la mayor influencia positiva, mientras que algunas variables geograficas pueden reducir la predicción estimada. Esta visualización es especialmente útil para comunicar predicciones individuales a usuarios no técnicos o para auditar el comportamiento del modelo.

XGBoost para clasificación

XGBoost es igualmente aplicable a problemas de clasificación, con soporte nativo para:

- **Clasificación binaria:** `objective="binary:logistic"` — el modelo genera probabilidades mediante la función sigmoide $p_i = \sigma(\hat{y}_i)$, y la función de pérdida es la log-verosimilitud negativa (log-loss). Los gradientes toman la forma $g_i = p_i - y_i$ y el hessiano $h_i = p_i(1 - p_i)$.
- **Clasificación multiclase:** `objective="multi:softprob"` con `num_class=K` — se entrena un conjunto de árboles por clase y se aplica softmax para normalizar las probabilidades.

La ventaja de usar el hessiano $h_i = p_i(1 - p_i)$ es que pesa menos las observaciones donde el modelo ya esta seguro (probabilidad cercana a 0 o 1), concentrando el aprendizaje en los casos difíciles (probabilidad cercana a 0.5).

El resultado muestra el excelente rendimiento de XGBoost incluso con hiperparámetros predeterminados. En el conjunto de cancer de mama (569 observaciones, 30 variables), se obtiene un AUC-ROC superior a 0.99, lo cual refleja tanto la calidad del clasificador como la separabilidad del conjunto de datos.

Curva ROC y análisis de umbrales

La curva ROC (*Receiver Operating Characteristic*) es una herramienta estándar para evaluar clasificadores binarios. Traza la tasa de verdaderos positivos (sensibilidad) frente a la tasa de falsos positivos (1 - especificidad) para todos los umbrales de decisión posibles. El área bajo la curva (AUC) cuantifica la capacidad discriminatoria del modelo en un único número: AUC = 1 indica clasificación perfecta, AUC = 0.5 indica predicción aleatoria.

La Figura **??** confirma visualmente el alto rendimiento del clasificador: la curva ROC se acerca al punto ideal (0, 1), y el área bajo la curva es prácticamente perfecta para este conjunto de datos. En aplicaciones reales, donde la separabilidad es menor, la curva ROC permite elegir el umbral de decisión adecuado según el costo relativo de falsos positivos y falsos negativos.

Buenas prácticas y comparación con alternativas

Comparación de métodos de ensamble

La siguiente tabla resume el posicionamiento de XGBoost en el contexto de otros métodos de aprendizaje supervisado.

Tabla 58.5.: Comparación de métodos de ensamble según complejidad, interpretabilidad y rendimiento típico.

Método	Complejidad	Interpretabilidad	Rendimiento típico
Árbol único	Baja	Alta	Bajo
Random Forest	Media	Media	Bueno
Gradient Boosting (sklearn)	Media	Media	Muy bueno
XGBoost	Media-alta	Media (+ SHAP)	Excelente

Método	Complejidad	Interpretabilidad	Rendimiento típico
LightGBM / CatBoost	Media-alta	Media (+ SHAP)	Excelente

Recomendaciones prácticas

Las siguientes guias resumen las mejores prácticas para usar XGBoost de forma efectiva:

1. **Estrategia de ajuste de hiperparámetros:** Comenzar con `learning_rate=0.1` y `n_estimators=200-500`. Una vez identificado un buen número de árboles mediante *early stopping*, reducir la tasa de aprendizaje a 0.05 o 0.01 y aumentar proporcionalmente los estimadores para refinar el modelo (Hastie et al. 2009).
2. **Early stopping obligatorio:** Siempre usar `early_stopping_rounds` con un conjunto de validación separado. Esto evita el sobreajuste sin necesidad de una búsqueda exhaustiva del número óptimo de árboles.
3. **Subsampling para reducir varianza:** Activar `subsample=0.8` y `colsample_bytree=0.8` por defecto. La aleatoriedad adicional que introducen, similar al Random Forest, generalmente mejora la generalización (Friedman 2002).
4. **Métrica de importancia:** Preferir `importance_type="gain"` sobre `"weight"`, ya que la frecuencia puede ser enganiosa cuando las variables tienen cardinalidades muy distintas.
5. **Regularización:** Mantener `reg_lambda=1` (valor por defecto) como punto de partida. Aumentar si hay seniales de sobreajuste. Usar `gamma` con moderación (valores entre 0 y 1) para podar divisiones poco informativas.
6. **Validación cruzada:** Para conjuntos de datos pequeños, usar `xgb.cv()` con k-fold para estimar el número óptimo de árboles de forma más robusta que con un único conjunto de validación.
7. **Manejo de valores faltantes:** XGBoost aprende automáticamente la dirección óptima para los valores faltantes durante el entrenamiento (sparsity-awareness (Chen y Guestrin 2016)). No es necesario imputar previamente.
8. **Comparar con LightGBM:** Para conjuntos de datos muy grandes (millones de filas o cientos de variables), LightGBM suele ser más rápido gracias a su estrategia de construcción de árboles basada en histogramas (*leaf-wise growth*). La API es muy similar a XGBoost.

Parte XXX.

Modelos Lineales

59. Regresión lineal y clasificadores lineales: Fisher y el perceptrón

Los árboles y los vecinos cercanos construyen fronteras flexibles a partir de reglas locales. Los **modelos lineales** hacen la apuesta contraria: resumen toda la información en un **puntaje** que es una suma ponderada de las variables,

$$g(\mathbf{x}) = w_0 + w_1 x_1 + \cdots + w_p x_p = \mathbf{w}^\top \mathbf{x} + w_0,$$

y deciden con ese puntaje. Para predecir un número, el puntaje es la predicción; para clasificar, basta mirar su **signo**. Su frontera de decisión es una recta en el plano y un hiperplano en general. Son menos flexibles que un árbol profundo, pero tienen poca varianza, se entrenan rápido, sus coeficientes se interpretan y son la base de la regresión logística, las máquinas de soporte vectorial y las redes neuronales.

Este capítulo recorre tres maneras de elegir $\mathbf{w}$: **mínimos cuadrados**, que ajusta números; el **discriminante de Fisher**, que busca la dirección que mejor separa dos clases; y el **perceptrón**, que corrige sus errores uno a uno. El capítulo siguiente presenta la cuarta y más usada, la regresión logística.

Regresión lineal

Intuición: la recta que menos se equivoca

Cinco estudiantes registraron sus horas de estudio x y su calificación y (sobre 10):

Tabla 59.1.: Horas de estudio y calificación de cinco estudiantes.

Estudiante	1	2	3	4	5
Horas x	1	2	3	4	5
Calificación y	2	3	5	4	6

Ninguna recta pasa por los cinco puntos. Para cada recta candidata $\widehat{y} = \beta_0 + \beta_1 x$, cada estudiante tiene un **residuo** $e_i = y_i - \widehat{y}_i$. El criterio de **mínimos cuadrados** (Legendre 1805; Gauss 1809) elige la recta que minimiza la suma de los residuos al cuadrado, $\mathrm{SSE} = \sum_i e_i^2$. Elevar al cuadrado impide que errores positivos y negativos se cancelen y castiga más los errores grandes.

Construcción paso a paso

Paso 1. Centrar. Las medias son $\bar{x} = 3$ y $\bar{y} = 20/5 = 4$.

Paso 2. Pendiente. Para una variable, la solución de mínimos cuadrados es el cociente entre la covariación de x con y y la variación de x:

Tabla 59.2.: Cálculo de la pendiente.

$x_i - \bar{x}$	-2	-1	0	1	2	Suma
$y_i - \bar{y}$	-2	-1	1	0	2	
$(x_i - \bar{x})(y_i - \bar{y})$	4	1	0	0	4	$S_{xy} = 9$
$(x_i - \bar{x})^2$	4	1	0	1	4	$S_{xx} = 10$

$$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}} = \frac{9}{10} = 0.9, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = 4 - 0.9 \cdot 3 = 1.3.$$

Cada hora adicional de estudio se asocia con 0.9 puntos más de calificación.

Paso 3. Predicciones y residuos. Con $\hat{y} = 1.3 + 0.9x$:

Tabla 59.3.: Predicciones y residuos de la recta de mínimos cuadrados.

x	1	2	3	4	5
$\hat{y}$	2.2	3.1	4.0	4.9	5.8
$e = y - \hat{y}$	-0.2	-0.1	1.0	-0.9	0.2
e^2	0.04	0.01	1.00	0.81	0.04

Paso 4. Métricas. SSE $= 1.90$, de modo que el **error cuadrático medio** es MSE $= 1.90/5 = 0.38$ y su raíz RMSE $= 0.616$ puntos. El **error absoluto medio** es MAE $= (0.2+0.1+1.0+0.9+0.2)/5 = 0.48$. La variación total respecto a la media es SST $= \sum_i (y_i - \bar{y})^2 = 4+1+1+0+4 = 10$, así que

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{1.9}{10} = 0.81 :$$

la recta explica el 81 % de la variación de las calificaciones respecto a predecir siempre el promedio.

Formalización

Con p variables, el modelo es $y_i = \beta_0 + \sum_j \beta_j x_{ij} + \varepsilon_i$. Apilando las observaciones en la **matriz de diseño** $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$, cuya primera columna es de unos, y en el vector $\mathbf{y}$, el problema es

$$\hat{\beta} = \arg\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2. \tag{59.1}$$

Derivando e igualando a cero se obtienen las **ecuaciones normales** $\mathbf{X}^\top\mathbf{X}\hat{\beta} = \mathbf{X}^\top\mathbf{y}$ y, si $\mathbf{X}^\top\mathbf{X}$ es invertible,

$$\hat{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}. \tag{59.2}$$

Geométricamente, $\hat{\mathbf{y}} = \mathbf{X}\hat{\beta}$ es la **proyección ortogonal** de $\mathbf{y}$ sobre el espacio generado por las columnas de $\mathbf{X}$, y por eso los residuos son perpendiculares a cada columna: suman cero y no están correlacionados con ninguna variable. Si dos variables son (casi) linealmente dependientes, $\mathbf{X}^\top\mathbf{X}$ es (casi) singular y los coeficientes se vuelven inestables (**multicolinealidad**); la solución práctica es eliminar variables redundantes o **regularizar** sumando $\lambda\|\beta\|^2$ a la pérdida (*ridge*) (Hoerl y Kennard 1970), tema que se retoma con la regresión logística.

Tabla 59.4.: Métricas de regresión.

Métrica	Fórmula	Unidades	Sensible a atípicos
MSE	$\frac{1}{n}\sum e_i^2$	cuadrado de y	mucho
RMSE	$\sqrt{\text{MSE}}$	las de y	mucho
MAE	$\frac{1}{n}\sum\lvert e_i\rvert$	las de y	poco
R^2	$1 - \text{SSE/SST}$	sin unidades	mucho

Laboratorio: ajusta la recta

Mueve la pendiente y el intercepto para minimizar la suma de cuadrados; cada residuo se dibuja como un cuadrado cuya área es su contribución a la SSE. El panel derecho muestra la SSE de todas las rectas posibles: el mínimo es la solución de mínimos cuadrados. Arrastra un punto para ver cuánto puede mover la recta un solo dato atípico.

En la versión web, este laboratorio permite mover la recta y los puntos. Valores de referencia con los cinco estudiantes: la recta de mínimos cuadrados es $\hat{y} = 1.3 + 0.9x$, con SSE $= 1.90$ y $R^2 = 0.81$. Si el quinto estudiante tuviera calificación 9 en lugar de 6, la recta pasaría a $\hat{y} = 0.1 + 1.5x$: un solo punto extremo cambia la pendiente en 67 %.

Guía del laboratorio.

1. Sin pulsar el botón, intenta llegar a una SSE menor que 2. Después compara con la solución y con la Tabla 59.3.
2. Observa el mapa: ¿por qué el valle es una franja diagonal y no un círculo?
3. Predice antes de mover: si arrastras al quinto estudiante hasta una calificación de 9, ¿la pendiente subirá mucho o poco? Pulsa *Mínimos cuadrados* y compruébalo.
4. ¿Qué residuo aporta más a la SSE en la solución original? ¿Por qué un error de 1 punto pesa 25 veces más que uno de 0.2?

De la regresión a la clasificación lineal

La geometría de un clasificador lineal

Con dos clases codificadas como $y \in \{-1, +1\}$, un clasificador lineal decide

$$\hat{y} = \operatorname{signo}\big(g(\mathbf{x})\big), \qquad g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0. \tag{59.3}$$

La frontera $g(\mathbf{x}) = 0$ es un hiperplano. Tres hechos geométricos se usarán en este capítulo y en el de máquinas de soporte vectorial:

- $\mathbf{w}$ es **perpendicular** a la frontera: apunta hacia la región positiva.
- w_0 **desplaza** la frontera sin girarla.
- La **distancia con signo** de un punto a la frontera es $g(\mathbf{x})/\|\mathbf{w}\|$. El puntaje no solo dice de qué lado está el punto, sino qué tan lejos.

Una primera idea para elegir $\mathbf{w}$ es usar mínimos cuadrados con $y \in \{-1, +1\}$ como respuesta. Funciona sorprendentemente bien cuando las clases son nubes compactas, pero tiene un defecto: castiga también a los puntos **bien clasificados y lejos** de la frontera, porque su puntaje se aleja de ± 1. Un grupo de puntos correctos pero extremos puede arrastrar la frontera y provocar errores. Las dos secciones siguientes, y la regresión logística, son maneras de evitar ese problema.

El discriminante lineal de Fisher

Intuición: la mejor sombra

Proyectemos todos los puntos sobre una recta que pasa por el origen con dirección $\mathbf{w}$: cada punto se convierte en un número, su «sombra» $z = \mathbf{w}^\top \mathbf{x}$. Una buena dirección es aquella en que las sombras de las dos clases quedan **lejos entre sí** y **apretadas dentro de cada clase**. Fisher (1936) propuso medirlo con el cociente

$$J(\mathbf{w}) = \frac{(\bar{z}_A - \bar{z}_B)^2}{s_A^2 + s_B^2}, \tag{59.4}$$

donde $\bar{z}$ y $s^2 = \sum (z - \bar{z})^2$ son la media y la dispersión de las sombras de cada clase. La dirección que une las medias parece la elección obvia, pero no considera la forma de las nubes: si una clase está estirada en diagonal, sus sombras sobre esa dirección se dispersan.

Construcción paso a paso

Dos clases de tres puntos: $A = \{(1,2),(2,3),(3,3)\}$ y $B = \{(4,5),(5,6),(6,5)\}$.

Paso 1. Medias. $\mathbf{m}_A = (2, 2.67)$ y $\mathbf{m}_B = (5, 5.33)$.

Paso 2. Dispersión dentro de cada clase. Con las desviaciones respecto a la media de cada clase, $\mathbf{S}_A = \sum(\mathbf{x} - \mathbf{m}_A)(\mathbf{x} - \mathbf{m}_A)^\top$:

$$\mathbf{S}_A = \begin{pmatrix} 2 & 1 \\ 1 & 0.67 \end{pmatrix}, \qquad \mathbf{S}_B = \begin{pmatrix} 2 & 0 \\ 0 & 0.67 \end{pmatrix}, \qquad \mathbf{S}_W = \mathbf{S}_A + \mathbf{S}_B = \begin{pmatrix} 4 & 1 \\ 1 & 1.33 \end{pmatrix}.$$

La clase A tiene covarianza positiva (sube en diagonal); la B, no.

Paso 3. Dirección de Fisher. La dirección que maximiza J es

$$\mathbf{w}^* \propto \mathbf{S}_W^{-1}(\mathbf{m}_A - \mathbf{m}_B) = \frac{1}{4.33}\begin{pmatrix} 1.33 & -1 \\ -1 & 4 \end{pmatrix}\begin{pmatrix} -3 \\ -2.67 \end{pmatrix} = \begin{pmatrix} -0.31 \\ -1.77 \end{pmatrix}.$$

Normalizada, $\mathbf{w}^* \approx (-0.17, -0.99)$: casi vertical, muy distinta de la diferencia de medias $(-3, -2.67)$, que apunta en diagonal.

Paso 4. Comparar. Proyectando sobre cada dirección (normalizada):

Tabla 59.5.: Proyecciones sobre la diferencia de medias y sobre la dirección de Fisher.

Dirección	Sombras de A	Sombras de B	J
Diferencia de medias	-2.08, -3.49, -4.24	-6.31, -7.72, -7.81	4.22
Fisher	-2.14, -3.30, -3.47	-5.61, -6.77, -5.95	**5.64**

Con la dirección de Fisher, las sombras de A quedan más juntas (la diagonal de A casi desaparece al proyectar) y el cociente sube un 34 %. La clasificación se completa con un **umbral** sobre z; el más simple es el punto medio de las medias proyectadas, $(-2.97 - 6.11)/2 = -4.54$.

Formalización

Con las matrices de dispersión **entre clases** $\mathbf{S}_B = (\mathbf{m}_A - \mathbf{m}_B)(\mathbf{m}_A - \mathbf{m}_B)^\top$ y **dentro de las clases** $\mathbf{S}_W$, el criterio es un cociente de Rayleigh generalizado:

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}, \qquad \mathbf{w}^* = \arg\max_{\mathbf{w}} J(\mathbf{w}) \propto \mathbf{S}_W^{-1}(\mathbf{m}_A - \mathbf{m}_B). \tag{59.5}$$

Es la misma estructura que el PCA del capítulo de preprocesamiento, con una diferencia crucial: PCA busca la dirección de **máxima varianza** sin mirar las clases; Fisher, la de **máxima separación** entre clases. Con K clases, Fisher encuentra hasta $K - 1$ direcciones, lo que lo convierte también en una técnica de reducción de dimensión supervisada. Cuando las clases son normales con la misma covarianza, la dirección de Fisher coincide con la del clasificador bayesiano óptimo, el **análisis discriminante lineal** (LDA) que se estudia en el capítulo de clasificadores bayesianos.

Laboratorio: la dirección de Fisher

En la versión web, este laboratorio gira la dirección de proyección. Con los seis puntos, la diferencia de medias (42°) da $J = 4.22$ y la dirección de Fisher (80°) da el máximo $J = 5.64$, como en la Tabla 59.5. Con dos nubes alargadas en diagonal la ventaja de Fisher es mucho mayor: la diferencia de medias deja ambas clases traslapadas.

Guía del laboratorio.

1. Con *Seis puntos*, pulsa *Diferencia de medias* y luego *Dirección de Fisher*. Comprueba los valores de J de la Tabla 59.5.
2. ¿Por qué con la dirección de Fisher las sombras de la clase A quedan más juntas?
3. Cambia a *Dos nubes alargadas*. Predice antes de pulsar: ¿la diferencia de medias separará bien las clases? Compara los errores con el umbral en el punto medio.
4. Busca un ángulo con J casi cero. ¿Qué tienen de especial las sombras en esa dirección?

El perceptrón

Intuición: aprender de los errores, uno a la vez

El perceptrón de Rosenblatt (1958), inspirado en una neurona, no resuelve ninguna ecuación: recorre los ejemplos y, cada vez que clasifica uno mal, **gira la frontera hacia él**. Si un punto positivo cae del lado negativo, se suma su vector a $\mathbf{w}$; si un negativo cae del lado positivo, se resta. Es el antecedente directo de las redes neuronales y del descenso de gradiente estocástico.

Construcción paso a paso

Usamos el **vector aumentado** $\tilde{\mathbf{x}} = (1, x_1, x_2)$ y $\tilde{\mathbf{w}} = (w_0, w_1, w_2)$, de modo que $g = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$. Cuatro puntos, recorridos en orden:

Tabla 59.6.: Cuatro puntos linealmente separables.

Punto	$\mathbf{x}$	y
P1	(2, 1)	$+1$
P2	(3, 4)	-1
P3	(1, 3)	-1
P4	(4, 2)	$+1$

Empezamos con $\tilde{\mathbf{w}} = (0, 0, 0)$. Un punto está **mal clasificado** si $y\, g \le 0$ (con $\mathbf{w} = 0$ todos lo están). La regla, con tasa de aprendizaje 1, es $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + y\, \tilde{\mathbf{x}}$:

Tabla 59.7.: Traza del perceptrón (en las épocas 2 y 3 se omiten los puntos sin error).

Época	Punto	g antes	¿Error?	$\tilde{\mathbf{w}}$ después
1	P1	0	sí	(0,0,0) + (1,2,1) = **(1, 2, 1)**
1	P2	$1+6+4=11$	sí ($y=-1$)	(1,2,1) − (1,3,4) = **(0, −1, −3)**
1	P3	$0-1-9=-10$	no	(0, −1, −3)
1	P4	$0-4-6=-10$	sí	(0,−1,−3) + (1,4,2) = **(1, 3, −1)**
2	P1	$1+6-1=6$	no	(1, 3, −1)
2	P2	$1+9-4=6$	sí	**(0, 0, −5)**
2	P4	$0+0-10=-10$	sí	**(1, 4, −3)**
3	P2	$1+12-12=1$	sí	**(0, 1, −7)**
3	P4	$0+4-14=-10$	sí	**(1, 5, −5)**
4	—		ninguno	(1, 5, −5)

Tras siete actualizaciones, la época 4 no comete errores: la frontera es $1+5x_1-5x_2=0$, es decir, $x_2=x_1+0.2$, y los positivos quedan debajo. Observa que el algoritmo oscila: P2 y P4 se «pelean» la frontera durante tres épocas porque cada corrección para uno estropea al otro.

Formalización

La regla de actualización ante un error es

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta\, y_i\, \tilde{\mathbf{x}}_i \quad \text{si } y_i\, \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i \le 0. \tag{59.6}$$

Tras la actualización, $y_i\, \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i$ aumenta en $\eta\|\tilde{\mathbf{x}}_i\|^2 > 0$: el punto queda «menos mal». Es un descenso de gradiente estocástico sobre la **pérdida del perceptrón** $\max(0, -y\, g(\mathbf{x}))$, que vale cero para los puntos bien clasificados y crece linealmente con lo mal clasificados que están.

Teorema de convergencia (Novikoff 1962). Si los datos son **linealmente separables** con margen γ (existe un $\mathbf{w}^*$ unitario con $y_i\, \mathbf{w}^{*\top} \tilde{\mathbf{x}}_i \ge \gamma$ para todo i) y $\|\tilde{\mathbf{x}}_i\| \le R$, el perceptrón comete a lo más $(R/\gamma)^2$ errores antes de encontrar un separador. Cuanto más «holgada» es la separación, más rápido converge.

Tres limitaciones motivan los modelos de los capítulos siguientes:

- Si los datos **no son separables**, el perceptrón nunca se detiene: la frontera cicla indefinidamente. El caso clásico es el **XOR** (Minsky y Papert 1969), que ninguna recta separa y que llevó a las redes multicapa.
- Si son separables, encuentra **una** frontera cualquiera, dependiente del orden de los datos; las máquinas de soporte vectorial buscan la de **margen máximo**.
- No produce probabilidades; la regresión logística sí.

Laboratorio: el perceptrón paso a paso

En la versión web, este laboratorio ejecuta el perceptrón paso a paso. Con los cuatro puntos reproduce la Tabla 59.7: siete actualizaciones, convergencia en la época 4 con $\tilde{\mathbf{w}} = (1, 5, -5)$. Con datos XOR nunca converge.

Guía del laboratorio.

1. Con *Cuatro puntos*, avanza punto por punto y comprueba cada fila de la Tabla 59.7.
2. ¿Por qué después de corregir P2 en la época 1, el punto P4 queda mal clasificado?
3. Predice antes de pulsar: con *Separable (20)*, ¿cuántas épocas crees que necesitará? Usa *Hasta converger*.
4. Con *XOR*, pulsa *Hasta converger*. Observa la gráfica de errores por época: ¿se estabiliza en cero?

Comparación

Tabla 59.8.: Tres maneras de elegir un clasificador lineal.

	Mínimos cuadrados	Fisher	Perceptrón
Qué optimiza	Suma de cuadrados del puntaje	Separación de las proyecciones	Pérdida del perceptrón
Solución	Cerrada (ecuaciones normales)	Cerrada ($\mathbf{S}_W^{-1}\Delta\mathbf{m}$)	Iterativa
Datos no separables	Sí	Sí	No converge
Sensible a puntos lejanos bien clasificados	Sí	Poco	No
Probabilidades	No	Con supuestos gaussianos (LDA)	No

Implementación en Python

Regresión lineal

Con varias variables, la interpretación de cada coeficiente es «manteniendo las demás constantes». Con el conjunto *diabetes* de `scikit-learn` (10 variables clínicas estandarizadas):

Figura 59.1.

Discriminante de Fisher

La dirección de `LinearDiscriminantAnalysis` es la misma (con el signo opuesto, porque apunta hacia la clase 1).

El perceptrón

	época	punto	w después
0	1	P1	(1.0, 2.0, 1.0)
1	1	P2	(0.0, -1.0, -3.0)
2	1	P4	(1.0, 3.0, -1.0)
3	2	P2	(0.0, 0.0, -5.0)
4	2	P4	(1.0, 4.0, -3.0)
5	3	P2	(0.0, 1.0, -7.0)
6	3	P4	(1.0, 5.0, -5.0)

`scikit-learn` incluye `Perceptron`. Con datos XOR no converge, como ningún clasificador lineal:

Resumen

- Un modelo lineal resume cada caso en un puntaje $\mathbf{w}^\top \mathbf{x} + w_0$; para clasificar se mira su signo y la frontera es un hiperplano perpendicular a $\mathbf{w}$.
- **Mínimos cuadrados** minimiza la suma de residuos al cuadrado; tiene solución cerrada $(\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}$ y se evalúa con MSE, RMSE, MAE y R^2. Es sensible a puntos extremos.
- **Fisher** proyecta sobre la dirección $\mathbf{S}_W^{-1}(\mathbf{m}_A - \mathbf{m}_B)$, que maximiza la separación de las medias en relación con la dispersión dentro de las clases; no es la diferencia de medias cuando las clases están correlacionadas.
- El **perceptrón** corrige la frontera con cada error; converge si los datos son linealmente separables, en a lo más $(R/\gamma)^2$ errores, y cicla si no lo son (XOR).
- La regresión logística (capítulo siguiente) resuelve las limitaciones de los tres: da probabilidades, converge siempre y no castiga a los puntos bien clasificados.

60. Regresión Logística

La regresión logística es uno de los modelos de clasificación más utilizados en estadística y aprendizaje automático. A pesar de su nombre, no es un método de regresión en el sentido predictivo clásico: su objetivo es modelar la probabilidad de que una observación pertenezca a una clase determinada. Su elegancia reside en combinar la interpretabilidad de un modelo lineal con una transformación que garantiza salidas en el intervalo $(0, 1)$, haciendo posible tratar las predicciones como probabilidades genuinas (Hosmer et al. 2013; McCullagh y Nelder 1989).

El modelo fue propuesto formalmente por David Cox en 1958 y desde entonces se ha convertido en la referencia de partida para problemas de clasificación binaria en medicina, epidemiología, economía y ciencias sociales. En aprendizaje automático, sigue siendo competitivo frente a métodos más complejos cuando los datos son aproximadamente linealmente separables o cuando la interpretabilidad de los coeficientes es una prioridad (Hastie et al. 2009).

Este capítulo desarrolla la regresión logística desde sus fundamentos hasta sus variantes más importantes. Comenzamos motivando el problema de clasificar con regresión lineal y mostrando sus limitaciones. Luego introducimos la función logística, derivamos el modelo, estimamos sus parámetros por máxima verosimilitud, interpretamos los coeficientes en términos de odds ratios, estudiamos la regularización y finalizamos con la extensión multiclase mediante softmax. Los ejemplos computacionales están escritos en Python usando únicamente `numpy` y `plotly.graph_objects`, implementando los algoritmos desde cero.

Motivación: el problema de clasificar con regresión lineal

Antes de introducir la regresión logística, conviene entender por qué la regresión lineal resulta inadecuada para la clasificación binaria. Supongamos que deseamos predecir si un correo electrónico es spam ($y = 1$) o no ($y = 0$) a partir de una característica numérica x, como la frecuencia de la palabra “oferta”.

Si ajustamos una regresión lineal ordinaria, obtenemos un modelo de la forma:

$$\hat{y} = \beta_0 + \beta_1 x,$$

y la regla de decisión natural es: predecir clase 1 si $\hat{y} > 0.5$, clase 0 en caso contrario. Esta estrategia tiene tres problemas fundamentales:

1. **Predicciones fuera de rango**: el modelo lineal puede producir valores $\hat{y} < 0$ o $\hat{y} > 1$, que no tienen interpretación como probabilidades.
2. **Sensibilidad a observaciones atípicas**: un punto muy alejado en el eje x puede arrastrar la recta de regresión de manera que la frontera de decisión se desplace hacia regiones incorrectas.

3. **Supuestos violados**: la regresión lineal asume homocedasticidad y residuos gaussianos, supuestos que son claramente inapropiados cuando la variable respuesta es binaria.

La siguiente figura ilustra este problema con un conjunto sintético (James et al. 2021).

Figura 60.1.

Nótese cómo para valores grandes de x la recta predice valores mayores que 1, y teóricamente podría predecir valores negativos si hubiera observaciones de clase 0 con valores muy pequeños de x. Estas predicciones fuera de $[0, 1]$ son inutilizables como estimaciones de probabilidad (Hastie et al. 2009).

> **!** Importante
>
> La regresión lineal puede usarse como clasificador en casos simples, pero sus predicciones no son probabilidades y su frontera de decisión puede ser inestable en presencia de observaciones atípicas. La regresión logística resuelve estos problemas de forma principiada.

La función logística (sigmoide)

La solución natural al problema anterior es transformar la salida lineal mediante una función que mapee $\mathbb{R}$ al intervalo $(0, 1)$. La función más utilizada con este propósito es la **función logística**, también llamada función sigmoide.

Definición. La función logística $\sigma : \mathbb{R} \to (0, 1)$ se define como

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{60.1}$$

Esta función tiene varias propiedades matemáticas que la hacen especialmente adecuada para modelar probabilidades:

Rango: Para cualquier $z \in \mathbb{R}$, se cumple $0 < \sigma(z) < 1$. Cuando $z \to +\infty$, $\sigma(z) \to 1$; cuando $z \to -\infty$, $\sigma(z) \to 0$.

Simetría alrededor del origen: $\sigma(0) = 1/2$. Esto significa que el punto $z = 0$ separa las dos clases con probabilidad exactamente igual.

Derivada conveniente: La derivada de la sigmoide admite una expresión cerrada en términos de la misma función:

$$\frac{d\sigma}{dz}(z) = \sigma(z)(1 - \sigma(z)). \tag{60.2}$$

Esta propiedad simplifica enormemente el cálculo del gradiente durante el entrenamiento.

Interpretación como log-odds: Si definimos $p = \sigma(z)$, entonces se puede verificar que

$$\log \frac{p}{1 - p} = z. \tag{60.3}$$

La cantidad $\log(p/(1-p))$ se denomina **logit** o log-odds de p. La Ecuación 60.3 establece que el argumento z de la sigmoide equivale al logaritmo de las probabilidades relativas entre la clase positiva y la negativa. De ahí el nombre "regresión logística": el modelo asume que el logit de la probabilidad es una función lineal de las características (Bishop 2006).

La siguiente figura muestra la curva sigmoide con la posibilidad de desplazar el umbral de decisión z_0, que corresponde al punto donde la probabilidad predicha cruza 0.5.

Intuición geométrica

La función sigmoide convierte la recta numérica $\mathbb{R}$ en el intervalo abierto $(0, 1)$. El argumento z actúa como un "puntaje de clase positiva": valores grandes de z corresponden a probabilidades cercanas a 1, valores pequeños corresponden a probabilidades cercanas a 0, y $z = 0$ corresponde exactamente a probabilidad 0.5.

Cuando $z = \beta_0 + \beta^\top \mathbf{x}$, el umbral $z = 0$ corresponde al hiperplano $\beta_0 + \beta^\top \mathbf{x} = 0$, que es precisamente la **frontera de decisión** del modelo. Exploraremos esto con detalle en la sección siguiente.

El modelo de regresión logística

Con la función logística en mano, el modelo de regresión logística modela la probabilidad condicional de que la respuesta sea 1 dada una observación $\mathbf{x} \in \mathbb{R}^d$:

Definición. El modelo de regresión logística establece que

$$P(y = 1 \mid \mathbf{x}) = \sigma(\beta_0 + \beta^\top \mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top \mathbf{x})}}, \tag{60.4}$$

donde $\beta_0 \in \mathbb{R}$ es el intercepto y $\beta \in \mathbb{R}^d$ es el vector de coeficientes.

Y consecuentemente:

$$P(y = 0 \mid \mathbf{x}) = 1 - \sigma(\beta_0 + \beta^\top \mathbf{x}) = \sigma(-(\beta_0 + \beta^\top \mathbf{x})).$$

Frontera de decisión

La regla de clasificación más natural consiste en asignar la clase 1 cuando la probabilidad posterior supera 0.5:

$$\hat{y} = \begin{cases} 1 & \text{si } P(y = 1 \mid \mathbf{x}) \geq 0.5, \\ 0 & \text{en caso contrario.} \end{cases}$$

Como $\sigma(z) \geq 0.5$ si y solo si $z \geq 0$, la condición anterior equivale a:

$$\beta_0 + \beta^\top \mathbf{x} \geq 0. \tag{60.5}$$

El conjunto de puntos $\mathbf{x}$ donde $\beta_0 + \beta^\top \mathbf{x} = 0$ define un **hiperplano** en $\mathbb{R}^d$, que es la frontera de decisión del modelo. Esto implica que la regresión logística es un **clasificador lineal**: separa las clases mediante una frontera plana (Bishop 2006; Hastie et al. 2009).

> **Nota**
>
> La frontera de decisión de la regresión logística siempre es un hiperplano en el espacio original de características. Para capturar fronteras no lineales, es necesario incluir transformaciones de las características originales (polinomios, interacciones, etc.) antes de ajustar el modelo.

La siguiente figura muestra la frontera de decisión y el mapa de probabilidades posterior en dos dimensiones con datos sintéticos de dos clases gaussianas.

Notación compacta con vector aumentado

Es habitual absorber el intercepto β_0 en el vector de coeficientes definiendo $\tilde{\beta} = (\beta_0, \beta_1, \dots, \beta_d)^\top$ y aumentando cada observación con una coordenada fija igual a 1: $\tilde{\mathbf{x}} = (1, x_1, \dots, x_d)^\top$. Con esta convención, el modelo se escribe de forma compacta como:

$$P(y = 1 \mid \mathbf{x}) = \sigma\left(\tilde{\beta}^\top \tilde{\mathbf{x}}\right).$$

De aquí en adelante, cuando no haya ambigüedad, omitiremos la tilde y escribiremos simplemente $\beta^\top \mathbf{x}$ entendiendo que el vector $\mathbf{x}$ ya incluye la coordenada 1.

Construcción paso a paso: seis estudiantes

Antes de formalizar la estimación, sigamos el procedimiento con un ejemplo que cabe en una hoja. Seis estudiantes registraron sus horas de estudio y si aprobaron:

Tabla 60.1.: Horas de estudio y aprobación de seis estudiantes.

Horas x	1	2	3	4	5	6
Aprobó y	0	0	1	0	1	1

Paso 1. Un modelo inicial y su pérdida. Con $\beta_0 = \beta_1 = 0$, todos reciben $p = \sigma(0) = 0.5$. Cada estudiante aporta a la **log-loss** $-\ln p$ si aprobó y $-\ln(1-p)$ si no; aquí todos aportan $-\ln 0.5 = 0.693$, y la pérdida promedio es 0.693. Es la pérdida de quien no sabe nada.

Paso 2. El gradiente. La derivada de la pérdida promedio respecto a cada coeficiente es el promedio de $(p_i - y_i)$ multiplicado por la variable correspondiente (se deduce más abajo, en la Ecuación 60.8):

Tabla 60.2.: Componentes del gradiente en $= (0, 0)$.

x	1	2	3	4	5	6	Promedio
$p - y$	0.5	0.5	-0.5	0.5	-0.5	-0.5	$\partial/\partial\beta_0 = 0$
$x(p-y)$	0.5	1.0	-1.5	2.0	-2.5	-3.0	$\partial/\partial\beta_1 = -0.583$

La derivada respecto a β_1 es negativa: **subir la pendiente reduce la pérdida**, porque quienes estudian más tienden a aprobar.

Paso 3. Un paso de descenso. Con tasa de aprendizaje $\eta = 0.5$, $\beta_1 \leftarrow 0 - 0.5(-0.583) = 0.292$ y β_0 no cambia. Las probabilidades pasan a $\sigma(0.292x) = 0.57, 0.64, 0.71, 0.76, 0.81, 0.85$ y la pérdida baja a 0.672.

Paso 4. Hasta el óptimo. Repitiendo cientos de pasos (o con el método de Newton, más abajo) se llega a $\widehat{\beta}_0 = -4.25$ y $\widehat{\beta}_1 = 1.21$, con pérdida 0.413:

Tabla 60.3.: Probabilidades ajustadas.

x	1	2	3	4	5	6
$\widehat{p} = \sigma(-4.25 + 1.21x)$	0.05	0.14	0.35	0.65	0.86	0.95

La frontera $\widehat{p} = 0.5$ está en $x^* = 4.25/1.21 = 3.5$ horas: justo entre los dos casos «raros» (el estudiante de 3 horas que aprobó y el de 4 que no). Y $e^{1.21} = 3.37$: cada hora adicional **multiplica por 3.37 las probabilidades a favor** (*odds*) de aprobar, una interpretación que se desarrolla en la sección de *odds ratio*.

Estimación por máxima verosimilitud

Para aprender los parámetros β del modelo se usa el principio de **máxima verosimilitud** (MLE): se buscan los valores de β que hacen más probable el conjunto de observaciones de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ (Murphy 2012).

Dado que cada $y_i \in \{0, 1\}$, la distribución de y_i dado $\mathbf{x}_i$ es Bernoulli con parámetro $p_i = \sigma(\beta^\top \mathbf{x}_i)$:

$$P(y_i \mid \mathbf{x}_i; \beta) = p_i^{y_i}(1 - p_i)^{1-y_i}.$$

Asumiendo independencia entre las observaciones, la verosimilitud conjunta es:

$$L(\beta) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}.$$

Tomando logaritmo se obtiene la **log-verosimilitud**:

$$\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \tag{60.6}$$

Equivalencia con entropía cruzada

La Ecuación 60.6 es exactamente el negativo de la **entropía cruzada binaria** (binary cross-entropy), que es la función de pérdida estándar para clasificación binaria en redes neuronales y otros modelos. Maximizar la log-verosimilitud equivale a minimizar la entropía cruzada:

$$\mathcal{L}(\beta) = -\frac{1}{n}\ell(\beta) = -\frac{1}{n}\sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \tag{60.7}$$

Gradiente de la log-verosimilitud

A diferencia de la regresión lineal, la log-verosimilitud logística no admite una solución de forma cerrada. Sin embargo, la función es **cóncava** (o equivalentemente, la entropía cruzada es convexa), lo que garantiza que el máximo global existe y es único, y que los métodos de optimización basados en gradiente convergen al óptimo global (Hastie et al. 2009).

El gradiente de la log-verosimilitud respecto a β es:

$$\nabla_\beta \, \ell(\beta) = \mathbf{X}^\top (\mathbf{y} - \hat{\mathbf{p}}), \tag{60.8}$$

donde $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$ es la matriz de diseño (con columna de unos), $\mathbf{y} = (y_1, \dots, y_n)^\top$ y $\hat{\mathbf{p}} = (p_1, \dots, p_n)^\top$ con $p_i = \sigma(\beta^\top \mathbf{x}_i)$.

Derivación del gradiente. Para la j-ésima componente:

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^{n} \left[y_i \frac{1}{p_i} \frac{\partial p_i}{\partial \beta_j} - (1 - y_i) \frac{1}{1 - p_i} \frac{\partial p_i}{\partial \beta_j} \right].$$

Usando la propiedad de la derivada de la sigmoide (Ec. Ecuación 60.2):

$$\frac{\partial p_i}{\partial \beta_j} = p_i (1 - p_i) \, x_{ij}.$$

Sustituyendo:

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^{n} \left[y_i (1 - p_i) - (1 - y_i) p_i \right] x_{ij} = \sum_{i=1}^{n} (y_i - p_i) \, x_{ij},$$

lo que en forma matricial da exactamente la Ecuación 60.8.

Descenso de gradiente

El algoritmo de **descenso de gradiente** actualiza los parámetros en la dirección del gradiente de la log-verosimilitud (o el negativo del gradiente de la pérdida):

$$\beta^{(t+1)} = \beta^{(t)} + \eta \cdot \mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{p}}^{(t)}), \tag{60.9}$$

donde $\eta > 0$ es la **tasa de aprendizaje**. Para minimizar la entropía cruzada, la actualización sería con signo negativo.

Algoritmo IRLS

El método de Newton-Raphson aplicado a la log-verosimilitud logística da lugar al algoritmo **IRLS** (*Iteratively Reweighted Least Squares*). En cada iteración, el problema se reduce a una regresión lineal ponderada con matriz de pesos $\mathbf{W} = \operatorname{diag}(p_i(1-p_i))$:

$$\beta^{(t+1)} = \left(\mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{z}^{(t)},$$

donde $\mathbf{z}^{(t)} = \mathbf{X}\beta^{(t)} + (\mathbf{W}^{(t)})^{-1}(\mathbf{y} - \hat{\mathbf{p}}^{(t)})$ es la respuesta ajustada. IRLS converge cuadráticamente cerca del óptimo, mientras que el descenso de gradiente converge linealmente (McCullagh y Nelder 1989; Murphy 2012).

La siguiente figura muestra la evolución de la función de pérdida (entropía cruzada) durante el entrenamiento por descenso de gradiente.

Observamos que una tasa de aprendizaje grande ($\eta = 0.5$) converge rápidamente pero puede ser inestable, mientras que una tasa pequeña ($\eta = 0.02$) converge de manera más suave pero requiere más iteraciones. La elección de η es un hiperparámetro crítico del entrenamiento (Hastie et al. 2009).

Laboratorio: ajustar la logística

El laboratorio repite la construcción anterior. Mueve β_0 y β_1 a mano o da pasos de descenso de gradiente desde $(0, 0)$; cada barra vertical es la pérdida $-\ln \hat{p}(y_i)$ de un estudiante. El modo *Separable* elimina el traslape (los tres primeros reprueban y los tres últimos aprueban): sin regularización los coeficientes crecen sin límite.

En la versión web, este laboratorio ajusta la regresión logística a los seis estudiantes. Valores de referencia: en $(0, 0)$ la log-loss es 0.693 y el gradiente es $(0, -0.583)$; tras un paso con $\eta = 0.5$, $\beta = (0, 0.292)$ y la pérdida es 0.672; el óptimo es $(-4.25, 1.21)$ con pérdida 0.413, frontera en 3.5 horas y *odds ratio* 3.37. Con datos separables y $\lambda = 0$ los coeficientes crecen sin límite.

Guía del laboratorio.

1. Desde $(0, 0)$, da un paso con $\eta = 0.5$ y comprueba los números del Paso 3.
2. Pulsa *200 pasos* varias veces. ¿Por qué la trayectoria avanza en zigzag por un valle diagonal? ¿Qué pasaría si centraras x restándole su media?
3. Predice antes de mover: con $\eta = 1.5$, ¿el descenso converge más rápido o se vuelve inestable?

4. En el óptimo, ¿qué estudiante aporta más pérdida? ¿Por qué el modelo no puede evitarlo?
5. Cambia a *Separable* con $\lambda = 0$ y da muchos pasos. ¿Qué pasa con β_1 y con la pérdida? Sube λ a 0.05: ¿dónde queda ahora el óptimo?

Interpretación de coeficientes: odds ratio

Una de las grandes ventajas de la regresión logística sobre otros clasificadores es que sus coeficientes admiten una **interpretación directa en términos de probabilidades y razones de momios** (*odds ratios*) (Hosmer et al. 2013).

Log-odds y odds ratio

De la Ecuación 60.3 sabemos que el modelo puede escribirse en la escala del logit:

$$\log \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \beta_0 + \beta_1 x_1 + \cdots + \beta_d x_d. \tag{60.10}$$

Esta expresión revela que la regresión logística es, en realidad, un modelo **lineal en la escala del log-odds**. El coeficiente β_j indica el cambio en el log-odds asociado a un incremento unitario en x_j, manteniendo las demás variables constantes.

Para obtener el **odds ratio** asociado a x_j, basta con exponenciar el coeficiente:

$$OR_j = e^{\beta_j}. \tag{60.11}$$

Interpretación: Si x_j aumenta en una unidad, los odds de la clase positiva se **multiplican** por e^{β_j}:

- $e^{\beta_j} > 1$: aumentan los odds de clase positiva.
- $e^{\beta_j} < 1$: disminuyen los odds de clase positiva.
- $e^{\beta_j} = 1$: el predictor no tiene efecto sobre los odds.

Intervalos de confianza

Los intervalos de confianza para los coeficientes se construyen usando la matriz de información de Fisher, que es la Hessiana negativa de la log-verosimilitud:

$$\mathbf{I}(\beta) = \mathbf{X}^\top \mathbf{W} \mathbf{X},$$

donde $\mathbf{W} = \text{diag}(p_i(1 - p_i))$. Los errores estándar de los coeficientes son las raíces cuadradas de los elementos diagonales de $\mathbf{I}(\beta)^{-1}$, y un intervalo de confianza aproximado al 95 % es:

$$\beta_j \pm 1.96 \cdot \text{SE}(\beta_j).$$

La siguiente figura muestra un **forest plot** con los coeficientes estimados e intervalos de confianza al 95 %, estilo habitual en publicaciones epidemiológicas (Hosmer et al. 2013).

La tabla siguiente resume la interpretación de los coeficientes del ejemplo anterior:

Tabla 60.4.: Interpretación de los coeficientes del modelo logístico.

Variable	$\hat{\beta}_j$	$e^{\hat{\beta}_j}$	Interpretación
Edad (estand.)	≈ 0.80	≈ 2.23	Cada SD de edad multiplica por 2.2 los odds
Colesterol	≈ 0.50	≈ 1.65	Asociación positiva moderada
Presión arterial	≈ 0.60	≈ 1.82	Riesgo elevado con mayor presión
Tabaquismo	≈ 1.20	≈ 3.32	Mayor factor de riesgo individual
Ejercicio físico	≈ -0.70	≈ 0.50	Factor protector (reduce a la mitad los odds)

> **Tip**
>
> Cuando las variables continuas están estandarizadas (media 0, desviación estándar 1), los coeficientes son directamente comparables entre sí y el odds ratio refleja el efecto de cambiar la variable en una desviación estándar.

Regularización

El problema de la separabilidad perfecta

Un supuesto implícito en la estimación por MLE es que los datos no son **perfectamente separables**. Si existe un hiperplano que separa perfectamente las dos clases, el gradiente de la log-verosimilitud nunca se anula: siempre es posible aumentar el módulo de β para hacer las predicciones más extremas y aumentar la verosimilitud. El resultado es que los coeficientes divergen hacia $\pm\infty$, haciendo el modelo numéricamente inestable e inutilizable (Hastie et al. 2009).

Este fenómeno ocurre con frecuencia cuando:

- El número de variables d es grande en relación con n.
- Las variables predictoras están altamente correlacionadas (colinealidad).
- Se incluyen variables binarias o indicadoras que predicen perfectamente un evento raro.

La solución estándar es añadir un término de **regularización** que penalice coeficientes grandes.

Regularización L2 (Ridge)

La penalización L2 añade el cuadrado de la norma euclidiana de β a la función de pérdida:

$$\mathcal{L}_{\text{Ridge}}(\beta) = -\ell(\beta) + \lambda \|\beta\|_2^2 = -\ell(\beta) + \lambda \sum_{j=1}^{d} \beta_j^2. \tag{60.12}$$

El parámetro $\lambda > 0$ controla la intensidad de la regularización. A mayor λ, mayor penalización a coeficientes grandes, y el modelo tiende hacia $\beta = \mathbf{0}$.

El gradiente de la pérdida regularizada es:

$$\nabla \mathcal{L}_{\text{Ridge}} = -\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{p}}) + 2\lambda\beta.$$

La regularización L2 corresponde, en un marco bayesiano, a colocar una distribución prior gaussiana $\beta \sim \mathcal{N}(\mathbf{0}, (2\lambda)^{-1}\mathbf{I})$ sobre los coeficientes.

Regularización L1 (Lasso)

La penalización L1 añade la norma ℓ_1 de β:

$$\mathcal{L}_{\text{Lasso}}(\beta) = -\ell(\beta) + \lambda\|\beta\|_1 = -\ell(\beta) + \lambda \sum_{j=1}^{d} |\beta_j|. \tag{60.13}$$

La diferencia clave con L2 es que L1 produce soluciones **dispersas** (sparse): para λ suficientemente grande, muchos coeficientes se vuelven exactamente cero, realizando selección automática de variables. Bayesianamente, corresponde a una distribución prior de Laplace sobre β (Murphy 2012).

La penalización L1 no es diferenciable en $\beta_j = 0$, por lo que requiere algoritmos especiales como el descenso de subgradiente o el descenso coordinado.

Elastic Net

La **Elastic Net** combina ambas penalizaciones:

$$\mathcal{L}_{\text{EN}}(\beta) = -\ell(\beta) + \lambda \left[\alpha\|\beta\|_1 + \frac{1-\alpha}{2}\|\beta\|_2^2\right], \tag{60.14}$$

donde $\alpha \in [0, 1]$ controla el balance entre L1 y L2. Cuando $\alpha = 1$ se recupera el Lasso; cuando $\alpha = 0$ se recupera Ridge. La Elastic Net es útil cuando hay grupos de variables correlacionadas, pues tiende a seleccionar o descartar grupos completos de variables correlacionadas.

Regularization path

El **regularization path** muestra cómo evolucionan los coeficientes estimados en función de λ. Para $\lambda \to 0$ se recuperan los estimadores MLE sin regularización; para $\lambda \to \infty$ todos los coeficientes convergen a cero.

Nótese la diferencia fundamental entre los dos paneles: en Ridge, los coeficientes se encogen suavemente hacia cero pero nunca llegan a serlo exactamente (excepto en el límite $\lambda \to \infty$). En Lasso, los coeficientes se vuelven exactamente cero a partir de un valor de λ determinado, realizando selección de variables implícita (Hastie et al. 2009).

> **Nota**
>
> En la práctica, el parámetro λ se elige mediante validación cruzada. Una mala elección puede llevar a sobreajuste (λ muy pequeño) o a un modelo demasiado simple (λ muy grande).

Extensión multiclase

La regresión logística puede extenderse a problemas con $K > 2$ clases mediante dos estrategias principales: **One-vs-Rest** y **regresión logística multinomial** (softmax).

One-vs-Rest (OvR)

La estrategia **One-vs-Rest** (también llamada One-vs-All, OvA) entrena K clasificadores binarios independientes. El k-ésimo clasificador aprende a distinguir la clase k de todas las demás clases combinadas. Para una nueva observación $\mathbf{x}$, se calcula la probabilidad $p_k = P(y = k \mid \mathbf{x})$ de cada clasificador binario, y se asigna la clase con mayor probabilidad:

$$\hat{y} = \arg \max_{k \in \{1,\dots,K\}} \hat{p}_k(\mathbf{x}).$$

La ventaja es la simplicidad: solo se requieren K modelos logísticos estándar. La desventaja es que las K probabilidades no suman 1 necesariamente, ya que provienen de modelos independientes.

Regresión logística multinomial (softmax)

La regresión logística multinomial extiende el modelo de manera conjunta y probabilísticamente coherente. Se aprenden $K - 1$ conjuntos de parámetros β_k (uno de los K sirve como referencia, típicamente $\beta_K = \mathbf{0}$) y la probabilidad de cada clase se calcula mediante la función **softmax**:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\beta_k^\top \mathbf{x}}}{\sum_{j=1}^{K} e^{\beta_j^\top \mathbf{x}}}. \tag{60.15}$$

La función softmax garantiza que las probabilidades sumen 1 y sean positivas, siendo una generalización directa de la función sigmoide al caso multiclase. La log-verosimilitud se convierte en:

$$\ell(\{\beta_k\}) = \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbb{1}[y_i = k] \log P(y_i = k \mid \mathbf{x}_i), \tag{60.16}$$

que es la **entropía cruzada categórica** generalizada. El gradiente respecto a β_k es:

$$\nabla_{\beta_k} \ell = \sum_{i=1}^{n} \left(\mathbb{1}[y_i = k] - P(y_i = k \mid \mathbf{x}_i)\right) \mathbf{x}_i = \mathbf{X}^\top (\mathbf{r}_k - \hat{\mathbf{p}}_k),$$

donde $\mathbf{r}_k = (\mathbb{1}[y_1 = k], \ldots, \mathbb{1}[y_n = k])^\top$ (Bishop 2006; Murphy 2012).

La siguiente figura muestra la frontera de decisión multiclase obtenida con el modelo softmax en un problema 2D con tres clases.

Comparación OvR vs. Softmax

Tabla 60.5.: Comparación entre One-vs-Rest y regresión logística multinomial.

Aspecto	One-vs-Rest	Softmax multinomial
Coherencia probabilística	No garantizada	Garantizada ($\sum_k p_k = 1$)
Número de modelos	K modelos separados	1 modelo conjunto
Costo computacional	$K\times$ costo binario	Similar, pero conjunto
Interpretación	Directa por modelo	Requiere clase de referencia
Separabilidad perfecta	Cada modelo por separado	Puede ocurrir globalmente

En la práctica, para K grande (como clasificación de texto con muchas categorías), OvR suele ser preferible por su paralelizabilidad. Para K pequeño o cuando se requiere coherencia probabilística, el modelo multinomial es superior (Murphy 2012).

Calibración de probabilidades

Una propiedad deseable pero no siempre garantizada en los clasificadores es que sus **predicciones de probabilidad estén bien calibradas**: si el modelo predice $P(y = 1 \mid \mathbf{x}) = 0.7$, entonces aproximadamente el 70 % de las observaciones con esa predicción deberían pertenecer a la clase 1.

Definición. Un clasificador está **perfectamente calibrado** si para todo valor $p \in (0, 1)$:

$$P(y = 1 \mid \hat{p}(\mathbf{x}) = p) = p.$$

Calibración de la regresión logística

La regresión logística está **bien calibrada por diseño** bajo sus propios supuestos: el entrenamiento por máxima verosimilitud minimiza la pérdida de entropía cruzada, que es precisamente la pérdida asociada a la calibración probabilística. En contraste, otros clasificadores como los bosques aleatorios o las SVM no están calibrados de forma natural (Hastie et al. 2009; Murphy 2012).

Sin embargo, esta garantía de calibración solo se mantiene si:

1. El modelo está correctamente especificado (la verdadera frontera de decisión es lineal en $\mathbf{x}$).
2. No hay sobreajuste severo.
3. Se usa un conjunto de evaluación independiente del conjunto de entrenamiento.

La calibración se visualiza mediante la **curva de calibración** (o *reliability diagram*): se dividen las predicciones en B contenedores (*bins*) según el valor de $\hat{p}$, y se grafica la fracción real de positivos en cada contenedor contra el promedio de $\hat{p}$ en ese contenedor. Un clasificador perfectamente calibrado sigue la diagonal.

Cuando la curva de calibración se aleja sistemáticamente de la diagonal, se puede aplicar **calibración post-hoc** mediante regresión isotónica o escala de Platt (una regresión logística adicional sobre las puntuaciones del modelo) para corregir el sesgo sin reentrenar el modelo completo (Murphy 2012).

Ventajas y limitaciones

Ventajas

Ventajas de la regresión logística

Interpretabilidad: Los coeficientes tienen una interpretación directa en términos de log-odds y odds ratios, lo que facilita la comunicación de resultados en contextos aplicados como la medicina o las ciencias sociales.
Eficiencia computacional: El entrenamiento por descenso de gradiente o IRLS es eficiente incluso con conjuntos de datos grandes. No requiere almacenar la matriz completa de datos durante la predicción.
Salidas probabilísticas calibradas: A diferencia de SVM o clasificadores de árbol, la regresión logística produce estimaciones de probabilidad que están bien calibradas bajo sus supuestos.
Pocos hiperparámetros: El único hiperparámetro principal es el parámetro de regularización λ (y el tipo de penalización). Esto simplifica el ajuste del modelo.
Estabilidad numérica: La función de pérdida es convexa, lo que garantiza convergencia al óptimo global con métodos de optimización estándar.
Línea de base sólida: En muchos problemas prácticos, la regresión logística es difícil de superar significativamente con métodos más complejos, especialmente cuando los datos son aproximadamente linealmente separables (James et al. 2021).

Limitaciones

Limitaciones de la regresión logística

Frontera de decisión lineal: La regresión logística solo puede aprender fronteras lineales en el espacio de características. Para problemas con fronteras no lineales complejas (como clasificación de imágenes o texto), otros métodos pueden ser más adecuados.
Sensibilidad a la colinealidad: Cuando dos o más variables predictoras están altamente correlacionadas, los coeficientes estimados pueden tener alta varianza y ser difíciles de interpretar. La regularización L2 mitiga este problema.
Separabilidad perfecta: En dimensiones altas o con pocas observaciones, los datos pueden ser perfectamente separables y el MLE no está definido. Requiere regularización obligatoria.
Supuesto de linealidad en el logit: Si la relación entre las características y el logit no es lineal, el modelo estará mal especificado. Es necesario incluir transformaciones o interacciones

manualmente.
Sensibilidad a datos atípicos: Los puntos atípicos en el espacio de características pueden influir desproporcionadamente en los coeficientes estimados, aunque menos que en regresión lineal.

Recomendaciones prácticas

Las siguientes recomendaciones sintetizan buenas prácticas para el uso de la regresión logística en proyectos reales de clasificación.

1. **Estandarizar las variables continuas.** Antes de ajustar el modelo, escalar las variables a media 0 y desviación estándar 1 mejora la comparabilidad de los coeficientes, acelera la convergencia del gradiente y hace que el intercepto sea más interpretable.

2. **Usar regularización L2 por defecto.** A menos que se tenga una razón específica para selección de variables, la penalización Ridge es la opción más robusta: estabiliza el entrenamiento, reduce la varianza y es diferenciable en todo punto.

3. **Elegir λ por validación cruzada.** Explorar un grid logarítmico de valores de λ (por ejemplo, 10^{-4} a 10^{2}) usando k-fold cross-validation (típicamente $k = 5$ o $k = 10$) para seleccionar el valor óptimo.

4. **Verificar la calibración.** Construir la curva de calibración en un conjunto de validación independiente. Si la calibración es pobre (curva alejada de la diagonal), aplicar calibración post-hoc antes de publicar probabilidades.

5. **Revisar la separabilidad.** Si el algoritmo de optimización no converge o los coeficientes son extremadamente grandes, sospechar separabilidad perfecta. La solución es incrementar la regularización.

6. **Explorar transformaciones de características.** Si el modelo tiene baja capacidad predictiva, considerar añadir términos cuadráticos, interacciones o transformaciones logarítmicas de las variables predictoras antes de migrar a modelos más complejos.

7. **Interpretar con intervalo de confianza.** Siempre reportar los odds ratios con sus intervalos de confianza, no solo las estimaciones puntuales. Un coeficiente estadísticamente no significativo no implica ausencia de efecto, sino incertidumbre.

8. **Comparar con una línea de base simple.** Antes de declarar que un modelo más complejo supera a la regresión logística, asegurarse de que la comparación se hace con la versión regularizada y con características bien preprocesadas.

9. **Cuidar el desbalanceo de clases.** Cuando una clase es mucho menos frecuente que la otra (por ejemplo, fraude, enfermedad rara), considerar ajustar el umbral de clasificación o usar métricas como AUC-ROC o F1 en lugar de la exactitud (*accuracy*).

10. **Documentar el proceso de selección del modelo.** Registrar qué variables se incluyeron, qué transformaciones se aplicaron, qué valor de λ se seleccionó y por qué. La reproducibilidad es esencial en análisis de datos serios.

Resumen del capítulo

La regresión logística es un modelo de clasificación que combina la simplicidad de los modelos lineales con la capacidad de producir estimaciones de probabilidad bien calibradas. Sus ingredientes fundamentales son la función logística (Ecuación 60.1), el modelo de probabilidad (Ecuación 60.4), la estimación por máxima verosimilitud (Ecuación 60.6) y la interpretación de los coeficientes como log-odds (Ecuación 60.10).

A lo largo del capítulo hemos visto que:

- La regresión lineal falla para clasificación porque produce predicciones fuera de $[0, 1]$ y es inestable ante observaciones atípicas.
- La función sigmoide transforma la recta real en $(0, 1)$ y su inversa, el logit, da nombre al modelo.
- La frontera de decisión es un hiperplano en el espacio de características.
- El gradiente de la log-verosimilitud tiene la forma elegante $\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{p}})$, que permite optimización eficiente.
- Los coeficientes se interpretan como cambios en el log-odds; exponenciados dan los odds ratios.
- La regularización (Ridge, Lasso o Elastic Net) es esencial en alta dimensión o cuando los datos son separables.
- La extensión multiclase se logra mediante softmax, una generalización directa de la sigmoide.
- La regresión logística está bien calibrada por diseño, a diferencia de muchos otros clasificadores.

Para profundizar en los fundamentos matemáticos se recomienda (Bishop 2006) y (Hastie et al. 2009); para aplicaciones en ciencias de la salud, (Hosmer et al. 2013); para una perspectiva bayesiana, (Murphy 2012); y para una introducción más accesible con ejemplos en R, (James et al. 2021).

61. Máquinas de Soporte Vectorial

Las Máquinas de Soporte Vectorial (SVM, del inglés *Support Vector Machines*) son uno de los algoritmos de aprendizaje supervisado más elegantes y teóricamente sólidos que existen. Desarrolladas por Vapnik y colaboradores en los años noventa (Vapnik 1995; Cortes y Vapnik 1995), combinan tres ideas fundamentales: la búsqueda de un hiperplano de margen máximo, la reformulación del problema mediante dualidad de Lagrange, y el llamado *kernel trick*, que permite operar en espacios de alta dimensión sin calcular explícitamente las coordenadas en ese espacio.

A diferencia de métodos como la regresión logística, que buscan cualquier frontera de decisión que separe bien los datos, las SVM buscan la frontera *óptima*: aquella que maximiza la distancia mínima a los puntos de entrenamiento. Esta propiedad geométrica tiene consecuencias profundas sobre la capacidad de generalización del modelo, vinculando la clasificación a nociones de la teoría estadística del aprendizaje (Hastie et al. 2009).

Este capítulo desarrolla la teoría de SVM desde sus fundamentos geométricos hasta sus extensiones prácticas, incluyendo el manejo de datos no separables, el kernel trick, la regresión con SVM y la calibración de probabilidades. Los ejemplos computacionales están escritos en Python usando `numpy` y `plotly`, construyendo las visualizaciones desde primeros principios para reforzar la intuición geométrica.

Motivación: clasificar con margen máximo

El problema de los múltiples hiperplanos separadores

Supongamos que tenemos un conjunto de datos linealmente separable: existen puntos de dos clases que pueden dividirse perfectamente mediante una línea recta (en 2D) o un hiperplano (en dimensiones superiores). La primera pregunta natural es: ¿cuántos hiperplanos separadores existen?

La respuesta es que, en general, existen infinitos. Si un hiperplano H separa perfectamente las clases, también lo hace cualquier hiperplano obtenido al desplazar o rotar H ligeramente, siempre que no cruce ningún punto de entrenamiento. Ante esta ambigüedad, surge la pregunta clave: **¿cuál de todos esos hiperplanos debería elegirse?**

Por qué el margen importa

La respuesta de las SVM es elegir el hiperplano que maximiza el **margen**: la distancia mínima entre el hiperplano y los puntos de entrenamiento más cercanos de cada clase. La intuición es geométrica y probabilística a la vez. Un hiperplano con margen grande es más robusto frente a perturbaciones de los datos: si un punto nuevo cae cerca de la frontera de decisión, un margen amplio le da más "espacio" para ser clasificado correctamente. Un hiperplano con margen pequeño, aunque separe

perfectamente los datos de entrenamiento, estará demasiado cerca de algunos puntos y será frágil ante datos nuevos.

La teoría de aprendizaje estadístico de Vapnik-Chervonenkis formaliza este argumento: la complejidad efectiva de un clasificador de margen máximo depende del margen, no de la dimensión del espacio de características (Vapnik 1995). Esto significa que las SVM pueden generalizar bien incluso en dimensiones muy altas, siempre que el margen sea grande.

Intuición geométrica: margen = 2/||w||

Un hiperplano en $\mathbb{R}^d$ se define como el conjunto de puntos $\mathbf{x}$ que satisfacen:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

donde $\mathbf{w} \in \mathbb{R}^d$ es el vector normal al hiperplano y $b \in \mathbb{R}$ es el sesgo. La distancia con signo de un punto $\mathbf{x}_i$ al hiperplano es:

$$d_i = \frac{\mathbf{w}^\top \mathbf{x}_i + b}{\|\mathbf{w}\|}$$

Si normalizamos la escala de $\mathbf{w}$ de modo que los puntos más cercanos de cada clase satisfagan $|\mathbf{w}^\top \mathbf{x}_i + b| = 1$, entonces la distancia del hiperplano a esos puntos es $1/\|\mathbf{w}\|$. El margen total —la distancia entre las dos "bandas" de clase— es:

$$\text{margen} = \frac{2}{\|\mathbf{w}\|}$$

Maximizar el margen equivale a minimizar $\|\mathbf{w}\|$.

Figura 61.1.

SVM de margen duro (datos linealmente separables)

Formulación primal

Dado un conjunto de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $y_i \in \{-1, +1\}$, queremos encontrar el hiperplano $(\mathbf{w}, b)$ que maximice el margen $2/\|\mathbf{w}\|$ sujeto a que todos los puntos estén correctamente clasificados con al menos una distancia de $1/\|\mathbf{w}\|$ al hiperplano. Esto equivale a:

$$\min_{\mathbf{w},\, b} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{sujeto a} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall\, i = 1, \ldots, n \tag{61.1}$$

La restricción $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ garantiza que los puntos de clase $+1$ queden al lado positivo del hiperplano (con valor funcional ≥ 1) y los de clase -1 queden al lado negativo (con valor funcional ≤ -1). Minimizar $\frac{1}{2}\|\mathbf{w}\|^2$ equivale a maximizar el margen $2/\|\mathbf{w}\|$.

Vectores de soporte

Los **vectores de soporte** son los puntos de entrenamiento que satisfacen la restricción con igualdad: $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$. Estos puntos están exactamente sobre las bandas de margen (los hiperplanos $\mathbf{w}^\top \mathbf{x} + b = \pm 1$) y son los únicos que determinan la solución. Si se eliminaran todos los demás puntos, el hiperplano óptimo no cambiaría. Esta propiedad de escasez (*sparsity*) es una de las características más importantes de las SVM.

Formulación dual y el rol del Lagrangiano

Para derivar la formulación dual, se introduce un multiplicador de Lagrange $\alpha_i \geq 0$ por cada restricción de la formulación primal (Ecuación 61.1). El Lagrangiano es:

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \left[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1 \right]$$

Tomando las condiciones de optimalidad de primer orden respecto a $\mathbf{w}$ y b:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \implies \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{i=1}^{n} \alpha_i y_i = 0$$

Sustituyendo de regreso en el Lagrangiano, se obtiene el **problema dual**:

$$\max_{\alpha} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \tag{61.2}$$

$$\text{sujeto a} \quad \alpha_i \geq 0, \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

La función dual depende de los datos sólo a través de productos internos $\mathbf{x}_i^\top \mathbf{x}_j$. Esta observación es el punto de partida del kernel trick.

Condiciones KKT

Las condiciones de Karush-Kuhn-Tucker (KKT) para este problema de optimización implican, entre otras cosas, la condición de complementariedad:

$$\alpha_i[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1] = 0, \quad \forall\, i$$

Esta condición dice que, para cada punto, o bien $\alpha_i = 0$ (el punto no es vector de soporte y no contribuye a la solución) o bien $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$ (el punto está exactamente sobre el margen y es vector de soporte). En la práctica, la mayoría de los α_i son cero, lo que hace que la solución sea dispersa.

Función de decisión en forma dual

Una vez resuelto el problema dual y obtenidos los α_i, la función de decisión para un nuevo punto $\mathbf{x}$ es:

$$f(\mathbf{x}) = \operatorname{sgn}\left(\sum_{i \in \mathcal{S}} \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b\right) \tag{61.3}$$

donde $\mathcal{S}$ es el conjunto de vectores de soporte (aquellos con $\alpha_i > 0$). El sesgo b se recupera de cualquier vector de soporte usando $y_s(\mathbf{w}^\top \mathbf{x}_s + b) = 1$.

Un ejemplo a mano

Cuatro puntos: los negativos $\mathbf{x}_1 = (1, 2)$ y $\mathbf{x}_2 = (2, 1)$, y los positivos $\mathbf{x}_3 = (3, 3)$ y $\mathbf{x}_4 = (4, 4)$. Por simetría respecto a la diagonal, la frontera óptima debe ser perpendicular a $(1, 1)$: $\mathbf{w} = (a, a)$. Los puntos más cercanos a la frontera serán $\mathbf{x}_1$, $\mathbf{x}_2$ (que dan el mismo valor $\mathbf{w}^\top \mathbf{x} = 3a$) y $\mathbf{x}_3$ ($6a$). Si son vectores de soporte, cumplen las restricciones con igualdad:

$$-(3a + b) = 1, \qquad 6a + b = 1 \quad \Longrightarrow \quad a = \tfrac{2}{3}, \quad b = -3.$$

La frontera es $\frac{2}{3}x_1 + \frac{2}{3}x_2 - 3 = 0$, es decir, $x_1 + x_2 = 4.5$, justo a la mitad entre $x_1 + x_2 = 3$ y $x_1 + x_2 = 6$. El margen es $2/\|\mathbf{w}\| = 2/(\frac{2}{3}\sqrt{2}) = 2.12$, la distancia entre las dos rectas paralelas que tocan a los vectores de soporte. El punto $\mathbf{x}_4$ cumple $\frac{2}{3}(8) - 3 = 2.33 > 1$: está fuera del margen y no influye.

Los multiplicadores se obtienen de $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ y $\sum_i \alpha_i y_i = 0$. Por simetría $\alpha_1 = \alpha_2 = \alpha$; la segunda condición da $\alpha_3 = 2\alpha$ y la primera, $-\alpha(1, 2) - \alpha(2, 1) + 2\alpha(3, 3) = (3\alpha, 3\alpha) = (\frac{2}{3}, \frac{2}{3})$, de donde

$$\alpha_1 = \alpha_2 = \tfrac{2}{9} \approx 0.222, \qquad \alpha_3 = \tfrac{4}{9} \approx 0.444, \qquad \alpha_4 = 0.$$

Es la condición KKT en acción: el único punto con $\alpha = 0$ es el que no toca el margen. Si se moviera $\mathbf{x}_4$ a cualquier lugar lejos de la frontera, la solución no cambiaría; si se agregara un punto negativo cerca, por ejemplo en $(3.5, 1.5)$, la frontera giraría por completo. El laboratorio permite comprobarlo.

SVM de margen suave (datos no separables)

En la práctica, los datos raramente son linealmente separables. Incluso cuando la separación es posible, un pequeño número de puntos atípicos puede impedir encontrar cualquier hiperplano válido. La SVM de **margen suave** aborda este problema introduciendo variables de holgura que permiten violar las restricciones de manera controlada (Cortes y Vapnik 1995).

Variables de holgura

Para cada punto de entrenamiento se introduce una variable de holgura $\xi_i \geq 0$ que mide cuánto viola la restricción de margen ese punto:

- $\xi_i = 0$: el punto está correctamente clasificado y fuera del margen.
- $0 < \xi_i \leq 1$: el punto está correctamente clasificado pero dentro del margen.
- $\xi_i > 1$: el punto está mal clasificado.

Formulación primal con margen suave

El problema de optimización se modifica para penalizar las violaciones:

$$\min_{\mathbf{w},\, b,\, \xi} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \tag{61.4}$$

$$\text{sujeto a} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall\, i$$

El parámetro $C > 0$ controla el equilibrio entre maximizar el margen y minimizar las violaciones:

- $C \to \infty$: se penalizan fuertemente los errores, equivale al margen duro.
- $C \to 0$: se permiten muchos errores, el modelo ignora casi todos los puntos y el margen se hace arbitrariamente grande.

La formulación dual correspondiente es similar a la del margen duro, con la diferencia de que los multiplicadores están acotados superiormente por C:

$$\max_{\alpha} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j$$

$$\text{sujeto a} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

La cota superior $\alpha_i \leq C$ es la única diferencia respecto al problema dual de margen duro, y es una consecuencia directa de las variables de holgura en el Lagrangiano.

El kernel trick

El problema de la no separabilidad lineal

La SVM de margen duro y suave opera con hiperplanos: fronteras de decisión lineales en el espacio de características. Sin embargo, muchos problemas del mundo real tienen estructuras no lineales: dos clases pueden estar dispuestas en anillos concéntricos, en espirales, o en cualquier forma que ningún hiperplano puede separar.

La solución clásica sería transformar los datos a un espacio de mayor dimensión donde sí sean linealmente separables. Si $\phi : \mathbb{R}^d \to \mathbb{R}^D$ es un mapeo a un espacio de dimensión $D \gg d$, entonces se aplicaría la SVM sobre los puntos transformados $\phi(\mathbf{x}_i)$. El problema es que si D es muy grande (incluso infinito), calcular $\phi(\mathbf{x})$ explícitamente puede ser computacionalmente inviable.

El kernel trick: producto interno implícito

La clave está en observar que la formulación dual de la SVM (Ecuación 61.2) y la función de decisión (Ecuación 61.3) dependen de los datos únicamente a través de productos internos $\mathbf{x}_i^\top \mathbf{x}_j$. Si se aplica el mapeo ϕ, estos productos se convierten en $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$.

Una **función kernel** es una función $K : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ que computa este producto interno de manera implícita:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) \tag{61.5}$$

El kernel evalúa el producto interno en el espacio de alta dimensión sin necesidad de calcular $\phi(\mathbf{x})$ explícitamente. Basta con sustituir $\mathbf{x}_i^\top \mathbf{x}_j$ por $K(\mathbf{x}_i, \mathbf{x}_j)$ en todas las expresiones. Esto se conoce como el *kernel trick* (Schölkopf y Smola 2002).

Kernels más utilizados

Kernel lineal:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$$

Equivale a no hacer transformación alguna. Produce fronteras de decisión lineales en el espacio original.

Kernel polinomial:

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma\, \mathbf{x}_i^\top \mathbf{x}_j + r)^d$$

Donde d es el grado del polinomio, $\gamma > 0$ y $r \geq 0$. El mapeo implícito incluye todos los monomios de grado hasta d. Por ejemplo, con $d = 2$ y $\mathbf{x} \in \mathbb{R}^2$, el espacio transformado incluye términos $x_1^2, x_2^2, x_1 x_2, x_1, x_2, 1$.

Kernel RBF (Gaussiano):

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\big(-\gamma \, \|\mathbf{x}_i - \mathbf{x}_j\|^2\big) \tag{61.6}$$

El parámetro $\gamma > 0$ controla el ancho del gaussiano. Este kernel corresponde a un mapeo a un espacio de dimensión infinita: su expansión en serie de Taylor genera todos los polinomios de todos los grados. Geométricamente, $K(\mathbf{x}_i, \mathbf{x}_j)$ mide la similitud entre dos puntos: vale 1 cuando son idénticos y decae a 0 conforme se alejan. El radio de influencia de cada punto de entrenamiento es $\sim 1/\sqrt{\gamma}$.

Kernel sigmoide:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \, \mathbf{x}_i^\top \mathbf{x}_j + r)$$

Este kernel no cumple las condiciones de Mercer para todos los valores de γ y r, lo que puede causar problemas numéricos en algunos casos.

Condición de Mercer

No toda función simétrica puede ser un kernel válido. La **condición de Mercer** establece que K es un kernel válido (positivo semidefinido) si y sólo si, para cualquier conjunto finito de puntos $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, la matriz de Gram $\mathbf{K}$ con entradas $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ es positiva semidefinida (Schölkopf y Smola 2002):

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j K(\mathbf{x}_i, \mathbf{x}_j) \geq 0, \quad \forall \, c_i \in \mathbb{R}$$

Esta condición garantiza que existe un mapeo ϕ tal que $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$, y que el problema de optimización dual es cóncavo (tiene solución única).

Efecto de los hiperparámetros C y (kernel RBF)

Cuando se usa el kernel RBF (Ecuación 61.6), la SVM tiene dos hiperparámetros principales que deben ajustarse mediante validación cruzada (Hastie et al. 2009):

Parámetro C (regularización):

- **C grande**: Se penalizan fuertemente los errores de clasificación. El modelo intenta clasificar correctamente casi todos los puntos de entrenamiento, lo que puede producir fronteras muy irregulares y sobreajuste.
- **C pequeño**: Se permite más margen a cambio de tolerar más errores en entrenamiento. El modelo generaliza mejor en datos ruidosos.

Parámetro (ancho del kernel RBF):

- **grande**: El radio de influencia de cada punto de soporte es pequeño ($\sim 1/\sqrt{\gamma}$). La frontera de decisión se vuelve muy irregular, capturando la forma local de cada punto. Alto riesgo de sobreajuste.

- **pequeño**: Cada punto de soporte influye sobre una región amplia. La frontera es suave. Alto riesgo de subajuste si es demasiado pequeño.

La interacción entre C y es compleja: un C grande con un grande produce el modelo más complejo (muy propenso a sobreajuste). Un C pequeño con un pequeño produce el modelo más simple (frontera suave, posible subajuste).

Comparación de kernels en datos reales

Un paso importante en la práctica es comparar diferentes kernels sobre los mismos datos para determinar cuál captura mejor la estructura subyacente. En datos con estructura circular o no lineal, el kernel lineal fracasará, mientras que el kernel RBF con parámetros adecuados puede lograr una separación casi perfecta (Bishop 2006).

Laboratorio: margen, C y núcleos

El laboratorio resuelve el problema dual con una versión simplificada de SMO. Las líneas discontinuas son las bandas $f(\mathbf{x}) = \pm 1$; los puntos con anillo son vectores de soporte. Arrastra los puntos para ver qué cambia la solución y qué no. El estado inicial reproduce el ejemplo a mano.

En la versión web, este laboratorio resuelve la SVM con SMO y permite arrastrar los puntos. Valores de referencia con los cuatro puntos y C = 1000: tres vectores de soporte con $\alpha = 0.222, 0.222, 0.444$, $\mathbf{w} = (0.667, 0.667)$, $b = -3$ y margen 2.12. Con datos en anillo, solo el núcleo RBF produce una frontera adecuada.

Guía del laboratorio.

1. Comprueba en la tabla los multiplicadores del ejemplo a mano y el margen 2.12.
2. Predice antes de mover: si arrastras $\mathbf{x}_4$ hacia $(5, 5)$, ¿cambia la frontera? ¿Y si lo arrastras a $(3.2, 2.0)$?
3. Arrastra $\mathbf{x}_2$ hasta $(3.5, 1.5)$. ¿Cuánto gira la frontera? Compara con el resultado de `scikit-learn` en el texto.
4. Con *Traslape*, baja C de 1000 a 0.01. ¿Qué pasa con el número de vectores de soporte, el margen y los errores de entrenamiento?
5. Con *Anillo* y RBF, sube hasta el máximo. ¿Qué forma toma la frontera? ¿Por qué un grande sobreajusta?

SVM para regresión (SVR)

Las ideas de las SVM se extienden naturalmente al problema de regresión mediante la **Support Vector Regression** (SVR) (Vapnik 1995). La diferencia fundamental respecto a la regresión lineal clásica es que la SVR no penaliza errores pequeños: introduce un margen de tolerancia ε alrededor de la función de regresión, y sólo penaliza las predicciones que caen fuera de ese tubo.

El tubo -insensible

La función de pérdida ε-insensible es:

$$L_\varepsilon(y, f(\mathbf{x})) = \text{máx}(0,\, |y - f(\mathbf{x})| - \varepsilon)$$

Esta función vale cero cuando el error de predicción es menor que ε, y crece linealmente con el error por encima de ese umbral. Geométricamente, define un **tubo** de ancho 2ε alrededor de la función de regresión: los puntos dentro del tubo no contribuyen a la función de pérdida.

Formulación primal de SVR

Introduciendo variables de holgura $\xi_i^+, \xi_i^- \geq 0$ para las violaciones por encima y por debajo del tubo respectivamente, el problema de optimización es:

$$\min_{\mathbf{w},\, b,\, \xi^+,\, \xi^-} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}(\xi_i^+ + \xi_i^-)$$

$$\text{sujeto a} \quad y_i - f(\mathbf{x}_i) \leq \varepsilon + \xi_i^+$$

$$f(\mathbf{x}_i) - y_i \leq \varepsilon + \xi_i^-$$

$$\xi_i^+,\, \xi_i^- \geq 0$$

donde $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$. Los **vectores de soporte en SVR** son los puntos que caen fuera o exactamente sobre el borde del tubo ($\xi_i^+ > 0$ o $\xi_i^- > 0$, o que satisfacen la igualdad). Igual que en clasificación, la solución es dispersa: sólo los vectores de soporte determinan $\mathbf{w}$.

Aspectos computacionales

Complejidad del entrenamiento

El entrenamiento de una SVM implica resolver un problema de programación cuadrática (QP) con n variables (α_i) y $n+1$ restricciones. Los algoritmos de QP de propósito general tienen complejidad $O(n^3)$ en tiempo y $O(n^2)$ en memoria debido a la necesidad de calcular y almacenar la matriz de Gram $\mathbf{K}$ de tamaño $n \times n$.

Esto hace que las SVM estándar sean impracticables para $n > 10{,}000$ a $50{,}000$ puntos, dependiendo de la memoria disponible. Para n grande, existen varias estrategias:

- **SMO (Sequential Minimal Optimization)**: Algoritmo de Platt (Platt 1999) que descompone el problema QP en subproblemas de tamaño 2 que se resuelven analíticamente. Es el algoritmo más usado en implementaciones modernas como LIBSVM (Chang y Lin 2011).

- **Chunking y decomposición**: Divide el problema en subproblemas más pequeños, solucionando sólo los pares de variables con mayor violación de las condiciones KKT en cada iteración.
- **Aproximaciones de bajo rango**: Métodos como Nyström o random features que aproximan la matriz kernel con una representación de rango bajo.

SMO (Sequential Minimal Optimization)

El algoritmo SMO selecciona en cada iteración el par de multiplicadores (α_i, α_j) que más viola las condiciones KKT. Dado que la restricción $\sum_i \alpha_i y_i = 0$ debe mantenerse, cambiar un solo α_i obliga a cambiar al menos otro. El subproblema de optimización para dos variables tiene solución analítica explícita:

$$\alpha_j^{\text{nuevo}} = \alpha_j^{\text{viejo}} + \frac{y_j(E_i - E_j)}{\eta}$$

donde $E_i = f(\mathbf{x}_i) - y_i$ es el error de clasificación del punto i, y $\eta = K_{ii} + K_{jj} - 2K_{ij}$.

Escalabilidad: SVM vs modelos lineales

Tabla 61.1.: Comparación de complejidad computacional

Modelo	Entrenamiento	Predicción	Memoria
SVM (kernel)	$O(n^2)$ a $O(n^3)$	$O(n_{\text{sv}} \cdot d)$	$O(n^2)$
SVM (lineal)	$O(n \cdot d)$	$O(d)$	$O(d)$
Regresión logística	$O(n \cdot d)$	$O(d)$	$O(d)$
Redes neuronales	Variable	$O(d \cdot \text{capas})$	$O(\text{parámetros})$

Para n muy grande (millones de puntos), los modelos lineales como la regresión logística con regularización ℓ_2 son preferibles. Las SVM con kernel son más competitivas cuando n es moderado (miles) y la dimensión del espacio de características es alta.

Cuándo usar SVM

Las SVM son especialmente adecuadas cuando:

1. El número de características es alto respecto al número de muestras ($d \gg n$), como en clasificación de texto o genómica.
2. Se busca una solución dispersa: sólo los vectores de soporte son necesarios.
3. Los datos tienen una estructura de margen claro y el kernel captura bien la geometría.
4. La memoria es limitada y no se pueden cargar todos los datos simultáneamente (implementaciones que usan sólo los vectores de soporte).

Ventajas y limitaciones

Ventajas de las SVM

- **Efectivas en alta dimensión**: El principio de margen máximo no depende de la dimensión del espacio de entrada. Las SVM funcionan bien cuando $d \gg n$, lo que las hace populares en clasificación de texto y bioinformática.
- **Robustez frente a sobreajuste**: El margen máximo actúa como una forma de regularización implícita. La complejidad efectiva del clasificador depende del margen, no de la dimensión.
- **Solución global única**: El problema dual es estrictamente cóncavo (cuando el kernel es definido positivo), garantizando que la solución es única y global.
- **Solución dispersa**: Sólo los vectores de soporte determinan la frontera. En la predicción, sólo se necesitan estos puntos.
- **Flexibilidad mediante kernels**: El kernel trick permite capturar estructuras no lineales arbitrarias sin aumentar explícitamente la dimensión.
- **Buena generalización cuando** $n < d$: En regímenes donde hay pocas muestras pero muchas características, las SVM suelen superar a redes neuronales.

Limitaciones de las SVM

- **Escala mal con** n **grande**: La complejidad $O(n^2)$ a $O(n^3)$ hace que el entrenamiento sea lento para conjuntos de más de 50,000–100,000 puntos. Para grandes datos, métodos como regresión logística o redes neuronales con SGD son más prácticos.
- **Elección del kernel y sus hiperparámetros**: La selección del kernel adecuado y el ajuste de C, γ (y posiblemente d, r) requiere validación cruzada exhaustiva. Una mala elección puede llevar a un rendimiento muy pobre.
- **No da probabilidades directamente**: La salida de la SVM es una función de decisión $f(\mathbf{x})$, no una probabilidad. Para obtener probabilidades se necesita calibración adicional (ver Sección 61).
- **Sensible a la escala de las características**: El kernel RBF y el kernel lineal son sensibles a las escalas relativas de las características. Es necesario estandarizar los datos antes de entrenar.
- **Interpretabilidad limitada**: En espacios de alta dimensión con kernel no lineal, es difícil interpretar qué características son importantes para la decisión.
- **Un solo umbral de decisión**: Las SVM son clasificadores binarios por naturaleza. Para problemas multiclase se necesitan estrategias como uno-contra-uno o uno-contra-todos.

Obtener probabilidades de SVM: calibración de Platt

Una limitación práctica de las SVM es que no producen estimaciones de probabilidad directamente: la salida es una puntuación real $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ (o su versión kernelizada), no una probabilidad $P(y = 1 \mid \mathbf{x})$. En muchas aplicaciones —como sistemas de diagnóstico médico o modelos de riesgo— se necesita saber no sólo la clase predicha sino también la confianza en la predicción.

El método de calibración de Platt

John Platt propuso en 1999 un método elegante para convertir las puntuaciones de la SVM en probabilidades calibradas (Platt 1999). La idea es ajustar un modelo de regresión logística sobre las puntuaciones de la SVM:

$$P(y = 1 \mid f(\mathbf{x})) = \frac{1}{1 + \exp(A f(\mathbf{x}) + B)} \tag{61.7}$$

donde A y B son parámetros que se estiman por máxima verosimilitud usando los scores $f(\mathbf{x}_i)$ y las etiquetas y_i. Para evitar sobreoptimismo, Platt recomienda usar **validación cruzada** para obtener los scores $f(\mathbf{x}_i)$: se entrena la SVM en un subconjunto y se obtienen los scores en los puntos dejados fuera.

Relación con la regresión logística

La calibración de Platt transforma la SVM en algo conceptualmente similar a un clasificador probabilístico. De hecho, la regresión logística puede verse como una SVM "calibrada" desde el origen: en lugar de maximizar el margen de manera directa, minimiza la pérdida logística que es una aproximación suave de la pérdida de bisagra (*hinge loss*) usada en las SVM.

La pérdida de bisagra de la SVM es:

$$L_{\text{hinge}}(y, f(\mathbf{x})) = \max(0,\, 1 - y\, f(\mathbf{x}))$$

La pérdida logística es:

$$L_{\log}(y, f(\mathbf{x})) = \log(1 + \exp(-y\, f(\mathbf{x})))$$

Ambas penalizan las clasificaciones incorrectas, pero la pérdida logística es diferenciable en todas partes y produce estimaciones de probabilidad directamente. La pérdida de bisagra tiene gradiente cero para los puntos bien clasificados con margen suficiente, lo que explica por qué la solución SVM es dispersa.

La calibración de Platt es un puente entre ambas perspectivas: usa la estructura de margen de la SVM (que es geométricamente óptima) y añade encima una capa logística para producir probabilidades bien calibradas.

Recomendaciones prácticas

Al aplicar SVM a un problema real, las siguientes recomendaciones pueden mejorar significativamente los resultados (Chang y Lin 2011; Hastie et al. 2009):

1. **Estandarizar las características**: Escalar cada característica a media cero y desviación estándar uno antes de entrenar. El kernel RBF depende de distancias euclídeas; sin estandarización, las características con mayor varianza dominarán.

2. **Comenzar con el kernel RBF**: A menos que se sepa de antemano que los datos son linealmente separables, el kernel RBF con parámetros ajustados es un buen punto de partida. Es flexible, tiene pocos hiperparámetros y funciona bien en la mayoría de los casos.

3. **Ajustar C y mediante validación cruzada en grid**: Explorar C en $\{10^{-3}, 10^{-2}, \ldots, 10^{3}\}$ y γ en $\{10^{-4}, \ldots, 10^{2}\}$. Usar búsqueda en grid con 5-fold o 10-fold CV. El logaritmo de los parámetros es más informativo que la escala lineal.

4. **Verificar la escala del problema**: Si $n > 50{,}000$, considerar alternativas más escalables como regresión logística con regularización ℓ_2 (equivalente a la SVM lineal). Las SVM con kernel son difíciles de entrenar a esa escala.

5. **Usar implementaciones eficientes**: LIBSVM (Chang y Lin 2011) es la implementación de referencia, accesible desde Python vía `scikit-learn` (`SVC`, `SVR`). Para datos linealmente separables de gran escala, `LinearSVC` usa una formulación diferente (LIBLINEAR) que escala a millones de puntos.

6. **Calibrar probabilidades si es necesario**: Si el problema requiere probabilidades (no sólo clases), activar la calibración de Platt (`probability=True` en scikit-learn, que usa CV interna) o usar `CalibratedClassifierCV`.

7. **Considerar el desbalance de clases**: Si las clases están muy desbalanceadas, usar `class_weight='balanced'` que ajusta los parámetros C de cada clase inversamente proporcional a su frecuencia.

8. **Validar los vectores de soporte**: Un número muy alto de vectores de soporte (cercano a n) indica que el modelo está subajustando o que C es demasiado pequeño. Un número muy bajo podría indicar sobreajuste si el error de generalización es alto.

9. **Inspeccionar las curvas de aprendizaje**: Para diagnosticar sobreajuste vs subajuste, graficar el error de entrenamiento y validación en función del tamaño del conjunto de entrenamiento. Las SVM con kernel RBF frecuentemente tienen alta varianza con pocos datos y se benefician de más muestras.

10. **Explorar SVR para regresión con outliers**: Cuando los datos de regresión contienen outliers, SVR con tubo ε amplio es más robusto que la regresión por mínimos cuadrados, que asigna pérdida cuadrática a todos los residuos.

Parte XXXI.

Clasificadores Probabilísticos

62. Clasificadores bayesianos

Los métodos de aprendizaje supervisado pueden verse desde dos perspectivas complementarias. En una perspectiva determinista, el objetivo es aprender directamente una función de decisión $f(\mathbf{x})$ que asigne cada observación a una clase, produzca una predicción continua o aproxime una relación funcional. En una perspectiva probabilística, en cambio, el objetivo es modelar explícitamente la incertidumbre: no sólo se decide una clase, sino que se estima qué tan plausible es cada clase dada la evidencia observada. Esto se traduce en que, si no observamos niguna caracteristica del objeto, la mejor decisión que podemos tomar depende de la probabilidad previa de cada clase.

Para ilustrar este hecho, supongamos que somos participantes dentro del famoso juego *En familia con chabelo*, donde se nos da la oportunidad de ganar un auto, para esto debemos de escoger dentro de 3 posibles puertas donde detrás se encuentra el auto o una cabra. Si no se nos da ninguna pista previa es razonable pensar que contamos con una probabilidad de $\frac{1}{3}$ de ganar el auto, por lo que la mejor decisión es escoger una puerta al azar.

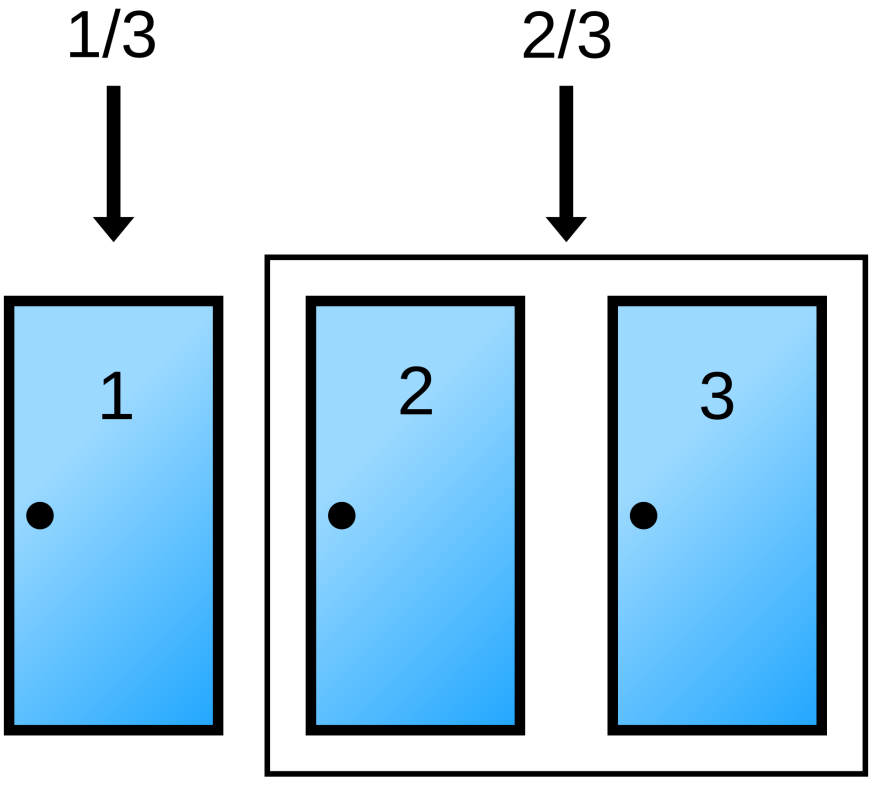


Figura 62.1.: Notemos como la probabilidad de encontrar el auto detrás de las puertas cerradas es de $\frac{1}{3}$, mientras que la probabilidad de encontrar la cabra es de $\frac{2}{3}$

Asumiendo que nuestra elección es la puerta 1, como sucede en muchas ocasiones el presentador nos hará notar que detras de la puerta tres hay una cabra, con lo cual es natural preguntarse: ¿Bajo que condiciones tendremos la mejor probabilidad de ganar?, es común pensar que dado que sabemos que detrás de las puerta de nuestra elección hay o una cabra o un auto la probabilidad de encntrar nuestro premio tras estas 2 puertas es de $\frac{1}{2}$, sin embargo nuestra intución en ese caso sería incorrecta.

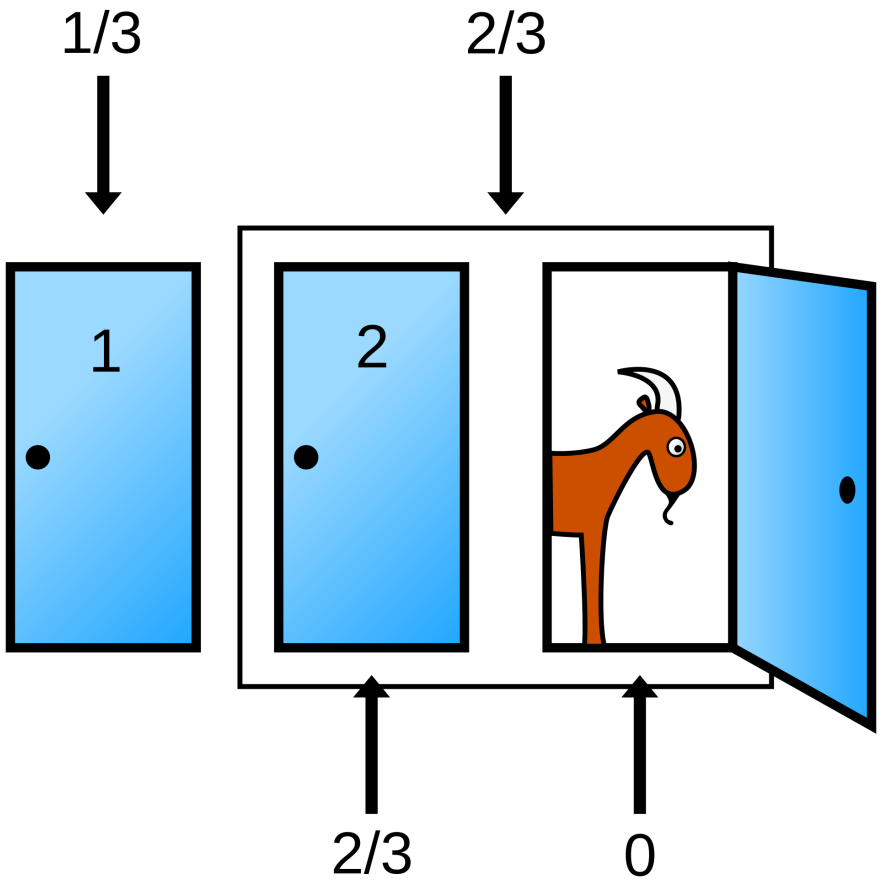


Figura 62.2.: Una vez se ha abierto una puerta con una cabra, dado qeu el presentador solo podría haber abierto la puerta que tuviera detrás una cabra obliga a que la probabilidad de ganar el auto si cambiamos a la puerta 2 es de $\frac{2}{3}$

Para ver con más detalle por qué conviene cambiar de puerta, hagamos las cuentas paso a paso. Antes de que Chabelo abra una puerta, el auto puede estar en cualquiera de las tres puertas con la misma probabilidad:

$$P(A_1) = P(A_2) = P(A_3) = \frac{1}{3}$$

donde A_i significa que el auto está detrás de la puerta i.

Ahora bien, Chabelo sabe dónde está el auto y siempre abre una puerta que tiene una cabra. Además, si tiene dos puertas con cabra disponibles, asumimos que escoge una de ellas al azar. Entonces las posibilidades son las siguientes:

Las ramas donde cambiar gana valen por 2 porque tienen probabilidad $\frac{1}{3}$, mientras que las ramas donde cambiar pierde valen $\frac{1}{6}$. Esto ocurre porque si el auto está en nuestra puerta inicial, Chabelo tiene dos puertas posibles para abrir y divide ese caso en dos ramas de probabilidad $\frac{1}{6}$. En cambio, si el auto está en una puerta distinta a la nuestra, Chabelo no tiene libertad: sólo puede abrir la única puerta restante con cabra. Por eso cada una de esas ramas conserva probabilidad $\frac{1}{3}$, equivalente a dos partes de tamaño $\frac{1}{6}$.

Tabla 62.1.: Los cuatro caminos posibles cuando elegimos la puerta 1. Quedarse gana con probabilidad $1/6 + 1/6 = 1/3$; cambiar, con $1/3 + 1/3 = 2/3$.

Auto detrás de	Chabelo abre	Probabilidad	Si nos quedamos	Si cambiamos
Puerta 1	Puerta 2	$\frac{1}{3} \cdot \frac{1}{2} = \frac{1}{6}$	gana	pierde
Puerta 1	Puerta 3	$\frac{1}{3} \cdot \frac{1}{2} = \frac{1}{6}$	gana	pierde
Puerta 2	Puerta 3	$\frac{1}{3} \cdot 1 = \frac{1}{3}$	pierde	gana
Puerta 3	Puerta 2	$\frac{1}{3} \cdot 1 = \frac{1}{3}$	pierde	gana

En la versión web, un laboratorio simula el juego con 3 a 10 puertas. Con tres puertas, quedarse gana con probabilidad 1/3 y cambiar con 2/3; con n puertas, $1/n$ y $(n-1)/n$.

Como en el ejemplo Chabelo abre la puerta 3 y muestra una cabra, debemos quedarnos únicamente con los casos compatibles con esa información. Es decir, descartamos todos los casos donde Chabelo abre la puerta 2 o donde el auto estaba detrás de la puerta 3.

Los casos posibles después de observar que Chabelo abrió la puerta 3 son:

Caso	Ubicación del auto	Probabilidad inicial	Probabilidad de que Chabelo abra la puerta 3	Probabilidad conjunta
1	Puerta 1	$\frac{1}{3}$	$\frac{1}{2}$	$\frac{1}{3} \cdot \frac{1}{2} = \frac{1}{6}$
2	Puerta 2	$\frac{1}{3}$	1	$\frac{1}{3} \cdot 1 = \frac{1}{3}$
3	Puerta 3	$\frac{1}{3}$	0	$\frac{1}{3} \cdot 0 = 0$

La probabilidad total de que Chabelo abra la puerta 3 es:

$$P(\text{Chabelo abre puerta 3}) = \frac{1}{6} + \frac{1}{3} + 0$$

$$P(\text{Chabelo abre puerta 3}) = \frac{1}{6} + \frac{2}{6} = \frac{3}{6} = \frac{1}{2}$$

Ahora actualizamos las probabilidades usando la información nueva. Primero, calculemos la probabilidad de que el auto esté en la puerta 1 dado que Chabelo abrió la puerta 3:

$$P(A_1 \mid \text{Chabelo abre puerta 3}) = \frac{P(A_1 \cap \text{Chabelo abre puerta 3})}{P(\text{Chabelo abre puerta 3})}$$

$$P(A_1 \mid \text{Chabelo abre puerta 3}) = \frac{\frac{1}{6}}{\frac{1}{2}} = \frac{1}{6} \cdot \frac{2}{1} = \frac{2}{6} = \frac{1}{3}$$

Por lo tanto, si nos quedamos con la puerta 1, la probabilidad de ganar sigue siendo:

$$P(\text{ganar si nos quedamos}) = \frac{1}{3}$$

Ahora calculemos la probabilidad de que el auto esté en la puerta 2 dado que Chabelo abrió la puerta 3:

$$P(A_2 \mid \text{Chabelo abre puerta 3}) = \frac{P(A_2 \cap \text{Chabelo abre puerta 3})}{P(\text{Chabelo abre puerta 3})}$$

$$P(A_2 \mid \text{Chabelo abre puerta 3}) = \frac{\frac{1}{3}}{\frac{1}{2}} = \frac{1}{3} \cdot \frac{2}{1} = \frac{2}{3}$$

Por lo tanto, si cambiamos a la puerta 2, la probabilidad de ganar es:

$$P(\text{ganar si cambiamos}) = \frac{2}{3}$$

La clave está en que la puerta que elegimos inicialmente conserva su probabilidad original de $\frac{1}{3}$. La información nueva no reparte la probabilidad en partes iguales entre las dos puertas cerradas. Más bien, la probabilidad que antes estaba distribuida entre las dos puertas que no escogimos se concentra en la única puerta no abierta por Chabelo.

En resumen:

$$P(\text{ganar si nos quedamos}) = \frac{1}{3}$$

$$P(\text{ganar si cambiamos}) = \frac{2}{3}$$

Así que la mejor estrategia es cambiar de puerta. La pista de Chabelo no es una simple eliminación de una opción: es evidencia nueva que cambia nuestras probabilidades posteriores. Esta es precisamente la idea central del razonamiento bayesiano: actualizar nuestras creencias iniciales cuando observamos nueva información.

En esta simulación podemos observar como las probabilidades del problema a lo largo de las diferentes realizaciones convergen a los valores teóricos de $\frac{1}{3}$ para la estrategia de quedarse y $\frac{2}{3}$ para la estrategia de cambiar.

Esta es la diferencia central en los clasificadores bayesianos, la decisión se deriva de probabilidades a priori, densidades condicionales y probabilidades posteriores (Duda et al. 2001; Bishop 2006). Por ejemplo, si una caja contiene monedas de dos tipos y una de ellas es más frecuente, la decisión con menor probabilidad de error antes de observar la moneda es escoger la clase más probable. Sin embargo, una vez que se observa una característica, como el peso, la decisión debe incorporar esa evidencia. La pregunta deja de ser "¿qué clase es más frecuente?" y pasa a ser "¿qué clase es más probable dado el valor observado?".

Los clasificadores bayesianos usan el teorema de Bayes para decidir a qué clase pertenece una observación. Para cada clase ω_i, se considera qué tan probable es observar los datos $\mathbf{x}$ si esa clase fuera la correcta, junto con qué tan probable era esa clase antes de ver los datos. Con esta información se calcula la probabilidad de cada clase después de observar $\mathbf{x}$. La decisión más sencilla consiste en elegir la clase con mayor probabilidad posterior; si algunos errores son más costosos que otros, se puede modificar la decisión para tomar en cuenta esos costos (Duda et al. 2001; Jaynes 2003).

Este capítulo desarrolla la clasificación bayesiana desde sus fundamentos hasta algunas variantes prácticas: Naïve Bayes, clasificadores gaussianos, modelos de Markov para secuencias discretas, decisión bayesiana con costos y el criterio minimax cuando las probabilidades previas son inciertas. Los ejemplos computacionales están escritos en Python y buscan conectar la teoría con implementaciones reproducibles.

Si tuvieramos
monedas de u
monedas de di
de observar el
decisión sería
más frecuente.
vez que se obs
decisión debe i
evidencia. La p
ser "¿qué clase
frecuente?" y p
clase es más p
valor observad



Vista probabilística de la clasificación

Sea $\mathbf{x} \in \mathbb{R}^d$ una observación y sea $\Omega = \{\omega_1, \dots, \omega_c\}$ el conjunto de clases posibles. En clasificación probabilística se supone que $\mathbf{x}$ no determina de manera absoluta la clase, sino que modifica nuestras creencias sobre ella. Antes de observar $\mathbf{x}$, la información disponible está representada por las probabilidades previas $P(\omega_i)$. Después de observar $\mathbf{x}$, la información relevante se resume en las probabilidades posteriores $P(\omega_i \mid \mathbf{x})$.

Definición. La probabilidad posterior de la clase ω_i dada una observación $\mathbf{x}$ se obtiene mediante el teorema de Bayes:

$$P(\omega_i \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid \omega_i) P(\omega_i)}{p(\mathbf{x})} \tag{62.1}$$

En la ecuación Ecuación 62.1, $p(\mathbf{x} \mid \omega_i)$ es la densidad condicional de la observación bajo la clase ω_i, $P(\omega_i)$ es la probabilidad previa de esa clase y $p(\mathbf{x})$ es la densidad marginal de la observación:

$$p(\mathbf{x}) = \sum_{j=1}^{c} p(\mathbf{x} \mid \omega_j) P(\omega_j).$$

Como $p(\mathbf{x})$ no depende de la clase que se compara, muchas reglas de decisión pueden escribirse en términos de la cantidad no normalizada $p(\mathbf{x} \mid \omega_i)P(\omega_i)$.

La decisión de clasificación.

Si no hay observaciones, una regla razonable para dos clases es escoger la clase más probable a priori:

$$\text{decidir } \omega_1 \quad \text{si} \quad P(\omega_1) > P(\omega_2).$$

El error esperado de esta regla, cuando sólo existen dos clases, es

$$P(e) = \min\{P(\omega_1), P(\omega_2)\}.$$

Una vez observada $\mathbf{x}$, la decisión cambia: se debe comparar $P(\omega_1 \mid \mathbf{x})$ contra $P(\omega_2 \mid \mathbf{x})$. Esta distinción es la esencia de la clasificación bayesiana: la evidencia observada actualiza las probabilidades de las clases.

Definición. La regla de máxima probabilidad posterior, o regla MAP, asigna la observación $\mathbf{x}$ a la clase con posterior máxima:

$$\hat{\omega}(\mathbf{x}) = \arg\max_{\omega_i \in \Omega} P(\omega_i \mid \mathbf{x}) \tag{62.2}$$

Ejemplo básico: El dilema del Cine

Esta persona dejó caer su boleto en el pasillo.

¿Le dices?

> “Disculpe, señora”

o

> “Disculpe, señor”

Tienes que hacer una conjetura.



Para responder de la manera mas acertada posible podemos hacer uso del teorema de Bayes, para esto definamos el evento H como "hombre" y el evento M como "mujer", mientras que la evidencia observada es el tipo de cabello de la persona. Entonces, la probabilidad de que la persona sea hombre o mujer dado el tipo de cabello se calcula con Bayes:

$$P(\text{sexo} \mid \text{cabello}) = \frac{P(\text{cabello} \mid \text{sexo})P(\text{sexo})}{\sum_{\text{sexo}} P(\text{sexo})P(\text{cabello} \mid \text{sexo})}$$

Donde bajo una asumpción justa podemos suponer que dentro del cine:

De cada 100 mujeres en el cine

- 50 tienen cabello corto
- 50 tienen cabello largo

De cada 100 hombres en el cine

- 96 tienen cabello corto
- 4 tienen cabello largo

Hagamos los calculos paso a paso, si asumimos que la proporción de hombres y mujeres en el cine es la misma, entonces las probabilidades previas son:

$$P(H) = P(M) = 0.5$$

Mientras que las verosimilitudes son:

$$P(\text{cabello largo} \mid H) = 0.04$$

$$P(\text{cabello largo} \mid M) = 0.50$$

teniendo esto, Entonces los numeradores son:

$$P(\text{largo} \mid H)P(H) = 0.04(0.5) = 0.02$$

$$P(\text{largo} \mid M)P(M) = 0.50(0.5) = 0.25$$

Entonces:

$$P(H \mid \text{cabello largo}) = 0.02A \approx 0.07$$

$$P(M \mid \text{cabello largo}) = 0.25A \approx 0.93$$

donde

$$A = \frac{1}{0.02 + 0.25}$$

Así, bajo la regla de máxima probabilidad posterior, la decisión es:

$$\Rightarrow \textbf{Decisión: mujer}$$

> **Nota**
>
> La idea central es que no decidimos solo por la característica observada, sino por cómo esa característica modifica nuestras creencias previas sobre cada clase.

Dilema en el cine: el contexto importa

Es importante notar que la decisión anterior depende de las probabilidades previas. Si el contexto cambia, las probabilidades previas también cambian, lo que puede llevar a una decisión diferente incluso con la misma evidencia observada. Por ejemplo, si esta situación ocurre mientras estás formado en la fila del baño de hombres, la proporción de hombres y mujeres cambia drásticamente, lo que afecta las probabilidades previas y, por ende, la decisión bayesiana.

¿Le dices?

> "Disculpe, señora"

o

> "Disculpe, señor"



Ahora las probabilidades previas cambian:

$$P(H) = 0.98, \qquad P(M) = 0.02$$

donde H representa "hombre" y M representa "mujer". De nuevo asumiendo de manera justa que dentro de la fila:

De cada 2 mujeres en la fila

- 1 tiene cabello corto
- 1 tiene cabello largo

De cada 98 hombres en la fila

- 94 tienen cabello corto
- 4 tienen cabello largo

De los datos anteriores:

$$P(\text{cabello largo} \mid H) = \frac{4}{98}$$

$$P(\text{cabello largo} \mid M) = \frac{1}{2}$$

Calculamos los numeradores de Bayes:

$$P(\text{cabello largo} \mid H)P(H) = \frac{4}{98}(0.98) = 0.04$$

$$P(\text{cabello largo} \mid M)P(M) = \frac{1}{2}(0.02) = 0.01$$

La constante de normalización es:

$$B = \frac{1}{0.04 + 0.01} = 20$$

Entonces, bajo la misma regla de máxima probabilidad posterior, las probabilidades posteriores son:

$$P(H \mid \text{cabello largo}) = 0.04B = 0.04(20) = 0.80$$

$$P(M \mid \text{cabello largo}) = 0.01B = 0.01(20) = 0.20$$

Como:

$$P(H \mid \text{cabello largo}) > P(M \mid \text{cabello largo})$$

la decisión bayesiana es:

$$\Rightarrow \textbf{Decisión: hombre}$$

! Importante

La misma evidencia observada (cabello largo) puede llevar a una decisión distinta cuando cambian las probabilidades previas.
En clasificación bayesiana, el contexto modifica la inferencia.

Implementación básica: algoritmo general de un clasificador bayesiano

A continuación se muestra una implementación básica de un clasificador bayesiano para cualquier número de clases. El código calcula las probabilidades posteriores y toma la decisión de clasificación basada en la clase con mayor probabilidad posterior.

En este ejemplo, aunque la verosimilitud bajo la clase de enfermedad es mayor, el prior de la clase normal domina la decisión de error mínimo. Esto ilustra una idea importante: una evidencia local no se interpreta de manera aislada, sino en relación con la frecuencia previa de cada clase.

En la versión web, un laboratorio recalcula la posterior al mover la previa y las verosimilitudes. Con $P(H) = 0.5$, $P(\text{largo} \mid H) = 0.04$ y $P(\text{largo} \mid M) = 0.5$ se obtiene $P(H \mid \text{largo}) = 0.074$; en la fila del baño de hombres, con $P(H) = 0.98$, la posterior sube a 0.80 y la decisión cambia a «señor».

Clasificador bayesiano de error mínimo

La regla MAP tiene una interpretación óptima cuando todos los errores tienen el mismo costo: minimiza la probabilidad promedio de clasificación incorrecta. Para verlo, consideremos inicialmente dos clases. Si se asigna $\mathbf{x}$ a ω_1, el error condicional es $P(\omega_2 \mid \mathbf{x})$; si se asigna a ω_2, el error condicional es $P(\omega_1 \mid \mathbf{x})$. Por tanto, para minimizar el error en cada punto $\mathbf{x}$ se elige la clase con mayor posterior.

Definición. El error condicional de clasificación para dos clases es

$$P(e \mid \mathbf{x}) = \begin{cases} P(\omega_2 \mid \mathbf{x}), & \text{si se decide } \omega_1, \\ P(\omega_1 \mid \mathbf{x}), & \text{si se decide } \omega_2. \end{cases} \tag{62.3}$$

El error promedio se obtiene integrando sobre todo el espacio de observaciones:

$$P(e) = \int P(e \mid \mathbf{x}) p(\mathbf{x}) \, d\mathbf{x}.$$

Dado que $p(\mathbf{x}) \geq 0$, minimizar el error promedio se logra minimizando el error condicional punto a punto. Así, para dos clases:

$$P(\omega_1 \mid \mathbf{x}) > P(\omega_2 \mid \mathbf{x}) \quad \Longrightarrow \quad \mathbf{x} \in \omega_1.$$

Ejemplo en Python: decisión de error mínimo

El siguiente código muestra la regla de error mínimo para varias observaciones. Para cada observación se tienen las verosimilitudes $p(\mathbf{x} \mid \omega_i)$ bajo dos clases. Después se calculan las posteriores, se elige la clase con mayor posterior y se reporta el error condicional mínimo.

En este ejemplo, una observación se clasifica como `enfermedad` sólo cuando su posterior supera a la posterior de `normal`. No basta con que $p(\mathbf{x} \mid \text{enfermedad})$ sea grande de manera aislada: también importa el prior de cada clase. Por eso la decisión se basa en el producto $p(\mathbf{x} \mid \omega_i)P(\omega_i)$ y no únicamente en la verosimilitud.

Formas equivalentes de la regla de decisión

Usando el teorema de Bayes, la comparación de posteriores puede escribirse como una comparación de productos entre verosimilitud y prior:

$$p(\mathbf{x} \mid \omega_1)P(\omega_1) > p(\mathbf{x} \mid \omega_2)P(\omega_2).$$

Para dos clases, también puede expresarse mediante el cociente de verosimilitudes.

Definición. El cociente de verosimilitudes entre dos clases se define como

$$\ell(\mathbf{x}) = \frac{p(\mathbf{x} \mid \omega_1)}{p(\mathbf{x} \mid \omega_2)} \tag{62.4}$$

La regla de decisión se vuelve

$$\ell(\mathbf{x}) \gtrless_{\omega_2}^{\omega_1} \frac{P(\omega_2)}{P(\omega_1)}.$$

Si se trabaja en escala logarítmica, se evita el subdesbordamiento numérico y se transforman productos en sumas:

$$\log \ell(\mathbf{x}) = \log p(\mathbf{x} \mid \omega_1) - \log p(\mathbf{x} \mid \omega_2).$$

Esta forma es especialmente útil en alta dimensión, en secuencias largas y en modelos donde las densidades son productos de muchas probabilidades pequeñas.

Clasificador Naïve Bayes

En alta dimensión, estimar la densidad conjunta $p(\mathbf{x} \mid \omega_i)$ puede ser difícil. Si $\mathbf{x} = (x_1, \dots, x_d)$, una densidad conjunta general requiere modelar dependencias entre todas las variables. Por ejemplo, en clasificación de textos no basta con preguntar si aparece una palabra: un documento puede contener cientos o miles de términos, y cada término puede aparecer varias veces.

Naïve Bayes simplifica el problema suponiendo independencia condicional de las características dada la clase (McCallum y Nigam 1998; Murphy 2012). La palabra *naïve* no significa que el método sea inútil, sino que usa una suposición fuerte: una vez que conocemos la clase, tratamos a las características como si aportaran evidencia de manera separada.

Definición. El supuesto de independencia condicional de Naïve Bayes establece que

$$p(\mathbf{x} \mid \omega_i) = p(x_1, x_2, \dots, x_d \mid \omega_i) = \prod_{k=1}^{d} p(x_k \mid \omega_i) \tag{62.5}$$

Con este supuesto, la regla MAP se escribe como:

$$\hat{\omega}(\mathbf{x}) = \arg\max_i \left[\log P(\omega_i) + \sum_{k=1}^{d} \log p(x_k \mid \omega_i) \right].$$

Esta expresión tiene una lectura sencilla. El término $\log P(\omega_i)$ representa qué tan probable era la clase antes de observar el documento. La suma $\sum_{k=1}^{d} \log p(x_k \mid \omega_i)$ acumula la evidencia que aportan las características observadas. La clase elegida es aquella que obtiene la puntuación total más alta.

Ejemplo manual con atributos categóricos

Volvamos a los quince días de béisbol del capítulo de árboles de decisión. Queremos decidir si se juega un día **soleado, fresco, con humedad alta y viento fuerte**. Naive Bayes solo necesita contar.

Paso 1. Previas. De 15 días, 9 se jugó y 6 no: $P(\text{Sí}) = 9/15 = 0.6$ y $P(\text{No}) = 6/15 = 0.4$.

Paso 2. Verosimilitudes de cada atributo dentro de cada clase.

Tabla 62.2.: Conteos por clase para el día nuevo.

Atributo = valor	Conteo en «Sí» (9 días)	$P(\cdot \mid \text{Sí})$	Conteo en «No» (6 días)	$P(\cdot \mid \text{No})$
Pronóstico = Soleado	2	2/9	4	4/6
Temperatura = Fresco	3	3/9	2	2/6
Humedad = Alta	3	3/9	5	5/6
Viento = Fuerte	2	2/9	3	3/6

Paso 3. Multiplicar (suponiendo independencia condicional).

$$P(\text{Sí}) \prod_j P(x_j \mid \text{Sí}) = 0.6 \cdot \tfrac{2}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} \cdot \tfrac{2}{9} = 0.6 \cdot 0.00549 = 0.00329,$$
$$P(\text{No}) \prod_j P(x_j \mid \text{No}) = 0.4 \cdot \tfrac{4}{6} \cdot \tfrac{2}{6} \cdot \tfrac{5}{6} \cdot \tfrac{3}{6} = 0.4 \cdot 0.0926 = 0.0370.$$

Paso 4. Normalizar. $P(\text{No} \mid \mathbf{x}) = 0.0370/(0.0370 + 0.00329) = 0.918$. Se predice **No**.

El problema de la frecuencia cero. Si el día nuevo fuera *nublado*, la verosimilitud $P(\text{Nublado} \mid \text{No}) = 0/6$ anularía todo el producto de la clase «No», sin importar lo que digan los demás atributos. El **suavizado de Laplace** suma α (habitualmente 1) a cada conteo:

$$\hat{P}(x_j = v \mid c) = \frac{n_{v,c} + \alpha}{n_c + \alpha\,|V_j|}, \tag{62.6}$$

donde $|V_j|$ es el número de valores posibles del atributo. Con $\alpha = 1$, $P(\text{Nublado} \mid \text{No}) = (0+1)/(6+3) = 0.111$: improbable, pero no imposible.

En la versión web, un laboratorio calcula Naive Bayes con los quince días para cualquier combinación de atributos y suavizado. Con el día soleado, fresco, de humedad alta y viento fuerte y $\alpha = 0$: productos 0.00329 («Sí») y 0.0370 («No»), $P(\text{Sí} \mid \text{día}) = 0.082$. Con un día nublado y $\alpha = 0$, la clase «No» queda con probabilidad cero.

Guía del laboratorio.

1. Comprueba los cuatro pasos del ejemplo manual con $\alpha = 0$.
2. Cambia el pronóstico a *Nublado*. ¿Qué celda de la tabla queda en cero y qué consecuencia tiene? Sube a 1 y compara.
3. Deja la temperatura *sin observar*. ¿Cambia mucho la posterior? ¿Por qué Naive Bayes maneja datos faltantes con tanta naturalidad?
4. En el gráfico de log-odds, ¿qué atributo aporta más evidencia en contra de jugar?

Naïve Bayes multinomial para texto

Una de las versiones más usadas en clasificación de documentos es Naïve Bayes multinomial. En este caso, cada documento se representa como un vector de conteos:

$$\mathbf{x} = (x_1, x_2, \dots, x_d),$$

donde x_k indica cuántas veces aparece la palabra k del vocabulario en el documento. Si θ_{ik} representa la probabilidad de observar la palabra k dentro de documentos de la clase ω_i, entonces el modelo asigna la siguiente puntuación:

$$g_i(\mathbf{x}) = \log P(\omega_i) + \sum_{k=1}^{d} x_k \log \theta_{ik}.$$

La decisión se toma escogiendo la clase con mayor puntuación:

$$\widehat{\omega}(\mathbf{x}) = \arg\max_i g_i(\mathbf{x}).$$

En problemas de texto, las probabilidades θ_{ik} se estiman contando palabras en los documentos de entrenamiento. Si N_{ik} es el número de veces que aparece la palabra k en documentos de la clase ω_i, una estimación directa sería:

$$\theta_{ik} = \frac{N_{ik}}{\sum_{r=1}^{d} N_{ir}}.$$

El problema es que si una palabra nunca aparece en una clase durante el entrenamiento, entonces $N_{ik} = 0$ y su probabilidad estimada sería cero. Como el modelo multiplica probabilidades, una sola palabra con probabilidad cero puede anular toda la puntuación de una clase. Para evitarlo se usa suavizado de Laplace:

$$\theta_{ik} = \frac{N_{ik} + \alpha}{\sum_{r=1}^{d} N_{ir} + \alpha d}.$$

Cuando $\alpha = 1$, se interpreta como si agregáramos una aparición ficticia de cada palabra en cada clase. Esto evita probabilidades cero y vuelve más estable al clasificador.

> **Tip**
>
> En la práctica se trabaja con logaritmos, no con productos directos. Esto evita problemas numéricos cuando se multiplican muchas probabilidades pequeñas y convierte el producto de evidencias en una suma de evidencias.

Ejemplo manual: clasificación de un documento corto

Supongamos que queremos clasificar documentos en dos clases: `spam` y `ham`. Después de entrenar el modelo, podríamos obtener las siguientes probabilidades:

Palabra	$P(\text{palabra} \mid \text{spam})$	$P(\text{palabra} \mid \text{ham})$
free	0.30	0.05
prize	0.25	0.05
project	0.05	0.30

Si los priors son iguales,

$$P(\text{spam}) = P(\text{ham}) = 0.5,$$

y observamos el documento:

$$\mathbf{x} = \text{“free project prize”},$$

entonces las puntuaciones logarítmicas son:

$$g_{\text{spam}}(\mathbf{x}) = \log(0.5) + \log(0.30) + \log(0.05) + \log(0.25),$$

$$g_{\text{ham}}(\mathbf{x}) = \log(0.5) + \log(0.05) + \log(0.30) + \log(0.05).$$

Aunque la palabra `project` favorece a `ham`, las palabras `free` y `prize` favorecen a `spam`. El clasificador suma toda la evidencia y decide por la clase con mayor puntuación final.

Ejemplo en Python: Naïve Bayes para texto

El parámetro `alpha=1.0` introduce suavizado de Laplace. Este suavizado evita que una palabra ausente en el entrenamiento de una clase anule completamente la probabilidad de un documento.

También podemos inspeccionar el vocabulario aprendido y las probabilidades estimadas por clase:

Este segundo bloque muestra de manera explícita qué aprendió el modelo: para cada clase, estima qué palabras son más probables. Por eso Naïve Bayes suele ser fácil de interpretar en texto. Una palabra como `free` debería tener mayor probabilidad bajo `spam`, mientras que una palabra como `meeting` debería tener mayor probabilidad bajo `ham`.

> **!** Importante
>
> Naïve Bayes no necesita que el supuesto de independencia sea perfectamente cierto para clasificar bien. Lo importante es que las puntuaciones relativas separen razonablemente las clases. Por eso puede funcionar bien incluso cuando las palabras de un documento claramente no son independientes entre sí.

Variantes comunes de Naïve Bayes

La misma idea puede adaptarse a distintos tipos de características:

Variante	Tipo de característica	Ejemplo de uso
Bernoulli Naïve Bayes	Presencia o ausencia de una característica	Si una palabra aparece o no aparece
Multinomial Naïve Bayes	Conteos discretos	Frecuencias de palabras en documentos
Gaussian Naïve Bayes	Variables continuas	Mediciones físicas, señales o atributos numéricos

La diferencia entre estas variantes está en cómo se modela $p(x_k \mid \omega_i)$. El principio de decisión sigue siendo el mismo: estimar una probabilidad por clase, sumar la evidencia de las características y elegir la clase con mayor posterior.

Aunque el supuesto de independencia suele ser falso, el clasificador puede funcionar muy bien en tareas de texto, filtrado de spam y clasificación de documentos, porque para decidir no siempre se necesita estimar perfectamente las probabilidades; basta con obtener fronteras de decisión útiles (McCallum y Nigam 1998).

Clasificadores bayesianos gaussianos

Una familia muy importante de clasificadores bayesianos se obtiene cuando las densidades condicionales de clase se modelan mediante distribuciones normales. Este supuesto es razonable cuando las características observadas resultan de la agregación de muchos factores independientes o débilmente

dependientes, de acuerdo con la intuición asociada al teorema central del límite (Wasserman 2004; Bishop 2006).

Definición. Una variable aleatoria continua unidimensional x tiene distribución normal con media μ y varianza σ^2, denotada $\mathcal{N}(\mu, \sigma^2)$, si su densidad es

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right] \tag{62.7}$$

En dimensión d, la distribución normal multivariada queda determinada por un vector de medias μ y una matriz de covarianza Σ.

Definición. Una observación $\mathbf{x} \in \mathbb{R}^d$ sigue una distribución normal multivariada $\mathcal{N}(\mu, \Sigma)$ si

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}-\mu)^T\Sigma^{-1}(\mathbf{x}-\mu)\right] \tag{62.8}$$

Funciones discriminantes gaussianas

Si cada clase ω_i tiene una densidad condicional gaussiana

$$p(\mathbf{x} \mid \omega_i) = \mathcal{N}(\mathbf{x}; \mu_i, \Sigma_i),$$

entonces la decisión MAP puede obtenerse maximizando la función discriminante

$$g_i(\mathbf{x}) = \log p(\mathbf{x} \mid \omega_i) + \log P(\omega_i).$$

Eliminando términos constantes comunes a todas las clases:

$$g_i(\mathbf{x}) = -\frac{1}{2}\log|\Sigma_i| - \frac{1}{2}(\mathbf{x}-\mu_i)^T\Sigma_i^{-1}(\mathbf{x}-\mu_i) + \log P(\omega_i).$$

Cuando las matrices de covarianza son distintas entre clases, la frontera $g_i(\mathbf{x}) = g_j(\mathbf{x})$ es cuadrática. Este caso se conoce como análisis discriminante cuadrático, o QDA (Hastie et al. 2009).

Casos especiales

Covarianza esférica común y priors iguales.

Si $\Sigma_i = \sigma^2\mathbf{I}$ para todas las clases y los priors son iguales, entonces maximizar $g_i(\mathbf{x})$ equivale a minimizar la distancia euclidiana a la media de la clase:

$$\hat{\omega}(\mathbf{x}) = \arg\min_i \|\mathbf{x} - \mu_i\|^2.$$

Este caso corresponde a un clasificador de distancia mínima o template matching.

Covarianza común y priors arbitrarios.

Si $\Sigma_i = \Sigma$ para todas las clases, pero los priors no necesariamente son iguales, la función discriminante se vuelve lineal:

$$g_i(\mathbf{x}) = \mathbf{w}_i^T \mathbf{x} + b_i,$$

con

$$\mathbf{w}_i = \Sigma^{-1}\mu_i, \qquad b_i = -\frac{1}{2}\mu_i^T \Sigma^{-1}\mu_i + \log P(\omega_i).$$

Este caso corresponde al análisis discriminante lineal, o LDA (Fisher 1936; Hastie et al. 2009). El término $\log P(\omega_i)$ desplaza la frontera de decisión hacia la clase con menor prior, porque se requiere más evidencia para asignar una observación a una clase poco frecuente.

Ejemplo en Python: clasificador gaussiano con covarianza común

Este código implementa la regla discriminante lineal sin llamar directamente a una clase de `scikit-learn`. La implementación deja visible la estructura bayesiana: estimación de medias, covarianza común, priors y maximización de $g_i(\mathbf{x})$.

En la versión web, un laboratorio dibuja dos densidades normales ponderadas por sus previas y la frontera de decisión. Con $\mu_1 = -1$, $\mu_2 = 1.5$, $\sigma_1 = \sigma_2 = 1$ y previas iguales, la frontera es el punto medio $x = 0.25$ y el error de Bayes es $\Phi(-1.25) \approx 0.106$. Al aumentar $P(\omega_1)$ la frontera se desplaza hacia μ_2; con varianzas distintas aparecen dos fronteras; con costos asimétricos, la región de la clase más costosa de confundir crece.

Dos gaussianas en 1D: el papel de la previa

El siguiente ejemplo muestra el caso más simple de clasificación gaussiana: una sola variable x y dos clases. Cada clase se modela con una distribución normal distinta:

$$p(x \mid \omega_1) = \mathcal{N}(x; \mu_1, \sigma_1^2), \qquad p(x \mid \omega_2) = \mathcal{N}(x; \mu_2, \sigma_2^2).$$

Para clasificar un valor observado x, no comparamos únicamente las densidades $p(x \mid \omega_i)$. La regla MAP compara las densidades ponderadas por sus probabilidades previas:

$$p(x \mid \omega_1)P(\omega_1) \quad \text{contra} \quad p(x \mid \omega_2)P(\omega_2).$$

La frontera de decisión aparece en los valores de x donde ambas cantidades son iguales. En esos puntos el clasificador está indiferente entre las dos clases:

$$p(x \mid \omega_1)P(\omega_1) = p(x \mid \omega_2)P(\omega_2).$$

El control interactivo modifica $P(\omega_1)$. Cuando aumenta el prior de una clase, esa clase necesita menos evidencia local para ser seleccionada. Por eso la frontera se desplaza hacia la otra clase: se requiere evidencia más fuerte para vencer a la clase que ahora es más probable a priori.

La gráfica muestra tres ideas importantes. Primero, las curvas $p(x \mid \omega_i)P(\omega_i)$ son las cantidades que realmente se comparan para decidir. Segundo, las posteriores $P(\omega_i \mid x)$ cambian suavemente con x: cerca de la media de una clase, su posterior suele aumentar. Tercero, si las varianzas son distintas, puede aparecer más de una frontera de decisión, porque las curvas gaussianas pueden cruzarse en más de un punto.

Clasificación gaussiana en 2D

En dos dimensiones, cada observación ya no es un número sino un vector:

$$\mathbf{x} = (x_1, x_2)^T.$$

Cada clase se describe con tres elementos:

- Un centro μ_i.
- Una matriz de covarianza Σ_i.
- Un prior $P(\omega_i)$.

El centro μ_i indica dónde se concentra la clase. La matriz Σ_i indica la forma de la nube: qué tan dispersa está, si está alargada en alguna dirección y si las variables x_1 y x_2 están correlacionadas. El prior $P(\omega_i)$ desplaza la decisión hacia la clase menos frecuente, porque se necesita más evidencia para asignar una observación a una clase con menor probabilidad previa.

La función discriminante para cada clase es:

$$g_i(\mathbf{x}) = \log p(\mathbf{x} \mid \omega_i) + \log P(\omega_i).$$

La frontera se obtiene donde las dos clases tienen la misma puntuación:

$$g_1(\mathbf{x}) = g_2(\mathbf{x}).$$

Si las covarianzas son iguales, la frontera es lineal. Si las covarianzas son distintas, la frontera puede ser curva, porque cada clase mide la cercanía a su centro con una geometría diferente.

Frontera en 2D

El siguiente ejemplo simula dos nubes gaussianas en el plano. El fondo indica la región donde gana cada clase y la línea de contorno muestra los puntos donde $g_1(\mathbf{x}) - g_2(\mathbf{x}) = 0$. A un lado de esa frontera se decide ω_1 y al otro lado se decide ω_2.

La frontera no depende sólo de qué media esté más cerca en distancia euclidiana. También depende de la orientación y dispersión de cada nube. Por ejemplo, una clase muy dispersa puede asignar probabilidad razonable a puntos alejados de su centro, mientras que una clase muy compacta exige que los puntos estén cerca de su media. Esa diferencia aparece en la forma curva de la frontera.

Interpretación geométrica

La cantidad

$$(\mathbf{x} - \mu_i)^T \Sigma_i^{-1} (\mathbf{x} - \mu_i)$$

es una distancia de Mahalanobis.

Significa que el modelo no mide sólo cercanía euclidiana, sino cercanía relativa a la forma de la nube de datos.

> **Tip**
>
> La covarianza codifica orientación, escala y correlación.

¿Cómo evaluar el supuesto gaussiano?

El supuesto de normalidad rara vez debe aceptarse sin diagnóstico. En aplicaciones reales conviene combinar razonamiento sustantivo, visualización y pruebas estadísticas. La visualización suele ser más informativa que una prueba aislada, porque permite detectar asimetrías, colas pesadas, multimodalidad o valores atípicos.

Algunas herramientas útiles son:

- histogramas y estimaciones de densidad;
- diagramas de caja;
- gráficos Q-Q;
- pruebas de bondad de ajuste, como Kolmogorov–Smirnov, Shapiro–Wilk o pruebas basadas en χ^2.

Una advertencia importante es que las pruebas de normalidad son sensibles al tamaño de muestra. Con muestras grandes pueden detectar desviaciones pequeñas sin relevancia práctica; con muestras pequeñas pueden carecer de potencia. Por ello, el diagnóstico debe vincularse con el objetivo de clasificación.

Clasificación bayesiana de secuencias discretas

Los clasificadores bayesianos no se limitan a variables continuas. También pueden aplicarse a datos discretos estructurados, como texto, secuencias genómicas o series de símbolos. En biología computacional, un ejemplo clásico es la identificación de islas CpG en el genoma humano (Durbin et al. 1998).

Las islas CpG son regiones cortas del ADN donde el dinucleótido CG aparece con mayor frecuencia que en otras regiones. Una definición determinista clásica considera regiones de longitud mayor a 200 pares de bases, contenido $G + C$ superior a 50 % y razón observada/esperada de CpG mayor a 0.60 (Gardiner-Garden y Frommer 1987). Sin embargo, una perspectiva probabilística permite

construir modelos generativos para regiones CpG y no CpG, y decidir con base en la razón de verosimilitudes.

Definición. El contenido $G+C$ de una secuencia de longitud N se define como

$$\%GC = \frac{N(C)+N(G)}{N} \tag{62.9}$$

Definición. La razón CpG observada/esperada puede escribirse como

$$\text{CpG ratio} = \frac{N(CpG)/N}{(N(C)/N)(N(G)/N)} \tag{62.10}$$

Cadenas de Markov de primer orden

Una cadena de Markov de primer orden modela una secuencia suponiendo que el símbolo actual depende sólo del símbolo inmediatamente anterior. Para ADN, el alfabeto es $\mathcal{A} = \{A, C, G, T\}$.

Definición. Una secuencia de variables aleatorias $X_1, X_2, \dots, X_n$ forma una cadena de Markov de primer orden si

$$P(X_n = x_n \mid X_1 = x_1, \dots, X_{n-1} = x_{n-1}) = P(X_n = x_n \mid X_{n-1} = x_{n-1}) \tag{62.11}$$

Las probabilidades de transición se denotan

$$a_{st} = P(X_i = t \mid X_{i-1} = s),$$

donde $s, t \in \mathcal{A}$. Para una secuencia $\mathbf{x} = (x_1, \dots, x_L)$, su probabilidad bajo una cadena de Markov es

$$P(\mathbf{x}) = P(x_1) \prod_{i=2}^{L} a_{x_{i-1}x_i}.$$

Log-odds para discriminar secuencias

Supongamos que se entrenan dos cadenas de Markov: una para islas CpG, denotada $+$, y otra para regiones no CpG, denotada $-$. La puntuación log-odds de una secuencia es

Definición. La puntuación log-odds entre dos modelos de Markov para una secuencia $\mathbf{x}$ es

$$S(\mathbf{x}) = \log \frac{P(\mathbf{x} \mid +)}{P(\mathbf{x} \mid -)} = \sum_{i=2}^{L} \log \frac{a^{+}_{x_{i-1}x_i}}{a^{-}_{x_{i-1}x_i}} \tag{62.12}$$

Si $S(\mathbf{x})$ supera un umbral, la secuencia se clasifica como CpG; si no, como no CpG. El umbral puede fijarse por priors, por costos de error o por validación empírica.

Ejemplo en Python: puntuación de una secuencia de ADN

La normalización por longitud permite comparar secuencias de tamaños distintos. En problemas reales se recomienda estimar las matrices de transición con suavizado, por ejemplo mediante pseudo-conteos, para evitar probabilidades cero.

En la versión web, un laboratorio permite construir secuencias y acumular los log-odds de transición. La transición C→G aporta $\ln(0.274/0.078) = +1.26$; la secuencia ACGCGCGTACGCGT obtiene una puntuación positiva y se clasifica como isla CpG.

Clasificadores bayesianos de riesgo mínimo

La decisión de error mínimo supone que todos los errores cuestan lo mismo. Sin embargo, en muchas aplicaciones esto no es cierto. En diagnóstico médico, por ejemplo, un falso negativo puede ser mucho más costoso que un falso positivo. En detección de fraude, bloquear una transacción legítima y dejar pasar una transacción fraudulenta tampoco tienen el mismo costo. Por ello se introduce una función de pérdida.

Definición. La pérdida $\lambda(\alpha_i, \omega_j)$ representa el costo de tomar la decisión α_i cuando el verdadero estado de la naturaleza es ω_j:

$$\lambda_{ij} = \lambda(\alpha_i, \omega_j) \tag{62.13}$$

Dada una observación $\mathbf{x}$, el riesgo condicional de tomar la decisión α_i es la pérdida esperada bajo la distribución posterior de las clases.

Definición. El riesgo condicional de decidir α_i dado $\mathbf{x}$ es

$$R(\alpha_i \mid \mathbf{x}) = \sum_{j=1}^{c} \lambda(\alpha_i, \omega_j) P(\omega_j \mid \mathbf{x}) \tag{62.14}$$

La regla de riesgo mínimo selecciona la decisión con menor riesgo condicional:

$$\alpha^*(\mathbf{x}) = \arg\min_i R(\alpha_i \mid \mathbf{x}).$$

Cuando $\lambda_{ii} = 0$ y $\lambda_{ij} = 1$ para $i \neq j$, la regla de riesgo mínimo se reduce a la regla de error mínimo. Es decir, la clasificación MAP es un caso particular de la teoría de decisión bayesiana (Berger 1985; Duda et al. 2001).

Dos clases con matriz de pérdidas

Para dos clases y dos decisiones, la matriz de pérdidas puede escribirse como

	ω_1	ω_2
α_1	λ_{11}	λ_{12}
α_2	λ_{21}	λ_{22}

La decisión α_1 se toma si

$$\lambda_{11}P(\omega_1 \mid \mathbf{x}) + \lambda_{12}P(\omega_2 \mid \mathbf{x}) < \lambda_{21}P(\omega_1 \mid \mathbf{x}) + \lambda_{22}P(\omega_2 \mid \mathbf{x}).$$

De manera equivalente, el umbral del cociente de verosimilitudes se modifica por los costos:

$$\ell(\mathbf{x}) \gtrless_{\alpha_2}^{\alpha_1} \frac{P(\omega_2)}{P(\omega_1)} \cdot \frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}}.$$

Ejemplo en Python: error mínimo frente a riesgo mínimo

El resultado muestra que la decisión de error mínimo puede ser “normal”, mientras que la decisión de riesgo mínimo puede ser “enfermedad” si el costo de no detectar la enfermedad es suficientemente alto. Esta diferencia es fundamental en sistemas de decisión sensibles al costo.

En la versión web, un laboratorio grafica los riesgos condicionales de cada acción. Con $\lambda_{12} = 6$ y $\lambda_{21} = 1$, el umbral es $p^* = 1/7 \approx 0.143$: un paciente con $P(\omega_2 \mid x) = 0.3$ se diagnostica como enfermo, aunque la regla de error mínimo lo declararía normal.

Minimizar un tipo de error condicionado al otro

En algunos problemas no se busca minimizar una suma ponderada de errores, sino controlar explícitamente un tipo de error mientras se minimiza otro. Esta idea aparece en pruebas de hipótesis, detección de señales y clasificación con restricciones operativas. El caso clásico es minimizar la probabilidad de falso negativo sujeto a una cota sobre la probabilidad de falso positivo, o viceversa. La regla resultante se relaciona con el lema de Neyman–Pearson (Neyman y Pearson 1933).

Para dos clases, sea $\mathcal{R}_1$ la región donde se decide ω_1 y $\mathcal{R}_2$ la región donde se decide ω_2. Las probabilidades de error condicionales pueden escribirse como

$$P_1(e) = \int_{\mathcal{R}_2} p(\mathbf{x} \mid \omega_1)d\mathbf{x}, \qquad P_2(e) = \int_{\mathcal{R}_1} p(\mathbf{x} \mid \omega_2)d\mathbf{x}.$$

Si se desea minimizar $P_1(e)$ sujeto a $P_2(e) = \epsilon$, se obtiene una regla basada en umbrales del cociente de verosimilitudes:

$$\frac{p(\mathbf{x} \mid \omega_1)}{p(\mathbf{x} \mid \omega_2)} \gtrless \lambda.$$

El valor de λ se ajusta para cumplir la restricción sobre el error. En términos prácticos, esto equivale a desplazar el umbral de decisión hasta alcanzar una sensibilidad, especificidad, tasa de falsos positivos o tasa de falsos negativos deseada.

Ejemplo en Python: elegir umbral por restricción de falsos positivos

Este enfoque es más adecuado que maximizar accuracy cuando una clase es rara o cuando los errores tienen consecuencias asimétricas.

Criterio minimax cuando los priors son desconocidos

La regla bayesiana clásica requiere conocer o estimar los priors $P(\omega_i)$. Cuando estos priors son inciertos, una alternativa conservadora es el criterio minimax. La idea es escoger la regla de decisión que minimiza el peor riesgo posible bajo los priors admisibles (Berger 1985; Duda et al. 2001).

> Definición. Una regla minimax selecciona la decisión o regla de decisión α que minimiza el máximo riesgo posible:
>
> $$\alpha_{\text{minimax}} = \arg\min_{\alpha} \max_{\pi \in \Pi} R(\alpha; \pi) \tag{62.15}$$

Aquí π representa el vector de priors y Π el conjunto de priors considerados plausibles. En problemas de dos clases, puede estudiarse el riesgo como función de $P(\omega_1)$, ya que $P(\omega_2) = 1 - P(\omega_1)$. La regla minimax evita depender de una estimación puntual del prior y protege contra escenarios adversos. Su desventaja es que puede ser demasiado conservadora si sí existe información confiable sobre las frecuencias reales de clase.

Recomendaciones prácticas

La clasificación bayesiana es conceptualmente elegante, pero su desempeño depende de la calidad del modelo probabilístico. Algunas recomendaciones prácticas son:

1. Estimar priors con cuidado. En datos desbalanceados, los priors empíricos pueden reflejar sesgos de muestreo y no prevalencias reales.
2. Trabajar en escala logarítmica para evitar problemas numéricos.
3. Validar supuestos distribucionales cuando se usan modelos gaussianos.
4. Usar suavizado en modelos discretos, especialmente en texto y secuencias.
5. Elegir la métrica o función de pérdida de acuerdo con el problema real, no sólo con accuracy.
6. Separar conceptualmente probabilidad, decisión y costo: una buena estimación probabilística no implica automáticamente una buena regla de decisión si el umbral no corresponde al costo operativo.

Conclusiones

Los clasificadores bayesianos proporcionan un marco unificado para tomar decisiones bajo incertidumbre. La regla MAP minimiza la probabilidad de error cuando los errores tienen el mismo costo, mientras que la regla de riesgo mínimo permite incorporar pérdidas asimétricas. Naïve Bayes muestra cómo construir clasificadores prácticos en alta dimensión mediante una suposición de independencia condicional. Los clasificadores gaussianos muestran cómo los supuestos sobre las densidades condicionales determinan la geometría de las fronteras de decisión: fronteras lineales bajo covarianza común

y fronteras cuadráticas bajo covarianzas distintas. Por otro lado, los modelos de Markov muestran que el mismo principio puede aplicarse a secuencias discretas con dependencia entre símbolos.

La idea central del capítulo es que clasificar no consiste únicamente en trazar una frontera, sino en articular tres elementos: un modelo de cómo se generan los datos, una actualización probabilística de las creencias y una regla de decisión adaptada al objetivo. Esta separación hace que los métodos bayesianos sean especialmente útiles en problemas donde la incertidumbre, los costos de error y la interpretabilidad son componentes esenciales del sistema de aprendizaje.

63. Modelos de Mezcla Gaussiana

Los **modelos de mezcla gaussiana** (GMM, por sus siglas en inglés) son modelos probabilísticos generativos que representan la distribución de los datos como una suma ponderada de distribuciones normales, lo que los hace ideales para capturar estructuras multimodales que una sola gaussiana no puede describir (Bishop 2006; Murphy 2012). El algoritmo de Expectation-Maximization (EM), introducido formalmente por Dempster et al. (1977), proporciona el marco de optimización para estimar los parámetros del modelo a partir de datos incompletos (en este caso, los datos están "incompletos" porque la asignación de cada observación a cada componente es desconocida). Los GMM pueden entenderse como una generalización probabilística del algoritmo K-Means: cuando las covarianzas son esféricas e iguales y se toma el límite de varianza cero, las asignaciones blandas del GMM convergen a las asignaciones duras de K-Means (Hastie et al. 2009). En este capítulo se desarrolla la teoría completa del modelo, el algoritmo EM desde cero, y todos los ejemplos computacionales se implementan exclusivamente con `numpy` y `plotly`.

Motivación: ¿por qué un solo gaussiano no basta?

La distribución normal multivariada es, sin duda, el modelo probabilístico más utilizado en aprendizaje automático. Su elegancia matemática y sus propiedades analíticas la hacen atractiva, pero su aplicabilidad está fundamentalmente limitada a datos **unimodales**: datos cuya densidad tiene un único máximo y una forma aproximadamente elipsoidal. Cuando los datos provienen de varias poblaciones distintas, o cuando la verdadera densidad tiene múltiples crestas, un único gaussiano ajustado a todos los datos producirá una descripción pobre que coloca masa de probabilidad en regiones vacías y asigna poca densidad a las regiones donde realmente se concentran las observaciones.

Considérese un conjunto de datos unidimensional formado por dos grupos bien separados. La figura Figura 63.1 ilustra el contraste entre un único gaussiano ajustado a todos los datos (panel izquierdo) y una mezcla de dos gaussianas (panel derecho). El modelo unimodal coloca la mayor parte de su masa en el valle entre los dos grupos, mientras que la mezcla captura fielmente la estructura bimodal. Este ejemplo simple motiva la necesidad de modelos más flexibles que puedan representar distribuciones multimodales de manera principiada.

Figura 63.1.

El modelo de mezcla gaussiana

Definición formal

Un modelo de mezcla gaussiana con K componentes define la siguiente densidad de probabilidad sobre $\mathbf{x} \in \mathbb{R}^D$:

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) \tag{63.1}$$

donde $\mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$ es la densidad gaussiana multivariada con media μ_k y matriz de covarianza Σ_k:

$$\mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2} |\Sigma_k|^{1/2}} \exp\!\left(-\frac{1}{2} (\mathbf{x} - \mu_k)^\top \Sigma_k^{-1} (\mathbf{x} - \mu_k) \right) \tag{63.2}$$

Los **coeficientes de mezcla** π_k satisfacen las restricciones $\pi_k \geq 0$ para todo k y $\sum_{k=1}^{K} \pi_k = 1$, de modo que pueden interpretarse como probabilidades a priori sobre las componentes: $\pi_k = p(z = k)$.

Variable latente y verosimilitud completa

La formulación con variable latente es fundamental para entender el algoritmo EM. Se introduce una variable aleatoria discreta $\mathbf{z} \in \{1, \dots, K\}$ (o equivalentemente, un vector one-hot $\mathbf{z} \in \{0,1\}^K$ con $\sum_k z_k = 1$) que indica cuál componente generó la observación $\mathbf{x}$. La distribución conjunta del modelo generativo es:

$$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})\, p(\mathbf{x} \mid \mathbf{z}) = \prod_{k=1}^{K} \left[\pi_k \, \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k) \right]^{z_k} \tag{63.3}$$

La densidad marginal Ecuación 63.1 se obtiene sumando (integrando) sobre todos los posibles valores de $\mathbf{z}$:

$$p(\mathbf{x}) = \sum_{k=1}^{K} p(z = k)\, p(\mathbf{x} \mid z = k) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$

Para un conjunto de N observaciones i.i.d. $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, la **log-verosimilitud observada** es:

$$\ell(\theta) = \sum_{i=1}^{N} \log \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_i \mid \mu_k, \Sigma_k) \tag{63.4}$$

donde $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$ son todos los parámetros del modelo. El logaritmo de una suma no tiene solución analítica en forma cerrada, lo que motiva el uso del algoritmo EM.

Densidad de una mezcla en 2D

La figura Figura **??** muestra la densidad de una mezcla con $K = 3$ componentes en $\mathbb{R}^2$. Se observa cómo cada componente contribuye una "joroba" en la densidad total, y la mezcla puede representar distribuciones con formas geométricas arbitrariamente complejas.

El algoritmo EM

Principio general: ascenso coordinado sobre el ELBO

El algoritmo EM (Expectation-Maximization) es un procedimiento iterativo para encontrar estimadores de máxima verosimilitud en modelos con variables latentes (Dempster et al. 1977; McLachlan y Peel 2000). La dificultad central es que la log-verosimilitud observada Ecuación 63.4 contiene el logaritmo de una suma, que no tiene solución analítica. La clave del EM es introducir una cota inferior (ELBO, *Evidence Lower BOund*) que sí es maximizable en forma cerrada.

Para cualquier distribución $q(\mathbf{z})$ sobre la variable latente, la desigualdad de Jensen garantiza:

$$\log p(\mathbf{x} \mid \theta) \geq \mathbb{E}_q\left[\log \frac{p(\mathbf{x}, \mathbf{z} \mid \theta)}{q(\mathbf{z})}\right] \equiv \mathcal{L}(q, \theta) \tag{63.5}$$

El EM alterna entre dos pasos que maximizan este funcional de manera coordinada.

Paso E: cálculo de responsabilidades

El **paso E** fija los parámetros $\theta^{(t)}$ y maximiza $\mathcal{L}$ con respecto a $q(\mathbf{z})$. La solución óptima es la distribución posterior verdadera $q^*(\mathbf{z}) = p(\mathbf{z} \mid \mathbf{x}, \theta^{(t)})$. Para el GMM, esta posterior factoriza y la **responsabilidad** de la componente k sobre la observación $\mathbf{x}_i$ es:

$$r_{ik} = p(z_i = k \mid \mathbf{x}_i, \theta^{(t)}) = \frac{\pi_k^{(t)}\, \mathcal{N}(\mathbf{x}_i \mid \mu_k^{(t)}, \Sigma_k^{(t)})}{\displaystyle\sum_{j=1}^{K} \pi_j^{(t)}\, \mathcal{N}(\mathbf{x}_i \mid \mu_j^{(t)}, \Sigma_j^{(t)})} \tag{63.6}$$

Nótese que $r_{ik} \geq 0$ y $\sum_k r_{ik} = 1$ para todo i; las responsabilidades son asignaciones **blandas** (probabilísticas) de cada punto a cada componente.

Paso M: actualización de parámetros

El **paso M** fija $q(\mathbf{z})$ y maximiza $\mathcal{L}$ con respecto a θ. Definiendo $N_k = \sum_{i=1}^{N} r_{ik}$ (número efectivo de puntos en la componente k), las actualizaciones en forma cerrada son:

$$\pi_k^{(t+1)} = \frac{N_k}{N} \tag{63.7}$$

$$\mu_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^{N} r_{ik}\, \mathbf{x}_i \tag{63.8}$$

$$\Sigma_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^{N} r_{ik}\, (\mathbf{x}_i - \mu_k^{(t+1)})(\mathbf{x}_i - \mu_k^{(t+1)})^{\top} \tag{63.9}$$

EM a mano: cinco puntos en una recta

Cinco observaciones, $x = (1, 2, 4, 6, 7)$, y una mezcla de dos componentes inicializada con $\mu_1 = 2$, $\mu_2 = 6$, $\sigma_1^2 = \sigma_2^2 = 1$ y $\pi_1 = \pi_2 = 0.5$. La log-verosimilitud inicial es $\ell = -10.367$.

Paso E. Para cada punto, la responsabilidad de la componente 1 compara las dos densidades ponderadas. Para $x = 2$: $\mathcal{N}(2 \mid 2, 1) = 0.3989$ y $\mathcal{N}(2 \mid 6, 1) = e^{-8}/\sqrt{2\pi} = 0.000134$, así que $r_{21} = 0.3989/(0.3989 + 0.000134) = 0.9997$. El punto $x = 4$ está exactamente a la misma distancia de ambas medias: $r_{31} = 0.5$.

Tabla 63.1.: Responsabilidades del primer paso E.

x	1	2	4	6	7	$N_k = \sum_i r_{ik}$
r_{i1}	1.0000	0.9997	0.5000	0.0003	0.0000	2.5
r_{i2}	0.0000	0.0003	0.5000	0.9997	1.0000	2.5

Paso M. Cada componente se reajusta como un promedio **ponderado por las responsabilidades**:

$$\mu_1 = \frac{1(1) + 0.9997(2) + 0.5(4) + 0.0003(6) + 0(7)}{2.5} = 2.0006, \qquad \sigma_1^2 = \frac{\sum_i r_{i1}(x_i - \mu_1)^2}{2.5} = 1.202, \qquad \pi_1 = \frac{2.5}{5} = 0$$

y simétricamente $\mu_2 = 5.9994$ y $\sigma_2^2 = 1.202$. El punto ambiguo $x = 4$ contribuye **la mitad** a cada componente y ensancha ambas varianzas. La log-verosimilitud sube a -10.320 y, en las iteraciones siguientes, apenas cambia: EM convergió en dos pasos a $\mu \approx (2.002, 5.998)$ y $\sigma^2 \approx 1.209$. A diferencia de k-medias, que habría asignado el punto 4 por completo a uno de los grupos, el GMM reconoce explícitamente su ambigüedad.

Convergencia y propiedad monótona

Una propiedad fundamental del EM es que la log-verosimilitud observada $\ell(\theta^{(t)})$ es **monótonamente no decreciente** en cada iteración: $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$. Esto se demuestra formalmente usando la desigualdad de Jensen y el hecho de que ambos pasos maximizan el ELBO. En la práctica, el algoritmo se detiene cuando el incremento en log-verosimilitud cae por debajo de una tolerancia ε (por ejemplo, 10^{-6}).

Implementación desde cero en numpy

El siguiente bloque implementa el algoritmo EM completo, incluyendo un truco numérico para evitar desbordamiento: se calcula el logaritmo de las densidades y se aplica la sustracción del máximo antes de exponenciar (equivalente al truco log-sum-exp).

Curva de log-verosimilitud

La figura Figura **??** confirma la propiedad de convergencia monótona del EM: la log-verosimilitud crece de manera no decreciente en cada iteración hasta alcanzar una meseta, señal de convergencia al óptimo local.

Laboratorio: EM paso a paso

El laboratorio alterna los pasos E y M sobre datos unidimensionales. Cada punto se colorea mezclando el azul y el naranja según su responsabilidad; las curvas son las componentes ponderadas $\pi_k \mathcal{N}(x \mid \mu_k, \sigma_k^2)$ y su suma. El estado inicial reproduce el ejemplo a mano.

En la versión web, un laboratorio alterna los pasos E y M. Con los cinco puntos y el inicio $\mu = (2, 6)$, $\sigma^2 = (1, 1)$: tras el primer paso E, $r(x = 4) = 0.5$; tras el primer paso M, $\mu = (2.001, 5.999)$, $\sigma^2 = (1.202, 1.202)$ y ℓ sube de -10.367 a -10.320.

Guía del laboratorio.

1. Con *Cinco puntos*, da un paso E y comprueba la Tabla 63.1; después un paso M y comprueba μ y σ^2.
2. Predice antes de mover: ¿cuántas iteraciones más hacen falta para converger?
3. Cambia a *Mezcla* y pulsa *Hasta converger*. ¿Recupera las dos componentes (medias cercanas a 3 y 8, pesos 0.625 y 0.375)?
4. Usa *Inicio aleatorio* varias veces. ¿Siempre llega a la misma solución? ¿Qué ocurre si una componente empieza lejos de todos los datos?

Tipos de covarianza

La flexibilidad de un GMM depende en gran medida de la estructura impuesta a las matrices de covarianza Σ_k. Restringir la forma de Σ_k reduce el número de parámetros a estimar, lo que puede mejorar la generalización cuando los datos son escasos, pero a costa de capacidad expresiva (Murphy 2012; Hastie et al. 2009). La siguiente tabla resume las cuatro estructuras más comunes en la práctica:

Tipo	Descripción	Parámetros por componente	Forma del contorno
Esférica (*spherical*)	$\Sigma_k = \sigma_k^2 \mathbf{I}$	1	Círculos
Diagonal (*diagonal*)	$\Sigma_k = \text{diag}(\sigma_{k1}^2, \dots, \sigma_{kD}^2)$	D	Elipses alineadas con los ejes
Completa (*full*)	Σ_k simétrica definida positiva	$D(D+1)/2$	Elipses rotadas arbitrariamente
Atada (*tied*)	$\Sigma_k = \Sigma$ (igual para todo k)	$D(D+1)/2$ en total	Elipses con la misma forma

Para un GMM con $D = 2$ y $K = 3$: la covarianza esférica usa solo 3 parámetros de covarianza, la diagonal usa 6, la completa usa 9, y la atada usa 3 (compartidos). La figura Figura **??** ilustra el efecto visual de cada estructura.

Selección del número de componentes

El problema del sobreajuste

Una limitación fundamental del GMM es que el número de componentes K debe especificarse de antemano. Aumentar K siempre incrementa la log-verosimilitud en los datos de entrenamiento (mayor flexibilidad), pero puede llevar a sobreajuste: el modelo captura ruido particular de la muestra en lugar de la estructura real de la distribución subyacente (Bishop 2006; Reynolds 2009). Un caso extremo es que con $K = N$, cada componente puede colapsar sobre un solo punto, llevando la log-verosimilitud al infinito pero sin poder generalizar.

Criterios de información: AIC y BIC

Los criterios de información penalizan la log-verosimilitud por el número de parámetros libres p, buscando un equilibrio entre ajuste y parsimonia:

$$\text{AIC} = -2\,\ell(\hat{\theta}) + 2p \tag{63.10}$$

$$\text{BIC} = -2\,\ell(\hat{\theta}) + p \log N \tag{63.11}$$

Para un GMM con covarianza completa en D dimensiones y K componentes, el número de parámetros libres es:

$$p = \underbrace{K \cdot D}_{\text{medias}} + \underbrace{K \cdot \frac{D(D+1)}{2}}_{\text{covarianzas}} + \underbrace{(K-1)}_{\text{mezclas}} \tag{63.12}$$

El BIC penaliza más fuertemente que el AIC cuando $N > e^2 \approx 7$, tendiendo a seleccionar modelos más parsimoniosos. Se elige el K que minimiza el criterio. La figura Figura **??** muestra la curva de AIC y BIC para $K = 1, \ldots, 8$.

GMM como clasificador vs. K-Means

K-Means como caso límite del GMM

El algoritmo K-Means puede derivarse formalmente como un caso límite del GMM bajo las siguientes restricciones: (1) coeficientes de mezcla iguales $\pi_k = 1/K$ para todo k, (2) covarianzas esféricas e iguales $\Sigma_k = \sigma^2 \mathbf{I}$ para todo k, y (3) se toma el límite $\sigma^2 \to 0$ (Bishop 2006; Hastie et al. 2009). En este límite, las responsabilidades r_{ik} colapsan a asignaciones duras: $r_{ik} \to 1$ para el componente más cercano y $r_{ik} \to 0$ para todos los demás. La actualización de medias del paso M del GMM se convierte exactamente en la actualización de centroides de K-Means.

La diferencia fundamental es que el GMM produce **asignaciones blandas** (probabilísticas) mediante las responsabilidades r_{ik}, mientras que K-Means produce **asignaciones duras** mediante la regla del vecino más cercano. Para clasificación, la regla de decisión del GMM asigna $\mathbf{x}$ a la clase $k^* = \arg\max_k r_{ik}$, pero también cuantifica la incertidumbre mediante la distribución completa $(r_{i1}, \ldots, r_{iK})$.

La figura Figura **??** ilustra la diferencia entre las asignaciones duras de K-Means y las asignaciones blandas del GMM sobre el mismo conjunto de datos.

GMM como modelo generativo

Una ventaja clave del GMM sobre los métodos de clustering no probabilísticos es que define una **distribución de probabilidad completa** sobre el espacio de datos. Esto significa que, una vez ajustado el modelo, es posible generar nuevas observaciones que son estadísticamente indistinguibles de los datos originales, siguiendo el procedimiento ancestral del modelo generativo (Bishop 2006; Murphy 2012).

El procedimiento de muestreo es directo y se realiza en dos etapas:

1. **Muestrear la componente**: extraer $k \sim \text{Categórica}(\pi_1, \ldots, \pi_K)$, es decir, seleccionar la componente k con probabilidad π_k.
2. **Muestrear la observación**: extraer $\mathbf{x} \sim \mathcal{N}(\mu_k, \Sigma_k)$ de la gaussiana correspondiente.

Este proceso genera muestras de la densidad marginal $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$, lo que confirma que el GMM es un modelo generativo válido. La figura Figura **??** compara los datos originales con muestras nuevas generadas del modelo ajustado.

Consideraciones prácticas

Inicialización

La calidad de la solución encontrada por el EM depende sensiblemente de la inicialización, ya que el algoritmo solo garantiza convergencia a un óptimo **local** de la log-verosimilitud. Existen dos estrategias principales:

- **Inicialización aleatoria**: seleccionar K observaciones al azar como medias iniciales y usar $\Sigma_k = \mathbf{I}$ y $\pi_k = 1/K$. Es simple pero puede conducir a mínimos locales pobres.
- **Arranque en caliente con K-Means**: ejecutar K-Means primero y usar los centroides resultantes como medias iniciales, con las covarianzas empíricas de cada cluster y proporciones iguales. Esta estrategia tiende a producir mejores puntos de partida y convergencia más rápida (Bishop 2006).

En la práctica se recomienda ejecutar el EM múltiples veces con diferentes inicializaciones y quedarse con la solución de mayor log-verosimilitud.

Singularidades y regularización

El GMM tiene un problema de **singularidades**: si una componente colapsa sobre un único punto de datos, su covarianza tiende a cero ($|\Sigma_k| \to 0$) y la densidad gaussiana en ese punto tiende al infinito, llevando la log-verosimilitud a $+\infty$. Este no es un máximo legítimo del modelo, sino una degeneración numérica. La solución estándar es añadir una regularización $\varepsilon\mathbf{I}$ a la covarianza estimada en cada paso M:

$$\hat{\Sigma}_k \leftarrow \hat{\Sigma}_k + \varepsilon\mathbf{I}$$

con $\varepsilon \approx 10^{-6}$ típicamente. Esto garantiza que todas las matrices de covarianza sean definidas positivas y evita la degeneración numérica.

Óptimos locales y múltiples reinicios

El EM es un algoritmo de ascenso de gradiente generalizado que no garantiza convergencia al óptimo global. La función de log-verosimilitud del GMM tiene en general múltiples óptimos locales, y el EM puede quedar atrapado en cualquiera de ellos dependiendo de la inicialización (McLachlan y Peel 2000). La estrategia estándar es ejecutar el algoritmo R veces (por ejemplo, $R = 10$) con diferentes semillas aleatorias y seleccionar el modelo con la mayor log-verosimilitud al converger.

Costo computacional

El costo por iteración del EM para un GMM con covarianza completa es $\mathcal{O}(K \cdot N \cdot D^2)$: para cada componente ($K$ términos) y cada observación (N términos) se evalúa la densidad gaussiana, que requiere $\mathcal{O}(D^2)$ operaciones (producto cuadrático con la inversa de Σ_k). La actualización de Σ_k en el paso M también cuesta $\mathcal{O}(N \cdot D^2)$ por componente. Para aplicaciones con D muy grande o K grande, las covarianzas diagonales o esféricas reducen el costo a $\mathcal{O}(K \cdot N \cdot D)$.

Variational Bayes GMM para selección automática de K

Una extensión bayesiana natural del GMM es el **Variational Bayes GMM** (VB-GMM), que coloca distribuciones a priori sobre todos los parámetros (π_k, μ_k, Σ_k) y realiza inferencia variacional en lugar de estimación puntual (Bishop 2006). Una ventaja clave del VB-GMM es que puede determinar automáticamente el número efectivo de componentes: componentes superfluas reciben pesos $\pi_k \to 0$ durante la inferencia, evitando la necesidad de ejecutar el modelo para múltiples valores de K y comparar con AIC/BIC. Sin embargo, la implementación es considerablemente más compleja y queda fuera del alcance de este capítulo.

Parte XXXII.

Redes Neuronales

64. Redes Neuronales para Clasificación

Las redes neuronales artificiales son modelos computacionales inspirados en la estructura del sistema nervioso biológico. Desde el perceptrón de Rosenblatt en 1958 (Rosenblatt 1958) hasta las arquitecturas profundas que dominan el estado del arte actual (LeCun et al. 2015), el esqueleto matemático fundamental permanece invariante: una composición de transformaciones lineales seguidas de no linealidades, aplicadas de manera jerárquica sobre los datos de entrada. Esta arquitectura modular permite que el modelo aprenda representaciones cada vez más abstractas de los datos, pasando de características de bajo nivel (como bordes en una imagen) hasta conceptos de alto nivel (como la clase de un objeto).

El interés renovado por las redes neuronales en la última década no es accidental. Se sustenta en tres pilares convergentes: la disponibilidad de grandes volúmenes de datos etiquetados, la capacidad de cómputo paralelo ofrecida por las unidades de procesamiento gráfico (GPU), y avances teóricos y prácticos en el entrenamiento de redes profundas. El teorema de aproximación universal (Hornik et al. 1989) formaliza por que estas arquitecturas son tan poderosas: una red de una sola capa oculta con suficientes unidades puede aproximar cualquier función continua sobre un conjunto compacto con precisión arbitraria. Este capitulo desarrolla la teoría desde el perceptrón hasta las redes convolucionales, construyendo cada componente desde primeros principios con implementaciones en Python puro y visualizaciones interactivas.

El Perceptrón

Modelo matemático

El perceptrón es la unidad de cómputo más elemental de las redes neuronales. Recibe d entradas escalares $x_1, \dots, x_d$ y computa una combinación lineal ponderada seguida de una función de activación binaria. Formalmente, dados un vector de pesos $\mathbf{w} \in \mathbb{R}^d$ y un sesgo $b \in \mathbb{R}$, el perceptrón calcula:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{d} w_i x_i + b \tag{64.1}$$

$$\hat{y} = \operatorname{sign}(z) \in \{-1, +1\} \tag{64.2}$$

La interpretación geométrica es inmediata: el conjunto $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ define un hiperplano en $\mathbb{R}^d$ que divide el espacio en dos semiespacios. Los puntos con $\mathbf{w}^\top \mathbf{x} + b > 0$ se clasifican como clase $+1$ y los puntos con $\mathbf{w}^\top \mathbf{x} + b < 0$ como clase -1. El vector $\mathbf{w}$ es el vector normal al hiperplano, y su magnitud $\|\mathbf{w}\|$ determina la escala del margen de decisión. El sesgo b desplaza el hiperplano respecto al origen, permitiendo que la frontera de decisión no pase necesariamente por el punto cero.

Esta representación es exactamente la misma que la de las Máquinas de Soporte Vectorial, con la diferencia fundamental de que el perceptrón no busca el hiperplano de margen máximo: simplemente busca cualquier hiperplano que clasifique correctamente todos los ejemplos de entrenamiento. Esta distinción es crucial y explica por que el perceptrón puede encontrar diferentes soluciones dependiendo del orden en que se presenten los datos, mientras que la SVM produce una solución única y óptima en el sentido del margen.

Regla de aprendizaje del perceptrón

El algoritmo de entrenamiento del perceptrón es sorprendentemente simple. Se procesan los ejemplos de entrenamiento de forma iterativa y, cuando el modelo comete un error de clasificación en el ejemplo $(\mathbf{x}^{(i)}, y^{(i)})$, se actualiza el vector de pesos en la dirección que corrige ese error:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\, y^{(i)}\mathbf{x}^{(i)}, \quad b \leftarrow b + \eta\, y^{(i)} \tag{64.3}$$

donde $\eta > 0$ es la tasa de aprendizaje. La lógica de esta regla es directa: si se predijo $\hat{y} = -1$ cuando el verdadero valor es $y^{(i)} = +1$ (falso negativo), entonces $\mathbf{w}$ se desplaza en la dirección de $\mathbf{x}^{(i)}$, lo que aumenta $\mathbf{w}^\top \mathbf{x}^{(i)}$ y acerca la predicción al valor correcto. Si se predijo $\hat{y} = +1$ cuando el verdadero es $y^{(i)} = -1$ (falso positivo), el desplazamiento es en la dirección opuesta a $\mathbf{x}^{(i)}$.

El teorema de convergencia del perceptrón establece que si los datos son linealmente separables con margen geométrico $\gamma > 0$ (es decir, existe un hiperplano que clasifica todos los puntos correctamente con distancia mínima γ a cualquier punto), entonces el algoritmo converge en a lo sumo R^2/γ^2 pasos, donde $R = \max_i \|\mathbf{x}^{(i)}\|$ es el radio de los datos (Rosenblatt 1958). Si los datos no son linealmente separables, el algoritmo no converge: oscila indefinidamente. Este es el límite fundamental del perceptrón simple que motivo el desarrollo de las redes multicapa.

Figura 64.1.

La limitación del perceptrón: el problema XOR

Aunque el perceptrón es elegante en su simplicidad, tiene una limitación fundamental: solo puede separar clases que son linealmente separables. En 1969, Minsky y Papert demostraron en su libro *Perceptrons* que la función lógica XOR (o exclusivo) no puede ser implementada por un perceptrón simple. Esta constatación genero un período de pesimismo en la investigación sobre redes neuronales conocido como el "invierno del IA", hasta que el desarrollo del algoritmo de retropropagación en los anos ochenta revivio el campo al demostrar que las redes multicapa podían aprender representaciones no lineales.

Funciones de Activación

Las funciones de activación son el ingrediente esencial que dota a las redes neuronales de su capacidad expresiva. Sin ellas, cualquier composición de capas lineales colapsaría a una única transformación lineal: si $f(\mathbf{x}) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2$, esto es simplemente $(\mathbf{W}_2\mathbf{W}_1)\mathbf{x}+\mathbf{W}_2\mathbf{b}_1+\mathbf{b}_2$, una transformación

lineal con una sola capa equivalente. La no linealidad introducida por la función de activación permite que las capas aprendan representaciones jerárquicas genuinamente no lineales.

La elección de la función de activación afecta tanto la expresividad del modelo como la estabilidad del entrenamiento. Las funciones clásicas como la sigmoide y la tangente hiperbólica fueron populares en las primeras decadas, pero sufren del problema del gradiente desvaneciente en redes profundas. Las funciones rectificadoras (ReLU y sus variantes) han dominado la práctica moderna precisamente por mitigar este problema. Las cinco funciones principales se definen y comparan a continuación.

La **sigmoide** $\sigma(z) = \frac{1}{1+e^{-z}}$ mapea cualquier valor real al intervalo $(0, 1)$ y se usa típicamente en la capa de salida para clasificación binaria. Su derivada es $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$, lo que significa que el gradiente nunca supera 0.25 en ninguna unidad. La **tangente hiperbólica** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ produce salidas en $(-1, 1)$ y tiene derivada $1 - \tanh^2(z) \leq 1$, con gradientes maximos en $z = 0$. La **ReLU** $\text{ReLU}(z) = \max(0, z)$ tiene derivada 1 para $z > 0$ y 0 para $z < 0$, lo que elimina el problema de gradientes exponencialmente pequeños en la parte positiva. La **Leaky ReLU** $\text{LReLU}(z) = \max(\alpha z, z)$ con $\alpha = 0.1$ resuelve el problema de las neuronas muertas (unidades que quedan atascadas con $z < 0$ y nunca se actualizan). La **Softplus** $\text{softplus}(z) = \ln(1 + e^z)$ es una aproximación suave y diferenciable en todas partes de la ReLU.

El **problema del gradiente desvaneciente** ocurre cuando se apilan muchas capas con activaciones cuya derivada es menor que 1 en valor absoluto. Si la red tiene L capas y cada una tiene derivada de activación $\delta < 1$, el gradiente que llega a las capas iniciales escala como δ^L, que decrece exponencialmente con la profundidad. Para sigmoid, $\delta \leq 0.25$, por lo que en una red de 10 capas el gradiente puede reducirse en un factor de $0.25^{10} \approx 10^{-6}$. La ReLU, al tener derivada 1 para entradas positivas, elimina este factor multiplicativo en las unidades activas.

Red Neuronal Multicapa (MLP)

Arquitectura

Una red neuronal multicapa (*Multilayer Perceptrón*, MLP) es una composición de L capas de transformaciones. Cada capa $\ell \in \{1, \ldots, L\}$ tiene n_ℓ unidades (neuronas). La capa 0 corresponde a la entrada con $n_0 = d$ unidades. Los parámetros de la capa ℓ son la matriz de pesos $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$ y el vector de sesgos $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$. La capa L es la capa de salida y las capas $1, \ldots, L-1$ son capas ocultas. El número total de parámetros es $\sum_{\ell=1}^{L} (n_{\ell-1} \cdot n_\ell + n_\ell)$, que puede escalar rápidamente con la profundidad y el ancho de la red.

La elección de la arquitectura (número de capas y unidades por capa) es uno de los hiperparámetros más importantes del diseño de una red. Redes más profundas pueden aprender representaciones más complejas, pero son más difíciles de entrenar y más propensas al sobreajuste con conjuntos de datos pequeños. Redes más anchas tienen mayor capacidad memorística pero son menos eficientes en términos de parámetros por unidad de complejidad. En la práctica, la arquitectura se elige mediante validación cruzada o mediante heurística basada en el dominio del problema.

Propagación hacia adelante

Dada una matriz de datos $\mathbf{X} \in \mathbb{R}^{m \times n_0}$ con m ejemplos, la propagación hacia adelante (*forward pass*) calcula la predicción de la red aplicando las transformaciones de cada capa en orden. Se define $\mathbf{A}^{(0)} = \mathbf{X}$ y para cada capa $\ell = 1, \dots, L$:

$$\mathbf{z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \tag{64.4}$$

$$\mathbf{A}^{(\ell)} = g^{(\ell)}(\mathbf{z}^{(\ell)}) \tag{64.5}$$

donde $g^{(\ell)}$ es la función de activación de la capa ℓ. Para las capas ocultas se usa típicamente ReLU o tanh; para la capa de salida en clasificación binaria se usa sigmoide σ (que produce probabilidades en $(0,1)$) y para clasificación multiclase se usa softmax $\mathrm{softmax}(\mathbf{z})_k = e^{z_k} / \sum_j e^{z_j}$. La predicción final es $\hat{\mathbf{Y}} = \mathbf{A}^{(L)}$.

Teorema de aproximación universal

El resultado fundamental que justifica el poder de las redes neuronales fue demostrado independientemente por Cybenko (1989) para activaciones sigmoideas (Cybenko 1989) y por Hornik, Stinchcombe y White para una clase más amplia de activaciones no polinomiales (Hornik et al. 1989):

Teorema (Aproximación Universal). Sea g una función de activación continua y no polinomial. Para cualquier función continua $f : [0,1]^d \to \mathbb{R}$ y cualquier $\varepsilon > 0$, existe una red neuronal de una sola capa oculta con suficientes unidades que aproxima f con error máximo ε en todo su dominio.

Este teorema es de existencia, no constructivo: garantiza que la red *puede* aproximar cualquier función, pero no dice como encontrar los pesos correctos. Tampoco dice cuantas unidades se necesitan (que puede ser exponencialmente grande), ni garantiza que el algoritmo de entrenamiento las encuentre. La profundidad importa precisamente porque redes profundas pueden representar ciertas funciones con exponencialmente menos unidades que redes superficiales, una propiedad conocida como ventaja de profundidad.

Ejemplo a mano: una red que resuelve XOR

Una capa oculta basta para resolver el XOR que el perceptrón no puede. Con activaciones sigmoide y pesos elegidos a mano, la neurona h_1 calcula un OR, la h_2 un NAND y la salida un AND de ambas:

$$h_1 = \sigma(20x_1 + 20x_2 - 10), \qquad h_2 = \sigma(-20x_1 - 20x_2 + 30), \qquad \hat{y} = \sigma(20h_1 + 20h_2 - 30).$$

Tabla 64.1.: Propagación hacia adelante de una red 2-2-1 que calcula XOR.

x_1	x_2	h_1 (OR)	h_2 (NAND)	$\hat{y}$	XOR
0	0	0.00	1.00	0.00	0
0	1	1.00	1.00	1.00	1
1	0	1.00	1.00	1.00	1
1	1	1.00	0.00	0.00	0

La capa oculta **cambia la representación**: en el espacio (h_1, h_2) los cuatro puntos son $(0,1)$, $(1,1)$, $(1,1)$ y $(1,0)$, y ahí sí una recta separa las clases. Esa es la idea central de las redes profundas: cada capa transforma los datos hasta que el problema se vuelve linealmente separable para la última.

Ejemplo a mano: un paso de retropropagación

Una red mínima con una entrada, una neurona oculta y una salida, ambas sigmoides: $h = \sigma(w_1 x + b_1)$ y $\hat{y} = \sigma(w_2 h + b_2)$. Con $x = 1$, $y = 1$, $w_1 = 0.5$, $w_2 = 1$ y sesgos en cero:

Hacia adelante. $z_1 = 0.5$, $h = \sigma(0.5) = 0.6225$; $z_2 = 0.6225$, $\hat{y} = \sigma(0.6225) = 0.6508$. La pérdida es $-\ln 0.6508 = 0.4296$.

Hacia atrás. Con entropía cruzada y salida sigmoide, el error de la salida es simplemente $\delta_2 = \hat{y} - y = -0.3492$. De ahí:

$$\frac{\partial \mathcal{L}}{\partial w_2} = \delta_2 \, h = -0.2174, \qquad \frac{\partial \mathcal{L}}{\partial b_2} = \delta_2 = -0.3492.$$

El error se propaga a la capa oculta multiplicando por el peso que la conecta y por la derivada de su activación, $\sigma'(z_1) = h(1-h) = 0.2350$:

$$\delta_1 = \delta_2 \, w_2 \, h(1-h) = -0.3492 \cdot 1 \cdot 0.2350 = -0.0821, \qquad \frac{\partial \mathcal{L}}{\partial w_1} = \delta_1 \, x = -0.0821.$$

Actualización con $\eta = 1$: $w_2 = 1.217$, $b_2 = 0.349$, $w_1 = 0.582$, $b_1 = 0.082$. La nueva predicción es $\hat{y} = 0.760$ y la pérdida baja de 0.430 a 0.274. Observa que el gradiente de la capa oculta es unas cuatro veces menor que el de la salida: el factor $h(1-h) \leq 0.25$ de la sigmoide lo encoge en cada capa, el origen del **gradiente desvaneciente** que se discute a continuación.

Retropropagación del Error

Función de pérdida

El entrenamiento de la red se formula como la minimización de una función de pérdida $\mathcal{L}$ que mide la discrepancia entre las predicciones del modelo y los valores reales. Para clasificación binaria, la pérdida de entropía cruzada (cross-entropy) es la elección estándar, derivada del principio de máxima verosimilitud bajo el modelo de Bernoulli:

$$\mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\hat{y}^{(i)} + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right] \tag{64.6}$$

donde $\hat{y}^{(i)} = \sigma(z_L^{(i)})$ es la probabilidad predicha para el ejemplo i. Esta pérdida es convexa como función de la capa de salida (aunque no convexa en los pesos de todas las capas), y tiene la propiedad de que su gradiente respecto a la entrada de la capa de salida es simplemente $\hat{y} - y$, lo que simplifica las derivaciones del gradiente.

Para minimizar $\mathcal{L}$, se usa descenso de gradiente, que requiere calcular $\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L}$ para cada capa ℓ. El algoritmo de retropropagación (Rumelhart et al. 1986) computa estos gradientes de forma eficiente mediante la regla de la cadena, propagando el error desde la capa de salida hacia las capas iniciales. Su complejidad computacional es proporcional a la del paso hacia adelante, lo que lo hace tractable incluso para redes con millones de parámetros.

Regla de la cadena — derivación capa a capa

Definimos la señal de error en la capa ℓ como $\delta^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}}$. Esta cantidad captura cuanto contribuye la activación pre-lineal de la capa ℓ a la pérdida total. La recurrencia de retropropagación se establece de la siguiente manera. Para la capa de salida L con sigmoide y pérdida cross-entropy:

$$\delta^{(L)} = \hat{\mathbf{Y}} - \mathbf{Y} \tag{64.7}$$

Para las capas ocultas $\ell < L$, aplicando la regla de la cadena en forma matricial:

$$\delta^{(\ell)} = \left(\delta^{(\ell+1)}\mathbf{W}^{(\ell+1)\top}\right) \odot g'^{(\ell)}(\mathbf{z}^{(\ell)}) \tag{64.8}$$

donde $\odot$ denota el producto elemento a elemento (producto de Hadamard) y $g'^{(\ell)}$ es la derivada de la función de activación de la capa ℓ. Los gradientes de los parámetros son:

$$\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L} = \frac{1}{m}\mathbf{A}^{(\ell-1)\top}\delta^{(\ell)}, \quad \nabla_{\mathbf{b}^{(\ell)}}\mathcal{L} = \frac{1}{m}\sum_{i=1}^{m}\delta_i^{(\ell)} \tag{64.9}$$

Esta derivación revela por que la profundidad es un desafio: el gradiente de la pérdida respecto a los pesos de la primera capa es el producto de L matrices jacobianas $g'^{(\ell)}$. Si cada una tiene norma espectral menor que 1 (como ocurre con sigmoid o tanh en la mayor parte de su dominio), el gradiente decrece exponencialmente, impidiendo el aprendizaje en las capas profundas.

Optimización

Descenso de gradiente estocástico (SGD)

El descenso de gradiente estocástico (*Stochastic Gradient Descent*, SGD) es el algoritmo base de optimización para el entrenamiento de redes neuronales. En lugar de calcular el gradiente exacto sobre todo el conjunto de entrenamiento (lo que sería computacionalmente prohibitivo para conjuntos grandes), se calcula el gradiente sobre un mini-lote aleatorio de B ejemplos:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}_{\text{mini-lote}} \tag{64.10}$$

La tasa de aprendizaje η controla el tamaño de cada paso en el espacio de parámetros. Si es demasiado grande, el algoritmo puede divergir u oscilar alrededor del mínimo; si es demasiado pequeña, el entrenamiento es lento. El tamaño del mini-lote B introduce un trade-off entre precisión del gradiente (lotes grandes) y eficiencia computacional y regularización implícita (lotes pequeños). El ruido del gradiente introducido por el muestreo aleatorio actua como regularizador: impide que el optimizador converja a minimos excesivamente agudos del paisaje de pérdida, lo que favorece soluciones con mejor generalización.

Momentum

El SGD con momentum introduce una variable de velocidad $\mathbf{v}$ que acumula gradientes pasados, similar a un objeto con inercia que acelera en direcciones de descenso consistente y amortigua las oscilaciones:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + (1 - \beta) \nabla \mathcal{L} \tag{64.11}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \mathbf{v} \tag{64.12}$$

Con β típicamente en $[0.85, 0.99]$, el momentum efectivo que llega al parámetro es la suma geometricamente ponderada de gradientes pasados, lo que suaviza la trayectoria de optimización y permite tasas de aprendizaje más grandes. En paisajes con curvatura muy anisotrópica (donde algunas dimensiones tienen gradientes mucho mayores que otras), el momentum reduce las oscilaciones en las dimensiones de alta curvatura y acelera el progreso en las de baja curvatura.

Adam

El optimizador Adam (Adaptive Moment Estimation) (Kingma y Ba 2015) combina el momentum con la adaptación de la tasa de aprendizaje por coordenada, usando estimaciones de primer y segundo momento del gradiente:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla \mathcal{L} \tag{64.13}$$

$$\mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2) (\nabla \mathcal{L})^2 \tag{64.14}$$

$$\hat{\mathbf{m}} = \mathbf{m} / (1 - \beta_1^t), \quad \hat{\mathbf{v}} = \mathbf{v} / (1 - \beta_2^t) \tag{64.15}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \hat{\mathbf{m}} / (\sqrt{\hat{\mathbf{v}}} + \epsilon) \tag{64.16}$$

Los términos de corrección de sesgo $(1 - \beta_k^t)$ compensan el hecho de que al inicio del entrenamiento $\mathbf{m}$ y $\mathbf{v}$ están inicializados en cero y subestiman los momentos verdaderos. Los valores típicos son $\beta_1 = 0.9$, $\beta_2 = 0.999$ y $\epsilon = 10^{-8}$. La división por $\sqrt{\hat{\mathbf{v}}} + \epsilon$ normaliza el gradiente por su magnitud típica en cada coordenada, lo que hace al optimizador invariante a la escala del gradiente y permite usar la misma tasa de aprendizaje para todos los parámetros.

Laboratorio: entrena una red en el navegador

Una red con una capa oculta se entrena aquí mismo con descenso de gradiente de lote completo y entropía cruzada. El color de fondo es la probabilidad predicha de la clase azul; los puntos con borde son de prueba.

En la versión web, un laboratorio entrena una red de una capa oculta en el navegador sobre cuatro conjuntos bidimensionales (XOR, círculo, lunas y espiral). Con cuatro neuronas tanh y 0.3, XOR, el círculo y las lunas superan el 90 % de exactitud en unas mil épocas; con una sola neurona la frontera es casi lineal y la exactitud se queda cerca del 50–70 %; la espiral necesita muchas más neuronas.

Guía del laboratorio.

1. Con XOR y una sola neurona oculta, entrena 1000 épocas. ¿Por qué no supera a un clasificador lineal? Sube a 2 neuronas y repite (recuerda la Tabla 64.1).
2. Compara tanh, ReLU y sigmoide con el mismo número de neuronas. ¿Cuál converge más lento? Relaciónalo con el factor $h(1-h)$ del ejemplo de retropropagación.
3. Predice antes de mover: con = 3, ¿la pérdida bajará más rápido o se volverá errática?
4. Con *Espiral*, entrena con 4 y con 16 neuronas durante varios miles de épocas. ¿Hasta dónde llega cada una? Una frontera con muchas vueltas necesita muchos «pliegues»: es el tipo de problema donde más capas ayudan más que más neuronas.

Implementación desde Cero

Para consolidar la comprensión matemática de los algoritmos descritos, se presenta a continuación una implementación completa de un MLP en Python puro con numpy, sin utilizar ninguna biblioteca de aprendizaje automático para el entrenamiento. La implementación sigue exactamente las ecuaciones de propagación hacia adelante (Ecuación 64.4, Ecuación 64.5) y retropropagación (Ecuación 64.7, Ecuación 64.8, Ecuación 64.9). La inicialización de pesos usa la inicialización de He, que escala los pesos por $\sqrt{2/n_{\ell-1}}$ para capas ReLU, lo que asegura que la varianza de las activaciones se preserve a través de las capas en el paso hacia adelante.

Esta implementación sirve como puente entre la teoría matemática y las bibliotecas de alto nivel como PyTorch o TensorFlow. En la práctica, esas bibliotecas ofrecen diferenciación automática, ejecución en GPU y múltiples optimizadores, pero la estructura lógica es idéntica a la presentada aquí. Entender la implementación desde cero es fundamental para diagnosticar problemas de entrenamiento, interpretar el comportamiento de la red y adaptar el modelo a casos de uso no estándar.

Redes Convolucionales (CNN)

Motivación

Las redes completamente conectadas tratan cada pixel de una imagen como una entrada independiente, ignorando por completo la estructura espacial. Para una imagen en escala de grises de 32×32 píxeles, la capa de entrada tiene 1024 unidades. Si la primera capa oculta tiene 512 unidades, solo esta capa requiere $1024 \times 512 = 524{,}288$ parámetros. Para imágenes en color de 256×256 píxeles, una sola capa con 512 unidades necesitaría más de 100 millones de parámetros. Además, si la imagen se traslada un pixel a la derecha, la red la ve como una entrada completamente diferente: no tiene ninguna noción de invarianza a la traslación, que es una propiedad fundamental de las imágenes naturales.

Las Redes Convolucionales (CNN) resuelven estos problemas mediante tres principios fundamentales (LeCun et al. 1998). El primero es la **conectividad local**: cada neurona de una capa convolucional se conecta solo a una región pequeña de la capa anterior (el campo receptivo), reduciendo drasticamente el número de conexiones. El segundo es el **compartimiento de pesos**: el mismo filtro (kernel) se aplica en todas las posiciones espaciales de la imagen, por lo que los parámetros del filtro se comparten entre todas las posiciones. El tercero es la **equivarianza a la traslación**: si el objeto se mueve en la imagen, la representación aprendida se mueve de manera correspondiente, lo que permite al modelo reconocer el mismo patrón independientemente de donde aparezca.

La operación de convolución

La operación fundamental de las CNN es la correlación cruzada 2D entre una imagen de entrada I y un filtro K de tamaño $k \times k$. Para cada posición (i, j) en la salida:

$$(I \star K)[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} I[i+u, j+v] \cdot K[u, v] \tag{64.17}$$

Si la entrada tiene dimensiones $H \times W$ y el filtro es $k \times k$, la salida tiene dimensiones $(H - k + 1) \times (W - k + 1)$ (sin relleno). Es importante notar que en la literatura de *deep learning* se llama "convolución" a lo que tecnicamente es correlación cruzada: la verdadera convolución matemática voltea el kernel antes de aplicarlo, pero dado que los pesos se aprenden durante el entrenamiento, esta distinción no tiene consecuencias prácticas (Goodfellow et al. 2016).

Capas de pooling

Las capas de pooling reducen las dimensiones espaciales de los mapas de características, introduciendo invarianza local a pequeñas traslaciones y reduciendo el número de parámetros en las capas subsiguientes. El **max-pooling** sobre una ventana de $p \times p$ retiene el valor máximo de cada ventana, capturando la presencia del patrón más activado en cada región. El **average-pooling** calcula el promedio, produciendo una representación más suave. Para una entrada de tamaño $H \times W$, el max-pooling con ventana $p \times p$ y stride p produce una salida de tamaño $\lfloor H/p \rfloor \times \lfloor W/p \rfloor$.

La operación de max-pooling tiene una derivada simple: en el paso hacia atras, el gradiente se propaga solo a la posición que fue el máximo en la ventana (las demas reciben gradiente cero). Esto

es eficiente computacionalmente y favorece la especialización de las neuronas: cada unidad del mapa de características aprende a detectar un patrón específico en alguna región del campo receptivo.

Arquitectura CNN clásica: LeNet-5

La arquitectura LeNet-5 (LeCun et al. 1998) establece el patrón básico de las CNN modernas: bloques de convolución seguidos de pooling, finalizados por capas completamente conectadas para la clasificación. El patrón general es:

$$\text{Conv} \to \text{Pool} \to \text{Conv} \to \text{Pool} \to \text{FC} \to \text{FC} \to \text{Softmax}$$

Las capas convolucionales aprenden filtros que detectan características locales (bordes, texturas, formas) mientras que las capas completamente conectadas integran estas características espaciales para la decisión final. Las arquitecturas modernas como VGG, ResNet e Inception siguen este mismo principio, pero con mayor profundidad, conexiones residuales y bloques de atención.

Regularización en Redes Neuronales

Las redes neuronales con muchos parámetros tienen alta capacidad de memorizar el conjunto de entrenamiento, lo que puede llevar al sobreajuste: el modelo obtiene baja pérdida en entrenamiento pero generaliza pobremente a datos nuevos. La regularización comprende un conjunto de técnicas que reducen la brecha entre el error de entrenamiento y el error de generalización, ya sea penalizando la complejidad del modelo, perturbando el proceso de entrenamiento o limitando la co-adaptacion de las neuronas.

Regularización L2 (weight decay)

La regularización L2 anade un término de penalización proporcional a la norma cuadrada de los pesos a la función de pérdida:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{datos}} + \frac{\lambda}{2}\|\mathbf{W}\|_F^2 \tag{64.18}$$

donde $\|\mathbf{W}\|_F^2 = \sum_{i,j} W_{ij}^2$ es la norma de Frobenius y $\lambda > 0$ es el coeficiente de regularización. El efecto sobre la regla de actualización es una contracción multiplicativa de los pesos en cada paso:

$$\mathbf{W} \leftarrow (1 - \eta\lambda)\mathbf{W} - \eta\nabla_{\mathbf{W}}\mathcal{L}_{\text{datos}} \tag{64.19}$$

El factor $(1 - \eta\lambda) < 1$ "decae" los pesos hacia cero en cada iteración, lo que da nombre al método (*weight decay*). Esta penalización favorece soluciones con pesos pequeños, que tienden a ser más suaves y generalizan mejor. La magnitud de λ controla el trade-off entre ajuste a los datos y suavidad del modelo.

Dropout

El Dropout (Srivastava et al. 2014) es una técnica de regularización específica de las redes neuronales. Durante el entrenamiento, en cada paso hacia adelante, cada neurona se desactiva de forma aleatoria con probabilidad $1 - p$ (o equivalentemente, se mantiene activa con probabilidad p, típicamente $p \in [0.5, 0.9]$). Las neuronas desactivadas no contribuyen a la salida ni reciben actualizaciones de gradiente en ese paso. Durante la inferencia, todas las neuronas están activas, pero sus salidas se escalan por p (o alternativamente, se dividen por p durante el entrenamiento, técnica conocida como *inverted dropout*).

El Dropout puede interpretarse como el entrenamiento implícito de un ensamble exponencialmente grande de sub-redes. Con n neuronas en total, existen 2^n posibles sub-redes, y en cada paso se entrena una de ellas. En inferencia, la red completa actua como un promedio aproximado de todas estas sub-redes, lo que reduce la varianza del predictor de manera análoga al promediado de ensambles. Esta interpretación de ensamble implícito explica por que el Dropout mejora la generalización de forma tan sistemática.

Batch Normalization

La Normalización por Lote (*Batch Normalization*) es una técnica que normaliza las pre-activaciones de cada capa dentro del mini-lote de entrenamiento, seguido de una transformación lineal con parámetros aprendibles γ y β. Para cada capa ℓ y mini-lote $\mathcal{B}$ de tamaño m:

$$\mu_B = \frac{1}{m} \sum_{i \in \mathcal{B}} z_i^{(\ell)}, \quad \sigma_B^2 = \frac{1}{m} \sum_{i \in \mathcal{B}} (z_i^{(\ell)} - \mu_B)^2$$

$$\hat{z}^{(\ell)} = \frac{z^{(\ell)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \tilde{z}^{(\ell)} = \gamma \hat{z}^{(\ell)} + \beta \tag{64.20}$$

La normalización estabiliza la distribución de las activaciones a lo largo del entrenamiento (reduciendo el *internal covariate shift*), lo que permite tasas de aprendizaje más altas y hace al modelo menos sensible a la inicialización de pesos. Los parámetros γ y β son aprendibles y permiten al modelo recuperar la distribución óptima si la normalización no es beneficiosa en alguna capa.

Más alla: Tipos de Redes

Redes Recurrentes (RNN/LSTM)

Las redes completamente conectadas y las CNN asumen que las entradas son independientes entre si. Muchos problemas prácticos, sin embargo, involucran datos secuenciales donde el orden temporal es fundamental: series de tiempo, texto natural, señales de audio, trayectorias. Las Redes Neuronales Recurrentes (RNN) abordan este problema manteniendo un estado oculto $\mathbf{h}_t$ que actua como memoria de la historia de la secuencia:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

El estado oculto $\mathbf{h}_t$ encapsula información relevante de todos los pasos anteriores $\mathbf{x}_1, \ldots, \mathbf{x}_t$, permitiendo que la red haga predicciones que dependen del contexto histórico. Sin embargo, las RNN simples sufren de gradiente desvaneciente a través del tiempo: para secuencias largas, los gradientes que fluyen desde el tiempo T hasta el tiempo 1 son el producto de T matrices jacobianas, que pueden ser exponencialmente pequeñas.

Las redes LSTM (*Long Short-Term Memory*) (Hochreiter y Schmidhuber 1997) resuelven este problema mediante el mecanismo de compuertas: una celda de memoria $\mathbf{c}_t$ con flujo de gradiente regulado por compuertas de olvido ($\mathbf{f}_t$), entrada ($\mathbf{i}_t$) y salida ($\mathbf{o}_t$). La compuerta de olvido decide que información de la celda anterior se descarta, la compuerta de entrada decide que nueva información se escribe, y la compuerta de salida determina que parte de la celda se expone como estado oculto. Este mecanismo permite que los gradientes fluyan sin atenuación a través de cientos de pasos de tiempo, haciendo posible el aprendizaje de dependencias de largo alcance.

Mecanismo de Atención y Transformers

El mecanismo de atención (Vaswani et al. 2017) revoluciono el procesamiento de secuencias al permitir que cada posición de la secuencia de salida consulte directamente cualquier posición de la secuencia de entrada, sin importar la distancia entre ellas. La atención escalada de producto punto (*Scaled Dot-Product Attention*) se define como:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

donde $\mathbf{Q}$ (*queries*), $\mathbf{K}$ (*keys*) y $\mathbf{V}$ (*values*) son proyecciones lineales de la secuencia de entrada, y d_k es la dimensión de las claves (el factor $\sqrt{d_k}$ previene que los productos internos se vuelvan muy grandes, lo que saturaría el softmax). La atención asigna a cada posición un peso que refleja su relevancia para la posición de consulta, permitiendo que el modelo capture dependencias directas sin importar la distancia.

Los Transformers, introducidos en 2017, reemplazaron a las RNN como arquitectura dominante en procesamiento de lenguaje natural y, más recientemente, en visión computacional (Visión Transformers, ViT) y otros dominios. La arquitectura consiste en bloques de atención multi-cabeza (*multi-head attention*) combinados con redes de retroalimentación posición a posición y conexiones residuales, permitiendo el entrenamiento paralelo eficiente y la captura de dependencias de largo alcance sin el cuello de botella secuencial de las RNN.

Aprendizaje por Transferencia

El aprendizaje por transferencia (*Transfer Learning*) es el paradigma dominante del aprendizaje profundo moderno. En lugar de entrenar una red desde cero sobre el conjunto de datos objetivo (lo que requeriría millones de ejemplos y días de cómputo), se parte de un modelo pre-entrenado en un conjunto de datos masivo (como ImageNet para visión o Common Crawl para lenguaje) y se ajusta (*fine-tuning*) sobre los datos del problema objetivo.

El fundamento teórico es que las representaciones aprendidas en dominios de gran escala son transferibles: los filtros de bordes, texturas y formas aprendidos sobre ImageNet son útiles para

clasificar imágenes médicas; las representaciones lingüísticas de BERT son útiles para análisis de sentimiento en dominios específicos. El ajuste puede ser total (se actualizan todos los pesos del modelo) o parcial (se congelan las capas iniciales y solo se actualizan las capas finales), dependiendo del tamaño del conjunto de datos objetivo y su similitud con el conjunto de pre-entrenamiento. Este paradigma ha democratizado el aprendizaje profundo, permitiendo obtener resultados de estado del arte con cientos (no millones) de ejemplos etiquetados.

Resumen

Este capitulo ha recorrido el desarrollo de las redes neuronales desde sus fundamentos más elementales hasta las arquitecturas modernas. El perceptrón de Rosenblatt establece el esqueleto matemático: sumas ponderadas seguidas de no linealidades, entrenadas mediante una regla de actualización basada en errores. La extensión a redes multicapa, habilitada por el algoritmo de retropropagación, permite aprender representaciones jerárquicas que resuelven problemas no lineales arbitrariamente complejos, respaldado por el teorema de aproximación universal. Las funciones de activación ReLU y la inicialización cuidadosa de pesos hacen posible el entrenamiento de redes profundas sin sufrir el colapso del gradiente. Los optimizadores modernos como Adam aceleran la convergencia adaptando la tasa de aprendizaje a la geometría local del paisaje de pérdida. Las redes convolucionales explotan la estructura espacial de las imágenes mediante conectividad local y compartimiento de pesos, aprendiendo detectores de características jerárquicos que escalan desde píxeles hasta objetos. Finalmente, técnicas de regularización como L2 y Dropout permiten que redes con millones de parámetros generalicen más alla del conjunto de entrenamiento.

La implementación desde cero presentada en la Sección 6 demuestra que toda esta teoría se traduce en unas pocas decenas de lineas de álgebra lineal en numpy, consolidando la comprensión profunda de los mecanismos subyacentes que es necesaria para aplicar, diagnosticar y extender estas arquitecturas en contextos reales.

65. Implementación Práctica: PyTorch y JAX

En el capitulo anterior construimos redes neuronales desde cero con NumPy: derivamos la regla de la cadena a mano, escribimos cada operación de convolución a mano y controlamos cada actualización de pesos. Ese ejercicio es indispensable para entender *por qué* funcionan los algoritmos.

Este capitulo es el complemento práctico. Tomamos exactamente los mismos conceptos — forward pass, backprop, descenso por gradiente — y los reimplementamos en frameworks de producción que automatizan lo que ya entendemos.

Hay tres alternativas principales según el nivel de abstracción:

- **scikit-learn** — Un `MLPClassifier` con la interfaz `fit/predict` de siempre. Ideal para tablas pequeñas; sin control sobre el ciclo de entrenamiento.
- **PyTorch** (Paszke et al. 2019) — Grafos computacionales dinámicos, diferenciación automática y una API orientada a objetos. El estándar de facto en investigación académica.
- **JAX** (Bradbury et al. 2018) — Paradigma funcional puro: funciones sin estado, PRNG explícito, compilación XLA transparente. Sobre JAX se construye Flax (Heek et al. 2023) (módulos de redes) y Optax (DeepMind et al. 2020) (optimizadores).

La diferencia entre PyTorch y JAX no es solo sintáctica: reflejan dos paradigmas de programación opuestos. PyTorch es **imperativo** — escribes instrucciones sobre objetos con estado. JAX es **funcional** — las funciones no tienen estado y se pueden transformar automáticamente. Comprender esta diferencia es el objetivo central de este capítulo.

El ecosistema de aprendizaje profundo

scikit-learn: MLPClassifier

Scikit-learn (Pedregosa et al. 2011) incluye `sklearn.neural_network.MLPClassifier`, un perceptrón multicapa que se integra sin fricciones en cualquier pipeline de sklearn. Acepta los mismos hiperparámetros conceptuales que ya conocemos: número y tamaño de capas ocultas (`hidden_layer_sizes`), función de activación, optimizador (lbfgs, sgd o adam) y tasa de aprendizaje. La interfaz es idéntica a la de cualquier otro clasificador de sklearn: `fit` para entrenar, `predict` para inferencia y `score` para exactitud. Esta uniformidad hace que sea trivial sustituirlo por un árbol de decisión o una SVM en un experimento comparativo.

Las limitaciones son igualmente claras. No permite definir arquitecturas personalizadas: capas convolucionales, mecanismos de atención, conexiones residuales o cualquier topología no secuencial están fuera de alcance. El entrenamiento ocurre en CPU y no escala a conjuntos de datos de imagen o texto de tamaño real. La ausencia de control sobre el ciclo de entrenamiento impide aplicar técnicas avanzadas como curriculum learning, ciclos de tasa de aprendizaje o pérdidas compuestas. Para todo esto se requieren frameworks de bajo nivel.

PyTorch

PyTorch (Paszke et al. 2019) introdujo en 2017 el **grafo computacional dinámico** (*define-by-run*): el grafo se construye al vuelo mientras se ejecutan operaciones sobre tensores, no antes. A diferencia del grafo estático de TensorFlow 1.x, el código de PyTorch es Python normal — se puede depurar con `print` y `pdb`, y las estructuras de control funcionan exactamente como se esperaría.

El motor de diferenciación automática se llama **Autograd**. Cuando un tensor tiene `requires_grad=True`, cada operación sobre él registra un nodo en el grafo. Al llamar `.backward()` sobre la pérdida escalar, Autograd recorre el grafo en sentido inverso y acumula los gradientes en el atributo `.grad` de cada parámetro.

La clase base para todo modelo es `nn.Module`. Su contrato: el constructor `__init__` declara las capas, y el método `forward` describe el cómputo. El ciclo de entrenamiento es explícito — el programador escribe cada paso. Esa explicitación es una característica, no un defecto: da control total.

JAX

JAX (Bradbury et al. 2018) parte de una premisa diferente: las **transformaciones funcionales** son ciudadanos de primera clase. El núcleo son cuatro transformaciones componibles:

Transformación	Qué hace
`jax.grad(f)`	Diferenciación automática de funciones escalares
`jax.jit(f)`	Compilación XLA transparente
`jax.vmap(f)`	Vectorización sobre dimensiones de lote
`jax.pmap(f)`	Paralelismo en múltiples dispositivos

Para beneficiarse de `jax.jit`, las funciones deben ser **puras**: sin efectos secundarios, sin estado global mutable. Los pesos del modelo se pasan como *pytrees* (diccionarios/listas de arrays JAX) que JAX recorre automáticamente.

Flax (Heek et al. 2023) añade una API de módulos similar a `nn.Module`, manteniendo la disciplina funcional: `model.init(key, x)` devuelve los parámetros y `model.apply(params, x)` ejecuta el forward pass sin modificar nada internamente. Optax (DeepMind et al. 2020) provee los optimizadores.

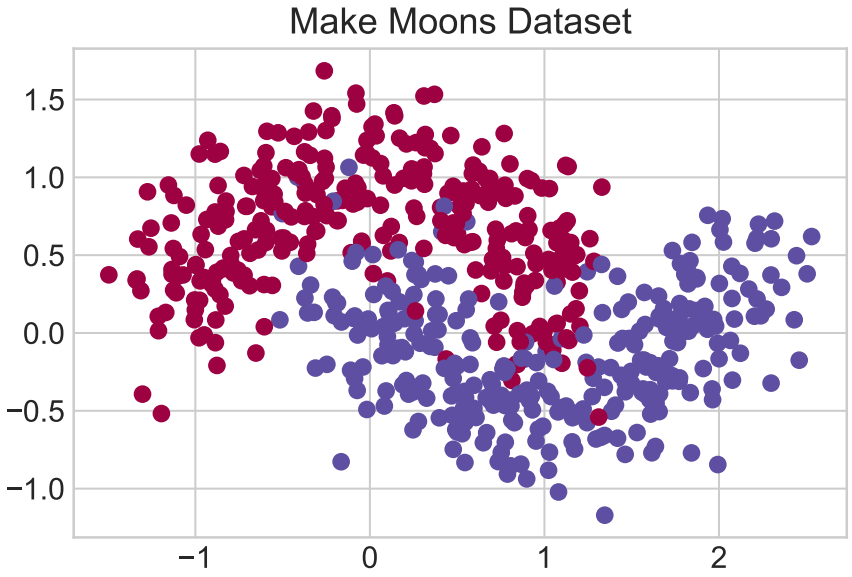
Comparativa de frameworks

La elección entre frameworks depende del caso de uso. Para prototipado rápido con datos tabulares, sklearn es imbatible en simplicidad. Para investigación en arquitecturas novedosas o fine-tuning de modelos preentrenados del ecosistema HuggingFace, PyTorch es el estándar de facto. Para investigación que requiere transformaciones funcionales, diferenciación de orden superior o integración nativa con TPUs de Google, JAX es la opción más poderosa. El siguiente radar resume estas características de forma comparativa.

Figura 65.1.

Dataset a explorar

Para los siguientes ejemplos utilizaremos el conocido dataset `make_moons`, un clásico para ilustrar clasificadores no lineales.



Preparación de datos

Antes de entrenar con PyTorch o JAX, los datos deben dividirse en conjuntos de entrenamiento y validación, y convertirse al formato esperado por cada framework.

Train-test split

Una partición típica es 80 % entrenamiento, 20 % validación, con la misma semilla aleatoria para reproducibilidad.

Comparativa: sklearn MLPClassifier

Como punto de partida, entrenemos un MLP de sklearn sobre el mismo dataset. Este es el clasificador "caja negra" que posteriormente comparamos con PyTorch y JAX.

La ventaja de sklearn es la sencillez: una linea para instanciar, una linea para entrenar, una linea para predecir. La desventaja es que no vemos las curvas de convergencia ni tenemos control sobre el ciclo de entrenamiento.

PyTorch: fundamentos

Tensores y autograd

Los tensores de PyTorch son analogos a los arrays de NumPy: admiten las mismas operaciones aritméticas, indexación y broadcasting. La diferencia fundamental es que un tensor puede residir en GPU (mediante `.to("cuda")`) y puede tener activado el rastreo de gradientes mediante `requires_grad=True`.

¿Que significa "rastreo de gradientes"? Cuando estableces `requires_grad=True` en un tensor, PyTorch comienza a registrar cada operación que le aplicas. Cada operación crea un nodo en un grafo computacional, junto con la función que permitira calcular el gradiente local cuando sea necesario. Es como dejar un rastro de "breadcrumbs" a través de tus cálculos.

El ciclo de gradientes: backward() y accumulation.

Para un parámetro θ y una pérdida escalar $\mathcal{L}$, la llamada `loss.backward()` computa todos los gradientes simultaneamente:

$$\frac{\partial \mathcal{L}}{\partial \theta} \quad \forall \theta \text{ con } \texttt{requires_grad=True} \tag{65.1}$$

El gradiente resultante se almacena en `theta.grad`.

Atrapas y detalles implementación: `.backward()` no *reemplaza* el gradiente en `.grad`; *acumula* (suma). Esto significa que si llamas `.backward()` dos veces sin limpiar gradientes intermedios, los valores se suman:

Por eso **cada ciclo de entrenamiento debe llamar `optimizer.zero_grad()` antes de `.backward()`**. Sin esta linea, el gradiente del epoch t se sumaría al del epoch $t-1$, produciendo actualizaciones completamente incorrectas que oscilan sin convergencia.

La regla de la cadena, automatizada.

Para funciones compuestas, Autograd aplica la regla de la cadena automáticamente. Consideremos:

$$z = (w \cdot x + b)^2 \quad \text{y} \quad \mathcal{L} = z$$

El gradiente respecto a w es:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial (wx+b)} \cdot \frac{\partial (wx+b)}{\partial w} = 1 \cdot 2(wx+b) \cdot x = 2(wx+b) \cdot x$$

Autograd calcula esto recorriendo el grafo en sentido inverso, aplicando la regla de la cadena nodo a nodo. La siguiente figura ilustra este grafo para valores concretos $w = 0.5$, $x = 2.0$, $b = 0.3$.

El patrón nn.Module

La clase `nn.Module` es la pieza central de PyTorch para definir modelos. Toda arquitectura, ya sea un MLP simple o un transformador con miles de millones de parámetros, es una subclase de `nn.Module`. El contrato es sencillo: el constructor `__init__` declara las capas y sub-modulos (que deben ser atributos de la instancia para que PyTorch los registre), y el método `forward` describe el cómputo que convierte una entrada en una salida.

`nn.Linear(in, out)` implementa $y = xW^T + b$ con inicialización de Kaiming. `nn.ReLU()` aplica máx$(0, z)$ elemento a elemento. `nn.Sequential` encadena módulos: la salida de cada uno es la entrada del siguiente.

Los métodos más útiles de cualquier `nn.Module`:

- `model.parameters()` — itera sobre todos los tensores con `requires_grad=True`. Se pasa directamente al optimizador.
- `model.state_dict()` — diccionario con los valores actuales de todos los parámetros. Permite guardar y cargar con `torch.save` / `torch.load`.
- `model.train()` / `model.eval()` — cambia el **modo** del modelo.

`model.train()` vs `model.eval()` — crítico en inferencia.
`model.eval()` no solo es estético. Capas como `nn.Dropout` y `nn.BatchNorm` se comportan diferente según el modo:

- **Dropout en `.train()`** → desactiva neuronas aleatoriamente (regularización).
- **Dropout en `.eval()`** → pasa todos los valores sin cambio (inferencia determinista).
- **BatchNorm en `.train()`** → usa estadísticas del lote actual.
- **BatchNorm en `.eval()`** → usa medias/varianzas acumuladas durante el entrenamiento.

Si olvidas llamar `model.eval()` en evaluación, tus métricas de validación serán ruidosas e irreproducibles.

El ciclo de entrenamiento

El ciclo de entrenamiento explícito es una de las características más apreciadas de PyTorch. A diferencia de los frameworks que ocultan el bucle interno (como Keras o sklearn), PyTorch expone cada paso, lo que facilita la implementación de lógicas de entrenamiento no estándar y permite debuggear exactamente que sucede en cada momento.

El patrón de 5 pasos.

El ciclo de entrenamiento en PyTorch sigue un patrón rigido que se repite en cada epoch:

El siguiente laboratorio recorre cada paso:

Cada iteración de entrenamiento en PyTorch consta de cinco pasos: `model.train()`, `optimizer.zero_grad()`, la propagación hacia adelante, `loss.backward()` y `optimizer.step()`. Si se omite `zero_grad()`, los gradientes se acumulan: al minimizar $(w-3)^2$ con SGD y $\eta = 0.1$ desde $w = 0$, el peso converge a 3 con la limpieza y oscila sin converger sin ella.

El error más frecuente: olvidar `optimizer.zero_grad()`.
PyTorch **acumula** gradientes: `.backward()` suma el gradiente calculado al valor existente en `.grad`. Si no limpias antes de cada paso:
El gradiente crece iterativamente. Los pesos reciben actualizaciones enormes y el entrenamiento diverge o produce modelos completamente distintos de lo esperado. La corrección es siempre llamar `optimizer.zero_grad()` antes de `loss.backward()`.

Paso 5: `optimizer.step()` — Usar gradientes para actualizar.

El optimizador lee los gradientes en `.grad` y actualiza los parámetros. Para SGD simple:

$$\theta := \theta - \eta \cdot \nabla \mathcal{L}(\theta)$$

Para Adam (el optimizador que usamos), mantiene exponential moving averages de los gradientes y sus cuadrados, y calcula updates adaptativos por parámetro.

Función de pérdida numéricamente estable.

¿Por qué `BCEWithLogitsLoss` y no `sigmoid` + `BCELoss`?
Aplicar sigmoide manualmente y luego `BCELoss` produce desbordamientos cuando el logit es muy grande. Por ejemplo, con $z = 100$, `sigmoid(100)` devuelve exactamente `1.0` y `log(1.0) = 0` — pero internamente ya hubo overflow.
`BCEWithLogitsLoss` combina ambas operaciones usando el truco log-sum-exp, que es numéricamente estable para cualquier valor:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left[\max(z_i, 0) - z_i y_i + \log\left(1 + e^{-|z_i|}\right) \right] \tag{65.2}$$

Regla práctica: siempre usa `BCEWithLogitsLoss` con logits crudos (sin aplicar sigmoide en `forward`).

Clasificación con PyTorch

Convertir arrays NumPy a tensores PyTorch

El primer paso es convertir los arrays NumPy a tensores PyTorch. Los tensores permiten diferenciación automática (rastreo de gradientes) y pueden ejecutarse en GPU para velocidad.

¿Por que `unsqueeze(1)`?

Note que `y_train` comienza como un vector 1D de shape `(480,)`. Lo convertimos a shape `(480, 1)` con `.unsqueeze(1)`.

¿Por que? Porque `nn.BCEWithLogitsLoss` espera que la predicción del modelo y la etiqueta tengan exactamente la misma forma. Nuestro modelo es un MLP de arquitectura $[2 \to 32 \to 32 \to 1]$, que para un lote de $N = 480$ produce un tensor de shape `(480, 1)`. Para que el cálculo de pérdida funcione correctamente, la etiqueta debe tener la misma forma.

Sin el `unsqueeze(1)`, intentaras hacer `BCEWithLogitsLoss` con shapes `(480, 1)` y `(480,)`, que causaría un error de broadcasting.

Definición del modelo MLP

En PyTorch, todo modelo es una subclase de `nn.Module`. La arquitectura es $[2 \to 32 \to 32 \to 1]$ con activaciones ReLU en las capas ocultas y una salida lineal (logit).

El modelo tiene: capa 1 ($2 \times 32 + 32$ parámetros) + capa 2 ($32 \times 32 + 32$) + capa 3 ($32 \times 1 + 1$) = $64 + 32 + 1024 + 32 + 32 + 1 = 1185$ parámetros en total.

Ciclo de entrenamiento con PyTorch

Aplicamos el patrón descrito en la sección anterior a un problema de clasificación binaria con `make_moons`. Este conjunto de datos es un clásico de prueba para fronteras de decisión no lineales: dos semilunares entrelazadas que no son separables linealmente. Una red con dos capas ocultas de 32 neuronas con activación ReLU debería aprender la frontera correctamente.

La arquitectura es $[2 \to 32 \to 32 \to 1]$: la capa de entrada recibe las dos coordenadas (x_1, x_2), las dos capas ocultas aprenden representaciones no lineales, y la capa de salida produce un único logit que se convierte en probabilidad mediante la sigmoide. Para la clasificación, el umbral es 0.5: si $\sigma(z) > 0.5$ se predice clase 1, de lo contrario clase 0.

El ciclo de entrenamiento sigue el patrón de **cinco pasos** que se repite en cada epoch:

Desglose detallado de cada sección.

Fase de entrenamiento:

1. `model_pt.train()` — Activa modo entrenamiento para capas como Dropout.
2. `optimizer.zero_grad()` — **Crucial:** Limpia los gradientes antiguos. Sin esto, `.backward()` acumularía gradientes sobre los anteriores.
3. `logits = model_pt(X_tr)` — Forward pass: propaga el lote a través de la red. Output shape `(480, 1)`.
4. `loss = criterion(logits, y_tr)` — Calcula pérdida comparando predicciones vs etiquetas verdaderas.
5. `loss.backward()` — Backward pass: recorre el grafo en reversa, acumulando gradientes en `.grad` de cada parámetro.
6. `optimizer.step()` — Lee los gradientes y actualiza los pesos según la regla del optimizador (Adam).

Fase de evaluación:

- `with torch.no_grad()`: Desactiva el rastreo de gradientes. No necesitamos gradientes para evaluación, así que esto ahorra memoria y tiempo.
- `model_pt.eval()`: Desactiva comportamientos aleatorios como Dropout y congela estadísticas de BatchNorm.
- Convertimos logits a probabilidades con `torch.sigmoid()` y aplicamos umbral 0.5 para clasificación binaria.
- `.item()` convierte un tensor escalar a un número Python para poder imprimir o almacenar.

Puntos críticos para entender.

1. **Los gradientes solo se calculan en `loss.backward()`**, no en `loss.backward()` + `optimizer.step()`. `step()` solo lee los gradientes que ya fueron calculados.
2. **`torch.no_grad()` es esencial en evaluación** porque no necesitamos gradientes y ahorrar memoria/tiempo es importante con lotes grandes.
3. **La evaluación usa el mismo modelo pero modo diferente** (`model_pt.eval()` vs `model_pt.train()`), lo que afecta como se comportan capas especiales.

Las curvas de pérdida muestran el comportamiento típico de un modelo bien ajustado: la pérdida de entrenamiento disminuye monotonamente mientras que la pérdida de validación converge a un valor cercano pero ligeramente superior, indicando que el modelo generaliza sin sobreajuste significativo. Las curvas de exactitud muestran que ambas fracciones (entrenamiento y validación) alcanzan valores por encima del 90 % tras los primeros 50 epochs, con una mejora marginal en los epochs restantes.

Sensibilidad a la tasa de aprendizaje

La tasa de aprendizaje η es el hiperparámetro más crítico de cualquier red neuronal. Un valor demasiado bajo produce convergencia lenta o estancamiento en minimos locales poco favorables; un valor demasiado alto provoca oscilaciones en la pérdida o incluso divergencia. La siguiente figura entrena cuatro instancias del mismo MLP con tasas de aprendizaje que cubren tres ordenes de magnitud, permitiendo observar estos regímenes de forma directa.

La frontera de decisión aprendida por la red se puede visualizar evaluando el modelo sobre una malla densa de puntos en el espacio de entrada. Cada punto de la malla recibe una probabilidad predicha $\hat{p} = \sigma(f_\theta(x))$, y la isocurva $\hat{p} = 0.5$ define la frontera de decisión.

La frontera de decisión resultante es claramente no lineal: la red ha aprendido a separar las dos lunas con una curva suave que sigue la estructura geométrica de los datos. La isocurva blanca en $\hat{p} = 0.5$ delimita la región de incertidumbre máxima, donde el modelo considera igualmente probables ambas clases. Las regiones de color intenso corresponden a zonas de alta confianza en la predicción.

Animación del aprendizaje

Una de las visualizaciones más reveladoras del proceso de entrenamiento de una red neuronal es observar como evoluciona la frontera de decisión a lo largo de las épocas. Al comienzo del entrenamiento, con pesos aleatorios pequeños, la frontera es casi lineal porque las activaciones ReLU operan en su región lineal. A medida que los pesos crecen y las no linealidades se activan, la frontera adquiere curvatura progresivamente hasta ajustarse a la geometría de los datos. El deslizador permite navegar por este proceso epoch a epoch.

CNN con PyTorch

Arquitectura CNN en PyTorch

Para ilustrar las redes convolucionales en PyTorch utilizamos `digits` de sklearn: 1797 imágenes de dígitos manuscritos en escala de grises de 8×8 píxeles y 10 clases. Es considerablemente más

pequeño que MNIST (28×28), pero suficiente para demostrar la arquitectura sin tiempos de espera largos.

Convención de forma: channels-first.

> **PyTorch es *channels-first*; JAX/Flax es *channels-last*.**
> PyTorch espera tensores de imagen con forma `(N, C, H, W)`: - `N` = tamaño del lote - `C` = canales (1 para escala de grises, 3 para RGB) - `H`, `W` = alto y ancho en píxeles
> Para `digits` en escala de grises: `(N, 1, 8, 8)`.
> JAX/Flax usa el orden opuesto `(N, H, W, C)`. Si portas código entre frameworks sin cambiar el reshape, obtendrás errores de forma confusos.

Las capas clave de la arquitectura:

- `nn.Conv2d(in, out, kernel_size, padding)` — aplica `out` filtros de `kernel × kernel` a la entrada. Con `padding=1` y `kernel_size=3`, las dimensiones espaciales se conservan.
- `nn.MaxPool2d(2)` — reduce dimensiones a la mitad seleccionando el máximo en ventanas 2×2.
- `nn.Flatten()` — convierte el tensor 4D a un vector 1D para capas densas.
- `nn.Dropout(0.3)` — desactiva aleatoriamente el 30 % de las neuronas. Solo activo en `model.train()`.

El flujo de dimensiones con la arquitectura abajo es:

```
Entrada:          (N, 1, 8, 8)
Conv1 + ReLU:     (N, 16, 8, 8)   ← 16 filtros 3×3, padding conserva tamaño
MaxPool:          (N, 16, 4, 4)   ← reducción ×2
Conv2 + ReLU:     (N, 32, 4, 4)   ← 32 filtros 3×3
MaxPool:          (N, 32, 2, 2)   ← reducción ×2
Flatten:          (N, 128)        ← 32 × 2 × 2 = 128
Dense + ReLU:     (N, 64)
Dropout:          (N, 64)
Salida:           (N, 10)         ← 10 logits para 10 dígitos
```

La CNN converge rápidamente en las imágenes de 8×8 de `digits`. La exactitud de validación supera el 95 % en pocos epochs, lo que refleja que estas imágenes son relativamente fáciles para una red convolucional: los patrones de cada digito son altamente consistentes y la resolución baja no introduce demasiada variabilidad. El `Dropout` con tasa 0.3 previene el sobreajuste, como se observa en la cercanía entre las curvas de entrenamiento y validación.

Una métrica especialmente útil para la clasificación multiclase es la matriz de confusión, que muestra cuantas instancias de la clase verdadera i fueron predichas como clase j. Los elementos diagonales corresponden a predicciones correctas; los elementos fuera de la diagonal revelan que pares de clases se confunden entre si, lo que puede guiar el análisis de errores y el ajuste de la arquitectura.

La diagonal dominante en la matriz de confusión confirma el buen desempeno del modelo. Las confusiones más frecuentes suelen ocurrir entre digitos con morfología similar: el 4 y el 9 comparten una curvatura superior cerrada, y el 3 y el 8 tienen regiones de activación superpuestas. Estos errores son consistentes con los que comete un clasificador humano cuando la escritura es ambigua,

lo que sugiere que el modelo ha aprendido representaciones genuinamente relevantes y no artefactos espurios.

Filtros aprendidos por la primera capa convolucional

Una propiedad notable de las CNNs es que sus filtros no son disenados manualmente: emergen del proceso de optimización como los detectores de características más útiles para la tarea. En los primeros niveles de una CNN entrenada en imágenes naturales (ImageNet), los filtros suelen especializarse en bordes orientados, manchas de color y texturas de baja frecuencia. En nuestra CNN entrenada sobre digitos 8x8, la primera capa aprende 16 filtros 3x3 que capturan los patrones más discriminativos de la escritura numérica.

JAX: fundamentos

El paradigma funcional

JAX (Bradbury et al. 2018) redefine completamente la relación entre Python y el hardware de aceleración. La diferencia no es solo técnica; es una filosofía de programación fundamentalmente diferente.

La diferencia central, en concreto:

PyTorch: imperativo

El modelo es un **objeto con estado mutable**: los pesos viven dentro de él y el optimizador los modifica en su lugar.

Ventaja: intuitivo y fácil de depurar. **Costo:** más difícil de compilar y de transformar.

JAX: funcional

Las funciones son **puras**: el estado (los pesos) vive fuera y se pasa explícitamente.

Ventaja: componible, compilable y reproducible. **Costo:** más verboso; depurar es más difícil.

¿Que es programación funcional pura?

La diferencia central: en PyTorch el estado *vive dentro del objeto modelo* y muta con cada paso. En JAX el estado son *datos externos* que pasas explícitamente y que nunca se modifican — cada operación produce valores nuevos.

Las cuatro transformaciones de JAX.

JAX proporciona cuatro transformaciones componibles:

1. `jax.grad(f)` — Diferenciación automática

Retorna directamente los gradientes como pytree. No acumula en `.grad` — los gradientes son valores de retorno, no efectos secundarios.

2. `jax.jit(f)` — Compilación XLA

3. `jax.vmap(f)` — Vectorización automática

Transforma una función que opera sobre *un solo ejemplo* en una que opera sobre *un lote entero*, sin escribir índices de lote.

4. `jax.pmap(f)` — Paralelismo entre dispositivos

Ejecuta en paralelo sobre múltiples GPUs/TPUs. Poco común para usuarios nuevos.

La potencia de la composición funcional.
Porque todas las funciones JAX son puras, se pueden anidar libremente:
En PyTorch, componer estas operaciones requiere configuraciones no estándar. En JAX, es la forma natural de trabajar.

Parámetros como pytrees: estado explícito

En JAX no existe un objeto "modelo" que contenga estado interno. En su lugar, los parámetros son **estructuras de datos Python explicitas** (diccionarios, listas, tuplas, o combinaciones anidadas) que contienen los arrays de JAX. Estos se pasan explícitamente como argumentos a cada función.

¿Por qué es ventajoso?

1. **Inspeccionables:** Puedes ver exactamente qué parámetros tiene tu modelo en cualquier momento.
2. **Serializables:** Guardar y cargar es trivial — solo serializa el diccionario.
3. **Transformables:** JAX entiende estas estructuras y puede aplicar transformaciones automáticamente.

Pytrees: contenedores anidados de datos.

Un pytree es cualquier estructura de datos Python donde las "hojas" (elementos más profundos) son arrays JAX. Ejemplos:

JAX transforma automáticamente cada hoja.

La magia de JAX es que cuando usas transformaciones como `jax.grad()` sobre una función que recibe un pytree, **JAX retorna un pytree con la misma estructura**.

Esto significa que actualizaciones de parámetros son triviales:

Ejemplo completo: forward pass con parámetros pytree.

Visualización: grad de la entropía cruzada

Para ilustrar `jax.grad` de forma concreta, calculamos la pérdida de entropía cruzada binaria y su gradiente respecto a un peso escalar w variando en $[-3, 3]$, con entrada fija $x = 1$ e etiqueta $y = 1$. La pérdida es:

$$\mathcal{L}(w) = -\left[y \log \sigma(wx) + (1 - y) \log(1 - \sigma(wx))\right] \tag{65.3}$$

donde $\sigma(z) = 1/(1 + e^{-z})$. Con $y = x = 1$, la pérdida simplifica a $\mathcal{L}(w) = -\log \sigma(w)$, que es una función decreciente (a mayor w, la predicción $\sigma(w)$ se acerca a 1 y la pérdida se acerca a 0). El gradiente es $\partial \mathcal{L}/\partial w = \sigma(w) - 1 \leq 0$, siempre negativo: el descenso por gradiente siempre incrementa w para que la predicción se acerque a $y = 1$.

La figura confirma la intuición matemática: la pérdida es alta y decreciente para w negativo (la red esta haciendo predicciones incorrectas con alta confianza) y se aproxima a cero a medida que w crece. El gradiente es siempre negativo, lo que significa que el descenso por gradiente siempre empuja w hacia valores más grandes. En $w = 0$ (inicialización típica), el gradiente vale exactamente -0.5, lo que corresponde al caso de incertidumbre máxima donde $\sigma(0) = 0.5$.

Derivadas de orden superior: una ventaja exclusiva de JAX

Una capacidad que distingue a JAX de otros frameworks es la diferenciación de orden superior mediante composición de `jax.grad`. En PyTorch es posible pero requiere configuraciones especiales (`create_graph=True`); en JAX es natural: `jax.grad(jax.grad(f))` es exactamente lo que parece. Esta capacidad es fundamental para métodos de optimización de segundo orden (Newton-Raphson, Gauss-Newton), regularización basada en la traza del Hessiano, y meta-aprendizaje (donde se diferencia a través de un bucle de entrenamiento completo).

La segunda derivada de la pérdida respecto a un peso, $\partial^2 \mathcal{L}/\partial w^2$, mide la curvatura del paisaje de optimización. Una curvatura alta indica que pequeños cambios en el peso producen grandes cambios en la pérdida; una curvatura baja indica una región plana. Los optimizadores de segundo orden utilizan esta información para dar pasos adaptados a la geometría local, a diferencia de SGD que usa el mismo tamaño de paso en todas las direcciones.

Clasificación con JAX puro

Cambio de paradigma: funciones puras y pytrees

Implementamos un MLP completo en JAX puro (sin Flax) sobre el mismo conjunto `make_moons` usado en la sección de PyTorch. Esto permite una comparación directa del código y los resultados. La arquitectura es idéntica: $[2 \to 32 \to 32 \to 1]$.

En JAX puro, no existe un objeto "modelo" con estado interno. Los parámetros son diccionarios Python (pytrees) que se pasan explícitamente como argumentos. Esta disciplina funcional tiene ventajas: los parámetros son inspeccionables, se pueden serializar como dicts, y las transformaciones de JAX pueden recorrerlos automáticamente.

Inicialización de parámetros JAX

En PyTorch, la inicialización de pesos ocurre automáticamente dentro de `nn.Linear()`. En JAX puro, debes ser explícito y escribir tu propia función de inicialización.

¿Por qué inicialización explícita?

Porque en JAX **no hay objeto modelo con estado**. Los parámetros son solo datos que debes crear tú mismo. Esto parece verboso, pero te da control total.

PRNG management — ¿Por qué `jax.random.split`?

JAX requiere **determinismo puro**: la misma función con los mismos argumentos debe retornar exactamente lo mismo. Por eso no hay un generador global mutable como en NumPy — la aleatoriedad debe ser **explícita**.

> **Error frecuente: reusar la misma key.**
> En JAX, si llamas `jax.random.normal` dos veces con la **misma** `key`, obtienes el **mismo** resultado — no valores distintos:
> Para obtener valores independientes en cada capa, debes **dividir** la key:
> En un bucle de inicialización de capas, el patrón estándar es:

Función de inicialización con He initialization.

He Initialization — ¿por qué?

Sin inicialización cuidadosa, los parámetros pueden ser demasiado grandes o demasiado pequeños: - Si W es muy pequeño: las activaciones desaparecen (gradientes vanishing). - Si W es muy grande: las activaciones saturan o explotan.

He initialization escala cada matriz de pesos para mantener una varianza consistente. Para ReLU, usamos std = $\sqrt{2/\text{fan_in}}$, donde `fan_in` es el número de entradas a la capa.

Comparación: PyTorch vs JAX

Aspecto	PyTorch	JAX
Inicialización	Automática en `nn.Linear()`	Manual con `init_params()`
Acceso a parámetros	`model.parameters()`	`params` diccionario
Aleatoriedad	PRNG global implícito	PRNG key explícito
Ventaja	Menos código	Más control, reproducible

Forward pass y función de pérdida JAX

Forward pass — operaciones lineales + activaciones.

¿Por qué `.squeeze(-1)`?

La salida de la capa final antes del `squeeze` es shape `(batch, 1)`. En JAX, es convención que funciones de pérdida escalar (como binary cross-entropy) reciben vectores 1D `(batch,)` para compatibilidad con operaciones de reducción.

Comparación de shapes a lo largo del forward pass con entrada `X_tr_j` de shape `(480, 2)`:

```
Entrada x:                  (480, 2)
Despues de capa 1 + ReLU:  (480, 32)
Despues de capa 2 + ReLU:  (480, 32)
Despues de capa 3:         (480, 1)  ← logit antes de sigmoid
Despues de sigmoid:        (480, 1)
Despues de squeeze(-1):    (480,)    ← vector 1D de probabilidades
```

Loss function — binary cross-entropy.

La forma de las variables.

Cuando llamas `bce_loss_jax(params, X_tr_j, y_tr_j)`: - `x` tiene shape `(480, 2)` — 480 ejemplos, 2 features - `y` tiene shape `(480,)` — 480 etiquetas binarias - `yhat` tiene shape `(480,)` — 480 predicciones - `loss` es un escalar — el promedio de la pérdida de todos los ejemplos

Este es el valor que luego diferenciaras con `jax.grad()` para obtener los gradientes.

Preparación de datos JAX

Paso de entrenamiento con JIT compilation

Diferencia filosófica: PyTorch vs JAX.

PyTorch:

- El bucle es imperativo — instrucciones paso a paso.
- El estado del modelo y optimizador cambia en cada paso.
- Fácil de debuggear porque el estado es observable en cualquier momento.

JAX:

- Todo el paso está encapsulado en una función pura que retorna nuevos valores.
- Ningun estado mutable — los parámetros y estado del optimizador son argumentos de entrada y salida.
- Se puede compilar con JIT para velocidad.

El paso de entrenamiento JAX.

Desglose línea por línea.

```
loss, grads = jax.value_and_grad(bce_loss_jax)(params, x, y)
```

Esto es equivalente a lo que PyTorch hace en dos llamadas separadas:

```
updates, new_opt_state = optimizer_jax.update(grads, opt_state, params)
```

El optimizador Lee los gradientes e internamente mantiene momentos (moving averages). Para Adam, computa:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(first moment)}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(second moment)}$$

$$\text{update} = \frac{m_t}{\sqrt{v_t} + \epsilon}$$

Esto es todo lo que PyTorch hace en `optimizer.step()`, pero de forma explícita y retornando los valores.

```
new_params = optax.apply_updates(params, updates)
```

Aplica los updates:

$$\theta_{\text{new}} = \theta_{\text{old}} + \text{learning_rate} \times \text{updates}$$

¿Por qué `@jax.jit`?

El decorador `@jax.jit` compila la función con XLA la primera vez que se llama: - **Primera llamada:** JAX traza la función, observa las formas de entrada, y genera código XLA optimizado. Lento — puede tomar varios segundos. - **Llamadas subsiguientes con mismas formas:** Reutilizan el código compilado. Mucho más rápido.

Para datos de tamaño fijo (como en este ejemplo), la compilación ocurre una sola vez y los 299 epochs siguientes son significativamente más rápidos que en PyTorch eager.

Cuando `@jax.jit` sorprende — tracing y efectos secundarios.
Dentro de una función JIT-compilada, `print()` **no funciona como esperas**. JAX traza la función estáticamente, y `print` solo se ejecuta una vez durante el trazado, no en cada llamada: Para debuggear dentro de JIT, usa `jax.debug.print(...)` que sí propaga el valor real en cada ejecución.

Bucle de entrenamiento completo

En JAX puro, cada transformación es explícita: compilación (`@jax.jit`), diferenciación (`jax.grad`), vectorización (`jax.vmap`). Esto da total control pero requiere verbosidad. El optimizador `optax` es funcional: mantiene su estado en `opt_state` en lugar de modificar un objeto.

Las curvas de entrenamiento en JAX son prácticamente idénticas a las de PyTorch bajo las mismas condiciones: misma arquitectura, mismo optimizador (Adam con `lr=0.01`), mismo conjunto de datos y misma partición. Esto ilustra que los dos frameworks, a pesar de sus filosofías radicalmente distintas, convergen a resultados equivalentes cuando los hiperparámetros son los mismos. La diferencia observable es que la primera llamada a `train_step_jax` es más lenta (trazado y compilación JIT), pero las 299 iteraciones siguientes son sustancialmente más rápidas que su equivalente en PyTorch eager.

La frontera de decisión aprendida por la red JAX debería ser visualmente indistinguible de la del MLP de PyTorch, confirmando que el aprendizaje de representaciones no esta ligado al framework sino a la arquitectura y los datos.

La frontera aprendida por JAX captura la misma estructura curvilinea que la de PyTorch. Las pequeñas diferencias visuales, si las hay, se deben a diferencias en la inicialización aleatoria (JAX usa claves PRNG explicitas mientras que PyTorch usa un generador global) y posiblemente a diferencias en el orden de las operaciones de punto flotante producidas por XLA versus la cadena de herramientas de PyTorch. En ninguno de los dos casos estas diferencias son significativas para la calidad del clasificador final.

CNN con Flax

flax.linen.Module

Flax (Heek et al. 2023) ofrece una API de módulos sobre JAX que es **superficialmente** similar a `nn.Module` de PyTorch, pero mantiene la disciplina funcional en su núcleo.

La diferencia clave: una instancia de `flax.linen.Module` **no almacena parámetros**. Es solo una descripción del cómputo. Los parámetros se obtienen con `model.init(key, x)` y se pasan explícitamente en cada `model.apply(params, x)`.

El decorador `@nn.compact` permite declarar las capas inline en `__call__` en lugar de en `__init__`. Flax nombra automáticamente cada capa según su posición, produciendo claves como `"Conv_0"`, `"Dense_1"`.

Error frecuente al portar código PyTorch → Flax: el orden de canales.
PyTorch espera `(N, C, H, W)` — canales primero. JAX/Flax espera `(N, H, W, C)` — canales último.

Framework	Formato	Reshape correcto
PyTorch	`(N, 1, 8, 8)`	`.reshape(-1, 1, 8, 8)`
JAX/Flax	`(N, 8, 8, 1)`	`.reshape(-1, 8, 8, 1)`

Si olvidas este cambio, obtendrás un error de forma confuso en `nn.Conv` porque el número de canales no coincide con el esperado.

La CNN de Flax alcanza una exactitud de validación comparable a la versión de PyTorch en el mismo número de epochs, lo que confirma que la capacidad del modelo es equivalente. Una diferencia observable es que usamos `avg_pool` en lugar de `MaxPool2d`: el average pooling es ligeramente menos agresivo en la selección de características pero igualmente efectivo para reducir la dimensión espacial. En redes más profundas, la elección entre max y average pooling puede tener un impacto measurable en el rendimiento.

El flujo de trabajo con Flax merece un comentario adicional. La llamada `model_flax.apply(params_flax, x)` es una función pura: produce la misma salida para los mismos parámetros y la misma entrada, sin ninguna dependencia de estado oculto. Esto facilita la verificación formal, el testing y la reproducción de resultados. El precio es la verbosidad: hay que pasar `params_flax` explícitamente en cada llamada, lo que contrasta con el `model(x)` de PyTorch donde los parámetros están implícitamente encapsulados en el objeto.

Comparativa PyTorch vs JAX

Filosofía de diseño: Imperative vs Functional

La diferencia más profunda entre PyTorch y JAX no es técnica sino **filosófica de programación**. Estos dos frameworks encarnan dos paradigmas completamente diferentes.

PyTorch: Programación Imperativa Orientada a Objetos.

PyTorch sigue el paradigma imperativo: escribes instrucciones paso a paso sobre objetos que tienen estado mutable.

Ventajas: - Ergonomía excelente — el código se lee como una serie natural de instrucciones. - Debugging fácil — inspecciona el estado en cualquier momento con `print` o un debugger. - Curva de aprendizaje suave para programadores acostumbrados a OOP.

Desventajas: - No es composable con transformaciones automáticas (diferenciar través de múltiples pasos es complicado). - El estado mutable hace difícil razonar sobre que código hace que.

JAX: Programación Funcional Pura.

JAX implementa programación funcional: las funciones no tienen estado, siempre retornan lo mismo para los mismos argumentos, y se pueden transformar de forma automática.

Ventajas: - Composición de transformaciones — puedes anidar `jax.grad`, `jax.jit`, `jax.vmap` libremente. - Compilación automática con XLA — velocidad. - Determinismo garantizado — mismos argumentos → mismo resultado. - Paralelización automática posible.

Desventajas: - Curva de aprendizaje más empinada — requiere cambiar tu forma de pensar. - Debugging más difícil — no puedes poner `print` dentro de `@jax.jit`. - Más verboso — tienes que ser explícito con el estado (parámetros, keys PRNG, etc).

Tabla comparativa de mentalidad.

Aspecto	PyTorch	JAX
Modelo	Objeto con estado mutable	Función pura + datos externos
Parámetros	Encapsulados en `model.parameters()`	Datos explícitos (pytrees)
Ciclo de entrenamiento	Instrucciones imperativas	Transformación de datos
Debugging	Trivial — inspecciona en cualquier punto	Difícil dentro de `@jax.jit`
Composición	Limitada	Ilimitada
Velocidad	Eager por defecto, JIT opcional	JIT por defecto
Uso típico	La mayoría de practicionantes	Investigación, TPUs, órdenes altas

Flujo de trabajo típico

Veamos las operaciones más comunes lado a lado.

1. Definición del modelo.

PyTorch:

JAX / Flax:

2. Forward pass.

PyTorch:

JAX / Flax:

3. Cálculo de gradientes.

PyTorch:

JAX:

4. Optimizador.

PyTorch:

JAX:

5. Compilación.

PyTorch:

JAX:

6. Debugging.

PyTorch — fácil:

JAX — más complejo:

Resumen de flujos de trabajo.

Operación	PyTorch	JAX
Crear modelo	`MyNet()`	`MyNet()` + `model.init(key, x)`
Forward	`model(x)`	`model.apply(params, x)`
Gradientes	`loss.backward()`	`jax.grad(loss_fn)(params)`
Optimizador	`optimizer.step()`	`optimizer.update(grads, state)`
Compilación	`torch.compile()` (opcional)	`@jax.jit` (automática internamente)
Debugging	Trivial	Requiere técnicas especiales

¿Cuál elegir?

Regla práctica para elegir framework.
Empieza con PyTorch si:

- Estás aprendiendo aprendizaje profundo por primera vez.
- Necesitas modelos preentrenados (HuggingFace Transformers, torchvisión).
- Quieres debuggear con `print()` y puntos de ruptura normales.
- Trabajas en equipo donde PyTorch ya es el estándar.

Considera JAX si:

- Necesitas diferenciación de orden superior (Hessiano, derivadas de derivadas).
- Quieres vectorización automática eficiente con `jax.vmap`.
- Trabajas con TPUs de Google Cloud (JAX tiene integración nativa).

- Investigas meta-aprendizaje, NTK o algoritmos que diferencian a través del entrenamiento.

La realidad del mercado (2025): ~80 % del código de investigación publicado usa PyTorch. JAX es dominante en DeepMind/Google y en investigación teórica. Para la industria general, PyTorch es la elección casi universal.

Comparativa cuantitativa

La siguiente figura presenta una comparación subjetiva pero estructurada de las dos opciones principales en seis dimensiones relevantes para el practicante de aprendizaje profundo.

La comparativa revela los patrones complementarios de los dos frameworks. PyTorch destaca en facilidad de debugging, tamaño del ecosistema y accesibilidad para nuevos usuarios (curva de aprendizaje más suave). JAX destaca en velocidad de ejecución gracias a la compilación XLA, que produce código significativamente más rápido en hardware especializado. Ambos frameworks son equivalentes en flexibilidad de arquitecturas (cualquier cómputo diferenciable es expresable en ambos) y soporte para investigación (las dos opciones son ampliamente usadas en publicaciones de primer nivel). La puntuación baja de JAX en "Curva de aprendizaje" no refleja una deficiencia del framework sino la naturaleza del paradigma funcional, que requiere un cambio conceptual sustancial para programadores acostumbrados al estilo imperativo.

Convergencia directa: PyTorch vs JAX

La prueba definitiva de que ambos frameworks son equivalentes en capacidad expresiva es comparar sus curvas de convergencia sobre el mismo problema, con la misma arquitectura y los mismos hiperparámetros. La siguiente figura superpone la exactitud de validación de los dos modelos MLP entrenados a lo largo de este capitulo — el de PyTorch (azul solido) y el de JAX puro (verde discontinuo) — sobre `make_moons` con arquitectura $[2 \to 32 \to 32 \to 1]$, optimizador Adam con $\eta = 0.01$ y 300 épocas.

Ejercicios

Los siguientes ejercicios están diseñados para profundizar tu comprensión de PyTorch y JAX experimentando con variaciones de arquitecturas e hiperparámetros. El objetivo no es solo "hacer que funcione", sino entender **qué sucede** cuando cambias cosas.

Ejercicio 1: Sensibilidad a la tasa de aprendizaje

Concepto: La tasa de aprendizaje η es el hiperparámetro más crítico. Controla qué tan grandes son los pasos que toma el optimizador: - Demasiado bajo → Convergencia lentísima (podrías esperar para siempre). - Demasiado alto → Oscilaciones o divergencia (los pesos saltan sin control).

Tarea: Entrena el MLP de PyTorch con cuatro tasas diferentes: `lr [0.0005, 0.005, 0.05, 0.5]`. Gráfica las curvas de `val_loss` para las cuatro en la misma figura, asegurándote de usar colores distintos para cada una.

Pasos sugeridos: 1. Define una función `train_with_lr(lr, n_epochs=300)` que devuelva las listas `val_losses`. 2. Itera sobre `learning_rates` y acumula los resultados. 3. Gráfica con Plotly (puedes usar `go.Scatter` y agregar múltiples traces).

Observaciones esperadas: - Con `lr = 0.0005`: Curva plana durante muchos epochs — convergencia extremadamente lenta. - Con `lr = 0.005` y `lr = 0.05`: Curvas suaves y decrecientes — zona dorada. - Con `lr = 0.5`: Oscilaciones erráticas o divergencia — la pérdida aumenta en lugar de disminuir.

Pregunta de reflexión: ¿Por qué Adam es más robusto a tasas de aprendizaje altas que SGD simple?

Ejercicio 2: Efecto del Dropout en regularización

Concepto: Dropout es una técnica de regularización que desactiva aleatoriamente neuronas durante el entrenamiento. La idea es prevenir que el modelo "memorice" patrones raros y, en su lugar, aprenda características más robustas.

Pregunta: ¿Reduce realmente la brecha entre pérdida de entrenamiento y validación?

Tarea: Define una clase `MLPWithDropout` que agregue `nn.Dropout(p=0.3)` **después de cada activación ReLU** (excepto la última capa). Entrena tanto el MLP original como la versión con Dropout durante 300 epochs, y gráfica `train_loss` y `val_loss` para ambos modelos en la misma figura.

Pasos sugeridos: 1. Define `MLPWithDropout` con la arquitectura `[2 -> 32 -> 32 -> 1]` y `Dropout(0.3)` después de cada ReLU. 2. Entrena ambos modelos con la misma semilla y tasa de aprendizaje. 3. Gráfica las 4 curvas: `train_loss (sin dropout)`, `val_loss (sin dropout)`, `train_loss (con dropout)`, `val_loss (con dropout)`. 4. Calcula el gap final: `gap = val_loss_final - train_loss_final` para ambos modelos.

Observaciones esperadas: - **Sin Dropout:** La pérdida de entrenamiento desciende rápidamente pero el gap con validación puede ser grande (sobreajuste). - **Con Dropout:** Ambas curvas convergen más juntas porque el modelo está más regularizado.

Ejercicio 3: Cambiar el dataset: make_circles

Concepto: Diferentes datasets requieren diferentes arquitecturas. `make_circles` es fundamentalmente diferente a `make_moons`: los datos forman dos círculos concéntricos, no semilunares. Esto requiere capas más poderosas.

Pregunta: ¿Es suficiente una red pequeña `[32, 32]` o necesitamos una arquitectura más grande?

Tarea: 1. Crea el dataset: `X_c, y_c = make_circles(n_samples=600, noise=0.1, factor=0.5, random_state=42)` 2. Haz train/test split (80/20) como lo hiciste con `make_moons`. 3. Entrena **tres** modelos: - `MLP(2, [16, 16], 1)` — arquitectura pequeña - `MLP(2, [32, 32], 1)` — arquitectura mediana - `MLP(2, [64, 64], 1)` — arquitectura grande 4. Para cada modelo, calcula la exactitud final en validación. 5. Visualiza las tres fronteras de decisión lado a lado usando tres subplots.

Observaciones esperadas: - `[16, 16]` - Posiblemente insuficiente; el modelo podría no capturar bien la estructura radial. - `[32, 32]` - Debería funcionar bastante bien. - `[64, 64]` - Captura los círculos con mayor precisión.

Ejercicio 4: Derivadas de orden superior en JAX — Una exclusiva de JAX

Concepto: JAX permite componer `jax.grad` de forma ilimitada. Puedes calcular: - Primera derivada: `grad1 = jax.grad(f)` - Segunda derivada: `grad2 = jax.grad(jax.grad(f))` - Tercera, cuarta, … — tantas como quieras.

Esto es **casi imposible** en PyTorch sin configuración especial, pero en JAX es completamente natural.

Por qué importa: Las derivadas de segundo orden (Hessiano) son fundamentales para: - Optimizadores de segundo orden (Newton-Raphson). - Análisis de curvatura del paisaje de optimización. - Meta-aprendizaje (diferenciar a través de un bucle de entrenamiento completo).

Tarea: Define la pérdida BCE con un solo peso escalar: `bce_escalar(w)` con entrada fija `x=1` y etiqueta `y=1`. Luego: 1. Calcula `grad1 = jax.grad(bce_escalar)` — primera derivada. 2. Calcula `grad2 = jax.grad(grad1)` — segunda derivada (Hessiano). 3. Evalúa las tres funciones para `w [-5, 5]` con 300 puntos. 4. Gráfica en tres subplots: - Arriba: Pérdida `L(w)` - Medio: Primera derivada `dL/dw` - Abajo: Segunda derivada d^2L/dw^2

Observaciones esperadas: - La pérdida es **convexa** — forma de U. - El gradiente es **siempre negativo** para `y=1` — descenso por gradiente siempre incrementa `w`. - La curvatura es **máxima en `w=0`** — máxima incertidumbre. - En $w \to \pm\infty$, la curvatura se aproxima a cero — la función se vuelve plana.

Pregunta de reflexión: En un optimizador de segundo orden (como Newton-Raphson), usarías la Hessiana para dar pasos más grandes donde la curvatura es pequeña y pasos más pequeños donde es grande. ¿Tiene sentido esto intuitivamente? ¿Por qué?

Resumen

Este capítulo presentó la implementación práctica de redes neuronales en tres frameworks — scikit-learn, PyTorch y JAX+Flax — con énfasis en entender *por qué* funciona cada pieza, no solo *cómo* usarla.

PyTorch — el patrón central:

JAX — el patrón central:

La diferencia filosófica que importa:

	PyTorch	JAX
Estado del modelo	Dentro del objeto	Datos externos (pytree)
Gradientes	Acumulan en `.grad`	Valores de retorno
Aleatoriedad	Generador global	PRNG key explícita
Compilación	Eager por defecto	`@jax.jit` explícito

	PyTorch	JAX
Debugging	Trivial	Requiere `jax.debug`

Los experimentos confirman que ambos frameworks producen modelos equivalentes con la misma arquitectura y optimizador. La elección es principalmente una decisión de ergonomía y ecosistema (PyTorch) versus expresividad funcional y velocidad en hardware especializado (JAX).

Referencias

Parte XXXIII.

Práctica Integradora

66. Clasificadores Supervisados: Experimento Práctico

Introducción

La clasificación supervisada es una de las tareas centrales del aprendizaje automático. Dado un conjunto de observaciones etiquetadas $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$, donde $\mathbf{x}_i \in \mathbb{R}^d$ es el vector de características y $y_i \in \{0, 1, \ldots, K-1\}$ es la clase objetivo, el problema de clasificación consiste en aprender una función $f : \mathbb{R}^d \to \mathcal{Y}$ que generalice correctamente a nuevas observaciones no vistas durante el entrenamiento.

Es el cierre de la parte de Clasificación: reúne en un solo experimento lo estudiado en los capítulos anteriores. La referencia mínima y el control de la complejidad vienen del capítulo introductorio; las métricas y el umbral, del capítulo de métricas; la validación cruzada sin fuga de datos, del capítulo de validación; y los modelos, de los capítulos de árboles, ensambles, modelos lineales y redes neuronales. Conviene leerlo como una lista de verificación aplicada.

Este capítulo no busca derivar los fundamentos teóricos de cada algoritmo desde cero, sino mostrar cómo se diseña, ejecuta y evalúa un experimento de clasificación real. El énfasis está en los retos prácticos: cómo preparar los datos, cómo evitar errores comunes, cómo comparar modelos de forma justa y cómo interpretar los resultados en función del contexto del problema.

Dataset: Breast Cancer Wisconsin

El dataset Breast Cancer Wisconsin (**wolberg1995breast?**) contiene 569 muestras de biopsias de tejido mamario, cada una descrita por 30 variables numéricas derivadas de imágenes de células. La variable objetivo indica si el tumor es maligno (0) o benigno (1). Este dataset tiene varias propiedades didácticas deseables:

- Está disponible directamente en scikit-learn, sin necesidad de descarga externa
- Todas las variables son numéricas, lo que simplifica el preprocesamiento
- El contexto médico es motivador: los errores tienen consecuencias asimétricas
- El desbalance moderado (37 % maligno, 63 % benigno) permite discutir la elección de métricas

Un aspecto importante es identificar correctamente la clase positiva. Por convención en scikit-learn, `target=1` corresponde a benigno y `target=0` a maligno. Cuando se reportan métricas como precision y recall, es necesario especificar con respecto a qué clase se calculan, ya que los valores pueden cambiar sustancialmente dependiendo de esta elección.

Flujo general de un experimento de clasificación

Un experimento bien diseñado sigue estos pasos:

1. **Entender el problema** y la variable objetivo
2. **Explorar y preparar los datos**
3. **Dividir correctamente** en entrenamiento y prueba
4. **Elegir modelos** y configurar hiperparámetros iniciales
5. **Entrenar** usando únicamente los datos de entrenamiento
6. **Evaluar** con métricas coherentes con el objetivo del problema
7. **Comparar modelos** con la misma partición de datos
8. **Ajustar thresholds** si el contexto lo requiere
9. **Interpretar resultados** y reconocer limitaciones
10. **Documentar** el flujo para que sea reproducible

Cada uno de estos pasos involucra decisiones que afectan la validez del experimento. En las secciones siguientes se desarrolla cada uno con detalle.

Preparación de datos

Separación entrenamiento/prueba

La regla fundamental es que el conjunto de prueba no debe usarse de ninguna manera durante el diseño, el entrenamiento o el ajuste de los modelos. Usar información del test set para tomar decisiones de diseño introduce **data leakage**, que produce una evaluación optimista que no se replicará en datos reales.

El parámetro `stratify=y` garantiza que la proporción de clases se mantiene tanto en train como en test, lo que es especialmente importante con datasets desbalanceados.

Pipelines para preprocesamiento sin leakage

La forma correcta de aplicar transformaciones como el escalamiento es dentro de un `Pipeline` de scikit-learn (Pedregosa et al. 2011). Un pipeline asegura que el ajuste de cualquier transformación (por ejemplo, `StandardScaler`) se realiza únicamente sobre los datos de entrenamiento, y la misma transformación ya ajustada se aplica automáticamente a los datos de prueba.

El error clásico de data leakage por escalamiento ocurre cuando se transforma todo el dataset antes del split:

Validación cruzada

Cuando el tamaño del dataset es limitado, la evaluación con una sola partición train/test puede ser inestable. La validación cruzada k-fold (Kohavi 1995) divide el conjunto de entrenamiento en k partes (*folds*), entrena k veces usando $k-1$ folds y evalúa en el fold restante, promediando los resultados.

La validación cruzada se usa para seleccionar hiperparámetros, no para evaluar el rendimiento final: el test set sigue siendo el árbitro final del rendimiento.

Clasificadores: intuición, sesgos y hiperparámetros

Regresión Logística (baseline)

La regresión logística modela la probabilidad de pertenencia a la clase positiva mediante una función sigmoide aplicada a una combinación lineal de las variables:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

Es el baseline lineal interpretable por excelencia. Sus coeficientes se pueden inspeccionar directamente para entender la contribución de cada variable. **Requiere escalamiento** para que el optimizador converja adecuadamente y para que los coeficientes sean comparables.

Hiperparámetros principales:

- `C`: inverso de la regularización (mayor C → menos regularización)
- `penalty`: tipo de regularización (`l1`, `l2`, `elasticnet`)
- `max_iter`: número máximo de iteraciones del optimizador

Árboles de decisión: CART e ID3

CART (Breiman et al. 1984).

CART (*Classification and Regression Trees*) construye árboles binarios mediante particiones recursivas del espacio de características. En cada nodo, selecciona el atributo y el umbral de corte que minimizan la impureza del nodo resultante. La impureza Gini se define como:

$$G = 1 - \sum_{k=0}^{K-1} p_k^2$$

donde p_k es la proporción de muestras de la clase k en el nodo.

ID3 (quinlan1986id3?).

ID3 (*Iterative Dichotomiser 3*) utiliza la ganancia de información para seleccionar el atributo de partición:

$$\text{Ganancia}(S, A) = H(S) - \sum_{v \in \text{Valores}(A)} \frac{|S_v|}{|S|} H(S_v)$$

donde $H(S) = -\sum_k p_k \log_2 p_k$ es la entropía del conjunto S. En scikit-learn, ID3 se aproxima con `criterion="entropy"` en `DecisionTreeClassifier`.

En la práctica, la diferencia entre Gini y entropía es pequeña en la mayoría de los datasets. La entropía tiende a producir árboles ligeramente más profundos en datasets con muchas categorías nominales.

Hiperparámetros principales de los árboles:

- `max_depth`: profundidad máxima (controla sobreajuste)
- `min_samples_split`: mínimo de muestras para dividir un nodo
- `min_samples_leaf`: mínimo de muestras en una hoja

Random Forest (Breiman 2001b)

Random Forest es un ensamble de árboles de decisión construidos en paralelo mediante *bootstrap aggregating* (bagging). Cada árbol se entrena sobre una muestra con reemplazo del conjunto de entrenamiento, y en cada nodo solo se considera un subconjunto aleatorio de variables (`max_features`). El resultado final es la moda o el promedio de las predicciones de todos los árboles.

La aleatorización reduce la varianza del modelo compuesto, ya que los árboles individuales son decorrelacionados entre sí. Random Forest no requiere escalamiento y es relativamente robusto a hiperparámetros.

Hiperparámetros principales:

- `n_estimators`: número de árboles (más árboles → menor varianza, pero mayor costo)
- `max_features`: fracción de variables consideradas por árbol (`sqrt`, `log2`, fracción)
- `max_depth`: profundidad máxima de cada árbol
- `min_samples_leaf`: mínimo de muestras en hojas

Gradient Boosting (friedman2001gradient?)

Gradient Boosting construye un ensamble de modelos de forma **secuencial**. Cada modelo nuevo se entrena para corregir los errores (residuales) del ensamble acumulado hasta ese momento. El algoritmo puede verse como un descenso por gradiente en el espacio de funciones, donde la función de pérdida (por ejemplo, log-loss para clasificación) guía la dirección de corrección.

A diferencia de Random Forest, Gradient Boosting es sensible a la tasa de aprendizaje (`learning_rate`) y a la profundidad de los árboles individuales. Una tasa de aprendizaje baja requiere más estimadores para alcanzar la misma capacidad, pero suele generalizar mejor.

XGBoost (Chen y Guestrin 2016) es una implementación optimizada de Gradient Boosting que añade regularización explícita, manejo eficiente de valores faltantes y aceleración mediante paralelización de la construcción de árboles.

Hiperparámetros principales:

- `n_estimators`: número de árboles secuenciales
- `learning_rate`: paso del descenso por gradiente (típicamente 0.01–0.3)
- `max_depth`: profundidad de cada árbol (típicamente 3–6)
- `subsample`: fracción del dataset usada por árbol

Red neuronal multicapa (MLP)

Una red neuronal multicapa (MLP, *Multilayer Perceptron*) aproxima funciones mediante composición de transformaciones afines y no lineales (Bishop 2006; Goodfellow et al. 2016):

$$\mathbf{h}^{(l)} = \sigma\big(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\big)$$

donde σ es la función de activación (ReLU, tanh, sigmoide), $\mathbf{W}^{(l)}$ son los pesos de la capa l y $\mathbf{h}^{(0)} = \mathbf{x}$ es la entrada. Los pesos se optimizan mediante retropropagación del gradiente.

Las redes neuronales son modelos muy flexibles, pero son más sensibles que los árboles a los hiperparámetros y al escalamiento de las variables de entrada. Si las entradas tienen escalas muy distintas, los gradientes pueden ser inestables y el optimizador converge lentamente o a mínimos deficientes.

Hiperparámetros principales en MLPClassifier:

- `hidden_layer_sizes`: arquitectura de capas ocultas (p.ej., `(64, 32)`)
- `activation`: función de activación (`relu`, `tanh`)
- `alpha`: regularización L2
- `max_iter`: épocas máximas de entrenamiento

Recordatorio de métricas

Las métricas de clasificación se derivan de la matriz de confusión, que organiza las predicciones en cuatro categorías:

	Predicho Positivo	Predicho Negativo
Real Positivo	TP	FN
Real Negativo	FP	TN

Las métricas principales son:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall (Sensibilidad)} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

(Powers 2011; Pedregosa et al. 2011)

Curva ROC y AUC

La curva ROC (*Receiver Operating Characteristic*) (Fawcett 2006) representa la tasa de verdaderos positivos (recall) en función de la tasa de falsos positivos para todos los umbrales de decisión posibles:

$$\text{TPR} = \frac{TP}{TP + FN} \qquad \text{FPR} = \frac{FP}{FP + TN}$$

El área bajo la curva ROC (AUC, *Area Under the Curve*) resume el rendimiento del clasificador en todos los thresholds. Un clasificador perfecto tiene AUC = 1.0; un clasificador aleatorio tiene AUC = 0.5.

Curva Precision-Recall y Average Precision

La curva Precision-Recall (Saito y Rehmsmeier 2015) representa precision en función de recall para todos los thresholds. Es más informativa que la curva ROC cuando la clase positiva es rara, ya que la curva ROC puede ser engañosamente optimista en presencia de muchos verdaderos negativos.

La métrica *Average Precision* (AP) es el área bajo la curva Precision-Recall, calculada como:

$$\text{AP} = \sum_{k} (R_k - R_{k-1}) \cdot P_k$$

donde P_k y R_k son la precisión y el recall para el threshold k.

Selección de métricas según el contexto

La elección de la métrica de evaluación depende del contexto del problema:

- **Clases balanceadas:** accuracy puede ser una métrica razonable
- **Clases desbalanceadas:** F1-score, ROC-AUC y AP son más informativas
- **Falsos negativos costosos** (p.ej., diagnóstico de enfermedades): maximizar recall
- **Falsos positivos costosos** (p.ej., spam, fraude con revisión manual costosa): maximizar precision
- **Balance entre ambos:** F1-score o F-beta con β ajustado al costo relativo
- **Análisis del comportamiento para todos los thresholds:** curvas ROC o Precision-Recall

---

Ajuste de thresholds de decisión

Motivación

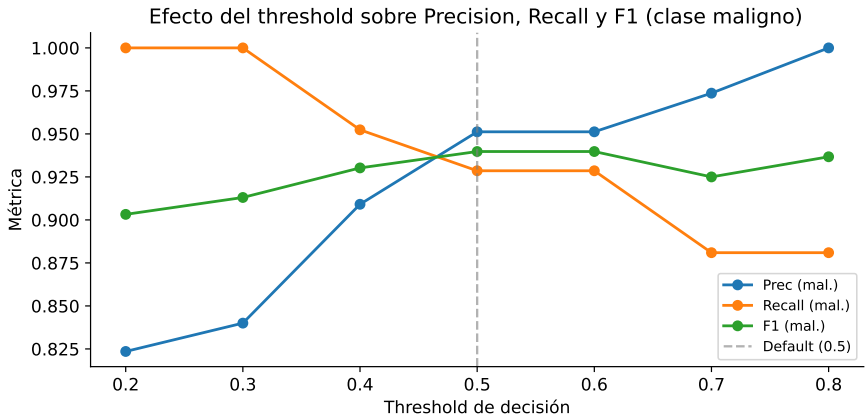
Por defecto, `predict()` clasifica una observación como positiva si su probabilidad estimada supera 0.5. Sin embargo, este umbral es arbitrario y no necesariamente óptimo para el objetivo del problema. En diagnóstico médico, donde un falso negativo puede tener consecuencias graves, puede convenir bajar el threshold para aumentar el recall a costa de reducir la precision.

predict_proba y decision_function

Los clasificadores de scikit-learn exponen las probabilidades estimadas mediante `predict_proba()`. Para algunos modelos (como SVM), también está disponible `decision_function()`, que produce un *score* continuo cuyo signo determina la clase predicha.

Comparación de thresholds

Para clasificar una observación como maligna usamos $P(\text{maligno}) \geq \theta$, donde θ es el threshold. Al reducir θ, clasificamos más observaciones como malignas, lo que aumenta el recall pero reduce la precision.



Consideraciones prácticas sobre el threshold

- El threshold óptimo depende del **costo relativo** de FP y FN, no de las métricas en el test set
- El threshold debe elegirse sobre un **conjunto de validación**, no sobre el test set
- La curva ROC y la curva Precision-Recall son herramientas para analizar el comportamiento del clasificador en **todos** los thresholds posibles antes de comprometerse con uno

---

Comparación entre modelos

La comparación válida entre modelos requiere que todos sean evaluados sobre el **mismo conjunto de prueba**, con el **mismo preprocesamiento** y el **mismo criterio de evaluación**.

Buenas prácticas para evaluación

Las siguientes prácticas garantizan que los resultados del experimento sean válidos, reproducibles e interpretables:

1. **Usar pipelines** que encapsulen preprocesamiento y modelo, evitando leakage
2. **Fijar semillas aleatorias** con `random_state` en todos los modelos y la partición
3. **Usar `stratify`** al dividir el dataset cuando las clases están desbalanceadas
4. **Comparar modelos con la misma partición** y el mismo preprocesamiento
5. **Reportar múltiples métricas**, eligiendo las más relevantes para el contexto del problema
6. **Usar validación cruzada** para seleccionar hiperparámetros, no el test set
7. **No usar el test set repetidamente** para ajustar decisiones de diseño
8. **Analizar la matriz de confusión** junto con las métricas agregadas
9. **Verificar el balance de clases** antes de elegir las métricas de evaluación
10. **Documentar** todos los parámetros, versiones de librerías y semillas usadas

Errores comunes en experimentos de clasificación

A continuación se describen los errores más frecuentes al diseñar y ejecutar experimentos de clasificación:

Evaluación en datos de entrenamiento. Evaluar el modelo sobre los mismos datos con los que fue entrenado produce resultados artificialmente optimistas que no reflejan la capacidad de generalización.

Data leakage por escalamiento. Ajustar el scaler sobre todo el dataset (incluyendo test) antes del split permite que información del test set "contamine" el entrenamiento. La solución es usar pipelines.

Evaluar solo con accuracy. En datasets con desbalance de clases, un modelo que siempre predice la clase mayoritaria puede tener una accuracy alta pero un rendimiento nulo en la clase minoritaria. Métricas como F1-score, ROC-AUC y AP son más informativas en estos casos.

No revisar el balance de clases. Ignorar el desbalance de clases puede llevar a elegir métricas incorrectas y a modelos con rendimiento deficiente en la clase minoritaria.

Comparar modelos con particiones distintas. Si dos modelos se evalúan sobre particiones distintas, las diferencias observadas pueden deberse al azar, no a diferencias reales de rendimiento.

No usar `random_state`. Los resultados no son reproducibles y no pueden ser comparados entre ejecuciones.

Redes neuronales sin escalamiento. Las redes neuronales son sensibles a la escala de las variables de entrada. Sin escalamiento, el optimizador puede converger lentamente o a un mínimo deficiente.

Ajustar hiperparámetros mirando el test set. Si se usa el test set para elegir hiperparámetros, el modelo está implícitamente siendo ajustado sobre el test, lo que produce una evaluación optimista.

Interpretar AUC sin contexto. Un AUC alto no garantiza que el modelo sea útil para el problema si el threshold elegido produce demasiados falsos negativos o falsos positivos.

Retos prácticos del flujo de clasificación

Más allá de los errores técnicos, existen retos conceptuales y de diseño que aparecen en cualquier proyecto de clasificación real:

Identificación de la clase positiva. La elección de qué clase se considera "positiva" afecta directamente el cálculo de precision, recall y F1. Esta decisión debe tomarse en función del objetivo del problema.

Costo asimétrico de los errores. En muchos problemas reales, un falso negativo no es equivalente a un falso positivo. El diseño del experimento debe reflejar estas asimetrías en la elección de métricas y thresholds.

Escalas heterogéneas. Variables con escalas muy distintas afectan a modelos sensibles como regresión logística, KNN y redes neuronales. Los árboles de decisión y Random Forest son invariantes al escalamiento.

Variables categóricas y numéricas mixtas. Cuando el dataset contiene ambos tipos de variables, es necesario usar `ColumnTransformer` para aplicar transformaciones distintas a cada grupo.

Reproducibilidad del experimento. Un experimento reproducible requiere fijar semillas, documentar versiones de librerías y guardar el estado del pipeline entrenado.

Selección del threshold en producción. El threshold óptimo para producción debe decidirse con datos de validación, considerando el costo relativo de FP y FN en el contexto del negocio o la aplicación.

Conclusión

Este capítulo ha mostrado que diseñar un experimento de clasificación correcto es más complejo que simplemente llamar `.fit()`. Las decisiones sobre la partición de datos, el preprocesamiento, la elección de métricas, la comparación entre modelos y el ajuste de thresholds tienen un impacto directo sobre la validez y la utilidad de los resultados.

Los modelos estudiados — regresión logística, CART, ID3, Random Forest, Gradient Boosting y MLP — representan distintos sesgos inductivos y compromisos entre interpretabilidad, flexibilidad y rendimiento. Ninguno es universalmente superior; la elección del modelo depende del dataset, el objetivo del problema y los recursos disponibles.

La práctica rigurosa de clasificación supervisada combina conocimiento algorítmico, diseño experimental cuidadoso e interpretación crítica de los resultados.

Parte XXXIV.

Aprendizaje No Supervisado

Parte XXXV.

Fundamentos del Aprendizaje No Supervisado

67. Aprendizaje no supervisado

En la parte de clasificación cada ejemplo de entrenamiento traía una **etiqueta**: el cliente pagó o no pagó, el correo era spam o no lo era. El trabajo del algoritmo consistía en aprender la relación entre las variables predictoras y esa etiqueta. En muchos problemas reales, sin embargo, la etiqueta **no existe**. Una cadena de tiendas tiene millones de tickets, pero nadie ha escrito junto a cada cliente «este es un comprador ocasional de alto gasto». Un banco registra transacciones, pero casi ninguna viene marcada como fraude. Un laboratorio mide la expresión de veinte mil genes en cada célula sin saber de antemano cuántos tipos celulares hay.

El **aprendizaje no supervisado** reúne los métodos que buscan **estructura** en datos sin variable objetivo: grupos de casos parecidos, direcciones en las que los datos varían, regiones donde se concentran, casos que no se parecen a ningún otro. Ya conocemos dos ejemplos. El análisis de componentes principales (PCA), del capítulo de preprocesamiento, encuentra las direcciones de máxima varianza sin mirar ninguna etiqueta, y la extracción de patrones frecuentes descubre combinaciones de productos que se compran juntos. Esta parte del libro estudia la tarea no supervisada más conocida, el **agrupamiento** o *clustering*, y la completa con la detección de anomalías y la reducción de dimensionalidad no lineal.

La pregunta que guía este capítulo es incómoda: **si no hay una respuesta correcta contra la cual comparar, ¿cómo sabemos que encontramos algo real?** La responderemos con un ejemplo de ocho clientes que se puede resolver a mano y con un laboratorio que muestra cómo una decisión aparentemente técnica, la escala de las variables, cambia por completo los grupos que «existen» en los datos.

Supervisado y no supervisado

Tabla 67.1.: Diferencias entre aprendizaje supervisado y no supervisado.

	Supervisado	No supervisado
Datos	pares $(\mathbf{x}_i, y_i)$	solo $\mathbf{x}_i$
Pregunta	¿qué valor de y corresponde a un $\mathbf{x}$ nuevo?	¿qué estructura tienen los $\mathbf{x}$?
Objeto que se estima	$P(y \mid \mathbf{x})$ o una regla $f(\mathbf{x})$	$p(\mathbf{x})$, una partición, una representación
Evaluación	error en datos de prueba	criterios internos, estabilidad, utilidad
Ejemplo	predecir si un cliente abandonará	descubrir segmentos de clientes

La diferencia de la última fila de la tabla es la más importante. En clasificación, el error en un conjunto de prueba es un juez imparcial: si el modelo memorizó en lugar de aprender, el error de prueba lo delata. En aprendizaje no supervisado **no hay error de prueba**. Un algoritmo de clustering siempre entrega grupos, aunque los datos sean ruido uniforme, y ninguna métrica calculada solo con los datos puede decidir por sí misma si esos grupos son útiles. Por eso esta parte insiste tanto en la validación, en la estabilidad y en la interpretación.

Entre ambos extremos existen variantes intermedias: el aprendizaje **semisupervisado**, con pocas etiquetas y muchos datos sin etiquetar, y el **autosupervisado**, que fabrica etiquetas a partir de los propios datos (por ejemplo, ocultar una palabra de una oración y pedir al modelo que la adivine). No las tratamos aquí, pero se apoyan en las ideas de esta parte.

Las tareas del aprendizaje no supervisado

Agrupamiento (*clustering*). Dividir los casos en grupos de modo que los casos de un mismo grupo se parezcan entre sí más que a los de otros grupos. Puede ser una partición **dura** (cada caso en exactamente un grupo, como k-medias), **blanda** (cada caso con una probabilidad de pertenecer a cada grupo, como las mezclas gaussianas), **jerárquica** (grupos anidados dentro de grupos) o **basada en densidad** (grupos como regiones densas separadas por zonas vacías, con casos que no pertenecen a ningún grupo).

Estimación de densidad. Estimar la distribución $p(\mathbf{x})$ que generó los datos[1]. El histograma es el estimador más simple; el estimador de **núcleo** (*kernel density estimation*, KDE) lo suaviza colocando una pequeña campana sobre cada observación,

$$\hat{p}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\Big(\frac{x - x_i}{h}\Big), \tag{67.1}$$

donde K es una densidad simétrica (por ejemplo la normal estándar) y h es el **ancho de banda**, que cumple el papel de k en k-NN: un h pequeño produce una densidad llena de picos y uno grande la aplana. Las **mezclas gaussianas**, estudiadas en la parte de clasificación, son estimadores de densidad paramétricos, y DBSCAN define sus grupos a partir de una estimación de densidad.

Reducción de dimensionalidad. Representar cada caso con pocas variables nuevas que conserven lo esencial: la varianza (PCA), las distancias (escalamiento multidimensional) o los vecindarios (t-SNE, UMAP). Sirve para visualizar, para quitar ruido y como paso previo a otros algoritmos.

Detección de anomalías. Encontrar casos que no se parecen al resto: transacciones fraudulentas, fallas de un sensor, errores de captura. Es, en cierto sentido, la tarea inversa del clustering: interesa lo que **no** forma grupo.

Patrones frecuentes. Combinaciones de valores que aparecen juntas con frecuencia, como en la parte de extracción de patrones. Es aprendizaje no supervisado sobre datos transaccionales.

[1] Una **densidad de probabilidad** $p(\mathbf{x})$ describe cómo se reparten los datos en el espacio: donde p es alta hay muchos casos, donde es baja hay pocos. La probabilidad de que un caso caiga en una región es la integral de p sobre esa región. En clasificación nos interesaba $P(y \mid \mathbf{x})$; aquí, a falta de y, el objeto de interés es $p(\mathbf{x})$ mismo.

Intuición: ocho clientes, ¿cuántos grupos?

Una tienda registra, para ocho clientes, el número de **visitas** al mes y el **ticket promedio** en pesos.

Tabla 67.2.: Ocho clientes de una tienda.

Cliente	1	2	3	4	5	6	7	8
Visitas al mes	2	3	2	4	10	11	12	9
Ticket promedio (pesos)	900	1 000	300	250	950	1 050	280	320

Mirando la tabla con calma se reconocen **cuatro** tipos de cliente: ocasionales de ticket alto (1 y 2), ocasionales de ticket bajo (3 y 4), frecuentes de ticket alto (5 y 6) y frecuentes de ticket bajo (7 y 8). Para el área de mercadotecnia esos cuatro segmentos piden estrategias distintas: a los frecuentes de ticket bajo conviene ofrecerles productos complementarios; a los ocasionales de ticket alto, motivos para volver. Nadie le dijo a la tienda que hubiera cuatro grupos; la estructura está en los datos. Veamos si un algoritmo la encuentra.

Construcción paso a paso

Paso 1. ¿Quién se parece a quién? Distancias en unidades originales

La forma natural de medir el parecido entre dos clientes es la distancia euclidiana entre sus vectores. Entre los clientes 1 y 3, que visitan la tienda lo mismo pero gastan muy distinto,

$$d(\mathbf{x}_1, \mathbf{x}_3) = \sqrt{(2-2)^2 + (900-300)^2} = 600,$$

y entre los clientes 1 y 5, que gastan parecido pero visitan la tienda con frecuencia muy distinta,

$$d(\mathbf{x}_1, \mathbf{x}_5) = \sqrt{(2-10)^2 + (900-950)^2} = \sqrt{64 + 2\,500} \approx 50.6.$$

Según esta distancia, el cliente 1 se parece **doce veces más** al cliente 5 que al 3. La diferencia de ocho visitas aporta 64 a la suma de cuadrados; la de 50 pesos aporta 2 500. Las visitas casi no cuentan porque se miden en una escala con números pequeños. Si sumamos los cuadrados de las desviaciones respecto a la media de cada variable, el ticket aporta 960 487.5 y las visitas 127.9: el **99.99 %** de la variación total que «ve» la distancia viene del ticket.

Paso 2. Un algoritmo de agrupamiento en unidades originales

El algoritmo de k-medias, que estudiaremos en detalle más adelante, busca k grupos que minimicen la **suma de cuadrados dentro de los grupos** (WCSS),

$$W = \sum_{c=1}^{k} \sum_{i \in C_c} \|\mathbf{x}_i - \mu_c\|^2,$$

donde μ_c es el promedio de los clientes del grupo c. Con ocho clientes podemos revisar **todas** las particiones posibles y quedarnos con la mejor. En unidades originales, la mejor partición en dos grupos es $\{1, 2, 5, 6\}$ y $\{3, 4, 7, 8\}$: ticket alto contra ticket bajo, con las visitas ignoradas. Hasta aquí parece razonable. Pero la mejor partición en **cuatro** grupos es $\{1, 5\}$, $\{2, 6\}$, $\{3, 8\}$ y $\{4, 7\}$: el algoritmo junta al cliente 1, que viene dos veces al mes, con el 5, que viene diez. Sigue partiendo **solo por el ticket**, ahora en rebanadas más finas. Los cuatro segmentos que vimos a simple vista no aparecen.

Paso 3. Estandarizar

Para que ambas variables cuenten lo mismo, las **estandarizamos**[2]: a cada valor le restamos la media de su variable y lo dividimos entre su desviación estándar. Las visitas tienen media 6.625 y desviación 4.00; el ticket, media 631.25 y desviación 346.5. El cliente 1 queda en

$$z_{\text{visitas}} = \frac{2 - 6.625}{4.00} = -1.16, \qquad z_{\text{ticket}} = \frac{900 - 631.25}{346.5} = 0.78.$$

Tabla 67.3.: Clientes estandarizados (desviación estándar poblacional).

Cliente	1	2	3	4	5	6	7	8
z_{visitas}	−1.16	−0.91	−1.16	−0.66	0.84	1.09	1.34	0.59
z_{ticket}	0.78	1.06	−0.96	−1.10	0.92	1.21	−1.01	−0.90

Ahora $d(\mathbf{z}_1, \mathbf{z}_3) = 1.73$ y $d(\mathbf{z}_1, \mathbf{z}_5) = 2.01$: el cliente 1 queda más cerca del 3 que del 5, y su vecino más cercano es el 2, a 0.38. Ambas variables pesan lo mismo en la distancia.

Paso 4. Agrupar en la escala estandarizada

Repitiendo la búsqueda exhaustiva sobre los datos estandarizados se obtienen las mejores particiones para cada número de grupos:

[2]La **estandarización** o puntuación z transforma cada variable a media 0 y desviación estándar 1: $z = (x - \bar{x})/s$. Aquí usamos la desviación estándar **poblacional** (dividiendo entre n), que es la que usa `StandardScaler` de `scikit-learn`. Después de estandarizar, una diferencia de una unidad significa lo mismo en todas las variables: una desviación estándar.

Tabla 67.4.: Mejores particiones de los clientes estandarizados y su suma de cuadrados dentro de los grupos.

k	Mejor partición	W
1	$\{1, \dots, 8\}$	16.00
2	$\{1, 2, 5, 6\}$, $\{3, 4, 7, 8\}$	8.12
3	$\{1, 2\}$, $\{5, 6\}$, $\{3, 4, 7, 8\}$	4.09
4	$\{1, 2\}$, $\{3, 4\}$, $\{5, 6\}$, $\{7, 8\}$	0.57
5	$\{1, 2\}$, $\{3, 4\}$, $\{5, 6\}$, $\{7\}$, $\{8\}$	0.28

Con $k = 4$ aparecen exactamente los cuatro segmentos. Con datos estandarizados, W para $k = 1$ vale 16, el número de clientes por el número de variables, porque cada variable estandarizada tiene varianza 1.

Paso 5. ¿Cuántos grupos? Una decisión, no un cálculo

W **siempre baja** al aumentar k: con ocho grupos, uno por cliente, valdría cero. No podemos elegir k minimizando W. Lo que sí revela la tabla es *dónde* deja de valer la pena agregar grupos: pasar de 3 a 4 grupos reduce W de 4.09 a 0.57, una caída enorme; pasar de 4 a 5 solo la reduce de 0.57 a 0.28, porque parte un segmento que ya era compacto. Esa rodilla en la curva de W contra k es el **método del codo**, y sugiere $k = 4$.

Observa además la fila $k = 2$. La partición por ticket tiene $W = 8.12$, pero la partición por **visitas**, $\{1, 2, 3, 4\}$ y $\{5, 6, 7, 8\}$, tiene $W = 8.46$, casi lo mismo. Ambas son descripciones legítimas de los datos («gastan mucho o poco», «vienen mucho o poco») y el criterio numérico apenas las distingue. Cuál conviene depende de **para qué** se quieren los grupos, algo que ningún algoritmo sabe.

Lo que enseña el ejemplo

1. **La representación decide.** Con las mismas ocho filas, el agrupamiento en unidades originales y el agrupamiento estandarizado describen mundos distintos. Elegir variables, unidades y escala es elegir qué significa «parecido».
2. **Un algoritmo siempre responde.** Pedimos cuatro grupos en unidades originales y obtuvimos cuatro grupos, aunque fueran absurdos. Ningún mensaje de error nos avisó.
3. **El número de grupos es una decisión informada**, que combina criterios numéricos como el codo con el conocimiento del problema.

Formalización

El problema de agrupamiento

Sea $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ un conjunto de n observaciones con p variables. Un **agrupamiento duro** en k grupos es una función de asignación $\ell : \{1, \dots, n\} \to \{1, \dots, k\}$, o equivalentemente una partición de

$\mathcal{D}$ en conjuntos no vacíos y disjuntos $C_1, \dots, C_k$ con $C_c = \{\mathbf{x}_i : \ell(i) = c\}$. Un agrupamiento **blando** asigna en cambio un vector de pertenencias $(r_{i1}, \dots, r_{ik})$ con $r_{ic} \geq 0$ y $\sum_c r_{ic} = 1$.

Casi todos los algoritmos se pueden leer como la optimización, exacta o aproximada, de un **criterio** que combina dos deseos opuestos:

- **cohesión**: los casos de un mismo grupo deben estar cerca entre sí;
- **separación**: los casos de grupos distintos deben estar lejos.

k-medias mide la cohesión con W; el clustering jerárquico, con distintas definiciones de la distancia entre grupos; DBSCAN reemplaza ambas ideas por la de **densidad**. Cada criterio favorece cierto tipo de forma: el de k-medias, grupos redondos y de tamaño parecido.

El número de particiones posibles

¿Por qué no revisar siempre todas las particiones, como hicimos con los ocho clientes? El número de maneras de partir n casos en exactamente k grupos no vacíos es el **número de Stirling de segunda especie**[3] $S(n, k)$, que cumple la recurrencia

$$S(n,k) = k\,S(n-1,k) + S(n-1,k-1), \qquad S(n,n) = 1, \quad S(n,0) = 0 \ (n > 0). \tag{67.2}$$

La recurrencia se lee así: el caso n forma un grupo nuevo él solo (los otros $n-1$ forman $k-1$ grupos) o se agrega a uno de los k grupos que ya formaron los demás. Para nuestros clientes, $S(8,2) = 127$, $S(8,4) = 1\,701$, y el total de particiones con cualquier número de grupos (el **número de Bell**) es $4\,140$. Una computadora las revisa en un instante. Pero $S(20,4) \approx 4.5 \times 10^{10}$ y $S(100,4) \approx 6.7 \times 10^{58}$, más que el número de átomos de la Tierra. Encontrar la partición que minimiza W es un problema NP-difícil (Aloise et al. 2009). Por eso todos los algoritmos prácticos son **heurísticos**: mejoran una partición paso a paso o la construyen de manera ávida, y no garantizan el óptimo global.

No existe el algoritmo perfecto: el teorema de Kleinberg

Podríamos esperar que hubiera un algoritmo de clustering «correcto» que todos debiéramos usar. Kleinberg (2002) mostró que no. Consideró funciones de agrupamiento que reciben una matriz de distancias d entre n puntos y devuelven una partición, y propuso tres propiedades deseables:

1. **Invariancia de escala.** Si se multiplican todas las distancias por la misma constante $\alpha > 0$, la partición no cambia. (Medir en metros o en kilómetros no debería importar).
2. **Riqueza.** Para cualquier partición imaginable de los n puntos, existe alguna matriz de distancias que la produce. (El algoritmo no tiene prohibido ningún resultado de antemano).
3. **Consistencia.** Si se acercan los puntos de un mismo grupo y se alejan los de grupos distintos, la partición no cambia. (Hacer los grupos más evidentes no debería alterarlos).

[3] Los **números de Stirling de segunda especie** cuentan particiones de un conjunto en bloques no vacíos sin orden. Por ejemplo, $S(4,2) = 7$: los cuatro elementos $\{a, b, c, d\}$ se pueden partir en dos grupos de 4 formas «uno contra tres» y 3 formas «dos contra dos». Para n grande, $S(n,k) \approx k^n/k!$, lo que muestra el crecimiento exponencial.

El teorema afirma que, para $n \geq 2$, **ninguna** función de agrupamiento cumple las tres a la vez. Cada algoritmo sacrifica alguna. Por ejemplo, el clustering jerárquico de enlace simple que se detiene cuando hay exactamente k grupos es invariante a la escala y consistente, pero no es rico: nunca produce un número de grupos distinto de k. El mensaje práctico es que elegir un algoritmo es elegir qué propiedades se está dispuesto a perder, y que la elección debe guiarse por el problema.

Evaluar sin respuesta correcta

Como no hay error de prueba, un resultado de clustering se juzga desde varios ángulos complementarios:

- **Criterios internos**, calculados solo con los datos y la partición: cohesión y separación (silueta, Davies-Bouldin, Calinski-Harabasz). Responden «¿los grupos son compactos y están separados?».
- **Criterios externos**, cuando por casualidad existe una clasificación de referencia (la especie de un pingüino, el diagnóstico de un paciente): índice de Rand ajustado, información mutua. Sirven para comparar algoritmos en datos conocidos, no para validar un hallazgo nuevo.
- **Estabilidad**: si se repite el análisis con otra muestra, otra semilla u otro subconjunto de variables, ¿aparecen los mismos grupos? Un grupo que desaparece al quitar el 10 % de los datos probablemente es ruido.
- **Utilidad e interpretación**: ¿se pueden describir los grupos con palabras?, ¿sugieren acciones distintas?, ¿un experto del dominio los reconoce?

El capítulo de validación de clustering desarrolla los tres primeros con cálculos a mano; el cuarto se practica en el caso integrador del final de esta parte.

Laboratorio: la escala crea la estructura

El laboratorio agrupa clientes con k-medias. En lugar de elegir entre «unidades originales» y «estandarizadas», controla con un deslizador **cuánto pesa el ticket** frente a las visitas en la distancia. Con peso 1 ambas variables estandarizadas cuentan lo mismo; con peso de alrededor de 87 se reproducen las unidades originales, porque esa es la razón entre las desviaciones estándar del ticket (346.5 pesos) y de las visitas (4 visitas). El estado inicial reproduce la Tabla 67.4 con $k = 4$.

Laboratorio interactivo disponible en la versión HTML. Valores de referencia con los ocho clientes estandarizados (peso 1): $W = 16.00, 8.12, 4.09, 0.57, 0.28$ para $k = 1, \ldots, 5$; con $k = 4$ los grupos son $\{1, 2\}$, $\{3, 4\}$, $\{5, 6\}$ y $\{7, 8\}$. En unidades originales (peso 87) el ticket explica el 99.99 % de la variación y la partición en cuatro grupos es $\{1, 5\}$, $\{2, 6\}$, $\{3, 8\}$, $\{4, 7\}$, que mezcla clientes ocasionales y frecuentes en todos sus grupos.

El laboratorio muestra que el peso de cada variable es un **hiperparámetro escondido** de cualquier método basado en distancias. Estandarizar no es «la» respuesta correcta: es la elección por omisión que da a cada variable la misma voz. Si el negocio considera que el gasto importa más que la frecuencia, puede darle más peso de forma deliberada. Lo que no se debe hacer es dejar que lo decidan las unidades de medida por accidente.

El flujo de trabajo no supervisado

El agrupamiento rara vez es un único paso. Un proyecto típico, dentro de la fase de modelado de CRISP-DM, recorre estas etapas:

1. **Definir el propósito.** ¿Para qué se quieren los grupos? ¿Campañas distintas, inventarios distintos, detectar casos raros? El propósito decide las variables y cómo validar.
2. **Preparar los datos.** Tratar faltantes y atípicos, codificar categóricas, elegir la escala. Como vimos, este paso **define** la geometría.
3. **Elegir una representación.** Una distancia adecuada al tipo de datos y, si hay muchas variables, una reducción de dimensionalidad previa.
4. **Aplicar uno o varios algoritmos** con distintos supuestos sobre la forma de los grupos.
5. **Validar.** Criterios internos, estabilidad ante remuestreo y, si existe, comparación con información externa.
6. **Interpretar.** Describir cada grupo con estadísticas de sus variables (*perfilado*), darle un nombre y comprobar con expertos que tiene sentido.
7. **Usar y monitorear.** Asignar casos nuevos a los grupos y vigilar si la estructura cambia con el tiempo.

Un mapa de esta parte

Tabla 67.5.: Capítulos de la parte de aprendizaje no supervisado.

Capítulo	Pregunta	Idea central
Similitud y distancias	¿Qué significa «parecido»?	Métricas para datos numéricos, categóricos y mixtos; escala
k-medias y k-medoides	¿Cómo partir en k grupos compactos?	Minimizar la suma de cuadrados; algoritmo de Lloyd
Validación de clustering	¿Son buenos los grupos? ¿Cuántos hay?	Silueta, índices internos, índices externos, estabilidad
Clustering jerárquico	¿Y si los grupos están anidados?	Fusiones sucesivas, criterios de enlace, dendrograma
DBSCAN, OPTICS y HDBSCAN	¿Y si los grupos tienen formas arbitrarias?	Regiones densas, puntos núcleo, ruido
Detección de anomalías	¿Qué casos no se parecen a ninguno?	Distancia, densidad local, aislamiento
Escalamiento multidimensional e Isomap	¿Cómo dibujar datos de muchas dimensiones?	Conservar distancias, geodésicas sobre variedades
t-SNE, UMAP y autoencoders	¿Cómo conservar los vecindarios?	Probabilidades de vecindad, grafos difusos, redes
Práctica integradora	¿Cómo se combina todo?	Segmentación de pingüinos, de la preparación al perfilado

Las **mezclas gaussianas** (GMM), presentadas en la parte de clasificación porque permiten construir clasificadores generativos, son también un método de clustering blando; el capítulo de validación las

usa para elegir el número de grupos con el criterio BIC.

Implementación en Python

La interfaz no supervisada de `scikit-learn`

Los estimadores no supervisados de `scikit-learn` comparten la interfaz de los supervisados, pero `fit` recibe solo `X`:

- `fit(X)` ajusta el modelo; los algoritmos de clustering guardan las etiquetas en `labels_`.
- `fit_predict(X)` ajusta y devuelve las etiquetas en un solo paso.
- `predict(X_nuevo)` asigna casos nuevos a los grupos, **solo** en los métodos que lo permiten (k-medias y GMM sí; el jerárquico aglomerativo y DBSCAN no).
- `transform(X)` en los métodos de reducción de dimensionalidad (PCA, Isomap) devuelve la nueva representación.

En la columna `grupo_crudo`, los clientes 1 y 5 comparten grupo aunque uno visita la tienda dos veces al mes y el otro diez; en `grupo_estandar` aparecen los cuatro segmentos. Los números de los grupos son arbitrarios: el algoritmo no sabe qué significa cada uno.

La curva del codo

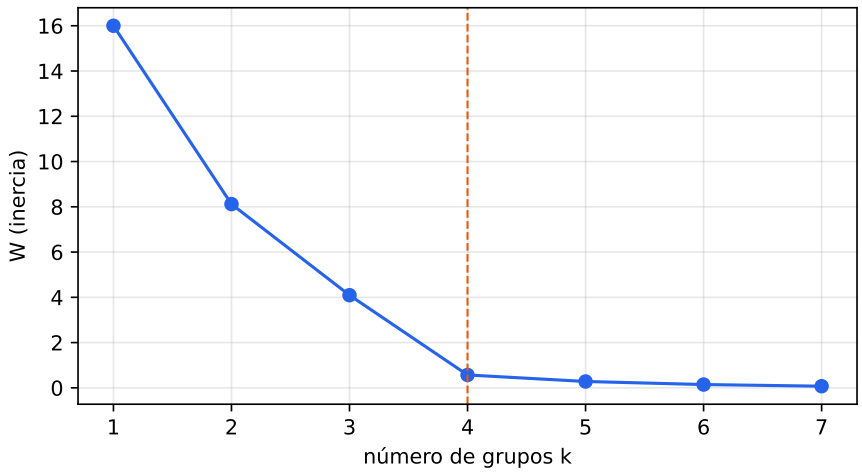


Figura 67.1.: Suma de cuadrados dentro de los grupos (W) de k-medias según el número de grupos, con los ocho clientes estandarizados. La caída se detiene después de k = 4.

Perfilar los grupos

Un grupo solo es útil cuando se puede describir. El **perfilado** resume cada grupo con estadísticas de las variables originales, no de las estandarizadas, para que el resultado se pueda comunicar.

Estimar una densidad

La misma estructura se ve como cuatro «montañas» en una estimación de densidad de núcleo (Ecuación 67.1). Con datos simulados de 120 clientes:

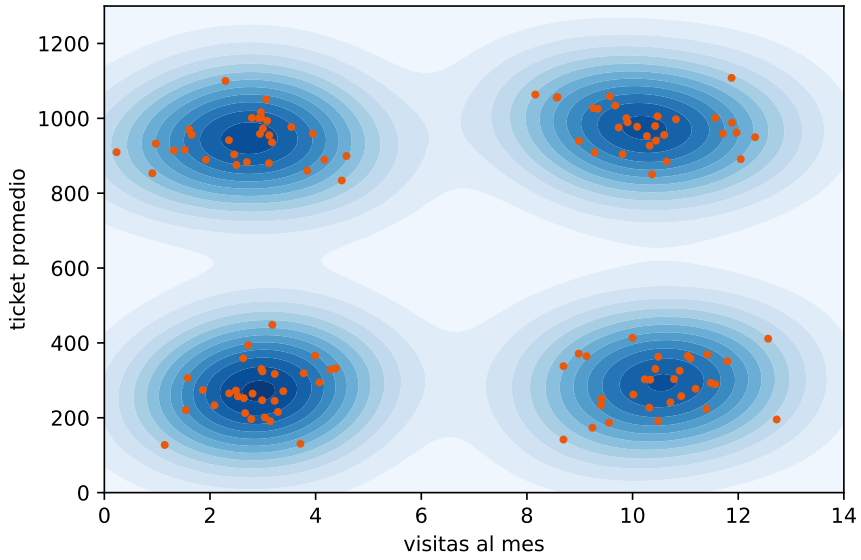


Figura 67.2.: Estimación de densidad de núcleo de 120 clientes simulados con cuatro segmentos. Cada máximo local de la densidad corresponde a un segmento; los métodos basados en densidad, como DBSCAN, definen los grupos de esta manera.

Resumen

- El aprendizaje no supervisado busca estructura en datos **sin variable objetivo**: grupos, densidades, representaciones de baja dimensión y anomalías.
- **La representación decide el resultado.** Las variables elegidas, sus unidades y su escala definen qué significa «parecido»; con unidades originales una sola variable puede dominar la distancia.
- Un algoritmo de clustering **siempre** entrega grupos. No existe un error de prueba que avise si son absurdos.
- El número de particiones crece de forma explosiva ($S(100, 4) \approx 6.7 \times 10^{58}$), así que los algoritmos son heurísticos; además, el teorema de Kleinberg muestra que **ningún** algoritmo satisface a la vez invariancia de escala, riqueza y consistencia.
- La validación combina criterios internos, externos, estabilidad e interpretación, y el número de grupos es una decisión informada, no un cálculo.

El siguiente capítulo estudia el primer ingrediente de cualquier método no supervisado: cómo medir la **similitud** entre casos con variables numéricas, categóricas o mixtas.

68. Similitud y distancias

Todo método de agrupamiento responde, explícita o implícitamente, a una pregunta previa: **¿qué tan parecidos son dos casos?** k-medias mide el parecido con la distancia euclidiana; el clustering jerárquico y DBSCAN aceptan cualquier matriz de distancias; t-SNE convierte distancias en probabilidades de ser vecinos. El capítulo anterior mostró que, con las mismas ocho filas, cambiar la escala de una variable transforma por completo los grupos. Este capítulo estudia la otra mitad del problema: **cómo** medir la distancia según el tipo de dato, que en la práctica rara vez es una tabla de números con unidades comparables (Aggarwal y Reddy 2013; Han et al. 2011).

Intuición: ¿quién se parece más a Ana?

Una tienda quiere ofrecer a Ana los productos que compran los clientes más parecidos a ella. Tiene cinco registros con variables de tipos distintos: dos **numéricas**, una **categórica** y una **binaria**.

Tabla 68.1.: Cinco clientes con variables mixtas.

Cliente	Edad	Ingreso (miles de pesos)	Canal de compra	Tarjeta de la tienda
Ana	34	28	app	sí
Beto	36	75	app	no
Carla	58	30	tienda	sí
Daniel	31	26	web	no
Elena	55	70	tienda	no

A simple vista, Daniel tiene casi la misma edad y el mismo ingreso que Ana, pero compra por otro canal y no tiene tarjeta. Beto comparte el canal pero gana casi el triple. Carla comparte la tarjeta pero le lleva 24 años. ¿Quién es el más parecido? La respuesta depende de qué variables se usen y de cuánto pese cada una, y en este capítulo la construiremos paso a paso.

Construcción paso a paso

Paso 1. Solo las variables numéricas

La distancia euclidiana con las unidades originales da, desde Ana, 3.61 a Daniel, 24.1 a Carla, 47.0 a Beto y 47.0 a Elena. Como en el capítulo anterior, las unidades importan: el ingreso varía en un rango de 49 mil pesos y la edad en uno de 27 años. Una alternativa a la estandarización es la normalización por **rango** (min-max), que lleva cada variable a $[0, 1]$:

$$\tilde{x} = \frac{x - \text{mín}}{\text{máx} - \text{mín}}, \qquad \tilde{x}_{\text{edad}} = \frac{\text{edad} - 31}{27}, \qquad \tilde{x}_{\text{ingreso}} = \frac{\text{ingreso} - 26}{49}.$$

Entre Ana y Daniel las diferencias normalizadas son $3/27 = 0.111$ en edad y $2/49 = 0.041$ en ingreso. Tres formas de combinarlas:

$$\begin{aligned} d_1 &= 0.111 + 0.041 = 0.152 && \text{(Manhattan: suma de diferencias)}, \\ d_2 &= \sqrt{0.111^2 + 0.041^2} = 0.118 && \text{(euclidiana)}, \\ d_\infty &= \text{máx}(0.111,\, 0.041) = 0.111 && \text{(Chebyshev: la mayor diferencia)}. \end{aligned}$$

Con cualquiera de las tres, **Daniel** es el más cercano a Ana; Carla queda a $d_2 = 0.89$, Beto a 0.96 y Elena a 1.16. Pero estas distancias ignoran el canal y la tarjeta.

Paso 2. Las variables categóricas: coincidencia simple

Para una variable **nominal** como el canal no tiene sentido restar: «app menos tienda» no es un número. Lo natural es preguntar si los valores **coinciden**:

$$\delta(a, b) = \begin{cases} 0 & \text{si } a = b, \\ 1 & \text{si } a \neq b. \end{cases}$$

Con dos variables categóricas (canal y tarjeta), la **distancia de coincidencia simple** es la proporción de variables en las que difieren dos casos[1]. Ana y Beto difieren en la tarjeta: 1/2. Ana y Carla difieren en el canal: 1/2. Ana y Daniel difieren en ambas: $2/2 = 1$. Si solo miráramos estas variables, Daniel sería el **menos** parecido.

Paso 3. Combinar: la distancia de Gower

Gower (1971) propuso promediar contribuciones de cada variable, todas en la escala $[0, 1]$: para las numéricas, la diferencia absoluta dividida entre el rango; para las categóricas, la coincidencia δ. Para Ana y Daniel:

$$d_G(\text{Ana}, \text{Daniel}) = \frac{1}{4}\Big(\underbrace{\tfrac{3}{27}}_{\text{edad}} + \underbrace{\tfrac{2}{49}}_{\text{ingreso}} + \underbrace{1}_{\text{canal}} + \underbrace{1}_{\text{tarjeta}} \Big) = \frac{0.111 + 0.041 + 1 + 1}{4} = 0.538.$$

Tabla 68.2.: Contribuciones de cada variable a la distancia de Gower desde Ana.

Desde Ana	Edad	Ingreso	Canal	Tarjeta	d_G	Orden
Beto	0.074	0.959	0	1	0.508	2
Carla	0.889	0.041	1	0	**0.482**	1
Daniel	0.111	0.041	1	1	0.538	3

[1] Para vectores de la misma longitud, el número de posiciones en que difieren es la **distancia de Hamming**; la coincidencia simple es la distancia de Hamming dividida entre el número de variables.

Desde Ana	Edad	Ingreso	Canal	Tarjeta	d_G	Orden
Elena	0.778	0.857	1	1	0.909	4

Con Gower, la más parecida a Ana es **Carla** y Daniel cae al tercer lugar. ¿Tiene sentido? Cada desacuerdo categórico aporta 1, lo mismo que la máxima diferencia posible de edad (27 años). Que Daniel compre por web y no tenga tarjeta pesa tanto como si tuviera 27 años más **y** 49 mil pesos más de ingreso. Es una decisión implícita del método.

Paso 4. Ponderar de forma deliberada

Gower admite pesos w_j por variable. Si el negocio considera que canal y tarjeta deben pesar la mitad que las variables demográficas, con $w = (1, 1, 0.5, 0.5)$:

$$d_G^{w}(\text{Ana}, \text{Daniel}) = \frac{0.111 + 0.041 + 0.5(1) + 0.5(1)}{1 + 1 + 0.5 + 0.5} = \frac{1.152}{3} = 0.384,$$

y ahora el orden es Daniel (0.384), Carla (0.477), Beto (0.511) y Elena (0.878). Ninguna de las dos respuestas es la correcta en abstracto; cada una refleja una idea distinta de «parecido». Lo importante es que la decisión sea **explícita**.

Paso 5. La matriz de distancias

Los algoritmos de clustering jerárquico, k-medoides y DBSCAN no necesitan las variables originales: les basta la **matriz de distancias D**, con $D_{ij} = d(\mathbf{x}_i, \mathbf{x}_j)$. Con Gower de pesos iguales:

Tabla 68.3.: Matriz de distancias de Gower entre los cinco clientes.

	Ana	Beto	Carla	Daniel	Elena
Ana	0	0.508	0.482	0.538	0.909
Beto	0.508	0	0.933	0.546	0.451
Carla	0.482	0.933	0	0.770	0.482
Daniel	0.538	0.546	0.770	0	0.697
Elena	0.909	0.451	0.482	0.697	0

La matriz es **simétrica** y tiene ceros en la diagonal. Para n casos tiene $n(n-1)/2$ valores distintos: 10 aquí, casi 50 millones con $n = 10\,000$. Esa es la razón por la que los métodos que la necesitan completa tienen problemas de memoria con muchos datos.

Formalización

¿Qué es una distancia?

Una función d que asigna un número a cada par de casos es una **métrica** si, para todos $\mathbf{x}, \mathbf{y}, \mathbf{z}$:

1. $d(\mathbf{x}, \mathbf{y}) \geq 0$, con igualdad si y solo si $\mathbf{x} = \mathbf{y}$ (positividad);
2. $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ (simetría);
3. $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$ (desigualdad triangular).

La desigualdad triangular dice que ir directamente nunca es más largo que dar un rodeo. No es un capricho matemático: los índices espaciales que aceleran la búsqueda de vecinos (árboles KD, árboles de bolas) la usan para descartar regiones enteras sin calcular distancias. Algunas medidas útiles **no** son métricas: la distancia euclidiana **al cuadrado** (que usa k-medias) viola la desigualdad triangular, y la distancia coseno también. Se les llama **disimilitudes**. Muchas veces se trabaja con una **similitud** s en lugar de una distancia; las conversiones más comunes son $d = 1 - s$ cuando $s \in [0, 1]$, y en sentido inverso el **núcleo gaussiano**

$$s(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{d(\mathbf{x}, \mathbf{y})^2}{2\sigma^2}\right), \tag{68.1}$$

que vale 1 cuando los casos coinciden y decae a 0 cuando se alejan, a un ritmo controlado por σ. Lo usarán el clustering espectral y t-SNE.

Distancias para datos numéricos

Minkowski. Para vectores con p variables, la distancia de orden $s \geq 1$ es

$$d_s(\mathbf{x}, \mathbf{y}) = \Big(\sum_{j=1}^{p} |x_j - y_j|^s\Big)^{1/s}. \tag{68.2}$$

Con $s = 1$ es la distancia **Manhattan** (la que recorre un taxi en una ciudad cuadriculada), con $s = 2$ la **euclidiana** y con $s \to \infty$ la de **Chebyshev**, $\max_j |x_j - y_j|$. Al crecer s, la distancia se concentra en la variable con mayor diferencia: Manhattan es la más tolerante con una diferencia grande en una sola variable y Chebyshev la menos. El conjunto de puntos a distancia menor que r de un centro, la **bola**, es un rombo para Manhattan, un círculo para la euclidiana y un cuadrado para Chebyshev; el laboratorio de este capítulo las dibuja.

Mahalanobis. Cuando las variables están **correlacionadas**, la euclidiana cuenta dos veces la información compartida. La distancia de Mahalanobis (1936) corrige escala y correlación a la vez:

$$d_M(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top \Sigma^{-1} (\mathbf{x} - \mathbf{y})}, \tag{68.3}$$

donde Σ es la matriz de covarianza[2] de los datos. Un ejemplo con dos variables estandarizadas y correlación 0.8:

$$\Sigma = \begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}, \qquad \Sigma^{-1} = \frac{1}{1-0.64}\begin{pmatrix} 1 & -0.8 \\ -0.8 & 1 \end{pmatrix} = \frac{1}{0.36}\begin{pmatrix} 1 & -0.8 \\ -0.8 & 1 \end{pmatrix}.$$

Los puntos $\mathbf{a} = (1, 1)$ y $\mathbf{b} = (1, -1)$ están a la misma distancia euclidiana del origen, $\sqrt{2} = 1.414$. Pero

$$d_M(\mathbf{a}, \mathbf{0})^2 = \frac{1 - 1.6 + 1}{0.36} = \frac{0.4}{0.36} = 1.11, \qquad d_M(\mathbf{b}, \mathbf{0})^2 = \frac{1 + 1.6 + 1}{0.36} = \frac{3.6}{0.36} = 10,$$

es decir, $d_M(\mathbf{a}, \mathbf{0}) = 1.05$ y $d_M(\mathbf{b}, \mathbf{0}) = 3.16$. El punto $\mathbf{a}$ sigue la tendencia de los datos (ambas variables altas a la vez, como es habitual con correlación positiva) y $\mathbf{b}$ la contradice. Mahalanobis lo reconoce: su bola es una **elipse** alineada con la nube de datos. Esta idea será central en la detección de anomalías.

Coseno. Cuando importa la **dirección** del vector y no su longitud, como en textos representados por conteos de palabras, se usa

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \, \|\mathbf{y}\|}. \tag{68.4}$$

Tres documentos con conteos de las palabras (*datos*, *minería*, *fútbol*): $\mathbf{d}_1 = (3, 2, 0)$, $\mathbf{d}_2 = (6, 4, 0)$, que es el mismo texto escrito dos veces, y $\mathbf{d}_3 = (0, 1, 3)$. La euclidiana separa a $\mathbf{d}_1$ de su propia copia por 3.61, casi tanto como de un texto sobre otro tema ($\mathbf{d}_3$, a 4.36), y la copia larga queda a 7.35 de $\mathbf{d}_3$: la **longitud** del texto domina la distancia. El coseno pone a $\mathbf{d}_1$ y $\mathbf{d}_2$ a distancia **cero**, y a $\mathbf{d}_1$ y $\mathbf{d}_3$ a $1 - 2/(\sqrt{13}\sqrt{10}) = 0.825$.

Correlación. Para comparar la **forma** de dos series (ventas semanales de dos tiendas de tamaño distinto), $d = 1 - r(\mathbf{x}, \mathbf{y})$, donde r es el coeficiente de correlación de Pearson entre los valores de ambas series. Es el coseno aplicado a las series centradas.

Escala y pesos

Como vimos en el capítulo anterior, **la escala es un peso implícito**. Las tres opciones habituales son:

Tabla 68.4.: Transformaciones de escala antes de medir distancias.

Transformación	Fórmula	Rango resultante	Sensibilidad a atípicos
Estandarización	$(x - \bar{x})/s$	sin límites, media 0 y desviación 1	alta (la media y s se mueven)

[2]La **matriz de covarianza** Σ tiene en la diagonal las varianzas de cada variable y fuera de ella las covarianzas entre pares. Su inversa Σ^{-1} «deshace» la escala y la correlación: si los datos tienen covarianza Σ, la transformación $\Sigma^{-1/2}\mathbf{x}$ produce variables sin correlación y con varianza 1, y la distancia de Mahalanobis es la euclidiana en ese espacio transformado.

Transformación	Fórmula	Rango resultante	Sensibilidad a atípicos
Min-max	$(x-\text{mín})/(\text{máx}-\text{mín})$	$[0,1]$	muy alta (un atípico fija el rango)
Robusta	$(x-\text{mediana})/\text{RIC}$	sin límites	baja

Con un atípico, min-max comprime a todos los demás casos en una franja estrecha y la variable deja de contar. Después de escalar, cualquier peso adicional w_j se aplica multiplicando la variable por $\sqrt{w_j}$ antes de la euclidiana, o directamente en la fórmula de Gower.

Distancias para datos categóricos y binarios

Nominales. La coincidencia simple cuenta desacuerdos. Si se codifican con *one-hot*, la euclidiana al cuadrado entre dos casos es exactamente **el doble** del número de desacuerdos, porque cada desacuerdo cambia dos columnas indicadoras. Con muchas categorías, la coincidencia simple es más directa y no infla la dimensión.

Ordinales. Si las categorías tienen orden (básico < plus < premium), se reemplazan por sus rangos $1,\dots,M$ y se tratan como numéricas divididas entre $M-1$.

Binarias. Para dos casos descritos por m atributos binarios se cuenta la tabla

Tabla 68.5.: Coincidencias entre dos vectores binarios.

	$y_j=1$	$y_j=0$
$x_j=1$	a	b
$x_j=0$	c	d

La similitud de coincidencia simple es $(a+d)/m$ y la de **Jaccard** es $a/(a+b+c)$, que ignora las coausencias d. Supón un catálogo de 10 productos. Ana compró pan y leche; Beto, vino y queso. No comparten nada, y sin embargo coinciden en **no** haber comprado los otros seis productos: la coincidencia simple da $6/10 = 0.6$ de similitud. Jaccard da $0/4 = 0$. Cuando la presencia es lo informativo y la ausencia es lo normal (canastas, síntomas, palabras), los atributos son **asimétricos** y conviene Jaccard; es la misma medida que aparece en la extracción de patrones.

Datos mixtos: Gower

La distancia de Gower (1971) para p variables de cualquier tipo es

$$d_G(\mathbf{x},\mathbf{y}) = \frac{\sum_{j=1}^{p} w_j\,\delta_j(\mathbf{x},\mathbf{y})\,d_j(x_j,y_j)}{\sum_{j=1}^{p} w_j\,\delta_j(\mathbf{x},\mathbf{y})}, \qquad d_j = \begin{cases} |x_j-y_j|/R_j & \text{numérica, con rango } R_j, \\ \mathbb{1}[x_j \neq y_j] & \text{categórica,} \end{cases} \tag{68.5}$$

donde $\delta_j = 0$ si el valor falta en alguno de los dos casos (o si ambos son 0 en un atributo binario asimétrico) y $\delta_j = 1$ en otro caso. Así, Gower **maneja faltantes** de forma natural: promedia solo las variables disponibles. Su punto débil es el que vimos con Ana: un desacuerdo categórico vale tanto como la diferencia numérica máxima, y el rango R_j depende de los extremos.

Cadenas y secuencias

La **distancia de edición** (de Levenshtein) entre dos cadenas es el mínimo número de inserciones, eliminaciones y sustituciones de un carácter que transforman una en la otra. Se calcula con programación dinámica: $D_{i,j}$ es la distancia entre los primeros i caracteres de una cadena y los primeros j de la otra, y

$$D_{i,j} = \min\left(D_{i-1,j} + 1,\ D_{i,j-1} + 1,\ D_{i-1,j-1} + \mathbb{1}[s_i \neq t_j]\right). \tag{68.6}$$

Para «casa» y «cosas» la tabla termina en $D_{4,5} = 2$: sustituir la *a* por *o* e insertar una *s* al final. Se usa para limpiar nombres capturados a mano, comparar secuencias de ADN y agrupar rutas de navegación. Para series de tiempo desfasadas existe una idea análoga, el alineamiento temporal dinámico (DTW).

Distancias en alta dimensión

En el capítulo de vecinos cercanos vimos que, con muchas variables, **todas las distancias se parecen**. Con 500 puntos uniformes en el hipercubo, el cociente entre la diferencia del vecino más lejano y el más cercano y la distancia al más cercano, $(d_{\text{máx}} - d_{\text{mín}})/d_{\text{mín}}$, vale 82 en dos dimensiones, 1.7 en diez, 0.51 en cien y 0.11 en mil (Beyer et al. 1999). Cuando el más lejano está apenas un 11 % más lejos que el más cercano, la idea de «grupo de casos cercanos» pierde sentido. Por eso, antes de agrupar datos con muchas variables, conviene seleccionar variables o reducir la dimensión.

Laboratorio: la forma de la distancia

Siete puntos en el plano (con la misma escala en ambos ejes). Elige un punto de referencia haciendo clic sobre él, cambia la métrica y arrastra los puntos. El contorno es la **bola** de la métrica que pasa por el vecino más cercano: todo lo que queda dentro está más cerca que ese vecino. La matriz de la derecha muestra todas las distancias, sombreadas de menor (oscuro) a mayor.

Laboratorio interactivo disponible en la versión HTML. En la configuración inicial (nube correlacionada, referencia A), el vecino más cercano de A con la euclidiana es G, mientras que con Mahalanobis es B, que sigue la dirección de la correlación. Con el coseno, la distancia solo depende del ángulo de cada punto respecto al origen.

Implementación en Python

Matrices de distancias con `scipy`

`scipy.spatial.distance.pdist` calcula las $n(n-1)/2$ distancias de una matriz de datos y `squareform` las acomoda en la matriz cuadrada. Acepta, entre otras, `"euclidean"`, `"cityblock"` (Manhattan), `"chebyshev"`, `"mahalanobis"`, `"cosine"`, `"jaccard"` y `"hamming"`.

Distancia de Gower desde cero

`scikit-learn` no trae Gower, pero se programa en pocas líneas siguiendo la Ecuación 68.5. Los pesos permiten reproducir los dos órdenes del ejemplo manual.

La matriz de Gower se puede pasar directamente a los algoritmos que aceptan distancias precalculadas, como `AgglomerativeClustering(metric="precomputed")` o `DBSCAN(metric="precomputed")`.

Mahalanobis, coseno y Jaccard

Distancia de edición

Concentración de distancias

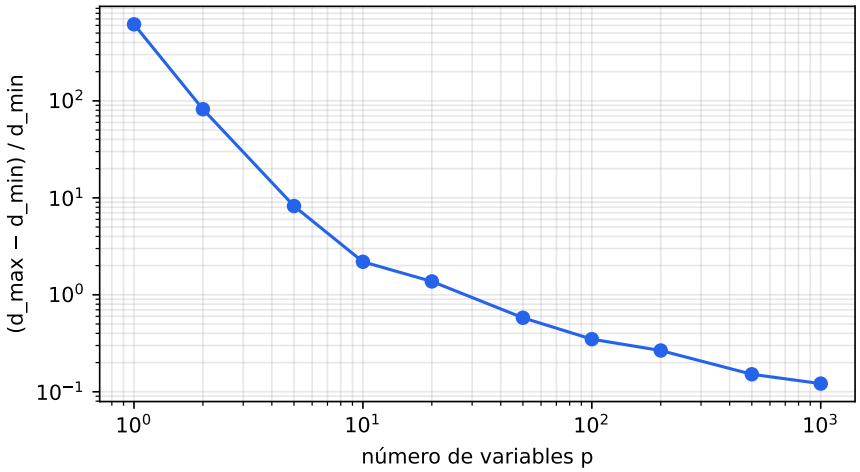


Figura 68.1.: Contraste relativo (d_max − d_min)/d_min entre un punto de consulta y 500 puntos uniformes, según la dimensión. Con muchas variables, el vecino más lejano está casi tan cerca como el más cercano.

Guía para elegir una distancia

Tabla 68.6.: Elección de la distancia según el tipo de datos.

Tipo de datos	Distancia recomendada	Antes de calcular
Numéricos, unidades comparables	Euclidiana o Manhattan	revisar atípicos
Numéricos, unidades distintas	Euclidiana sobre datos escalados	estandarizar o escalado robusto
Numéricos correlacionados	Mahalanobis, o euclidiana tras PCA	estimar la covarianza con suficientes datos
Conteos, textos	Coseno	normalizar por longitud (TF-IDF)

Tipo de datos	Distancia recomendada	Antes de calcular
Formas de series	$1 - r$ (correlación), DTW	alinear frecuencias
Binarios asimétricos (canastas)	Jaccard	—
Categóricos	Coincidencia simple	agrupar categorías raras
Mixtos	Gower	decidir pesos de forma explícita
Cadenas	Edición (Levenshtein)	normalizar mayúsculas y acentos

Resumen

- Una **métrica** cumple positividad, simetría y desigualdad triangular; algunas medidas útiles (euclidiana al cuadrado, coseno) no la cumplen y se llaman disimilitudes.
- Minkowski reúne Manhattan, euclidiana y Chebyshev; Mahalanobis corrige escala y correlación; coseno compara direcciones.
- Para categóricas se cuenta coincidencia; para binarias asimétricas, Jaccard ignora las coausencias.
- **Gower** combina variables mixtas y maneja faltantes, pero cada desacuerdo categórico pesa tanto como la diferencia numérica máxima. Con Ana, cambiar los pesos cambió al cliente más parecido de Carla a Daniel.
- La escala y los pesos son decisiones explícitas; en alta dimensión las distancias se concentran y conviene reducir la dimensión antes de agrupar.

Con una forma de medir el parecido, el siguiente capítulo presenta el algoritmo de agrupamiento más usado: k-medias.

Parte XXXVI.

Clustering por Particiones

69. K-medias y K-medoides

k-medias (*k-means*) es el algoritmo de agrupamiento más usado. La idea apareció de forma independiente varias veces: Stuart Lloyd la describió en 1957 para cuantizar señales en los laboratorios Bell, aunque su trabajo se publicó hasta 1982 (Lloyd 1982), y MacQueen (1967) le dio el nombre con el que se conoce. Su atractivo es una combinación difícil de superar: se entiende en una frase, se programa en diez líneas y escala a millones de casos.

La frase es: **cada grupo se representa por su promedio, y cada caso pertenece al grupo cuyo promedio tiene más cerca.** El algoritmo alterna esas dos afirmaciones hasta que dejan de cambiar. En este capítulo lo ejecutaremos a mano sobre seis puntos, veremos por qué siempre termina pero no siempre en la mejor solución, cómo elegir buenos puntos de partida y qué ocurre cuando los datos no tienen la forma que el algoritmo supone.

Intuición: ubicar dos almacenes

Una empresa de reparto tiene seis puntos de entrega en una ciudad, con coordenadas en kilómetros, y quiere abrir **dos** almacenes. Cada punto se abastecerá desde el almacén más cercano, y el costo de transporte crece con el cuadrado de la distancia (por tiempo, combustible y desgaste).

Tabla 69.1.: Seis puntos de entrega.

Punto	P1	P2	P3	P4	P5	P6
x (km)	1	2	1	5	6	4
y (km)	1	1	3	4	6	6

Dos problemas están entrelazados. Si supiéramos **qué puntos atiende cada almacén**, pondríamos cada almacén en el promedio de sus puntos, porque el promedio es el lugar que minimiza la suma de distancias al cuadrado. Y si supiéramos **dónde están los almacenes**, cada punto elegiría el más cercano. No sabemos ninguna de las dos cosas, así que empezamos con una suposición y alternamos.

Construcción paso a paso

Paso 0. Un punto de partida

Colocamos los almacenes provisionalmente sobre dos puntos de entrega, $\mu_1 = P1 = (1, 1)$ y $\mu_2 = P3 = (1, 3)$. La elección es mala a propósito: ambos almacenes quedan en la esquina inferior izquierda.

Paso 1. Asignar cada punto al almacén más cercano

Calculamos las distancias de cada punto a los dos almacenes. Para P4 = (5, 4):

$$d(P4, \mu_1) = \sqrt{(5-1)^2 + (4-1)^2} = 5, \qquad d(P4, \mu_2) = \sqrt{(5-1)^2 + (4-3)^2} = \sqrt{17} \approx 4.12,$$

así que P4 va al almacén 2. Repitiendo para todos, el almacén 1 atiende a P1 y P2, y el almacén 2 a P3, P4, P5 y P6. La **suma de cuadrados dentro de los grupos**, que es el costo total, vale

$$W = \underbrace{0 + 1}_{\text{almacén 1}} + \underbrace{0 + 17 + 34 + 18}_{\text{almacén 2}} = 70.$$

Paso 2. Mover cada almacén al promedio de sus puntos

$$\mu_1 = \tfrac{1}{2}\big[(1,1) + (2,1)\big] = (1.5,\ 1), \qquad \mu_2 = \tfrac{1}{4}\big[(1,3) + (5,4) + (6,6) + (4,6)\big] = (4,\ 4.75).$$

Con los mismos grupos y los almacenes en su nuevo lugar, el costo baja de 70 a $W = 21.25$. El almacén 2 se desplazó mucho hacia arriba y a la derecha, arrastrado por P4, P5 y P6.

Paso 3. Reasignar

Con los almacenes nuevos, algunas asignaciones cambian:

Tabla 69.2.: Distancias a los almacenes después del primer movimiento.

Punto	P1	P2	P3	P4	P5	P6
$d(\cdot, \mu_1)$, con $\mu_1 = (1.5, 1)$	**0.50**	**0.50**	**2.06**	4.61	6.73	5.59
$d(\cdot, \mu_2)$, con $\mu_2 = (4, 4.75)$	4.80	4.25	3.47	**1.25**	**2.36**	**1.25**

P3 ahora está más cerca del almacén 1 ($2.06 < 3.47$) y cambia de grupo. Los grupos quedan $\{P1, P2, P3\}$ y $\{P4, P5, P6\}$, y con los almacenes todavía en $(1.5, 1)$ y $(4, 4.75)$ el costo baja a $W = 13.44$.

Paso 4. Mover otra vez y comprobar la convergencia

$$\mu_1 = \tfrac{1}{3}\big[(1,1)+(2,1)+(1,3)\big] = (1.33,\ 1.67), \qquad \mu_2 = \tfrac{1}{3}\big[(5,4)+(6,6)+(4,6)\big] = (5,\ 5.33),$$

y $W = 8.00$. Al reasignar, ningún punto cambia de grupo; los promedios, por lo tanto, tampoco cambian. El algoritmo **convergió** en dos iteraciones completas. La secuencia de costos fue

$$70 \xrightarrow{\text{mover}} 21.25 \xrightarrow{\text{asignar}} 13.44 \xrightarrow{\text{mover}} 8.00 \xrightarrow{\text{asignar}} 8.00.$$

Nunca subió. No es casualidad, como veremos en la formalización.

Paso 5. ¿Es la mejor solución?

La variación total de los seis puntos alrededor de su promedio general $(3.17, 3.5)$ es $T = 48.33$. Con dos almacenes quedan $W = 8.00$ dentro de los grupos; los otros $B = 40.33$ son la variación **entre** los grupos. Con seis puntos podemos revisar las $S(6,2) = 31$ particiones posibles: ninguna tiene un W menor que 8. Además, si se repite el algoritmo empezando en cualquiera de los 15 pares de puntos posibles, siempre termina en la misma solución. En datos tan bien separados, el punto de partida no importa. No siempre será así.

Paso 6. Cuando el punto de partida sí importa

Seis clientes en una recta, en las posiciones $0, 2, 10, 12, 20, 22$, y $k = 3$. La solución evidente es $\{0,2\}$, $\{10,12\}$, $\{20,22\}$, con centros $1, 11, 21$ y $W = 6$. Si empezamos con centros en 0, 2 y 16:

- asignar: $0 \to 0$, $2 \to 2$ y los otros cuatro a 16 (el 10 está a 8 del centro 2 y a 6 del 16);
- mover: los centros quedan en 0, 2 y $(10+12+20+22)/4 = 16$, exactamente donde estaban.

El algoritmo se detiene de inmediato con $W = 0 + 0 + (36 + 16 + 16 + 36) = 104$, diecisiete veces peor que la solución evidente. Es un **óptimo local**: ningún paso del algoritmo puede mejorarlo, aunque exista una solución mucho mejor. La defensa es doble: empezar en puntos bien repartidos y repetir el algoritmo desde varios puntos de partida, quedándose con el menor W.

Formalización

El objetivo

Dados n casos $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^p$ y un número de grupos k, k-medias busca una partición $C_1, \dots, C_k$ y centros $\mu_1, \dots, \mu_k$ que minimicen la **suma de cuadrados dentro de los grupos**, también llamada **inercia**:

$$W(C, \mu) = \sum_{c=1}^{k} \sum_{i \in C_c} \|\mathbf{x}_i - \mu_c\|^2. \tag{69.1}$$

El promedio es el mejor centro. Para un grupo fijo C, la función $f(\mu) = \sum_{i \in C} \|\mathbf{x}_i - \mu\|^2$ es una parábola en cada coordenada. Su gradiente es $\nabla f = -2 \sum_{i \in C} (\mathbf{x}_i - \mu)$, que se anula en

$$\mu = \frac{1}{|C|} \sum_{i \in C} \mathbf{x}_i = \bar{\mathbf{x}}_C. \tag{69.2}$$

Por eso el método se llama k-**medias**: con distancia euclidiana al cuadrado, el mejor representante de un grupo es su media, o **centroide**.

Descomposición de la variación. La variación total no depende de la partición y se reparte en una parte dentro de los grupos y otra entre ellos[1]:

$$\underbrace{\sum_{i=1}^{n} \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2}_{T} = \underbrace{\sum_{c=1}^{k} \sum_{i \in C_c} \|\mathbf{x}_i - \bar{\mathbf{x}}_c\|^2}_{W} + \underbrace{\sum_{c=1}^{k} |C_c| \, \|\bar{\mathbf{x}}_c - \bar{\mathbf{x}}\|^2}_{B}. \tag{69.3}$$

En el ejemplo, $48.33 = 8.00 + 40.33$. Como T es fijo, **minimizar la cohesión W equivale a maximizar la separación B**. La razón $B/T = 0.83$ dice que dos grupos explican el 83 % de la variación de los puntos, un análogo del R^2 de la regresión.

El algoritmo de Lloyd

Algoritmo de Lloyd. Entrada: datos $\mathbf{x}_1, \dots, \mathbf{x}_n$, número de grupos k, centros iniciales $\mu_1, \dots, \mu_k$.

1. **Asignación.** Para cada i: $\ell(i) \leftarrow \arg\min_c \|\mathbf{x}_i - \mu_c\|^2$.
2. **Actualización.** Para cada c: $\mu_c \leftarrow$ promedio de los $\mathbf{x}_i$ con $\ell(i) = c$.
3. Si ninguna asignación cambió, terminar; si no, volver al paso 1.

Convergencia. Cada paso minimiza W respecto a una parte de las incógnitas con la otra fija: la asignación elige, para centros fijos, el mejor grupo de cada punto; la actualización elige, para grupos fijos, el mejor centro de cada grupo (Ecuación 69.2). Ninguno de los dos puede aumentar W. Como hay un número finito de particiones y W nunca sube, el algoritmo no puede visitar dos veces la misma partición con un W distinto, y **termina en un número finito de pasos**. Lo que no garantiza es llegar al mínimo global: termina en un punto donde ningún cambio individual mejora, como en el Paso 6. Encontrar el mínimo global es NP-difícil incluso con $k = 2$ (Aloise et al. 2009).

Costo. Cada iteración calcula $n \times k$ distancias en p dimensiones: $O(nkp)$. En la práctica bastan pocas decenas de iteraciones, por lo que el algoritmo es lineal en el número de casos.

Empates y grupos vacíos. Si un punto está a la misma distancia de dos centros, se asigna al de menor índice. Si un grupo se queda sin puntos, su centro no tiene promedio; las implementaciones lo reubican, por ejemplo en el punto más alejado de su centro.

[1] La descomposición es el **teorema de Huygens** (o de König-Huygens) y es la misma que se usa en el análisis de varianza (ANOVA): la suma de cuadrados total se divide en una parte «dentro» y otra «entre». Se demuestra sumando y restando $\bar{\mathbf{x}}_c$ dentro de $\|\mathbf{x}_i - \bar{\mathbf{x}}\|^2$ y observando que los productos cruzados suman cero porque $\sum_{i \in C_c} (\mathbf{x}_i - \bar{\mathbf{x}}_c) = \mathbf{0}$.

La geometría: regiones de Voronoi

Para centros fijos, el conjunto de puntos del plano que quedarían asignados al centro c es su **región de Voronoi**,

$$V_c = \{\mathbf{x} : \|\mathbf{x} - \mu_c\| \leq \|\mathbf{x} - \mu_{c'}\| \text{ para todo } c'\}.$$

La frontera entre dos centros es la **mediatriz** del segmento que los une: una recta en dos dimensiones, un hiperplano en general. Por lo tanto, **las regiones de k-medias son polígonos convexos**. De ahí sale su principal limitación: no puede producir grupos con forma de media luna, de anillo o de serpiente. Una vez ajustado, k-medias sirve también para asignar casos nuevos: cada uno va a la región de Voronoi donde cae. En procesamiento de señales esto se llama **cuantización vectorial**: cada caso se resume con el índice de su centro.

K-Means++: empezar bien

La inicialización aleatoria, eligiendo k casos al azar como centros, a menudo pone dos centros en el mismo grupo natural y deja otro grupo sin centro, como en el Paso 6. Arthur y Vassilvitskii (2007) propusieron elegir los centros uno por uno, favoreciendo los puntos lejanos a los ya elegidos:

1. El primer centro es un caso elegido al azar con probabilidad uniforme.
2. Para cada caso, sea $D(\mathbf{x}_i)^2$ su distancia al cuadrado al centro **más cercano** ya elegido.
3. El siguiente centro es el caso i con probabilidad $D(\mathbf{x}_i)^2 / \sum_j D(\mathbf{x}_j)^2$.
4. Repetir 2 y 3 hasta tener k centros; luego ejecutar Lloyd.

En el ejemplo de los almacenes, si el primer centro es P1, las distancias al cuadrado son

Tabla 69.3.: Probabilidades de K-Means++ para el segundo centro, con P1 como primero.

Punto	P1	P2	P3	P4	P5	P6	Suma
D^2	0	1	4	25	50	34	114
Probabilidad	0	0.009	0.035	0.219	0.439	0.298	1

Con probabilidad $0.219 + 0.439 + 0.298 = 0.956$ el segundo centro cae en el grupo de arriba. Los autores demostraron que, en promedio, W con esta inicialización (aun **antes** de Lloyd) es a lo más $8(\ln k + 2)$ veces el óptimo; en la práctica, la diferencia es mucho menor. Es la inicialización por omisión de `scikit-learn`, que además repite el algoritmo `n_init` veces y se queda con el menor W.

El número de grupos

W siempre baja al aumentar k, así que no sirve para elegir k por sí solo. El **método del codo** busca el k a partir del cual las reducciones de W se vuelven pequeñas; el siguiente capítulo desarrolla criterios más finos (silueta, estadístico de brecha, estabilidad).

Qué supone k-medias

Minimizar W con distancia euclidiana equivale a suponer que los grupos son nubes **esféricas**, de **tamaño parecido** y con la **misma dispersión**. Formalmente, es el caso límite de una mezcla gaussiana con covarianzas $\sigma^2\mathbf{I}$ iguales cuando $\sigma \to 0$, como se mostró en el capítulo de mezclas gaussianas. De ahí sus fallas típicas:

- **Formas no convexas** (medias lunas, anillos): las fronteras rectas las cortan.
- **Grupos alargados** o con variables correlacionadas: la distancia euclidiana no los respeta; conviene estandarizar, usar PCA o una mezcla gaussiana con covarianza completa.
- **Tamaños o densidades muy distintos**: para reducir W conviene partir el grupo grande y difuso en lugar de respetar los pequeños.
- **Escala**: las variables con mayor varianza dominan, como vimos en el capítulo introductorio.
- **Atípicos**: el cuadrado amplifica las distancias grandes; un solo punto lejano puede capturar un centro completo.
- **Variables categóricas**: el promedio no está definido. Las variantes k-modas y k-prototipos (Huang 1998) usan la moda y la coincidencia simple.

Laboratorio: el algoritmo de Lloyd

Ejecuta k-medias paso a paso. El fondo muestra las regiones de Voronoi de los centros actuales; las líneas discontinuas, el camino que han recorrido. Con «K-Means++», el tamaño de cada punto es proporcional a su probabilidad de ser elegido como siguiente centro. El estado inicial reproduce el ejemplo de los almacenes.

Laboratorio interactivo disponible en la versión HTML. Con los seis puntos y los centros iniciales en P1 y P3, la secuencia de W es $70 \to 21.25 \to 13.44 \to 8.00 \to 8.00$ y el algoritmo converge en dos iteraciones a los grupos $\{P1, P2, P3\}$ y $\{P4, P5, P6\}$. En la recta con centros iniciales en 0, 2 y 16, el algoritmo se detiene de inmediato en $W = 104$, mientras que el óptimo es $W = 6$.

K-medoides: un representante real y robusto

El problema del atípico

Agreguemos al ejemplo de los almacenes un séptimo punto de entrega muy alejado, P7 = (14, 5). La partición «natural» en dos grupos sigue siendo $\{P1, P2, P3\}$ y $\{P4, P5, P6, P7\}$, pero su W es $3.33 + 65.5 = 68.83$: P7 arrastra el centro del segundo grupo hasta $(7.25, 5.25)$ y aporta él solo casi todo el costo. En cambio, la partición que **aísla** a P7, $\{P1, \dots, P6\}$ contra $\{P7\}$, tiene $W = 48.33 + 0 = 48.33$. La solución óptima de k-medias es dedicarle un almacén completo al punto atípico y juntar los dos grupos reales. El responsable es el **cuadrado**: P7 está a unas 7 unidades del centro natural, y su contribución de ≈ 49 pesa más que la de todos los demás puntos juntos.

Medoides

k-**medoides** (Kaufman y Rousseeuw 1990) hace dos cambios. El representante de cada grupo debe ser **uno de los casos** (el *medoide*) y el costo usa distancias **sin elevar al cuadrado**, o cualquier otra disimilitud:

$$\text{costo} = \sum_{c=1}^{k} \sum_{i \in C_c} d(\mathbf{x}_i, \mathbf{m}_c), \qquad \mathbf{m}_c \in \{\mathbf{x}_1, \dots, \mathbf{x}_n\}. \tag{69.4}$$

El medoide de un grupo es el caso con la menor suma de distancias a los demás. En el grupo $\{P4, P5, P6, P7\}$, las sumas de distancias euclidianas desde cada candidato son 13.53, 12.30, 14.29 y 27.17: el medoide es P5 = (6, 6). A diferencia del promedio, que P7 desplazó más de dos unidades, el medoide **no puede ser arrastrado** a una zona vacía. Con distancias sin cuadrado, la partición natural (medoides P1 y P5) cuesta 15.30 y la que aísla a P7 cuesta 17.84: k-medoides prefiere la partición natural.

El algoritmo PAM

El algoritmo clásico, PAM (*Partitioning Around Medoids*), tiene dos fases:

1. **Construcción.** Elegir el primer medoide como el caso con menor suma de distancias a todos; luego, agregar uno a uno el caso que más reduzca el costo.
2. **Intercambio.** Para cada par (medoide **m**, no medoide **o**), calcular cuánto cambia el costo si **o** reemplaza a **m**. Aplicar el mejor intercambio si reduce el costo y repetir hasta que ninguno lo reduzca.

Cada fase de intercambio evalúa $k(n-k)$ pares, cada uno con costo $O(n)$: $O(k(n-k)^2)$ por iteración, mucho más que k-medias. A cambio, k-medoides:

- funciona con **cualquier matriz de distancias** (Gower, Jaccard, edición), sin necesidad de promedios;
- es más **robusto** a atípicos;
- produce representantes **interpretables**: un cliente real, no un cliente promedio que no existe.

Para datos grandes existen aproximaciones como CLARA, que aplica PAM a submuestras, y FastPAM.

Variantes a escala

- **Mini-batch k-medias** (Sculley 2010): en cada paso usa una muestra pequeña de casos para mover los centros con un promedio móvil. Es mucho más rápido con millones de casos, a cambio de un W ligeramente mayor.
- **k-medias bisectante**: empieza con un grupo y parte repetidamente el de mayor W en dos con k-medias de dos grupos. Produce una jerarquía y suele dar grupos de tamaño más parejo.
- **Algoritmo de Elkan**: usa la desigualdad triangular para evitar calcular distancias que no pueden cambiar la asignación; da el mismo resultado que Lloyd, más rápido con pocos grupos.

Implementación en Python

El ejemplo de los almacenes

`KMeans` acepta los centros iniciales como un arreglo, lo que permite reproducir el cálculo manual. Con `max_iter=1` se detiene después de una asignación y una actualización.

`inertia_` se calcula con las asignaciones finales a los centros finales; por eso, con `max_iter=1`, `scikit-learn` informa el W de los nuevos grupos con los centros ya movidos.

Lloyd desde cero

Cuándo funciona y cuándo no

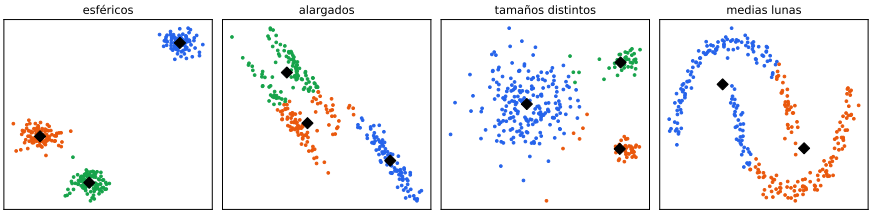


Figura 69.1.: k-medias con k-means++ y 10 reinicios en cuatro situaciones. Funciona con grupos esféricos y de tamaño parecido; falla con grupos alargados, con tamaños muy distintos (parte el grupo grande) y con formas no convexas.

K-medoides con PAM

`scikit-learn` no incluye k-medoides (está en el paquete externo `scikit-learn-extra`). Una versión compacta de PAM sobre una matriz de distancias, aplicada al ejemplo con el atípico:

Una aplicación: cuantización de color

Cada píxel de una imagen es un punto en el espacio de color (R, G, B). Agrupar los píxeles con k-medias y reemplazar cada uno por su centro reduce la imagen a k colores: es la cuantización vectorial en acción.

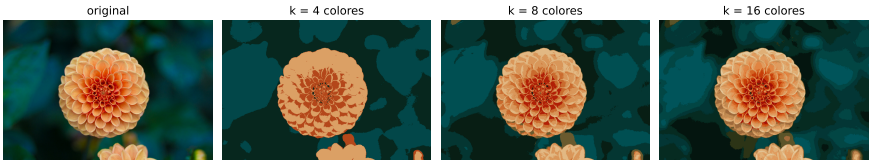


Figura 69.2.: Cuantización de color con k-medias: la imagen original (miles de colores) y sus versiones con 4, 8 y 16 colores, que son los centros de los grupos de píxeles. Se ajusta con una muestra de 5 000 píxeles y se asigna el resto con predict.

Mini-batch para datos grandes

Ventajas y limitaciones

Tabla 69.4.: Ventajas y limitaciones de k-medias.

Ventajas	Limitaciones
Simple, rápido y escalable: $O(nkp)$ por iteración	Requiere fijar k de antemano
Converge siempre en pocos pasos	Solo garantiza un óptimo local; depende del inicio
Asigna casos nuevos (cuantización vectorial)	Supone grupos esféricos, de tamaño y dispersión parecidos
Centros interpretables como «caso típico»	Sensible a la escala y a los atípicos
Base de muchas variantes (mini-batch, bisectante, k-medoides)	Solo datos numéricos con distancia euclidiana

Resumen

- k-medias minimiza la suma de cuadrados dentro de los grupos W; para grupos fijos, el mejor centro es el **promedio**.
- La variación total se descompone en $T = W + B$: minimizar la cohesión equivale a maximizar la separación.
- El algoritmo de Lloyd alterna **asignar** y **mover**; W nunca sube y el algoritmo termina, pero solo en un **óptimo local**.
- K-Means++ elige centros iniciales con probabilidad proporcional a D^2 y, junto con varios reinicios, evita casi siempre los malos óptimos.
- Las regiones son de Voronoi (convexas); k-medias falla con formas no convexas, tamaños distintos y atípicos.
- k-medoides usa casos reales como representantes y distancias sin cuadrado; es más robusto y acepta cualquier disimilitud, a un costo computacional mayor.

El siguiente capítulo responde a las dos preguntas que k-medias deja abiertas: **¿son buenos estos grupos?** y **¿cuántos grupos hay?**

70. Validación de clustering

En clasificación, la pregunta «¿qué tan bueno es el modelo?» tiene una respuesta operativa: el error en datos que el modelo no vio. En clustering no existe ese juez. El capítulo introductorio de esta parte adelantó las cuatro miradas complementarias con que se evalúa un agrupamiento: criterios **internos**, criterios **externos**, **estabilidad** e **interpretación**. Este capítulo desarrolla las tres primeras con cálculos que se pueden hacer a mano, y añade una pregunta que debería hacerse antes que todas: **¿hay grupos en los datos, para empezar?**

Usaremos los seis puntos de entrega del capítulo de k-medias, cuyas distancias ya conocemos, y compararemos dos particiones en dos grupos: la que encontró k-medias y una deliberadamente mala.

Intuición: dos maneras de partir seis puntos

Tabla 70.1.: Los seis puntos y dos particiones en dos grupos.

Punto	P1	P2	P3	P4	P5	P6
x	1	2	1	5	6	4
y	1	1	3	4	6	6
Partición buena	1	1	1	2	2	2
Partición mala	1	1	2	2	2	2

La partición mala es la que k-medias tenía después de su primera asignación (con los centros en P1 y P3): manda a P3 con los puntos de arriba a la derecha. A simple vista es peor. El objetivo es construir números que lo digan, **punto por punto y en conjunto**, sin usar ninguna etiqueta externa.

Las distancias euclidianas entre los seis puntos son:

Tabla 70.2.: Matriz de distancias euclidianas entre los seis puntos.

	P1	P2	P3	P4	P5	P6
P1	0	1.00	2.00	5.00	7.07	5.83
P2	1.00	0	2.24	4.24	6.40	5.39
P3	2.00	2.24	0	4.12	5.83	4.24
P4	5.00	4.24	4.12	0	2.24	2.24
P5	7.07	6.40	5.83	2.24	0	2.00
P6	5.83	5.39	4.24	2.24	2.00	0

Construcción paso a paso

Paso 1. La silueta de un punto

Rousseeuw (1987) propuso comparar, para cada punto, dos distancias promedio:

- $a(i)$: la distancia promedio de i a los **otros puntos de su grupo** (¿qué tan bien acompañado está?);
- $b(i)$: la distancia promedio de i a los puntos del **grupo vecino**, el grupo ajeno más cercano en promedio (¿qué tan cerca está la alternativa?).

Para P3 en la partición buena, su grupo es $\{P1, P2, P3\}$ y el único otro grupo es $\{P4, P5, P6\}$:

$$a(P3) = \frac{2.00 + 2.24}{2} = 2.12, \qquad b(P3) = \frac{4.12 + 5.83 + 4.24}{3} = 4.73.$$

La **silueta** compara ambas en una escala de -1 a 1:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \qquad s(P3) = \frac{4.73 - 2.12}{4.73} = 0.55.$$

Si $s(i)$ es cercano a 1, el punto está mucho más cerca de su grupo que de la alternativa. Cercano a 0, está en la frontera. **Negativo**, está más cerca, en promedio, de otro grupo que del suyo: probablemente está mal asignado. En la partición mala, P3 está con P4, P5 y P6:

$$a(P3) = \frac{4.12 + 5.83 + 4.24}{3} = 4.73, \qquad b(P3) = \frac{2.00 + 2.24}{2} = 2.12, \qquad s(P3) = \frac{2.12 - 4.73}{4.73} = -0.55.$$

Los papeles de a y b se intercambiaron y la silueta cambió de signo: el índice **señala al punto mal asignado**.

Paso 2. Siluetas de todos los puntos

Tabla 70.3.: Siluetas de cada punto en las dos particiones.

	P1	P2	P3	P4	P5	P6	Promedio
Buena: $a(i)$	1.50	1.62	2.12	2.24	2.12	2.12	
Buena: $b(i)$	5.97	5.34	4.73	4.46	6.44	5.15	
Buena: $s(i)$	0.75	0.70	0.55	0.50	0.67	0.59	**0.63**
Mala: $s(i)$	0.80	0.78	**−0.55**	0.38	0.50	0.50	**0.40**

El promedio de las siluetas, el **coeficiente de silueta**, resume la partición: 0.63 contra 0.40. En la partición buena, el punto peor ubicado es P4 (0.50), el más cercano al otro grupo. En la mala, P1 y P2 mejoran ligeramente (su grupo es más compacto), pero P3 se hunde y P4, P5 y P6 empeoran porque cargan con un compañero lejano.

Paso 3. Dos índices basados en centroides

La silueta requiere todas las distancias entre pares, $O(n^2)$. Otros índices solo usan los centroides, lo que los hace baratos con muchos datos.

Calinski-Harabasz. Compara la variación entre grupos B y dentro de los grupos W de la descomposición $T = W + B$ del capítulo de k-medias, cada una dividida entre sus grados de libertad, como en un análisis de varianza (Caliński y Harabasz 1974):

$$\mathrm{CH} = \frac{B/(k-1)}{W/(n-k)}. \tag{70.1}$$

Para la partición buena, $W = 8.00$ y $B = 48.33 - 8.00 = 40.33$, así que $\mathrm{CH} = \dfrac{40.33/1}{8.00/4} = 20.17$. Para la mala, $W = 21.25$ y $B = 27.08$: $\mathrm{CH} = \dfrac{27.08}{21.25/4} = 5.10$. **Mayor es mejor.**

Davies-Bouldin. Para cada grupo c calcula su dispersión S_c, la distancia promedio de sus puntos a su centroide, y lo compara con el grupo que más se le parece (Davies y Bouldin 1979):

$$\mathrm{DB} = \frac{1}{k}\sum_{c=1}^{k} \max_{c' \neq c} \frac{S_c + S_{c'}}{\|\bar{\mathbf{x}}_c - \bar{\mathbf{x}}_{c'}\|}. \tag{70.2}$$

En la partición buena, los centroides son $(1.33, 1.67)$ y $(5, 5.33)$, a 5.19 uno del otro, y las dispersiones son $S_1 = (0.75 + 0.94 + 1.37)/3 = 1.02$ y $S_2 = 1.25$. Con dos grupos, ambos términos son iguales: $\mathrm{DB} = (1.02 + 1.25)/5.19 = 0.44$. En la mala, $S_1 = 0.50$, $S_2 = 2.08$ y los centroides están a 4.51: $\mathrm{DB} = 2.58/4.51 = 0.57$. **Menor es mejor.**

Tabla 70.4.: Índices internos de las dos particiones.

Partición	Silueta (↑)	Calinski-Harabasz (↑)	Davies-Bouldin (↓)
Buena	0.63	20.17	0.44
Mala	0.40	5.10	0.57

Los tres índices coinciden. No siempre ocurre; cada uno mide la cohesión y la separación de manera distinta.

Paso 4. Si existiera una respuesta de referencia

Supongamos que alguien conoce la **zona** real de cada punto: P1, P2 y P3 están en la zona norte, y P4, P5 y P6 en la sur. Esa es exactamente la partición buena, así que comparemos la **mala** con la referencia. Los índices externos se basan en **pares** de puntos: con seis puntos hay $\binom{6}{2} = 15$ pares, y cada uno cae en una de cuatro casillas.

Tabla 70.5.: Clasificación de los 15 pares de puntos según la referencia y la partición mala.

	Juntos en la partición	Separados en la partición
Juntos en la referencia	$a = 4$	$b = 2$
Separados en la referencia	$c = 3$	$d = 6$

Por ejemplo, el par (P1, P2) está junto en ambas (a); (P1, P3) está junto en la referencia y separado en la partición (b); (P3, P4) está separado en la referencia y junto en la partición (c). El **índice de Rand** es la proporción de pares en los que ambas particiones están de acuerdo (Rand 1971):

$$\mathrm{RI} = \frac{a + d}{a + b + c + d} = \frac{4 + 6}{15} = 0.67.$$

Un 0.67 parece aceptable, pero el índice de Rand de dos particiones **al azar** rara vez es bajo, porque la mayoría de los pares están separados en ambas. El **índice de Rand ajustado** (ARI) resta lo que se esperaría por azar con los mismos tamaños de grupo (Hubert y Arabie 1985). Se calcula con la tabla de contingencia de ambas particiones:

Tabla 70.6.: Tabla de contingencia de la referencia contra la partición mala.

	Grupo 1 de la partición	Grupo 2 de la partición	Total
Norte	2	1	3
Sur	0	3	3
Total	2	4	6

Con $\binom{m}{2}$ el número de pares que se pueden formar con m elementos:

$$\begin{aligned}
&\textstyle\sum_{ij} \binom{n_{ij}}{2} = \binom{2}{2} + \binom{1}{2} + \binom{0}{2} + \binom{3}{2} = 1 + 0 + 0 + 3 = 4 \quad (= a),\\
&\textstyle\sum_{i} \binom{n_{i\cdot}}{2} = 3 + 3 = 6, \qquad \sum_{j} \binom{n_{\cdot j}}{2} = 1 + 6 = 7,\\
&\text{esperado} = \frac{6 \times 7}{15} = 2.8, \qquad \text{máximo} = \frac{6 + 7}{2} = 6.5,\\
&\mathrm{ARI} = \frac{4 - 2.8}{6.5 - 2.8} = \frac{1.2}{3.7} = 0.32.
\end{aligned}$$

El ARI vale 1 con concordancia perfecta y alrededor de 0 para particiones al azar (puede ser negativo). Aquí, 0.32: la partición mala recupera la estructura solo parcialmente, algo que el 0.67 del índice sin ajustar ocultaba.

Formalización

¿Hay estructura? La tendencia de agrupamiento

Todo algoritmo entrega grupos, incluso con datos uniformes. Con 300 puntos uniformes en un cuadrado, k-medias con $k = 4$ produce particiones con coeficiente de silueta cercano a 0.40, el mismo que la partición «mala» de nuestro ejemplo. Antes de agrupar conviene preguntar si los datos son más agrupados que el azar.

El **estadístico de Hopkins** compara dos tipos de distancias al vecino más cercano en los datos:

- $u_1, \dots, u_m$: desde m puntos generados **uniformemente** en la región que ocupan los datos hasta el dato más cercano;
- $w_1, \dots, w_m$: desde m **datos** elegidos al azar hasta su vecino más cercano entre los demás datos.

$$H = \frac{\sum_{i} u_i}{\sum_{i} u_i + \sum_{i} w_i}. \tag{70.3}$$

Si los datos son uniformes, ambos tipos de distancia se parecen y $H \approx 0.5$. Si están agrupados, los datos tienen vecinos muy cerca (w pequeñas) mientras que los puntos uniformes caen a menudo en zonas vacías (u grandes), y H se acerca a 1. Valores por encima de 0.7 indican una tendencia clara a formar grupos. Se suele usar m alrededor del 5 al 10 % de n.

Criterios internos

Tabla 70.7.: Índices internos más usados.

Índice	Mide	Rango	Mejor	Costo	Sesgo
Silueta	cohesión y separación de cada punto	$[-1, 1]$	mayor	$O(n^2)$	grupos convexos
Calinski-Harabasz	varianza entre / varianza dentro	$[0, \infty)$	mayor	$O(nk)$	grupos esféricos
Davies-Bouldin	dispersión / distancia entre centroides	$[0, \infty)$	menor	$O(nk)$	grupos esféricos

La silueta, además de un número global, produce un **diagnóstico por punto** y por grupo: el gráfico de silueta muestra las barras $s(i)$ ordenadas dentro de cada grupo. Grupos con muchas barras negativas, o mucho más delgados que el resto, delatan problemas. Su definición formal, para $|C_{\ell(i)}| > 1$, es

$$a(i) = \frac{1}{|C_{\ell(i)}| - 1} \sum_{j \in C_{\ell(i)},\, j \neq i} d(i, j), \qquad b(i) = \min_{c \neq \ell(i)} \frac{1}{|C_c|} \sum_{j \in C_c} d(i, j), \tag{70.4}$$

y $s(i) = 0$ por convención si el grupo de i tiene un solo punto. La silueta acepta cualquier distancia, por ejemplo la de Gower.

Advertencia. Los tres índices premian grupos **compactos y convexos**. En dos medias lunas entrelazadas, la partición de k-medias (que corta ambas lunas con una recta) tiene silueta 0.49, y la partición correcta, que encuentra DBSCAN, solo 0.33. Un índice interno mide qué tan bien se ajusta la partición a **su idea** de grupo, no si la partición es la verdadera. Por eso no deben usarse para comparar algoritmos con supuestos muy distintos.

Criterios externos

Sean U y V dos particiones de los mismos n casos, n_{ij} el número de casos en el grupo i de U y el j de V, y $a_i = \sum_j n_{ij}$, $b_j = \sum_i n_{ij}$. El índice de Rand ajustado es (Hubert y Arabie 1985)

$$\mathrm{ARI} = \frac{\sum_{ij}\binom{n_{ij}}{2} - \Big[\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}\Big]\Big/\binom{n}{2}}{\frac{1}{2}\Big[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\Big] - \Big[\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}\Big]\Big/\binom{n}{2}}. \tag{70.5}$$

La **información mutua normalizada** (NMI) mide cuánta información comparten las dos particiones[1]:

$$\mathrm{NMI}(U,V) = \frac{I(U;V)}{\frac{1}{2}[H(U)+H(V)]}, \qquad I(U;V) = \sum_{ij}\frac{n_{ij}}{n}\ln\frac{n\,n_{ij}}{a_i b_j}. \tag{70.6}$$

En el ejemplo, $I = 0.318$, $H(\text{referencia}) = \ln 2 = 0.693$ y $H(\text{partición}) = 0.637$, de modo que $\mathrm{NMI} = 0.318/0.665 = 0.48$. A diferencia del ARI, la NMI no está corregida por azar; con muchos grupos pequeños tiende a ser optimista, y existe una versión ajustada (AMI) (Vinh et al. 2010).

Tres reglas para usar índices externos:

1. Son **invariantes a los nombres** de los grupos: «grupo 1» y «grupo 2» pueden estar intercambiados.
2. Sirven para **comparar algoritmos** en datos con una clasificación conocida, o para ver si los grupos descubiertos coinciden con una variable que **no** se usó al agrupar (la especie de un pingüino, la región de un cliente).
3. La clasificación de referencia **no es la verdad**: los datos pueden tener una estructura distinta, igualmente válida. Si k-medias agrupa pingüinos por sexo y no por especie, no está «equivocado».

[1] En teoría de la información, la **información** (o sorpresa) de un evento con probabilidad p es $-\log p$: un evento seguro ($p = 1$) no aporta información, y uno raro aporta mucha. El idf es la sorpresa de que un documento contenga el término. La justificación probabilística formal del idf fue desarrollada después de su propuesta empírica.

¿Cuántos grupos?

Método del codo. Graficar $W(k)$ y buscar el punto donde la curva se aplana. Es simple, pero el codo a menudo es ambiguo.

Máximo de un índice. Elegir el k que maximiza la silueta o CH, o minimiza DB. Ninguno está definido para $k = 1$, de modo que no pueden decir «no hay grupos»; para eso sirve el estadístico de Hopkins o el de brecha.

Estadístico de brecha (*gap*). Tibshirani et al. (2001) compararon $\log W_k$ con el valor que tendría en datos **sin estructura**, generados uniformemente en la caja que contiene a los datos:

$$\mathrm{Gap}(k) = \mathbb{E}^*\big[\log W_k^*\big] - \log W_k, \tag{70.7}$$

donde la esperanza se estima con B conjuntos de referencia. Si hay k grupos reales, W_k será mucho menor que en datos uniformes y la brecha será grande. La regla elige el menor k tal que $\mathrm{Gap}(k) \geq \mathrm{Gap}(k+1) - s_{k+1}$, con s_{k+1} el error estándar de la simulación. A diferencia de los índices anteriores, admite $k = 1$.

Criterios de información. Si el agrupamiento es un modelo probabilístico, como una mezcla gaussiana, se puede usar el **BIC**, que penaliza la verosimilitud por el número de parámetros (ver el capítulo de mezclas gaussianas).

Estabilidad. Si los grupos son reales, deben reaparecer al repetir el análisis con otras muestras. Un procedimiento simple: tomar muchos pares de submuestras (o muestras *bootstrap*), agrupar cada una y medir el ARI entre las dos particiones sobre los casos comunes. Un k con ARI promedio alto y poca variación es estable (Luxburg 2010). La estabilidad no garantiza que los grupos sean verdaderos (un algoritmo que siempre parte por la misma recta es muy estable), pero su ausencia es una alerta seria.

Ningún criterio es definitivo. En la práctica se calculan varios, se buscan coincidencias y la decisión final la toman la **interpretabilidad** y el **uso** de los grupos.

Laboratorio: la silueta punto por punto

Haz clic en cualquier punto para ver sus distancias: líneas continuas hacia su propio grupo, que promedian $a(i)$, y discontinuas hacia el grupo vecino, que promedian $b(i)$. Las barras de la derecha son las siluetas de todos los puntos, ordenadas dentro de cada grupo. El estado inicial reproduce la partición buena y el punto P3.

Laboratorio interactivo disponible en la versión HTML. Valores de referencia: en la partición buena de los seis puntos, P3 tiene $a = 2.12$, $b = 4.73$ y $s = 0.55$; el coeficiente de silueta es 0.63, CH = 20.17 y DB = 0.44. En la partición mala, $s(P3) = -0.55$, la silueta media baja a 0.40, CH = 5.10 y DB = 0.57. Con ruido uniforme, k-medias produce siluetas cercanas a 0.4 y el estadístico de Hopkins queda cerca de 0.5.

Implementación en Python

Los índices del ejemplo

Tendencia de agrupamiento

Índices según el número de grupos

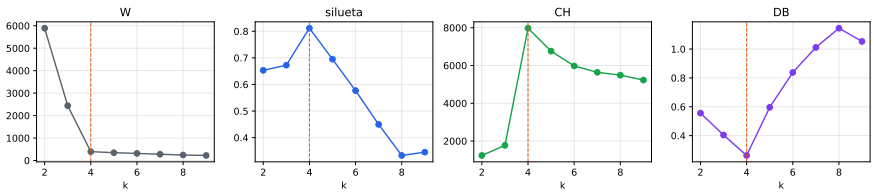


Figura 70.1.: Cuatro criterios para elegir k con k-medias sobre datos con cuatro grupos. El codo de W, el máximo de la silueta y de Calinski-Harabasz y el mínimo de Davies-Bouldin coinciden en k = 4.

El gráfico de silueta

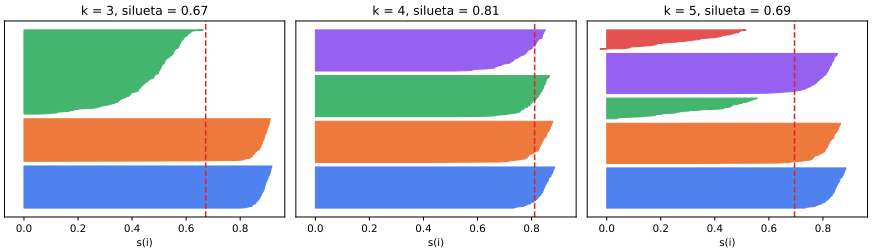


Figura 70.2.: Gráficos de silueta para k = 3, 4 y 5. Con k = 4 todos los grupos tienen barras anchas y por encima del promedio (línea roja). Con k = 3 un grupo junta dos grupos reales; con k = 5 un grupo real se parte y aparecen siluetas bajas.

El estadístico de brecha

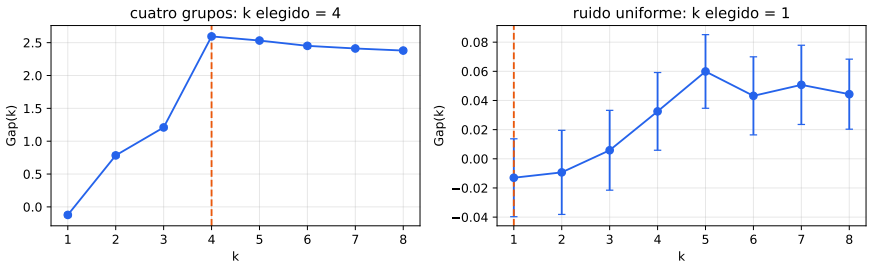


Figura 70.3.: Estadístico de brecha para los datos con cuatro grupos (izquierda) y para ruido uniforme (derecha), con barras de error estándar. La regla de Tibshirani elige k = 4 en el primer caso y k = 1 en el segundo.

Estabilidad por remuestreo

Para cada k se toman 30 pares de submuestras del 80 % de los datos, se agrupa cada una con una semilla distinta y se mide el ARI sobre los casos que ambas comparten.

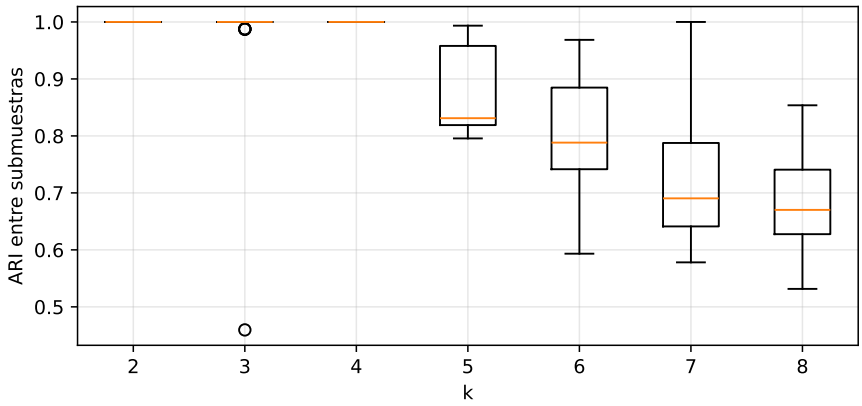


Figura 70.4.: Estabilidad de k-medias según k: ARI entre agrupamientos de pares de submuestras (80 %). Hasta k = 4 las particiones se reproducen casi siempre (con k = 2 y 3 el algoritmo junta grupos reales, pero siempre de la misma forma: estable no significa correcto); con más grupos, la forma de partir los grupos reales cambia de una submuestra a otra.

BIC con mezclas gaussianas

Cuando los índices internos engañan

La silueta prefiere la partición de k-medias, aunque la de DBSCAN recupera exactamente las dos lunas (ARI = 1).

Resumen

- Antes de agrupar, el **estadístico de Hopkins** indica si los datos son más agrupados que el azar ($H \approx 0.5$: no; H cercano a 1: sí).
- La **silueta** $s(i) = (b - a)/\max(a, b)$ evalúa cada punto: negativa señala puntos mal asignados. Su promedio resume la partición.
- **Calinski-Harabasz** (mayor es mejor) y **Davies-Bouldin** (menor es mejor) solo usan centroides; los tres índices internos favorecen grupos compactos y convexos.
- Los índices **externos** (ARI, NMI) comparan con una clasificación de referencia; el ARI corrige por azar y por eso es preferible al índice de Rand.
- Para elegir k se combinan el codo, el máximo de los índices, el **estadístico de brecha** (que admite $k = 1$), el BIC y la **estabilidad** ante remuestreo; la decisión final depende del uso de los grupos.

El siguiente capítulo presenta una familia de algoritmos que no exige fijar k de antemano: el clustering jerárquico.

Parte XXXVII.

Clustering Jerárquico y por Densidad

71. Clustering jerárquico

k-medias obliga a decidir el número de grupos antes de empezar y entrega una única partición. Muchas estructuras reales, en cambio, son **anidadas**: las especies se agrupan en géneros y los géneros en familias; los clientes de un segmento «familias jóvenes» se dividen en «con hijos pequeños» y «con adolescentes». El **clustering jerárquico** construye toda una familia de particiones anidadas, desde una en la que cada caso es su propio grupo hasta otra en la que todos forman uno solo, y la resume en un diagrama de árbol llamado **dendrograma**. El número de grupos se elige **después**, mirando el árbol.

La versión más usada es la **aglomerativa** (de abajo hacia arriba): empieza con n grupos de un caso y, en cada paso, fusiona los dos grupos más cercanos. Todo depende de una decisión: **¿qué significa que dos grupos estén cerca?** Las distintas respuestas, llamadas criterios de **enlace**, producen árboles muy diferentes. En este capítulo las calcularemos a mano sobre cinco puntos.

Intuición: cinco sucursales

Una cadena tiene cinco sucursales en una ciudad, con coordenadas en kilómetros, y quiere organizarlas en zonas de supervisión.

Tabla 71.1.: Cinco sucursales.

Sucursal	A	B	C	D	E
x (km)	3	6	2.5	7	7
y (km)	0	0	4	5	6

Las distancias euclidianas entre ellas son:

Tabla 71.2.: Matriz de distancias entre las sucursales.

	A	B	C	D	E
A	0	3.00	4.03	6.40	7.21
B	3.00	0	5.32	5.10	6.08
C	4.03	5.32	0	4.61	4.92
D	6.40	5.10	4.61	0	1.00
E	7.21	6.08	4.92	1.00	0

D y E están casi juntas; A y B también, a 3 km. La sucursal C es el caso dudoso: está a 4.03 km de A, pero a 4.61 km de D. ¿Pertenece a la zona de A y B o a la de D y E? Veremos que la respuesta depende del criterio de enlace.

Construcción paso a paso

Paso 1. Las dos primeras fusiones

El par más cercano de toda la matriz es (D, E), a distancia 1: se fusiona en el grupo $\{D, E\}$. Para continuar necesitamos la distancia de $\{D, E\}$ a cada uno de los demás, y aquí entra el criterio de enlace:

- **Enlace simple** (*single*): la distancia entre dos grupos es la de su par de puntos **más cercano**.
- **Enlace completo** (*complete*): la de su par **más lejano**.
- **Enlace promedio** (*average*, UPGMA): el **promedio** de todas las distancias entre un punto de un grupo y uno del otro.

Tabla 71.3.: Distancias del grupo

Distancia a $\{D, E\}$ desde	A	B	C
Simple: mín	6.40	5.10	4.61
Completo: máx	7.21	6.08	4.92
Promedio	6.81	5.59	4.77

En los tres criterios, el par más cercano que queda es (A, B), a 3: se fusiona en $\{A, B\}$. Hasta aquí, todos los criterios coinciden. Las dos primeras fusiones son las «obvias».

Paso 2. La decisión sobre C

Quedan tres grupos: $\{A, B\}$, $\{C\}$ y $\{D, E\}$. Las distancias entre ellos son

Tabla 71.4.: Tercera fusión según el criterio de enlace.

Criterio	$d(\{A, B\}, C)$	$d(C, \{D, E\})$	$d(\{A, B\}, \{D, E\})$	Fusión elegida
Simple	mín(4.03, 5.32) = **4.03**	mín(4.61, 4.92) = 4.61	5.10	C con $\{A, B\}$
Completo	máx(4.03, 5.32) = 5.32	máx(4.61, 4.92) = **4.92**	7.21	C con $\{D, E\}$
Promedio	(4.03 + 5.32)/2 = **4.67**	(4.61 + 4.92)/2 = 4.77	6.20	C con $\{A, B\}$

El **enlace simple** mira solo el vecino más cercano de C, que es A, y lo une a $\{A, B\}$. El **enlace completo** mira el peor caso: unir C con $\{A, B\}$ significaría tener en el mismo grupo dos sucursales a 5.32 km (C y B), mientras que unirla con $\{D, E\}$ solo exige 4.92 km. Así que la une a $\{D, E\}$. El **promedio** queda entre ambos y, por poco (4.67 contra 4.77), coincide con el simple.

Paso 3. La última fusión y las alturas

Con el enlace simple, la última fusión une $\{A, B, C\}$ con $\{D, E\}$ a la altura mín(...) = 4.61 (la distancia entre C y D). Con el completo, une $\{A, B\}$ con $\{C, D, E\}$ a la altura 7.21 (entre A y E). Con el promedio, a 5.72, el promedio de las seis distancias entre $\{A, B, C\}$ y $\{D, E\}$.

Tabla 71.5.: Historia de fusiones y alturas para cada criterio.

Fusión	Simple	Completo	Promedio
1	$\{D, E\}$ a 1.00	$\{D, E\}$ a 1.00	$\{D, E\}$ a 1.00
2	$\{A, B\}$ a 3.00	$\{A, B\}$ a 3.00	$\{A, B\}$ a 3.00
3	$\{A, B, C\}$ a 4.03	$\{C, D, E\}$ a 4.92	$\{A, B, C\}$ a 4.67
4	todos a 4.61	todos a 7.21	todos a 5.72

Paso 4. El dendrograma y su corte

El **dendrograma** dibuja esta historia: las hojas son los casos y cada fusión es un trazo horizontal a la altura en que ocurrió. Para obtener una partición se **corta** el árbol con una línea horizontal: cada rama que cruza la línea es un grupo. Con el enlace simple, cortar a la altura 4.3 deja dos grupos, $\{A, B, C\}$ y $\{D, E\}$; con el completo, cortar a 6 deja $\{A, B\}$ y $\{C, D, E\}$. Cortar más abajo, entre 3 y 4, da tres grupos en ambos casos: $\{A, B\}$, $\{C\}$, $\{D, E\}$.

Un buen lugar para cortar es donde hay un **salto grande** de altura entre fusiones consecutivas: significa que se empezaron a unir grupos que estaban lejos. Con el completo, el salto de 4.92 a 7.21 sugiere dos grupos. Con el simple, las alturas $1, 3, 4.03, 4.61$ crecen sin un salto claro: el enlace simple no ve dos zonas bien separadas, porque C hace de **puente** entre ellas.

Paso 5. Ward: fusionar lo que menos empeora

El criterio de **Ward** (Ward 1963) no mide distancias entre puntos, sino el **aumento de la suma de cuadrados dentro de los grupos** W al fusionar: el mismo criterio de k-medias, aplicado de forma ávida. Al unir los grupos G y H, con n_G y n_H casos y centroides $\bar{\mathbf{x}}_G$ y $\bar{\mathbf{x}}_H$, W aumenta en

$$\Delta(G, H) = \frac{n_G\, n_H}{n_G + n_H}\, \|\bar{\mathbf{x}}_G - \bar{\mathbf{x}}_H\|^2. \tag{71.1}$$

Al fusionar D y E: $\Delta = \frac{1}{2} \cdot 1^2 = 0.5$. Al fusionar A y B: $\frac{1}{2} \cdot 3^2 = 4.5$. Para C, con $\{A, B\}$ de centroide $(4.5, 0)$ y $\{D, E\}$ de centroide $(7, 5.5)$:

$$\Delta(C, \{A, B\}) = \tfrac{2}{3}[(2.5{-}4.5)^2{+}(4{-}0)^2] = \tfrac{2}{3}(20) = 13.33, \qquad \Delta(C, \{D, E\}) = \tfrac{2}{3}[(2.5{-}7)^2{+}(4{-}5.5)^2] = \tfrac{2}{3}(22.5) = 1$$

Ward une C con $\{A, B\}$. El factor $\frac{n_G n_H}{n_G + n_H}$ hace que fusionar dos grupos grandes cueste más que agregar un punto suelto; por eso Ward tiende a producir grupos de **tamaño parecido** y forma redondeada, como k-medias. `scipy` reporta como altura $\sqrt{2\Delta}$, de modo que la primera fusión de dos puntos aparece a la altura de su distancia euclidiana: las alturas de Ward son $1, 3, 5.16$ y 8.11.

Formalización

El algoritmo aglomerativo

Clustering jerárquico aglomerativo. Entrada: una matriz de distancias $\mathbf{D}$ entre n casos y un criterio de enlace.

1. Empezar con n grupos, uno por caso.
2. Encontrar el par de grupos (G, H) con la menor distancia de enlace $d(G, H)$.
3. Fusionarlos en $G \cup H$ y registrar la altura $d(G, H)$.
4. Actualizar las distancias del nuevo grupo a todos los demás.
5. Repetir 2–4 hasta que quede un solo grupo ($n - 1$ fusiones).

Los criterios de enlace más usados, para grupos G y H:

Tabla 71.6.: Criterios de enlace.

Enlace	$d(G, H)$	Forma de los grupos
Simple	$\min_{i \in G, j \in H} d(i, j)$	alargados, en cadena
Completo	$\max_{i \in G, j \in H} d(i, j)$	compactos, de diámetro parecido
Promedio (UPGMA)	$\frac{1}{n_G n_H} \sum_{i \in G} \sum_{j \in H} d(i, j)$	intermedio
Centroide	$\lVert \bar{\mathbf{x}}_G - \bar{\mathbf{x}}_H \rVert$	puede producir inversiones
Ward	$\sqrt{2\Delta(G, H)}$, con Δ de la Ecuación 71.1	compactos, de tamaño parecido

Simple, completo y promedio aceptan **cualquier** matriz de distancias (Gower, Jaccard, edición); centroide y Ward requieren distancias euclidianas, porque usan promedios de vectores.

Una fórmula para todos: Lance-Williams

Recalcular la distancia del nuevo grupo a los demás desde los puntos originales sería costoso. Lance y Williams (1967) observaron que todos estos criterios se pueden actualizar con una sola fórmula que usa solo las distancias entre grupos ya conocidas. Si se fusionan G y H, la distancia del nuevo grupo a otro grupo Q es

$$d(G \cup H, Q) = \alpha_G \, d(G, Q) + \alpha_H \, d(H, Q) + \beta \, d(G, H) + \gamma \, |d(G, Q) - d(H, Q)|, \tag{71.2}$$

con coeficientes según el criterio:

Tabla 71.7.: Coeficientes de Lance-Williams.

Enlace	α_G	α_H	β	γ
Simple	$1/2$	$1/2$	0	$-1/2$

Enlace	α_G	α_H	β	γ
Completo	$1/2$	$1/2$	0	$1/2$
Promedio	$\frac{n_G}{n_G+n_H}$	$\frac{n_H}{n_G+n_H}$	0	0
Ward (sobre d^2)	$\frac{n_G+n_Q}{n_G+n_H+n_Q}$	$\frac{n_H+n_Q}{n_G+n_H+n_Q}$	$\frac{-n_Q}{n_G+n_H+n_Q}$	0

Por ejemplo, para el enlace completo con $G = A$, $H = B$, $Q = C$: $\frac{1}{2}(4.03)+\frac{1}{2}(5.32)+\frac{1}{2}|4.03-5.32| = 2.015 + 2.66 + 0.645 = 5.32$, que es el máximo, como debía. El truco es que $\frac{1}{2}(a+b) \pm \frac{1}{2}|a-b|$ es máx(a, b) o mín(a, b).

Monotonía e inversiones

Con enlace simple, completo, promedio y Ward, las alturas de las fusiones **nunca bajan**: cada fusión ocurre a una altura mayor o igual que la anterior. Por eso el dendrograma se puede cortar con una línea horizontal sin ambigüedad. El enlace de **centroide** no tiene esta propiedad: al fusionar dos grupos, el centroide nuevo puede quedar más cerca de un tercero que los centroides anteriores, y aparece una **inversión** (una fusión más baja que la anterior), que hace al dendrograma difícil de leer.

La distancia cofenética

El dendrograma define una nueva distancia entre casos: la **distancia cofenética** $c(i, j)$ es la altura a la que i y j quedan por primera vez en el mismo grupo. Con el enlace simple en las sucursales, $c(A, B) = 3$, $c(A, C) = c(B, C) = 4.03$, $c(D, E) = 1$ y la distancia cofenética entre cualquier sucursal de $\{A, B, C\}$ y cualquiera de $\{D, E\}$ es 4.61. Esta distancia es una **ultramétrica**: cumple $c(i, k) \leq$ máx$\{c(i, j), c(j, k)\}$, una versión más fuerte de la desigualdad triangular en la que todo triángulo es isósceles con los dos lados mayores iguales.

¿Qué tan fielmente representa el árbol a los datos? La **correlación cofenética** (Sokal y Rohlf 1962) es la correlación de Pearson entre las $n(n-1)/2$ distancias originales $d(i, j)$ y las cofenéticas $c(i, j)$. En las sucursales vale 0.88 con enlace simple, 0.87 con completo, 0.89 con promedio y 0.87 con Ward. El enlace promedio suele dar la correlación cofenética más alta, porque precisamente promedia distancias. Valores por encima de 0.8 indican que el árbol resume bien las distancias, pero una correlación alta no garantiza grupos útiles.

El efecto cadena y el árbol de expansión mínima

El enlace simple tiene una propiedad notable: sus fusiones son exactamente las aristas del **árbol de expansión mínima** (MST)[1] de los datos, agregadas de menor a mayor peso. Cortar el dendrograma de enlace simple a la altura h equivale a borrar del MST las aristas de peso mayor que h: los grupos son las componentes que quedan conectadas. Por eso el enlace simple puede encontrar grupos de

[1] Un **árbol de expansión mínima** de un grafo con pesos es un conjunto de aristas que conecta todos los vértices sin formar ciclos y cuya suma de pesos es la menor posible. Aquí el grafo es completo: cada par de casos está unido por una arista cuyo peso es su distancia. El algoritmo de Kruskal lo construye agregando aristas de menor a mayor peso siempre que no cierren un ciclo, que es exactamente lo que hace el enlace simple.

cualquier forma (medias lunas, anillos), pero también sufre el **efecto cadena**: una fila de puntos intermedios, como la sucursal C, une dos grupos que deberían estar separados. Esta conexión con el MST reaparecerá en HDBSCAN, en el siguiente capítulo.

Clustering divisivo

La alternativa **divisiva** (de arriba hacia abajo) empieza con todos los casos en un grupo y lo parte sucesivamente. DIANA (Kaufman y Rousseeuw 1990) separa en cada paso el caso más «inconforme» de su grupo y le va sumando los casos que están más cerca de él que del resto; k-medias bisectante, mencionado en el capítulo anterior, es otro ejemplo. Los métodos divisivos son atractivos cuando interesan sobre todo las primeras divisiones (pocos grupos grandes), pero son más costosos y menos usados.

Costo y escalabilidad

El algoritmo aglomerativo necesita la matriz de distancias completa: $O(n^2)$ de memoria. Con $n = 50\,000$ son más de mil millones de distancias, unos 10 GB en doble precisión. El algoritmo ingenuo cuesta $O(n^3)$ en tiempo; con estructuras adecuadas se reduce a $O(n^2 \log n)$ y, para el enlace simple (algoritmo SLINK) y Ward, a $O(n^2)$ (Murtagh y Contreras 2012). En la práctica, el clustering jerárquico es cómodo hasta unas decenas de miles de casos. Para más, se agrupa una muestra, se usan restricciones de **conectividad** (solo se permite fusionar grupos vecinos en un grafo de k vecinos), o se combina con k-medias: primero cientos de microgrupos, luego un árbol sobre sus centros.

Laboratorio: construye el dendrograma

Avanza fusión por fusión y observa el dendrograma crecer. En el plano, el segmento resaltado muestra **qué par de puntos** decide la última fusión: el par más cercano en el enlace simple, el más lejano en el completo; en el promedio y en Ward, que dependen de todos los puntos, se dibuja como referencia la línea entre los centroides de los dos grupos. Mueve la línea de corte para obtener una partición. El estado inicial reproduce las cinco sucursales con enlace simple.

Laboratorio interactivo disponible en la versión HTML. Con las cinco sucursales, las alturas de las fusiones son $1, 3, 4.03, 4.61$ (simple), $1, 3, 4.92, 7.21$ (completo), $1, 3, 4.67, 5.72$ (promedio) y $1, 3, 5.16, 8.11$ (Ward). Solo el enlace completo une la sucursal C con $\{D, E\}$. Las correlaciones cofenéticas son 0.884, 0.865, 0.891 y 0.868.

Implementación en Python

El ejemplo con `scipy`

`scipy.cluster.hierarchy.linkage` devuelve la matriz de enlace $\mathbf{Z}$, de $n-1$ filas: cada fila contiene los dos grupos fusionados (los índices mayores o iguales a n se refieren a grupos creados antes), la altura y el tamaño del nuevo grupo.

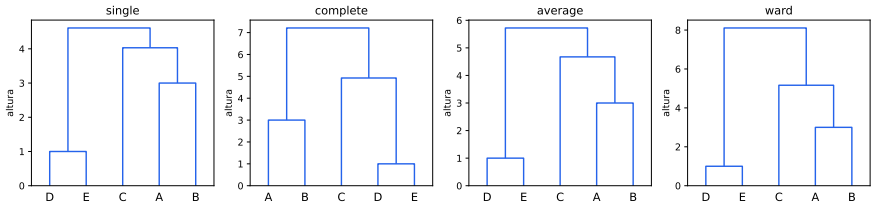


Figura 71.1.: Dendrogramas de las cinco sucursales con cuatro criterios de enlace. Solo el enlace completo une C con D y E. El enlace completo y Ward muestran un salto grande antes de la última fusión (dos grupos claros); el simple no.

Lance-Williams desde cero

La implementación siguiente aplica la Ecuación 71.2 y reproduce las alturas de `scipy`, incluida la convención de Ward ($\sqrt{2\Delta}$, aplicando la fórmula sobre distancias al cuadrado).

El efecto cadena y las formas no convexas

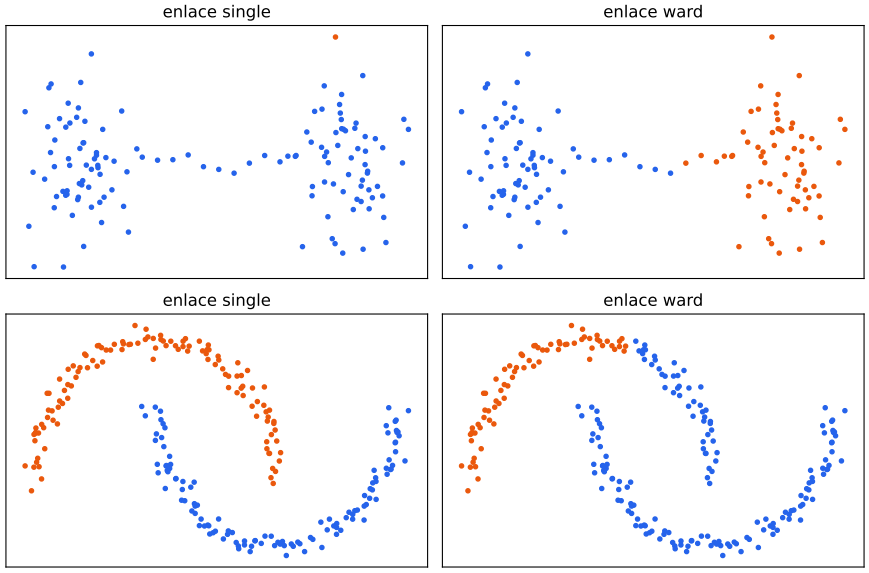


Figura 71.2.: Enlace simple frente a Ward cortando en dos grupos. Arriba, dos grupos unidos por un puente de puntos: el enlace simple separa un punto aislado y junta todo lo demás (efecto cadena), mientras que Ward separa los dos grupos. Abajo, dos medias lunas: el enlace simple las recupera y Ward las corta.

Restricciones de conectividad

`AgglomerativeClustering` acepta un grafo de **conectividad**: solo se fusionan grupos que son vecinos en él. Con un grafo de k vecinos, incluso Ward respeta formas alargadas, y el cálculo se acelera porque solo se consideran fusiones locales.

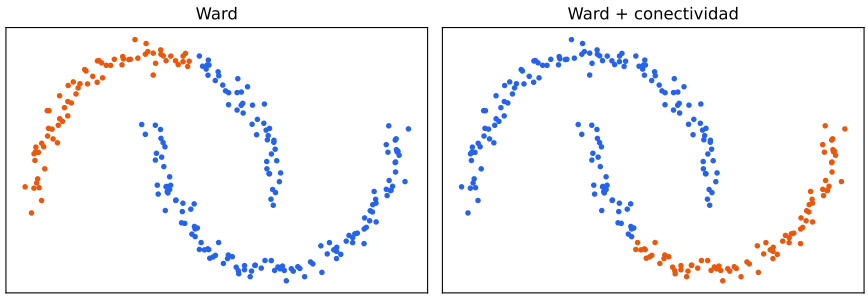


Figura 71.3.: Ward sin restricciones (izquierda) y con un grafo de 10 vecinos como restricción de conectividad (derecha), sobre dos medias lunas. La restricción obliga a fusionar solo grupos contiguos.

Agrupar variables: un mapa de correlaciones ordenado

El clustering jerárquico no solo agrupa casos. Con la distancia $1 - |r|$ entre variables, el dendrograma reúne las variables que miden cosas parecidas, y su orden de hojas sirve para reordenar una matriz de correlaciones y hacer visibles los bloques.

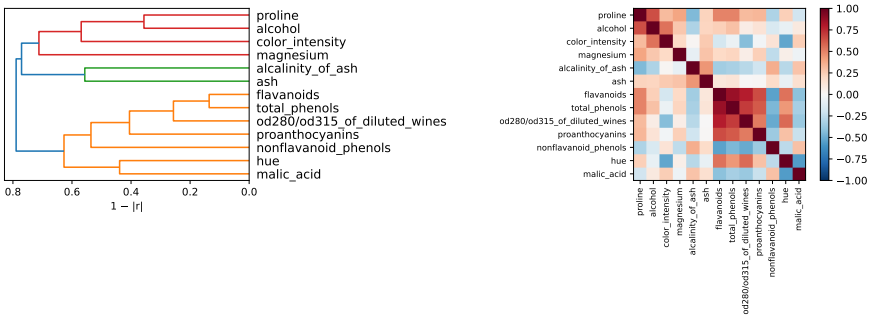


Figura 71.4.: Dendrograma de las 13 variables del conjunto wine con distancia $1 - |r|$ y enlace promedio (izquierda) y matriz de correlaciones reordenada según las hojas del dendrograma (derecha). Los bloques de variables correlacionadas aparecen juntos.

Cómo leer (y no leer) un dendrograma

- La información está en la **altura** de las fusiones, no en la distancia horizontal entre hojas. Dos hojas vecinas en el dibujo pueden estar muy lejos: el orden de las hojas es en parte arbitrario (cada fusión se puede «voltear»).
- Un **salto grande** entre alturas consecutivas sugiere un buen corte; varios saltos parecidos indican que no hay un número de grupos claramente mejor.
- Las ramas largas y aisladas suelen ser **atípicos**: casos que se unen muy tarde a todo lo demás.
- El dendrograma **siempre** existe, aunque los datos no tengan estructura jerárquica. La correlación cofenética y la validación del capítulo anterior siguen siendo necesarias.

Resumen

- El clustering jerárquico aglomerativo fusiona en cada paso los dos grupos más cercanos y registra la altura; el resultado es un **dendrograma**, que se corta para obtener grupos.
- El **criterio de enlace** define la distancia entre grupos: simple (par más cercano), completo (más lejano), promedio y Ward (aumento de la suma de cuadrados). Con las cinco sucursales, solo el enlace completo unió C con D y E.
- La fórmula de **Lance-Williams** actualiza todas estas distancias con cuatro coeficientes.
- El enlace simple equivale al **árbol de expansión mínima**: encuentra formas arbitrarias pero sufre el efecto cadena. Completo y Ward producen grupos compactos.
- La **correlación cofenética** mide qué tan bien el árbol conserva las distancias originales.
- Necesita $O(n^2)$ de memoria; para muchos datos se usan muestras, restricciones de conectividad o combinaciones con k-medias.

El siguiente capítulo toma otro camino: definir los grupos como **regiones densas** separadas por zonas vacías, sin fijar k y reconociendo explícitamente el ruido.

72. DBSCAN, OPTICS y HDBSCAN

k-medias y el clustering jerárquico con enlace completo o Ward comparten una idea de grupo: una nube **compacta** alrededor de un centro. Muchas estructuras reales no son así. Los reportes de delitos se concentran a lo largo de avenidas; los clientes de una tienda en línea forman manchas de formas irregulares en un mapa; las órbitas de satélites dibujan anillos. Además, en casi cualquier conjunto de datos hay casos que **no pertenecen a ningún grupo**, y obligarlos a entrar en uno distorsiona el resultado.

El clustering **basado en densidad** cambia la definición: un grupo es una **región densa** del espacio, separada de otras regiones densas por zonas de baja densidad, y los casos en zonas vacías son **ruido**. DBSCAN (Ester et al. 1996), el algoritmo más conocido de esta familia, no necesita saber cuántos grupos hay, encuentra formas arbitrarias y separa el ruido. A cambio, pide dos parámetros que definen qué significa «denso». Este capítulo lo construye a mano y luego presenta sus dos sucesores, OPTICS y HDBSCAN, que resuelven su principal debilidad: grupos de densidades distintas.

Intuición: reportes de fallas en el alumbrado

Una alcaldía recibe reportes georreferenciados de luminarias fundidas. Quiere identificar **zonas** con fallas concentradas, para enviar una cuadrilla a cada una, y distinguirlas de reportes aislados. Estos son 13 reportes, con coordenadas en cientos de metros:

Tabla 72.1.: Trece reportes de fallas de alumbrado.

Reporte	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13
x	1	2	1	2	3	4	6	7	6	7	8	4.3	8.5
y	1	1	2	2	2	2.5	5	5	6	6	6.5	4.2	2

Un criterio razonable: **una zona tiene fallas concentradas si un reporte tiene al menos otros tres reportes a 150 m o menos**. Esos dos números, el radio y el número mínimo de vecinos, son los parámetros de DBSCAN: $\varepsilon = 1.5$ (en cientos de metros) y $\text{minPts} = 4$, contando al propio reporte[1].

[1] Hay dos convenciones: algunos textos cuentan al propio punto en minPts y otros no. Aquí, como en `scikit-learn` (`min_samples`), **se cuenta**: un punto con exactamente tres vecinos a distancia $\leq \varepsilon$ tiene una vecindad de tamaño 4 y es núcleo con $\text{minPts} = 4$.

Construcción paso a paso

Paso 1. Contar vecinos dentro del radio

La **vecindad** ε de un reporte es el conjunto de reportes a distancia menor o igual que ε, incluido él mismo. Calculamos, para cada reporte, cuántos tiene:

Tabla 72.2.: Vecindades de radio 1.5 y tipo de cada reporte con minPts = 4.

Reporte	Vecinos a ≤ 1.5 (distancia)	Tamaño de la vecindad	Tipo
Q1	Q2 (1.00), Q3 (1.00), Q4 (1.41)	4	núcleo
Q2	Q1 (1.00), Q3 (1.41), Q4 (1.00), Q5 (1.41)	5	núcleo
Q3	Q1 (1.00), Q2 (1.41), Q4 (1.00)	4	núcleo
Q4	Q1 (1.41), Q2 (1.00), Q3 (1.00), Q5 (1.00)	5	núcleo
Q5	Q2 (1.41), Q4 (1.00), Q6 (1.12)	4	núcleo
Q6	Q5 (1.12)	2	borde
Q7	Q8 (1.00), Q9 (1.00), Q10 (1.41)	4	núcleo
Q8	Q7 (1.00), Q9 (1.41), Q10 (1.00)	4	núcleo
Q9	Q7 (1.00), Q8 (1.41), Q10 (1.00)	4	núcleo
Q10	Q7 (1.41), Q8 (1.00), Q9 (1.00), Q11 (1.12)	5	núcleo
Q11	Q10 (1.12)	2	borde
Q12	—	1	ruido
Q13	—	1	ruido

Paso 2. Núcleo, borde y ruido

- Un punto es **núcleo** si su vecindad tiene al menos minPts puntos. Está en el interior de una zona densa. Aquí: Q1–Q5 y Q7–Q10.
- Un punto es **borde** si no es núcleo pero está en la vecindad de algún núcleo. Está en la orilla de una zona densa. Q6 tiene solo dos puntos en su vecindad, pero uno de ellos, Q5, es núcleo. Lo mismo Q11 con Q10.
- Un punto es **ruido** si no es núcleo ni borde. Q12 está a 1.73 de su vecino más cercano y Q13 a 3.35: nadie está a 1.5 o menos.

Paso 3. Formar los grupos

Los grupos se forman encadenando núcleos. Empezamos en Q1, que es núcleo: abre el **grupo 1**. Agregamos sus vecinos Q2, Q3 y Q4; como Q2 y Q4 también son núcleos, agregamos sus vecinos, entre ellos Q5; Q5 es núcleo y agrega a Q6. Q6 entra al grupo, pero como no es núcleo **no sigue expandiendo**: la cadena se detiene en la orilla. No quedan puntos por revisar en esta cadena: el grupo 1 es $\{Q1, \dots, Q6\}$.

El siguiente punto sin revisar es Q7, núcleo: abre el **grupo 2**, que por el mismo proceso crece a $\{Q7, \dots, Q11\}$. Q12 y Q13 no son núcleos ni están en la vecindad de ninguno: son ruido.

Hay una asimetría importante. Q6 es **alcanzable** desde Q5 (está en la vecindad de un núcleo), pero Q5 **no** es alcanzable desde Q6, porque Q6 no es núcleo: desde un borde no se puede «saltar». Los bordes pertenecen a un grupo, pero no lo extienden.

Paso 4. El efecto de los parámetros

Tabla 72.3.: Resultados de DBSCAN con distintos parámetros.

ε	minPts	Grupo 1	Grupo 2	Ruido
1.0	4	Q2–Q5	—	los otros nueve
1.2	4	Q2–Q5	Q8–Q11	Q1, Q6, Q7, Q12, Q13
1.5	**4**	**Q1–Q6**	**Q7–Q11**	**Q12, Q13**
2.0	4	Q1–Q6	Q7–Q12	Q13
2.6	4	Q1–Q12	—	Q13
1.5	5	Q1–Q5	Q7–Q11	Q6, Q12, Q13

Con ε muy pequeño casi todo es ruido; con ε muy grande las dos zonas se funden en una. Subir minPts exige más densidad: con 5, Q5 deja de ser núcleo y Q6 queda como ruido.

Paso 5. Elegir ε con las k-distancias

Para cada punto calculamos su k**-distancia**: la distancia a su k-ésimo vecino más cercano, con $k = \text{minPts} - 1 = 3$. Un punto es núcleo exactamente cuando su k-distancia es menor o igual que ε. Ordenadas de menor a mayor:

$$1.00,\ 1.12,\ \underbrace{1.41,\ 1.41,\ 1.41,\ 1.41,\ 1.41,\ 1.41,\ 1.41}_{\text{núcleos típicos}},\ 2.06,\ 2.06,\ 2.48,\ 4.27.$$

Los puntos dentro de las zonas densas tienen k-distancias parecidas y pequeñas; los bordes y el ruido, grandes. La curva da un **salto** entre 1.41 y 2.06: cualquier ε en ese intervalo deja como núcleos a los nueve puntos de las zonas densas. Esta es la heurística del **gráfico de k-distancias** propuesta por los autores de DBSCAN: ordenar las k-distancias y elegir ε en el «codo».

Formalización

Definiciones

Sea d una distancia, $\varepsilon > 0$ y $\text{minPts} \geq 1$.

- **Vecindad ε:** $N_\varepsilon(p) = \{q : d(p,q) \leq \varepsilon\}$, que incluye a p.
- **Punto núcleo:** $|N_\varepsilon(p)| \geq \text{minPts}$.
- **Directamente alcanzable por densidad:** q lo es desde p si p es núcleo y $q \in N_\varepsilon(p)$. No es simétrica.
- **Alcanzable por densidad:** q lo es desde p si existe una cadena $p = p_1, p_2, \ldots, p_m = q$ en la que cada p_{i+1} es directamente alcanzable desde p_i. Todos los eslabones, salvo quizá el último, son núcleos.
- **Conectados por densidad:** p y q lo están si existe un núcleo o desde el que ambos son alcanzables. Esta relación sí es simétrica.

Un **grupo** es un conjunto no vacío C que es **maximal** (si $p \in C$ y q es alcanzable desde p, entonces $q \in C$) y **conexo** (todos sus puntos están conectados por densidad entre sí). El **ruido** son los puntos que no pertenecen a ningún grupo.

El algoritmo

DBSCAN. Para cada punto p todavía sin etiqueta:

1. Calcular $N_\varepsilon(p)$. Si $|N_\varepsilon(p)| < \text{minPts}$, marcar p como ruido (provisionalmente) y continuar.
2. Si no, crear un grupo nuevo C, asignarle p y poner los vecinos de p en una cola.
3. Mientras la cola no esté vacía, sacar un punto q:
   - si q estaba marcado como ruido, reasignarlo a C (es un borde);
   - si q no tenía etiqueta, asignarlo a C y, si q es núcleo, agregar sus vecinos a la cola.

Costo. El trabajo dominante son las consultas de vecindad, una por punto. Con un índice espacial (árbol KD o de bolas) cada consulta cuesta del orden de $\log n$ en dimensiones bajas, y el total es aproximadamente $O(n \log n)$; sin índice, o en alta dimensión, es $O(n^2)$ (Schubert et al. 2017). A diferencia del clustering jerárquico, no necesita guardar la matriz de distancias completa.

Determinismo. Los núcleos y el ruido no dependen del orden en que se recorren los puntos. Los **bordes** sí pueden depender: un borde que está en la vecindad de núcleos de dos grupos distintos se queda en el primero que lo alcance. Es un efecto menor, pero explica pequeñas diferencias entre implementaciones.

Elegir los parámetros

- **minPts** controla cuántos puntos hacen falta para hablar de densidad y suaviza el resultado. Una regla común es $\text{minPts} \geq p + 1$ en p dimensiones, y $\text{minPts} = 2p$ como punto de partida; con datos ruidosos o muchos casos conviene aumentarlo (Schubert et al. 2017).

- ε se elige con el gráfico de k-distancias, con $k = \text{minPts} - 1$. Si el codo no es claro, probablemente los grupos tienen **densidades distintas**.
- Ambos dependen de la **escala** de las variables: hay que estandarizar antes, como en cualquier método basado en distancias.

Fortalezas y limitaciones

DBSCAN encuentra grupos de **forma arbitraria**, no requiere k e identifica **ruido**. Pero un único ε define una única densidad: si una zona es mucho más densa que otra, no hay un valor que funcione para ambas. Con ε chico, la zona dispersa se vuelve ruido; con ε grande, las zonas densas cercanas se funden. Además, en alta dimensión las distancias se concentran y las vecindades pierden sentido.

Laboratorio: DBSCAN paso a paso

Recorre el algoritmo punto por punto. El círculo es la vecindad ε del punto que se está procesando; la cola muestra los puntos pendientes de expandir. A la derecha, el gráfico de k-distancias con la línea horizontal en el ε actual: los puntos bajo la línea son núcleos. El estado inicial reproduce los 13 reportes.

Laboratorio interactivo disponible en la versión HTML. Con los 13 reportes, $\varepsilon = 1.5$ y $\text{minPts} = 4$: dos grupos, $\{Q1, \dots, Q6\}$ y $\{Q7, \dots, Q11\}$, con 9 núcleos, 2 bordes (Q6 y Q11) y 2 puntos de ruido (Q12 y Q13). El gráfico de k-distancias ($k = 3$) tiene su codo entre 1.41 y 2.06.

OPTICS: todas las densidades a la vez

DBSCAN responde a una pregunta con un ε fijo. OPTICS (Ankerst et al. 1999) calcula, en una sola pasada, un **orden** de los puntos y una **distancia de alcanzabilidad** para cada uno, de los que se puede leer el resultado de DBSCAN para **cualquier** ε.

Distancia de núcleo. La menor distancia que haría a p ser núcleo: la distancia a su $(\text{minPts} - 1)$-ésimo vecino, que es su k-distancia. Para Q1 es 1.41; para Q6, 2.06; para Q13, 4.27.

Distancia de alcanzabilidad de o desde p:

$$\text{reach}(o \mid p) = \text{máx}\left\{\text{core}(p),\ d(p, o)\right\}. \tag{72.1}$$

Es la distancia de p a o, pero nunca menor que lo que p necesita para ser núcleo. Desde un punto en zona densa, alcanzar a un vecino cuesta poco; desde un punto en zona dispersa, cuesta al menos su distancia de núcleo.

El orden. OPTICS empieza en un punto cualquiera y, en cada paso, visita el punto pendiente con la **menor** alcanzabilidad desde los ya visitados, como el algoritmo de Prim para árboles de expansión mínima. Así recorre por completo una zona densa antes de saltar a otra. Con los 13 reportes y $\text{minPts} = 4$, el orden y las alcanzabilidades son:

Tabla 72.4.: Orden de OPTICS y distancias de alcanzabilidad.

Orden	Q1	Q2	Q3	Q4	Q5	Q6	Q12	Q7	Q8	Q9	Q10	Q11	Q13
Alcanzabilidad	∞	1.41	1.41	1.41	1.00	1.41	2.06	2.48	1.41	1.41	1.41	1.12	3.35

El **gráfico de alcanzabilidad** dibuja estas barras en ese orden. Los **valles** son grupos: Q1–Q6 y Q7–Q11 forman dos valles a altura ≈ 1.4, separados por los picos de Q12 (2.06) y Q7 (2.48), que es el «salto» de una zona a otra. Cortar el gráfico con una línea horizontal a altura ε' equivale a ejecutar DBSCAN con ese ε': cada valle por debajo de la línea es un grupo. Y si hay grupos de densidades distintas, aparecen como valles de **profundidades** distintas, todos visibles en el mismo gráfico.

Laboratorio: el gráfico de alcanzabilidad

Mueve el umbral sobre el gráfico de alcanzabilidad y observa los grupos que resultan en el plano. Con los datos de densidades distintas, prueba cortes a distintas alturas: el gráfico muestra a la vez los valles profundos de los grupos densos y el valle ancho y poco profundo del disperso.

Laboratorio interactivo disponible en la versión HTML. Con los 13 reportes y minPts = 4, el orden de OPTICS es Q1–Q6, Q12, Q7–Q11, Q13. Cortar el gráfico de alcanzabilidad a $\varepsilon' = 1.8$ produce los grupos $\{Q1, \dots, Q6\}$ y $\{Q7, \dots, Q11\}$; Q12 y Q13 quedan como ruido.

HDBSCAN: elegir la densidad de cada grupo

HDBSCAN (Campello et al. 2013) lleva la idea de OPTICS a una jerarquía completa y elige automáticamente los grupos más **persistentes**. Sus pasos:

1. **Distancia de alcanzabilidad mutua.** Para cada par de puntos,

$$d_{\text{mr}}(a, b) = \max\{\text{core}(a),\ \text{core}(b),\ d(a, b)\}. \tag{72.2}$$

   Los puntos en zonas densas conservan sus distancias; los de zonas dispersas quedan «empujados» lejos de todos. Por ejemplo, $d_{\text{mr}}(Q5, Q6) = \max\{1.41, 2.06, 1.12\} = 2.06$: aunque Q5 y Q6 están a 1.12, Q6 vive en una zona más dispersa.
2. **Árbol de expansión mínima** sobre d_{mr} y, a partir de él, una **jerarquía de enlace simple**, como en el capítulo anterior. Con d_{mr}, el efecto cadena se atenúa porque los puntos de puente quedan lejos.
3. **Árbol condensado.** Se recorre la jerarquía de arriba hacia abajo, aumentando la densidad $\lambda = 1/d$. Cuando un grupo se divide, solo cuentan como grupos nuevos las ramas con al menos `min_cluster_size` puntos; las ramas más pequeñas se consideran puntos que «se caen» del grupo.

4. **Estabilidad y selección.** Cada grupo C tiene una estabilidad $\sum_{p \in C}(\lambda_p - \lambda_{\text{nace}}(C))$, donde λ_p es la densidad a la que el punto p abandona el grupo. Un grupo que persiste en un rango amplio de densidades es estable. HDBSCAN elige el conjunto de grupos, sin anidarse entre sí, con la mayor estabilidad total.

El resultado es un DBSCAN con un ε **distinto para cada grupo**, sin necesidad de fijarlo. HDBSCAN solo pide `min_cluster_size` (el tamaño mínimo de un grupo de interés), que es mucho más intuitivo que ε, y opcionalmente `min_samples`. Además entrega una **probabilidad de pertenencia** de cada punto a su grupo y una puntuación de atipicidad (GLOSH), útil en el capítulo de detección de anomalías.

Comparación de algoritmos de clustering

Tabla 72.5.: Comparación de los algoritmos de clustering de esta parte.

	k-medias	Jerárquico (Ward)	Jerárquico (simple)	DBSCAN	HDBSCAN	Mezcla gaussiana
Parámetro principal	k	corte o k	corte o k	ε, minPts	tamaño mínimo	k, covarianza
Formas	convexas	convexas	arbitrarias	arbitrarias	arbitrarias	elipses
Densidades distintas	regular	regular	mal	mal	bien	bien
Ruido explícito	no	no	no	sí	sí	no (baja probabilidad)
Casos nuevos	`predict`	no	no	no	aproximado	`predict`
Escala	millones	decenas de miles	decenas de miles	millones (con índice)	cientos de miles	cientos de miles

Implementación en Python

El ejemplo de los reportes

En `scikit-learn`, `min_samples` cuenta al propio punto, igual que en este capítulo, y el ruido recibe la etiqueta -1.

El gráfico de k-distancias

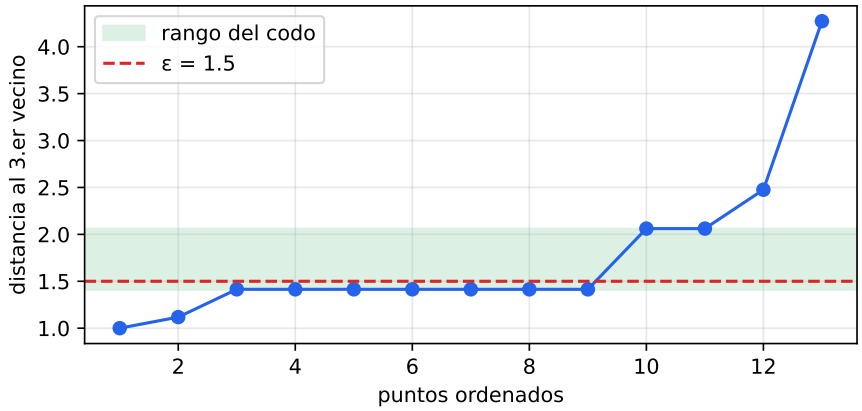


Figura 72.1.: Gráfico de k-distancias (k = 3) de los 13 reportes. El salto entre 1.41 y 2.06 indica el rango razonable de .

Formas arbitrarias y ruido

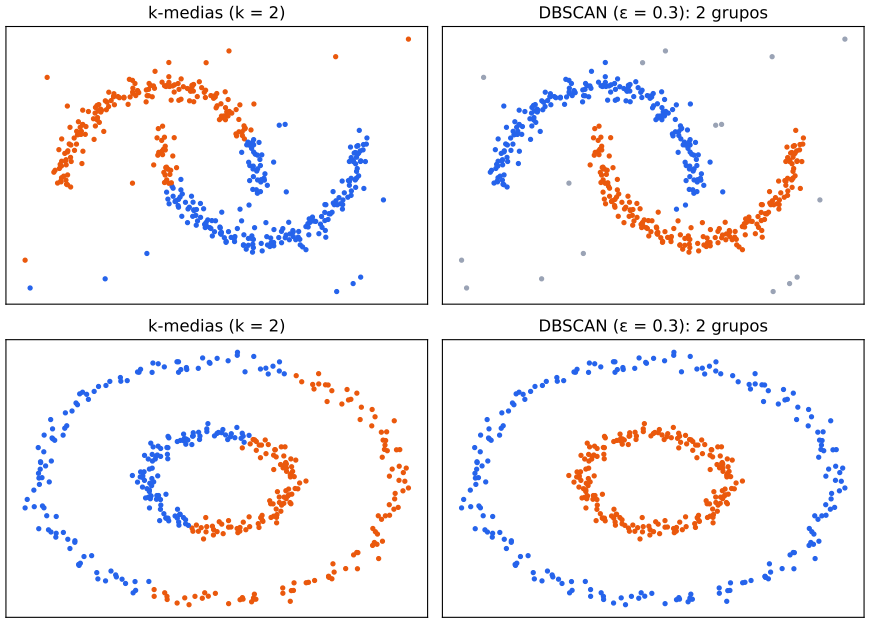


Figura 72.2.: k-medias frente a DBSCAN en dos conjuntos no convexos con ruido. DBSCAN recupera las lunas y los anillos y marca el ruido en gris; k-medias corta las formas con rectas.

Densidades distintas: DBSCAN, OPTICS y HDBSCAN

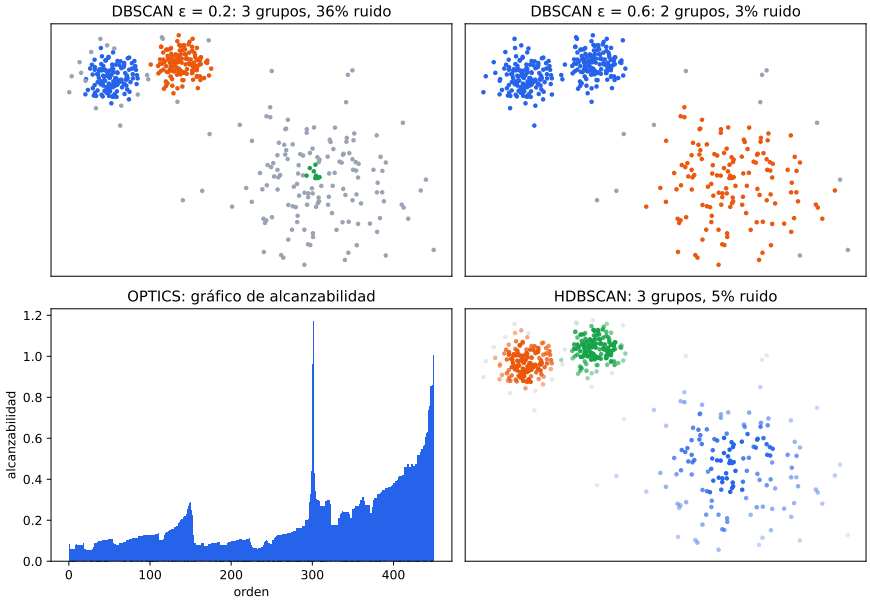


Figura 72.3.: Tres grupos, dos densos y uno disperso. Arriba: DBSCAN con un pequeño (el grupo disperso se vuelve ruido) y con uno grande (los grupos densos se funden). Abajo: gráfico de alcanzabilidad de OPTICS, con valles de distinta profundidad, y HDBSCAN, que encuentra los tres grupos.

La transparencia de los puntos en el panel de HDBSCAN es su probabilidad de pertenencia: los puntos en la periferia del grupo disperso son menos seguros.

HDBSCAN en los reportes

HDBSCAN coincide con DBSCAN en los dos grupos y en Q13 como ruido, pero incorpora a Q12 al primer grupo con una probabilidad baja: con densidad variable, Q12 es una orilla lejana más que un punto aislado.

Coordenadas geográficas

Con latitud y longitud, la distancia correcta es la de **haversine** (sobre la esfera), que `DBSCAN` acepta con `metric="haversine"` y coordenadas en radianes. El radio se expresa en radianes: $\varepsilon = \text{metros}/6\,371\,000$.

Resumen

- DBSCAN define un grupo como una región densa: puntos **núcleo** (al menos minPts vecinos a distancia $\leq \varepsilon$), puntos **borde** (en la vecindad de un núcleo) y **ruido** (todo lo demás).

- Los grupos se forman encadenando núcleos; los bordes se agregan pero no expanden. No requiere k y encuentra formas arbitrarias.
- ε se elige en el codo del **gráfico de k-distancias**; minPts suele ser al menos la dimensión más uno. Ambos dependen de la escala.
- Un único ε no sirve para grupos de densidades distintas. **OPTICS** resume todas las ε en un gráfico de alcanzabilidad; **HDBSCAN** construye la jerarquía con la distancia de alcanzabilidad mutua y elige los grupos más estables, con un parámetro intuitivo: el tamaño mínimo de grupo.

Los puntos de ruido de DBSCAN y las puntuaciones de HDBSCAN son una primera forma de encontrar casos raros. El siguiente capítulo convierte esa idea en una tarea en sí misma: la **detección de anomalías**.

Parte XXXVIII.

Detección de Anomalías

73. Detección de anomalías

Una **anomalía** (*outlier*, valor atípico) es un caso que se aparta tanto del resto que hace sospechar que lo generó un mecanismo distinto: un fraude entre miles de compras legítimas, una falla incipiente en las lecturas de un sensor, un error de captura en una base de datos, un intruso en el tráfico de una red. Detectarlas es una de las tareas más valiosas de la minería de datos, y casi siempre es **no supervisada**: las anomalías son raras, cambian con el tiempo y casi nunca vienen etiquetadas (Chandola et al. 2009).

Ya nos hemos topado con ellas varias veces. En el análisis exploratorio usamos el rango intercuartílico para marcar valores extremos de **una** variable. En k-medias vimos que un solo punto lejano puede quedarse con un centro completo. DBSCAN etiqueta como **ruido** a los puntos en zonas vacías. Este capítulo convierte esas observaciones en métodos: cada uno define «raro» de una manera distinta, y la pregunta central es **¿raro respecto a qué?**

Intuición: una lectura que no está fuera de rango

Una máquina registra la **temperatura** (°C) y la **presión** (kPa) de un proceso. Las dos variables se mueven juntas: al subir la temperatura, sube la presión. Estas son diez lecturas; la décima llegó esta mañana.

Tabla 73.1.: Diez lecturas de un sensor.

Lectura	1	2	3	4	5	6	7	8	9	**10**
Temperatura (°C)	60	62	64	66	68	70	72	74	76	**69**
Presión (kPa)	120	125	127	133	135	141	143	149	151	**126**

Ni 69 °C ni 126 kPa son valores extremos: ambos están dentro del rango habitual. Pero 69 °C suele venir con unos 137 kPa, no con 126. Una presión baja para esa temperatura puede indicar una fuga. ¿Qué métodos la detectan?

Construcción paso a paso

Paso 1. Una variable a la vez

La **puntuación** z mide a cuántas desviaciones estándar está un valor de la media de su variable. Con las diez lecturas, la temperatura tiene media 68.1 y desviación 5.17, y la presión media 135 y desviación 10.68. Para la lectura 10:

$$z_{\text{temp}} = \frac{69 - 68.1}{5.17} = 0.17, \qquad z_{\text{pres}} = \frac{126 - 135}{10.68} = -0.84.$$

La regla habitual marca como atípico un $|z| > 3$ (o > 2, siendo más estrictos). La lectura 10 pasa sin problema; de hecho, las lecturas 1 y 9 tienen puntuaciones más extremas (± 1.5). Las **cercas de Tukey** del capítulo de EDA, $[Q_1 - 1.5\,\text{RIC},\ Q_3 + 1.5\,\text{RIC}]$, dan $[54, 82]$ para la temperatura y $[101.9, 166.9]$ para la presión: tampoco la marcan. Los métodos univariados **no pueden** ver una anomalía que solo existe en la relación entre variables.

Paso 2. La distancia de Mahalanobis con todos los datos

La distancia de Mahalanobis (1936), vista en el capítulo de similitud, mide la distancia al centro de los datos **teniendo en cuenta su forma**: su escala y su correlación. Con las diez lecturas, la correlación es 0.93 y la distancia de la lectura 10 al centro resulta $d_M = 2.76$.

¿Es mucho? Si los datos fueran normales en p dimensiones, d_M^2 seguiría aproximadamente una distribución χ^2 con p grados de libertad[1]. Con $p = 2$, el percentil 97.5 de χ^2_2 es 7.38, es decir, $d_M = 2.72$. La lectura 10 apenas supera el umbral. Algo no cuadra: en la gráfica salta a la vista que está fuera de la nube.

Paso 3. El enmascaramiento: estimar sin el sospechoso

El problema es que la media y la covarianza se calcularon **incluyendo** a la lectura 10. Al estar fuera de la tendencia, ella misma ensancha la covarianza en su dirección y reduce la correlación (de 0.996 a 0.93): la anomalía **se esconde a sí misma**. A esto se le llama **enmascaramiento** (*masking*).

Si estimamos el modelo con las nueve lecturas de referencia, la media es $\bar{\mathbf{x}} = (68, 136)$ y

$$\mathbf{S} = \begin{pmatrix} 30 & 59 \\ 59 & 117 \end{pmatrix}, \qquad \mathbf{S}^{-1} = \frac{1}{30 \cdot 117 - 59^2} \begin{pmatrix} 117 & -59 \\ -59 & 30 \end{pmatrix} = \frac{1}{29} \begin{pmatrix} 117 & -59 \\ -59 & 30 \end{pmatrix}.$$

Para la lectura 10, la diferencia con la media es $\delta = (1, -10)$ y

$$d_M^2 = \delta^\top \mathbf{S}^{-1} \delta = \frac{117(1)^2 - 2(59)(1)(-10) + 30(-10)^2}{29} = \frac{117 + 1180 + 3000}{29} = 148.2, \qquad d_M = 12.17.$$

[1] Si $\mathbf{x}$ sigue una normal multivariada, $\Sigma^{-1/2}(\mathbf{x} - \mu)$ tiene p componentes normales estándar independientes, y d_M^2 es la suma de sus cuadrados. La suma de los cuadrados de p normales estándar independientes sigue, por definición, una distribución **ji cuadrada** con p grados de libertad. Con parámetros estimados y pocos datos la aproximación es solo eso, una aproximación.

Ahora la lectura está a más de doce «desviaciones» del comportamiento normal, cuatro veces el umbral. El término dominante es el cruzado, +1180: la temperatura sube un poco mientras la presión baja mucho, justo lo contrario de lo que dice la correlación. Este es el principio de los estimadores **robustos** de covarianza: calcular la media y la covarianza con la parte de los datos que parece «limpia», para que las anomalías no contaminen la referencia.

Paso 4. Anomalías locales: el factor LOF

Mahalanobis supone una sola nube elíptica. Con varios grupos de densidades distintas se necesita otra idea. Consideremos siete valores en una recta: un grupo **denso** $\{0, 0.2, 0.4\}$, un grupo **disperso** $\{5, 7, 9\}$ y un punto $p = 1.4$.

Una primera medida de rareza es la **distancia al k-ésimo vecino**: lejos de todos, raro. Con $k = 2$:

Tabla 73.2.: Distancia al segundo vecino más cercano.

Punto	0	0.2	0.4	**1.4**	5	7	9
Distancia al 2.º vecino	0.4	0.2	0.4	**1.2**	3.6	2.0	4.0

Según esta medida, los más raros son 9, 5 y 7, y p ocupa el cuarto lugar. Pero 5, 7 y 9 son un grupo normal, solo que disperso; en cambio, p está a 1 de su vecino más cercano cuando en su vecindario los puntos están a 0.2 unos de otros. Es raro **para su zona**.

El **factor de anomalía local** (LOF) (Breunig et al. 2000) compara la densidad de cada punto con la de sus vecinos:

1. **Distancia de alcanzabilidad** de p a su vecino o: $\text{reach}_k(p, o) = \max\{k\text{-dist}(o),\ d(p, o)\}$, la misma idea que en OPTICS. Para p y sus dos vecinos, 0.4 y 0.2: $\text{reach}(p, 0.4) = \max\{0.4, 1.0\} = 1.0$ y $\text{reach}(p, 0.2) = \max\{0.2, 1.2\} = 1.2$.
2. **Densidad local de alcanzabilidad**: el inverso de la alcanzabilidad promedio, $\text{lrd}(p) = 1/\frac{1.0+1.2}{2} = 0.909$. Para el punto 0, con vecinos 0.2 y 0.4: alcanzabilidades $\max\{0.2, 0.2\} = 0.2$ y $\max\{0.4, 0.4\} = 0.4$, así que $\text{lrd}(0) = 1/0.3 = 3.33$. De la misma forma, $\text{lrd}(0.2) = 2.5$ y $\text{lrd}(0.4) = 3.33$.
3. **LOF**: el promedio de las densidades de los vecinos dividido entre la densidad propia,

$$\text{LOF}(p) = \frac{\frac{1}{2}\big[\text{lrd}(0.4) + \text{lrd}(0.2)\big]}{\text{lrd}(p)} = \frac{(3.33 + 2.5)/2}{0.909} = 3.21.$$

Tabla 73.3.: Densidad local y factor LOF con k = 2.

Punto	0	0.2	0.4	**1.4**	5	7	9
lrd	3.33	2.50	3.33	0.91	0.36	0.26	0.33
LOF	0.88	1.33	0.88	**3.21**	1.64	1.31	0.93

Un LOF cercano a 1 significa «tan denso como mis vecinos»: normal, sea cual sea la densidad de la zona. Mayor que 1, menos denso que ellos. Ahora p es, con diferencia, el más anómalo, y el grupo disperso tiene valores cercanos a 1.

Paso 5. Aislar en lugar de medir: Isolation Forest

Liu et al. (2008) partieron de otra observación: las anomalías son **pocas y diferentes**, así que son **fáciles de aislar**. Si partimos el intervalo $[0, 9]$ con un corte al azar, el punto 9 queda solo cuando el corte cae entre 7 y 9, con probabilidad $2/9 \approx 0.22$ en el primer intento. Para aislar el punto 0.2, en cambio, hacen falta varios cortes muy precisos. Repitiendo muchos árboles de cortes aleatorios, la **profundidad promedio** a la que se aísla cada punto es

Tabla 73.4.: Profundidad promedio de aislamiento (20 000 árboles simulados) y puntuación de Isolation Forest.

Punto	0	0.2	0.4	1.4	5	7	9
Profundidad promedio $\mathbb{E}[h]$	3.43	4.10	3.81	2.93	2.80	2.86	**2.14**
Puntuación s	0.46	0.39	0.42	0.51	0.53	0.52	**0.61**

La puntuación $s = 2^{-\mathbb{E}[h]/c(n)}$ normaliza la profundidad (la fórmula está en la formalización): cerca de 1 es anómalo y por debajo de 0.5, normal. Isolation Forest señala al punto 9, el más alejado de todo, y no distingue mucho a p del grupo disperso. Como la distancia al k-ésimo vecino, es un método más bien **global**. Cada detector tiene su idea de rareza.

Formalización

Tipos de anomalía y de problema

- **Puntual**: un caso raro respecto al resto (la lectura 10).
- **Contextual**: raro solo en un contexto. 30 °C es normal en mayo y anómalo en enero en la misma ciudad; las variables de contexto (fecha, ubicación) se usan para definir con quién comparar.
- **Colectiva**: un conjunto de casos que solo es anómalo junto, como una secuencia de transacciones pequeñas que juntas forman un patrón de fraude.

Según la información disponible, el problema puede ser:

Tabla 73.5.: Escenarios de detección de anomalías.

Escenario	Datos de entrenamiento	Enfoque
Supervisado	etiquetas de normal y anómalo	clasificación con clases muy desbalanceadas (parte de clasificación)
Semisupervisado (*novelty detection*)	solo casos normales	aprender lo normal y medir el alejamiento de casos nuevos
No supervisado (*outlier detection*)	datos mezclados, sin etiquetas	suponer que las anomalías son pocas y diferentes

El Paso 3 fue semisupervisado: usamos nueve lecturas que sabíamos normales. El Paso 2, no supervisado, sufrió enmascaramiento.

Métodos estadísticos

Univariados. La puntuación z, su versión **robusta** con la mediana y la desviación absoluta mediana[2], $z^* = 0.6745\,(x - \text{mediana})/\text{MAD}$, y las cercas de Tukey. Son simples y explicables, pero solo ven una variable a la vez.

Mahalanobis. Para datos aproximadamente normales con media μ y covarianza Σ,

$$d_M(\mathbf{x})^2 = (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \sim \chi^2_p, \tag{73.1}$$

y se marca como anómalo un caso con $d_M^2 > \chi^2_{p,1-\alpha}$. Los contornos de igual d_M son elipses alineadas con la nube.

Robustez: el determinante de covarianza mínimo (MCD). Para evitar el enmascaramiento, el estimador MCD busca el subconjunto de h casos (por ejemplo, el 75 %) cuya matriz de covarianza tiene el **menor determinante**, es decir, la nube más compacta, y estima la media y la covarianza solo con ellos (Rousseeuw y Van Driessen 1999). El algoritmo FAST-MCD alterna «pasos C»: calcular la media y la covarianza de un subconjunto, ordenar todos los casos por su distancia de Mahalanobis a ese modelo y quedarse con los h más cercanos. Cada paso C reduce el determinante, igual que cada paso de Lloyd reduce W en k-medias. En `scikit-learn` se usa con `MinCovDet` o `EllipticEnvelope`.

Métodos basados en distancia y densidad

Distancia a los k vecinos (Ramaswamy et al. 2000): la puntuación es la distancia al k-ésimo vecino (o el promedio de las distancias a los k vecinos). No supone ninguna forma, pero usa una escala **global**: confunde grupos dispersos con anomalías.

LOF (Breunig et al. 2000). Con $N_k(p)$ los k vecinos de p:

[2]La **desviación absoluta mediana** es $\text{MAD} = \text{mediana}(|x_i - \text{mediana}(x)|)$. El factor 0.6745 hace que, para datos normales, z^* sea comparable con la puntuación z habitual. A diferencia de la media y la desviación estándar, la mediana y la MAD apenas cambian con unos pocos valores extremos.

$$\text{reach}_k(p,o) = \text{máx}\{k\text{-dist}(o), d(p,o)\}, \qquad \text{lrd}_k(p) = \left(\frac{1}{k}\sum_{o\in N_k(p)} \text{reach}_k(p,o)\right)^{-1}, \qquad \text{LOF}_k(p) = \frac{1}{k}\sum_{o\in N_k(p)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(p)} \tag{73.2}$$

Usar k-dist(o) en la alcanzabilidad suaviza la estimación: todos los puntos muy cercanos a o reciben la misma alcanzabilidad. Un LOF de 1 es normal; valores de 1.5 o 2 suelen considerarse sospechosos, pero el umbral depende de los datos. El parámetro k define qué tan «local» es la comparación: valores entre 10 y 50 son habituales.

Ruido de DBSCAN y GLOSH. Los puntos de ruido de DBSCAN son anomalías con una regla binaria; HDBSCAN produce una puntuación continua (GLOSH) basada en la densidad a la que cada punto abandona su grupo.

Isolation Forest

Un **árbol de aislamiento** se construye sobre una submuestra de ψ casos (por omisión, 256): en cada nodo se elige una variable al azar y un punto de corte uniforme entre el mínimo y el máximo de esa variable en el nodo, hasta aislar cada caso o alcanzar una profundidad máxima de $\lceil \log_2 \psi \rceil$. La **longitud de camino** $h(\mathbf{x})$ es el número de cortes hasta llegar a la hoja de $\mathbf{x}$, más una corrección si la hoja tiene varios casos. Para comparar entre tamaños de muestra se normaliza con la longitud promedio de una búsqueda fallida en un árbol binario de búsqueda de n elementos:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \qquad H(i) \approx \ln i + 0.5772,$$

y la puntuación es

$$s(\mathbf{x}, n) = 2^{-\mathbb{E}[h(\mathbf{x})]/c(n)}. \tag{73.3}$$

Si $\mathbb{E}[h] \to 0$, $s \to 1$ (anómalo); si $\mathbb{E}[h] = c(n)$, $s = 0.5$. Para $n = 7$, $c(7) = 3.02$; para $\psi = 256$, $c(256) = 10.24$. Isolation Forest no calcula distancias, escala linealmente con n, funciona bien con muchas variables y no le afecta la escala de cada variable por separado. Submuestrear es parte del diseño: con pocos casos, las anomalías agrupadas no se enmascaran unas a otras.

Otros enfoques

- **One-Class SVM** (Schölkopf et al. 2001): aprende una frontera que encierra a la mayoría de los datos en el espacio de un núcleo, en la línea de las máquinas de soporte vectorial.
- **Error de reconstrucción**: con PCA o con un autoencoder (capítulo de t-SNE, UMAP y autoencoders), los casos normales se reconstruyen bien con pocas componentes y los anómalos no.
- **Modelos de densidad**: una mezcla gaussiana o una estimación de densidad de núcleo; anómalo es lo que tiene baja densidad $\widehat{p}(\mathbf{x})$.

Evaluar con pocas etiquetas

Casi todos los detectores producen una **puntuación**, no una decisión. El umbral se fija con el parámetro de **contaminación** (la proporción esperada de anomalías) o con la capacidad de revisión: si un equipo puede investigar 50 alertas al día, se revisan las 50 de mayor puntuación. Si existen algunas etiquetas, se usan las métricas del capítulo de evaluación de clasificadores, con dos preferencias: la **precisión en los primeros** N (de las N alertas más altas, cuántas eran reales) y el **área bajo la curva de precisión-exhaustividad**, mucho más informativa que la ROC cuando las anomalías son el 1 % de los datos.

Laboratorio: cuatro detectores, cuatro ideas de rareza

Los datos tienen un grupo denso con correlación fuerte, un grupo disperso y seis **anomalías plantadas** (marcadas con un anillo). El fondo muestra la puntuación del detector en cada punto del plano: cuanto más intenso, más anómalo. Los puntos con contorno grueso son las alertas: el porcentaje de casos con mayor puntuación según la contaminación elegida. Arrastra el rombo para medir cualquier posición.

Laboratorio interactivo disponible en la versión HTML. El laboratorio compara cuatro detectores sobre un grupo denso correlacionado, un grupo disperso y seis anomalías plantadas. La covarianza clásica de Mahalanobis se infla con todos los datos y deja escapar anomalías entre los grupos; la distancia a los k vecinos marca como sospechosa la periferia del grupo disperso; LOF detecta las anomalías pegadas al grupo denso; Isolation Forest produce contornos rectangulares porque corta en paralelo a los ejes.

Implementación en Python

El sensor: puntuación z, Mahalanobis y covarianza robusta

`EmpiricalCovariance` usa la covarianza con divisor n y `MinCovDet` aplica además una corrección de consistencia, por lo que sus valores difieren ligeramente de los calculados a mano con divisor $n-1$; la conclusión es la misma: solo con una referencia limpia o robusta la lectura 10 destaca con claridad.

LOF y la distancia a los vecinos en el ejemplo de la recta

`scikit-learn` guarda el LOF con signo negativo (`negative_outlier_factor_`) para que «mayor sea más normal», la convención de todos sus detectores.

Comparación en datos con anomalías plantadas

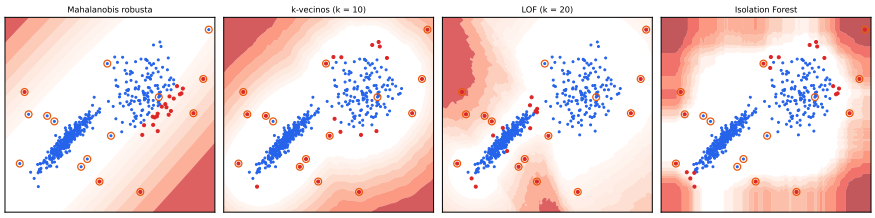


Figura 73.1.: Puntuación de cuatro detectores sobre un grupo denso correlacionado, un grupo disperso y 15 anomalías plantadas (anillos naranjas). El color de fondo es la puntuación (más oscuro, más anómalo) y los puntos rojos son las alertas con contaminación del 5 %.

Evaluar con las etiquetas plantadas

La referencia de la precisión promedio (AP) es la proporción de anomalías: un detector al azar obtiene ese valor. Una ROC AUC alta puede convivir con una AP modesta cuando las anomalías son escasas, por eso conviene mirar ambas.

Detección de novedades

Cuando se tiene un conjunto de entrenamiento que se sabe normal, los detectores se ajustan con él y se aplican a datos nuevos. En `LocalOutlierFactor` hay que pedirlo explícitamente con `novelty=True`.

`predict` devuelve 1 para normal y -1 para anómalo. La envolvente elíptica, ajustada solo con el grupo denso, considera anómalo todo lo que no se le parezca, incluido el centro del grupo disperso: un modelo de «lo normal» solo sabe de lo que vio.

Guía de elección

Tabla 73.6.: Elección de un detector de anomalías.

Situación	Detector sugerido
Una variable, explicación sencilla	puntuación z robusta, cercas de Tukey
Pocas variables, nube elíptica	Mahalanobis con covarianza robusta (MCD)
Grupos de densidades distintas	LOF, HDBSCAN (GLOSH)
Muchos casos o muchas variables	Isolation Forest
Datos de referencia que se saben normales	detección de novedades (LOF con `novelty=True`, One-Class SVM)
Imágenes, señales, texto	error de reconstrucción de un autoencoder

En la práctica conviene combinar varios detectores y revisar a mano las alertas de mayor puntuación: una anomalía es una **hipótesis** que un experto debe confirmar.

Resumen

- Una anomalía es un caso que parece generado por otro mecanismo; puede ser puntual, contextual o colectiva, y el problema puede ser supervisado, semisupervisado o no supervisado.
- Los métodos **univariados** no ven anomalías en la relación entre variables. **Mahalanobis** sí, pero sufre **enmascaramiento**: con todos los datos, la lectura del sensor tenía $d_M = 2.76$; con una referencia limpia, 12.17. La covarianza **robusta** (MCD) resuelve el problema.
- La distancia a los k vecinos es global; **LOF** compara la densidad de cada punto con la de sus vecinos y detecta anomalías **locales** (LOF = 3.21 en el ejemplo).
- **Isolation Forest** mide qué tan fácil es aislar un caso con cortes aleatorios; es rápido, escala bien y es más bien global.
- Los detectores producen puntuaciones; el umbral sale de la contaminación esperada o de la capacidad de revisión, y con pocas etiquetas se evalúa con precisión en los primeros N y área PR.

Los últimos capítulos de esta parte cambian de tarea: en lugar de agrupar o señalar casos, buscan **representaciones** de pocas dimensiones que conserven la estructura de los datos.

Parte XXXIX.

Reducción de Dimensionalidad

74. Escalamiento multidimensional e Isomap

El capítulo de PCA, en la parte de preprocesamiento, presentó la reducción de dimensionalidad como un problema de **varianza**: encontrar las direcciones en las que los datos varían más y proyectarlos sobre ellas. Hay otra forma de plantear el mismo deseo: **dibujar los datos en pocas dimensiones de manera que las distancias del dibujo se parezcan a las distancias reales**. Es lo que hace un cartógrafo al dibujar un mapa, y lo que necesitamos cada vez que queremos «ver» datos de cincuenta variables en una pantalla de dos.

Este capítulo desarrolla esa idea en dos etapas. Primero, el **escalamiento multidimensional** (MDS), que reconstruye coordenadas a partir de una matriz de distancias y que, con distancias euclidianas, coincide exactamente con PCA. Después, los métodos de **variedades** (*manifold learning*), que reconocen que la distancia en línea recta puede ser engañosa cuando los datos viven sobre una superficie curva, y la reemplazan por la distancia **a lo largo** de los datos. El siguiente capítulo completará el panorama con t-SNE y UMAP, que renuncian a conservar distancias y se concentran en los vecindarios.

Intuición: reconstruir un mapa a partir de distancias

Imagina que alguien te da la tabla de distancias en kilómetros entre cuatro ciudades, pero no sus coordenadas, y te pide dibujar el mapa. Si las ciudades forman un rectángulo de 4 por 3 km, la tabla sería:

Tabla 74.1.: Distancias entre cuatro ciudades.

	A	B	C	D
A	0	4	5	3
B	4	0	3	5
C	5	3	0	4
D	3	5	4	0

Con un poco de ingenio se reconstruye a mano: A y B a 4 km, C a 3 km de B y a 5 km de A (un triángulo 3-4-5, rectángulo), y D cierra el rectángulo. Pero con cien ciudades el ingenio no alcanza. El MDS clásico convierte este rompecabezas en un problema de álgebra lineal con solución exacta.

Construcción paso a paso

Paso 1. Elevar al cuadrado

El MDS clásico, propuesto por Torgerson (1952), trabaja con las distancias **al cuadrado**, $D_{ij}^{(2)} = d_{ij}^2$:

$$\mathbf{D}^{(2)} = \begin{pmatrix} 0 & 16 & 25 & 9 \\ 16 & 0 & 9 & 25 \\ 25 & 9 & 0 & 16 \\ 9 & 25 & 16 & 0 \end{pmatrix}.$$

Paso 2. Doble centrado: de distancias a productos punto

Si las ciudades tuvieran coordenadas $\mathbf{x}_i$ con centro en el origen, el **producto punto** $b_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$ se relacionaría con la distancia por la ley de los cosenos: $d_{ij}^2 = b_{ii} + b_{jj} - 2b_{ij}$. Despejar b_{ij} requiere conocer b_{ii} y b_{jj}, y aquí entra el truco del **doble centrado**: restar a cada elemento el promedio de su fila y el de su columna, sumarle el promedio general y multiplicar por $-\frac{1}{2}$,

$$\mathbf{B} = -\tfrac{1}{2}\,\mathbf{J}\,\mathbf{D}^{(2)}\,\mathbf{J}, \qquad \mathbf{J} = \mathbf{I} - \tfrac{1}{n}\mathbf{1}\mathbf{1}^\top.$$

En nuestro caso, cada fila y cada columna de $\mathbf{D}^{(2)}$ suma 50 (promedio 12.5) y el promedio general es 12.5. Para el elemento (A, B): $b_{AB} = -\frac{1}{2}(16 - 12.5 - 12.5 + 12.5) = -1.75$. Para (A, A): $b_{AA} = -\frac{1}{2}(0 - 12.5 - 12.5 + 12.5) = 6.25$. Completo:

$$\mathbf{B} = \begin{pmatrix} 6.25 & -1.75 & -6.25 & 1.75 \\ -1.75 & 6.25 & 1.75 & -6.25 \\ -6.25 & 1.75 & 6.25 & -1.75 \\ 1.75 & -6.25 & -1.75 & 6.25 \end{pmatrix}.$$

$\mathbf{B}$ es la matriz de productos punto de unas coordenadas centradas que reproducirían exactamente las distancias de la tabla.

Paso 3. Descomposición en valores propios

Como $\mathbf{B} = \mathbf{X}\mathbf{X}^\top$, sus vectores propios dan las direcciones de las coordenadas y sus valores propios[1], la varianza en cada una. Los valores propios de $\mathbf{B}$ son $\lambda_1 = 16$, $\lambda_2 = 9$ y dos ceros. Que haya exactamente dos valores propios distintos de cero dice que las distancias se pueden reproducir **exactamente en dos dimensiones**, y no menos.

[1] Un **vector propio** $\mathbf{v}$ de una matriz $\mathbf{B}$ es una dirección que la matriz solo estira: $\mathbf{B}\mathbf{v} = \lambda\mathbf{v}$, y el factor de estiramiento λ es su **valor propio**. Para una matriz de productos punto $\mathbf{X}\mathbf{X}^\top$, los valores propios miden cuánta variación de los datos hay en cada dirección, como en el capítulo de PCA.

Paso 4. Las coordenadas

Las coordenadas son los vectores propios unitarios multiplicados por la raíz de su valor propio, $\mathbf{X} = \mathbf{V}\Lambda^{1/2}$:

Tabla 74.2.: Coordenadas recuperadas por el MDS clásico.

Ciudad	A	B	C	D
Coordenada 1 ($\sqrt{16} = 4$ por el vector propio)	-2	2	2	-2
Coordenada 2 ($\sqrt{9} = 3$ por el vector propio)	1.5	1.5	-1.5	-1.5

Es el rectángulo de 4 por 3, centrado en el origen. El mapa es correcto salvo por una rotación o un reflejo: las distancias no tienen orientación, de modo que cualquier giro del mapa es igualmente válido.

Paso 5. ¿Y si solo hubiera una dimensión?

Quedarnos con la primera coordenada da $(-2, 2, 2, -2)$: A y D caen en el mismo punto, igual que B y C, aunque estén a 3 km. La proporción de la «varianza» que conserva una dimensión es $\lambda_1/(\lambda_1 + \lambda_2) = 16/25 = 64\,\%$, exactamente como en PCA.

Paso 6. Cuando la línea recta engaña

Ahora siete estaciones de un tren que recorre un arco de circunferencia de radio 2 km, separadas cada 45°, desde $-135°$ hasta 135°. Entre estaciones consecutivas hay 1.53 km. Pero la distancia **en línea recta** entre la primera y la última es solo 2.83 km, porque el arco casi se cierra sobre sí mismo. Si aplicamos MDS a las distancias en línea recta y pedimos una sola dimensión, obtenemos las coordenadas

$$(-1.41,\ -2.00,\ -1.41,\ 0,\ 1.41,\ 2.00,\ 1.41),$$

que **pliegan** el recorrido: las estaciones 1 y 3 caen en el mismo lugar, y la 1 queda más cerca de la 7 que de la 4. Una dimensión conserva solo el 58 % de la variación.

Para un pasajero, la distancia relevante es la del recorrido del tren, la **geodésica**. Construyamos un grafo que une solo a las estaciones vecinas (las que están a menos de 1.6 km) y midamos las distancias como caminos más cortos en ese grafo: de la estación 1 a la 7 hay $6 \times 1.53 = 9.18$ km. Con estas distancias geodésicas, el MDS en una dimensión da

$$(4.59,\ 3.06,\ 1.53,\ 0,\ -1.53,\ -3.06,\ -4.59),$$

las estaciones **en orden** y con la separación correcta, y la primera dimensión conserva el 100 % de la variación. Hemos «desenrollado» el arco. Este es el algoritmo **Isomap**.

Formalización

MDS clásico

Dada una matriz de distancias $\mathbf{D}$ entre n casos:

1. Calcular $\mathbf{B} = -\frac{1}{2}\mathbf{J}\mathbf{D}^{(2)}\mathbf{J}$.
2. Obtener los q valores propios mayores de $\mathbf{B}$, $\lambda_1 \geq \cdots \geq \lambda_q$, y sus vectores propios unitarios $\mathbf{v}_1, \ldots, \mathbf{v}_q$.
3. Las coordenadas son $\mathbf{Y} = [\sqrt{\lambda_1}\mathbf{v}_1, \ldots, \sqrt{\lambda_q}\mathbf{v}_q]$.

Si $\mathbf{D}$ proviene de puntos en un espacio euclidiano, $\mathbf{B}$ es semidefinida positiva y la solución minimiza el error $\sum_{ij}(b_{ij} - \mathbf{y}_i^\top \mathbf{y}_j)^2$ entre productos punto. Si $\mathbf{D}$ no es euclidiana (por ejemplo, una distancia de Gower o de edición), aparecen valores propios **negativos**, que indican cuánto se aleja la tabla de cualquier configuración euclidiana; se descartan.

MDS y PCA son lo mismo cuando $\mathbf{D}$ es la distancia euclidiana entre los datos. Si $\mathbf{X}$ es la matriz de datos centrada, PCA diagonaliza la covarianza $\frac{1}{n}\mathbf{X}^\top\mathbf{X}$ ($p \times p$) y MDS diagonaliza $\mathbf{B} = \mathbf{X}\mathbf{X}^\top$ ($n \times n$). Ambas matrices tienen los mismos valores propios distintos de cero y las coordenadas de MDS son exactamente los puntajes de PCA[2]. La ventaja de MDS es que solo necesita las **distancias**, no las variables: sirve para mapas de similitud entre productos calificados por consumidores, entre textos o entre secuencias.

MDS métrico y no métrico

El MDS clásico tiene solución cerrada, pero conserva mejor las distancias grandes que las pequeñas. El **MDS métrico** minimiza directamente el **estrés**, la discrepancia entre las distancias originales y las del dibujo:

$$\text{estrés}(\mathbf{Y}) = \sqrt{\frac{\sum_{i<j}\left(d_{ij} - \|\mathbf{y}_i - \mathbf{y}_j\|\right)^2}{\sum_{i<j} d_{ij}^2}}, \tag{74.1}$$

mediante un algoritmo iterativo (SMACOF, en `scikit-learn`). El **MDS no métrico** de Kruskal (1964) solo pide conservar el **orden** de las distancias, útil cuando provienen de juicios humanos («A se parece más a B que a C») cuya escala no es confiable. Un estrés menor que 0.05 se considera excelente y mayor que 0.2, pobre.

[2]Si $\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^\top$ es la descomposición en valores singulares de los datos centrados, entonces $\mathbf{X}^\top\mathbf{X} = \mathbf{V}\Sigma^2\mathbf{V}^\top$ (PCA) y $\mathbf{X}\mathbf{X}^\top = \mathbf{U}\Sigma^2\mathbf{U}^\top$ (MDS). Los puntajes de PCA son $\mathbf{X}\mathbf{V} = \mathbf{U}\Sigma$, exactamente las coordenadas $\mathbf{V}_{\text{MDS}}\Lambda^{1/2}$ de MDS con $\Lambda = \Sigma^2$.

La hipótesis de la variedad

Muchos datos de alta dimensión viven, aproximadamente, sobre una **variedad** de dimensión mucho menor[3]. Las fotografías de una cara que gira tienen miles de píxeles, pero solo dos grados de libertad reales (los ángulos de giro): el conjunto de esas imágenes es una superficie de dimensión 2 doblada dentro de un espacio de miles de dimensiones. Sobre una variedad curva, la distancia en línea recta (euclidiana) puede «atajar» por fuera de los datos, como en el arco del tren, mientras que la distancia **geodésica**, medida a lo largo de la variedad, refleja la estructura real.

Isomap

Tenenbaum et al. (2000) propusieron aproximar las geodésicas con un grafo de vecinos:

1. **Grafo de vecindad.** Unir cada punto con sus k vecinos más cercanos (o con los que estén a menos de ε), con peso igual a su distancia euclidiana. Localmente, en distancias cortas, la línea recta y la geodésica casi coinciden.
2. **Caminos más cortos.** Calcular la distancia de camino más corto entre todos los pares de puntos del grafo (algoritmos de Dijkstra o Floyd-Warshall). Esa es la aproximación de la geodésica.
3. **MDS clásico** sobre la matriz de distancias geodésicas.

El parámetro delicado es k. Si es **demasiado pequeño**, el grafo se desconecta y algunas distancias son infinitas. Si es **demasiado grande**, aparecen **atajos** (*short circuits*): aristas que saltan entre dos partes de la variedad que están cerca en línea recta pero lejos a lo largo de ella, como si el tren pudiera cruzar el hueco del arco. Un solo atajo puede arruinar toda la representación. Isomap también es sensible al ruido, que crea atajos, y su costo es alto: $O(n^2)$ de memoria y hasta $O(n^3)$ de tiempo.

LLE: conservar la geometría local

El **embebido localmente lineal** (LLE) de Roweis y Saul (2000) parte de otra idea: en un vecindario pequeño, la variedad es casi plana, de modo que cada punto se puede **reconstruir** como combinación lineal de sus vecinos. LLE

1. calcula para cada punto los pesos w_{ij} que mejor lo reconstruyen a partir de sus k vecinos, con $\sum_j w_{ij} = 1$, minimizando $\sum_i \|\mathbf{x}_i - \sum_j w_{ij}\mathbf{x}_j\|^2$;
2. busca coordenadas de baja dimensión $\mathbf{y}_i$ que se reconstruyan **con los mismos pesos**, minimizando $\sum_i \|\mathbf{y}_i - \sum_j w_{ij}\mathbf{y}_j\|^2$, lo que se resuelve con los vectores propios de una matriz dispersa.

LLE no calcula distancias entre puntos lejanos, por lo que es más rápido que Isomap, pero tiende a comprimir regiones y es sensible a k y al ruido.

[3] Una **variedad** de dimensión d es un objeto que, visto de cerca, se parece a un espacio plano de dimensión d: una curva ($d = 1$) parece una recta en un tramo corto, y la superficie de la Tierra ($d = 2$) parece un plano a escala de una ciudad, aunque globalmente sea una esfera.

Embebido espectral y clustering espectral

Los **mapas propios del laplaciano** (*Laplacian eigenmaps*) (Belkin y Niyogi 2003) construyen un grafo de similitud con pesos w_{ij} (por ejemplo, el núcleo gaussiano entre vecinos) y buscan coordenadas que mantengan cerca a los puntos con peso alto, minimizando $\sum_{ij} w_{ij}\|\mathbf{y}_i - \mathbf{y}_j\|^2$. La solución son los vectores propios de menor valor propio (sin contar el trivial) del **laplaciano** del grafo, $\mathbf{L} = \mathbf{G} - \mathbf{W}$, con $\mathbf{G}$ la matriz diagonal de grados $g_i = \sum_j w_{ij}$.

El laplaciano tiene una propiedad notable: el número de valores propios iguales a cero es el número de **componentes conexas** del grafo. Si los datos forman dos grupos sin aristas entre ellos, los dos primeros vectores propios son indicadores de cada grupo. El **clustering espectral** (Luxburg 2007) aprovecha esto: calcula el embebido espectral y aplica k-medias sobre él. Como el grafo sigue la conectividad local, separa formas no convexas (anillos, medias lunas) que k-medias no puede separar en el espacio original.

¿Qué tan buena es una representación?

- **Varianza explicada** o proporción de valores propios, en MDS clásico e Isomap.
- **Estrés**, en MDS métrico.
- **Confiabilidad** (*trustworthiness*): mide si los vecinos en el dibujo eran vecinos en los datos originales, penalizando a los «falsos vecinos». Vale 1 si se conservan todos los vecindarios.
- Si existe una variable de referencia (como el orden de las estaciones), la **correlación de rangos** entre ella y la coordenada obtenida.

Laboratorio: desenrollar una espiral

Los puntos siguen una espiral: una variedad de dimensión 1 dibujada en el plano. El color indica la posición real a lo largo de la curva. A la derecha, la coordenada que asigna cada método frente a esa posición real: si la curva se desenrolla bien, los puntos forman una línea monótona. Las aristas rojas del grafo son **atajos** entre vueltas distintas de la espiral. El estado inicial reproduce el arco de siete estaciones.

Laboratorio interactivo disponible en la versión HTML. Con el arco de siete estaciones, MDS sobre distancias euclidianas da $(-1.41, -2, -1.41, 0, 1.41, 2, 1.41)$ salvo el signo, con 58 % de la variación en la primera dimensión; Isomap con $k = 1$ (cada estación unida a su vecina) da coordenadas igualmente espaciadas cada 1.53 con el 100 % de la variación. En la espiral, valores pequeños de k desconectan el grafo y valores grandes crean atajos entre vueltas.

Implementación en Python

MDS clásico desde cero y su relación con PCA

Un mapa a partir de distancias geográficas

Con las distancias de gran círculo entre ocho ciudades mexicanas, calculadas a partir de sus coordenadas aproximadas, el MDS reconstruye el mapa del país sin conocer latitudes ni longitudes.

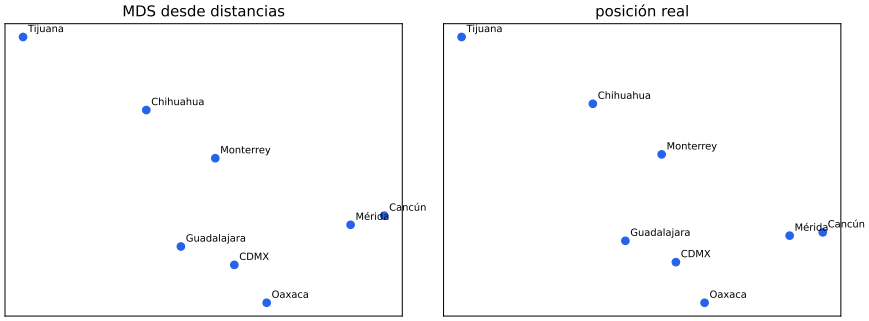


Figura 74.1.: Mapa de ocho ciudades reconstruido por MDS clásico a partir de su matriz de distancias en kilómetros (izquierda), comparado con su posición real en longitud y latitud (derecha). La reconstrucción es correcta salvo una rotación o un reflejo, que aquí se corrigen con un análisis de Procrustes.

MDS métrico (SMACOF) y estrés

La función `estres` calcula la Ecuación 74.1 a partir de las distancias del dibujo (el atributo `stress_` de `scikit-learn` es la suma de cuadrados sin normalizar). En una dimensión, el mapa de México no cabe; en dos, el estrés es casi nulo.

El arco y la espiral con Isomap

Con un grafo de radio 1.6, como en el ejemplo manual, se recuperan las coordenadas equiespaciadas (el signo es arbitrario). Con $k = 2$ vecinos, cada estación también se une con la que está a dos posiciones (a 2.83 km, menos que dos tramos de 1.53 km): son pequeños atajos que acortan las geodésicas, así que las coordenadas ya no quedan equiespaciadas, aunque el orden se conserva.

Comparación de métodos en el rollo suizo

El **rollo suizo** es una hoja rectangular enrollada en tres dimensiones: una variedad de dimensión 2. Un buen método debería desenrollarla, de modo que el color (la posición a lo largo del rollo) varíe de forma suave en una dirección.

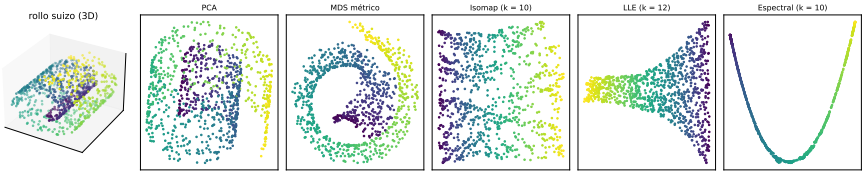


Figura 74.2.: El rollo suizo (1 000 puntos en 3D) representado en 2D por PCA, MDS métrico, Isomap, LLE y embebido espectral. El color es la posición a lo largo del rollo. PCA y MDS lo aplastan; Isomap lo desenrolla; LLE y el embebido espectral conservan el orden con distorsiones.

Sensibilidad de Isomap a k

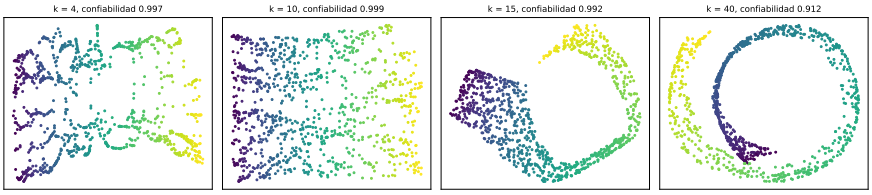


Figura 74.3.: Isomap del rollo suizo con distintos números de vecinos. Con k = 4 el grafo se fragmenta y la representación se distorsiona; con k entre 8 y 15 el rollo se desenrolla; con k = 40 aparecen atajos entre capas y la hoja se pliega.

Clustering espectral

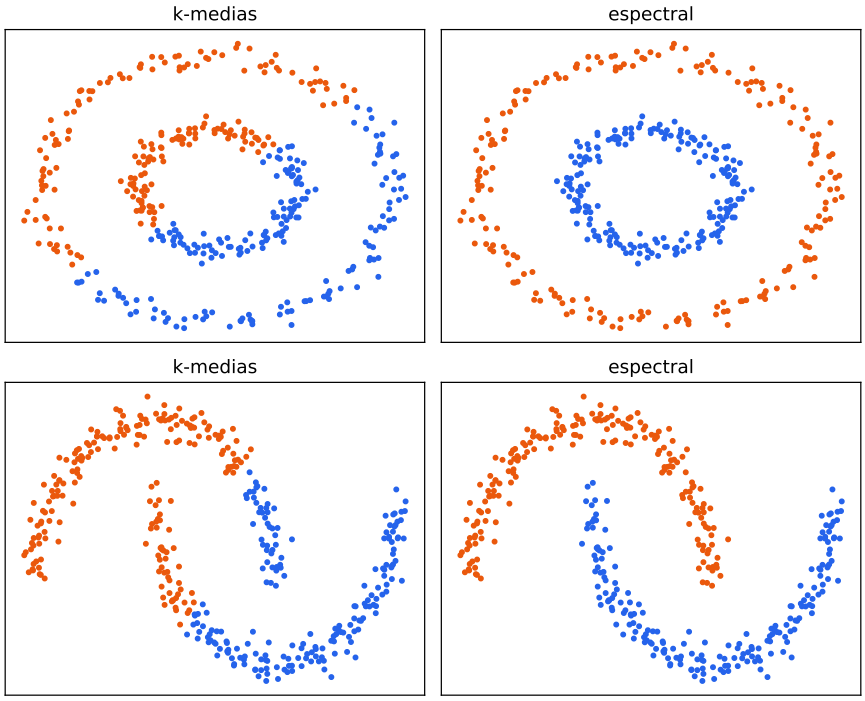


Figura 74.4.: k-medias frente a clustering espectral (grafo de 10 vecinos) en anillos concéntricos y medias lunas. El clustering espectral separa las formas porque agrupa en el espacio de los vectores propios del laplaciano, donde cada componente bien conectada queda compacta.

Resumen

- El **MDS clásico** reconstruye coordenadas a partir de distancias: eleva al cuadrado, hace un doble centrado para obtener productos punto y toma los vectores propios. Con distancias euclidianas coincide con **PCA**; con otras distancias permite «dibujar» cualquier tabla de disimilitudes.
- El **MDS métrico** minimiza el estrés y el **no métrico** solo conserva el orden de las distancias.
- Cuando los datos viven sobre una **variedad** curva, la distancia en línea recta engaña. **Isomap** aproxima la distancia **geodésica** con caminos más cortos en un grafo de vecinos y luego aplica MDS: desenrolló el arco del tren y conservó el 100 % de la variación en una dimensión.
- El número de vecinos es crítico: pocos desconectan el grafo; muchos crean **atajos**.
- **LLE** conserva reconstrucciones locales; el **embebido espectral** usa los vectores propios del laplaciano de un grafo, y el **clustering espectral** aplica k-medias sobre él para separar formas no convexas.

El siguiente capítulo presenta los métodos más usados hoy para visualizar datos de alta dimensión, t-SNE y UMAP, que dejan de lado las distancias globales y se concentran en conservar **quién es vecino de quién**.

75. t-SNE, UMAP y autoencoders

PCA, MDS e Isomap intentan conservar **distancias**: las de todos los pares, incluidos los muy lejanos. En dos dimensiones eso es casi siempre imposible. Si los datos tienen diez grupos en cincuenta dimensiones, no hay forma de dibujarlos en un plano respetando todas sus distancias, y el intento suele producir una mancha en la que los grupos se enciman. Los métodos de este capítulo hacen una renuncia deliberada: **olvidan las distancias grandes y se concentran en conservar quién es vecino de quién**. El resultado son mapas en los que los grupos se ven con una claridad asombrosa. Esa misma claridad es su peligro, porque invita a leer en el mapa cosas que no conserva.

Estudiaremos t-SNE (Maaten y Hinton 2008), el método que popularizó estas visualizaciones; UMAP (McInnes et al. 2018), su sucesor más rápido; y los **autoencoders**, redes neuronales que aprenden una representación comprimida y que, a diferencia de los otros dos, pueden reconstruir los datos.

Intuición: ¿quién es vecino de quién?

Cinco puntos en una recta, en las posiciones

Tabla 75.1.: Cinco puntos en una recta.

Punto	A	B	C	D	E
Posición	0	1	3	5	6

Si le pidiéramos a A que eligiera **un vecino al azar**, favoreciendo a los cercanos, casi siempre elegiría a B, a veces a C y casi nunca a D o E. Esa es la idea de t-SNE: describir los datos con **probabilidades de ser vecinos** y buscar un dibujo en el que esas probabilidades se parezcan.

Construcción paso a paso

Paso 1. Probabilidades de vecindad

t-SNE convierte las distancias desde cada punto i en probabilidades con una campana gaussiana de ancho σ_i:

$$p_{j|i} = \frac{\exp(-d_{ij}^2/2\sigma_i^2)}{\sum_{k\neq i}\exp(-d_{ik}^2/2\sigma_i^2)}.$$

Para A con $\sigma = 1$, los pesos $e^{-d^2/2}$ hacia B, C, D y E son 0.6065, 0.0111, 3.7×10^{-6} y 1.5×10^{-8}. Normalizando, $p_{B|A} = 0.982$ y $p_{C|A} = 0.018$: A «ve» prácticamente un solo vecino. Con $\sigma = 2$, los pesos son 0.883, 0.325, 0.044 y 0.011, y las probabilidades 0.699, 0.257, 0.035 y 0.009: ahora A reparte su atención entre más puntos.

Paso 2. La perplejidad: cuántos vecinos efectivos

¿Qué σ usar? Un σ fijo no sirve: en una zona densa abarcaría cientos de puntos y en una dispersa ninguno. t-SNE elige un σ_i **distinto para cada punto** de modo que todos tengan el mismo **número efectivo de vecinos**, medido con la **perplejidad**[1]:

$$\mathrm{Perp}(P_i) = 2^{H(P_i)}, \qquad H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}.$$

Con $\sigma = 1$, la entropía de A es $H = -(0.982 \log_2 0.982 + 0.018 \log_2 0.018) = 0.13$ bits y la perplejidad $2^{0.13} = 1.09$: un vecino efectivo. Con $\sigma = 2$, la perplejidad es 2.13. El usuario fija la perplejidad (típicamente entre 5 y 50) y t-SNE busca, para cada punto, el σ_i que la produce mediante una **búsqueda binaria**: si la perplejidad es muy alta, reduce σ_i; si es muy baja, lo aumenta. Las probabilidades se simetrizan como $p_{ij} = (p_{j|i} + p_{i|j})/2n$, para que cada par tenga un único peso.

Paso 3. El dibujo usa colas pesadas

En el dibujo de baja dimensión, con posiciones $\mathbf{y}_i$, t-SNE define probabilidades análogas, pero con una distribución t **de Student** con un grado de libertad en lugar de una gaussiana:

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}.$$

¿Por qué? Compara cuánto más «vecino» es un punto a distancia 1 que uno a distancia 5. Con la gaussiana ($\sigma = 1$) el cociente es $e^{-0.5}/e^{-12.5} = e^{12} \approx 162\,755$; con la t es $\frac{1/(1+1)}{1/(1+25)} = 13$. La t tiene **colas pesadas**: decae lentamente, de modo que dos puntos moderadamente lejanos en los datos pueden quedar **muy** lejos en el dibujo sin que su q_{ij} cambie mucho. Esto resuelve el **problema de aglomeración** (*crowding*): en diez dimensiones caben muchos puntos a la misma distancia moderada de un centro, pero en un plano no hay sitio para todos. Las colas pesadas permiten empujarlos hacia afuera y dejar espacio entre los grupos.

[1] La **perplejidad** de una distribución es 2^H, con H su entropía en bits. Una distribución uniforme sobre m resultados tiene entropía $\log_2 m$ y perplejidad exactamente m. Por eso se interpreta como un número efectivo de opciones: una perplejidad de 30 equivale a repartir la probabilidad por igual entre unos 30 vecinos.

Paso 4. Ajustar el dibujo

t-SNE mueve los puntos del dibujo para que las q_{ij} se parezcan a las p_{ij}, minimizando la **divergencia de Kullback-Leibler**[2]

$$C = \mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}, \tag{75.1}$$

con descenso de gradiente. El gradiente tiene una interpretación física:

$$\frac{\partial C}{\partial \mathbf{y}_i} = 4 \sum_j (p_{ij} - q_{ij}) \frac{\mathbf{y}_i - \mathbf{y}_j}{1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2}. \tag{75.2}$$

Cada par actúa como un **resorte**: si $p_{ij} > q_{ij}$ (son vecinos en los datos pero están lejos en el dibujo), se **atraen**; si $p_{ij} < q_{ij}$, se **repelen**. La asimetría de la KL es clave: poner lejos a dos vecinos reales (p grande, q pequeño) cuesta mucho; poner cerca a dos puntos lejanos (p pequeño, q grande) cuesta poco. Por eso t-SNE conserva bien los vecindarios y mal las distancias grandes.

Formalización

t-SNE completo

t-SNE. Entrada: datos $\mathbf{x}_1, \ldots, \mathbf{x}_n$, perplejidad, número de iteraciones.

1. Para cada i, buscar σ_i tal que $\mathrm{Perp}(P_i)$ sea la perplejidad deseada; calcular $p_{j|i}$ y simetrizar p_{ij}.
2. Inicializar $\mathbf{y}_i$ (al azar con varianza pequeña o con las dos primeras componentes de PCA).
3. Durante las primeras iteraciones, multiplicar las p_{ij} por un factor de **exageración temprana** (12 por omisión), que acentúa las atracciones y ayuda a formar los grupos.
4. Iterar el descenso de gradiente con momento, calculando q_{ij} y el gradiente de la Ecuación 75.2.

Costo. La versión exacta calcula todos los pares: $O(n^2)$ por iteración. La aproximación de **Barnes-Hut**, la opción por omisión de `scikit-learn`, agrupa los puntos lejanos con un árbol y reduce el costo a $O(n \log n)$, lo que permite decenas de miles de puntos. Con muchas variables conviene reducir antes a unas 50 con PCA.

Hiperparámetros. La **perplejidad** es el más importante: valores bajos (2–5) muestran estructura muy local y pueden partir los grupos en fragmentos; valores altos (50–100) muestran una estructura más global pero funden grupos pequeños. Debe ser menor que el número de puntos. La tasa de aprendizaje y el número de iteraciones también importan: con pocas iteraciones, el mapa no ha convergido.

[2] La **divergencia de Kullback-Leibler** $\mathrm{KL}(P\|Q) = \sum p \log(p/q)$ mide cuánta información se pierde al aproximar la distribución P con Q. Vale 0 si son iguales y es siempre positiva en otro caso. No es simétrica: $\mathrm{KL}(P\|Q) \neq \mathrm{KL}(Q\|P)$, y esa asimetría es la que hace a t-SNE priorizar los vecindarios.

Cómo leer (y no leer) un mapa de t-SNE

Wattenberg et al. (2016) documentaron con ejemplos las trampas más comunes:

1. **El tamaño de los grupos no significa nada.** t-SNE adapta σ_i a la densidad local: un grupo denso y uno disperso con el mismo número de puntos aparecen del mismo tamaño.
2. **Las distancias entre grupos pueden no significar nada.** Dos grupos lejanos en el mapa no están necesariamente más separados en los datos que dos grupos cercanos.
3. **El ruido puede parecer estructura.** Con perplejidad baja, datos uniformes sin grupos producen manchas aparentes.
4. **Las formas engañan.** Una línea en los datos puede aparecer partida en segmentos o curvada.
5. **Hay que mirar varias perplejidades y varias semillas.** Un patrón que aparece en una sola configuración no es confiable.

UMAP

UMAP (McInnes et al. 2018) comparte la idea de t-SNE, conservar vecindarios, con otra construcción matemática (topología de conjuntos simpliciales difusos) y una implementación mucho más rápida.

El grafo en alta dimensión. Para cada punto i se toman sus k vecinos más cercanos (`n_neighbors`), se calcula ρ_i, la distancia a su vecino **más cercano**, y se asigna a cada arista un peso

$$v_{j|i} = \exp\!\left(-\frac{d_{ij} - \rho_i}{\sigma_i}\right), \tag{75.3}$$

con σ_i elegido para que $\sum_j v_{j|i} = \log_2 k$. Restar ρ_i garantiza que cada punto esté conectado con peso 1 a su vecino más cercano: nadie queda aislado, lo que hace a UMAP robusto en zonas dispersas. Para A en nuestra recta, con $k = 3$, sus vecinos B, C y D están a 1, 3 y 5, de modo que $\rho_A = 1$ y hay que resolver

$$e^{0} + e^{-2/\sigma} + e^{-4/\sigma} = \log_2 3 = 1.585.$$

Con $u = e^{-2/\sigma}$, la ecuación es $u + u^2 = 0.585$, de donde $u = 0.414$ y $\sigma_A = -2/\ln 0.414 = 2.27$. Los pesos de A hacia B, C y D son 1, 0.414 y 0.171. Como $v_{j|i}$ y $v_{i|j}$ pueden diferir, se combinan como una unión difusa, $v_{ij} = v_{j|i} + v_{i|j} - v_{j|i}v_{i|j}$.

El dibujo. En baja dimensión, la similitud es $w_{ij} = (1 + a\|\mathbf{y}_i - \mathbf{y}_j\|^{2b})^{-1}$, una familia de curvas de cola pesada cuyos parámetros a y b se ajustan a partir de `min_dist`, la distancia mínima a la que se permite que queden los puntos. El objetivo es la **entropía cruzada** entre los pesos de ambos grafos,

$$\sum_{i \neq j} \Big[v_{ij} \log \frac{v_{ij}}{w_{ij}} + (1 - v_{ij}) \log \frac{1 - v_{ij}}{1 - w_{ij}} \Big], \tag{75.4}$$

cuyo primer término atrae a los vecinos y el segundo repele a los no vecinos. Para no calcular todas las repulsiones, UMAP usa **muestreo negativo**: en cada paso repele cada punto de unos cuantos

puntos elegidos al azar. Además se inicializa con un embebido espectral (capítulo anterior), lo que mejora la estructura global.

Hiperparámetros. `n_neighbors` (5–50) cumple el papel de la perplejidad: pocos vecinos, estructura local; muchos, global. `min_dist` (0–0.99) controla qué tan apretados quedan los grupos en el dibujo, sin cambiar qué es vecino de qué.

t-SNE y UMAP frente a frente

Tabla 75.2.: Comparación de t-SNE y UMAP.

	t-SNE	UMAP
Objetivo	KL entre probabilidades de vecindad	entropía cruzada entre grafos difusos
Hiperparámetro principal	perplejidad	`n_neighbors`, `min_dist`
Costo	$O(n \log n)$ (Barnes-Hut)	aproximadamente lineal (vecinos aproximados y muestreo negativo)
Estructura global	débil	algo mejor (inicialización espectral)
Casos nuevos	no (hay que recalcular)	sí, con `transform`
Dimensiones de salida	2 o 3 en la práctica	cualquiera; útil como paso previo a clustering

Kobak y Berens (2019) mostraron que buena parte de la diferencia en estructura global se debe a la **inicialización**: t-SNE inicializado con PCA conserva la estructura global casi tan bien como UMAP. En ambos casos, las advertencias sobre tamaños y distancias siguen aplicando.

Autoencoders

Un **autoencoder** (Hinton y Salakhutdinov 2006) es una red neuronal entrenada para **copiar su entrada en su salida** a través de un cuello de botella:

$$\mathbf{z} = f_{\text{cod}}(\mathbf{x}) \in \mathbb{R}^q, \qquad \hat{\mathbf{x}} = f_{\text{dec}}(\mathbf{z}), \qquad \min \sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2, \tag{75.5}$$

con q mucho menor que p. Para reconstruir bien a través de solo q números, el codificador debe aprender una representación compacta de lo esencial. Si el codificador y el decodificador son lineales, el mejor autoencoder abarca el mismo subespacio que las primeras q componentes de PCA; con capas no lineales, puede aprender variedades curvas. A diferencia de t-SNE y UMAP:

- tiene un **decodificador**, que reconstruye datos a partir de la representación;
- asigna **casos nuevos** con solo pasarlos por el codificador;
- su **error de reconstrucción** sirve para detectar anomalías: los casos parecidos a los del entrenamiento se reconstruyen bien y los raros, no.

Sus variantes (autoencoders variacionales, de eliminación de ruido) son la base de muchos modelos generativos. El capítulo de PyTorch y JAX muestra cómo entrenar redes de este tipo.

Laboratorio: t-SNE en el navegador

El laboratorio ejecuta t-SNE exacto sobre 90 puntos de **diez dimensiones**. En «Tres grupos», los grupos A y B están cerca entre sí y C está casi tres veces más lejos; además, C es tres veces más disperso. La tabla compara esas proporciones con las del mapa. Cambia la perplejidad y reinicia para ver cómo cambia el dibujo.

Laboratorio interactivo disponible en la versión HTML. El laboratorio ejecuta t-SNE exacto sobre 90 puntos en diez dimensiones. Con tres grupos en los que C está 2.9 veces más lejos de A que B y es 3 veces más disperso, el mapa separa los tres grupos, pero la razón de dispersiones queda cerca de 1 y la de distancias se comprime a alrededor de 2: t-SNE conserva los vecindarios, no los tamaños ni las distancias entre grupos. Con ruido uniforme y perplejidad baja aparecen manchas que no son grupos.

Implementación en Python

Perplejidad y σ_i a mano

t-SNE en los dígitos escritos a mano

El conjunto `digits` de `scikit-learn` tiene 1 797 imágenes de 8×8 píxeles: 64 variables. Ningún método usa las etiquetas; el color solo sirve para juzgar el resultado.

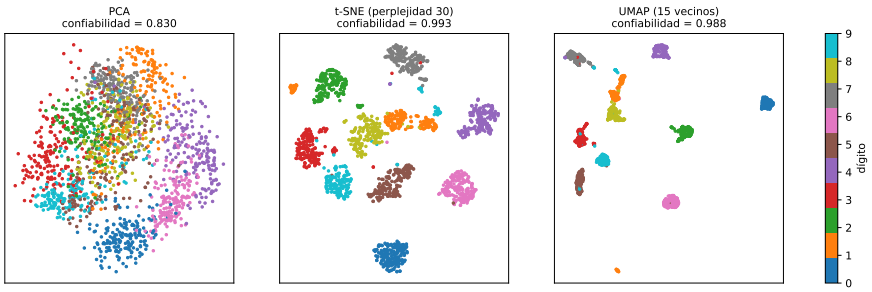


Figura 75.1.: Los dígitos escritos a mano (64 dimensiones) proyectados con PCA, t-SNE (perplejidad 30) y UMAP (15 vecinos). PCA conserva la varianza global pero encima los dígitos; t-SNE y UMAP separan casi todos los dígitos sin haber visto las etiquetas.

El efecto de la perplejidad y de `n_neighbors`

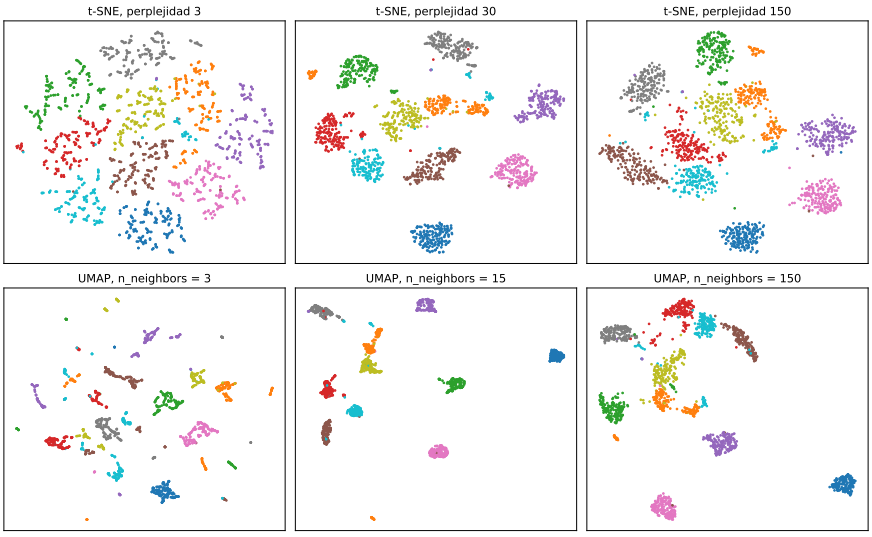


Figura 75.2.: t-SNE de los dígitos con perplejidad 3, 30 y 150 (arriba) y UMAP con 3, 15 y 150 vecinos (abajo). Valores pequeños fragmentan los grupos; valores grandes conservan más estructura global y funden los detalles.

Tamaños y distancias que no significan nada

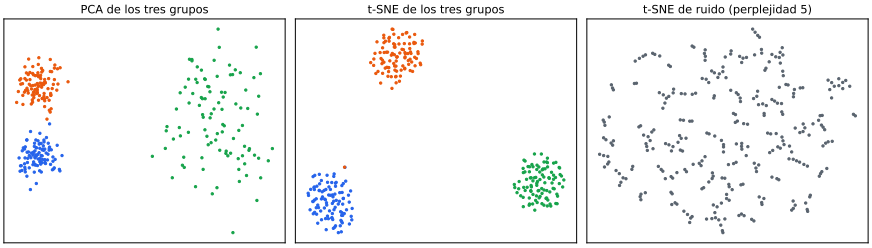


Figura 75.3.: Izquierda: tres grupos en 10 dimensiones; C está tres veces más lejos de A que B y es tres veces más disperso (proyección PCA). Centro: t-SNE iguala tamaños y distancias. Derecha: ruido uniforme en 10 dimensiones con perplejidad 5, en el que t-SNE dibuja manchas que no son grupos.

UMAP para casos nuevos y como paso previo al clustering

UMAP aprende una transformación que se puede aplicar a datos nuevos, y con más de dos dimensiones de salida es un buen preprocesamiento para algoritmos basados en densidad como HDBSCAN.

Agrupar sobre una representación de UMAP puede funcionar muy bien, pero hereda sus distorsiones: los tamaños y las densidades del espacio reducido no son los originales. Conviene validar los grupos en el espacio original.

Un autoencoder con `scikit-learn`

Un perceptrón multicapa entrenado para predecir su propia entrada es un autoencoder. Con la arquitectura $64 \to 256 \to 64 \to q \to 64 \to 256 \to 64$, la capa central es la representación de q dimensiones; la extraemos aplicando a mano las tres primeras capas.

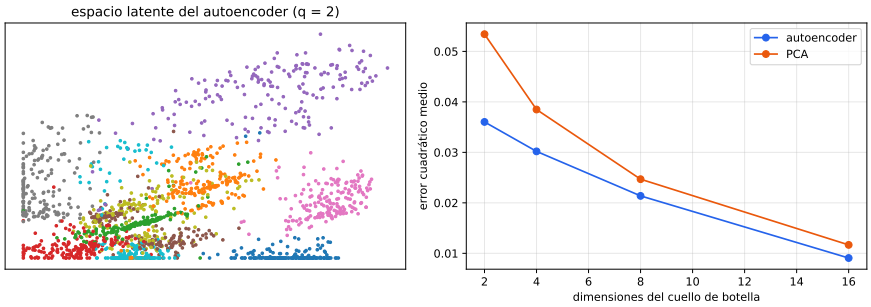


Figura 75.4.: Representación de dos dimensiones de los dígitos aprendida por un autoencoder (izquierda) y error de reconstrucción según el número de dimensiones del cuello de botella, comparado con PCA (derecha). Con un cuello de botella estrecho, el autoencoder profundo reconstruye mejor que PCA porque puede seguir la curvatura de los datos.

Cuándo usar cada método

Tabla 75.3.: Elección de un método de reducción de dimensionalidad.

Objetivo	Recomendación
Ejes interpretables, reducción reversible antes de un modelo	PCA
Dibujar una tabla de distancias o disimilitudes	MDS
Desenrollar una variedad suave y con poco ruido	Isomap
Visualizar grupos en datos de alta dimensión	t-SNE (perplejidad 30, inicialización PCA) o UMAP
Muchos casos, o transformar casos nuevos	UMAP
Reducción previa a clustering por densidad	UMAP con 5–10 dimensiones y `min_dist = 0`
Reconstruir datos, detectar anomalías por reconstrucción	autoencoder

Y tres reglas para cualquier mapa de t-SNE o UMAP: probar **varias** perplejidades o números de vecinos, probar **varias semillas**, y **confirmar** en los datos originales cualquier patrón que se quiera reportar.

Resumen

- t-SNE convierte distancias en **probabilidades de vecindad** con un ancho σ_i por punto, elegido para que todos tengan la misma **perplejidad** (número efectivo de vecinos).

- En el dibujo usa una distribución t **de Student**, de colas pesadas, que resuelve el problema de aglomeración; minimiza la divergencia KL, que castiga separar vecinos y tolera acercar puntos lejanos.
- Por eso t-SNE conserva vecindarios, pero **no** tamaños de grupo ni distancias entre grupos, y con perplejidad baja puede inventar grupos en el ruido.
- **UMAP** construye un grafo difuso de k vecinos (con ρ_i y σ_i por punto) y optimiza una entropía cruzada con muestreo negativo; es más rápido, transforma casos nuevos y sirve como paso previo al clustering.
- Los **autoencoders** aprenden una representación comprimida con una red neuronal; tienen decodificador, asignan casos nuevos y su error de reconstrucción detecta anomalías.

El último capítulo de esta parte reúne todas las herramientas en un caso completo: descubrir grupos en los pingüinos del archipiélago Palmer sin mirar su especie.

Parte XL.

Práctica Integradora

76. Práctica integradora: segmentar pingüinos

Los capítulos anteriores estudiaron cada herramienta por separado y con ejemplos pequeños. Esta práctica las reúne en un análisis completo, siguiendo el flujo de trabajo del capítulo introductorio: propósito, preparación, representación, algoritmos, validación e interpretación. Usaremos los datos de los **pingüinos del archipiélago Palmer**, en la Antártida, recolectados por la Dra. Kristen Gorman entre 2007 y 2009 (Gorman et al. 2014; Horst et al. 2020), que ya aparecieron en el capítulo de clasificación asociativa.

Los datos traen la **especie** de cada pingüino. La guardaremos bajo llave: el análisis no la usará en ningún momento y solo al final la sacaremos para comparar. Así reproducimos la situación de un biólogo que mide animales sin saber cuántas especies hay, y podemos comprobar, por una vez, qué tan bien funcionan los métodos.

1. Propósito y datos

Pregunta. A partir de cuatro medidas corporales, ¿existen grupos naturales de pingüinos? ¿Cuántos? ¿Cómo se describen?

Hay 344 pingüinos. Dos no tienen ninguna medida y nueve más no tienen el sexo registrado. Como el sexo servirá para una validación posterior, conservamos los 333 registros completos y separamos las variables que **no** se usarán para agrupar.

2. Preparación: la escala

Las cuatro medidas tienen unidades muy distintas: la masa ronda los 4 000 g con una desviación de 800 g, mientras que el alto del pico tiene una desviación de 2 mm. Como vimos en el capítulo introductorio, sin escalar, la masa dominaría cualquier distancia. Estandarizamos.

Sin escalar, la masa aporta el 99.96 % de la varianza: un análisis en unidades originales sería un análisis de la masa.

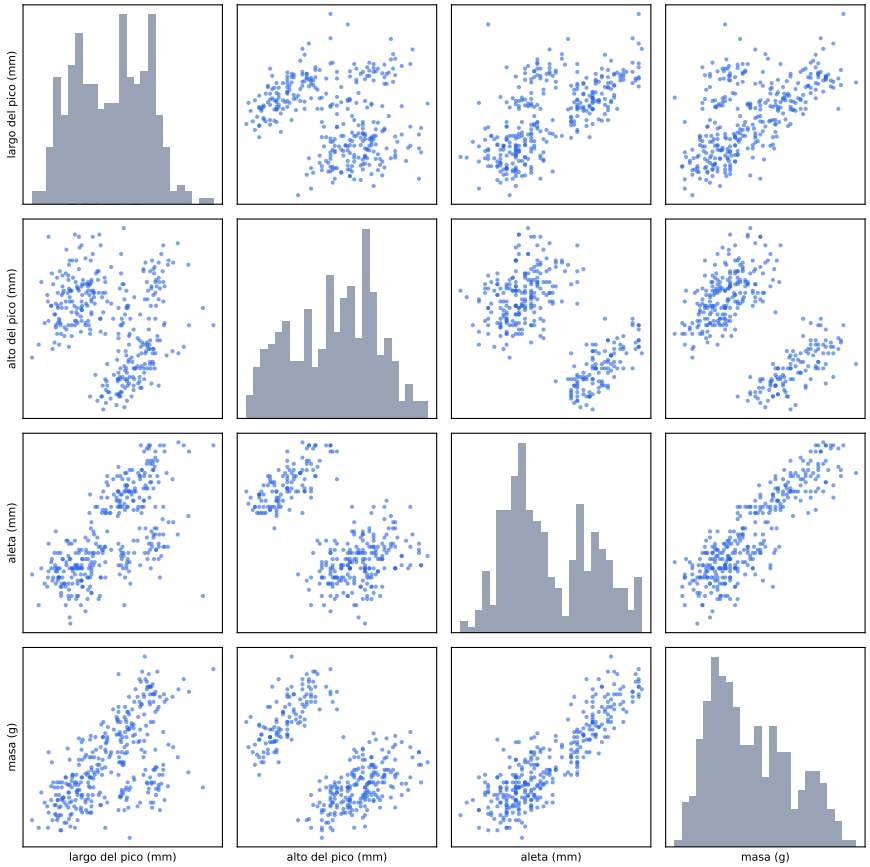


Figura 76.1.: Diagramas de dispersión de las cuatro medidas estandarizadas, sin colores: así es como ve los datos el análisis no supervisado. Ya se adivinan al menos dos grupos, muy claros en las combinaciones con la aleta o el alto del pico.

3. ¿Hay grupos? Tendencia de agrupamiento

Un valor de Hopkins muy por encima de 0.5 confirma que vale la pena buscar grupos.

4. Representación: PCA y UMAP

Con solo cuatro variables podríamos agrupar directamente, pero una representación de dos dimensiones ayuda a ver la estructura y a revisar los resultados.

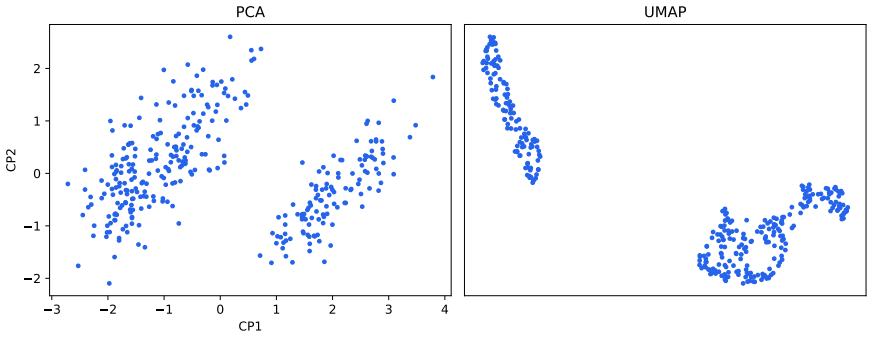


Figura 76.2.: Los pingüinos en dos dimensiones: PCA (izquierda; las dos primeras componentes explican la mayor parte de la varianza) y UMAP (derecha). Ambos muestran un grupo muy separado y una masa mayor que parece contener dos subgrupos.

5. ¿Cuántos grupos?

Calculamos los criterios del capítulo de validación para k de 2 a 7 con k-medias, y el BIC con mezclas gaussianas.

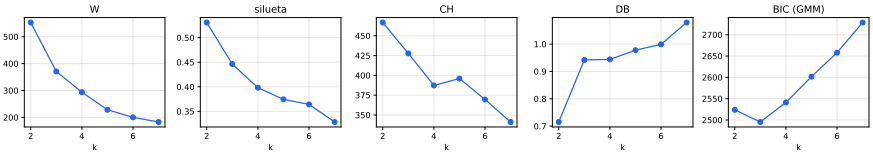


Figura 76.3.: Criterios para elegir el número de grupos. La silueta, Calinski-Harabasz y Davies-Bouldin favorecen k = 2 (un grupo muy separado frente al resto) y el codo de W es suave; el BIC de la mezcla gaussiana tiene su mínimo en k = 3.

Los criterios no coinciden, y eso es información: hay un nivel de estructura muy claro con **dos** grupos y otro, más sutil, con **tres**. Antes de decidir, revisemos la estabilidad.

Con $k = 2$ la partición es idéntica en todas las submuestras y con $k = 3$ es muy estable. Con $k = 4$ la estabilidad baja y varía mucho: el cuarto grupo se forma de manera distinta cada vez. Con $k = 5$ vuelve a subir, una pista de que existe otro nivel de estructura (la sección 9 mostrará cuál) y un recordatorio de que la estabilidad es una condición necesaria, no suficiente. Tomamos $k = 3$ como hipótesis principal, sin olvidar que el nivel de dos grupos es el más evidente.

6. Comparar algoritmos

Con los criterios internos, todos los métodos parecen razonables, y HDBSCAN, que decide solo el número de grupos, encuentra **dos**. Sin la especie, aquí terminaría la evidencia numérica y empezaría la interpretación.

7. Abrir la caja: validación externa

Ahora sí sacamos la especie y el sexo. Recordemos que ninguno se usó para agrupar.

Varios hallazgos:

- La **mezcla gaussiana** con covarianza completa recupera las especies casi a la perfección. Las especies son nubes **elípticas** inclinadas (las medidas están correlacionadas dentro de cada especie), justo lo que supone ese modelo.
- ***k*-medias** separa perfectamente a los Gentoo, pero confunde a una parte de los Adelia con los Barbijo (*Chinstrap*): su supuesto de grupos esféricos y de tamaño parecido no encaja, porque los Barbijo son pocos (68) y se traslapan con los Adelia en tres de las cuatro medidas.
- **HDBSCAN** ve dos grupos: Gentoo contra el resto. Desde la densidad, Adelia y Barbijo forman una sola región continua.
- Ningún método se acerca al **sexo**: la estructura dominante en estas medidas es la especie. La asociación moderada con la **isla** (ARI cercano a 0.35) es indirecta: los Gentoo solo viven en una isla (ejercicio 4).

8. Perfilar los grupos

Para comunicar los grupos se describen en unidades originales. Usamos la mezcla gaussiana.

Un biólogo describiría los grupos así: uno de pingüinos **grandes** con aletas largas y pico **bajo** (los Gentoo); otro de pico **corto y alto** (Adelia); y otro de pico **largo y alto**, de tamaño similar al anterior (Barbijo). La variable que distingue a los dos últimos es el largo del pico, que en el diagrama de pares se veía como una separación sutil.

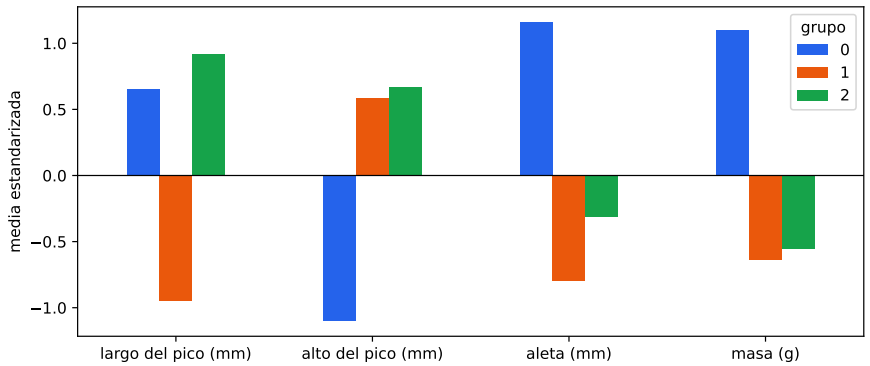


Figura 76.4.: Perfil de los tres grupos de la mezcla gaussiana: media estandarizada de cada medida. Las barras resumen en una sola figura lo que distingue a cada grupo.

9. Estructura dentro de los grupos

El clustering jerárquico sugería que la estructura es anidada. ¿Hay subgrupos dentro de cada especie? Agrupamos con $k = 2$ dentro de cada grupo, estandarizando de nuevo con los datos del propio grupo.

Dentro de cada especie, los dos subgrupos corresponden aproximadamente a **machos y hembras**: los machos son más grandes en todas las medidas. El sexo no aparecía en el análisis global porque la diferencia entre especies es mucho mayor que la diferencia entre sexos, pero emerge al mirar dentro de cada grupo. Es un ejemplo de por qué conviene explorar varios niveles de una jerarquía.

10. Casos raros

Los pingüinos con mayor LOF tienen combinaciones inusuales para su grupo, por ejemplo un pico excepcionalmente largo para su tamaño. Antes de descartarlos habría que verificar con el equipo de campo si son errores de medición o individuos genuinamente atípicos: una anomalía es una hipótesis, no un veredicto.

11. ¿Y si agregamos variables categóricas?

La isla y el sexo son categóricos. Con la distancia de Gower del capítulo de similitud se pueden combinar con las medidas. Al agregar el sexo con el mismo peso que cada medida, el resultado cambia por completo:

Con peso 1, el sexo, que es binario y aporta un desacuerdo completo de 1 a la mitad de los pares, se vuelve la variable dominante. Es el mismo fenómeno que vimos con Ana y sus vecinos: **la elección de variables y pesos decide qué estructura se encuentra**.

Laboratorio: las variables deciden los grupos

Elige qué medidas entran al análisis (siempre estandarizadas), el algoritmo y el número de grupos. Las tablas cruzan los grupos con la especie y con el sexo, que el algoritmo nunca ve. El estado inicial usa las cuatro medidas con k-medias y $k = 3$, como en la sección 7.

Laboratorio interactivo disponible en la versión HTML. Con las cuatro medidas estandarizadas, k-medias con $k = 3$ obtiene un ARI de 0.80 con la especie y prácticamente 0 con el sexo; usando solo largo del pico, alto del pico y aleta, el ARI con la especie sube a 0.92. La combinación de alto del pico y masa con $k = 3$ es la que más se asocia con el sexo (ARI de 0.23).

12. Lo que aprendimos

Tabla 76.1.: Decisiones del análisis y su efecto.

Etapa	Decisión	Por qué importó
Preparación	estandarizar	sin escalar, la masa explicaba el 99.96 % de la varianza
Tendencia	Hopkins	confirmó que había estructura antes de agrupar

Etapa	Decisión	Por qué importó
Número de grupos	combinar criterios y estabilidad	la silueta veía 2 grupos; el BIC, 3; ambos niveles eran reales
Algoritmo	comparar supuestos	la mezcla gaussiana (elipses) superó a k-medias (esferas)
Validación	variables «bajo llave»	permitió medir qué estructura se había encontrado: especie, no sexo ni isla
Interpretación	perfiles en unidades originales	convirtió etiquetas numéricas en descripciones biológicas
Jerarquía	agrupar dentro de los grupos	reveló el sexo como segundo nivel de estructura
Datos mixtos	pesos explícitos en Gower	el peso de una variable binaria cambió por completo el resultado

En un problema real no tendríamos la especie para comprobar. Lo que sí tendríamos son las herramientas que nos llevaron a la respuesta correcta antes de verla: escalar con cuidado, verificar la tendencia, mirar varios criterios y la estabilidad, comparar algoritmos con supuestos distintos, explorar varios niveles de la jerarquía, perfilar los grupos y **contrastarlos con el conocimiento del dominio**.

Ejercicios

1. Repite la sección 6 sin estandarizar. ¿Qué algoritmo se ve más afectado? Explica con la tabla de varianzas de la sección 2.
2. Ajusta mezclas gaussianas con los cuatro tipos de covarianza (`full`, `tied`, `diag`, `spherical`) y $k = 3$. Compara su BIC y su ARI con la especie. ¿Qué tipo de covarianza se parece a k-medias?
3. Aplica DBSCAN a las medidas estandarizadas eligiendo ε con el gráfico de k-distancias. ¿Cuántos grupos encuentra? ¿Por qué coincide con HDBSCAN?
4. Usa la isla como variable de validación externa en lugar de la especie. ¿Qué relación hay entre isla y especie en estos datos? (Pista: haz una tabla cruzada.)
5. Repite la sección 9 con clustering jerárquico de Ward y dibuja el dendrograma de cada especie. ¿El corte en dos grupos es claro?

Parte XLI.

Minería de Textos

77. Minería de Textos: de las palabras a los vectores

Todas las técnicas del libro trabajan sobre una tabla: filas que son observaciones y columnas que son atributos numéricos o categóricos. Pero una parte enorme de lo que una organización registra es **texto libre**: reseñas de clientes, tickets de soporte, correos, notas de un expediente, descripciones de productos, transcripciones de llamadas. Ese texto no cabe en ninguna columna de un modelo hasta que alguien decide cómo convertirlo en números.

La **minería de textos** es el proceso de extraer patrones y conocimiento de colecciones de documentos (Feldman y Dagan 1995; Hearst 1999). En el marco de CRISP-DM no es una fase nueva, sino una variante de la **preparación de datos**: el paso crítico es la **representación**, que transforma cada documento en un vector. Una vez hecho eso, todo lo que ya conocemos se aplica sin cambios: clasificación para detectar quejas, clustering y reducción de dimensionalidad para descubrir temas, reglas de asociación para encontrar palabras que aparecen juntas (Aggarwal y Zhai 2012).

En el almacén de datos, el texto suele vivir como un atributo de baja calidad analítica: un campo `comentario` en la tabla de hechos de pedidos o una tabla de reseñas con llave de cliente, sucursal y fecha. La minería de textos cierra el ciclo en sentido inverso: sus resultados (una probabilidad de queja, un tema) **regresan al almacén** como nuevas medidas y dimensiones que sí se pueden agregar con un cubo OLAP.

Intuición: cuatro reseñas de Café Norte

La aplicación de Café Norte, la cadena del caso integrador de la Parte I, pide una reseña breve después de cada pedido. Estas son cuatro:

Tabla 77.1.: Cuatro reseñas de la aplicación de Café Norte.

Reseña	Texto	Estrellas
d_1	¡Qué rico café! Y el pan, rico también.	
d_2	Café frío y servicio lento.	
d_3	Servicio rápido y café rico.	
d_4	Café frío, pan duro.	

Una persona las lee y, sin pensarlo, hace varias cosas:

- Distingue **de qué hablan**: d_1 y d_4 hablan de la comida; d_2 y d_3, del servicio (y del café).
- Distingue **cómo se sienten**: d_1 y d_3 son positivas; d_2 y d_4 son quejas.
- Ignora palabras que no aportan, como «el», «y» o «qué».

- Nota que **«café» aparece en las cuatro**: es lo esperable en una cafetería y, por eso mismo, no sirve para distinguir una reseña de otra.

Un algoritmo no ve nada de esto; ve cadenas de caracteres. El objetivo de este capítulo es construir, paso a paso, una representación numérica que capture al menos las tres últimas observaciones.

Construcción numérica paso a paso

Del texto a los tokens

El primer paso es partir cada reseña en unidades llamadas **tokens**, normalmente palabras. Seguimos tres reglas:

1. Pasar todo a **minúsculas**, para que «Café» y «café» sean la misma palabra.
2. Quedarse solo con **secuencias de letras** (incluidas las acentuadas y la ñ); la puntuación y los signos «¡!» desaparecen.
3. Eliminar las **palabras vacías** (*stopwords*): artículos, preposiciones y conjunciones muy frecuentes que casi no tienen contenido propio[1].

Con d_1:

Tabla 77.2.: Preprocesamiento de la reseña d_1.

Paso	Resultado	Tokens
Texto original	¡Qué rico café! Y el pan, rico también.	—
Minúsculas y solo letras	qué · rico · café · y · el · pan · rico · también	8
Sin palabras vacías	rico · café · pan · rico	4

Las otras tres quedan como d_2 = (café, frío, servicio, lento), d_3 = (servicio, rápido, café, rico) y d_4 = (café, frío, pan, duro). De 22 tokens originales quedan 16, y el **vocabulario** (las palabras distintas) tiene 8 términos.

Hay pasos adicionales que se usan con frecuencia: quitar los acentos («cafe» y «café» como la misma palabra) y reducir cada palabra a una forma base con **lematización** o **truncamiento** (*stemming*)[2]. Aquí no los necesitamos, pero el laboratorio permite ver qué pasa sin ellos.

[1] No existe una lista universal de palabras vacías. Las listas genéricas (como las de NLTK o spaCy para español) suelen incluir «no», «nada» y «nunca», lo cual es un error grave para el análisis de sentimiento. Además, cada dominio tiene sus propias palabras vacías: en las reseñas de una cafetería, «café» casi no informa; en las de una librería, sí.

[2] El **truncamiento** (*stemming*) corta sufijos con reglas, sin diccionario: «rico», «rica» y «ricos» quedan como «ric». Es rápido, pero puede unir palabras distintas o producir raíces que no son palabras. La **lematización** usa un diccionario y el análisis gramatical para llevar cada palabra a su forma de diccionario: «estaba» → «estar», «ricas» → «rico». En Python, el truncador Snowball para español está en NLTK y la lematización, en spaCy.

La bolsa de palabras

Ahora contamos cuántas veces aparece cada término en cada reseña. El resultado es la **matriz documento-término**: una fila por documento, una columna por término.

Tabla 77.3.: Matriz documento-término con conteos (columnas en orden alfabético, como las ordena Python).

	café	duro	frío	lento	pan	rico	rápido	servicio
d_1	1	0	0	0	1	**2**	0	0
d_2	1	0	1	1	0	0	0	1
d_3	1	0	0	0	0	1	1	1
d_4	1	1	1	0	1	0	0	0

Este modelo se llama **bolsa de palabras** (*bag of words*): cada documento se reduce a cuántas veces contiene cada palabra, **sin importar el orden**. «Café frío y servicio lento» y «servicio frío y café lento» tienen exactamente la misma fila. Es una simplificación brutal y, aun así, sorprendentemente útil.

Observa dos rasgos que se vuelven extremos en corpus reales. Primero, de las 32 celdas solo 15 son distintas de cero (47 %); con miles de documentos y decenas de miles de términos, más del 99 % de las celdas son cero y la matriz se guarda en formato **disperso**. Segundo, cada término nuevo es una dimensión nueva: el espacio tiene tantas dimensiones como palabras distintas.

Pesar los términos: TF-IDF

Los conteos tratan igual a todas las palabras, pero no todas informan lo mismo. «Café» está en las cuatro reseñas: saber que una reseña dice «café» no ayuda a distinguirla. «Lento» solo está en una: es muy distintiva.

La **frecuencia de documento** df cuenta en cuántas reseñas aparece cada término, y la **frecuencia inversa de documento** la convierte en un peso (Spärck Jones 1972):

$$\mathrm{idf}(t) = \ln \frac{N}{\mathrm{df}(t)},$$

donde $N = 4$ es el número de documentos.

Tabla 77.4.: Frecuencia de documento e idf de cada término.

Término	df	idf
café	4	$\ln(4/4) = 0$
frío, pan, rico, servicio	2	$\ln(4/2) = 0.693$
duro, lento, rápido	1	$\ln(4/1) = 1.386$

El peso **TF-IDF** multiplica el conteo dentro del documento (*term frequency*, tf) por el idf. Por ejemplo, en d_1 el término «rico» aparece dos veces: $2 \times 0.693 = 1.386$.

Tabla 77.5.: Matriz TF-IDF. La columna de «café» quedó en cero.

	café	duro	frío	lento	pan	rico	rápido	servicio
d_1	0	0	0	0	0.693	1.386	0	0
d_2	0	0	0.693	1.386	0	0	0	0.693
d_3	0	0	0	0	0	0.693	1.386	0.693
d_4	0	1.386	0.693	0	0.693	0	0	0

La columna de «café» desapareció: el idf aprendió, solo contando, la observación de la sección anterior.

Similitud coseno

¿Qué tan parecidas son dos reseñas? Con vectores, la medida estándar es el **coseno del ángulo** entre ellos: el producto punto dividido entre el producto de las normas. Para d_1 y d_3, el único término que comparten con peso distinto de cero es «rico»:

$$\cos(d_1, d_3) = \frac{1.386 \times 0.693}{\|d_1\|\,\|d_3\|} = \frac{0.961}{1.550 \times 1.698} = 0.365,$$

con $\|d_1\| = \sqrt{0.693^2 + 1.386^2} = 1.550$ y $\|d_3\| = \sqrt{1.386^2 + 0.693^2 + 0.693^2} = 1.698$. Haciendo lo mismo con todos los pares, y comparando con lo que se obtendría usando los conteos crudos:

Tabla 77.6.: Similitud coseno entre las reseñas.

Par	Comparten	Coseno con conteos	Coseno con TF-IDF
d_1–d_3	café, rico	0.612	**0.365**
d_1–d_4	café, pan	0.408	0.183
d_2–d_3	café, servicio	0.500	0.167
d_2–d_4	café, frío	0.500	0.167
d_3–d_4	café	0.250	0
d_1–d_2	café	0.204	0

Con conteos, todos los pares se parecen algo porque todos comparten «café». Con TF-IDF, los pares que **solo** comparten «café» quedan en cero, y el par más parecido es d_1–d_3: las dos dicen que algo estaba «rico», y d_1 lo dice dos veces.

Una consulta: buscar «café frío»

La misma maquinaria sirve para **buscar**. La consulta «café frío» se trata como un documento más: su vector TF-IDF tiene $0 \times 1 = 0$ en «café» y 0.693 en «frío». Solo d_2 y d_4 contienen «frío», y las dos empatan:

$$\cos(q, d_2) = \frac{0.693 \times 0.693}{0.693 \times 1.698} = \frac{1}{\sqrt{6}} = 0.408 = \cos(q, d_4).$$

Ordenar los documentos por su coseno con la consulta es el **modelo de espacio vectorial** de recuperación de información (Salton et al. 1975), la idea detrás de los primeros buscadores y de muchos sistemas de búsqueda internos.

Formalización

Sea un **corpus** de N documentos $d_1, \dots, d_N$ y un **vocabulario** $V = \{t_1, \dots, t_m\}$ con los términos que aparecen en ellos después del preprocesamiento.

Definición (matriz documento-término). Es la matriz $X \in \mathbb{N}^{N \times m}$ con $x_{ij} = \mathrm{tf}(t_j, d_i)$, el número de veces que el término t_j aparece en el documento d_i. Cada fila es la **bolsa de palabras** de un documento.

Definición (TF-IDF). Con $\mathrm{df}(t) = |\{d : \mathrm{tf}(t, d) > 0\}|$, el peso de t en d es

$$w(t, d) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t), \qquad \mathrm{idf}(t) = \ln \frac{N}{\mathrm{df}(t)}. \tag{77.1}$$

El idf tiene una lectura probabilística: $\mathrm{df}(t)/N$ estima la probabilidad de que un documento al azar contenga t, y $\mathrm{idf}(t) = -\ln P(t \in d)$ es la **información** (sorpresa) de encontrarlo[3]. Un término que está en todas partes no sorprende y pesa cero.

Hay muchas variantes de la fórmula (Salton y Buckley 1988; Manning et al. 2008). Las dos más comunes son el **tf sublineal**, $1 + \ln \mathrm{tf}$ si $\mathrm{tf} > 0$, que evita que una palabra repetida diez veces pese diez veces más, y el **idf suavizado** que usa `scikit-learn` por omisión[4]:

$$\mathrm{idf}_{\mathrm{sk}}(t) = \ln \frac{1 + N}{1 + \mathrm{df}(t)} + 1.$$

Definición (similitud coseno). Para dos vectores $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ distintos de cero,

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} = \sum_{j=1}^{m} \hat{u}_j \hat{v}_j, \qquad \hat{\mathbf{u}} = \frac{\mathbf{u}}{\|\mathbf{u}\|}. \tag{77.2}$$

Si los vectores se normalizan a norma 1 de antemano (como hace `TfidfVectorizer`), el coseno es simplemente el producto punto. Con pesos no negativos, el coseno está entre 0 (sin términos en común) y 1 (misma proporción de términos).

[3] En teoría de la información, la **información** (o sorpresa) de un evento con probabilidad p es $-\log p$: un evento seguro ($p = 1$) no aporta información, y uno raro aporta mucha. El idf es la sorpresa de que un documento contenga el término. La justificación probabilística formal del idf fue desarrollada después de su propuesta empírica.

[4] El suavizado equivale a agregar un documento imaginario que contiene todos los términos una vez, lo que evita divisiones entre cero cuando un término de una consulta no aparece en el corpus. El $+1$ final hace que un término presente en todos los documentos conserve un peso pequeño en lugar de anularse. Con `smooth_idf=False`, `scikit-learn` usa $\ln(N/\mathrm{df}) + 1$, que tampoco coincide con la fórmula clásica.

¿Por qué coseno y no distancia euclidiana? Porque el coseno ignora la **longitud** del documento. Si una reseña es la otra copiada dos veces, sus vectores apuntan en la misma dirección (coseno 1), aunque la distancia euclidiana entre ellos sea grande. En texto, la dirección (qué palabras y en qué proporción) importa más que la magnitud (cuántas palabras)[5].

Propiedades del texto como dato

- **Alta dimensión y dispersión.** m crece con el corpus y casi todas las entradas de X son cero. Los algoritmos de texto aprovechan la dispersión: guardan solo los valores no nulos.
- **Ley de Zipf.** En un corpus natural, la frecuencia de la palabra que ocupa el lugar r es aproximadamente proporcional a $1/r$ (Zipf 1949): unas pocas palabras (las vacías) concentran gran parte de los tokens y la mayoría aparece una o dos veces. Por eso se filtran términos muy frecuentes (`max_df`) y muy raros (`min_df`).
- **Orden y negación.** La bolsa de palabras no distingue «no estaba rico» de «estaba rico, no lo dudo». Una solución parcial es incluir **n-gramas**: secuencias de n tokens consecutivos. Con bigramas, «no estaba» y «estaba rico» son nuevos términos. El precio es un vocabulario mucho más grande.

El preprocesamiento depende de la tarea

No hay un preprocesamiento correcto en abstracto. La misma decisión ayuda en una tarea y estorba en otra:

Tabla 77.7.: Decisiones de preprocesamiento según la tarea.

Decisión	Clasificar quejas	Descubrir temas	Buscar documentos
Quitar «no», «nada», «nunca»	**No**: invierten el sentido	Sí: no indican tema	Depende
Bigramas	Sí: capturan negaciones	Opcional	Útiles para frases
Quitar acentos y truncar	Opcional	Sí: agrupa variantes	Sí: tolera errores de escritura
Filtrar términos muy frecuentes	Con cuidado	Sí	Lo hace el idf

Laboratorio: TF-IDF y similitud entre reseñas

El laboratorio calcula en el navegador todo lo de la sección anterior con las reseñas de la Tabla 77.1. Cambia la ponderación, desactiva la eliminación de palabras vacías, escribe una consulta o edita las reseñas.

[5] Si los vectores están normalizados a norma 1, la distancia euclidiana y el coseno dan el mismo orden: $\|\hat{\mathbf{u}} - \hat{\mathbf{v}}\|^2 = 2 - 2\cos(\mathbf{u}, \mathbf{v})$. Por eso k-means sobre TF-IDF normalizado se comporta como un agrupamiento por coseno.

En la versión web, este laboratorio calcula la matriz documento-término, TF-IDF, las similitudes coseno y una búsqueda con las reseñas de la Tabla 77.1. Valores de referencia con TF-IDF clásico y sin palabras vacías: vocabulario de 8 términos, 22 → 16 tokens, 15 de 32 celdas no nulas; normas 1.550 (d_1) y 1.698 (las demás); par más parecido d_1–d_3 con coseno 0.365; la consulta «café frío» empata d_2 y d_4 con 0.408. Con el idf de `scikit-learn`, «café» tiene idf 1, d_1–d_3 sube a 0.520 y la consulta da 0.596 a d_2 y d_4.

Guía del laboratorio.

1. Comprueba el estado inicial contra las tablas de la sección manual: los idf 0, 0.693 y 1.386, las normas 1.550 y 1.698 y el coseno 0.365 entre d_1 y d_3.
2. Antes de cambiar a **Conteo**, predice: ¿qué pares que hoy valen 0 dejarán de valer 0? Comprueba. ¿Por qué d_1–d_2 vale ahora 0.204?
3. Con TF-IDF clásico, cambia la consulta a solo «café». ¿Por qué la búsqueda no puede ordenar nada? Cambia a **TF-IDF (sklearn)**: ¿qué cambia y por qué ese +1 en la fórmula es una decisión de diseño razonable?
4. Desactiva **Quitar palabras vacías**. ¿Cuántos términos nuevos aparecen y cuál tiene el idf más pequeño (después de «café»)? Ahora d_1 y d_3 comparten también «y», pero su similitud **baja** de 0.365 a 0.211. ¿Por qué? (Pista: mira el idf de «qué», «el» y «también», que solo están en d_1.)
5. Rómpelo: escribe d_4 sin acentos, «Cafe frio, pan duro». ¿Qué le pasa al vocabulario y a la similitud entre d_2 y d_4? ¿Qué paso de preprocesamiento lo evitaría?
6. Escribe d_3 como «Servicio nada rápido y café no rico». Con palabras vacías eliminadas, ¿sigue siendo d_3 la reseña más parecida a d_1? ¿Qué parte de la representación falla?

Tareas sobre la representación vectorial

Una vez que cada documento es un vector, las tareas de minería son las mismas de los capítulos anteriores. Las tres más comunes en texto son clasificar, agrupar y descubrir temas.

Clasificación de textos

Detectar quejas, enrutar tickets de soporte a la mesa correcta o filtrar spam son problemas de **clasificación** cuyos atributos son las columnas de la matriz TF-IDF. Dos modelos funcionan especialmente bien con esta representación:

- **Naive Bayes multinomial**, que modela la probabilidad de cada palabra dada la clase y suma la evidencia de cada token (McCallum y Nigam 1998). Su desarrollo completo, con un ejemplo de texto, está en el capítulo de clasificadores bayesianos.
- **Regresión logística** (o SVM lineal) sobre TF-IDF, que asigna un peso a cada término. En alta dimensión y con datos dispersos, los modelos lineales suelen ser difíciles de superar (Pang et al. 2002).

El análisis de **sentimiento** es el caso particular en que la clase es la polaridad de la opinión. Es también donde más duele ignorar el orden: «no estaba rico» contiene «rico». Los bigramas, como veremos en la práctica, recuperan buena parte de esa información.

Modelado de temas

La segunda pregunta de la intuición, *de qué habla cada reseña*, no tiene etiquetas: nadie marcó las reseñas como «comida» o «servicio». Un **modelo de temas** descubre esos grupos a partir de qué palabras **aparecen juntas** en los mismos documentos.

Ejemplo manual. Considera cuatro reseñas con solo cuatro términos (después de quitar palabras vacías):

Tabla 77.8.: Una matriz documento-término que se factoriza exactamente con dos temas.

	café	rico	barista	amable
A: «Café rico.»	1	1	0	0
B: «Barista amable.»	0	0	1	1
C: «Café rico y barista amable.»	1	1	1	1
D: «Café rico, muy rico café.»	2	2	0	0

Esta matriz de 4×4 se puede escribir como el producto de una matriz W de 4×2 y una matriz H de 2×4:

$$\underbrace{\begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 2 & 2 & 0 & 0 \end{pmatrix}}_{X} = \underbrace{\begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & 0 \end{pmatrix}}_{W\ (\text{documentos} \times \text{temas})} \underbrace{\begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}}_{H\ (\text{temas} \times \text{términos})}.$$

Cada fila de H es un **tema**: el primero es {café, rico} (comida) y el segundo {barista, amable} (servicio). Cada fila de W dice **cuánto de cada tema** tiene un documento: A es solo comida, B solo servicio, C mitad y mitad, y D es comida «con el doble de intensidad». Comprueba una celda: la fila de C en X es $1 \cdot (1,1,0,0) + 1 \cdot (0,0,1,1) = (1,1,1,1)$.

En un corpus real la factorización nunca es exacta y hay que buscar la mejor aproximación. Eso es la **factorización de matrices no negativas**.

Definición (NMF). Dada $X \in \mathbb{R}^{N \times m}_{\geq 0}$ y un número de temas $k \ll \min(N, m)$, la factorización de matrices no negativas busca

$$\min_{W \geq 0,\ H \geq 0} \|X - WH\|_F^2, \qquad W \in \mathbb{R}^{N \times k},\ H \in \mathbb{R}^{k \times m}. \tag{77.3}$$

La restricción de **no negatividad** es lo que hace interpretables los temas (Lee y Seung 1999): un documento solo puede *sumar* temas, nunca restarlos, así que cada tema tiene que ser un conjunto de palabras que efectivamente aparecen juntas. Con PCA, en cambio, las componentes mezclan pesos positivos y negativos y son difíciles de leer como temas[6].

[6] Aplicar SVD truncada (la versión de PCA sin centrar) a la matriz TF-IDF se conoce como **análisis semántico latente** (LSA). Captura sinónimos parcialmente, pero sus componentes tienen pesos negativos y no se leen como temas. En `scikit-learn` es `TruncatedSVD`, que acepta matrices dispersas.

El algoritmo clásico alterna dos **actualizaciones multiplicativas** (Lee y Seung 2001), donde $\odot$ y $\oslash$ son el producto y la división elemento a elemento:

$$H \leftarrow H \odot (W^\top X) \oslash (W^\top W H), \qquad W \leftarrow W \odot (X H^\top) \oslash (W H H^\top). \tag{77.4}$$

Si las entradas empiezan positivas, siguen siéndolo (solo se multiplican por cocientes de números no negativos), y el error $\|X - WH\|_F$ nunca aumenta. Como el problema no es convexo, el resultado depende de la inicialización: distintas semillas pueden dar temas distintos[7].

LDA. La alternativa probabilística más conocida es la **asignación latente de Dirichlet** (Blei et al. 2003). LDA supone que cada documento se genera así: se elige una mezcla de temas θ_d (como una fila de W que suma 1); para cada palabra, se elige un tema z según θ_d y luego una palabra según la distribución de ese tema φ_z (como una fila de H normalizada)[8]. Ajustar el modelo es invertir ese proceso: inferir las θ_d y φ_k que mejor explican los conteos observados. LDA trabaja con **conteos** y necesita documentos con suficientes palabras para estimar la mezcla de cada uno; en textos cortos, como reseñas o tuits, NMF sobre TF-IDF suele dar temas más limpios.

Laboratorio: temas con NMF

Este laboratorio ejecuta las actualizaciones multiplicativas de la Ecuación 77.4 sobre diez reseñas cortas. Cada barra de color es un tema; la tabla inferior muestra qué mezcla de temas tiene cada reseña. La reseña r10 habla a la vez de la comida y del servicio.

En la versión web, este laboratorio ejecuta NMF con actualizaciones multiplicativas sobre diez reseñas cortas (25 términos). Valores de referencia con TF-IDF, semilla 1 y 100 iteraciones: con $k = 3$ el error baja de 3.003 a 2.155 y los temas son {delicioso, aroma, latte}, {barista, amable, atención} y {app, pagar, fácil}; la reseña «Latte delicioso y barista amable» queda con 41 % del primer tema y 59 % del segundo. Con $k = 2$ el error final es 2.430 y dos grupos se fusionan; con $k = 4$ baja a 1.92, pero un tema se divide en una versión positiva y otra negativa, y cuál se divide depende de la semilla.

Guía del laboratorio.

1. Con $k = 3$, lleva las iteraciones a 0 y luego súbelas poco a poco. ¿En qué iteración se estabilizan los tres temas? Compara con la curva del error.
2. Comprueba la mezcla de r10. ¿Por qué tiene sentido que no sea 100 % de un solo tema? Observa también que r3 («Café amargo y con mal aroma») cae en el mismo tema que los elogios al sabor: ¿un tema mide de qué se habla o cómo se siente el cliente?
3. Predice qué pasará con $k = 2$: ¿qué dos grupos de palabras se fusionarán? Comprueba con varias semillas.

[7] La factorización tampoco es única aun encontrando el mínimo: si D es una matriz diagonal con entradas positivas, $(WD)(D^{-1}H) = WH$, y cualquier permutación de los temas da la misma reconstrucción. Por eso los temas no tienen un orden natural y su escala es arbitraria. La inicialización NNDSVD (la que usa `scikit-learn` con `init="nndsvda"`) parte de la SVD de X en lugar de valores aleatorios, lo que hace los resultados mucho más estables.

[8] La **distribución de Dirichlet** es una distribución sobre vectores de probabilidades (vectores no negativos que suman 1). Un parámetro pequeño favorece vectores concentrados en pocas componentes: documentos con pocos temas y temas con pocas palabras dominantes. La inferencia en LDA se hace por métodos variacionales (la opción de `scikit-learn`) o por muestreo de Gibbs.

4. Con $k = 4$, prueba cinco semillas. ¿Qué temas se dividen? ¿La división en «sabor bueno» y «sabor malo» es un tema o un sentimiento?
5. Cambia a **Conteos**. ¿Cambian los temas? Observa el peso de «app» y «pagar», que aparecen en tres reseñas: ¿qué hace el idf con ellos?

Implementación en Python

El ejemplo manual

Empezamos reproduciendo la sección manual. La función `tokenizar` aplica las tres reglas del preprocesamiento; la lista de palabras vacías es corta a propósito y, como veremos, **no** incluye «no».

En la práctica no se calcula a mano. `TfidfVectorizer` hace todo en un paso y devuelve una **matriz dispersa**, que guarda solo los valores distintos de cero. Le pasamos nuestro `tokenizar` para usar exactamente el mismo preprocesamiento; las diferencias numéricas vienen del idf suavizado y de la normalización de las filas.

Con el idf suavizado, «café» pesa 1 en lugar de 0, y por eso d_1 y d_3 obtienen un coseno pequeño, pero positivo, con la consulta. El orden de los resultados no cambia: d_2 y d_4 siguen empatadas en primer lugar.

Un corpus de reseñas de Café Norte

Para trabajar a escala simulamos 3 000 reseñas de las cinco sucursales de Café Norte (las mismas de `data/sucursales.csv`). Cada reseña habla principalmente de uno de cinco **aspectos** (sabor, servicio, espera, precio o la app), a veces menciona un segundo aspecto y tiene una calificación de 1 a 5 estrellas. Las sucursales tienen perfiles distintos: en el kiosco de Oriente predominan los comentarios sobre la espera y en Poniente, sobre la app.

Conocer el aspecto y la calificación verdaderos nos permitirá **evaluar** lo que descubran los modelos, algo imposible con reseñas reales sin etiquetar. Definimos como **queja** una reseña de 1 o 2 estrellas. Las reseñas de 3 estrellas son mixtas: elogian un aspecto y critican otro.

	sucursal_id	estrellas	texto
1957	S002	4	El grano tiene un tostado excelente. El capuchino con mucho sabor y buen aroma
2087	S004	4	Primera vez que voy. No tardaron nada, cero espera. Me atendieron rápido y sin
1394	S004	4	Pedir en la app es fácil. La app funcionó perfecto.
1520	S004	5	No tardaron nada, cero espera.
1098	S004	1	Fui por la mañana. La fila larguísima y lenta. Esperé veinte minutos.
393	S005	1	El cold brew amargo y con mal aroma.

Antes de modelar conviene mirar la matriz. Aun con un vocabulario pequeño, casi todas las celdas son cero, y los bigramas multiplican el número de columnas por diez.

El término más frecuente del corpus es «no». Si lo hubiéramos tratado como palabra vacía, como hacen muchas listas genéricas, habríamos borrado la señal más directa de una negación.

Clasificar quejas

Comparamos tres modelos con validación cruzada estratificada de cinco pliegues. Todos son *pipelines*, así que el vocabulario y el idf se aprenden **solo** con los pliegues de entrenamiento, sin fuga de información hacia el de prueba.

Tabla 77.9.: Detección de quejas con validación cruzada de cinco pliegues (media ± desviación estándar). La clase positiva es «queja».

modelo	precision	recall	f1
Naive Bayes · unigramas (conteos)	0.931 ± 0.020	0.975 ± 0.010	0.952 ± 0.015
Logística · unigramas (TF-IDF)	0.948 ± 0.019	0.977 ± 0.004	0.962 ± 0.010
Logística · uni + bigramas (TF-IDF)	0.963 ± 0.017	0.982 ± 0.005	0.972 ± 0.009

Los tres modelos son buenos, porque las quejas usan un vocabulario muy distinto del de los elogios. Los bigramas dan la mejora más clara en precisión. Para ver por qué, miramos los pesos de la regresión logística: un peso positivo empuja hacia «queja» y uno negativo hacia «no queja».

Tabla 77.10.: Términos con mayor peso en la regresión logística con unigramas y bigramas.

	hacia «queja»	peso (queja)	hacia «no queja»	peso (no queja)
0	sabía	3.78	no caro	-7.29
1	estaba rico	3.74	barista amable	-5.61
2	rico	3.74	había	-4.76
3	incompleto	3.73	había fila	-4.76
4	llegó incompleto	3.73	no había	-4.76
5	grosero	3.62	atendieron	-4.60
6	personal grosero	3.62	excelente	-4.58
7	excesivo	3.26	volveré	-4.49
8	precio excesivo	3.26	recomendado	-4.38
9	atención no	3.25	estaba delicioso	-4.26

Dos lecciones salen de estos pesos:

- **«rico» empuja hacia la queja.** Parece absurdo, pero en este corpus los elogios dicen «delicioso» o «riquísimo», y «rico» solo aparece en «no estaba rico». El modelo aprendió una regularidad real de los datos, no el significado de la palabra. Es un recordatorio de que los coeficientes describen el corpus de entrenamiento.
- **El bigrama «no caro» es la señal más fuerte de «no queja»**, mientras que «caro» solo empuja hacia la queja. Sin bigramas, «no es caro» y «muy caro» comparten el término «caro» y el modelo tiene que adivinar.

¿Dónde se equivoca el mejor modelo? Usamos predicciones de validación cruzada (cada reseña se predice con un modelo que no la vio):

	estrellas	queja	texto
87	3	0	El cold brew aguado y sin sabor. El latte amargo y con mal aroma. No había fila. Qué de
93	1	1	El personal fue grosero. El barista ni saludó. Acumulé puntos en la app sin problema.
102	3	0	Cliente frecuente. El brownie sabía a recalentado. El americano no estaba rico. Pagar con
248	3	0	Cliente frecuente. Precio justo para la calidad. Olvidaron mi orden y tuve que esperar ma

La mayoría de los errores son reseñas de **3 estrellas**: textos que critican un aspecto y elogian otro, y que por definición no son quejas. El límite del modelo es el límite de la etiqueta: con una clase intermedia (o con una etiqueta por aspecto) el problema estaría mejor planteado. En datos reales se suma otra fuente de error que aquí no simulamos: la ironía («¡qué rapidez, solo 40 minutos!»).

Descubrir temas con NMF

Para los temas cambiamos el preprocesamiento, como indica la Tabla 77.7: «no» y «nada» no dicen de qué habla una reseña, así que los quitamos. También descartamos términos que aparecen en menos de 5 reseñas o en más del 30 %.

Tabla 77.11.: Temas encontrados por NMF (k = 5) sobre TF-IDF: ocho términos con mayor peso en cada fila de H.

	nombre asignado	términos con mayor peso
T0	app	app · llegó · pedido · pagar · puntos · completo · pedir · fácil
T1	espera	espera · rápido · hora · pico · tardaron · cero · atendieron · incluso
T2	sabor	estaba · amargo · frappé · aroma · sabor · americano · cheesecake · riquísimo
T3	servicio	personal · atención · excelente · amable · barista · gerente · respetuoso · atento
T4	precio	precio · barato · caro · promoción · zona · justo · calidad · pena

Los cinco temas corresponden a los cinco aspectos de la simulación, aunque NMF nunca vio la columna `aspecto`. ¿Qué tan bien coinciden? Asignamos a cada reseña su tema dominante (el máximo de su fila de W) y lo cruzamos con el aspecto verdadero:

Figura 77.1.

El índice de Rand ajustado[9] resume la coincidencia en un número. Ahora comparamos con LDA, que trabaja con conteos:

Tabla 77.12.: Temas encontrados por LDA (k = 5) sobre conteos.

	términos con mayor peso
T0	hora · personal · espera · fila · rápido · tardaron · minutos · pico
T1	app · precio · llegó · caro · pedido · completo · barato · promoción

[9] El **índice de Rand ajustado** compara dos particiones contando los pares de elementos que ambas ponen juntos o separados, corregido para que una asignación al azar valga 0 en promedio. Vale 1 si las particiones coinciden exactamente (sin importar los nombres de los grupos). Se estudia en el capítulo de introducción al clustering.

Tabla 77.12.: Temas encontrados por LDA (k = 5) sobre conteos.

	términos con mayor peso
T2	atendieron · sin · espera · rápido · cajera · menú · explicó · paciencia
T3	estaba · aroma · amargo · frappé · sabía · cheesecake · muffin · sabor
T4	app · gerente · pagar · problema · más · resolvió · amabilidad · cómodo

LDA mezcla aspectos en varios temas (por ejemplo, la app con el precio). No es un defecto de implementación: con 7 u 8 palabras útiles por reseña, hay muy poca información para estimar la mezcla de temas de cada documento, que es justo lo que LDA necesita. En documentos largos (artículos, contratos, expedientes) la situación se invierte con frecuencia. En cualquier caso, **el número de temas y su interpretación son decisiones del analista**: conviene probar varios valores de k y revisar los temas con alguien que conozca el negocio.

Palabras que aparecen juntas

Si cada reseña es una «canasta» de palabras, todo lo de la Parte V se aplica sin cambios. Minamos reglas de asociación entre palabras dentro de las quejas:

Tabla 77.13.: Reglas entre palabras dentro de las quejas con mayor lift (soporte 3 %). Cada par aparece una vez.

regla	support	confidence	lift	PMI (bits)
otra → subieron	0.034	1.0	29.333	4.874
aplicó → pagué	0.044	1.0	22.957	4.521
notificación → nunca	0.045	1.0	22.468	4.490
pagué → completo	0.044	1.0	22.468	4.490
aplicó → completo	0.044	1.0	22.468	4.490
trabó → pagar	0.043	1.0	22.000	4.459
tarjeta → aceptó	0.048	1.0	20.706	4.372
pagué → promoción	0.044	1.0	20.706	4.372

Las reglas con lift más alto son **colocaciones**: palabras que forman una expresión fija («subieron … otra vez», «la promoción no se aplicó y pagué»). En lingüística computacional, el logaritmo del lift tiene nombre propio: es la **información mutua puntual** (PMI)[10], la medida clásica para detectar colocaciones y la base de muchos métodos de representación de palabras.

[10] La **información mutua puntual** de dos palabras a y b es $\mathrm{PMI}(a,b) = \log_2 \frac{P(a,b)}{P(a)\,P(b)}$, es decir, el logaritmo del lift. Vale 0 si las palabras son independientes y es positiva si aparecen juntas más de lo esperado. Tiene el mismo defecto que el lift: favorece pares raros, por lo que se combina con un soporte mínimo.

De vuelta al almacén de datos

El resultado de todo lo anterior es estructurado: para cada reseña, una **probabilidad de queja** y un **tema**. Esas columnas pueden cargarse en una tabla de hechos `hecho_resena` con las llaves de sucursal y fecha, y consultarse como cualquier otra medida. Usamos probabilidades de validación cruzada, para que cada reseña se califique con un modelo que no la vio (en producción, el modelo se aplicaría a las reseñas nuevas, que pueden no traer estrellas: menciones en redes sociales, correos, tickets).

		resenas	tasa_queja_estimada
nombre	formato		
Centro 1	Tienda	593	0.287
Norte 1	Tienda	620	0.252
Oriente 1	Kiosco	588	0.481
Poniente 1	Kiosco	600	0.338
Sur 1	Tienda	599	0.417

(a)

Figura 77.2.

Este es el tipo de resultado que un gerente puede usar: no «hay 1 000 quejas», sino **qué** se queja la gente **en cada sucursal**. Con el tema como dimensión, el análisis admite *drill-down* por fecha, por formato de tienda o por producto mencionado, exactamente como en el capítulo de cubos OLAP.

Más allá de la bolsa de palabras

TF-IDF trata «delicioso» y «riquísimo» como columnas sin relación: si una reseña dice una y otra reseña la otra, su coseno es cero. Los métodos modernos representan cada palabra, o cada texto completo, con un **vector denso** de unos cientos de dimensiones (un *embedding*) en el que textos de significado parecido quedan cerca.

- **word2vec** (Mikolov et al. 2013) aprende un vector por palabra prediciendo cada palabra a partir de sus vecinas. Palabras que aparecen en contextos parecidos («delicioso», «riquísimo», «sabroso») terminan cerca.
- **BERT** (Devlin et al. 2019) y los modelos tipo *transformer* producen vectores que dependen del contexto: «rico» en «café rico» y en «país rico» recibe vectores distintos.
- **Sentence-BERT** (Reimers y Gurevych 2019) ajusta esos modelos para que el coseno entre vectores de oraciones mida similitud semántica. Es la base de la búsqueda semántica actual.
- **BERTopic** (Grootendorst 2022) agrupa esos vectores (con UMAP y HDBSCAN, los métodos de la Parte VII) y describe cada grupo con un TF-IDF por clase.

El flujo es el mismo de este capítulo, con una representación distinta. El siguiente código no se ejecuta al compilar el libro porque requiere `sentence-transformers` y PyTorch (ver `requirements-torch-jax.txt`):

Los grandes modelos de lenguaje permiten además clasificar o extraer temas **sin ejemplos etiquetados**, describiendo la tarea en una instrucción. Las reglas de este libro siguen valiendo: hay que

medir el desempeño contra un conjunto etiquetado, cuidar la fuga de información y considerar el costo, la latencia y la privacidad de enviar textos de clientes a un servicio externo. Para muchos problemas de negocio con vocabulario estable, un TF-IDF con regresión logística sigue siendo una línea base rápida, barata, interpretable y difícil de superar por mucho margen (Jurafsky y Martin 2025).

Ventajas y limitaciones

Tabla 77.14.: Comparación de representaciones de texto.

Aspecto	Bolsa de palabras y TF-IDF	Embeddings densos
Representación	Dispersa, una dimensión por término	Densa, unos cientos de dimensiones
Sinónimos y contexto	No los captura	Sí
Interpretabilidad	Alta: cada peso es una palabra	Baja
Costo	Muy bajo, sin GPU	Requiere un modelo preentrenado; idealmente GPU
Datos necesarios	Funciona con pocos miles de documentos	Aprovecha el preentrenamiento
Riesgo principal	Decisiones de preprocesamiento (vacías, negación, acentos)	Sesgos del modelo preentrenado; difícil de auditar

Tabla 77.15.: Tareas de minería de textos y su relación con el resto del libro.

Tarea	Técnica en este capítulo	Capítulo de referencia
Buscar documentos	TF-IDF y coseno	—
Detectar quejas o sentimiento	Naive Bayes, regresión logística con n-gramas	Clasificación
Descubrir temas	NMF, LDA	No supervisado
Encontrar expresiones	Reglas de asociación, PMI	Extracción de patrones
Analizar por sucursal o periodo	Probabilidad y tema como medida y dimensión	Almacenes de datos y OLAP

A. Presentaciones de clase

Herramientas para proyectos profesionales

Quarto: un proyecto, muchas salidas

Taller práctico de 60 minutos · entorno local · Python · formatos · GitHub Pages

Ver taller de Quarto →

Entendimiento de negocio

Antes del modelo

Roles, ciclo de vida de los datos y *Business Understanding Canvas*.

35 diapositivas en formato PDF.

Abrir presentación

De una necesidad de negocio a un problema analítico

Identificación de interesados, preguntas de negocio y formulación del problema analítico.

25 diapositivas en formato PDF.

Abrir presentación

Arquitecturas para datos analíticos

Construyamos un Data Warehouse

De las transacciones operativas a una arquitectura preparada para análisis.

39 diapositivas en formato PDF.

Abrir presentación

De silos a Lakehouse

Evolución de las arquitecturas de datos y criterios para adoptar un *data lakehouse*.

30 diapositivas en formato PDF.

Abrir presentación

Análisis Exploratorio de Datos

Medidas de variabilidad · repaso animado

Rango · IQR · boxplot · varianza · desviación estándar

Abrir presentación animada →

Medidas de heterogeneidad · repaso animado

Gini–Simpson · Shannon · categorías efectivas · eta cuadrada

Abrir presentación animada → Abrir simulador interactivo →

Medidas de concentración · clase animada (60 min)

CR · HHI · Lorenz · Gini · práctica guiada

Abrir presentación animada →

Datos faltantes

Tratamiento, detección e imputación de valores ausentes.

Ver presentación →

EDA BigMart

Heterogeneidad · Concentración · Correlación · PCA

EDA BigMart →

EDA Multivariado

PCA · Correlación · T-SNE · UMAP

EDA Multivariado →

Componentes principales · clase animada 3D (60 min)

Resorte y tres cámaras · máxima varianza · autovectores · nube 3D · límites

Abrir presentación animada → Abrir laboratorio 3D →

Chi cuadrado

Correlacion categorica · Test de independencia

Chi cuadrado →

Reglas de asociación

Relación entre variables categóricas · Soporte · Confianza · Lift

Reglas de asociación →

Reglas de asociación · clase animada (60 min)

Construir reglas · probabilidad como área · lift, leverage, conviction y Zhang · decidir

Abrir presentación animada →

FP-Growth · clase animada (60 min)

Costo de Apriori · FP-tree · bases y árboles condicionales · comparación y Python

Abrir presentación animada →

Evaluación de clasificadores

Matriz de confusión · Métricas de evaluación · Curvas ROC y PR

Evaluación de clasificadores →

Ejemplo de clasificación

Matriz de confusión · Métricas de evaluación · Curvas ROC y PR

Ejemplo de clasificación →

Clasificadores bayesianos

Teorema de Bayes · Naive Bayes · Modelos gaussianos

Clasificadores bayesianos →

Redes neuronales con PyTorch y JAX

Entrenamiento práctico · Autodiferenciación · Comparación de frameworks

PyTorch y JAX →

B. Banco de Proyectos

Esta sección reúne diez propuestas de proyecto final para poner en práctica el ciclo completo del libro: entendimiento de negocio, construcción (o consumo) de un almacén de datos, EDA, preprocesamiento y modelado (Marco Metodológico de Minería de Datos).

Cada propuesta describe un problema de negocio real, uno o más datasets con los que arrancar, qué la hace desafiante (para que no se quede en un ejercicio de manual) y las consideraciones legales/éticas de scraping cuando aplican. Ningún dataset está atado a un proyecto: úsalo como punto de partida y ajusta el alcance con tu equipo durante la fase de entendimiento de negocio.

> **!** Importante
>
> **Antes de scrapear cualquier sitio.** Revisa siempre el `robots.txt` y los Términos de Servicio del sitio, prefiere APIs oficiales o datasets ya compilados antes que scraping directo, respeta límites de tasa (*rate limiting*) si automatizas descargas, y nunca recolectes ni publiques datos personales identificables. Estas reglas se repiten en cada propuesta porque son no negociables, no porque cambien de un proyecto a otro.

1. Value bets en Liga MX

Problema de negocio. Una plataforma de apuestas o un boletín deportivo quiere detectar cuándo las casas de apuestas "se equivocan" al fijar los momios de un partido.

Dataset(s) sugerido(s).

- LigaMX Matches 2016–2024 (Kaggle) — resultados históricos de Liga MX.
- Historical Football Results/Betting Odds Data (Kaggle) — resultados y momios históricos multi-liga; revisa si incluye México o combínalo con el dataset anterior por fecha/equipos.

Qué lo hace desafiante. No existe una etiqueta de "value bet": el equipo tiene que construirla comparando la probabilidad implícita del momio contra el resultado real observado. Exige pensar en calibración de probabilidades, no solo en accuracy.

Técnicas del curso que cubre. Reglas de asociación, clasificación, evaluación con métricas de negocio (ROI simulado, no accuracy).

> **⚠** Advertencia
>
> Si decides scrapear momios en vivo en lugar de usar un dataset ya compilado, revisa el `robots.txt` del sitio de la casa de apuestas: muchas prohíben explícitamente el scraping en sus Términos de Servicio. Prefiere siempre una API pública o un dataset ya cerrado.

2. Priorización de contratos públicos sospechosos

Problema de negocio. Una ONG anticorrupción necesita priorizar qué licitaciones revisar a mano con recursos limitados.

Dataset(s) sugerido(s).

- Contratos y expedientes del sistema histórico de CompraNet — datos.gob.mx
- Datos Abiertos de las Unidades Compradoras (ComprasMX)

Qué lo hace desafiante. No existe *ground truth* de "corrupción": el éxito se mide por qué tan útil es la priorización con recursos limitados, no por accuracy. Alto riesgo ético de falsos positivos (señalar a un proveedor honesto).

Técnicas del curso que cubre. Reglas de asociación, detección de anomalías, clasificación con datos débilmente etiquetados.

> **Advertencia**
>
> CompraNet/ComprasMX ya expone datos vía portal de datos abiertos — úsalo en vez de scrapear HTML. Si automatizas la descarga, respeta límites de tasa y guarda evidencia de la fecha de descarga, porque los datos se actualizan y corrigen con el tiempo.

3. Detección temprana de réplicas sísmicas anómalas

Problema de negocio. Protección Civil o un medio de análisis de riesgo quiere anticipar clústers de actividad sísmica inusual.

Dataset(s) sugerido(s).

- Catálogo de sismos — Servicio Sismológico Nacional, UNAM

Qué lo hace desafiante. Series de tiempo espaciales con eventos raros por definición; exige justificar el umbral de "anómalo" en términos que Protección Civil pueda entender y accionar.

Técnicas del curso que cubre. Clustering espacial, detección de anomalías, reglas de asociación (magnitud/profundidad/zona).

> **Nota**
>
> El SSN ofrece el catálogo en descarga directa (CSV/texto) sin necesidad de scraping. Si automatizas la descarga periódica, evita solicitudes masivas simultáneas al servidor.

4. Calidad del aire y “días de contingencia” en CDMX

Problema de negocio. SEDEMA o una app de salud pública quiere predecir y explicar picos de contaminación.

Dataset(s) sugerido(s).

- Índice de la Calidad del Aire — Portal de Datos Abiertos CDMX
- Red Automática de Monitoreo Atmosférico (RAMA) — Portal de Datos Abiertos CDMX
- Portal SIMAT / Dirección de Monitoreo Atmosférico — histórico por estación, incluye descargas en Excel.

Qué lo hace desafiante. Fuerte estacionalidad, correlación espacial entre estaciones y variables de tráfico/clima como confusores; distinguir correlación de causa real.

Técnicas del curso que cubre. EDA con series de tiempo, PCA, clustering de estaciones, detección de anomalías.

> **Nota**
>
> SIMAT tiene descarga directa por estación/año — no requiere scraping. Si combinas con datos de tráfico de terceros (p. ej. Waze), revisa los términos de uso de esa API antes de automatizar la extracción.

---

5. Fraude en reembolsos/transacciones fintech

Problema de negocio. Una startup fintech LATAM quiere reducir el fraude sin bloquear a usuarios legítimos.

Dataset(s) sugerido(s).

- PaySim — Synthetic Financial Datasets For Fraud Detection (Kaggle)
- IEEE-CIS Fraud Detection (Kaggle)

Qué lo hace desafiante. Desbalance de clases extremo (fraude $<1\,\%$); obliga a discutir el costo asimétrico de falsos negativos vs. falsos positivos, no solo accuracy.

Técnicas del curso que cubre. Clasificación con clases desbalanceadas, detección de anomalías, evaluación con métricas apropiadas (recall, PR-AUC).

> **Nota**
>
> No aplica scraping: son datasets ya publicados en Kaggle. Verifica la licencia específica de cada dataset (algunas restringen uso comercial) antes de usarlo más allá del curso.

---

6. Segmentación de incidentes de inseguridad por alcaldía/colonia

Problema de negocio. Un observatorio ciudadano quiere apoyar decisiones de patrullaje con evidencia, no percepción.

Dataset(s) sugerido(s).

- Datos Abiertos de Incidencia Delictiva — SESNSP
- Incidencia delictiva — datos.gob.mx

Qué lo hace desafiante. Dato sensible políticamente: exige comunicación responsable (evitar estigmatizar zonas) y separar tendencia real de subregistro/denuncia.

Técnicas del curso que cubre. Clustering geográfico/temporal, EDA con Gini/entropía por tipo de delito, reglas de asociación.

> **!** Importante
>
> El SESNSP publica los datos en descarga directa — no requiere scraping. Si cruzas con datos socioeconómicos por colonia (INEGI), cuida el nivel de agregación para no exponer o inferir datos de personas identificables.

---

7. Reseñas falsas o manipuladas en e-commerce

Problema de negocio. Un marketplace (estilo Mercado Libre) quiere detectar reseñas fabricadas o campañas de reputación.

Dataset(s) sugerido(s).

- amazon_reviews_multi_es (Hugging Face) — reseñas de Amazon en español.
- Mercado Libre Data Challenge (Kaggle) — como fuente adicional de texto/metadatos de publicaciones.

Qué lo hace desafiante. No hay una etiqueta confiable de "reseña falsa": el equipo debe operacionalizar señales indirectas (ráfagas temporales, texto genérico repetido, sentimiento desalineado con la calificación).

Técnicas del curso que cubre. Minería de texto, TF-IDF + sentimiento, detección de anomalías, clustering de patrones de escritura.

> **!** Importante
>
> Prefiere datasets ya recolectados antes que scrapear Mercado Libre o Amazon directamente: ambos prohíben el scraping en sus Términos de Servicio y usan detección activa de bots. Si de todas formas construyes un dataset propio, hazlo solo con datos públicos ya visibles sin login, con retraso entre solicitudes, y sin recolectar datos personales de quienes escriben las reseñas.

8. Predicción de retrasos y saturación en el Metro/transporte público

Problema de negocio. Una autoridad de movilidad quiere anticipar la saturación de líneas y recomendar rutas alternas.

Dataset(s) sugerido(s).

- Afluencia diaria del Metro CDMX — Portal de Datos Abiertos CDMX
- Afluencia diaria de Metrobús CDMX — Portal de Datos Abiertos CDMX

Qué lo hace desafiante. Estacionalidad múltiple (hora, día, evento especial), datos con huecos y calidad heterogénea; el reto de negocio es "accionable en tiempo real", no solo descriptivo.

Técnicas del curso que cubre. EDA con visualización temporal, clustering de patrones de demanda, detección de anomalías (incidentes atípicos).

> **Nota**
>
> El STC Metro y Metrobús publican afluencia en datos abiertos con descarga directa — no requiere scraping. Si agregas datos de Waze u otro proveedor externo, usa su API oficial de partners en vez de scrapear el mapa web: Waze prohíbe explícitamente el scraping de su plataforma.

9. Deserción escolar en educación media superior

Problema de negocio. Una Secretaría de Educación estatal quiere identificar estudiantes en riesgo antes de que abandonen.

Dataset(s) sugerido(s).

- Estadística educativa — DGPPYEE, SEP
- Resultados PLANEA Básica
- Datasets de educación — datos.gob.mx

Qué lo hace desafiante. Etiquetas retrospectivas (se sabe quién desertó después de que ya pasó); exige pensar en qué variables están disponibles a tiempo para intervenir, no solo cuáles correlacionan.

Técnicas del curso que cubre. Clasificación, reglas de asociación, PCA/reducción de dimensionalidad, checkpoint de stakeholder realista (política pública).

> **!** Importante
>
> La SEP publica estos indicadores agregados en descarga directa (no a nivel de estudiante individual) — no requiere scraping. Si buscas microdatos, usa solo bases públicas anonimizadas oficiales y nunca intentes reconstruir la identidad de estudiantes individuales.

---

10. Discurso de odio/polarización en debates electorales

Problema de negocio. Un medio de comunicación o una autoridad electoral quiere monitorear el tono del discurso público sin censurar.

Dataset(s) sugerido(s).

- HatEval — SemEval-2019 Task 5 (Hugging Face) — tuits en inglés y español anotados para discurso de odio contra inmigrantes y mujeres.

Qué lo hace desafiante. Ambigüedad lingüística fuerte en español (sarcasmo, regionalismos); exige justificar el umbral entre "polarizado" y "de odio", con implicaciones éticas serias.

Técnicas del curso que cubre. Minería de texto, TF-IDF, sentimiento con dos métodos comparados, discusión ética explícita en la presentación final.

> **!** Importante
>
> X/Twitter restringe fuertemente el scraping y el acceso gratuito a su API desde 2023 — usa el dataset académico HatEval en vez de scraping directo. Si de todas formas recolectas comentarios propios de otra fuente, anonimiza a los usuarios, no publiques el dataset crudo, y documenta el criterio ético de qué se considera "discurso de odio" antes de anotar datos.

C. Glosario Unificado de Términos de Datos y Computación en la Nube

Este glosario consolida los términos y definiciones fundamentales abordados en los módulos teóricos y técnicos del curso, incluyendo conceptos de infraestructura en la nube, almacenamiento, procesamiento, y gobernanza.

A

- **Advanced Provisioning (Aprovisionamiento Avanzado)**: Modelo de entrega en la nube en el que el usuario firma un acuerdo formal con el proveedor de servicios en la nube y paga un precio fijo o se le factura mensualmente.
- **Apache Hadoop**: Framework de código abierto que permite el almacenamiento y procesamiento distribuido de grandes conjuntos de datos a través de clústeres de computadoras utilizando modelos de programación simples.
- **Archival Policy (Política de Archivo)**: Lineamiento que define dónde, cómo y por cuánto tiempo se deben almacenar los datos una vez que un proyecto de análisis ha finalizado.
- **Audit (Auditoría de Datos)**: Examen formal y sistemático de cómo los usuarios acceden y utilizan los datos, diseñado para garantizar el cumplimiento de las políticas de seguridad e identificar vulnerabilidades.
- **Autoscaling (Escalabilidad Automática)**: Servicio en la nube que monitorea las aplicaciones y ajusta automáticamente la capacidad de cómputo (escalando hacia arriba o hacia abajo) según la demanda en tiempo real.

B

- **Backend Platform (Plataforma de Backend)**: Conjunto de componentes físicos y lógicos de la arquitectura de la nube que soportan la ejecución del sistema (servidores, redes, almacenamiento).
- **Batch Processing (Procesamiento por Lotes)**: Método de procesamiento de datos que consiste en recolectar un gran volumen de transacciones u operaciones durante un periodo de tiempo y procesarlas de forma conjunta en una sola ejecución.
- **Business Data Request (Solicitud de Datos de Negocio)**: Pregunta o requerimiento formal planteado por un área operativa que puede ser respondido o solucionado mediante la extracción y análisis de datos.

C

- **Cloud Data Storage (Almacenamiento de Datos en la Nube)**: Solución tecnológica que permite a las organizaciones guardar, acceder y mantener datos digitales en servidores remotos administrados por un proveedor de servicios (ej. Google Cloud Storage).
- **Complex Data Types (Tipos de Datos Complejos)**: Estructuras de datos no primitivas (como Arrays y Structs) utilizadas en bases de datos modernas como BigQuery para almacenar registros anidados y optimizar el rendimiento de las consultas desnormalizadas.

D

- **Data Catalog (Catálogo de Datos)**: Inventario centralizado de los activos de datos de una organización que proporciona metadatos detallados para que los analistas puedan descubrir, comprender y evaluar la calidad de las fuentes disponibles.
- **Data Governance (Gobierno de Datos)**: Conjunto de procesos, políticas, roles y estándares que garantizan el uso seguro, confiable y eficiente de los datos dentro de una organización.
- **Data Lake (Lago de Datos)**: Repositorio de almacenamiento que aloja grandes volúmenes de datos brutos en su formato original (estructurado, semi-estructurado o no estructurado) hasta que sean requeridos para análisis.
- **Data Warehouse (Almacén de Datos)**: Sistema de base de datos relacional optimizado para el análisis de datos (OLAP) que consolida información depurada proveniente de múltiples sistemas transaccionales (OLTP).

N

- **Normalization (Normalización)**: Técnica de diseño de bases de datos que organiza las tablas relacionales para minimizar la redundancia de datos y proteger la integridad referencial.

S

- **Streaming Processing (Procesamiento en Tiempo Real)**: Método de procesamiento de datos que analiza y transforma los registros de forma continua, instantáneamente a medida que son generados por las fuentes emisoras.

D. Preparación para Entrevistas Analíticas

La Importancia del Proceso de Pensamiento

En el reclutamiento para roles de analítica, minería e ingeniería de datos, las pruebas técnicas y las entrevistas ya no se limitan a evaluar si recuerdas la sintaxis exacta de una consulta SQL o los parámetros de un clasificador en Scikit-Learn. Los entrevistadores buscan comprender **cómo piensas y resuelves problemas complejos**.

Demostrar tu proceso de pensamiento de forma estructurada le indica al empleador que: * Puedes lidiar con la ambigüedad (problemas de negocio mal definidos). * No saltas a conclusiones sin explorar hipótesis. * Priorizas el valor de negocio antes que la complejidad del algoritmo. * Eres capaz de comunicar conceptos complejos a stakeholders no técnicos.

El Marco de Resolución de Problemas en 4 Pasos

Cuando te enfrentes a un caso de negocio o pregunta abierta en una entrevista técnica, evita dar una respuesta directa e inmediata. Utiliza el siguiente marco estructurado para guiar al entrevistador a través de tu solución:

```
graph TD
    P1["Paso 1: Entender y Acotar<br>(Hacer preguntas de negocio)"]
    --> P2["Paso 2: Datos e Hipótesis<br>(Identificar fuentes y métricas)"]
    --> P3["Paso 3: Metodología Analítica<br>(ETL, EDA, Modelado)"]
    --> P4["Paso 4: Impacto y Comunicación<br>(Resultados para Stakeholders)"]
```

Paso 1: Entender y Acotar el Problema

- **Qué hacer**: Haz preguntas aclaratorias para definir el alcance del problema. Nunca asumas.
- **Frases útiles**:
    - *"Antes de estructurar la base de datos, ¿cuál es el objetivo principal de este análisis para el negocio?"*
    - *"¿Estamos midiendo la tasa de retención de forma mensual o anual?"*

Paso 2: Formular Hipótesis e Identificar Datos

- **Qué hacer**: Propón las variables y fuentes de datos necesarias. Explica qué supuestos quieres validar.
- **Frases útiles**:
  - *"Mi hipótesis es que los clientes que cancelan su servicio tienen una baja frecuencia de inicio de sesión en los últimos 15 días. Para validar esto, necesitaría acceder a los logs de actividad del usuario y a la tabla de suscripciones."*

Paso 3: Definir la Metodología Analítica

- **Qué hacer**: Explica paso a paso cómo procesarás y modelarás la información (desde la ingesta hasta la validación del modelo).
- **Frases útiles**:
  - *"Primero realizaría una limpieza para tratar los valores nulos en el historial. Luego, aplicaría un análisis exploratorio (EDA) para visualizar correlaciones. Si el objetivo es predictivo, entrenaría un XGBoost y evaluaría su desempeño utilizando la métrica Recall, porque en este caso los falsos negativos son muy costosos."*

Paso 4: Traducir al Impacto de Negocio

- **Qué hacer**: Concluye explicando cómo presentarías los resultados a un director de área o stakeholder clave.
- **Frases útiles**:
  - *"Finalmente, presentaría los resultados mediante un dashboard interactivo enfocado en el ahorro financiero estimado al retener al 10 % de los clientes identificados en riesgo por el modelo."*

Preguntas de Práctica Comunes y Guías de Respuesta

Caso 1: Diseñar un Pipeline de Ingesta

> **Pregunta del Entrevistador**: *"Nuestra aplicación móvil genera millones de eventos de clics por segundo y queremos analizarlos diariamente para ajustar ofertas. ¿Cómo diseñarías esta arquitectura?"*

- **Enfoque de respuesta**:
  1. Clasifica el procesamiento: Identifica que es un caso mixto (ingesta continua/streaming, análisis en lote/batch).

2. Propón herramientas del ecosistema en la nube: Sugiere un almacenamiento intermedio de bajo costo (ej. Google Cloud Storage) para almacenar el flujo continuo, y un motor de procesamiento distribuido (ej. PySpark en Databricks) para procesar los datos agregados cada noche.
3. Añade gobernanza: Menciona la necesidad de un catálogo de datos para indexar estos esquemas semi-estructurados (JSON).

Caso 2: Selección de Métricas Analíticas

Pregunta del Entrevistador: *"Tenemos un modelo para detectar fraudes en transacciones de e-commerce. La precisión general (Accuracy) es del 99 %. ¿Deberíamos ponerlo en producción?"*

- **Enfoque de respuesta**:

1. Desafía la métrica: Explica que en conjuntos de datos altamente desbalanceados (como el fraude, donde solo el 0.1 % de las transacciones son fraudulentas), la precisión general es una métrica engañosa (un modelo que prediga 'No Fraude' para todo tendrá 99.9 % de precisión).
2. Propón métricas alternativas: Sugiere usar **F1-Score**, **Precision** o **Recall** dependiendo del costo relativo de los Falsos Positivos (bloquear a un cliente legítimo) frente a los Falsos Negativos (dejar pasar un fraude).

E. Bibliografía y Recursos Adicionales

Este apéndice recopila las referencias bibliográficas, documentación oficial de herramientas y lecturas recomendadas citadas a lo largo de las diferentes unidades del curso.

Referencias Bibliográficas

- **Attaran, M. (2017)**. *Cloud computing technology: Leveraging the power of the internet to improve business performance*. Journal of Systems and Information Technology, 19(3/4), 232-254. (Referenciado en la introducción a la nube y arquitecturas de almacenamiento).
- **Kimball, R., & Ross, M. (2013)**. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. John Wiley & Sons. (Lectura fundamental para la Parte I: Modelado Dimensional y Esquemas de Estrella/Copo de Nieve).
- **Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000)**. *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS. (Metodología analítica base para la Parte II).

Documentación Oficial y Guías de Tecnologías

Para profundizar en las herramientas prácticas utilizadas en los laboratorios del curso, consulta los siguientes recursos en línea:

- **Google Cloud Architecture Framework**: Guías de diseño estructurado para sistemas en la nube de Google. Ver documentación.
- **Guía de Inicio Rápido de BigQuery**: Documentación de referencia para almacenamiento analítico y consultas a gran escala. Ver documentación.
- **PySpark SQL and DataFrame API**: Referencia completa para el desarrollo de transformaciones distribuidas a escala. Ver documentación.
- **dbt (Data Build Tool) Documentation**: Guía para la transformación de datos SQL dentro del Data Warehouse y orquestación. Ver documentación.

Conjuntos de Datos y Casos de Estudio

- **TheLook eCommerce Dataset**: Conjunto de datos público que simula transacciones, usuarios, envíos e inventario de una tienda de ropa en línea. Hospedado en BigQuery Public Data (`bigquery-public-data.thelook_ecommerce`).
- **BigMart Sales Prediction**: Caso de estudio de ventas minoristas utilizado en los laboratorios de modelado predictivo y preprocesamiento de la Parte II.

References

Aggarwal, Charu C., y Chandan K. Reddy. 2013. *Data Clustering: Algorithms and Applications*. CRC Press.

Aggarwal, Charu C., y ChengXiang Zhai, eds. 2012. *Mining Text Data*. Springer.

Agrawal, Rakesh, y Ramakrishnan Srikant. 1994. «Fast Algorithms for Mining Association Rules in Large Databases». *Proceedings of the 20th International Conference on Very Large Data Bases*, VLDB '94, 487-99.

Agrawal, Rakesh, y Ramakrishnan Srikant. 1995. «Mining Sequential Patterns». *Proceedings of the Eleventh International Conference on Data Engineering (ICDE)*, 3-14.

Aloise, Daniel, Amit Deshpande, Pierre Hansen, y Preyas Popat. 2009. «NP-Hardness of Euclidean Sum-of-Squares Clustering». *Machine Learning* 75 (2): 245-48. https://doi.org/10.1007/s10994-009-5103-0.

Amazon Web Services. 2024. *Amazon SageMaker Documentation*. AWS Documentation. https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html.

Analytics Vidhya. 2016. *BigMart Sales Prediction (Practice Problem)*. DataHack Platform. https://www.analyticsvidhya.com/datahack/contest/big-mart-sales-prediction/.

Ankerst, Mihael, Markus M. Breunig, Hans-Peter Kriegel, y Jörg Sander. 1999. «OPTICS: Ordering Points to Identify the Clustering Structure». *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, 49-60. https://doi.org/10.1145/304182.304187.

Anscombe, Francis J. 1973. «Graphs in Statistical Analysis». *The American Statistician* 27 (1): 17-21. https://doi.org/10.1080/00031305.1973.10478966.

Arthur, David, y Sergei Vassilvitskii. 2007. «k-means++: the advantages of careful seeding». *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027-35.

Bamber, Donald. 1975. «The area above the ordinal dominance graph and the area below the receiver operating characteristic graph». *Journal of Mathematical Psychology* 12 (4): 387-415. https://doi.org/10.1016/0022-2496(75)90001-2.

Bayardo, Roberto J. 1998. «Efficiently Mining Long Patterns from Databases». *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data*, 85-93.

Belkin, Mikhail, y Partha Niyogi. 2003. «Laplacian Eigenmaps for Dimensionality Reduction and

Data Representation». *Neural Computation* 15 (6): 1373-96. https://doi.org/10.1162/089976603321780317.

Bellman, Richard E. 1961. *Adaptive Control Processes: A Guided Tour*. Princeton University Press.

Berger, James O. 1985. *Statistical Decision Theory and Bayesian Analysis*. 2.ª ed. Springer.

Bergstra, James, y Yoshua Bengio. 2012. «Random Search for Hyper-Parameter Optimization». *Journal of Machine Learning Research* 13: 281-305.

Beyer, Kevin, Jonathan Goldstein, Raghu Ramakrishnan, y Uri Shaft. 1999. «When Is "Nearest Neighbor" Meaningful?» *Database Theory – ICDT'99*, Lecture Notes en Computer Science, vol. 1540: 217-35. https://doi.org/10.1007/3-540-49257-7_15.

Biostatsquid. 2026. *Diversity metrics easily explained: Shannon, Simpson, Gini, Chao1 and Hill numbers!* Video de YouTube. https://youtu.be/CZRdULKPG2M.

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Information Science y Statistics. Springer.

Blei, David M., Andrew Y. Ng, y Michael I. Jordan. 2003. «Latent Dirichlet Allocation». *Journal of Machine Learning Research* 3: 993-1022.

Borgelt, Christian. 2005. «An Implementation of the FP-growth Algorithm». *Proceedings of the 1st International Workshop on Open Source Data Mining: Frequent Pattern Mining Implementations*, 1-5. https://doi.org/10.1145/1133905.1133907.

Bradbury, James, Roy Frostig, Peter Hawkins, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*. Released. http://github.com/jax-ml/jax.

Breiman, Leo. 1996. «Bagging Predictors». *Machine Learning* 24 (2): 123-40. https://doi.org/10.1007/BF00058655.

Breiman, Leo. 2001b. «Random Forests». *Machine Learning* 45 (1): 5-32. https://doi.org/10.1023/A:1010933404324.

Breiman, Leo. 2001a. «Random Forests». *Machine Learning* 45 (1): 5-32. https://doi.org/10.1023/A:1010933404324.

Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, y Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth International Group.

Breunig, Markus M., Hans-Peter Kriegel, Raymond T. Ng, y Jörg Sander. 2000. «LOF: Identifying Density-Based Local Outliers». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 93-104. https://doi.org/10.1145/342009.335388.

Brier, Glenn W. 1950. «Verification of Forecasts Expressed in Terms of Probability». *Monthly Weather Review* 78 (1): 1-3. https://doi.org/10.1175/1520-0493(1950)078%3C0001:VOFEIT%3E2.0.CO;2.

Bryson, John M. 2004. «What to do when stakeholders matter: Stakeholder identification and analysis techniques». *Public Management Review* 6 (1): 21-53. https://doi.org/10.1080/14719030410001675722.

Caliński, Tadeusz, y Józef Harabasz. 1974. «A dendrite method for cluster analysis». *Communications in Statistics* 3 (1): 1-27.

Campello, Ricardo J. G. B., Davoud Moulavi, y Jörg Sander. 2013. «Density-based clustering based on hierarchical density estimates». *Advances in Knowledge Discovery and Data Mining*, Lecture Notes en Computer Science, vol. 7819: 160-72.

Cawley, Gavin C., y Nicola L. C. Talbot. 2010. «On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation». *Journal of Machine Learning Research* 11: 2079-107.

Center for Community Health and Development, University of Kansas. 2023. *Chapter 7, Section 8: Identifying and Analyzing Stakeholders and Their Interests*. Community Tool Box. https://ctb.ku.edu/en/table-of-contents/participation/encouraging-involvement/identify-stakeholders/main.

Chandola, Varun, Arindam Banerjee, y Vipin Kumar. 2009. «Anomaly Detection: A Survey». *ACM Computing Surveys* 41 (3): 15:1-58. https://doi.org/10.1145/1541880.1541882.

Chang, Chih-Chung, y Chih-Jen Lin. 2011. «LIBSVM: A Library for Support Vector Machines». *ACM Transactions on Intelligent Systems and Technology* 2 (3): 27:1-27. https://doi.org/10.1145/1961189.1961199.

Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.

Chaudhuri, Surajit, y Umeshwar Dayal. 1997. «An Overview of Data Warehousing and OLAP Technology». *ACM SIGMOD Record* 26 (1): 65-74. https://doi.org/10.1145/248603.248616.

Chawla, Nitesh V., Kevin W. Bowyer, Lawrence O. Hall, y W. Philip Kegelmeyer. 2002. «SMOTE: Synthetic Minority Over-sampling Technique». *Journal of Artificial Intelligence Research* 16: 321-57. https://doi.org/10.1613/jair.953.

Chen, Tianqi, y Carlos Guestrin. 2016. «XGBoost: A Scalable Tree Boosting System». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-94. https://doi.org/10.1145/2939672.2939785.

Chicco, Davide, y Giuseppe Jurman. 2020. «The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation». *BMC Genomics* 21 (6). https://doi.org/10.1186/s12864-019-6413-7.

Congreso de los Estados Unidos Mexicanos. 2010. *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf.

Cortes, Corinna, y Vladimir N. Vapnik. 1995. «Support-Vector Networks». *Machine Learning* 20 (3):

273-97. https://doi.org/10.1007/BF00994018.

Cover, Thomas M., y Peter E. Hart. 1967. «Nearest Neighbor Pattern Classification». *IEEE Transactions on Information Theory* 13 (1): 21-27. https://doi.org/10.1109/TIT.1967.1053964.

Cube Dev, Inc. 2024. *Cube Documentation*. Cube Dev. https://cube.dev/docs/.

Cybenko, George. 1989. «Approximation by Superpositions of a Sigmoidal Function». *Mathematics of Control, Signals, and Systems* 2 (4): 303-14.

DAMA International. 2017. *DAMA-DMBOK: Data Management Body of Knowledge*. 2nd ed. Technics Publications.

Databricks. 2024. *Databricks: The Data and AI Company*. Databricks. https://www.databricks.com.

Davenport, Thomas H., y D. J. Patil. 2012. «Data Scientist: The Sexiest Job of the 21st Century». *Harvard Business Review* 90 (10): 70-76.

Davies, David L., y Donald W. Bouldin. 1979. «A cluster separation measure». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1 (2): 224-27.

dbt Labs. 2024. *dbt Semantic Layer Documentation*. Dbt Labs. https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl.

DeepMind, Igor Babuschkin, Kate Baumli, et al. 2020. *The DeepMind JAX Ecosystem*. Released. http://github.com/google-deepmind.

Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.

Dempster, Arthur P., Nan M. Laird, y Donald B. Rubin. 1977. «Maximum Likelihood from Incomplete Data via the EM Algorithm». *Journal of the Royal Statistical Society, Series B* 39 (1): 1-38.

Devlin, Barry A., y Paul T. Murphy. 1988. «An Architecture for a Business and Information System». *IBM Systems Journal* 27 (1): 60-80. https://doi.org/10.1147/sj.271.0060.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, y Kristina Toutanova. 2019. «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding». *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 4171-86.

Dietterich, Thomas G. 1998. «Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms». *Neural Computation* 10 (7): 1895-923. https://doi.org/10.1162/089976698300017197.

Dietterich, Thomas G. 2000. «Ensemble Methods in Machine Learning». *Multiple Classifier Systems*, Lecture Notes en Computer Science, vol. 1857: 1-15. https://doi.org/10.1007/3-540-45014-9_1.

Duda, Richard O., Peter E. Hart, y David G. Stork. 2001. *Pattern Classification*. 2.ª ed. Wiley.

Durbin, Richard, Sean R. Eddy, Anders Krogh, y Graeme Mitchison. 1998. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press.

Ester, Martin, Hans-Peter Kriegel, Jörg Sander, y Xiaowei Xu. 1996. «A density-based algorithm for discovering clusters in large spatial databases with noise». *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226-31.

Fawcett, Tom. 2006. «An introduction to ROC analysis». *Pattern Recognition Letters* 27 (8): 861-74. https://doi.org/10.1016/j.patrec.2005.10.010.

Fayyad, Usama M., y Keki B. Irani. 1992. «On the Handling of Continuous-Valued Attributes in Decision Tree Generation». *Machine Learning* 8 (1): 87-102. https://doi.org/10.1007/BF00994007.

Fayyad, Usama, Gregory Piatetsky-Shapiro, y Padhraic Smyth. 1996. «From data mining to knowledge discovery in databases». *AI Magazine* 17 (3): 37-54. https://doi.org/10.1609/aimag.v17i3.1230.

Feldman, Ronen, y Ido Dagan. 1995. «Knowledge Discovery in Textual Databases (KDT)». *Proceedings of the First International Conference on Knowledge Discovery and Data Mining (KDD-95)*, 112-17.

Fielding, Roy Thomas. 2000. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral, University of California, Irvine.

Fisher, Ronald A. 1936. «The Use of Multiple Measurements in Taxonomic Problems». *Annals of Eugenics* 7 (2): 179-88. https://doi.org/10.1111/j.1469-1809.1936.tb02137.x.

Fix, Evelyn, y Joseph L. Hodges. 1951. *Discriminatory Analysis. Nonparametric Discrimination: Consistency Properties*. No. 4. USAF School of Aviation Medicine, Randolph Field, Texas.

Fournier-Viger, Philippe, Jerry Chun-Wei Lin, Rage Uday Kiran, Yun Sing Koh, y Rincy Thomas. 2017. «A Survey of Sequential Pattern Mining». *Data Science and Pattern Recognition* 1 (1): 54-77.

Fowler, Martin. 2010. *Richardson Maturity Model*. Martinfowler.com. https://martinfowler.com/articles/richardsonMaturityModel.html.

Freedman, David, Robert Pisani, y Roger Purves. 2007. *Statistics*. 4.ª ed. W. W. Norton.

Freeman, R. Edward. 1984. *Strategic Management: A Stakeholder Approach*. Pitman.

Freund, Yoav, y Robert E. Schapire. 1997. «A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting». *Journal of Computer and System Sciences* 55 (1): 119-39. https://doi.org/10.1006/jcss.1997.1504.

Friedman, Jerome H. 2001. «Greedy Function Approximation: A Gradient Boosting Machine». *The Annals of Statistics* 29 (5): 1189-232. https://doi.org/10.1214/aos/1013203451.

Friedman, Jerome H. 2002. «Stochastic Gradient Boosting». *Computational Statistics & Data*

Analysis 38 (4): 367-78. https://doi.org/10.1016/S0167-9473(01)00065-2.

Friedman, Jerome H., Jon Louis Bentley, y Raphael Ari Finkel. 1977. «An Algorithm for Finding Best Matches in Logarithmic Expected Time». *ACM Transactions on Mathematical Software* 3 (3): 209-26. https://doi.org/10.1145/355744.355745.

Friedman, Jerome, Trevor Hastie, y Robert Tibshirani. 2000. «Additive Logistic Regression: A Statistical View of Boosting». *The Annals of Statistics* 28 (2): 337-407. https://doi.org/10.1214/aos/1016218223.

Fukunaga, Keinosuke, y Patrenahalli M. Narendra. 1975. «A Branch and Bound Algorithm for Computing k-Nearest Neighbors». *IEEE Transactions on Computers* C-24 (7): 750-53. https://doi.org/10.1109/T-C.1975.224297.

Ganter, Bernhard, y Rudolf Wille. 1999. *Formal Concept Analysis: Mathematical Foundations*. Springer.

Gardiner-Garden, Margaret, y Marianne Frommer. 1987. «CpG Islands in Vertebrate Genomes». *Journal of Molecular Biology* 196 (2): 261-82. https://doi.org/10.1016/0022-2836(87)90689-9.

Gauss, Carl Friedrich. 1809. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Friedrich Perthes; I. H. Besser.

Geman, Stuart, Elie Bienenstock, y René Doursat. 1992. «Neural Networks and the Bias/Variance Dilemma». *Neural Computation* 4 (1): 1-58. https://doi.org/10.1162/neco.1992.4.1.1.

Gini, Corrado. 1912. «Variabilità e mutabilità». *Studi Economico-Giuridici della Regia Università di Cagliari* 3: 1-158.

Goodfellow, Ian, Yoshua Bengio, y Aaron Courville. 2016. *Deep Learning*. MIT Press.

Google. 2022. *Foundations: Data, Data, Everywhere*. Google Data Analytics Professional Certificate, Coursera.

Google Cloud. 2024. *Vertex AI Documentation*. Google Cloud Documentation. https://cloud.google.com/vertex-ai/docs.

Gorman, Kristen B., Tony D. Williams, y William R. Fraser. 2014. «Ecological Sexual Dimorphism and Environmental Variability within a Community of Antarctic Penguins (Genus *Pygoscelis*)». *PLOS ONE* 9 (3): e90081. https://doi.org/10.1371/journal.pone.0090081.

Gower, John C. 1971. «A General Coefficient of Similarity and Some of Its Properties». *Biometrics* 27 (4): 857-71. https://doi.org/10.2307/2528823.

Grahne, Gösta, y Jianfei Zhu. 2003. «Efficiently Using Prefix-trees in Mining Frequent Itemsets». *Proceedings of the ICDM 2003 Workshop on Frequent Itemset Mining Implementations (FIMI'03)*, CEUR Workshop Proceedings, vol. 90.

GraphQL Foundation. 2021. *GraphQL Specification*. GraphQL Foundation. https://spec.graphql.org/.

Gray, Jim, Adam Bosworth, Andrew Layman, y Hamid Pirahesh. 1996. «Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals». *Proceedings of the Twelfth International Conference on Data Engineering*, 152-59. https://doi.org/10.1109/ICDE.1996.492099.

Grootendorst, Maarten. 2022. «BERTopic: Neural Topic Modeling with a Class-Based TF-IDF Procedure». *arXiv preprint arXiv:2203.05794*.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2011. *Data Mining: Concepts and Techniques*. 3.ª ed. Morgan Kaufmann. https://doi.org/10.1016/B978-0-12-381479-1.00001-0.

Han, Jiawei, Micheline Kamber, y Jian Pei. 2012. *Data Mining: Concepts and Techniques*. 3.ª ed. Morgan Kaufmann.

Han, Jiawei, Jian Pei, y Yiwen Yin. 2000. «Mining Frequent Patterns without Candidate Generation». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, 1-12. https://doi.org/10.1145/342009.335372.

Hart, Peter E. 1968. «The Condensed Nearest Neighbor Rule». *IEEE Transactions on Information Theory* 14 (3): 515-16. https://doi.org/10.1109/TIT.1968.1054155.

Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.ª ed. Springer Series en Statistics. Springer. https://doi.org/10.1007/978-0-387-84858-7.

Hearst, Marti A. 1999. «Untangling Text Data Mining». *Proceedings of the 37th Annual Meeting of the Association for Computational Linguistics (ACL)*, 3-10.

Heckert, N. Alan, James J. Filliben, Carroll M. Croarkin, et al. 2012. *NIST/SEMATECH e-Handbook of Statistical Methods*. National Institute of Standards; Technology. https://www.itl.nist.gov/div898/handbook/.

Heek, Jonathan, Anselm Levskaya, Avital Oliver, et al. 2023. *Flax: A Neural Network Library and Ecosystem for JAX*. Released. http://github.com/google/flax.

Hill, Mark O. 1973. «Diversity and Evenness: A Unifying Notation and Its Consequences». *Ecology* 54 (2): 427-32. https://doi.org/10.2307/1934352.

Hinton, Geoffrey E., y Ruslan R. Salakhutdinov. 2006. «Reducing the Dimensionality of Data with Neural Networks». *Science* 313 (5786): 504-7. https://doi.org/10.1126/science.1127647.

Hirschman, Albert O. 1945. *National Power and the Structure of Foreign Trade*. University of California Press.

Hochreiter, Sepp, y Jürgen Schmidhuber. 1997. «Long Short-Term Memory». *Neural Computation* 9 (8): 1735-80.

Hoerl, Arthur E., y Robert W. Kennard. 1970. «Ridge Regression: Biased Estimation for Nonorthogonal Problems». *Technometrics* 12 (1): 55-67. https://doi.org/10.1080/00401706.1970.10488634.

Hornik, Kurt, Maxwell Stinchcombe, y Halbert White. 1989. «Multilayer Feedforward Networks are Universal Approximators». *Neural Networks* 2 (5): 359-66.

Horst, Allison Marie, Alison Presmanes Hill, y Kristen B. Gorman. 2020. «palmerpenguins: Palmer Archipelago (Antarctica) Penguin Data». *R package version 0.1.0*, advance online publication. https://doi.org/10.5281/zenodo.3960218.

Hosmer, David W., Stanley Lemeshow, y Rodney X. Sturdivant. 2013. *Applied Logistic Regression*. 3.ª ed. John Wiley & Sons.

Huang, Zhexue. 1998. «Extensions to the k-Means Algorithm for Clustering Large Data Sets with Categorical Values». *Data Mining and Knowledge Discovery* 2 (3): 283-304. https://doi.org/10.1023/A:1009769707641.

Huber, Peter J., y Elvezio M. Ronchetti. 2009. *Robust Statistics*. 2.ª ed. Wiley. https://doi.org/10.1002/9780470434697.

Hubert, Lawrence, y Phipps Arabie. 1985. «Comparing partitions». *Journal of Classification* 2 (1): 193-218.

Hunt, Earl B., Janet Marin, y Philip J. Stone. 1966. *Experiments in Induction*. Academic Press.

Hyndman, Rob J., y Yanan Fan. 1996. «Sample Quantiles in Statistical Packages». *The American Statistician* 50 (4): 361-65. https://doi.org/10.1080/00031305.1996.10473566.

IBM. 2015. *Analytics Solutions Unified Method (ASUM-DM)*. IBM Corporation, internal methodology derived from CRISP-DM. https://public.dhe.ibm.com/software/data/sw-library/services/ASUM.pdf.

Inmon, William H. 2005. *Building the Data Warehouse*. 4.ª ed. Wiley.

James, Gareth, Daniela Witten, Trevor Hastie, y Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2.ª ed. Springer Texts en Statistics. Springer. https://doi.org/10.1007/978-1-0716-1418-1.

Jaynes, Edwin T. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press.

Joanes, D. N., y C. A. Gill. 1998. «Comparing Measures of Sample Skewness and Kurtosis». *Journal of the Royal Statistical Society: Series D (The Statistician)* 47 (1): 183-89. https://doi.org/10.1111/1467-9884.00122.

Johnson, Richard A., y Dean W. Wichern. 2007. *Applied Multivariate Statistical Analysis*. 6.ª ed. Prentice Hall.

Jost, Lou. 2006. «Entropy and Diversity». *Oikos* 113 (2): 363-75. https://doi.org/10.1111/j.2006.0030-

1299.14714.x.

Jurafsky, Daniel, y James H. Martin. 2025. *Speech and Language Processing*. 3.ª ed.

Kaufman, Leonard, y Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

Kendall, Maurice G. 1938. «A New Measure of Rank Correlation». *Biometrika* 30 (1/2): 81-93. https://doi.org/10.2307/2332226.

Kimball, Ralph, y Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3.ª ed. Wiley.

Kingma, Diederik P., y Jimmy Ba. 2015. «Adam: A Method for Stochastic Optimization». *International Conference on Learning Representations (ICLR)*.

Kleinberg, Jon. 2002. «An Impossibility Theorem for Clustering». *Advances in Neural Information Processing Systems 15 (NIPS 2002)*, 463-70.

Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O'Reilly Media.

Kobak, Dmitry, y Philipp Berens. 2019. «The Art of Using t-SNE for Single-Cell Transcriptomics». *Nature Communications* 10: 5416. https://doi.org/10.1038/s41467-019-13056-x.

Kohavi, Ron. 1995. «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection». *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* 2: 1137-43.

Kohavi, Ron, Diane Tang, y Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.

Kruskal, Joseph B. 1964. «Multidimensional Scaling by Optimizing Goodness of Fit to a Nonmetric Hypothesis». *Psychometrika* 29 (1): 1-27. https://doi.org/10.1007/BF02289565.

Lance, G. N., y W. T. Williams. 1967. «A General Theory of Classificatory Sorting Strategies: 1. Hierarchical Systems». *The Computer Journal* 9 (4): 373-80. https://doi.org/10.1093/comjnl/9.4.373.

LeCun, Yann, Yoshua Bengio, y Geoffrey Hinton. 2015. «Deep Learning». *Nature* 521: 436-44.

LeCun, Yann, Léon Bottou, Yoshua Bengio, y Patrick Haffner. 1998. «Gradient-Based Learning Applied to Document Recognition». *Proceedings of the IEEE* 86 (11): 2278-324.

Lee, Daniel D., y H. Sebastian Seung. 1999. «Learning the Parts of Objects by Non-negative Matrix Factorization». *Nature* 401: 788-91.

Lee, Daniel D., y H. Sebastian Seung. 2001. «Algorithms for Non-negative Matrix Factorization». *Advances in Neural Information Processing Systems 13 (NIPS 2000)*, 556-62.

Legendre, Adrien-Marie. 1805. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot.

Li, Wenmin, Jiawei Han, y Jian Pei. 2001. «CMAR: Accurate and Efficient Classification Based on Multiple Class-Association Rules». *Proceedings of the 2001 IEEE International Conference on Data Mining (ICDM)*, 369-76.

Liu, Bing, Wynne Hsu, y Yiming Ma. 1998. «Integrating Classification and Association Rule Mining». *Proceedings of the Fourth International Conference on Knowledge Discovery and Data Mining (KDD-98)*, 80-86.

Liu, Fei Tony, Kai Ming Ting, y Zhi-Hua Zhou. 2008. «Isolation Forest». *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 413-22. https://doi.org/10.1109/ICDM.2008.17.

Lloyd, Stuart P. 1982. «Least squares quantization in PCM». *IEEE Transactions on Information Theory* 28 (2): 129-37.

Lorenz, Max O. 1905. «Methods of Measuring the Concentration of Wealth». *Publications of the American Statistical Association* 9 (70): 209-19. https://doi.org/10.2307/2276207.

Lundberg, Scott M., Gabriel Erion, Hugh Chen, et al. 2020. «From Local Explanations to Global Understanding with Explainable AI for Trees». *Nature Machine Intelligence* 2: 56-67. https://doi.org/10.1038/s42256-019-0138-9.

Lundberg, Scott M., y Su-In Lee. 2017. «A Unified Approach to Interpreting Model Predictions». *Advances in Neural Information Processing Systems* 30.

Luxburg, Ulrike von. 2007. «A Tutorial on Spectral Clustering». *Statistics and Computing* 17 (4): 395-416. https://doi.org/10.1007/s11222-007-9033-z.

Luxburg, Ulrike von. 2010. «Clustering Stability: An Overview». *Foundations and Trends in Machine Learning* 2 (3): 235-74. https://doi.org/10.1561/2200000008.

Maaten, Laurens van der, y Geoffrey Hinton. 2008. «Visualizing data using t-SNE». *Journal of Machine Learning Research* 9: 2579-605.

MacQueen, James. 1967. «Some methods for classification and analysis of multivariate observations». *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281-97.

Mahalanobis, Prasanta Chandra. 1936. «On the generalized distance in statistics». *Proceedings of the National Institute of Sciences of India* 2 (1): 49-55.

Manning, Christopher D., Prabhakar Raghavan, y Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.

Mardia, K. V. 1970. «Measures of multivariate skewness and kurtosis with applications». *Biometrika* 57 (3): 519-30. https://doi.org/10.1093/biomet/57.3.519.

Matthews, Brian W. 1975. «Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme». *Biochimica et Biophysica Acta (BBA) – Protein Structure* 405 (2): 442-51. https://doi.org/10.1016/0005-2795(75)90109-9.

McCallum, Andrew, y Kamal Nigam. 1998. «A Comparison of Event Models for Naive Bayes Text Classification». *AAAI-98 Workshop on Learning for Text Categorization*, 41-48.

McCullagh, Peter, y John A. Nelder. 1989. *Generalized Linear Models*. 2.ª ed. Chapman & Hall.

McInnes, Leland, John Healy, y James Melville. 2018. «UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction». *arXiv preprint arXiv:1802.03426*.

McKinney, Wes. 2010. «Data Structures for Statistical Computing in Python». *Proceedings of the 9th Python in Science Conference*, 56-61. https://doi.org/10.25080/Majora-92bf1922-00a.

McLachlan, Geoffrey J., y David Peel. 2000. *Finite Mixture Models*. Wiley Series en Probability y Statistics. John Wiley & Sons.

McNemar, Quinn. 1947. «Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages». *Psychometrika* 12 (2): 153-57. https://doi.org/10.1007/BF02295996.

Mendelow, Aubrey L. 1981. «Environmental Scanning – The Impact of the Stakeholder Concept». *Proceedings of the Second International Conference on Information Systems (ICIS)* (Cambridge, MA).

Microsoft. 2023. *What Is the Team Data Science Process (TDSP)?* GitHub (Azure/Microsoft-TDSP), archived repository. https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/README.md.

Microsoft. 2024. *Azure Machine Learning Documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/machine-learning/.

Mikolov, Tomas, Kai Chen, Greg Corrado, y Jeffrey Dean. 2013. «Efficient Estimation of Word Representations in Vector Space». *arXiv preprint arXiv:1301.3781*.

Minsky, Marvin, y Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Mitchell, Tom M. 1997. *Machine Learning*. McGraw-Hill.

Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.

Murtagh, Fionn, y Pedro Contreras. 2012. «Algorithms for Hierarchical Clustering: An Overview». *WIREs Data Mining and Knowledge Discovery* 2 (1): 86-97. https://doi.org/10.1002/widm.53.

Netflix Technology Blog. 2016. *It's All A/Bout Testing: The Netflix Experimentation Platform*. https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15.

Neyman, Jerzy, y Egon S. Pearson. 1933. «On the Problem of the Most Efficient Tests of Statistical Hypotheses». *Philosophical Transactions of the Royal Society of London. Series A* 231: 289-337. https://doi.org/10.1098/rsta.1933.0009.

Niculescu-Mizil, Alexandru, y Rich Caruana. 2005. «Predicting Good Probabilities with Supervised Learning». *Proceedings of the 22nd International Conference on Machine Learning*, 625-32. https://doi.org/10.1145/1102351.1102430.

Novikoff, Albert B. J. 1962. «On Convergence Proofs on Perceptrons». *Proceedings of the Symposium on the Mathematical Theory of Automata* 12: 615-22.

Osterwalder, Alexander, y Yves Pigneur. 2010. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley.

Pang, Bo, Lillian Lee, y Shivakumar Vaithyanathan. 2002. «Thumbs up? Sentiment Classification using Machine Learning Techniques». *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 79-86.

Pasquier, Nicolas, Yves Bastide, Rafik Taouil, y Lotfi Lakhal. 1999. «Discovering Frequent Closed Itemsets for Association Rules». *Database Theory – ICDT'99*, Lecture Notes en Computer Science, vol. 1540: 398-416.

Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. «PyTorch: An Imperative Style, High-Performance Deep Learning Library». *Advances in Neural Information Processing Systems (NeurIPS)* 32.

Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2nd ed. Cambridge University Press.

Pearson, Karl. 1895. «Notes on Regression and Inheritance in the Case of Two Parents». *Proceedings of the Royal Society of London* 58: 240-42. https://doi.org/10.1098/rspl.1895.0041.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12: 2825-30. https://jmlr.org/papers/v12/pedregosa11a.html.

Pei, Jian, Jiawei Han, Behzad Mortazavi-Asl, et al. 2001. «PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth». *Proceedings of the 17th International Conference on Data Engineering (ICDE)*, 215-24.

Pei, Jian, Jiawei Han, Behzad Mortazavi-Asl, et al. 2004. «Mining Sequential Patterns by Pattern-Growth: The PrefixSpan Approach». *IEEE Transactions on Knowledge and Data Engineering* 16 (11): 1424-40.

Platt, John C. 1999. «Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods». En *Advances in Large Margin Classifiers*, editado por Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, y Dale Schuurmans. MIT Press.

Powers, David M. W. 2011. «Evaluation: From Precision, Recall and F-Factor to ROC, Informedness,

Markedness and Correlation». *Journal of Machine Learning Technologies* 2 (1): 37-63. https://arxiv.org/abs/2010.16061.

Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 6th ed. Project Management Institute.

Provost, Foster, y Tom Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.

Quinlan, J. Ross. 1986. «Induction of Decision Trees». *Machine Learning* 1 (1): 81-106. https://doi.org/10.1007/BF00116251.

Quinlan, J. Ross. 1987. «Simplifying Decision Trees». *International Journal of Man-Machine Studies* 27 (3): 221-34. https://doi.org/10.1016/S0020-7373(87)80053-6.

Quinlan, J. Ross. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.

Ramaswamy, Sridhar, Rajeev Rastogi, y Kyuseok Shim. 2000. «Efficient Algorithms for Mining Outliers from Large Data Sets». *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 427-38. https://doi.org/10.1145/342009.335437.

Ramírez, Sebastián. 2024. *FastAPI Documentation*. FastAPI. https://fastapi.tiangolo.com/.

Rand, William M. 1971. «Objective Criteria for the Evaluation of Clustering Methods». *Journal of the American Statistical Association* 66 (336): 846-50. https://doi.org/10.1080/01621459.1971.10482356.

Raschka, Sebastian. 2018. «MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack». *The Journal of Open Source Software* 3 (24): 638. https://doi.org/10.21105/joss.00638.

Reimers, Nils, y Iryna Gurevych. 2019. «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks». *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982-92.

Reynolds, Douglas A. 2009. «Gaussian Mixture Models». *Encyclopedia of Biometrics* (Boston, MA), 659-63. https://doi.org/10.1007/978-0-387-73003-5_196.

Rosenblatt, Frank. 1958. «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain». *Psychological Review* 65 (6): 386-408. https://doi.org/10.1037/h0042519.

Rousseeuw, Peter J. 1984. «Least Median of Squares Regression». *Journal of the American Statistical Association* 79 (388): 871-80. https://doi.org/10.1080/01621459.1984.10477105.

Rousseeuw, Peter J. 1987. «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis». *Journal of Computational and Applied Mathematics* 20: 53-65.

Rousseeuw, Peter J., y Christophe Croux. 1993. «Alternatives to the Median Absolute Deviation». *Journal of the American Statistical Association* 88 (424): 1273-83. https://doi.org/10.1080/01621459.1993.10476408.

Rousseeuw, Peter J., y Katrien Van Driessen. 1999. «A Fast Algorithm for the Minimum Covariance Determinant Estimator». *Technometrics* 41 (3): 212-23. https://doi.org/10.1080/00401706.1999.10485670.

Roweis, Sam T., y Lawrence K. Saul. 2000. «Nonlinear Dimensionality Reduction by Locally Linear Embedding». *Science* 290 (5500): 2323-26. https://doi.org/10.1126/science.290.5500.2323.

Rumelhart, David E., Geoffrey E. Hinton, y Ronald J. Williams. 1986. «Learning Representations by Back-propagating Errors». *Nature* 323: 533-36.

Saito, Takaya, y Marc Rehmsmeier. 2015. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». *PLOS ONE* 10 (3): e0118432. https://doi.org/10.1371/journal.pone.0118432.

Salton, Gerard, y Christopher Buckley. 1988. «Term-Weighting Approaches in Automatic Text Retrieval». *Information Processing & Management* 24 (5): 513-23.

Salton, Gerard, Anita Wong, y Chung-Shu Yang. 1975. «A Vector Space Model for Automatic Indexing». *Communications of the ACM* 18 (11): 613-20.

Santizo Galicia, Jessica. 2026. *Computación de Cubos de datos*. Presentación de clase, Facultad de Ciencias, Universidad Nacional Autónoma de México.

SAS Institute. 2017. *Introduction to SEMMA*. SAS Enterprise Miner Reference Help. https://documentation.sas.com/doc/en/emref/14.3/n061bzurmej4j3n1jnj8bbjjm1a2.htm.

Schapire, Robert E. 1990. «The Strength of Weak Learnability». *Machine Learning* 5 (2): 197-227. https://doi.org/10.1007/BF00116037.

Schölkopf, Bernhard, John C. Platt, John Shawe-Taylor, Alex J. Smola, y Robert C. Williamson. 2001. «Estimating the Support of a High-Dimensional Distribution». *Neural Computation* 13 (7): 1443-71. https://doi.org/10.1162/089976601750264965.

Schölkopf, Bernhard, y Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.

Schubert, Erich, Jörg Sander, Martin Ester, Hans-Peter Kriegel, y Xiaowei Xu. 2017. «DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN». *ACM Transactions on Database Systems* 42 (3): 19:1-21. https://doi.org/10.1145/3068335.

Sculley, D. 2010. «Web-Scale K-Means Clustering». *Proceedings of the 19th International Conference on World Wide Web*, 1177-78. https://doi.org/10.1145/1772690.1772862.

Shannon, Claude E. 1948. «A Mathematical Theory of Communication». *Bell System Technical*

Journal 27 (3): 379-423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x.

Shearer, Colin. 2000. «The CRISP-DM Model: The New Blueprint for Data Mining». *Journal of Data Warehousing* 5 (4): 13-22.

Simpson, Edward H. 1949. «Measurement of Diversity». *Nature* 163: 688. https://doi.org/10.1038/163688a0.

Simpson, Edward H. 1951. «The Interpretation of Interaction in Contingency Tables». *Journal of the Royal Statistical Society: Series B (Methodological)* 13 (2): 238-41. https://doi.org/10.1111/j.2517-6161.1951.tb00088.x.

Small, Christopher G. 1990. «A Survey of Multidimensional Medians». *International Statistical Review* 58 (3): 263-77. https://doi.org/10.2307/1403809.

Sokal, Robert R., y F. James Rohlf. 1962. «The Comparison of Dendrograms by Objective Methods». *Taxon* 11 (2): 33-40. https://doi.org/10.2307/1217208.

Spärck Jones, Karen. 1972. «A Statistical Interpretation of Term Specificity and Its Application in Retrieval». *Journal of Documentation* 28 (1): 11-21.

Spearman, Charles. 1904. «The Proof and Measurement of Association between Two Things». *The American Journal of Psychology* 15 (1): 72-101. https://doi.org/10.2307/1412159.

Srikant, Ramakrishnan, y Rakesh Agrawal. 1996. «Mining Sequential Patterns: Generalizations and Performance Improvements». *Advances in Database Technology – EDBT'96*, Lecture Notes en Computer Science, vol. 1057: 1-17.

Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, y Ruslan Salakhutdinov. 2014. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». *Journal of Machine Learning Research* 15: 1929-58.

Stone, Charles J. 1977. «Consistent Nonparametric Regression». *The Annals of Statistics* 5 (4): 595-620. https://doi.org/10.1214/aos/1176343886.

Stone, M. 1974. «Cross-Validatory Choice and Assessment of Statistical Predictions». *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111-33. https://doi.org/10.1111/j.2517-6161.1974.tb00994.x.

Strobl, Carolin, Anne-Laure Boulesteix, Achim Zeileis, y Torsten Hothorn. 2007. «Bias in Random Forest Variable Importance Measures: Illustrations, Sources and a Solution». *BMC Bioinformatics* 8: 25. https://doi.org/10.1186/1471-2105-8-25.

Studer, Stefan, Thanh Binh Bui, Christian Drescher, et al. 2021. «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology». *Machine Learning and Knowledge Extraction* 3 (2): 392-413.

Tan, Pang-Ning, Michael Steinbach, Anuj Karpatne, y Vipin Kumar. 2019. *Introduction to Data*

Mining. 2.ª ed. Pearson.

Tan, Pang-Ning, Michael Steinbach, y Vipin Kumar. 2006. *Introduction to Data Mining*. Addison-Wesley.

Tenenbaum, Joshua B., Vin de Silva, y John C. Langford. 2000. «A Global Geometric Framework for Nonlinear Dimensionality Reduction». *Science* 290 (5500): 2319-23. https://doi.org/10.1126/science.290.5500.2319.

The pandas development team. 2024. *pandas Documentation*. Pandas. https://pandas.pydata.org/docs/.

Tibshirani, Robert, Guenther Walther, y Trevor Hastie. 2001. «Estimating the Number of Clusters in a Data Set via the Gap Statistic». *Journal of the Royal Statistical Society: Series B* 63 (2): 411-23. https://doi.org/10.1111/1467-9868.00293.

Torgerson, Warren S. 1952. «Multidimensional Scaling: I. Theory and Method». *Psychometrika* 17 (4): 401-19. https://doi.org/10.1007/BF02288916.

Tukey, John W. 1975. «Mathematics and the picturing of data». *Proceedings of the International Congress of Mathematicians* 2: 523-31.

Tukey, John W. 1977. *Exploratory Data Analysis*. Addison-Wesley.

Vapnik, Vladimir N. 1995. *The Nature of Statistical Learning Theory*. Springer.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems (NeurIPS)* 30.

Vinh, Nguyen Xuan, Julien Epps, y James Bailey. 2010. «Information Theoretic Measures for Clusterings Comparison: Variants, Properties, Normalization and Correction for Chance». *Journal of Machine Learning Research* 11: 2837-54.

Wang, Jianyong, Jiawei Han, y Jian Pei. 2003. «CLOSET+: Searching for the Best Strategies for Mining Frequent Closed Itemsets». *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 236-45.

Ward, Joe H. 1963. «Hierarchical grouping to optimize an objective function». *Journal of the American Statistical Association* 58 (301): 236-44.

Wasserman, Larry. 2004. *All of Statistics: A Concise Course in Statistical Inference*. Springer.

Wattenberg, Martin, Fernanda Viégas, y Ian Johnson. 2016. «How to Use t-SNE Effectively». *Distill*, advance online publication. https://doi.org/10.23915/distill.00002.

Wilcox, Rand R. 2012. *Introduction to Robust Estimation and Hypothesis Testing*. 3.ª ed. Academic Press.

Wilks, Samuel S. 1932. «Certain generalizations in the analysis of variance». *Biometrika* 24 (3/4): 471-94. https://doi.org/10.1093/biomet/24.3-4.471.

Wilson, Dennis L. 1972. «Asymptotic Properties of Nearest Neighbor Rules Using Edited Data». *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2 (3): 408-21. https://doi.org/10.1109/TSMC.1972.4309137.

Wirth, Rüdiger, y Jochen Hipp. 2000. «CRISP-DM: Towards a Standard Process Model for Data Mining». *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29-39.

Wolpert, David H. 1996. «The Lack of A Priori Distinctions Between Learning Algorithms». *Neural Computation* 8 (7): 1341-90. https://doi.org/10.1162/neco.1996.8.7.1341.

Yan, Xifeng, Jiawei Han, y Ramin Afshar. 2003. «CloSpan: Mining Closed Sequential Patterns in Large Datasets». *Proceedings of the 2003 SIAM International Conference on Data Mining*, 166-77.

Zaki, Mohammed J. 2001. «SPADE: An Efficient Algorithm for Mining Frequent Sequences». *Machine Learning* 42 (1–2): 31-60.

Zaki, Mohammed J., y Ching-Jui Hsiao. 2002. «CHARM: An Efficient Algorithm for Closed Itemset Mining». *Proceedings of the 2002 SIAM International Conference on Data Mining*, 457-73.

Zipf, George Kingsley. 1949. *Human Behavior and the Principle of Least Effort*. Addison-Wesley.